

CANN 社区版
9.0.0-beta1

Ascend C 算子开发指南

文档版本 01
发布日期 2026-02-27



版权所有 © 华为技术有限公司 2026。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

安全声明

产品生命周期政策

华为公司对产品生命周期的规定以“产品生命周期终止政策”为准，该政策的详细内容请参见如下网址：
<https://support.huawei.com/ecolumnsweb/zh/warranty-policy>

漏洞处理流程

华为公司对产品漏洞管理的规定以“漏洞处理流程”为准，该流程的详细内容请参见如下网址：
<https://www.huawei.com/cn/psirt/vul-response-process>
如企业客户须获取漏洞信息，请参见如下网址：
<https://securitybulletin.huawei.com/enterprise/cn/security-advisory>

华为初始证书权责说明

华为公司对随设备出厂的初始数字证书，发布了“华为设备初始数字证书权责说明”，该说明的详细内容请参见如下网址：
<https://support.huawei.com/enterprise/zh/bulletins-service/ENEWS2000015766>

华为企业业务最终用户许可协议(EULA)

本最终用户许可协议是最终用户（个人、公司或其他任何实体）与华为公司就华为软件的使用所缔结的协议。最终用户对华为软件的使用受本协议约束，该协议的详细内容请参见如下网址：
<https://e.huawei.com/cn/about/eula>

产品资料生命周期策略

华为公司针对随产品版本发布的售后客户资料（产品资料），发布了“产品资料生命周期策略”，该策略的详细内容请参见如下网址：
<https://support.huawei.com/enterprise/zh/bulletins-website/ENEWS2000017760>

目 录

1 入门教程.....	1
1.1 什么是 Ascend C.....	1
1.2 环境准备.....	4
1.3 快速入门.....	4
1.3.1 SIMD.....	5
1.3.1.1 HelloWorld.....	5
1.3.1.2 Add 自定义算子开发.....	6
2 编程指南.....	16
2.1 本文档组织结构.....	16
2.2 编程模型.....	17
2.2.1 异构并行编程模型.....	17
2.2.2 抽象硬件架构.....	19
2.2.3 核函数.....	20
2.2.4 编程范式.....	24
2.2.4.1 AI Core SIMD 编程.....	24
2.2.4.1.1 基于 TPipe 和 TQue 编程.....	24
2.2.4.1.2 静态 Tensor 编程.....	30
2.2.4.1.3 基于语言扩展层 C API 编程.....	41
2.2.4.2 AI CPU 编程.....	45
2.3 编译与运行.....	47
2.3.1 AI Core 算子编译.....	47
2.3.1.1 算子编译简介.....	47
2.3.1.2 通过 bisheng 命令行编译.....	47
2.3.1.3 常用的编译选项.....	50
2.3.1.4 通过 CMake 编译.....	51
2.3.1.5 RTC.....	53
2.3.1.6 约束说明.....	57
2.3.2 AI CPU 算子编译.....	58
2.3.3 算子运行.....	63
2.4 语言扩展层.....	64
2.4.1 SIMD BuiltIn 关键字和 API.....	65
2.5 C++类库 API.....	72
2.5.1 编程接口概述.....	72

2.5.2 基础 API.....	73
2.5.2.1 概述.....	73
2.5.2.2 接口分类说明.....	74
2.5.2.2.1 连续计算 API.....	75
2.5.2.2.2 高维切分 API.....	75
2.5.2.3 常用操作速查指导.....	80
2.5.2.3.1 如何使用掩码操作 API.....	80
2.5.2.3.2 如何使用归约计算 API.....	83
2.5.3 高阶 API.....	85
2.5.3.1 概述.....	85
2.5.3.2 常用操作速查指导.....	86
2.5.3.2.1 如何使用 Tiling 依赖的头文件.....	86
2.5.3.2.2 如何使用 Kernel 侧临时空间.....	86
2.5.4 Utils API.....	88
2.6 硬件实现.....	89
2.6.1 基本架构.....	89
2.6.2 架构规格.....	97
2.6.2.1 NPU 架构版本 200x.....	97
2.6.2.2 NPU 架构版本 220x.....	100
2.6.2.3 NPU 架构版本 300x.....	107
2.6.3 硬件约束.....	113
2.6.3.1 NPU 架构版本 200x.....	113
2.6.3.2 NPU 架构版本 220x.....	113
2.7 调试调优.....	116
2.7.1 概述.....	116
2.7.2 功能调试.....	118
2.7.2.1 CPU 域孪生调试.....	118
2.7.2.2 NPU 域上板调试.....	121
2.7.3 性能调优.....	123
2.8 兼容性指南.....	123
2.9 概念原理和术语.....	124
2.9.1 术语表.....	124
2.9.2 神经网络和算子.....	131
2.9.2.1 算子基本概念.....	131
2.9.2.2 数据排布格式.....	134
2.9.3 编程模型设计原理.....	148
2.9.4 内存访问原理.....	153
2.9.4.1 Scalar 读写数据.....	153
2.9.5 性能优化技术原理.....	155
2.9.5.1 DoubleBuffer.....	155
2.10 附录.....	157
2.10.1 C++标准支持.....	157

2.10.1.1 概述.....	157
2.10.1.2 C/C++语法限制.....	157
2.10.1.2.1 特性.....	157
2.10.2 工程化算子开发.....	157
2.10.2.1 概述.....	157
2.10.2.2 创建算子工程.....	158
2.10.2.3 算子原型定义.....	161
2.10.2.4 Kernel 侧算子实现.....	166
2.10.2.5 Host 侧 Tiling 实现.....	169
2.10.2.5.1 基本流程.....	169
2.10.2.5.2 通过 TilingData 传递属性信息.....	175
2.10.2.5.3 使用高阶 API 时配套的 Tiling 实现.....	176
2.10.2.5.4 使用标准 C++语法定义 Tiling 结构体.....	177
2.10.2.5.5 Tiling 模板编程.....	181
2.10.2.6 算子包编译.....	183
2.10.2.6.1 算子工程编译.....	183
2.10.2.6.2 算子包部署.....	190
2.10.2.7 算子动态库和静态库编译.....	192
2.10.2.8 算子工程编译拓展.....	194
2.10.2.9 单算子 API 调用.....	202
2.10.3 算子入图（GE 图）开发.....	208
2.10.3.1 概述.....	208
2.10.3.2 基本开发流程.....	209
2.10.3.3 使能 Tiling 下沉.....	218
2.10.3.4 SuperKernel 开发.....	220
2.10.3.5 图编译和图执行.....	223
2.10.4 AI 框架算子适配.....	227
2.10.4.1 概述.....	227
2.10.4.2 PyTorch 框架.....	228
2.10.4.3 ONNX 框架.....	231
2.10.4.3.1 适配插件开发.....	231
2.10.4.3.2 调用样例.....	233
2.10.4.4 TensorFlow 框架.....	235
2.10.5 show_kernel_debug_data 工具.....	247
2.10.6 msobjdump 工具.....	249
2.10.7 基于样例工程完成 Kernel 直调.....	262
2.10.8 简易自定义算子工程.....	273
2.10.9 常用操作.....	279
2.10.9.1 如何开发动态输入算子.....	279
2.10.9.2 如何在矢量编程时使能 Vector Core.....	281
2.10.9.3 如何使用 workspace.....	282
2.10.9.4 如何进行 Tiling 调测.....	283

2.10.9.5 如何使用 Tensor 原地操作提升算子性能.....	284
2.10.10 FAQ.....	287
2.10.10.1 核函数运行验证时算子存在精度问题.....	287
2.10.10.2 运行验证时 AllocTensor/FreeTensor 失败.....	289
2.10.10.3 kernel 侧获取 Tiling 信息不正确.....	290
2.10.10.4 Kernel 编译时报错 “error: out of jump/jumprange”	291
2.10.10.5 使用跨版本的自定义算子包时，含有 Matmul 高阶 API 的算子存在编译或执行报错.....	291
2.10.10.6 含有 Matmul 高阶 API 的算子精度问题.....	292
2.10.10.7 算子工程编译时出现文件名过长报错.....	302
2.10.10.8 调用算子时出现无法打开 config.ini 的报错.....	303
2.10.10.9 算子包部署时出现权限不足报错.....	303
3 算子实践参考.....	305
3.1 本文档组织结构.....	305
3.2 异构计算.....	307
3.3 SIMD 算子实现.....	308
3.3.1 概述.....	308
3.3.2 矢量编程.....	309
3.3.2.1 概述.....	309
3.3.2.2 基础矢量算子.....	309
3.3.2.3 TBuf 的使用.....	315
3.3.2.4 多核&Tiling 切分.....	317
3.3.2.4.1 概述.....	318
3.3.2.4.2 多核 Tiling.....	319
3.3.2.4.3 尾块 Tiling.....	323
3.3.2.4.4 尾核 Tiling.....	327
3.3.2.4.5 尾核&尾块.....	329
3.3.2.5 DoubleBuffer 场景.....	331
3.3.2.6 Broadcast 场景.....	333
3.3.2.7 非对齐场景.....	337
3.3.3 矩阵编程（高阶 API）.....	347
3.3.3.1 基础知识.....	347
3.3.3.2 算子实现.....	352
3.3.3.3 特性场景.....	357
3.3.3.3.1 Matmul 特性介绍.....	357
3.3.3.3.2 多核对齐切分.....	358
3.3.3.3.3 多核非对齐切分.....	360
3.3.3.3.4 异步场景处理.....	362
3.3.3.3.5 矩阵乘输出的量化/反量化.....	363
3.3.3.3.6 矩阵乘输出的 Channel 拆分.....	366
3.3.3.3.7 矩阵向量乘.....	368
3.3.3.3.8 4:2 稀疏矩阵乘.....	370
3.3.3.3.9 TSCM 输入的矩阵乘.....	372

3.3.3.3.10 矩阵乘输出的 N 方向对齐.....	373
3.3.3.3.11 单次矩阵乘局部输出.....	375
3.3.3.3.12 AIC 和 AIV 独立运行机制.....	377
3.3.3.3.13 Batch Matmul 基础功能.....	378
3.3.3.3.14 Batch Matmul 复用 Bias 矩阵.....	381
3.3.4 矩阵编程（基础 API）.....	383
3.3.4.1 耦合模式.....	383
3.3.4.2 分离模式.....	397
3.3.5 融合算子编程.....	399
3.3.5.1 CV 融合.....	399
3.3.5.1.1 基础知识.....	399
3.3.5.1.2 算子实现.....	403
3.3.5.2 通算融合.....	410
3.3.5.2.1 基础知识.....	411
3.3.5.2.2 算子实现.....	413
3.3.5.2.3 特性场景.....	429
3.4 功能调试.....	430
3.4.1 运行正常.....	430
3.4.2 精度正常.....	430
3.4.3 算子调试.....	432
3.5 性能分析.....	432
3.5.1 获取性能数据.....	432
3.5.2 分析性能数据.....	434
3.6 SIMD 算子性能优化.....	436
3.6.1 优化建议总览表.....	437
3.6.2 Tiling 策略.....	438
3.6.2.1 核间负载均衡.....	438
3.6.3 头尾开销优化.....	439
3.6.3.1 设置合适的核数和算子 Kernel 类型.....	439
3.6.3.2 限制 TilingData 结构大小.....	440
3.6.3.3 避免 TPipe 在对象内创建和初始化.....	442
3.6.3.4 核函数内删除 Workspace 相关冗余操作.....	445
3.6.4 流水编排.....	446
3.6.4.1 使能 DoubleBuffer.....	446
3.6.4.2 使能 Iterate 或 IterateAll 异步接口避免 AIC/AIV 同步依赖.....	448
3.6.5 内存访问.....	449
3.6.5.1 尽量一次搬运较大的数据块.....	449
3.6.5.2 GM 地址尽量 512B 对齐.....	451
3.6.5.3 高效的使用搬运 API.....	452
3.6.5.4 避免同地址访问.....	453
3.6.5.5 设置合理的 L2 CacheMode.....	457
3.6.5.6 算子与高阶 API 共享临时 Buffer.....	460

3.6.5.7 纯搬运类算子 VECIN 和 VECOUT 建议复用.....	461
3.6.5.8 通过缩减 Tensor ShapeInfo 维度, 优化栈空间.....	463
3.6.5.9 避免 Unified Buffer 的 bank 冲突.....	464
3.6.5.9.1 避免 bank 冲突 (NPU 架构版本 220x)	464
3.6.5.10 L2 Cache 切分.....	470
3.6.6 矢量计算.....	473
3.6.6.1 通过 Unified Buffer 融合实现连续 vector 计算.....	473
3.6.6.2 Vector 算子灵活运用 Counter 模式.....	475
3.6.6.3 选择低延迟指令, 优化归约操作性能.....	478
3.6.7 矩阵计算.....	482
3.6.7.1 通过 BT Buffer 实现高效的 bias 计算.....	482
3.6.7.2 通过 FP Buffer 存放量化参数实现高效随路量化.....	487
3.6.7.3 通过 LOC Buffer 数据暂存实现高效的矩阵乘结果累加.....	493
3.6.7.4 较小矩阵长驻 L1 Buffer, 仅分次搬运较大矩阵.....	498
3.6.7.5 Matmul 使能 AtomicAdd 选项.....	503
3.7 优秀实践.....	505
3.7.1 FlashAttention 算子性能调优案例.....	505
3.7.2 GroupedMatmul 算子性能调优案例.....	515
3.7.3 MC ² 算子性能调优案例.....	520
3.7.4 Matmul 性能调优案例.....	526
3.7.4.1 Matmul 性能优化策略总览.....	526
3.7.4.2 Matmul 算子优化 Tiling 策略.....	528
3.7.4.3 Matmul 高阶 API 使能纯 Cube 模式.....	531
3.7.4.4 Matmul 高阶 API 使能 MDL 模板.....	535
3.7.4.5 Matmul 高阶 API 使能 UnitFlag.....	539
3.7.4.6 Matmul 高阶 API 使能 Tiling 全量常量化.....	542
3.7.4.7 Matmul 高阶 API 使能 L2 Cache 切分.....	546
3.7.4.8 Matmul 高阶 API 使能多核切 K.....	551
3.7.4.9 Matmul 高阶 API 使能多核 K 轴错峰访问内存.....	554
3.7.4.10 Matmul 高阶 API 使能 NBuffer33 模板.....	559
3.7.4.11 Matmul 高阶 API 使能 IBShare 模板共享 B 矩阵数据.....	561
3.7.4.12 Matmul 高阶 API 使能 IBShare 模板共享 A 和 B 矩阵数据.....	564
3.7.4.13 AIV 核上的 ND2NZ 格式转换.....	568
3.7.4.14 Matmul 高阶 API 使能 MTE2 Preload.....	572

1 入门教程

1.1 什么是Ascend C

1.2 环境准备

1.3 快速入门

1.1 什么是 Ascend C

Ascend C是CANN针对算子开发场景推出的编程语言，原生支持C和C++标准规范，兼具开发效率和运行性能。基于Ascend C编写的算子程序，通过编译器编译和运行时调度，运行在昇腾AI处理器上。使用Ascend C，开发者可以基于昇腾AI硬件，高效的实现自定义的创新算法。您可以通过[Ascend C主页](#)了解更详细的内容。

Ascend C提供多层次API，满足多维场景算子开发诉求。

- **语言扩展层 C API**：开放芯片完备编程能力，支持数组分配内存，一般基于指针编程，提供与业界一致的C语言编程体验。
- **基础API**：基于Tensor进行编程的C++类库API，实现单指令级抽象，为底层算子开发提供灵活控制能力。
- **高阶API**：封装单核公共算法，涵盖一些常见的计算算法（如卷积、矩阵运算等），显著降低复杂算法开发门槛。
- **算子模板库**：基于模板提供算子完整实现参考，简化Tiling（切分算法）开发，支撑用户自定义扩展。
- **Python前端**：[PyAsc编程语言](#)基于Python原生接口，提供芯片底层完备编程能力，支持基于Python接口开发高性能Ascend C算子。

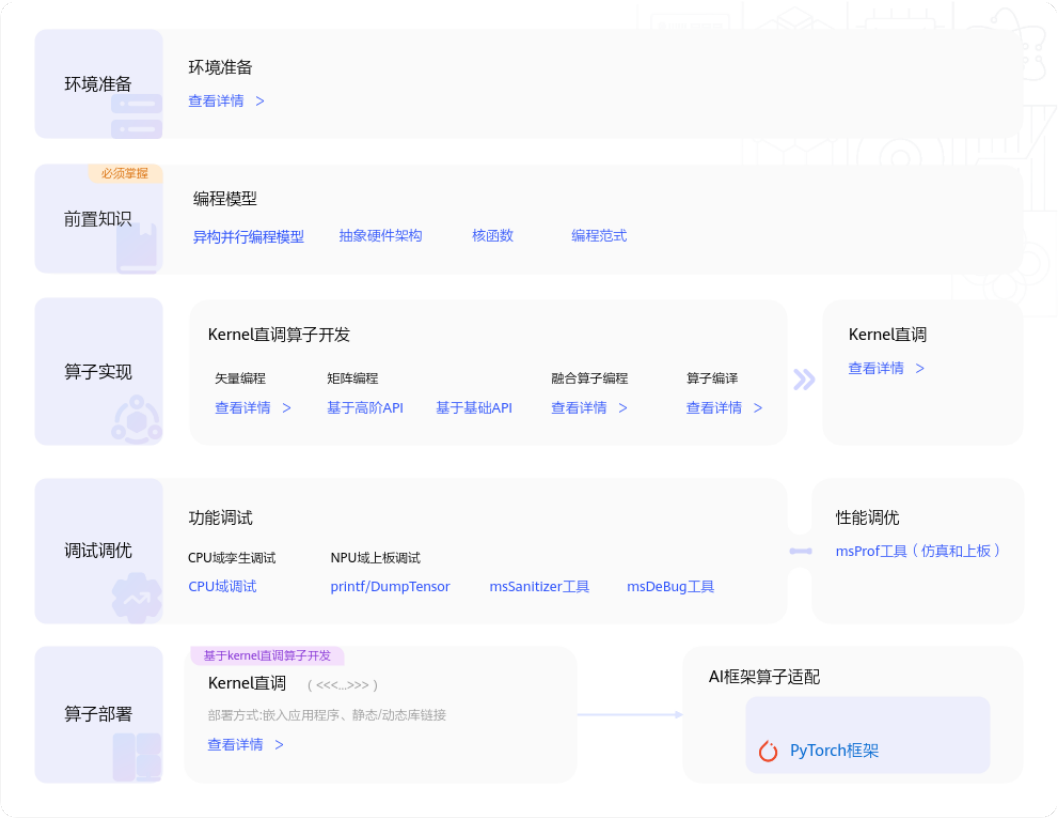


快速入门

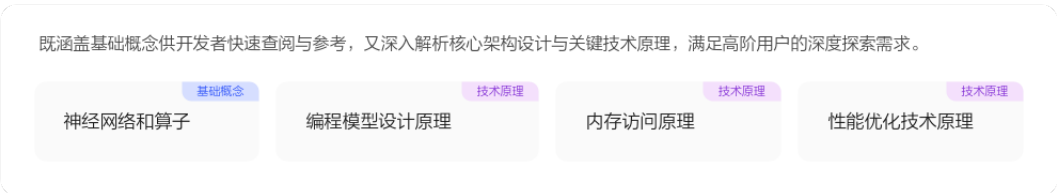
从一个简单的样例出发，带您体验Ascend C算子开发的基本流程。

[查看详情 >](#)

成长地图



概念原理



API 参考



算子实践参考

结合经验总结的性能优化手段和性能优化案例 [查看更多 >](#)


性能优化
建议


FlashAttention
算子性能调优案例


Matmul
算子性能调优案例


MC2
算子性能调优案例

Ascend C支持在如下AI处理器型号使用：

- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品
- Atlas 200I/500 A2 推理产品
- Atlas 推理系列产品
- Atlas 训练系列产品

1.2 环境准备

- 进行Ascend C算子开发之前，需要安装**驱动固件**和**CANN软件包**，请参考《[CANN 软件安装指南](#)》完成环境准备。
 - a. **安装驱动固件**（仅昇腾设备需要），安装步骤请参见“安装NPU驱动和固件”章节。
 - b. **安装CANN软件包**，可参考“安装CANN”完成快速安装，可参考其他章节了解更多场景的安装步骤。

📖 说明

安装CANN软件后，使用CANN运行用户进行编译、运行时，需要以CANN运行用户登录环境，执行`source ${INSTALL_DIR}/set_env.sh`命令设置环境变量。`${INSTALL_DIR}`请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：`/usr/local/Ascend/cann`。

- 安装cmake。通过cmake编译Ascend C算子时，要求安装3.16及以上版本的cmake，如果版本不符合要求，可以参考如下示例安装满足要求的版本。

示例：安装3.16.0版本的cmake（x86_64架构）。

```
mkdir -p cmake-3.16 && wget -qO- "https://cmake.org/files/v3.16/cmake-3.16.0-Linux-x86_64.tar.gz" |  
tar --strip-components=1 -xz -C cmake-3.16  
export PATH=`pwd`/cmake-3.16/bin:$PATH
```

📖 说明

对于Ascend C算子的开发，并非必须安装驱动固件。在非昇腾设备上，可以利用CPU仿真环境先行进行算子开发和测试，并在准备就绪后，利用昇腾设备进行加速计算。非昇腾设备的安装请参考《[CANN 软件安装指南](#)》中“附录B：常用操作 > 在非昇腾设备上安装CANN”章节。

1.3 快速入门

1.3.1 SIMD

1.3.1.1 HelloWorld

本示例展示了如何使用Ascend C编写一个简单的"Hello World"程序，包括核函数（设备侧实现的入口函数）的实现、调用流程以及编译运行的完整步骤。通过本示例，您可以快速了解Ascend C的基本开发流程。完整样例请参考[LINK](#)。

代码文件hello_world.asc包括核函数实现和主函数实现。

- 核函数实现：该核函数的核心逻辑是输出"Hello World!!!"字符串。
- 主函数实现：在主函数中，进行初始化环境、申请资源、通过<<<>>>调用核函数以及释放资源等操作。完整的代码流程和逻辑可以通过代码注释查看。

```
// Host侧应用程序需要包含的头文件
#include "acl/acl.h"
// Kernel侧需要包含的头文件
#include "kernel_operator.h"
__global__ __vector__ void hello_world()
{
    AscendC::printf("[Block (%lu/%lu)]: Hello World!!!\n", AscendC::GetBlockIdx(), AscendC::GetBlockNum());
}

int32_t main(int argc, char const *argv[])
{
    // 初始化
    aclInit(nullptr);
    // 运行管理资源申请
    int32_t deviceId = 0;
    aclrtSetDevice(deviceId);
    aclrtStream stream = nullptr;
    aclrtCreateStream(&stream);

    // 设置参与计算的核数为8（核数可根据实际需求设置）
    constexpr uint32_t numBlocks = 8;
    // 用内核调用符<<<>>>调用核函数
    hello_world<<<numBlocks, nullptr, stream>>>();
    aclrtSynchronizeStream(stream);
    // 资源释放和去初始化
    aclrtDestroyStream(stream);
    aclrtResetDevice(deviceId);
    aclFinalize();
    return 0;
}
```

完成代码实现后，可以通过两种方式，对上述代码进行编译：

- 使用**bisheng**命令行进行编译
bisheng hello_world.asc --npu-arch=dav-2201 -o demo
./demo

- 使用**CMake**进行编译

CMake编译配置如下：

```
cmake_minimum_required(VERSION 3.16)
# find_package(ASC)是CMake中用于查找和配置Ascend C编译工具链的命令
find_package(ASC REQUIRED)
# 指定项目支持的语言包括ASC和CXX，ASC表示支持使用毕昇编译器对Ascend C编程语言进行编译
project(kernel_samples LANGUAGES ASC CXX)
add_executable(demo
    hello_world.asc
)
# 通过编译选项设置NPU架构
target_compile_options(demo PRIVATE
```

```
$(<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-2201>  
)
```

编译和运行步骤如下：

```
mkdir -p build && cd build;  
cmake .;make -j;  
./demo
```

📖 说明

- 该样例仅支持如下型号：
 - Atlas A3 训练系列产品/Atlas A3 推理系列产品
 - Atlas A2 训练系列产品/Atlas A2 推理系列产品
- `--npu-arch`用于指定NPU的架构版本，dav-后为架构版本号，各AI处理器型号对应的架构版本号请通过[AI处理器型号和_NPU_ARCH_的对应关系](#)进行查询。

运行结果如下，本样例共调度8个核，打印了核号和"Hello World!!!"等信息。

```
[Block (0/8)]: Hello World!!!  
[Block (1/8)]: Hello World!!!  
[Block (2/8)]: Hello World!!!  
[Block (3/8)]: Hello World!!!  
[Block (4/8)]: Hello World!!!  
[Block (5/8)]: Hello World!!!  
[Block (6/8)]: Hello World!!!  
[Block (7/8)]: Hello World!!!
```

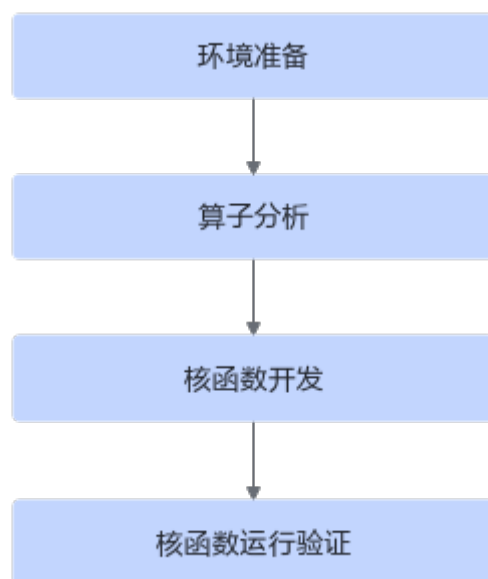
1.3.1.2 Add 自定义算子开发

本入门教程，将会引导你完成以下任务，体验Ascend C算子开发基本流程。

1. 算子分析，明确数学表达式和计算逻辑等内容；
2. Add算子核函数开发；
3. 算子核函数运行验证。

在正式的开发之前，还需要先完成环境准备工作，开发Ascend C算子的基本流程如下图所示：

图 1-1 开发 Ascend C 算子的基本流程



📖 说明

- 请点击[LINK](#)获取样例代码。
- 使用本教程只需要您具有一定的C/C++基础，在此基础上，如果您已经对Ascend C编程模型有一定的了解，您可以在实际操作的过程中加深对理论的理解；如果您还没有开始了解Ascend C编程模型，也无需担心，您可以先尝试跑通教程中的样例，参考教程最后的[指引](#)进行进一步的学习。

环境准备

- CANN软件安装
开发算子前，需要先准备好开发环境和运行环境，开发环境和运行环境的介绍和具体的安装步骤可参见《[CANN 软件安装指南](#)》。
- 环境变量配置
安装CANN软件后，使用CANN运行用户进行编译、运行时，需要以CANN运行用户登录环境，执行`source ${INSTALL_DIR}/set_env.sh`命令设置环境变量。`${INSTALL_DIR}`请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：`/usr/local/Ascend/cann`。

算子分析

主要分析算子的数学表达式、输入输出的数量、Shape范围以及计算逻辑的实现，明确需要调用的Ascend C接口。下文以Add算子为例，介绍具体的分析过程。

步骤1 明确算子的数学表达式及计算逻辑。

Add算子的数学表达式为：

$$\vec{z} = \vec{x} + \vec{y}$$

计算逻辑是：从外部存储Global Memory搬运数据至内部存储Local Memory，然后使用Ascend C计算接口完成两个输入参数相加，得到最终结果，再搬运到Global Memory上。

步骤2 明确输入和输出。

- Add算子有两个输入：`x`与`y`，输出为`z`。
- 本样例中算子输入支持的数据类型为float，算子输出的数据类型与输入数据类型相同。
- 算子输入支持的shape为（8，2048），输出shape与输入shape相同。
- 算子输入支持的format为：ND。

步骤3 确定核函数名称和参数。

- 本样例中核函数命名为`add_custom`。
- 根据对算子输入输出的分析，确定核函数有3个参数`x`，`y`，`z`；`x`，`y`为输入参数，`z`为输出参数。

步骤4 确定算子实现所需接口。

- 实现涉及外部存储和内部存储间的数据搬运，查看Ascend C API参考中的数据搬运接口，需要使用DataCopy来实现数据搬运。
- 本样例只涉及矢量计算的加法操作，查看Ascend C API参考中的矢量计算接口Memory矢量计算，初步分析可使用Add接口Add实现`x+y`。

- 计算中使用到的Tensor数据结构，使用AllocTensor、FreeTensor进行申请和释放。
- 并行流水任务之间使用Queue队列完成同步，会使用到EnQue、DeQue等接口。

----结束

通过以上分析，得到Ascend C Add算子的设计规格如下：

表 1-1 Ascend C Add 算子设计规格

算子类型 (OpType)	AddCustom			
算子输入	name	shape	data type	format
	x	(8, 2048)	float	ND
	y	(8, 2048)	float	ND
算子输出	z	(8, 2048)	float	ND
核函数名称	add_custom			
使用的主要接口	DataCopy：数据搬运接口			
	Add：矢量基础算术接口			
	AllocTensor、FreeTensor：内存管理接口			
	EnQue、DeQue接口：Queue队列管理接口			
算子实现文件名称	add_custom.asc			

核函数开发

完成环境准备和初步的算子分析后，即可开始Ascend C核函数的开发。开发之前请先从[LINK](#)获取样例代码，以下样例代码在add_custom.asc中实现。

本样例中使用多核并行计算，即把数据进行分片，分配到多个核上进行处理。Ascend C核函数是在一个核上的处理函数，所以只处理部分数据。分配方案是：假设共启用8个核，数据整体长度为8 * 2048个元素，平均分配到8个核上运行，每个核上处理的数据大小为2048个元素。对于单核上的处理数据，也可以进行数据切块，实现对数据的流水并行处理。

步骤1 根据分配方案设计一个结构体AddCustomTilingData，用于保存并行数据切分相关的参数。AddCustomTilingData定义了两个参数：totalLength、tileNum。totalLength指待处理的数据总大小为（8 * 2048）个元素，tileNum指每个核需要计算的数据块个数。

```
struct AddCustomTilingData
{
    uint32_t totalLength;
    uint32_t tileNum;
};
```

步骤2 根据[核函数定义和调用](#)中介绍的规则进行核函数的定义，并在核函数中调用算子类的Init和Process函数，算子类实现在后续步骤中介绍。

```
__global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData tiling)
{
    KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_AIV_ONLY); // 设置Kernel类型为Vector核（用于矢量计算）
    KernelAdd op;
    op.Init(x, y, z, tiling.totalLength, tiling.tileNum);
    op.Process();
}
```

- 使用__global__函数类型限定符来标识它是一个核函数，可以被<<<>>>调用；使用__aicore__函数类型限定符来标识该核函数在设备端AI Core上执行。指针入参变量需要增加变量类型限定符__gm__，表明该指针变量指向Global Memory上某处内存地址。为了统一表达，使用GM_ADDR宏来修饰入参，GM_ADDR宏定义如下：

```
#define GM_ADDR __gm__ uint8_t*
```

- 算子类的Init函数，完成内存初始化相关工作，Process函数完成算子实现的核心逻辑。

步骤3 根据矢量编程范式实现算子类，本样例中定义KernelAdd算子类，其具体成员如下：

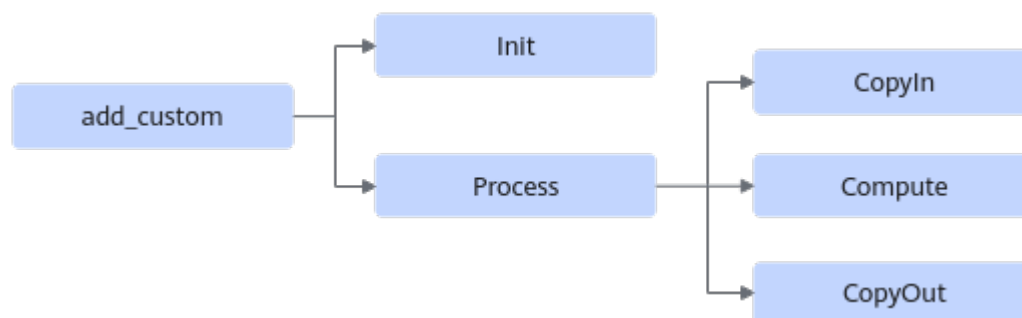
```
class KernelAdd {
public:
    __aicore__ inline KernelAdd(){}
    // 初始化函数，完成内存初始化相关操作
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z, uint32_t totalLength, uint32_t tileNum)
{}
    // 核心处理函数，实现算子逻辑，调用私有成员函数CopyIn、Compute、CopyOut完成矢量算子的三级流水操作
    __aicore__ inline void Process(){}

private:
    // 搬入函数，从Global Memory搬运数据至Local Memory，被核心Process函数调用
    __aicore__ inline void CopyIn(int32_t progress){}
    // 计算函数，完成两个输入参数相加，得到最终结果，被核心Process函数调用
    __aicore__ inline void Compute(int32_t progress){}
    // 搬出函数，将最终结果从Local Memory搬运到Global Memory上，被核心Process函数调用
    __aicore__ inline void CopyOut(int32_t progress){}

private:
    AscendC::TPipe pipe; // TPipe内存管理对象
    AscendC::TQue<AscendC::TPosition::VECIN, BUFFER_NUM> inQueueX, inQueueY; // 输入数据Queue队列管理对象，TPosition为VECIN
    AscendC::TQue<AscendC::TPosition::VECOUT, BUFFER_NUM> outQueueZ; // 输出数据Queue队列管理对象，TPosition为VECOUT
    AscendC::GlobalTensor<float> xGm; // 管理输入输出Global Memory内存地址的对象，其中xGm, yGm为输入，zGm为输出
    AscendC::GlobalTensor<float> yGm;
    AscendC::GlobalTensor<float> zGm;
    uint32_t blockLength; // 每个核的计算数据长度
    uint32_t tileNum; // 每个核需要计算的数据块个数
    uint32_t tileLength; // 每个核内每个数据块的长度
};
```

内部函数的调用关系示意图如下：

图 1-2 核函数调用关系图



由此可见除了Init函数完成初始化外，Process中完成了对流水任务“搬入、计算、搬出”的调用，开发者可以重点关注三个流水任务的实现。

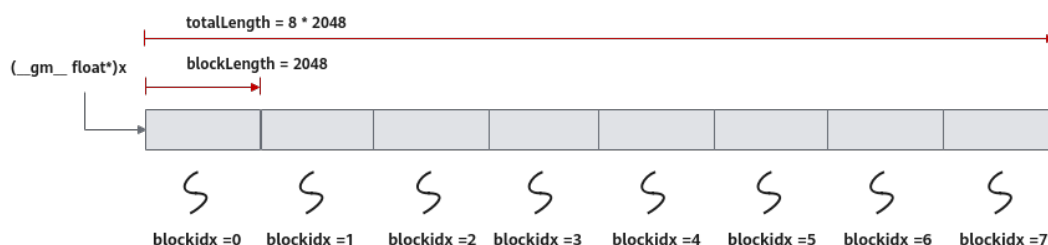
步骤4 初始化函数Init主要完成以下内容：设置输入输出Global Tensor的Global Memory内存地址，通过TPipe内存管理对象为输入输出Queue分配内存。

上文我们介绍到，本样例将数据切分成8块，平均分配到8个核上运行，每个核上处理的数据大小为2048个元素。那么我们是如何实现这种切分的呢？

每个核上处理的数据地址需要在起始地址上增加GetBlockIdx() * blockLength（每个block处理的数据长度）的偏移来获取。这样也就实现了多核并行计算的数据切分。

以输入x为例， $x + \text{blockLength} * \text{GetBlockIdx}()$ 即为单核处理程序中x在Global Memory上的内存偏移地址，获取偏移地址后，使用GlobalTensor类的SetGlobalBuffer接口设定该核上Global Memory的起始地址以及长度。具体示意图如下。

图 1-3 多核并行处理示意图

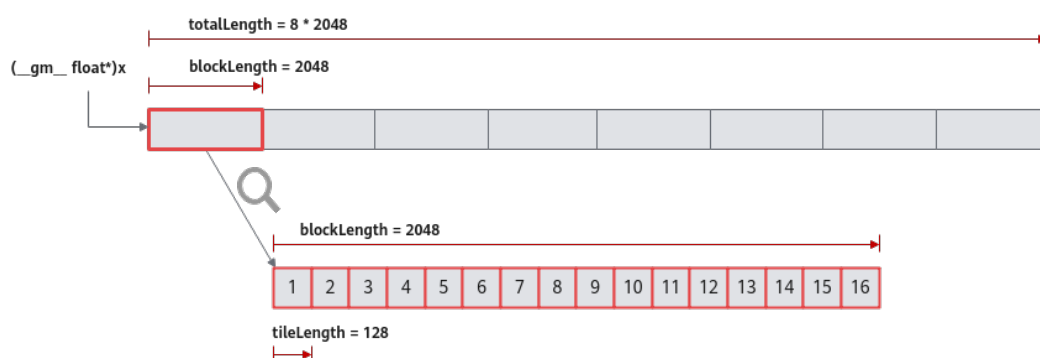


上面已经实现了多核数据的切分，那么单核上的处理数据如何进行切分？

对于单核上的处理数据，可以进行数据切块（Tiling），在本示例中，仅作为参考，将数据切分成8块（并不意味着8块就是性能最优）。切分后的每个数据块再次切分成2块，即可开启double buffer，实现流水线之间的并行。

这样单核上的数据（2048个数）被切分成16块，每块tileLength（128）个数据。TPipe为inQueueX分配了两块大小为tileLength * sizeof(float)个字节的内存块，每个内存块能容纳tileLength（128）个float类型数据。数据切分示意图如下。

图 1-4 单核数据切分示意图



具体的初始化函数代码如下：

```
// Kernel侧所需头文件
#include "kernel_operator.h"

constexpr int32_t BUFFER_NUM = 2; // tensor num for each queue
```



```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z, uint32_t totalLength, uint32_t tileNum)
{
    this->blockLength = totalLength / AscendC::GetBlockNum(); // length computed of each core
    this->tileNum = tileNum; // split data into 8 tiles for each core
    this->tileLength = this->blockLength / tileNum / BUFFER_NUM; // separate to 2 parts, due to double
    buffer
    // get start index for current core, core parallel
    xGm.SetGlobalBuffer((__gm__ float *)x + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
    yGm.SetGlobalBuffer((__gm__ float *)y + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
    zGm.SetGlobalBuffer((__gm__ float *)z + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
    // pipe alloc memory to queue, the unit is Bytes
    pipe.InitBuffer(inQueueX, BUFFER_NUM, this->tileLength * sizeof(float));
    pipe.InitBuffer(inQueueY, BUFFER_NUM, this->tileLength * sizeof(float));
    pipe.InitBuffer(outQueueZ, BUFFER_NUM, this->tileLength * sizeof(float));
}
```

步骤5 基于矢量编程范式，将核函数的实现分为3个基本任务：CopyIn，Compute，CopyOut。**Process**函数中通过如下方式调用这三个函数。

```
__aicore__ inline void Process()
{
    // loop count need to be doubled, due to double buffer
    int32_t loopCount = this->tileNum * BUFFER_NUM;
    // tiling strategy, pipeline parallel
    for (int32_t i = 0; i < loopCount; i++) {
        CopyIn(i);
        Compute(i);
        CopyOut(i);
    }
}
```

1. CopyIn函数实现。

- 使用DataCopy接口将GlobalTensor数据拷贝到LocalTensor。
- 使用EnQue将LocalTensor放入VecIn的Queue中。

```
__aicore__ inline void CopyIn( int32_t progress)
{
    // alloc tensor from queue memory
    AscendC::LocalTensor<float> xLocal = inQueueX.AllocTensor<float>();
    AscendC::LocalTensor<float> yLocal = inQueueY.AllocTensor<float>();
    // copy progress_th tile from global tensor to local tensor
    AscendC::DataCopy(xLocal, xGm[progress * this->tileLength], this->tileLength);
    AscendC::DataCopy(yLocal, yGm[progress * this->tileLength], this->tileLength);
    // enqueue input tensors to VECIN queue
    inQueueX.EnQue(xLocal);
    inQueueY.EnQue(yLocal);
}
```

2. Compute函数实现。

- 使用DeQue从VecIn中取出LocalTensor。
- 使用Ascend C接口Add完成矢量计算。
- 使用EnQue将计算结果LocalTensor放入到VecOut的Queue中。
- 使用FreeTensor将释放不再使用的LocalTensor。

```
__aicore__ inline void Compute(int32_t progress)
{
    // deque input tensors from VECIN queue
    AscendC::LocalTensor<float> xLocal = inQueueX.DeQue<float>();
    AscendC::LocalTensor<float> yLocal = inQueueY.DeQue<float>();
    AscendC::LocalTensor<float> zLocal = outQueueZ.AllocTensor<float>();
    // call Add instr for computation
    AscendC::Add(zLocal, xLocal, yLocal, this->tileLength);
    // enqueue the output tensor to VECOUT queue
    outQueueZ.EnQue<float>(zLocal);
    // free input tensors for reuse
}
```



```
inQueueX.FreeTensor(xLocal);  
inQueueY.FreeTensor(yLocal);  
}
```

3. CopyOut函数实现。

- 使用DeQue接口从VecOut的Queue中取出LocalTensor。
- 使用DataCopy接口将LocalTensor拷贝到GlobalTensor上。
- 使用FreeTensor将不再使用的LocalTensor进行回收。

```
__aicore__ inline void CopyOut(int32_t progress)  
{  
    // deque output tensor from VECOUT queue  
    AscendC::LocalTensor<float> zLocal = outQueueZ.DeQue<float>();  
    // copy progress_th tile from local tensor to global tensor  
    AscendC::DataCopy(zGm[progress * this->tileLength], zLocal, this->tileLength);  
    // free output tensor for reuse  
    outQueueZ.FreeTensor(zLocal);  
}
```

----结束

核函数运行验证

完成Kernel侧核函数开发后，即可编写Host侧的核函数调用程序。实现从Host侧的APP程序调用算子，执行计算过程。

步骤1 Host侧应用程序框架的编写。

```
// Host侧应用程序需要包含的头文件  
#include "acl/acl.h"  
// Kernel侧需要包含的头文件  
#include "kernel_operator.h"  
// 核函数开发部分  
...  
  
__global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData tiling)  
{  
    KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_AIV_ONLY);  
    KernelAdd op;  
    op.Init(x, y, z, tiling.totalLength, tiling.tileNum);  
    op.Process();  
}  
  
// 通过<<<...>>>内核调用符调用算子  
std::vector<float> kernel_add(std::vector<float> &x, std::vector<float> &y)  
{  
    ...  
}  
  
// 计算结果比对  
uint32_t VerifyResult(std::vector<float> &output, std::vector<float> &golden)  
{  
    auto printTensor = [](std::vector<float> &tensor, const char *name) {  
        constexpr size_t maxPrintSize = 20;  
        std::cout << name << ": ";  
        std::copy(tensor.begin(), tensor.begin() + std::min(tensor.size(), maxPrintSize),  
            std::ostream_iterator<float>(std::cout, " "));  
        if (tensor.size() > maxPrintSize) {  
            std::cout << "...";  
        }  
        std::cout << std::endl;  
    };  
    printTensor(output, "Output");  
    printTensor(golden, "Golden");  
    if (std::equal(golden.begin(), golden.end(), output.begin())) {  
        std::cout << "[Success] Case accuracy is verification passed." << std::endl;  
    }  
}
```

```
        return 0;
    } else {
        std::cout << "[Failed] Case accuracy is verification failed!" << std::endl;
        return 1;
    }
    return 0;
}

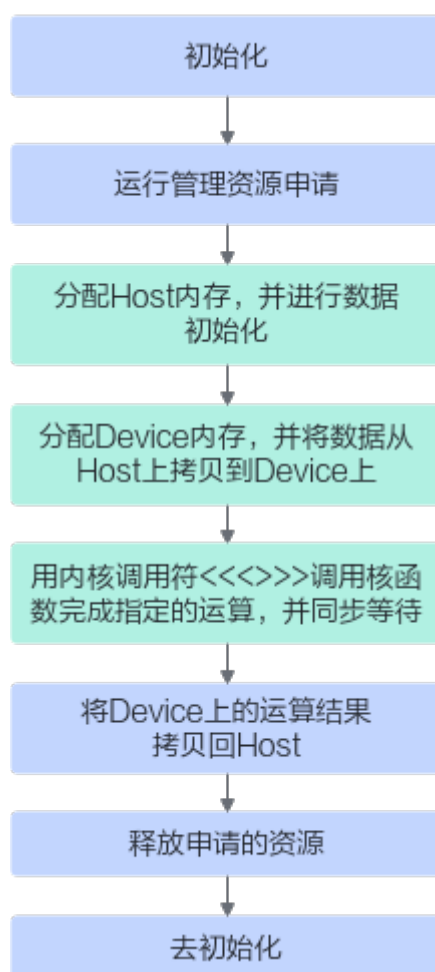
// 算子验证主程序
int32_t main(int32_t argc, char *argv[])
{
    constexpr uint32_t totalLength = 8 * 2048;
    constexpr float valueX = 1.2f;
    constexpr float valueY = 2.3f;
    std::vector<float> x(totalLength, valueX);
    std::vector<float> y(totalLength, valueY);

    std::vector<float> output = kernel_add(x, y);

    std::vector<float> golden(totalLength, valueX + valueY);
    return VerifyResult(output, golden);
}
```

步骤2 编写通过<<<...>>>内核调用符调用算子的代码。

图 1-5 调用步骤



如下示例中的acl API使用方法请参考“acl API（C&C++）”章节。

```
std::vector<float> kernel_add(std::vector<float> &x, std::vector<float> &y)
{
    constexpr uint32_t numBlocks = 8;
    uint32_t totalLength = x.size();
    size_t totalByteSize = totalLength * sizeof(float);
    int32_t deviceId = 0;
    aclrtStream stream = nullptr;
    AddCustomTilingData tiling = { /*totalLength:*/totalLength, /*tileNum:*/8};
    uint8_t *xHost = reinterpret_cast<uint8_t *>(x.data());
    uint8_t *yHost = reinterpret_cast<uint8_t *>(y.data());
    uint8_t *zHost = nullptr;
    uint8_t *xDevice = nullptr;
    uint8_t *yDevice = nullptr;
    uint8_t *zDevice = nullptr;

    // 初始化
    aclInit(nullptr);
    // 运行管理资源申请
    aclrtSetDevice(deviceId);
    aclrtCreateStream(&stream);
    // 分配Host内存
    aclrtMallocHost((void **)&zHost, totalByteSize);
    // 分配Device内存
    aclrtMalloc((void **)&xDevice, totalByteSize, ACL_MEM_MALLOC_HUGE_FIRST);
    aclrtMalloc((void **)&yDevice, totalByteSize, ACL_MEM_MALLOC_HUGE_FIRST);
    aclrtMalloc((void **)&zDevice, totalByteSize, ACL_MEM_MALLOC_HUGE_FIRST);
    // 将Host上的输入数据拷贝到Device侧
    aclrtMemcpy(xDevice, totalByteSize, xHost, totalByteSize, ACL_MEMCPY_HOST_TO_DEVICE);
    aclrtMemcpy(yDevice, totalByteSize, yHost, totalByteSize, ACL_MEMCPY_HOST_TO_DEVICE);
    // 用内核调用符<<<...>>>调用核函数完成指定的运算
    add_custom<<<numBlocks, nullptr, stream>>>(xDevice, yDevice, zDevice, tiling);
    aclrtSynchronizeStream(stream);
    // 将Device上的运算结果拷贝回Host
    aclrtMemcpy(zHost, totalByteSize, zDevice, totalByteSize, ACL_MEMCPY_DEVICE_TO_HOST);
    std::vector<float> z((float *)zHost, (float *)zHost + totalByteSize);
    // 释放申请的资源
    aclrtFree(xDevice);
    aclrtFree(yDevice);
    aclrtFree(zDevice);
    aclrtFreeHost(zHost);
    // 去初始化
    aclrtDestroyStream(stream);
    aclrtResetDevice(deviceId);
    aclFinalize();
    return z;
}
```

步骤3 CMake编译配置如下:

```
cmake_minimum_required(VERSION 3.16)
# find_package(ASC)是CMake中用于查找和配置Ascend C编译工具链的命令
find_package(ASC REQUIRED)
# 指定项目支持的语言包括ASC和CXX, ASC表示支持使用昇腾编译器对Ascend C编程语言进行编译
project(kernel_samples LANGUAGES ASC CXX)

add_executable(demo
    add_custom.asc
)

# 通过编译选项设置NPU架构
target_compile_options(demo PRIVATE
    $<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-2201>
)
```

步骤4 编译和运行步骤如下

```
mkdir -p build && cd build;
cmake ..;make -j;
./demo
```

说明

- 该样例仅支持如下型号：
 - Atlas A3 训练系列产品/Atlas A3 推理系列产品
 - Atlas A2 训练系列产品/Atlas A2 推理系列产品
- `--npu-arch`用于指定NPU的架构版本，`dev-`后为架构版本号，各AI处理器型号对应的架构版本号请通过[AI处理器型号和_NPU_ARCH_的对应关系](#)进行查询。

----结束

接下来的引导

如果您对教程中的多核并行、流水编程等概念不了解，导致阅读过程有些吃力，可以参考[2.2 编程模型](#)学习基本概念，再来回顾本教程；如果您已经了解相关概念，并跑通了该样例，您可以参考[3.3.2 向量编程](#)了解Ascend C向量编程中的更多细节。

2 编程指南

2.1 本文档组织结构

2.2 编程模型

2.3 编译与运行

2.4 语言扩展层

2.5 C++类库API

2.6 硬件实现

2.7 调试调优

2.8 兼容性指南

2.9 概念原理和术语

2.10 附录

2.1 本文档组织结构

本指南作为AscendC 算子开发综合性技术文档，详细阐述编程模型、语言扩展特性、C++类库API、硬件实现、以及调试调优等。无论是入门开发者还是资深工程师，均能通过本指南充分掌握如何高效释放昇腾AI处理器的完整计算潜能。

本文档的组织结构和各章内容如下：

- **编程模型**：介绍Ascend C编程模型和编程范式。
- **编译与运行**：介绍如何编译Ascend C算子，以及算子运行需要的常用接口。
- **语言扩展层**：介绍语言扩展层特性，包括内置变量和函数、以及Ascend C扩展的C API。
- **C++类库API**：介绍Ascend C多层次类库API，包括单指令封装抽象的基础API、基于单核公共算法抽象的高阶API等。
- **硬件实现**：介绍Ascend C硬件实现，包括昇腾AI处理器的硬件架构、硬件规格和使用约束。
- **调试调优**：介绍AscendC算子如何进行功能调试和性能调优。

2.2 编程模型

2.2.1 异构并行编程模型

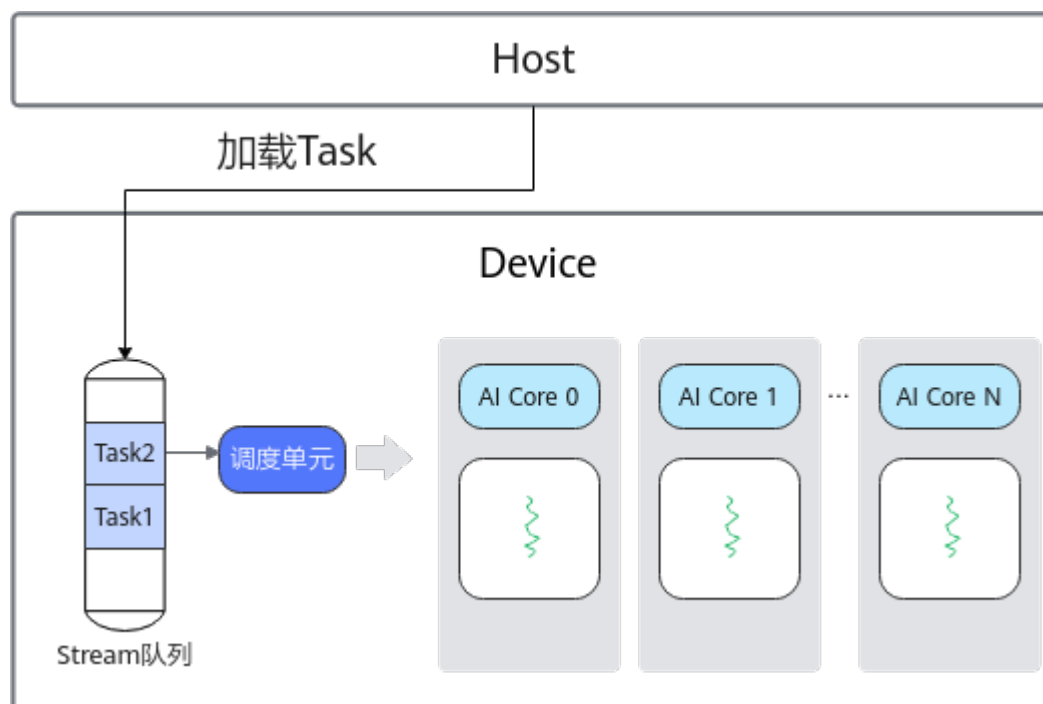
Host-Device 异构协同机制

Ascend C异构并行编程模型是为应对异构计算架构的挑战而设计的，旨在解决传统编程模型在处理复杂计算任务时的效率和可扩展性问题。

异构计算架构分为Host侧和Device侧（Device侧对应AI处理器），两者协同完成计算任务。Host侧主要负责运行时管理，包括存储管理、设备管理以及Stream管理等，确保任务的高效调度与资源的合理分配；Device侧，会执行开发者基于Ascend C语法编写的**Kernel核函数**，主要完成批量数据的矩阵运算、向量运算等计算密集型的任务，用于计算加速。

如下图所示，当一个Kernel下发到AI Core（AI处理器的计算核心）上执行时，运行时管理模块根据开发者设置的核数和任务类型启动对应的Task，该Task从Host加载到Device的Stream运行队列，调度单元会把就绪的Task分配到空闲AI Core上执行。这里将需要处理的数据拆分并同时在多个计算核心上运行的方式（也就是下文介绍的**SPMD并行计算**），可以获取更高的性能。

图 2-1 Kernel 调度示意图



Host和Device拥有不同的内存空间，Host无法直接访问Device内存，反之亦然。所以，输入数据需要从Host侧拷贝至Device侧内存空间，供Device侧进行计算，输出结果需要从Device侧内存拷贝回Host侧，便于在Host侧继续使用。

说明

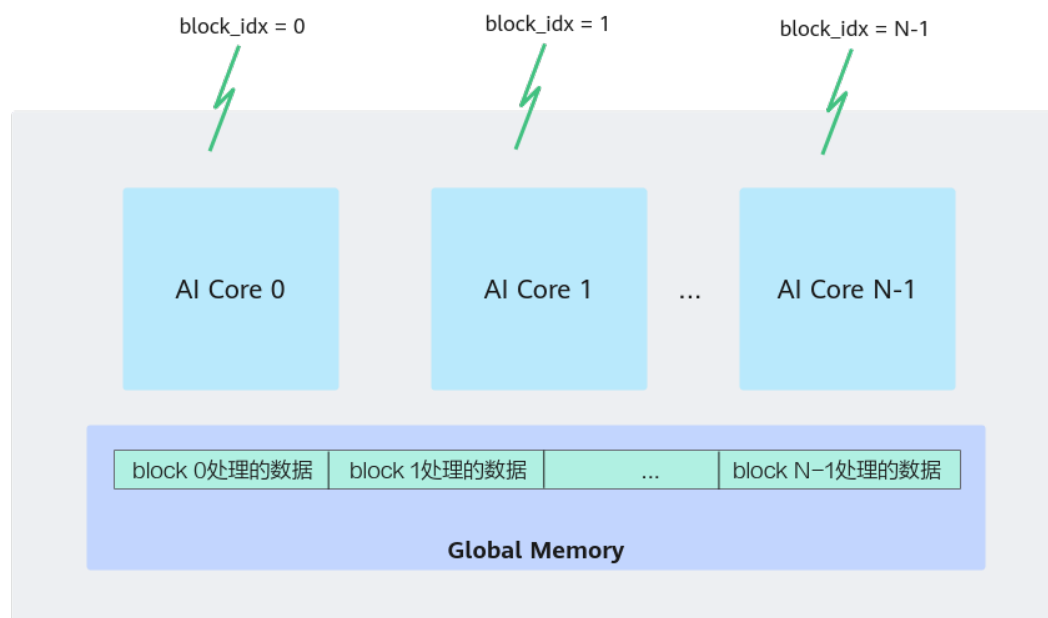
关于运行时管理的详细介绍请参考“运行时管理”章节。

SPMD 并行计算

Ascend C算子编程是SPMD（Single-Program Multiple-Data）编程，通俗来讲就是“一份代码，多处执行，处理的数据不同”。SPMD是一种常用的并行计算的方法，是提高计算速度的有效手段。

具体到Ascend C编程模型中的应用，是将需要处理的数据拆分并同时在多个计算核心（类比为上文介绍中的多个进程）上运行，从而获取更高的性能。多个AI Core共享相同的指令代码，每个核上的运行实例唯一的区别是block_idx不同，每个核通过不同的block_idx来识别自己的身份。block的概念类似于上文中进程的概念，block_idx就是标识进程唯一性的进程ID。并行计算过程如下图所示。

图 2-2 SPMD 并行计算示意图



下面的代码片段取自于Ascend C Add算子的实现代码，算子被调用时，所有的计算核心都执行相同的实现代码，入口函数的入参也是相同的。每个核上处理的数据地址需要在起始地址上增加**GetBlockIdx()*BLOCK_LENGTH**（每个核处理的数据长度）的偏移来获取。这样也就实现了多核并行计算的数据切分。代码中GetBlockIdx接口用于获取每个核的block_idx。

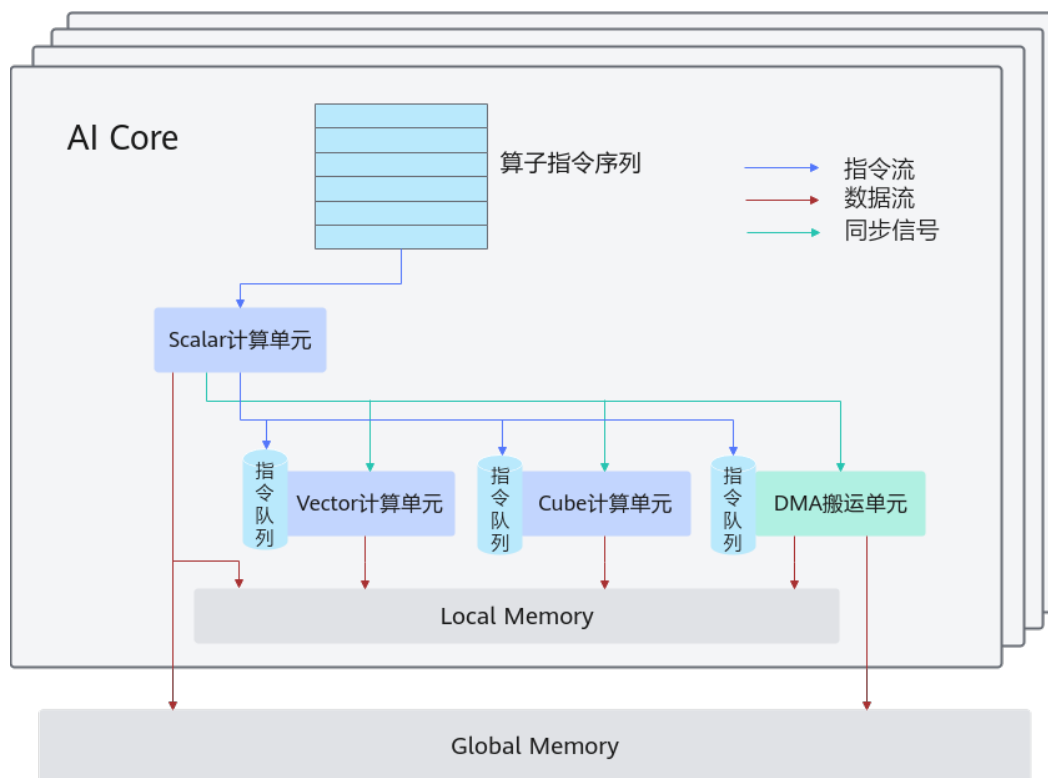
```
class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z)
    {
        // 不同核根据各自的block_idx设置数据地址
        xGm.SetGlobalBuffer((__gm__ half*)x + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);
        yGm.SetGlobalBuffer((__gm__ half*)y + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);
        zGm.SetGlobalBuffer((__gm__ half*)z + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);
        // Queue初始化，单位为字节
        pipe.InitBuffer(inQueueX, BUFFER_NUM, TILE_LENGTH * sizeof(half));
        pipe.InitBuffer(inQueueY, BUFFER_NUM, TILE_LENGTH * sizeof(half));
        pipe.InitBuffer(outQueueZ, BUFFER_NUM, TILE_LENGTH * sizeof(half));
    }
}
```

```
...  
}  
  
// 实现核函数  
__global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z)  
{  
    // 初始化算子类，算子类提供算子初始化和核心处理等方法  
    KernelAdd op;  
    // 初始化函数，获取该核函数需要处理的输入输出地址，同时完成必要的内存初始化工作  
    op.Init(x, y, z);  
    // 核心处理函数，完成算子的数据搬运与计算等核心逻辑  
    op.Process();  
}
```

2.2.2 抽象硬件架构

AI Core是AI处理器的计算核心，AI处理器内部有多个AI Core。本章节将介绍AI Core的并行计算架构抽象，该抽象架构屏蔽了不同硬件之间的差异。使用Ascend C进行编程时，基于抽象硬件架构，可以简化硬件细节，显著降低开发门槛。如需了解更详细的硬件架构信息或者原理，请参考[2.6 硬件实现](#)。

图 2-3 抽象硬件架构



AI Core中包含**计算单元**、**存储单元**、**搬运单元**等核心组件。

- 计算单元包括了三种基础计算资源：Cube计算单元、Vector计算单元和Scalar计算单元。
- 存储单元包括内部存储和外部存储：
 - AI Core的内部存储，统称为Local Memory，对应的数据类型为LocalTensor。

- AI Core能够访问的外部存储称之为Global Memory，对应的数据类型为GlobalTensor。
- DMA (Direct Memory Access) 搬运单元：负责数据搬运，包括Global Memory和Local Memory之间的数据搬运，以及不同层级Local Memory之间的数据搬运。

AI Core内部核心组件及组件功能详细说明如下表。

表 2-1 AI Core 内部核心组件

组件分类	组件名称	组件功能
计算单元	Scalar	执行地址计算、循环控制等标量计算工作，并把向量计算、矩阵计算、数据搬运、同步指令发射给对应单元执行。
	Vector	负责执行向量运算。
	Cube	负责执行矩阵运算。
存储单元	Local Memory	AI Core的内部存储。
搬运单元	DMA (Direct Memory Access)	负责数据搬运，包括Global Memory和Local Memory之间的数据搬运以及不同层级Local Memory之间的数据搬运。

开发者在理解硬件架构的抽象时，需要重点关注如下**异步指令流**、**同步信号流**、**计算数据流**三个过程：

- AI Core内部的异步并行计算过程：Scalar计算单元读取指令序列，并把向量计算、矩阵计算、数据搬运指令发射给对应单元的指令队列，向量计算单元、矩阵计算单元、数据搬运单元异步的并行执行接收到的指令。该过程可以参考图1中蓝色箭头所示的指令流。
- 不同的指令间有可能存在依赖关系，为了保证不同指令队列间的指令按照正确的逻辑关系执行，Scalar计算单元也会给对应单元下发同步指令。各单元之间的同步过程可以参考图1中的绿色箭头所示的同步信号流。
- AI Core内部数据处理的基本过程：DMA搬入单元将数据从Global Memory搬运到Local Memory，Vector/Cube计算单元完成数据计算，并把计算结果写回Local Memory，DMA搬出单元把处理好的数据从Local Memory搬运回Global Memory。该过程可以参考图1中的红色箭头所示的数据流。

2.2.3 核函数

核函数 (Kernel Function) 是Ascend C算子设备侧实现的入口。Ascend C允许用户使用C/C++函数的语法扩展来编写设备端的运行代码，用户在核函数中进行数据访问和计算操作，由此实现该算子的所有功能。区别于普通的C++函数调用时仅执行一次，当核函数被调用时，多个核都执行相同的核函数代码，具有相同的函数入参，并行执行。

核函数定义时需要使用函数类型限定符 `_global_` 和 `_aicore_`；其指针入参变量需要增加变量类型限定符 `_gm_`，表明该指针变量指向Global Memory上某处内存地址；使用`<<<...>>>`内核调用符调用执行核函数，并指定调用时的执行核数。

以下是一个Add算子的核函数示例（代码片段）。

```
// 实现核函数
__global__ __aicore__ void add_custom(__gm__ uint8_t* x, __gm__ uint8_t* y, __gm__ uint8_t* z)
{
    // 初始化算子类，算子类提供算子初始化和核心处理等方法
    KernelAdd op;
    // 初始化函数，获取该核函数需要处理的输入输出地址，同时完成必要的内存初始化工作
    op.Init(x, y, z);
    // 核心处理函数，完成算子的数据搬运与计算等核心逻辑
    op.Process();
}

// 调用核函数
void add_custom_do(uint32_t numBlocks, void* l2ctrl, void* stream, uint8_t* x, uint8_t* y, uint8_t* z)
{
    add_custom<<<numBlocks, l2ctrl, stream>>>(x, y, z);
}
```

核函数定义和调用

定义核函数时需要遵循以下规则。

- **使用函数类型限定符**

除了需要按照C/C++函数声明的方式定义核函数之外，还要为核函数加上额外的函数类型限定符，包含__global__和__aicore__。

使用__global__函数类型限定符来标识它是一个核函数，可以被<<<...>>>调用；

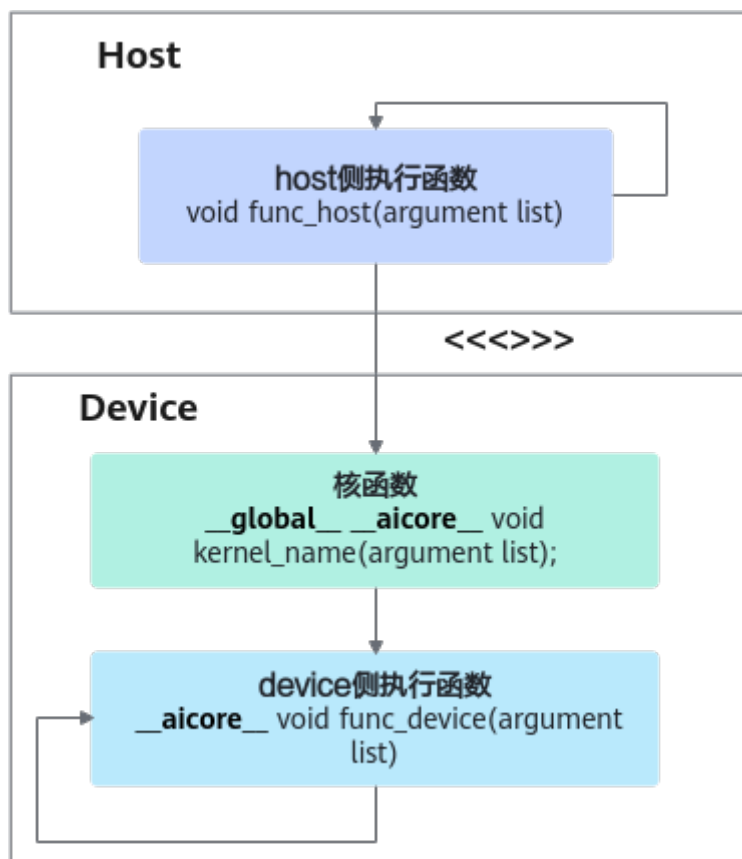
使用__aicore__函数类型限定符来标识该核函数在设备端AI Core上执行：

```
__global__ __aicore__ void kernel_name(argument list);
```

编程中使用到的函数可以分为三类：核函数（device侧执行）、host侧执行函数、device侧执行函数（除核函数之外）。下图以Kernel直调算子开发方式为例描述三者的调用关系：

- host侧执行函数可以调用同类的host执行函数，也就是通用C/C++编程中的函数调用；也可以通过<<<...>>>调用核函数。
- device侧执行函数（除核函数之外）可以调用同类的device侧执行函数。
- 核函数可以调用device侧执行函数（除核函数之外）。

图 2-4 核函数、host 侧执行函数、device 侧执行函数调用关系



- **使用变量类型限定符**

指针入参变量需要增加变量类型限定符 `_gm_`，表明该指针变量指向 Global Memory 上某处内存地址。

- **其他规则或建议**

- 规则：核函数必须具有 `void` 返回类型。
- 规则：仅支持入参为指针或 C/C++ 内置数据类型（Primitive data types），如：`half* s0`、`float* s1`、`int32_t c`。
- 建议：为了统一表达，建议使用 `GM_ADDR` 宏来修饰入参，`GM_ADDR` 宏定义如下：

```
#define GM_ADDR __gm__ uint8_t*
```

使用 `GM_ADDR` 修饰入参的样例如下：

```
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z)
```

这里统一使用 `uint8_t` 类型的指针，在后续的使用中需要将其转化为实际的指针类型。

常见的函数调用方式是如下的形式：

```
function_name(argument list);
```

核函数使用内核调用符 `<<<...>>>` 这种语法形式，来规定核函数的执行配置：

```
kernel_name<<<numBlocks, l2ctrl, stream>>>(argument list);
```

内核调用符仅可在 NPU 侧编译时调用，CPU 侧编译无法识别该符号。

执行配置由 3 个参数决定：

- numBlocks, 规定了核函数将会在几个核上执行。每个执行该核函数的核会被分配一个逻辑ID, 即block_idx, 可以在核函数的实现中调用GetBlockIdx来获取block_idx;

说明

numBlocks是逻辑核的概念, 取值范围为[1,65535]。为了充分利用硬件资源, 一般设置为物理核的核数或其倍数。

- 对于耦合模式和分离模式, numBlocks在运行时的意义和设置规则有一些区别, 具体说明如下:
 - 耦合模式: 由于其Vector、Cube单元是集成在一起的, numBlocks用于设置启动多个AI Core核实例执行, 不区分Vector、Cube。AI Core的核数可以通过GetCoreNumAiv或者GetCoreNumAic获取。
 - 分离模式
 - 针对仅包含Vector计算的算子, numBlocks用于设置启动多少个Vector (AIV) 实例执行, 比如某款AI处理器上有40个Vector核, 建议设置为40。
 - 针对仅包含Cube计算的算子, numBlocks用于设置启动多少个Cube (AIC) 实例执行, 比如某款AI处理器上有20个Cube核, 建议设置为20。
 - 针对Vector/Cube融合计算的算子, 启动时, 按照AIV和AIC组合启动, numBlocks用于设置启动多少个组合执行, 比如某款AI处理器上有40个Vector核和20个Cube核, 一个组合是2个Vector核和1个Cube核, 建议设置为20, 此时会启动20个组合, 即40个Vector核和20个Cube核。**注意: 该场景下, 设置的numBlocks逻辑核的核数不能超过物理核 (2个Vector核和1个Cube核组合为1个物理核) 的核数。**
 - AIC/AIV的核数分别通过GetCoreNumAic和GetCoreNumAiv接口获取。
- 如果开发者使用了Device资源限制特性, 那么算子设置的numBlocks不应超过PlatformAscendC提供核数的API (GetCoreNum/GetCoreNumAic/GetCoreNumAiv等) 返回的核数。例如, 使用aclrtSetStreamResLimit设置Stream级别的Vector核数为8, 那么GetCoreNumAiv接口返回值为8, 针对Vector算子设置的numBlocks不应超过8, 否则会抢占其他Stream的资源, 导致资源限制失效。
- l2ctrl, 保留参数, 暂时设置为固定值nullptr, 开发者无需关注;
- stream, 类型为aclrtStream, stream用于维护一些异步操作的执行顺序, 确保按照应用程序中的代码调用顺序在device上执行。stream创建等管理接口请参考“Stream管理”章节。

如下名为add_custom的核函数, 实现两个矢量的相加, 调用示例如下:

```
// numBlocks设置为8表示在8个核上调用了add_custom核函数, 每个核都会独立且并行地执行该核函数, 该核函数的参数列表为x, y, z。  
add_custom<<<8, nullptr, stream>>>(x, y, z);
```

核函数的调用是异步的, 核函数的调用结束后, 控制权立刻返回给主机端, 可以调用以下aclrtSynchronizeStream函数来强制主机端程序等待所有核函数执行完毕。

```
aclError aclrtSynchronizeStream(aclrtStream stream);
```

模板核函数定义和调用

支持开发者使用模板定义核函数, 核函数定义示例如下, 它有两个模板参数: a和T。a是一个非类型模板参数, T是一个类型模板参数。

```
template<int a, typename T>  
__global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z)  
{  
    ...  
    AscendC::printf("Print Template a: %d\n", a);  
    ...  
    xGm.SetGlobalBuffer((__gm_T*)x + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);  
}
```

```
yGm.SetGlobalBuffer((__gm__T*)y + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);  
zGm.SetGlobalBuffer((__gm__T*)z + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);  
...  
}
```

模板核函数的调用方式如下：add_custom<20, float>这部分代码调用了名为 add_custom 的核函数，并为其模板参数提供了具体值。

```
add_custom<20, float><<<numBlocks, nullptr, stream>>>(x, y, z);
```

2.2.4 编程范式

2.2.4.1 AI Core SIMD 编程

2.2.4.1.1 基于 TPipe 和 TQue 编程

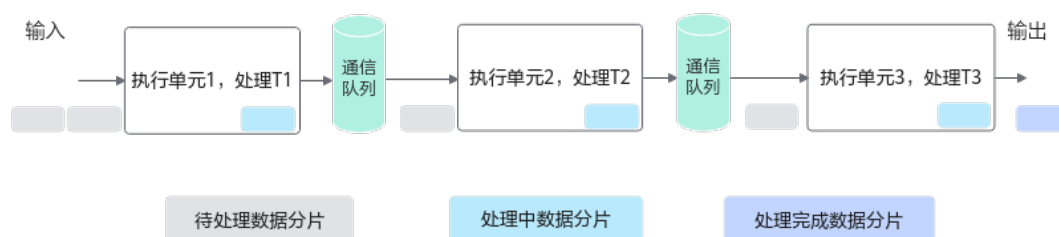
典型算子的编程范式

编程范式描述了算子核函数实现的固定流程，基于编程范式进行编程，可以快速搭建算子实现的代码框架。

根据[2.2.2 抽象硬件架构](#)，AI Core内部的执行单元异步并行地执行接收到的指令，各执行单元配合，以一种流水线的方式完成完整的算子执行过程。

通过下图可以更直观地理解流水并行的概念。示意图中，从输入数据到输出数据需要经过3个阶段任务的处理（T1、T2、T3），多个执行单元并行处理，每个执行单元只会专注于一个任务的处理，会处理所有的数据分片；执行单元完成对某个数据分片的处理后，将其加入到通信队列，下一个执行单元空闲时就会从队列中取出数据继续处理；可以类比为生产流水线中的工人只完成某一项固定工序，完成后就交由下一项工序负责人继续处理。

图 2-5 流水线并行示意图

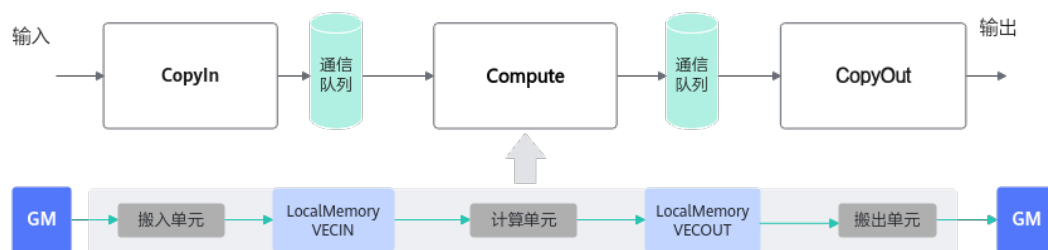


Ascend C编程范式正是这样一种流水线式的编程范式，把算子核内的处理程序，分成多个流水任务，通过队列（TQue）完成任务间通信和同步，并通过统一的资源管理模块（TPipe）来统一管理内存、事件等资源。

下文将从三种典型的算子类型出发，对这种基于TPipe和TQue的编程范式进行详细介绍。

- [矢量编程范式](#)
- [矩阵编程范式](#)
- [融合算子编程范式](#)

矢量编程范式

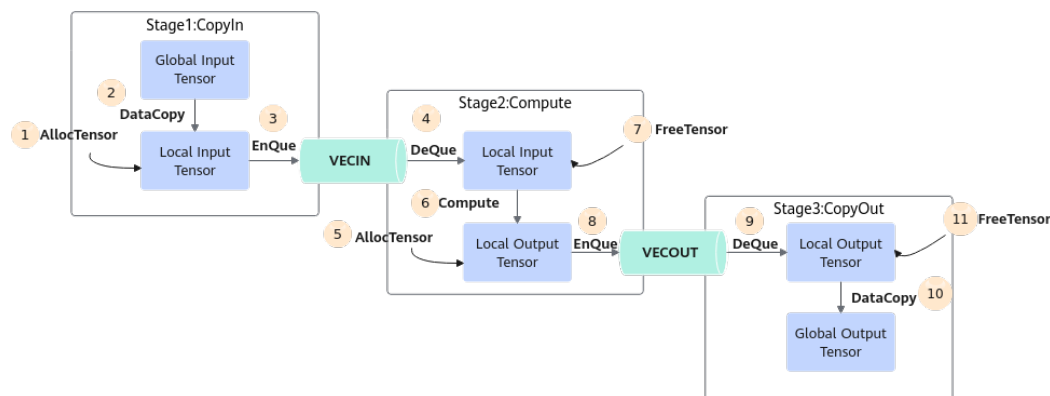


如上图所示，矢量编程范式把算子的实现流程分为3个基本任务：CopyIn，Compute，CopyOut。

- **CopyIn**负责搬入操作：将输入数据从Global Memory搬运到Local Memory（VECIN用于表达矢量计算搬入数据的存放位置），完成搬运后执行入队列操作；
- **Compute**负责矢量指令计算操作：完成队列出队后，从Local Memory获取数据并计算，计算完成后执行入队操作；
- **CopyOut**负责搬出操作：完成队列出队后，将计算结果从Local Memory（VEECOUT用于表达矢量计算搬出数据的存放位置）搬运到Global Memory。

上文中提到的VECIN/VEECOUT是TPosition的概念。Ascend C管理不同层级的物理内存时，用一种抽象的逻辑位置（TPosition）来表达各级别的存储，代替了片上物理存储的概念，达到隐藏硬件架构的目的。除了VECIN/VEECOUT，矢量编程中还会使用到VECCALC，一般在定义临时变量时使用此位置。TPosition与物理内存的映射关系请参考表1。

从编程的角度来讲，具体流程（如下文的伪代码）和流程图如下：



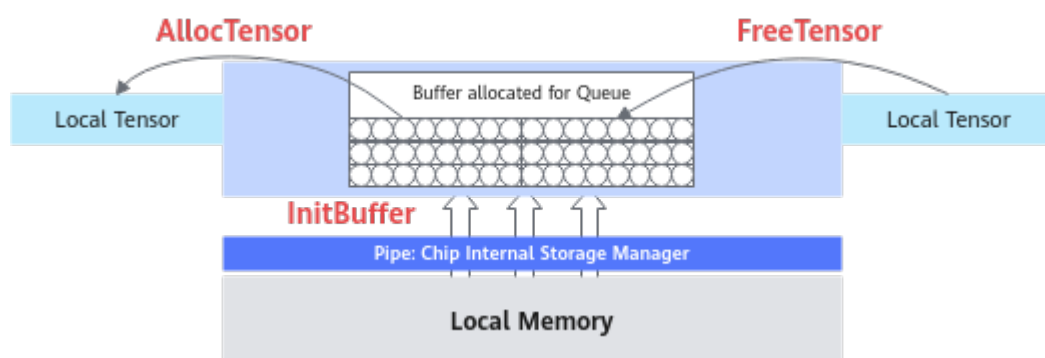
```
AscendC::TPipe pipe; // 创建全局的资源管理
AscendC::TQue<AscendC::TPosition::VecIn, 1> queIn; // 创建CopyIn阶段的队列
AscendC::TQue<AscendC::TPosition::VecOut, 1> queOut; // 创建CopyOut阶段的队列
// Init 阶段
pipe.InitBuffer(queIn, 2, 1024); // 开启DoubleBuffer，将待处理的数据一分为二，实现流水并行
pipe.InitBuffer(queOut, 2, 1024);
for-loop {
    // CopyIn 阶段
    {
        auto tensor = queIn.AllocTensor<half>(); // 从Que上申请资源，长度1024
        AscendC::DataCopy(tensor, gm, 1024); // 搬运数据从GM到VECIN
        queIn.Enqueue(tensor);
    }
    // Compute阶段
    {
```

```
auto tensor = queIn.DeQue<half>();  
auto tensorOut = queOut.AllocTensor<half>();  
AscendC::Abs(tensorOut, tensor, 1024);    // 计算  
queIn.FreeTensor(tensor);  
queOut.EnQue(tensorOut);  
}  
// CopyOut 阶段  
{  
auto tensor = queOut.DeQue<half>();  
AscendC::DataCopy(gmOut, tensor, 1024);    // 搬运数据从VECOUT到GM  
queOut.FreeTensor(tensor);                // 释放资源  
}  
}
```

任务间数据传递使用到的内存、事件等资源统一由管理模块Pipe进行管理。如下所示的内存管理示意图，TPipe通过InitBuffer接口对外提供队列内存初始化功能，开发者可以通过该接口为指定的队列分配内存。

队列内存初始化完成后，需要使用内存时，通过调用AllocTensor来为LocalTensor分配内存，当创建的LocalTensor完成相关计算无需再使用时，再调用FreeTensor来回收LocalTensor的内存。

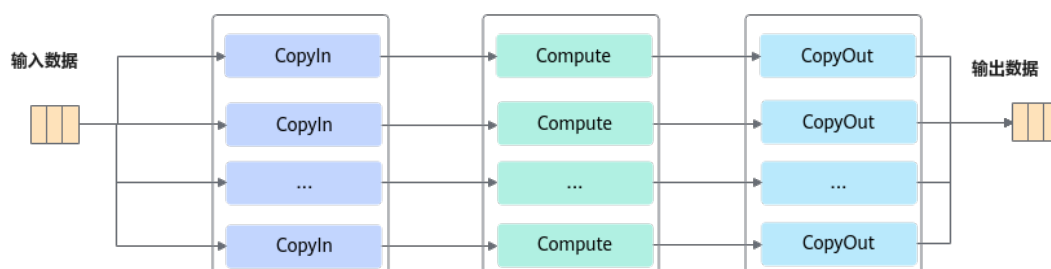
图 2-6 内存管理示意图



编程过程中使用到的临时变量内存同样通过Pipe进行管理。临时变量可以使用TBuf数据结构来申请指定TPosition上的存储空间。使用TBuf申请的内存空间只能参与计算，无法执行队列的入队出队操作。具体的接口使用说明请参考TBuf。

按照上述编程范式进行编程即可实现单核上数据的并行处理。需要处理的数据被切分成n片，每个并行任务需要依次完成n个数据切片的处理。任务间的箭头表达数据间的依赖关系，比如CopyIn处理完第一个数据切片之后，Compute才能对该切片进行处理。

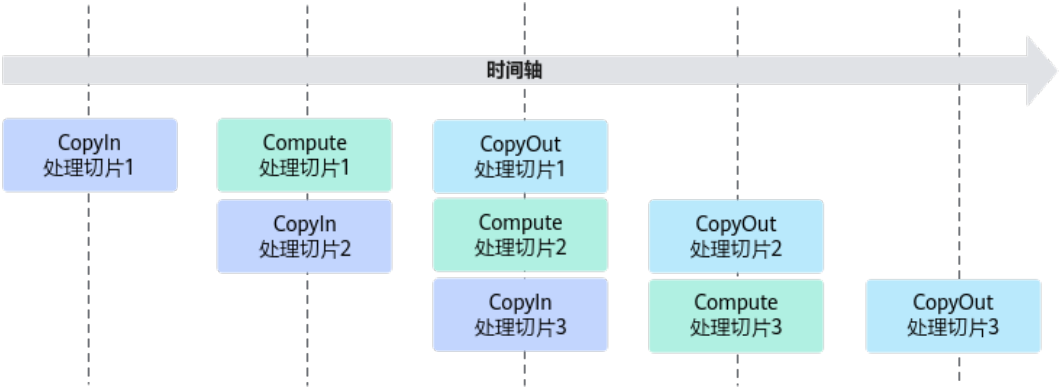
图 2-7 流水任务示意图



上图中的流水任务运行起来的示意图如下，从运行图中可以看出，对于同一片数据，CopyIn、Compute、CopyOut之间的处理具有依赖关系，需要串行处理；不同的数据

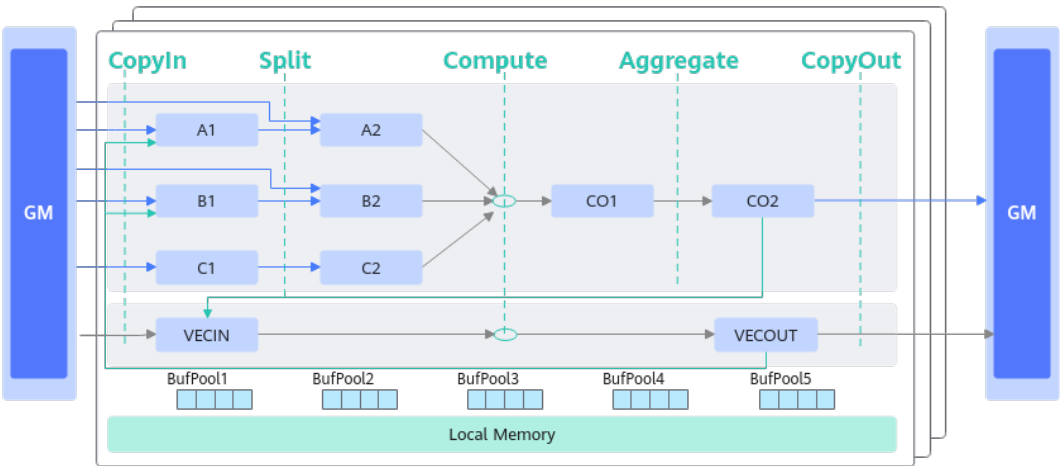
切片，同一时间点，可以有多个任务在并行处理，由此达到任务并行、提升性能的目的。

图 2-8 流水任务运行示意图



矩阵编程范式

Cube计算的典型数据流图如下所示：



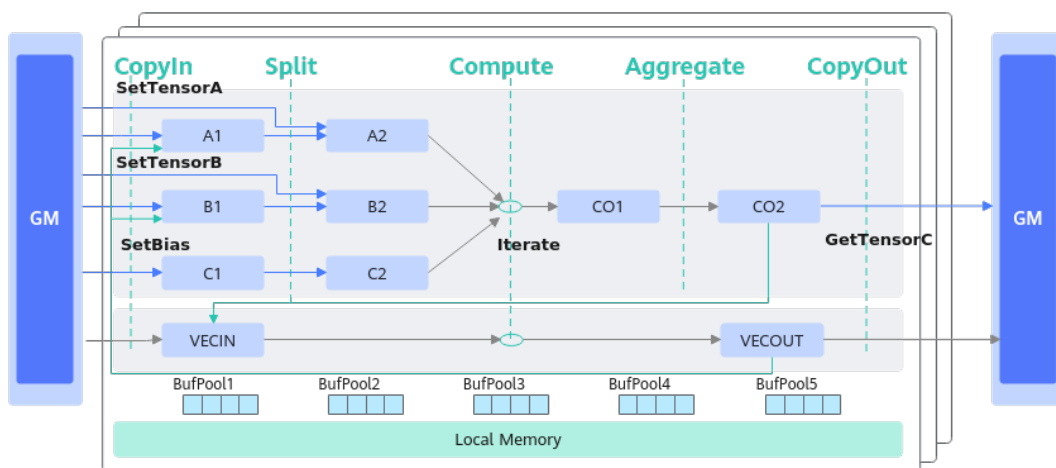
和矢量编程范式一样，同样也使用逻辑位置（TPosition）来表达数据流，Cube编程范式中主要使用的逻辑位置定义如下：

- A1：代表设备上用于矩阵计算的逻辑内存，用于存放左矩阵，物理存储对应AI Core的L1 Buffer。
- B1：代表设备上用于矩阵计算的逻辑内存，用于存放右矩阵，物理存储对应AI Core的L1 Buffer。
- C1：代表设备上用于矩阵计算的逻辑内存，用于存放Bias（偏置）数据，物理存储对应AI Core的L1 Buffer或Unified Buffer。
- A2：代表设备上用于矩阵计算的逻辑内存，用于存放小块左矩阵（如经过分割、适配LOA Buffer容量的分块），物理存储对应AI Core的LOA Buffer。
- B2：代表设备上用于矩阵计算的逻辑内存，用于存放小块右矩阵（如经过分割、适配LOB Buffer容量的分块），物理存储对应AI Core的LOB Buffer。
- C2：代表设备上用于矩阵计算的逻辑内存，用于存放小块Bias（偏置）数据（如经过分割、适配BT Buffer容量的分块），物理存储对应AI Core的BT Buffer或LOC Buffer。

- CO1: 代表设备上用于矩阵计算的逻辑内存, 用于存放小块矩阵计算结果(如经过分割的矩阵计算结果分块), 物理存储对应AI Core的LOC Buffer。
- CO2: 代表设备上用于矩阵计算的逻辑内存, 用于存放矩阵计算结果(如原始矩阵的最终计算结果), 物理存储对应Global Memory或AI Core的Unified Buffer。
- VECIN: 代表设备上用于向量计算的逻辑内存, 用于存放向量计算的输入数据, 物理存储对应AI Core的Unified Buffer。
- VECCALC: 代表设备上用于向量计算的逻辑内存, 用于存放临时变量, 物理存储对应AI Core的Unified Buffer。
- VECOUT: 代表设备上用于向量计算的逻辑内存, 用于存放向量计算的输出数据, 物理存储对应AI Core的Unified Buffer。

TPosition与物理内存的映射关系请参考表1。

Cube计算流程同样也可以理解为CopyIn、Compute、CopyOut这几个阶段, 因为流程相对复杂, Matmul高阶API提供对此的高阶封装, 简化了编程范式。



如上图所示: CopyIn阶段对应SetTensorA、SetTensorB、SetBias接口; Compute阶段对应Iterate接口; CopyOut阶段对应GetTensorC接口。具体流程可参考如下示例:

```
// 创建Matmul对象 创建对象时需要传入A、B、C、Bias的参数类型信息, 类型信息通过MatmulType来定义,
// 包括: 内存逻辑位置、数据格式、数据类型。
typedef MatmulType<TPosition::GM, CubeFormat::ND, half> aType;
typedef MatmulType<TPosition::GM, CubeFormat::ND, half> bType;
typedef MatmulType<TPosition::GM, CubeFormat::ND, float> cType;
typedef MatmulType<TPosition::GM, CubeFormat::ND, float> biasType;
Matmul<aType, bType, cType, biasType> mm;

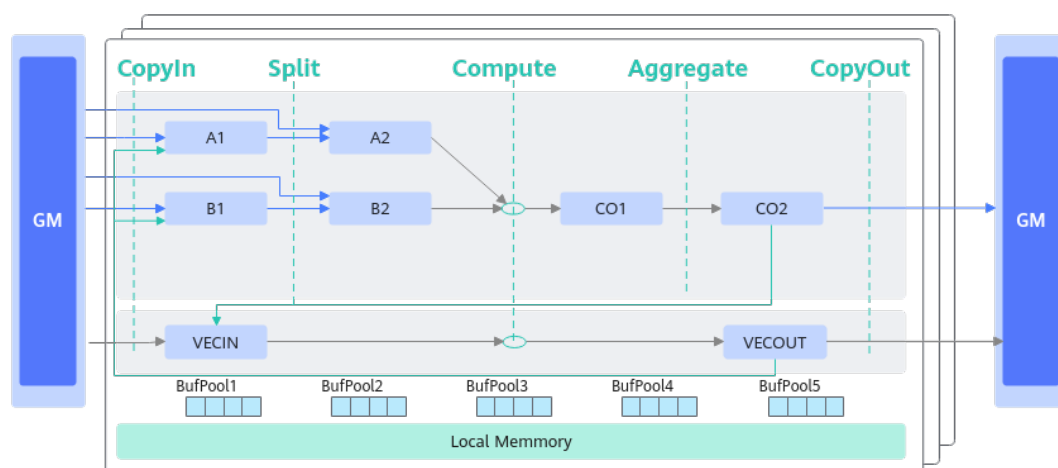
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling); // 初始化
// CopyIn阶段: 完成从GM到LocalMemory的搬运
mm.SetTensorA(gm_a); // 设置左矩阵A
mm.SetTensorB(gm_b); // 设置右矩阵B
mm.SetBias(gm_bias); // 设置Bias
// Compute阶段: 完成矩阵乘计算
while (mm.Iterate()) {
    // CopyOut阶段: 完成从LocalMemory到GM的搬运
    mm.GetTensorC(gm_c);
}
// 结束矩阵乘操作
mm.End();
```

融合算子编程范式

支持Vector与Cube混合计算的算子称之为融合算子。Ascend C提供融合算子的编程范式，方便开发者基于该范式表达融合算子的数据流，快速实现自己的融合算子。

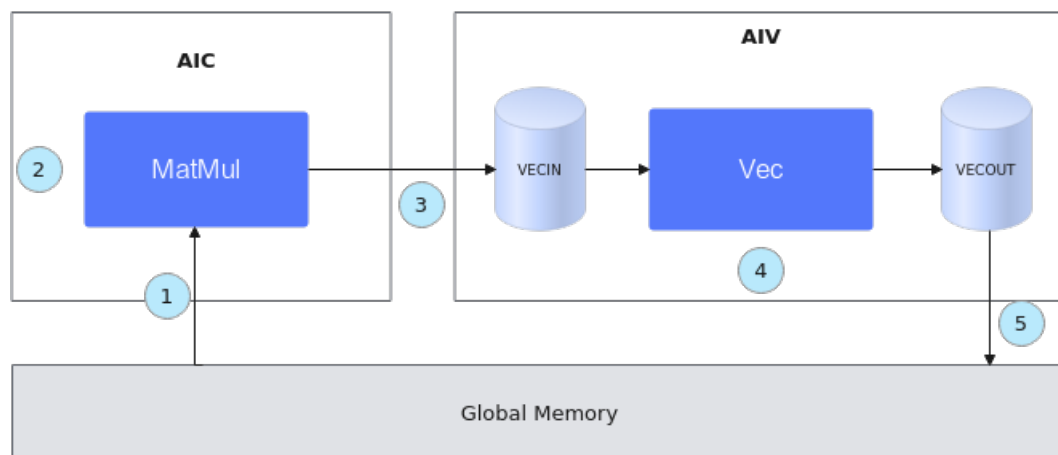
融合算子数据流指融合算子的输入输出在各存储位置间的流向。以一个典型的Cube和Vector融合算子为例，逻辑位置间的数据流向如下图所示（为了简化描述，没有列出bias）：

- Cube的输出可以作为Vector的输入：CO2->VECIN
- Vector的输出可以作为Cube的输入：VEECOUT->A1->A2、VEECOUT->B1->B2



基于Matmul高阶API的融合算子编程范式，对上述数据流简化表达如下：

图 2-9 融合算子编程范式



1. 初始化一个MatMul对象，将输入数据从Global Memory搬运到Cube核上。
2. 进行MatMul内部的计算。
3. 将MatMul的计算结果搬运到Vector核上。
4. 进行Vector矢量计算。
5. 将输出结果搬运到Global Memory上。

整个过程的示例代码如下（伪代码）：

```
template<typename aType, typename bType, typename cType, typename biasType>
__acore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::Process()
{
    // 步骤1: 初始化一个MatMul对象, 将输入数据从Global Memory搬运到Cube核上。
    uint32_t computeRound = 0;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), matmulObj);
    matmulObj.Init(&tiling);
    matmulObj.SetTensorA(aGlobal);
    matmulObj.SetTensorB(bGlobal);
    matmulObj.SetBias(biasGlobal);

    while (matmulObj.template Iterate<true>()) { // 步骤2: 进行MatMul内部的计算。
        // 步骤3: 将MatMul的计算结果搬运到Vector核上。
        reluOutLocal = reluOutQueue_.AllocTensor<cType>();
        matmulObj.template GetTensorC<true>(reluOutLocal, false, true);
        // 步骤4: 进行Vector向量计算。
        AscendC::LeakyRelu(reluOutLocal, reluOutLocal, (cType)alpha, tiling.baseM * tiling.baseN);
        reluOutQueue_.EnQue(reluOutLocal);
        // 步骤5: 将输出结果搬运到Global Memory上
        reluOutQueue_.DeQue<cType>();
        ...
        AscendC::DataCopy(cGlobal[startOffset], reluOutLocal, copyParam);
        reluOutQueue_.FreeTensor(reluOutLocal);

        computeRound++;
    }
    matmulObj.End();
}
```

2.2.4.1.2 静态 Tensor 编程

在基于Pipe进行算子开发的方式中, 由Pipe (TPipe类) 统一管理Device端内存等资源, 开发者无需感知内存管理、DoubleBuffer流水、同步等处理, 只需要按照计算流编写算子即可, 但由此也带来了一些运行时开销 (如TPipe创建、InitBuffer等)。

基于以上原因, Ascend C提供了静态Tensor编程方式, 相比基于Pipe的编程方式, 这种方式避免了TPipe内存管理初始化过程 (约数百纳秒), 从而减少了运行时开销, 更有助于开发者实现极致性能。通过直接构造指定地址和存储位置的LocalTensor, 并将其传递给计算、搬运等API进行编程, 提供了更高的灵活性。然而, 这种编程方式也带来了更高的开发复杂性, 需要开发者自行管理DoubleBuffer和同步流水, 并且只能使用Ascend C的基础API, 而非全部功能。

两种编程方式的对比如下:

基于TPipe和TQue自动管理内存与同步

```
AscendC::TPipe pipe; // 创建全局的资源管理对象
AscendC::TQue<AscendC::TPosition::VecIn, 1> queIn;
AscendC::TQue<AscendC::TPosition::VecOut, 1> queOut;
// 初始化队列的内存
pipe.InitBuffer(queIn, 2, bufferSize);
pipe.InitBuffer(queOut, 2, bufferSize);

auto xLocal = queIn.AllocTensor<half>();
auto yLocal = queOut.AllocTensor<half>();
AscendC::DataCopy(xLocal, xGm[offset], bufferSize);
queIn.Enqueue(xLocal);
xLocal = queIn.DeQue<half>();
AscendC::Exp(yLocal, xLocal, bufferSize);
// ...
```

运行时开销

静态Tensor编程自主管理内存与同步

```
// 直接定义LocalTensor
AscendC::LocalTensor<half> xLocal(AscendC::TPosition::VECIN, xAddr,
bufferSize);
AscendC::LocalTensor<half> yLocal(AscendC::TPosition::VEECOUT, yAddr,
bufferSize);
AscendC::DataCopy(xLocal, xGm[offset], bufferSize);
AscendC::SetFlag<AscendC::HardEvent::MTE2_V>(EVENT_ID0);
AscendC::WaitFlag<AscendC::HardEvent::MTE2_V>(EVENT_ID0);
AscendC::Exp(yLocal, xLocal, bufferSize);
// ...
```

说明

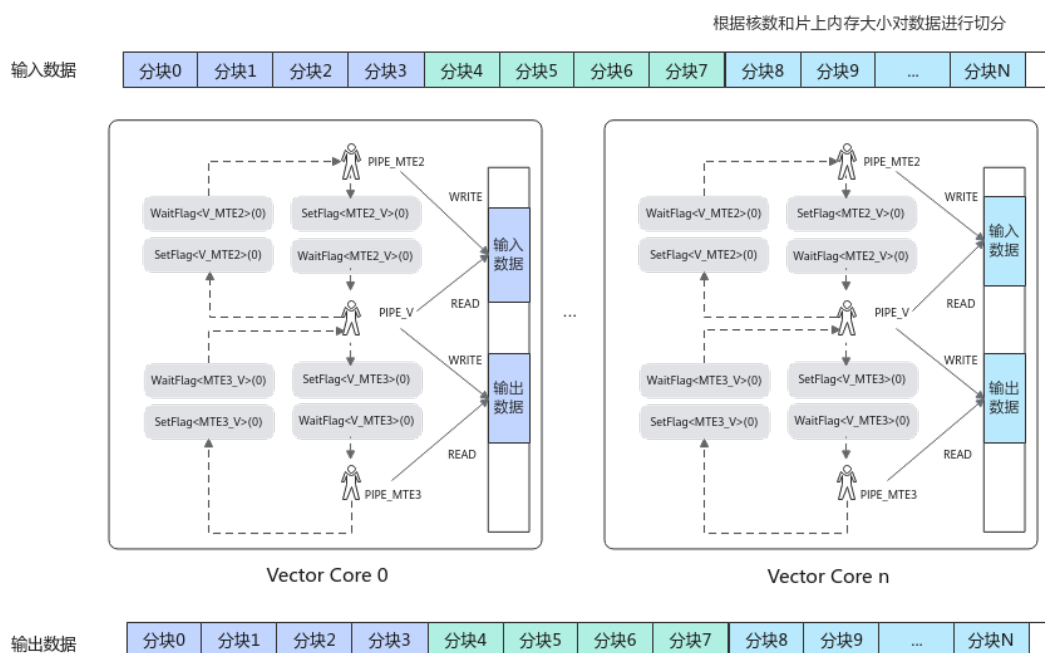
- 静态Tensor编程的使用约束和限制请参考[使用约束和限制](#)。
- 本节涉及的完整样例请参考[静态Tensor编程样例](#)。

编程范式

- AI Core包括多种内存单元，比如用于矢量计算的Unified Buffer和用于矩阵计算的L1 Buffer、LOA Buffer、LOB Buffer、LOC Buffer等内存资源。
- AI Core包括多条流水，比如Vector/Cube/Scalar计算流水，MTE1、MTE2、MTE3搬运流水等，每条流水并行执行。

静态Tensor编程方式下，开发者完全自主的管理AI Core上的所有内存资源，调用Ascend C提供的搬运或者计算类API编写算子，并根据数据依赖关系插入对应的同步事件，以达成最优性能。

下图是一个典型矢量算子的示意图，开发者首先根据业务计算量进行数据分块处理，之后根据核内的数据依赖关系完成同步事件的插入：



内存管理

静态Tensor编程方式下，开发者可以使用两种方式创建Tensor：

- 通过LocalMemAllocator指定硬件位置进行Tensor分配。
LocalMemAllocator是一种线性内存分配器，开发者可以调用Alloc方法进行内存分配，地址分配从0开始，根据调用次序依次向后进行线性分配，LocalMemAllocator只是一个简单的线性分配器，并不提供内存释放以及其它内存管理的能力。在不关注Bank冲突场景或者算子初始功能开发时，可以使用LocalMemAllocator简化算子编写，在后续性能优化时切换到使用LocalTensor进行地址分配的方式。
- 通过LocalTensor构造函数创建Tensor，极致性能场景推荐使用此方式。

开发者可以使用LocalTensor构造函数直接指定内存地址，实现内存的完全自主管理（本质上无需申请和释放内存）。使用时，需根据需求合理指定地址（不超过物理存储上限），并在保证功能正确的前提下进行内存复用。如果需要通过规避Bank冲突或者复用内存来获得极致性能时，推荐使用该方式。

```
// 方式1：使用LocalMemAllocator进行内存分配
AscendC::LocalMemAllocator<AscendC::Hardware::UB> ubAllocator;
AscendC::LocalTensor<float> xLocalPing = ubAllocator.Alloc<float, TILE_LENGTH>();
AscendC::LocalTensor<float> yLocalPing = ubAllocator.Alloc<float, TILE_LENGTH>();
AscendC::LocalTensor<float> zLocalPing = ubAllocator.Alloc<float, TILE_LENGTH>();

// 方式2：直接使用LocalTensor构造函数构造Tensor
AscendC::LocalTensor<float> xLocalPing(AscendC::TPosition::VECCALC, xAddrPing, TILE_LENGTH);
AscendC::LocalTensor<float> yLocalPing(AscendC::TPosition::VECCALC, yAddrPing, TILE_LENGTH);
AscendC::LocalTensor<float> zLocalPing(AscendC::TPosition::VECCALC, zAddrPing, TILE_LENGTH);
```

同步管理

根据前文介绍的硬件架构，AI Core内部异步并行计算存在多条流水（包括矢量计算、矩阵计算、数据搬入、数据搬出等），多条流水之间存在数据依赖时，需要插入对应的同步事件。静态Tensor编程方式下，开发者使用SetFlag/WaitFlag(ISASI)和PipeBarrier(ISASI)手动插入同步，事件的类型和事件ID由开发者自行管理，但需要注意事件ID不能使用6和7（可能与内部使用的事件ID出现冲突，进而出现未定义行为）。另外由于需要使用SetFlag/WaitFlag/PipeBarrier底层同步接口（属于ISASI硬件体系结构相关的接口），无法保证跨硬件版本兼容。

在同步依赖中，根据数据依赖和指令执行关系，存在两种依赖关系，即正向同步（循环内依赖）与反向同步（循环间依赖）：

- 正向同步

在本次数据搬入和计算之间，插入MTE2_V（矢量计算流水等待MT2搬运流水）同步事件，确保数据搬入之后再计算；在本次数据计算和搬出之间，插入V_MTE3（MTE3搬运流水等待矢量计算流水）同步事件，确保数据计算完成后再进行搬出。

- 反向同步

在上一次的数据计算和本次数据搬入之间，插入V_MTE2（MT2搬运流水等待矢量计算流水）同步事件，确保上一次的数据计算完成后，本次的数据再进行搬入。防止本次的数据会覆盖掉上一次未计算完成的数据；在上一次的数据搬出和本次数据计算之间，插入MTE3_V（矢量计算流水等待MT3搬运流水）同步事件，确保上一次的数据搬出后，再进行本次数据的计算。防止本次的数据会覆盖掉上一次未搬出的数据。

上述的同步逻辑在使用Pipe编程框架时，框架会使用EnQue/DeQue/AllocTensor/FreeTensor进行封装。您可以通过[2.9.3 编程模型设计原理](#)来了解应该如何在静态Tensor编程方式时手动进行同步控制。

```
AscendC::LocalTensor<float> xLocal(AscendC::TPosition::VECCALC, xAddr, TILE_LENGTH);
AscendC::LocalTensor<float> yLocal(AscendC::TPosition::VECCALC, yAddr, TILE_LENGTH);
AscendC::LocalTensor<float> zLocal(AscendC::TPosition::VECCALC, zAddr, TILE_LENGTH);
for (int i = 0; i < loopCount; i++) {
    // dependency of PIPE_V & PIPE_MTE2 caused by xLocal/yLocal between 2 sequential loops
    if (i != 0) {
        AscendC::WaitFlag<AscendC::HardEvent::V_MTE2>(EVENT_ID0);
    }
    AscendC::DataCopy(xLocal, xGm[i * TILE_LENGTH], TILE_LENGTH);
    AscendC::DataCopy(yLocal, yGm[i * TILE_LENGTH], TILE_LENGTH);
    // dependency of PIPE_MTE2 & PIPE_V caused by xLocal/yLocal in one single loop
    AscendC::SetFlag<AscendC::HardEvent::MTE2_V>(EVENT_ID0);
    AscendC::WaitFlag<AscendC::HardEvent::MTE2_V>(EVENT_ID0);
    if (i != 0) {
        // dependency of PIPE_MTE3 & PIPE_V caused by zLocal between 2 sequential loops
```

```
AscendC::WaitFlag<AscendC::HardEvent::MTE3_V>(EVENT_ID0);
}
AscendC::Add(zLocal, xLocal, yLocal, TILE_LENGTH);
if (i != (loopCount - 1)) {
    // dependency of PIPE_V & PIPE_MTE2 caused by xLocal/yLocal between 2 sequential loops
    AscendC::SetFlag<AscendC::HardEvent::V_MTE2>(EVENT_ID0);
}
// dependency of PIPE_V & PIPE_MTE3 caused by zLocal in one single loop
AscendC::SetFlag<AscendC::HardEvent::V_MTE3>(EVENT_ID0);
AscendC::WaitFlag<AscendC::HardEvent::V_MTE3>(EVENT_ID0);
AscendC::DataCopy(zGm[i * TILE_LENGTH], zLocal, TILE_LENGTH);
if (i != (loopCount - 1)) {
    // dependency of PIPE_MTE3 & PIPE_V caused by zLocal between 2 sequential loops
    AscendC::SetFlag<AscendC::HardEvent::MTE3_V>(EVENT_ID0);
}
}
```

流水优化

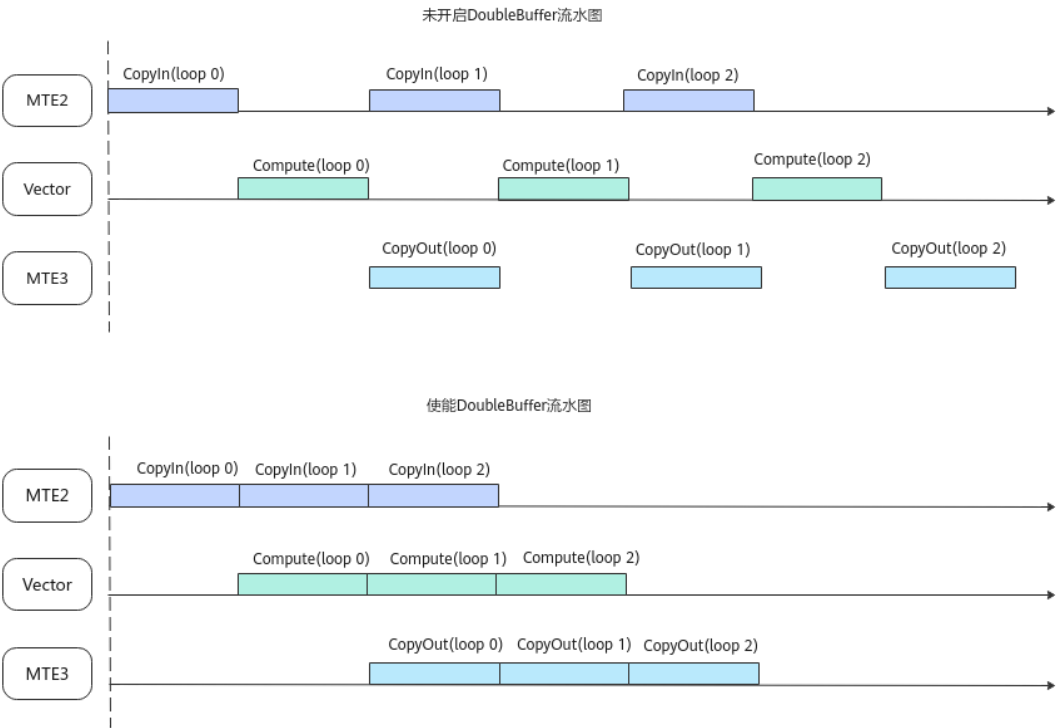
在基于TPipe的编程范式中，开发者只需要在InitBuffer时指定buffer数量为2，即可自动开启Double Buffer。但是静态Tensor编程方式下，开发者需要手动开启Double Buffer，具体示例如下，完整样例请参考[静态Tensor编程样例](#)中的Double Buffer示例。

```
// ping
AscendC::LocalTensor<float> xLocalPing(AscendC::TPosition::VECCALC, xAddrPing, TILE_LENGTH);
AscendC::LocalTensor<float> yLocalPing(AscendC::TPosition::VECCALC, yAddrPing, TILE_LENGTH);
AscendC::LocalTensor<float> zLocalPing(AscendC::TPosition::VECCALC, zAddrPing, TILE_LENGTH);
// pong
AscendC::LocalTensor<float> xLocalPong(AscendC::TPosition::VECCALC, xAddrPong, TILE_LENGTH);
AscendC::LocalTensor<float> yLocalPong(AscendC::TPosition::VECCALC, yAddrPong, TILE_LENGTH);
AscendC::LocalTensor<float> zLocalPong(AscendC::TPosition::VECCALC, zAddrPong, TILE_LENGTH);

// double buffer
AscendC::SetFlag<AscendC::HardEvent::MTE3_MTE2>(EVENT_ID0);
AscendC::SetFlag<AscendC::HardEvent::MTE3_MTE2>(EVENT_ID1);
for (int i = 0; i < loopCount; i++) {
    int32_t eventID = (i % 2 == 0 ? EVENT_ID0 : EVENT_ID1);
    AscendC::LocalTensor<float> &xLocal = (i % 2 == 0 ? xLocalPing : xLocalPong);
    AscendC::LocalTensor<float> &yLocal = (i % 2 == 0 ? yLocalPing : yLocalPong);
    AscendC::LocalTensor<float> &zLocal = (i % 2 == 0 ? zLocalPing : zLocalPong);
    // dependency of PIPE_MTE3 & PIPE_MTE2 caused by xLocal/yLocal between 2 sequential loops
    AscendC::WaitFlag<AscendC::HardEvent::MTE3_MTE2>(eventID);
    AscendC::DataCopy(xLocal, xGm[i * TILE_LENGTH], TILE_LENGTH);
    AscendC::DataCopy(yLocal, yGm[i * TILE_LENGTH], TILE_LENGTH);

    // dependency of PIPE_MTE2 & PIPE_V caused by xLocal/yLocal in one single loop
    AscendC::SetFlag<AscendC::HardEvent::MTE2_V>(eventID);
    AscendC::WaitFlag<AscendC::HardEvent::MTE2_V>(eventID);
    AscendC::Add(zLocal, xLocal, yLocal, TILE_LENGTH);
    // dependency of PIPE_V & PIPE_MTE3 caused by zLocal in one single loop
    AscendC::SetFlag<AscendC::HardEvent::V_MTE3>(eventID);
    AscendC::WaitFlag<AscendC::HardEvent::V_MTE3>(eventID);
    AscendC::DataCopy(zGm[i * TILE_LENGTH], zLocal, TILE_LENGTH);
    // dependency of PIPE_MTE3 & PIPE_MTE2 caused by zLocal between 2 sequential loops
    AscendC::SetFlag<AscendC::HardEvent::MTE3_MTE2>(eventID);
}
AscendC::WaitFlag<AscendC::HardEvent::MTE3_MTE2>(EVENT_ID0);
AscendC::WaitFlag<AscendC::HardEvent::MTE3_MTE2>(EVENT_ID1);
```

以下为不使能DoubleBuffer和使能DoubleBuffer的流水示意图。多数情况下，采用DoubleBuffer能有效提升Vector的时间利用率，缩减算子执行时间，详细内容可参考[2.9.5.1 DoubleBuffer](#)。



使用约束和限制

静态Tensor编程方式需要遵循以下约束和限制：

- 开发者不能使用TPipe/TQue/TQueBind/TBufPool等框架接口，和上述框架接口混用可能会出现未定义行为。
- 只能使用部分API。具体支持的API列表见[支持的API范围](#)。因为不在列表范围内的API内部依赖TPipe分配事件ID，可能会和开发者定义的事件ID产生冲突。
- 同步事件需要由开发者使用SetFlag/WaitFlag(ISASI)和PipeBarrier(ISASI)手动插入，事件的类型和事件ID由开发者自行管理，但需要注意事件ID不能使用6和7（可能与内部使用的事件ID出现冲突，进而出现未定义行为）。
- 由于需要使用SetFlag/WaitFlag/PipeBarrier底层同步接口（属于ISASI硬件体系结构相关的接口），无法保证算子跨硬件版本兼容。
- Kernel入口处需要开发者手动调用InitSocState接口用来初始化全局状态寄存器。因为全局状态寄存器处于不确定状态，如果不调用该接口，可能导致算子执行过程中出现未定义行为。在TPipe框架编程中，初始化过程由TPipe完成，无需开发者关注。

支持的 API 范围

表 2-2 针对 Atlas 推理系列产品 AI Core，支持的 API 范围

接口分类	接口名称
基础API > 标量计算	ScalarGetCountOfValue、ScalarCountLeadingZero、ScalarCast、CountBitsCntSameAsSignBit、ScalarGetSFFValue

接口分类	接口名称
基础API > 矢量计算 > 基础算术	Exp、Ln、Abs、Reciprocal、Sqrt、Rsqrt、Relu、VectorPadding、Add、Sub、Mul、Div、Max、Min、BilinearInterpolation、Adds、Muls、Maxs、Mins、LeakyRelu
基础API > 矢量计算 > 逻辑计算	Not、And、Or
基础API > 矢量计算 > 复合计算	Axpy、CastDeq、AddRelu、AddReluCast、AddDeqRelu、SubRelu、SubReluCast、MulAddDst、MulCast、FusedMulAdd、FusedMulAddRelu
基础API > 矢量计算 > 比较与选择	Compare、Compare（结果存入寄存器）、CompareScalar、GetCmpMask、SetCmpMask、Select、GatherMask
基础API > 矢量计算 > 类型转换	Cast
基础API > 矢量计算 > 归约计算	WholeReduceMax、WholeReduceMin、WholeReduceSum、BlockReduceMax、BlockReduceMin、BlockReduceSum、PairReduceSum、RepeatReduceSum、GetReduceMaxMinCount
基础API > 矢量计算 > 数据转换	Transpose、TransDataTo5HD
基础API > 矢量计算 > 数据填充	Duplicate
基础API > 矢量计算 > 排序组合	ProposalConcat、ProposalExtract、RpSort16、MrgSort4、GetMrgSortResult
基础API > 矢量计算 > 离散与聚合	Gather、Scatter
基础API > 矢量计算 > 掩码操作	SetMaskCount、SetMaskNorm、SetVectorMask、ResetMask
基础API > 矢量计算 > 量化设置	SetDeqScale
基础API > 数据搬运 > DataCopy	基础数据搬运
基础API > 同步控制 > 核内同步	SetFlag/WaitFlag、PipeBarrier
基础API > 缓存控制	DataCachePreload、DataCacheCleanAndInvalid、ICachePreLoad
基础API > 系统变量访问	GetBlockNum、GetBlockIdx、GetDataBlockSizeInBytes、GetArchVersion、GetTaskRatio、InitSocState、GetProgramCounter、CheckLocalMemoryIA

接口分类	接口名称
基础API > 原子操作	SetAtomicAdd、SetAtomicNone
基础API > 矩阵计算	InitConstValue、LoadData、SetAippFunctions、LoadImageToLocal、LoadUnzipIndex、LoadDataUnzip、SetLoadDataBoundary、SetLoadDataPaddingValue、Mmad

表 2-3 针对 Atlas A2 训练系列产品/Atlas A2 推理系列产品，支持的 API 范围

接口分类	接口名称	备注
基础API > 标量计算	ScalarGetCountOfValue、ScalarCountLeadingZero、ScalarCast、CountBitsCntSameAsSignBit、ScalarGetSFFValue、ToBfloat16、ToFloat	-
基础API > 矢量计算 > 基础算术	Exp、Ln、Abs、Reciprocal、Sqrt、Rsqr、Relu、Add、Sub、Mul、Div、Max、Min、BilinearInterpolation、Adds、Muls、Maxs、Mins、LeakyRelu	-
基础API > 矢量计算 > 逻辑计算	Not、And、Or、ShiftLeft、ShiftRight	-
基础API > 矢量计算 > 复合计算	Axpy、CastDeq、AddRelu、AddReluCast、AddDeqRelu、SubRelu、SubReluCast、MulAddDst、MulCast、FusedMulAdd、FusedMulAddRelu	-
基础API > 矢量计算 > 比较与选择	Compare、Compare（结果存入寄存器）、CompareScalar、GetCmpMask、SetCmpMask、Select、GatherMask	-
基础API > 矢量计算 > 类型转换	Cast	-
基础API > 矢量计算 > 归约计算	WholeReduceMax、WholeReduceMin、WholeReduceSum、BlockReduceMax、BlockReduceMin、BlockReduceSum、PairReduceSum、RepeatReduceSum、GetAccVal、GetReduceMaxMinCount	-

接口分类	接口名称	备注
基础API > 矢量计算 > 数据转换	Transpose、TransDataTo5HD	-
基础API > 矢量计算 > 数据填充	Duplicate、Brcb	-
基础API > 矢量计算 > 排序组合	Sort32、MrgSort、GetMrgSortResult	-
基础API > 矢量计算 > 离散与聚合	Gather、Gatherb	-
基础API > 矢量计算 > 掩码操作	SetMaskCount、SetMaskNorm、SetVectorMask、ResetMask	-
基础API > 矢量计算 > 量化设置	SetDeqScale	-
基础API > 数据搬运 > DataCopy	基础数据搬运	不支持VECIN/VECCALC/VECCOUT -> TSCM通路的数据搬运。
	增强数据搬运	不支持VECIN/VECCALC/VECCOUT -> TSCM通路的数据搬运。
	切片数据搬运	-
	随路转换ND2NZ搬运 随路转换NZ2ND搬运 随路量化激活搬运	不支持VECIN/VECCALC/VECCOUT -> TSCM通路的数据搬运。
基础API > 数据搬运	Copy、DataCopyPad、SetPadValue	-
基础API > 同步控制 > 核内同步	SetFlag/WaitFlag、PipeBarrier、DataSyncBarrier	-
基础API > 同步控制 > 核间同步	CrossCoreSetFlag、CrossCoreWaitFlag	-
基础API > 缓存控制	DataCachePreload、DataCacheCleanAndInvalid、ICachePreLoad、GetICachePreloadStatus	-

接口分类	接口名称	备注
基础API > 系统变量访问	GetBlockNum、GetBlockIdx、GetDataBlockSizeInBytes、GetArchVersion、GetTaskRatio、InitSocState、GetProgramCounter、GetSubBlockNum、GetSubBlockIdx、GetSystemCycle、CheckLocalMemoryIA	-
基础API > 原子操作	SetAtomicAdd、SetAtomicType、SetAtomicNone、SetAtomicMax、SetAtomicMin、SetStoreAtomicConfig、GetStoreAtomicConfig	-
基础API > 矩阵计算	Mmad、MmadWithSparse、SetHF32Mode、SetHF32TransMode、SetMMLLayoutTransform、SetFixPipeConfig、SetFixpipeNz2ndFlag、SetFixpipePreQuantFlag、InitConstValue、LoadData、LoadDataWithTranspose、SetAippFunctions、LoadImageToLocal、LoadDataWithSparse、SetFmatrix、SetLoadDataBoundary、SetLoadDataRepeat、SetLoadDataPaddingValue、Fixpipe	-
Utils API > C++标准库 > 算法	max、min、index_sequence	-
Utils API > C++标准库 > 容器函数	tuple、get、make_tuple	-
Utils API > C++标准库 > 类型特性	is_convertible、is_base_of、is_same、enable_if、conditional	-

表 2-4 针对 Atlas A3 训练系列产品/Atlas A3 推理系列产品，支持的 API 范围

接口分类	接口名称	备注
基础API > 标量计算	ScalarGetCountOfValue、 ScalarCountLeadingZero、 ScalarCast、 CountBitsCntSameAsSignBit、 ScalarGetSFFValue、ToBfloat16、 ToFloat	-
基础API > 矢量计算 > 基础算术	Exp、Ln、Abs、Reciprocal、 Sqrt、Rsqr、Relu、Add、Sub、 Mul、Div、Max、Min、 BilinearInterpolation、Adds、 Muls、Maxs、Mins、LeakyRelu	-
基础API > 矢量计算 > 逻辑计算	Not、And、Or、ShiftLeft、 ShiftRight	-
基础API > 矢量计算 > 复合计算	Axpy、CastDeq、AddRelu、 AddReluCast、AddDeqRelu、 SubRelu、SubReluCast、 MulAddDst、MulCast、 FusedMulAdd、FusedMulAddRelu	-
基础API > 矢量计算 > 比较与选择	Compare、Compare（结果存入寄存器）、 CompareScalar、GetCmpMask、 SetCmpMask、Select、GatherMask	-
基础API > 矢量计算 > 类型转换	Cast	-
基础API > 矢量计算 > 归约计算	WholeReduceMax、WholeReduceMin、 WholeReduceSum、BlockReduceMax、 BlockReduceMin、BlockReduceSum、 PairReduceSum、RepeatReduceSum、 GetAccVal、GetReduceMaxMinCount	-
基础API > 矢量计算 > 数据转换	Transpose、TransDataTo5HD	-
基础API > 矢量计算 > 数据填充	Duplicate、Brcb	-
基础API > 矢量计算 > 排序组合	Sort32、MrgSort、GetMrgSortResult	-
基础API > 矢量计算 > 离散与聚合	Gather、Gatherb	-

接口分类	接口名称	备注
基础API > 矢量计算 > 掩码操作	SetMaskCount、SetMaskNorm、SetVectorMask、ResetMask	-
基础API > 矢量计算 > 量化设置	SetDeqScale	-
基础API > 数据搬运 > DataCopy	基础数据搬运	不支持VECIN/VECCALC/VEECOUT -> TSCM通路的数据搬运。
基础API > 数据搬运	增强数据搬运	不支持VECIN/VECCALC/VEECOUT -> TSCM通路的数据搬运。
	切片数据搬运	-
	随路转换ND2NZ搬运 随路转换NZ2ND搬运 随路量化激活搬运	不支持VECIN/VECCALC/VEECOUT -> TSCM通路的数据搬运。
	Copy、DataCopyPad、SetPadValue	-
基础API > 同步控制 > 核内同步	SetFlag/WaitFlag、PipeBarrier、DataSyncBarrier	-
基础API > 同步控制 > 核间同步	CrossCoreSetFlag、CrossCoreWaitFlag	-
基础API > 缓存控制	DataCachePreload、DataCacheCleanAndInvalid、ICachePreLoad、GetICachePreloadStatus	-
基础API > 系统变量访问	GetBlockNum、GetBlockIdx、GetDataBlockSizeInBytes、GetArchVersion、GetTaskRatio、InitSocState、GetProgramCounter、GetSubBlockNum、GetSubBlockIdx、GetSystemCycle、CheckLocalMemoryIA	-
基础API > 原子操作	SetAtomicAdd、SetAtomicType、SetAtomicNone、SetAtomicMax、SetAtomicMin、SetStoreAtomicConfig、GetStoreAtomicConfig	-

接口分类	接口名称	备注
基础API > 矩阵计算	Mmad、MmadWithSparse、SetHF32Mode、SetHF32TransMode、SetMMLLayoutTransform、SetFixPipeConfig、SetFixpipeNz2ndFlag、SetFixpipePreQuantFlag、InitConstValue、LoadData、LoadDataWithTranspose、SetAippFunctions、LoadImageToLocal、LoadDataWithSparse、SetFmatrix、SetLoadDataBoundary、SetLoadDataRepeat、SetLoadDataPaddingValue、Fixpipe	-
Utils API > C++标准库 > 算法	max、min、index_sequence	-
Utils API > C++标准库 > 容器函数	tuple、get、make_tuple	-
Utils API > C++标准库 > 类型特性	is_convertible、is_base_of、is_same、enable_if、conditional	-
高阶API > C++标准库 > 类型特性	is_convertible、is_base_of、is_same、enable_if、conditional	-
高阶API > 模板库函数 > type_traits	is_convertible、is_base_of、is_same、enable_if、conditional	-

2.2.4.1.3 基于语言扩展层 C API 编程

基于语言扩展层C API编程时，通过提供纯C风格的接口，符合C语言算子开发习惯，提供与业界类似编程体验。本节主要介绍C API编程范式，通过内存管理、同步控制、计算及搬运接口相关的介绍，使开发者更好地理解和使用C API进行编程。

内存管理

C API通过C风格的地址限定符描述不同层级内存，并且可以通过指针直接操作内存地址，从而精准控制数据存放位置。不同存储单元的地址限定符介绍如下：

表 2-5 不同存储单元的地址限定符

存储单元	地址限定符	描述
Global Memory	__gm__	表示被修饰的变量位于Global Memory地址空间。

存储单元	地址限定符	描述
Unified Buffer	<code>__ubuf__</code>	表示被修饰的变量位于Unified Buffer地址空间。
L1 Buffer	<code>__cbuf__</code>	表示被修饰的变量位于L1 Buffer地址空间。
L0A Buffer	<code>__ca__</code>	表示被修饰的变量位于L0A Buffer地址空间。
L0B Buffer	<code>__cb__</code>	表示被修饰的变量位于L0B Buffer地址空间。
L0C Buffer	<code>__cc__</code>	表示被修饰的变量位于L0C Buffer地址空间。

地址空间限定符可以在数组或指针变量声明中使用，用于指定对象分配的区域。同一个类型上不允许使用多个地址空间限定符。

基于C API编程时，开发者需要自行通过显式的内存管理来控制内存，不同层级的内存申请介绍如下：

- 全局内存（Global Memory）：一般通过Device侧接口[aclrtMalloc](#)接口分配传入，需要增加对应地址限定符使用。
- 内部存储（包含Unified Buffer、L1 Buffer等）：由用户自行申请空间，通过地址限定符关键字在Kernel内声明。无自动垃圾回收机制，需开发者严格控制生命周期。以申请UB空间为例：

```
// 在数组变量声明中使用地址空间限定符
// total_length 指参与计算的数据长度
constexpr uint64_t total_length = 256;
__ubuf__ float xLocal[ total_length ];
__ubuf__ float yLocal[ total_length ];
__ubuf__ float zLocal[ total_length ];

// 在指针变量声明中使用地址空间限定符
uint64_t offset = 0; // 首先为src0申请内存，从0开始。
__ubuf__ half* src0 = (__ubuf__ half*)asc_get_phy_buf_addr(offset); // 获取src0的地址，通过__ubuf__关键字指定该地址指向UB内存。
```

同步控制

NPU内部有不同的计算单元，在计算前往往需要把计算数据搬运到计算单元上。不同计算单元上的计算过程、数据搬运过程可划分为不同的流水线。如下表所示：

表 2-6 指令流水类型和相关说明

流水类型	含义
PIPE_S	标量流水线
PIPE_V	矢量计算流水及部分硬件架构下的L0C Buffer->UB数据搬运流水
PIPE_M	矩阵计算流水
PIPE_MTE1	L1 Buffer ->L0A Buffer、L1 Buffer->L0B Buffer数据搬运流水
PIPE_MTE2	GM->L1 Buffer、GM->UB等数据搬运流水
PIPE_MTE3	UB->GM等数据搬运流水

流水类型	含义
PIPE_FIX	L0C Buffer->GM、L0C Buffer ->L1等数据搬运流水

在调用C API提供的搬运或者计算类API编写算子时，需要根据流水线之间的数据依赖关系插入对应的同步事件。C API提供了两种不同的同步控制接口，同步控制粒度由浅到深，帮助开发者精准适配硬件架构，挖掘异构计算的性能极限。

第一种：和静态Tensor编程方式一致的同步接口，主要通过asc_sync_notify/asc_sync_wait接口来精细化管理，需要手动管理事件的类型和事件ID，还需要考虑正向同步（循环内依赖）与反向同步（循环间依赖）。极致性能场景推荐使用此方式。使用示例如下：

```
// 本片段仅用于说明数据搬运、向量计算、同步操作间的关系。各接口的完整参数及上下文请参考下文中的编程示例。
```

```
asc_copy_gm2ub(); // GM->UB的搬运流水
asc_sync_notify(PIPE_MTE2, PIPE_V, EVENT_ID0);
asc_sync_wait(PIPE_MTE2, PIPE_V, EVENT_ID0);
asc_add(); // 向量计算流水
asc_sync_notify(PIPE_V, PIPE_MTE3, EVENT_ID0);
asc_sync_wait(PIPE_V, PIPE_MTE3, EVENT_ID0);
asc_copy_ub2gm(); // UB->GM的搬运流水
```

第二种：不感知流水类型的同步接口，将asc_sync接口添加在对应流水类型的指令后面来实现。使用这类同步接口时，不需要考虑指令流水类型，接口内部会自动管理所有指令流水的同步，简化同步指令。性能不敏感场景下，可以使用此方式。使用示例如下：

```
// 本片段仅用于说明数据搬运、向量计算、同步操作间的关系。各接口的完整参数及上下文请参考下文中的编程示例。
```

```
asc_copy_gm2ub(); // GM->UB的搬运流水
asc_sync(); // 全同步 无需考虑后面的指令流水
asc_add(); // 向量计算流水
asc_sync(); // 全同步 无需考虑后面的指令流水
asc_copy_ub2gm(); // UB->GM的搬运流水
```

另外，C API还提供了一组包含同步能力的搬运及计算接口，开发者无需显式手动管理同步，同步操作隐藏在相应的接口中。性能不敏感场景下，推荐使用此方式。使用示例如下：

```
// 本片段仅用于说明数据搬运、向量计算、同步操作间的关系。各接口的完整参数及上下文请参考下文中的编程示例。
```

```
asc_copy_gm2ub_sync(); // GM->UB的搬运流水同时包含了和后面的任意指令流水的同步
asc_add_sync(); // 向量计算流水同时包含了和后面的任意指令流水的同步
asc_copy_ub2gm_sync(); // UB->GM的搬运流水同时包含了和后面的任意指令流水的同步
```

编程示例

内存管理与精细化同步完整示例：

```
#include <stdint>
#include "c_api/asc_simd.h"

constexpr uint32_t C_API_ONE_BLOCK_SIZE = 32;
constexpr uint32_t C_API_ONE_REPEAT_BYTE_SIZE = 256;
constexpr uint32_t C_API_TOTAL_LENGTH = 16384;
constexpr uint32_t C_API_TILE_NUM = 8;
constexpr uint32_t C_API_TILE_LENGTH = 256;

__vector__ __global__ __aicore__ void add_custom(__gm__ float* x, __gm__ float* y, __gm__ float* z)
{
```



```
asc_init();

uint32_t blockLength = C_API_TOTAL_LENGTH / asc_get_block_num();
uint32_t tileLength = blockLength / C_API_TILE_NUM;

__gm__ float* xGm = x + asc_get_block_idx() * blockLength;
__gm__ float* yGm = y + asc_get_block_idx() * blockLength;
__gm__ float* zGm = z + asc_get_block_idx() * blockLength;

__ubuf__ float xLocal[C_API_TILE_LENGTH];
__ubuf__ float yLocal[C_API_TILE_LENGTH];
__ubuf__ float zLocal[C_API_TILE_LENGTH];

uint16_t burst_len = tileLength;
for (uint32_t i = 0; i < C_API_TILE_NUM; i++) {
    if (i != 0) {
        asc_sync_wait(PIPE_V, PIPE_MTE2, EVENT_ID0);
    }

    burst_len = tileLength * sizeof(float) / C_API_ONE_BLOCK_SIZE;
    asc_copy_gm2ub(xLocal, xGm + i * tileLength, 0, 1, burst_len, 0, 0);
    asc_copy_gm2ub(yLocal, yGm + i * tileLength, 0, 1, burst_len, 0, 0);

    asc_sync_notify(PIPE_MTE2, PIPE_V, EVENT_ID0);
    asc_sync_wait(PIPE_MTE2, PIPE_V, EVENT_ID0);

    if (i != 0) {
        asc_sync_wait(PIPE_MTE3, PIPE_V, EVENT_ID0);
    }

    asc_add(zLocal, xLocal, yLocal, tileLength * sizeof(float) / C_API_ONE_REPEAT_BYTE_SIZE, 1, 1, 1, 8, 8);

    if (i != (C_API_TILE_NUM-1)) {
        asc_sync_notify(PIPE_V, PIPE_MTE2, EVENT_ID0);
    }

    asc_sync_notify(PIPE_V, PIPE_MTE3, EVENT_ID0);
    asc_sync_wait(PIPE_V, PIPE_MTE3, EVENT_ID0);

    asc_copy_ub2gm(zGm + i * tileLength, zLocal, 0, 1, burst_len, 0, 0);

    if (i != (C_API_TILE_NUM-1)) {
        asc_sync_notify(PIPE_MTE3, PIPE_V, EVENT_ID0);
    }
}
}
```

内存管理与不感知流水类型的同步管理完整示例如下：

```
#include <cstdint>
#include "c_api/asc_simd.h"

constexpr uint32_t TILE_LENGTH = 2048;
constexpr uint32_t NUM_BLOCKS = 8;

__vector__ __global__ __aicore__ void add_custom(__gm__ float* x, __gm__ float* y, __gm__ float* z)
{
    asc_init();

    uint32_t blockLength = NUM_BLOCKS * TILE_LENGTH / asc_get_block_num();

    __gm__ float* xGm = x + asc_get_block_idx() * blockLength;
    __gm__ float* yGm = y + asc_get_block_idx() * blockLength;
    __gm__ float* zGm = z + asc_get_block_idx() * blockLength;

    __ubuf__ float xLocal[TILE_LENGTH];
    __ubuf__ float yLocal[TILE_LENGTH];
    __ubuf__ float zLocal[TILE_LENGTH];
```

```
asc_copy_gm2ub((__ubuf__ void*)xLocal, (__gm__ void*)xGm, blockLength * sizeof(float));
asc_copy_gm2ub((__ubuf__ void*)yLocal, (__gm__ void*)yGm, blockLength * sizeof(float));
asc_sync();

asc_add(zLocal, xLocal, yLocal, blockLength);
asc_sync();

asc_copy_ub2gm((__gm__ void*)zGm, (__ubuf__ void*)zLocal, blockLength * sizeof(float));
asc_sync();
}
```

内存管理与使用带同步能力的接口完整示例如下：

```
#include <cstdint>
#include "c_api/asc_simd.h"

constexpr uint32_t TILE_LENGTH = 2048;
constexpr uint32_t NUM_BLOCKS = 8;

__vector__ __global__ __aicore__ void add_custom(__gm__ float* x, __gm__ float* y, __gm__ float* z)
{
    asc_init();

    __ubuf__ float xLocal[TILE_LENGTH];
    __ubuf__ float yLocal[TILE_LENGTH];
    __ubuf__ float zLocal[TILE_LENGTH];

    uint32_t blockLength = TILE_LENGTH * NUM_BLOCKS / asc_get_block_num();

    asc_copy_gm2ub_sync((__ubuf__ void*)xLocal, (__gm__ void*)(x + asc_get_block_idx() * blockLength),
        blockLength * sizeof(float));
    asc_copy_gm2ub_sync((__ubuf__ void*)yLocal, (__gm__ void*)(y + asc_get_block_idx() * blockLength),
        blockLength * sizeof(float));

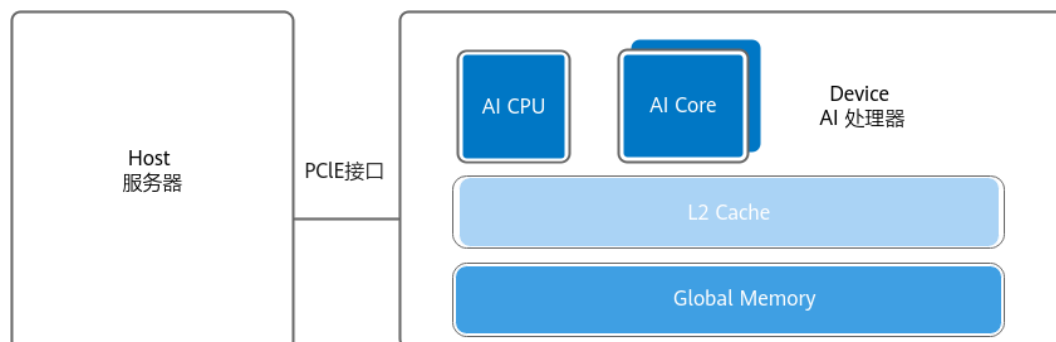
    asc_add_sync(zLocal, xLocal, yLocal, blockLength);

    asc_copy_ub2gm_sync((__gm__ void*)(z + asc_get_block_idx() * blockLength), (__ubuf__ void*)zLocal,
        blockLength * sizeof(float));
}
```

2.2.4.2 AI CPU 编程

AI CPU是位于Device侧ARM64架构的处理器，其具备与AI Core相同的内存访问能力，可直接访问Device侧内存资源；也可以与Host侧的CPU一样，进行类似的数据计算，通常作为AI Core的补充，主要承担非矩阵类、逻辑比较复杂的分支密集型计算。AI CPU的运行环境为基础的Linux环境，编程时可使用libc库，C++标准库，STL模板库等。其硬件架构图如下所示：

图 2-10 AI CPU 硬件架构图



本节介绍的AI CPU编程仅支持如下产品型号：

- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品

AI CPU 核函数定义

在进行AI CPU编程时，与AI Core类似，同样需要定义设备侧函数入口（即核函数），该函数必须通过__aicpu__标识符进行声明，并且需与__global__标识符联合使用以表明其只能被Host侧调用。AI CPU的Device侧实现文件需要以.aicpu为后缀（或者在编译时增加-x aicpu选项）。该实现文件中包括上面介绍的核函数以及AI CPU普通函数定义，AI CPU普通函数无需添加执行空间标识符。

如下是一个AI CPU “Hello World”程序的示例，hello_world.aicpu文件内容如下：

```
// 调用printf接口需要包含的头文件
#include "aicpu_api.h"

__global__ __aicpu__ uint32_t hello_world(void *args)
{
    AscendC::printf("Hello World!!!\n");
    return 0;
}
```

说明

编程时需要遵循如下规范：

- __aicpu__ __global__函数不能是void返回类型，并且入参只能是一个指针。
- __aicpu__ __global__函数不能是类的成员函数，也不能存在于匿名空间下。
- 尽管AI CPU的Kernel函数有返回值，但该返回值仅用于Runtime组件报告运行状态，开发者无需编写返回逻辑，也无法使用该返回值。因此，对于用户而言，AI CPU Kernel函数等同于void类型，不能作为右值使用。

AI CPU 核函数调用

AI CPU核函数的调用需要在.asc文件中进行，和AI Core的算子调用类似，同样使用<<<>>>语法。

```
hello_world<<<numBlocks, nullptr, stream>>>(&args, sizeof(KernelArgs));
```

- numBlocks：AI CPU Device侧暂不支持分核逻辑，因此Host侧调用多核无实际意义。建议设置为1。
- l2ctrl：保留参数，当前固定为nullptr，开发者无需关注。
- stream：类型为aclrtStream，stream用于维护一些异步操作的执行顺序，确保按照应用程序中的代码调用顺序在Device上执行。stream创建等管理接口请参考“应用开发接口 > AscendCL API（C&C++）> 运行时管理 > Stream管理”章节。

说明

在编写调用代码时需要遵循如下规范：

- __aicpu__ __global__函数不能在.asc文件中进行定义，只能声明，且需要使用extern。
- Host侧调用__global__ __aicpu__函数时必须使用<<<>>>异构调用语法，输入的函数入参在入参指针的基础上需要输入从指针中读取的数据大小。
- 在Host侧使用内核调用符<<<...>>>调用AI Core与AI CPU算子时不能使用同一条stream。

加载和运行算子时，需要使用Runtime API，完成运行时管理和配置，详细内容请参考[2.3.3 算子运行](#)。AI CPU算子的编译请参考[2.3.2 AI CPU算子编译](#)。

AI CPU 模板核函数

若需要使用模板核函数，则需要[在.aicpu文件中给出模板核函数的实例化声明](#)，参考如下：

```
template<typename T, int BUFF_SIZE>
__global__ __aicpu__ uint32_t hello_world(void *args)
{
    AscendC::printf("Hello World!!!\n");
    AscendC::printf("buffer_size is %d\n", BUFF_SIZE);
    return 0;
}
template __global__ __aicpu__ uint32_t hello_world<KernelArgs, 4096>(void *args);
```

并在.asc文件中新增模板核函数实例化的extern声明：

```
template<typename T, int BUFF_SIZE>
extern __global__ __aicpu__ uint32_t hello_world(void *args);

template extern __global__ __aicpu__ uint32_t hello_world<KernelArgs, 4096>(void *args);
```

更多进阶用法

说明

更多AI CPU API的使用方法请参考AI CPU API。

2.3 编译与运行

2.3.1 AI Core 算子编译

2.3.1.1 算子编译简介

本章节介绍的算子编译方法支持开发者通过bisheng命令行和CMake进行手动配置编译选项，或编写CMake脚本来实现编译。开发者可以将Host侧main.cpp和Device侧Kernel核函数置于同一实现文件中，以实现异构编译。

- 目前，该编译方法仅支持如下型号：
 - Atlas A3 训练系列产品/Atlas A3 推理系列产品
 - Atlas A2 训练系列产品/Atlas A2 推理系列产品
 - Atlas 推理系列产品
- 当前版本暂不支持CPU孪生调试功能，NPU仿真调试功能。
- 异构编译场景中的编程相关约束请参考[2.3.1.6 约束说明](#)。

2.3.1.2 通过 bisheng 命令行编译

毕昇编译器是一款专为AI处理器设计的编译器，支持异构编程扩展，可以将用户编写的昇腾算子代码编译成二进制可执行文件和动态库等形式。毕昇编译器的可执行程序命名为bisheng，支持x86、aarch64等主机系统，并且原生支持设备侧AI Core架构指令集编译。通过使用毕昇编译器，用户可以更加高效地进行针对昇腾AI处理器的编程和开发工作。

入门示例

以下是一个使用毕昇编译器编译静态Shape的add_custom算子入门示例。该示例展示了如何编写源文件add_custom.asc以及具体的编译命令。通过这个示例，您可以了解如何使用毕昇编译器进行算子编译。

步骤1 包含头文件。

在编写算子源文件时，需要包含必要的头文件。

```
// 头文件
#include "acl/acl.h"
#include "kernel_operator.h"
```

步骤2 核函数实现。

- 核函数支持模板。
- 核函数入参支持传入用户自定义的结构体，比如示例中用户自定义的AddCustomTilingData结构体。

```
// 用户自定义的TilingData结构体
struct AddCustomTilingData {
    uint32_t totalLength;
    uint32_t tileNum;
};

// Kernel核心实现逻辑，包括搬运，计算等
class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    // ...
};

template <typename T>
__global__ __vector__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData
tiling) // 该算子执行时仅启动AI Core上的Vector核
{
    KernelAdd<T> op;
    op.Init(x, y, z, tiling.totalLength, tiling.tileNum);
    op.Process();
}
```

步骤3 Host侧调用函数逻辑，包括内存申请和释放，初始化和去初始化，内核调用符调用核函数等。

```
// Host侧应用程序需要包含的头文件
#include "acl/acl.h"
// Kernel侧需要包含的头文件
#include "kernel_operator.h"
// 核函数开发部分
...

__global__ __vector__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData tiling)
{
    KernelAdd op;
    op.Init(x, y, z, tiling.totalLength, tiling.tileNum);
    op.Process();
}

// 通过<<<...>>>内核调用符调用算子
std::vector<float> kernel_add(std::vector<float> &x, std::vector<float> &y)
{
    ...
}
```

```
// 计算结果比对
uint32_t VerifyResult(std::vector<float> &output, std::vector<float> &golden)
{
    ...
}

// 算子验证主程序
int32_t main(int32_t argc, char *argv[])
{
    constexpr uint32_t totalLength = 8 * 2048;
    constexpr float valueX = 1.2f;
    constexpr float valueY = 2.3f;
    std::vector<float> x(totalLength, valueX);
    std::vector<float> y(totalLength, valueY);

    std::vector<float> output = kernel_add(x, y);

    std::vector<float> golden(totalLength, valueX + valueY);
    return VerifyResult(output, golden);
}
```

步骤4 采用如下的编译命令进行编译。

- -o demo：指定输出文件名为demo。
- --npu-arch=dav-2201：指定NPU的架构版本为dav-2201。dav-后为NPU架构版本号，各产品型号对应的架构版本号请通过[对应关系表](#)进行查询。

```
bisheng add_custom.asc -o demo --npu-arch=dav-2201
```

步骤5 执行可执行文件。

```
./demo
```

----结束

程序的编译与执行

通过毕昇编译器可以将算子源文件（以.asc为后缀）编译为当前平台的可执行文件或算子动态库，静态库。此外，也支持编译以.cpp/.c等为后缀的C++/C源文件，但需要增加-x asc编译选项。

- 编译生成可执行文件

```
# 1.编译hello_world.cpp为当前平台可执行文件
# bisheng [算子源文件] -o [输出产物名称] --npu-arch=[NPU架构版本号]，常见参数顺序与g++保持一致。
bisheng -x asc add_custom.cpp -o add_custom --npu-arch=dav-xxxx
```

生成的可执行文件可通过如下方式执行：

```
./add_custom
```

- 编译生成算子动态库

```
# 2.编译add_custom_base.cpp生成算子动态库
# bisheng -shared [算子源文件] -o [输出产物名称] --npu-arch=[NPU架构版本号]
# 动态库
bisheng -shared -x asc add_custom_base.cpp -o libadd.so --npu-arch=dav-xxxx
```

- 编译生成算子静态库

```
# 3.编译add_custom_base.cpp生成算子静态库
bisheng -lib [算子源文件] -o [输出产物名称] --npu-arch=[NPU架构版本号]
# 静态库
bisheng -lib -x asc add_custom_base.cpp -o libadd.a --npu-arch=dav-xxxx
```

在命令行编译场景下，可以按需链接需要的库文件，常见的库文件请参考[常用的链接库](#)。编译时会默认链接[表2-8](#)中列出的库文件。注意如下例外场景：在使用g++链接asc代码编译生成的静态库时，需要手动链接默认链接库。

2.3.1.3 常用的编译选项

常用的编译选项

常用的编译选项说明如下，全量的编译选项请参考毕昇编译器编译选项。

选项	是否必需	说明
-help	否	查看帮助。
--npu-arch	是	编译时指定的AI处理器架构，取值为dav- <arch-version>，其中<arch-version>为NPU 架构版本号，各产品型号对应的架构版本号请 通过 对应关系表 进行查询。
--npu-soc	否	编译时指定的AI处理器型号，npu-soc和npu- arch同时配置时，优先使能npu-arch。 AI处理器的型号请通过如下方式获取： - 针对如下产品：在安装AI处理器的服务器执行 npu-smi info命令进行查询，获取Name 信息。实际配置值为AscendName，例如 Name取值为xxxxyy，实际配置值为 Ascendxxxxyy。 Atlas A2 训练系列产品/Atlas A2 推理系列 产品 Atlas 200I/500 A2 推理产品 Atlas 推理系列产品 Atlas 训练系列产品 - 针对如下产品，在安装AI处理器的服务器执行 npu-smi info -t board -i id -c chip_id 命令进行查询，获取Chip Name和NPU Name信息，实际配置值为Chip Name_NPU Name。例如Chip Name取值 为Ascendxxx，NPU Name取值为1234， 实际配置值为Ascendxxx_1234。其中： - id：设备id，通过npu-smi info -l命令 查出的NPU ID即为设备id。 - chip_id：芯片id，通过npu-smi info - m命令查出的Chip ID即为芯片id。 Atlas A3 训练系列产品/Atlas A3 推理系列 产品
-x	否	指定编译语言。 指定为asc时表示Ascend C编程语言。
-o <file>	否	指定输出文件的名称和位置。
-c	否	编译生成目标文件。
-shared, --shared	否	编译生成动态链接库。

选项	是否必需	说明
-lib, --cce-build-static-lib	否	编译生成静态链接库。编译器会将Device侧的代码进行编译链接，生成Device侧二进制文件，随后将该文件作为Host侧编译的输入进行编译，最后链接生成静态链接库。
-g	否	编译时增加调试信息。
--sanitizer	否	编译时增加代码正确性校验信息。使用 sanitizer选项时，需要同步添加-g选项，且不能在-O0场景下使用。 注意，启用该选项后GlobalTensor默认使能L2 Cache，无法通过AscendC::SetL2CacheHint接口设置不使能L2 Cache的模式。
-fPIC	否	告知编译器产生位置无关代码。
-O	否	用于指定编译器的优化级别，当前支持-O3，-O2，-O0。

内置编译宏开关

内置编译宏开关列表如下：

- ASCENDC_DUMP用于控制Dump开关，默认开关打开，开发者调用printf/DumpTensor/assert后会有信息打印（需要注意直调工程的kernel文件内存在host函数，如果在host函数内调用了printf接口，也会触发kernel内的printf相关初始化动作，进而影响kernel的执行性能）；设置为0后，表示开关关闭。示例如下：

```
// 关闭所有算子的printf打印功能
ascendc_compile_definitions(ascendc_kernels_${RUN_MODE} PRIVATE
    ASCENDC_DUMP=0
)
```

- ASCENDC_DEBUG用于控制Ascend C API的调测开关，默认开关关闭；增加该编译宏后，表示开关打开，此时接口内部的assert校验生效，校验不通过会有assert日志打屏。开启该功能会对算子实际运行的性能带来一定影响，通常在调测阶段使用。示例如下：

```
ascendc_compile_definitions(ascendc_kernels_${RUN_MODE} PRIVATE
    ASCENDC_DEBUG
)
```

当前ASCENDC_DEBUG功能支持的产品型号为：

Atlas 推理系列产品

Atlas A2 训练系列产品/Atlas A2 推理系列产品

2.3.1.4 通过 CMake 编译

项目中可以使用CMake来更简便地使用昇昇编译器编译Ascend C算子，生成可执行文件、动态库、静态库或二进制文件。

以下是CMake脚本的示例及其核心步骤说明：

```
# 1、find_package(ASC)是CMake中用于查找和配置Ascend C编译工具链的命令
find_package(ASC)
```



```
# 2、指定项目支持的语言包括ASC和CXX，ASC表示支持使用毕昇编译器对Ascend C编程语言进行编译
project(kernel_samples LANGUAGES ASC CXX)

# 3、使用CMake接口编译可执行文件、动态库、静态库、二进制文件
add_executable(demo
    add_custom.asc
)
#.....
target_compile_options(demo PRIVATE
    # --npu-arch用于指定NPU的架构版本，dav-后为架构版本号，各产品型号对应的架构版本号请通过对应关系表进行查询。
    # <COMPILE_LANGUAGE:ASC>:表明该编译选项仅对语言ASC生效
    $<$<COMPILE_LANGUAGE:ASC>: --npu-arch=dav-2201>
)

```

以下是动态库、静态库编译示例，同时展示如何将源文件切换为用语言ASC编译：

- 编译.cpp文件生成动态库

将.cpp文件置为ASC属性，启用Ascend C语言进行编译

```
set_source_files_properties(
    add_custom_base.cpp
    sub_custom_base.cpp
    PROPERTIES LANGUAGE ASC
)

add_library(kernel_lib SHARED
    add_custom_base.cpp
    sub_custom_base.cpp
)

add_executable(demo
    main.cpp
)
target_compile_definitions(demo PRIVATE
    ASCENDC_DUMP=0
)
target_compile_options(demo PRIVATE
    -g
)
target_include_directories(demo PRIVATE
    include
)
target_link_libraries(demo PRIVATE
    kernel_lib
)

```

- 编译.asc文件生成静态库

.asc文件会默认启用Ascend C语言进行编译，不需要通过set_source_files_properties进行设置

```
add_library(kernel_lib STATIC
    add_custom_base.asc
    sub_custom_base.asc
)

add_executable(demo
    main.cpp
)
target_compile_definitions(demo PRIVATE
    ASCENDC_DUMP=0
)
target_compile_options(demo PRIVATE
    -g
)
target_include_directories(demo PRIVATE
    include
)
target_link_libraries(demo PRIVATE
    kernel_lib
)

```

下文列出了使用CMake编译时常用的链接库、以及默认链接库。

表 2-7 常用的链接库（在使用高阶 API 时，必须链接以下库，因为这些库是高阶 API 功能所依赖的。在其他场景下，可以根据具体需求选择是否链接这些库。）

名称	作用描述	使用场景
libtiling_api.a	Tiling函数相关库。	使用高阶API相关的Tiling接口时需要链接。
libregister.so	Tiling注册相关库。	使用高阶API相关的Tiling接口时需要链接。
libgraph_base.so	基础数据结构和接口库。	调用ge::Shape，ge::DataType等基础结构体时需要链接。
libplatform.so	硬件平台信息库。	使用PlatformAscendC相关硬件平台信息接口时需要链接。

表 2-8 默认链接库

名称	作用描述
libascendc_runtime.a	Ascend C算子参数等组装库。
libruntime.so	Runtime运行库。
libprofapi.so	Ascend C算子运行性能数据采集库。
libunified_dlog.so	CANN日志收集库。
libmmpa.so	CANN系统接口库。
libascend_dump.so	CANN维测信息库。
libc_sec.so	CANN安全函数库。
liberror_manager.so	CANN错误信息管理库。
libascendcl.so	acl相关接口库。

2.3.1.5 RTC

RTC是Ascend C运行时编译库，通过aclrtc接口，在程序运行时，将中间代码动态编译成目标机器码，提升程序运行性能。

运行时编译库提供以下核心接口：

- aclrtcCreateProg：根据输入参数（字符串形式表达的Ascend C源代码等）创建aclrtcProg程序实例。
- aclrtcCompileProg：编译给定的程序，支持用户自定义编译选项，比如指定NPU架构版本号：--npu-arch=dav-2201。支持的编译选项可以参考昇腾编译器编译选项。

- `aclrtcGetBinDataSize`: 获取编译后的Device侧二进制数据的大小。
- `aclrtcGetBinData`: 获取编译后的Device侧二进制数据。
- `aclrtcDestroyProg`: 在编译和执行过程结束后, 销毁给定的程序。

编译完成后需要调用如下接口完成(仅列出核心接口) Kernel加载与执行。完整流程和详细接口说明请参考“Kernel加载与执行”章节。

1. 通过aclrtBinaryLoadFromData接口解析由aclrtcGetBinData接口获取的算子二进制数据。
2. 获取核函数句柄并根据核函数句柄操作其参数列表, 相关接口包括aclrtBinaryGetFunction(获取核函数句柄)、aclrtKernelArgsInit(初始化参数列表)、aclrtKernelArgsAppend(追加拷贝用户设置的参数值如`xDevice`, `yDevice`, `zDevice`)等。
3. 调用aclrtLaunchKernelWithConfig接口, 启动对应算子的计算任务。

如下是一个使用aclrtc接口编译并运行Add自定义算子的完整样例:

```
#include <iostream>
#include <fstream>
#include <vector>
#include "acl/acl.h"
// 使用aclrtc接口需要包含的头文件
#include "acl/acl_rt_compile.h"

#define CHECK_ACL(x)
do {
    aclError __ret = x;
    if (__ret != ACL_ERROR_NONE) {
        std::cerr << __FILE__ << ": " << __LINE__ << " aclError: " << __ret << std::endl;
    }
} while (0);

int main(int argc, char *argv[])
{
    // ----- aclrtc part -----
    const char *src = R"("""(
#include "kernel_operator.h"
constexpr int32_t TOTAL_LENGTH = 8 * 1024;           // total length of data
constexpr int32_t USE_CORE_NUM = 8;                 // num of core used
constexpr int32_t BLOCK_LENGTH = TOTAL_LENGTH / USE_CORE_NUM; // length computed of each core
constexpr int32_t TILE_NUM = 8;                     // split data into 8 tiles for each core
constexpr int32_t BUFFER_NUM = 2;                   // tensor num for each queue
constexpr int32_t TILE_LENGTH = BLOCK_LENGTH / TILE_NUM / BUFFER_NUM; // separate to 2 parts, due to double buffer

class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z)
    {
        xGm.SetGlobalBuffer((__gm__ float *)x + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);
        yGm.SetGlobalBuffer((__gm__ float *)y + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);
        zGm.SetGlobalBuffer((__gm__ float *)z + BLOCK_LENGTH * AscendC::GetBlockIdx(), BLOCK_LENGTH);
        pipe.InitBuffer(inQueueX, BUFFER_NUM, TILE_LENGTH * sizeof(float));
        pipe.InitBuffer(inQueueY, BUFFER_NUM, TILE_LENGTH * sizeof(float));
        pipe.InitBuffer(outQueueZ, BUFFER_NUM, TILE_LENGTH * sizeof(float));
    }
    __aicore__ inline void Process()
    {
        int32_t loopCount = TILE_NUM * BUFFER_NUM;
        for (int32_t i = 0; i < loopCount; i++) {
            CopyIn(i);
            Compute(i);
            CopyOut(i);
        }
    }
};
```

```
}

private:
__aicore__ inline void CopyIn(int32_t progress)
{
    AscendC::LocalTensor<float> xLocal = inQueueX.AllocTensor<float>();
    AscendC::LocalTensor<float> yLocal = inQueueY.AllocTensor<float>();
    AscendC::DataCopy(xLocal, xGm[progress * TILE_LENGTH], TILE_LENGTH);
    AscendC::DataCopy(yLocal, yGm[progress * TILE_LENGTH], TILE_LENGTH);
    inQueueX.EnQue(xLocal);
    inQueueY.EnQue(yLocal);
}
__aicore__ inline void Compute(int32_t progress)
{
    AscendC::LocalTensor<float> xLocal = inQueueX.DeQue<float>();
    AscendC::LocalTensor<float> yLocal = inQueueY.DeQue<float>();
    AscendC::LocalTensor<float> zLocal = outQueueZ.AllocTensor<float>();
    AscendC::Add(zLocal, xLocal, yLocal, TILE_LENGTH);

    outQueueZ.EnQue<float>(zLocal);
    inQueueX.FreeTensor(xLocal);
    inQueueY.FreeTensor(yLocal);
}
__aicore__ inline void CopyOut(int32_t progress)
{
    AscendC::LocalTensor<float> zLocal = outQueueZ.DeQue<float>();
    AscendC::DataCopy(zGm[progress * TILE_LENGTH], zLocal, TILE_LENGTH);
    outQueueZ.FreeTensor(zLocal);
}
}

private:
    AscendC::TPipe pipe;
    AscendC::TQue<AscendC::TPosition::VECIN, BUFFER_NUM> inQueueX, inQueueY;
    AscendC::TQue<AscendC::TPosition::VECOUT, BUFFER_NUM> outQueueZ;
    AscendC::GlobalTensor<float> xGm;
    AscendC::GlobalTensor<float> yGm;
    AscendC::GlobalTensor<float> zGm;
};
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z)
{
    KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_AIV_ONLY);
    KernelAdd op;
    op.Init(x, y, z);
    op.Process();
}
)""";
// aclrtc流程, src为用户Device侧源码, 通过aclrtcCreateProg来创建编译程序
aclrtcProg prog;
CHECK_ACL(aclrtcCreateProg(&prog, src, "add_custom", 0, nullptr, nullptr));

// aclrtc流程, 传入毕昇编译器的编译选项, 调用aclrtcCompileProg进行编译
const char *options[] = {
    "--npu-arch=dav-2201",
};
int numOptions = sizeof(options) / sizeof(options[0]);
CHECK_ACL(aclrtcCompileProg(prog, numOptions, options));

// aclrtc流程, 获取Device侧二进制内容和大小
size_t binDataSizeRet;
CHECK_ACL(aclrtcGetBinDataSize(prog, &binDataSizeRet));
std::vector<char> deviceELF(binDataSizeRet);
CHECK_ACL(aclrtcGetBinData(prog, deviceELF.data()));

const char *funcName = "add_custom";
// ----- aclrt part -----
uint32_t numBlocks = 8;
size_t inputByteSize = 8 * 1024 * sizeof(uint32_t);
size_t outputByteSize = 8 * 1024 * sizeof(uint32_t);
CHECK_ACL(aclInit(nullptr));
```

```
int32_t deviceId = 0;
CHECK_ACL(aclrtSetDevice(deviceId));
aclrtStream stream = nullptr;
CHECK_ACL(aclrtCreateStream(&stream));

uint8_t *xHost, *yHost, *zHost;
uint8_t *xDevice, *yDevice, *zDevice;
CHECK_ACL(aclrtMallocHost((void **)&xHost, inputByteSize));
CHECK_ACL(aclrtMallocHost((void **)&yHost, inputByteSize));
CHECK_ACL(aclrtMallocHost((void **)&zHost, outputByteSize));
CHECK_ACL(aclrtMalloc((void **)&xDevice, inputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
CHECK_ACL(aclrtMalloc((void **)&yDevice, inputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
CHECK_ACL(aclrtMalloc((void **)&zDevice, outputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
CHECK_ACL(aclrtMemcpy(xDevice, inputByteSize, xHost, inputByteSize,
ACL_MEMCPY_HOST_TO_DEVICE));
CHECK_ACL(aclrtMemcpy(yDevice, inputByteSize, yHost, inputByteSize,
ACL_MEMCPY_HOST_TO_DEVICE));

aclrtBinHandle binHandle = nullptr;
aclrtBinaryLoadOptions loadOption;
loadOption.numOpt = 1;
aclrtBinaryLoadOption option;
option.type = ACL_RT_BINARY_LOAD_OPT_LAZY_MAGIC;
option.value.magic = ACL_RT_BINARY_MAGIC_ELF_VECTOR_CORE; // 设置magic值, 表示算子在Vector
Core上执行
loadOption.options = &option;
CHECK_ACL(aclrtBinaryLoadFromData(deviceELF.data(), binDataSizeRet, &loadOption, &binHandle));
aclrtFuncHandle funcHandle = nullptr;

CHECK_ACL(aclrtBinaryGetFunction(binHandle, funcName, &funcHandle));

aclrtArgsHandle argsHandle = nullptr;
aclrtParamHandle paramHandle = nullptr;
CHECK_ACL(aclrtKernelArgsInit(funcHandle, &argsHandle));
CHECK_ACL(aclrtKernelArgsAppend(argsHandle, (void **)&xDevice, sizeof(uintptr_t), &paramHandle));
CHECK_ACL(aclrtKernelArgsAppend(argsHandle, (void **)&yDevice, sizeof(uintptr_t), &paramHandle));
CHECK_ACL(aclrtKernelArgsAppend(argsHandle, (void **)&zDevice, sizeof(uintptr_t), &paramHandle));
CHECK_ACL(aclrtKernelArgsFinalize(argsHandle));
// 核函数入口
CHECK_ACL(aclrtLaunchKernelWithConfig(funcHandle, numBlocks, stream, nullptr, argsHandle, nullptr));

CHECK_ACL(aclrtSynchronizeStream(stream));
CHECK_ACL(aclrtMemcpy(zHost, outputByteSize, zDevice, outputByteSize,
ACL_MEMCPY_DEVICE_TO_HOST));

// 获取日志大小并得到日志字符串
size_t logSize;
CHECK_ACL(aclrtcGetCompileLogSize(prog, &logSize));
char* log = (char*)malloc(logSize);
CHECK_ACL(aclrtcGetCompileLog(prog, log));
// 将日志字符串存到文件中
/*
std::ofstream logFile("compile.log");
if (logFile.is_open()) {
logFile << log << std::endl;
logFile.close();
std::cout << "already write to compile.log!" << std::endl;
}
*/
free(log);
CHECK_ACL(aclrtBinaryUnLoad(binHandle));
CHECK_ACL(aclrtFree(xDevice));
CHECK_ACL(aclrtFree(yDevice));
CHECK_ACL(aclrtFree(zDevice));
CHECK_ACL(aclrtFreeHost(xHost));
CHECK_ACL(aclrtFreeHost(yHost));
CHECK_ACL(aclrtFreeHost(zHost));
CHECK_ACL(aclrtDestroyStream(stream));
CHECK_ACL(aclrtResetDevice(deviceId));
```

```
CHECK_ACL(aclFinalize());

// 编译和运行均已结束，销毁程序
CHECK_ACL(aclrtcDestroyProg(&prog));

return 0;
}
```

编译命令如下，编译时需要设置-I路径为\${INSTALL_DIR}/include，用于找到aclrtc相关头文件，并需要链接alc_rtc动态库。

```
g++ add_custom.cpp -I${INSTALL_DIR}/include -L${INSTALL_DIR}/lib64 -lascendcl -lalc_rtc -o main
```

\${INSTALL_DIR}请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。

2.3.1.6 约束说明

- 在同一个编译单元，若存在多个核函数，暂不支持自动推导Kernel类型，需要开发者手动设置Kernel类型。
 - 特别地，针对如下型号，无论是否是同一个编译单元多个核函数的场景，均不支持在开发者未设置Kernel类型时进行自动推导。建议开发者手动设置Kernel类型。
 - Atlas 推理系列产品
 - 针对Atlas 推理系列产品，暂不支持设置Kernel类型为KERNEL_TYPE_MIX_VECTOR_CORE。
- KERNEL_TASK_TYPE_DEFAULT接口需在核函数或者inline的aicore函数中进行调用，禁止在非inline的aicore函数中进行调用。
- 纯Scalar算子无法实现自动推导
需手动标记Kernel函数类型，推荐设置为纯Vector类型，添加__vector__ attribute 进行标记：

```
__global__ __vector__ __aicore__ void func0(__gm__ uint8* Addr) {
    Addr[1] = Addr[0];
    AscendC::printf("Hello world");
}
```

- Kernel函数不支持特化
使用特化核函数调用，目前依然会使用主模板核函数。

```
#include "kernel_operator.h"
#include "acl/acl.h"

// 测试__mix__(x, y)中x, y由模板参数控制是否可行
template<int32_t cube, int32_t vec>
__mix__(cube, vec) __global__ void hello_world()
{
    if ASCEND_IS_AIC {
        AscendC::printf("Hello World AIC with cube, vec !!!\n");
    } else {
        AscendC::printf("Hello World AIV with cube, vec !!!\n");
    }
}

// 特化写法，不支持
template<>
__mix__(1, 0) __global__ void hello_world<1, 0>()
{
    if ASCEND_IS_AIC {
        AscendC::printf("1:0 Hello World AIC with vec !!!\n");
    } else {
```

```
        AscendC::printf("1:0 Hello World AIV with vec !!!\n");
    }
}

int32_t main(int argc, char const *argv[])
{
    uint8_t *aHost;
    uint8_t *aDevice;

    aclInit(nullptr);
    int32_t deviceId = 0;
    aclrtSetDevice(deviceId);
    aclrtStream stream = nullptr;
    aclrtCreateStream(&stream);
    aclrtMallocHost((void **)&aHost, sizeof(uint32_t));
    aclrtMalloc((void **)&aDevice, sizeof(uint32_t), ACL_MEM_MALLOC_HUGE_FIRST);
    *aHost = 12;
    aclrtMemcpy(aDevice, sizeof(uint32_t), aHost, sizeof(uint32_t), ACL_MEMCPY_HOST_TO_DEVICE);
    constexpr uint32_t numBlocks = 8;

    hello_world<1, 2><<<numBlocks, nullptr, stream>>>();
    aclrtSynchronizeStream(stream);
    hello_world<1, 0><<<numBlocks, nullptr, stream>>>();
    aclrtSynchronizeStream(stream);

    aclrtFree(aDevice);
    aclrtFreeHost(aHost);
    aclrtDestroyStream(stream);
    aclrtResetDevice(deviceId);
    aclFinalize();
    return 0;
}
```

- bfloat16_t等数据类型在Host侧不支持，使用这些数据类型时，Host和Device不能写在同一个实现文件里。Host侧不支持的数据类型如下：

Atlas A2 训练系列产品/Atlas A2 推理系列产品：bfloat16_t。

Atlas A3 训练系列产品/Atlas A3 推理系列产品：bfloat16_t。

- 不支持可变参数模板和可变参数函数

// 不支持以下写法

```
template <typename... Args> // Args 是一个“类型参数包”
void func(Args... args);    // args 是一个“函数参数包”
```

- 不支持#line 预处理

// 不支持以下写法

#line number // 更改行号

#line number "filename" // 更改文件名

2.3.2 AI CPU 算子编译

通过 bisheng 命令行编译

下文基于一个Hello World打印样例来讲解如何通过bisheng命令行编译AI CPU算子。

hello_world.aicpu文件内容如下：

```
#include "aicpu_api.h"

__global__ __aicpu__ uint32_t hello_world(void *args)
{
    AscendC::printf("Hello World!!!\n");
    return 0;
}
```

Host侧使用内核调用符<<<...>>>进行AI CPU算子的调用，main.asc示例代码如下：

```
#include "acl/acl.h"

struct KernelArgs {
    int mode;
};

extern __global__ __aicpu__ uint32_t hello_world(void *args);

int32_t main(int argc, char const *argv[])
{
    aclInit(nullptr);
    int32_t deviceId = 0;
    aclrtSetDevice(deviceId);
    aclrtStream stream = nullptr;
    aclrtCreateStream(&stream);

    struct KernelArgs args = {0};
    constexpr uint32_t numBlocks = 1;
    hello_world<<<numBlocks, nullptr, stream>>>(&args, sizeof(KernelArgs));
    aclrtSynchronizeStream(stream);

    aclrtDestroyStream(stream);
    aclrtResetDevice(deviceId);
    aclFinalize();
    return 0;
}
```

开发者可以使用bisheng命令行将hello_world.aicpu与main.asc分别编译成.o，再链接成为可执行文件，编译命令如下：

- 编译hello_world.aicpu时，通过-l指定依赖头文件所在路径；通过--cce-aicpu-laicpu_api为Device链接依赖的库libaicpu_api.a，通过--cce-aicpu-L指定libaicpu_api.a的库路径。
- 编译main.asc时，通过--npu-arch编译选项指定对应的架构版本号。

`\${INSTALL\_DIR}`请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。

```
$bisheng -O2 hello_world.aicpu --cce-aicpu-L${INSTALL_DIR}/lib64/device/lib64 --cce-aicpu-laicpu_api -I${INSTALL_DIR}/include/ascendc/aicpu_api -c -o hello_world.aicpu.o
# --npu-arch用于指定NPU的架构版本，dav-后为架构版本号，各产品型号对应的架构版本号请通过对应关系表
# 进行查询。
$bisheng --npu-arch=dav-2201 main.asc -c -o main.asc.o
$bisheng hello_world.aicpu.o main.asc.o -o demo
```

上文我们通过一个入门示例介绍了使用bisheng命令行编译生成可执行文件的示例。除此之外，使用bisheng命令行也支持编译生成AI CPU算子的动态库与静态库，用户可在asc代码中通过内核调用符<<<...>>>调用AI CPU算子的核函数，并在编译asc代码源文件生成可执行文件的时候，链接AI CPU动态库或者静态库，注意：若单独编译AI CPU算子代码生成动态库、静态库时，需要手动链接[表2-8](#)。

- 编译生成算子动态库
编译test_aicpu.cpp生成算子动态库
-lxxx表示默认链接库
bisheng -shared -x aicpu test_aicpu.cpp -o libtest_aicpu.so -lxxx ...
- 编译生成算子静态库
编译test_aicpu.cpp生成算子静态库
-lxxx表示默认链接库
bisheng -lib -x aicpu test_aicpu.cpp -o libtest_aicpu.a -lxxx ...

AI CPU 算子常用编译选项

AI CPU算子常用的编译选项说明如下：

选项	是否必需	说明
-help	否	查看帮助。
-x	否	指定编译语言。 指定为aicpu时表示AI CPU算子编程语言。
-o <file>	否	指定输出文件的名称和位置。
-c	否	编译生成目标文件。
-shared, --shared	否	编译生成动态链接库。
-lib	否	编译生成静态链接库。
-g	否	编译时增加调试信息。
-fPIC	否	告知编译器产生位置无关代码。
-O	否	用于指定编译器的优化级别，当前支持-O3, -O2, -O0。
--cce-aicpu-L	否	指定AI CPU Device依赖的库路径。
--cce-aicpu-l	否	指定AI CPU Device依赖的库。

通过 CMake 编译

项目中可以使用CMake来更简便地使用毕昇编译器编译AI CPU算子，生成可执行文件、动态库、静态库或二进制文件。

仍以[通过bisheng命令行编译](#)中介绍的Hello World打印样例为例，除了代码实现文件，还需要在工程目录下准备一个CMakeLists.txt。

```
├── hello_world.aicpu // AI CPU算子核函数定义
├── main.asc // AI CPU算子核函数调用
└── CMakeLists.txt
```

CMakeLists.txt内容如下：

```
cmake_minimum_required(VERSION 3.16)
# 1、find_package()是CMake中用于查找和配置Ascend C编译工具链的命令
find_package(ASC REQUIRED)
find_package(AICPU REQUIRED)

# 2、指定项目支持的语言包括ASC、AICPU和CXX，ASC表示支持使用毕昇编译器对Ascend C编程语言进行编译，AI CPU表示支持使用毕昇编译器对AI CPU算子进行编译
project(kernel_samples LANGUAGES ASC AICPU CXX)

# 3、使用CMake接口编译可执行文件
add_executable(demo
    hello_world.aicpu
    main.asc
)

#4、由于存在ASC与AI CPU语言，需要指定链接器
set_target_properties(demo PROPERTIES LINKER_LANGUAGE ASC) //指定链接使用语言

target_include_directories(demo PUBLIC
    ${INSTALL_DIR}/include/ascendc/aicpu_api
```

```
)  
target_compile_options(demo PRIVATE  
    # --npu-arch用于指定NPU的架构版本，dav-后为架构版本号，各产品型号对应的架构版本号请通过对应关系表进行查询。  
    # <COMPILE_LANGUAGE:ASC>:表明该编译选项仅对语言ASC生效  
    $<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-2201>  
)
```

如果需要CMake编译生成动态库、静态库，下面提供了更详细具体的编译示例：

- 编译.cpp文件生成动态库

```
# 将.cpp文件置为ASC属性，启用Ascend C语言进行编译  
set_source_files_properties(  
    add_custom_base.cpp  
    sub_custom_base.cpp  
    PROPERTIES LANGUAGE ASC  
)  
  
# 将.cpp文件置为AICPU属性，支持AI CPU算子编译  
set_source_files_properties(  
    aicpu_kernel.cpp  
    PROPERTIES LANGUAGE AICPU  
)  
  
add_library(kernel_lib SHARED  
    add_custom_base.cpp  
    sub_custom_base.cpp  
    aicpu_kernel.cpp # 支持AI CPU算子与AI Core算子一起打包为动态库  
)  
  
# AI CPU算子编译时，需要手动链接以下依赖库（若指定链接语言为ASC时，不需要手动链接以下库）  
target_link_libraries(kernel_lib PRIVATE  
    ascendc_runtime  
    profapi  
    unified_dlog  
    ascendcl  
    runtime  
    c_sec  
    mmpa  
    error_manager  
    ascend_dump  
)  
  
add_executable(demo  
    main.cpp  
)  
target_compile_definitions(demo PRIVATE  
    ASCENDC_DUMP=0  
)  
target_compile_options(demo PRIVATE  
    -g  
)  
target_include_directories(demo PRIVATE  
    include  
)  
target_link_libraries(demo PRIVATE  
    kernel_lib  
)
```

- 编译.asc文件与.aicpu文件生成静态库

```
# .asc文件会默认启用Ascend C语言进行编译，.aicpu文件会默认启用AICPU语言进行编译，不需要通过  
set_source_files_properties进行设置  
add_library(kernel_lib STATIC  
    add_custom_base.asc  
    sub_custom_base.asc  
    aicpu_kernel.aicpu # 可支持AI CPU算子与AI Core算子一起打包为静态库  
)  
  
add_executable(demo  
    main.cpp
```

```
)  
target_compile_definitions(demo PRIVATE  
    ASCENDC_DUMP=0  
)  
target_compile_options(demo PRIVATE  
    -g  
)  
target_include_directories(demo PRIVATE  
    include  
)  
target_link_libraries(demo PRIVATE  
    kernel_lib  
)
```

下文列出了使用CMake编译时常用的变量配置说明、常用的链接库。

表 2-9 常用的变量配置说明

变量	配置说明
CMAKE_BUILD_TYPE	编译模式选项，可配置为： “Release”，Release版本，不包含调试信息，编译最终发布的版本。 “Debug”，Debug版本，包含调试信息，便于开发者开发和调试。
CMAKE_INSTALL_PREFIX	用于指定CMake执行install时，安装的路径前缀，执行install后编译产物（ascendc_library中指定的target以及对应的头文件）会安装在该路径下。默认路径为当前目录的out目录下。
CMAKE_CXX_COMPILER_LAUNCHER	用于配置C++语言编译器（如g++）、毕昇编译器的启动器程序为ccache，配置后即可开启cache缓存编译，加速重复编译并提高构建效率。使用该功能前需要安装ccache。 配置方法如下，在对应的CMakeLists.txt进行设置： set(CMAKE_CXX_COMPILER_LAUNCHER <launcher_program>) 其中<launcher_program>是ccache的安装路径，比如ccache的安装路径为/usr/bin/ccache，示例如下： set(CMAKE_CXX_COMPILER_LAUNCHER /usr/bin/ccache)

表 2-10 常用的链接库（在使用高阶 API 时，必须链接以下库，因为这些库是高阶 API 功能所依赖的。在其他场景下，可以根据具体需求选择是否链接这些库。）

名称	作用描述	使用场景
libtiling_api.a	Tiling函数相关库。	使用高阶API相关的Tiling接口时需要链接。
libregister.so	Tiling注册相关库。	使用高阶API相关的Tiling接口时需要链接。
libgraph_base.so	基础数据结构和接口库。	调用ge::Shape，ge::DataType等基础结构体时需要链接。
libplatform.so	硬件平台信息库。	使用PlatformAscendC相关硬件平台信息接口时需要链接。

表 2-11 编译 AI CPU 算子需要手动链接的库

名称	作用描述
libascendc_runtime.a	Ascend C算子参数等组装库。
libruntime.so	Runtime运行库。
libprofapi.so	Ascend C算子运行性能数据采集库。
libunified_dlog.so	CANN日志收集库。
libmmpa.so	CANN系统接口库。
libascend_dump.so	CANN维测信息库。
libc_sec.so	CANN安全函数库。
liberror_manager.so	CANN错误信息管理库。
libascendcl.so	acl相关接口库。

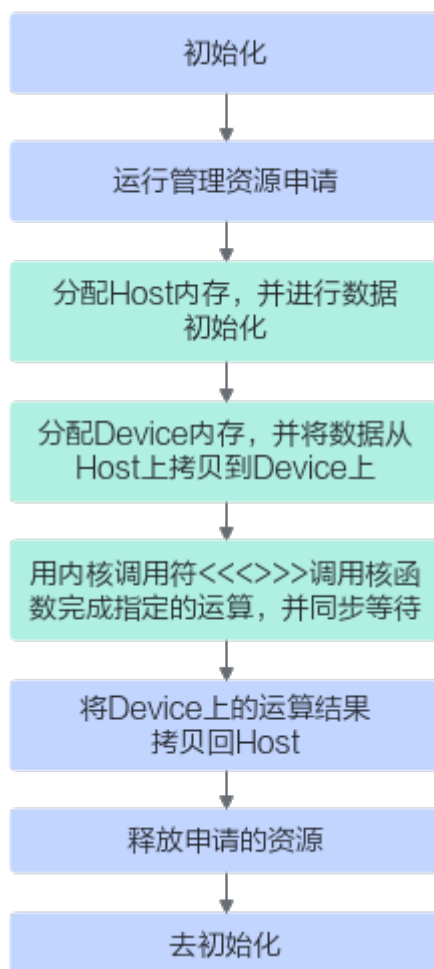
2.3.3 算子运行

算子的计算算法实现通过Ascend C API来完成，而算子的加载调用则使用Runtime API来完成。本章节将结合核函数调用介绍CANN软件栈中Ascend C算子运行时常用的Runtime接口。Runtime接口更多信息与细节可以参考“acl API (C&C++)”章节。

加载和运行代码

加载和运行算子时，需要使用Runtime API，完成运行时管理和配置。主要流程和使用到的API如下：

1. 初始化：aclInit。
2. 运行时资源申请：通过aclrtSetDevice和aclrtCreateStream分别申请Device、Stream运行管理资源。
3. 使用aclrtMallocHost分配Host内存，并进行数据初始化。
4. 使用aclrtMalloc分配Device内存，并通过aclrtMemcpy将数据从Host上拷贝到Device上，参与核函数计算。
5. 使用<<<>>>调用算子核函数。
6. 执行核函数后，将Device上的运算结果拷贝回Host。
7. 异步等待核函数执行完成：aclrtSynchronizeStream。
8. 资源释放：通过aclrtDestroyStream和aclrtResetDevice分别释放Stream、Device运行管理资源。
9. 去初始化：aclFinalize。



Kernel 加载与执行的更多方式

Kernel的加载与执行也可以通过二进制加载方式实现，这是最底层的接口实现方式。内核调用符<<<...>>>为对底层接口的封装实现。使用时需要bisheng命令行编译将算子源文件编译为二进制.o文件，再通过aclrtLaunchKernelWithConfig等Kernel加载与执行接口完成算子调用。

- Kernel加载与执行接口的具体说明请参考“Kernel加载与执行”章节。
- bisheng命令行编译选项的使用介绍请参考[常用的编译选项](#)。
- 完整样例请参考[Kernel加载与执行（加载二进制）样例](#)。

📖 说明

核函数的调用是异步的，核函数的调用结束后，控制权立刻返回给主机端，可以调用以下[aclrtSynchronizeStream](#)函数来强制主机端程序等待所有核函数执行完毕。

```
aclError aclrtSynchronizeStream(aclrtStream stream);
```

2.4 语言扩展层

2.4.1 SIMD BuiltIn 关键字和 API

预定义宏

如其他语言一样，会提供一些内置的宏方便用户编写程序。预定义宏一节着重介绍一些用户在做异构编程时会经常用到的宏，以及宏的解释。

- **__NPU_ARCH__**

__NPU_ARCH__ 是 Device 侧 AI Core 代码中的预处理宏，用于标识 AI 处理器的架构版本。该宏由四位数字组成，其中前三位数字用于标识 AI Core 的 IP 核 (Intellectual Property Core) 类型，第四位数字标识该 AI Core 同一个 IP 核的配置版本。通过该宏，开发者可以针对不同 AI 处理器，差异化进行代码适配和优化。AI 处理器型号和 __NPU_ARCH__ 对应关系如下表所示：

表 2-12 AI 处理器型号和 __NPU_ARCH__ 的对应关系

AI 处理器型号	__NPU_ARCH__
Atlas A3 训练系列产品/Atlas A3 推理系列产品	2201
Atlas A2 训练系列产品/Atlas A2 推理系列产品	2201
Atlas 200I/500 A2 推理产品	3002
Atlas 推理系列产品	2002
Atlas 训练系列产品	1001

以下为通过 __NPU_ARCH__ 控制在不同 AI 处理器上算子输出值舍入模式的示例。

```
__aicore__ static inline void CopyOut(uint64_t mulLen)
{
    #if __NPU_ARCH__ == 2002
        Cast(dstLocal, srcLocal, RoundMode::CAST_NONE, mulLen); // CAST_NONE 表示舍入模式在转换有精度损失时使用 CAST_RINT 模式，在不涉及精度损失时不进行舍入
    #elif __NPU_ARCH__ == 2201
        Cast(dstLocal, srcLocal, RoundMode::CAST_RINT, mulLen); // CAST_RINT 表示舍入模式为四舍六入五成双舍入
    #endif
    event_t eventVToMTE3 = static_cast<event_t>(GetTPipePtr()->FetchEventID(HardEvent::V_MTE3));
    SetFlag<HardEvent::V_MTE3>(eventVToMTE3);
    WaitFlag<HardEvent::V_MTE3>(eventVToMTE3);
    CommonCopyOut<float>(dstLocal, mulLen); // 拷贝 LocalTensor 至 GlobalTensor
}
```

- **ASCEND_IS_AIV、ASCEND_IS_AIC**

ASCEND_IS_AIV 和 ASCEND_IS_AIC 是通过 C++ 宏实现的条件判断语句，用于在 __aicore__ 修饰的函数中实现代码的条件编译。基于分离模式（AIC 核和 AIV 核分离）开发融合算子时，算子逻辑中同时涉及 AIV 核和 AIC 核的处理逻辑，并需要进行核间同步，此时需要通过 ASCEND_IS_AIV/ASCEND_IS_AIC 进行 AIV 和 AIC 核代码的隔离。

说明

当使用高阶 API Matmul 时，其内部已通过 REGIST_MATMUL_OBJ 宏方式实现了 AIV 与 AIC 核代码的隔离，用户无需再使用该宏进行处理。

以MatmulNzCustom算子为例，该算子在分离模式下需要分别在AIV核和AIC核上实现不同的逻辑。具体而言，AIV核负责将矩阵数据搬入Unified Buffer，完成数据的重排（将矩阵数据转换为NZ格式），并将其写入Global Memory。而AIC核则直接从Global Memory读取已经重排好的NZ格式数据，并执行矩阵乘法（Matmul）计算。由于AIV核和AIC核的代码逻辑不同，需要通过ASCEND_IS_AIV和ASCEND_IS_AIC宏进行代码隔离，确保在编译时分别生成适用于AIV核和AIC核的代码。

示例伪码如下：

```
template <typename AType, typename BType, typename CType, typename BiasType>
__aicore__ inline void MatmulKernel<AType, BType, CType, BiasType>::Process(AscendC::TPipe *pipe)
{
    // 利用AIV核的Vector计算单元实现ND2NZ格式转换。如下代码中MatrixBtoNZ为将B矩阵进行ND2NZ
    // 格式转换的函数。
    if ASCEND_IS_AIV {
        pipe->InitBuffer(ubBuf, TOTAL_UB_SIZE);
        MatrixBtoNZ<typename B_TYPE::T>(tempGM,
            bGMNZ,
            tiling,
            isTransB,
            ubBuf,
            tiling.baseK,
            tiling.baseN); // Vector侧实现的ND2NZ函数
        SyncAll();
        // AIC核和AIV核同步
        AscendC::CrossCoreSetFlag<0x2, PIPE_MTE3>(0x4);
        return;
    }
    if ASCEND_IS_AIC {
        AscendC::CrossCoreWaitFlag(0x4); // 等待AIV核完成ND2NZ格式转换
    }
    ...
    // 设置左矩阵A、右矩阵B、Bias。
    matmulObj.SetTail(tailM, tailN);
    matmulObj.SetTensorA(aGlobal, false);
    matmulObj.SetTensorB(bGlobal, false);
    if (tiling.isBias) {
        matmulObj.SetBias(biasGlobal);
    }
    // 完成矩阵乘操作
    matmulObj.IterateAll(cGlobal);
    // 结束矩阵乘操作
    matmulObj.End();
}
```

- **ASCENDC_CUBE_ONLY**

ASCENDC_CUBE_ONLY是通过C++宏实现的条件判断语句，用于在__aicore__修饰的函数中实现代码的条件编译。

基于分离模式开发非融合算子时，在只有矩阵计算的算子场景下，可以通过设置ASCENDC_CUBE_ONLY，使能纯Cube模式完成Matmul计算，减少消息通信的性能开销，提升算子性能。



注意

ASCENDC_CUBE_ONLY宏必须在#include "lib/matmul_intf.h"之前设置。

以matmul_custom算子为例，高阶API Matmul默认使用MIX模式，即用户从AIV侧发起消息，通过消息通信框架中转消息后，在AIC侧执行Matmul计算。这套消息处理机制会带来额外的Scalar性能开销。相较于MIX模式，纯Cube模式可以直接跳过消息通信框架，完成Matmul计算，提升算子性能。

示例伪码如下：

```
#define ASCENDC_CUBE_ONLY
#include "lib/matmul_intf.h"

using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, AType>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BType>;
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_NORM> matmulObj;
```

函数执行空间限定符

函数执行空间限定符（Function Execution Space Qualifier）指示函数是在Host侧执行还是在Device侧执行，以及它是否可从Host侧或Device侧调用。

- **__global__**

__global__ 执行空间限定符声明一个Kernel函数。Kernel函数有如下性质：在Device上执行；只能被Host侧函数调用；__global__ 只是表示这是Device侧函数的入口，并不表示具体的设备类型，具体的设备类型由__aicore__ 标记。具有如下使用约束：

- 一个__global__ 函数必须返回void类型，并且不能是class的成员函数。
- 主机侧调用__global__ 函数必须使用<<<>>>异构调用语法。
- __global__ 的调用是异步的，意味着函数返回，并不表示kernel函数在device侧已经执行完成，如果需要同步，需要使用Runtime同步接口显式同步，如aclrtSynchronizeStream接口。

- **__aicore__**

__aicore__ 执行空间限定符声明一个函数，它具有如下属性：

- 在Device侧执行
- 只能被__global__ 函数，或者其他__aicore__ 函数调用

```
// Only callable from device functions with same kind
// of execution space
__aicore__ void bar() {}

// Define a kernel function execute on AI Core device
__global__ __aicore__ void foo() {
    bar(); // OK.
}
```

- **__host__**

__host__ 执行空间限定符声明一个函数，它具有如下属性：

- 只能在Host侧执行
- 只能被Host侧函数调用
- __global__ 和__host__ 不能一起使用

__host__ 限定符是可选项，无函数执行空间限定符定义的函数，默认是host函数。

```
__aicore__ int f() {}

// defines a host side function
int foo() {}

// defines a host side function
__host__ int bar() {
    f(); // Error.
    foo(); // OK.
}

// Error.
__global__ __host__ void kfunc() {}
```


- **__aicpu__**

AI CPU函数执行空间限定符__aicpu__用于指示函数是否为AI CPU Kernel函数，它具有如下属性：

- 在Device侧执行且只能被Host侧函数调用，因此必须与__global__同时声明。
- 一个__global__ __aicpu__函数不能是void返回类型，并且入参只能是一个指针。
- 一个__global__ __aicpu__函数不能在.asc文件中进行定义，只能声明，且需要使用extern。
- Host侧调用__global__ __aicpu__函数时必须使用<<<>>>异构调用语法，输入的函数入参在入参指针的基础上需要输入从指针中读取的数据大小。
- __global__的调用是异步的，意味着函数返回，并不表示kernel函数在Device侧已经执行完成，如果需要同步，需要使用Runtime同步接口显式同步，如aclrtSynchronizeStream接口

```
// Define a AI CPU kernel function in AI CPU device file
__aicpu__ void foo() {} // Error, single __aicpu__ identifier without __global__
__global__ void foo() {} // Error, single __global__ identifier without __aicpu__
__global__ __aicpu__ void foo() {} // Error, return type is void
__global__ __aicpu__ int foo(void *a) {} // OK
__global__ __aicpu__ int foo(int a) {} // Error, input param is not pointer
__global__ __aicpu__ int foo(void *a, void *b) {} // Error, input param num is not one
// Declare a AI CPU kernel function in .asc file
extern __global__ __aicpu__ uint32_t hello_world(void *args); // OK
}
```

- **__inline__**

__inline__限定符声明一个函数，它具有如下属性：

- 标识Device侧函数强制内联，可以减少函数频繁调用产生的指令压栈、出栈的开销，但可能会导致算子二进制增加。
- 和C++函数修饰符inline的主要区别是Device侧__inline__是强制内联，C++的inline则是根据编译器优化选择性内联。
- AI Core对函数嵌套深度有限制，一般推荐嵌套深度不超过4层。使用强制内联可以减少调用层次。

- **__cube__**

标识该核函数仅在Cube核执行。针对耦合模式的硬件架构，该修饰符不生效。

```
__vector__ __global__ __aicore__ void add_custom(){}
```

- **__vector__**

标识该核函数仅在Vector核执行。针对耦合模式的硬件架构，该修饰符不生效。

- **__mix__(cube, vec)**

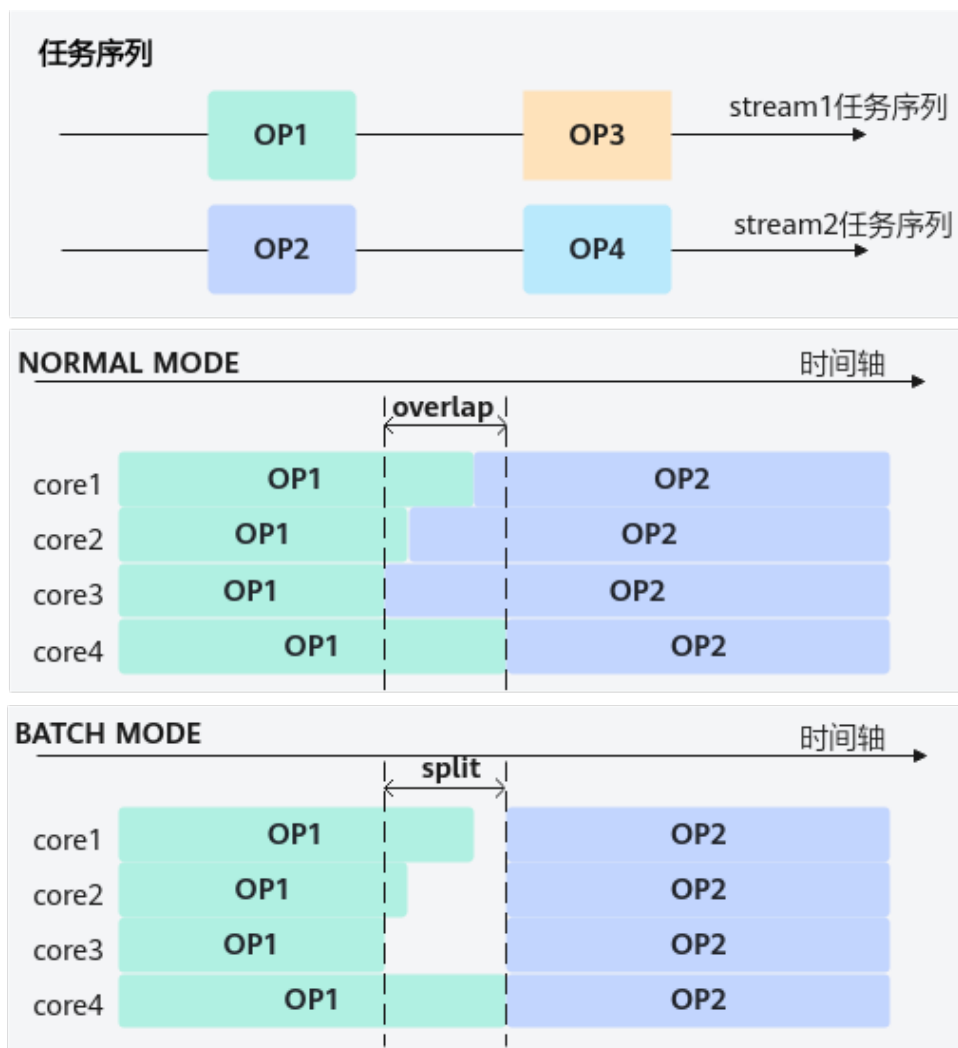
标识该核函数同时在Cube核和Vector核上执行。(cube, vec)分别表示核函数启动的Cube核和Vector核的配比，支持的配比为(1, 0)，(0, 1)，(1, 1)，(1, 2)。针对耦合模式的硬件架构，该修饰符不生效。

- **__schedmode__(mode)**

标识该核函数的执行调度模式。如下图所示：

- mode = 0 : normal mode，尽可能选择空闲物理核下发执行核函数，若空闲物理核数无法满足当前核函数的需要，没有下发的部分等待核心空闲后执行。此时OP1和OP2算子会存在交叠执行（overlap）的情况。
- mode = 1 : batch mode，在下发核函数时先进行判断，若空闲物理核数无法满足当前核函数的需要，则等待至空闲物理核数满足该核函数所需要的所有

物理核时，同时下发执行，OP1和OP2的执行被切分（split）开，不会出现交叠执行的情况。



在多流并发场景，多算子并行执行时，若执行总核数超过最大物理核数，且多个算子逻辑使用SyncALL等核间同步接口时，建议设置mode为1，防止多个算子之间互相等待空闲核调度，导致死锁。默认值mode为0。

```
__schedmode__(1) __global__ __mix__(1, 2) void OP1() // OP1使用了SyncAll接口，且存在多流并发的可能，需要设置batch mode ( mode 1 )
{
    AscendC::SyncAll();
    ....
}
__schedmode__(1) __global__ __mix__(1, 2) void OP2() // OP2使用了SyncAll接口，且存在多流并发的可能，需要设置batch mode ( mode 1 )
{
    AscendC::SyncAll();
    ....
}
__schedmode__(0) __global__ __vector__ void OP3() {...} // OP3没有使用SyncAll接口，可以设置为normal mode(mode 0)，按照正常规则执行算子。
or
__global__ __vector__ void OP3() {...} // 不设置__schedmode__，默认为normal mode。
```

地址空间限定符

AI Core具备多级独立片上存储，各个地址空间独立编址，具备各自的访存指令，根据架构差异，有些存储空间具备统一地址空间（Generic Address Space），有些则没有。设备侧编程基于语法扩展允许地址空间作为合法的类型限定符，以提供针对不同地址空间的访问能力和地址空间合法性检查。

表 2-13 地址空间映射关系

地址空间限定符	AI Core物理存储空间
<code>__gm__</code>	设备侧内存GM
<code>__ubuf__</code>	Vector Unified Buffer
<code>__ca__</code>	Cube L0A Buffer
<code>__cb__</code>	Cube L0B Buffer
<code>__cc__</code>	Cube L0C Buffer
<code>__cbuf__</code>	Cube L1 Buffer
<code>__fbuf__</code>	Fixpipe Buffer

地址空间限定符可以在变量声明中使用，用于指定对象分配的区域。如果对象的类型被地址空间名称限定，那么该对象将被分配在指定的地址空间中。同样地，对于指针，指向的类型可以通过地址空间进行限定，以指示所指向的对象所在的地址空间。

```
// declares a pointer p in the __gm__ address space that
// points to an object(has int type) in the __gm__ address space
__gm__ int *p;

__global__ __aicore__ void foo(...)
{
    // declares an array of 4 floats in the private address space.
    float x[4];
}
```

地址空间限定符不能用于非指针返回类型，非指针函数参数，函数类型，同一个类型上不允许使用多个地址空间限定符。

```
// OK.
__aicore__ int f() {...}

// Error. Address space qualifier cannot be used with a non-pointer return type.
__ubuf__ int f() { ... }

// OK. Address space qualifier can be used with a pointer return type.
__ubuf__ int *f() { ... }

// Error. Multiple address spaces specified for a type.
__ubuf__ __gm__ int i;

// OK. The first address space qualifies the object pointed to and the second
// qualifies the pointer.
__ubuf__ int * __gm__ ptr;
```

 说明

重要：不同地址空间指针的大小可能不同。例如，不能认为 `sizeof(__gm__ int *)` 总是等于 `sizeof(__ubuf__ int *)`，譬如编译器或许可能在某些系统上以32bit存储 `__ubuf__` 指针。

- **private地址空间**

private地址空间是大多数变量的默认地址空间，特别是局部变量。

```
// m is in a specific kernel parameter address space,
// it's physical location is implementation determined.
__global__ __aicore__ void foo(int m) {
    // OK. i is an int variable allocated in private address space
    int i;
}

__aicore__ void bar(int k) { //OK. k is in private address space
    // OK. i is an int variable allocated in private address space
    int i;
}
```

- **__gm__ 地址空间**

`__gm__` 地址空间限定符用来表示分配于设备侧全局内存的对象，全局内存对象可以声明为标量、用户自定义结构体的指针。

```
__gm__ int *var; // var point to an array of int elements

typedef struct {
    float a[3];
    int b[2];
} foo_t;

__gm__ foo_t *info; // info point to an array of foo_t elements
```

- **__ubuf__ 地址空间**

`__ubuf__` 地址空间用来描述存储于AI Core核内UB存储空间的变量。

```
__global__ __aicore__ void foo() {
    // ptr is in private address space, point to __ubuf__
    __ubuf__ int *ptr;
}
```

- **__ca__, __cb__, __cc__, __cbuf__ 地址空间**

上述几个地址空间主要用于特定的DMA指令访问，不具备标量直接访问能力。

```
class ObjTy{
    ObjTy(){...}
    void print(){...}

private:
    int a;
    int b;
};

__global__ __aicore__
void foo(__ca__ int * ptr) { // Error. Cannot have __ca__
                           // qualifier in kernel arguments
    // OK
    __ca__ int *ptr;
}
```

内置常量

常量名	取值	功能
constexpr int32_t g_coreType	- AscendC::AIC - AscendC::AIV	常量值由框架自动设置，AIC核下，配置为AscendC::AIC，AIV核下，配置为AscendC::AIV。 可以通过对该常量值的判断，来实现了AIV与AIC核代码的区分和隔离。功能等同于直接使用ASCEND_IS_AIV、ASCEND_IS_AIC。

内置变量

变量名	对应API	功能
block_num	GetBlockNum	当前任务配置的核数，用于代码内部的多核逻辑控制等。
block_idx	GetBlockIdx	当前核的索引，用于代码内部的多核逻辑控制及多核偏移量计算等。

通常，建议用户使用内置变量对应的API获取所需值，不建议用户直接使用内置变量。因为内置变量反映的是单个硬件资源的配置信息，对于软件栈整合硬件资源、扩展硬件的功能，内置变量的值与实际语义可能不符。

例如，在Atlas 推理系列产品中，当启用KERNEL_TYPE_MIX_VECTOR_CORE时，算子会同时运行在AI Core和Vector Core上。此时，block_idx在这两种核心上都是从0开始计数，用户无法直接通过block_idx来切分数据和控制多核逻辑。而GetBlockIdx在Vector Core上对block_idx增加偏移量（AI Core的block_num），从而保证返回的值能够正确反映多核环境下的实际逻辑。

BuiltIn API

具体API列表请参见《CCE Intrinsic开发接口》。

2.5 C++类库 API

2.5.1 编程接口概述

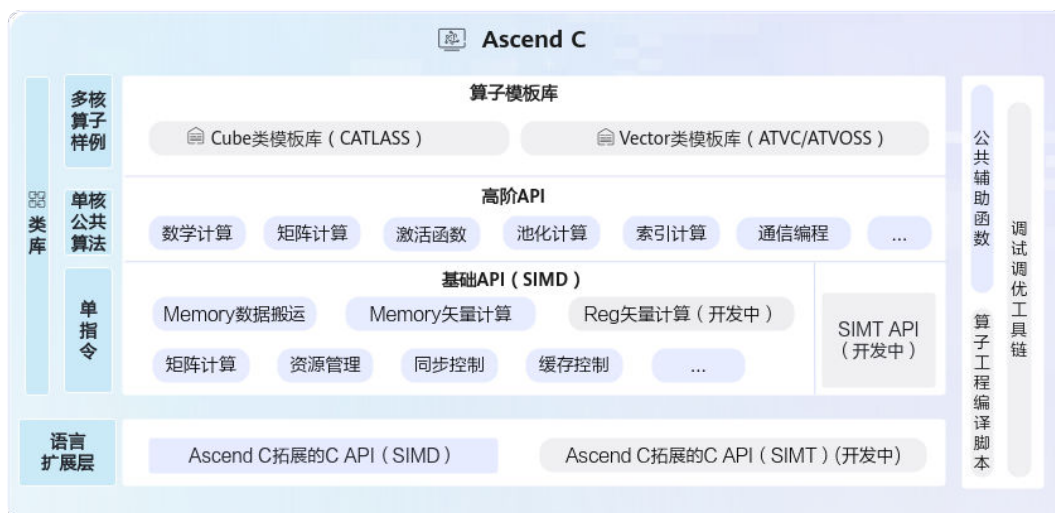
说明

本章提供编程API的概述。具体API参考见Ascend C API。

Ascend C提供一组类库API，开发者使用标准C++语法和类库API进行编程。Ascend C编程类库API示意图如下所示，分为：

- 基础数据结构**：kernel API中使用到的基础数据结构，比如GlobalTensor和LocalTensor。

- **基础API**：实现对硬件能力的抽象，开放芯片的能力，保证完备性和兼容性。标注为ISASI（Instruction Set Architecture Special Interface，硬件体系结构相关的接口）类别的API，不能保证跨硬件版本兼容。
- **高阶API**：实现一些常用的计算算法，用于提高编程开发效率，通常会调用多种基础API实现。高阶API包括数学库、Matmul、Softmax等API。高阶API可以保证兼容性。
- **Utils API（公共辅助函数）**：丰富的通用工具类，涵盖标准库、平台信息获取、运行时编译及日志输出等功能，支持开发者高效实现算子开发与性能优化。



说明

Ascend C API所在头文件目录为：

- 基础API：\${INSTALL_DIR}/include/ascendc/basic_api/interface
- 高阶API：（注意，如下目录头文件中包含的接口如果未在资料中声明，属于间接调用接口，开发者无需关注）
 - \${INSTALL_DIR}/include/ascendc/highlevel_api/lib
 - \${INSTALL_DIR}/include/tiling

\${INSTALL_DIR}请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。

使用Ascend C API依赖的库文件说明如下：

- 基础API：不涉及
- 高阶API：因高阶API配套Host Tiling接口，需要链接libtiling_api.a。

2.5.2 基础 API

2.5.2.1 概述

基础API实现对硬件能力的抽象，开放芯片的能力，保证完备性和兼容性。标注为ISASI（Instruction Set Architecture Special Interface，硬件体系结构相关的接口）类别的API，不能保证跨硬件版本兼容。

根据功能的不同，主要分为以下几类：

- 标量计算API，实现调用Scalar计算单元执行计算的功能。

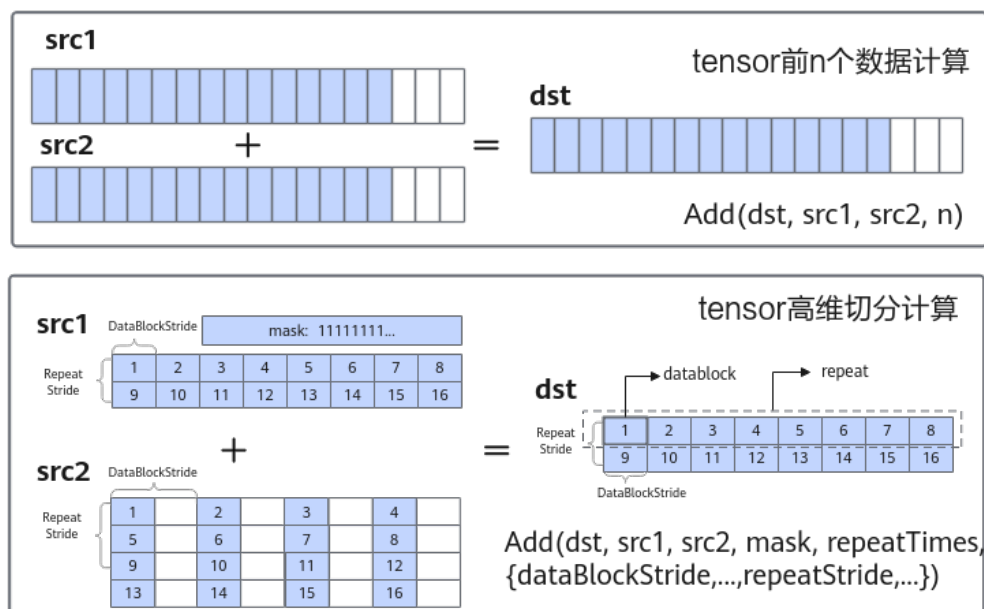
- Memory矢量计算API，实现调用Vector计算单元执行计算的功能。
- 矩阵计算API，实现调用Cube计算单元执行计算的功能。
- 数据搬运API，计算API基于Local Memory数据进行计算，所以数据需要先从Global Memory搬运至Local Memory，再使用计算API完成计算，最后从Local Memory搬出至Global Memory。执行搬运过程的接口称之为数据搬运API，比如DataCopy接口。
- 资源管理API，用于分配管理内存，比如AllocTensor、FreeTensor接口；
- 同步控制API，完成任务间的通信和同步，比如EnQue、DeQue接口。不同的API指令间有可能存在依赖关系，从2.2.2 抽象硬件架构可知，不同的指令异步并行执行，为了保证不同指令队列间的指令按照正确的逻辑关系执行，需要向不同的组件发送同步指令。同步控制API内部即完成这个发送同步指令的过程，开发者无需关注内部实现逻辑，使用简单的API接口即可完成。

根据对数据操作方法的的不同，分为以下几类：

- 连续计算API：支持Tensor前n个数据计算。针对源操作数的连续n个数据进行计算并连续写入目的操作数，解决一维tensor的连续计算问题。
`Add(dst, src1, src2, n);`
- 高维切分API：支持Repeat和Stride。功能灵活的计算API，提供与Builtin API完全对等编程能力，充分发挥硬件优势，支持对每个操作数的DataBlock Stride，Repeat Stride，Mask等参数的操作。

下图以矢量加法为例，展示了连续计算API和高维切分API的特点。

图 2-11 计算 API 几种计算方式的特点



2.5.2.2 接口分类说明

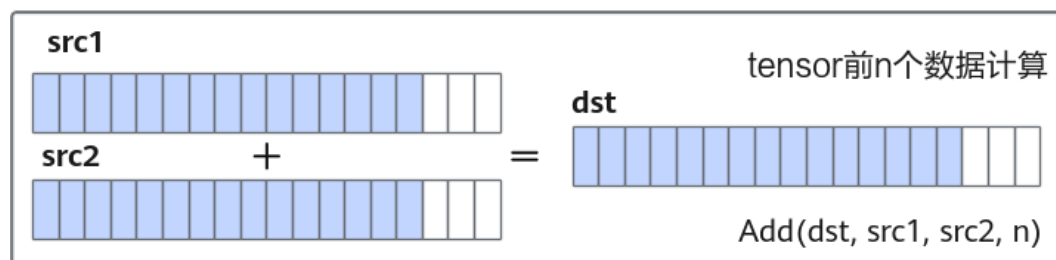
2.5.2.2.1 连续计算 API

连续计算API，支持Tensor前n个数据计算。针对源操作数的连续n个数据进行计算并连续写入目的操作数，解决一维tensor的连续计算问题。

```
Add(dst, src1, src2, n);
```

下图以矢量加法为例，展示了**连续计算API**的特点。

图 2-12 连续计算 API



2.5.2.2.2 高维切分 API

说明

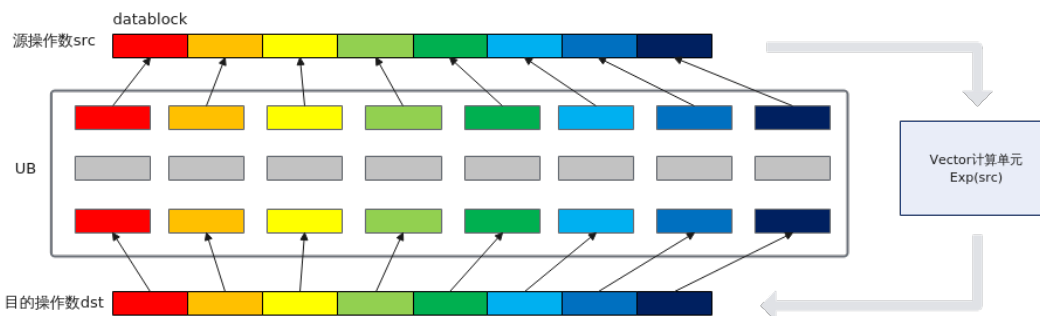
- 本章节对**矢量计算基础API**中的**tensor高维切分计算**接口做解释说明。如果您不需要使用此类接口，可略过该章节。
- 下文中的repeatTime、dataBlockStride、repeatStride、mask为通用描述，其命名不一定与具体指令中的参数命名完全对应。
比如，单次迭代内不同datablock间地址步长dataBlockStride参数，在单目API中，对应为dstBlkStride、srcBlkStride参数；在双目API中，对应为dstBlkStride、src0BlkStride、src1BlkStride参数。

您可以在具体接口的参数说明中，找到参数含义的描述。

使用**tensor高维切分计算API**可充分发挥硬件优势，支持开发者控制指令的**迭代执行**和**操作数的地址间隔**，功能更加灵活。

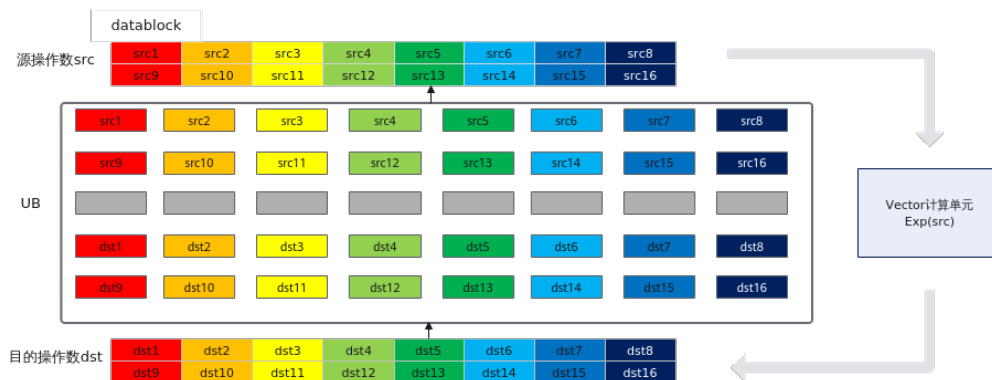
矢量计算通过Vector计算单元完成，矢量计算的源操作数和目的操作数均通过Unified Buffer（UB）来进行存储。Vector计算单元每个迭代会从UB中取出8个**datablock**（每个datablock数据块内部地址连续，长度32Byte），进行计算，并写入对应的8个datablock中。下图为单次迭代内的8个datablock进行Exp计算的示意图。

图 2-13 单次迭代内的 8 个 datablock 进行 Exp 计算示意图



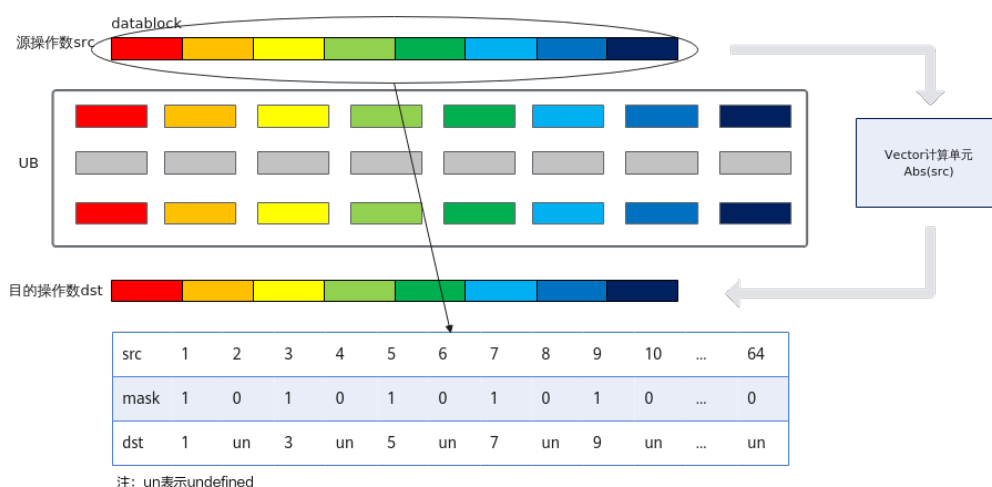
- 矢量计算API支持开发者通过**repeatTime**来配置迭代次数，从而控制指令的多次迭代执行。假设repeatTime设置为2，矢量计算单元会进行2个迭代的计算，可计算出 $2 * 8$ （每个迭代8个datablock）* 32Byte（每个datablock32Byte）= 512Byte的结果。如果数据类型为half，则计算了256个元素。下图展示了2次迭代Exp计算的示意图。由于硬件限制，**repeatTime不能超过255**。

图 2-14 2 次迭代 Exp 计算



- 针对同一个迭代中的数据，可以通过**mask**参数进行掩码操作来控制实际参与计算的个数。下图为进行Abs计算时通过mask逐比特模式按位控制哪些元素参与计算的示意图，1表示参与计算，0表示不参与计算。

图 2-15 通过 mask 参数进行掩码操作示意图（以 float 数据类型为例）

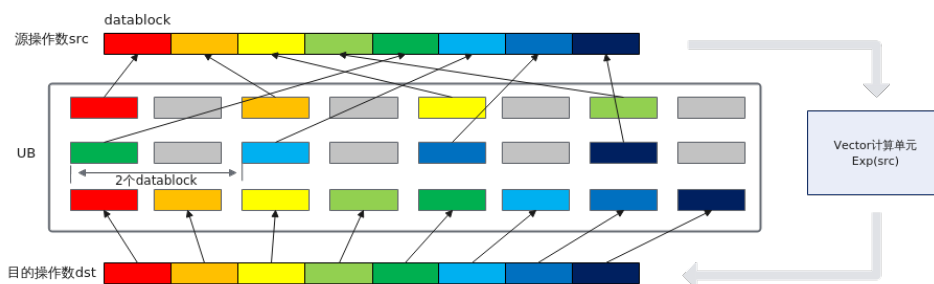


- 矢量计算单元还支持带间隔的向量计算，通过**dataBlockStride**（单次迭代内不同datablock间地址步长）和**repeatStride**（相邻迭代间相同datablock的地址步长）来进行配置。

dataBlockStride

如果需要控制单次迭代内，数据处理的步长，可以通过设置同一迭代内不同datablock的地址步长dataBlockStride来实现。下图给出了单次迭代内非连续场景的示意图，示例中源操作数的dataBlockStride配置为2，表示单次迭代内不同datablock间地址步长（起始地址之间的间隔）为2个datablock。

图 2-16 单次迭代内非连续场景的示意图



– repeatStride

当repeatTime大于1，需要多次迭代完成矢量计算时，您可以根据不同的使用场景合理设置相邻迭代间相同datablock的地址步长repeatStride的值。

下图给出了多次迭代间非连续场景的示意图，示例中源操作数和目的操作数的repeatStride均配置为9，表示相邻迭代间相同datablock起始地址之间的间隔为9个datablock。相同datablock是指datablock在迭代内的位置相同，比如下图中的src1和src9处于相邻迭代，在迭代内都是第一个datablock的位置，其间隔即为repeatStride的数值。

图 2-17 多次迭代间非连续场景的示意图



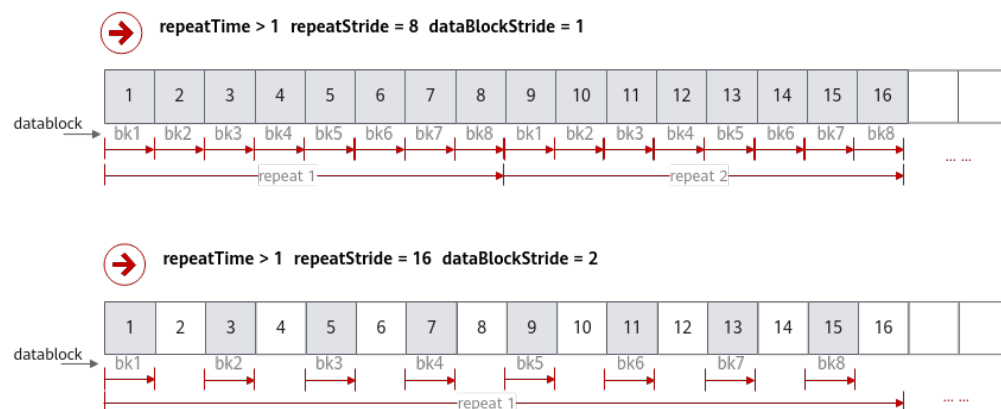
下文中给出了dataBlockStride、repeatStride、mask的详细配置说明和示例。

dataBlockStride

dataBlockStride是指同一迭代内不同datablock的地址步长。

- 连续计算，dataBlockStride设置为1，对同一迭代内的8个datablock数据连续进行处理。
- 非连续计算，dataBlockStride值大于1（如取2），同一迭代内不同datablock之间在读取数据时出现一个datablock的间隔，如下图所示。

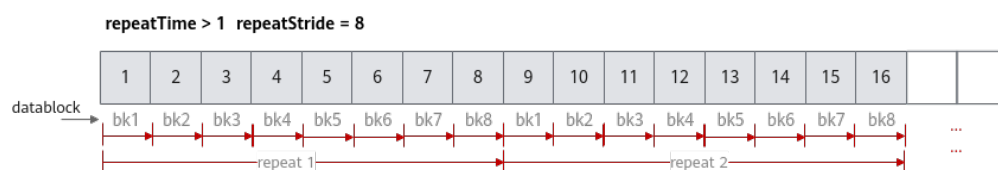
图 2-18 dataBlockStride 不同取值举例



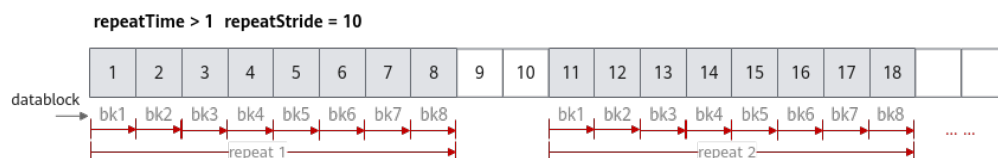
repeatStride

repeatStride是指相邻迭代间相同datablock的地址步长。

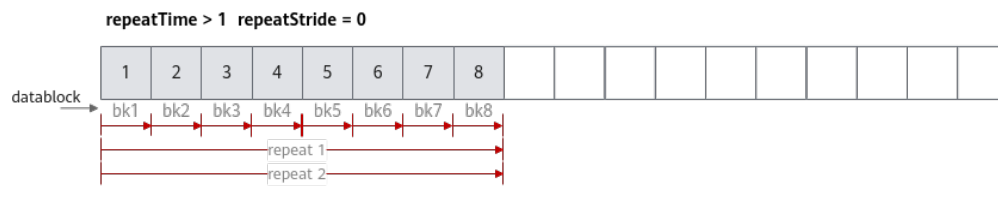
- 连续计算场景：**假设定义一个Tensor供目的操作数和源操作数同时使用（即地址重叠），repeatStride取值为8。此时，矢量计算单元第一次迭代读取连续8个datablock，第二轮迭代读取下一个连续的8个datablock，通过多次迭代即可完成所有输入数据的计算。



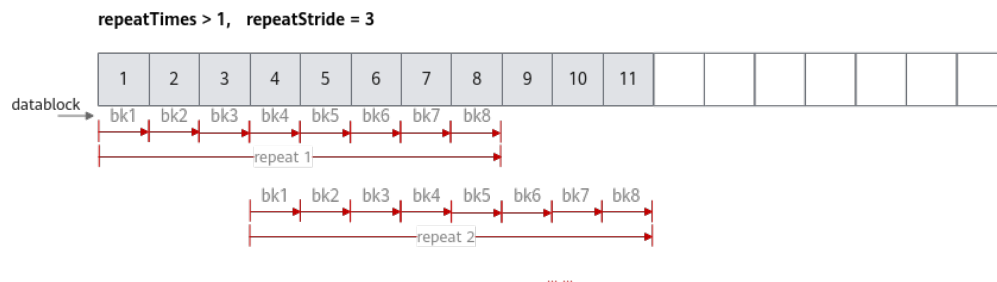
- 非连续计算场景：**repeatStride取值大于8（如取10）时，则相邻迭代间矢量计算单元读取的数据在地址上不连续，出现2个datablock的间隔。



- 反复计算场景：**repeatStride取值为0时，矢量计算单元会对首个连续的8个datablock进行反复读取和计算。



- 部分重复计算：**repeatStride取值大于0且小于8时，相邻迭代间部分数据会被矢量计算单元重复读取和计算，此种情形一般场景不涉及。



mask 参数

mask用于控制每次迭代内参与计算的元素。可通过连续模式和逐bit模式两种方式进行设置。

- 连续模式：表示前面连续的多少个元素参与计算。数据类型为uint64_t。取值范围和源操作数的数据类型有关，数据类型不同，每次迭代内能够处理的元素个数最大值不同（当前数据类型单次迭代时能处理的元素个数最大值为： $256 / \text{sizeof}(\text{数据类型})$ ）。当操作数的数据类型占bit位16位时（如half/uint16_t），mask $\in [1, 128]$ ；当操作数为32位时（如float/int32_t），mask $\in [1, 64]$ 。

具体样例如下：

```
// int16_t数据类型单次迭代能处理的元素个数最大值为256/sizeof(int16_t) = 128, mask = 64, mask  $\in$  [1, 128], 所以是合法输入
// repeatTime = 1, 共128个元素, 单次迭代能处理128个元素, 故repeatTime = 1
// dstBlkStride, src0BlkStride, src1BlkStride = 1, 单次迭代内连续读取和写入数据
// dstRepStride, src0RepStride, src1RepStride = 8, 迭代间的数据连续读取和写入
uint64_t mask = 64;
AscendC::Add(dstLocal, src0Local, src1Local, mask, 1, { 1, 1, 1, 8, 8, 8 });
```

结果示例如下：

```
输入数据(src0Local): [1 2 3 ... 64 ...128]
输入数据(src1Local): [1 2 3 ... 64 ...128]
输出数据(dstLocal): [2 4 6 ... 128 undefined...undefined]
```

```
// int32_t数据类型单次迭代能处理的元素个数最大值为256/sizeof(int32_t) = 64, mask = 64, mask  $\in$  [1, 64], 所以是合法输入
// repeatTime = 1, 共64个元素, 单次迭代能处理64个元素, 故repeatTime = 1
// dstBlkStride, src0BlkStride, src1BlkStride = 1, 单次迭代内连续读取和写入数据
// dstRepStride, src0RepStride, src1RepStride = 8, 迭代间的数据连续读取和写入
uint64_t mask = 64;
AscendC::Add(dstLocal, src0Local, src1Local, mask, 1, { 1, 1, 1, 8, 8, 8 });
```

结果示例如下：

```
输入数据(src0Local): [1 2 3 ... 64]
输入数据(src1Local): [1 2 3 ... 64]
输出数据(dstLocal): [2 4 6 ... 128]
```

- 逐bit模式：可以按位控制哪些元素参与计算，bit位的值为1表示参与计算，0表示不参与。

mask为数组形式，数组长度和数组元素的取值范围和操作数的数据类型有关。当操作数为16位时，数组长度为2，mask[0]、mask[1] $\in [0, 2^{64}-1]$ 并且不同时为0；当操作数为32位时，数组长度为1，mask[0] $\in (0, 2^{64}-1]$ ；当操作数为64位时，数组长度为1，mask[0] $\in (0, 2^{32}-1]$ 。

具体样例如下：

```
// 数据类型为int16_t
uint64_t mask[2] = {6148914691236517205, 6148914691236517205};
// repeatTime = 1, 共128个元素, 单次迭代能处理128个元素, 故repeatTime = 1。
// dstBlkStride, src0BlkStride, src1BlkStride = 1, 单次迭代内连续读取和写入数据。
// dstRepStride, src0RepStride, src1RepStride = 8, 迭代间的数据连续读取和写入。
AscendC::Add(dstLocal, src0Local, src1Local, mask, 1, { 1, 1, 1, 8, 8, 8 });
```

结果示例如下：

```
输入数据(src0Local): [1 2 3 ... 64 ...127 128]
输入数据(src1Local): [1 2 3 ... 64 ...127 128]
输出数据(dstLocal): [2 undefined 6 ... undefined ...254 undefined]
```

mask过程如下：

mask={6148914691236517205, 6148914691236517205} (注：
6148914691236517205表示64位二进制数0b010101....01，mask按照低位到高位
的顺序排布)

src0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...	64	...	127	128
src1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...	64	...	127	128
mask	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	...	0	...	1	0
dst	2	un	6	un	10	un	14	un	18	un	22	un	26	un	30	un	34	un	38	un	...	un	...	254	un

注：un表示undefined

```
// 数据类型为int32_t
uint64_t mask[1] = {6148914691236517205};
// repeatTime = 1, 共64个元素，单次迭代能处理64个元素，故repeatTime = 1。
// dstBlkStride, src0BlkStride, src1BlkStride = 1, 单次迭代内连续读取和写入数据。
// dstRepStride, src0RepStride, src1RepStride = 8, 迭代间的数据连续读取和写入。
AscendC::Add(dstLocal, src0Local, src1Local, mask, 1, { 1, 1, 1, 8, 8, 8 });
```

结果示例如下：

```
输入数据(src0Local): [1 2 3 ... 63 64]
输入数据(src1Local): [1 2 3 ... 63 64]
输出数据(dstLocal): [2 undefined 6 ... 126 undefined]
```

mask过程如下：

mask={6148914691236517205, 0} (注：6148914691236517205表示64位二进
制数0b010101....01)

src0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...	63	64
src1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...	63	64
mask	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	...	1	0
dst	2	un	6	un	10	un	14	un	18	un	22	un	26	un	30	un	34	un	38	un	...	126	un

注：un表示undefined

2.5.2.3 常用操作速查指导

2.5.2.3.1 如何使用掩码操作 API

Mask用于控制矢量计算中参与计算的元素个数，支持以下工作模式及配置方式：

表 2-14 Mask 工作模式

工作模式	说明
Normal 模式	默认模式，支持单次迭代内的Mask能力，需要开发者配置迭代次数，额外进行尾块的计算。 Normal模式下，Mask用来控制单次迭代内参与计算的元素个数。 通过调用SetMaskNorm设置Normal模式。

工作模式	说明
Counter 模式	简化模式，直接传入计算数据量，自动推断迭代次数，不需要开发者去感知迭代次数、处理非对齐尾块的操作；但是不具备单次迭代内的Mask能力。 Counter模式下，Mask表示整个矢量计算参与计算的元素个数。 通过调用SetMaskCount设置Counter模式。

表 2-15 Mask 配置方式

配置方式	说明
接口传参（默认）	通过矢量计算API的入参直接传递Mask值。矢量计算API的模板参数isSetMask（仅部分API支持）用于控制接口传参还是外部API配置，默认值为true，表示接口传参。Mask对应于高维切分计算API中的mask/mask[]参数或者tensor前n个数据计算API中的calCount参数。
外部API配置	调用SetVectorMask接口设置Mask值，矢量计算API的模板参数isSetMask设置为false，接口入参中的Mask参数（对应于高维切分计算API中的mask/mask[]参数或者tensor前n个数据计算API中的calCount参数）不生效。适用于Mask参数相同，多次重复使用的场景，无需在矢量计算API内部反复设置，会有一定的性能优势。

Mask操作的使用方式如下：

表 2-16 Mask 操作的使用方式

配置方式	工作模式	前n个数据计算API	高维切分计算API
接口传参	Normal 模式	不涉及。	isSetMask模板参数设置为true，通过接口入参传入Mask，根据使用场景配置dataBlockStride、repeatStride、repeatTime参数。
	Counter 模式	isSetMask模板参数设置为true，通过接口入参传入Mask。	- isSetMask模板参数设置为true，通过接口入参传入Mask。 - 根据使用场景配置dataBlockStride、repeatStride参数。repeatTime传入固定值即可，建议统一设置为1，该值不生效。

配置方式	工作模式	前n个数据计算API	高维切分计算API
外部API配置	Normal模式	不涉及。	调用SetVectorMask设置Mask，之后调用高维切分计算API。 - isSetMask模板参数设置为false，接口入参中的mask值设置为占位符MASK_PLACEHOLDER，用于占位，无实际含义。 - 根据使用场景配置repeatTime、dataBlockStride、repeatStride参数。
	Counter模式	调用SetVectorMask设置Mask，之后调用前n个数据计算API，isSetMask模板参数设置为false；接口入参中的calCount建议设置成1。	调用SetVectorMask设置Mask，之后调用高维切分计算API。 - isSetMask模板参数设置为false；接口入参中的mask值设置为MASK_PLACEHOLDER，用于占位，无实际含义。 - 根据使用场景配置dataBlockStride、repeatStride参数。repeatTime传入固定值即可，建议统一设置为1，该值不生效。

典型场景的使用示例如下：

- 场景1：Normal模式 + 外部API配置 + 高维切分计算API

```
AscendC::LocalTensor<half> dstLocal;  
AscendC::LocalTensor<half> src0Local;  
AscendC::LocalTensor<half> src1Local;  
  
// 1、设置Normal模式  
AscendC::SetMaskNorm();  
// 2、设置Mask  
AscendC::SetVectorMask<half, AscendC::MaskMode::NORMAL>(0xffffffffffffffff, 0xffffffffffffffff); // 逐bit模式  
// SetVectorMask<half, MaskMode::NORMAL>(128); // 连续模式  
  
// 3、多次调用矢量计算API，isSetMask模板参数设置为false，接口入参中的mask值设置为占位符  
MASK_PLACEHOLDER，用于占位，无实际含义  
// 根据使用场景配置repeatTime、dataBlockStride、repeatStride参数  
// dstBlkStride, src0BlkStride, src1BlkStride = 1, 单次迭代内数据连续读取和写入  
// dstRepStride, src0RepStride, src1RepStride = 8, 相邻迭代间数据连续读取和写入  
AscendC::Add<half, false>(dstLocal, src0Local, src1Local, AscendC::MASK_PLACEHOLDER, 1, { 2, 2, 2, 8,  
8, 8 });  
AscendC::Sub<half, false>(src0Local, dstLocal, src1Local, AscendC::MASK_PLACEHOLDER, 1, { 2, 2, 2, 8,  
8, 8 });  
AscendC::Mul<half, false>(src1Local, dstLocal, src0Local, AscendC::MASK_PLACEHOLDER, 1, { 2, 2, 2, 8,  
8, 8 });  
// 4、恢复Mask值为默认值  
AscendC::ResetMask();
```

- 场景2：Counter模式 + 外部API配置 + 高维切分计算API

```
AscendC::LocalTensor<half> dstLocal;  
AscendC::LocalTensor<half> src0Local;  
AscendC::LocalTensor<half> src1Local;  
int32_t len = 128; // 参与计算的元素个数  
// 1、设置Counter模式
```

```
AscendC::SetMaskCount();  
// 2、设置Mask  
AscendC::SetVectorMask<half, AscendC::MaskMode::COUNTER>(len);  
// 3、多次调用矢量计算API, isSetMask模板参数设置为false; 接口入参中的mask值设置为  
MASK_PLACEHOLDER, 用于占位, 无实际含义  
// 根据使用场景正确配置dataBlockStride、repeatStride参数。repeatTime传入固定值即可, 建议统一设置  
为1, 该值不生效  
AscendC::Add<half, false>(dstLocal, src0Local, src1Local, AscendC::MASK_PLACEHOLDER, 1, { 1, 1, 1, 8,  
8, 8 });  
AscendC::Sub<half, false>(src0Local, dstLocal, src1Local, AscendC::MASK_PLACEHOLDER, 1, { 1, 1, 1, 8,  
8, 8 });  
AscendC::Mul<half, false>(src1Local, dstLocal, src0Local, AscendC::MASK_PLACEHOLDER, 1, { 1, 1, 1, 8,  
8, 8 });  
// 4、恢复工作模式  
AscendC::SetMaskNorm();  
// 5、恢复Mask值为默认值  
AscendC::ResetMask();
```

- 场景3: Counter模式 + 外部API配置 + 前n个数据计算接口配合使用

```
AscendC::LocalTensor<half> dstLocal;  
AscendC::LocalTensor<half> src0Local;  
half num = 2;  
// 1、设置Mask  
AscendC::SetVectorMask<half, AscendC::MaskMode::COUNTER>(128); // 参与计算的元素个数为128  
// 2、调用前n个数据计算API, isSetMask模板参数设置为false; 接口入参中的calCount建议设置成1。  
AscendC::Adds<half, false>(dstLocal, src0Local, num, 1);  
AscendC::Muls<half, false>(dstLocal, src0Local, num, 1);  
// 3、恢复工作模式  
AscendC::SetMaskNorm();  
// 4、恢复Mask值为默认值  
AscendC::ResetMask();
```

📖 说明

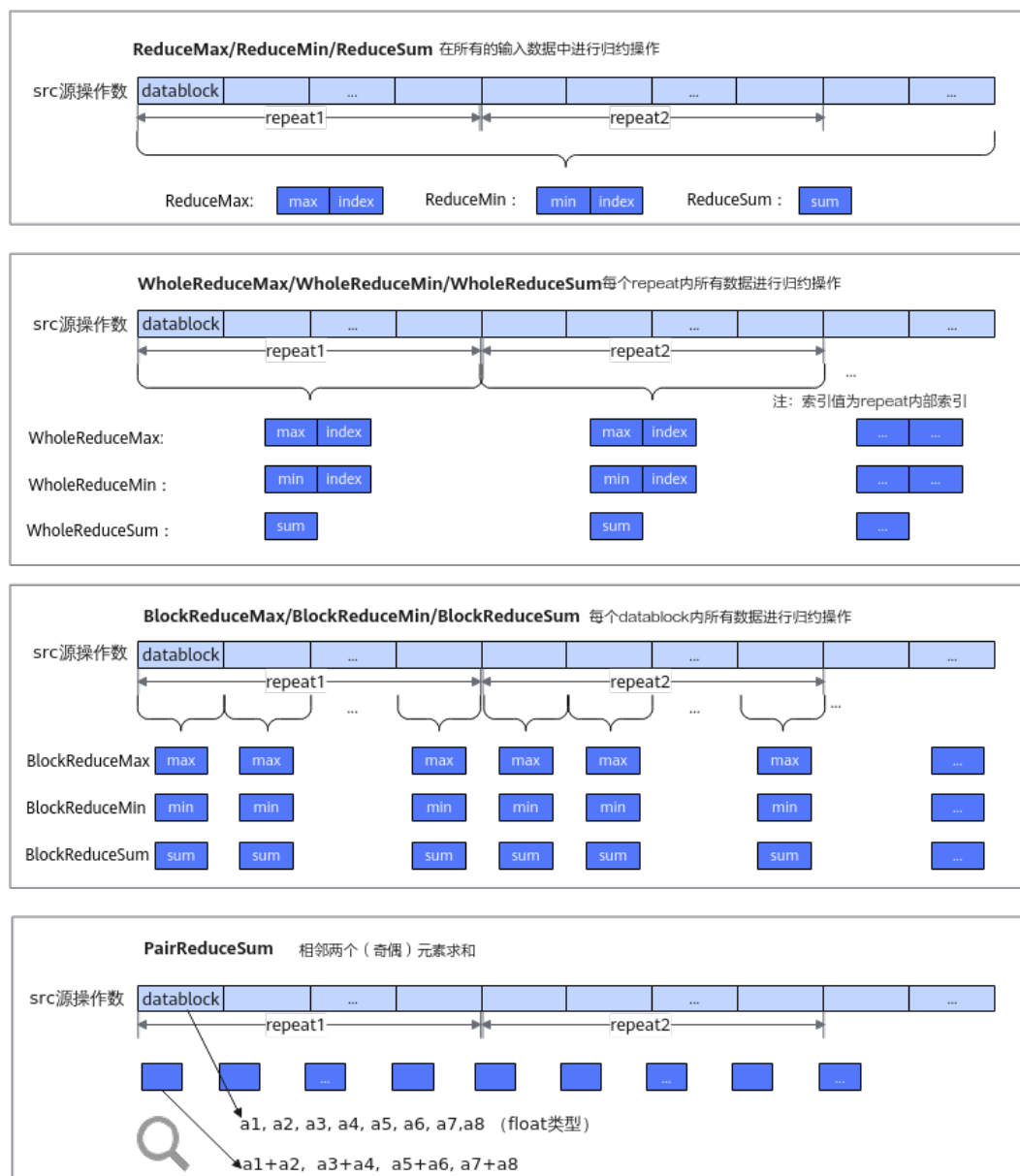
- 前n个数据计算API接口内部会设置工作模式为Counter模式, 所以如果前n个数据计算API配合Counter模式使用时, 无需手动调用SetMaskCount设置Counter模式。
- 所有手动使用Counter模式的场景, 使用完毕后, 需要调用SetMaskNorm恢复工作模式。
- 调用SetVectorMask设置Mask, 使用完毕后, 需要调用ResetMask恢复Mask值为默认值。
- 使用高维切片计算API配套Counter模式使用时, 比前n个数据计算API增加了可间隔的计算, 支持dataBlockStride、repeatStride参数。

2.5.2.3.2 如何使用归约计算 API

归约指令将数据集合简化为单一值或者更小的集合。按照归约操作的数据范围的不同, 归约指令分为以下几种, 可参考[归约指令示意图](#):

- ReduceMax/ReduceMin/ReduceSum: 对所有的输入数据做归约操作, 得到最大值和最大值索引/最小值和最小值索引/数据总和。
- WholeReduceMax/WholeReduceMin/WholeReduceSum: 对每个repeat内的输入数据做归约操作, 得到每个repeat内的最大值和最大值索引/最小值和最小值索引/数据总和。返回索引时返回的是repeat内部索引。
- BlockReduceMax/BlockReduceMin/BlockReduceSum: 对每个datablock内的输入数据做归约操作, 得到每个datablock内的最大值/最小值/数据总和。
- PairReduce: 相邻两个 (奇偶) 元素求和, 例如 (a1, a2, a3, a4, a5, a6...), 归约后结果为 (a1+a2, a3+a4, a5+a6,)。

图 2-19 归约指令示意图



针对归约指令，和其他的基础API一样也提供了**tensor高维切分**计算接口，可充分发挥硬件优势，支持开发者控制指令的**迭代执行**和操作数的**地址间隔**，功能更加灵活。但具体参数的单位和约束与**基础API**略有不同，下文将对这些差异点进行介绍。

- **repeatTime**: 迭代次数，开发者通过repeatTime来配置迭代次数，从而控制指令的多次迭代执行。
 - ReduceMax/ReduceMin/ReduceSum对于repeatTime超过255的情况，在API内部进行了处理，所以repeatTime支持更大的取值范围，保证不超过int32_t最大值的范围即可。
 - WholeReduceMax/WholeReduceMin/WholeReduceSum/BlockReduceMax/BlockReduceMin/BlockReduceSum/PairReduce和其他基础API一样，repeatTime要求不超过255。
- **mask**: 用于控制每次迭代内参与计算的元素，mask参数的使用方法和基础API通用的使用方法一致。

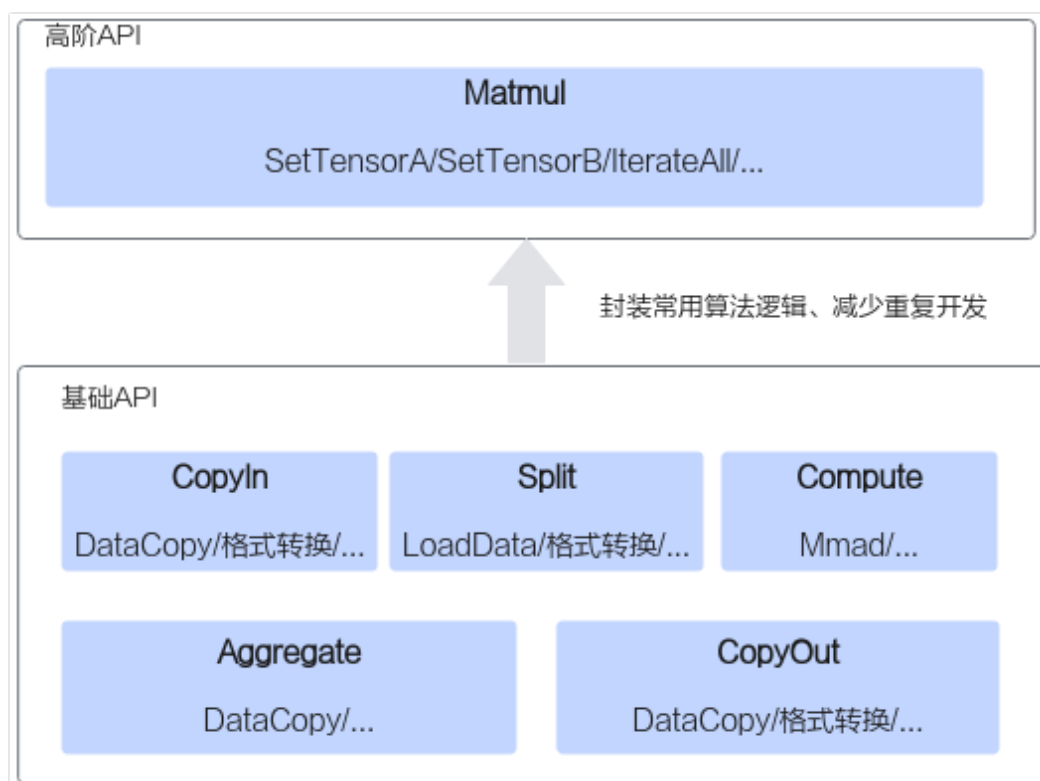
- **repeatStride**: 表示相邻迭代间的地址步长。
 - ReduceMax/ReduceMin/ReduceSum指令的目的操作数会归约成一个最大值/最小值/总和，所以其目的操作数不支持配置repeatStride。仅源操作数支持repeatStride，其含义、单位（datablock）和基础API通用说明一致。
 - WholeReduceMax/WholeReduceMin/WholeReduceSum/BlockReduceMax/BlockReduceMin/BlockReduceSum/PairReduce源操作数和目的操作数都支持配置repeatStride，源操作数repeatStride的含义、单位（datablock）和基础API通用说明一致。目的操作数repeatStride的含义、单位和基础API通用说明有差异，因为归约后，目的操作数的长度会变短，比如WholeReduceSum归约后每个repeat会合并为一个值，所以迭代之间的间隔不能再使用一个datablock为单位，而以一个repeat归约后的长度为单位。
- **dataBlockStride**: 表示单次迭代内datablock的地址步长。
 - ReduceMax/ReduceMin/ReduceSum指令的目的操作数会归约成一个最大值/最小值/总和，所以其目的操作数不支持配置dataBlockStride。源操作数也不支持dataBlockStride。
 - WholeReduceMax/WholeReduceMin/WholeReduceSum/BlockReduceMax/BlockReduceMin/BlockReduceSum/PairReduce源操作数支持配置dataBlockStride，源操作数dataBlockStride的含义、单位（datablock）和基础API通用说明一致。目的操作数不支持dataBlockStride，因为归约后，目的操作数的长度会变短，比如WholeReduceSum归约后每个repeat会合并为一个值，不再有迭代内datablock和地址间隔的概念。

2.5.3 高阶 API

2.5.3.1 概述

高阶API基于单核对常见算法进行抽象和封装，实现了一些常用的计算算法，旨在提高编程开发效率。高阶API一般通过调用多种基础API实现。高阶API包括数学计算、矩阵计算、激活函数等API。

如下图所示，实现一个矩阵乘操作，使用基础API需要的步骤较多，需要关注格式转换、数据切分等逻辑；使用高阶API则无需关注这些逻辑，可以快速实现功能。



2.5.3.2 常用操作速查指导

2.5.3.2.1 如何使用 Tiling 依赖的头文件

由于AI处理器的Scalar计算单元执行能力有限，为减少算子Kernel侧的Scalar计算，将部分计算在Host端执行，这需要编写**Host端Tiling代码**。注意，在程序中调用高阶API的Tiling接口或者使用高阶API的Tiling结构体参数时，需要引入依赖的头文件。在不同的Tiling实现方式下，具体为：

- **使用标准C++语法定义Tiling结构体**

这种方式需要引入依赖的头文件如下。所有高阶API的Tiling结构体定义在AscendC::tiling命名空间下，因此需要通过AscendC::tiling访问具体API的Tiling结构体。

```
#include "kernel_tiling/kernel_tiling.h"

// ...
AscendC::tiling::TCubeTiling cubeTilingData;
```

- **使用TILING_DATA_DEF宏定义Tiling结构体**

这种方式需要引入依赖的头文件如下。所有高阶API的Tiling结构体和Tiling函数定义在optiling命名空间下。

```
#include "tiling/tiling_api.h"

namespace optiling {
// ...
}
```

2.5.3.2.2 如何使用 Kernel 侧临时空间

Kernel侧接口的内部实现一般涉及复杂的数学计算，需要额外的临时空间来存储计算过程中的中间变量。除矩阵计算、HCCL通信类、卷积计算等，对于多数高阶API中临

时空间的处理，开发者可以通过Kernel侧接口的入参sharedTmpBuffer传入提前申请的临时空间、通过接口框架申请临时空间两种方式。

- 通过sharedTmpBuffer入参传入，Kernel侧接口使用该传入的Tensor作为临时空间。该方式下，开发者可以自行管理sharedTmpBuffer内存空间，并在接口调用完成后，复用该部分内存，内存不会反复申请释放，灵活性较高，内存利用率也较高。
- 接口框架申请临时空间，开发者无需在Kernel侧申请临时空间，但是需要预留临时空间的大小，即在分配内存空间时，应在可用空间大小中减去需预留的临时空间大小。

无论开发者采用上述哪种方式，在申请Tensor空间或预留临时空间时，都需要提前获取Kernel侧接口所需的临时空间大小BufferSize，为此相应类别API中提供了GetxxxMaxMinTmpSize接口，用于获取所需预留空间的大小范围，其中xxx为对应的Kernel侧接口。开发者在Host侧调用GetxxxMaxMinTmpSize接口，获取预留/申请的最大和最小临时空间大小，基于此范围选择合适的空间大小作为Tiling参数传递到Kernel侧使用。

- 为保证功能正确，预留/申请的临时空间大小不能小于最小临时空间大小；
- 在最小临时空间-最大临时空间范围内，随着临时空间增大，Kernel侧接口计算性能会有一定程度的优化提升。为了达到更好的性能，开发者可以根据实际的内存使用情况进行空间预留/申请。

以Asin接口为例：

```
// 算子输入的数据类型T为half, isReuseSource传入默认值false
auto shape_input = context->GetInputTensor(0)->GetOriginShape();
std::vector<int64_t> srcDims = {shape_input.GetDim(0), shape_input.GetDim(1)};
uint32_t srcSize = 1;
for (auto dim : srcDims) {
    srcSize *= dim;
}
uint32_t typeSize = 2;
ge::Shape shape(srcDims);
uint32_t minVal = 0;
uint32_t maxVal = 0;
AscendC::GetAsinMaxMinTmpSize(shape, typeSize, false, maxVal, minVal);

auto platformInfo = context->GetPlatformInfo();
auto ascendcPlatform = platform_ascendc::PlatformAscendC(platformInfo);
uint64_t tailSize = 0; // UB剩余空间大小
ascendcPlatform.GetCoreMemSize(platform_ascendc::CoreMemType::UB, tailSize); // 本样例中使用完整的ub
空间，实际情况下tailSize需要减掉用户已使用的UB空间
auto tmpSize = tailSize >= maxVal ? maxVal : tailSize;

AsinCustomTilingData tiling;
tiling.set_tmpBufferSize(tmpSize); // 将临时空间大小设置为Tiling参数
```

另外，多数高阶API中提供了GetxxxTmpBufferFactorSize接口，该接口用于获取maxLiveNodeCnt和extraBuf，maxLiveNodeCnt表示临时空间是单次计算数据量所占空间的多少倍；extraBuf表示Kernel侧接口所需的临时空间大小。在固定空间大小的情况下，通过maxLiveNodeCnt和extraBuf可以推算算子单次最大计算元素数量。

推算示例如下：

- 算子实现需要调用Mean接口，开发者为其预留currBuff大小的空间（即总可用空间），利用GetMeanTmpBufferFactorSize接口得到maxLiveNodeCnt、extraBuf输出值，可推导算子单次最大计算元素数量为：
$$\text{currentShapeSize} = (\text{currBuff} - \text{extraBuf}) / \text{maxLiveNodeCnt} / \text{typeSize}$$

- 算子实现需要调用两个Kernel侧API KernelIntf1、KernelIntf2，利用两个GetXxxTmpBufferFactorSize（其中Xxx为需要调用的两个高阶API）接口的两组输出值(maxLiveNodeCnt、extraBuf)以及当前现有的临时空间currBuff，推导单次最大计算元素数量currentShapeSize为：

$$\text{currentShapeSize1} = (\text{currBuff} - \text{extraBuf1}) / \text{maxLiveNodeCnt1} / \text{typeSize}$$
$$\text{currentShapeSize2} = (\text{currBuff} - \text{extraBuf2}) / \text{maxLiveNodeCnt2} / \text{typeSize}$$
$$\text{currentShapeSize} = \min(\text{currentShapeSize1}, \text{currentShapeSize2})$$

注意上文中的currBuff表示接口计算可用的空间，需要去除用户输入输出等空间。

以算子中需要同时调用Asin、Acos接口为例：

```
// 算子输入的数据类型T为half
auto shape_input = context->GetInputTensor(0)->GetOriginShape();
std::vector<int64_t> srcDims = { shape_input.GetDim(0), shape_input.GetDim(1) };
uint32_t srcSize = 1;
uint32_t srcCurSize = 1;
for (auto dim : srcDims) {
    srcSize *= dim;
}
uint32_t typeSize = 2;

auto platformInfo = context->GetPlatformInfo();
auto ascendcPlatform = platform_ascendc::PlatformAscendC(platformInfo);
uint64_t tailSize = 0; // UB剩余空间大小
ascendcPlatform.GetCoreMemSize(platform_ascendc::CoreMemType::UB, tailSize);

uint32_t asinMaxLiveNodeCount = 0;
uint32_t asinExtraBuf = 0;

uint32_t acosMaxLiveNodeCount = 0;
uint32_t acosExtraBuf = 0;

AscendC::GetAsinTmpBufferFactorSize(typeSize, asinMaxLiveNodeCount, asinExtraBuf);
AscendC::GetAcosTmpBufferFactorSize(typeSize, acosMaxLiveNodeCount, acosExtraBuf);
// tmp的大小需要减去UB上调用api接口输入和输出占用的大小
// 该示例中包括Asin接口的输入输出，以及Acos的输入输出，其中Asin接口的输出作为Acos的输入，因此一共需要3份src的空间大小
auto tmpSize = tailSize - srcSize * typeSize * 3;
assert(tmpSize >= asinExtraBuf);
assert(tmpSize >= acosExtraBuf);
// 计算Asin算子单次最大计算元素数量
if (asinMaxLiveNodeCount != 0) {
    srcAsinCurSize = (tmpSize - asinExtraBuf) / asinMaxLiveNodeCount / typeSize;
} else {
    srcAsinCurSize = srcSize;
}
// 计算Acos算子单次最大计算元素数量
if (acosMaxLiveNodeCount != 0) {
    srcAcosCurSize = (tmpSize - acosExtraBuf) / acosMaxLiveNodeCount / typeSize;
} else {
    srcAcosCurSize = srcSize;
}
srcCurSize = std::min(srcAsinCurSize, srcAcosCurSize);

AsinCustomTilingData tiling;
tiling.set_srcCurSize(srcCurSize); // 将单次最大计算元素数量设置为Tiling参数
```

2.5.4 Utils API

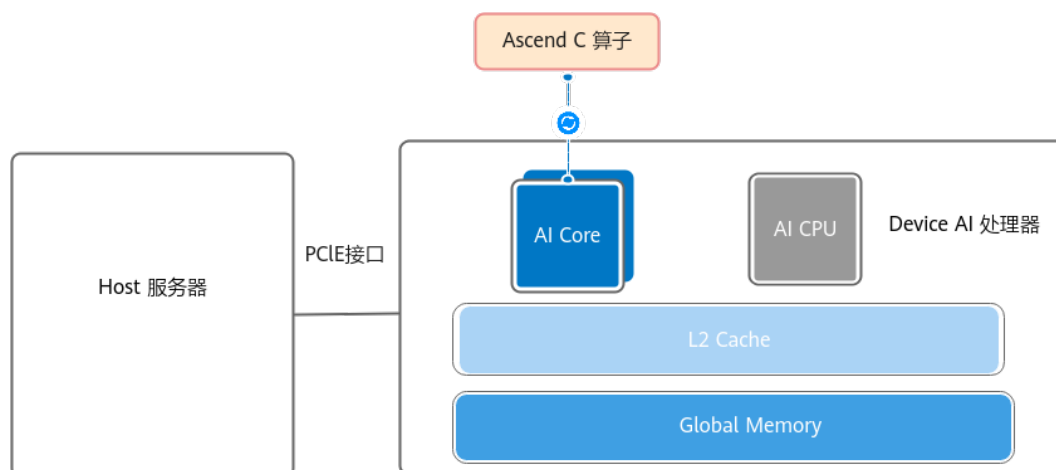
Ascend C开发提供了丰富的通用工具类，涵盖标准库、平台信息获取、上下文构建、运行时编译及日志输出等功能，支持开发者高效实现算子开发与性能优化。

- C++标准库API：提供算法、数学函数、容器函数等C++标准库函数。
- 平台信息获取API：提供获取平台信息的功能，比如获取硬件平台的核数等信息。
- RTC API：Ascend C运行时编译库，通过aclrtc接口，在程序运行时，将中间代码动态编译成目标机器码，提升程序运行性能。
- log API：提供Host侧打印Log的功能。开发者可以在算子的TilingFunc代码中使用ASC_CPU_LOG_XXX接口来输出相关内容。

2.6 硬件实现

2.6.1 基本架构

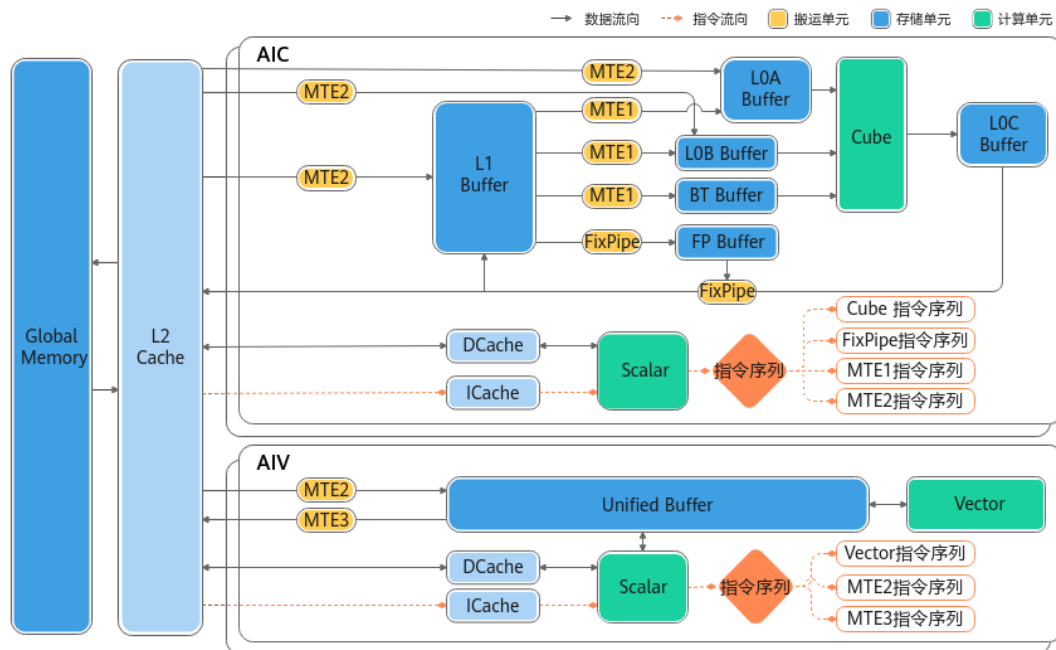
如下图所示，基于Ascend C开发的算子运行在AI Core上。Ascend C编程模型基于AI Core硬件架构的抽象进行介绍，了解硬件架构能够帮助开发者更好的理解编程模型；对于需要完成高性能编程的深度开发者，更需要了解硬件架构相关知识，算子实践参考中很多内容都以本章为基础进行介绍。



AI Core负责执行矩阵、矢量计算密集的任务，其包括以下组成部分：

- **计算单元**：包括Cube（矩阵）计算单元、Vector（矢量）计算单元和Scalar（标量）计算单元。
- **存储单元**：包括L1 Buffer、LOA Buffer、LOB Buffer、LOC Buffer、Unified Buffer、BiasTable Buffer、Fixpipe Buffer等专为高效计算设计的存储单元。
- **搬运单元**：包括MTE1、MTE2、MTE3和FixPipe，用于数据在不同存储单元之间的高效传输。

以Atlas A2 训练系列产品/Atlas A2 推理系列产品为例，硬件架构图如下：



本章节首先介绍硬件架构相关的关键概念和术语，以及AI Core的工作模式，为理解后续内容奠定基础。随后以Atlas A2 训练系列产品/Atlas A2 推理系列产品为例，提供AI Core基本架构的介绍：首先介绍计算单元、存储单元和搬运单元的基本功能与结构，然后通过典型的数据流和控制流示例，帮助开发者深入理解硬件架构的工作原理。针对不同产品型号对应的具体架构规格和细节说明需要参考后续[2.6.2 架构规格](#)章节。

关键概念和术语

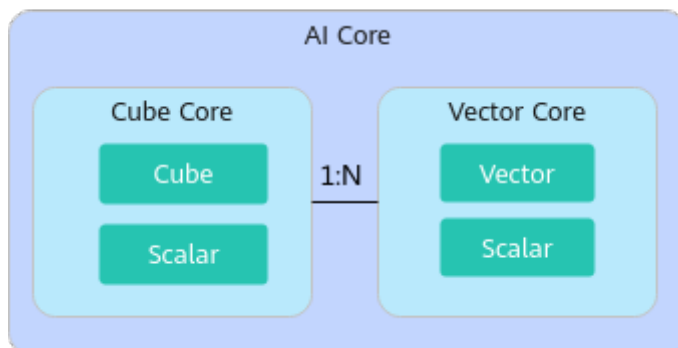
- Core
拥有独立Scalar计算单元的计算核。Scalar计算单元是核内的调度单元，承担了核内的指令发射等功能。
- AI Core
AI处理器的计算核，负责执行矩阵、矢量计算密集的任务。
- Cube Core
矩阵计算核，专注于矩阵计算。由Scalar调度单元、矩阵计算单元、搬运单元等组成，不包括矢量计算单元。
- Vector Core
矢量计算核，专注于矢量计算，由Scalar调度单元、矢量计算单元、搬运单元等组成，不包括矩阵计算单元。
- AIC
在AI Core[分离模式](#)下，一组Cube Core和Vector Core组合中的Cube Core。
- AIV
在AI Core[分离模式](#)下，一组Cube Core和Vector Core组合中的Vector Core。

AI Core 的工作模式

- 分离模式
AI Core的一种工作模式，矩阵计算单元、矢量计算单元各自对应独立的Scalar调度单元，分离部署在Cube Core和Vector Core上。将Cube Core和Vector Core按

照一定比例（1：N）进行组合，这样的组合视为一个AI Core，AI Core的核数以Cube Core为准。

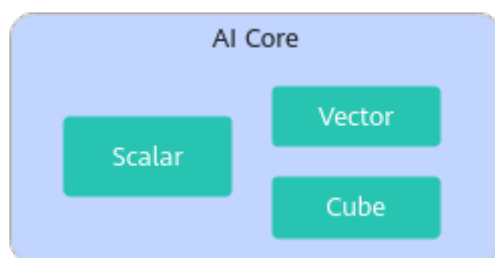
图 2-20 分离模式示意图（N 的取值以硬件平台信息获取接口获取的数值为准）



- 耦合模式

AI Core的一种工作模式，矩阵计算单元、矢量计算单元对应同一个Scalar调度单元，部署在一个AI Core上。

图 2-21 耦合模式示意图



说明

Ascend C编程中，不同产品的工作模式如下：

- Atlas 推理系列产品：耦合模式
- Atlas 训练系列产品：耦合模式
- Atlas A2 训练系列产品/Atlas A2 推理系列产品：分离模式
- Atlas A3 训练系列产品/Atlas A3 推理系列产品：分离模式
- Atlas 200I/500 A2 推理产品：耦合模式

注意：针对Atlas 200I/500 A2 推理产品，硬件的工作模式既可以支持耦合模式，又可以支持分离模式。耦合模式下，开发者仅需关注AI Core数量，无需关注Vector Core和Cube Core数量；分离模式下，需要关注AI Core、Vector Core、Cube Core的数量。Ascend C编程场景下，仅支持耦合模式。

计算单元

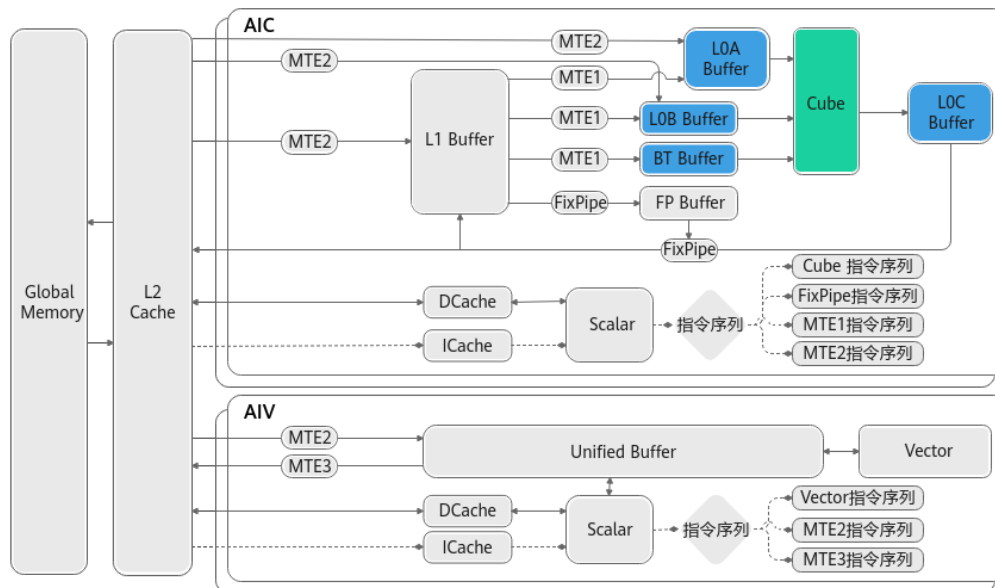
计算单元是AI Core中提供强大算力的核心单元，包括三种基础**计算单元**：Cube（矩阵）计算单元、Vector（矢量）计算单元和Scalar（标量）计算单元，完成AI Core中不同类型的数据计算。

- Cube

Cube计算单元负责执行矩阵运算，以float16数据类型为例，Cube每次执行可完成两个float16类型的16x16矩阵的乘法操作。如下图所示，高亮部分为Cube计算

单元及其访问的存储单元，其中LOA存储左矩阵，LOB存储右矩阵，LOC存储矩阵乘的结果和中间结果。

图 2-22 Cube 计算单元数据访问

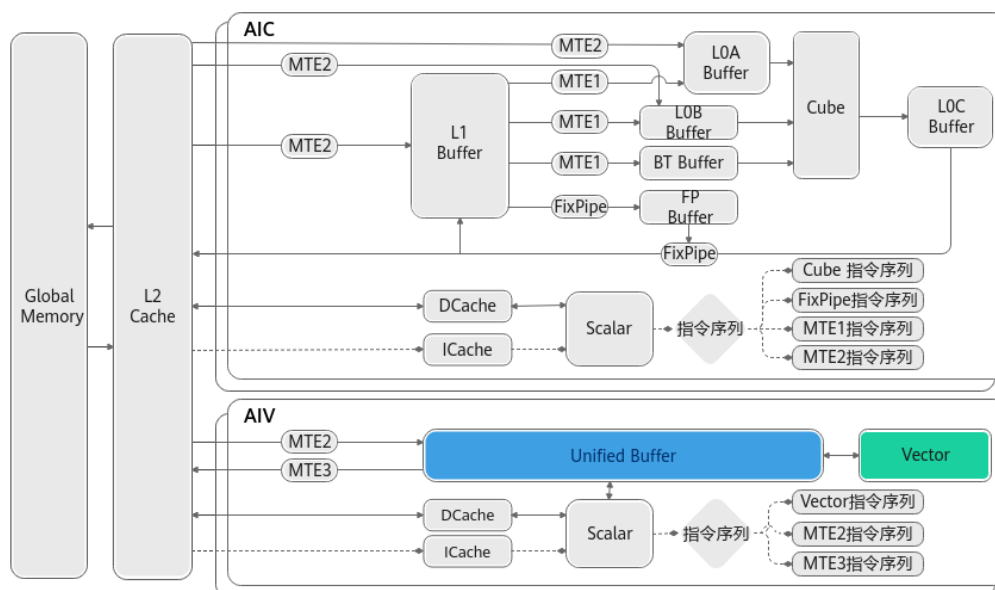


- Vector

Vector负责执行向量运算。向量计算单元执行向量指令，类似于传统的单指令多数据（Single Instruction Multiple Data, SIMD）指令，每个向量指令可以完成多个操作数的同一类型运算。向量计算单元可以快速完成两个float16类型的向量相加或者相乘。向量指令支持多次迭代执行，也支持对带有间隔的向量直接进行运算。

如下图所示，Vector所有计算的源数据以及目标数据都要求存储在Unified Buffer中，Vector指令的首地址和操作长度有对齐要求，通常要求32B对齐，具体对齐要求参考API的约束描述。

图 2-23 Vector 计算单元数据访问

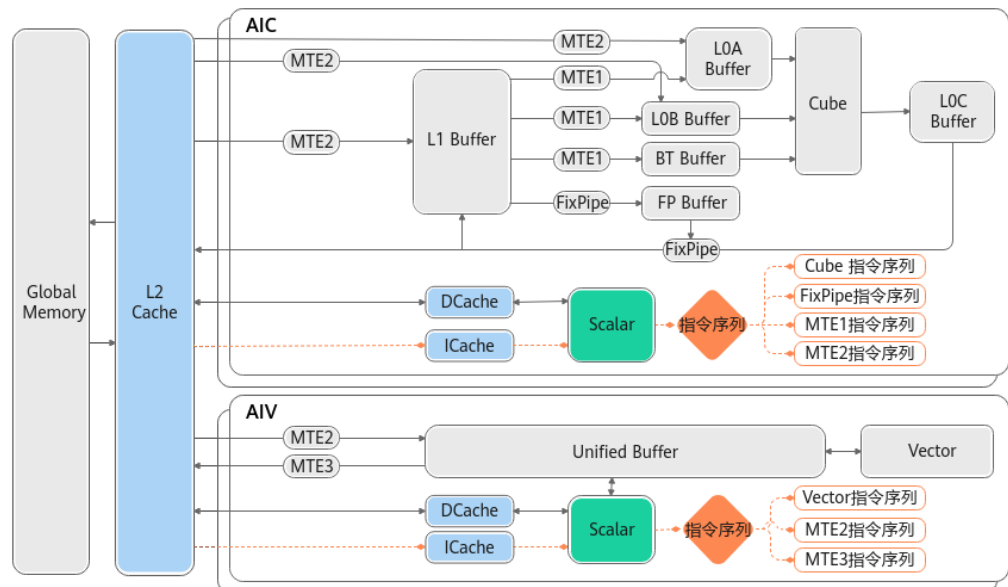


- Scalar

Scalar负责各类型的标量数据运算和程序的流程控制。功能上可以看做一个小CPU，完成整个程序的循环控制、分支判断、Cube/Vector等指令的地址和参数计算以及基本的算术运算，并且可以通过在事件同步模块中插入同步符的方式来控制AI Core中其他执行单元的流水。相对于Host CPU，AI Core中的Scalar计算能力较弱，重点用于发射指令，所以在实际应用场景中应尽量减少Scalar计算，比如性能调优时尽量减少if/else等分支判断及变量运算。

如下图所示：Scalar执行标量运算指令时，执行标准的ALU(Arithmetic Logic Unit)语句，ALU需要的代码段和数据段（栈空间）都来自于GM，ICache(Instruction Cache)用于缓存代码段，缓存大小与硬件规格相关，比如为16K或32K，以2K为单位加载；DCache(Data Cache)用于缓存数据段，大小也与硬件规格相关，比如为16K，以Cache Line（64Byte）为单位加载。考虑到核内访问效率最高，应尽量保证代码段和数据段被缓存在ICache和DCache，避免核外访问；同时根据数据加载单位不同，编程时可以考虑单次加载数据大小，来提升加载效率。例如在DCache加载数据时，当数据内存首地址与Cache Line（64Byte）对齐时，加载效率最高。

图 2-24 Scalar 对指令和数据的访问

 **说明**

硬件提供L2Cache用于缓存访问GM的数据（包括代码段、数据段），以此加快访问速度，提高访问效率。核外L2Cache以Cache Line为单位加载数据，根据硬件规格不同，Cache Line大小不同（128/256/512Byte等）。

存储单元和搬运单元

AI处理器中的计算资源要想发挥强劲算力，必要条件是保证输入数据能够及时准确地出现在计算单元中，需要精心设计存储系统，保证计算单元所需的数据供应。

如下图所示：AI Core中包含多级内部存储，AI Core需要把外部存储中的数据加载到内部存储中，才能完成相应的计算。AI Core的主要内部存储包括：L1 Buffer（L1缓冲区），L0 Buffer（L0缓冲区），Unified Buffer（统一缓冲区）等。为了配合AI Core中的数据传输和搬运，AI Core中还包含MTE（Memory Transfer Engine，数据传递引擎）搬运单元，在搬运过程中可执行随路数据格式/类型转换。

内部存储单元和搬运单元的具体介绍请参考表2-17和表2-18。

图 2-25 存储单元

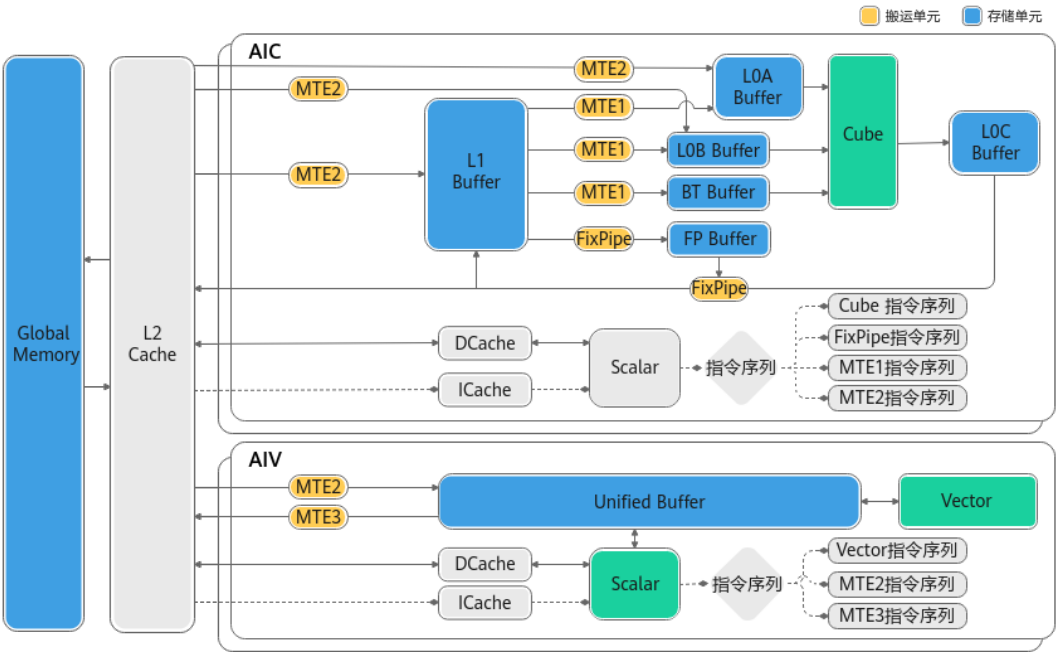


表 2-17 存储单元介绍

存储单元	描述
L1 Buffer	L1缓冲区，通用内部存储，是AI Core内比较大的一块数据中转区，可暂存Cube计算单元需要反复使用的一些数据从而减少从总线读写的次数。
L0A Buffer / L0B Buffer	Cube指令的输入。
L0C Buffer	Cube指令的输出，但进行累加计算的时候，也是输入的一部分。
Unified Buffer	统一缓冲区，向量和标量计算的输入和输出。
BT Buffer	BiasTable Buffer，存放矩阵计算中的Bias。
FP Buffer	Fixpipe Buffer，存放量化参数、Relu参数等。

表 2-18 搬运单元介绍

搬运单元	描述
MTE1	负责如下通路的数据搬运： - L1->L0A/L0B - L1->BT Buffer

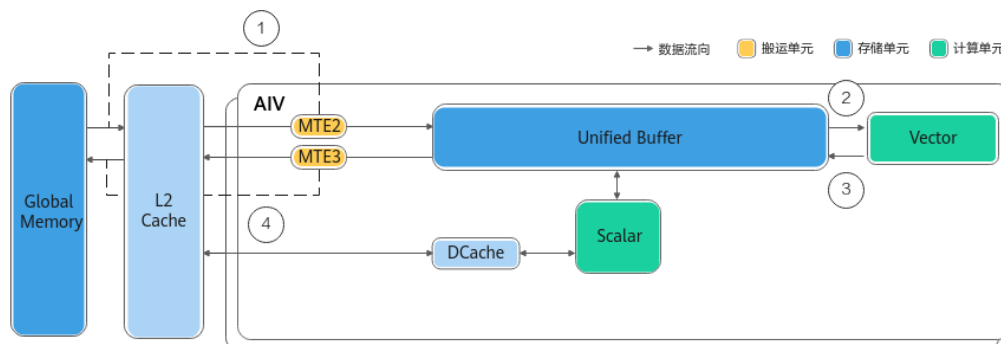
搬运单元	描述
MTE2	负责如下通路的数据搬运： - GM->{L1, L0A/B}，在该通路下，基于分形大小搬运，搬运时满足Cache Line大小对齐，性能更优。 - GM->UB，基于Cache Line大小搬运性能更优。
MTE3	负责如下通路的数据搬运： UB -> GM
FixPipe	负责如下通路的数据搬运，搬运过程中可以完成随路数据格式/类型转换： - L0C->{GM/L1} - L1->FP Buffer

说明

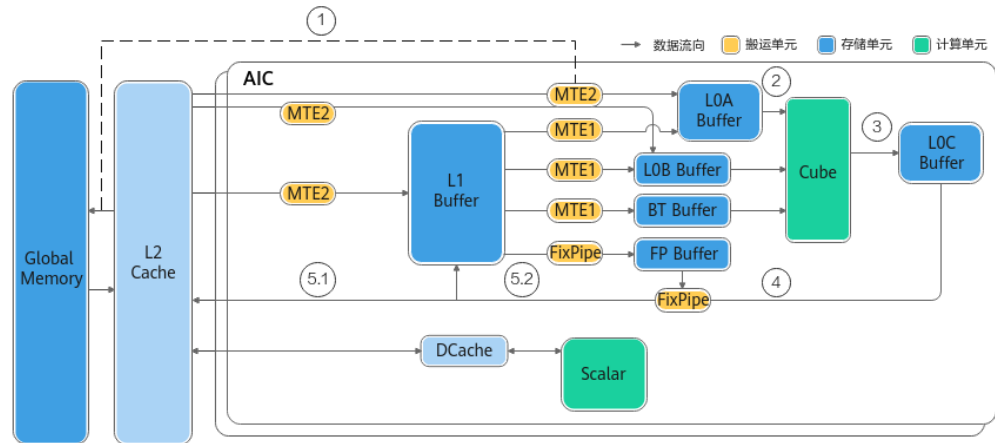
- 不同类型的AI处理器，存储单元大小不同，开发者可通过GetCoreMemSize接口获取。
- 所有通过搬运单元读写GM的数据都缺省被缓存在L2Cache，以此加快访问速度，提高访问效率。核外L2Cache以Cache Line为单位加载数据，根据硬件规格不同，Cache Line大小不同（128/256/512Byte等）。

典型的数据流

- Vector计算典型的数据流如下：
GM → UB → Vector → UB → GM



- Cube计算典型的数据流如下：
 - GM → L1 → L0A/L0B → Cube → L0C → FixPipe → GM
 - GM → L1 → L0A/L0B → Cube → L0C → FixPipe → L1

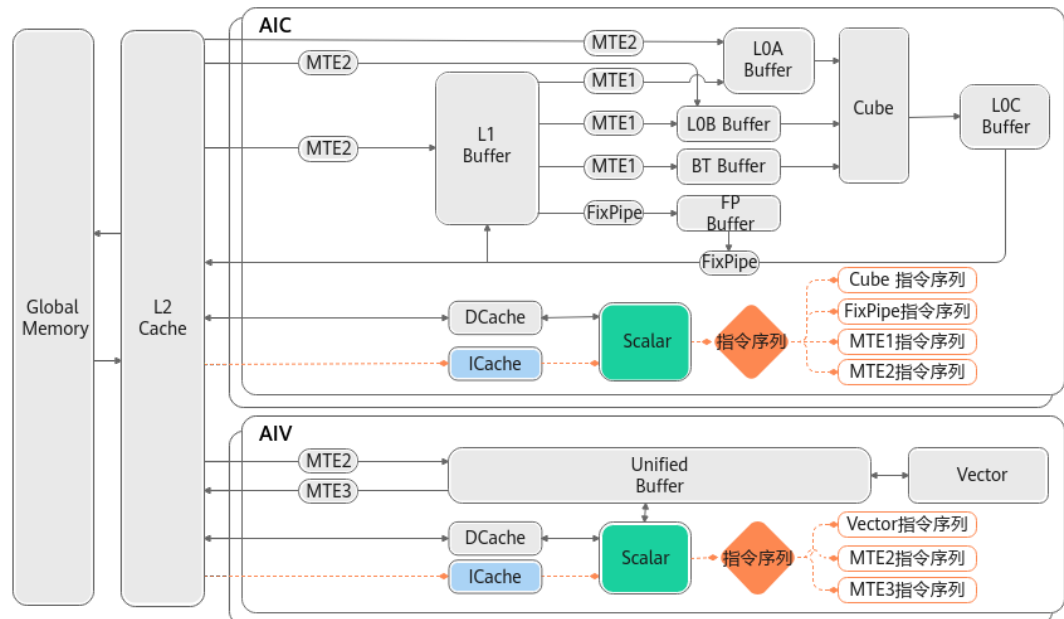


典型的指令流

多条指令从系统内存通过总线接口进入到ICache(Instruction Cache)中，后续的指令执行过程，根据指令的类型，有两种可能：

- 如果指令是Scalar指令，指令会被Scalar单元直接执行。
- 其他指令会被Scalar单元调度到独立的分类序列（Vector指令序列、Cube指令序列、MTE1/MTE2/MTE3指令序列等），然后再被对应的执行单元执行。

图 2-26 指令分类处理机制



同一个指令序列中的指令是按照进入指令序列的顺序执行的，不同指令序列之间可以并行执行，通过多个指令序列的并行执行可以提升整体执行效率。对于并行执行过程中可能出现的数据依赖，通过事件同步模块插入同步指令来控制流水线的同步，提供PipeBarrier、SetFlag/WaitFlag两种API，保证序列内部以及序列之间按照逻辑关系执行。

- PipeBarrier本身是一条指令，用于在序列内部约束执行顺序（虽然指令是顺序执行，但并不意味着后一条指令开始执行时前一条指令执行结束）。PipeBarrier指令可以保证前序指令中所有数据读写全部完成，后序指令才开始执行。

- SetFlag/WaitFlag为两条指令，在SetFlag/WaitFlag的指令中，可以指定一对指令序列的关系，表示两个序列之间完成一组“锁”机制，其作用方式为：
 - SetFlag：当前序指令的所有读写操作都完成之后，当前指令开始执行，并将硬件中的对应标志位设置为1。
 - WaitFlag：当执行到该指令时，如果发现对应标志位为0，该序列的后续指令将一直被阻塞；如果发现对应标志位为1，则将对标志位设置为0，同时后续指令开始执行。

Ascend C提供同步控制API，开发者可以使用这类API来自行完成同步控制。需要注意的是，通常情况下，开发者基于2.2 编程模型中介绍的编程模型和范式进行编程时不需要关注同步，编程模型帮助开发者完成了同步控制；使用编程模型和范式是我们推荐的编程方式，自行同步控制可能会带来一定的编程复杂度。

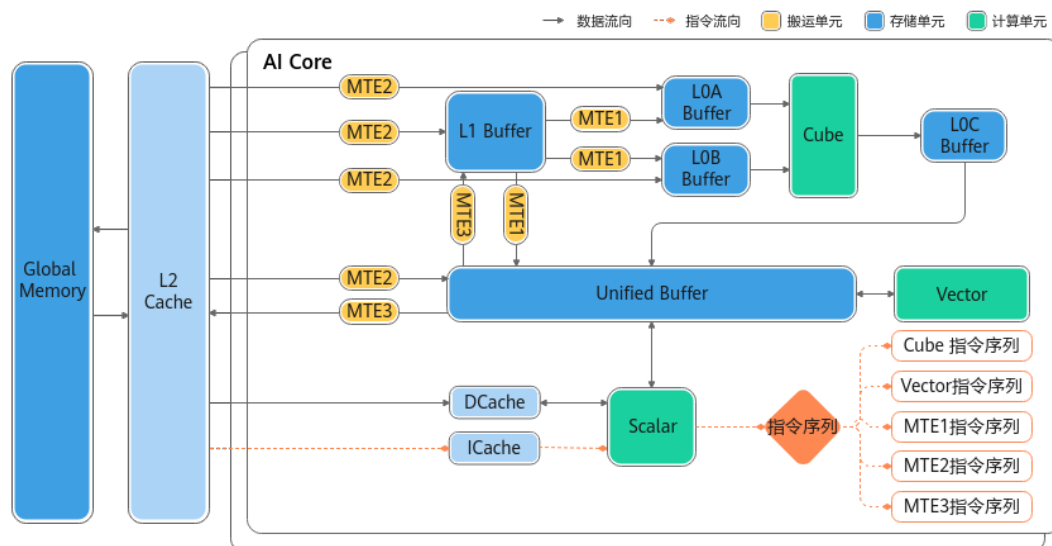
但是我们仍然希望开发者可以理解同步的基本原理，便于后续更好的理解设计并行计算程序；同时少数情况需要开发者手动插入同步，您可以通过什么时候需要开发者手动插入同步来了解具体内容。

2.6.2 架构规格

2.6.2.1 NPU 架构版本 200x

本节介绍_NPU_ARCH_版本号200x的硬件架构和其功能说明，其中200代表IP核编号，x表示同一个IP核的配置版本号。对应的产品型号为Atlas 推理系列产品。

硬件架构图



计算单元

Cube计算单元和Vector计算单元同核部署

本架构中，Cube计算单元和Vector计算单元同核部署，共享同一个Scalar计算单元。

Vector计算单元

- Vector计算单元的数据来源来自于Unified Buffer，要求32字节对齐。

- 数据从L0C Buffer传输至Unified Buffer需要以Vector计算单元作为中转。

Cube计算单元

- Cube计算单元可以访问的存储单元有L0A Buffer、L0B Buffer、L0C Buffer，其中L0A Buffer存储左矩阵，L0B Buffer存储右矩阵，L0C Buffer存储矩阵乘的结果和中间结果。

存储单元

获取存储单元的内存空间大小

开发者可以通过平台信息获取接口查询各存储单元的内存空间大小。

各存储单元的最小访问粒度（对齐要求）

存储单元	对齐要求
Unified Buffer	32Byte对齐。
L1 Buffer	32Byte对齐。
L0A Buffer	512Byte对齐。
L0B Buffer	512Byte对齐。
L0C Buffer	64Byte对齐。

各存储单元推荐使用的数据排布格式

- L0A Buffer、L0B Buffer和L0C Buffer推荐分别采用以下分形格式：
 - L0A Buffer: FRACTAL_ZZ
 - L0B Buffer: FRACTAL_ZN
 - L0C Buffer: FRACTAL_NZ这些格式针对矩阵乘法等计算密集型任务进行优化，可显著提升计算效率。
- L1 Buffer缓存推荐使用FRACTAL_NZ格式。当L1采用NZ格式时，数据搬运到L0A/L0B Buffer（需分别转换为ZZ和ZN格式）时，可降低格式转换开销。
- Unified Buffer对数据格式没有要求。

解决存储单元的访问冲突，提升读写性能

当多个操作尝试同时访问Unified Buffer同一个bank或者bank group时，可能会发生bank冲突，包括读写冲突、写写冲突、读读冲突，这种冲突会导致访问排队，降低性能。可以通过优化bank分配的方式来提升读写性能，具体信息请参考[3.6.5.9 避免Unified Buffer的bank冲突](#)章节。

搬运单元

搬运时的对齐要求

由于搬运后的数据用于参与数据计算，因此对搬运数据大小有要求，搬运到Unified Buffer的数据大小需要按照DataBlock对齐，其余存储单元的数据搬运必须按[分形要求](#)进行搬运。例如，数据从L1 Buffer搬运到L0A Buffer时，数据格式需要从NZ转换为ZZ

格式，搬运数据的大小要按分形大小对齐，如果L1 Buffer的剩余大小不足1个分形，则硬件执行中会出现异常。

同步控制

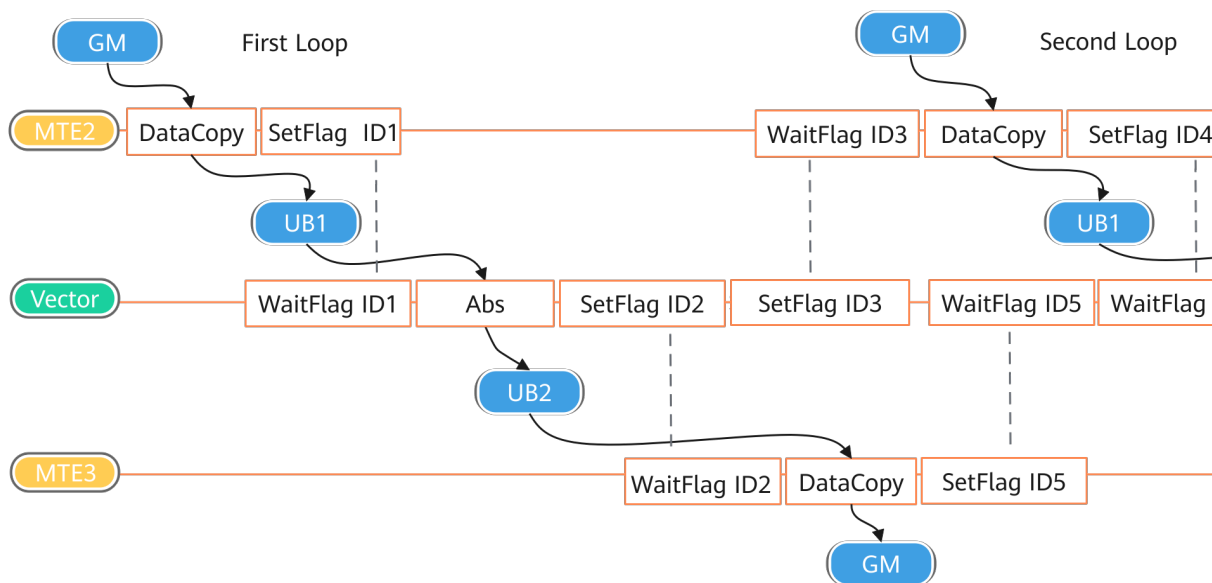
核内同步

由于AI Core内部的执行单元（如MTE2搬运单元、Vector计算单元等）以异步并行的方式运行，在读写Local Memory（如Unified Buffer）时可能存在数据依赖关系。为确保数据一致性及计算正确性，需通过同步控制协调操作时序。

以MTE2从GM搬运数据至UB，进行Vector计算单元的Abs计算，再搬运回GM的流程为例，需满足以下同步条件：

1. 数据搬运与计算顺序
 - GM → UB搬运完成后再启动Vector单元的Abs计算（避免计算时未完成搬运导致的数据缺失）；
 - Vector计算完成后执行UB → GM的数据搬运（确保结果数据已就绪）。
2. 循环搬运计算场景的同步规则
 - 前序计算完成后启动新搬运：上一次计算未完成时，不得触发新数据搬运（防止UB中旧数据被覆盖）；
 - 前序数据搬出完成后启动新计算：上一次数据未完全从UB搬出时，不得触发新计算任务（避免目标内存区域的覆盖冲突）。

同步控制流程如下图所示：



上图中，ID1、ID2、ID3、ID4、ID5、ID6表示事件ID（EventID），每个EventID对应一块存储数据的搬运状态，确保数据操作的正确性和一致性。

需要注意以下几点：

- 建议通过AllocEventID或者FetchEventID接口获取EventID，以确保其合法性和有效性。
- EventID的数量有限，使用后应立即调用ReleaseEventID释放资源，避免EventID耗尽，影响系统正常运行。

- SetFlag和WaitFlag必须成对使用，且SetFlag和WaitFlag的参数必须完全一致（包括模板参数和事件ID）。如果不匹配，可能导致当前核的计算异常，或影响下一个核的算子执行，引发timeout问题。

例如，SetFlag<HardEvent::S_MTE3>(1)和SetFlag<HardEvent::MTE3_MTE1>(1)设置的不是同一个EventID，因为其模板参数不同。只有当模板参数和事件ID完全一致时，才表示同一个EventID。

- 不允许连续设置同一个EventID，因为这可能导致事件状态混乱或未被正确处理。

核间同步

该硬件架构不支持核间同步。

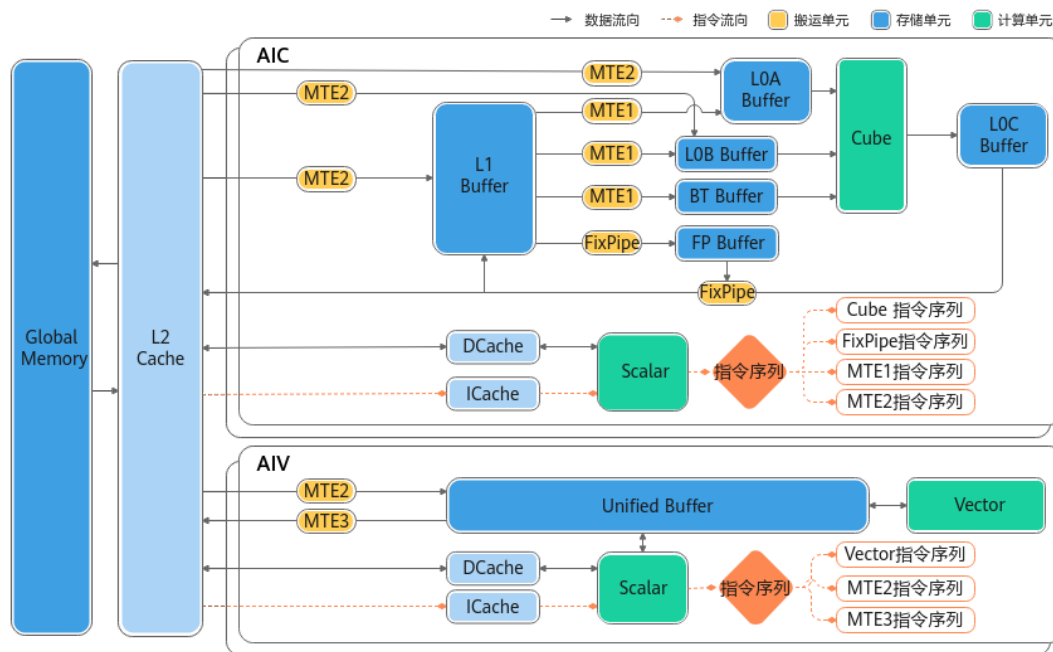
2.6.2.2 NPU 架构版本 220x

本节介绍__NPU_ARCH__版本号为220x的硬件架构和其功能说明，其中220代表IP核编号，x表示同一个IP核的配置版本号。对应的产品型号为：

- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品

硬件架构图

如下图所示，本架构中AI Core分为AIC和AIV两个独立的核，分别用于矩阵计算和向量计算。每个核都有自己的Scalar单元，能独立加载自己的代码段。AIV与AIC之间通过Global Memory进行数据传递。



计算单元

Cube计算单元和Vector计算单元分离部署

本架构中，Cube计算单元和Vector计算单元分别部署在AIC核和AIV核上，每个核都有自己的Scalar单元，能独立加载自己的代码段。

Vector计算单元

- Vector计算单元的数据来自于Unified Buffer，要求32字节对齐。

Cube计算单元

- Cube计算单元可以访问的存储单元有LOA Buffer、LOB Buffer、LOC Buffer，其中LOA Buffer存储左矩阵，LOB Buffer存储右矩阵，LOC Buffer存储矩阵乘的结果和中间结果。

存储单元

获取存储单元的内存空间大小

开发者可以通过平台信息获取接口查询各存储单元的内存空间大小。

各存储单元的最小访问粒度（对齐要求）

核	存储单元	对齐要求
AIV	Unified Buffer	32Byte对齐。
AIC	L1 Buffer	32Byte对齐。
	LOA Buffer	512Byte对齐。
	LOB Buffer	512Byte对齐。
	LOC Buffer	64Byte对齐。
	BiasTable Buffer	64Byte对齐。
	Fixpipe Buffer	64Byte对齐。

各存储单元推荐使用的数据排布格式

- LOA Buffer、LOB Buffer和LOC Buffer推荐分别采用以下分形格式：
 - LOA Buffer: FRACTAL_ZZ
 - LOB Buffer: FRACTAL_ZN
 - LOC Buffer: FRACTAL_NZ这些格式针对矩阵乘法等计算密集型任务进行优化，可显著提升计算效率。
- L1 Buffer缓存推荐使用FRACTAL_NZ格式。当L1 Buffer采用NZ格式时，数据搬运到LOA/LOB Buffer（需分别转换为ZZ和ZN格式）时，可降低格式转换开销。
- Unified Buffer对数据格式没有要求。

解决存储单元的访问冲突，提升读写性能

当多个操作尝试同时访问Unified Buffer同一个bank或者bank group时，可能会发生bank冲突，包括读写冲突、写写冲突、读读冲突，这种冲突会导致访问排队，降低性能。可以通过优化bank分配的方式来提升读写性能，具体信息请参考[3.6.5.9 避免 Unified Buffer的bank冲突](#)章节。

搬运单元

搬运时的对齐要求

由于搬运后的数据用于参与数据计算，因此对搬运数据大小有要求，搬运到Unified Buffer的数据大小需要按照DataBlock对齐，其余存储单元的数据搬运必须按**分形要求**进行搬运。例如，数据从L1 Buffer搬运到L0A Buffer时，数据格式需要从NZ转换为ZZ格式，搬运数据的大小要按分形大小对齐，如果L1 Buffer的剩余大小不足1个分形，则硬件执行中会出现异常。

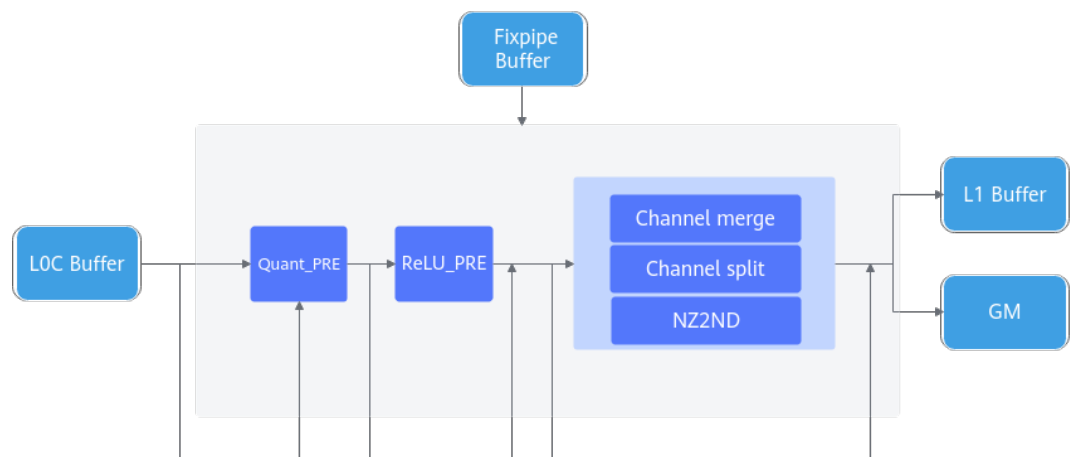
支持跨卡数据搬运（Hccs物理链路）

在跨卡通信算子开发场景，DataCopy类接口支持跨卡数据搬运，在Atlas A2 训练系列产品/Atlas A2 推理系列产品设备上，仅支持Hccs物理链路，不支持其他通路；开发者开发过程中，请关注涉及卡间通信的物理通路；通过npd-smi info -t topo指令查询Hccs 物理通路。

支持Fixpipe硬件化加速

Fixpipe是NPU将典型操作进行硬化的加速模块，位于AIC内部，配合Cube计算单元完成随路计算，主要功能如下：

- 量化反量化：包括S322FP16、S322S32、S322S4、S322S8、S322S16、FP322FP16、FP322BF16、FP322S8、FP322S4、FP322FP32。
- Relu功能，包括ReLU、PReLU和Leaky ReLU等典型的激活函数。
- 数据格式转换，包括：
 - 通过Channel merge、Channel split可以实现分形大小的转换，保证输出到L1 Buffer/GM的分形满足诉求。
 - NZ2ND数据格式转换。

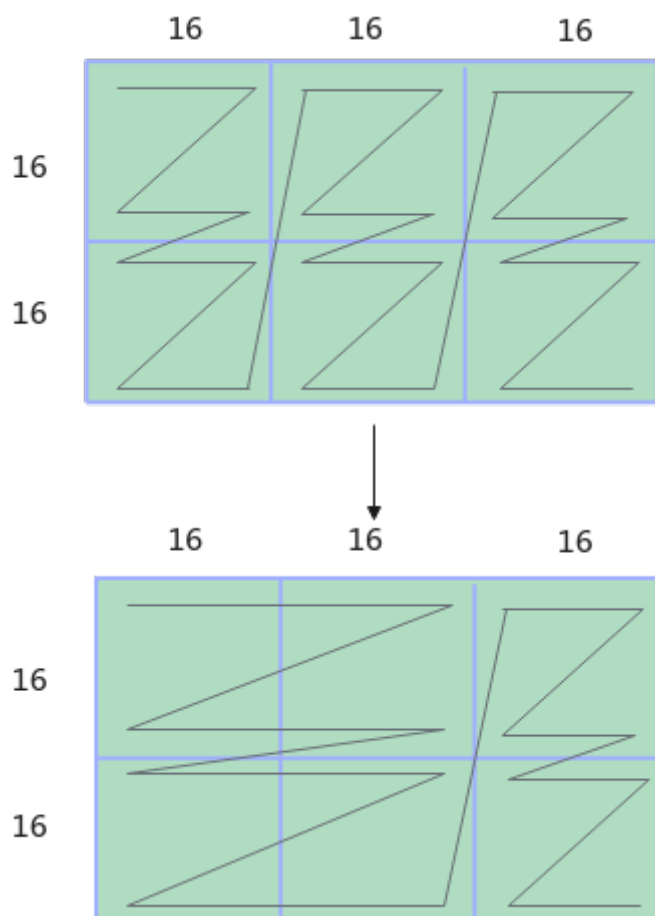


上图中，Channel merge支持S8、U8、S4和U4数据类型，而Channel split支持FP32数据类型。

- Channel merge（S8和U8数据类型）

对于转换为S8或U8的目标数据类型，分形矩阵通过硬件从16x16转换为16x32，如果输出通道数N是16的偶数倍，则N方向上每2个相邻的16x16分形矩阵将合并为1个16x32分形矩阵。如果N是16的奇数倍，则将通道1到通道（N-16）合并，最后16个通道保持未合并。

如下所示，目标数据类型为S8，M为32，N为48，首先将前2列16x16分形矩阵合并为一个16x32矩阵，然后将剩余的16x16分形矩阵直接移入L1 Buffer。

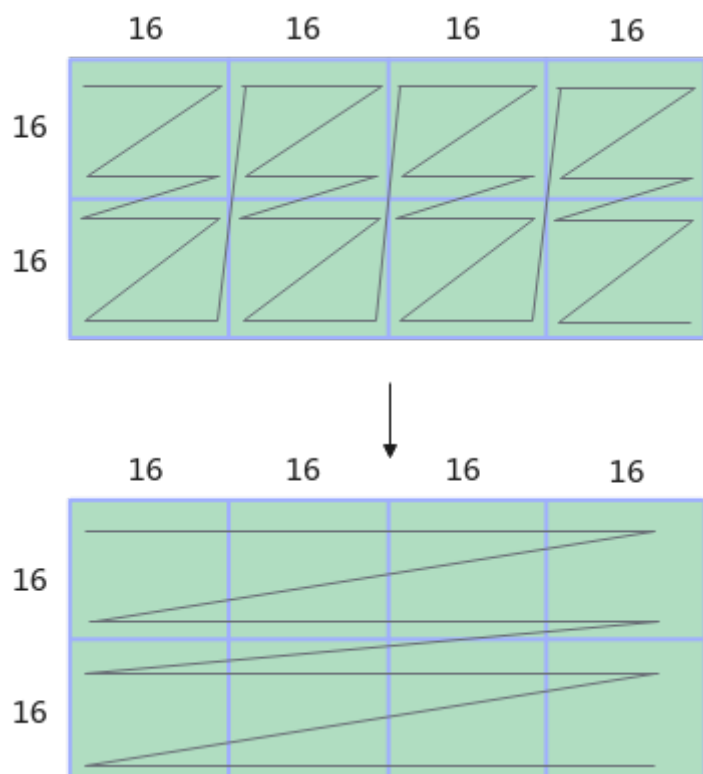


- Channel merge (S4和U4数据类型)

对于转换为S4或U4的目标数据类型，分形矩阵通过硬件从16x16转换为16x64，如果输出通道数N是64的倍数，则N方向上每4个相邻的16x16分形矩阵将合并为1个16x64分形矩阵。

例如，这里目标数据类型为S4，M为32，N为64，首先将第1行16x16分形矩阵合并为一个16x64矩阵，然后将第2行16x16分形矩阵也合并。

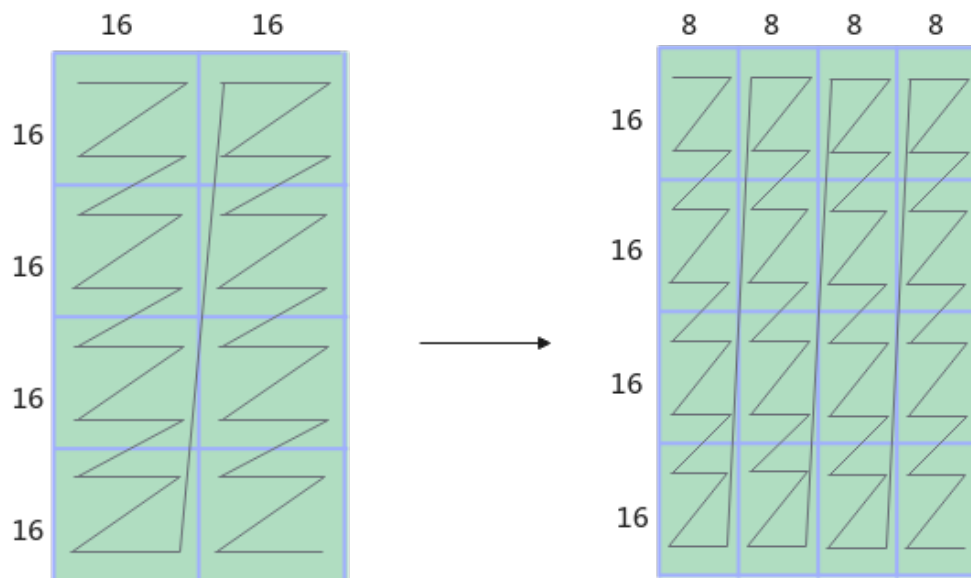
在这种情况下，N的配置必须是64的倍数。



- FP32 Channel split:

对于目标类型为FP32，分形矩阵可以通过硬件从16x16转换为16x8，如果使能Channel split，则每个16x16分形矩阵将被分裂为2个16x8分形矩阵。

如下图所示，这里的目标数据类型是FP32，M是64，N是32，它将被拆分为16个16x8的分形。



同步控制

- 核内同步

由于AI Core内部的执行单元（如MTE2搬运单元、Vector计算单元等）以异步并行的方式运行，在读写Local Memory（如Unified Buffer）时可能存在数据依赖关系。为确保数据一致性及计算正确性，需通过同步控制协调操作时序。

以MTE2从GM搬运数据至UB，进行Vector计算单元的Abs计算，再搬运回GM的流程为例，需满足以下同步条件：

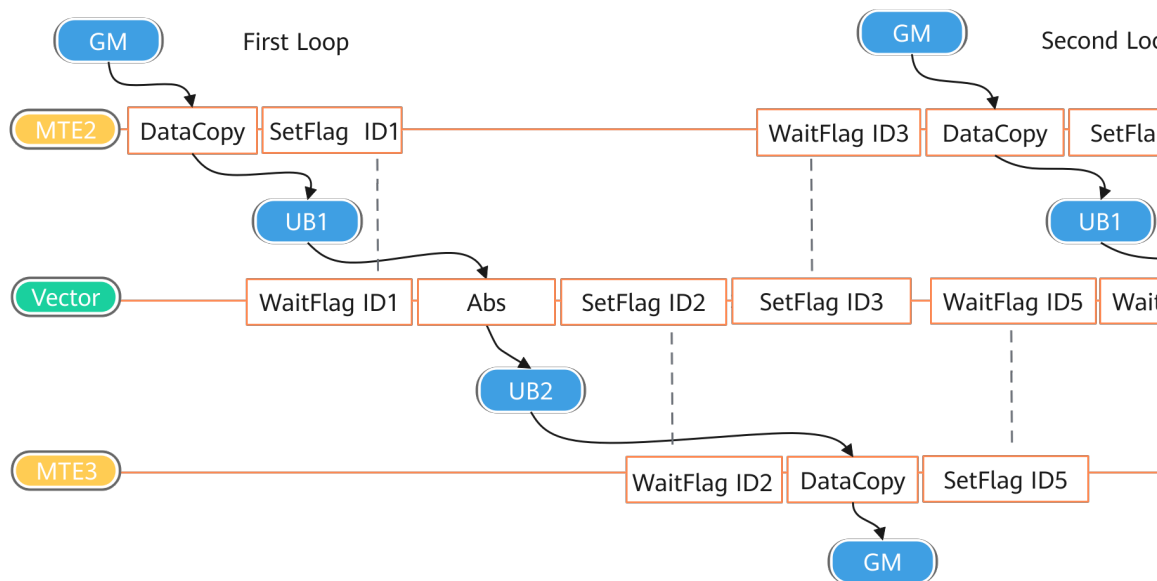
a. 数据搬运与计算顺序

- GM → UB搬运完成后启动Vector单元的Abs计算（避免计算时未完成搬运导致的数据缺失）；
- Vector计算完成后执行UB → GM的数据搬运（确保结果数据已就绪）。

b. 循环搬运计算场景的同步规则

- 前序计算完成后启动新搬运：上一次计算未完成时，不得触发新数据搬运（防止UB中旧数据被覆盖）；
- 前序数据搬出完成后启动新计算：上一次数据未完全从UB搬出时，不得触发新计算任务（避免目标内存区域的覆盖冲突）。

同步控制流程如下图所示：



上图中，ID1、ID2、ID3、ID4、ID5、ID6表示事件ID（EventID），每个EventID对应一块存储数据的搬运状态，确保数据操作的正确性和一致性。

需要注意以下几点：

- 建议通过AllocEventID或者FetchEventID接口获取EventID，以确保其合法性和有效性。
- EventID的数量有限，使用后应立即调用ReleaseEventID释放资源，避免EventID耗尽，影响系统正常运行。
- SetFlag和WaitFlag必须成对使用，且SetFlag和WaitFlag的参数必须完全一致（包括模板参数和事件ID）。如果不匹配，可能导致当前核的计算异常，或影响下一个核的算子执行，引发timeout问题。

例如，SetFlag<HardEvent::S_MTE3>(1)和

SetFlag<HardEvent::MTE3_MTE1>(1)设置的不是同一个EventID，因为其模板参数不同。只有当模板参数和事件ID完全一致时，才表示同一个EventID。

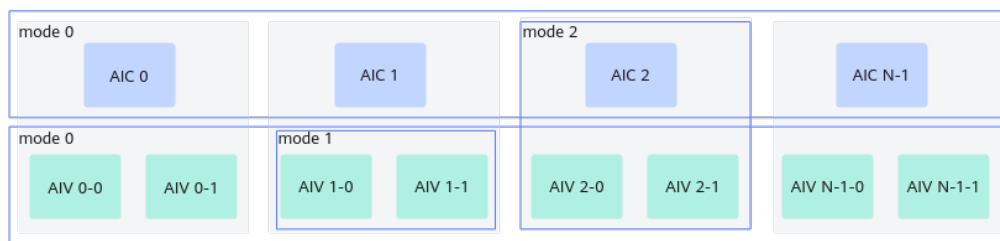
- 不允许连续设置同一个EventID，因为这可能导致事件状态混乱或未被正确处理。
- 不建议手动插入 TEventID，不能手动插入6和7的TEventID，因为它们可能被系统预留或用于特殊用途。

- 核间同步

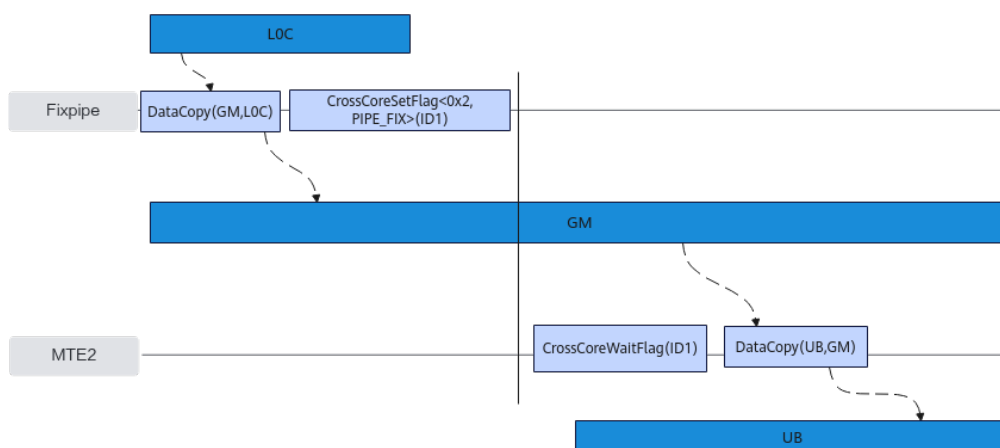
当不同核之间操作同一块全局内存时，可能存在读后写、写后读以及写后写等数据依赖问题，需要进行核间同步控制。

核间同步控制分为以下几种模式，如下图所示：

- 模式0：AI Core核间的同步控制。对于AIC场景，同步所有的AIC核，直到所有的AIC核都执行到CrossCoreSetFlag时，CrossCoreWaitFlag后续的指令才会执行；对于AIV场景，同步所有的AIV核，直到所有的AIV核都执行到CrossCoreSetFlag时，CrossCoreWaitFlag后续的指令才会执行。
- 模式1：AI Core内部，AIV核之间的同步控制。如果两个AIV核都运行了CrossCoreSetFlag，CrossCoreWaitFlag后续的指令才会执行。
- 模式2：AI Core内部，AIC与AIV之间的同步控制。在AIC核执行CrossCoreSetFlag之后，两个AIV上CrossCoreWaitFlag后续的指令才会继续执行；两个AIV都执行CrossCoreSetFlag后，AIC上CrossCoreWaitFlag后续的指令才能执行。



例如，在AIC中将LOC的计算结果搬运到GM后，AIV需要将GM的数据搬运到UB。此时，可以使用CrossCoreSetFlag和CrossCoreWaitFlag命令，确保数据从LOC成功搬运到GM后，再从GM搬运到UB，流程如下图所示。



CrossCoreSetFlag和CrossCoreWaitFlag接口配合使用。使用时需传入核间同步的标记ID(flagId)，即上图中的ID1，每个ID对应一个初始值为0的计数器。执行CrossCoreSetFlag后ID对应的计数器增加1；执行CrossCoreWaitFlag时如果对应的计数器数值为0则阻塞不执行；如果对应的计数器大于0，则计数器减一，同时后续指令开始执行。flagId取值范围是0-10。

需要注意以下几点：

- 成对使用

CrossCoreSetFlag和CrossCoreWaitFlag必须成对使用，否则可能导致算子超时问题。

- **一致性要求**

CrossCoreSetFlag 的模板参数和flagId必须与CrossCoreWaitFlag完全一致，否则视为不同的flagId。例如，CrossCoreSetFlag<0x0, PIPE_MTE3>(0x8) 和 CrossCoreSetFlag<0x2, PIPE_FIX>(0x8) 设置的不是同一个flagId。

- **避免连续设置**

不允许连续设置同一个flagId，以防止计数器状态混乱。

- **与高阶 API 的使用冲突**

Matmul高阶API内部实现中使用了本接口进行核间同步控制，所以不建议开发者同时使用该接口和Matmul高阶API，否则会有flagId冲突的风险。

- **计数器限制**

同一flagId的计数器最多可以设置15次。

- **默认流水类型**

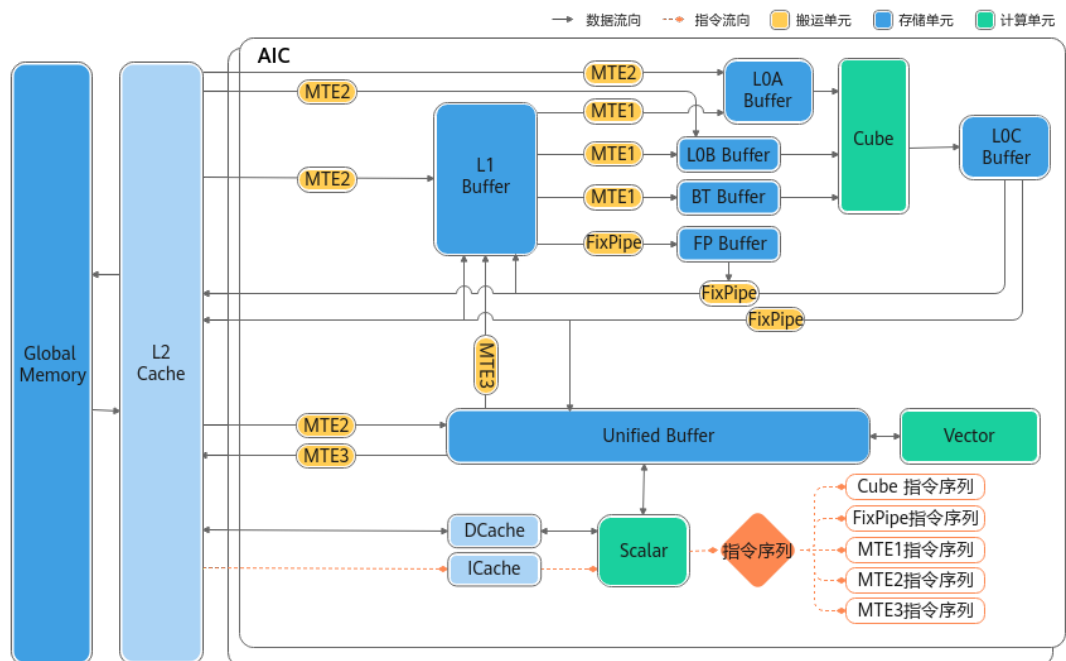
CrossCoreWaitFlag不需要显式设置指令所在的流水类型，默认使用PIPE_S。

2.6.2.3 NPU 架构版本 300x

本节介绍_NPU_ARCH_版本号为300x的硬件架构和其功能说明，其中300代表IP核编号，x表示同一个IP核的配置版本号。对应的产品型号为：

- Atlas 200I/500 A2 推理产品

硬件架构图



计算单元

Cube计算单元和Vector计算单元同核部署

本架构中，Cube计算单元和Vector计算单元同核部署，共享同一个Scalar计算单元。

Vector计算单元

- Vector计算单元的数据来源来自于Unified Buffer，要求32字节对齐。

Cube计算单元

- Cube计算单元可以访问的存储单元有LOA Buffer、LOB Buffer、LOC Buffer，其中LOA Buffer存储左矩阵，LOB Buffer存储右矩阵，LOC Buffer存储矩阵乘的结果和中间结果。

存储单元

获取存储单元的内存空间大小

开发者可以通过平台信息获取接口查询各存储单元的内存空间大小。

各存储单元的最小访问粒度（对齐要求）

核	存储单元	对齐要求
AIV	Unified Buffer	32Byte对齐。
AIC	L1 Buffer	32Byte对齐。
	LOA Buffer	512Byte对齐。
	LOB Buffer	512Byte对齐。
	LOC Buffer	64Byte对齐。
	BiasTable Buffer	64Byte对齐。
	Fixpipe Buffer	64Byte对齐。

各存储单元推荐使用的数据排布格式

- LOA Buffer、LOB Buffer和LOC Buffer推荐分别采用以下分形格式：
 - LOA Buffer: FRACTAL_ZZ
 - LOB Buffer: FRACTAL_ZN
 - LOC Buffer: FRACTAL_NZ这些格式针对矩阵乘法等计算密集型任务进行优化，可显著提升计算效率。
- L1 Buffer缓存推荐使用FRACTAL_NZ格式。当L1 Buffer采用NZ格式时，数据搬运到LOA/LOB Buffer（需分别转换为ZZ和ZN格式）时，可降低格式转换开销。
- Unified Buffer对数据格式没有要求。

解决存储单元的访问冲突，提升读写性能

当多个操作尝试同时访问Unified Buffer同一个bank或者bank group时，可能会发生bank冲突，包括读写冲突、写写冲突、读读冲突，这种冲突会导致访问排队，降低性能。可以通过优化bank分配的方式来提升读写性能，具体信息请参考[3.6.5.9 避免 Unified Buffer的bank冲突](#)章节。

搬运单元

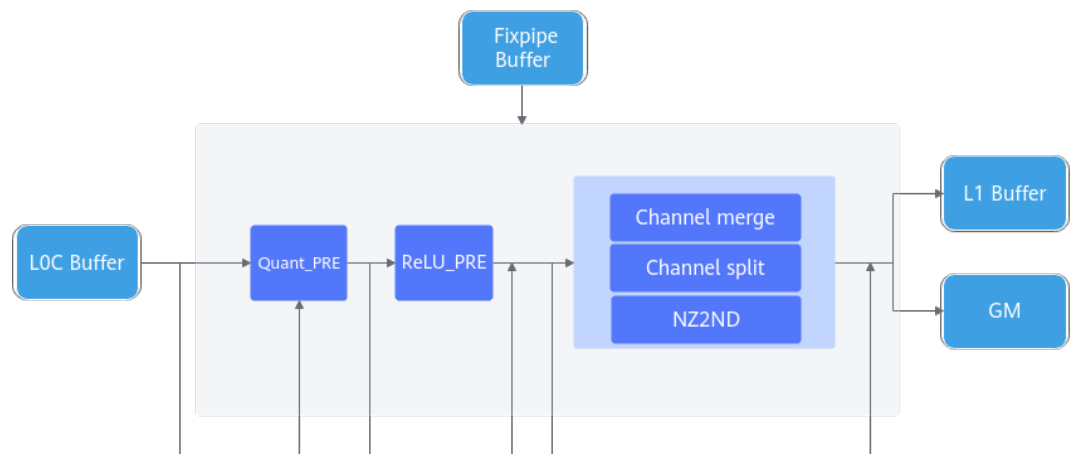
搬运时的对齐要求

由于搬运后的数据用于参与数据计算，因此对搬运数据大小有要求，搬运到Unified Buffer的数据大小需要按照DataBlock对齐，其余存储单元的数据搬运必须按[分形要求](#)进行搬运。例如，数据从L1 Buffer搬运到LOA Buffer时，数据格式需要从NZ转换为ZZ格式，搬运数据的大小要按分形大小对齐，如果L1 Buffer的剩余大小不足1个分形，则硬件执行中会出现异常。

支持Fixpipe硬件化加速

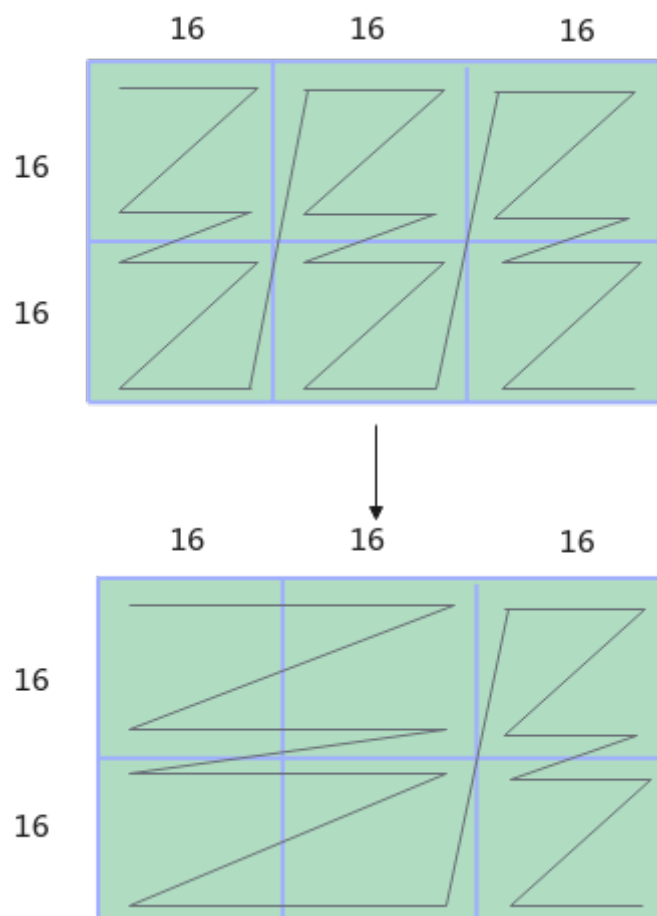
Fixpipe是NPU将典型操作进行硬化的加速模块，位于AIC内部，配合Cube计算单元完成随路计算，主要功能如下：

- 量化反量化：包括S322FP16、S322S32、S322S4、S322S8、S322S16、FP322FP16、FP322BF16、FP322S8、FP322S4、FP322FP32。
- Relu功能，包括ReLu、PReLu和Leaky ReLu等典型的激活函数。
- 数据格式转换，包括：
 - 通过Channel merge、Channel split可以实现分形大小的转换，保证输出到L1 Buffer/GM的分形满足诉求。
 - NZ2ND数据格式转换。



上图中，Channel merge支持S8、U8、S4和U4数据类型，而Channel split支持FP32数据类型。

- Channel merge（S8和U8数据类型）
对于转换为S8或U8的目标数据类型，分形矩阵通过硬件从16x16转换为16x32，如果输出通道数N是16的偶数倍，则N方向上每2个相邻的16x16分形矩阵将合并为1个16x32分形矩阵。如果N是16的奇数倍，则将通道1到通道（N-16）合并，最后16个通道保持未合并。
如下所示，目标数据类型为S8，M为32，N为48，首先将前2列16x16分形矩阵合并为一个16x32矩阵，然后将剩余的16x16分形矩阵直接移入L1 Buffer。

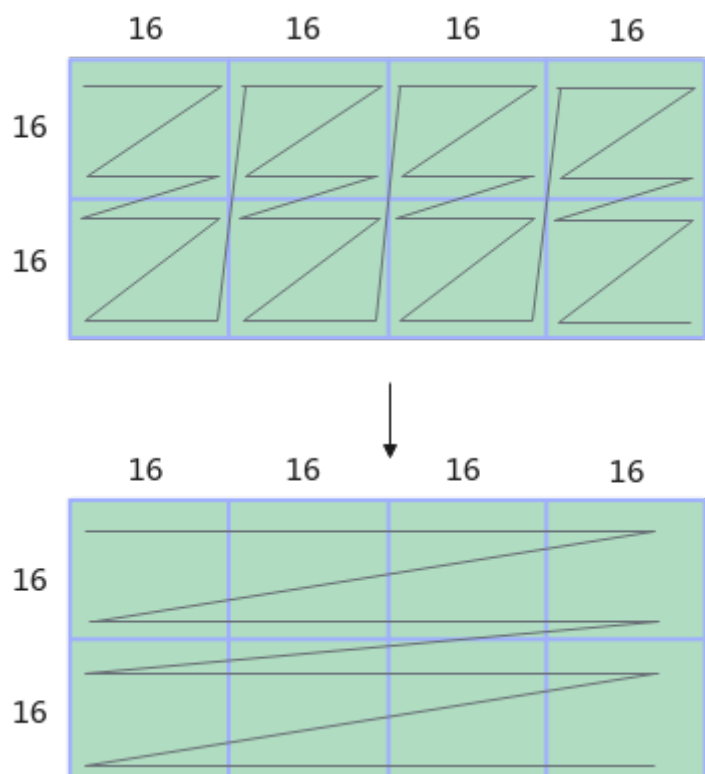


- Channel merge (S4和U4数据类型)

对于转换为S4或U4的目标数据类型，分形矩阵通过硬件从16x16转换为16x64，如果输出通道数N是64的倍数，则N方向上每4个相邻的16x16分形矩阵将合并为1个16x64分形矩阵。

例如，这里目标数据类型为S4，M为32，N为64，首先将第1行16x16分形矩阵合并为一个16x64矩阵，然后将第2行16x16分形矩阵也合并。

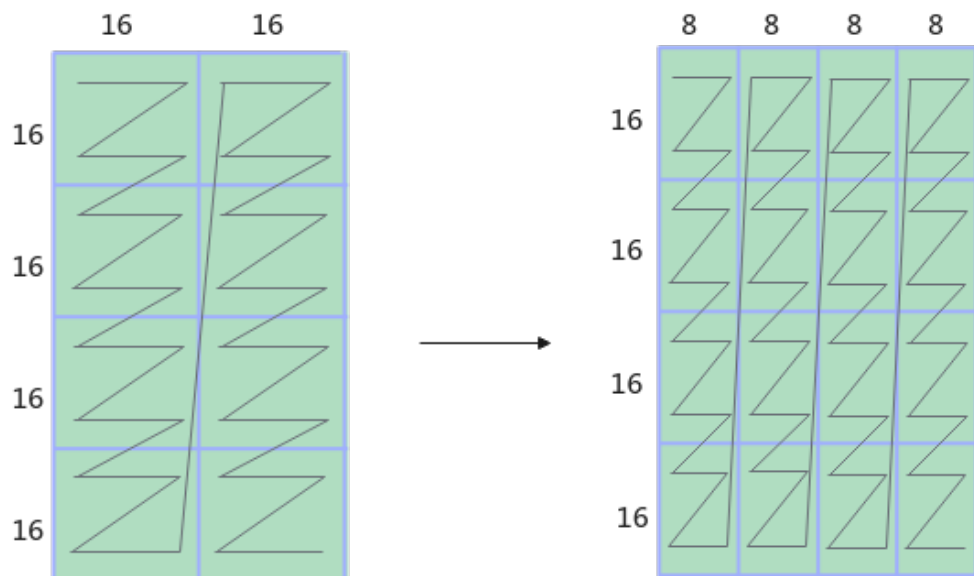
在这种情况下，N的配置必须是64的倍数。



- FP32 Channel split:

对于目标类型为FP32，分形矩阵可以通过硬件从16x16转换为16x8，如果使能Channel split，则每个16x16分形矩阵将被分裂为2个16x8分形矩阵。

如下图所示，这里的目标数据类型是FP32，M是64，N是32，它将被拆分为16个16x8的分形。



同步控制

- 核内同步

由于AI Core内部的执行单元（如MTE2搬运单元、Vector计算单元等）以异步并行的方式运行，在读写Local Memory（如Unified Buffer）时可能存在数据依赖关系。为确保数据一致性及计算正确性，需通过同步控制协调操作时序。

以MTE2从GM搬运数据至UB，进行Vector计算单元的Abs计算，再搬运回GM的流程为例，需满足以下同步条件：

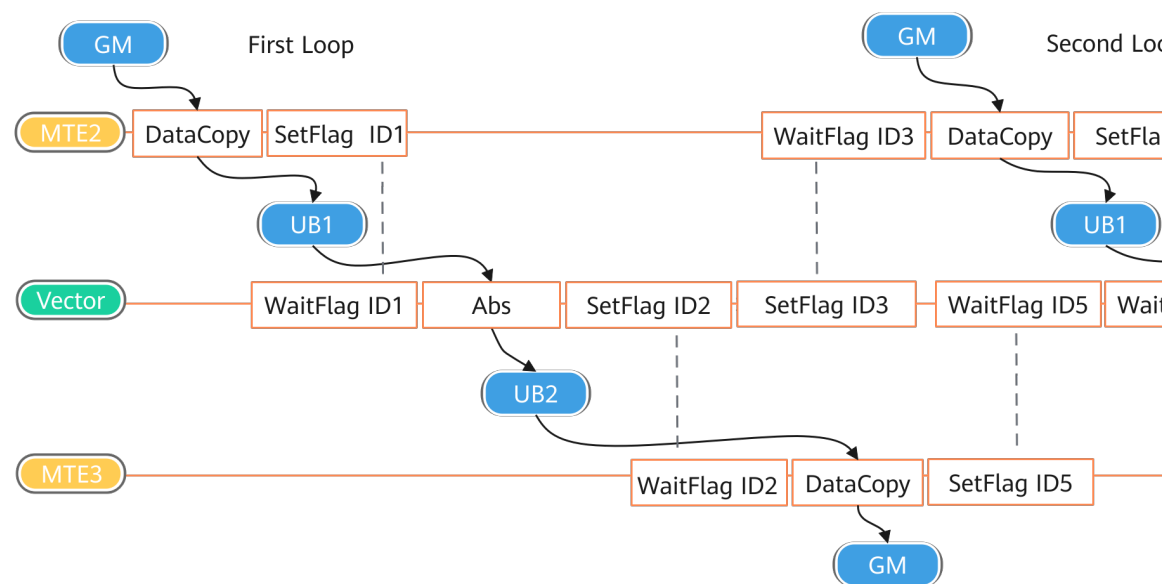
a. 数据搬运与计算顺序

- GM → UB搬运完成后启动Vector单元的Abs计算（避免计算时未完成搬运导致的数据缺失）；
- Vector计算完成后执行UB → GM的数据搬运（确保结果数据已就绪）。

b. 循环搬运计算场景的同步规则

- 前序计算完成后启动新搬运：上一次计算未完成时，不得触发新数据搬运（防止UB中旧数据被覆盖）；
- 前序数据搬出完成后启动新计算：上一次数据未完全从UB搬出时，不得触发新计算任务（避免目标内存区域的覆盖冲突）。

同步控制流程如下图所示：



上图中，ID1、ID2、ID3、ID4、ID5、ID6表示事件ID（EventID），每个EventID对应一块存储数据的搬运状态，确保数据操作的正确性和一致性。

需要注意以下几点：

- 建议通过AllocEventID或者FetchEventID接口获取EventID，以确保其合法性和有效性。
- EventID的数量有限，使用后应立即调用ReleaseEventID释放资源，避免EventID耗尽，影响系统正常运行。
- SetFlag和WaitFlag必须成对使用，且SetFlag和WaitFlag的参数必须完全一致（包括模板参数和事件ID）。如果不匹配，可能导致当前核的计算异常，或影响下一个核的算子执行，引发timeout问题。

例如，SetFlag<HardEvent::S_MTE3>(1)和

SetFlag<HardEvent::MTE3_MTE1>(1)设置的不是同一个EventID，因为其模板参数不同。只有当模板参数和事件ID完全一致时，才表示同一个EventID。

- 不允许连续设置同一个EventID，因为这可能导致事件状态混乱或未被正确处理。
- 不建议手动插入 TEventID，不能手动插入6和7的TEventID，因为它们可能被系统预留或用于特殊用途。
- 核间同步
该硬件架构不支持核间同步。

2.6.3 硬件约束

2.6.3.1 NPU 架构版本 200x

本节介绍硬件约束以及解决方案建议。对应的产品型号为：Atlas 推理系列产品。

- 全局变量使用约束

NPU架构版本200x不支持Generic Addressing通用寻址（针对UB、stack、GM等地址空间），因此语言层面地址空间必须匹配。不同的地址空间信息不允许转换，不符合语法。全局变量位于GM上，传参位于stack上，函数内传参使用全局变量时会报错。目前编译器仅在优化级别为O0的场景下，对constexpr做了适配处理：将全局变量先从DDR上放到stack上。所以全局变量仅支持在O0下使用constexpr进行定义和使用，在其他场景均不支持。

- 支持场景

在O0下支持constexpr全局变量小规模类型（整型、浮点型）数据的引用，支持数组取元素。

```
constexpr int a=1;
int *pa=&a;
// 数组
constexpr uint8_t padList[4] = {0, 1, 3, 5};
__aicore__ uint64_t Compute(uint32_t padNumber){
    uint64_t regFMatrix =3;
    for (uint32_t i = 0; i < padNumber; i++) {
        regFMatrix |= uint64_t(padList[i]);
    }
    return regFMatrix;
}
```

- 不支持场景

对于使用constexpr之外的全局参数将报错。

```
const float aa = 3.141592653589; // 不支持const 应修改为constexpr
template<typename T>
__global__ __aicore__ void hello_world3()
{
    AscendC::printf("global var is %f\n", aa);
};
```

2.6.3.2 NPU 架构版本 220x

本节介绍硬件约束以及解决方案建议。对应的产品型号为：

- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品

表 2-19 硬件约束以及解决方案建议

分类	硬件约束描述	解决方案建议
内存访问 (L0 Buffer/L 1 Buffer/U B等)	各存储单元的最小访问粒度/地址对齐要求： Unified Buffer: 32Byte对齐。 L1 Buffer: 32Byte对齐。 LOA Buffer/LOB Buffer: 512Byte对齐。 LOC Buffer: 64Byte对齐。 BiasTable Buffer: 64Byte对齐。 Fixpipe Buffer: 128Byte对齐。	- 进行数据搬运时，需要感知对齐约束。 - 针对UB，遇到一些非对齐的场景，可以使用非对齐搬运的接口或者通过一些技巧（比如搬入时包含冗余数据，搬出时去除冗余数据）来解决。详情见3.3.2.7 非对齐场景。
内存访问 (UB)	UB bank访问冲突 (Vector计算访问/搬运访问)。	需要按照芯片要求，在软件实现时错开处理的地址，从而解决bank冲突。具体解决方案可参考 3.6.5.9 避免Unified Buffer的bank冲突 章节。
内存访问 (GM)	多核并行同地址访问 GM，会被硬件串行化。	对相同地址的访问，会被硬件串行化，性能为排队时间，大约下降10%-20%； 多核访问通过错峰访问（调整数据访问顺序和修改切分策略等），使得第一次加载数据到L2 Cache，后续访问性能反而提升。具体解决方案可参考 3.6.5.4 避免同地址访问 章节。
内存访问 (GM)	单次搬运数据长度16KB以上时，可发挥带宽的最佳性能。	根据实测经验，单次搬运数据长度16KB以上时，通常能较好地发挥出带宽的最佳性能。因此对于单次搬运，应考虑尽可能的搬运较大的数据块（不同芯片不一样）。 具体解决方案可参考 3.6.5.1 尽量一次搬运较大的数据块 章节。
内存访问 (GM-->L1)	DataCopy中源操作数相邻连续数据块的间隔（前面一个数据块的尾与后面数据块的头的间隔）不要超出65535，单位为DataBlock（32字节）。	前面一个数据块的尾与后面数据块的头的间隔超出65535时，需要拆分成多条指令来实现。
内存访问 (GM)	数据搬运会被拆成128B/256B/512B不同长度进行搬运，非对齐会向上取整。	Tiling尽量让搬运的内轴128B、256B、512B对齐。 具体解决方案可参考 3.6.5.2 GM地址尽量512B对齐 章节。

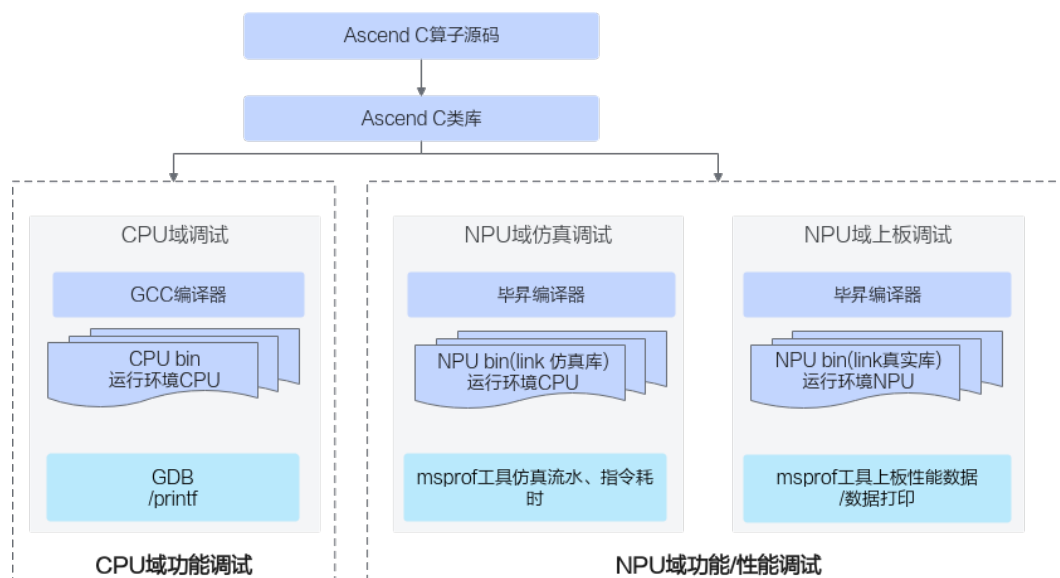
分类	硬件约束描述	解决方案建议
Cube	MTE1和MMAD指令队列的深度为32。	对应指令队列容易满，会阻塞其他指令下发，引起流水断流。 Load2D从L1 Buffer搬运到L0 Buffer，需要发射32条指令，Load3D只需要一条指令，建议使用Load3D来实现。
ICache	ICache硬件规格限制32KB。	拆分Tiling_key或使用模板函数来减少代码段。详情见 2.10.2.5.5 Tiling模板编程 。
ICache	多核并行同地址访问ICache，会被硬件串行化。	小Shape场景尽量减少启动核数，减少多核同地址访问问题。
DCache	DCache硬件规格限制32KB	无
Scalar	标量写GM时，数据会被缓存在Dcache。硬件不保证DCache和GM一致性，需要用户保证。	使用DataCacheCleanAndInvalid来保证一致性。
Cube	L0C Buffer容量128KB。	无
Cube	BiasTable Buffer为1KB。	无
Cube	Cube计算场景float算力为half算力的1/4。	无
Cube	Cube输出随路量化场景，不支持int32_t到bfloat16_t类型量化。	- AIV上存在int32_t->float、float->bfloat16_t的转换。 - AIC支持float->bfloat16_t的随路量化。
Vector	Reduce接口half比float性能差。	half写回UB时存在非32Byte对齐，导致性能劣化，建议把half转float计算，此场景建议使用float数据类型。
Vector	Exp/Ln接口处理同样数量的half/float的耗时是一样的	内部对float做了优化，故两者性能相当，开发者可以根据实际情况选择合适的精度类型。
流水同步（核内）	set/wait同步不匹配，状态会残留，影响后续算子。	通过孪生调试/mssanitizer工具，提前识别此类问题。
流水同步（核间）	CrossCoreSetFlag计数器存在限制，超出15次需要做一次反向同步，否则会出现卡死。	通过孪生调试/mssanitizer工具，超出场景提前报错。

分类	硬件约束描述	解决方案建议
API通用约束	使用Ascend C API时，源操作数和目的操作数的地址重叠通用约束	使用基础API的Tensor高维切分计算接口时，为了节省地址空间，开发者可以定义一个Tensor，供源操作数与目的操作数同时使用（即地址重叠）。使用时需要注意以下约束： • 单次迭代内：源操作数与目的操作数必须100%完全重叠，不支持部分重叠。 • 多次迭代间：不支持前序迭代的目的操作数与后序迭代的源操作数重叠。例如，第N次迭代的目的操作数是第N+1次的源操作数。在这种情况下，第N次迭代可能会改写覆盖源操作数的数值，导致无法得到预期结果。特别地，对于部分双目计算类的API（Add、Sub、Mul、Max、Min、AddRelu、SubRelu），当数据类型为half、int32_t、float时，支持前序迭代的目的操作数与后序迭代的源操作数重叠：仅针对目的操作数和第二个源操作数重叠的情况，且src1RepStride或者dstRepStride必须为0。

2.7 调试调优

2.7.1 概述

Ascend C算子调试的整体方案如下：开发者通过调用Ascend C类库编写Ascend C算子Kernel侧源码，Kernel侧源码通过通用的GCC编译器进行编译，编译生成通用的CPU域的二进制，可以通过gdb通用调试工具等调试手段进行调试；Kernel侧源码通过毕昇编译器进行编译，编译生成NPU域的二进制文件，可以通过仿真打点图或者Profiling工具进行上板数据采集等方式进行调试。



具体的调试调优方法和使用的工具列表如下：

表 2-20 调试调优方法和使用的工具列表

分类	子分类	方法
功能调试	CPU域孪生调试	孪生调试： 相同的算子代码可以在CPU域调试精度，NPU域调试性能。在CPU域可以进行gdb调试、使用printf命令打印。
	NPU域上板调试	printf/assert： printf主要用于打印标量和字符串信息；assert主要用于在代码中设置检查点，当某个条件不满足时，程序会立即终止并报错。
		DumpTensor： 使用DumpTensor接口打印指定Tensor的数据。
		上板调试工具： 使用msDebug工具调试NPU侧运行的算子程序，在真实的硬件环境中，对算子的输入输出进行测试，以验证算子的功能是否正确。具体功能包括断点设置、打印变量和内存、单步调试、中断运行等。
性能调优	-	msprof工具： msProf工具用于采集和分析运行在AI处理器上算子的关键性能指标，用户可根据输出的性能数据，快速定位算子的软、硬件性能瓶颈，提升算子性能的分析效率。 当前支持基于不同运行模式（上板或仿真）和不同文件形式（可执行文件或算子二进制.o文件）进行性能数据的采集和自动解析。

2.7.2 功能调试

2.7.2.1 CPU 域孪生调试

本节介绍CPU域调试的方法：CPU侧验证核函数，gdb调试、使用printf命令打印。

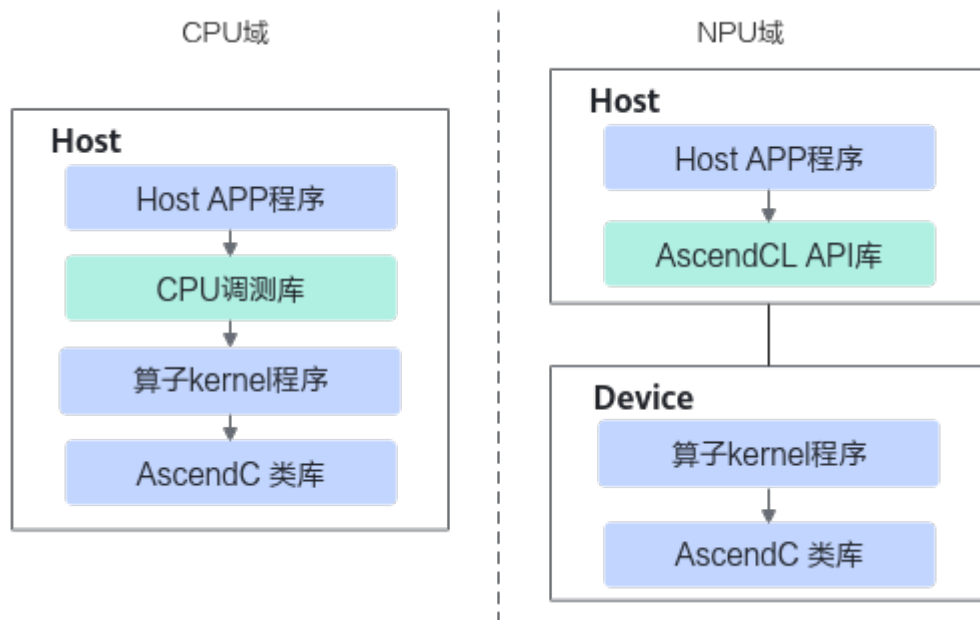
说明

- 当前您可以基于[Kernel直调样例工程](#)实现CPU域调试功能。
异构编译场景，开发者使用命令行或者编写Cmake文件进行编译的情况，暂不支持CPU孪生调试并将在后续的版本中逐步支持。
- CPU调测过程中，配置日志相关环境变量，可以记录程序的运行过程及异常信息，有助于开发者进行功能调测。
关于环境变量的使用约束以及详细说明，可参见日志。

CPU 侧验证核函数

在非昇腾设备上，开发者可以利用CPU仿真环境先行进行算子开发和测试，并在准备就绪后，利用昇腾设备进行加速计算。在[2.3 编译与运行](#)章节，我们已经介绍了算子Kernel程序NPU域的编译运行。相比于NPU域的算子运行逻辑，CPU域调试，实际上是通过标准的GCC编译器编译算子Kernel程序。此时算子Kernel程序链接CPU调测库，执行编译生成的可执行文件，可以完成算子CPU域的运行验证。CPU侧的运行程序，通过GDB通用调试工具进行单步调试，可以精准验证程序执行流程是否符合预期。

图 2-27 CPU 域和 NPU 域的核函数运行逻辑对比



基于Kernel直调样例工程，通过ACLRT_LAUNCH_KERNEL接口调用核函数时，可实现CPU与NPU域的代码的统一，且该方式仅支持以下型号：

- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品

- Atlas 推理系列产品

如果通过<<<>>>调用核函数，则需要使用单独的调测接口进行内存分配等操作，并对CPU域和NPU域的代码进行宏隔离。

下面代码以add_custom算子为例，介绍算子核函数在CPU侧验证时，算子调用的应用程序如何编写（通过ACLRT_LAUNCH_KERNEL接口调用核函数的方式）。您在实现自己的应用程序时，需要关注由于算子核函数不同带来的修改，包括算子核函数名，入参出参的不同等，合理安排相应的内存分配、内存拷贝和文件读写等，相关API的调用方式直接复用即可。

步骤1 按需包含头文件。

```
#include "data_utils.h"
#include "acl/acl.h"
#include "aclrtlaunch_add_custom.h"
```

步骤2 应用程序框架编写。

```
int32_t main(int32_t argc, char* argv[])
{
    uint32_t numBlocks = 8;
    size_t inputByteSize = 8 * 2048 * sizeof(uint16_t);
    size_t outputByteSize = 8 * 2048 * sizeof(uint16_t);
    // 运行算子的调用程序
    return 0;
}
```

步骤3 运行验证。

```
// 初始化
CHECK_ACL(aclInit(nullptr));
// 运行管理资源申请
int32_t deviceId = 0;
CHECK_ACL(aclrtSetDevice(deviceId));
aclrtStream stream = nullptr;
CHECK_ACL(aclrtCreateStream(&stream));
// 分配Host内存
uint8_t *xHost, *yHost, *zHost;
uint8_t *xDevice, *yDevice, *zDevice;

CHECK_ACL(aclrtMallocHost((void**)&xHost, inputByteSize));
CHECK_ACL(aclrtMallocHost((void**)&yHost, inputByteSize));
CHECK_ACL(aclrtMallocHost((void**)&zHost, outputByteSize));
// 分配Device内存
CHECK_ACL(aclrtMalloc((void**)&xDevice, inputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
CHECK_ACL(aclrtMalloc((void**)&yDevice, inputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
CHECK_ACL(aclrtMalloc((void**)&zDevice, outputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
// Host内存初始化
ReadFile("./input/input_x.bin", inputByteSize, xHost, inputByteSize);
ReadFile("./input/input_y.bin", inputByteSize, yHost, inputByteSize);
// 将数据从Host上拷贝到Device上
CHECK_ACL(aclrtMemcpy(xDevice, inputByteSize, xHost, inputByteSize,
ACL_MEMCPY_HOST_TO_DEVICE));
CHECK_ACL(aclrtMemcpy(yDevice, inputByteSize, yHost, inputByteSize,
ACL_MEMCPY_HOST_TO_DEVICE));
// 用ACLRT_LAUNCH_KERNEL接口调用核函数完成指定的运算
ACLRT_LAUNCH_KERNEL(add_custom)(numBlocks, stream, xDevice, yDevice, zDevice);
CHECK_ACL(aclrtSynchronizeStream(stream));
// 将Device上的运算结果拷贝回Host
CHECK_ACL(aclrtMemcpy(zHost, outputByteSize, zDevice, outputByteSize,
ACL_MEMCPY_DEVICE_TO_HOST));
WriteFile("./output/output_z.bin", zHost, outputByteSize);
// 释放申请的资源
CHECK_ACL(aclrtFree(xDevice));
CHECK_ACL(aclrtFree(yDevice));
CHECK_ACL(aclrtFree(zDevice));
CHECK_ACL(aclrtFreeHost(xHost));
CHECK_ACL(aclrtFreeHost(yHost));
```

```
CHECK_ACL(aclrtFreeHost(zHost));  
// 去初始化  
CHECK_ACL(aclrtDestroyStream(stream));  
CHECK_ACL(aclrtResetDevice(deviceId));  
CHECK_ACL(aclFinalize());
```

----结束

📖 说明

为了实现CPU域与NPU域代码归一，仅对部分acl接口进行适配，开发者在使用CPU域调测功能时，仅支持使用如下acl接口：

- 有实际功能接口，支持CPU域调用
 - aclDataTypeSize、aclFloat16ToFloat、aclFloatToFloat16。
 - aclrtMalloc、aclrtFree、aclrtMallocHost、aclrtFreeHost、aclrtMemset、aclrtMemsetAsync、aclrtMemcpy、aclrtMemcpyAsync、aclrtMemcpy2d、aclrtMemcpy2dAsync、aclrtCreateContext、aclrtDestroyContext。
- 无实际功能接口，打桩实现。
 - Profiling数据采集
aclprofInit、aclprofSetConfig、aclprofStart、aclprofStop、aclprofFinalize。
 - 系统配置
aclInit、aclFinalize、aclrtGetVersion。
 - 运行时管理
aclrtSetDevice、aclrtResetDevice、aclrtCreateStream、aclrtCreateStreamWithConfig、aclrtDestroyStream、aclrtDestroyStreamForce、aclrtSynchronizeStream、aclrtCreateContext、aclrtDestroyContext。

gdb 调试

可使用gdb单步调试算子计算精度。由于cpu调测已转为多进程调试，每个核都会拉起独立的子进程，故gdb需要转换成子进程调试的方式。针对Atlas 推理系列产品、Atlas 训练系列产品，每个核会拉起1个子进程。针对Atlas A2 训练系列产品/Atlas A2 推理系列产品，每个核会拉起3个子进程，1个Cube，2个Vector。

- 调试单独一个子进程

启动gdb，示例中的add_custom_cpu为CPU域的算子可执行文件，参考[修改并执行一键式编译运行脚本](#)，将一键式编译运行脚本中的run-mode设置成cpu，即可编译生成CPU域的算子可执行文件。

gdb启动后，首先设置跟踪子进程，之后再打断点，就会停留在子进程中，但是这种方式只会停留在遇到断点的第一个子进程中，其余子进程和主进程会继续执行直到退出。涉及到核间同步的算子无法使用这种方法进行调试。

```
gdb --args add_custom_cpu // 启动gdb，add_custom_cpu为算子可执行文件  
(gdb) set follow-fork-mode child
```

- 调试多个子进程

如果涉及到核间同步，那么需要能同时调试多个子进程。

在gdb启动后，首先设置调试模式为只调试一个进程，挂起其他进程。设置的命令如下：

```
(gdb) set detach-on-fork off
```

查看当前调试模式的命令为：

```
(gdb) show detach-on-fork
```

中断gdb程序要使用捕捉事件的方式，即gdb程序捕捉fork这一事件并中断。这样在每一次起子进程时就可以中断gdb程序。设置的命令为：

```
(gdb) catch fork
```

当执行r后，可以查看当前的进程信息：

```
(gdb) info inferiors
Num Description
* 1 process 19613
```

可以看到，当第一次执行fork的时候，程序断在了主进程fork的位置，子进程还未生成。

执行c后，再次查看info inferiors，可以看到此时第一个子进程已经启动。

```
(gdb) info inferiors
Num Description
* 1 process 19613
  2 process 19626
```

这个时候可以使用切换到第二个进程，也就是第一个子进程，再打上断点进行调试，此时主进程是暂停状态：

```
(gdb) inferior 2
[Switching to inferior 2 [process 19626] ($HOME/demo)]
(gdb) info inferiors
Num Description
  1 process 19613
* 2 process 19626
```

请注意，inferior后跟的数字是进程的序号，而不是进程号。

如果遇到同步阻塞，可以切换回主进程继续生成子进程，然后再切换到新的子进程进行调试，等到同步条件完成后，再切回第一个子进程继续执行。

如下是调试一个单独子进程的命令样例：

```
gdb --args add_custom_cpu
set follow-fork-mode child
break add_custom.cpp:45
run
list
backtrace
print i
break add_custom.cpp:56
continue
display xLocal
quit
```

使用 printf 打印命令打印

在代码中直接编写printf(...)来观察数值的输出。样例代码如下：

```
printf("xLocal size: %d\n", xLocal.GetSize());
printf("tileLength: %d\n", tileLength);
```

2.7.2.2 NPU 域上板调试

通过 DumpTensor、printf 打印进行调试

NPU域上板数据打印功能包括DumpTensor、printf两种，其中DumpTensor用于打印指定Tensor的数据，printf主要用于打印标量和字符串信息。

具体的使用方法如下：

在算子kernel侧实现代码中需要输出日志信息的地方调用DumpTensor接口或者printf接口打印相关内容。

- DumpTensor示例，srcLocal表示待打印的Tensor；5表示用户的自定义附加信息，比如当前的代码行号；dataLen表示元素个数。DumpTensor接口的使用说明和具体约束请参考DumpTensor。

```
DumpTensor(srcLocal,5, dataLen);
```

Dump时，每个block核的dump信息前会增加对应信息头DumpHead（32字节大小），用于记录核号和资源使用信息；每次Dump的Tensor数据前也会添加信息头DumpTensorHead（32字节大小），用于记录Tensor的相关信息。打印结果的示例如下：

```
DumpTensor: desc=5, addr=0, data_type=float16, position=UB, dump_size=32
[19.000000, 4.000000, 38.000000, 50.000000, 39.000000, 67.000000, 84.000000, 98.000000,
21.000000, 36.000000, 18.000000, 46.000000, 10.000000, 92.000000, 26.000000, 38.000000,
39.000000, 9.000000, 82.000000, 37.000000, 35.000000, 65.000000, 97.000000, 59.000000, 89.000000,
63.000000, 70.000000, 57.000000, 35.000000, 3.000000, 16.000000,
42.000000]
DumpTensor: desc=5, addr=100, data_type=float16, position=UB, dump_size=32
[6.000000, 34.000000, 52.000000, 38.000000, 73.000000, 38.000000, 35.000000, 14.000000,
67.000000, 62.000000, 30.000000, 49.000000, 86.000000, 37.000000, 84.000000, 18.000000,
38.000000, 18.000000, 44.000000, 21.000000, 86.000000, 99.000000, 13.000000, 79.000000,
84.000000, 9.000000, 48.000000, 74.000000, 52.000000, 99.000000, 80.000000,
53.000000]
...
DumpTensor: desc=5, addr=0, data_type=float16, position=UB, dump_size=32
[35.000000, 41.000000, 41.000000, 22.000000, 84.000000, 49.000000, 60.000000, 0.000000,
90.000000, 14.000000, 67.000000, 80.000000, 16.000000, 46.000000, 16.000000, 83.000000, 6.000000,
70.000000, 97.000000, 28.000000, 97.000000, 62.000000, 80.000000, 22.000000, 53.000000,
37.000000, 23.000000, 58.000000, 65.000000, 28.000000, 4.000000,
29.000000]
```

- printf示例如下，printf接口的使用说明和具体约束请参考printf。
`printf("fmt string %d", 0x123);`

说明

DumpTensor、printf接口打印功能会对算子实际运行的性能带来一定影响，通常在调测阶段使用。开发者可以按需关闭打印功能。具体方法请参考DumpTensor、printf。

使用 msSanitizer 工具进行异常检测

msSanitizer工具是基于AI处理器的异常检测工具，包含了单算子开发场景下的内存检测、竞争检测、未初始化检测和同步检测四个子功能。

- 内存检测：工具可以在用户开发算子的过程中，协助定位非法读写、多核踩踏、非对齐访问、内存泄漏以及非法释放等内存问题。同时工具也支持对CANN软件栈的内存检测，帮助用户定界软件栈内存异常发生的模块。
- 竞争检测：工具可以协助用户定位由于竞争风险可能导致的数据竞争问题，包含核内竞争和核间竞争问题。其中，核内竞争包含流水线竞争和流水内竞争。
- 未初始化检测：工具可以协助用户定位由于内存未初始化可能导致的脏数据读取问题。
- 同步检测：工具可以协助用户定位由于前序算子中的未配对同步指令导致的后续算子同步失败的问题。

具体使用方法请参考异常检测（msSanitizer）。

该功能仅在如下场景支持：

- 通过[2.10.7 基于样例工程完成Kernel直调](#)方式调用算子。
- 通过[单算子API调用](#)方式调用算子。
- 间接调用单算子API(aclnnxxx)接口：Pytorch框架单算子直调的场景。

使用 msDebug 工具进行算子调试

msDebug是一款面向昇腾设备的算子调试工具，用于调试NPU侧运行的算子程序，为算子开发人员提供调试手段。msDebug工具支持调试所有的昇腾算子，包含Ascend C

算子（Vector、Cube以及融合算子）程序。具体功能包括断点设置、打印变量和内存、单步调试、中断运行、核切换、检查程序状态、调试信息展示、解析Core dump文件，用户可根据实际情况进行选择。具体使用方法请参考算子调试（msDebug）。

- 通过[2.10.7 基于样例工程完成Kernel直调](#)方式调用算子。
- 通过[单算子API调用](#)方式调用算子。
- 间接调用单算子API(aclnnxxx)接口：Pytorch框架单算子直调的场景。

2.7.3 性能调优

NPU 模式下的性能采集与分析

基于NPU域算子的调用接口编写的算子程序，通过毕昇编译器编译后生成可执行程序，使用算子调优工具运行NPU模式下生成的可执行文件从而采集Ascend C算子在AI处理器上执行的性能数据，进行性能精细调优。

- Profiling性能数据采集：使用msProf工具采集Ascend C算子在AI处理器上执行的性能数据。
- Roofline瓶颈分析：通过msprof op生成的visualize_data.bin文件可通过MindStudio Insight进行可视化呈现，Roofline瓶颈分析图可构建出处理器的性能模型，然后利用该性能模型快速评估出算子的理论性能极限，协助开发者快速识别瓶颈类型。
- 指令流水图：通过msprof op simulator生成的visualize_data.bin文件或trace.json文件，并进行可视化呈现。指令流水图以指令维度展示时序关系，并关联调用栈快速定位瓶颈位置。

性能调优工具的具体使用方法请参考算子调优（msProf）。

2.8 兼容性指南

兼容性是确保开发者能够高效迁移算子的关键。为了保证在不同AI处理器之间迁移时，Ascend C算子的语义和功能保持一致，开发者需要根据目标平台的架构进行代码适配和优化。如果开发者希望在新平台上运行其他平台开发的Ascend C算子程序，需要在新平台上重新编译并运行。请参考[迁移指南](#)，了解如何将昇腾平台上开发的算子迁移到麒麟平台。

说明

为了保证兼容性，建议开发者在开发过程中尽量避免使用如下接口或编程方式，否则开发者需自行保证兼容性。

- 非Ascend C公开接口、结构体，例如impl目录下的接口、结构体等；
- 对芯片规格进行硬编码；例如：

```
constexpr uint32_t MAX_UB_SIZE = 256 * 1024;
// 建议获取方式：ascendcPlatform.GetCoreMemSize(platform_ascendc::CoreMemType::UB,
MAX_UB_SIZE);
...
```
- 编译器BuiltIn API；例如：

```
copy_gm_to_ubuf(input_1_local_UB, input_1, 0, 1, 8, 0, 0);
```
- TilingKey编程。

建议开发者使用如下接口或编程方式。

- 兼容的API；
- 若涉及特定领域扩展特性，在Device侧NPU_ARCH编译宏进行隔离，在Host侧用SocVersion进行隔离；
- 使用<<<>>>直调或Tiling模板编程。

2.9 概念原理和术语

2.9.1 术语表

表 2-21 术语表

术语/缩略语	含义
A1	AscendC::TPosition::A1代表设备上用于矩阵计算的逻辑内存，用于存放左矩阵，物理存储对应AI Core的L1 Buffer。
A2	AscendC::TPosition::A2代表设备上用于矩阵计算的逻辑内存，用于存放小块左矩阵（如经过分割、适配LOA Buffer容量的分块），物理存储对应AI Core的LOA Buffer。
AI Core	AI处理器的计算核，负责执行矩阵、向量计算密集的任务。
AIC	在AI Core分离模式下，一组Cube Core和Vector Core组合中的Cube Core。
AIV	在AI Core分离模式下，一组Cube Core和Vector Core组合中的Vector Core。
Ascend IR	Ascend Intermediate Representation，AI处理器专用的、用于表达计算流程的抽象数据结构。在本文档中，若无特殊说明，IR默认指代Ascend IR。
B1	AscendC::TPosition::B1代表设备上用于矩阵计算的逻辑内存，用于存放右矩阵，物理存储对应AI Core的L1 Buffer。
B2	AscendC::TPosition::B2代表设备上用于矩阵计算的逻辑内存，用于存放小块右矩阵（如经过分割、适配LOB Buffer容量的分块），物理存储对应AI Core的LOB Buffer。

术语/缩略语	含义
Block	Block在不同场景下具有多种含义，通常情况下指AI Core的逻辑核。典型场景有： - AI Core逻辑核：一个Block表示一个AI Core的逻辑核，其BlockID是以0为起始的逻辑编号。 - DataBlock：一个DataBlock表示一条NPU矢量计算指令处理的数据单元，大小通常为32字节，一条指令可同时处理多个DataBlock。 - 基本块：表示一次计算需要的典型数据块大小。
BlockID	以0为起始的AI Core逻辑编号，可以比实际硬件核数大。
numBlocks	参与计算的逻辑AI Core核数，在调用核函数时由开发者指定，其值一般等于或大于实际物理核数。
BiasTable Buffer	偏置存储，AI Core内部物理存储单元，通常用于存储矩阵计算所需的Bias（偏置）数据，与逻辑内存AscendC::TPosition::C2相对应。
Broadcast	广播，一种张量操作机制。通过广播，较小的张量可以自动扩展以匹配较大的张量的形状。
C1	AscendC::TPosition::C1代表设备上用于矩阵计算的逻辑内存，用于存放Bias（偏置）数据，物理存储对应AI Core的L1 Buffer或Unified Buffer。
C2	AscendC::TPosition::C2代表设备上用于矩阵计算的逻辑内存，用于存放小块Bias（偏置）数据（如经过分割、适配BT Buffer容量的分块），物理存储对应AI Core的BT Buffer或L0C Buffer。
C2PIPE2GM	AscendC::TPosition::C2PIPE2GM代表设备上用于矩阵计算的逻辑内存，用于存放量化参数，物理存储对应AI Core的Fixpipe Buffer。
Cache Line	缓存（DCache、ICache、L2 Cache）中的最小数据单位。
Core	拥有独立Scalar计算单元的计算核，Scalar计算单元承担了核内的指令发射等功能，也称之为核内的调度单元。
CO1	AscendC::TPosition::CO1代表设备上用于矩阵计算的逻辑内存，用于存放小块矩阵计算结果（如经过分割的矩阵计算结果分块），物理存储对应AI Core的L0C Buffer。
CO2	AscendC::TPosition::CO2代表设备上用于矩阵计算的逻辑内存，用于存放矩阵计算结果（如原始矩阵的最终计算结果），物理存储对应Global Memory或AI Core的Unified Buffer。
Compute	Ascend C算子编程范式中典型的三个阶段之一，负责完成计算任务。
CopyIn	Ascend C算子编程范式中典型的三个阶段之一，负责将待计算数据从Global Memory搬运到Local Memory。
CopyOut	Ascend C算子编程范式中典型的三个阶段之一，负责将计算结果从Local Memory搬运到Global Memory。

术语/缩略语	含义
Core ID	AI Core核的物理编号，与实际硬件核数一一对应。
Cube	AI Core上的Cube计算单元，负责执行矩阵运算。以float16数据类型为例，Cube每次执行可完成两个float16类型的16x16矩阵的乘法操作。
Cube Core	矩阵计算核，专注于矩阵计算。由Scalar调度单元、矩阵计算单元、搬运单元等组成，不包括矢量计算单元。
DataBlock	矢量计算指令处理的数据单元，大小通常为32字节，矢量计算指令执行一次，可同时处理多个DataBlock。
DataBlock Stride	矢量计算指令单次Repeat内DataBlock的间隔大小，即下次处理的起始数据地址与本次处理的起始数据地址之间的DataBlock个数。
DCache	Data Cache，数据缓存。 用于缓存Scalar计算单元近期可能被重复访问的数据段，以提升访问效率。
Device	Device指安装了昇腾AI处理器的硬件设备，利用PCIe接口与主机Host侧连接，为Host提供神经网络计算能力。若存在多个Device，多个Device之间的内存资源不能共享。
DMA	Direct Memory Access，直接内存访问单元。 负责数据搬运，包括Global Memory和Local Memory之间的数据搬运以及不同层级Local Memory之间的数据搬运，包含搬运单元MTE2、MTE3等。
DoubleBuffer/DB	双缓冲，并行领域常用的优化方式，通过创建多个持有数据的缓冲区（Buffer）提高数据处理的并行性。
Elementwise	元素级操作是对张量的每个元素独立进行的操作。每个元素的结果仅依赖于对应的输入元素。
Fixpipe	AI Core中负责将矩阵计算结果从L0C Buffer搬运到Global Memory或L1 Buffer的单元，搬运过程中随路完成量化、激活等操作。
Fixpipe Buffer	AI Core内部物理存储单元，通常用于存储Fixpipe搬运过程中所需的量化参数等数据，与逻辑内存AscendC::TPosition::C2PIPE2GM相对应。
Global Memory/GM	设备端的主内存，AI Core的外部存储，用于存储大规模数据，但需要优化访问模式以提升性能。
GlobalTensor	存放Global Memory全局数据的Tensor。
Host	指与设备端Device相连接的X86服务器、ARM服务器，会利用Device提供的NN（Neural-Network）计算能力，完成业务。
ICache	Instruction Cache，指令缓存。 用于缓存最近或频繁使用的指令。极致性能优化时，需要关注如何降低ICache Miss（指令缓存未命中）。

术语/缩略语	含义
InferShape	算子shape推导，仅在GE图模式时才使用。实际的网络模型生成过程中，会先进行Tensor shape以及datatype的推导。这样可以在图执行之前，就知道各Tensor的数据类型和形状，提前校验其正确性；同时提前推理出算子的输出张量描述，包括张量的形状、数据类型及数据排布格式等信息，算子构图准备阶段就可以为所有的张量静态分配内存，避免动态内存分配带来的开销。
Kernel	核函数，是Device设备上执行的并行函数。核函数通过__global__修饰，多个核并行执行相同的核函数，其主要区别是不同核函数运行时具有不同的BlockID。
Kernel Launch	将kernel程序提交至硬件进行启动执行的过程。
LOA Buffer	AI Core内部物理存储单元，通常用于存储矩阵计算的左矩阵，与逻辑内存AscendC::TPosition::A2相对应。
LOB Buffer	AI Core内部物理存储单元，通常用于存储矩阵计算的右矩阵，与逻辑内存AscendC::TPosition::B2相对应。
LOC Buffer	AI Core内部物理存储单元，通常用于存储矩阵计算的结果，与逻辑内存AscendC::TPosition::CO1相对应。
L1 Buffer	AI Core内部物理存储单元，空间相对较大，通常用于缓存矩阵计算的输入数据。矩阵计算的输入一般需要从GM搬运到L1 Buffer，然后分别搬运到LOA Buffer和LOB Buffer。L1 Buffer与逻辑内存AscendC::TPosition::A1、AscendC::TPosition::B1相对应。
L2 Cache	二级缓存，专门用于存储频繁访问的数据，以便减少对Global Memory的读写。
LCM	Local Cache Memory，AscendC::TPosition::LCM代表临时共享的Unified Buffer空间，与VECCALC实现同样的功能。
Local Memory	AI Core的内部存储，包括L1 Buffer、LOA Buffer、LOB Buffer、LOC Buffer、Unified Buffer等存储单元。
LocalTensor	存放AI Core中Local Memory本地数据的Tensor。
Mask	用于控制矢量计算指令每次Repeat内参与计算的元素，可通过连续模式和逐比特模式两种方式进行设置。
MTE1	Memory Transfer Engine 1，AI Core的数据传递引擎，负责将数据从L1 Buffer搬运到LOA Buffer或LOB Buffer等。注意：不同硬件能力可能有差异。
MTE2	Memory Transfer Engine 2，AI Core的数据传递引擎，负责将数据从GM搬运到L1 Buffer、LOA Buffer、LOB Buffer、Unified Buffer等。注意：不同硬件能力可能有差异。
MTE3	Memory Transfer Engine 3，AI Core的数据传递引擎，负责将数据从Unified Buffer搬运到Global Memory、L1 Buffer等。注意：不同硬件能力可能有差异。

术语/缩略语	含义
NC1HWC0	一种五维数据格式，其中C0与硬件架构强相关，采用该格式可提升矩阵乘法的计算效率。
NCHW	按照[Batch, Channels, Height, Width]的排列顺序存储特征图数据。
ND	普通格式，N维张量。
NHWC	按照[Batch, Height, Width, Channels]的排列顺序存储特征图数据。
NPU	Neural-Network Processing Unit，神经网络处理器单元。采用“数据驱动并行计算”的架构，专门用于处理人工智能应用中的大量计算任务。
OP	算子（Operator，简称OP），是深度学习算法中执行特定数学运算或操作的基础单元，例如激活函数（如ReLU）、卷积（Conv）、池化（Pooling）以及归一化（如Softmax）。通过组合这些算子，可以构建神经网络模型。
OpType	算子类型，一类算子的统称。例如，在网络中可能会出现多个Add算子，名称分别为Add1、Add2，但这类算子的OpType均为Add。
Pipe	Ascend C编程范式核心概念之一，用于统一管理Device端内存等资源，一个Kernel函数必须且只能初始化一个Pipe对象。
Preload	在计算任务开始前，预先将必要的指令或数据加载到缓存中，用于减少指令或数据访问的延迟，提高计算效率。
Reduce	减维操作，用于减少多维张量的维度。常见的减维操作包括求和、求平均、求最大值、求最小值等。
Repeat	矢量计算指令执行一次，读取8个DataBlock数据进行计算，称之为一个迭代（Repeat）。通常情况下，需要循环执行多次才能完成所有数据的读取与计算。
Repeat Stride	矢量计算指令循环执行时，下一次Repeat起始数据地址与当前Repeat起始数据地址之间的DataBlock个数。
Repeat Times	矢量计算指令循环执行的次数。
Scalar	AI Core上的标量计算单元，主要负责标量数据运算和对其他单元（如MTE数据搬运单元、Vector矢量计算单元、Cube矩阵计算单元）的指令发射。
SIMD	SIMD: Single Instruction, Multiple Data，一条指令同时对多个数据进行相同的操作。
SPMD	Single-Program Multiple-Data，一种并行程序设计模型，其主要思想是使用同一个程序在多个核上并行执行，但每个核处理不同数据。

术语/缩略语	含义
SuperKernel	SuperKernel是一种算子的二进制融合技术，与源码融合不同，它聚焦于内核函数 (Kernel) 的二进制的调度方案，展开深度优化，于已编译的二进制代码基础上融合创建一个超级Kernel函数 (SuperKernel)，以调用子函数的方式调用多个其他内核函数，也就是子Kernel。相对于单算子下发，SuperKernel技术可以减少任务调度等待时间和调度开销，同时利用Task间隙资源进一步优化算子头开销。
Tensor	Tensor张量是算子计算数据的容器，是N维数据结构，最常见的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
Tiling	Tiling指数据的切分和分块。计算数据量较大时，需要将数据进行多核切分、每个核也需要分块多次计算。
TilingData	TilingData指数据切分和分块的相关参数（如每次搬运的块大小、循环次数）。鉴于设备端Scalar计算能力限制，一般Tiling参数在Host侧计算完成，然后传输到设备侧供Kernel函数使用。
TilingFunc	算子工程提供的在Host侧计算Tiling的默认函数。
TilingKey	用来区分Kernel函数不同版本的特例实现，不同的TilingKey会编译生成不同二进制。
TPosition	Ascend C管理不同层级的物理内存时，用一种抽象的逻辑位置 (TPosition) 来表达各级别的存储，代替了片上物理存储的概念，达到隐藏硬件架构的目的。TPosition类型包括：VEGIN、VEGOUT、VECCALC、A1、A2、B1、B2、CO1、CO2等，其中VEGIN、VECCALC、VEGOUT主要用于向量编程，A1、A2、B1、B2、CO1、CO2用于矩阵编程。
TSCM	AscendC::TPosition::TSCM表示L1 Buffer空间对应的逻辑内存，需开发者自行管理以高效利用硬件资源，主要用于Matmul计算。比如，开发者可缓存一份TSCM数据，在不同使用场景中灵活配置为Matmul操作的A矩阵、B矩阵或Bias偏置矩阵，实现内存复用与计算效率优化。
Unified Buffer/UB	AI Core内部存储单元，主要用于向量计算，与逻辑内存AscendC::TPosition::VEGIN、AscendC::TPosition::VEGOUT、AscendC::TPosition::VECCALC相对应。
VECCALC	Vector Calculation，AscendC::TPosition::VECCALC代表设备上用于向量计算的逻辑内存，用于存放临时变量，物理存储对应AI Core的Unified Buffer。
VEGIN	Vector Input，AscendC::TPosition::VEGIN代表设备上用于向量计算的逻辑内存，用于存放向量计算的输入数据，物理存储对应AI Core的Unified Buffer。
VEGOUT	Vector Output，AscendC::TPosition::VEGOUT代表设备上用于向量计算的逻辑内存，用于存放向量计算的输出数据，物理存储对应AI Core的Unified Buffer。

术语/缩略语	含义
Vector	AI Core上的Vector计算单元，负责执行矢量运算。其算力低于Cube，但灵活度高于Cube（如支持数学中的求倒数，求平方根等）。
Vector Core	矢量计算核，专注于矢量计算，由Scalar调度单元、矢量计算单元、搬运单元等组成，不包括矩阵计算单元。
Workspace	通常情况下指一个预分配的、临时使用的Global Memory内存，用于存储中间结果或临时数据。
CPU域调试	Ascend C提供的一种孪生调试方法，在CPU上模拟设备侧Kernel函数的执行和调试，仅调试算子功能和精度。
基本块	一次计算需要的典型数据块大小。
静态Tensor编程	静态Tensor编程方式，相比基于Pipe的编程方式，这种方式避免了TPipe内存管理初始化过程（约数百纳秒），从而减少了运行时开销，更有助于开发者实现极致性能。通过直接构造指定地址和存储位置的LocalTensor，并将其传递给计算、搬运等API进行编程，提供了更高的灵活性。
Kernel直调	一种简单直接的Kernel调用方式。 完成Kernel侧算子实现和Host侧Tiling实现后，即可通过运行时接口，完成算子Kernel直调。该方式下Tiling开发不受CANN框架的限制，简单直接，多用于算子功能的快速验证。
NPU域调试	Ascend C提供的一种孪生调试方法，指基于NPU仿真软件或NPU硬件调试。
Tiling下沉	Tiling下沉是指将Tiling计算下沉至Device侧的AI CPU上执行，从而实现计算全程在Device侧高效完成。
分离模式	AI Core的一种工作模式，其矩阵计算单元和矢量计算单元分别由独立的Scalar调度单元进行调度，并分离部署在Cube Core和Vector Core上。将Cube Core和Vector Core按照一定比例（1:N）进行组合，这样的组合视为一个AI Core，AI Core的核数以Cube Core为准。
孪生调试	Ascend C提供的算子调试方法，支持在CPU域调试精度和NPU域调试精度/性能。
流水任务	Ascend C编程范式是一种流水线式的编程范式，把算子核内的处理程序，分成多个流水任务。流水任务是指单核处理程序中主程序调度的并行任务。在核函数内部，可以通过流水任务实现数据的并行处理，进一步提升性能。
连续模式	使用Mask控制矢量计算每次Repeat内参与计算的元素时，可选择的模式之一，表示前面连续的多少个元素参与计算。
耦合模式	AI Core的一种工作模式，采用同一个Scalar调度单元同时调度矩阵计算单元、矢量计算单元，所有的单元部署在一个AI Core上。

术语/缩略语	含义
融合算子	融合算子由多个独立的小算子融合而成，其功能与多个小算子的功能等价，性能方面通常优于独立的小算子。用户可以根据实际业务场景诉求，按照具体算法自由融合矢量（Vector）、矩阵（Cube）算子以达到性能上的收益。
算子入图	算子入图指通过GE图模式运行算子，在图模式下首先将所有算子构造成一张图，然后通过GE将图下发到AI处理器执行。
算子原型	算子原型是算子的抽象描述，定义了算子的输入、输出、属性等信息。
通算融合算子	通算融合算子是融合集合通信任务和计算任务的算子，在算子执行过程中，计算和通信任务可以实现部分流水并行，从而提升性能。
逐比特模式	使用Mask控制矢量计算每次Repeat内参与计算的元素时，可选择的模式之一，可以按位控制哪些元素参与计算，bit位的值为1表示参与计算，0表示不参与。
自定义算子工程	Ascend C提供的基于msOpGen工具生成的算子工程。

2.9.2 神经网络和算子

2.9.2.1 算子基本概念

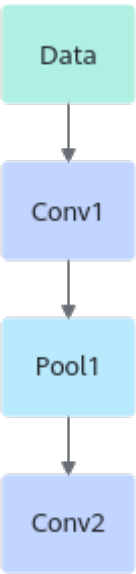
算子（Operator，简称OP），是深度学习算法中执行特定数学运算或操作的基础单元，例如激活函数（如ReLU）、卷积（Conv）、池化（Pooling）以及归一化（如Softmax）。通过组合这些算子，可以构建神经网络模型。

本章节介绍算子中常用的基本概念。

算子名称（Op Name）

算子的名称，用于标识网络中的某个算子，同一网络中算子的名称需要保持唯一。如下图所示Conv1、Pool1、Conv2都是此网络中的算子名称，其中Conv1与Conv2算子的类型为Convolution，表示分别做一次卷积运算。

图 2-28 网络拓扑示例



算子类型（Op Type）

网络中每一个算子根据算子类型进行算子实现的匹配，相同类型算子的实现逻辑相同。在一个网络中同一类型的算子可能存在多个，例如上图中的Conv1算子与Conv2算子的类型都为Convolution。

张量（Tensor）

Tensor是算子计算数据的容器，包含如下属性信息。

表 2-22 Tensor 属性信息

属性	定义
形状	Tensor的形状，比如(10,)或者(1024, 1024)或者(2, 3, 4)等。如形状(3, 4)表示第一维有3个元素，第二维有4个元素，(3, 4)表示一个3行4列的矩阵数组。 形式：(i1, i2, ..., in)，其中i1到in均为正整数。
数据类型	指定Tensor对象的数据类型。 取值范围：float16, float32, int8, int16, int32, uint8, uint16, bfloat16, bool等。
数据排布格式	数据的物理排布格式，详细请参见 2.9.2.2 数据排布格式 。

形状（Shape）

张量的形状，以(D0, D1, ..., Dn-1)的形式表示，D0到Dn是任意的正整数。

如形状(3,4)表示第一维有3个元素，第二维有4个元素，(3,4)表示一个3行4列的矩阵数组。

形状的第一个元素对应张量最外层中括号中的元素个数，形状的第二个元素对应张量中从左边开始数第二个中括号中的元素个数，依此类推。例如：

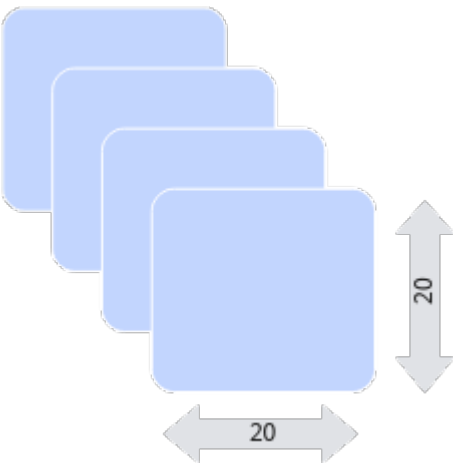
表 2-23 张量的形状举例

张量	形状	描述
1	(0,)	0维张量，也是一个标量
[1,2,3]	(3,)	1维张量
[[1,2],[3,4]]	(2, 2)	2维张量
[[[1,2],[3,4]], [[5,6],[7,8]]]	(2, 2, 2)	3维张量

物理含义我们应该怎么理解呢？假设我们有这样一个shape=(4, 20, 20, 3)。

假设有一些照片，每个像素点都由红/绿/蓝3色组成，即shape里面3的含义，照片的宽和高都是20，也就是20*20=400个像素，总共有4张照片，这就是shape=(4, 20, 20, 3)的物理含义。

图 2-29 示意图



如果体现在编程上，可以简单把shape理解为操作Tensor的各层循环，比如我们要对shape=(4, 20, 20, 3)的A tensor进行操作，循环语句如下：

```
produce A {
  for (i, 0, 4) {
    for (j, 0, 20) {
      for (p, 0, 20) {
        for (q, 0, 3) {
          A[((((i*20) + j)*20) + p)*3 + q] = a_tensor[((((i*20) + j)*20) + p)*3 + q]
        }
      }
    }
  }
}
```

轴 (axis)

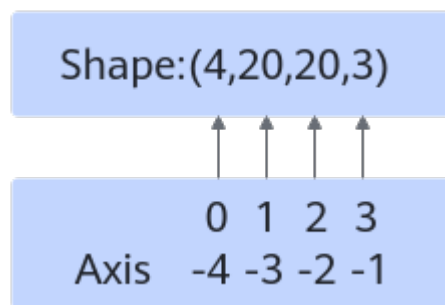
轴是相对shape来说的，轴代表张量的shape的下标，比如张量a是一个5行6列的二维数组，即shape是(5,6)，则axis=0表示是张量中的第一维，即行。axis=1表示是张量中的第二维，即列。

例如张量数据[[[1,2],[3,4]], [[5,6],[7,8]]]，Shape为(2,2,2)，则轴0代表第一个维度的数据即[[1,2],[3,4]]与[[5,6],[7,8]]这两个矩阵，轴1代表第二个维度的数据即[1,2]、[3,4]、[5,6]、[7,8]这四个数组，轴2代表第三个维度的数据即1, 2, 3, 4, 5, 6, 7, 8这八个数。

轴axis可以为负数，此时表示是倒数第axis个维度。

N维Tensor的轴有：0, 1, 2, …, N-1。

图 2-30 轴示意图



2.9.2.2 数据排布格式

数据排布格式 (Data Layout Format) 是深度学习中对多维Tensor在内存中存储方式的描述。

常见的数据格式包括ND、NHWC和NCHW等，为Tensor的每个轴赋予了特定的业务语义。

除了上述NHWC和NCHW格式外，还存在一些特殊的私有数据格式，如FRACTAL_NZ (也简称NZ)、NC1HWC0、FRACTAL_Z、NDC1HWC0、FRACTAL_Z_3D等。这些格式的引入是为了满足AI Core中Cube计算单元的高性能计算需求，通过优化内存布局，这些格式能够提升计算效率。在使用矩阵乘、卷积API开发相关算子的过程中，您可以看到这些格式的具体应用。

普通格式

- **ND、NHWC和NCHW**

数据排布格式最初用于表示图像在内存中的存储方式，其中常见的包括ND、NHWC和NCHW。在一般情况下，所有的Tensor都是N维的 (ND)，而NHWC和NCHW则是为四维Tensor中的每个轴赋予了特定的业务语义，例如高度 (Height)、宽度 (Width) 和通道数 (Channels)。

NHWC和NCHW的主要区别在于通道 (Channel) 维度的位置：

- NHWC格式中，通道维度位于最后一个位置。
- NCHW格式中，通道维度位于高度和宽度之前。

具体解释每个轴的含义：

- **N**：Batch数量，表示图像的数目。

- **H**: Height, 图像的高度, 即垂直方向的像素个数。
- **W**: Width, 图像的宽度, 即水平方向的像素个数。
- **C**: Channels, 图像的通道数, 例如彩色RGB图像的Channels为3。

如图2-31所示, 以一张格式为RGB的图片为例, NCHW中, C排列在外层, 实际存储的是“RRRRRRGGGGGGBBBBBB”, 即同一通道的所有像素值顺序存储在一起; 而NHWC中C排列在最内层, 实际存储的则是“RGBRGBRGBRGBRGB”, 即多个通道的同一位置的像素值顺序存储在一起。

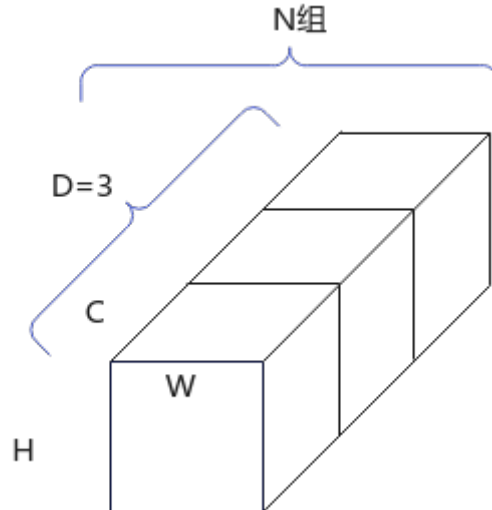
图 2-31 NCHW 和 NHWC 存储示例



尽管存储的数据相同, 但不同的存储顺序会导致数据的访问特性不一致, 因此即便进行同样的运算, 相应的计算性能也会不同。

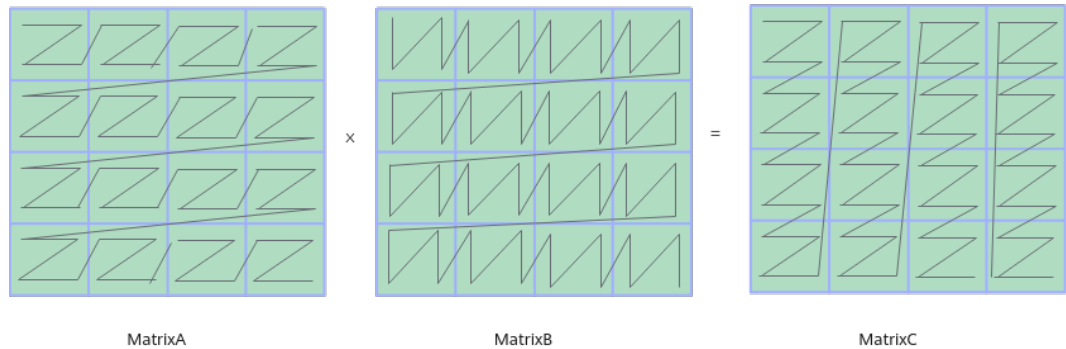
- **NDHWC和NCDHW**

NDHWC和NCDHW是五维Tensor, 较NHWC和NCHW多了一个D的维度, D代表特征深度 (Depth), 表示数据在深度方向上的扩展, 如视频的时间步或医学图像的深层, 因此该类格式便于在时间维度上进行卷积操作。以NDHWC为例, 其数据格式如下图所示:



矩阵乘相关特殊格式

使用Mmad基础API进行矩阵乘计算时, 对矩阵输入输出的数据排布格式有一定的要求, 如下图所示, 要求A矩阵 (位于LOA Buffer) 为FRACTAL_ZZ, B矩阵 (位于LOB Buffer) 为FRACTAL_ZN, C矩阵 (位于LOC Buffer) 为FRACTAL_NZ。这些格式将矩阵划分成了一些分形 (Fractal Matrix), 适配Cube计算单元每次读取(16, 16)×(16, 16)的数据进行计算的硬件特点 (以half数据类型为例), 从而提高矩阵计算的效率。分形的大小和数据类型有关, 也和所在的存储位置有关, 具体可参见下文的详细介绍。



- **FRCTAL_NZ/NZ**

FRCTAL_NZ格式，简称NZ格式，是对一个Tensor最低两维（一个Tensor的所有维度，右侧为低维，左侧为高维）进行填充（pad）、拆分（reshape）和转置（transpose）操作后得到的格式。具体的转换过程如下：

(M, N)大小的矩阵被分为 $M1 * N1$ 个分形，按照column major（列优先）排布，形状如N字形；每个分形内部有 $M0 * N0$ 个元素，按照row major（行优先）排布，形状如Z字形，所以这种数据格式称为NZ格式。其中，(M0, N0)表示一个分形的大小。

通过公式表达为：

$(\dots, B, M, N) \rightarrow \text{pad} \rightarrow (\dots, B, M1 * M0, N1 * N0) \rightarrow \text{reshape} \rightarrow (\dots, B, M1, M0, N1, N0) \rightarrow \text{transpose} \rightarrow (\dots, B, N1, M1, M0, N0)$

📖 说明

通常情况下，NZ格式在L0C Buffer和L1 Buffer中分别用于不同的场景：

- 在L0C Buffer中，NZ格式用于存储矩阵乘法的结果。其分形形状为16x16，包含256个元素，这种结构非常适合Cube计算单元进行高效的矩阵乘法运算。
- 在L1 Buffer中，NZ格式被采用以便于将数据搬运到LOA Buffer和LOB Buffer时，能够方便地转换为对应的ZZ和ZN格式。此时，分形形状为16 x (32B / sizeof(Datatype))，大小为512字节。

因此，当数据从L0C Buffer搬运到L1 Buffer时，其分形大小可能会发生变化。

下面通过一个具体的例子来了解ND格式转换为NZ格式的过程。

原始Tensor的Shape为(20, 28)：

```
data = [x for x in range(20 * 28)]
data_a = data * np.ones((20 * 28), dtype="float16")
tensor_a = data_a.reshape((20, 28))
print(tensor_a)
```

原始Tensor数据打印如下：

```
[[ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11. 12. 13.
 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27.]
 [28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41.
 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55.]
 [56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69.
 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83.]
 [84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97.
 98. 99. 100. 101. 102. 103. 104. 105. 106. 107. 108. 109. 110. 111.]
 [112. 113. 114. 115. 116. 117. 118. 119. 120. 121. 122. 123. 124. 125.
 126. 127. 128. 129. 130. 131. 132. 133. 134. 135. 136. 137. 138. 139.]
 [140. 141. 142. 143. 144. 145. 146. 147. 148. 149. 150. 151. 152. 153.
 154. 155. 156. 157. 158. 159. 160. 161. 162. 163. 164. 165. 166. 167.]
 [168. 169. 170. 171. 172. 173. 174. 175. 176. 177. 178. 179. 180. 181.
 182. 183. 184. 185. 186. 187. 188. 189. 190. 191. 192. 193. 194. 195.]
 [196. 197. 198. 199. 200. 201. 202. 203. 204. 205. 206. 207. 208. 209.
 210. 211. 212. 213. 214. 215. 216. 217. 218. 219. 220. 221. 222. 223.]
 [224. 225. 226. 227. 228. 229. 230. 231. 232. 233. 234. 235. 236. 237.]
```

```
238. 239. 240. 241. 242. 243. 244. 245. 246. 247. 248. 249. 250. 251.]  
[252. 253. 254. 255. 256. 257. 258. 259. 260. 261. 262. 263. 264. 265.  
266. 267. 268. 269. 270. 271. 272. 273. 274. 275. 276. 277. 278. 279.]  
[280. 281. 282. 283. 284. 285. 286. 287. 288. 289. 290. 291. 292. 293.  
294. 295. 296. 297. 298. 299. 300. 301. 302. 303. 304. 305. 306. 307.]  
[308. 309. 310. 311. 312. 313. 314. 315. 316. 317. 318. 319. 320. 321.  
322. 323. 324. 325. 326. 327. 328. 329. 330. 331. 332. 333. 334. 335.]  
[336. 337. 338. 339. 340. 341. 342. 343. 344. 345. 346. 347. 348. 349.  
350. 351. 352. 353. 354. 355. 356. 357. 358. 359. 360. 361. 362. 363.]  
[364. 365. 366. 367. 368. 369. 370. 371. 372. 373. 374. 375. 376. 377.  
378. 379. 380. 381. 382. 383. 384. 385. 386. 387. 388. 389. 390. 391.]  
[392. 393. 394. 395. 396. 397. 398. 399. 400. 401. 402. 403. 404. 405.  
406. 407. 408. 409. 410. 411. 412. 413. 414. 415. 416. 417. 418. 419.]  
[420. 421. 422. 423. 424. 425. 426. 427. 428. 429. 430. 431. 432. 433.  
434. 435. 436. 437. 438. 439. 440. 441. 442. 443. 444. 445. 446. 447.]  
[448. 449. 450. 451. 452. 453. 454. 455. 456. 457. 458. 459. 460. 461.  
462. 463. 464. 465. 466. 467. 468. 469. 470. 471. 472. 473. 474. 475.]  
[476. 477. 478. 479. 480. 481. 482. 483. 484. 485. 486. 487. 488. 489.  
490. 491. 492. 493. 494. 495. 496. 497. 498. 499. 500. 501. 502. 503.]  
[504. 505. 506. 507. 508. 509. 510. 511. 512. 513. 514. 515. 516. 517.  
518. 519. 520. 521. 522. 523. 524. 525. 526. 527. 528. 529. 530. 531.]  
[532. 533. 534. 535. 536. 537. 538. 539. 540. 541. 542. 543. 544. 545.  
546. 547. 548. 549. 550. 551. 552. 553. 554. 555. 556. 557. 558. 559.]
```

转换过程通过伪代码表达如下：

```
N0 = 16  
N1 = (28 + N0 - 1) // N0  
pad_n = N1 * N0 - 28  
M0 = 16  
M1 = (20 + M0 - 1) // M0  
pad_m = M1 * M0 - 20  
tensor_b = np.pad(tensor_a, [[0, pad_m], [0, pad_n]])  
tensor_b = tensor_b.reshape((M1, M0, N1, N0))  
tensor_b = tensor_b.transpose((2, 0, 1, 3))  
print(tensor_b)
```

转换过程示意图如下：

0	1	2	3	4	...	11	12	13	14	15	16	17	18	19	20	...
28	29	30	31	32	...	39	40	41	42	43	44	45	46	47	48	...
56	57	58	59	60	...	67	68	69	70	71	72	73	74	75	76	...
84	85	86	87	88	...	95	96	97	98	99	100	101	102	103	104	...
112	113	114	115	116	...	123	124	125	126	127	128	129	130	131	132	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
308	309	310	311	312	...	319	320	321	322	323	324	325	326	327	328	...
336	337	338	339	340	...	347	348	349	350	351	352	353	354	355	356	...
364	365	366	367	368	...	375	376	377	378	379	380	381	382	383	384	...
392	393	394	395	396	...	403	404	405	406	407	408	409	410	411	412	...
420	421	422	423	424	...	431	432	433	434	435	436	437	438	439	440	...
448	449	450	451	452	...	459	460	461	462	463	464	465	466	467	468	...
476	477	478	479	480	...	487	488	489	490	491	492	493	494	495	496	...
504	505	506	507	508	...	515	516	517	518	519	520	521	522	523	524	...
532	533	534	535	536	...	543	544	545	546	547	548	549	550	551	552	...
0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	...
0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	...
0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	...
0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	...
0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	...
0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	...

转换后Tensor打印如下：

```

[[[ 0.  1.  2. ... 13. 14. 15.]
 [ 28. 29. 30. ... 41. 42. 43.]
 [ 56. 57. 58. ... 69. 70. 71.]
 ...
 [364. 365. 366. ... 377. 378. 379.]
 [392. 393. 394. ... 405. 406. 407.]
 [420. 421. 422. ... 433. 434. 435.]]

[[448. 449. 450. ... 461. 462. 463.]
 [476. 477. 478. ... 489. 490. 491.]
 [504. 505. 506. ... 517. 518. 519.]
 ...
 [ 0.  0.  0. ...  0.  0.  0.]
 [ 0.  0.  0. ...  0.  0.  0.]
 [ 0.  0.  0. ...  0.  0.  0.]]

[[[ 16. 17. 18. ...  0.  0.  0.]
 [ 44. 45. 46. ...  0.  0.  0.]
 [ 72. 73. 74. ...  0.  0.  0.]
 ...
 [380. 381. 382. ...  0.  0.  0.]
 [408. 409. 410. ...  0.  0.  0.]
 [436. 437. 438. ...  0.  0.  0.]]

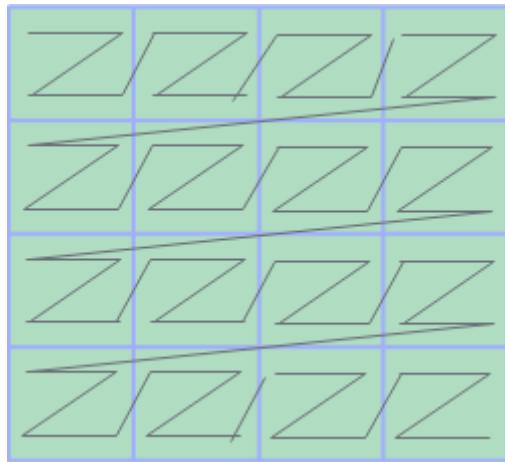
```

```
[[464. 465. 466. ... 0. 0. 0.]  
[492. 493. 494. ... 0. 0. 0.]  
[520. 521. 522. ... 0. 0. 0.]  
...  
[ 0. 0. 0. ... 0. 0. 0.]  
[ 0. 0. 0. ... 0. 0. 0.]  
[ 0. 0. 0. ... 0. 0. 0.]]]
```

- **FRACTAL_ZZ/ZZ**

FRACTAL_ZZ格式，简称ZZ格式，是对一个Tensor最低两维（一个Tensor的所有维度，右侧为低维，左侧为高维）进行填充（pad）、拆分（reshape）和转置（transpose）操作后得到的格式。具体转换过程如下：

(M, K)大小的矩阵被分为 $M1 * K1$ 个分形，按照row major排布，形状如Z字形；每个分形内部有 $M0 * K0$ 个元素，按照row major排布，形状如Z字形，所以这种数据格式称为ZZ格式。其中，(M0, K0)表示一个分形的大小，分形Shape为 $16 \times (32B / \text{sizeof}(\text{Datatype}))$ ，大小为512字节。



通过公式表达转换过程如下：

$(\dots, B, M, K) \rightarrow \text{pad} \rightarrow (\dots, B, M1 * M0, K1 * K0) \rightarrow \text{reshape} \rightarrow (\dots, B, M1, M0, K1, K0) \rightarrow \text{transpose} \rightarrow (\dots, B, M1, K1, M0, K0)$

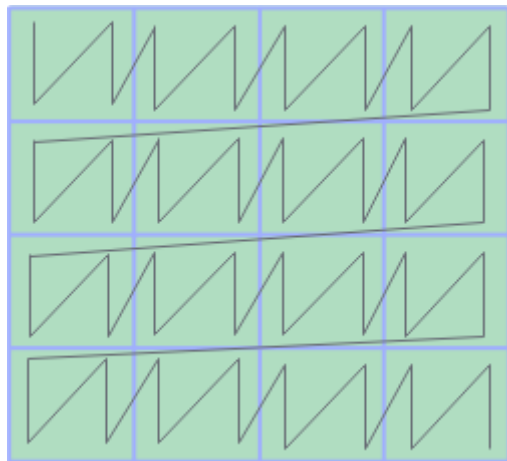
对于不同的数据类型，M0和K0的大小不同：

- 位宽为4的数据类型：M0=16，K0=64。
- 位宽为8的数据类型：M0=16，K0=32。
- 位宽为16的数据类型：M0=16，K0=16。
- 位宽为32的数据类型：M0=16，K0=8。

- **FRACTAL_ZN/ZN**

FRACTAL_ZN格式，简称ZN格式，是对一个Tensor最低两维（一个Tensor的所有维度，右侧为低维，左侧为高维）进行填充（pad）、拆分（reshape）和转置（transpose）操作后得到的格式。具体转换过程如下：

(K, N)大小的矩阵被分为 $K1 * N1$ 个分形，按照row major排布，形状如Z字形；每个分形内部有 $K0 * N0$ 个元素，按照column major排布，形状如N字形，所以这种数据格式称为ZN格式。其中，(K0, N0)表示一个分形的大小，分形shape为 $(32B / \text{sizeof}(\text{Datatype})) \times 16$ ，大小为512字节。



通过公式表达转换过程如下：

$(\dots, B, K, N) \rightarrow \text{pad} \rightarrow (\dots, B, K1 * K0, N1 * N0) \rightarrow \text{reshape} \rightarrow (\dots, B, K1, K0, N1, N0) \rightarrow \text{transpose} \rightarrow (\dots, B, K1, N1, K0, N0)$

对于不同的数据类型，K0和N0的大小不同：

- 位宽为4的数据类型：K0=64，N0=16；
- 位宽为8的数据类型：K0=32，N0=16；
- 位宽为16的数据类型：K0=16，N0=16；
- 位宽为32的数据类型：K0=8，N0=16。

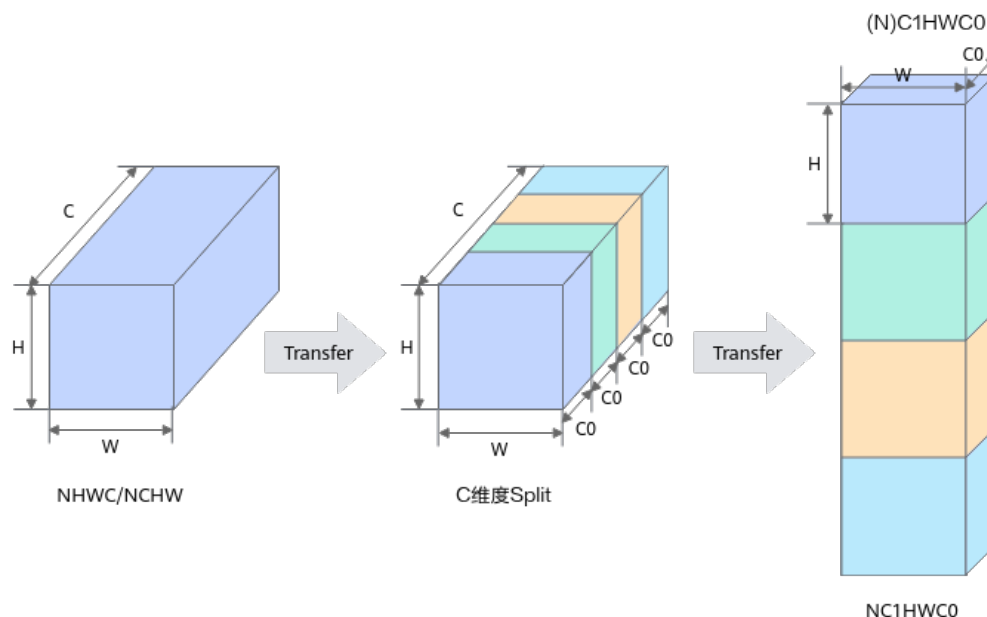
卷积相关特殊格式

- **NC1HWC0**

AI处理器中，为了提高通用矩阵乘法（GEMM）运算数据块的访问效率，所有张量数据统一采用NC1HWC0的五维数据格式。其中C0与微架构强相关，等于AI Core中矩阵计算单元的大小。

$C1 = (C + C0 - 1) / C0$ 。如果结果不整除，向下取整。

NHWC/NCHW -> NC1HWC0的转换过程为：将数据在C维度进行分割，变成C1份NHWC0/NC0HW，再将C1份NHWC0/NC0HW在内存中连续排列成NC1HWC0，其格式转换示意图如下图所示。

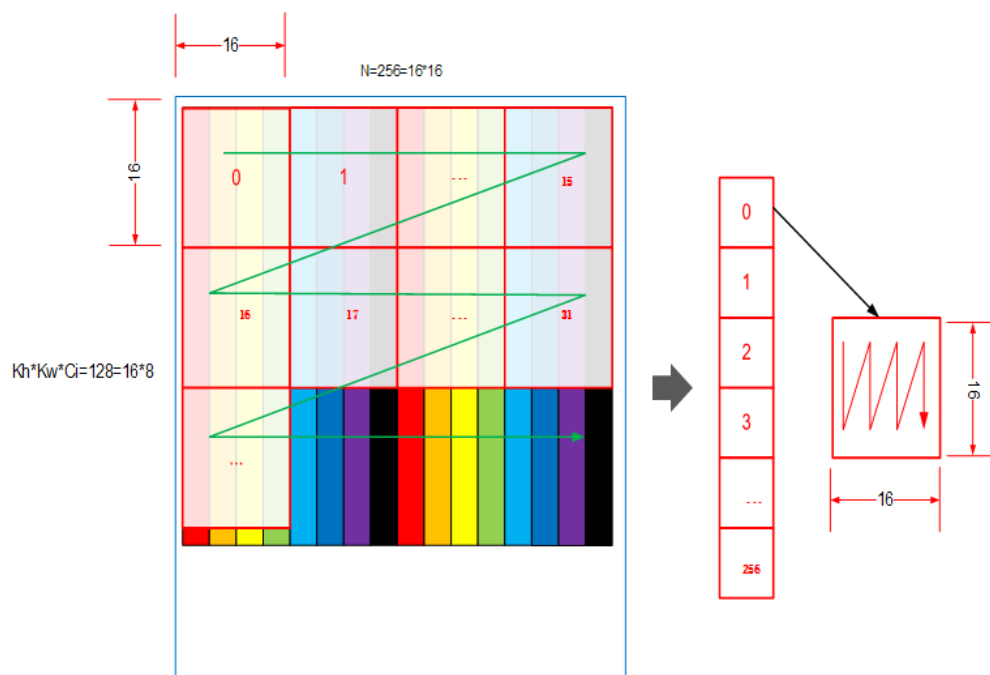


- NHWC -> NC1HWC0的转换公式如下：
`Tensor.reshape( [N, H, W, C1, C0]).transpose( [0, 3, 1, 2, 4] )`
- NCHW -> NC1HWC0的转换公式如下：
`Tensor.reshape( [N, C1, C0, H, W]).transpose( [0, 1, 3, 4, 2] )`

● FRACTAL_Z

FRACTAL_Z是用于定义卷积权重的数据格式，由FT Matrix（FT：Filter，卷积核）变换得到。FRACTAL_Z是送往Cube的最终数据格式，采用“C1HW,N1,N0,C0”的4维数据排布。

数据有两层Tiling，如下图所示：



第一层与Cube的Size相关，数据按照列的方向连续（小n）；第二层与矩阵的Size相关，数据按照行的方向连续（大Z）。

例如: $HWCN = (2, 2, 32, 32)$, 将其变成 $FRACTAL_Z(C1HW, N1, N0, C0) = (8, 2, 16, 16)$ 。

HWCN变换FRACTAL_Z的过程为:

```
Tensor.padding([ [0,0], [0,0], [0,(C0-C%CO)%C0], [0,(N0-N%N0)%N0] ]).reshape([H, W, C1, C0, N1, N0]).transpose([2, 0, 1, 4, 5, 3]).reshape([C1*H*W, N1, N0, C0])
```

NCHW变换FRACTAL_Z的过程为:

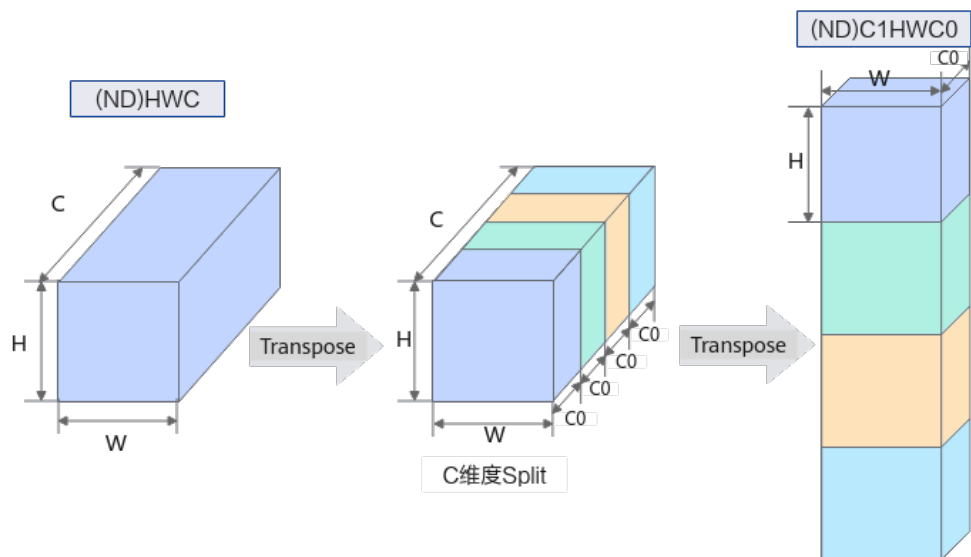
```
Tensor.padding([ [0,(N0-N%N0)%N0], [0,(C0-C%CO)%C0], [0,0], [0,0] ]).reshape([N1, N0, C1, C0, H, W]).transpose([2, 4, 5, 0, 1, 3]).reshape([C1*H*W, N1, N0, C0])
```

- **NDC1HWC0**

为了提高矩阵乘法运算数据块的访问效率, 将NDHWC转换为NDC1HWC0格式。其中C0与微架构强相关, 等于AI Core中矩阵计算单元的大小, 对于float16_t类型为16, 对于int8_t类型则为32, 这部分数据需要连续存储。

$C1 = (C + C0 - 1) / C0$ 。如果结果不整除, 向下取整。

NDHWC -> NDC1HWC0的转换过程为: 将数据在C维度进行分割, 变成C1份NDHWC0, 再将C1份NDHWC0在内存中连续排列成NDC1HWC0, 其格式转换意图如下图所示。



- **FRACTAL_Z_3D**

FRACTAL_Z_3D是3D卷积权重格式, 例如Conv3D算子都会涉及到用这种格式来表达3D卷积的权重。

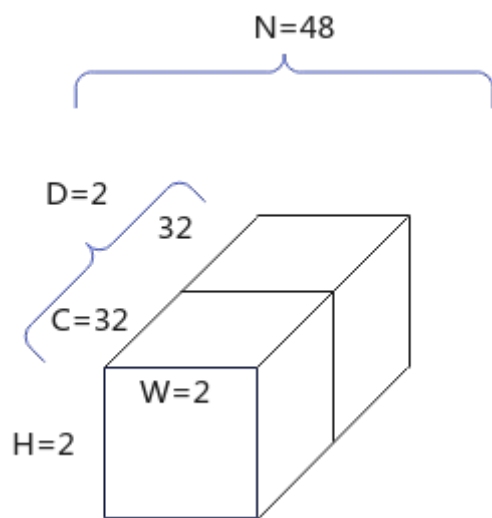
NDHWC -> FRACTAL_Z_3D的变换过程通过公式表达如下:

```
(..., N, D, H, W, C) -> pad -> (... , N1 * N0, D, H, W, C1 * C0) -> reshape -> (... , N1, N0, D, H, W, C1, C0) -> transpose -> (D, C1, H, W, N1, N0, C0) -> reshape -> (... , D * C1 * H * W, N1, N0, C0)
```

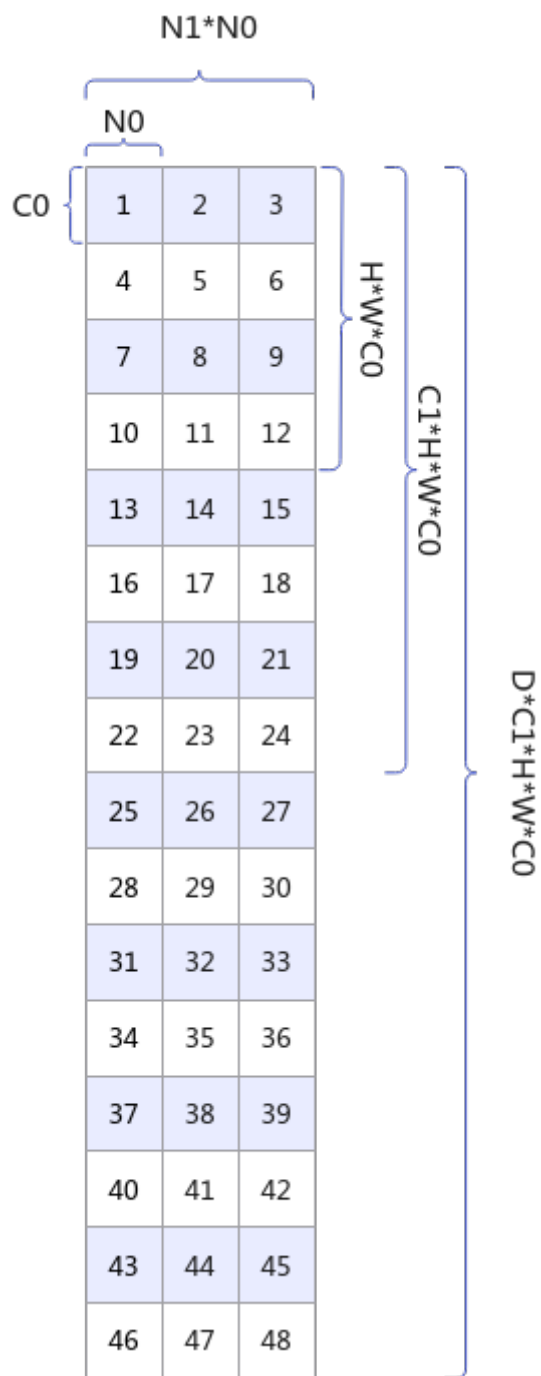
对于不同的数据类型, C0和N0的大小不同:

- 位宽为4的数据类型: $C0=64, N0=16$;
- 位宽为8的数据类型: $C0=32, N0=16$;
- 位宽为16的数据类型: $C0=16, N0=16$;
- 位宽为32的数据类型: $C0=8, N0=16$ 。

输入一个NDHWC格式的Tensor, Shape大小为(48, 2, 2, 2, 32):



转换后，得到FRACTAL_Z_3D格式如下所示：



Matmul 高阶 API 相关格式

- **BSH/SBH**: B: Batch, 批处理的大小; S: sequence length, 序列长度; $H = N \times D$, 其中, N为head的数量, D为head的大小, 此格式通常用于Matmul矩阵乘。数据排布格式如下图所示:

BSH (B = 2, S = 2, H = 12)

		H											
B	S	1	2	3	4	5	6	7	8	9	10	11	12
		13	14	15	16	17	18	19	20	21	22	23	24
	S	25	26	27	28	29	30	31	32	33	34	35	36
		37	38	39	40	41	42	43	44	45	46	47	48

SBH (S = 2, B = 2, H = 12)

		B											
S	H	1	2	3	4	5	6	7	8	9	10	11	12
		13	14	15	16	17	18	19	20	21	22	23	24
	H	25	26	27	28	29	30	31	32	33	34	35	36
		37	38	39	40	41	42	43	44	45	46	47	48

- **BMNK**: 通用数据格式; B: Batch, 批处理的大小; M、N、K为矩阵乘[M, K]*[K, N]的矩阵维度; 其数据排布格式如下:

BMNK (B = 2, M = 3, N = 4, K = 4)

		K			
B	M	1	2	3	4
		5	6	7	8
		9	10	11	12
	M	13	14	15	16
		17	18	19	20
		21	22	23	24

矩阵A

		N			
B	K	1	2	3	4
		5	6	7	8
		9	10	11	12
	K	13	14	15	16
		17	18	19	20
		21	22	23	24
		25	26	27	28
		29	30	31	32

矩阵B

- **BSNGD**: 为原始BSH shape做reshape后的shape, S和D为单Batch的矩阵乘的M轴 (或N轴) 和K轴, 一个SD为一个batch的计算数据, 此格式通常用于Matmul矩阵乘, 数据排布格式如下图所示:

BSNGD (B = 2 , S = 3 , N = 3 , G = 2 , D = 2)

		N											
		G				G				G			
		D		D		D		D		D		D	
B	S	1	2	3	4	5	6	7	8	9	10	11	12
		13	14	15	16	17	18	19	20	21	22	23	24
		25	26	27	28	29	30	31	32	33	34	35	36
	S	37	38	39	40	41	42	43	44	45	46	47	48
		49	50	51	52	53	54	55	56	57	58	59	60
		61	62	63	64	65	66	67	68	69	70	71	72

- **SBNGD**: 为原始SBH shape做reshape后的shape, S和D为单Batch的矩阵乘的M轴 (或N轴) 和K轴, 一个SD为一个Batch的计算数据, 此格式通常用于Matmul矩阵乘, 数据排布格式如下图所示:

SBNGD (S = 3 , B = 2 , N = 3 , G = 2 , D = 2)

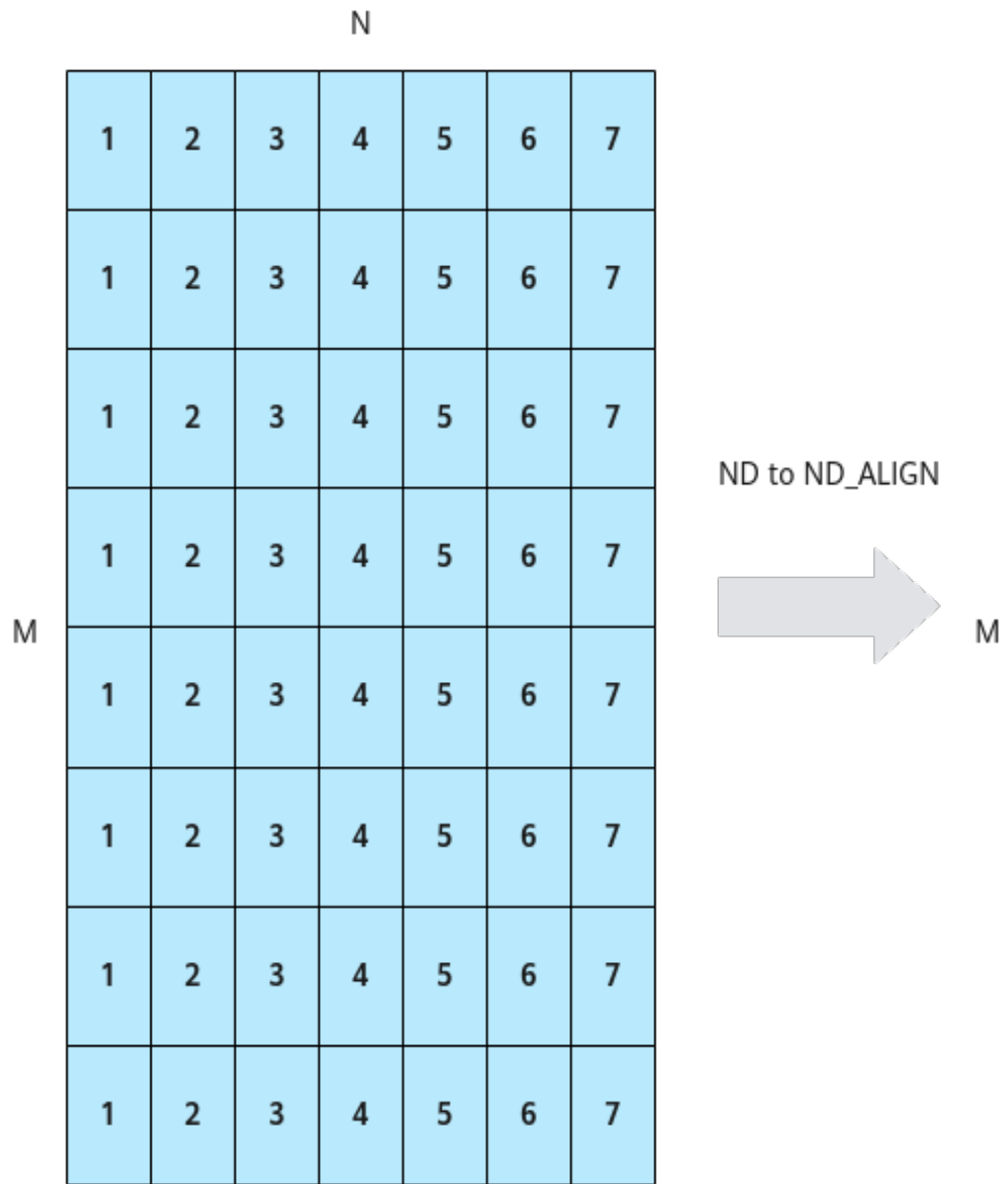
		B																							
		N												N											
		G				G				G				G				G				G			
		D		D		D		D		D		D		D		D		D		D		D		D	
S		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
		25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
		49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72

- **BNGS1S2**: 一般为前两种数据排布进行矩阵乘的输出, S1S2数据连续存放, 一个S1S2为一个Batch的计算数据, 此格式通常用于Matmul矩阵乘, 数据排布格式如下图所示:

BNGS1S2 (B = 2 , N = 3 , G = 2 , S1 = 3 , S2 = 2)

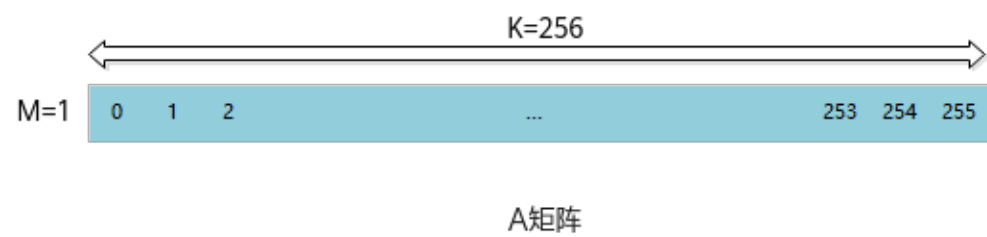
		B																							
		N												N											
		G				G				G				G				G				G			
		S2		S2		S2		S2		S2		S2		S2		S2		S2		S2		S2		S2	
S1		1	2	7	8	13	14	19	20	25	26	31	32	37	38	43	44	49	50	55	56	61	62	67	68
		3	4	9	10	15	16	21	22	27	28	33	34	39	40	45	46	51	52	57	58	63	64	69	70
		5	6	11	12	17	18	23	24	29	30	35	36	41	42	47	48	53	54	59	60	65	66	71	72

- ND_ALIGN:** ND_ALIGN是ND数据格式的一种变换数据格式。输出矩阵乘的结果矩阵C时，用于配置C矩阵按照N方向32字节对齐的规则进行输出。
ND->ND_ALIGN变换过程如下图所示，假设矩阵乘结果矩阵C的数据类型是int32_t，输出到VECOUT，原矩阵N方向没有32字节对齐，设置ND_ALIGN后则在其后补0，将其对齐到32字节。



- VECTOR:** VECTOR是**GEMV**（矩阵向量乘，General Matrix-Vector Multiply）场景使用的一种数据格式，配置矩阵为VECTOR数据排布格式即代表输入数据是一个向量。

图 2-32 GEMV 场景输入 Vector 格式的 A 矩阵示意图



2.9.3 编程模型设计原理

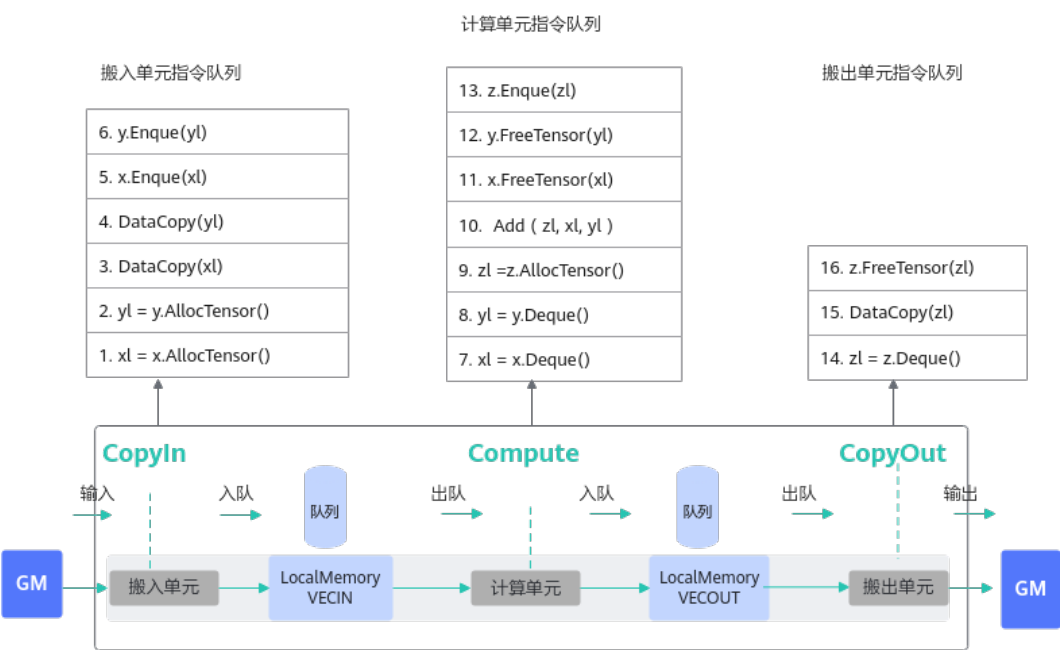
Ascend C编程模型中，并行编程范式核心要素是：一组并行计算任务、通过队列实现任务之间的同步、开发者自主表达对并行计算任务和资源的调度。本节介绍编程模型的设计原理，作为扩展阅读，便于开发者更好的理解编程模型的设计思路和优势，对于后续的深度开发也会有所帮助。

每个并行任务Stage的编程范式如下：

- 1. 获取Local Memory的内存：调用**AllocTensor**申请内存，或者从上游队列**DeQue**一块内存数据。
- 2. 完成计算或者数据搬运。
- 3. 把上一步处理好的数据调用**EnQue**入队。
- 4. 调用**FreeTensor**释放不再需要的内存。

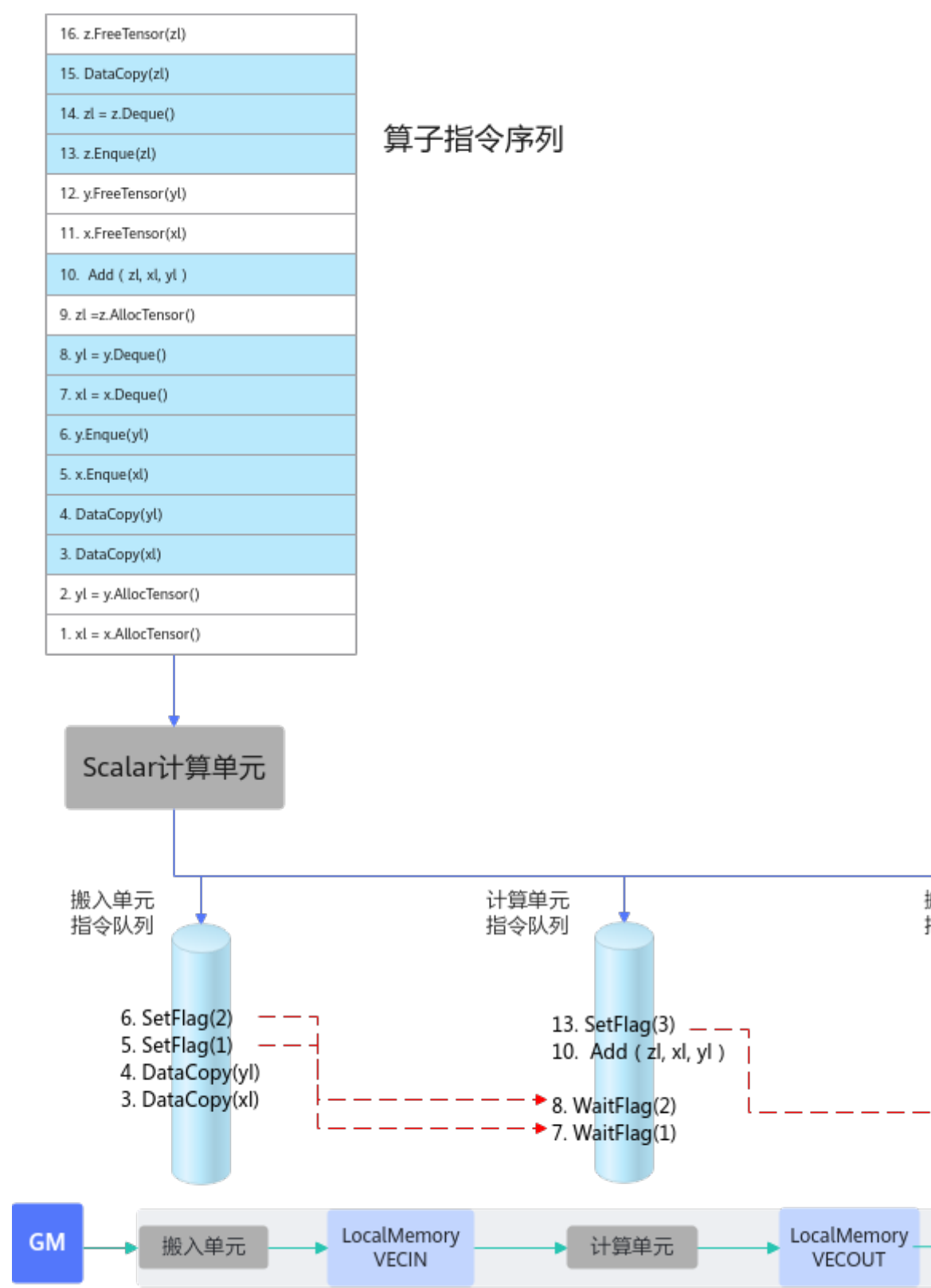
以最简单的矢量编程范式为例，在调用上述接口时，实际上会给各执行单元下发一些指令，如下图所示：

图 2-33 Vector 编程范式指令队列示例

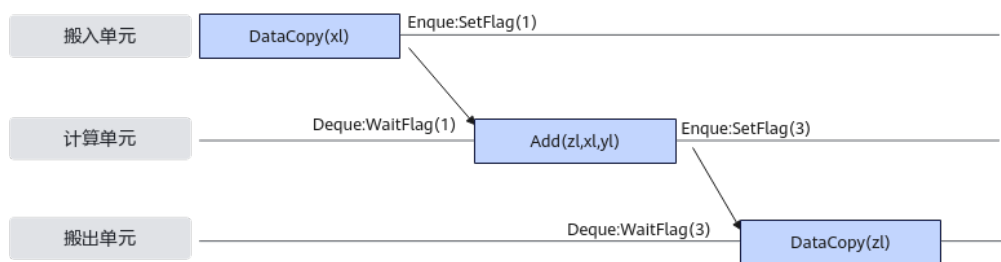


- EnQue/DeQue的具体处理流程：

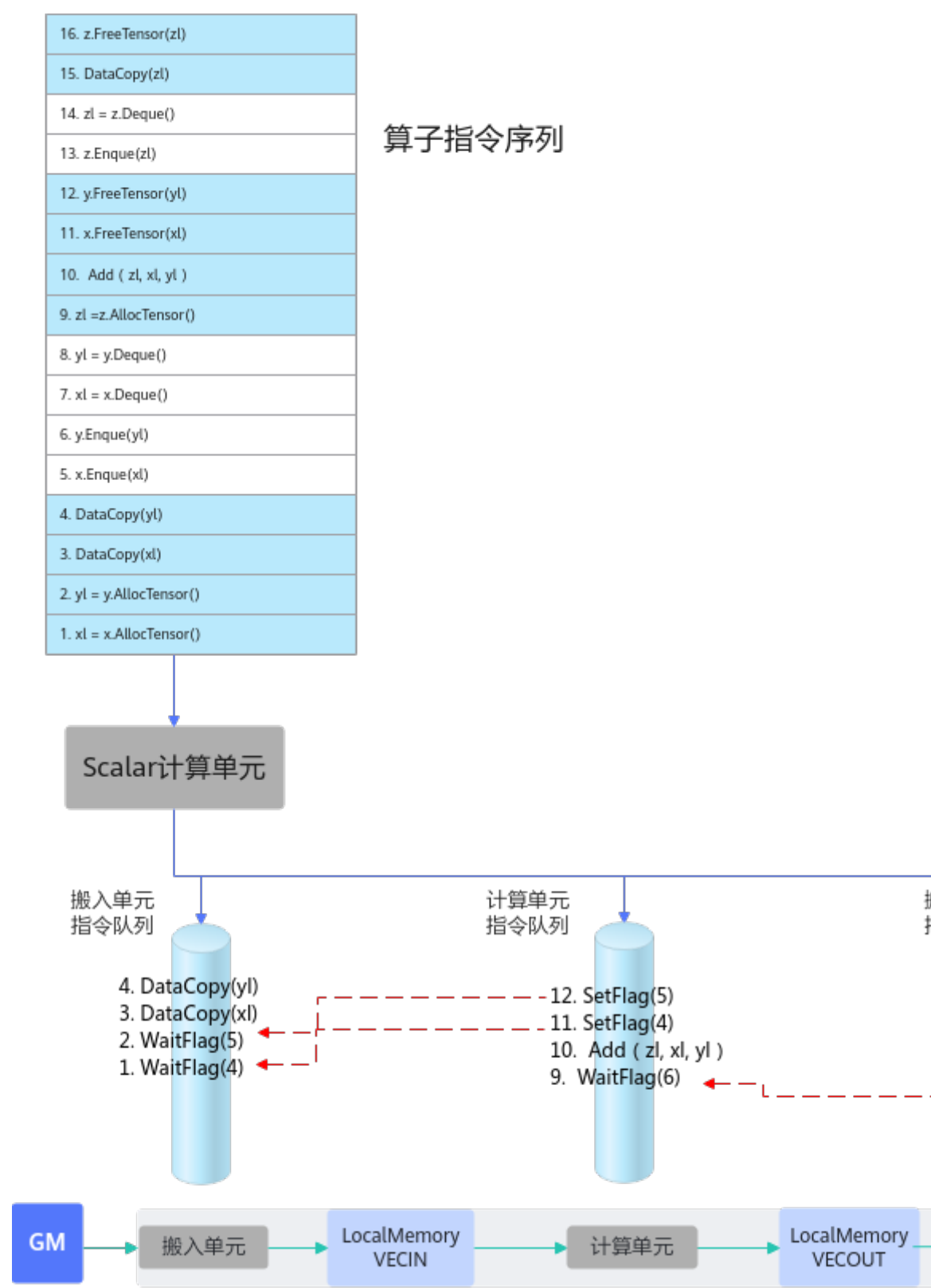
- a. 标量执行单元读取算子指令序列
- b. 把这些指令发射到对应的执行单元的指令队列
- c. 各个执行单元并行执行这些指令序列
- d. EnQue/DeQue解决对内存的写后读问题
 - EnQue调用会发射同步指令set，发送信号激活wait
 - DeQue调用会发射同步指令wait，等待数据写入完成
 - wait需要等到set指令执行完成后才能执行否则阻塞



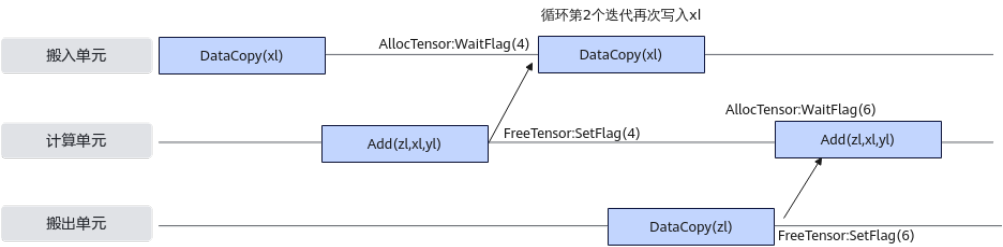
由此可以看出，EnQue/DeQue主要解决了存在数据依赖时，并行执行单元的写后读同步控制问题。



- AllocTensor/FreeTensor的具体处理流程
 - a. 标量执行单元读取算子指令序列
 - b. 把这些指令发射到对应的执行单元的指令队列
 - c. 各个执行单元并行执行这些指令序列
 - d. AllocTensor/FreeTensor，解决对内存的读后写问题
 - AllocTensor调用会发射同步指令wait等待内存被读完成
 - FreeTensor调用会发射同步指令set，通知内存释放，可以重复写
 - wait需要等到set指令执行完成后才能执行否则阻塞



由此可以看出，AllocTensor/FreeTensor主要解决了存在数据依赖时，并行执行单元的读后写同步控制问题。



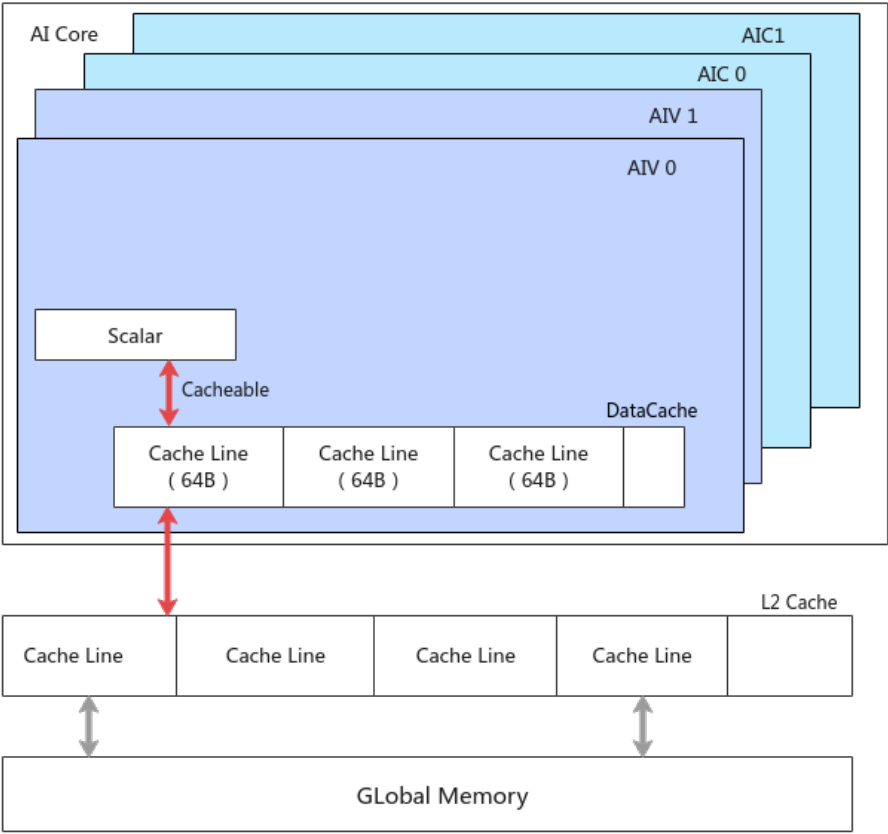
通过上文的详细说明，可以看出异步并行程序需要考虑复杂的同步控制，而Ascend C 编程模型将这些流程进行了封装，通过EnQue/DeQue/AllocTensor/FreeTensor这种开发者熟悉的资源控制方式来体现，达到简化编程和易于理解的目的。

2.9.4 内存访问原理

2.9.4.1 Scalar 读写数据

AI Core中Scalar计算单元负责各类型的标量数据运算和程序的流程控制。根据[硬件架构](#)设计，Scalar仅支持对Global Memory和Unified Buffer的读写操作，而不支持对L1 Buffer、LOA Buffer、LOB Buffer和LOC Buffer等其他类型存储的访问。下文分别介绍了Scalar读写Global Memory和Unified Buffer的方式和Scalar读写数据时的同步机制。

Scalar 读写 Global Memory



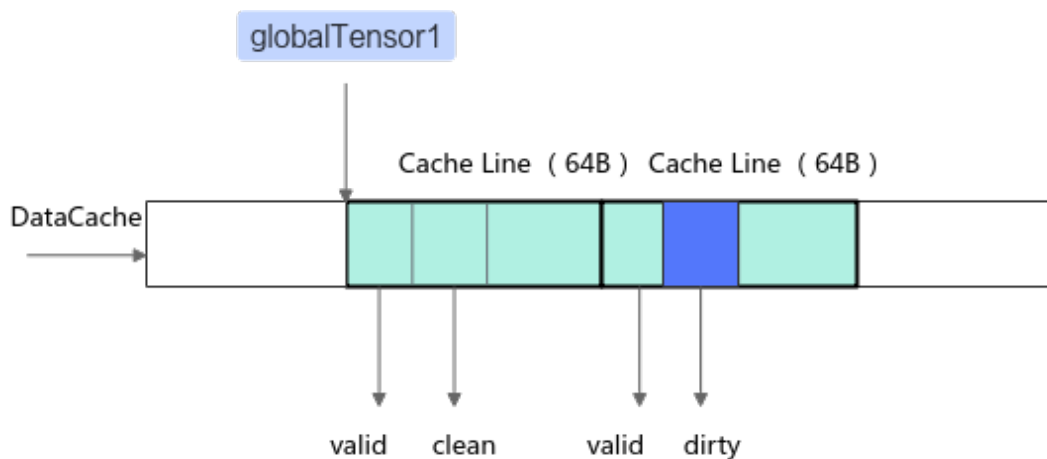
如上图所示，Scalar读写GM数据时会经过DataCache，DataCache主要用于提高标量访存指令的执行效率，每一个AIC/AIV核内均有一个独立的DataCache。下面通过一个具体示例来讲解DataCache的具体工作机制。

globalTensor1是位于GM上的Tensor：

- 执行完GetValue(0)后，globalTensor1的前8个元素会进入DataCache，后续GetValue(1)~GetValue(7)不需要再访问GM，而可以直接从DataCache的Cache Line中读取数据，提高了标量连续访问的效率。
- 执行完SetValue(8, val)后，globalTensor1的index为8~15的元素会进入DataCache，SetValue只会修改DataCache中的Cache Line数据，同时将Cache Line的状态设置为Dirty，表明Cache Line中的数据与GM中的数据不一致。

```
AscendC::GlobalTensor<int64_t> globalTensor1;  
globalTensor1.SetGlobalBuffer((__gm__ int64_t *)input);  
// 从0~7共计8个uint64_t类型，DataCache的Cache Line长度为64字节  
// 执行完GetValue(0)后，GetValue(1)~GetValue(7)可以直接从Cache Line中读取，不需要再访问GM  
globalTensor1.GetValue(0);  
globalTensor1.GetValue(1);  
globalTensor1.GetValue(2);  
globalTensor1.GetValue(3);  
globalTensor1.GetValue(4);  
globalTensor1.GetValue(5);  
globalTensor1.GetValue(6);  
globalTensor1.GetValue(7);  
  
// 执行完SetValue(8)后，不会修改GM上的数据，只会修改DataCache中Cache Line数据  
// 同时Cache Line的状态置为dirty，dirty表示DataCache中Cache Line数据与GM中的数据不一致  
int64_t val = 32;  
globalTensor1.SetValue(8, val);  
globalTensor1.GetValue(8);
```

根据上文的工作机制（如下图所示），多核间访问globalTensor1会出现数据不一致的情况，如果其余核需要获取GM数据的变化，则需要开发者手动调用DataCacheCleanAndInvalid来保证数据的一致性。



Scalar 读写 Unified Buffer

Scalar读写Unified Buffer时，可以使用LocalTensor的SetValue和GetValue接口。示例如下：

```
for (int32_t i = 0; i < 16; ++i) {  
    inputLocal.SetValue(i, i); // 对inputLocal中第i个位置进行赋值为i  
}
```

```
for (int32_t i = 0; i < srcLen; ++i) {  
    auto element = inputLocal.GetValue(i); // 获取inputLocal中第i个位置的数值  
}
```

Scalar 读写数据时的同步

Scalar读写Global MemoryUnified Buffer时属于PIPE_S（Scalar流水）操作，当用户使用SetValue或者GetValue接口，且算子工程使能自动同步时，不需要手动插入同步事件。

如果用户关闭算子工程的自动同步功能时，则需要手动插入同步事件：

```
// GetValue为Scalar操作，与后续的Duplicate存在数据依赖  
// 因此Vector流水需要等待Scalar操作结束  
float inputVal = srcLocal.GetValue(0);  
SetFlag<HardEvent::S_V>(eventID1);  
WaitFlag<HardEvent::S_V>(eventID1);  
AscendC::Duplicate(dstLocal, inputVal, srcDataSize);  
  
// SetValue为Scalar操作，与后续的数据搬运操作存在数据依赖  
// 因此MTE3流水需要等待Scalar操作结束  
srcLocal.SetValue(0, value);  
SetFlag<HardEvent::S_MTE3>(eventID2);  
WaitFlag<HardEvent::S_MTE3>(eventID2);  
AscendC::DataCopy(dstGlobal, srcLocal, srcDataSize);
```

2.9.5 性能优化技术原理

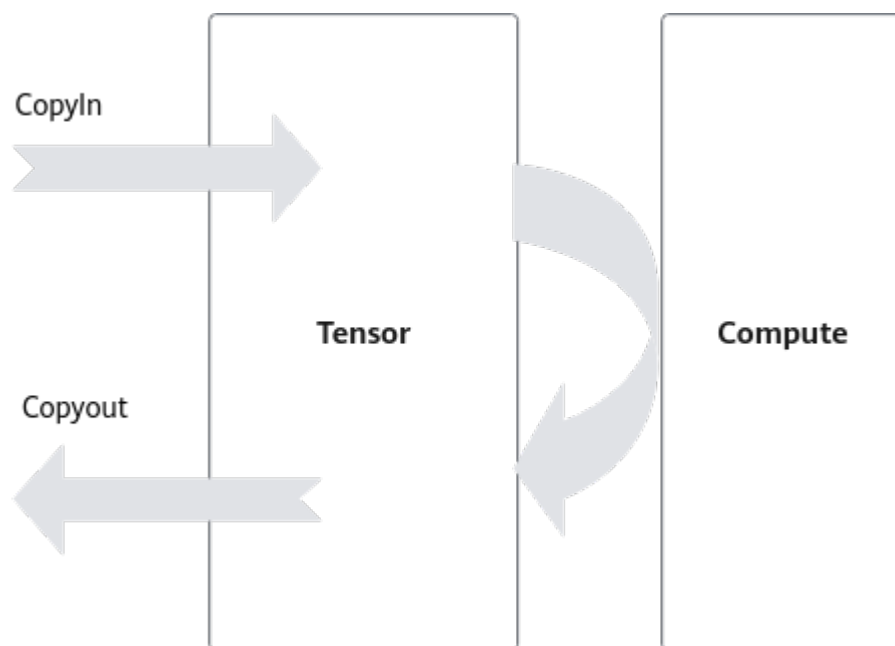
2.9.5.1 DoubleBuffer

执行于AI Core上的指令队列主要包括如下几类，即Vector指令队列、Cube指令队列和MTE指令队列。不同指令队列间的相互独立性和可并行执行性，是DoubleBuffer优化机制的基石。

矢量计算CopyIn、CopyOut过程使用MTE指令队列（MTE2、MTE3），Compute过程使用Vector指令队列（V），意味着CopyIn、CopyOut过程和Compute过程是可以并行的。

如图2-34所示，考虑一个完整的数据搬运和计算过程，CopyIn过程将数据从Global Memory搬运到Local Memory，Vector计算单元完成计算后，经过CopyOut过程将计算结果搬回Global Memory。

图 2-34 数据搬运与 Vector 计算过程



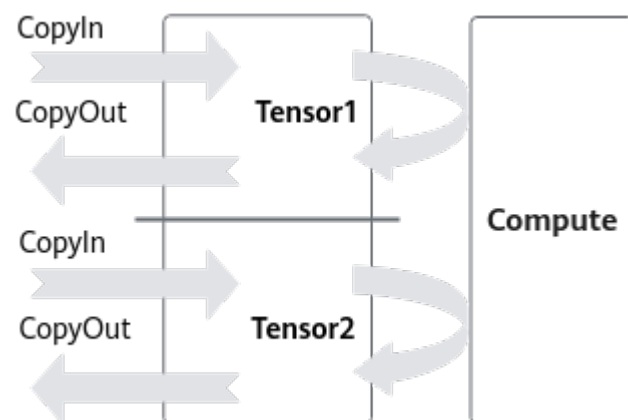
在此过程中，数据搬运与Vector计算串行执行，Vector计算单元无可避免存在资源闲置问题。举例而言，若CopyIn、Compute、CopyOut三阶段分别耗时 t ，则Vector的时间利用率仅为1/3，等待时间过长，Vector利用率严重不足。

为减少Vector等待时间，DoubleBuffer机制将待处理的数据一分为二，比如Tensor1、Tensor2。如图2-35所示，当Vector对Tensor1中数据进行Compute时，Tensor2可以执行CopyIn的过程；而当Vector切换到计算Tensor2时，Tensor1可以执行CopyOut的过程。由此，数据的进出搬运和Vector计算实现并行执行，Vector闲置问题得以有效缓解。

总体来说，DoubleBuffer是基于MTE指令队列与Vector指令队列的独立性和可并行性，通过将数据搬运与Vector计算并行执行以隐藏数据搬运时间并降低Vector指令的等待时间，最终提高Vector单元的利用效率，您可以通过为队列申请内存时设置内存块的个数来实现数据并行，简单代码示例如下：

```
pipe.InitBuffer(inQueueX, 2, 256);
```

图 2-35 DoubleBuffer 机制



需要注意：

多数情况下，采用DoubleBuffer能有效提升Vector的时间利用率，缩减算子执行时间。然而，DoubleBuffer机制缓解Vector闲置问题并不代表它总能带来整体的性能提升。例如：

- 当数据搬运时间较短，而Vector计算时间显著较长时，由于数据搬运在整个计算过程中的时间占比较低，DoubleBuffer机制带来的性能收益会偏小。
- 又如，当原始数据较小且Vector可一次性完成所有计算时，强行使用DoubleBuffer会降低Vector计算资源的利用率，最终效果可能适得其反。

因此，DoubleBuffer的性能收益需综合考虑Vector算力、数据量大小、搬运与计算时间占比等多种因素。

2.10 附录

2.10.1 C++标准支持

2.10.1.1 概述

Host侧与clang15一致，支持完整的C/C++标准。

Device侧，默认支持C++11标准，支持指定C++14、C++17、C++20。由于硬件限制，部分C++运行时能力无法支持。

2.10.1.2 C/C++语法限制

2.10.1.2.1 特性

- 不支持虚函数
- 不支持虚继承
- 不支持运行时递归
- 不支持动态malloc、new/free
- 不支持STL
- 不支持运行时typeid
- 不支持文件系统IO
- 不支持标准库下的tuple及算法类运算（相关库函数调用需要标记aicore）

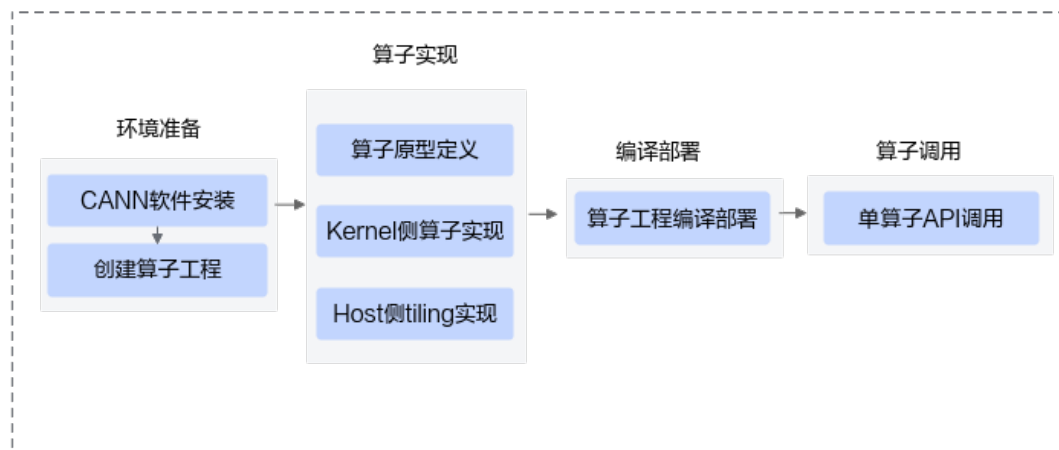
2.10.2 工程化算子开发

2.10.2.1 概述

工程化算子开发是指基于自动生成的**自定义算子工程**完成算子实现、编译部署、单算子调用代码自动生成等一系列流程。

该开发流程是标准的开发流程，建议开发者按照该流程进行算子开发。该方式下，算子开发的代码会更规范、统一、易于维护；同时该方式考虑了单算子API调用、算子入图、AI框架调用等功能的集成，使得开发者易于借助CANN框架实现上述功能。

工程化算子开发流程如下图所示：



步骤1 环境准备。

1. CANN软件安装请参考[1.2 环境准备](#)。
2. [创建算子工程](#)。使用msOpGen工具创建算子开发工程。

步骤2 算子实现。

- [算子原型定义](#)。通过原型定义来描述算子输入输出、属性等信息以及算子在AI处理器上相关实现信息，并关联tiling实现等函数。
- Kernel侧算子实现和host侧tiling实现请参考[3.3 SIMD算子实现](#)；工程化算子开发，支持开发者调用Tiling API基于CANN提供的编程框架进行tiling开发，kernel侧也提供对应的接口方便开发者获取tiling参数，具体内容请参考[2.10.2.4 Kernel侧算子实现](#)和[2.10.2.5 Host侧Tiling实现](#)，由此而带来的额外约束也在上述章节说明。

步骤3 编译部署。通过工程编译脚本完成算子的编译部署，分为[算子包编译](#)和[算子动态库编译](#)两种方式。

步骤4 [单算子API调用](#)：调用单算子API接口，基于C语言的API执行算子。

----结束

2.10.2.2 创建算子工程

CANN开发套件包中提供了自定义算子工程生成工具msOpGen，可基于算子原型定义输出算子工程：包括算子host侧代码实现文件、算子kernel侧实现文件以及工程编译配置文件等。

📖 说明

使用msOpGen工具创建算子工程之前，需要参考[1.2 环境准备](#)章节安装驱动固件和CANN软件包，完成开发环境和运行环境的准备。

使用msOpGen工具创建算子开发工程的步骤如下：

步骤1 编写算子的原型定义json文件，用于生成算子开发工程。json文件的配置参数详细说明请参考表1。

例如，AddCustom算子的json文件命名为add_custom.json，文件内容如下：

```
[
{
```

```
"op": "AddCustom",
"input_desc": [
  {
    "name": "x",
    "param_type": "required",
    "format": [
      "ND",
      "ND",
      "ND"
    ],
    "type": [
      "float16",
      "float",
      "int32"
    ]
  },
  {
    "name": "y",
    "param_type": "required",
    "format": [
      "ND",
      "ND",
      "ND"
    ],
    "type": [
      "float16",
      "float",
      "int32"
    ]
  }
],
"output_desc": [
  {
    "name": "z",
    "param_type": "required",
    "format": [
      "ND",
      "ND",
      "ND"
    ],
    "type": [
      "float16",
      "float",
      "int32"
    ]
  }
]
}
```

例如，ReduceMaxCustom算子（包含属性）的json文件命名为reduce_max_custom.json，文件内容如下：

```
[
  {
    "op": "ReduceMaxCustom",
    "input_desc": [
      {
        "name": "x",
        "param_type": "required",
        "format": ["ND"],
        "type": ["float16"]
      }
    ],
    "output_desc": [
      {
        "name": "y",
        "param_type": "required",
        "format": ["ND"],
        "type": ["float16"]
      }
    ]
  }
]
```

```
    },  
    {  
      "name": "idx",  
      "param_type": "required",  
      "format": ["ND"],  
      "type": ["int32"]  
    }  
  ],  
  "attr": [  
    {  
      "name": "reduceDim",  
      "param_type": "required",  
      "type": "int"  
    },  
    {  
      "name": "isKeepDim",  
      "param_type": "optional",  
      "type": "int",  
      "default_value": 1  
    }  
  ]  
}
```

说明

原型定义json文件中的算子类型字段op需要采用**大驼峰**的命名方式，即采用大写字符区分不同的语义。

步骤2 使用msOpGen工具生成算子的开发工程。以生成AddCustom的算子工程为例，下文仅针对关键参数进行解释，详细参数说明请参见算子工程创建（msOpGen）。

```
${INSTALL_DIR}/python/site-packages/bin/msopgen gen -i $HOME/sample/add_custom.json -c ai_core-  
<soc_version> -lan cpp -out $HOME/sample/AddCustom
```

- **\${INSTALL_DIR}**为CANN软件安装后文件存储路径，请根据实际环境进行替换。
- **-i**：指定算子原型定义文件`add_custom.json`所在路径，请根据实际情况修改。
- **-c**：ai_core-**<soc_version>**代表算子在AI Core上执行，**<soc_version>**为AI处理器的型号。

说明

AI处理器的型号**<soc_version>**请通过如下方式获取：

- 针对如下产品：在安装AI处理器的服务器执行**npusmi info**命令进行查询，获取**Name**信息。实际配置值为AscendName，例如**Name**取值为xxxxyy，实际配置值为Ascendxxxxyy。
Atlas A2 训练系列产品/Atlas A2 推理系列产品
Atlas 200I/500 A2 推理产品
Atlas 推理系列产品
Atlas 训练系列产品
- 针对如下产品，在安装AI处理器的服务器执行**npusmi info -t board -i id -c chip_id**命令进行查询，获取**Chip Name**和**NPU Name**信息，实际配置值为Chip Name_NPU Name。例如**Chip Name**取值为Ascendxxx，**NPU Name**取值为1234，实际配置值为Ascendxxx_1234。其中：
 - **id**：设备id，通过**npusmi info -l**命令查出的NPU ID即为设备id。
 - **chip_id**：芯片id，通过**npusmi info -m**命令查出的Chip ID即为芯片id。Atlas A3 训练系列产品/Atlas A3 推理系列产品

基于同系列的AI处理器型号创建的算子工程，其基础功能（基于该工程进行算子开发、编译和部署）通用。

- -lan: 参数cpp代表算子基于Ascend C编程框架, 使用C/C++编程语言开发。
- -out: 生成文件所在路径, 可配置为绝对路径或者相对路径, 并且工具执行用户对路径具有可读写权限。若不配置, 则默认生成在执行命令的当前路径。

步骤3 命令执行完后, 会在-out指定目录或者默认路径下生成算子工程目录, 工程中包含算子实现的模板文件, 编译脚本等, 以AddCustom算子为例, 目录结构如下所示:

```
AddCustom
├── build.sh           // 编译入口脚本
├── cmake              // 算子工程编译所需脚本及公共编译文件存放目录
├── CMakeLists.txt     // 算子工程的CMakeLists.txt
├── CMakePresets.json  // 编译配置项
├── framework          // AI框架适配时, 算子插件实现文件目录
├── op_host            // host侧实现文件
│   ├── add_custom_tiling.h // 算子tiling定义文件
│   ├── add_custom.cpp      // 算子原型注册、shape推导、信息库、tiling实现等内容文件
│   └── CMakeLists.txt
├── op_kernel          // kernel侧实现文件
│   ├── CMakeLists.txt
│   └── add_custom.cpp      // 算子代码实现文件
└── scripts            // 自定义算子工程打包相关脚本所在目录
```

说明

- 上述目录结构中的粗体文件为后续算子开发过程中需要修改的文件, 其他文件无需修改。
- kernel侧实现依赖的所有文件需全部放置在op_kernel目录下, 不遵循该约束会导致[源码发布](#)场景在线编译失败。因为在后续的算子源码发布时, 只会打包发布op_kernel目录下的文件。

----结束

工程目录中的op_kernel和op_host包含了算子的核心实现文件。op_kernel下存放[kernel侧算子实现](#)。op_host下存放host侧代码实现, 包括[算子原型定义](#)、[host侧tiling实现](#)。其中kernel侧算子实现和host侧tiling实现在[3.3 SIMD算子实现](#)章节已经介绍了其核心的实现方法, 在该章节会侧重于介绍接入CANN框架后的编程模式和API的使用。工程目录中的CMakePresets.json, 用于开发者完成工程编译相关配置, 之后即可进行编译部署。

2.10.2.3 算子原型定义

算子原型主要描述了算子的输入输出、属性等信息以及算子在AI处理器上相关实现信息, 并关联[tiling实现](#)等函数。算子原型通过自定义的算子类来承载, 该算子类继承自OpDef类。完成算子的原型定义等操作后, 需要调用OP_ADD接口, 传入算子类型(自定义算子类的类名), 进行算子原型注册。下面是一个简单的Add算子原型定义和注册的例子。

```
namespace ops {
class AddCustom : public OpDef {
public:
    AddCustom(const char* name) : OpDef(name)
    {
        this->Input("x")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
        this->Input("y")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
        this->Output("z")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
        // 如下的shape/datatype推导函数仅在算子入图场景使用
    }
};
}
```

```
this->SetInferShape(ge::InferShape);
this->SetInferDataType(ge::InferDataType);
this->AICore()
    .SetTiling(optiling::TilingFunc);
// 请替换为实际的昇腾AI处理器型号
this->AICore().AddConfig("ascendxxx");
}
};
OP_ADD(AddCustom);
} // namespace ops
```

说明

- 基于算子原型定义，自定义算子工程可以实现如下自动化能力：
 - 自动生成**单算子API调用**的实现和接口，开发者可以直接使用生成的API实现单算子调用。
 - 自动生成图模式场景使用的算子原型定义REG_OP，开发者可以使用生成的算子原型进行构图、图编译、图执行等操作。
- 注册算子类型后，框架会根据算子类型获取算子注册信息，同时在编译和运行时按照一定的规则匹配算子实现文件名称和kernel侧核函数名称。为了保证正确匹配，算子类型、算子实现文件名称和核函数名称需要遵循如下定义规则。通常情况下，开发者只需要保证创建算子工程时原型定义json文件中算子类型op的参数值为大驼峰命名方式即可，工程创建后自动生成的代码即满足该规则。在手动编写算子原型定义和算子实现文件时需要按照如下规则定义。
 - 算子类型需要采用**大驼峰**的命名方式，即采用大写字符区分不同的语义。
 - 算子实现文件名称、核函数名称需相同，均为算子类型转换为**下划线**命名方式后的值。下文描述了通过算子类型转换成算子实现文件名称和核函数名称的过程：
 - 首字符的大写字符转换为小写字符。例如：Abc -> abc。
 - 大写字符的前一个字符为小写字符或数字，则在大写字符前插一个下划线“_”，并将该字符转换为小写字符。例如：AbcDef -> abc_def。
 - 大写字符前一个字符为大写字符且后一个字符是小写字符，则在大写字符前插一个下划线“_”，并将该字符转换为小写字符。例如：AbcAAc -> abc_a_ac。
 - 其他大写字符转换为小写字符，小写字符保持不变。

算子输入/输出/属性定义

算子原型定义描述了算子的输入输出、属性等信息。输入输出支持的datatype、format格式的数量需要一致，并保持一一对应的关系。

如下的代码片段呈现了Add算子输入x的描述信息。

```
this->Input("x")
    .ParamType(REQUIRED)
    .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
    .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
```

表 2-24 输入输出参数说明

原型定义	参数	具体描述
Input/ Output	ParamType	参数类型，Option取值为：OPTIONAL（可选）、REQUIRED（必选）、DYNAMIC（动态输入）。 - 类似于上文中的Add样例，其输入输出是必选的。 - 有些算子的输入或者输出个数是动态的，例如AddN，将N个输入Tensor累加到一起，输出一个Tensor；SplitV，将一个Tensor在某个轴上，拆分为N个Tensor输出。 - 有些算子的输入/输出是可选的，例如BatchNorm算子，在训练的时候没有均值和方差输入，在推理的时候有均值和方差的输入。
	DataType	算子输入输出支持的datatype。datatype的取值请参考DataType。
	Format	算子输入输出支持的format。format的取值请参考Format。

从上文的原型定义中可以看出，列出了输入输出所有datatype和format的组合，保持一一对应。使用如下接口，可以达到简化这种代码逻辑的目的。

- 在指定输入/输出的datatype信息时，如果某个输入/输出的datatype支持和其他所有输入/输出的datatype/format组合使用，其datatype可以通过DataTypeList来表达；在指定输入/输出的format信息时，如果某个输入/输出的format支持和其他所有输入/输出的datatype/format组合使用，其format可以通过FormatList来表达。示例如下，以下两种代码表达含义相同。

```
// 列出所有一一对应的组合
class XxxCustom : public OpDef {
public:
    XxxCustom(const char* name) : OpDef(name)
    {
        this->Input("x")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT16, ge::DT_FLOAT16, ge::DT_FLOAT16});
        this->Input("y")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
        this->Output("z")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
        ...
    }
};

// 通过DataTypeList和FormatList来表达，无需重复列出
class XxxCustom : public OpDef {
public:
    XxxCustom(const char* name) : OpDef(name)
    {
        this->Input("x")
            .ParamType(REQUIRED)
            .DataTypeList({ge::DT_FLOAT16})
            .FormatList({ge::FORMAT_ND});
        this->Input("y")
```



```
.ParamType(REQUIRED)
.DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
.Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
this->Output("z")
.ParamType(REQUIRED)
.DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
.Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
...
}
};
```

- 通过Follow接口指定当前输入/输出的datatype/format/shape信息与之前定义过的某个输入一致。示例如下：输出“y1” Follow输入“x1”场景，此时“y1”的datatype、format以及shape都将会和“x1”保持一致。使用Follow接口指定shape一致时通常比shape推导函数逻辑更加简单，能用Follow表达的逻辑，建议使用Follow接口，则无需再编写注册InferShape函数。

```
this->Input("x1")
.ParamType(REQUIRED)
.DataType({ge::DT_FLOAT, ge::DT_FLOAT})
.Format({ge::FORMAT_ND, ge::FORMAT_ND});
this->Input("x2")
.ParamType(REQUIRED)
.DataType({ge::DT_FLOAT, ge::DT_FLOAT})
.Format({ge::FORMAT_ND, ge::FORMAT_ND});
this->Output("y1")
.ParamType(REQUIRED)
.Follow("x1")
.OutputShapeDependOnCompute();
```

原型定义中还包括算子属性信息，如下的代码片段呈现了ReduceMax算子的属性reduceDim和isKeepDim的描述信息。

```
this->Attr("reduceDim")
.AttrType(REQUIRED)
.Int();
this->Attr("isKeepDim")
.AttrType(OPTIONAL)
.Int(1);
```

具体参数说明如下：

表 2-25 属性参数说明

原型定义	注册方式	具体描述
Attr	AttrType	设置算子属性类型，取值为：OPTIONAL（可选）、REQUIRED（必选）。
	Bool/ Float/ Int...	设置算子属性数据类型为Bool/Float/Int...。具体说明请参考OpAttrDef。

AI 处理器上相关实现信息

通过AddConfig注册算子支持的AI处理器型号以及相关的配置信息。AddConfig接口原型如下：soc参数表示AI处理器型号，aicore_config表示其他配置信息。

```
void AddConfig(const char *soc);
void AddConfig(const char *soc, OpAICoreConfig &aicore_config);
```

通过该接口注册AI处理器型号的样例如下，ascendxxx填写规则请参考算子工程目录下编译配置项文件CMakePresets.json中的ASCEND_COMPUTE_UNIT字段，该字段取值在[使用msOpGen创建工程](#)时自动生成。

```
this->AICore().AddConfig("ascendxxx");
```

其他AI Core配置信息的配置方式请参考OpAICoreConfig。

注册 Tiling 实现、Shape 推导等函数

通过SetInferShape、SetInferDataType、SetTiling接口来注册对应的Tiling实现和Shape推导等函数，样例如下。注册的Tiling实现等函数由框架侧进行调用，并在调用时传入对应的Context上下文，供开发者使用。Tiling函数的实现方法请参考[2.10.2.5 Host侧Tiling实现](#)，入图相关的Shape推导等函数实现请参考[2.10.3 算子入图（GE图）开发](#)。

```
// 如下的shape/datatype推导函数仅在算子入图场景使用
this->SetInferShape(ge::InferShape);
this->SetInferDataType(ge::InferDataType);
this->AICore()
    .SetTiling(optiling::TilingFunc);
```

多硬件平台注册差异化的算子原型

算子类继承基类OpDef，使用Input、Output、Attr等注册算子原型信息，硬件平台支持相同的算子原型的情况下，直接通过AICore().AddConfig添加支持的AI处理器型号即可；不同的硬件形态算子原型定义不同的情况，可以通过新增OpAICoreConfig的方式，针对不同的AI处理器型号注册差异化的算子原型。

差异化的算子原型生效规则如下：

- 对于算子类的输入输出原型信息，OpAICoreConfig未配置的会继承OpDef定义的原型，比如算子类中定义了输出y，OpAICoreConfig中没有定义输出y，OpAICoreConfig会继承y的原型定义；
- 对于算子类和新增OpAICoreConfig中定义的算子原型相同的情况，新增OpAICoreConfig中定义的算子原型信息会覆盖OpDef定义的原型信息，比如算子类中定义了输入x支持DT_FLOAT16数据类型，新增OpAICoreConfig中也定义了输入x，但是支持DT_FLOAT16、DT_BF16数据类型，则以OpAICoreConfig新增定义为准。

如下样例中ascendxxx1、ascendxxx2（AI处理器型号）使用相同的算子原型，算子类通过继承基类OpDef，使用Input、Output、Attr等注册算子原型信息，再通过AICore().AddConfig添加支持的AI处理器型号；对于ascendxxx3支持的算子原型需要定制化处理，新增了DT_BF16的类型，通过新增OpAICoreConfig的方式进行注册，x，y，z的定义会覆盖算子类中对应定义的原型信息。

```
namespace ops {
class MyAdd : public OpDef {
public:
    MyAdd(const char* name) : OpDef(name)
    {
        // ascendxxx1 ascendxxx2 AI处理器型号原型定义
        this->Input("x")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT16})
            .Format({ge::FORMAT_ND});
        this->Input("y")
            .ParamType(OPTIONAL)
            .DataType({ge::DT_INT64})
            .Format({ge::FORMAT_ND});
    }
};
```

```
this->Output("z")
    .ParamType(REQUIRED)
    .DataType({ge::DT_FLOAT16})
    .Format({ge::FORMAT_ND});
this->AICore()
    .SetTiling(optiling::TilingFunc);
this->AICore().AddConfig("ascendxxx1");
this->AICore().AddConfig("ascendxxx2");
// ascendxxx3 AI处理器定义OpAICoreConfig变量，定制化原型
OpAICoreConfig config;
config.Input("x")
    .ParamType(REQUIRED)
    .DataType({ge::DT_FLOAT16, ge::DT_BF16})
    .Format({ge::FORMAT_ND, ge::FORMAT_ND});
config.Input("y")
    .ParamType(REQUIRED)
    .DataType({ge::DT_FLOAT16, ge::DT_BF16})
    .Format({ge::FORMAT_ND, ge::FORMAT_ND});
config.Output("z")
    .ParamType(REQUIRED)
    .DataType({ge::DT_FLOAT16, ge::DT_BF16})
    .Format({ge::FORMAT_ND, ge::FORMAT_ND});
this->AICore().AddConfig("ascendxxx3", config);
}
};
OP_ADD(MyAdd);
}
```

如下的样例中，只有几个参数原型信息在不同硬件平台不一致，开发者也可以通过OpAICoreConfig定制部分算子原型信息，复用OpDef定义的其他算子原型信息，达到部分原型信息硬件平台定制化的目的。

```
class AddCustom : public OpDef {
public:
    AddCustom(const char* name) : OpDef(name)
    {
        this->Input("x").DataType({ ge::DT_FLOAT16 }).ParamType(OPTIONAL);
        this->Output("y").DataType({ ge::DT_FLOAT16 });
        OpAICoreConfig aicConfig1;
        OpAICoreConfig aicConfig2;
        aicConfig1.Input("x")
            .ParamType(OPTIONAL)
            .DataType({ ge::DT_FLOAT })
            .Format({ ge::FORMAT_ND });
        aicConfig2.Input("x")
            .ParamType(REQUIRED)
            .DataType({ ge::DT_INT32 })
            .Format({ ge::FORMAT_ND });
        this->AICore().AddConfig("ascendxxx1", aicConfig1);
        this->AICore().AddConfig("ascendxxx2", aicConfig2);
    }
};
```

2.10.2.4 Kernel 侧算子实现

在[2.2.3 核函数](#)章节已经介绍了kernel侧算子核心的实现方法，本章节侧重于介绍接入CANN框架时编程模式和API的使用。

自动生成 kernel 侧算子实现模板

在算子工程目录下的“op_kernel/xxx.cpp”文件中实现算子的核函数。核函数的定义模板已通过msOpGen工具自动生成，样例如下所示。**注意这里参数的顺序按照“输入、输出、workspace、tiling”的顺序排布，开发者不要调整其顺序。**

```
#include "kernel_operator.h"
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR
```

```
workspace, GM_ADDR tiling) {  
    GET_TILING_DATA(tiling_data, tiling); // 获取Tiling参数，详见下文介绍  
    // TODO: user kernel impl  
}
```

说明

算子原型定义中的输入和输出同名的情况下，自动生成的核函数中，输出参数增加ref后缀予以区分。示例如下：

```
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR x_ref,  
GM_ADDR workspace, GM_ADDR tiling) {  
    ...  
}
```

GET_TILING_DATA 获取 Tiling 参数

提供GET_TILING_DATA，用于获取算子kernel入口函数传入的tiling信息，并填入注册的Tiling结构体中，此函数会以宏展开的方式进行编译。注意，对应的算子host实现中需要定义TilingData结构体，实现并注册计算TilingData的Tiling函数。具体请参考[2.10.2.5 Host侧Tiling实现](#)。

核函数中调用GET_TILING_DATA获取TilingData的样例如下：

```
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR  
workspace, GM_ADDR tiling)  
{  
    GET_TILING_DATA(tilingData, tiling);  
    KernelAdd op;  
    op.Init(x, y, z, tilingData.totalLength, tilingData.tileNum);  
    if (TILING_KEY_IS(1)) {  
        op.Process();  
    }  
}
```

核函数内获取算子输入输出的数据类型和格式

算子工程在核函数内提供了DTYPE_<Arg>、ORIG_DTYPE_<Arg>、FORMAT_<Arg>三种宏用于表示核函数入参（算子的输入输出）的数据类型、原始数据类型和数据格式。其中<Arg>为入参的大写格式。

- DTYPE_<Arg>，入参的数据类型。是指在Device侧实际可使用的数据类型，比如half、float等。
- ORIG_DTYPE_<Arg>，入参的原始数据类型。是指在Host侧进行原型定义时，指定的数据类型（不包含命名空间），比如DT_FLOAT16、DT_FLOAT等。
- FORMAT_<Arg>，入参的数据格式。是指在Host侧进行原型定义时，指定的数据格式（不包含命名空间），比如FORMAT_ND、FORMAT_NZ等。

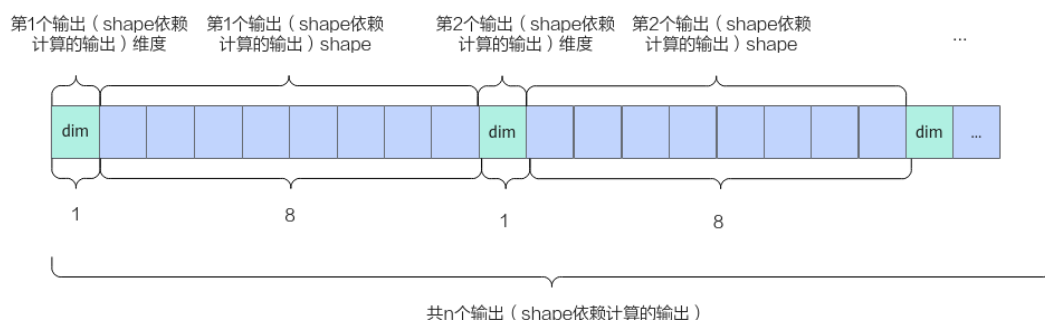
样例如下：

```
template<class T> func() {}  
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR  
workspace, GM_ADDR tiling)  
{  
    DTYPE_X temp;  
    func<DTYPE_Z>();  
    if (FORMAT_Y == FORMAT_ND) {  
        ...  
    }  
    if (ORIG_DTYPE_Y == DT_FLOAT) {  
        ...  
    }  
}
```

输出 shape 依赖计算的算子 kernel 实现

某些算子，比如NonZero（统计tensor中非零值的个数），计算完成前无法得知算子输出的shape信息，算子计算完成后才能获取。该类算子在原型定义时，需要使用OutputShapeDependOnCompute接口进行标识，同时在算子核函数中将实际输出shape写入到出参中，便于框架侧基于该信息进行输出内存的管理。

在核函数所有输出的最后增加一个GM_ADDR类型的输出参数，并在核函数计算完成后，将输出shape信息写入到该出参中。shape信息的排布格式如下，大小为 $n * (8 + 1)$ ，每个元素的数据类型为uint64_t。其中n表示待刷新shape信息的输出个数，每个输出的shape信息都通过第1个元素来保存实际的shape维度（dim），后续的8个元素来保存具体每个维度的shape信息。



说明

- 输出的顺序和原型定义中输出的顺序保持一致。
- 对于uint64_t的输出数据类型（对于tensor而言），需要将dim的uint32_t的高位设置为1，表示以uint64_t类型解析该tensor。
- 如下示例中，算子中有一个输出依赖计算得出，输出tensor的数据类型为uint32_t，计算完成后，得到输出的shape为（32, 64），出参shape_out用于存放该shape信息，值为（2, 32, 64）。代码示例如下：

```
extern "C" __global__ __aicore__ void xxx_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR shape_out, GM_ADDR workspace, GM_ADDR tiling) {  
    ...  
    constexpr uint32_t SHAPEOUT_SIZE = 9;  
    // 输出数据为2维([32, 64])，tensor类型为uint32_t  
    // shapeoutGlobal_uint32用于存放输出Shape信息，数据类型固定为uint64_t  
    GlobalTensor<uint64_t> shapeoutGlobal_uint32;  
    shapeoutGlobal_uint32.SetGlobalBuffer((__gm__ uint64_t*)shape_out, SHAPEOUT_SIZE);  
    shapeoutGlobal_uint32.SetValue(0, 2);  
    shapeoutGlobal_uint32.SetValue(1, 32);  
    shapeoutGlobal_uint32.SetValue(2, 64);  
    ...  
}
```
- 如下示例中，算子中有一个输出依赖计算得出，输出tensor的数据类型为uint64_t，计算完成后，得到输出的shape为（1, 64, 128, 128），出参shape_out用于存放该shape信息，值为（0x0000000080000000 | 4, 1, 64, 128, 128）。代码示例如下：

```
extern "C" __global__ __aicore__ void xxx_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR shape_out, GM_ADDR workspace, GM_ADDR tiling) {  
    ...  
    constexpr uint32_t SHAPEOUT_SIZE = 9;  
    // 输出数据为4维([1, 64, 128, 128])，tensor类型为uint64_t  
    // shapeoutGlobal_uint64用于存放输出Shape信息，数据类型固定为uint64_t  
    GlobalTensor<uint64_t> shapeoutGlobal_uint64;  
    shapeoutGlobal_uint64.SetGlobalBuffer((__gm__ uint64_t*)shape_out, SHAPEOUT_SIZE);  
    shapeoutGlobal_uint64.SetValue(0, 0x0000000080000000 | 4);  
    ...  
}
```

```
shapeoutGlobal_uint64.SetValue(1, 1);  
shapeoutGlobal_uint64.SetValue(2, 64);  
shapeoutGlobal_uint64.SetValue(3, 128);  
shapeoutGlobal_uint64.SetValue(4, 128);  
...  
}
```

- 如下示例中，算子中有两个输出依赖计算得出，输出tensor的数据类型为uint64_t，计算完成后，得到输出的shape为（16, 32）和（1, 16, 16, 32），出参shape_out用于存放该shape信息。示例如下：

```
extern "C" __global__ __aicore__ void xxx_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR  
shape_out, GM_ADDR workspace, GM_ADDR tiling) {  
    ...  
    // 有两个输出需要刷新shape，一个维度为2维[16, 32]，一个维度为4维[1, 16, 16, 32]  
    // 输出tensor类型为uint64_t  
    constexpr uint32_t SHAPEOUT_SIZE_2 = 18;  
    // shapeoutGlobal_uint64_2用于存放输出Shape信息，数据类型固定为uint64_t  
    GlobalTensor<uint64_t> shapeoutGlobal_uint64_2;  
    shapeoutGlobal_uint64_2.SetGlobalBuffer((__gm__ uint64_t*)shape_out, SHAPEOUT_SIZE_2);  
    shapeoutGlobal_uint64_2.SetValue(0, 0x0000000080000000 | 2);  
    shapeoutGlobal_uint64_2.SetValue(1, 16);  
    shapeoutGlobal_uint64_2.SetValue(2, 32);  
    // index[3]~index[8]数据为占位  
    shapeoutGlobal_uint64_2.SetValue(9, 0x0000000080000000 | 4);  
    shapeoutGlobal_uint64_2.SetValue(10, 1);  
    shapeoutGlobal_uint64_2.SetValue(11, 16);  
    shapeoutGlobal_uint64_2.SetValue(12, 16);  
    shapeoutGlobal_uint64_2.SetValue(13, 32);  
    ...  
}
```

2.10.2.5 Host 侧 Tiling 实现

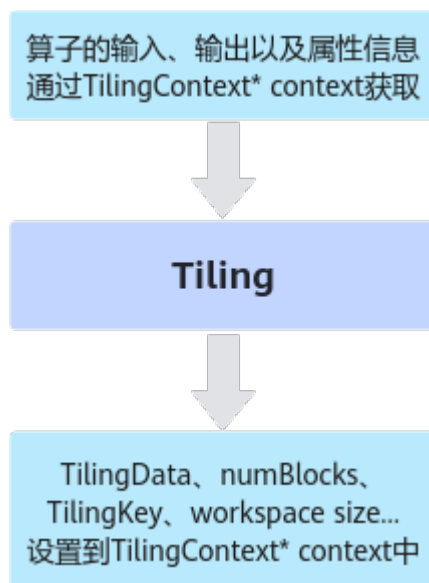
2.10.2.5.1 基本流程

在[3.3 SIMD算子实现](#)章节已经介绍了host侧Tiling核心的实现方法，本章节侧重于介绍接入CANN框架时编程模式和API的使用。

大多数情况下，Local Memory的存储，无法完整的容纳算子的输入与输出，需要每次搬运一部分输入进行计算然后搬出，再搬运下一部分输入进行计算，直到得到完整的最终结果，这个数据切分、分块计算的过程称之为**Tiling**。根据算子的shape等信息来确定数据切分算法相关参数（比如每次搬运的块大小，以及总共循环多少次）的计算程序，称之为**Tiling实现**。

Tiling实现完成后，获取到的Tiling切分算法相关参数，会传递给kernel侧，用于指导并行数据的切分。由于Tiling实现中完成的均为标量计算，AI Core并不擅长，所以我们将其独立出来放在host CPU上执行。

图 2-36 Tiling 实现的输入输出



如上图所示，Tiling实现即为根据算子shape等信息来确定切分算法相关参数的过程，这里的算子shape等信息可以理解为是**Tiling实现的输入**，切分算法相关参数可以理解为是**Tiling实现的输出**。输入和输出都通过Tiling函数的参数（TilingContext* context 上下文结构）来承载。也就是说，开发者可以从上下文结构中获取算子的输入、输出以及属性信息，也就是**Tiling实现的输入**，经过Tiling计算后，获取到TilingData数据结构（切分算法相关参数）、numBlocks变量、用于选择不同的kernel实现分支的TilingKey、算子workspace的大小，也就是**Tiling实现的输出**，并将这些输出设置到上下文结构中。

TilingData、numBlocks、TilingKey、workspace这些概念的具体解释如下：

- **TilingData**：切分算法相关参数，比如每次搬运的块大小，以及总共循环多少次，通过结构体存储，由开发者自行设计。

TilingData结构定义支持单结构定义方法，也支持结构体嵌套：

- 单结构定义方法，以平铺的形式定义：

```
namespace optiling {  
BEGIN_TILING_DATA_DEF(MyAddTilingData) // 声明tiling结构名字  
    TILING_DATA_FIELD_DEF(uint32_t, field1); // 结构成员的类型和名字  
    TILING_DATA_FIELD_DEF(uint32_t, field2);  
    TILING_DATA_FIELD_DEF(uint32_t, field3);  
END_TILING_DATA_DEF;  
  
REGISTER_TILING_DATA_CLASS(MyAdd, MyAddTilingData) // tiling结构注册给算子  
}
```

Tiling实现函数中对tiling结构成员赋值的方式如下：

```
MyAddTilingData myTiling;  
myTiling.set_field1(1);  
myTiling.set_field2(2);
```

- 支持结构体嵌套：

```
namespace optiling {  
BEGIN_TILING_DATA_DEF(MyStruct1) // 声明结构1名字  
    TILING_DATA_FIELD_DEF(uint32_t, field1); // 结构成员的类型和名字  
    TILING_DATA_FIELD_DEF(uint32_t, field2); // 结构成员的类型和名字  
END_TILING_DATA_DEF;  
REGISTER_TILING_DATA_CLASS(MyStruct1Op, MyStruct1) // 注册结构体到<op_type>Op  
  
BEGIN_TILING_DATA_DEF(MyStruct2) // 声明结构2名字
```

```
TILING_DATA_FIELD_DEF(uint32_t, field3); // 结构成员的类型和名字
TILING_DATA_FIELD_DEF(uint32_t, field4); // 结构成员的类型和名字
END_TILING_DATA_DEF;
REGISTER_TILING_DATA_CLASS(MyStruct2Op, MyStruct2) // 注册结构体到<op_type>Op

BEGIN_TILING_DATA_DEF(MyAddTilingData) // 声明tiling结构名字
TILING_DATA_FIELD_DEF_STRUCT(MyStruct1, st1); // 结构成员的引用结构体
TILING_DATA_FIELD_DEF_STRUCT(MyStruct2, st2); // 结构成员的引用结构体
END_TILING_DATA_DEF;
REGISTER_TILING_DATA_CLASS(MyAdd, MyAddTilingData) // tiling结构注册给算子
}
```

Tiling实现函数中对tiling结构成员赋值的方式如下：

```
MyAddTilingData myTiling;
myTiling.st1.set_field1(1);
myTiling.st1.set_field2(2);
myTiling.st2.set_field3(3);
myTiling.st2.set_field4(4);
```

- **numBlocks**：规定了核函数将会在几个核上执行。例如，需要计算8M的数据，每个核上计算1M的数据，numBlocks设置为8，但是为了充分利用硬件资源，一般将numBlocks设置为硬件平台的核数，根据核数进行数据切分。

📖 说明

numBlocks是逻辑核的概念，取值范围为[1,65535]。为了充分利用硬件资源，一般设置为物理核的核数或其倍数。

- 对于耦合模式和分离模式，numBlocks在运行时的意义和设置规则有一些区别，具体说明如下：
 - 耦合模式：由于其Vector、Cube单元是集成在一起的，numBlocks用于设置启动多个AI Core核实例执行，不区分Vector、Cube。AI Core的核数可以通过GetCoreNumAiv或者GetCoreNumAic获取。
 - 分离模式
 - 针对仅包含Vector计算的算子，numBlocks用于设置启动多少个Vector（AIV）实例执行，比如某款AI处理器上有40个Vector核，建议设置为40。
 - 针对仅包含Cube计算的算子，numBlocks用于设置启动多少个Cube（AIC）实例执行，比如某款AI处理器上有20个Cube核，建议设置为20。
 - 针对Vector/Cube融合计算的算子，启动时，按照AIV和AIC组合启动，numBlocks用于设置启动多少个组合执行，比如某款AI处理器上有40个Vector核和20个Cube核，一个组合是2个Vector核和1个Cube核，建议设置为20，此时会启动20个组合，即40个Vector核和20个Cube核。**注意：该场景下，设置的numBlocks逻辑核的核数不能超过物理核（2个Vector核和1个Cube核组合为1个物理核）的核数。**
 - AIC/AIV的核数分别通过GetCoreNumAic和GetCoreNumAiv接口获取。
- 如果开发者使用了Device资源限制特性，那么算子设置的numBlocks不应超过PlatformAscendC提供核数的API（GetCoreNum/GetCoreNumAic/GetCoreNumAiv等）返回的核数。例如，使用aclrtSetStreamResLimit设置Stream级别的Vector核数为8，那么GetCoreNumAiv接口返回值为8，针对Vector算子设置的numBlocks不应超过8，否则会抢占其他Stream的资源，导致资源限制失效。
- **TilingKey（可选）**：TilingKey是一个算子内为了区分不同的实现而将kernel代码进行区分的方法，该方法类似于C++的Template模板机制，可减少不必要的icache miss以及scalar耗时，有助于优化单次调用kernel的性能。不同的kernel实现分支可以通过TilingKey来标识，host侧设置TilingKey后，可以选择对应的分支。例如，一个算子在不同的shape下，有不同的算法逻辑，kernel侧可以通过TilingKey来选择不同的算法逻辑，在host侧Tiling算法也有差异，host/kernel侧通过相同的TilingKey进行关联。

假如有如下kernel代码：

```
if (condition) {
    ProcessA();
}
```



```
} else {  
    ProcessB();  
}
```

如果函数ProcessA、ProcessB两个函数是个非常大的函数，那么上述代码在编译后会变得更大，而每次kernel运行只会选择1个分支，条件的判断和跳转在代码大到一定程度（16-32K，不同芯片存在差异）后会出现icache miss。通过TilingKey可以对这种情况进行优化，给2个kernel的处理函数设置不同的TilingKey 1和2：

```
if (TILING_KEY_IS(1)) {  
    ProcessA();  
} else if (TILING_KEY_IS(2)) {  
    ProcessB();  
}
```

这样device kernel编译时会自动识别到2个TilingKey并编译2个kernel入口函数，将条件判断进行常量折叠。同时需要和host tiling函数配合，判断走ProcessA的场景设置TilingKey为1，走ProcessB的场景设置TilingKey为2：

```
static ge::graphStatus TilingFunc(gert::TilingContext* context)  
{  
    // some code  
    if (condition) {  
        context->SetTilingKey(1);  
    } else {  
        context->SetTilingKey(2);  
    }  
    return ge::GRAPH_SUCCESS;  
}
```

📖 说明

编译时，可以通过设置`--tiling_key`编译选项指定TilingKey，编译时只编译指定TilingKey相关的kernel代码，用于加速编译过程。

- **WorkspaceSize**：workspace是设备侧Global Memory上的一块内存。在Tiling函数中可以设置workspace的大小。设置后：单算子API执行场景，可以通过单算子API调用第一段接口获取workspace的大小，然后由开发者申请对应大小的Global Memory；入图场景，框架会根据设置的大小自动申请对应大小的Global Memory。申请workspace后，在算子Kernel实现时，可以使用这块workspace内存。

workspace内存分为两部分：Ascend C API需要的workspace内存和算子实现使用到的workspace内存（按需）。

- Ascend C API需要预留workspace内存

API在计算过程需要一些workspace内存作为缓存，因此算子Tiling函数需要为API预留workspace内存，预留内存大小通过GetLibApiWorkSpaceSize接口获取。

- 算子实现使用到的workspace内存（按需）

算子内部需要通过额外的device内存进行数据交换或者缓存的时候才需要分配，根据算子计算的空间自行分配。

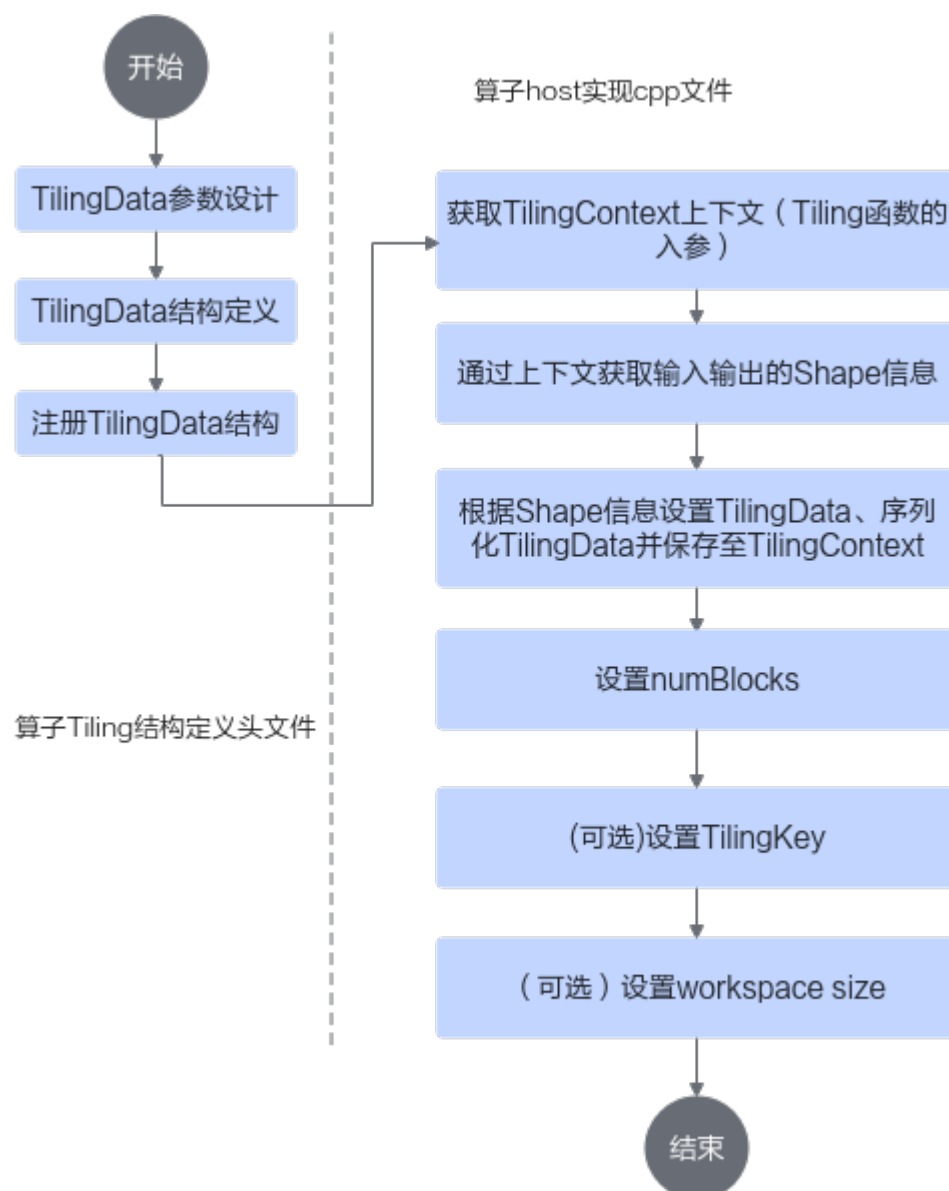
整体的workspace内存就是上述两部分之和，在Tiling函数中设置方法如下：

```
auto workspaceSizes = context->GetWorkspaceSizes(1); // 只使用1块workspace  
workspaceSizes[0] = sysWorkspaceSize + usrWorkspaceSize;
```

Tiling 实现基本流程

Tiling实现开发的流程图如下：

图 2-37 Tiling 开发流程图



下面将从一个简单的 Add 算子为例介绍 Tiling 的实现流程。本样例中待处理数据的 Shape 大小可以平均分配到每个核上，并且可以对齐到一个 datablock（32B）的大小。

首先完成算子 TilingData 结构定义头文件的编写，该文件命名为“算子名称_tiling.h”，位于算子工程的 op_host 目录下。样例代码如下：

```
#ifndef ADD_CUSTOM_TILING_H
#define ADD_CUSTOM_TILING_H
#include "register/tilingdata_base.h"

namespace optiling {
BEGIN_TILING_DATA_DEF(TilingData) // 注册一个tiling的类，以tiling的名字作为入参
    TILING_DATA_FIELD_DEF(uint32_t, totalLength); // 添加tiling字段，总计算数据量
    TILING_DATA_FIELD_DEF(uint32_t, tileNum); // 添加tiling字段，每个核上总计算数据分块个数
END_TILING_DATA_DEF;
// 注册算子tilingdata类到对应的AddCustom算子
REGISTER_TILING_DATA_CLASS(AddCustom, TilingData)
```

```
}  
#endif // ADD_CUSTOM_TILING_H
```

具体的编写步骤如下：

- 步骤1** 代码框架编写，需要增加#ifndef...的判断条件，防止头文件的重复包含；需要包含register/tilingdata_base.h头文件，tilingdata_base.h中定义了多个用于tilingdata注册的宏。样例代码如下：

```
#ifndef ADD_CUSTOM_TILING_H  
#define ADD_CUSTOM_TILING_H  
#include "register/tilingdata_base.h"  
  
namespace optiling {  
// tiling结构定义和注册代码  
// ...  
}  
#endif // ADD_CUSTOM_TILING_H
```

- 步骤2** TilingData参数设计，TilingData参数本质上是和并行数据切分相关的参数，本示例算子使用了2个tiling参数：totalLength、tileNum。totalLength是指需要计算的数据量大小，tileNum是指每个核上总计算数据分块个数。比如，totalLength这个参数传递到kernel侧后，可以通过除以参与计算的核数，得到每个核上的计算量，这样就完成了多核数据的切分。

- 步骤3** TilingData结构定义，通过BEGIN_TILING_DATA_DEF接口定义一个TilingData的类，通过TILING_DATA_FIELD_DEF接口增加TilingData的两个字段totalLength、tileNum，通过END_TILING_DATA_DEF接口结束TilingData定义。相关接口的详细说明请参考TilingData结构定义。

```
BEGIN_TILING_DATA_DEF(TilingData) // 注册一个tiling的类，以tiling的名字作为入参  
TILING_DATA_FIELD_DEF(uint32_t, totalLength); // 添加tiling字段，总计算数据量  
TILING_DATA_FIELD_DEF(uint32_t, tileNum); // 添加tiling字段，每个核上总计算数据分块个数  
END_TILING_DATA_DEF;
```

- 步骤4** 注册TilingData结构，通过REGISTER_TILING_DATA_CLASS接口，注册TilingData类，和自定义算子相关联。REGISTER_TILING_DATA_CLASS第一个参数为op_type(算子类型)，本样例中传入AddCustom，第二个参数为TilingData的类名。REGISTER_TILING_DATA_CLASS接口介绍请参考TilingData结构注册。

```
// 注册算子tilingdata类到对应的AddCustom算子  
REGISTER_TILING_DATA_CLASS(AddCustom, TilingData)
```

----结束

然后完成算子host实现cpp文件中Tiling函数实现，该文件命名为“算子名称.cpp”，位于算子工程的op_host目录下。Tiling函数的原型是固定的，接受一个TilingContext作为输入，在此context上可以获取到输入、输出的Shape指针等内容。注册的Tiling函数由框架调用，调用时会传入TilingContext参数。样例代码如下：

```
namespace optiling {  
const uint32_t NUM_BLOCKS = 8;  
const uint32_t TILE_NUM = 8;  
static ge::graphStatus TilingFunc(ge::TilingContext *context)  
{  
    TilingData tiling;  
    uint32_t totalLength = context->GetInputShape(0)->GetOriginShape().GetShapeSize();  
    context->SetBlockDim(NUM_BLOCKS);  
    tiling.set_totalLength(totalLength);  
    tiling.set_tileNum(TILE_NUM);  
    tiling.SaveToBuffer(context->GetRawTilingData()->GetData(), context->GetRawTilingData()->GetCapacity());  
    context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());  
    size_t *currentWorkspace = context->GetWorkspaceSizes(1);  
    currentWorkspace[0] = 0;  
    return ge::GRAPH_SUCCESS;  
}
```

```
}  
} // namespace optiling
```

具体步骤如下：

步骤1 获取TilingContext的上下文，即Tiling函数的入参gert::TilingContext* context。

步骤2 设置TilingData。在**步骤3**中定义了TilingData类后，可以创建该类的一个实例，并通过调用set_{field_name}方法来设置各个字段值（其中field_name是**步骤3**中定义的tiling字段名）。设置完tiling字段后，通过调用SaveToBuffer方法完成TilingData实例的序列化和保存。

1. 通过上下文获取输入输出shape信息。本样例中通过TilingContext的GetInputShape接口获取输入的shape大小。

```
// 获取输入shape信息  
uint32_t totalLength = context->GetInputShape(0)->GetOriginShape().GetShapeSize();
```

2. 设置TilingData。通过调用set_{field_name}方法来设置TilingData的字段值。

```
// 用TilingData定义一个具体的实例  
TilingData tiling;  
// 设置TilingData  
tiling.set_totalLength(totalLength);  
tiling.set_tileNum(TILE_NUM);
```

3. 调用TilingData类的SaveToBuffer接口完成序列化并保存至TilingContext上下文。SaveToBuffer的第一个参数为存储Buffer的首地址，第二个参数为Buffer的长度。通过调用GetRawTilingData获取无类型的TilingData的地址，再通过GetData获取数据指针，作为Buffer的首地址；通过调用GetRawTilingData获取无类型的TilingData的地址，再通过GetCapacity获取TilingData的长度，作为Buffer的长度。完成SaveToBuffer操作后需要通过SetDataSize设置TilingData的长度，该长度通过TilingData类的GetDataSize接口获取。

```
// 序列化并保存  
tiling.SaveToBuffer(context->GetRawTilingData()->GetData(), context->GetRawTilingData()->GetCapacity());  
context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());
```

步骤3 通过SetBlockDim接口设置numBlocks。

```
context->SetBlockDim(NUM_BLOCKS);
```

步骤4 （可选）通过SetTilingKey设置TilingKey。

```
context->SetTilingKey(1);
```

步骤5 （可选）通过GetWorkspaceSizes获取workspace size指针，并设置size大小。此处仅作为举例，设置workspace的大小为0。

```
size_t *currentWorkspace = context->GetWorkspaceSizes(1);  
currentWorkspace[0] = 0;
```

----结束

2.10.2.5.2 通过 TilingData 传递属性信息

如果算子包含属性信息，该属性信息可以通过TilingData传递到kernel侧，参与kernel侧算子核函数的计算。以ReduceMaxCustom算子为例，该算子用于对输入数据按维度dim返回最大值，并且返回索引。ReduceMaxCustom算子有两个属性，reduceDim和isKeepDim，reduceDim表示按照哪一个维度进行reduce操作；isKeepDim表示是否需要保持输出的维度与输入一样。本样例仅支持对最后一维做reduce操作，输入数据类型为half。

1. ReduceMaxCustom算子TilingData的定义如下：这里我们重点关注reduceAxisLen。参数reduceAxisLen表示获取reduceDim轴的长度，这里也就是最后一维的长度。该参数后续会通过TilingData传递到kernel侧参与计算。

```
#ifndef REDUCE_MAX_CUSTOM_TILING_H
#define REDUCE_MAX_CUSTOM_TILING_H
#include "register/tilingdata_base.h"
namespace optiling {
BEGIN_TILING_DATA_DEF(ReduceMaxTilingData)
    TILING_DATA_FIELD_DEF(uint32_t, reduceAxisLen); // 添加tiling字段, reduceDim轴的长度
    //其他TilingData参数的定义
    ...
END_TILING_DATA_DEF;
// 注册算子tilingdata类到对应的ReduceMaxCustom算子
REGISTER_TILING_DATA_CLASS(ReduceMaxCustom, ReduceMaxTilingData)
}
#endif // REDUCE_MAX_CUSTOM_TILING_H
```

2. ReduceMaxCustom算子的Tiling实现如下。这里我们重点关注属性信息通过TilingData传递的过程：首先通过TilingContext上下文从attr获取reduceDim属性值；然后根据reduceDim属性值获取reduceDim轴的长度并设置到TilingData中。

```
namespace optiling {
static ge::graphStatus TilingFunc(ge::TilingContext* context)
{
    ReduceMaxTilingData tiling;
    // 从attr获取reduceDim属性值, 因为reduceDim是第一个属性, 所以GetAttrPointer传入的索引值为0
    const ge::RuntimeAttrs* attrs = context->GetAttrs();
    const uint32_t* reduceDim = attrs->GetAttrPointer<uint32_t>(0);
    // 获取reduceDim轴的长度
    const ge::StorageShape* xShapePtr = context->GetInputShape(0);
    const ge::Shape& xShape = xShapePtr->GetStorageShape();
    const uint32_t reduceAxisLen = xShape.GetDim(*reduceDim);
    // 计算TilingData中除了reduceAxisLen之外其他成员变量的值
    ...
    // 将reduceAxisLen设置到tiling结构体中, 传递到kernel函数使用
    tiling.set_reduceAxisLen(reduceAxisLen);
    // 设置TilingData中除了reduceAxisLen之外其他成员变量的值
    ...
    // TilingData序列化保存
    tiling.SaveToBuffer(context->GetRawTilingData()->GetData(), context->GetRawTilingData()->GetCapacity());
    context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());
    ...
    return ge::GRAPH_SUCCESS;
} // namespace optiling
```

2.10.2.5.3 使用高阶 API 时配套的 Tiling 实现

1. 首先进行tiling结构定义：

```
namespace optiling {
BEGIN_TILING_DATA_DEF(MyAddTilingData) // 声明tiling结构名字
    TILING_DATA_FIELD_DEF_STRUCT(TCubeTiling, cubeTilingData); // 引用高阶API的tiling结构体
    TILING_DATA_FIELD_DEF(uint32_t, field); // 结构成员的引用结构体
END_TILING_DATA_DEF;
REGISTER_TILING_DATA_CLASS(MyAdd, MyAddTilingData) // tiling结构注册给算子
}
```

2. 通过高阶API配套的tiling函数对tiling结构初始化：

```
static ge::graphStatus TilingFunc(ge::TilingContext* context) {
    int32_t M = 1024;
    int32_t N = 640;
    int32_t K = 256;
    int32_t baseM = 128;
    int32_t baseN = 128;
    auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
    MultiCoreMatmulTiling cubeTiling(ascendcPlatform);
    cubeTiling.SetDim(2);
    cubeTiling.SetAType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT16);
    cubeTiling.SetBType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT16);
    cubeTiling.SetCTYPE(TPosition::LCM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT);
    cubeTiling.SetBiasType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT);
    cubeTiling.SetShape(M, N, K);
    cubeTiling.SetOrgShape(M, N, K);
}
```

```
cubeTiling.SetFixSplit(baseM, baseN, -1);
cubeTiling.SetBias(true);
cubeTiling.SetBufferSpace(-1, -1, -1);
MyAddTilingData tiling;
if (cubeTiling.GetTiling(tiling.cubeTilingData) == -1){
    return ge::GRAPH_FAILED;
}
// some code
}
```

2.10.2.5.4 使用标准 C++语法定义 Tiling 结构体

具体步骤

在定义Tiling结构体时，可以使用标准C++语法定义一个**POD类型（Plain Old Data）**，即与C语言兼容的数据类型。具体步骤如下。完整样例请参考[标准C++语法定义Tiling结构体样例](#)。

步骤1 使用C++语法定义Tiling结构体。

说明

该结构体定义所在的头文件应放置在算子工程的op_kernel目录下。由于只有该目录下的文件会被打包进算子包，供在线编译场景中使用，若将文件放置在其他目录中，可能导致在线编译因找不到相关文件而失败。

用户在使用高阶API的Tiling结构体时，通过AscendC::tiling命名空间引用"kernel_tiling/kernel_tiling.h"中预定义的Tiling结构体，如下代码所示。

```
#ifndef MATMUL_CUSTOM_TILING_H
#define MATMUL_CUSTOM_TILING_H
#include <cstdint>
#include "kernel_tiling/kernel_tiling.h" // for TCubeTiling

struct MatmulCustomTilingData {
    uint64_t localMemSize;
    AscendC::tiling::TCubeTiling cubeTilingData;
};
#endif // MATMUL_CUSTOM_TILING_H
```

步骤2 Host侧Tiling函数中对Tiling结构体赋值。

- 需要包含Tiling结构体定义头文件。
- 通过GetTilingData获取Tiling结构体指针，并对其成员变量进行赋值。

```
#include "../op_kernel/matmul_custom_tiling.h" // 包含Tiling结构体定义头文件
...

namespace optiling {
static ge::graphStatus TilingFunc(ge::TilingContext *context)
{
    ...
    MultiCoreMatmulTiling cubeTiling(ascendcPlatform);
    ...
    // 获取Tiling结构体指针
    MatmulCustomTilingData *tiling = context->GetTilingData<MatmulCustomTilingData>();
    // 对tiling的成员变量赋值
    if (cubeTiling.GetTiling(tiling->cubeTilingData) == -1) {
        return ge::GRAPH_FAILED;
    }
    uint64_t localMemSize;
    ascendcPlatform.GetCoreMemSize(platform_ascend::CoreMemType::UB, localMemSize);
    tiling->localMemSize = localMemSize;
    ...
    return ge::GRAPH_SUCCESS;
}
} // namespace optiling
```

步骤3 Kernel侧注册Tiling结构体，解析Tiling数据至TilingData结构并使用。

- 需要包含Tiling结构体定义头文件。
- 通过REGISTER_TILING_DEFAULT或者REGISTER_TILING_FOR_TILINGKEY注册Tiling结构体；通过GET_TILING_DATA解析Tiling数据至TilingData结构并使用。其中REGISTER_TILING_DEFAULT同时也用于标识使用标准C++语法定义TilingData结构体。

```
#include "kernel_operator.h"
#include "matmul_custom_tiling.h" // 包含Tiling结构体定义头文件

extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR bias,
GM_ADDR c, GM_ADDR workspace, GM_ADDR tiling)
{
    REGISTER_TILING_DEFAULT(MatmulCustomTilingData);
    GET_TILING_DATA(tilingData, tiling);
    MatmulKernel<half, half, float, float> matmulKernel;
    AscendC::TPipe pipe;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), matmulKernel.matmulObj,
    &tilingData.cubeTilingData); // Initialize the matmul object.
    matmulKernel.Init(a, b, bias, c, workspace, tilingData.localMemSize, tilingData.cubeTilingData);
    ...
}
```

----结束

使用标准 C++语法定义 Tiling 结构体的优势

相比较使用BEGIN_TILING_DATA_DEF等宏进行定义的方式，该方式不仅更符合C++开发者的开发习惯，并且提供了强大的灵活性。

- 支持bool类型，支持数组、结构体数组及列表初始化。

```
class A {
public:
    bool xxx;
    uint32_t xxx[2][128] = {0};
};

class B {
public:
    bool xxx = false;
    uint8_t xxx[2][2]{0};
    A[10];
};
```

- 不同算子可以支持定义同名但结构不同的Tiling结构体，通过算子引用对应的头文件即可实现区分。这种方式允许每个算子使用符合自身需求的Tiling结构定义，而不会与其他算子产生冲突。

相比之下，使用BEGIN_TILING_DATA_DEF等宏方式定义同名但结构不同的Tiling结构体时，由于这些结构体会被注册到全局的Tiling结构体管理变量中，可能导致后续通过结构体名称访问时，无法准确获取当前算子实际使用的Tiling结构体，从而引发未定义行为。

算子A:

```
class TilingData {
public:
    uint32_t length;
};
```

算子B:

```
class TilingData {
public:
    uint32_t length;
    uint16_t coreNum;
};
```

- 支持自定义Tiling赋值，用户可以通过访问Tiling结构体成员变量直接赋值，或自定义Tiling赋值函数（宏定义方式下，用户仅可通过框架生成的set_xx/get_xx方法赋值/访问）

Tiling结构体定义：

```
class TilingData {
public:
    uint32_t xxx1;
    uint32_t xxx2;
    uint8_t xxx3;
    bool xxx4;
};
```

Host侧Tiling函数：

```
#include "../op_kernel/xxx_custom_tiling.h" // 包含Tiling结构体定义头文件
...

namespace optiling {
static void ComputeTiling(TilingData* tiling, ...)
{
    // 计算Tiling逻辑
    ...
    tiling->xxx1 = xxx;
    tiling->xxx2 = xxx;
    tiling->xxx3 = xxx;
    tiling->bool = xxx;
}
static ge::graphStatus TilingFunc(gert::TilingContext *context)
{
    ...
    TilingData *tiling = context->GetTilingData<TilingData>();
    ...
    ComputeTiling(tiling, ...)
    ...

    return ge::GRAPH_SUCCESS;
}
} // namespace optiling
```

使用约束

使用标准C++语法定义Tiling结构体时存在如下约束限制：

- Tiling结构体内不支持定义成员函数，因为成员函数存在Device侧和Host侧的差异（Device侧的函数需要__aicore__修饰符），而Tiling结构体Device侧和Host侧共用，所以会在编译或执行时出现问题：

```
class TilingData {
public:
    uint32_t xxx;

    __aicore__ funcA() { ... } // 错误，host侧编译时不支持__aicore__修饰符，会出现编译错误
    void func() { ... }        // 错误，device侧缺少__aicore__修饰符，无法执行
};
```

- Tiling结构体成员变量不支持指针、引用类型，此类数据类型会导致Host侧到Device侧数据解析异常：

```
class TilingData {
public:
    uint32_t* totalLength; // 指针场景不支持，Host无法传递指针到Device
    uint32_t& tileNum;      // 引用场景不支持，Host无法传递指针到Device
};
```

- Tiling结构体仅支持POD类型，没有虚函数、虚继承等面向对象特性，也不支持模板类：

```
class A {
public:
    uint32_t totalLength;
    uint32_t tileNum;
```



```
};  
class B: public A {  
public:  
    uint32_t xxx;  
    uint32_t xxx;  
};  
static ge::graphStatus TilingFunc(ge::TilingContext* context)  
{  
    // 错误用法  
    B *tiling = context->GetTilingData<A>(); // 不支持，会触发未知问题  
    // 正确用法  
    B *tiling = context->GetTilingData<B>();  
    .....  
    return ge::GRAPH_SUCCESS;  
}
```

- GetTilingData获取的Tiling不包含初值，需显式赋值或在Tiling结构体定义并调用Tiling赋值函数；

```
static ge::graphStatus TilingFunc(ge::TilingContext* context)  
{  
    TilingData *tiling = context->GetTilingData<TilingData>(); //获取Tiling结构体，此时totalLength、  
    tileNum为0，并不会带入初始值  
    .....  
    // 需显式赋值  
    tiling->totalLength = totalLength; // 赋值Tiling结构体成员变量  
    tiling->tileNum = TILE_NUM;      // 赋值Tiling结构体成员变量  
    .....  
    return ge::GRAPH_SUCCESS;  
}
```

如何将宏定义 Tiling 结构体修改为标准 C++语法

本节介绍如何将使用BEGIN_TILING_DATA_DEF等宏进行定义的方式改造成使用标准C++语法的方式。

- 步骤1** 首先将之前位于op_host目录下的Tiling结构体定义头文件移至op_kernel目录下，内容前后对比如下，注意此时包含的头文件变化，不需要再包含宏定义相关的头文件。

表 2-26 两种方式对比

宏定义方式	标准C++语法定义方式
<pre>#include "register/tilingdata_base.h" #include "tiling/tiling_api.h" // TCubeTiling结构体 通过宏定义 namespace optiling { BEGIN_TILING_DATA_DEF(MatmulCustomTilingData) TILING_DATA_FIELD_DEF(uint64_t, localMemSize); TILING_DATA_FIELD_DEF_STRUCT(TCubeTiling, cubeTilingData); END_TILING_DATA_DEF; REGISTER_TILING_DATA_CLASS(MatmulCustom, MatmulCustomTilingData) } // namespace optiling</pre>	<pre>#include <cstdint> #include "kernel_tiling/kernel_tiling.h" // TCubeTiling结构体通过C++语法定义 struct MatmulCustomTilingData { uint64_t localMemSize; AscendC::tiling::TCubeTiling cubeTilingData; };</pre>

- 步骤2** 然后修改Host侧的Tiling函数实现，此时对Tiling结构体的成员变量赋值无需使用宏定义生成的set方法，而是使用用户熟悉的C++指针赋值方式。

表 2-27 两种方式对比

宏定义方式	标准C++语法定义方式
<pre>namespace optiling { static ge::graphStatus TilingFunc(gert::TilingContext *context) { ... MultiCoreMatmulTiling cubeTiling(ascendcPlatform); ... MatmulCustomTilingData tiling; if (cubeTiling.GetTiling(tiling.cubeTilingData) == -1) { // Get matmul tiling. return ge::GRAPH_FAILED; } uint64_t localMemSize; ascendcPlatform.GetCoreMemSize(platform_ascen dc::CoreMemType::UB, localMemSize); tiling.set_localMemSize(localMemSize); // 需要 使用宏定义方式生成的set方法 ... // 需要将局部变量保存至context上下文 tiling.SaveToBuffer(context- >GetRawTilingData()->GetData(), context- >GetRawTilingData()->GetCapacity()); ... return ge::GRAPH_SUCCESS; } } // namespace optiling</pre>	<pre>#include "../op_kernel/matmul_custom_tiling.h" // 包含Tiling结构体定义头文件 ... namespace optiling { static ge::graphStatus TilingFunc(gert::TilingContext *context) { ... MultiCoreMatmulTiling cubeTiling(ascendcPlatform); ... MatmulCustomTilingData *tiling = context- >GetTilingData<MatmulCustomTilingData>(); if (cubeTiling.GetTiling(tiling->cubeTilingData) == -1) { return ge::GRAPH_FAILED; } uint64_t localMemSize; ascendcPlatform.GetCoreMemSize(platform_ascen dc::CoreMemType::UB, localMemSize); tiling->localMemSize = localMemSize; // 使用用 户友好的C++指针方式赋值成员变量 ... return ge::GRAPH_SUCCESS; } } // namespace optiling</pre>

步骤3 最后，在Kernel 函数入口处新增REGISTER_TILING_DEFAULT调用，用于注册Tiling结构体。该注册操作的作用是：告知框架用户已使用标准 C++ 语法定义Tiling结构体，并明确其类型，以便框架在进行Tiling数据解析时能够正确识别和使用该结构体。

```
#include "matmul_custom_tiling.h"
...

extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR bias,
GM_ADDR c, GM_ADDR workspace, GM_ADDR tiling)
{
    REGISTER_TILING_DEFAULT(MatmulCustomTilingData); // 新增REGISTER_TILING_DEFAULT调用注册Tiling
    结构体
    ...
}
```

----结束

2.10.2.5.5 Tiling 模板编程

在[TilingKey编程](#)章节介绍的TilingKey编程方式中，TilingKey不易于记忆和理解，因为它们通常是较长又没有明确含义的数字。

在涉及多个TilingKey的场景中，开发者依赖TilingKey来管理kernel的实现，无论是在管理还是使用上都会遇到相当大的复杂性。为了简化这一过程，可以采用模板编程的方法来替代传统的TilingKey编程，从而减少对TilingKey数值标识的依赖，使kernel的管理更加直观和高效。使用步骤如下：

步骤1 在[自定义算子工程](#)的op_kernel目录下，新增定义模板参数和模板参数组合的头文件，本示例中头文件命名为tiling_key_add_custom.h。

- 该头文件中需要包含模板头文件ascendc/host_api/tiling/template_argument.h。
- 定义模板参数ASCENDC_TPL_ARGS_DECL和模板参数组合ASCENDC_TPL_ARGS_SEL（即可使用的模板）。具体API参考见模板参数定义。

```
#include "ascendc/host_api/tiling/template_argument.h"

// 模板参数
ASCENDC_TPL_ARGS_DECL(AddCustomTemplate, // 算子OpType
ASCENDC_TPL_DATATYPE_DECL(D_T_X, C_DT_FLOAT, C_DT_FLOAT16, ASCENDC_TPL_INPUT(0)), //
DataType类型的模板参数定义：输入参数x的数据类型，取值范围为float16/float32, ASCENDC_TPL_INPUT(0)说明
对应Kernel侧第0个输入
ASCENDC_TPL_DATATYPE_DECL(D_T_Y, C_DT_FLOAT, C_DT_FLOAT16, ASCENDC_TPL_INPUT(1)), //
DataType类型的模板参数定义：输入参数y的数据类型，取值范围为float16/float32, ASCENDC_TPL_INPUT(1)说明
对应Kernel侧第1个输入
ASCENDC_TPL_DATATYPE_DECL(D_T_Z, C_DT_FLOAT, C_DT_FLOAT16, ASCENDC_TPL_OUTPUT(0)), //
DataType类型的模板参数定义：输入参数z的数据类型，取值范围为float16/float32, ASCENDC_TPL_OUTPUT(0)
说明对应Kernel侧第0个输出
ASCENDC_TPL_UINT_DECL(TILE_NUM, ASCENDC_TPL_8_BW, ASCENDC_TPL_UI_MIX, 2, 0, 2, 3, 5, 10, 12, 13,
9, 8), // 自定义UINT类型（无符号整形）的模板参数定义：模板参数为切分的块数，编码位宽为
ASCENDC_TPL_8_BW即8比特，表示该模板参数的个数不超过8比特能表达的范围；ASCENDC_TPL_UI_MIX表示
通过混合模式表达取值范围，有2组的数据{0-2}、{3-5}和穷举值10、12、13、9、8，最后结果为{0, 1, 2, 3, 4,
5, 10, 12, 13, 9, 8}
ASCENDC_TPL_BOOL_DECL(IS_SPLIT, 0, 1), // 自定义bool类型的模板参数定义：模板参数为是否切分标志位，
取值范围为0和1，1表示切分，0表示不切分
);

// 模板参数组合
// 用于调用GET_TPL_TILING_KEY获取TilingKey时，接口内部校验TilingKey是否合法
ASCENDC_TPL_SEL(
ASCENDC_TPL_ARGS_SEL(
ASCENDC_TPL_KERNEL_TYPE_SEL(ASCENDC_TPL_AIV_ONLY), // Kernel类型选择，无需在模板参数声明中定
义，在SEL中直接配置，所有ASCENDC_TPL_ARGS_SEL是否配置需要保持统一，如不配置将走自动推导流程
ASCENDC_TPL_DATATYPE_SEL(D_T_X, C_DT_FLOAT16),
ASCENDC_TPL_DATATYPE_SEL(D_T_Y, C_DT_FLOAT16),
ASCENDC_TPL_DATATYPE_SEL(D_T_Z, C_DT_FLOAT16),
ASCENDC_TPL_UINT_SEL(TILE_NUM, ASCENDC_TPL_UI_LIST, 1, 8),
ASCENDC_TPL_BOOL_SEL(IS_SPLIT, 0, 1)
),
ASCENDC_TPL_ARGS_SEL(
ASCENDC_TPL_KERNEL_TYPE_SEL(ASCENDC_TPL_AIV_ONLY),
ASCENDC_TPL_DATATYPE_SEL(D_T_X, C_DT_FLOAT),
ASCENDC_TPL_DATATYPE_SEL(D_T_Y, C_DT_FLOAT),
ASCENDC_TPL_DATATYPE_SEL(D_T_Z, C_DT_FLOAT),
ASCENDC_TPL_UINT_SEL(TILE_NUM, ASCENDC_TPL_UI_LIST, 1, 8),
ASCENDC_TPL_BOOL_SEL(IS_SPLIT, 0, 1)
),
);
```

步骤2 host侧调用ASCENDC_TPL_SEL_PARAM接口自动生成并配置TilingKey。

- host实现文件中包含步骤1中定义模板参数和模板参数组合的头文件。
- 调用ASCENDC_TPL_SEL_PARAM接口自动生成并配置TilingKey，ASCENDC_TPL_SEL_PARAM输入参数为模板参数的具体值，传入时需要与定义模板参数和模板参数组合的头文件中的模板参数顺序保持一致。

```
#include "tiling_key_add_custom.h"
static ge::graphStatus TilingFunc(gert::TilingContext *context)
{
    TilingData tiling;
    uint32_t totalLength = context->GetInputShape(0)->GetOriginShape().GetShapeSize();
    ge::DataType dtype_x = context->GetInputDesc(0)->GetDataType();
    ge::DataType dtype_y = context->GetInputDesc(1)->GetDataType();
    ge::DataType dtype_z = context->GetOutputDesc(1)->GetDataType();
    uint32_t D_T_X = static_cast<int>(dtype_x), D_T_Y = static_cast<int>(dtype_y), D_T_Z =
static_cast<int>(dtype_z), TILE_NUM = 1, IS_SPLIT = 0;
    if(totalLength < MIN_LENGTH_FOR_SPLIT){
        IS_SPLIT = 0;
        TILE_NUM = 1;
    }
```

```
    }else{
        IS_SPLIT = 1;
        TILE_NUM = DEFAULT_TILE_NUM;
    }
    context->SetBlockDim(NUM_BLOCKS);
    tiling.set_totalLength(totalLength);
    tiling.SaveToBuffer(context->GetRawTilingData()->GetData(), context->GetRawTilingData()->GetCapacity());
    context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());
    ASCENDC_TPL_SEL_PARAM(context, D_T_X, D_T_Y, D_T_Z, TILE_NUM, IS_SPLIT);
    size_t *currentWorkspace = context->GetWorkspaceSizes(1);
    currentWorkspace[0] = 0;
    return ge::GRAPH_SUCCESS;
}
```

步骤3 kernel侧实现

- kernel实现文件中包含[步骤1](#)中定义模板参数和模板参数组合的头文件。
- 核函数添加template模板，以便支持模板参数的传入，参数顺序需要与定义模板参数和模板参数组合的头文件中的模板参数顺序保持一致。
- 通过对模板参数的分支判断，选择不同的kernel侧实现。

```
#include "tiling_key_add_custom.h"
...
...
template<typename D_T_X, typename D_T_Y, typename D_T_Z, int TILE_NUM, int IS_SPLIT>
__global__ __aicore__ void add_custom_template(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR workspace, GM_ADDR tiling)
{
    GET_TILING_DATA(tiling_data, tiling);
    KernelAdd<D_T_X, D_T_Y, D_T_Z> op;
    op.Init(x, y, z, tiling_data.totalLength, TILE_NUM);
    if constexpr (std::is_same_v<D_T_X, float> && std::is_same_v<D_T_Y, float> && std::is_same_v<D_T_Z, float>) {
        op.Process1();
    } else if constexpr (std::is_same_v<D_T_X, half> && std::is_same_v<D_T_Y, half> && std::is_same_v<D_T_Z, half>){
        if (IS_SPLIT == 0) {
            op.Process1();
        } else if (IS_SPLIT == 1) {
            op.Process2();
        }
    }
}
```

----结束

说明

完整样例请参考[Tiling模板编程样例](#)。

2.10.2.6 算子包编译

2.10.2.6.1 算子工程编译

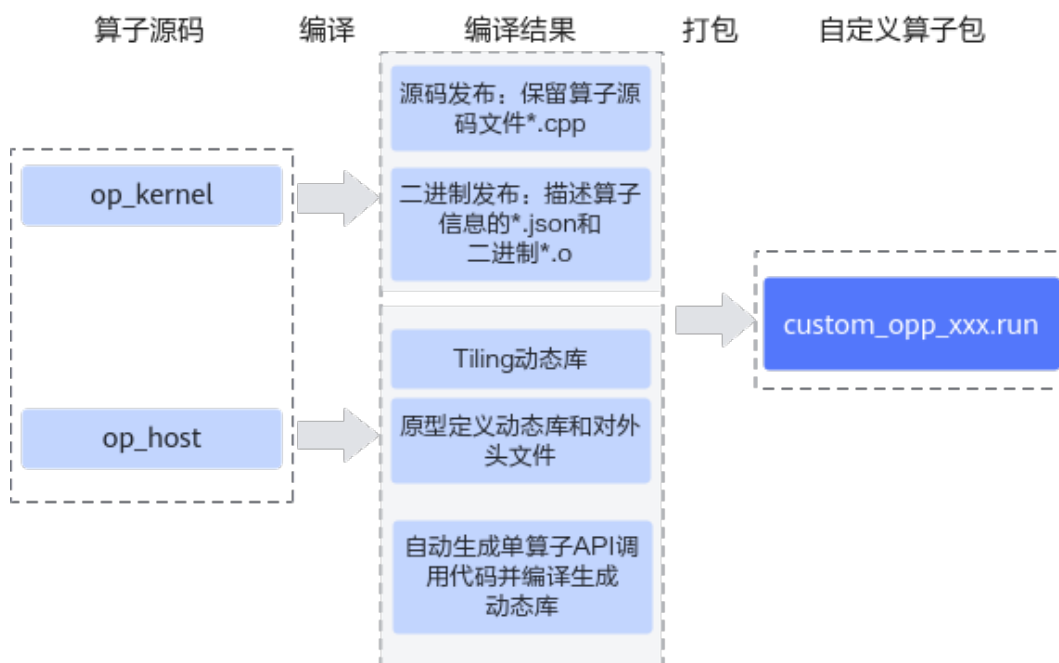
算子kernel侧和host侧实现开发完成后，需要对算子工程进行编译，生成自定义算子安装包*.run，详细的编译操作包括：

- 编译Ascend C算子kernel侧代码实现文件*.cpp，分为源码发布和二进制发布两种方式。
 - **源码发布：**不对算子kernel侧实现进行编译，保留算子kernel源码文件*.cpp。该方式可以支持算子的在线编译、通过ATC模型转换的方式编译算子的场景。

- **二进制发布**：对算子kernel侧实现进行编译，生成描述算子相关信息的json文件*.json和算子二进制文件*.o。算子调用时，如果需要直接调用算子二进制，则使用该编译方式，比如通过[2.10.2.9 单算子API调用](#)的方式完成单算子的调用，PyTorch框架中单算子调用的场景，动态网络中调用算子的场景。
- 编译Ascend C算子host侧代码实现文件*.cpp、*.h。
 - 将原型定义和shape推导实现编译成算子原型定义动态库libcust_opsproto_*.so，并生成算子原型对外接口op_proto.h。
 - 将Tiling实现编译成Tiling动态库liboptiling.so等。
 - 基于算子原型定义，自动生成单算子API调用代码和头文件aclnn_*.h，并编译生成单算子API调用的动态库libcust_opapi.so。

上述编译过程示意图如下：

图 2-38 算子工程编译示意图



编译步骤

步骤1 完成工程编译相关配置。

- 修改工程目录下的CMakePresets.json cacheVariables的配置项。CMakePresets.json文件内容如下，需要配置的参数请参考[表2-28](#)，其他参数会在工程创建时自动生成。

```
{
  "version": 1,
  "cmakeMinimumRequired": {
    "major": 3,
    "minor": 19,
    "patch": 0
  },
  "configurePresets": [
    {
      "name": "default",
      "displayName": "Default Config",
      "description": "Default build using Unix Makefiles generator",
    }
  ]
}
```

```
"generator": "Unix Makefiles",
"binaryDir": "${sourceDir}/build_out",
"cacheVariables": {
  "CMAKE_BUILD_TYPE": {
    "type": "STRING",
    "value": "Release"
  },
  "ENABLE_SOURCE_PACKAGE": {
    "type": "BOOL",
    "value": "True"
  },
  "ENABLE_BINARY_PACKAGE": {
    "type": "BOOL",
    "value": "True"
  },
  "ASCEND_COMPUTE_UNIT": {
    "type": "STRING",
    "value": "ascendxxx"
  },
  "ENABLE_TEST": {
    "type": "BOOL",
    "value": "True"
  },
  "vendor_name": {
    "type": "STRING",
    "value": "customize"
  },
  "ASCEND_PYTHON_EXECUTABLE": {
    "type": "STRING",
    "value": "python3"
  },
  "CMAKE_INSTALL_PREFIX": {
    "type": "PATH",
    "value": "${sourceDir}/build_out"
  },
  "ENABLE_CROSS_COMPILE": {
    "type": "BOOL",
    "value": "False"
  },
  "CMAKE_CROSS_PLATFORM_COMPILER": {
    "type": "PATH",
    "value": "/usr/bin/aarch64-linux-gnu-g++"
  },
  "ASCEND_PACK_SHARED_LIBRARY": {
    "type": "BOOL",
    "value": "False"
  }
}
```

表 2-28 需要开发者配置的参数列表

参数名称	参数描述	默认值
CMAKE_BUILD_TYPE	编译模式选项，可配置为： “Release”，Release版本，不包含调试信息，编译最终发布的版本。 “Debug”，“Debug”版本，包含调试信息，便于开发者开发和调试。	“Release”

参数名称	参数描述	默认值
ENABLE_SOURCE_PACKAGE	是否开启源码编译	“True”
ENABLE_BINARY_PACKAGE	是否开启二进制编译	“True”
vendor_name	标识自定义算子所属厂商的名称。建议开发者自行指定所属厂商名称，避免和其他厂商提供的算子包冲突。	“customize”
ASCEND_PACK_SHARED_LIBRARY	是否开启动态库编译功能。	“False”

- 配置编译相关环境变量（可选）

表 2-29 环境变量说明

环境变量	配置说明
CMAKE_CXX_COMPILER_LAUNCHER	用于配置C++语言编译器（如g++）、毕昇编译器的启动器程序为ccache，配置后即可开启cache缓存编译，加速重复编译并提高构建效率。使用该功能前需要安装ccache。 配置方法如下，在对应的CMakeLists.txt进行设置： set(CMAKE_CXX_COMPILER_LAUNCHER <launcher_program>) 其中<launcher_program>是ccache的安装路径，比如ccache的安装路径为/usr/bin/ccache，示例如下： set(CMAKE_CXX_COMPILER_LAUNCHER /usr/bin/ccache)

步骤2 在算子工程目录下执行如下命令，进行算子工程编译。

./build.sh

编译成功后，会在当前目录下创建build_out目录，并在build_out目录下生成自定义算子安装包**custom_opp_<target os>_<target architecture>.run**。

用户如果需要编译过程日志存盘，可以使用环境变量ASCENDC_BUILD_LOG_DIR来控制存储路径。用户设置该选项之后，如果编译过程中无错误产生，则对应的log文件后缀会添加“_success”，若编译过程有错误产生，则会在屏幕打印对应的报错信息，以及指示用户log文件的具体路径与文件名，同时，对应log文件后缀会添加“_error”。

```
# 如希望编译日志存储在/home/build_log/，则可以按照如下设置，默认不打开日志存储
export ASCENDC_BUILD_LOG_DIR=/home/build_log/
```

----结束

算子包交叉编译

完成算子代码实现后，如果当前平台架构和运行环境一致则参考上一节的内容进行编译即可，如果需要实现算子包的交叉编译，您可以参考如下流程。

步骤1 交叉编译工具下载，下表以Ubuntu系列操作系统为例，展示了编译工具下载命令的样例。其他操作系统，请替换为实际的下载命令。

表 2-30 Ubuntu 系列操作系统交叉编译工具下载命令样例

当前平台架构	运行环境平台架构	编译工具下载命令
x86_64	aarch64	sudo apt-get install -y g++-aarch64-linux-gnu
aarch64	x86_64	sudo apt-get install g++-x86-64-linux-gnu

步骤2 自定义算子工程交叉编译，构建生成自定义算子包。

1. 修改CMakePresets.json中**ENABLE_CROSS_COMPILE**为True，使能交叉编译。

```
"ENABLE_CROSS_COMPILE": {  
  "type": "BOOL",  
  "value": "True"  
}
```

2. 修改CMakePresets.json中**CMAKE_CROSS_PLATFORM_COMPILER**为安装后的交叉编译工具路径。

```
"CMAKE_CROSS_PLATFORM_COMPILER": {  
  "type": "PATH",  
  "value": "/usr/bin/aarch64-linux-gnu-g++"  
}
```

3. 在算子工程目录下执行如下命令，进行算子工程交叉编译。

./build.sh

编译成功后，会在当前目录下创建build_out目录，并在build_out目录下生成自定义算子安装包**custom_opp_<target os>_<target architecture>.run**

----结束

支持自定义编译选项

在算子工程中，如果开发者想对算子kernel侧代码增加一些自定义的编译选项，可以参考如下内容进行编译选项的定制。

修改算子工程op_kernel目录下的CMakeLists.txt，使用add_ops_compile_options来增加编译选项，方法如下：

```
add_ops_compile_options(OpType COMPUTE_UNIT soc_version1 soc_version2 ... OPTIONS option1 option2 ...)
```

具体参数的介绍如下：

表 2-31 具体参数介绍

参数名称	可选/必选	参数描述
OpType（算子类型）	必选	第一个参数应传入算子类型，如果需要对算子工程中的所有算子生效，需要配置为ALL。

参数名称	可选/必选	参数描述
COMPUTE_UNIT	可选	标识编译选项在哪些AI处理器型号上生效，多个型号之间通过空格间隔。不配置时表示对所有AI处理器型号生效。 说明 COMPUTE_UNIT具体配置如下： - 针对如下产品：在安装AI处理器的服务器执行npusmi info命令进行查询，获取Name信息。实际配置值为AscendName，例如Name取值为xxxxyy，实际配置值为Ascendxxxxyy。 Atlas A2 训练系列产品/Atlas A2 推理系列产品 Atlas 200I/500 A2 推理产品 Atlas 推理系列产品 Atlas 训练系列产品 - 针对如下产品，在安装AI处理器的服务器执行npusmi info -t board -i id -c chip_id命令进行查询，获取Chip Name和NPU Name信息，实际配置值为Chip Name_NPU Name。例如Chip Name取值为Ascendxxx，NPU Name取值为1234，实际配置值为Ascendxxx_1234。其中： - id：设备id，通过npusmi info -l命令查出的NPU ID即为设备id。 - chip_id：芯片id，通过npusmi info -m命令查出的Chip ID即为芯片id。 Atlas A3 训练系列产品/Atlas A3 推理系列产品

参数名称	可选/必选	参数描述
OPTIONS	必选	自定义的编译选项。多个编译选项之间通过空格间隔。 - 增加-D编译选项，用于在编译时定义宏。 OPTIONS -Dname=definition - 增加-g -O0等调试用编译选项。 - 支持传入毕昇编译器编译选项： 比如--cce-auto-sync=off，设置该选项可以关闭自动同步功能，自定义算子工程已默认开启，通常无需开发者手动设置。详细内容参见支持自动同步。 更多编译选项可以参考毕昇编译器编译选项。 - Ascend C框架提供的编译选项介绍如下： --tiling_key，设置该选项后，只编译指定的TilingKey相关的kernel代码，用于加速编译过程。若不指定TilingKey编译，则默认编译所有的TilingKey。配置多个TilingKey时，TilingKey之间不能有空格。示例如下，其中1、2为tiling_key。 --tiling_key=1,2 - 编译宏开关请参考内置编译宏开关。 --op_relocatable_kernel_binary，设置该选项为true时，会额外编译一份可被重新链接的二进制文件；不配置或设置为false时该选项均不生效。该选项用于自定义Tiling下沉算子使能SuperKernel的场景，配置该选项所生成的二进制文件，可以使算子在SuperKernel编译时直接复用二进制文件，降低编译耗时。

须知

- 编译选项是基于“算子类型+AI处理器型号系列”进行配置的，也就是说不同的“算子类型+AI处理器型号系列”可以配置不同的编译选项。
add_ops_compile_options(AddCustom COMPUTE_UNIT Ascendxxxxyy ... OPTIONS -DNEW_MACRO1=xx)
add_ops_compile_options(AddCustom COMPUTE_UNIT Ascendxxxxyy ... OPTIONS -DNEW_MACRO2=xx)
add_ops_compile_options(AddCustom COMPUTE_UNIT Ascendxxxxyy ... OPTIONS -DNEW_MACRO3=xx)
- 对相同算子类型+AI处理器型号系列，做多次编译选项配置，以后配置的为准。
- 对ALL生效的编译选项和对单一算子生效的编译选项如果没有冲突，同时生效，如果有冲突，以单一算子的编译选项为准。

2.10.2.6.2 算子包部署

算子包部署指执行自定义算子包的安装，算子工程的编译结果会自动部署到算子包安装目录下。

算子包部署

步骤1 自定义算子包安装部署。

在自定义算子包所在路径下，执行如下命令，安装自定义算子包。

```
./custom_opp_<target os>_<target architecture>.run --install-path=<path>
```

--install-path为可选参数，用于指定自定义算子包的安装目录。支持指定绝对路径，运行用户需要对指定的安装路径有可读写权限。

下文描述中的<vendor_name>为算子工程编译时CMakePresets.json配置文件中字段“vendor_name”的取值，默认为“customize”。

- 默认安装场景，不配置--install-path参数，安装成功后会将编译生成的自定义算子相关文件部署到\${INSTALL_DIR}/opp/vendors/<vendor_name>目录。\${INSTALL_DIR}请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。

说明

自定义算子包默认安装路径\${INSTALL_DIR}/opp/vendors的目录权限与CANN软件包安装用户和安装配置有关。如果因权限不足导致自定义算子包安装失败，可使用--install-path参数并配置环境变量ASCEND_CUSTOM_OPP_PATH来指定安装目录（参考[指定目录安装](#)）或者联系CANN软件包的安装用户修改vendors目录权限来解决。详细的案例请参考[2.10.10.8 调用算子时出现无法打开config.ini的报错](#)和[2.10.10.9 算子包部署时出现权限不足报错](#)。

- 指定目录安装场景，配置--install-path参数，安装成功后会将编译生成的自定义算子相关文件部署到<path>/vendors/<vendor_name>目录，并在<path>/vendors/<vendor_name>/bin目录下新增set_env.bash，写入当前自定义算子包相关的环境变量。

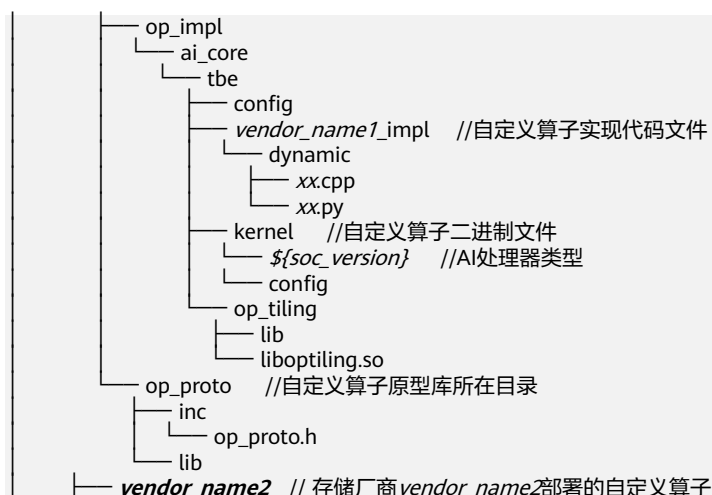
须知

如果部署算子包时通过配置--install-path参数指定了算子包的安装目录，则在使用自定义算子前，需要执行**source <path>/vendors/<vendor_name>/bin/set_env.bash**命令，set_env.bash脚本中将自定义算子包的安装路径追加到环境变量ASCEND_CUSTOM_OPP_PATH中，使自定义算子在当前环境中生效。

命令执行成功后，自定义算子包中的相关文件将部署至当前环境中。

步骤2 以默认安装场景为例，可查看部署后的目录结构，如下所示：

```
├── opp //算子库目录
│   ├── vendors //自定义算子所在目录
│   │   ├── config.ini
│   │   └── vendor_name1 // 存储对应厂商部署的自定义算子，此名字为编译自定义算子安装包时配置的
│   │       vendor_name, 若未配置，默认值为customize
│   │       ├── framework //自定义算子插件库
│   │       ├── op_api
│   │       │   ├── include
│   │       │   │   ├── aclnn_xx.h //算子调用API声明文件
│   │       │   └── lib
│   │       └── libcust_opapi.so
```



步骤3 配置自定义算子优先级。

多算子包共存的情况下，若不同的算子包目录下存在相同OpType的自定义算子，则以优先级高的算子包目录下的算子为准。下面介绍如何配置算子包优先级：

- 默认安装场景

当“opp/vendors”目录下存在多个厂商的自定义算子时，您可通过配置“opp/vendors”目录下的“config.ini”文件，配置自定义算子包的优先级。

“config.ini”文件的配置示例如下：

```
load_priority=vendor_name1,vendor_name2,vendor_name3
```

- “load_priority”：优先级配置序列的关键字，不允许修改。
- “vendor_name1,vendor_name2,vendor_name3”：自定义算子厂商的优先级序列，按照优先级从高到低的顺序进行排列。

- 指定目录安装场景

指定目录安装场景下，如果需要多个自定义算子包同时生效，分别执行各算子包安装路径下的set_env.bash脚本即可。每次脚本执行都会将当前算子包的安装路径追加到ASCEND_CUSTOM_OPP_PATH环境变量的最前面。因此可以按照脚本执行顺序确定优先级：脚本执行顺序越靠后，算子包优先级越高。

比如先执行source <path>/vendor_name1/bin/set_env.bash，后执行source <path>/vendor_name2/bin/set_env.bash，vendor_name2算子包的优先级高于vendor_name1。ASCEND_CUSTOM_OPP_PATH示例如下：

```
ASCEND_CUSTOM_OPP_PATH=<path>/vendor_name2:<path>/vendor_name1:
```

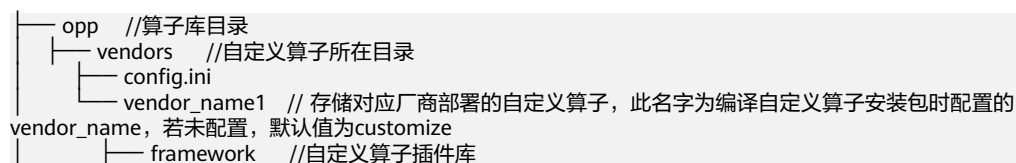
- 指定目录安装场景下安装的算子包优先级高于默认方式安装的算子包。

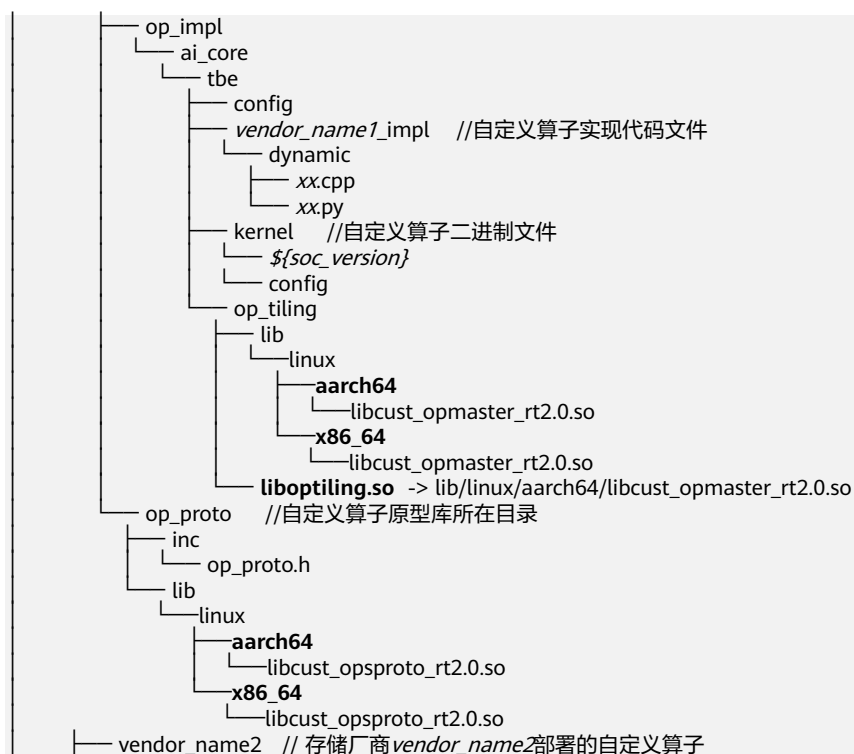
----结束

多平台算子包部署

支持安装多平台的自定义算子包，安装时同样支持默认路径和自定义路径安装。

以默认路径安装为例，在aarch64平台上分别安装aarch64平台和x86_64平台算子包，安装成功后可查看目录结构兼容两种平台类型，如下所示：





2.10.2.7 算子动态库和静态库编译

算子库编译

算子动态库和静态库编译是将算子实现代码及相关文件编译为动态库和静态库的过程。相比自定义算子包编译，动态库和静态库编译能够显著简化集成与部署流程。该过程包括将算子Kernel实现、Host侧Tiling实现、[入图适配文件](#)以及[自动生成的单算子调用实现代码](#)编译链接成动态库和静态库。

同时会自动生成以下头文件：

- 单算子调用aclnn头文件，用于[单算子调用场景](#)；
- 算子原型定义头文件，用于[算子入图场景](#)。

📖 说明

算子动态库和静态库编译支持如下型号：

- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品
- Atlas 推理系列产品

算子动态库和静态库编译的具体步骤如下：

步骤1 完成工程编译相关配置。

除了上文介绍的[基础配置](#)，算子动态库和静态库编译需要在工程目录下的CMakePresets.json cacheVariables的配置项中配置ASCEND_PACK_SHARED_LIBRARY为True，默认为False（会生成run包）。

```
"ASCEND_PACK_SHARED_LIBRARY": {  
  "type": "BOOL",  
  "value": "True"  
}
```

步骤2 在算子工程目录下执行如下命令，进行算子工程编译。

```
./build.sh
```

编译成功后，会在`${CMAKE_INSTALL_PREFIX}/op_api`目录生成以下文件：

- 算子原型定义头文件，用于算子入图场景，定义算子的原型。
- 单算子调用`aclnn`头文件，用于单算子调用场景，提供直接调用算子的接口。
- 动态库`libcust_opapi.so`，用于动态链接。
- 静态库`lib${vendor_name}.a`，用于静态链接。
- 安装文件`${vendor_name}-config.cmake`和`${vendor_name}-targets.cmake`，方便开发者将多个厂商的算子动态库或静态库集成到一个公共的动态库中，其中`${vendor_name}`是厂商名，也可以理解成同一个算子工程生成的算子package名称。具体集成方式可以参考[算子库集成和使用](#)。

具体目录结构如下：

```
├── op_api
│   ├── include
│   │   ├── aclnn_optype1.h           // 单算子调用aclnn头文件
│   │   ├── aclnn_optype2.h
│   │   ├── aclnn_optypexxx.h
│   │   └── op_proto.h               // 算子原型定义头文件
│   └── lib
│       ├── cmake
│       │   ├── ${vendor_name}
│       │   │   ├── ${vendor_name}-config.cmake // 安装文件，定义了项目中的配置选项和变量
│       │   │   └── ${vendor_name}-targets.cmake // 安装文件，定义静态库和动态库的构建目标
│       │   └── {vendor_name}::static和{vendor_name}::shared，便于开发者使用静态库和动态库，自动处理库的依赖关系
│       └── libcust_opapi.so          // 算子动态库
│           └── lib${vendor_name}.a  // 算子静态库
```

----结束

算子库集成和使用

- 单算子调用场景

单算子调用场景可以通过如下方式对[算子库编译](#)中生成的动态库和静态库进行集成和使用。

完整样例可参考[静态库集成和使用样例](#)。

- 动态库集成

```
find_package(${vendor_name} REQUIRED # ${vendor_name}为编译生成的算子package名称
PATHS ${CUST_PKG_PATH}           # ${CUST_PKG_PATH}为编译生成的算子package的存放路
径
NO_DEFAULT_PATH
)
target_link_libraries(op_runner PRIVATE
...
${vendor_name}::shared           # 已自动包含相关的target依赖
...
)
```

- 静态库集成

```
find_package(${vendor_name} REQUIRED # ${vendor_name}为编译生成的算子package名称
PATHS ${CUST_PKG_PATH}           # ${CUST_PKG_PATH}为编译生成的算子package的存放路
径
NO_DEFAULT_PATH
)
target_link_libraries(op_runner PRIVATE
...
${vendor_name}::static           # 已自动包含相关的target依赖
...
)
```

```
...
)
- 静态库和动态库混合使用场景
find_package(${vendor_name1} REQUIRED # ${vendor_name1}为编译生成的算子package名称
PATHS ${CUST_PKG_PATH_1} # ${CUST_PKG_PATH_1}为编译生成的算子package的存
放路径
NO_DEFAULT_PATH
)
find_package(${vendor_name2} REQUIRED # ${vendor_name2}为编译生成的算子package名称
PATHS ${CUST_PKG_PATH_2} # ${CUST_PKG_PATH_2}为编译生成的算子package的存
放路径
NO_DEFAULT_PATH
)

target_link_libraries(op_runner PRIVATE
...
${vendor_name1}::static # 已自动包含相关的静态库target依赖
${vendor_name2}::shared # 已自动包含相关的动态库target依赖
...
)
```

说明

- 上文中的算子package存放路径默认指算子工程`${CMAKE_INSTALL_PREFIX}/op_api`目录，开发者可以自行将`op_api`目录下的文件拷贝到自己的目录下，此时`${CUST_PKG_PATH}`可以设置为该自定义目录。
- 由于不同算子工程生成的算子动态库名称相同，如果需要将多个动态库进行集成，需要对`libcust_opapi.so`名称进行修改：

编译前，修改`op_host`目录中的`CMakeLists.txt`文件，添加如下代码，设置不同的输出文件名，从而可以区分不同动态库。

```
set_target_properties(cust_opapi PROPERTIES
  OUTPUT_NAME ${vendor_name}
)
```

编译后，修改生成的`op_api`目录中的`${vendor_name}-targets.cmake`文件，将其中的`libcust_opapi.so`修改为`lib${vendor_name}.so`。保证动态库名称的一致性。

- 算子入图场景
 - 配置`ASCEND_CUSTOM_OPP_PATH`环境变量，将动态库`libcust_opapi.so`的绝对路径追加到该环境变量。由GE框架在图编译和执行时根据该环境变量搜索算子动态库并使用。环境变量位置越靠前，算子的优先级越高。

```
export ASCEND_CUSTOM_OPP_PATH=${CMAKE_INSTALL_PREFIX}/op_api/lib/;$
{ASCEND_CUSTOM_OPP_PATH}
```

说明

动态库编译和算子包编译功能同时使用时，前者生成的动态库优先级更高。

如下示例中，`path1`和`path3`是算子包编译生成的目录，`path2`和`path4`是动态库编译产物的存放目录，则编译产物的优先级为`2>4>1>3`。

```
ASCEND_CUSTOM_OPP_PATH=<path1>/vendor_name1:<path2>/op_api/lib/:<path3>/
vendor_name3:<path4>/op_api/lib/
```

- 如果开发者通过Ascend Graph进行图开发，除了配置环境变量的方式也可以采用直接在应用程序的编译文件中链接动态库`libcust_opapi.so`的方式。Ascend Graph图开发的相关信息请参考《图模式开发指南》。动态库链接方式的`so`加载优先级高于环境变量配置方式。

2.10.2.8 算子工程编译拓展

使用`msOpGen`工具创建算子工程时，相关编译脚本被固化在本地，若需使用算子工程提供的新特性（比如支持MC2算子等），需重新运行`msOpGen`工具生成工程。为了解决开发者因功能更新而频繁重建工程的问题，将算子工程的`cmake`脚本打包到CANN软

件包中。开发者可通过find_package的形式来查找对应的cmake modules包，从而使用算子工程对外提供的cmake函数接口。

该开发方式下，开发者参考如下工程目录，自行组织算子工程。目录结构与通过msOpGen生成的目录结构类似，但无需再创建cmake与scripts目录，这些目录被打包至CANN软件包中。以AddCustom算子为例，目录结构如下：

```
AddCustom
├── CMakeLists.txt           // 算子工程顶层CMakeLists.txt
├── CMakePresets.json       // 编译配置项，cmake内置功能，若使用则需3.19及以上cmake版本
├── framework               // AI框架适配时，算子插件实现文件目录
├── op_host                 // Host侧实现文件
│   ├── add_custom.cpp      // 算子原型注册、Shape推导、Tiling实现等内容文件
│   ├── add_custom_tiling.h // 算子Tiling定义文件
│   └── CMakeLists.txt      // Host侧CMakeLists文件
├── op_kernel               // Kernel侧实现文件
│   ├── Add
│   │   ├── add_custom.cpp  // 算子代码实现文件
│   │   └── CMakeLists.txt  // Kernel侧CMakeLists文件
```

CMakeLists.txt 编写方法

- 算子工程顶层CMakeLists.txt

- 使用**find_package**找到对应的编译库。
- 使用**npu_op_package**设置算子工程的编译产物形态，支持RUN/SHARED/STATIC，分别对应算子run包形式、算子动态库形式与算子静态库形式，同时，该接口还可配置package（即编译产物）的内容和package的安装位置。
- 添加需要进行编译的子目录。

```
cmake_minimum_required(VERSION 3.16.0) # 指定所需的最低cmake版本号
project(opp)
# 1、使用find_package找到对应的编译库。
# ${ASCEND_CANN_PACKAGE_PATH}为对应的CANN软件包路径，开发者可直接使用
find_package(ASC REQUIRED HINTS ${ASCEND_CANN_PACKAGE_PATH}/compiler/tikcpp/
ascendc_kernel_cmake)
# 2、使用npu_op_package设置算子工程的编译产物形态。
set(package_name ${vendor_name})
npu_op_package(${package_name} # package name
# RUN模式下会使用该name命名算子部署后的目录，静态库模式下输出产物为lib${package_name}.a
TYPE RUN # 指定编译产物形态，[RUN|STATIC|SHARED]
CONFIG
    ENABLE_SOURCE_PACKAGE True # 是否将源码打包到run包中
    ENABLE_BINARY_PACKAGE True # 是否编译Kernel二进制，SHARED/STATIC模式下必须指定为True，默认值为True
    INSTALL_PATH ${CMAKE_BINARY_DIR}/ # package的安装位置
)
# 3、添加需要进行编译的子目录。
if(EXISTS ${CMAKE_CURRENT_SOURCE_DIR}/op_host)
    add_subdirectory(op_host) # 添加子目录op_host进行编译
endif()
if(EXISTS ${CMAKE_CURRENT_SOURCE_DIR}/op_kernel)
    add_subdirectory(op_kernel) # 添加子目录op_kernel进行编译
endif()
```

- Host侧CMakeLists.txt

- 使用**npu_op_code_gen**生成aclnn单算子调用代码、入图所需的原型定义代码等。
- 单算子调用场景，使用**npu_op_library**编译aclnn单算子调用库。
- 算子入图场景，使用**npu_op_library**编译算子入图所需的算子原型库。
- 使用**npu_op_library**编译Tiling相关库。

e. 使用`npu_op_package_add`添加上述Host侧库至对应package中。

```
aux_source_directory(${CMAKE_CURRENT_SOURCE_DIR} ops_srcs)
#1、使用npu_op_code_gen生成aclnn单算子调用代码、入图所需的原型定义代码等。
npu_op_code_gen(
    SRC ${ops_srcs}                                # Host侧实现文件集合
    PACKAGE ${package_name}                        # 与npu_op_package的
package_name对应
    COMPILE_OPTIONS -g                             # 编译选项
    OUT_DIR ${ASCEND_AUTOGEN_PATH}                 # 指定生成文件的保存路径
)
#2、单算子调用场景，使用npu_op_library编译aclnn单算子调用库。
file(GLOB autogen_aclnn_src ${ASCEND_AUTOGEN_PATH}/aclnn_*.cpp)      # 将通过
npu_op_code_gen生成的aclnn相关源文件添加至源文件列表中

set_source_files_properties(${autogen_aclnn_src} PROPERTIES GENERATED TRUE) # 设置对应的文件
属性为编译过程中生成的文件
npu_op_library(cust_opapi ACLNN                                         # 指定library名称与类型，以及参与
编译的源文件
    ${autogen_aclnn_src}
)
target_compile_options(cust_opapi PRIVATE                             # 为对应library添加编译选项，与
通用的cmake语法一致
    -fvisibility=hidden
)

#3、算子入图场景，使用npu_op_library编译算子入图所需的算子原型库。
file(GLOB proto_src ${ASCEND_AUTOGEN_PATH}/op_proto.cc)              # 将通过
npu_op_code_gen生成的原型定义相关源文件添加至源文件列表中
set_source_files_properties(${proto_src} PROPERTIES GENERATED TRUE) # 设置对应的文件属性
为编译过程中生成的文件
npu_op_library(cust_op_proto GRAPH                                     # 指定library名称与类型，以及参
与编译的源文件
    ${ops_srcs}
    ${proto_src}
)
target_compile_options(cust_op_proto PRIVATE                         # 为对应library添加编译选项，
与通用的cmake语法一致
    -fvisibility=hidden
)

#4、使用npu_op_library编译Tiling相关库。
file(GLOB fallback_src ${ASCEND_AUTOGEN_PATH}/fallback_*.cpp)        # 将通过
npu_op_code_gen生成的Tiling相关源文件添加至源文件列表中
# 若算子配置了EnableFallBack会生成该文件
set_source_files_properties(${fallback_src} PROPERTIES GENERATED TRUE) # 设置对应的文件属性
为编译过程中生成的文件
npu_op_library(cust_optiling TILING                                   # 指定library名称与类型，以及参与编
译的源文件
    ${ops_srcs}
    ${fallback_src}
)
target_compile_options(cust_optiling PRIVATE                         # 为对应library添加编译选项，与
通用的cmake语法一致
    -fvisibility=hidden
)

#5、使用npu_op_package_add添加上述Host侧库到对应package中。
npu_op_package_add(${package_name}                                   # 与npu_op_package的
package_name对应
    LIBRARY                                                         # 添加Host侧相关library，library名称与前述
步骤的library名称对应
    cust_optiling
    cust_opapi
    cust_op_proto
)
```

- Kernel侧CMakeLists.txt

- a. 使用`npu_op_kernel_options`添加算子编译选项。支持的编译选项可参考[编译选项](#)。
- b. 使用`npu_op_kernel_sources`指定算子特定目录与编译源文件。
 - 若算子的源码文件没有平铺在SRC_BASE目录（通过`npu_op_kernel_library`设置）下，可以通过`KERNEL_DIR`指定特定目录。
 - 若算子的Kernel实现cpp文件需要自定义命名，需同时指定`OP_TYPE`（算子类型）和`KERNEL_FILE`（Kernel实现cpp文件名），以配置两者之间的对应关系。不配置时，Kernel实现cpp文件名和OpType之间需满足[转换规则](#)。
- c. 使用`npu_op_kernel_library`编译Kernel库。
- d. 使用`npu_op_package_add`添加上述Kernel侧库到对应package中。

```
#1、使用npu_op_kernel_options添加算子编译选项。
npu_op_kernel_options(ascendc_kernels ALL OPTIONS --save-temp-files -g)# 为算子添加编译选项

#2、使用npu_op_kernel_sources指定算子特定目录与编译源文件。
npu_op_kernel_sources(ascendc_kernels                                # 对应的Kernel库名称，与
npu_op_kernel_library保持一致                                         # 若算子的Kernel实现cpp文件需要自定义命
    OP_TYPE AddCustom                                                  # 将KERNEL_DIR目录与SRC_BASE目录进行拼
    KERNEL_DIR ./Add                                                  # 设置KERNEL_FILE在Ascendxxxxy型号生
    COMPUTE_UNIT Ascendxxxxy                                          # 若算子的Kernel实现cpp文件需要自定义
    KERNEL_FILE add_custom.cpp                                        # 命名，需配置Kernel实现cpp文件名
)

#3、使用npu_op_kernel_library编译Kernel库。
npu_op_kernel_library(ascendc_kernels                                # Kernel库名称
    SRC_BASE ${CMAKE_SOURCE_DIR}/op_kernel/                          # 指定kernel侧源码的根目录
    TILING_LIBRARY cust_optiling                                       # 指定依赖的Tiling库
)

#4、添加kernel侧library到package
npu_op_package_add(${package_name}                                    # 与npu_op_package的
    package_name对应                                                  # 添加Kernel侧相关library，library名称与前
    LIBRARY ascendc_kernels                                           # 述步骤的library名称对应
)
```

编译环境选项说明

下表列出了供用户配置和使用的编译环境选项：

说明

- 暂不支持设置Release、Debug版本相关选项。
- 暂不支持交叉编译相关选项。

表 2-32 编译环境选项

类型	环境选项	默认值	说明
STRING	vendor_name	customize	标识自定义算子所属厂商的名称。建议开发者自行指定所属厂商名称，避免和其他厂商提供的算子包冲突。

类型	环境选项	默认值	说明
STRING	ASCEND_COMPUTE_UNIT	-	AI处理器型号。编译对应型号的package。
PATH	ASCEND_AUTOTGEN_PATH	<CMAKE_BINARY_DIR>/autogen	通过npu_op_code_gen生成的aclnn单算调用、入图所需算子原型库等源文件存放路径。
BOOL	ENABLE_SOURCE_PACKAGE	TRUE	是否开启源码编译。 如果使用npu_op_package配置了源码和二进制编译相关配置，npu_op_package配置的优先级更高。
BOOL	ENABLE_BINARY_PACKAGE	TRUE	是否开启二进制编译。 package的类型配置为SHARED或STATIC时，必须指定为TRUE。 如果使用npu_op_package配置了源码和二进制编译相关配置，npu_op_package配置的优先级更高。
PATH	ASCEND_CANN_PACKAGE_PATH	-	CANN软件包路径，默认情况下用户无需配置。开发者可以在编写cmake文件时直接使用。 路径示例如下：/usr/local/Ascend/cann。

上述编译环境选项可通过两种方式进行配置：

- 支持直接通过set命令进行设置。

```
# CMakeLists.txt直接使用
set(vendor_name "customize")
set(ASCEND_COMPUTE_UNIT "ascendxxxxy")
```

- 通过CMakePresets.json进行设置，若通过该方式设置，则需要安装3.19及以上的cmake。

```
{
  "version": 1,
  "cmakeMinimumRequired": {
    "major": 3,
    "minor": 19,
    "patch": 0
  },
  "configurePresets": [
    {
      "name": "default",
      "displayName": "Default Config",
      "description": "Default build using Unix Makefiles generator",
      "generator": "Unix Makefiles",
      "binaryDir": "${sourceDir}/build_out",
      "cacheVariables": {
        "ASCEND_COMPUTE_UNIT": {
          "type": "STRING",
          "value": "ascendxxxxy"
        }
      }
    }
  ]
}
```

```
    },  
    "vendor_name": {  
      "type": "STRING",  
      "value": "customize"  
    }  
  }  
}  
]  
}
```

然后使用如下命令进行构建，根据--preset指定的preset进行编译。

```
// Shell环境中cmake构建编译  
cmake -S . -B build_out --preset=default // 在当前目录(.)中寻找CMakePresets.json
```

编译命令说明

package的类型配置为RUN时，可参考如下编译命令进行编译，使用cmake的pack能力生成run包。

```
# 工程根目录下执行  
mkdir -p build_out  
rm -rf build_out/*  
cmake -S . -B build_out --preset=default  
cmake --build build_out --target binary -j${nproc}  
cmake --build build_out --target package -j${nproc}
```

package的类型配置为SHARED或STATIC时，可参考如下编译命令进行编译：

```
# 工程根目录下执行  
mkdir -p build_out  
rm -rf build_out/*  
cmake -S . -B build_out --preset=default  
cmake --build build_out --target binary -j${nproc}  
cmake --build build_out --target install -j${nproc}
```

cmake 函数说明

提供如下cmake函数供开发者使用：

类型	接口	功能
package	npu_op_package	创建一个package。
	npu_op_package_add	将目标或文件添加到package中。
library	npu_op_library	创建Host侧库。
	npu_op_kernel_library	创建Kernel侧库。
	npu_op_kernel_options	添加Kernel目标编译选项。
	npu_op_kernel_sources	描述Kernel目标的源码信息。
	npu_op_device_tiling_library	创建Device侧Tiling库。
其他	npu_op_code_gen	执行代码生成过程，生成aclnn单算子调用代码和入图所需的原型定义代码。

1. npu_op_package

创建一个package。

```
npu_op_package(<package_name> TYPE <type> [CONFIG] [ENABLE_SOURCE_PKG <value>]  
[ENABLE_BINARY_PACKAGE <value>] [INSTALL_PATH <path>])
```

参数说明如下：

- <package_name>：必选，package的名称。
- TYPE <type>：必选，package的类型，取值为RUN、SHARED、STATIC。分别对应算子run包形式、算子动态库形式与算子静态库形式。
- [CONFIG]：可选，用于配置package的内容和安装位置。
 - [ENABLE_SOURCE_PKG <value>]：可选，是否将源码打包到package中，默认为True。
 - [ENABLE_BINARY_PACKAGE <value>]：可选，是否将二进制文件打包到package中，默认为True。
 - [INSTALL_PATH <path>]：可选，指定包的安装路径，默认为CMAKE_BINARY_DIR。
 - [ENABLE_CPACK <value>]：可选，是否打包，默认为True。对于用户需要定制打包的场景，可以设置为False，配置为False的情况下直接在<path>路径下生成编译产物，不会打包为run包。

2. npu_op_package_add

将目标或文件添加到package中。

```
# 添加目标  
npu_op_package_add(<package_name> LIBRARY <target_name1> [<target_name2>...] )  
# 添加文件，仅给run包模式使用  
npu_op_package_add(<package_name> FILES <file_name1> [<file_name2>...] [TYPE <target_type>]  
[PACKAGE_PATH <pkg_path>])
```

参数说明如下：

- <package_name>：必选，package的名称。
- LIBRARY：必选，指定需要添加到package中的目标名称。
 - <target_name1> [<target_name2>...]：必选，目标名称列表。
- FILES：必选，指定需要添加到package中的文件名称。
 - <file_name1> [<file_name2>...]：必选，文件名称列表。
- [TYPE <target_type>]：可选，指定文件类型，将文件安装到对应的目录中，取值为ACLNN、GRAPH。配置为ACLNN，会将文件打包至run包目录下aclnn单算子调用头文件所在目录，配置为GRAPH，会将文件打包至run包目录下入图原型定义头文件目录下。
- [PACKAGE_PATH <pkg_path>]：可选，指定文件在包中的相对路径位置。TYPE和PACKAGE_PATH参数互斥，即只能选择其中一个进行配置。

3. npu_op_library

创建Host侧库。

```
npu_op_library(<library_name> TYPE <library_type> <files>)
```

参数说明如下：

- <library_name>：必选，Host侧库的名称。
- TYPE <library_type>：必选，Host库的类型，可选值为TILING、ACLNN、GRAPH、TF_PLUGIN。

- TILING, Tiling相关库。
 - ACLNN, aclnn单算子调用库。
 - GRAPH, 算子入图所需的算子原型库。
 - TF_PLUGIN, TensorFlow框架适配相关库。
 - <files>: 必选, 设置参与编译的源文件。
4. npu_op_kernel_library
- 创建Kernel侧库。
- ```
npu_op_kernel_library(<target_name> SRC_BASE <path> TILING_LIBRARY <tiling_target>)
```
- 参数说明如下:
- <target\_name>: 必选, 目标的名称。
  - SRC\_BASE <path>: 必选, 指定Kernel源码的base目录, 要求配置绝对路径。比如示例中的op\_kernel目录的绝对路径。
  - TILING\_LIBRARY <tiling\_target>: 必选, 指定依赖的Tiling目标。

5. npu\_op\_kernel\_options

添加Kernel目标编译选项。

```
npu_op_kernel_options(<target_name> <op_type> [COMPUTE_UNIT <soc_version>] OPTIONS ...)
```

参数说明如下:

- <target\_name>: 必选, 目标的名称。
- <op\_type>: 必选, 定义配置生效的范围, 取值为ALL、OP\_TYPE。ALL表示对所有算子生效, OP\_TYPE表示对特定算子生效。
- [COMPUTE\_UNIT <soc\_version>]: 可选, 用于设置算子在具体AI处理器型号上的编译选项, 不填写该选项时默认对所有型号生效。

### 说明

<soc\_version>具体配置如下:

- 针对如下产品: 在安装AI处理器的服务器执行**npu-smi info**命令进行查询, 获取**Name**信息。实际配置值为AscendName, 例如**Name**取值为xxxyy, 实际配置值为Ascendxxxyy。

Atlas A2 训练系列产品/Atlas A2 推理系列产品

Atlas 200I/500 A2 推理产品

Atlas 推理系列产品

Atlas 训练系列产品

- 针对如下产品, 在安装AI处理器的服务器执行**npu-smi info -t board -i id -c chip\_id**命令进行查询, 获取**Chip Name**和**NPU Name**信息, 实际配置值为Chip Name\_NPU Name。例如**Chip Name**取值为Ascendxxx, **NPU Name**取值为1234, 实际配置值为Ascendxxx\_1234。其中:

- id: 设备id, 通过**npu-smi info -l**命令查出的NPU ID即为设备id。

- chip\_id: 芯片id, 通过**npu-smi info -m**命令查出的Chip ID即为芯片id。

Atlas A3 训练系列产品/Atlas A3 推理系列产品

- OPTION...: 必选, 传递给编译器的编译选项。

6. npu\_op\_kernel\_sources

描述Kernel目标的源码信息, 包括设置算子的Kernel实现文件和源码路径等。

```
npu_op_kernel_sources(<target_name> [OP_TYPE <op_type>] [KERNEL_DIR <path>] [COMPUTE_UNIT <soc_version>] [KERNEL_FILE <file>])
```

参数说明如下：

- <target\_name>：必选，目标的名称。
- [OP\_TYPE <op\_type>]：可选，算子类型，必须与KERNEL\_FILE同时存在。
- [KERNEL\_DIR <path>]：可选，指定Kernel源码相对于SRC\_BASE的相对路径。

若算子的源码文件没有平铺在SRC\_BASE目录（通过npu\_op\_kernel\_library设置）下，可以通过KERNEL\_DIR指定特定目录。

- [COMPUTE\_UNIT <soc\_version>]：可选，设置KERNEL\_FILE在<soc\_version>型号生效。默认KERNEL\_FILE对所有型号生效。
- [KERNEL\_FILE <file>]：可选，指定算子入口的Kernel实现文件名。

若算子的Kernel实现cpp文件需要自定义命名，需同时指定OP\_TYPE（算子类型）和KERNEL\_FILE（Kernel实现cpp文件名），以配置两者之间的对应关系。不配置时，Kernel实现cpp文件名和OpType之间需满足[转换规则](#)。

#### 7. npu\_op\_device\_tiling\_library

创建Device侧Tiling库。使用该选项时，package的类型仅支持配置为RUN（run包模式）。

```
npu_op_device_tiling_library(<target_name> <type> <files>)
```

参数说明如下：

- <target\_name>：必选，目标的名称。
- <type>：必选，指定Tiling产物的类型。支持取值为SHARED、STATIC。
- <files>：必选，指定Tiling源码文件。

#### 8. npu\_op\_code\_gen

执行代码生成过程，生成aclnn单算子调用代码和入图所需的原型定义代码。

```
npu_op_code_gen(SRC <src_files> OUT_DIR <output_dir> PACKAGE <pkg_name>
[COMPILE_OPTIONS ...])
```

参数说明如下：

- SRC <src\_files>：必选，参与代码生成的源文件范围。
- OUT\_DIR <output\_dir>：必选，生成代码的输出路径。
- PACKAGE <pkg\_name>：必选，指定生成代码的package名称。
- [COMPILE\_OPTIONS ...]：可选，自定义编译过程中的编译选项。

### 2.10.2.9 单算子 API 调用

单算子API调用方式，是指直接调用单算子API接口，基于C语言的API执行算子。算子工程创建完成后，基于工程代码框架完成算子原型定义、kernel侧算子实现、host侧tiling实现，通过工程编译脚本完成算子的编译部署，之后再进行单算子API的调用。

#### 基本原理

完成自定义算子编译后，会自动生成单算子API，可以直接在应用程序中调用。

单算子API的形式一般定义为“两段式接口”，形如：

```
aclnnStatus aclnnXxxGetWorkspaceSize(const aclTensor *src, ..., aclTensor *out, uint64_t *workspaceSize,
aclOpExecutor **executor);
aclnnStatus aclnnXxx(void *workspace, uint64_t workspaceSize, aclOpExecutor *executor, aclrtStream
stream);
```

其中aclnnXxxGetWorkspaceSize/aclnnXxxTensorGetWorkspaceSize为第一段接口，主要用于计算本次API调用过程中需要多少workspace内存，获取到本次计算所需的

workspaceSize后, 按照workspaceSize申请NPU内存, 然后调用第二段接口执行计算。*Xxx*代表[算子原型注册](#)时传入的算子类型。

**aclnnXxxGetWorkspaceSize**接口的输入输出参数生成规则如下:

- 可选输入的命名增加Optional后缀。如下样例中, x是可选输入。  
`aclnnStatus aclnnXxxGetWorkspaceSize(const aclTensor *xOptional, ..., aclTensor *out, uint64_t *workspaceSize, aclOpExecutor **executor);`
- 输入输出同名、使用同一个Tensor承载的情况下, 生成的接口中只保留input参数同时去掉input的const修饰, 并以Ref作为后缀。如下样例中, 原型定义input、output都定义为x, xRef既作为输入, 又作为输出。  
`aclnnStatus aclnnXxxGetWorkspaceSize(aclTensor *xRef, ..., uint64_t *workspaceSize, aclOpExecutor **executor);`
- 如果仅有一个输出, 输出参数命名为out; 如果存在多个输出, 每个输出后面都以Out作为后缀。  
// 仅有一个输出  
`aclnnStatus aclnnXxxGetWorkspaceSize(const aclTensor *src, ..., aclTensor *out, uint64_t *workspaceSize, aclOpExecutor **executor);`  
// 存在多个输出  
`aclnnStatus aclnnXxxGetWorkspaceSize(const aclTensor *src, ..., aclTensor *yOut, aclTensor *y1Out, ..., uint64_t *workspaceSize, aclOpExecutor **executor);`
- 如果算子包含属性, 则属性参数的位置介于输入输出之间。如下示例中, x是算子输入, negativeSlope是算子属性, out是算子输出。  
`aclnnStatus aclnnXxxGetWorkspaceSize(const aclTensor *x, double negativeSlope, aclTensor *out, uint64_t *workspaceSize, aclOpExecutor **executor);`

当算子原型注册时使用ValueDepend接口标识输入为数据依赖输入时, 会额外生成一个API, 该API支持值依赖场景输入数据为空的一阶段计算。

```
aclnnStatus aclnnXxxTensorGetWorkspaceSize(const aclTensor *src, ..., aclTensor *out, uint64_t *workspaceSize, aclOpExecutor **executor);
```

在中, **aclnnXxxGetWorkspaceSize**参数的数据类型 (aclIntArray、aclFloatArray和aclBoolArray) 将被转换为aclTensor数据类型, 其他输入输出参数生成规则与**aclnnXxxGetWorkspaceSize**一致。如下示例中, x0、x1、x2是算子声明为数据依赖的输入, 数据类型分别为DT\_INT64、DT\_BOOL、DT\_FLOAT, out是算子输出。

```
aclnnStatus aclnnXxxGetWorkspaceSize(const aclIntArray *x0, const aclBoolArray *x1, const aclFloatArray *x2, aclTensor *out, uint64_t *workspaceSize, aclOpExecutor **executor);
aclnnStatus aclnnXxxTensorGetWorkspaceSize(const aclTensor *x0, const aclTensor *x1, const aclTensor *x2, aclTensor *out, uint64_t *workspaceSize, aclOpExecutor **executor);
```

## 前置步骤

- 参考[2.10.2.2 创建算子工程](#)完成自定义算子工程的创建。
- 参考[2.10.2.4 Kernel侧算子实现](#)完成kernel侧实现的相关准备, 参考[2.10.2.5 Host侧Tiling实现](#)、[2.10.2.3 算子原型定义](#)完成host侧实现相关准备。
- 对于算子包编译场景, 参考[2.10.2.6.1 算子工程编译](#)、[2.10.2.6.2 算子包部署](#)完成算子的编译部署, 编译部署时需要开启算子的二进制编译功能: 修改算子工程中的编译配置项文件CMakePresets.json, 将ENABLE\_BINARY\_PACKAGE设置为True。编译部署时可将算子的二进制部署到当前环境, 便于后续算子的调用。

```
"ENABLE_BINARY_PACKAGE": {
 "type": "BOOL",
 "value": "True"
},
```

算子编译部署后, 会在算子包安装目录下的op\_api目录生成单算子调用的头文件aclnn\_xx.h和动态库libcust\_opapi.so。

以默认安装场景为例, 单算子调用的头文件.h和动态库libcust\_opapi.so所在的目录结构, 如下所示:



```
├── opp //算子库目录
│ ├── vendors //自定义算子所在目录
│ │ ├── config.ini
│ │ └── vendor_name1 // 存储对应厂商部署的自定义算子，此名字为编译自定义算子安装包时配置
│ │ the vendor_name, 若未配置，默认值为customize
│ └── op_api
│ ├── include
│ │ └── aclnn_xx.h
│ ├── lib
│ └── libcust_opapi.so
...

```

- 对于算子动态库编译场景，参考[2.10.2.7 算子动态库和静态库编译](#)完成算子的编译。编译完成后会在如下路径生成单算子调用的头文件aclnn\_xx.h和动态库libcust\_opapi.so。其中CMAKE\_INSTALL\_PREFIX为开发者在cmake文件中配置的编译产物存放路径。
  - 动态库路径：\${CMAKE\_INSTALL\_PREFIX}/op\_api/lib/libcust\_opapi.so
  - 头文件路径：\${CMAKE\_INSTALL\_PREFIX}/op\_api/include

## 准备验证代码工程

代码工程目录结构如下，您可以单击[LINK](#)，获取样例工程的完整样例：

```
├── input // 存放脚本生成的输入数据目录
├── output // 存放算子运行输出数据和真值数据的目录
├── inc // 头文件目录
│ ├── common.h // 声明公共方法类，用于读取二进制文件
│ ├── operator_desc.h // 算子描述声明文件，包含算子输入/输出，算子类型以及输入描述与输出描述
│ └── op_runner.h // 算子运行相关信息声明文件，包含算子输入/输出个数，输入/输出大小等
├── src
│ ├── CMakeLists.txt // 编译规则文件
│ ├── common.cpp // 公共函数，读取二进制文件函数的实现文件
│ ├── main.cpp // 单算子调用应用的入口
│ ├── operator_desc.cpp // 构造算子的输入与输出描述
│ └── op_runner.cpp // 单算子调用主体流程实现文件
└── scripts
 ├── verify_result.py // 真值对比文件
 ├── gen_data.py // 输入数据和真值数据生成脚本文件
 └── acl.json // acl配置文件

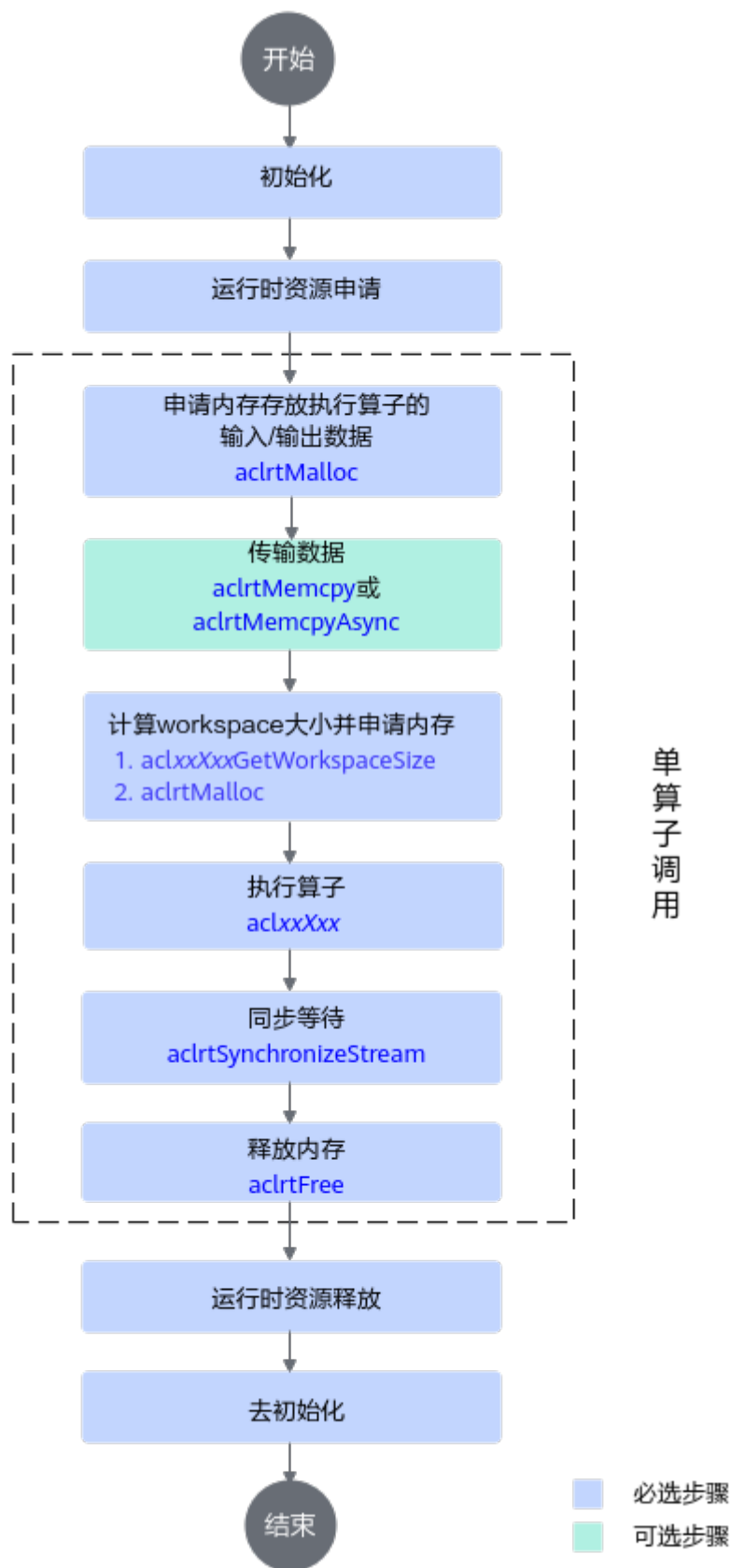
```

下文将重点介绍和单算子调用流程相关的主文件main.cpp、op\_runner.cpp文件、CMakeLists.txt文件如何编写，其他文件请自行参考。

## 单算子调用流程

单算子API执行流程如下：

图 2-39 单算子 API 执行接口调用流程



本节以**AddCustom自定义算子**调用为例，介绍如何编写单算子调用的代码逻辑。其他算子的调用逻辑与Add算子大致一样，请根据实际情况自行修改代码。

以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考，调用接口后，需增加异常处理的分支，并记录报错日志、提示日志，此处不一一列举。

### 说明

因为单算子API执行方式，会自动在编译工程的build\_out/autogen目录下生成.cpp和.h，编写单算子的调用代码时，要包含自动生成的单算子API执行接口头文件。示例如下：

```
#include "aclnn_add_custom.h"
```

```
// 1.初始化
aclRet = aclInit("../scripts/acl.json");

// 2.运行管理资源申请
int deviceId = 0;
aclRet = aclrtSetDevice(deviceId);
// 获取软件栈的运行模式，不同运行模式影响后续的接口调用流程（例如是否进行数据传输等）
aclrtRunMode runMode;
bool g_isDevice = false;
aclError aclRet = aclrtGetRunMode(&runMode);
g_isDevice = (runMode == ACL_DEVICE);

// 3.申请内存存放算子的输入输出
//

// 4.传输数据
if (aclrtMemcpy(devInputs_[i], size, hostInputs_[i], size, kind) != ACL_SUCCESS) {
 return false;
}

// 5.计算workspace大小并申请内存
size_t workspaceSize = 0;
aclOpExecutor *handle = nullptr;
auto ret = aclnnAddCustomGetWorkspaceSize(inputTensor_[0], inputTensor_[1], outputTensor_[0],
 &workspaceSize, &handle);

// ...
void *workspace = nullptr;
if (workspaceSize != 0) {
 if (aclrtMalloc(&workspace, workspaceSize, ACL_MEM_MALLOC_HUGE_FIRST) != ACL_SUCCESS) {
 ERROR_LOG("Malloc device memory failed");
 }
}

// 6.执行算子
if (aclnnAddCustom(workspace, workspaceSize, handle, stream) != ACL_SUCCESS) {
 (void)aclrtDestroyStream(stream);
 ERROR_LOG("Execute Operator failed. error code is %d", static_cast<int32_t>(ret));
 return false;
}

// 7.同步等待
aclrtSynchronizeStream(stream);

// 8.处理执行算子后的输出数据，例如在屏幕上显示、写入文件等，由用户根据实际情况自行实现
//

// 9.释放运行管理资源
aclRet = aclrtResetDevice(deviceId);
//

// 10.去初始化
aclRet = aclFinalize();
```

## CMakeLists 文件

算子编译后，会生成单算子调用的头文件和动态库libcust\_opapi.so。具体路径请参考前置步骤。

编译算子调用程序时，需要在头文件的搜索路径include\_directories中增加单算子调用的头文件目录，便于找到该头文件；同时需要链接cust\_opapi动态库并在库文件的搜索路径link\_directories中增加libcust\_opapi.so所在目录。

- 在头文件的搜索路径include\_directories中增加单算子调用的头文件目录。以下样例仅为参考，请根据头文件的实际目录位置进行设置。

```
include_directories(
 ${INC_PATH}/runtime/include
 ${INC_PATH}/atc/include
 ../inc
 ${OP_API_PATH}/include
)
```

- 链接cust\_opapi链接库。

```
target_link_libraries(execute_add_op
 ascendcl
 cust_opapi
 acl_op_compiler
 nnopbase
 stdc++
)
```

- 在库文件的搜索路径link\_directories中增加libcust\_opapi.so所在目录。以下样例仅为参考，请根据库文件的实际目录位置进行设置。

```
link_directories(
 ${LIB_PATH}
 ${LIB_PATH1}
 ${OP_API_PATH}/lib
)
```

## 生成测试数据

在样例工程目录下，执行如下命令：

```
python3 scripts/gen_data.py
```

会在工程目录下目录中生成两个shape为(8,2048)，数据类型为float16的数据文件\_0.bin与\_1.bin，用于进行AddCustom算子的验证。

代码样例如下：

```
import numpy as np
a = np.random.randint(100, size=(8, 2048,)).astype(np.float16)
b = np.random.randint(100, size=(8, 2048,)).astype(np.float16)
a.tofile('input_0.bin')
b.tofile('input_1.bin')
```

## 编译与运行

- 开发环境上，设置环境变量，配置单算子验证程序编译依赖的头文件与库文件路径，如下为设置环境变量的示例。\${INSTALL\_DIR}请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。{arch-os}为运行环境的架构和操作系统，arch表示操作系统架构，os表示操作系统，例如x86\_64-linux或aarch64-linux。

```
export DDK_PATH=${INSTALL_DIR}
export NPU_HOST_LIB=${INSTALL_DIR}/{arch-os}/devlib
```

- 编译样例工程，生成单算子验证可执行文件。

- a. 切换到样例工程根目录，然后在样例工程根目录下执行如下命令创建目录用于存放编译文件，例如，创建的目录为“build”。

```
mkdir -p build
```

- b. 进入build目录，执行cmake编译命令，生成编译文件

命令示例如下所示：

```
cd build
cmake ../src -DCMAKE_SKIP_RPATH=TRUE
```

- c. 执行如下命令，生成可执行文件。

```
make
```

会在工程目录的output目录下生成可执行文件**execute\_add\_op**。

### 3. 执行单算子

- a. 以运行用户（例如HwHiAiUser）拷贝开发环境中样例工程output目录下的**execute\_add\_op**到运行环境任一目录。

说明：若您的开发环境即为运行环境，此拷贝操作可跳过。

- b. 在运行环境中，执行**execute\_add\_op**文件：

```
chmod +x execute_add_op
./execute_add_op
```

会有如下屏显信息：

```
[INFO] Set device[0] success
[INFO] Get RunMode[1] success
[INFO] Init resource success
[INFO] Set input success
[INFO] Copy input[0] success
[INFO] Copy input[1] success
[INFO] Create stream success
[INFO] Execute aclnnAddCustomGetWorkspaceSize success, workspace size 0
[INFO] Execute aclnnAddCustom success
[INFO] Synchronize stream success
[INFO] Copy output[0] success
[INFO] Write output success
[INFO] Run op success
[INFO] Reset Device success
[INFO] Destroy resource success
```

如果有Run op success，表明执行成功，会在output目录下生成输出文件output\_z.bin。

### 4. 比较真值文件

切换到样例工程根目录，然后执行如下命令：

```
python3 scripts/verify_result.py output/output_z.bin output/golden.bin
```

会有如下屏显信息：

```
test pass
```

可见，AddCustom算子验证结果正确。

## 2.10.3 算子入图（GE 图）开发

### 2.10.3.1 概述

图模式是神经网络模型的一种运行模式，在图模式下用户首先将模型的计算过程构造成一张图，然后通过GE将图下发到昇腾硬件执行。相对于单个算子依次下发的方式，图模式下，GE可以通过计算图优化、多流并行、内存复用、模型下沉等技术手段，加速模型执行效率，减少模型内存占用。

算子入图的开发流程如下图所示：算子工程创建完成后，基于工程代码框架完成算子原型定义、kernel侧算子实现、host侧tiling实现并完成算子入图开发，通过工程编译

脚本完成算子的编译部署，之后即可基于图IR执行算子，比如单算子模型执行或者IR构图的方式调用自定义算子。该开发流程以[2.10.2 工程化算子开发](#)为基础，除了需要提供工程化算子开发中的算子实现文件外，还需要额外交付算子入图的代码文件。



### 步骤1 环境准备。

1. CANN软件安装请参考[1.2 环境准备](#)。
2. [创建算子工程](#)。使用msOpGen工具创建算子开发工程。

### 步骤2 算子实现。

- [算子原型定义](#)。通过原型定义来描述算子输入输出、属性等信息以及算子在AI处理器上相关实现信息，并关联tiling实现等函数。
- Kernel侧算子实现和host侧tiling实现请参考[3.3 SIMD算子实现](#)；工程化算子开发，支持开发者调用Tiling API基于CANN提供的编程框架进行tiling开发，kernel侧也提供对应的接口方便开发者获取tiling参数，具体内容请参考[2.10.2.4 Kernel侧算子实现](#)和[2.10.2.5 Host侧Tiling实现](#)，由此而带来的额外约束也在上述章节说明。

**步骤3 [算子入图（GE图）开发](#)。**算子入图场景下，需要提供shape推导等算子入图适配函数的实现。

**步骤4 编译部署。**通过工程编译脚本完成算子的编译部署，分为[算子包编译](#)和[算子动态库编译](#)两种方式。

**步骤5 [图编译和图执行](#)：**基于图IR执行算子，比如单算子模型执行或者IR构图的方式调用自定义算子。

----结束

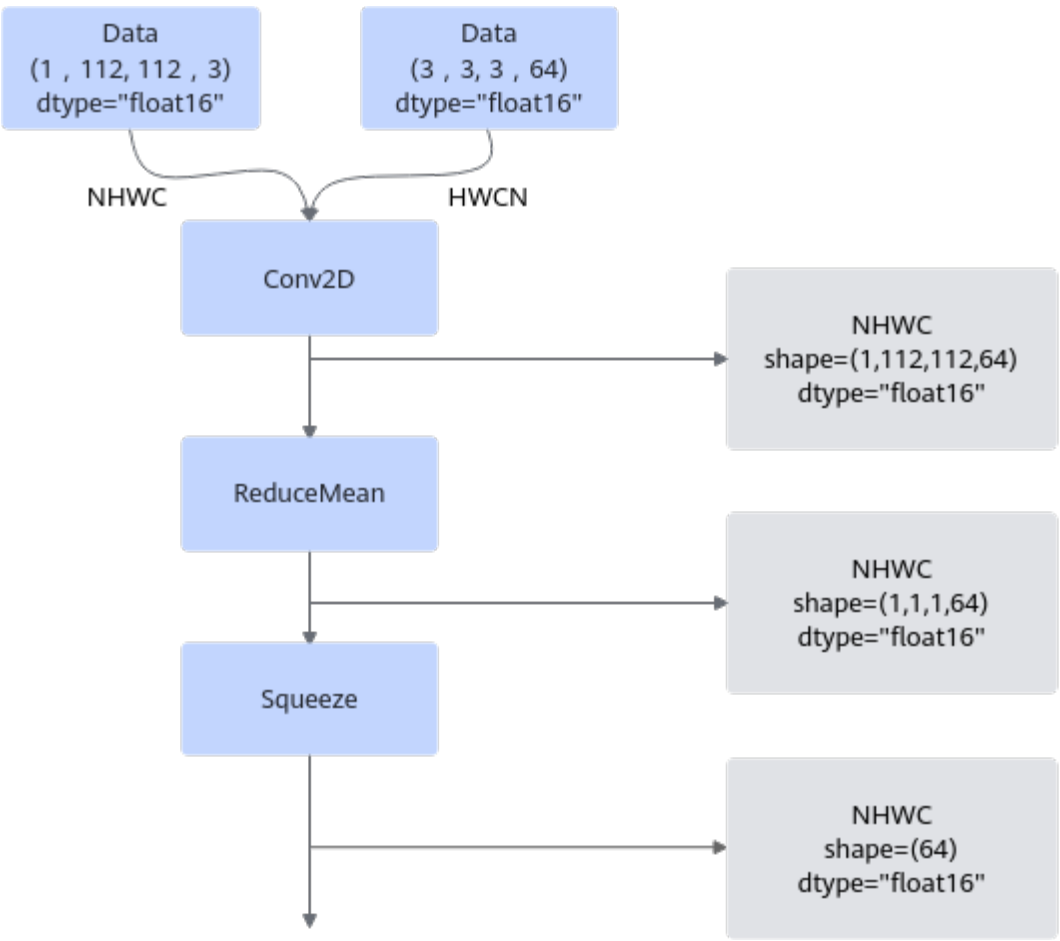
## 2.10.3.2 基本开发流程

该开发流程以[2.10.2 工程化算子开发](#)为基础，除了需要提供工程化算子开发中的算子实现文件外，还需要额外交付算子入图的代码文件。本节仅提供算子入图代码文件的开发指导。

假设下图是我们需要使用的网络模型，您可能会想直接逐个算子调用，根据输入tensor得到输出tensor就可以完成网络的运行，但在图模式场景下，实际的网络模型生成过程中，会先进行tensor shape以及datatype的推导。这样可以让我们在图执行之前，就知道各tensor的数据类型和形状，提前校验其正确性；同时提前推理出算子的输出张量描述，包括张量的形状、数据类型及数据排布格式等信息，算子构图准备阶段就可以为所有的张量静态分配内存，避免动态内存分配带来的开销。

下面的网络模型经过shape和datatype推导之后，可以得到灰色底纹框中的推导信息：

图 2-40 shape 与 datatype 推导示意图



除了tiling实现外，算子入图时需要额外提供的实现代码有以下几种：

- datatype推导：根据算子的输入datatype、算子逻辑及算子属性等信息，推理出算子的输出张量datatype。
- shape推导：根据算子的输入shape、算子逻辑及算子属性等信息，推理出算子的输出张量shape。
- ShapeRange推导：编译时无法推导输出shape，只能推导输出shape range，执行完才能得出输出shape。在下发时需要按照输出shape range来申请最大输出内存，该类算子需要提供ShapeRange推导函数。
- 声明数据依赖：部分算子在InferShape时，需要依赖某个输入的具体值才可以进行，这类算子被称为“数据依赖算子”，对应的输入被称为“数据依赖输入”。该类算子在注册时，需要声明其数据依赖输入。

下表列出了不同类型的算子对上述实现代码的要求。

表 2-33 不同的类型的算子对入图实现代码的要求

| 分类                     | 对入图实现代码的要求                                                                     |
|------------------------|--------------------------------------------------------------------------------|
| 根据输入shape可以推导出输出shape。 | <ul style="list-style-type: none"><li>• shape推导</li><li>• datatype推导</li></ul> |

| 分类                                                                     | 对入图实现代码的要求                                                                                                                             |
|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 依赖输入的value才能推导出输出shape，即数据依赖算子。如Reshape算子，依赖shape输入的value才能推导出输出shape。 | <ul style="list-style-type: none"><li>• shape推导</li><li>• datatype推导</li><li>• 声明数据依赖</li></ul>                                        |
| 编译时无法推导输出shape，只能推导输出shap range，执行完才能得出输出shape。                        | <ul style="list-style-type: none"><li>• Shape推导（必选）</li><li>• DataType推导（必选）</li><li>• ShapeRange推导（必选）</li><li>• 声明数据依赖（按需）</li></ul> |

实际开发时通过固定的datatype和shape推导原型实现推导函数，然后再通过SetInferShape、SetInferDataType接口来关联对应的shape推导函数，样例如下。

```
namespace ge {
static graphStatus InferShape(gert::InferShapeContext *context)
{
 ...
 return GRAPH_SUCCESS;
}

static graphStatus InferDataType(gert::InferDataTypeContext *context)
{
 ...
 return ge::GRAPH_SUCCESS;
}
} // namespace ge

namespace ops {
class AddCustom : public OpDef {
public:
 AddCustom(const char* name) : OpDef(name)
 {
 this->Input("x")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
 .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
 this->Input("y")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
 .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
 this->Output("z")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT16, ge::DT_FLOAT, ge::DT_INT32})
 .Format({ge::FORMAT_ND, ge::FORMAT_ND, ge::FORMAT_ND});
 // 根据用户的算子调用方式决定需不需要注册 图模式调用方式下需要
 this->SetInferShape(ge::InferShape);
 this->SetInferShapeRange(ge::InferShapeRange);
 this->SetInferDataType(ge::InferDataType);
 this->AICore()
 .SetTiling(optiling::TilingFunc);
 // 请替换为实际的昇腾AI处理器型号
 this->AICore().AddConfig("ascendxxx");
 }
};
OP_ADD(AddCustom);
} // namespace ops
```



## datatype 推导

以AddCustom算子为例，InferDataType的实现如下所示。该样例中输出tensor的数据类型与输入tensor的数据类型相同，所以直接将任意一个输入tensor的数据类型赋给输出tensor即可。

```
namespace ge {
static graphStatus InferDataType(gert::InferDataTypeContext* context)
{
 const auto inputDataType = context->GetInputDataType(0);
 context->SetOutputDataType(0, inputDataType);
 return ge::GRAPH_SUCCESS;
}
} // namespace ge
```

如下示例则给出了更灵活的datatype推导样例，当输入的数据类型为DT\_INT4时，其输出的数据类型为DT\_INT32。

```
ge::graphStatus InferDataTypeForFoo(gert::InferDataTypeContext* context) {
 if (context->GetInputDataType(0) == DT_INT4) {
 context->SetOutputDataType(0, DT_INT32);
 }
}
```

## shape 推导

简单的shape推导逻辑可以使用Follow接口来表达，比如输出shape和输入shape相同的情况。示例如下：输出“y1” Follow输入“x1” 场景，指定Follow模式为SHAPE，此时“y1”的shape将会和“x1”保持一致。

```
this->Input("x1")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT, ge::DT_FLOAT})
 .Format({ge::FORMAT_ND, ge::FORMAT_ND});
this->Input("x2")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT, ge::DT_FLOAT})
 .Format({ge::FORMAT_ND, ge::FORMAT_ND});
this->Output("y1")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT, ge::DT_FLOAT})
 .Format({ge::FORMAT_ND, ge::FORMAT_ND})
 .Follow("x1", FollowType::SHAPE);
```

无法在原型定义中通过Follow表达的情况需要开发者编写InferShape函数，InferShape函数的原型是固定的，如下示例，接受一个InferShapeContext作为输入，从此context上可以获取到输入、输出的shape指针等内容。输入shape为const类型，因此InferShape时，输入shape是只读、不允许修改的。InferShape成功后，返回ge::GRAPH\_SUCCESS，其他返回值被认为推导失败。推导失败后，执行过程结束退出。

以Reshape算子为例，InferShape的实现如下所示。根据第1个输入（shape输入）的值，Reshape算子将第0个输入（x输入）的shape做变换，并输出到其第0个输出（y输出）上。Reshape的InferShape实现为：

```
ge::graphStatus InferShapeForReshape(InferShapeContext *context) {
 const gert::Shape *x_shape = context->GetInputShape(0); // 获取第0个输入的shape
 const gert::Tensor *shape_tensor = context->GetInputTensor(1); // 获取第1个输入的tensor
 gert::Shape *output_shape = context->GetOutputShape(0);
 if (x_shape == nullptr || shape_tensor == nullptr || output_shape == nullptr) {
 // 防御式编程，不应该出现的场景，打印错误并返回失败
 return ge::GRAPH_FAILED;
 }
}
```

```
auto reshape_size = static_cast<int32_t>(shape_tensor->GetShapeSize());
if (reshape_size < 1) {
 // 防御式编程，不应该出现的场景，打印错误并返回失败
 return ge::GRAPH_FAILED;
}

// 根据原型信息，Reshape的shape输入支持INT32与INT64两类，根据不同的类型进入对应的模板函数中做真正的shape变换操作
if (shape_tensor->GetDataType() == ge::DT_INT32) {
 int32_t *reshape_data = shape_tensor->GetData<int32_t>();
 return ReshapeInferShapeImpl<int32_t>(reshape_data, *x_shape, *output_shape, reshape_size);
} else {
 int64_t *reshape_data = shape_tensor->GetData<int64_t>();
 return ReshapeInferShapeImpl<int64_t>(reshape_data, *x_shape, *output_shape, reshape_size);
}
}
```

InferShapeContext public继承自ExtendedKernelContext，因此ExtendedKernelContext中提供的方法如获取算子type、name、属性等接口均可以在InferShapeContext实例中调用。

### ⚠ 注意

- InferShape推导函数和Follow接口不能混用，即不支持部分输出采用InferShape推导、部分输出采用Follow推导的情况。若用户同时使用了InferShape函数和Follow接口，以用户的InferShape函数为准，需要在InferShape函数中能够推导出所有的输出shape。
- 为了效率考虑，调用InferShape函数时，框架不会为输出shape做初始化，因此，在InferShape函数中，可以认为输出是**未初始化的**状态。如果在InferShape时，希望通过Append方式操作输出shape，需要先将输出shape的DimNum清零，以防止出现未定义行为。

## InferShapeRange 实现

某些算子的输出Shape在计算完成后才能确定。比如unique算子，其Shape的推导逻辑如下：

给定一维Tensor x，找到其中不重复的元素，返回去重后的Tensor y，输出idx与输入x大小相同，保存x每个元素在y中的索引。

```
tensor 'x' is [1, 1, 2, 4, 4, 4, 7, 8, 8] x shape[9]
y, idx = unique(x)
y ==> [1, 2, 4, 7, 8] y shape[5]
idx ==> [0, 0, 1, 2, 2, 2, 3, 4, 4] idx shape[9]
```

由此可知，y的shape在编译时为[-1]，unique执行后shape才确定。

在入图场景执行时，需要在执行前分配输出内存，而内存的大小依赖于输出Shape和数据类型。对于此类算子，由于输出Shape在执行后才能确定，因此需要根据输出Shape的范围，按照最大范围申请输出内存，以确保有足够的空间供计算函数写入输出Tensor。

这种场景下，开发者需要自行实现InferShapeRange函数，来推导输出Shape的范围。下面以unique算子为例子，介绍InferShapeRange函数的实现方法。

```
ge::graphStatus UniqueInferShapeRangeFunc(gert::InferShapeRangeContext *context) {
 // 取输入的shape range
 auto x_shape_range = context->GetInputShapeRange(0U);
```

```
OPS_CHECK_NULL_WITH_CONTEXT(context, x_shape_range);
OPS_CHECK_NULL_WITH_CONTEXT(context, x_shape_range->GetMax());
OPS_CHECK_NULL_WITH_CONTEXT(context, x_shape_range->GetMin());

// 开始计算y输出的shape range
auto y_shape_range = context->GetOutputShapeRange(0U);
OPS_CHECK_NULL_WITH_CONTEXT(context, y_shape_range);
y_shape_range->GetMax()->SetDimNum(1); // 一维向量, rank为1
y_shape_range->GetMin()->SetDimNum(1);

auto x_max_shape = x_shape_range->GetMax();
auto x_shape_dimnum = x_max_shape->GetDim(0); // x为一维Tensor, 其shape为[n], x_shape_dimnum表示x输入的元素个数n
if (x_shape_dimnum == 1) {
 // 若x输入只有1个元素, 不存在去重, y的shape轴最小最大均为1. 因此range为[1~1]
 y_shape_range->GetMax()->SetDim(0, 1);
 y_shape_range->GetMin()->SetDim(0, 1);
} else {
 // 若x输入有0个元素, 或者大于1个元素, 去重后, y的元素个数最小为x的min, 最大为x的max
 y_shape_range->GetMax()->SetDim(0, x_shape_dimnum);
 y_shape_range->GetMin()->SetDim(0, x_shape_range->GetMin());
}

// 开始计算输出idx的shape range
// 输出idx表示x元素在y中的索引, 其元素个数与x相等, 因此shape range与x一致
auto idx_shape_range = context->GetOutputShapeRange(1U);
OPS_CHECK_NULL_WITH_CONTEXT(context, idx_shape_range);
*(idx_shape_range->GetMax()) = *(x_shape_range->GetMax());
*(idx_shape_range->GetMin()) = *(x_shape_range->GetMin());

return ge::GRAPH_SUCCESS;
}
```

## InferShape 时获取属性、输入

在InferShape、Tiling时, 可以通过context实例获取算子IR属性值, 所谓IR属性, 是指在IR注册时定义的属性, 以TransData算子为例:

```
namespace ops {
class TransData : public OpDef {
public:
 explicit TransData(const char *name) : OpDef(name)
 {
 this->Input("src")
 ...
 this->Output("dst")
 ...
 this->Attr("src_format")
 .AttrType(REQUIRED)
 .String();
 this->Attr("dst_format")
 .AttrType(REQUIRED)
 .String();
 this->Attr("group")
 .AttrType(OPTIONAL)
 .Int(1);
 ...
 }
};
OP_ADD(TransData);
} // namespace ops
```

其原型定义中声明了src\_format、dst\_format、group三个属性, 可以通过如下方式获取算子属性:

```
ge::graphStatus ExampleGetTransDataAttr(TilingContext *context) {
 // 获取所有属性
 const RuntimeAttrs *attrs = context->GetAttrs();
}
```

```
ASSERT_NOT_NULL(attrs);

// 按照在原型定义中的顺序, 使用index获取属性, index从0开始计数
const char *src_format = attrs->GetAttrPointer<char>(0); // 获取src_format, src_format是第一个属性, 因此index为0
const char *dst_format = attrs->GetAttrPointer<char>(1); // 获取dst_format, dst_format是第二个属性, 因此index为1
const int64_t group = attrs->GetAttrPointer<int64_t>(2); // 获取group, group是第三个属性, 因此index为2

return ge::GRAPH_SUCCESS;
}
```

通过index而不是字符串name来索引输入输出, 对于带有OPTIONAL、DYNAMIC类型输入的算子, 可能出现实例化后, 单纯通过index无法索引到具体输入的问题, 以DynamicRNNV3算子为例:

```
namespace ops {
class DynamicRNNV3 : public OpDef {
public:
 explicit DynamicRNNV3(const char *name) : OpDef(name)
 {
 this->Input("x")
 .ParamType(REQUIRED)
 ...
 this->Input("w")
 .ParamType(REQUIRED)
 ...
 this->Input("b")
 .ParamType(REQUIRED)
 ...
 this->Input("seq_length")
 .ParamType(OPTIONAL)
 ...
 this->Input("init_h")
 .ParamType(OPTIONAL)
 ...
 this->Input("init_c")
 .ParamType(OPTIONAL)
 ...
 this->Input("wci")
 .ParamType(OPTIONAL)
 ...
 this->Input("wcf")
 .ParamType(OPTIONAL)
 ...
 this->Input("mask")
 .ParamType(OPTIONAL)
 ...
 this->Input("mask")
 .ParamType(OPTIONAL)
 ...
 this->Input("project")
 .ParamType(OPTIONAL)
 ...
 }
};
OP_ADD(DynamicRNNV3);
} // namespace ops
```

由于DynamicRNNV3算子有连续的多个optional输入, 这导致init\_h及其后面的输入的实例化后index都是不确定的, 对于这种类型的算子, 可以通过GetOptionalInputShape传入原型对应的index来获取对应的输入shape等数据, 以InferShape为例:

```
ge::graphStatus InferShapeForDynamicRNNV3(InferShapeContext *context) {
 // 对于前两个输入, 不受到optional或dynamic的影响, 可以按照常规方法获取输入shape
```

```
auto x_shape = context->GetInputShape(0);
auto w_shape = context->GetInputShape(1);
if (x_shape == nullptr || w_shape == nullptr) {
 return ge::GRAPH_FAILED;
}

int64_t state_size = 0;
// 在原型定义上, project是第11个输入(从0开始计数)
constexpr int64_t kProjectInputIndex = 11;

// 受到前面optional输入影响的, project实例化后输入的index是不确定的, 通过GetOptionalInputShape来获取对应的输入shape,
// GetOptionalInputShape的入参为原型上对应的index
auto project_shape = context->GetOptionalInputShape(kProjectInputIndex);
if (project_shape != nullptr) {
 if (project_shape->GetDimNum() < 2) {
 return ge::GRAPH_FAILED;
 }
 state_size = project_shape->GetDim(1);
}
// 更多的inferred shape逻辑...
return ge::GRAPH_SUCCESS;
}
```

对于dynamic类型的输入, 实例化后的输入可能是一到多个, 对于此类输入, 获取方式为:

```
// ir_index: 此输入在原型定义中的index, 从0开始计数
// relative_index: 该输入实例化后的相对index, 从0开始计数, 例如某个DYNAMIC_INPUT实例化了3个, 要取第二个, 那么relative_index = 1
auto shape = context->GetDynamicInputShape(ir_index, relative_index);
```

本节举例的获取optional、dynamic输入的方式, 在InferShape、Tiling函数中均可以调用。

## 数据依赖

一般来说, 具备输入shape后, 算子可以通过InferShape推导出输出shape。然而部分算子在InferShape时, 需要依赖某个输入的具体值才可以进行, 这类算子被称为“数据依赖算子”, 对应的输入被称为“数据依赖输入”。以Reshape算子为例, 其依据shape输入的描述, 对输入的shape做调整, 因此Reshape算子依赖shape输入的值。这类算子需要在原型定义时通过ValueDepend接口声明对应的输入为数据依赖输入。

```
namespace ops {
class Reshape : public OpDef {
public:
 explicit Reshape(const char *name) : OpDef(name)
 {
 ...
 this->Input("shape")
 .ParamType(REQUIRED)
 ...
 .ValueDepend(REQUIRED) // 声明 Reshape算子的shape输入为数据依赖输入
 ...
 }
};
OP_ADD(Reshape);
} // namespace ops
```

根据第1个输入 (shape输入) 的值, Reshape算子将第0个输入 (x输入) 的shape做变换, 并输出到其第0个输出 (y输出) 上。Reshape的InferShape实现为:

```
// shape变换具体实现
template<typename T>
ge::graphStatus ReshapeInferShapeImpl(const T *reshape_dims, const gert::Shape &x_shape, gert::Shape &output_shape, int32_t reshape_rank) {
 constexpr T UNKNOWN_DIM = -1;
```

```
// 将算子输出的维度数设置为reshape后的维度数reshape_rank
output_shape.SetDimNum(reshape_rank);
auto x_shape_size = x_shape.GetShapeSize();
int64_t output_shapsize = 1;
size_t unknown_dim_idx = std::numeric_limits<size_t>::max();
for (int32_t i = 0; i < reshape_rank; i++) {
 if (reshape_dims[i] != UNKNOWN_DIM) { // reshape后某一轴的维度值不为-1
 output_shape.SetDim(i, reshape_dims[i]); // 设置输出的维度值为reshape后的维度值
 output_shapsize *= reshape_dims[i]; // 计算当前输出元素数量
 } else {
 output_shape.SetDim(i, 1); // reshape后某一轴的维度值为-1, 临时设置输出的维度值为1, 后续计算
 // 后看是否可以推导出确定值, 并记录未知维度的索引
 unknown_dim_idx = i;
 }
}
if (unknown_dim_idx == std::numeric_limits<size_t>::max() && output_shapsize == x_shape_size) {
 return ge::GRAPH_SUCCESS; // 不存在未知维度, 且输出shape size和输入x的shape size一致, 直接返回
成功
} else if (unknown_dim_idx != std::numeric_limits<size_t>::max() && x_shape_size % output_shapsize ==
0) {
 output_shape.SetDim(unknown_dim_idx, x_shape_size / output_shapsize); // 存在未知维度, 根据输入
shape动态调整未知维度值保持总元素个数不变
 return ge::GRAPH_SUCCESS;
}
return ge::GRAPH_FAILED;
}

ge::graphStatus InferShapeForReshape(InferShapeContext *context) {
 const gert::Shape *x_shape = context->GetInputShape(0); // 获取第0个输入的shape
 const gert::Tensor *shape_tensor = context->GetInputTensor(1); // 获取第1个输入的tensor
 gert::Shape *output_shape = context->GetOutputShape(0);
 if (x_shape == nullptr || shape_tensor == nullptr || output_shape == nullptr) {
 // 防御式编程, 不应该出现的场景, 打印错误并返回失败
 return ge::GRAPH_FAILED;
 }

 auto reshape_size = static_cast<int32_t>(shape_tensor->GetShapeSize());
 if (reshape_size < 1) {
 // 防御式编程, 不应该出现的场景, 打印错误并返回失败
 return ge::GRAPH_FAILED;
 }

 // 根据原型信息, Reshape的shape输入支持INT32与INT64两类, 根据不同的类型进入对应的模板函数中做真
 正的shape变换操作
 if (shape_tensor->GetDataType() == ge::DT_INT32) {
 int32_t *reshape_data = shape_tensor->GetData<int32_t>();
 return ReshapeInferShapeImpl<int32_t>(reshape_data, *x_shape, *output_shape, reshape_size);
 } else {
 int64_t *reshape_data = shape_tensor->GetData<int64_t>();
 return ReshapeInferShapeImpl<int64_t>(reshape_data, *x_shape, *output_shape, reshape_size);
 }
}
```

### 注意

- 只有声明过数据依赖的输入, 才可以在InferShape时调用GetInputTensor等获取tensor的接口获取其对应的tensor数据。若对一个未声明数据依赖的输入调用GetInputTensor等获取tensor的接口, 只能在tensor中获取到正确的shape、format、datatype信息, 无法获取到真实的tensor数据地址(获取到的地址为nullptr)。
- 从tensor中获取tensor\_data时(GetData<int32\_t>或GetData<int64\_t>), 使用者需要保证获取的数据类型是正确的, 否则行为是未定义的。

### 2.10.3.3 使能 Tiling 下沉

在静态图模式下，可以通过**整图下沉**优化调度性能。将完整的计算图一次性下发至 Device 侧，后续执行则无需 Host 参与，由 Device 自主完成计算，从而减少 Host-Device 交互开销，提升执行效率。部分算子的 Tiling 计算依赖运行时输入的具体数值（**Tiling 值依赖**），需在执行时动态计算 Tiling 参数。针对该场景，可采用**Tiling 下沉**优化方案：将 Tiling 计算下沉至 Device 侧的 AI CPU 上执行，从而实现计算全程在 Device 侧高效完成。

#### 说明

- 基于新版本 CANN 包（支持 Tiling 下沉特性）编译生成的 Tiling 下沉算子，不兼容旧版 CANN（不支持 Tiling 下沉特性）运行环境。
- 当前仅融合算子（向量计算和矩阵计算融合）支持进行 Tiling 下沉。
- Tiling 下沉功能仅支持如下产品型号：
  - Atlas A3 训练系列产品/Atlas A3 推理系列产品
  - Atlas A2 训练系列产品/Atlas A2 推理系列产品

自定义算子使能 Tiling 下沉的步骤如下，完整样例请参考[Tiling 下沉算子样例](#)。

Tiling 下沉场景下，算子工程的 op\_host 目录结构如下，Tiling 实现文件需单独放在在一个 cpp 文件中，示例中为 add\_custom\_tiling\_sink\_tiling.cpp。

```
├── op_host
│ ├── add_custom_tiling_sink.cpp // 算子原型定义、InferShape、InferDataType 实现
│ ├── add_custom_tiling_sink_tiling.cpp // Tiling 函数实现
│ ├── add_custom_tiling_sink_tiling.h // TilingData 结构体定义、Tiling 函数声明
│ └── CMakeLists.txt
```

以 AddCustom 算子为例，讲解关键代码文件的具体实现方法如下：

- TilingData 结构体定义、Tiling 函数声明头文件 add\_custom\_tiling\_sink\_tiling.h
  - 进行 TilingData 结构体的定义
  - 进行 Tiling 实现函数的声明

```
#ifndef ADD_CUSTOM_TILING_SINK_TILING_H
#define ADD_CUSTOM_TILING_SINK_TILING_H
#include "register/tilingdata_base.h"
#include "register/op_def_registry.h"

namespace optiling {
BEGIN_TILING_DATA_DEF(TilingSinkTilingData)
TILING_DATA_FIELD_DEF(uint32_t, totalLength);
TILING_DATA_FIELD_DEF(uint32_t, tileNum);
END_TILING_DATA_DEF;

REGISTER_TILING_DATA_CLASS(AddCustomTilingSink, TilingSinkTilingData) // Tiling 结构体定义

ge::graphStatus AddCustomSinkTilingFunc(gert::TilingContext* context); // Tiling 函数声明
} // namespace optiling
#endif // ADD_CUSTOM_TILING_SINK_TILING_H
```

- 算子原型定义、InferShape、InferDataType 实现文件 add\_custom\_tiling\_sink.cpp，需包含 add\_custom\_tiling\_sink\_tiling.h，进行 Tiling 函数和算子原型定义的关联。

Tiling 下沉仅适用于存在 Tiling 值依赖（即当 InferShape 不依赖输入值，仅 Tiling 计算需要输入值）且算子输入为非 Const 类型的场景，本示例中的输入 y 通过 ValueDepend 配置了非 Const 类型的 Tiling 值依赖。

```
#include "add_custom_tiling_sink_tiling.h" // 包含头文件

// ...
```

```
namespace ops {
class AddCustomTilingSink : public OpDef {
public:
 explicit AddCustomTilingSink(const char *name) : OpDef(name)
 {
 this->Input("x")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT})
 .Format({ge::FORMAT_ND});
 this->Input("y")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT})
 .Format({ge::FORMAT_ND})
 .ValueDepend(OPTIONAL, DependScope::TILING); // 表示输入y为Tiling值依赖
 this->Output("z")
 .ParamType(REQUIRED)
 .DataType({ge::DT_FLOAT})
 .Format({ge::FORMAT_ND});

 this->SetInferShape(ge::InferShape).SetInferDataType(ge::InferDataType);

 this->AICore().SetTiling(optiling::AddCustomSinkTilingFunc); // Tiling函数和算子原型定义的关联

 // 请替换为实际的昇腾AI处理器型号
 this->AICore().AddConfig("ascendxxx");
 }
};
OP_ADD(AddCustomTilingSink);
} // namespace ops
```

- Tiling函数的实现文件add\_custom\_tiling\_sink\_tiling.cpp

- Tiling函数中通过判断值依赖InputTensor即输入y的数据指针是否为空指针来确认当前是否处于编译期。Tiling下沉场景，编译期需要为算子分配内存，包括其所需的workspace。为了保证运行时的高效性，编译期应根据算子的执行需求，合理设置所需的workspace最大值，以避免内存不足或浪费。AddCustomTilingSink样例不需要用户workspace，不涉及设置，此处设置为固定值仅作为示例。
- 完成下沉Tiling函数注册：包含device\_op\_impl\_registry.h头文件，使用宏DEVICE\_IMPL\_OP\_OPTILING进行注册。

```
#include "add_custom_tiling_sink_tiling.h"
#include "register/device_op_impl_registry.h"
#include "tiling/platform/platform_ascendc.h"

namespace optiling {
static constexpr uint32_t NUM_BLOCKS = 8;
static constexpr uint32_t TILE_NUM = 3;
static constexpr size_t MAX_WORKSPACE_SIZE = 32; // 算子所需用户workspace空间最大值，
AddCustomTilingSink算子本身逻辑无需用户workspace空间，此处设置为固定值仅作为示例
static constexpr size_t DEFAULT_WORKSPACE_SIZE = 0;
ge::graphStatus AddCustomSinkTilingFunc(gert::TilingContext *context)
{
 TilingSinkTilingData tiling;
 uint32_t totalLength = context->GetInputTensor(0)->GetShapeSize();
 context->SetBlockDim(NUM_BLOCKS);
 tiling.set_totalLength(totalLength);
 tiling.set_tileNum(TILE_NUM);
 tiling.SaveToBuffer(context->GetRawTilingData()->GetData(), context->GetRawTilingData()->GetCapacity());
 context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());
 size_t *currentWorkspace = context->GetWorkspaceSizes(1);
 auto platform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
 size_t sysWorkspaceSize = platform.GetLibApiWorkSpaceSize();
 currentWorkspace[0] = sysWorkspaceSize + DEFAULT_WORKSPACE_SIZE; // 设置运行时workspace大小，
 此处为系统workspace空间 + 用户workspace空间
 if (context->GetInputTensor(1) != nullptr && context->GetInputTensor(1)->GetData<float>() ==
```



```
nullptr) {
 // 通过判断值依赖InputTensor的Data是否为空指针来确认当前是否处于编译期。
 // Tiling下沉场景，编译期需要为算子分配内存，包括其所需的workspace。为了保证运行时的高效性，编译期应根据算子的执行需求，合理设置所需的workspace最大值，以避免内存不足或浪费。
 currentWorkspace[0] = sysWorkspaceSize + MAX_WORKSPACE_SIZE; // 设置编译期workspace大小，此处为系统workspace空间 + 用户workspace空间最大值
}
return ge::GRAPH_SUCCESS;
}
DEVICE_IMPL_OP_OPTILING(AddCustomTilingSink).Tiling(optiling::AddCustomSinkTilingFunc); // 下沉Tiling函数注册
} // namespace optiling
```

- 算子核函数实现

当前Tiling下沉仅支持融合算子，为了模拟融合算子场景，通过

KERNEL\_TASK\_TYPE\_DEFAULT接口强制指定算子在AIC、AIV混合场景运行。

```
extern "C" __global__ __aicore__ void add_custom_tiling_sink(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR workspace, GM_ADDR tiling)
{
 GET_TILING_DATA(tiling_data, tiling);
 KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_MIX_AIC_1_2); // 将算子强制指定在AIC、AIV混合场景运行，模拟融合算子场景
 if ASCEND_IS_AIC {
 return;
 }
 AscendC::KernelAdd op;
 op.Init(x, y, z, tiling_data.totalLength, tiling_data.tileNum);
 op.Process();
}
```

- 修改op\_host目录下的编译脚本CMakeLists.txt，添加Tiling下沉编译命令。具体代码如下所示：

```
通过ascendc_device_library添加Tiling下沉编译任务
ascendc_device_library(TARGET cust_opmaster # 任务名称，固定为cust_opmaster
 OPTION SHARED # 动态库（当前仅支持动态库入图下沉）
 SRC ${CMAKE_CURRENT_SOURCE_DIR}/add_custom_tiling_sink_tiling.cpp) # Tiling函数实现代码源文件
```

### 2.10.3.4 SuperKernel 开发

SuperKernel是一种算子的二进制融合技术，与源码融合不同，它聚焦于内核函数(Kernel)的二进制的调度方案，展开深度优化，于已编译的二进制代码基础上融合创建一个超级Kernel函数（SuperKernel），以调用子函数的方式调用多个其他内核函数，也就是子Kernel。相对于单算子下发，SuperKernel技术可以减少任务调度等待时间和调度开销，同时利用Task间隙资源进一步优化算子头开销。

#### 📖 说明

- SuperKernel仅适用于静态图场景。
- SuperKernel适用于如下型号：
  - Atlas A3 训练系列产品/Atlas A3 推理系列产品

### 自定义算子支持 SuperKernel

自定义算子支持SuperKernel与普通算子在开发流程上并无显著差异，但需注意一些**特定约束**（详见下文）。当前SuperKernel特性仅支持在Pytorch框架使用，所以完成**算子入图（GE图）开发**后，还需要参考《[PyTorch图模式使用指南\(TorchAir\)](#)》中的“自定义算子入图”章节，完成Pytorch入图。同时，TorchAir提供标定SuperKernel范围的能力，用户可根据实际业务需求对融合范围内的算子进行标记和优化配置。具体内容请参考《[PyTorch图模式使用指南\(TorchAir\)](#)》中的“[max-autotune模式功能>图内标定SuperKernel范围](#)”章节。

开发时的**特定约束**说明如下：

- 自定义算子若进行全核同步，需注意子Kernel（即该算子）启动的核数与SuperKernel的核数一致。若子Kernel启动的核数少于SuperKernel的核数，全核同步会等待所有核完成，导致卡住超时。  
注：SuperKernel启动核数为子Kernel的最大启动核数。假设SuperKernel包括算子a（启动核数为4）和算子b（启动核数为2），此时SuperKernel启动核数为4。
  - 使用SyncAll时，为了解决该问题，可以通过在标定SuperKernel范围时开启feed-sync-all功能，此时系统会在SuperKernel内子Kernel的其余核中插入SyncAll指令，防止卡住超时。
  - 若使用CrossCoreSetFlag和CrossCoreWaitFlag硬同步接口实现全核同步，仅支持子Kernel启动核数与SuperKernel核数相同。
- 若自定义算子的Kernel类型设置为KERNEL\_TYPE\_MIX\_AIC\_1\_1，因为SuperKernel会根据启动核数等信息调整SuperKernel的启动比例，此时需特别注意该算子也可以适应SuperKernel的1:2启动比例，确保AIC与AIV之间的硬同步操作正确执行。比如：算子内部使用了AIC与AIV之间的硬同步接口（CrossCoreSetFlag和CrossCoreWaitFlag），不要单独指定某些AIV核调用硬同步接口，使所有AIV核均调用硬同步接口，防止因为硬同步数量不匹配而导致卡死超时；使用Matmul高阶API时，算子逻辑应保证仅有一个AIV0核调用Matmul接口，防止启动两个AIV核之后出现AIV1核消息无法接收导致卡死超时。
- 在开发自定义算子时，开发者必须确保所有对GM的标量读写操作都按需正确插入DataCacheCleanAndInvalid指令：在单算子编译场景下，毕昇编译器自动在算子末尾添加DataCacheCleanAndInvalid指令，刷新整个DCache（数据缓存）。在SuperKernel中，子Kernel被当做普通函数处理，编译器不会自动插入该指令来确保数据缓存一致性，开发者需要自行保证避免因容错机制改变而导致错误。

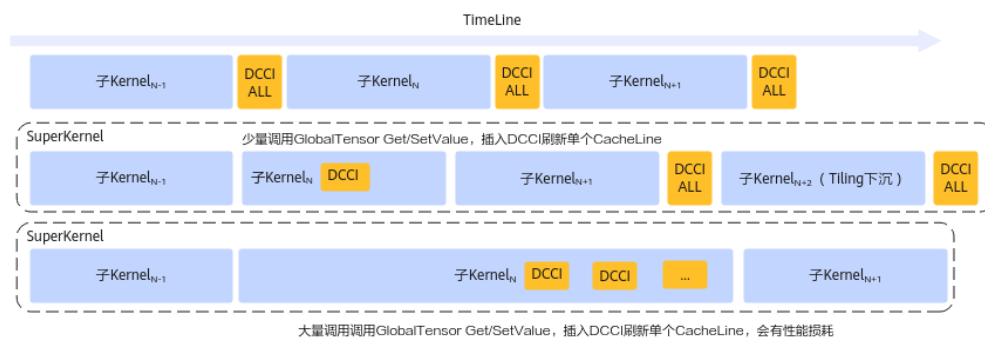
出于性能考虑，SuperKernel场景下Cache刷新机制如下：

如果开发者调用GlobalTensor的GetValue和SetValue接口对GM进行标量读写，SuperKernel编译时会自动在两个接口内部插入DataCacheCleanAndInvalid指令刷新单个Cache Line，保证一定的数据缓存一致性。不会在子Kernel调用前后插入DataCacheCleanAndInvalid。

但需要注意的是，过多调用GetValue和SetValue，在SuperKernel场景下会导致性能劣化，开发者需要尽量减少该接口调用。对于劣化过多的算子，SuperKernel提供了编译选项dcci-before-kernel-start、dcci-after-kernel-start、dcci-disable-on-kernel，可以关闭指定算子内GetValue/SetValue中自动插入的缓存刷新指令以提升模型性能，最终由模型用户决定是否在SuperKernel调用该算子前或后插入整个DCache刷新，编译选项具体内容请参考[图内标定SuperKernel范围](#)中编译选项说明。

特别地，对于Tiling下沉场景，通常会涉及二进制复用优化，无法在线选择上述的Cache刷新机制，SuperKernel框架统一在每个子Kernel调用前后都插入DataCacheCleanAndInvalid指令，刷新整个DCache。不会在GetValue和SetValue自动进行缓存刷新。

Cache刷新机制示意图如下图所示：



- 在子Kernel中调用GetBlockNum接口获取核数时，无论是否融合SuperKernel，获取的核数保持不变，不受SuperKernel启动核数的影响。因此，在使用该接口时，开发者无需特别关注SuperKernel的启动核数，使用方法和开发普通算子时一样。
- 针对Atlas A3 训练系列产品/Atlas A3 推理系列产品中，在不使能SuperKernel场景下，TPipe::Destroy接口内部最后会插入AscendC::PipeBarrier<PIPE\_ALL>()指令，额外保障多个TPipe之间的流水同步；模型中绝大部分算子只会使用一个TPipe对象，在对象析构时会调用Destroy，为不阻塞SetNextTaskStart和WaitPreTaskEnd性能提升，SuperKernel场景下默认关闭了TPipe::Destroy中插入的AscendC::PipeBarrier<PIPE\_ALL>()指令，所以当算子需要多个TPipe对象并手动调用Destroy函数时，开发者需自行保障TPipe对象间流水的同步。

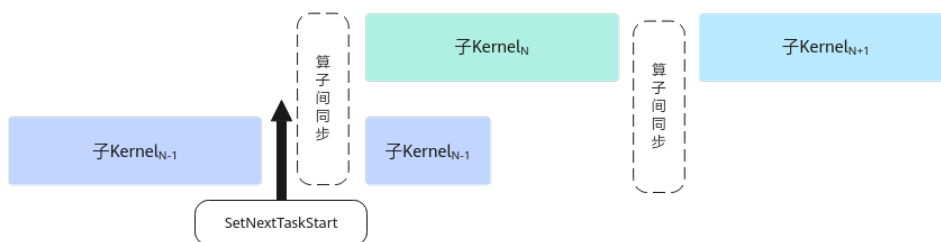
## 性能优化建议

- 任务间同步

此外，开发者在进行Kernel侧编程时，可以通过调用SetNextTaskStart和WaitPreTaskEnd两个任务间接口，进一步提升性能。

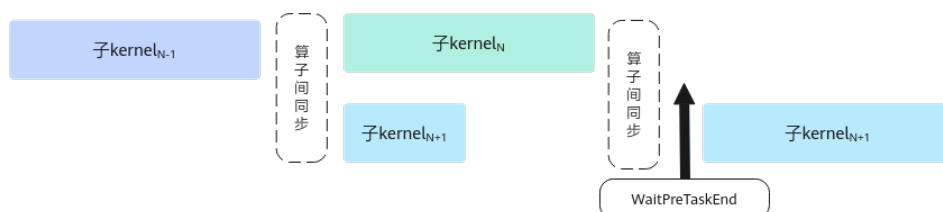
- 调用SetNextTaskStart后的指令可以和后续其他的子Kernel实现并行，提升整体性能。如图2-41所示，SuperKernel按序调用子Kernel，为保证子Kernel之间数据互不干扰，会在子Kernel间插入算子间同步进行保序，子Kernel<sub>N-1</sub>调用该接口后，之后的指令会和后续子Kernel<sub>N</sub>实现并行。

图 2-41 通过 SetNextTaskStart 实现并行示意图



- 调用WaitPreTaskEnd前的指令可以和前序其他的子Kernel实现并行，提升整体性能。如图2-42所示，SuperKernel按序调用子Kernel，为保证子Kernel之间数据互不干扰，会在子Kernel间插入算子间同步进行保序，子Kernel<sub>N+1</sub>调用该接口之前的指令会和前序子Kernel<sub>N</sub>实现并行。

图 2-42 通过 WaitPreTaskEnd 实现并行示意图



- Tiling下沉场景，可以通过--op\_relocatable\_kernel\_binary编译选项，开启二进制复用优化，提升编译性能，具体可参考[LINK](#)。

### 2.10.3.5 图编译和图执行

本节通过单算子模型执行的样例来介绍图模式下图编译和图执行流程。单算子模型执行是指基于图IR执行算子，先编译算子（例如，使用ATC工具将Ascend IR定义的单算子描述文件编译成算子om模型文件），再调用acl接口加载算子模型，最后调用acl接口执行算子。下文仅提供单算子模型执行的样例和基础内容讲解，详细内容请参考单算子模型执行。

### 环境要求

- 已参考[1.2 环境准备](#)，完成CANN驱动和软件的安装，配置CANN软件所需基本环境变量。

安装CANN软件后，使用CANN运行用户进行编译、运行时，需要以CANN运行用户登录环境，执行`source ${INSTALL_DIR}/set_env.sh`命令设置环境变量。`${INSTALL_DIR}`请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：`/usr/local/Ascend/cann`。

- 已参考[2.10.2 工程化算子开发](#)完成算子的开发和部署。

### 准备验证代码工程

代码工程目录结构如下，您可以单击[LINK](#)，获取样例工程的完整样例：

|                               |                                       |
|-------------------------------|---------------------------------------|
| input                         | // 存放脚本生成的输入数据目录                      |
| output                        | // 存放算子运行输出数据和真值数据的目录                 |
| inc                           | // 头文件目录                              |
| common.h                      | // 声明公共方法类，用于读取二进制文件                  |
| operator_desc.h               | // 算子描述声明文件，包含算子输入/输出，算子类型以及输入描述与输出描述 |
| op_runner.h                   | // 算子运行相关信息声明文件，包含算子输入/输出个数，输入/输出大小等  |
| src                           |                                       |
| CMakeLists.txt                | // 编译规则文件                             |
| common.cpp                    | // 公共函数，读取二进制文件函数的实现文件                |
| main.cpp                      | // 将单算子编译为om文件并加载om文件执行               |
| operator_desc.cpp             | // 构造算子的输入与输出描述                       |
| op_runner.cpp                 | // 单算子编译与运行函数实现文件                     |
| scripts                       |                                       |
| verify_result.py              | // 真值对比文件                             |
| gen_data.py                   | // 输入数据和真值数据生成脚本文件                    |
| acl.json                      | // acl配置文件                            |
| add_custom_static_shape.json  | // 算子描述文件，用于构造静态shape单算子模型文件          |
| add_custom_dynamic_shape.json | // 算子描述文件，用于构造动态shape单算子模型文件          |

### 生成单算子离线模型文件

- 步骤1** 构造静态shape单算子描述文件`add_custom_static_shape.json`，描述算子的输入、输出及属性等信息。

AddCustom静态shape算子的描述文件示例如下：

```
[
 {
 "op": "AddCustom",
 "input_desc": [
 {
 "name": "x",
 "param_type": "required",
 "format": "ND",
 "shape": [8, 2048],
 "type": "float16"
 },
 {
 "name": "y",
 "param_type": "required",
 "format": "ND",
 "shape": [8, 2048],
 "type": "float16"
 }
],
 "output_desc": [
 {
 "name": "z",
 "param_type": "required",
 "format": "ND",
 "shape": [8, 2048],
 "type": "float16"
 }
]
 }
]
```

AddCustom动态shape算子的描述文件示例如下：

```
[
 {
 "op": "AddCustom",
 "input_desc": [
 {
 "name": "x",
 "param_type": "required",
 "format": "ND",
 "shape": [-1, -1],
 "shape_range": [[1,-1],[1,-1]],
 "type": "float16"
 },
 {
 "name": "y",
 "param_type": "required",
 "format": "ND",
 "shape": [-1, -1],
 "shape_range": [[1,-1],[1,-1]],
 "type": "float16"
 }
],
 "output_desc": [
 {
 "name": "z",
 "param_type": "required",
 "format": "ND",
 "shape": [-1, -1],
 "shape_range": [[1,-1],[1,-1]],
 "type": "float16"
 }
]
 }
]
```

**步骤2** 使用ATC工具，将该算子描述文件编译成单算子模型文件（\*.om文件）

ATC工具的命令示例如下：

```
atc --singleop=$HOME/op_verify/run/out/test_data/config/add_custom_static_shape.json --
output=op_models/ --soc_version=<soc_version>
```

关键参数解释如下（详细参数说明，请参见《[ATC离线模型编译工具用户指南](#)》。）：

- --singleop：单算子描述文件（json格式）的路径。
- --output：存放om模型文件的目录。
- --soc\_version：配置为AI处理器的型号，请根据实际环境进行替换。

### 📖 说明

AI处理器的型号请通过如下方式获取：

- 针对如下产品：在安装AI处理器的服务器执行**npusmi info**命令进行查询，获取**Name**信息。实际配置值为AscendName，例如**Name**取值为xxxxyy，实际配置值为Ascendxxxxyy。  
Atlas A2 训练系列产品/Atlas A2 推理系列产品  
Atlas 200I/500 A2 推理产品  
Atlas 推理系列产品  
Atlas 训练系列产品
- 针对如下产品，在安装AI处理器的服务器执行**npusmi info -t board -i id -c chip\_id**命令进行查询，获取**Chip Name**和**NPU Name**信息，实际配置值为Chip Name\_NPU Name。例如**Chip Name**取值为Ascendxxx，**NPU Name**取值为1234，实际配置值为Ascendxxx\_1234。其中：
  - id：设备id，通过**npusmi info -l**命令查出的NPU ID即为设备id。
  - chip\_id：芯片id，通过**npusmi info -m**命令查出的Chip ID即为芯片id。  
Atlas A3 训练系列产品/Atlas A3 推理系列产品

以上命令执行后，会在output参数指定路径下生成\*.om后缀的离线模型文件。

----结束

## 生成测试数据

在样例工程目录下，执行如下命令：

```
python3 scripts/gen_data.py
```

会在input目录下生成两个shape为(8,2048)，数据类型为float16的数据文件input\_0.bin与input\_1.bin，用于进行AddCustom算子的验证。

代码样例如下：

```
import numpy as np
a = np.random.randint(100, size=(8, 2048,)).astype(np.float16)
b = np.random.randint(100, size=(8, 2048,)).astype(np.float16)
a.tofile('input_0.bin')
b.tofile('input_1.bin')
```

## 编写验证代码

您可以参考如下样例编写单算子加载、执行的代码逻辑。

以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考，调用接口后，需增加异常处理的分支，并记录报错日志、提示日志，此处不一一列举。

```
// 1.初始化
aclRet = aclInit("../scripts/acl.json");

// 2.运行管理资源申请
int deviceId = 0;
aclRet = aclrtSetDevice(deviceId);
// 获取软件栈的运行模式，不同运行模式影响后续的接口调用流程（例如是否进行数据传输等）
aclrtRunMode runMode;
bool g_isDevice = false;
aclError aclRet = aclrtGetRunMode(&runMode);
g_isDevice = (runMode == ACL_DEVICE);

// 3.加载单算子模型文件（*.om文件）
// 该目录相对可执行文件所在的目录，例如，编译出来的可执行文件存放在output目录下，此处就表示工程目录下的op_models目录
aclRet = aclOpSetModelDir("../op_models");

// 4.设置算子的输入，申请内存，然后读取输入数据input_0.bin与input_1.bin并保存至申请的内存中
//

// 5.创建Stream流
aclrtStream stream = nullptr;
aclrtCreateStream(&stream)

// 6.执行算子
// opType表示算子类型名称，例如AddCustom
// numInputs表示算子输入个数，例如AddCustom算子是2个输入
// inputDesc表示算子输入tensor描述的数组，描述每个输入的format、shape、数据类型
// inputs表示算子输入tensor数据
// numOutputs表示算子输出个数，例如AddCustom算子是1个输出
// outputDesc表示算子输出tensor描述的数组，描述每个输出的format、shape、数据类型
// outputs表示算子输出tensor数据
// attr表示算子属性，如果算子没有属性，也需要调用aclOpCreateAttr接口创建aclOpAttr类型的数据
// stream用于维护一些异步操作的执行顺序

aclOpExecuteV2(opType, numInputs, inputDesc, inputs,
 numOutputs, outputDesc, outputs, attr, nullptr);

// 7.阻塞应用运行，直到指定Stream中的所有任务都完成
aclrtSynchronizeStream(stream);

// 8.处理执行算子后的输出数据，例如在屏幕上显示、写入文件等，由用户根据实际情况自行实现，本示例会将结果写入output_z.bin文件中
//

// 9.释放stream流
aclrtDestroyStream(stream);

// 10.释放运行管理资源
aclRet = aclrtResetDevice(deviceId);
aclRet = aclFinalize();

//
```

## 运行和验证

1. 开发环境上，设置环境变量，配置单算子验证程序编译依赖的头文件与库文件路径，如下为设置环境变量的示例。\${INSTALL\_DIR}请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。{arch-os}为运行环境的架构和操作系统，arch表示操作系统架构，os表示操作系统，例如x86\_64-linux。

```
export DDK_PATH=${INSTALL_DIR}
export NPU_HOST_LIB=${INSTALL_DIR}/${arch-os}/devlib
```
2. 编译样例工程，生成单算子验证可执行文件。

- a. 切换到样例工程根目录，然后在样例工程根目录下执行如下命令创建目录用于存放编译文件，例如，创建的目录为“build”。  
**mkdir -p build**
  - b. 进入build目录，执行cmake编译命令，生成编译文件  
命令示例如下所示：  
**cd build**  
**cmake ../src -DCMAKE\_SKIP\_RPATH=TRUE**
  - c. 执行如下命令，生成可执行文件。  
**make**  
会在工程目录的output目录下生成可执行文件**execute\_add\_op**。
3. 执行单算子
- a. 以运行用户（例如HwHiAiUser）拷贝开发环境中样例工程output下的**execute\_add\_op**到运行环境任一目录。  
说明：若您的开发环境即为运行环境，此拷贝操作可跳过。
  - b. 在运行环境中，执行**execute\_add\_op**文件，验证单算子模型文件。  
**chmod +x execute\_add\_op**  
**./execute\_add\_op**  
会有如下屏显信息：

```
[INFO] static op will be called
[INFO] Set device[0] success
[INFO] Get RunMode[1] success
[INFO] aclOpSetModelDir op model success
[INFO] Init resource success
[INFO] Set input success
[INFO] Copy input[0] success
[INFO] Copy input[1] success
[INFO] Create stream success
[INFO] Execute AddCustom success
[INFO] Synchronize stream success
[INFO] Copy output[0] success
[INFO] Write output success
[INFO] Run op success
[INFO] Reset Device success
[INFO] Destroy resource success
```

如果有Run op success，表明执行成功，会在output目录下生成输出文件output\_z.bin。
4. 比较真值文件
- 切换到样例工程根目录，然后执行如下命令：
- ```
python3 scripts/verify_result.py output/output_z.bin output/golden.bin
```
- 会有如下屏显信息，可见AddCustom算子验证结果正确。
- ```
test pass
```

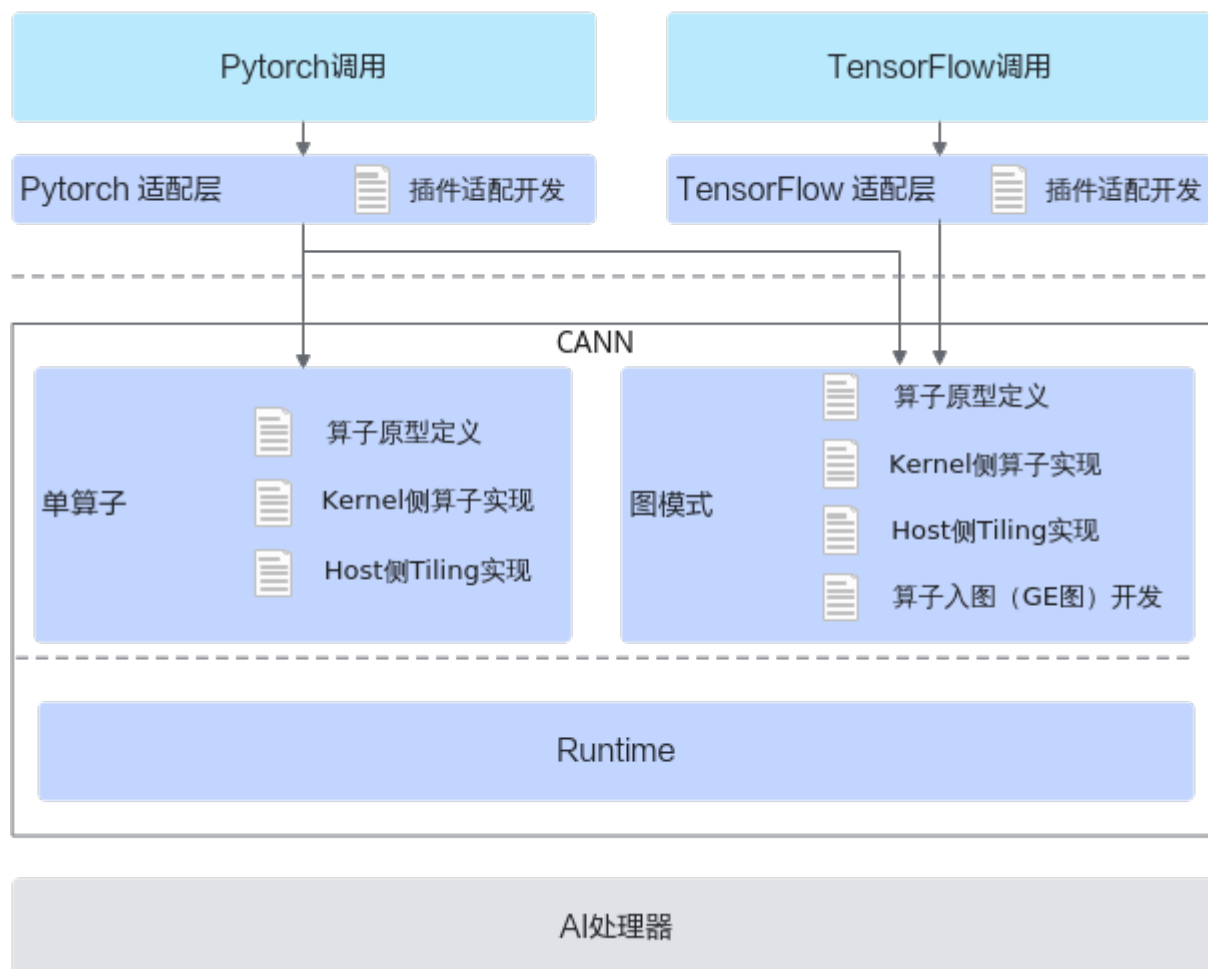
## 2.10.4 AI 框架算子适配

### 2.10.4.1 概述

本章节内容介绍AI框架调用自定义算子的方法。如下图所示，Pytorch支持单算子和图模式两种，TensorFlow仅支持图模式。

AI框架调用时，除了需要提供CANN框架调用时需要的代码实现文件，还需要进行插件适配开发。





#### 2.10.4.2 PyTorch 框架

通过PyTorch框架进行模型的训练、推理时，会调用很多算子进行计算。开发者开发的自定义算子如果需要集成部署到PyTorch框架，有如下几种方式：

- **Kernel直调：**通过适配torch.library或Pybind注册自定义算子，可以实现PyTorch框架调用算子Kernel程序。
- **单算子API调用：**该模式下的适配插件开发流程和具体样例请参见《[Ascend Extension for PyTorch 框架特性指南](#)》中的“基于OpPlugin算子适配开发”章节。
- **图模式调用：**自定义算子在Pytorch图模式下的适配开发指导请参见《[PyTorch图模式使用指南\(TorchAir\)](#)》中的“自定义算子入图”章节。

图 2-43 Pytorch 框架部署方式



本节主要提供通过torch.library与Pybind注册自定义算子并实现PyTorch框架调用算子Kernel程序的指导。

- torch.library是用于扩展PyTorch核心算子库的API集合。它允许开发者创建新的算子、并为其提供自定义实现。
- Pybind是一个开源的C++和Python之间的桥接工具，它旨在使C++代码能够无缝地集成到Python环境中。

Pybind适用于快速将C++函数暴露给Python，实现高效接口绑定。但其生成的算子无法被PyTorch的算子系统识别，不具备schema定义与图追踪能力，因此不支持torch.compile优化。相比之下，torch.library提供了与PyTorch核心算子系统深度集成的机制，支持算子注册、schema定义和图追踪能力，是支持torch.compile的必要条件。开发者可根据需求选择对应方式。

## torch.library

下面代码以add\_custom（Add自定义算子为例）算子为例，介绍通过torch.library如何调用算子Kernel程序，文档中仅介绍核心步骤，完整样例请参考[torch.library样例](#)。

### 步骤1 环境准备。

除了按照[1.2 环境准备](#)进行CANN软件包的安装，还需要安装以下依赖：

- [安装PyTorch框架](#)
- [安装torch\\_npu插件](#)

### 步骤2 实现NPU上的自定义算子。

包括算子Kernel实现，并使用<<<>>>接口调用算子核函数完成指定的运算。样例中的c10\_npu::getCurrentNPUStream接口用于获取当前npu流，返回值类型NPUStream，使用方式请参考《[Ascend Extension for PyTorch 自定义API参考](#)》中的“（beta）c10\_npu::getCurrentNPUStream”章节。

需要注意的是，本样例的输入x，y的内存是在外层的Python调用脚本中分配的。

```
namespace ascendc_ops {
at::Tensor ascendc_add(const at::Tensor &x, const at::Tensor &y)
{
 // 运行资源申请，通过c10_npu::getCurrentNPUStream()的函数获取当前NPU上的流
 auto aclStream = c10_npu::getCurrentNPUStream().stream(false);
 // 分配Device侧输出内存
 at::Tensor z = at::empty_like(x);
 uint32_t numBlocks = 8;
 uint32_t totalLength = 1;
 for (uint32_t size : x.sizes()) {
 totalLength *= size;
 }
}
```

```
// 用<<<>>>接口调用核函数完成指定的运算
auto xGm = static_cast<uint8_t*>(const_cast<void*>(x.storage().data()));
auto yGm = static_cast<uint8_t*>(const_cast<void*>(y.storage().data()));
auto zGm = static_cast<uint8_t*>(const_cast<void*>(z.storage().data()));
add_custom<<<numBlocks, nullptr, aclStream>>>(xGm, yGm, zGm, totalLength);
// 将Device上的运算结果拷贝回Host并释放申请的资源
return z;
}
} // namespace ascendc_ops
```

### 步骤3 自定义算子的注册。

PyTorch提供TORCH\_LIBRARY宏作为自定义算子注册的核心接口，用于创建并初始化自定义算子库，注册后在Python侧可以通过torch.ops.namespace.op\_name方式进行调用。TORCH\_LIBRARY\_IMPL用于将算子逻辑绑定到特定的DispatchKey（PyTorch设备调度标识），针对NPU设备，需要将算子实现注册到PrivateUse1这一专属的DispatchKey上。

```
// 注册算子到torch.library
TORCH_LIBRARY(ascendc_ops, m)
{
 m.def("ascendc_add(Tensor x, Tensor y) -> Tensor");
}

// 注册PrivateUse1实现，NPU设备
TORCH_LIBRARY_IMPL(ascendc_ops, PrivateUse1, m)
{
 m.impl("ascendc_add", TORCH_FN(ascendc_ops::ascendc_add));
}
```

### 步骤4 编译生成算子动态库。

### 步骤5 使用Python测试脚本进行测试。

在add\_custom\_test.py中，首先通过torch.ops.load\_library加载生成的自定义算子库，调用注册的ascendc\_add函数，并通过对比NPU输出与CPU标准加法结果来验证自定义算子的数值正确性。

----结束

## Pybind

下面代码以add\_custom算子为例，介绍通过Pybind方式实现Pytorch脚本中调用自定义算子的流程。文档中仅介绍核心步骤，完整样例请参考[Pybind样例](#)。

### 步骤1 环境准备。

除了按照[1.2 环境准备](#)进行CANN软件包的安装，还需要安装以下依赖：

- [安装PyTorch框架](#)
  - [安装torch\\_npu插件](#)
  - 安装pybind11
- ```
pip3 install pybind11
```

步骤2 实现NPU上的自定义算子。

包括算子Kernel实现，并使用<<<>>>接口调用算子核函数完成指定的运算。样例中的c10_npu::getCurrentNPUStream接口用于获取当前npu流，返回值类型NPUStream，使用方式请参考《[Ascend Extension for PyTorch 自定义API参考](#)》中的“（beta）c10_npu::getCurrentNPUStream”章节。

需要注意的是，本样例的输入x，y的内存是在Python调用脚本中分配的。

```
// Pybind和PyTorch调用所需的头文件
#include <pybind11/pybind11.h>
```

```
#include <torch/extension.h>

#include "torch_npu/csrc/core/npu/NPUStream.h"
// Kernel侧实现需要的头文件
#include "kernel_operator.h"
...

namespace my_add {
at::Tensor run_add_custom(const at::Tensor &x, const at::Tensor &y)
{
    // 运行资源申请, 通过c10_npu::getCurrentNPUStream()的函数获取当前NPU上的流
    auto aclStream = c10_npu::getCurrentNPUStream().stream(false);
    // 分配Device侧输出内存
    at::Tensor z = at::empty_like(x);
    uint32_t numBlocks = 8;
    uint32_t totalLength = 1;
    for (uint32_t size : x.sizes()) {
        totalLength *= size;
    }
    // 用<<<>>>接口调用核函数完成指定的运算
    auto xGm = static_cast<uint8_t*>(const_cast<void*>(x.storage().data()));
    auto yGm = static_cast<uint8_t*>(const_cast<void*>(y.storage().data()));
    auto zGm = static_cast<uint8_t*>(const_cast<void*>(z.storage().data()));
    add_custom<<<numBlocks, nullptr, aclStream>>>(xGm, yGm, zGm, totalLength);
    // 将Device上的运算结果拷贝回Host并释放申请的资源
    return z;
}
} // namespace my_add
```

步骤3 定义Pybind模块, 将C++函数封装成Python函数。PYBIND11_MODULE是Pybind11库中的一个宏, 用于定义一个Python模块。它接受两个参数, 第一个参数是封装后的模块名, 第二个参数是一个Pybind11模块对象, 用于定义模块中的函数、类、常量等。通过调用m.def()方法, 可以将上一步骤中函数my_add::run_add_custom()转成Python函数run_add_custom, 使其可以在Python代码中被调用。

```
PYBIND11_MODULE(add_custom, m) { // 模块名add_custom, 模块对象m
    m.doc() = "add_custom pybind11 interfaces"; // optional module docstring
    m.def("run_add_custom", &my_add::run_add_custom, ""); // 将函数run_add_custom与Pybind模块进行绑定
}
```

步骤4 编译生成算子动态库。

步骤5 在Python调用脚本中, 使用torch接口生成随机输入数据并分配内存, 通过导入封装的自定义模块add_custom, 调用自定义模块add_custom中的run_add_custom函数, 从而在NPU上执行算子。

----结束

2.10.4.3 ONNX 框架

2.10.4.3.1 适配插件开发

说明

针对Atlas A3 训练系列产品/Atlas A3 推理系列产品, 暂不支持ONNX框架算子调用。

您可以参考本章节进行算子适配插件的开发, 将ONNX框架的算子映射成适配AI处理器的算子(下文简称CANN算子), 从而完成从ONNX框架调用Ascend C自定义算子的过程。

完成[算子工程创建](#)后, 会在算子工程目录下生成framework/onnx_plugin目录, 用于存放ONNX框架适配插件实现文件。以自定义CANN算子LeakyReluCustom为例, 算子工程目录如下:

```
LeakyReluCustom
├── build.sh          // 编译入口脚本
├── cmake
├── CMakeLists.txt    // 算子工程的CMakeLists.txt
├── CMakePresets.json // 编译配置项
├── framework         // 框架适配插件实现文件目录
│   ├── onnx_plugin   // ONNX框架适配插件实现文件目录
│   │   ├── CMakeLists.txt
│   │   └── leaky_relu_custom_plugin.cc // ONNX框架适配插件实现文件
│   └── CMakeLists.txt
├── op_host           // host侧实现文件
├── op_kernel         // kernel侧实现文件
└── scripts           // 自定义算子工程打包相关脚本所在目录
```

下文主要讲解ONNX框架适配插件实现文件（`leaky_relu_custom_plugin.cc`）的开发流程。

```
#include "register/register.h"
#include "graph/operator.h"
#include "json.hpp"
namespace domi {
    Status ParseParamByOpFunc(const ge::Operator& op_src, ge::Operator& op_dest) {
        //...
    }
    REGISTER_CUSTOM_OP("OpType")
        .FrameworkType(ONNX)
        .OriginOpType("OriginOpType")
        .ParseParamsByOperatorFn(ParseParamByOpFunc) //用来注册解析算子属性的函数
        .ImpliedType(ImpliedType::TVM); // Ascend C算子实现类型设置为TVM
}
```

步骤1 包含所需头文件。

- `register.h`，存储在CANN软件安装后文件存储路径的“`include/register/`”目录下，包含该头文件，可使用算子注册相关类，调用算子注册相关的接口。
- `operator.h`（可选），存储在CANN软件安装后文件存储路径的“`include/graph/`”目录下，包含该头文件，可以使用`Operator`类相关接口，获取算子输入输出及属性等算子信息。
- `json.hpp`，用于进行ONNX数据定义的解析，将`String`类型的算子参数定义转换为`json`格式。若样例工程中未提供“`json.hpp`”文件，用户可以自行下载，并将“`json.hpp`”放在工程可以找到的任意路径下，然后包含此头文件即可，下载路径可参见[json.hpp](#)。

步骤2 使用`REGISTER_CUSTOM_OP`宏，完成CANN算子和ONNX框架的算子映射关系注册。使用方法如下：

- `REGISTER_CUSTOM_OP`：注册自定义算子，`OpType`为算子类型名称，需要与[算子原型注册](#)中的`OpType`保持一致。
- `FrameworkType`：ONNX代表原始框架为ONNX。
- `OriginOpType`：算子在原始框架中的类型。例如自定义算子`OpTypeA`，对应ONNX算子库版本`opset_version=11`，应传入“`ai.onnx::11::OpTypeA`”，当前支持的ONNX版本范围为9~15。
- `ParseParamsByOperatorFn(ParseParamByOpFunc)`：用来注册解析算子参数实现映射关系的回调函数，需要用户自定义实现回调函数`ParseParamByOpFunc`。具体实现方式参考[步骤3](#)。
- `ImpliedType`：指定算子的实现方式。Ascend C算子实现类型设置为TVM。

步骤3 实现回调函数`ParseParamByOpFunc`。其函数声明如下所示：

```
Status ParseParamByOpFunc(const ge::Operator& op_src, ge::Operator& op_dest)
```

- `ParseParamByOpFunc`：函数名称，用户自定义。

- `op_src`: ONNX框架定义的Operator类对象, 包含ONNX模型中自定义的算子属性信息, 定义来源于ONNX框架的原始模型文件。
- `op_dest`: CANN算子数据结构, 保存算子信息。

开发者需要在回调函数中实现属性的解析和映射, 具体实现方式如下:

ONNX原始模型中, 属性为repeated message类型, 对于repeated message类型的参数, 可使用`GetAttr(const char *name, ge::AscendString &attr_value)`接口获取其属性值, 然后将AscendString类型的属性值转换为String类型, 再将其转换为json格式进行属性字段的解析。

实现如下所示:

```
Status ParseParamLeakyReluAscend(const ge::Operator& op_src, ge::Operator& op_dest) {
    float negative_slope = 0.01f;
    string negative_slope_str;
    AscendString attrs_string;
    // 使用固定属性名称 "attribute" 获取ONNX算子中的属性, 并赋值给AscendString类型对象
    if (ge::GRAPH_SUCCESS == op_src.GetAttr("attribute", attrs_string)) {
        // 转换为json格式
        json attrs = json::parse(attrs_string.GetString());
        for (json attr : attrs["attribute"]) {
            if (attr["name"] == "alpha" && attr["type"] == kTypeFloat) {
                negative_slope_str = attr["f"]; // float type in json has accuracy loss, so we use string type to store it
                negative_slope = atof(negative_slope_str.c_str());
            }
        }
    }
    op_dest.SetAttr("negative_slope", negative_slope);
    return SUCCESS;
}
```

须知

- 当前版本GetAttr与SetAttr接口不支持对原始文件中数据类型为double和uint64的字段进行解析。
- 使用ATC工具执行模型转换时, 对属性的获取情况不会进行强校验。所以进行算子适配插件实现时, 若用户调用GetAttr失败, 建议根据算子实际情况增加相应的处理逻辑, 例如, 针对必选属性, 可返回失败, 针对可选属性, 可设置默认值。

----结束

2.10.4.3.2 调用样例

完成了ONNX框架的适配插件开发后, 即可实现从ONNX框架调用Ascend C自定义算子。下面以一个仅包含LeakyRelu算子的ONNX框架网络为例(该网络中的LeakyRelu算子通过适配插件映射为自定义的LeakyRelu算子), 呈现一个使用推理工具进行推理的过程, 目的在于让您快速体验推理场景下网络中自定义算子调用的过程。

在完成如下步骤之前, 您需要先参考上文内容完成自定义LeakyRelu算子kernel侧和host侧的开发、ONNX适配插件的开发, 并完成算子的编译部署。

LeakyRelu算子实现的完整样例请参考[LINK](#)。ONNX框架调用的完整示例请参考[LINK](#)。

步骤1 通过如下命令获取ONNX框架网络模型。作为示例, 该模型中仅包含一个LeakyRelu算子。

```
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/AscendC/leaky_relu.onnx
```


步骤2 执行如下命令生成离线模型。（如下命令中使用的目录以及文件均为样例，请以实际为准）

```
atc --model=$HOME/module/leaky_relu.onnx --framework=5 --soc_version=<soc_version> --output=$HOME/module/out/leaky_relu --input_shape="X:8,16,1024" --input_format=ND
```

关键参数的解释如下：

- --model：ONNX框架网络模型文件（*.onnx）的路径。
- --framework：原始框架类型。5表示ONNX。
- --output：转换后的离线模型的路径以及文件名。请注意，记录保存该om模型文件的路径，后续开发应用时需要使用。
- --soc_version：AI处理器的型号。

说明

AI处理器的型号请通过如下方式获取：

- 针对如下产品：在安装AI处理器的服务器执行**npu-smi info**命令进行查询，获取**Name**信息。实际配置值为AscendName，例如**Name**取值为xxxxyy，实际配置值为Ascendxxxxyy。

Atlas A2 训练系列产品/Atlas A2 推理系列产品

Atlas 200I/500 A2 推理产品

Atlas 推理系列产品

Atlas 训练系列产品

- 针对如下产品，在安装AI处理器的服务器执行**npu-smi info -t board -i id -c chip_id**命令进行查询，获取**Chip Name**和**NPU Name**信息，实际配置值为Chip Name_NPU Name。例如**Chip Name**取值为Ascendxxx，**NPU Name**取值为1234，实际配置值为Ascendxxx_1234。其中：

- id：设备id，通过**npu-smi info -l**命令查出的NPU ID即为设备id。

- chip_id：芯片id，通过**npu-smi info -m**命令查出的Chip ID即为芯片id。

Atlas A3 训练系列产品/Atlas A3 推理系列产品

- --input_shape：指定模型输入数据的shape，请基于算子支持的shape范围和实际使用场景进行设置，这里设置输入X为固定shape [8,16,1024]。
- --input_format：指定模型输入数据的格式，请基于算子支持的格式和实际使用场景进行设置，这里配置为ND。

步骤3 使用export ASCEND_GLOBAL_LOG_LEVEL=1改变日志级别为INFO，若出现如下提示信息，则说明进入了Ascend C自定义算子编译流程且模型转换成功。

```
...
start compile Ascend C operator LeakyReluCustom. kernel name is leaky_relu_custom
compile Ascend C operator: LeakyReluCustom success!
...
ATC run success
```

成功执行命令后，在--output参数指定的路径下，可查看离线模型（如：leaky_relu.om）。

步骤4 准备符合模型输入要求的*.bin格式的输入数据，单击[LINK](#)，获取msame工具，参考该工具配套的README，使用推理工具快速完成推理体验，样例如下。

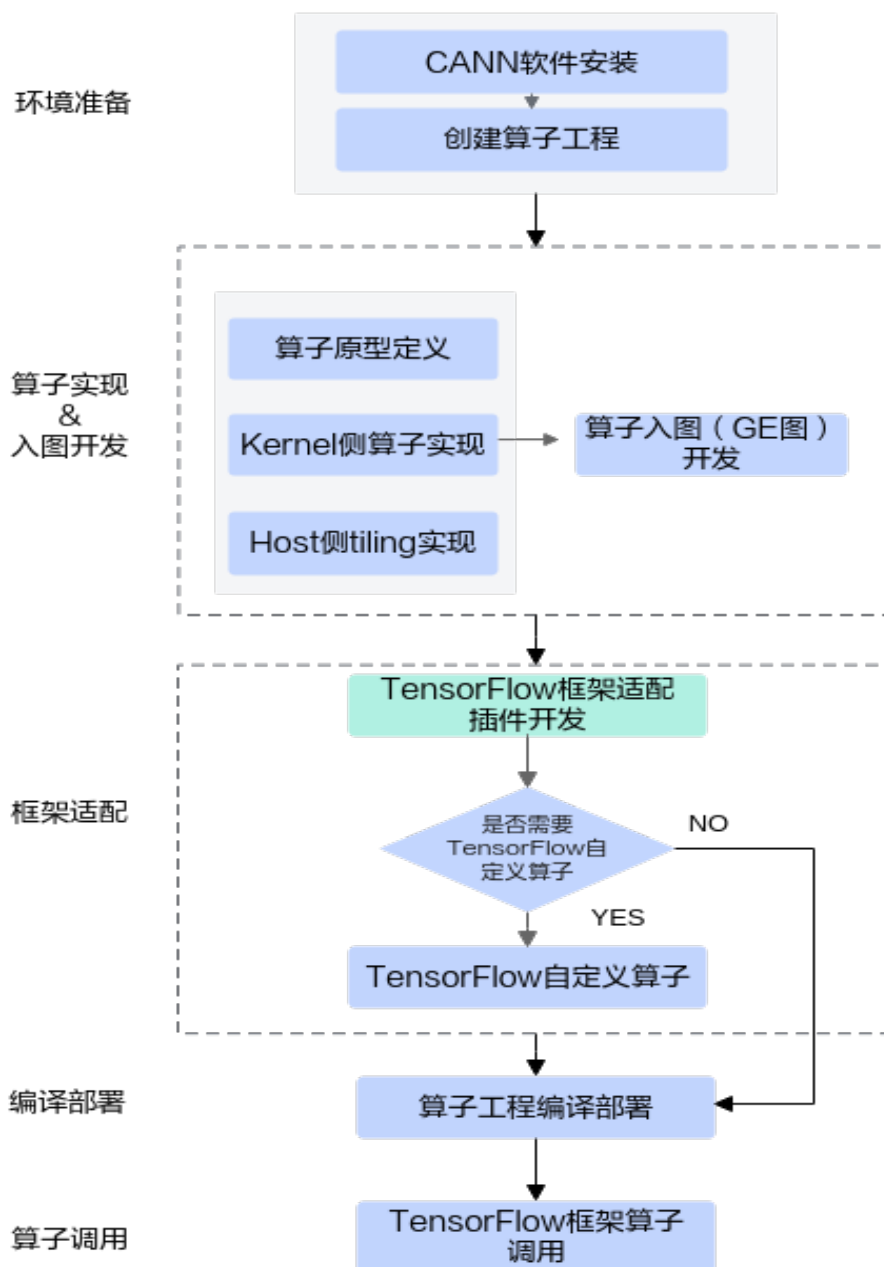
```
./msame --model "$HOME/module/out/leaky_relu.om" --output "$HOME/module/out/" --outfmt TXT --loop 1
```

----结束

2.10.4.4 TensorFlow 框架

本章节介绍TensorFlow框架算子适配的流程，用于将TensorFlow框架的算子映射成CANN算子（开发者基于CANN框架自定义开发的算子），从而完成从TensorFlow框架调用到CANN算子的过程。同时给出TensorFlow框架侧算子调用的示例，便于开发者了解完整流程。

下图展示了完整的开发流程，具体步骤如下：



步骤1 环境准备。

1. CANN软件安装请参考[1.2 环境准备](#)。
2. 安装框架插件包，请参考《[TensorFlow 1.15模型迁移指南](#)》或《[TensorFlow 2.6.5模型迁移指南](#)》中的环境准备 > 安装框架插件包章节，获取框架插件包详细的安装步骤。

3. **创建算子工程**。使用msOpGen工具创建算子开发工程。TensorFlow框架算子适配场景下，需要通过framework参数指定具体的框架为tf或者tensorflow，工具会自动生成框架适配代码。以自定义CANN算子AddCustom为例，使用msOpGen工具创建算子开发工程的具体命令如下：

```
`${INSTALL_DIR}/python/site-packages/bin/msopgen gen -i $HOME/sample/add_custom.json -f tf -c ai_core-<soc_version> -lan cpp -out $HOME/sample/AddCustom
```

步骤2 算子实现。

- **算子原型定义**。通过原型定义来描述算子输入输出、属性等信息以及算子在AI处理器上相关实现信息，并关联tiling实现等函数。
- Kernel侧算子实现和host侧tiling实现请参考[3.3 SIMD算子实现](#)；工程化算子开发，支持开发者调用Tiling API基于CANN提供的编程框架进行tiling开发，kernel侧也提供对应的接口方便开发者获取tiling参数，具体内容请参考[2.10.2.4 Kernel侧算子实现](#)和[2.10.2.5 Host侧Tiling实现](#)，由此而带来的额外约束也在上述章节说明。

步骤3 算子入图（GE图）开发。算子入图场景下，需要提供shape推导等算子入图适配函数的实现。

步骤4 TensorFlow框架适配插件开发。详细说明见[适配插件开发](#)。

步骤5 编译部署。通过工程编译脚本完成算子的编译部署，分为[算子包编译](#)和[算子动态库编译](#)两种方式。

步骤6 TensorFlow框架算子调用。详细说明见[TensorFlow原生算子映射到CANN算子](#)和[TensorFlow自定义算子开发并映射到CANN算子](#)。完整样例请参考[LINK](#)。

----结束

适配插件开发

完成[算子工程创建](#)后，会在算子工程目录下生成framework/tf_plugin目录，用于存放TensorFlow框架适配插件实现文件。以自定义CANN算子AddCustom为例，算子工程目录如下：

```
AddCustom
├── build.sh           // 编译入口脚本
├── cmake
├── CMakeLists.txt    // 算子工程的CMakeLists.txt
├── CMakePresets.json // 编译配置项
├── framework         // 框架适配插件实现文件目录
│   ├── tf_plugin     // TensorFlow框架适配插件实现文件目录
│   │   ├── CMakeLists.txt
│   │   └── tensorflow_add_custom_plugin.cc // TensorFlow框架适配插件实现文件
│   └── CMakeLists.txt
├── op_host           // host侧实现文件
├── op_kernel         // kernel侧实现文件
└── scripts           // 自定义算子工程打包相关脚本所在目录
```

当TensorFlow算子与CANN算子原型定义一致时，TensorFlow框架适配插件实现代码如下：

```
#include "register/register.h"
namespace domi {
REGISTER_CUSTOM_OP("AddCustom")
    .FrameworkType(TENSORFLOW)
    .OriginOpType("AddCustom")
    .ParseParamsByOperatorFn(AutoMappingByOpFn);
}
```

当TensorFlow算子与CANN算子原型定义不一致时，TensorFlow框架适配插件实现代码如下：

```
#include "register/register.h"
REGISTER_CUSTOM_OP("FlashAttentionScore")
    .FrameworkType(TENSORFLOW)
    .OriginOpType({"FlashAttentionScore"})
    .ParseParamsByOperatorFn(FlashAttentionScoreMapping)
    .ParseOpToGraphFn(AddOptionalPlaceholderForFA);
```

- 包含插件实现函数相关的头文件。
register.h存储在CANN软件安装后文件存储路径的“include/register/”目录下，包含该头文件，可使用算子注册相关类，调用算子注册相关的接口。
- REGISTER_CUSTOM_OP：注册自定义算子，传入算子的OpType，需要与算子原型注册中的OpType保持一致。
 - FrameworkType：TENSORFLOW代表原始框架为TensorFlow。
 - OriginOpType：算子在原始框架中的类型。对于TensorFlow自定义算子，还需要完成[TensorFlow自定义算子的开发](#)，这里的OriginOpType与REGISTER_OP注册算子名相同，对于TensorFlow原生算子，即为原生算子名。
 - ParseParamsByOperatorFn：用来注册解析算子参数实现映射关系的回调函数，需要用户自定义实现回调函数ParseParamByOpFunc。原始TensorFlow算子中参数与CANN算子中参数一一对应时，可直接使用自动映射回调函数AutoMappingByOpFn自动实现映射。
 - ParseOpToGraphFn：当TensorFlow算子与CANN算子原型定义不一致（比如CANN算子原型定义原型中有可选输入，但TensorFlow原型定义中不支持可选输入，没有可选输入）的情况时，用来注册调整算子原型映射关系的回调函数。

TensorFlow 原生算子映射到 CANN 算子

以自定义算子AddCustom为例，将该算子映射到TensorFlow内置算子Add上，需要先修改AddCustom自定义算子目录framework/tf_plugin下插件代码，完成算子名映射：

```
#include "register/register.h"
namespace domi {
REGISTER_CUSTOM_OP("AddCustom") // 当前Ascend C自定义算子名
    .FrameworkType(TENSORFLOW) // 第三方框架类型TENSORFLOW
    .OriginOpType("Add") // 映射到TensorFlow原生算子Add
    .ParseParamsByOperatorFn(AutoMappingByOpFn);
}
```

完成算子工程的编译部署后，构造单算子的TensorFlow 1.15版本测试用例进行验证。

步骤1 编写测试用例“tf_add.py”。

步骤2 导入python库。

```
import logging # Python标准库日志模块
import tensorflow as tf # 导入TensorFlow开源库
from npu_bridge.estimator import npu_ops # 导入TensorFlow开源库中的npu_ops模块
import numpy as np # 导入Python的数学基础库
```

步骤3 通过config()定义AI处理器和CPU上的运行参数。

当“execute_type”为“ai_core”时，代表在AI处理器上运行单算子网络，最终会调用到Ascend C算子。

当“execute_type”为“cpu”时，代表在Host侧的CPU运行单算子网络，调用的是TensorFlow算子。

```
def config(execute_type):
    if execute_type == 'ai_core':
```

```
session_config = tf.ConfigProto(
    allow_soft_placement=True,
    log_device_placement=False,)
custom_op = session_config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["enable_data_pre_proc"].b = True # 开启数据预处理下沉到Device侧执行
custom_op.parameter_map["mix_compile_mode"].b = True
custom_op.parameter_map["use_off_line"].b = True # True表示在AI处理器上执行训练

elif execute_type == 'cpu':
    session_config = tf.ConfigProto(
        allow_soft_placement=True,
        log_device_placement=False)

return session_config
```

步骤4 单算子网络测试用例主函数。

- 算子输入请根据算子实际输入个数及shape进行构造。
- 算子输出的计算，请根据算子逻辑调用TensorFlow相关接口进行实现。

```
#设置np.allclose比较函数的公差参数。
#np.allclose比较函数的相对公差参数
atol = 0.001
#np.allclose比较函数的绝对公差参数
rtol = 0.001

def main(unused_argv):
    shape_params = (8, 2048)
    dtype_params = np.float16

    # 构造Add算子的两个输入数据,shape为shape_params, 范围在[-2,2]之间的随机数
    x_data = np.random.uniform(-2, 2, size=shape_params).astype(dtype_params)
    y_data = np.random.uniform(-2, 2, size=shape_params).astype(dtype_params)
    # 分别对Add算子的两个输入数据进行占位
    x = tf.compat.v1.placeholder(dtype_params, shape=shape_params)
    y = tf.compat.v1.placeholder(dtype_params, shape=shape_params)
    # 计算算子输出
    out = tf.math.add(x, y)
    # 在Host侧CPU上运行单算子, 得到期望运行结果
    with tf.compat.v1.Session(config=config('cpu')) as session:
        result_cpu = session.run(out, feed_dict={x: x_data, y: y_data})
    # 在AI处理器上运行单算子, 得到实际运行结果
    with tf.compat.v1.Session(config=config('ai_core')) as session:
        result_ai_core = session.run(out, feed_dict={x: x_data, y: y_data})

    np.array(result_ai_core).astype(dtype_params)
    np.array(result_cpu).astype(dtype_params)
    print('=====')
    # 通过np.allclose比较AI处理器上运行的实际结果和cpu上运行的期望结果, 其中atol和rtol为np.allclose比较函数的相对公差参数和绝对公差参数
    cmp_result = np.allclose(result_ai_core, result_cpu, atol, rtol)
    print(cmp_result)
    print('=====')
```

步骤5 运行单算子网络。

```
if __name__ == "__main__":
    tf.app.run()
```

----结束

TensorFlow 自定义算子开发并映射到 CANN 算子

步骤1 适配插件代码开发。以自定义算子AddCustom为例，将该算子映射到TensorFlow自定义算子AddCustom上，需要先修改CANN AddCustom自定义算子工程目录framework/tf_plugin下插件代码，完成算子名映射：

```
REGISTER_CUSTOM_OP("AddCustom")
    .FrameworkType(TENSORFLOW)
    .OriginOpType("AddCustom")
    .ParseParamsByOperatorFn(AutoMappingByOpFn);
```

步骤2 TensorFlow自定义算子的开发。本节仅给出示例说明，详细内容请参考TensorFlow官方文档。

创建TensorFlow原型注册文件custom_assign_add_custom.cc，内容如下：

```
#include "tensorflow/core/framework/op.h"
#include "tensorflow/core/framework/shape_inference.h"
#include "tensorflow/core/framework/op_kernel.h"
#include "tensorflow/core/framework/common_shape_fns.h"
using namespace tensorflow;

// 通过TensorFlow提供的REGISTER_OP接口完成算子原型的注册
REGISTER_OP("AddCustom")      // TensorFlow 注册算子名
    .Input("x: T")           // 算子原型，输入参数x，类型为T
    .Input("y: T")           // 算子原型，输入参数y，类型为T
    .Output("z: T")          // 算子原型，输入参数z，类型为T
    .Attr("T: {half}")       // T类型支持范围
    .SetShapeFn(shape_inference::BroadcastBinaryOpShapeFn); // 算子shape信息推导，
// BroadcastBinaryOpShapeFn为TensorFlow提供的内置函数，输出shape信息由输入shape传播推导，即输入和输出shape保持一致

// 实现一个CPU版本的kernel函数，因为Tensorflow的计算图在构建时会检查所有的算子是否有任意设备上的
// kernel函数（NPU Kernel无法被感知），如果没有将会报错。这里实现一个固定返回错误的CPU kernel函数：
class AddCustomOp : public OpKernel {
public:
    explicit AddCustomOp(OpKernelConstruction* context) : OpKernel(context) {}

    void Compute(OpKernelContext* context) override {
        OP_REQUIRES_OK(context, errors::Unimplemented("AddCustomOp is not supported on CPU"));
    }
};

REGISTER_KERNEL_BUILDER(Name("AddCustom").Device(DEVICE_CPU), AddCustomOp); // 注册
// AddCustom算子的CPU实现内核，该函数当前仅打印日志提示CPU不支持
```

使用如下命令对上述代码进行编译，产物为libcustom_ops.so，后续的算子调用脚本中可通过load_op_library接口加载该so为python模块，从而调用自定义算子。

```
TF_CFLAGS=( $(python3 -c 'import tensorflow as tf; print(" ".join(tf.sysconfig.get_compile_flags()))' ) ) // 获取TensorFlow编译选项
TF_LFLAGS=( $(python3 -c 'import tensorflow as tf; print(" ".join(tf.sysconfig.get_link_flags()))' ) ) // 获取TensorFlow链接选项
SOURCE_FILES=custom_assign_add_custom.cc // 包含
// TensorFlow算子注册和CPU内核实现的cc文件
g++ -std=c++14 -shared $SOURCE_FILES -o ${Path}/libcustom_ops.so -fPIC ${TF_CFLAGS[@]} $
{TF_LFLAGS[@]} -O2 // 编译命令，产物为libcustom_ops.so，TensorFlow即可通过load_op_library加载该so为
python模块，调用自定义算子
```

步骤3 测试脚本中加载上一步骤编译好的动态库，实现自定义算子的调用。

- TensorFlow 1.15.0调用代码示例

```
import os
import tensorflow as tf
import numpy as np
from npu_bridge.npu_init import *
tf.enable_resource_variables()
#np.allclose比较函数的相对公差参数
atol = 0.001
#np.allclose比较函数的绝对公差参数
rtol = 0.001
def main(unused_argv):
    custom_op_lib = tf.load_op_library('./outputs/libcustom_ops.so') # 加载so为python模块
    shape_params = (8, 2048)
    dtype_params = np.float16
```

```
x_data = np.random.uniform(-2, 2, size=shape_params).astype(dtype_params)
y_data = np.random.uniform(-2, 2, size=shape_params).astype(dtype_params)
x = tf.compat.v1.placeholder(dtype_params, shape=shape_params)
y = tf.compat.v1.placeholder(dtype_params, shape=shape_params)
tf_z = tf.math.add(x, y) # 调用TensorFlow原生算子
ac_z = custom_op_lib.add_custom(x, y) # 调用AscendC AddCustom自定义算子;
add_custom是将REGISTER_OP(AddCustom)中的AddCustom由大驼峰命名转为下划线格式
config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer" # 配置在AI处理器上运行单算子
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF
config.graph_options.rewrite_options.memory_optimization = RewriterConfig.OFF

with tf.Session(config=config) as sess:
    sess.run(tf.global_variables_initializer())
    tf_golden = sess.run(tf_z, feed_dict={x: x_data, y: y_data})
with tf.Session(config=config) as sess:
    sess.run(tf.global_variables_initializer())
    ascend_out = sess.run(ac_z, feed_dict={x: x_data, y: y_data})
np.array(tf_golden).astype(dtype_params)
np.array(ascend_out).astype(dtype_params)
print('=====')
# 通过np.allclose比较AI处理器上运行的实际结果和使用TensorFlow原生算子运行的期望结果, 其中atol
和rtol为np.allclose比较函数的相对公差参数和绝对公差参数。
cmp_result = np.allclose(tf_golden, ascend_out, atol, rtol)
print(cmp_result)
print('=====')
if __name__ == "__main__":
    tf.app.run()
```

- TensorFlow 2.6.5调用代码

```
import os
import tensorflow as tf
import numpy as np
import npu_device
from npu_device.compat.v1.npu_init import *
npu_device.compat.enable_v1()
tf.compat.v1.enable_resource_variables()
#np.allclose比较函数的相对公差参数
atol = 0.001
#np.allclose比较函数的绝对公差参数
rtol = 0.001
def main(unused_argv):
    custom_op_lib = tf.load_op_library('./outputs/libcustom_ops.so') # 加载so为python模块

    shape_params = (8, 2048)
    dtype_params = np.float16
    x_data = np.random.uniform(-2, 2, size=shape_params).astype(dtype_params)
    y_data = np.random.uniform(-2, 2, size=shape_params).astype(dtype_params)
    x = tf.compat.v1.placeholder(dtype_params, shape=shape_params)
    y = tf.compat.v1.placeholder(dtype_params, shape=shape_params)
    tf_z = tf.math.add(x, y) # 调用TensorFlow原生算子
    ac_z = custom_op_lib.add_custom(x, y) # 调用AscendC AddCustom自定义算子;
    add_custom是将REGISTER_OP(AddCustom)中的AddCustom由大驼峰命名转为下划线格式
    config = tf.compat.v1.ConfigProto()
    custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
    custom_op.name = "NpuOptimizer"
    config.graph_options.rewrite_options.remapping = RewriterConfig.OFF
    config.graph_options.rewrite_options.memory_optimization = RewriterConfig.OFF

    with tf.compat.v1.Session(config=config) as sess:
        sess.run(tf.global_variables_initializer())
        tf_golden = sess.run(tf_z, feed_dict={x: x_data, y: y_data})
    with tf.compat.v1.Session(config=config) as sess:
        sess.run(tf.global_variables_initializer())
        ascend_out = sess.run(ac_z, feed_dict={x: x_data, y: y_data})
    np.array(tf_golden).astype(dtype_params)
    np.array(ascend_out).astype(dtype_params)
    print('=====')
    # 通过np.allclose比较AI处理器上运行的实际结果和使用TensorFlow原生算子运行的期望结果, 其中atol
```

```
和rtol为np.allclose比较函数的相对公差参数和绝对公差参数。  
    cmp_result = np.allclose(tf_golden, ascend_out, atol, rtol)  
    print(cmp_result)  
    print('=====')  
if __name__ == "__main__":  
    tf.app.run()
```

----结束

可选输入算子映射关系开发

TensorFlow的原型定义中不支持可选输入，对于包含可选输入的算子，其从TensorFlow到CANN的映射关系，不满足简单的一对一映射，需要在插件适配代码中，将输入转换为可选输入，调整原型的映射关系。下文以CANN算子库中的FlashAttentionScore算子为例，介绍针对此类算子的框架适配插件如何开发。

步骤1 适配插件开发

和上文中介绍的简单的一对一映射不同，进行插件适配开发时，需要调用ParseOpToGraphFn注册回调函数，回调函数中用于调整算子原型映射关系。此时：

- 通过ParseParamsByOperatorFn注册回调函数，回调函数中将TensorFlow原生算子映射到一个IR和TensorFlow一致的中间算子（调用AutoMappingByOpFn完成属性映射）。
- 通过ParseOpToGraphFn注册回调函数，调整算子原型映射关系，将中间算子最终映射到CANN算子库中的算子，这里映射到Graph图的概念是指一个算子构成的单算子图。

需要注意：在ParseParamsByOperatorFn的回调函数中，需要将TensorFlow算子名称设置到中间算子的original_type属性中，用于后续ParseOpToGraphFn回调函数的触发。示例代码如下：

```
#include <string>  
#include <vector>  
#include "register/register.h"  
#include "graph/operator.h"  
#include "graph/graph.h"  
#include "graph/operator_factory.h"  
  
namespace domi {  
using namespace ge;  
  
static Status AddOptionalPlaceholderForFA(const ge::Operator &tf_op, ge::Graph &graph) {  
    // 1. 创建一个FlashAttentionScore算子npu_fa_op  
    ge::AscendString op_name;  
    tf_op.GetName(op_name);  
    auto npu_fa_op = OperatorFactory::CreateOperator(op_name.GetString(), "FlashAttentionScore");  
    // 2. 将TensorFlow算子属性映射到npu_fa_op算子上  
    float scale_value = 1.0;  
    (void)tf_op.GetAttr("scale_value", scale_value);  
    (void)npu_fa_op.SetAttr("scale_value", scale_value);  
  
    float keep_prob = 1.0;  
    (void)tf_op.GetAttr("keep_prob", keep_prob);  
    (void)npu_fa_op.SetAttr("keep_prob", keep_prob);  
  
    int32_t pre_tokens = 2147483647;  
    (void)tf_op.GetAttr("pre_tokens", pre_tokens);  
    (void)npu_fa_op.SetAttr("pre_tokens", pre_tokens);  
  
    int32_t next_tokens = 2147483647;  
    (void)tf_op.GetAttr("next_tokens", next_tokens);  
    (void)npu_fa_op.SetAttr("next_tokens", next_tokens);  
  
    int32_t head_num = 0;
```

```
(void)tf_op.GetAttr("head_num", head_num);
(void)npu_fa_op.SetAttr("head_num", head_num);

std::string input_layout;
(void)tf_op.GetAttr("input_layout", input_layout);
(void)npu_fa_op.SetAttr("input_layout", input_layout);

int32_t inner_precise = 0;
(void)tf_op.GetAttr("inner_precise", inner_precise);
(void)npu_fa_op.SetAttr("inner_precise", inner_precise);

int32_t sparse_mode = 0;
(void)tf_op.GetAttr("sparse_mode", sparse_mode);
(void)npu_fa_op.SetAttr("sparse_mode", sparse_mode);

int32_t pse_type = 1;
(void)tf_op.GetAttr("pse_type", pse_type);
(void)npu_fa_op.SetAttr("pse_type", pse_type);

int32_t seed = 0;
(void)tf_op.GetAttr("seed", seed);
(void)npu_fa_op.SetAttr("seed", seed);
int32_t offset = 0;
(void)tf_op.GetAttr("offset", offset);
(void)npu_fa_op.SetAttr("offset", offset);
int32_t out_dtype = 0;
(void)tf_op.GetAttr("out_dtype", out_dtype);
(void)npu_fa_op.SetAttr("out_dtype", out_dtype);

// 3. 创建输入Data
std::vector<Operator> inputs;
for (size_t i = 0UL; i < tf_op.GetInputsSize(); i++) {
    const std::string data_name = "Data_" + std::to_string(i);
    Operator data_op = OperatorFactory::CreateOperator(data_name.c_str(), "Data");
    (void)data_op.SetAttr("index", static_cast<int32_t>(i));
    inputs.emplace_back(data_op);
}

size_t index = 0UL;
//4. 必选输入直接设置Data到算子输入
(void)npu_fa_op.SetInput("query", inputs[index++]);
(void)npu_fa_op.SetInput("key", inputs[index++]);
(void)npu_fa_op.SetInput("value", inputs[index++]);

// 5. 可选输入需要判断type属性的个数是否为0，不为0则表示可选输入已经使能
std::vector<DataType> real_shift_type;
(void)tf_op.GetAttr("real_shift_type", real_shift_type);
if (!real_shift_type.empty()) {
    (void)npu_fa_op.SetInput("real_shift", inputs[index++]);
}

std::vector<DataType> drop_mask_type;
(void)tf_op.GetAttr("drop_mask_type", drop_mask_type);
if (!drop_mask_type.empty()) {
    (void)npu_fa_op.SetInput("drop_mask", inputs[index++]);
}

std::vector<DataType> padding_mask_type;
(void)tf_op.GetAttr("padding_mask_type", padding_mask_type);
if (!padding_mask_type.empty()) {
    (void)npu_fa_op.SetInput("padding_mask", inputs[index++]);
}

std::vector<DataType> atten_mask_type;
(void)tf_op.GetAttr("atten_mask_type", atten_mask_type);
if (!atten_mask_type.empty()) {
    (void)npu_fa_op.SetInput("atten_mask", inputs[index++]);
}

std::vector<DataType> prefix_type;
(void)tf_op.GetAttr("prefix_type", prefix_type);
```

```
if (!prefix_type.empty()) {
    (void)npu_fa_op.SetInput("prefix", inputs[index++]);
}
std::vector<DataType> actual_seq_qlen_type;
(void)tf_op.GetAttr("actual_seq_qlen_type", actual_seq_qlen_type);
if (!actual_seq_qlen_type.empty()) {
    (void)npu_fa_op.SetInput("actual_seq_qlen", inputs[index++]);
}
std::vector<DataType> actual_seq_kvlen_type;
(void)tf_op.GetAttr("actual_seq_kvlen_type", actual_seq_kvlen_type);
if (!actual_seq_kvlen_type.empty()) {
    (void)npu_fa_op.SetInput("actual_seq_kvlen", inputs[index++]);
}

std::vector<DataType> q_start_idx_type;
(void)tf_op.GetAttr("q_start_idx_type", q_start_idx_type);
if (!q_start_idx_type.empty()) {
    (void)npu_fa_op.SetInput("q_start_idx", inputs[index++]);
}

std::vector<DataType> kv_start_idx_type;
(void)tf_op.GetAttr("kv_start_idx_type", kv_start_idx_type);
if (!kv_start_idx_type.empty()) {
    (void)npu_fa_op.SetInput("kv_start_idx", inputs[index++]);
}
std::vector<DataType> d_scale_q_type;
(void)tf_op.GetAttr("d_scale_q_type", d_scale_q_type);
if (!d_scale_q_type.empty()) {
    (void)npu_fa_op.SetInput("d_scale_q", inputs[index++]);
}
std::vector<DataType> d_scale_k_type;
(void)tf_op.GetAttr("d_scale_k_type", d_scale_k_type);
if (!d_scale_k_type.empty()) {
    (void)npu_fa_op.SetInput("d_scale_k", inputs[index++]);
}
std::vector<DataType> d_scale_v_type;
(void)tf_op.GetAttr("d_scale_v_type", d_scale_v_type);
if (!d_scale_v_type.empty()) {
    (void)npu_fa_op.SetInput("d_scale_v", inputs[index++]);
}
std::vector<DataType> query_rope_type;
(void)tf_op.GetAttr("query_rope_type", query_rope_type);
if (!query_rope_type.empty()) {
    (void)npu_fa_op.SetInput("queryRope", inputs[index++]);
}

std::vector<DataType> key_rope_type;
(void)tf_op.GetAttr("key_rope_type", key_rope_type);
if (!key_rope_type.empty()) {
    (void)npu_fa_op.SetInput("keyRope", inputs[index++]);
}
// 6. 使用npu_fa_op算子的输出构造图的输出。
std::vector<std::pair<Operator, std::vector<size_t>>> output_indexes;
std::vector<size_t> node_output_index;
for (size_t i = 0UL; i < npu_fa_op.GetOutputsSize(); i++) {
    node_output_index.emplace_back(i);
}
(void)output_indexes.emplace_back(std::make_pair(npu_fa_op, node_output_index));
(void)graph.SetInputs(inputs).SetOutputs(output_indexes);
return SUCCESS;
}

static Status FlashAttentionScoreMapping(const ge::Operator& op_src, ge::Operator& op_dst) {
    // 1. 调用默认映射函数即可
    if (AutoMappingByOpFn(op_src, op_dst) != ge::GRAPH_SUCCESS) {
        return FAILED;
    }
    // 2. 需要将TensorFlow算子名称设置到op_dst的original_type属性中，用于后续ParseOpToGraphFn回调函数的触发
}
```



```
op_dst.SetAttr("original_type", "FlashAttentionScore");  
return SUCCESS;  
}  
  
REGISTER_CUSTOM_OP("FlashAttentionScore")  
    .FrameworkType(TENSORFLOW)  
    .OriginOpType({"FlashAttentionScore"})  
    .ParseParamsByOperatorFn(FlashAttentionScoreMapping) // 注册此函数用于实现算子本身属性的映射  
    .ParseOpToGraphFn(AddOptionalPlaceholderForFA); // 注册此函数用于实现将tf中的输入转化为可选输入，  
    改变连边关系  
} // namespace domi
```

步骤2 在TensorFlow开源框架里注册FlashAttentionScore算子的原型定义，由于TensorFlow不支持可选输入，需要将其可选输入在TensorFlow原型中表示为动态输入，并通过属性来标记动态输入的个数，**这些可选输入需要放置在原型定义的最后**。示例代码（FlashAttentionScore.cc）如下：

```
#include <algorithm>  
#include <atomic>  
#include <map>  
#include "tensorflow/core/framework/common_shape_fns.h"  
#include "tensorflow/core/framework/op.h"  
#include "tensorflow/core/framework/op_kernel.h"  
using namespace tensorflow;  
using shape_inference::InferenceContext;  
using shape_inference::ShapeHandle;  
using namespace std;  
using namespace chrono;  
using OpKernelConstructionPtr = OpKernelConstruction*;  
using OpKernelContextPtr = OpKernelContext*;  
using InferenceContextPtr = ::tensorflow::shape_inference::InferenceContext*;  
namespace {  
class CustOps : public OpKernel {  
public:  
    explicit CustOps(OpKernelConstructionPtr context) : OpKernel(context) {}  
    void Compute(OpKernelContextPtr context) override  
    {  
        std::cout << "Cust Ops not installed!!" << std::endl;  
    }  
    ~CustOps() override = default;};  
} // namespace  
namespace tensorflow {  
REGISTER_OP("FlashAttentionScore")  
    .Input("query: T")  
    .Input("key: T")  
    .Input("value: T")  
    .Input("real_shift: real_shift_type") // 可选输入在TensorFlow原型中注册为动态输入  
    .Input("drop_mask: drop_mask_type")  
    .Input("padding_mask: padding_mask_type")  
    .Input("atten_mask: atten_mask_type")  
    .Input("prefix: prefix_type")  
    .Input("actual_seq_qlen: actual_seq_qlen_type")  
    .Input("actual_seq_kvlen: actual_seq_kvlen_type")  
    .Input("q_start_idx: q_start_idx_type")  
    .Input("kv_start_idx: kv_start_idx_type")  
    .Input("d_scale_q: d_scale_q_type")  
    .Input("d_scale_k: d_scale_k_type")  
    .Input("d_scale_v: d_scale_v_type")  
    .Input("query_rope: query_rope_type")  
    .Input("key_rope: key_rope_type")  
    .Output("softmax_max: float32")  
    .Output("softmax_sum: float32")  
    .Output("softmax_out: T")  
    .Output("attention_out: T")  
    .Attr("scale_value: float = 1.0")  
    .Attr("keep_prob: float = 1.0")  
    .Attr("pre_tokens: int = 2147483647")  
    .Attr("next_tokens: int = 2147483647")  
    .Attr("head_num: int")
```

```
.Attr("input_layout: string")
.Attr("inner_precise: int = 0")
.Attr("sparse_mode: int = 0")
.Attr("pse_type: int = 1")
.Attr("seed: int = 0")
.Attr("offset: int = 0")
.Attr("out_dtype: int = 0")
.Attr("T: {float16, float32, bfloat16} = DT_FLOAT")
.Attr("real_shift_type: list({float16, float32, bfloat16}) >= 0") // 通过属性来标记动态输入个数
.Attr("drop_mask_type: list({uint8}) >= 0")
.Attr("padding_mask_type: list({float16, float32, bfloat16}) >= 0")
.Attr("atten_mask_type: list({bool, uint8}) >= 0")
.Attr("prefix_type: list({int64}) >= 0")
.Attr("actual_seq_qlen_type: list({int64}) >= 0")
.Attr("actual_seq_kvlen_type: list({int64}) >= 0")
.Attr("q_start_idx_type: list({int64}) >= 0")
.Attr("kv_start_idx_type: list({int64}) >= 0")
.Attr("d_scale_q_type: list({float32}) >= 0")
.Attr("d_scale_k_type: list({float32}) >= 0")
.Attr("d_scale_v_type: list({float32}) >= 0")
.Attr("query_rope_type: list({float32}) >= 0")
.Attr("key_rope_type: list({float32}) >= 0")
.SetShapeFn([])(InferenceContext *c) {
    return Status::OK();
};
REGISTER_KERNEL_BUILDER(Name("FlashAttentionScore").Device(DEVICE_CPU), CustOps)}
```

使用如下命令对上述代码进行编译，产物为libcustom_ops.so，后续的算子调用脚本中可通过load_op_library接口加载该so为python模块，从而调用自定义算子。

```
TF_CFLAGS=( $(python3 -c 'import tensorflow as tf; print(" ".join(tf.sysconfig.get_compile_flags()))' ) ) // 获取TensorFlow编译选项
TF_LFLAGS=( $(python3 -c 'import tensorflow as tf; print(" ".join(tf.sysconfig.get_link_flags()))' ) ) // 获取TensorFlow链接选项
SOURCE_FILES=FlashAttentionScore.cc // 包含TensorFlow
算子注册和CPU内核实现的cc文件
g++ -std=c++14 -shared $SOURCE_FILES -o ${Path}/libflashattention.so -fPIC ${TF_CFLAGS[@]} $
{TF_LFLAGS[@]} -O2 // 编译命令，产物为libflashattention.so，${Path}为自定义的路径，后续TensorFlow可
通过load_op_library加载该so为python模块，调用自定义算子
```

步骤3 封装一个TensorFlow的算子调用接口，在此接口中处理可选输入。在该脚本需要加载上一步骤编译好的动态库。

```
from tensorflow.python.framework import ops
import tensorflow as tf
tfOpLib = tf.load_op_library("../build/tf_ops/libflashattention.so")
# 假如外部未使能该可选输入，则给底层传入空列表
def create_optional_input_list(input):
    input_list = []
    if not input is None:
        input_list.append(input)
    return input_list
# flash_attention_score 封装函数
def npu_flash_attention(query, key, value, head_num, input_layout, real_shift=None, drop_mask=None,
padding_mask=None,
                        atten_mask=None, prefix=None, actual_seq_qlen=None, actual_seq_kvlen=None,
                        q_start_idx=None, kv_start_idx=None,
d_scale_q=None, d_scale_k=None, d_scale_v=None, query_rope=None, key_rope=None, scale_value=1.0,
keep_prob=1.0,
                        pre_tokens=2147483647, next_tokens=2147483647, inner_precise=0, sparse_mode=0,
                        pse_type=1, seed=0, offset=0, out_dtype=0
):
    output = tfOpLib.flash_attention_score(query=query, key=key, value=value,
real_shift=create_optional_input_list(real_shift), drop_mask=create_optional_input_list(drop_mask),
padding_mask=create_optional_input_list(padding_mask),
atten_mask=create_optional_input_list(atten_mask),
prefix=create_optional_input_list(prefix),
actual_seq_qlen=create_optional_input_list(actual_seq_qlen),
actual_seq_kvlen=create_optional_input_list(actual_seq_kvlen),
q_start_idx=create_optional_input_list(q_start_idx),
```

```
kv_start_idx=create_optional_input_list(kv_start_idx),d_scale_q=create_optional_input_list(d_scale_q),
d_scale_k=create_optional_input_list(d_scale_k),d_scale_v=create_optional_input_list(d_scale_v),
query_rope=create_optional_input_list(query_rope),key_rope=create_optional_input_list(key_rope),
scale_value=scale_value, keep_prob=keep_prob, pre_tokens=pre_tokens, next_tokens=next_tokens,
head_num=head_num, input_layout=input_layout, inner_precise=inner_precise,
sparse_mode=sparse_mode,
pse_type=pse_type,seed=seed,offset=offset,out_dtype=out_dtype
)
return output
```

步骤4 测试脚本中实现自定义算子的调用。假设上一步骤中代码文件保存为ops.py，从ops中导入npu_flash_attention函数并使用。TensorFlow 2.6.5调用代码如下：

```
import sys
from ops import npu_flash_attention

import tensorflow as tf
import numpy as np
tf.compat.v1.disable_eager_execution()

import npu_device
from npu_device.compat.v1.npu_init import *
npu_device.compat.enable_v1()

def sess_config():
    config = tf.compat.v1.ConfigProto()
    custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
    custom_op.name = "NpuOptimizer"
    config.graph_options.rewrite_options.remapping = RewriterConfig.OFF
    config.graph_options.rewrite_options.memory_optimization = RewriterConfig.OFF
    return config

shape = [1, 32, 32]
query_np = np.random.randn(*shape).astype(np.float16)
key_np = np.random.randn(*shape).astype(np.float16)
value_np = np.random.randn(*shape).astype(np.float16)

query = tf.Variable(query_np, tf.float16)
key = tf.Variable(key_np, tf.float16)
value = tf.Variable(value_np, tf.float16)

mask = tf.zeros(shape=(shape[0], 1, shape[1], shape[1]), dtype=tf.uint8)

head_num = 1
input_layout = "BSH"
flash_result_t = npu_flash_attention(query, key, value, head_num, input_layout, atten_mask=mask)

with tf.compat.v1.Session(config=sess_config()) as sess:
    sess.run(tf.compat.v1.global_variables_initializer())
    flash_result = sess.run(flash_result_t)
    print(flash_result)
```

----结束

动态输入算子映射关系开发

对于存在动态输入/输出的算子，需要在插件的回调函数ParseParamByOpFunc中使用AutoMappingByOpFnDynamic实现TensorFlow算子和CANN算子的匹配。通过DynamicInputOutputInfo结构类描述动态输入/输出的信息，将动态输入/输出的名称和描述其个数的属性名绑定，再传入AutoMappingByOpFnDynamic实现自动匹配。

以ParseSingleExample算子为例，插件适配代码如下：

```
#include "register/register.h"
namespace domi {
Status ParseSingleExampleMapping(const ge::Operator& op_src, ge::Operator& op) {
    std::vector<DynamicInputOutputInfo> value;
    const std::string dynamic_input_name_dense_defaults = "dense_defaults";
    const std::string dynamic_input_attr_name_dense_defaults = "Tdense";
```

```
DynamicInputOutputInfo input(kInput, dynamic_input_name_dense_defaults.c_str(),
    dynamic_input_name_dense_defaults.size(), dynamic_input_attr_name_dense_defaults.c_str(),
    dynamic_input_attr_name_dense_defaults.size());
value.push_back(input);
const std::string dynamic_output_name_sparse_indices = "sparse_indices";
const std::string dynamic_output_attr_name_sparse_indices = "num_sparse";
DynamicInputOutputInfo output(kOutput,
    dynamic_output_name_sparse_indices.c_str(),
    dynamic_output_name_sparse_indices.size(), dynamic_output_attr_name_sparse_indices.c_str(),
    dynamic_output_attr_name_sparse_indices.size());
value.push_back(output);
const std::string dynamic_output_name_sparse_values = "sparse_values";
const std::string dynamic_output_attr_name_sparse_values = "sparse_types";
DynamicInputOutputInfo output1(kOutput,
    dynamic_output_name_sparse_values.c_str(),
    dynamic_output_name_sparse_values.size(), dynamic_output_attr_name_sparse_values.c_str(),
    dynamic_output_attr_name_sparse_values.size());
value.push_back(output1);
const std::string dynamic_output_name_sparse_shapes = "sparse_shapes";
const std::string dynamic_output_attr_name_sparse_shapes = "sparse_types";
DynamicInputOutputInfo output2(kOutput,
    dynamic_output_name_sparse_shapes.c_str(),
    dynamic_output_name_sparse_shapes.size(), dynamic_output_attr_name_sparse_shapes.c_str(),
    dynamic_output_attr_name_sparse_shapes.size());
value.push_back(output2);
const std::string dynamic_output_name_dense_values = "dense_values";
const std::string dynamic_output_attr_name_dense_values = "Tdense";
DynamicInputOutputInfo output3(kOutput,
    dynamic_output_name_dense_values.c_str(),
    dynamic_output_name_dense_values.size(), dynamic_output_attr_name_dense_values.c_str(),
    dynamic_output_attr_name_dense_values.size());
value.push_back(output3);
AutoMappingByOpFnDynamic(op_src, op, value);
return SUCCESS;
}

// register ParseSingleExample op to GE
REGISTER_CUSTOM_OP("ParseSingleExample")
    .FrameworkType(TENSORFLOW)
    .OriginOpType("ParseSingleExample")
    .ParseParamsByOperatorFn(ParseSingleExampleMapping)
    }
```

说明

暂不支持同时有可选输入和动态输入的算子映射。

2.10.5 show_kernel_debug_data 工具

静态图场景下，整图算子全部下沉到NPU侧执行，kernel侧单算子调试信息（通过printf接口）需要在模型执行结束后才能获取。本工具提供了离线解析能力，帮助用户获取并解析调试信息（将bin文件解析成可读格式）。

说明

show_kernel_debug_data支持多用户并发调用，但用户需要指定不同的落盘路径，否则可能出现落盘内容被覆盖等问题。

产品支持情况

产品	是否支持
Atlas A3 训练系列产品/Atlas A3 推理系列产品	√

产品	是否支持
Atlas A2 训练系列产品/Atlas A2 推理系列产品	√
Atlas 200I/500 A2 推理产品	√
Atlas 推理系列产品	√
Atlas 训练系列产品	x

工具安装

步骤1 安装工具。

工具跟随CANN软件包发布（参考[环境准备](#)完成CANN安装），其路径默认为“\${INSTALL_DIR}/tools/show_kernel_debug_data”，其中\${INSTALL_DIR}请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。

步骤2 设置环境变量。

- root用户安装Ascend-cann-toolkit包时
source /usr/local/Ascend/cann/set_env.sh
- 非root用户安装Ascend-cann-toolkit包时
source \${HOME}/Ascend/cann/set_env.sh

步骤3 检查工具是否安装成功。

执行如下命令，若能正常显示--help或-h信息，则表示工具环境正常，功能可正常使用。

```
show_kernel_debug_data -h
```

----结束

使用方法

• 命令行方式

```
show_kernel_debug_data <bin_file_path> [<output_path>]
```

参数	可选/必选	说明
<bin_file_path> h>	必选	kernel侧调试信息落盘的bin文件路径，例如“/input/dump_workspace.bin”。
<output_path> h>	可选	解析结果的保存路径，例如“/output_dir”。默认是当前命令行执行目录下。

• API方式

获取kernel侧调试信息并解析成可读文件。函数原型如下。

```
def show_kernel_debug_data(bin_file_path: str, output_path: str = './') -> None
```

其中，输入参数说明如下。函数无输出参数和返回值。

- bin_file_path: kernel侧调试信息落盘的bin文件路径，字符串类型。

- output_path: 解析结果的保存路径，字符串类型，默认是当前接口调用脚本所在目录下。

调用示例参考如下代码。

```
from show_kernel_debug_data import show_kernel_debug_data
show_kernel_debug_data('./input/dump_workspace.bin')
```

产物说明

工具解析结果文件目录结构如下：

```
{output_path}
├── PARSER_{{timestamp}}    // {{timestamp}}表示时间戳。
└── parser.log             // 工具解析的日志，包含kernel侧日常流程和printf打印信息。
```

2.10.6 msobjdump 工具

本工具主要针对[Kernel直调工程](#)（NPU模式）、[标准自定义算子工程](#)、[简易自定义算子工程](#)编译生成的算子ELF文件（Executable and Linkable Format）提供解析和解压功能，并将结果信息以可读形式呈现，方便开发者直观获得kernel文件信息。

说明

- ELF文件是一种用于二进制文件、可执行文件、目标代码、共享库和核心转储的文件格式，包括常见的*.a、*.so文件等。ELF文件常见构成如下：
 - ELF头部：描述了整个文件的组织结构，包括文件类型、机器类型、版本号等信息。
 - 程序头部表：描述了文件中各种段（segments）信息，包括程序如何加载到内存中执行的信息。
 - 节区头部表：描述了文件中各个节（sections）信息，包括程序的代码、数据、符号表等。
- 工具使用过程中，若出现如下场景，请根据日志提示信息，分析排查问题。
 - ELF文件未找到
 - ELF文件权限错误
 - ELF文件存在但不支持解析或解压

产品支持情况

产品	是否支持
Atlas A3 训练系列产品/Atlas A3 推理系列产品	√
Atlas A2 训练系列产品/Atlas A2 推理系列产品	√
Atlas 200I/500 A2 推理产品	√
Atlas 推理系列产品	√
Atlas 训练系列产品	√

工具安装

步骤1 安装msobjdump工具。

工具跟随CANN软件包发布（参考[环境准备](#)完成CANN安装），其路径默认为“\${INSTALL_DIR}/tools/msobjdump”，其中\${INSTALL_DIR}请替换为CANN软件安装

后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。

步骤2 设置环境变量。

- root用户安装Ascend-cann-toolkit包时
source /usr/local/Ascend/cann/set_env.sh
- 非root用户安装Ascend-cann-toolkit包时
source \${HOME}/Ascend/cann/set_env.sh

步骤3 检查工具是否安装成功。

执行如下命令，若能正常显示--help或-h信息，则表示工具环境正常，功能可正常使用。

```
msobjdump -h
```

----结束

命令格式

- **解析ELF文件的命令**
msobjdump --dump-elf <elf_file> [--verbose]

表 2-34 参数说明

参数（区分大小写）	可选/必选	说明
--dump-elf <elf_file>, -d	必选	解析ELF文件中包含的device信息，如文件名、文件类型、文件长度、符号表等，并终端打屏显示。 <elf_file>表示待解析ELF文件路径，如/home/op_api/lib_api.so。支持两种打印模式： 简单打印：默认仅打印部分device信息。 全量打印：与--verbose配套使用，开启全量device信息打屏显示。 不同工程打印字段信息不同，具体参见 表2-37 和 表2-38 。
--verbose, -V	可选	必须与--dump-elf配套使用，用于开启ELF文件中全量打印device信息功能。

- **解压ELF文件的命令**
msobjdump --extract-elf <elf_file> [--out-dir <out_path>]

表 2-35 参数说明

参数（区分大小写）	可选/必选	说明
--extract-elf <elf_file>, -e	必选	解压ELF文件中包含的device信息，并按原始文件夹规则落盘到输出路径下。 <elf_file>表示待解压ELF文件路径，如/home/op_api/lib_api.so。 默认路径：解压结果文件默认落盘到当前执行路径下。 自定义路径：可与--out-dir配套使用，设置落盘路径。
--out-dir <out_path>, -o	可选	必须与--extract-elf配套使用，用于设置解压文件的落盘路径。 <out_path>为落盘文件目录，如/home/extract/。 请注意： msobjdump支持多用户并发调用，但需要指定不同的--out-dir，否则可能出现落盘内容被覆盖的问题。

- 获取ELF文件列表的命令

```
msobjdump --list-elf <elf_file>
```

表 2-36 参数说明

参数（区分大小写）	可选/必选	说明
--list-elf <elf_file>, -l	可选	获取ELF文件中包含的device信息文件列表，并打印显示。 <elf_file>表示待打印的ELF文件路径，如/home/op_api/lib_api.so。

表 2-37 ELF 解析字段说明（Kernel 直调工程）

字段名	含义	是否必选	打印说明
VERSION	表示版本号。	是	不设置--verbose，默认打印。
TYPE COUNT	表示ELF文件中包含的kernel文件个数。	是	不设置--verbose，默认打印。

字段名	含义	是否必选	打印说明
ELF FILE \${id}	表示ELF文件中包含的kernel文件名，\${id}表示kernel文件序号。 kernel文件名的命名规则如下： 按\${sec_prefix}_\${file_index}_\${kernel_type}.o 拼接，其中\${sec_prefix}为section段名（工具根据“.ascend.kernel”关键字搜索获取），\${file_index}表示文件编号，\${kernel_type}表示kernel类型。	是	不设置--verbose，默认打印。
KERNEL LEN	表示kernel文件的长度。	是	不设置--verbose，默认打印。
KERNEL TYPE	表示kernel类型，映射关系为{0: 'mix', 1: 'aiv', 2: 'aic'}。	否	不设置--verbose，默认打印。
ASCEND META	表示算子执行时核间同步、Cube/Vector核占比（task_ration）等信息。 若没有获取到该信息，默认显示None。	否	不设置--verbose，默认打印。
elf heard infos	包括ELF Header、Section Headers、Key to Flags、Program Headers、Symbol表等信息。	否	设置--verbose，开启全量打印。

表 2-38 ELF 解析字段说明（标准/简易自定义算子工程）

字段名	含义	是否必选	打印说明
.ascend.m eta. \${id}	表示算子kernel函数名称，其中\${id}表示meta信息的索引值。	是	不设置--verbose，默认打印。
VERSION	表示版本号。	是	不设置--verbose，默认打印。

字段名	含义	是否 必选	打印说明
DEBUG	调试相关信息，包含如下两部分内容： debugBufSize：调试信息需要的内存空间。 debugOptions：调试开关状态。取值如下： 0：调试开关关闭。 1：通过DumpTensor、printf打印进行调试。 2：通过assert断言进行调试。 4：通过时间戳打点功能进行调试。 8：通过内存越界检测进行调试。	否	不设置--verbose，默认打印。
DYNAMIC_PARAM	算子kernel函数是否启用动态参数。取值分别为： 0：关闭动态参数模式。 1：开启动态参数模式。	否	不设置--verbose，默认打印。
OPTIONAL_PARAM	可选参数信息，包含如下两部分内容： optionalInputMode：可选输入在算子kernel函数中是否需要占位。取值分别为： 0：可选输入不占位。 1：可选输入占位。 optionalOutputMode：可选输出在算子kernel函数中是否需要占位。取值分别为： 0：可选输出不占位。 1：可选输出占位。	否	不设置--verbose，默认打印。
KERNEL_TYPE	表示kernel函数运行时core类型，取值参见表 表 2-39 。	否	不设置--verbose，默认打印。
CROSS_CORE_SYNC	表示硬同步syncall类型。 USE_SYNC：使用硬同步。 NO_USE_SYNC：不使用硬同步。	否	不设置--verbose，默认打印。
MIX_TASK_RATIO	表示kernel函数运行时的Cube核/Vector核占比分配类型。	否	不设置--verbose，默认打印。
DETERMINISTIC_INFO	表示算子是否为确定性计算。 0：不确定计算。 1：确定性计算。	否	不设置--verbose，默认打印。

字段名	含义	是否 必选	打印说明
BLOCK_NUM	表示算子执行核数，该字段当前暂不支持，只打印默认值0xFFFFFFFF。	否	不设置--verbose，默认打印。
FUNCTION_ENTRY	算子TilingKey的值。	否	不设置--verbose，默认打印。
elf heard infos	包括ELF Header、Section Headers、Key to Flags、Program Headers、Symbol表等信息。	否	设置--verbose，开启全量打印。

表 2-39 kernel type 信息

KERNEL_TYPE	说明
AICORE	该参数为预留参数，当前版本暂不支持。 算子执行时仅会启动AI Core，比如用户在host侧设置blocknum为5，则会启动5个AI Core。
AIC	算子执行时仅启动AI Core上的Cube核：比如用户在host侧设置blocknum为10，则会启动10个Cube核。
AIV	算子执行时仅启动AI Core上的Vector核：比如用户在host侧设置blocknum为10，则会启动10个Vector核。
MIX_AIC_MAIN	AIC、AIV混合场景下，设置核函数的类型为MIX，算子执行时会同时启动AI Core上的Cube核和Vector核，比如用户在host侧设置blocknum为10，且设置task_ratio为1：2，则会启动10个Cube核和20个Vector核。
MIX_AIV_MAIN	AIC、AIV混合场景下，使用了多核控制相关指令时，设置核函数的类型为MIX，算子执行时会同时启动AI Core上的Cube核和Vector核，比如用户在host侧设置blocknum为10，且设置task_ratio为1：2，则会启动10个Vector核和20个Cube核。
AIC_ROLLBACK	算子执行时会同时启动AI Core和Vector Core，此时AI Core会当成Cube Core使用。
AIV_ROLLBACK	算子执行时会同时启动AI Core和Vector Core，此时AI Core会当成Vector Core使用。

使用样例（Kernel 直调算子工程）

以MatMulInvocationNeo算子为例（NPU模式），完整的工程可参考[Matmul多核Kernel直调样例](#)。假设\${cmake_install_dir}为算子Cmake编译产物根目录，目录结构如下（仅为示例，具体以实际算子工程为准），类似[CMake编译配置文件编写](#)。

```
out
├── lib
│   └── libascendc_kernels_npu.so
├── include
│   ├── ascendc_kernels_npu
│   │   ├── aclrtlaunch_matmul_custom.h
│   │   └── aclrtlaunch_triple_chevrons_func.h
└── .....
```

工具对编译生成的库文件（如*.so、*.a等）进行解析和解压，功能实现命令样例如下：

- **解析包含device信息的库文件**

支持两种打印方式，请按需选取，解析字段含义参见[表2-37](#)。

- **简单打印**

```
msobjdump --dump-elf ${cmake_install_dir}/out/libascendc_kernels_npu.so
```

执行上述命令，终端打印基础device信息，示例如下：

```
=====
[VERSION]: 1
[TYPE COUNT]: 1
=====
[ELF FILE 0]: ascendxxx1_ascendc_kernels_npu_0_mix.o
[KERNEL TYPE]: mix
[KERNEL LEN]: 511560
[ASCEND META]: None
```

- **全量打印**

```
msobjdump --dump-elf ${cmake_install_dir}/out/libascendc_kernels_npu.so --verbose
```

执行上述命令，终端打印所有device信息，示例如下：

```
=====
[VERSION]: 1
[TYPE COUNT]: 1
=====
[ELF FILE 0]: ascendxxx1_ascendc_kernels_npu_0_mix.o
[KERNEL TYPE]: mix
[KERNEL LEN]: 511560
[ASCEND META]: None
===== [elf heard infos] =====
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                   EXEC (Executable file)
  Machine:                                <unknown>: 0x1029
  Version:                                0x1
  Entry point address:                    0x0
  Start of program headers:               64 (bytes into file)
  Start of section headers:              510280 (bytes into file)
  Flags:                                  0x940000
  Size of this header:                     64 (bytes)
  Size of program headers:                 56 (bytes)
  Number of program headers:                2
  Size of section headers:                 64 (bytes)
  Number of section headers:               20
  Section header string table index:      18

Section Headers:
```

```
[Nr] Name      Type      Address      Off  Size  ES Flg Lk Inf Al
[ 0]          NULL      0000000000000000 000000 000000 00  0  0  0
[ 1] .text      PROGBITS  0000000000000000 0000b0 010a08 00  AX  0  0  4
.....
[19] .strtab     STRTAB     0000000000000000 071278 00b6cb 00  0  0  1
Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), p (processor specific)

There are no section groups in this file.

Program Headers:
Type      Offset  VirtAddr      PhysAddr      FileSiz MemSiz  Flg Align
LOAD      0x0000b0 0x0000000000000000 0x0000000000000000 0x010aa8 0x010aa8 R E
0x1000
GNU_STACK 0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000
RW 0
.....
```

- **解压包含device信息的库文件并落盘**

```
msobjdump --extract-elf ${cmake_install_dir}/out/libascendc_kernels_npu.so
```

执行上述命令，默认在当前执行路径下落盘
ascendxxx1_ascendc_kernels_npu_0_mix.o文件。

- **获取包含device信息的库文件列表**

```
msobjdump --list-elf ${cmake_install_dir}/out/libascendc_kernels_npu.so
```

执行上述命令，终端会打印所有文件，屏显信息形如：

```
ELF file 0: ascendxxx1_ascendc_kernels_npu_0_mix.o
```

使用样例（标准/简易自定义算子工程）

以下面的算子工程为例（仅为示例，具体以实际算子工程为准），假设\${cmake_install_dir}为算子Cmake编译产物根目录，目录结构如下，类似[算子编译](#)。

```
├── op_api
│   ├── include
│   │   ├── aclnn_acos_custom.h
│   │   ├── aclnn_matmul_leakyrelu_custom.h
│   │   └── .....
│   └── lib
│       └── libcust_opapi.so
```

工具对编译生成的库文件（如*.so、*.a等）进行解析和解压，功能实现命令样例如下：

- **解析包含device信息的库文件**

支持两种打印方式，请按需选取，解析字段含义参见[表2-38](#)。

- 简单打印

```
msobjdump --dump-elf ${cmake_install_dir}/op_api/lib/libcust_opapi.so
```

执行上述命令，终端打印基础device信息，示例如下：

```
.ascend.meta META INFO
VERSION: 1
DEBUG: debugBufSize=0, debugOptions=0
DYNAMIC_PARAM: dynamicParamMode=0
OPTIONAL_PARAM: optionalInputMode=1, optionalOutputMode=1
.ascend.meta. [0]: AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26_1
KERNEL_TYPE: AIV
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 1
.ascend.meta. [0]: AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26_2_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
MIX_TASK_RATIO: [0:1]
```

```
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 2
.ascend.meta. [0]: AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26_3_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
MIX_TASK_RATIO: [0:1]
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 3
.....
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_1
KERNEL_TYPE: AIV
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 1
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_2_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
MIX_TASK_RATIO: [0:1]
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 2
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_3_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 3
```

— 全量打印

```
msobjdump --dump-elf ${cmake_install_dir}/op_api/lib/libcust_opapi.so --verbose
```

执行上述命令，终端打印基础device信息，示例如下：

```
.ascend.meta META INFO
VERSION: 1
DEBUG: debugBufSize=0, debugOptions=0
DYNAMIC_PARAM: dynamicParamMode=0
OPTIONAL_PARAM: optionalInputMode=1, optionalOutputMode=1
.ascend.meta. [0]: AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26_1
KERNEL_TYPE: AIV
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 1
.ascend.meta. [0]: AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26_2_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
MIX_TASK_RATIO: [0:1]
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 2
.ascend.meta. [0]: AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26_3_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
MIX_TASK_RATIO: [0:1]
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 3
.....
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_1
KERNEL_TYPE: AIV
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 1
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_2_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
MIX_TASK_RATIO: [0:1]
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 2
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_3_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
DETERMINISTIC_INFO: 1
BLOCK_NUM: 0xFFFFFFFF
FUNCTION_ENTRY: 3
```

- 解压包含device信息的库文件并落盘

执行上述命令，默认在当前执行路径下保存解压文件，产物目录如下：

```
-- config // 算子原型配置文件目录
├── ${soc_version}
│   ├── acos_custom.json
│   ├── matmul_leakyrelu_custom.json
│   └── .....
-- ${soc_version} // AI处理器名
├── acos_custom // 基础单算子编译文件*.o和对应的*.json文件
│   ├── AcosCustom_da824ede53d7e754f85c14b9446ec2fc.json // 命名规则: ${op_type}.${parm_info}.json或${op_type}.${parm_info}.o, ${parm_info}是基于算子输入/输出dtype、shape等信息生成的标识码
│   ├── AcosCustom_da824ede53d7e754f85c14b9446ec2fc.o
│   ├── AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26.json
│   └── AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26.o
├── matmul_leakyrelu_custom
│   ├── MatmulLeakyreluCustom_e052bee3255764ac919095f3bdf83389.json
│   └── MatmulLeakyreluCustom_e052bee3255764ac919095f3bdf83389.o
```

```
| |--- aypy_custom  
| |--- .....
```

以acos_custom算子编译产物解压为例：

- 查看算子原型（acos_custom.json）

```
{  
  "binList": [  
    {  
      "implMode": "high_performance",  
      "int64Mode": false,  
      "simplifiedKeyMode": 0,  
      "simplifiedKey": [...],  
      "staticKey": "96b2b4bb2e3xxx,ee37ce8796ef139dxxxx",  
      "inputs": [  
        {  
          "name": "x",  
          "index": 0,  
          "dtype": "float32",  
          "format": "ND",  
          "paramType": "required",  
          "shape": [  
            -2  
          ],  
          "format_match_mode": "FormatAgnostic"  
        }  
      ],  
      "outputs": [  
        {  
          "name": "y",  
          "index": 0,  
          "dtype": "float32",  
          "format": "ND",  
          "paramType": "required",  
          "shape": [  
            -2  
          ],  
          "format_match_mode": "FormatAgnostic"  
        }  
      ],  
      "attrs": [  
        {  
          "name": "tmp",  
          "dtype": "int",  
          "value": 0  
        },  
        .....  
      ],  
      "opMode": "dynamic",  
      "optionalInputMode": "gen_placeholder",  
      "deterministic": "ignore",  
      "binInfo": {  
        "jsonFilePath": "ascendxxx/acos_custom/  
AcosCustom_da824ede53d7e754f85c14b9446ec2fc.json"  
      }  
    },  
    {  
      "implMode": "high_performance",  
      "int64Mode": false,  
      "simplifiedKeyMode": 0,  
      "simplifiedKey": [  
      ],  
      "staticKey": "27d6f997f2f3551axxxx,1385590c47affa578eb429xxx",  
      "inputs": [  
        {  
          "name": "x",  
          "index": 0,  
          "dtype": "float16",  
          "format": "ND",  
          "paramType": "required",  
          "shape": [  
            -2  
          ],  
          "format_match_mode": "FormatAgnostic"  
        }  
      ],  
      "outputs": [  
        {  
          "name": "y",  
          "index": 0,  
          "dtype": "float16",  
          "format": "ND",  
          "paramType": "required",  
          "shape": [  
            -2  
          ],  
          "format_match_mode": "FormatAgnostic"  
        }  
      ],  
      "attrs": [  
        {  
          "name": "tmp",  
          "dtype": "int",  
          "value": 0  
        },  
        .....  
      ],  
      "opMode": "dynamic",  
      "optionalInputMode": "gen_placeholder",  
      "deterministic": "ignore",  
      "binInfo": {  
        "jsonFilePath": "ascendxxx/acos_custom/  
AcosCustom_da824ede53d7e754f85c14b9446ec2fc.json"  
      }  
    }  
  ],  
  "opMode": "dynamic",  
  "optionalInputMode": "gen_placeholder",  
  "deterministic": "ignore",  
  "binInfo": {  
    "jsonFilePath": "ascendxxx/acos_custom/  
AcosCustom_da824ede53d7e754f85c14b9446ec2fc.json"  
  }  
}
```



```
        "paramType": "required",
        "shape": [
            -2
        ],
        "format_match_mode": "FormatAgnostic"
    }
],
"outputs": [
    {
        "name": "y",
        "index": 0,
        "dtype": "float16",
        "format": "ND",
        "paramType": "required",
        "shape": [
            -2
        ],
        "format_match_mode": "FormatAgnostic"
    }
],
"attrs": [
    {
        "name": "tmp",
        "dtype": "int",
        "value": 0
    },
    .....
],
"opMode": "dynamic",
"optionalInputMode": "gen_placeholder",
"deterministic": "ignore",
"binInfo": {
    "jsonFilePath": "ascendxxx/acos_custom/
AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26.json"
}
}
]
```

- 解析\${op_type}_\${parm_info}.o文件获取.ascend.meta段信息。

msobjdump --dump-elf ./AcosCustom_da824ede53d7e754f85c14b9446ec2fc.o

执行上述命令，终端屏显如下，字段与库文件解析类似，参见表2-38。

```
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_1
KERNEL_TYPE: AIV
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_2_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
MIX_TASK_RATIO: [0:1]
.ascend.meta. [0]: AcosCustom_da824ede53d7e754f85c14b9446ec2fc_3_mix_aiv
KERNEL_TYPE: MIX_AIV_MAIN
MIX_TASK_RATIO: [0:1]
```

- 查看\${op_type}_\${parm_info}.json，直观获取device文件中算子信息。

```
{
    "binFileName": "AcosCustom_da824ede53d7e754f85c14b9446ec2fc",
    "binFileSuffix": ".o",
    "blockDim": -1,
    "coreType": "MIX",
    "intercoreSync": 1,
    "kernelName": "AcosCustom_da824ede53d7e754f85c14b9446ec2fc",
    "magic": "RT_DEV_BINARY_MAGIC_ELF",
    "memoryStamping": [],
    "opParaSize": 24,
    "parameters": [],
    "sha256": "94e32d04fc4f435411xxxxxxx",
    "workspace": {
        "num": 1,
        "size": [
            -1
        ],
    },
}
```

```
    "type": [
      0
    ]
  },
  "kernelList": [
    {
      "tilingKey": 1,
      "kernelType": "MIX_AIC",
      "taskRation": "0:1",
      "crossCoreSync": 0,
      "kernelName": "AcosCustom_da824ede53d7e754f85c14b9446ec2fc_1"
    },
    .....
  ],
  "taskRation": "tilingKey",
  "optionalInputMode": "gen_placeholder",
  "debugOptions": "printf",
  "debugBufSize": 78643200,
  "compileInfo": {},
  "supportInfo": { // 算子原型信息
    "implMode": "high_performance",
    "int64Mode": false,
    "simplifiedKeyMode": 0,
    "simplifiedKey": [.....],
    "staticKey": "96b2b4bb2e35fa3dxxx,ee37ce8796ef139dedxxxxxxxx",
    "inputs": [
      {
        "name": "x",
        "index": 0,
        "dtype": "float32",
        "format": "ND",
        "paramType": "required",
        "shape": [
          -2
        ],
        "format_match_mode": "FormatAgnostic"
      }
    ],
    "outputs": [
      {
        "name": "y",
        "index": 0,
        "dtype": "float32",
        "format": "ND",
        "paramType": "required",
        "shape": [
          -2
        ],
        "format_match_mode": "FormatAgnostic"
      }
    ],
    "attrs": [
      {
        "name": "tmp",
        "dtype": "int",
        "value": 0
      },
      .....
    ],
    "opMode": "dynamic",
    "optionalInputMode": "gen_placeholder",
    "deterministic": "ignore"
  },
  "filePath": "ascendxxx/acos_custom/AcosCustom_da824ede53d7e754f85c14b9446ec2fc.json"
}
```

- 获取包含device信息的库文件列表

```
msobjdump --list-elf ${cmake_install_dir}/op_api/lib/libcust_opapi.so
```

执行上述命令，终端会打印所有文件，屏显信息形如：

```
ELF file 0: ascendxxx_acos_custom_AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26.json
ELF file 1: ascendxxx_acos_custom_AcosCustom_dad9c8ca8fcbfd789010c8b1c0da8e26.o
.....
ELF file 2: ascendxxx_acos_custom_AcosCustom_da824ede53d7e754f85c14b9446ec2fc.json
ELF file 3: ascendxxx_acos_custom_AcosCustom_da824ede53d7e754f85c14b9446ec2fc.o
```

2.10.7 基于样例工程完成 Kernel 直调

说明

本章节介绍的基于样例工程完成Kernel直调的方式，后续不再演进。推荐开发者直接使用命令行或者编写Cmake文件进行编译，详细内容请参考[2.3.1 AI Core算子编译](#)。

下文将以Add矢量算子为例对Kernel直调算子开发流程进行详细介绍。

更多算子样例工程请通过如下链接获取：

- [矢量算子样例](#)
- [支持Tiling的矢量算子样例](#)
- [矩阵算子样例](#)
- [矢量+矩阵融合算子样例](#)

环境准备

- 使用Kernel Launch算子工程之前，需要参考[1.2 环境准备](#)章节安装驱动固件和CANN软件包，完成开发环境和运行环境的准备。
- 使用该算子工程要求cmake版本为3.16及以上版本，如不符合要求，请参考如下的命令示例更新cmake版本，如下示例以更新到3.16.0版本为例。

```
wget https://cmake.org/files/v3.16/cmake-3.16.0.tar.gz
tar -zxvf cmake-3.16.0.tar.gz
cd cmake-3.16.0
./bootstrap --prefix=/usr
sudo make
sudo make install
```

工程目录

您可以单击[矢量算子样例](#)，获取核函数开发和运行验证的完整样例。样例目录结构如下所示：

```
AddKernelInvocationNeo
|-- cmake                                // CMake编译文件
|-- scripts
|   |-- gen_data.py                    // 输入数据和真值数据生成脚本文件
|   |-- verify_result.py              // 验证输出数据和真值数据是否一致的验证脚本
|-- CMakeLists.txt                    // CMake编译配置文件
|-- add_custom.cpp                    // 矢量算子kernel实现
|-- data_utils.h                      // 数据读入写出函数
|-- main.cpp                          // 主函数，调用算子的应用程序，含CPU域及NPU域调用
|-- run.sh                            // 编译运行算子的脚本
```

基于该算子工程，开发者进行算子开发的步骤如下：

- 完成算子kernel侧实现。
- 编写算子调用应用程序main.cpp。
- 编写CMake编译配置文件CMakeLists.txt。
- 根据实际需要修改输入数据和真值数据生成脚本文件gen_data.py；验证输出数据和真值数据是否一致的验证脚本verify_result.py。

- 根据实际需要修改编译运行算子的脚本run.sh并执行该脚本，完成算子的编译运行和结果验证。

算子 Kernel 侧实现

请参考工程目录中的矢量算子、矩阵算子、融合算子的Kernel实现完成Ascend C算子实现文件的编写。

说明

一个算子Kernel实现文件中只支持定义一个核函数。

算子调用应用程序

下面代码以固定shape的add_custom算子为例，介绍算子核函数调用的应用程序main.cpp如何编写。您在实现自己的应用程序时，需要关注由于算子核函数不同带来的修改，包括算子核函数名，入参出参的不同等，合理安排相应的内存分配、内存拷贝和文件读写等，相关API的调用方式直接复用即可。

- 步骤1** 按需包含头文件，通过ASCENDC_CPU_DEBUG宏区分CPU/NPU侧需要包含的头文件。需要注意的是，NPU侧需要包含对应的核函数调用接口声明所在的头文件aclrtlaunch_{kernel_name}.h（该头文件为工程框架自动生成），kernel_name为算子核函数的名称。

```
#include "data_utils.h"
#ifdef ASCENDC_CPU_DEBUG
#include "acl/acl.h"
#include "aclrtlaunch_add_custom.h"
#else
#include "tikicplib.h"
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z);
#endif
```

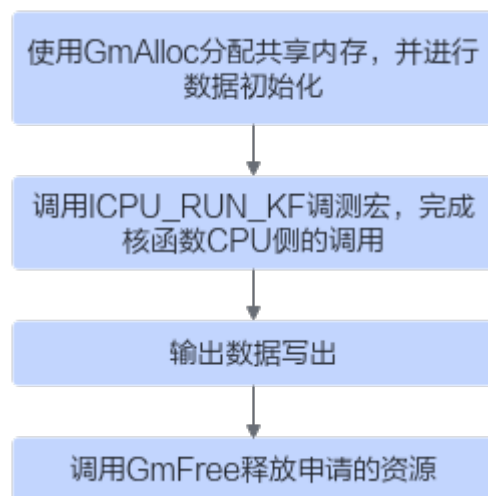
- 步骤2** 应用程序框架编写。该应用程序通过ASCENDC_CPU_DEBUG宏区分代码逻辑运行于CPU侧还是NPU侧。

```
int32_t main(int32_t argc, char* argv[])
{
    uint32_t numBlocks = 8;
    size_t inputByteSize = 8 * 2048 * sizeof(uint16_t);
    size_t outputByteSize = 8 * 2048 * sizeof(uint16_t);

#ifdef ASCENDC_CPU_DEBUG
    // 用于CPU调试的调用程序
#else
    // NPU侧运行算子的调用程序
#endif
    return 0;
}
```

- 步骤3** CPU侧运行验证。完成算子核函数CPU侧运行验证的步骤如下：

图 2-44 CPU 侧运行验证步骤



```
// 使用GmAlloc分配共享内存，并进行数据初始化
uint8_t* x = (uint8_t*)AscendC::GmAlloc(inputByteSize);
uint8_t* y = (uint8_t*)AscendC::GmAlloc(inputByteSize);
uint8_t* z = (uint8_t*)AscendC::GmAlloc(outputByteSize);

ReadFile("./input/input_x.bin", inputByteSize, x, inputByteSize);
ReadFile("./input/input_y.bin", inputByteSize, y, inputByteSize);
// 矢量算子需要设置内核模式为AIV模式
AscendC::SetKernelMode(KernelMode::AIV_MODE);
// 调用ICPU_RUN_KF调测宏，完成核函数CPU侧的调用
ICPU_RUN_KF(add_custom, numBlocks, x, y, z);
// 输出数据写出
WriteFile("./output/output_z.bin", z, outputByteSize);
// 调用GmFree释放申请的资源
AscendC::GmFree((void *)x);
AscendC::GmFree((void *)y);
AscendC::GmFree((void *)z);
```

步骤4 NPU侧运行验证。完成算子核函数NPU侧运行验证的步骤如下：

图 2-45 NPU 侧运行验证步骤



```
// 初始化
CHECK_ACL(aclInit(nullptr));
// 运行管理资源申请
int32_t deviceId = 0;
CHECK_ACL(aclrtSetDevice(deviceId));
aclrtStream stream = nullptr;
CHECK_ACL(aclrtCreateStream(&stream));
// 分配Host内存
uint8_t *xHost, *yHost, *zHost;
uint8_t *xDevice, *yDevice, *zDevice;

CHECK_ACL(aclrtMallocHost((void**)&xHost, inputByteSize));
CHECK_ACL(aclrtMallocHost((void**)&yHost, inputByteSize));
CHECK_ACL(aclrtMallocHost((void**)&zHost, outputByteSize));
// 分配Device内存
CHECK_ACL(aclrtMalloc((void**)&xDevice, inputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
CHECK_ACL(aclrtMalloc((void**)&yDevice, inputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
CHECK_ACL(aclrtMalloc((void**)&zDevice, outputByteSize, ACL_MEM_MALLOC_HUGE_FIRST));
// Host内存初始化
ReadFile("./input/input_x.bin", inputByteSize, xHost, inputByteSize);
ReadFile("./input/input_y.bin", inputByteSize, yHost, inputByteSize);
// 将数据从Host上拷贝到Device上
CHECK_ACL(aclrtMemcpy(xDevice, inputByteSize, xHost, inputByteSize,
ACL_MEMCPY_HOST_TO_DEVICE));
CHECK_ACL(aclrtMemcpy(yDevice, inputByteSize, yHost, inputByteSize,
ACL_MEMCPY_HOST_TO_DEVICE));
// 用内核调用符<<<<>>>>调用核函数完成指定的运算,add_custom_do中封装了<<<<>>>>调用
add_custom_do(numBlocks, nullptr, stream, xDevice, yDevice, zDevice);
```

```
// 用ACLRT_LAUNCH_KERNEL接口调用核函数完成指定的运算
// ACLRT_LAUNCH_KERNEL(add_custom)(numBlocks, stream, xDevice, yDevice, zDevice);
CHECK_ACL(aclrtSynchronizeStream(stream));
// 将Device上的运算结果拷贝回Host
CHECK_ACL(aclrtMemcpy(zHost, outputByteSize, zDevice, outputByteSize,
ACL_MEMCPY_DEVICE_TO_HOST));
WriteFile("./output/output_z.bin", zHost, outputByteSize);
// 释放申请的资源
CHECK_ACL(aclrtFree(xDevice));
CHECK_ACL(aclrtFree(yDevice));
CHECK_ACL(aclrtFree(zDevice));
CHECK_ACL(aclrtFreeHost(xHost));
CHECK_ACL(aclrtFreeHost(yHost));
CHECK_ACL(aclrtFreeHost(zHost));
// 去初始化
CHECK_ACL(aclrtDestroyStream(stream));
CHECK_ACL(aclrtResetDevice(deviceId));
CHECK_ACL(aclFinalize());
```

📖 说明

针对<<<>>>调用方式在[2.2.3 核函数](#)章节已有说明，这里仅对ACLRT_LAUNCH_KERNEL调用接口的使用方法介绍如下：

```
ACLRT_LAUNCH_KERNEL(kernel_name)(numBlocks, stream, argument list);
```

- kernel_name：算子核函数的名称。
- numBlocks：规定了核函数将会在几个核上执行。每个执行该核函数的核会被分配一个逻辑ID，即block_idx，可以在核函数的实现中调用GetBlockIdx来获取block_idx。
- stream，类型为aclrtStream，stream用于维护一些异步操作的执行顺序，确保按照应用程序中的代码调用顺序在Device上执行。stream创建等管理接口请参考“Stream管理”章节。
- argument list：参数列表，与核函数的参数列表保持一致。

----结束

CMake 编译配置文件编写

本节会介绍CMake文件中一些关键环境变量和Cmake命令参数的说明，通常情况下不需要开发者修改，但是这些参数可以帮助开发者更好的理解编译原理，方便有能力的开发者对Cmake进行定制化处理。

表 2-40 环境变量说明

环境变量	配置说明
SOC_VERSION	AI处理器的型号。 - 针对如下产品：在安装AI处理器的服务器执行npusmi info命令进行查询，获取Name信息。实际配置值为AscendName，例如Name取值为xxxxyy，实际配置值为Ascendxxxxyy。 Atlas A2 训练系列产品/Atlas A2 推理系列产品 Atlas 200I/500 A2 推理产品 Atlas 推理系列产品 Atlas 训练系列产品 - 针对如下产品，在安装AI处理器的服务器执行npusmi info -t board -i id -c chip_id命令进行查询，获取Chip Name和NPU Name信息，实际配置值为Chip Name_NPU Name。例如Chip Name取值为Ascendxxx，NPU Name取值为1234，实际配置值为Ascendxxx_1234。其中： - id：设备id，通过npusmi info -l命令查出的NPU ID即为设备id。 - chip_id：芯片id，通过npusmi info -m命令查出的Chip ID即为芯片id。 Atlas A3 训练系列产品/Atlas A3 推理系列产品
ASCEND_CANN_PACKAGE_PATH	CANN软件包安装后的实际路径。
CMAKE_BUILD_TYPE	编译模式选项，可配置为： “Release”，Release版本，不包含调试信息，编译最终发布的版本。 “Debug”，Debug版本，包含调试信息，便于开发者开发和调试。
CMAKE_INSTALL_PREFIX	用于指定CMAKE执行install时，安装的路径前缀，执行install后编译产物（ascendc_library中指定的target以及对应的头文件）会安装在该路径下。默认路径为当前目录的out目录下。
CMAKE_CXX_COMPILER_LAUNCHER	用于配置C++语言编译器（如g++）、毕昇编译器的启动器程序为ccache，配置后即可开启cache缓存编译，加速重复编译并提高构建效率。使用该功能前需要安装ccache。 配置方法如下，在对应的CMakeLists.txt进行设置： set(CMAKE_CXX_COMPILER_LAUNCHER <launcher_program>) 其中<launcher_program>是ccache的安装路径，比如ccache的安装路径为/usr/bin/ccache，示例如下： set(CMAKE_CXX_COMPILER_LAUNCHER /usr/bin/ccache)

表 2-41 Cmake 命令语法说明

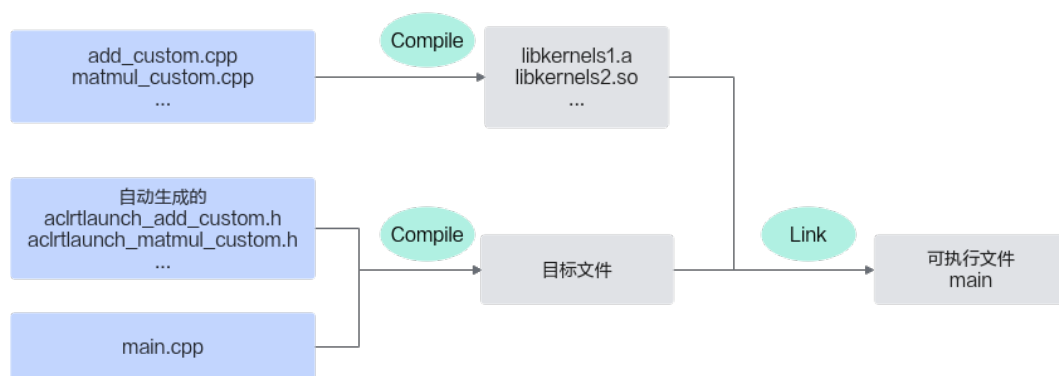
Cmake命令	语法说明
add_executable	使用指定的源文件将可执行文件添加到项目中。和Cmake通用的命令参数使用方法一致。
ascendc_library	使用指定的核函数源文件向项目（ project ）添加库。语法格式如下： <pre>ascendc_library(<target_name> [STATIC SHARED] [<source>...])</pre> 其中<target_name>表示库文件的名称，该库文件会根据命令里列出的源文件来建立。STATIC、SHARED的作用是生成库文件的类型。STATIC库是目标文件的归档文件，在连接其它目标的时候使用。SHARED库会被动态连接（动态连接库），在运行时会被加载。<source>表示核函数源文件。
ascendc_fatbin_library	使用指定的核函数源文件编译生成一个Kernel二进制文件，供Kernel加载和执行接口使用。语法格式如下： <pre>ascendc_fatbin_library(<target_name> [<source>...])</pre> • <target_name>表示库文件的名称，该库文件会根据命令里列出的核函数源文件编译生成<target_name>.o文件，放置于\${CMAKE_INSTALL_PREFIX}/fatbin/\${target_name}/路径下。 • <source>表示核函数源文件。 说明 • Kernel加载与执行接口的调用流程和上文介绍的<<<...>>>等调用流程有所差异，具体流程请参考《应用开发指南（C&C++）》中的“Kernel加载与执行”章节。 • 该编译选项暂不支持printf、DumpTensor、DumpAccChkPoint、assert接口。

Cmake命令	语法说明
ascendc_compile_definitions	添加编译宏。可以添加Ascend C提供的编译宏和开发者自定义的编译宏。语法格式如下： <pre>ascendc_compile_definitions(<target_name> [PRIVATE] [<xxx>...])</pre> Ascend C提供的编译宏介绍如下： ● HAVE_WORKSPACE用于表示kernel入口是否包含workspace入参。默认情况下为不包含；增加该编译宏后，表示包含，此时框架会获取kernel入参的倒数第一个参数（未配置HAVE_TILING），或倒数第二个参数（配置HAVE_TILING），自动在kernel侧设置系统workspace，开发者在kernel侧入参处获取的workspace为偏移了系统workspace后的用户workspace。当开发者使用了Matmul Kernel侧接口等需要系统workspace的高阶API时，建议开启此参数，入参排布、系统workspace的设置逻辑与2.10.2 工程化算子开发保持一致，可减少算子实现在不同开发方式间切换带来的修改成本。需要注意的是，host侧开发者仍需要自行申请workspace的空间，系统workspace大小可以通过PlatformAscendCManager的GetLibApiWorkSpaceSize接口获取。HAVE_WORKSPACE的设置样例如下： <pre>ascendc_compile_definitions(ascendc_kernels_\${RUN_MODE} PRIVATE HAVE_WORKSPACE)</pre> ● HAVE_TILING用于表示kernel入口是否含有tiling入参。在配置了HAVE_WORKSPACE之后，此编译宏才会生效。默认情况下为不包含，开关关闭；增加该编译宏后，表示包含，此时框架会将kernel入参的最后一个参数当做tiling，将倒数第二个参数当做workspace。框架不会对此tiling入参做任何处理，仅通过该入参来判断workspace参数的位置，使用此编译宏可以和2.10.2 工程化算子开发保持入参一致，减少算子实现在不同开发方式间切换带来的修改成本。设置样例如下： <pre>ascendc_compile_definitions(ascendc_kernels_\${RUN_MODE} PRIVATE HAVE_WORKSPACE HAVE_TILING)</pre>

Cmake命令	语法说明
ascendc_compile_options	添加编译选项。可以添加相应的编译选项用于host侧与device侧的编译过程。语法格式如下： <pre>ascendc_compile_options(<target_name> PRIVATE [<xxx>...])</pre> 默认情况下，指定的编译选项都将传递给device侧编译器进行编译。若想传递编译选项给host侧编译器，则需要使用“-forward-options-to-host-compiler”编译选项，该选项后的编译选项将传递给host侧编译器，示例如下： <pre>ascendc_compile_options(<target_name> PRIVATE -g -forward-options-to-host-compiler -gdwarf-4)</pre> 如上代码所示，在编译时，“-g”编译选项传递给device侧编译器，“-gdwarf-4”编译选项传递给host侧编译器。 备注：host侧编译选项只支持g++与clang编译器共同支持的编译选项。
ascendc_include_directories	添加开发者自定义的头文件搜索路径。语法格式如下： <pre>ascendc_include_directories(<target_name> [PRIVATE] [<xxx>...])</pre>

简化的编译流程图如下图所示：将算子核函数源文件编译生成kernel侧的库文件（*.so或*.a库文件）；工程框架自动生成核函数调用接口声明头文件；编译main.cpp（算子调用应用程序）时依赖上述头文件，将编译应用程序生成的目标文件和kernel侧的库文件进行链接，生成最终的可执行文件。

图 2-46 编译简化流程图



编译安装结束后在CMAKE_INSTALL_PREFIX目录下生成的编译产物示例如下；最终的可执行文件会生成在cmake命令的执行目录下。

```
out
├── lib
│   ├── libkernels1.a
│   └── libkernels2.so
└── include
```

```
├── kernels1
│   ├── aclrtlaunch_matmul_custom.h
│   └── aclrtlaunch_add_custom.h
├── kernels2
│   └── aclrtlaunch_xxx.h
└── ...
```

对于lib目录下生成的库文件可通过msobjdump工具进一步解析得到kernel信息，具体操作参见[2.10.6 msobjdump工具](#)。

输入数据和真值数据生成以及验证脚本文件

以固定shape的add_custom算子为例，输入数据和真值数据生成的脚本样例如下：根据算子的输入输出编写脚本，生成输入数据和真值数据。

```
#!/usr/bin/python3
# -*- coding:utf-8 -*-
import numpy as np

def gen_golden_data_simple():
    input_x = np.random.uniform(1, 100, [8, 2048]).astype(np.float16)
    input_y = np.random.uniform(1, 100, [8, 2048]).astype(np.float16)
    golden = (input_x + input_y).astype(np.float16)

    input_x.tofile("./input/input_x.bin")
    input_y.tofile("./input/input_y.bin")
    golden.tofile("./output/golden.bin")

if __name__ == "__main__":
    gen_golden_data_simple()
```

验证输出数据和真值数据是否一致的验证脚本样例如下：当前使用numpy接口计算了输出数据和真值数据的绝对误差和相对误差，误差在容忍偏差范围内，视为精度符合要求，输出"test pass"字样。

```
import os
import sys
import numpy as np

loss = 1e-3 # 容忍偏差，一般fp16要求绝对误差和相对误差均不超过千分之一
minimum = 10e-10

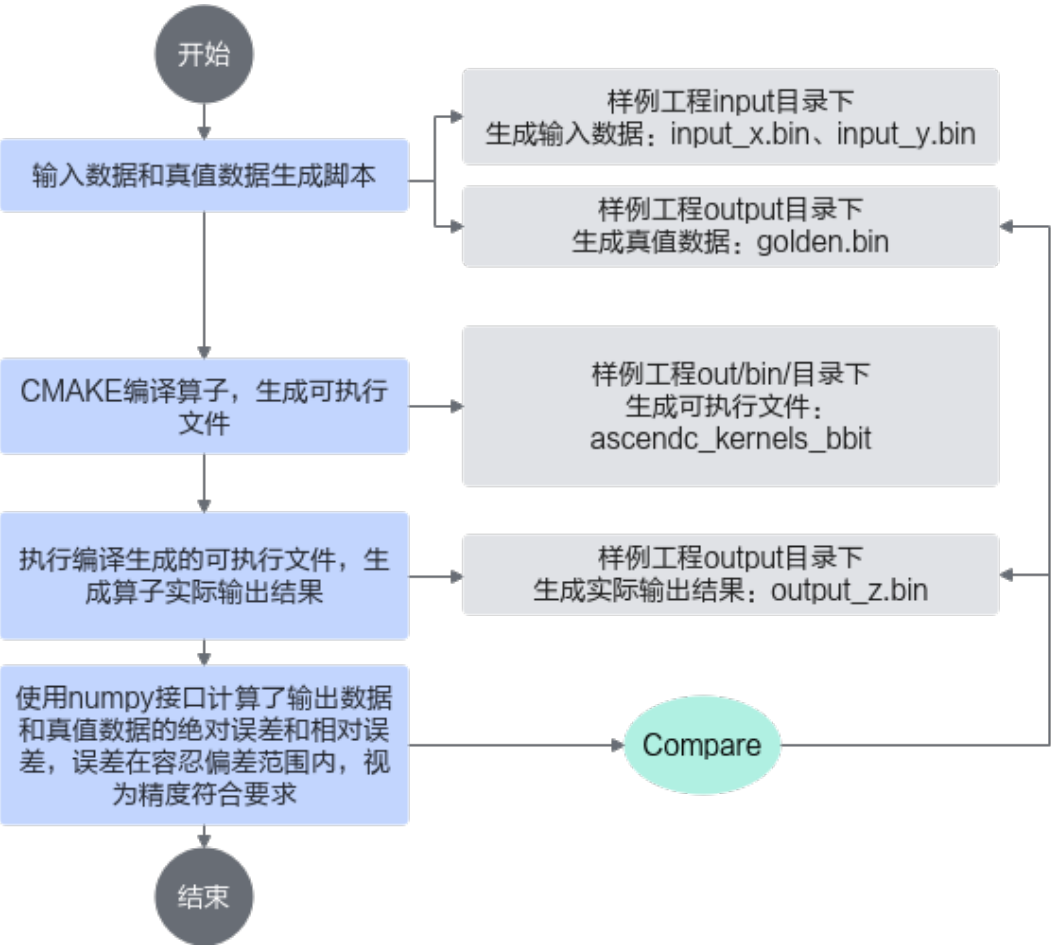
def verify_result(real_result, golden):
    real_result = np.fromfile(real_result, dtype=np.float16) # 从bin文件读取实际运算结果
    golden = np.fromfile(golden, dtype=np.float16) # 从bin文件读取预期运算结果
    result = np.abs(real_result - golden) # 计算运算结果和预期结果偏差
    deno = np.maximum(np.abs(real_result), np.abs(golden)) # 获取最大值并组成新数组
    result_atol = np.less_equal(result, loss) # 计算绝对误差
    result_rtol = np.less_equal(result / np.add(deno, minimum), loss) # 计算相对误差
    if not result_rtol.all() and not result_atol.all():
        if np.sum(result_rtol == False) > real_result.size * loss and np.sum(result_atol == False) >
real_result.size * loss:
            print("[ERROR] result error")
            return False
    print("test pass")
    return True

if __name__ == '__main__':
    verify_result(sys.argv[1], sys.argv[2])
```

修改并执行一键式编译运行脚本

您可以基于样例工程中提供的一键式编译运行脚本进行快速编译，并在CPU侧和NPU侧执行Ascend C算子。一键式编译运行脚本主要完成以下功能：

图 2-47 一键式编译运行脚本流程图



须知

样例中提供的一键式编译运行脚本并不能适用于所有的算子运行验证场景，使用时请根据实际情况进行修改。

- 根据Ascend C算子的算法原理的不同，自行实现输入和真值数据的生成脚本。

完成上述文件的编写后，可以执行一键式编译运行脚本，编译和运行应用程序。

脚本执行方式和脚本参数介绍如下：

```
bash run.sh --run-mode=npu --soc-version=<soc_version> --install-path=<install_path> --build-type=Debug --install-prefix=<install-prefix>

bash run.sh -r npu -v <soc_version> -i <install_path> -b Debug -p <install-prefix>
```

表 2-42 脚本参数介绍

参数名	参数简写	参数介绍
--run-mode	-r	表明算子以cpu模式或npu模式运行。 取值为cpu或npu。

参数名	参数简写	参数介绍
--soc-version	-v	算子运行的AI处理器型号。 说明 AI处理器的型号<soc_version>请通过如下方式获取： - 针对如下产品：在安装AI处理器的服务器执行npd-smi info命令进行查询，获取Name信息。实际配置值为AscendName，例如Name取值为xxxxyy，实际配置值为Ascendxxxxyy。 Atlas A2 训练系列产品/Atlas A2 推理系列产品 Atlas 200I/500 A2 推理产品 Atlas 推理系列产品 Atlas 训练系列产品 - 针对如下产品，在安装AI处理器的服务器执行npd-smi info -t board -i id -c chip_id命令进行查询，获取Chip Name和NPU Name信息，实际配置值为Chip Name_NPU Name。例如Chip Name取值为Ascendxxx，NPU Name取值为1234，实际配置值为Ascendxxx_1234。其中： - id：设备id，通过npd-smi info -l命令查出的NPU ID即为设备id。 - chip_id：芯片id，通过npd-smi info -m命令查出的Chip ID即为芯片id。 Atlas A3 训练系列产品/Atlas A3 推理系列产品
--install-path	-i	配置为CANN软件的安装路径，请根据实际安装路径进行修改。 默认值为\$HOME/Ascend/ascend-toolkit/latest。
--build-type	-b	编译模式选项，可配置为： - Release，Release版本，不包含调试信息，编译最终发布的版本。 - Debug，Debug版本，包含调试信息，便于开发者开发和调试。 默认值为Debug。
--install-prefix	-p	用于指定CMAKE执行install时，安装的路径前缀，执行install后编译产物（ascendc_library中指定的target以及对应的头文件）会安装在该路径下。默认路径为当前目录的out目录下。

脚本执行完毕输出"test pass"字样表示算子精度符合要求。

2.10.8 简易自定义算子工程

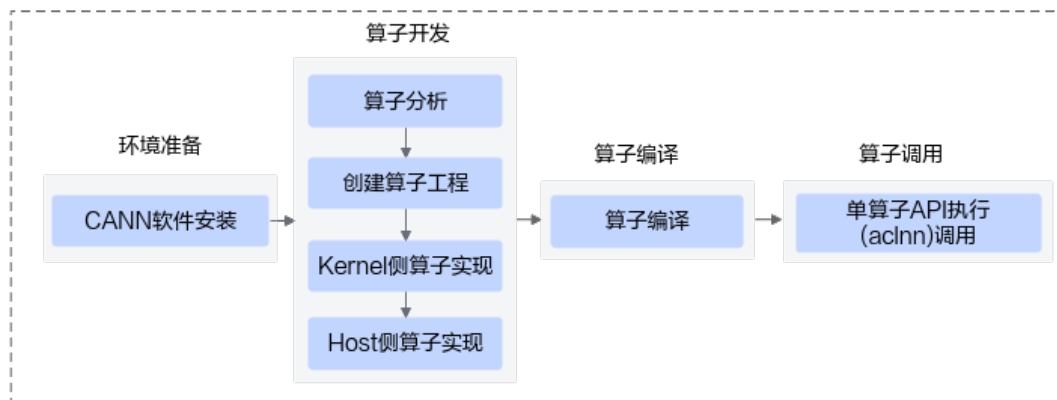
本章节介绍的简易自定义算子工程，是上文中介绍的自定义算子工程的简化版，对算子的编译、打包、部署过程进行简化，便于开发者将该工程集成到自己的算子工程。

基于简易自定义算子工程进行算子开发的完整样例请参考[简易自定义算子工程](#)。

说明

- 使用该工程，支持在如下平台进行自定义算子开发：
 - Atlas A2训练系列产品
 - Atlas 推理系列产品
- 使用本工程开发的算子，只支持通过单算子API执行（aclnn）方式进行调用。
- 本工程暂不支持算子的交叉编译功能。

基于简易自定义算子工程的算子开发流程图如下：



创建算子工程

和[2.10.2.2 创建算子工程](#)类似，简易自定义算子工程通过msOpGen生成，基于算子原型定义输出算子工程，包括算子host侧代码实现文件、算子kernel侧实现文件以及工程编译配置文件等。主要差异点在于：创建简易算子工程需要通过-f参数配置framework框架为aclnn。

使用msOpGen工具创建简易算子开发工程的步骤如下：

步骤1 编写算子的原型定义json文件，用于生成算子开发工程。

例如，AddCustom算子的json文件命名为add_custom.json，文件内容如下：

```
[
{
  "op": "AddCustom",
  "input_desc": [
    {
      "name": "x",
      "param_type": "required",
      "format": [
        "ND",
        "ND",
        "ND"
      ],
      "type": [
        "fp16",
        "float",
        "int32"
      ]
    }
  ],
  {
    "name": "y",
    "param_type": "required",
    "format": [
      "ND",
      "ND",
      "ND"
    ]
  }
}
```

```
    ],  
    "type": [  
        "fp16",  
        "float",  
        "int32"  
    ]  
  }  
],  
"output_desc": [  
  {  
    "name": "z",  
    "param_type": "required",  
    "format": [  
      "ND",  
      "ND",  
      "ND"  
    ],  
    "type": [  
      "fp16",  
      "float",  
      "int32"  
    ]  
  }  
]  
}  
]
```

步骤2 使用msOpGen工具生成算子的开发工程。以生成AddCustom的算子工程为例，下文仅针对关键参数进行解释，详细参数说明请参见msOpGen工具。

```
${INSTALL_DIR}/python/site-packages/bin/msopgen gen -i $HOME/sample/add_custom.json -c ai_core-  
<soc_version> -lan cpp -out $HOME/sample/AddCustom -f aclnn
```

- **\${INSTALL_DIR}**为CANN软件安装后文件存储路径，请根据实际环境进行替换。
- **-i**: 指定算子原型定义文件`add_custom.json`所在路径，请根据实际情况修改。
- **-c**: `ai_core-<soc_version>`代表算子在AI Core上执行，`<soc_version>`为昇腾AI处理器的型号。

📖 说明

AI处理器的型号`<soc_version>`请通过如下方式获取：

- 针对如下产品：在安装AI处理器的服务器执行**`npu-smi info`**命令进行查询，获取**Name**信息。实际配置值为AscendName，例如**Name**取值为`xxxxyy`，实际配置值为Ascend`xxxxyy`。

Atlas A2 训练系列产品/Atlas A2 推理系列产品

Atlas 200I/500 A2 推理产品

Atlas 推理系列产品

Atlas 训练系列产品

- 针对如下产品，在安装AI处理器的服务器执行**`npu-smi info -t board -i id -c chip_id`**命令进行查询，获取**Chip Name**和**NPU Name**信息，实际配置值为Chip Name_NPU Name。例如**Chip Name**取值为Ascend`xxx`，**NPU Name**取值为1234，实际配置值为Ascend`xxx_1234`。其中：

- **id**: 设备id，通过**`npu-smi info -l`**命令查出的NPU ID即为设备id。

- **chip_id**: 芯片id，通过**`npu-smi info -m`**命令查出的Chip ID即为芯片id。

Atlas A3 训练系列产品/Atlas A3 推理系列产品

基于同系列的AI处理器型号创建的算子工程，其基础功能（基于该工程进行算子开发、编译和部署）通用。

- **-lan**: 参数`cpp`代表算子基于Ascend C编程框架，使用C/C++编程语言开发。

- -out: 生成文件所在路径, 可配置为绝对路径或者相对路径, 并且工具执行用户对路径具有可读写权限。若不配置, 则默认生成在执行命令的当前路径。
- -f: 表示框架类型, aclnn表示生成简易工程。

步骤3 命令执行完后, 会在-out指定目录或者默认路径下生成算子工程目录, 工程中包含算子实现的模板文件, 编译脚本等, 以AddCustom算子为例, 目录结构如下所示:

```
AddCustom
├── build.sh                // 编译入口脚本
├── cmake
│   ├── config.cmake      // 编译配置项
│   ├── func.cmake
│   ├── intf.cmake
│   └── util                // 算子工程编译所需脚本及公共编译文件存放目录
├── CMakeLists.txt         // 算子工程的CMakeLists.txt
├── op_host                // host侧实现文件
│   ├── add_custom_tiling.h // 算子tiling定义文件
│   ├── add_custom.cpp    // 算子原型注册、tiling实现等内容文件
│   └── CMakeLists.txt
├── op_kernel              // kernel侧实现文件
│   ├── CMakeLists.txt
│   └── add_custom.cpp    // 算子代码实现文件
```

说明

上述目录结构中的粗体文件为后续算子开发过程中需要修改的文件, 其他文件无需修改。

----结束

算子实现

参考[2.10.2.4 Kernel侧算子实现](#)、[2.10.2.5 Host侧Tiling实现](#)、[2.10.2.3 算子原型定义](#)完成算子实现。

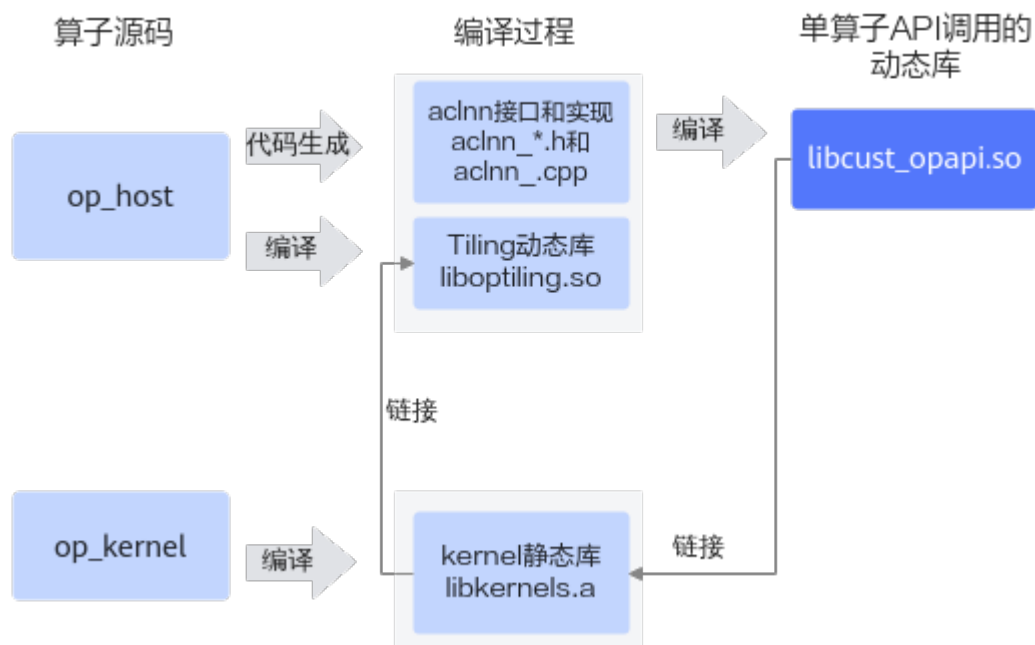
算子编译

算子kernel侧和host侧实现开发完成后, 需要对算子进行编译, 生成算子静态库; 自动生成aclnn调用实现代码和头文件, 链接算子静态库生成aclnn动态库, 以支持后续的单算子API执行方式 (aclnn) 的算子调用。编译过程如下:

- 根据host侧算子实现文件自动生成aclnn接口aclnn_*.h和aclnn实现文件aclnn_.cpp。
- 编译Tiling实现和算子原型定义生成Tiling动态库liboptiling.so (libcust_opmaster_rt2.0)。
- 编译kernel侧算子实现文件, 并加载Tiling动态库, 生成kernel静态库libkernels.a。
- 编译aclnn实现文件, 并链接kernel静态库libkernels.a生成单算子API调用的动态库libcust_opapi.so。

上述编译过程示意图如下:

图 2-48 编译过程示意图



上文描述的过程都封装在编译脚本中，开发者进行编译时参考如下的步骤进行操作：

步骤1 完成工程编译相关配置。

- 修改cmake目录下config.cmake中的配置项，config.cmake文件内容如下：

```
set(CMAKE_CXX_FLAGS_DEBUG "")
set(CMAKE_CXX_FLAGS_RELEASE "")

if (NOT DEFINED CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release CACHE STRING "")
endif()
if (CMAKE_INSTALL_PREFIX_INITIALIZED_TO_DEFAULT)
    set(CMAKE_INSTALL_PREFIX "${CMAKE_SOURCE_DIR}/build_out" CACHE PATH "" FORCE)
endif()
if (NOT DEFINED ASCEND_CANN_PACKAGE_PATH)
    set(ASCEND_CANN_PACKAGE_PATH /usr/local/Ascend/cann CACHE PATH "") //请替换为CANN软件包安装后的实际路径
endif()
if (NOT DEFINED ASCEND_PYTHON_EXECUTABLE)
    set(ASCEND_PYTHON_EXECUTABLE python3 CACHE STRING "")
endif()
if (NOT DEFINED ASCEND_COMPUTE_UNIT)
    set(ASCEND_COMPUTE_UNIT ascendxxx CACHE STRING "")
endif()
if (NOT DEFINED ENABLE_TEST)
    set(ENABLE_TEST FALSE CACHE BOOL "")
endif()
if (NOT DEFINED ENABLE_CROSS_COMPILE)
    set(ENABLE_CROSS_COMPILE FALSE CACHE BOOL "")
endif()
if (NOT DEFINED CMAKE_CROSS_PLATFORM_COMPILER)
    set(CMAKE_CROSS_PLATFORM_COMPILER "/your/cross/compiler/path" CACHE PATH "")
endif()
set(ASCEND_TENSOR_COMPILER_PATH ${ASCEND_CANN_PACKAGE_PATH}/compiler)
set(ASCEND_CCEC_COMPILER_PATH ${ASCEND_TENSOR_COMPILER_PATH}/ccec_compiler/bin)
set(ASCEND_AUTOGEN_PATH ${CMAKE_BINARY_DIR}/autogen)
file(MAKE_DIRECTORY ${ASCEND_AUTOGEN_PATH})
set(CUSTOM_COMPILE_OPTIONS "custom_compile_options.ini")
execute_process(COMMAND rm -rf ${ASCEND_AUTOGEN_PATH}/${CUSTOM_COMPILE_OPTIONS}
    COMMAND touch ${ASCEND_AUTOGEN_PATH}/${CUSTOM_COMPILE_OPTIONS})
```

表 2-43 需要开发者配置的常用参数列表

参数名称	参数描述	默认值
ASCEND_CANN_PACKAGE_PATH	CANN软件包安装路径，请根据实际情况进行修改。	“/usr/local/Ascend/cann”
CMAKE_BUILD_TYPE	编译模式选项，可配置为： “Release”，Release版本，不包含调试信息，编译最终发布的版本。 “Debug”，“Debug”版本，包含调试信息，便于开发者开发和调试。	“Release”
CMAKE_INSTALL_PREFIX	编译产物存放的目录，不指定则为默认值。	<code>\${CMAKE_SOURCE_DIR}/build_out</code> ： 算子工程目录下的 build_out目录

- 配置编译相关环境变量（可选）

表 2-44 环境变量说明

环境变量	配置说明
CMAKE_CXX_COMPILER_LAUNCHER	用于配置C++语言编译器（如g++）、毕昇编译器的启动器程序为ccache，配置后即可开启ccache缓存编译，加速重复编译并提高构建效率。用法如下，在对应的CMakeLists.txt进行设置： <code>set(CMAKE_CXX_COMPILER_LAUNCHER <launcher_program>)</code> 其中<launcher_program>是ccache的安装路径，比如ccache的安装路径为/usr/bin/ccache，示例如下： <code>set(CMAKE_CXX_COMPILER_LAUNCHER /usr/bin/ccache)</code>

步骤2 （可选）如果需要编译多个算子，在op_kernel目录下的CMakeLists.txt中增加要编译的算子。

```
# set custom compile options
if ("${CMAKE_BUILD_TYPE}" STREQUAL "Debug")
    add_ops_compile_options(ALL OPTIONS -g -O0)
endif()
#多算子编译通过add_kernel_compile命令增加算子源码文件即可
add_kernel_compile(AddCustom ${CMAKE_CURRENT_SOURCE_DIR}/add_custom.cpp)
add_kernel_compile(SubCustom ${CMAKE_CURRENT_SOURCE_DIR}/sub_custom.cpp)
if(ENABLE_TEST AND EXISTS ${CMAKE_CURRENT_SOURCE_DIR}/testcases)
    add_subdirectory(testcases)
endif()
```

步骤3 （可选）在算子工程中，如果开发者想对算子kernel侧代码增加一些自定义的编译选项，可以参考[支持自定义编译选项](#)进行编译选项的定制。

步骤4 在算子工程目录下执行如下命令，进行算子工程编译。

```
./build.sh
```

编译成功后，会在CMAKE_INSTALL_PREFIX/op_api目录存放生成的aclnn头文件和lib库，每一个算子都会对应一个单独的头文件。具体目录结构如下：

```
├── op_api
│   ├── include
│   │   ├── aclnn_optype1.h
│   │   ├── aclnn_optype2.h
│   │   └── aclnn_optypexxx.h
│   └── lib
│       └── libcust_opapi.so
```

对于lib目录下生成的库文件可通过msobjdump工具进一步解析得到kernel信息，具体操作参见[2.10.6 msobjdump工具](#)。

----结束

算子调用

完成[单算子API调用](#)。

2.10.9 常用操作

2.10.9.1 如何开发动态输入算子

动态输入算子是指算子的输入个数是动态的，例如AddN，将N个输入tensor累加到一起，输出一个tensor，输入tensor的个数是不固定的。动态输入算子的开发在构造和解析输入数据方面有差异：核函数的入参采用ListTensorDesc的结构存储输入数据信息，对应的，调用时需构造TensorList结构保存参数信息。下面基于kernel直调和工程化算子开发两种开发方式分别介绍具体开发流程。

📖 说明

下文仅列出代码片段，完整样例请参考[动态输入算子样例（工程化算子开发）](#)和[动态输入算子样例（kernel直调）](#)。

- kernel直调

- 参考ListTensorDesc数据结构自行定义ListTensorDesc和TensorDesc结构体，并将实际的输入数据保存至ListTensorDesc结构中。示例如下：

ptrOffset传入为ListTensorDesc首地址和数据指针首地址dataPtr之间的偏移量，tensorDesc中保存两个输入的tensor描述信息，dataPtr传入为保存输入数据的地址指针。

```
constexpr uint32_t SHAPE_DIM = 2;
struct TensorDesc {
    uint32_t dim{SHAPE_DIM};
    uint32_t index;
    uint64_t shape[SHAPE_DIM] = {8, 2048};
};

TensorDesc xDesc;
xDesc.index = 0;
TensorDesc yDesc;
yDesc.index = 1;

constexpr uint32_t TENSOR_DESC_NUM = 2;
struct ListTensorDesc {
    uint64_t ptrOffset;
    TensorDesc tensorDesc[TENSOR_DESC_NUM];
    uintptr_t dataPtr[TENSOR_DESC_NUM];
} inputDesc;
...
```

```
inputDesc = {(1 + (1 + SHAPE_DIM) * TENSOR_DESC_NUM) * sizeof(uint64_t), {xDesc, yDesc},  
{(uintptr_t)xDevice, (uintptr_t)yDevice}};
```

- kernel侧调用时，直接传入ListTensorDesc表达的输入信息。示例如下：

```
void *inputDescInDevice = nullptr;  
CHECK_ACL(aclrtMalloc((void **)&inputDescInDevice, sizeof(ListTensorDesc),  
ACL_MEM_MALLOC_HUGE_FIRST));  
CHECK_ACL(aclrtMemcpy(inputDescInDevice, sizeof(ListTensorDesc), &inputDesc,  
sizeof(ListTensorDesc),  
ACL_MEMCPY_HOST_TO_DEVICE));  
  
ACLRT_LAUNCH_KERNEL(addn_custom)(numBlocks, stream, inputDescInDevice, zDevice);
```

- kernel侧算子实现，通过ListTensorDesc和TensorDesc提供的接口解析ListTensorDesc输入信息，并处理。示例如下：

```
uint64_t buf[SHAPE_DIM] = {0};  
AscendC::TensorDesc<int32_t> tensorDesc;  
tensorDesc.SetShapeAddr(buf);  
listTensorDesc.GetDesc(tensorDesc, 0);  
uint64_t totalLength = tensorDesc.GetShape(0) * tensorDesc.GetShape(1);  
__gm__ uint8_t *x = listTensorDesc.GetDataPtr<__gm__ uint8_t>(0);  
__gm__ uint8_t *y = listTensorDesc.GetDataPtr<__gm__ uint8_t>(1);
```

- 工程化算子开发

- 单算子调用时，构造List类型tensor并传入。

使用aclCreateTensor创建tensor后，需调用aclCreateTensorList，将创建好的tensor组成List形式，如下所示。

```
inputTensorList = aclCreateTensorList(inputTensor_data(), inputTensor_size());
```

获取算子使用的workspace空间大小接口的入参，也需使用aclTensorList结构参数，用来计算workspace的大小，调用示例如下。

```
// 获取算子使用的workspace空间大小  
aclnnStatus aclnnAddNCustomGetWorkspaceSize(const aclTensorList *srcList, const aclTensor  
*out, uint64_t *workspaceSize, aclOpExecutor **executor);
```

- 算子原型定义中，输入数据的参数类型设置为动态，示例如下。

```
this->Input("srcList")  
.ParamType(DYNAMIC)  
.DataType({ge::DT_FLOAT16})  
.Format({ge::FORMAT_ND});
```

- host侧算子实现，获取动态输入信息的接口，需使用对应的动态接口。

例如，Tiling函数和InferShape函数中，GetDynamicInputShape接口用于获取动态输入的shape信息，InferDataType函数中，GetDynamicInputDataType接口用于获取动态输入的数据类型，示例如下。

```
namespace ge {  
static graphStatus InferShape(gert::InferShapeContext *context)  
{  
    const gert::Shape *x1_shape = context->GetDynamicInputShape(0, 0);  
    gert::Shape *y_shape = context->GetOutputShape(0);  
    *y_shape = *x1_shape;  
    return GRAPH_SUCCESS;  
}  
  
static graphStatus InferDataType(gert::InferDataTypeContext *context)  
{  
    const auto inputDataType = context->GetDynamicInputDataType(0, 0);  
    context->SetOutputDataType(0, inputDataType);  
    return ge::GRAPH_SUCCESS;  
}  
} // namespace ge
```

- kernel侧算子实现，入参需传入动态结构的数据，并使用AscendC::ListTensorDesc结构做解析。

核函数入参需传入动态结构的数据，例如GM_ADDR srcList，示例如下。

```
extern "C" __global__ __aicore__ void addn_custom(GM_ADDR srcList, GM_ADDR z, GM_ADDR workspace, GM_ADDR tiling)
```

对传入的参数srcList，需使用AscendC::ListTensorDesc结构做解析，得到每个tensor的具体信息，示例如下。

```
AscendC::ListTensorDesc keyListTensorDescInit((__gm__ void*)srcList);  
GM_ADDR x = (__gm__ uint8_t*)keyListTensorDescInit.GetDataPtr<__gm__ uint8_t>(0);  
GM_ADDR y = (__gm__ uint8_t*)keyListTensorDescInit.GetDataPtr<__gm__ uint8_t>(1);
```

2.10.9.2 如何在矢量编程时使能 Vector Core

针对Atlas 推理系列产品，其硬件架构除了AI Core外，还额外设置了单独的Vector Core，作为AI Core中Vector计算单元的补充，从而缓解Vector计算瓶颈。Vector Core只包括了两种基础计算资源：向量计算单元（Vector Unit）和标量计算单元（Scalar Unit），分别用于完成向量与标量的数据计算。矢量算子开发时，使能Vector Core，算子执行时会同时启动AI Core和Vector Core，这些核并行执行相同的核函数代码。

本节将重点介绍如何使能Atlas 推理系列产品中的Vector Core。学习本节内容之前，建议您先熟悉[算子实现](#)、[2.10.7 基于样例工程完成Kernel直调](#)、[2.10.2 工程化算子开发](#)的相关内容，掌握基于AI Core的算子端到端开发流程。在此基础上本章将重点阐述使能Vector Core时的差异点。具体如下：

1. 完成算子kernel侧开发时，需要通过宏KERNEL_TASK_TYPE_DEFAULT使能Vector Core，算子执行时会同时启动AI Core和Vector Core，此时AI Core会当成Vector Core使用。如下的代码样例展示了使能Vector Core的方法：

```
extern "C" __global__ __aicore__ void add_custom(__gm__ uint8_t *x, __gm__ uint8_t *y, __gm__ uint8_t *z, __gm__ uint8_t *workspace, __gm__ uint8_t *tiling)  
{  
    GET_TILING_DATA(tilingData, tiling);  
    if (workspace == nullptr) {  
        return;  
    }  
    GM_ADDR usr = AscendC::GetUserWorkspace(workspace);  
    KernelAdd op;  
    op.Init(x, y, z, tilingData.numBlocks, tilingData.totalLength, tilingData.tileNum);  
    KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_MIX_VECTOR_CORE); // 使能VectorCore  
    if (TILING_KEY_IS(1)) {  
        op.Process1();  
    } else if (TILING_KEY_IS(2)) {  
        op.Process2();  
    }  
    // ...  
}
```

2. 完成host侧tiling开发时，设置的numBlocks代表的是AI Core和Vector Core的总数，比如用户在host侧设置numBlocks为10，则会启动总数为10的AI Core和Vector Core；为保证启动Vector Core，设置数值应大于AI Core的核数。您可以通过GetCoreNumAic接口获取AI Core的核数，GetCoreNumVector接口获取Vector Core的核数。如下代码片段，分别为使用kernel直调工程和自定义算子工程时的设置样例，此处设置为AI Core和Vector Core的总和，表示所有AI Core和Vector Core都启动。

- kernel直调工程

```
auto ascendcPlatform = platform_ascendc::PlatformAscendCManager::GetInstance();  
auto totalCoreNum = ascendcPlatform.GetCoreNumAic();  
// ASCENDXXX请替换为实际的版本型号  
if (ascendcPlatform.GetSocVersion() == platform_ascendc::SocVersion::ASCENDXXX) {  
    totalCoreNum = totalCoreNum + ascendcPlatform.GetCoreNumVector();  
}  
...  
kernel_name<<<totalCoreNum, l2ctrl, stream>>>(argument list);
```

- 自定义算子工程

```
// 配套的host侧tiling函数示例：  
ge::graphStatus TilingFunc(gert::TilingContext* context)
```

```
{
    // 使能VectorCore，将numBlocks置为AI Core中vector核数 + Vector Core中的vector核数
    auto ascendcPlatform = platform_ascendc::PlatformAscendC(platformInfo);
    auto totalCoreNum = ascendcPlatform.GetCoreNumAic();
    // ASCENDXXX请替换为实际的版本型号
    if (ascendcPlatform.GetSocVersion() == platform_ascendc::SocVersion::ASCENDXXX) {
        totalCoreNum = totalCoreNum + ascendcPlatform.GetCoreNumVector();
    }
    context->SetBlockDim(totalCoreNum);
}
```

说明

- 请参考Ascend C API中具体API支持的型号，来判断API接口是否支持Atlas 推理系列产品 Vector Core。
- 支持Vector Core后，因为AI Core和Vector Core会分别执行，通过不同的任务进行调度，所以不支持核间同步指令，如IBSet、IBWait、SyncAll等。
- 算子计算溢出（输入inf/nan或计算结果超出范围）时，需注意AI Core和Vector Core结果表现不一致，AI Core仅支持饱和模式，Vector Core仅支持inf/nan模式。

2.10.9.3 如何使用 workspace

workspace是设备侧Global Memory上的一块内存。workspace内存分为两部分：系统workspace和用户workspace。

- 系统workspace：Ascend C API需要预留的workspace内存
API在计算过程需要一些workspace内存作为缓存，因此算子需要为API预留workspace内存，预留内存大小通过GetLibApiWorkSpaceSize接口获取。
- 用户workspace：算子实现使用到的workspace内存
算子内部需要通过额外的device内存进行数据交换或者缓存的时候才需要分配，根据实际情况自行分配。使用场景如下：
 - 需要使用Unified Buffer和L1 Buffer上的空间且空间不够用时，可以将数据暂存至workspace上。
 - 调用SyncAll等API接口时，需要workspace作为入参。
 - 其他需要使用Global Memory上内存空间的场景。

不同开发方式下，具体的使用方法如下：

- 工程化算子开发方式
在tiling函数中先通过GetWorkspaceSizes接口获取workspace大小的存放位置，再设置workspace的大小，框架侧会为其申请对应大小的设备侧Global Memory，在对应的算子kernel侧实现时可以使用这块workspace内存。在使用Matmul Kernel侧接口等需要系统workspace的高阶API时，设置的workspace空间大小为系统workspace和用户workspace之和。

```
// 用户自定义的tiling函数
static ge::graphStatus TilingFunc(ge::TilingContext* context)
{
    AddApiTiling tiling;
    ...
    size_t usrSize = 256; // 设置用户需要使用的workspace大小为256字节。
    // 如需要使用系统workspace需要调用GetLibApiWorkSpaceSize获取系统workspace的大小。
    auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
    uint32_t sysWorkspaceSize = ascendcPlatform.GetLibApiWorkSpaceSize();
    size_t *currentWorkspace = context->GetWorkspaceSizes(1); // 通过框架获取workspace的指针，
    GetWorkspaceSizes入参为所需workspace的块数。当前限制使用一块。
    currentWorkspace[0] = usrSize + sysWorkspaceSize; // 设置总的workspace的数值大小，总的
    workspace空间由框架来申请并管理。
    ...
}
```


在device侧kernel入口处的workspace为用户的workspace指针：

```
// 用户写的Kernel函数，核函数必须包括GM_ADDR workspace入参，位置需要放在tiling之前
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR
workspace, GM_ADDR tiling)
{
    ...
}
```

- Kernel直调算子开发场景

需要使用workspace空间时，建议开启编译选项[HAVE_WORKSPACE](#)。host侧开发者仍需要自行申请workspace的空间，并传入。在使用Matmul Kernel侧接口等需要系统workspace的高阶API时，设置的workspace空间大小为系统workspace和用户workspace之和。系统workspace大小可以通过PlatformAscendCManager的GetLibApiWorkSpaceSize接口获取。开启[HAVE_WORKSPACE](#)后，开发者在kernel侧入参处获取的workspace为偏移了系统workspace后的用户workspace。

2.10.9.4 如何进行 Tiling 调测

在[工程化算子开发](#)过程中，开发者需实现Tiling函数，该函数原型是固定的，接受TilingContext作为输入。框架负责构造TilingContext并调用Tiling函数。若需单独进行Tiling调测，开发者可通过OpTilingRegistry加载编译后的Tiling动态库，获取Tiling函数的指针并进行调用，调用时Tiling函数的TilingContext入参使用ContextBuilder构建。

以下是具体步骤：

步骤1 参考工程化算子开发的开发步骤，完成算子实现，并通过[算子包编译](#)或[算子动态库编译](#)获取对应的Tiling动态库文件。

- 算子包编译：Tiling实现对应的动态库为算子包部署目录下的liboptiling.so。具体路径可参考[2.10.2.6.2 算子包部署](#)。
- 动态库编译：Tiling实现集成在算子动态库libcust_opapi.so中。具体路径可参考[2.10.2.7 算子动态库和静态库编译](#)。

步骤2 编写测试代码。

- 使用ContextBuilder配置输入输出Tensor的形状、数据类型、格式及平台信息等，构建TilingContext。
- 通过OpTilingRegistry的LoadTilingLibrary接口加载Tiling动态库；使用GetTilingFunc接口获取Tiling函数指针。
- 执行Tiling函数，验证其正确性。

```
// test.cpp
#include <iostream>
#include "exe_graph/runtime/storage_shape.h"
#include "tiling/context/context_builder.h"

int main()
{
    gert::StorageShape x_shape = {{2, 32}, {2, 32}};
    gert::StorageShape y_shape = {{2, 32}, {2, 32}};
    gert::StorageShape z_shape = {{2, 32}, {2, 32}};

    auto param = gert::TilingData::CreateCap(4096);
    auto workspace_size_holder = gert::ContinuousVector::Create<size_t>(4096);
    auto ws_size = reinterpret_cast<gert::ContinuousVector *>(workspace_size_holder.get());

    auto holder = context_ascend::ContextBuilder()
        .NodeloNum(2, 1)
        .IriInstanceNum({1, 1})
```



```
.AddInputTd(0, ge::DT_FLOAT, ge::FORMAT_ND, ge::FORMAT_ND, x_shape)
.AddInputTd(1, ge::DT_FLOAT, ge::FORMAT_ND, ge::FORMAT_ND, y_shape)
.AddOutputTd(0, ge::DT_FLOAT, ge::FORMAT_ND, ge::FORMAT_ND, z_shape)
.TilingData(param.get())
.Workspace(ws_size)
.AddPlatformInfo("Ascendxxxxy")
.BuildTilingContext();
auto tilingContext = holder->GetContext<gert::TilingContext>();
context_ascendc::OpTilingRegistry tmpIns;
bool flag = tmpIns.LoadTilingLibrary("/your/path/to/so_path/liboptiling.so"); // 加载对应的Tiling动态库文件
if (flag == false) {
    std::cout << "Failed to load tiling so" << std::endl;
    return -1;
}
context_ascendc::TilingFunc tilingFunc = tmpIns.GetTilingFunc("AddCustom"); // 获取AddCustom算子对应的Tiling函数, 此处入参为OpType
if (tilingFunc != nullptr) {
    ge::graphStatus ret = tilingFunc(tilingContext); // 执行Tiling函数
    if (ret != ge::GRAPH_SUCCESS) {
        std::cout << "Exec tiling func failed." << std::endl;
        return -1;
    }
} else {
    std::cout << "Get tiling func failed." << std::endl;
    return -1;
}
return 0;
}
```

步骤3 编译测试代码。

```
g++ test.cpp -I${INSTALL_DIR}/include -L${INSTALL_DIR}/lib64 -Wl,-rpath,${INSTALL_DIR}/lib64 -ltiling_api -lc_sec -lgraph_base -lregister -lunified_dlog -lplatform -o test
```

- \${INSTALL_DIR}请替换为CANN软件安装后文件存储路径。以root用户安装为例，安装后文件默认存储路径为：/usr/local/Ascend/cann。
- 开发者根据需要链接依赖的动态库，必需链接的动态库有：
 - libtiling_api.so：Tiling功能相关的动态库，包含ContextBuilder类、OpTilingRegistry类等。
 - libc_sec.so：安全函数库，libtiling_api.so依赖该库。
 - libgraph_base.so：基础数据结构与接口库，libtiling_api.so依赖该库。
 - libregister.so：业务函数注册相关库（例如Tiling函数注册，算子原型注册等）。
 - libunified_dlog.so：log库，libtiling_api.so依赖该库。
 - libplatform.so：平台信息库，libtiling_api.so依赖该库；Tiling函数中使用硬件平台信息时，需要依赖该库。

步骤4 执行可执行文件。

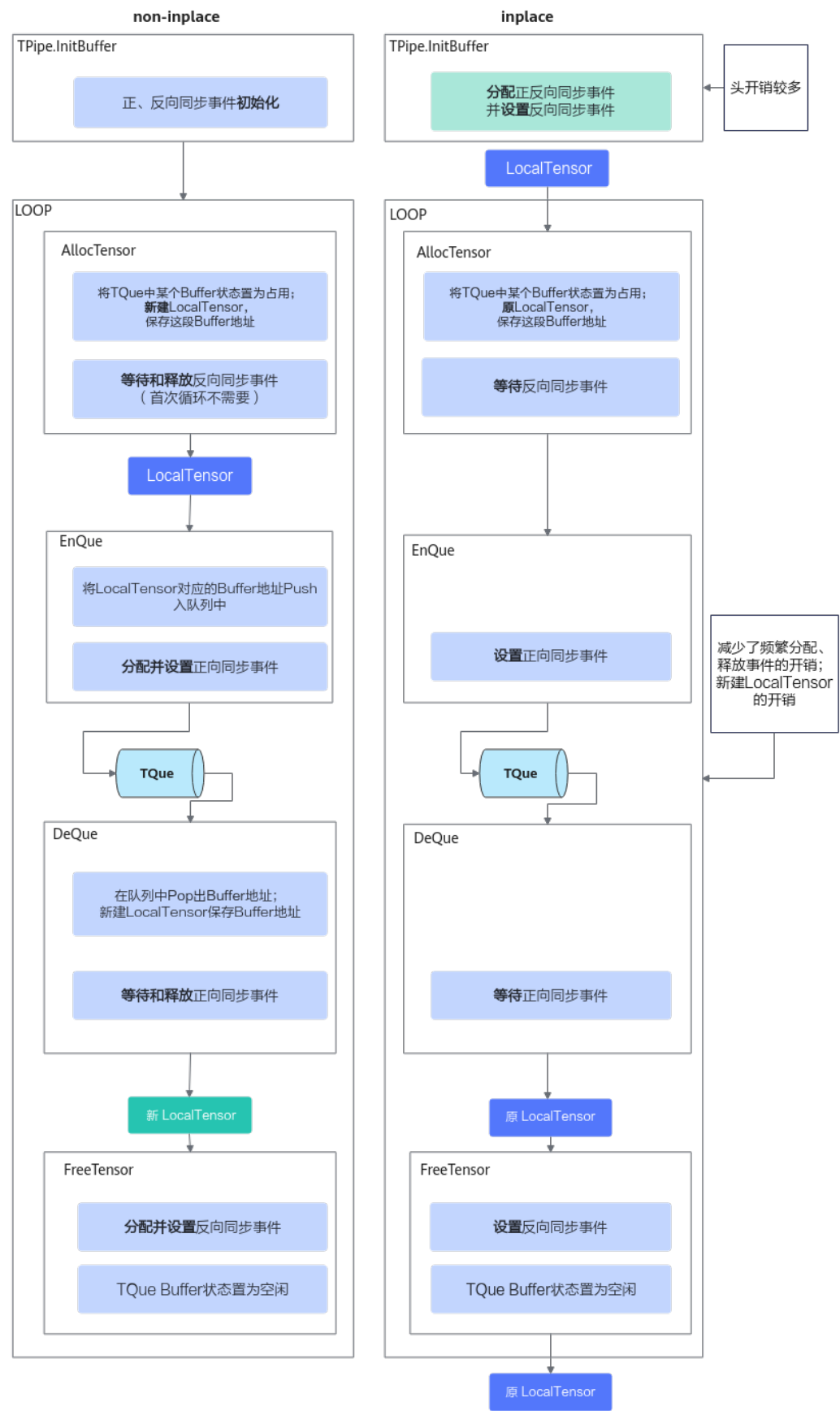
```
./test
```

----结束

2.10.9.5 如何使用 Tensor 原地操作提升算子性能

Tensor原地操作（inplace接口）是一种优化技术，全局申请、保留LocalTensor内存，避免了频繁创建和销毁LocalTensor对象。AllocTensor、FreeTensor、EnQue、DeQue接口不产生新的LocalTensor，而是在该全局LocalTensor上反复申请、释放、入队、出队。其实现原理如下图所示：

图 2-49 Tensor 原地操作实现原理



Tensor 原地操作的优势

- **减少栈变换**: 相比构造新Tensor的方式, inplace接口减少了LocalTensor的栈变换, 允许Tensor被反复使用。
- **减少入队/出队操作**: 在调用EnQue、DeQue的过程中, TQue对象没有存储该Tensor对应的Buffer地址, 实际没有真正入队、出队, 减少了反复入队、出队的Scalar指令。

保留 EnQue 和 DeQue 的原因

既然Tensor原地操作没有执行真正的入队出队操作, 为什么还需要保留EnQue和DeQue接口呢?

- **编程兼容性**: 为了保持编程接口的一致性, inplace接口仍然需要调用EnQue和DeQue, 确保代码结构的统一性和可维护性。
- **内存同步功能**: EnQue和DeQue操作中实现了内存读写同步功能, 确保数据的一致性和正确性, 即使没有实际的队列操作, 这些同步机制仍然需要保留。

适用场景

适合计算循环次数多的场景: 如图2-49所示, inplace接口虽然增加了TQue对象InitBuffer的初始化开销, 但显著减少了每次循环中AllocTensor、EnQue、DeQue和FreeTensor内部对LocalTensor和事件的操作次数, 特别适合需要多次循环来完成计算的场景。

使用方法

- **配置TQue对象**: 在创建TQue对象时, 设置深度 (depth) 为0, 启用inplace操作模式。
- **调用原地操作接口**: 使用inplace接口直接操作LocalTensor。
 - AllocTensor和DeQue区分non-inplace和inplace接口, 详情请参考AllocTensor、DeQue。
 - FreeTensor和EnQue不区分non-inplace和inplace接口。

示例代码

```
// ...
namespace AscendC {
class MyKernel {
public:
    __aicore__ inline MyKernel() {}
    __aicore__ inline void Init(__gm__ uint8_t* src0Gm, __gm__ uint8_t* src1Gm, __gm__ uint8_t* dstGm)
    {
        src0Global.SetGlobalBuffer((__gm__ half*)src0Gm);
        src1Global.SetGlobalBuffer((__gm__ half*)src1Gm);
        dstGlobal.SetGlobalBuffer((__gm__ half*)dstGm);
        pipe.InitBuffer(srcQue0, 1, BLOCK_SIZE * sizeof(half));
        pipe.InitBuffer(srcQue1, 1, BLOCK_SIZE * sizeof(half));
        pipe.InitBuffer(dstQue0, 1, BLOCK_SIZE * sizeof(half));
    }

    __aicore__ inline void Process()
    {
        for (int i = 0; i < REPTIMES; i++) {
            CopyIn(i);
            Compute(i);
            CopyOut(i);
        }
    }
};
```

```
    }  
private:  
    __aicore__ inline void CopyIn(int32_t i)  
    {  
        srcQue0.AllocTensor<half>(src0Local);  
        srcQue1.AllocTensor<half>(src1Local);  
        DataCopy(src0Local, src0Global[i*BLOCK_SIZE], BLOCK_SIZE);  
        DataCopy(src1Local, src1Global[i*BLOCK_SIZE], BLOCK_SIZE);  
        srcQue0.Enqueue(src0Local);  
        srcQue1.Enqueue(src1Local);  
    }  
    __aicore__ inline void Compute(int32_t i)  
    {  
        srcQue0.DeQueue<half>(src0Local);  
        srcQue1.DeQueue<half>(src1Local);  
        dstQue0.AllocTensor<half>(dstLocal);  
        Add(dstLocal, src0Local, src1Local, BLOCK_SIZE);  
        dstQue0.Enqueue<half>(dstLocal);  
        srcQue0.FreeTensor(src0Local);  
        srcQue1.FreeTensor(src1Local);  
    }  
    __aicore__ inline void CopyOut(int32_t i)  
    {  
        dstQue0.DeQueue<half>(dstLocal);  
        DataCopy(dstGlobal[i*BLOCK_SIZE], dstLocal, BLOCK_SIZE);  
        dstQue0.FreeTensor(dstLocal);  
    }  
private:  
    TPipe pipe;  
    TQue<QuePosition::VECIN, 0> srcQue0, srcQue1;  
    TQue<QuePosition::VECOUT, 0> dstQue0;  
    GlobalTensor<half> src0Global, src1Global, dstGlobal;  
    LocalTensor<half> src0Local;  
    LocalTensor<half> src1Local;  
    LocalTensor<half> dstLocal;  
};  
} // namespace AscendC  
  
// ...
```

2.10.10 FAQ

2.10.10.1 核函数运行验证时算子存在精度问题

现象描述

在进行算子NPU域的运行验证时，实际数据和真值数据不一致，算子存在精度问题。

问题根因

算子出现精度问题，一般是由于算子的实现逻辑有误。

定位步骤

Ascend C提供孪生调试的功能，通过CPU域的功能验证、gdb单步调试、printf数值打印来定位算子的实现逻辑问题。本样例仅展示了可能会出现的场景，便于演示定位步骤。实际使用过程中，请根据代码情况进行调试。

步骤1 进行CPU域的功能验证，观察是否有日志报错。

参考[2.10.7 基于样例工程完成Kernel直调](#)章节，编写CPU侧的运行验证代码，并进行运行验证，发现CPU域的精度比对也存在不一致的问题。

观察打屏日志中是否有报错信息，可搜索关键词"failed"。比如，下图的报错示例指示，错误出现在代码中调用LeakyRelu接口的地方。

```
leakyrelu_custom_cpu: /usr/local/Ascend/CANN-7.0/x86_64-linux/tikcpp/tikcfw/interface/  
kernel_operator_vec_binary_scalar_intf.h:447: void AscendC::LeakyRelu(const AscendC::LocalTensor<T>&,  
const AscendC::LocalTensor<T>&, const T&, const int32_t&) [with T = float16::Fp16T; int32_t = int]:  
Assertion `false && "check vrelu instr failed"' failed
```

通过上述报错日志，一般只能定位到报错的代码行，无法明确具体错误，接下来需要通过gdb调试的方式或者printf打印的方式进一步精确定位。

步骤2 gdb调试。下面的样例展示了拉起leakyrelu算子CPU侧运行程序的样例，该样例程序会直接抛出异常，直接gdb运行，查看调用栈信息分析定位即可。其他场景下您可以使用gdb打断点等基本操作进行调试。使用gdb调试Ascend C程序的详细内容请参考[2.7.2.1 CPU域孪生调试](#)。

1. 使用gdb拉起待调试程序，进入gdb界面进行debug。
gdb leakyrelu_custom_cpu
2. 单独调试一个子进程。
(gdb) set follow-fork-mode child
3. 运行程序。
(gdb) r
4. 通过bt查看程序调用栈。
(gdb) bt
5. 查看具体层的堆栈信息，打印具体变量的值。本示例中，打印了tileLength为1024，该程序中表示需要处理1024个half类型的数，大小为1024*sizeof(half)=2048字节；输入Tensor xLocal的值，其中dataLen表示LocalTensor的size大小为1024字节，只能计算1024字节的数据。可以看出两者的长度不匹配，由此可以定位问题。
(gdb) f 5
#5 0x000055555555d364 in KernelLeakyRelu::Compute (this=0x7fffffffd7d0, progress=0) at /root/AscendC_DemoCode-master/precision-error/vector/leakyrelu_custom.cpp:59
59 LeakyRelu(yLocal, xLocal, scalar, tileLength);
(gdb) p tileLength
\$1 = 1024
(gdb) p xLocal
\$1 = {<AscendC::BaseTensor<float16::Fp16T>> = {<No data fields>}, address_ = {logicPos = 9 '\t', bufferHandle = 0x7fffffffd930 "\003\005\377\377", dataLen = 1024,bufferAddr = 0,absAddr = ...}}

步骤3 printf打印。在合适的位置增加变量打印。样例代码如下：

```
printf("xLocal size: %d\n", xLocal.GetSize());  
printf("tileLength: %d\n", tileLength);
```

可以看到有如下打屏日志输出，打印了tileLength为1024，该程序中表示需要处理1024个half类型的数；输入Tensor xLocal的size大小，为512，表示只能计算512个half类型的数。可以看出两者的长度不匹配，由此可以定位问题。

```
xLocal size: 512  
tileLength: 1024
```

----结束

2.10.10.2 运行验证时 AllocTensor/FreeTensor 失败

现象描述

通过NPU进行核函数的运行验证时，出现挂死现象；通过CPU进行核函数的运行验证时，出现AllocTensor/FreeTensor失败的报错，日志报错和调用栈打印如下：

```
[ERROR][Core_0][usr/local/Ascend/cann/x86_64-linux/tikcpp/tikcfw/interface/kernel_tpipe.h:730]
[AllocEventID][321678] current size is 4, max buffer number in same queue position is 4
[ERROR][CORE_0][pid 321674] error happened! =====
SIGABRT Signal (Abort Signal from abort) caught, backtrace info:
[#0] 0x000000000001e7c0: handler(int) at /usr/local/Ascend/cann/tools/tikicpulib/lib/include/
kern_fw.h:105
[#1] 0x0000000000017c4f: signed char AscendC::TPipe::AllocEventID<(AscendC::HardEvent)5>() at /usr/
local/Ascend/cann/x86_64-linux/tikcpp/tikcfw/interface/kernel_tpipe.h:733
[#2] 0x000000000001426d: AscendC::TQueBind<(AscendC::TPosition)0, (AscendC::TPosition)9, 4,
0>::FreeBuffer(unsigned char*) at /usr/local/Ascend/cann/x86_64-linux/tikcpp/tikcfw/interface/
kernel_tpipe.h:1217
[#3] 0x0000000000011058: void AscendC::TQueBind<(AscendC::TPosition)0, (AscendC::TPosition)9, 4,
0>::FreeTensor<float16::Fp16T>(AscendC::LocalTensor<float16::Fp16T>&) at /usr/local/Ascend/cann/x86_64-
linux/tikcpp/tikcfw/interface/kernel_tpipe.h:1237
[#4] 0x0000000000000dfde: KernelAdd::Compute(int) at /home/xxxx/xxxx.cpp:59
[#5] 0x0000000000000dd1c: KernelAdd::Process() at /home/xxxx/xxxx.cpp:37 (discriminator 2)
...
```

问题根因

根据日志信息“current size is 4, max buffer number in same queue position is 4”可以明确该问题是因为同一个TPosition上QUE Buffer的数量超出限制导致。

同一个TPosition上的所有Queue，连续调用AllocTensor接口申请的Tensor数量，根据AI处理器型号的不同，有数量约束。申请Buffer时，需要满足该约束。

Atlas 训练系列产品不超过4个。

Atlas 推理系列产品AI Core不超过8个。

Atlas 推理系列产品Vector Core不超过8个。

Atlas A2 训练系列产品/Atlas A2 推理系列产品不超过8个。

Atlas A3 训练系列产品/Atlas A3 推理系列产品不超过8个。

Atlas 200I/500 A2 推理产品不超过8个。

不满足该约束，在后续使用AllocTensor/FreeTensor可能会出现分配资源失败。比如：

```
AscendC::TQue<AscendC::TPosition::VECIN, 1> que0;
AscendC::TQue<AscendC::TPosition::VECIN, 1> que1;
AscendC::TQue<AscendC::TPosition::VECIN, 1> que2;
AscendC::TQue<AscendC::TPosition::VECIN, 1> que3;
AscendC::TQue<AscendC::TPosition::VECIN, 1> que4;
AscendC::TQue<AscendC::TPosition::VECIN, 1> que5;
// 比如，算子有6个输入，需要申请6块buffer
// 通过6个队列为其申请内存，分别为que0~que5，每个que分配1块，申请VECIN TPosition上的buffer总数为6
// 假设，同一个Position上连续Alloc的Buffer数量限制为4，超出该限制后，使用AllocTensor/FreeTensor会出现
分配资源失败
// 在NPU上可能体现为卡死等异常行为，在CPU Debug场景会出现报错提示
pipe.InitBuffer(que0, 1, len);
pipe.InitBuffer(que1, 1, len);
pipe.InitBuffer(que2, 1, len);
pipe.InitBuffer(que3, 1, len);
pipe.InitBuffer(que4, 1, len);
pipe.InitBuffer(que5, 1, len);
```

```
AscendC::LocalTensor<T> local1 = que0.AllocTensor<T>();  
AscendC::LocalTensor<T> local2 = que1.AllocTensor<T>();  
AscendC::LocalTensor<T> local3 = que2.AllocTensor<T>();  
AscendC::LocalTensor<T> local4 = que3.AllocTensor<T>();  
// 第5个AllocTensor会出现资源分配失败, 同一个TPosition上同时Alloc出来的Tensor数量超出了4个的限制  
AscendC::LocalTensor<T> local5 = que4.AllocTensor<T>();
```

处理步骤

如果确实有多块buffer使用, 可以将多个buffer合并到一块buffer, 通过偏移使用。样例如下:

```
// 此时建议通过以下方法解决:  
// 如果确实有多块buffer使用, 可以将多个buffer合并到一块buffer, 通过偏移使用  
pipe.InitBuffer(que0, 1, len * 3);  
pipe.InitBuffer(que1, 1, len * 3);  
/*  
 * 分配出3块内存大小的LocalTensor, local1的地址为que0中buffer的起始地址,  
 * local2的地址为local1的地址偏移len后的地址, local3的地址为local1的地址偏移  
 * len * 2的地址  
 */  
int32_t offset1 = len;  
int32_t offset2 = len * 2;  
AscendC::LocalTensor<T> local1 = que0.AllocTensor<T>();  
AscendC::LocalTensor<T> local2 = local1[offset1];  
AscendC::LocalTensor<T> local3 = local1[offset2];
```

2.10.10.3 kernel 侧获取 Tiling 信息不正确

现象描述

通过算子在kernel侧实现代码中添加PRINTF打印发现kernel侧获取的Tiling信息不正确。

比如下文样例, 增加的打印代码如下:

```
PRINTF("tiling_data.totalLength: %d tiling_data.tileNum: %d.\n", tiling_data.totalLength,  
tiling_data.tileNum);
```

打印的Tiling数据如下, 全为0:

```
tiling_data.totalLength: 0 tiling_data.tileNum: 0.
```

问题根因

kernel侧获取Tiling信息不正确的原因一般有以下两种:

- host侧计算Tiling的逻辑不正确
- kernel侧核函数的参数未按照正确顺序填写

处理步骤

步骤1 参考如下示例, 打印TilingData的数据, 确认host侧序列化保存的TilingData是否正确。如果此时打印值有误, 说明Tiling的计算逻辑可能不正确, 需要进一步检查host侧Tiling实现代码, 排查计算逻辑是否有误。

```
std::cout<<*reinterpret_cast<uint32_t*>(context->GetRawTilingData()->GetData())<<std::endl; //按照实际数据类型打印TilingData第一个参数值, 如需确认其他值, 取值指针向后偏移即可
```

步骤2 如果上一步骤中打印的TilingData正确, 需要排查kernel侧核函数的参数是否按照正确顺序填写。

使用msOpGen工具创建算子工程，并基于工程进行kernel侧算子开发时，核函数的定义模板已通过msOpGen工具自动生成，样例如下所示。参数按照“输入、输出、workspace、tiling”的顺序排布。请检查是否调整过参数顺序导致和正确顺序不一致。

```
#include "kernel_operator.h"
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR
workspace, GM_ADDR tiling) {
    GET_TILING_DATA(tiling_data, tiling); // 获取Tiling参数
    // TODO: user kernel impl
}
```

----结束

2.10.10.4 Kernel 编译时报错 “error: out of jump/jumpc imm range”

现象描述

使用工程化算子开发方式，基于自定义算子工程进行算子开发。编译算子时失败，报如下错误：

```
[ERROR] [ascendxxxx] PowerCustom_88a695f03edfbc0af76b9eaae9e4556c error: out of jump/jumpc imm
range
```

问题根因

该编译错误的原因是算子kernel代码过大，导致在编译时跳转指令跳转的偏移值超过了限定的大小(int16_t的数据范围)，可通过添加编译选项“-mllvm -cce-aicore-jump-expand=true”通过间接跳转的方式来避免该问题，让编译器能够正常编译。

处理步骤

步骤1 在kernel侧的CMakeLists中通过add_ops_compile_options针对报错算子添加编译选项“-mllvm -cce-aicore-jump-expand=true”，示例如下：

```
add_ops_compile_options(PowerCustom OPTIONS -mllvm -cce-aicore-jump-expand=true)
```

add_ops_compile_options的具体使用方法请参考[支持自定义编译选项](#)。

步骤2 重新编译该算子。正常编译无报错。

----结束

2.10.10.5 使用跨版本的自定义算子包时，含有 Matmul 高阶 API 的算子存在编译或执行报错

现象描述

1. 基于CANN-7.2及之前版本（<=7.2）的CANN开发套件包，编译含有Matmul高阶API的自定义算子包，将编译后的自定义算子包安装至CANN-7.3及之后版本（>=7.3）的CANN包环境，然后对该含有Matmul高阶API的算子，执行图模式在线编译时，报如下错误：

```
res = struct.unpack_from(fmt_str, tiling_data, offset + unpack_size)
struct.error: unpack_from requires a buffer of at least 52 bytes for unpacking 4 bytes at offset 48
```

2. 基于CANN-7.2及之前版本（<=7.2）的CANN开发套件包，编译[sample样例仓](#)中含有Matmul高阶API的算子，例如[MatmulLeakyReluCustomSample](#)，将编译

后的自定义算子包安装至CANN-7.3及之后版本（ ≥ 7.3 ）的CANN包环境，然后对该含有Matmul高阶API的算子，执行单算子API的调用时，报如下错误：
ERROR: acl executable run failed! please check your project!

问题根因

该错误的原因是编译自定义算子包的软件版本过老，可通过更新自定义算子包编译环境上的CANN开发套件包版本，然后重新编译和部署自定义算子包，来避免出现该问题。

处理步骤

步骤1 查看自定义算子包编译时使用的CANN开发套件包版本号，示例如下：

```
cd ${CANN包安装路径}
cat version.cfg

# version: 1.0
runtime_running_version=[7.2.T11.0.B218:8.0.RC2.alpha001]
runtime_upgrade_version=[7.2.T11.0.B218:8.0.RC2.alpha001]
runtime_installed_version=[7.2.T11.0.B218:8.0.RC2.alpha001]
```

步骤2 基于CANN-7.3及之后版本（ ≥ 7.3 ）的CANN开发套件包，重新编译该自定义算子包。部署编译生成的自定义算子包后，正常编译或者执行算子，无报错。重新编译和部署自定义算子包的具体方法可参考[2.10.2.6 算子包编译](#)。

----结束

2.10.10.6 含有 Matmul 高阶 API 的算子精度问题

本节针对含有Matmul高阶API的算子，为排查算子精度问题是否为算子中Matmul高阶API调用方式导致，提供初步的问题定界和定位指导。如未特殊说明，下面均以Atlas A2 训练系列产品/Atlas A2 推理系列产品上的案例为例。

具体排查过程主要有如下六个步骤：

1. CPU域调试，观察报错信息；
2. Matmul Tiling是否有修改，修改是否合理；
3. 算子隐藏Vector计算，仅调用Matmul API，算子功能是否正确；
4. 单核执行，算子功能是否正确；
5. 排查Matmul API的使用是否正确；
6. 用于算子调测的golden脚本是否正确。

步骤1 CPU域调试，观察报错信息

在完成算子代码的开发后，优先通过[Kernel直调中的CPU调测工程](#)，调试算子的功能。在CPU域调试时，若编译或执行报错，日志中一般会有明显的报错信息。根据报错信息的提示内容，通常可以快速定位到问题所对应的代码位置。这种方法尤其对DataCopy参数设置错误导致的地址越界、算子Tiling参数设置不正确、其他内存越界访问等基础参数的使用问题，可以快速定位到具体原因。

1. 案例：

以下为matmul算子核函数的代码片段。该段代码实现了根据Global Memory上的A、B矩阵和Tiling信息，计算每个核要使用数据的地址偏移、创建Matmul对象，计算得到Matmul结果。

```
extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR c,
GM_ADDR workspace, GM_ADDR tilingGm)
```

```
{
    using A_T = half;
    using B_T = half;
    using C_T = float;

    AscendC::TPipe pipe;
    TCubeTiling tiling;
    CopyTiling(&tiling, tilingGm);

    AscendC::GlobalTensor<A_T> aGlobal;
    AscendC::GlobalTensor<B_T> bGlobal;
    AscendC::GlobalTensor<C_T> cGlobal;
    aGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ A_T *>(a), tiling.M * tiling.Ka);
    bGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ B_T *>(b), tiling.Ka * tiling.N);
    cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ C_T *>(c), tiling.M * tiling.N);

    int offsetA = 0;
    int offsetB = 0;
    int offsetC = 0;
    bool isTransA = false;
    bool isTransB = true;

    int tailM = 0;
    int tailN = 0;
    CalcGMOffset(GetBlockIdx(), tiling, offsetA, offsetB, offsetC, tailM, tailN, isTransA, isTransB);

    auto gmA = aGlobal[offsetA];
    auto gmB = bGlobal[offsetB];
    auto gmC = cGlobal[offsetC];

    AscendC::Matmul<AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, A_T>,
        AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, B_T>,
        AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, C_T>> mm;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling);

    mm.SetTensorA(gmA, isTransA);
    mm.SetTensorB(gmB, isTransB);
    mm.SetTail(tailM, tailN);
    mm.IterateAll(gmC);
    mm.End();
}
```

以下为上述代码在CPU域调试时输出的执行结果。以下示例中的路径请以实际情况为准。

```
[ASSERT] $HOME/Ascend/xxxxx/include/ascendc/highlevel_api/lib/matmul/matmul_client.h:268:
Assertion `isTransposeB <= B_TYPE::isTrans && "It is not allowed to do B transpose when matmul B
transpose is not defined."'
[ASSERT] $HOME/Ascend/xxxxx/include/ascendc/highlevel_api/lib/matmul/matmul_client.h:268:
Assertion `isTransposeB <= B_TYPE::isTrans && "It is not allowed to do B transpose when matmul B
transpose is not defined."'
```

本案例中的算子有精度问题，于是使用CPU调测该算子功能，CPU运行后，根据报错信息提示的矩阵B的transpose未定义，查看矩阵B的相关设置代码，发现Matmul对象定义时未设置矩阵B的B_TYPE::isTrans，而SetTensorB接口设置了isTransB = true，导致执行报错。所以，此问题的根因为SetTensorB设置的isTransB值与B_TYPE不符。

步骤2 Matmul Tiling是否有修改，修改是否合理

一般含有Matmul的算子Tiling实现中，通过调用GetTiling接口获取Matmul Tiling，其数据类型为TCubeTiling结构体，这时这组Tiling值是合法的。某些情况下，用户自定义了一组TCubeTiling参数值，或者，基于GetTiling接口返回的TCubeTiling，自行修改了其中的部分值，这样的修改需要满足参数间的制约条件。

为获取所有Tiling参数值，需要打印Tiling参数相关的日志。设置日志环境变量，获取MatmulTiling参数值。设置环境变量的命令如下：

```
export ASCEND_GLOBAL_LOG_LEVEL=1
export ASCEND_SLOG_PRINT_TO_STDOUT=1
```

在日志中搜索“MatmulTiling”关键字，参照TCubeTiling约束条件，检查Tiling取值是否合法。若不满足某条约束条件，需要修改对应的相关参数，使该组TCubeTiling参数值均合法。

```
cat test_tiling.log |grep MatmulTiling // test_tiling.log为示例日志文件名
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.864
[matmul_tiling_base.cpp:697][PrintTilingDataInfo] MatmulTiling: M = 1024
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.870
[matmul_tiling_base.cpp:698][PrintTilingDataInfo] MatmulTiling: N = 640
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.873
[matmul_tiling_base.cpp:699][PrintTilingDataInfo] MatmulTiling: Ka = 256
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.876
[matmul_tiling_base.cpp:700][PrintTilingDataInfo] MatmulTiling: Kb = 256
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.879
[matmul_tiling_base.cpp:701][PrintTilingDataInfo] MatmulTiling: singleCoreM = 512
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.882
[matmul_tiling_base.cpp:702][PrintTilingDataInfo] MatmulTiling: singleCoreN = 640
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.884
[matmul_tiling_base.cpp:703][PrintTilingDataInfo] MatmulTiling: singleCoreK = 256
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.887
[matmul_tiling_base.cpp:704][PrintTilingDataInfo] MatmulTiling: baseM = 256
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.890
[matmul_tiling_base.cpp:705][PrintTilingDataInfo] MatmulTiling: baseN = 128
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.893
[matmul_tiling_base.cpp:706][PrintTilingDataInfo] MatmulTiling: baseK = 64
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.896
[matmul_tiling_base.cpp:707][PrintTilingDataInfo] MatmulTiling: depthA1 = 10
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.899
[matmul_tiling_base.cpp:708][PrintTilingDataInfo] MatmulTiling: depthB1 = 2
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.902
[matmul_tiling_base.cpp:709][PrintTilingDataInfo] MatmulTiling: depthAL1CacheUB = 0
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.905
[matmul_tiling_base.cpp:710][PrintTilingDataInfo] MatmulTiling: depthBL1CacheUB = 0
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.908
[matmul_tiling_base.cpp:711][PrintTilingDataInfo] MatmulTiling: stepM = 2
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.912
[matmul_tiling_base.cpp:712][PrintTilingDataInfo] MatmulTiling: stepN = 1
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.915
[matmul_tiling_base.cpp:713][PrintTilingDataInfo] MatmulTiling: isBias = 1
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.917
[matmul_tiling_base.cpp:714][PrintTilingDataInfo] MatmulTiling: transLength = 0
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.920
[matmul_tiling_base.cpp:715][PrintTilingDataInfo] MatmulTiling: iterateOrder = 0
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.923
[matmul_tiling_base.cpp:716][PrintTilingDataInfo] MatmulTiling: shareMode = 0
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.926
[matmul_tiling_base.cpp:717][PrintTilingDataInfo] MatmulTiling: usedL1Size = 295424
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.929
[matmul_tiling_base.cpp:718][PrintTilingDataInfo] MatmulTiling: usedL0CSize = 131072
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.932
[matmul_tiling_base.cpp:719][PrintTilingDataInfo] MatmulTiling: usedUBSize = 0
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.935
[matmul_tiling_base.cpp:720][PrintTilingDataInfo] MatmulTiling: batchM = 1
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.938
[matmul_tiling_base.cpp:721][PrintTilingDataInfo] MatmulTiling: batchN = 1
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.941
[matmul_tiling_base.cpp:722][PrintTilingDataInfo] MatmulTiling: singleBatchM = 1
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.943
[matmul_tiling_base.cpp:723][PrintTilingDataInfo] MatmulTiling: singleBatchN = 1
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.946
[matmul_tiling_base.cpp:724][PrintTilingDataInfo] MatmulTiling: stepKa = 4
[INFO] ASCENDCKERNEL(1202803,ascendc_kernels_bbit):2024-10-12-08:53:59.636.949
[matmul_tiling_base.cpp:725][PrintTilingDataInfo] MatmulTiling: stepKb = 1
```

例如，根据如上打印出的TCubeTiling参数，对照TCubeTiling约束条件查看各个参数的取值，depthA1的取值应该等于stepM*stepKa或者stepM*stepKa*2，而depthA1的取

值为10，既不等于stepM*stepKa=8，也不等于stepM*stepKa*2=16，不满足约束条件，因此需要校正depthA1的值。

步骤3 算子隐藏Vector计算，仅调用Matmul API，算子功能是否正确

融合算子的代码既包含Matmul API，也包含Vector计算API。通过在算子代码中删除Vector计算API，只保留Matmul API，快速定界是否为Matmul API的错误使用导致了融合算子的精度问题。具体排查过程为：修改算子代码逻辑，删除Vector计算的代码，同步完成golden脚本相应修改，完成适配修改后，**CPU域或NPU域上执行算子**，观察算子结果是否正确。若算子结果正确，说明代码中Matmul API的使用方式正确，需要继续排查Vector计算是否正确；反之，若算子结果不正确，需要继续排查Matmul API的使用是否正确。

- 案例：

以融合算子matmul_leakyrelu为例，执行算子后，出现如下图所示的精度问题。

```
data index: 000195, expected: -0.693000019, actual: -69.300003052, rdiff: -99.000000
data index: 000196, expected: -0.209000006, actual: -20.899999619, rdiff: -99.000000
data index: 000197, expected: -0.517000020, actual: -51.700000763, rdiff: -99.000000
data index: 000200, expected: -0.193000004, actual: -19.300001144, rdiff: -99.000000
data index: 000202, expected: -0.684000015, actual: -68.400001526, rdiff: -99.000000
data index: 000204, expected: -0.422000021, actual: -42.200000763, rdiff: -98.999992
data index: 000209, expected: -0.109000005, actual: -10.900000572, rdiff: -99.000000
error ratio: 0.4517, tolerance: 0.0001
[ERROR] result error
```

修改算子代码，注释屏蔽LeakyRelu API计算，同时，需要适配修改相应的内存分配和涉及的同步等代码；然后，注释golden脚本中LeakyRelu计算，具体修改示例如下。

以下代码为算子核函数的代码片段。

```
template <typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::Process(AscendC::TPipe
*pipe)
{
    uint32_t computeRound = 0;

    matmulObj.SetTensorA(aGlobal);
    matmulObj.SetTensorB(bGlobal);
    matmulObj.SetBias(biasGlobal);
    while (matmulObj.template Iterate<true>()) {
        MatmulCompute();
        // LeakyReluCompute(); // 注释LeakyReluCompute Vector计算
        CopyOut(computeRound);
        computeRound++;
    }
    matmulObj.End();
}

template <typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::MatmulCompute()
{
    reluOutLocal = reluOutQueue_.AllocTensor<cType>();
    matmulObj.template GetTensorC<true>(reluOutLocal, false, true);
    reluOutQueue_.EnQue(reluOutLocal); // 将LeakyReluCompute()接口里的reluOutLocal结果输出提前
到这里
}

template <typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::LeakyReluCompute()
{
    LeakyRelu(reluOutLocal, reluOutLocal, (cType)0.1, tiling.baseM * tiling.baseN);
    reluOutQueue_.EnQue(reluOutLocal);
}

template <typename aType, typename bType, typename cType, typename biasType>
```

```
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::CopyOut(uint32_t count)
{
    reluOutQueue_.DeQueue<cType>();
    const uint32_t roundM = tiling.singleCoreM / tiling.baseM;
    const uint32_t roundN = tiling.singleCoreN / tiling.baseN;
    uint32_t startOffset = (count % roundM * tiling.baseM * tiling.N + count / roundM * tiling.baseN);
    AscendC::DataCopyParams copyParam = {(uint16_t)tiling.baseM, (uint16_t)(tiling.baseN *
    sizeof(cType) / AscendC::DEFAULT_C0_SIZE), 0,
    (uint16_t)((tiling.N - tiling.baseN) * sizeof(cType) /
    AscendC::DEFAULT_C0_SIZE)};
    DataCopy(cGlobal[startOffset], reluOutLocal, copyParam);
    reluOutQueue_.FreeTensor(reluOutLocal);
}
```

以下代码为golden生成脚本的代码片段。

```
def gen_golden_data():
    M = 1024
    N = 640
    K = 256

    input_a = np.random.randint(-10, 10, [M, K]).astype(np.float16)
    input_b = np.random.randint(-10, 10, [K, N]).astype(np.float16)
    input_bias = np.random.randint(-10, 10, [N]).astype(np.float32)
    alpha = 0.001
    golden = (np.matmul(input_a.astype(np.float32), input_b.astype(np.float32)) +
    input_bias).astype(np.float32)
    # golden = np.where(golden >= 0, golden, golden * alpha) # 与kernel保持一致，golden生成也需注
    释相应的LeakyRelu计算
    os.system("mkdir -p input")
    os.system("mkdir -p output")
    input_a.tofile("./input/x1_gm.bin")
    input_b.tofile("./input/x2_gm.bin")
    input_bias.tofile("./input/bias.bin")
    golden.tofile("./output/golden.bin")
```

删除LeakyRelu计算后，执行用例，算子结果比对正确，如下所示。

```
-- Installing: $HOME/samples/Precision_Check_Guide/samples-master/operator/
MatmulLeakyReluCustomSample/KernellLaunch/MatmulLeakyReluInvocation_cube_vec/out/bin/
ascendc_kernels_bbit
8901941eee314bcd64d24ff5f8d21247 output/golden.bin
8901941eee314bcd64d24ff5f8d21247 output/output.bin
error ratio: 0.0000, tolerance: 0.0001
test pass
```

由此可确定，算子代码中已正确使用Matmul API，并得到了正确的Matmul API 计算结果，需要继续定位LeakyReluCompute函数内LeakyRelu接口使用中存在的问题。

步骤4 单核执行，算子功能是否正确

验证单核场景下算子的功能是否正确，可以帮助快速定界是Matmul API的计算结果不符合预期，还是算子代码中错误调用Matmul API导致。由于Matmul API内部实现的是单核的计算逻辑，所以单核的计算结果正确，而多核的计算结果错误的情况，说明单核上的Matmul API的使用及计算正确，这时需要排查与多核切分相关的代码逻辑是否正确，比如每个核的输入和输出地址偏移是否正确，每个核上的尾块地址设置是否正确。如果验证单核场景下，算子精度不正确，需要排查Matmul API的使用是否正确，具体可参考[步骤5](#)。

提示，包含Matmul的算子的Tiling实现中，Matmul的多核Tiling需要使用MultiCoreMatmulTiling构造多核Tiling对象，通过SetDim接口设置Matmul计算所用的核数。注意：这里设置的核数为Matmul计算所用的核数，仅在多核场景下设置，用于计算tiling参数。如下两个案例为MIX模式的算子，SetDim的设置规则请参考[MIX场景核数设置规则](#)。

- 案例1：多核切分场景，输出地址偏移不正确

以M=512, N=1024, K=512的Matmul为例, MIX模式的算子代码中设置AIC核数为4, AIV核数为8, 因为本案例以分离模式为例, 所以SetDim设置为AIV核数的取值8。多核场景下执行该算子, 计算结果精度错误。

以下为算子Tiling计算的代码片段。

```
uint8_t *GenerateTiling(const char *socVersion)
{
    int M = 512;
    int N = 1024;
    int K = 512;

    TPosition leftPosition = TPosition::GM;
    CubeFormat leftFormat = CubeFormat::ND;
    DataType leftDtype = DataType::DT_FLOAT16;
    bool isTransA = false;

    TPosition rightPosition = TPosition::GM;
    CubeFormat rightFormat = CubeFormat::ND;
    DataType rightDtype = DataType::DT_FLOAT16;
    bool isTransB = false;

    TPosition resultPosition = TPosition::GM;
    CubeFormat resultFormat = CubeFormat::ND;
    DataType resultDtype = DataType::DT_FLOAT;

    bool isBias = false;

    int usedCoreNum = 8;
    int32_t baseM = 128;
    int32_t baseN = 256;

    optiling::TCubeTiling tilingData;
    auto ascendcPlatform = platform_ascendc::PlatformAscendCManager::GetInstance(socVersion);
    MultiCoreMatmulTiling tilingApi(*ascendcPlatform);

    tilingApi.SetDim(usedCoreNum); // 设置为AIV核数8
    tilingApi.SetAType(leftPosition, leftFormat, leftDtype, isTransA);
    tilingApi.SetBType(rightPosition, rightFormat, rightDtype, isTransB);
    tilingApi.SetCTYPE(resultPosition, resultFormat, resultDtype);

    tilingApi.SetOrgShape(M, N, K);
    tilingApi.SetShape(M, N, K);
    tilingApi.SetFixSplit(baseM, baseN, -1);
    tilingApi.SetBias(isBias);
    tilingApi.SetBufferSpace(-1, -1, -1);

    int64_t res = tilingApi.GetTiling(tilingData);
    if (res == -1) {
        std::cout << "gen tiling failed" << std::endl;
    }
    return GetTilingBuf(&tilingData);
}
```

以下为算子核函数的代码片段。

```
__aicore__ inline void CalcGMOffset(int blockIdx, const TCubeTiling &tiling, int &offsetA, int &offsetB,
int &offsetC,
                                int &tailM, int &tailN, bool isTransA, bool isTransB)
{
    uint32_t mSingleBlocks = CeilDiv(tiling.M, tiling.singleCoreM);
    uint32_t mCoreIndx = blockIdx % mSingleBlocks;
    uint32_t nCoreIndx = blockIdx / mSingleBlocks;

    offsetA = mCoreIndx * tiling.Ka * tiling.singleCoreM;
    if (isTransA) {
        offsetA = mCoreIndx * tiling.singleCoreM;
    }
    offsetB = nCoreIndx * tiling.singleCoreN;
    if (isTransB) {
        offsetB = nCoreIndx * tiling.Kb * tiling.singleCoreN;
    }
}
```

```
    }
    offsetC = mCoreIdx * tiling.singleCoreN * tiling.singleCoreM + nCoreIdx * tiling.singleCoreN; //此处的
    tiling.singleCoreN参数错误，应为tiling.N

    tailM = tiling.M - mCoreIdx * tiling.singleCoreM;
    tailM = tailM < tiling.singleCoreM ? tailM : tiling.singleCoreM;

    tailN = tiling.N - nCoreIdx * tiling.singleCoreN;
    tailN = tailN < tiling.singleCoreN ? tailN : tiling.singleCoreN;
}

extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR c,
GM_ADDR workspace,
GM_ADDR tilingGm)
{
    using A_T = half;
    using B_T = half;
    using C_T = float;

    AscendC::TPipe pipe;
    TCubeTiling tiling;
    CopyTiling(&tiling, tilingGm);

    AscendC::GlobalTensor<A_T> aGlobal;
    AscendC::GlobalTensor<B_T> bGlobal;
    AscendC::GlobalTensor<C_T> cGlobal;
    aGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ A_T *>(a), tiling.M * tiling.Ka);
    bGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ B_T *>(b), tiling.Ka * tiling.N);
    cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ C_T *>(c), tiling.M * tiling.N);

    int offsetA = 0;
    int offsetB = 0;
    int offsetC = 0;
    bool isTransA = false;
    bool isTransB = false;

    int tailM = 0;
    int tailN = 0;
    CalcGMOffset(GetBlockIdx(), tiling, offsetA, offsetB, offsetC, tailM, tailN, isTransA, isTransB);

    auto gmA = aGlobal[offsetA];
    auto gmB = bGlobal[offsetB];
    auto gmC = cGlobal[offsetC];

    AscendC::Matmul<AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, A_T>,
    AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, B_T>,
    AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, C_T>> mm;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling);

    mm.SetTensorA(gmA, isTransA);
    mm.SetTensorB(gmB, isTransB);
    mm.SetTail(tailM, tailN);
    mm.IterateAll(gmC);
    mm.End();
}
```

执行算子，精度校验失败：

```
data index: 000609, expected: 12979.000000000, actual: 0.000000000, rdifff: 1.000000
data index: 000610, expected: 12931.000000000, actual: 0.000000000, rdifff: 1.000000
data index: 000611, expected: 13120.000000000, actual: 0.000000000, rdifff: 1.000000
data index: 000612, expected: 12275.000000000, actual: 0.000000000, rdifff: 1.000000
error ratio: 0.8750, tolerance: 0.0001
[ERROR] result error
```

修改测试脚本和算子Tiling的代码，通过验证单核上的算子执行结果，快速定界。
具体如下：

修改算子调测代码，只启动单核，CPU调测代码中将ICPU_RUN_KF宏接口中的numBlocks设置为1（表示一组AIC和AIV）；算子的Tiling实现中，设置单核场景，AIC核数为1，AIV核数为2，SetDim设置为AIV核数的取值2。如下代码所示。以下为调测脚本的代码片段。

```
uint32_t numBlocks = 1;  
ICPU_RUN_KF(matmul_custom, numBlocks, a, b, c, workspace, tiling);
```

以下为算子Tiling计算的代码片段。

```
int usedCoreNum = 2;  
tilingApi.SetDim(usedCoreNum);
```

修改为单核场景后，执行算子：

```
-- Installing: $HOME/samples/Precision_Check_Guide/samples-master/operator/  
MatmulCustomSample/KernelLaunch/MatmulInvocationNeo-muticore/out/bin/ascendc_kernels_bbit  
efaf4dc1e484bc3778cac65f56244e59 output/golden.bin  
efaf4dc1e484bc3778cac65f56244e59 output/output.bin  
error ratio: 0.0000, tolerance: 0.0001  
test pass
```

从上述比对结果可看出，单核验证结果正确，此时可以定界导致精度的问题与多核逻辑相关。

首先排查多核切分后的输入和输出地址偏移。分析CalcGMOffset函数，定位到矩阵C的偏移地址offsetC计算错误，正确的偏移应该是mCoreIdx * tiling.N * tiling.singleCoreM + nCoreIdx * tiling.singleCoreN。将offsetC修改为正确的偏移地址后，执行算子，结果比对正确。

提示，在上述单核场景的修改验证中，AIC核数为1，AIV核数为2；若想进一步验证，不引入任何多核切分，AIC核数和AIV核数均修改为1，代码修改示例如下：

- 在核函数中REGIST_MATMUL_OBJ接口后，利用判断代码，BlockIdx不为0的AIV核退出。

以下为算子核函数的代码片段。

```
extern "C" __global__ __aicare__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR c,  
                                                    GM_ADDR workspace,  
                                                    GM_ADDR tilingGm)  
{  
    using A_T = half;  
    using B_T = half;  
    using C_T = float;  
  
    AscendC::TPipe pipe;  
    TCubeTiling tiling;  
    CopyTiling(&tiling, tilingGm);  
  
    AscendC::GlobalTensor<A_T> aGlobal;  
    AscendC::GlobalTensor<B_T> bGlobal;  
    AscendC::GlobalTensor<C_T> cGlobal;  
    aGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ A_T*>(a), tiling.M * tiling.Ka);  
    bGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ B_T*>(b), tiling.Ka * tiling.N);  
    cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ C_T*>(c), tiling.M * tiling.N);  
  
    int offsetA = 0;  
    int offsetB = 0;  
    int offsetC = 0;  
    bool isTransA = false;  
    bool isTransB = false;  
  
    int tailM = 0;  
    int tailN = 0;  
    CalcGMOffset(GetBlockIdx(), tiling, offsetA, offsetB, offsetC, tailM, tailN, isTransA, isTransB);  
  
    auto gmA = aGlobal[offsetA];  
    auto gmB = bGlobal[offsetB];  
    auto gmC = cGlobal[offsetC];
```



```
AscendC::Matmul<AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, A_T>,  
AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, B_T>,  
AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, C_T>> mm;  
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling);  
if (GetBlockIdx() == 1) {  
    return;  
}  
  
mm.SetTensorA(gmA, isTransA);  
mm.SetTensorB(gmB, isTransB);  
mm.SetTail(tailM, tailN);  
mm.IterateAll(gmC);  
mm.End();  
}
```

- 算子调测脚本的ICPU_RUN_KF中numBlocks和算子Tiling中SetDim的usedCoreNum均设置为1。

以下为算子调测代码片段。

```
uint32_t numBlocks = 1;  
ICPU_RUN_KF(matmul_custom, numBlocks, a, b, c, workspace, tiling);
```

以下为算子Tiling计算的代码片段。

```
int usedCoreNum = 1;  
tilingApi.SetDim(usedCoreNum);
```

- 案例2：尾块设置不正确

多核场景下，当最后一个核的singleCoreM/singleCoreN/singleCoreK值与前面的核取值不同时，需要在最后一个核上，即尾核，调用SetTail接口，调整singleCoreM/singleCoreN/singleCoreK为实际尾核上的对应取值；若尾核未设置这些参数值，或者设置的参数值大小不正确，也会导致多核精度错误，单核精度正确。

```
data index: 100254, expected: 13605.000000000, actual: 13137.000000000, rdiff: 0.034399  
data index: 101277, expected: 13268.000000000, actual: 13419.000000000, rdiff: 0.011381  
data index: 102300, expected: 13509.000000000, actual: 13114.000000000, rdiff: 0.029240  
data index: 103323, expected: 13526.000000000, actual: 13400.000000000, rdiff: 0.009315  
error ratio: 0.0010, tolerance: 0.0001  
[ERROR] result error
```

以下为算子核函数的代码片段。

```
__aicore__ inline void CalcGMOffset(int blockIdx, const TCubeTiling &tiling, int &offsetA, int &offsetB,  
int &offsetC,  
int &tailM, int &tailN, bool isTransA, bool isTransB)  
{  
    uint32_t mSingleBlocks = CeilDiv(tiling.M, tiling.singleCoreM);  
    uint32_t mCoreIdx = blockIdx % mSingleBlocks;  
    uint32_t nCoreIdx = blockIdx / mSingleBlocks;  
  
    offsetA = mCoreIdx * tiling.Ka * tiling.singleCoreM;  
    if (isTransA) {  
        offsetA = mCoreIdx * tiling.singleCoreM;  
    }  
    offsetB = nCoreIdx * tiling.singleCoreN;  
    if (isTransB) {  
        offsetB = nCoreIdx * tiling.Kb * tiling.singleCoreN;  
    }  
    offsetC = mCoreIdx * tiling.N * tiling.singleCoreM + nCoreIdx * tiling.singleCoreN;  
  
    // 尾核对应的M/N计算，此处为正确的计算方式  
    tailM = tiling.M - mCoreIdx * tiling.singleCoreM;  
    tailM = tailM < tiling.singleCoreM ? tailM : tiling.singleCoreM;  
  
    tailN = tiling.N - nCoreIdx * tiling.singleCoreN;  
    tailN = tailN < tiling.singleCoreN ? tailN : tiling.singleCoreN;  
}  
  
extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR c,  
GM_ADDR workspace,
```

```
GM_ADDR tilingGm)
{
    using A_T = half;
    using B_T = half;
    using C_T = float;

    AscendC::TPipe pipe;
    TCubeTiling tiling;
    CopyTiling(&tiling, tilingGm);

    AscendC::GlobalTensor<A_T> aGlobal;
    AscendC::GlobalTensor<B_T> bGlobal;
    AscendC::GlobalTensor<C_T> cGlobal;
    aGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ A_T*>(a), tiling.M * tiling.Ka);
    bGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ B_T*>(b), tiling.Ka * tiling.N);
    cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ C_T*>(c), tiling.M * tiling.N);

    int offsetA = 0;
    int offsetB = 0;
    int offsetC = 0;
    bool isTransA = false;
    bool isTransB = false;

    int tailM = 0;
    int tailN = 0;
    CalcGMOffset(GetBlockIdx(), tiling, offsetA, offsetB, offsetC, tailM, tailN, isTransA, isTransB);

    auto gmA = aGlobal[offsetA];
    auto gmB = bGlobal[offsetB];
    auto gmC = cGlobal[offsetC];

    AscendC::Matmul<AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, A_T>,
        AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, B_T>,
        AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, C_T>> mm;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling);

    mm.SetTensorA(gmA, isTransA);
    mm.SetTensorB(gmB, isTransB);
    // mm.SetTail(tailM, tailN); 尾核设置接口，若此处未更新尾块会导致单核精度正确，多核失败
    mm.IterateAll(gmC);
    mm.End();
}
```

步骤5 排查Matmul API的使用是否正确

经过上述步骤，可定界出是否为Matmul API使用问题。如果由于Matmul API使用错误导致了算子的精度问题，需要根据Matmul各接口的使用说明、约束条件等，检查接口的使用是否正确。

- 案例1：未遵循接口约束条件

在Matmul MDL模板下，调用IterateBatch接口，导致算子执行失败。这是由于不满足该接口的约束条件，IterateBatch接口仅支持Norm模板。

此类问题，应仔细阅读Matmul各接口中的约束条件，并排查算子实现使用的相关接口，是否满足对应接口的约束条件。

- 案例2：未遵循模板约束条件

在使能doMTE2Preload预加载模板时，若K方向非全载，不满足模板约束条件，则会导致精度比对失败。

除了满足函数接口约束条件外，也需要满足模板参数相应的约束条件，排查模板参数的使用。

步骤6 用于算子调测的golden脚本是否正确

算子的golden生成脚本，根据自定义算子的功能逻辑自行实现，用于比对算子执行结果是否正确。因此，该golden脚本的逻辑需要与算子的实现逻辑保持一致，如果

golden脚本实现错误，会导致算子计算结果的精度比对失败，这种情况是golden数据不可信。

所以，在算子精度定界定位的过程中，用户需要自行根据自定义算子的逻辑，检查golden脚本的正确性，尤其是对于复杂计算逻辑的算子，需重点排查该项。

----结束

2.10.10.7 算子工程编译时出现文件名过长报错

现象描述

工程化算子开发场景，在进行[算子工程编译](#)时，提示以下报错信息中的一种：

- file name is too long (cannot be split); not dumped
ERROR: failed to create temporary archive: /tmp/mkself336430.tar
CMake Error at /addcustom/cmake/makeself.cmake:12 (message):
CPack Command error: 1

Header is 672 lines long

tar:
./packages/vendors/customize/op_proto/inc/
my_very_long_and_detailed_c_plus_plus_project_source_code_file_containing_advanced_features_and_implementations.h:
file name is too long (cannot be split); not dumped

tar: Exiting with failure status due to previous errors
- file name is too long (max 256); not dumped
ERROR: failed to create temporary archive: /tmp/mkself133003.tar
CMake Error at /root/r00882787/Customproj/addcustom/cmake/makeself.cmake:12 (message):
CPack Command error: 1

Header is 672 lines long

tar:
./packages/vendors/
ProjectX_FeatureY_Z_20241118_Monday_Development_BranchA1_Commit56789_AlphaVersion_ABCDEFGHIJKLM/op_impl/ai_core/tbe/
ProjectX_FeatureY_Z_20241118_Monday_Development_BranchA1_Commit56789_AlphaVersion_ABCDEFGHIJKLM_impl/dynamic/add_custom.cpp:
file name is too long (max 256); not dumped

问题根因

在构建过程中，由于文件名或路径长度超出系统限制，使用tar命令打包算子工程生成的文件时发生了错误。

定位步骤

出现此类报错，需要根据提示的报错信息（通常包含超长的文件名或者路径），减少对应的文件名长度或路径长度。

下面列出了常见错误的解决方案：

- 文件名过长报错
 - 位于算子工程op_kernel目录下的kernel侧代码和位于op_host目录下的host侧代码等文件，文件名是根据创建算子工程时传入的算子OpType自动生成的。如果因为此类文件名过长报错，应减少OpType的长度。

- 使用Comment接口设置算子分组名称后，会对应生成同名的供GE调用的原型定义代码文件。如果因为此类文件导致文件名过长报错，应减少算子分组名称的长度。
- 文件路径过长报错
完成工程编译相关配置时，如果在CMakePresets.json文件中配置vendor_name，编译时会在vendor目录下生成以vendor_name为名称的路径。如果因为此类文件路径过长报错，应减少配置的vendor_name长度。

2.10.10.8 调用算子时出现无法打开 config.ini 的报错

现象描述

自定义算子包安装部署后，在调用已部署的算子时，出现如下json文件获取失败的报错信息：

```
[INFO] Start get path and read binary_info_config.json.  
[WARNING] Get jsonfile path for */binary_info_config.json failed, errmsg:No such file or directory.  
[ERROR] Get path and read binary_info_config.json failed, please check if the opp_kernel package is installed!
```

通过查询前文的报错信息，上述json文件获取失败的原因是前置流程中无法打开config.ini，提示信息如下：

```
[INFO] Start to get opp kernel base path, default custom opp kernel is in ASCEND_OPP_PATH.  
[INFO] The real path of config.ini is */opp/vendors/config.ini.  
[WARNING] Can not open file: */opp/vendors/config.ini.
```

问题根因

根因在于当前调用算子的用户缺少对算子包部署目录下的config.ini（*/opp/vendors/config.ini）文件的读权限。config.ini文件权限默认为640，仅允许部署用户和同属组用户访问，当前执行用户与安装用户非同一属组，缺少读权限，导致算子调用失败。

比如下述场景即会导致调用算子时出现执行报错：root用户安装部署自定义算子包，HwHiAiUser属组用户调用已部署的自定义算子。

处理步骤

联系自定义算子包安装用户修改config.ini权限为644：
chmod 644 config.ini

2.10.10.9 算子包部署时出现权限不足报错

现象描述

部署自定义算子包时，出现如下报错信息：

```
[WARNING] The directory /usr/local/Ascend/cann/opp does not have sufficient permissions. Please check and modify the folder permissions (e.g., using chmod), or use the --install-path option to specify an installation path and change the environment variable ASCEND_CUSTOM_OPP_PATH to the specified path.  
...  
[ERROR] create /usr/local/Ascend/cann/opp/vendors/customize/framework failed
```

问题根因

当前操作用户缺少对部署路径下vendors目录的写权限。

自定义算子包默认安装路径\${INSTALL_DIR}/opp/vendors的目录权限与CANN软件包安装用户和安装配置有关：root用户安装CANN，\${INSTALL_DIR}/opp/vendors权限为755；非root用户携带--install for all参数安装CANN，该目录权限为755，非root用户不带--install for all参数安装CANN时，该目录权限为750。

例如，root用户安装CANN软件包后，HwHiAiUser属组用户在对应目录部署自定义算子包，因为其他用户没有写权限，会出现上述报错信息，提示权限不足导致自定义算子包部署失败。

处理步骤

- 方法一：使用--install-path参数并配置环境变量ASCEND_CUSTOM_OPP_PATH来指定安装目录（参考[指定目录安装](#)）。运行用户需要对指定的安装路径有可读写权限。

```
./custom_opp_<target os>_<target architecture>.run --install-path=<path>  
source <path>/vendors/<vendor_name>/bin/set_env.bash
```
- 方法二：联系CANN软件包安装用户修改默认安装路径下的vendors目录权限，比如修改为777：

```
chmod 777 /usr/local/Ascend/cann/opp/vendors/
```

3 算子实践参考

- 3.1 本文档组织结构
- 3.2 异构计算
- 3.3 SIMD算子实现
- 3.4 功能调试
- 3.5 性能分析
- 3.6 SIMD算子性能优化
- 3.7 优秀实践

3.1 本文档组织结构

需要具备的知识

本文档主要用于指导开发者基于Ascend C编程语言在昇腾AI处理器上开发高性能的算子。在阅读此文档时，需要开发者具备如下能力：

- 熟练使用C++编程语言。
- 了解计算机架构。
- 理解昇腾AI处理器硬件架构。
- 完成Ascend C编程相关文档、课程学习。
- 可以完成Ascend C开发、调试环境搭建。
- 可以独立完成Ascend C算子的开发。
- 熟练使用性能分析工具获取性能数据。

您可以通过[LINK](#)获取如上内容的学习资料。

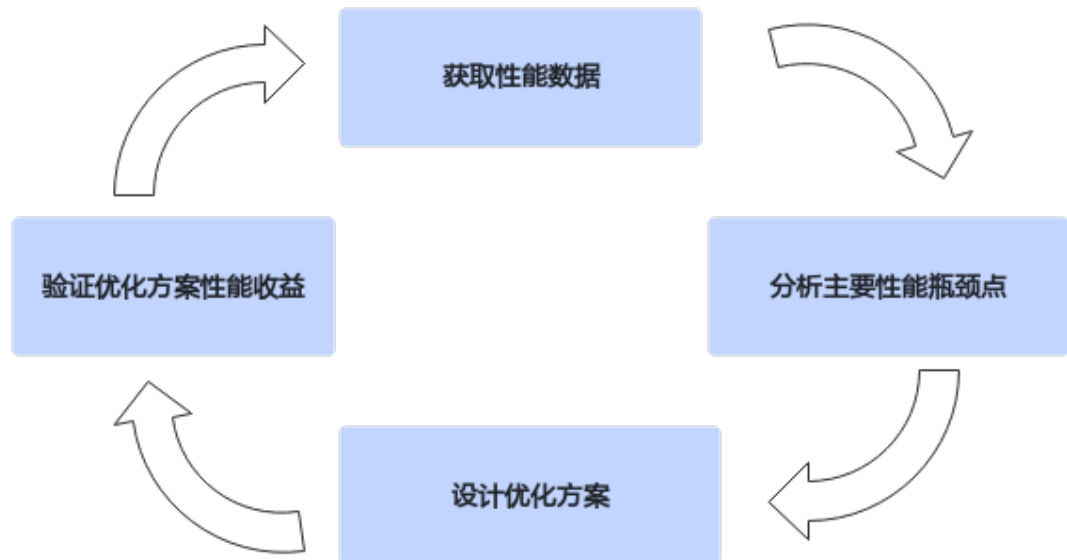
可以解决的问题

开发者完成Ascend C算子（本文后续提到的算子均指使用Ascend C开发的算子）的开发后，如果需要对算子性能进行进一步优化，那么通过阅读本文档可以得到有效帮助。

本文档首先介绍异构计算的硬件特点及运行的数据交互，然后介绍使用Ascend C编程时调试调优的思路以及各种性能优化手段，最后介绍具体的性能优化案例。

优化算子性能是一个持续迭代的流程，将如下步骤不断循环迭代，直至达成性能目标。在[优秀实践](#)章节，开发者可以进一步了解基于以下四个步骤的具体实践。

图 3-1 算子性能优化流程



本文档分为五个章节，各章的内容以及目标如下：

- **异构计算**：介绍算子在硬件上的部署以及运行的数据流。目的是让开发者在宏观上了解硬件架构上可能影响算子执行性能的流程。
- **SIMD算子实现**：介绍矢量编程、矩阵编程、融合算子编程三种典型场景下的算子 Tiling、Kernel实现，是对Ascend C编程范式的具体应用。
- **功能调试**：介绍部分比较常见的影响算子功能的场景。目的是让开发者可以快速解决功能问题，便于进行性能优化，以及快速解决在性能优化过程中可能出现的功能问题。
- **性能分析**：介绍了算子性能数据分析的方向。目的是让开发者能够通过分析性能数据，识别性能的优化方向。算子性能数据的测试方法，以及性能工具使用方式的详细说明请参考《[性能调优工具用户指南](#)》，本章不做赘述。
- **SIMD算子性能优化**：介绍性能优化的手段。目的是让开发者结合算子性能瓶颈点，开展性能优化工作。将主要的优化建议分成搬运优化、内存优化、API使用优化、流水优化以及Tiling优化。有一些优化建议是对上述分类的综合体现，则写在了相关性比较大的章节。这些建议按优先级进行分类，优先级是综合考虑性能效果及其范围来设定：为大多数Ascend C算子带来性能收益的建议具有最高优先级，而仅影响特定情况的手段被给予较低优先级。开发者不必熟悉所有优化手段，可以根据分析得到的算子性能瓶颈，获取对应的优化手段，逐渐了解优化策略全貌。
- **优秀实践**：介绍算子性能优化的优秀实践案例。目的是让开发者结合实例更深入的理解上文内容，同时参考其中的优化手段和优化思路，完成算子的性能优化，实现从理论到实践的过渡。

3.2 异构计算

使用Ascend C进行编程时，会涉及到在两个不同的平台（Host、Device）上开发代码。本章简单介绍Host、Device之间的差异，便于开发者宏观地了解这个异构系统；同时给出算子相关的数据流，结合异构架构的特点，开发者可以进一步了解应该如何合理安排算子代码的执行位置，便于得到更好的性能。

Host 侧 CPU 和 Device 侧 NPU 的主要区别

- 不同的硬件资源
CPU是为了执行通用计算任务而设计的，但在处理大量的并行计算（如矩阵乘、批数据处理）时效率不高。NPU是为了加速机器学习和深度学习任务而设计的，它擅长执行大量的并行计算。NPU包含了大量的专用硬件，比如：支持矩阵计算的Cube单元，NPU中一个核可以支持一个时钟周期内完成数据量为 $16 * 16 * 16$ 、数据类型为float16的乘累加运算；支持向量计算的Vector单元，NPU中一个核可以支持一个时钟周期内完成数据量为 $128 + 128$ 、数据类型为float16的加法运算。
- 不同的物理内存空间
Host和Device的物理内存是分离的，通常需要在Host侧内存和Device侧内存之间进行数据交换。

如何合理安排算子代码

开发者进行Ascend C算子开发时，可以将Host和Device视为一个协同的异构系统，为每个处理单元分配其擅长的工作。在Host侧推荐执行非计算密集型任务，一般为标量计算任务。在Device侧推荐进行计算密集型任务，利用Device侧NPU的SIMD(Single Instruction Multiple Data)指令可以高效的实现批量数据的矩阵运算、向量运算等。

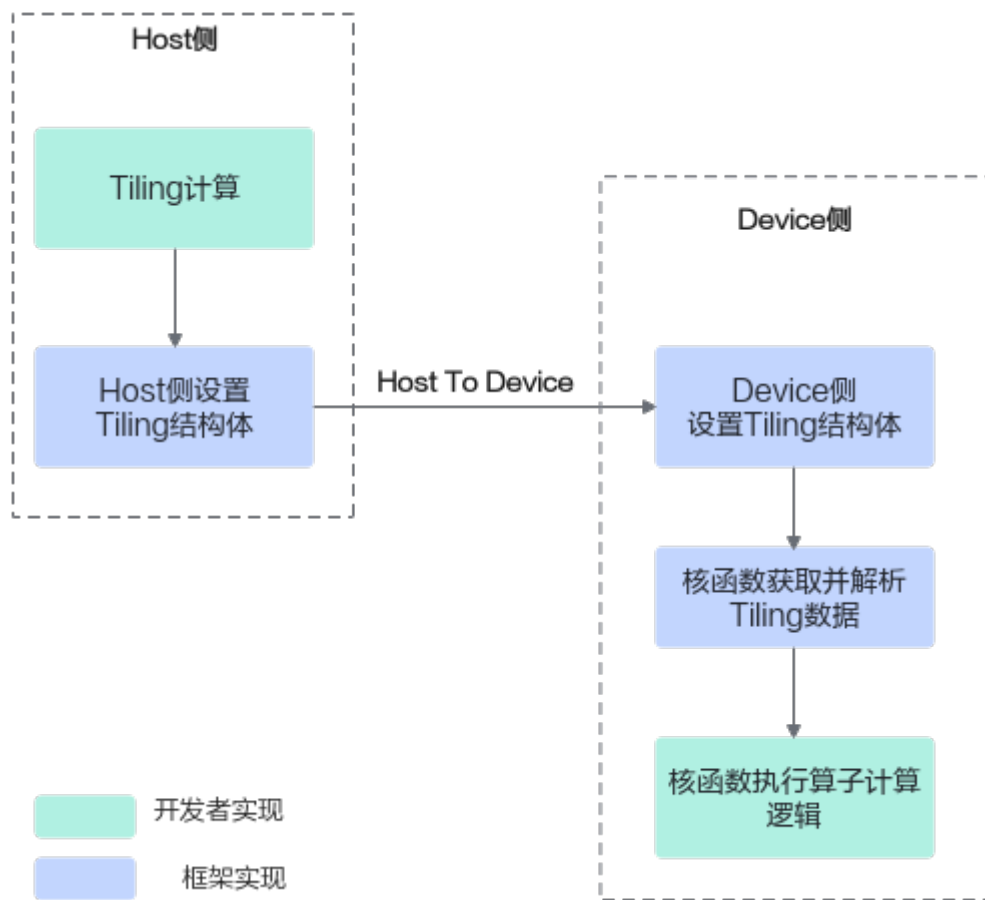
Ascend C算子的实现主要包含两个部分：

- Host侧Tiling实现
由于NPU中AI Core内部存储无法完全容纳算子输入输出的所有数据，需要每次搬运一部分输入数据进行计算然后搬出，再搬运下一部分输入数据进行计算，这个过程就称之为Tiling。切分数据的算法称为Tiling算法或者Tiling策略。根据算子的shape等信息来确定数据切分算法相关参数（比如每次搬运的块大小，循环的总次数）的计算程序，称之为Tiling实现，也叫Tiling函数（Tiling Function）。由于Tiling实现中均为标量计算，AI Core并不擅长，所以我们将其独立出来放在Host侧CPU上执行。
- Device侧Kernel实现
Kernel实现即算子核函数实现，在Kernel函数内部通过解析Host侧传入的Tiling结构体获取Tiling信息，根据Tiling信息控制数据搬入搬出Local Memory的流程；通过调用计算、数据搬运、内存管理、任务同步等API实现算子逻辑。其核心逻辑基本上都为计算密集型任务，适合在Device侧NPU上执行。

算子数据流

算子执行过程中涉及到Host和Device的数据交换。这里仅针对Tiling参数的传递，给出具体的数据流：Host侧Tiling算法根据算子具体输入输出的信息，完成Tiling参数的计算，并存放在Tiling结构体中；将Host侧的Tiling结构体发送到Device侧，Device侧的算子获取并解析Tiling结构体，基于该信息执行后续的算子计算逻辑。

图 3-2 算子 Tiling 传递数据流



3.3 SIMD 算子实现

3.3.1 概述

Ascend C的算子实现主要包含两个部分：

- Host侧Tiling实现

由于NPU中AI Core内部存储无法完全容纳算子输入输出的所有数据，需要每次搬运一部分输入数据进行计算然后搬出，再搬运下一部分输入数据进行计算，这个过程就称之为Tiling。切分数据的算法称为Tiling算法或者Tiling策略。根据算子的shape等信息来确定数据切分算法相关参数（比如每次搬运的块大小，以及总共循环多少次）的计算程序，称之为Tiling实现，也叫Tiling函数（Tiling Function）。由于Tiling实现中完成的均为标量计算，AI Core并不擅长，所以我们将其独立出来放在Host侧CPU上执行。

- Device侧Kernel实现

Kernel实现即算子核函数实现，在Kernel函数内部通过解析Host侧传入的Tiling结构体获取Tiling信息，根据Tiling信息控制数据搬入搬出Local Memory的流程；通过调用计算、数据搬运、内存管理、任务同步API，实现算子逻辑。其核心逻辑基本上都为计算密集型任务，需要在NPU上执行。

本章介绍矢量编程、矩阵编程、融合算子编程三种典型场景下的算子Tiling、Kernel实现，是对上文中三种典型编程范式的具体应用，同时也介绍了编程的更多细节、API的使用方法等。

3.3.2 矢量编程

3.3.2.1 概述

本节将以Add算子为例，带您快速构建Ascend C矢量算子程序，并学习矢量算子开发的典型场景以及处理方式。涉及的场景包括：

- **基础矢量算子**：开发一个简单的Add矢量算子。
- **TBuf的使用**：在算子计算过程中使用临时空间存储运算的中间结果。
- **多核Tiling**：算子在AI处理器的多个核上运行，所有核的计算数据量相等且32字节对齐。
- **尾块Tiling**：算子在AI处理器的多个核上运行，所有核的计算数据量相等，每个核上除最后一个数据块（尾块）外，其余数据块的数据量相等，每个核都需要处理尾块数据的计算。
- **尾核Tiling**：算子在AI处理器的多个核上运行，数据无法平均分配到每个核。将所有核分为多个整核和多个尾核，整核的计算数据量相等，尾核的计算数据量相等。
- **尾核&尾块**：算子在AI处理器的多个核上运行，数据无法平均分配到每个核，同时每个核内的数据无法均分，除最后一个数据块（尾块）外，其余数据块的数据量相等，每个核都需要单独处理尾块数据的计算。
- **DoubleBuffer场景**：使能double buffer，算子中的多条流水并行执行。
- **Broadcast场景**：算子中两个输入的shape（形状）不相等，需要将一个输入的shape进行Broadcast（广播）后，再执行计算。
- **非对齐场景**：更多数据非32字节对齐场景的处理方案。

须知

进行数据搬运和Vector计算时，对于搬运的数据长度和操作数的起始地址有如下的对齐要求：

- 使用DataCopy接口进行数据搬运，搬运的数据长度和操作数的起始地址（Unified Buffer上）必须保证32字节对齐。
- 通常情况下，进行Vector计算时，操作数的起始地址必须保证32字节对齐，执行计算的基本单位为32字节。

3.3.2.2 基础矢量算子

基于Ascend C方式实现基础矢量算子核函数的流程如下图所示。

图 3-3 矢量算子核函数实现流程



- 算子分析：分析算子的数学表达式、输入、输出以及计算逻辑的实现，明确需要调用的Ascend C接口。
- 核函数定义：定义Ascend C算子入口函数。
- 根据[矢量编程范式](#)实现算子类：完成核函数的内部实现，包括3个基本任务：CopyIn, Compute, CopyOut。

下文以输入为half数据类型且shape的最后一维为32Bytes对齐、在单核上运行的、一次完成计算的Add算子为例，对上述步骤进行详细介绍。本样例中介绍的算子完整代码请参见[基础Add算子样例](#)。

算子分析

算子分析具体步骤如下：

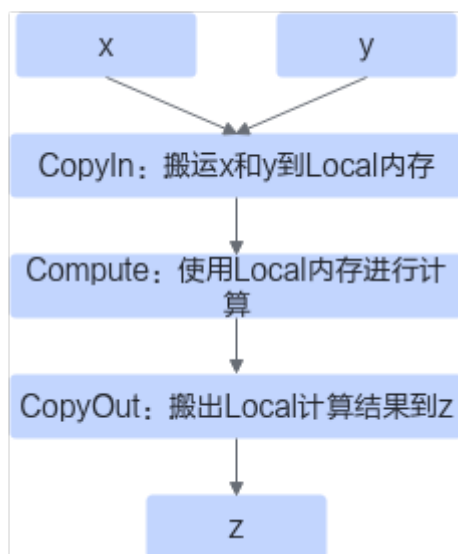
步骤1 明确算子的数学表达式及计算逻辑。

Add算子的数学表达式为：

$$z = x + y$$

计算逻辑是：Ascend C提供的矢量计算接口的操作元素都为LocalTensor，输入数据需先从外部存储（Global Memory）搬运进片上存储（Unified Buffer），然后使用计算接口完成两个输入参数相加，得到最终结果，再搬出到外部存储上。Ascend C Add算子的计算逻辑如下图所示。

图 3-4 算子计算逻辑

**步骤2** 明确输入和输出。

- Add算子有两个输入：x与y；输出为z。
- 本样例中算子的输入支持的数据类型为half（float16），算子输出的数据类型与输入的数据类型相同。
- 算子输入支持的shape为（1，2048），输出shape与输入shape相同。
- 算子输入支持的**format**为：ND。

步骤3 确定核函数名称和参数。

- 您可以自定义核函数名称，本样例中核函数命名为add_custom。
- 根据对算子输入输出的分析，确定核函数有3个参数x，y，z；x，y为输入在Global Memory上的内存地址，z为输出在Global Memory上的内存地址。

步骤4 确定算子实现所需接口。

- 实现涉及外部存储和内部存储间的数据搬运，查看Ascend C API参考中的数据搬运接口，需要使用DataCopy来实现数据搬运。
- 本样例只涉及矢量计算的加法操作，查看Ascend C API参考中的矢量计算接口，初步分析可使用基础算术Add接口Add实现x+y。
- 使用Queue队列管理计算中使用的Tensor数据结构，具体使用EnQue、DeQue等接口。

----结束

通过以上分析，得到Ascend C Add算子的设计规格如下：

- 算子类型（OpType）：Add
- 算子输入输出：

表 3-1 Add 算子输入输出规格

name	shape	data type	format
x（输入）	(1, 2048)	half	ND

name	shape	data type	format
y (输入)	(1, 2048)	half	ND
z (输出)	(1, 2048)	half	ND

- 核函数名称：add_custom
- 使用的主要接口：
 - DataCopy：数据搬移接口
 - Add：矢量基础算术接口
 - EnQue、DeQue等接口：Queue队列管理接口
- 算子实现文件名称：add_custom.cpp

核函数定义

根据[核函数](#)中介绍的规则进行核函数的定义。

步骤1 函数原型定义

本样例中，函数名为add_custom（核函数名称可自定义），根据[算子分析](#)中对算子输入输出的分析，确定有3个参数x, y, z，其中x, y为输入内存，z为输出内存。根据[核函数](#)的规则介绍，函数原型定义如下所示：使用__global__函数类型限定符标识它是一个核函数，可以被<<<>>>调用；使用__aicore__函数类型限定符标识该核函数在设备端aicore上执行；为方便起见，统一使用GM_ADDR宏修饰入参，GM_ADDR宏定义请参考[核函数](#)。

```
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z)
{
}
```

步骤2 调用算子类的Init和Process函数。

算子类的Init函数，完成内存初始化相关工作，Process函数完成算子实现的核心逻辑，具体介绍参见[算子类实现](#)。

```
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z)
{
    KernelAdd op;
    op.Init(x, y, z);
    op.Process();
}
```

步骤3 对核函数的调用进行封装，得到add_custom_do函数，便于主程序调用。#ifndef ASCENDC_CPU_DEBUG表示该封装函数仅在编译运行NPU侧的算子时会用到，编译运行CPU侧的算子时，可以直接调用add_custom函数。根据[核函数定义和调用](#)章节，调用核函数时，除了需要传入参数x, y, z，还需要传入numBlocks（核函数执行的核数），l2ctrl（保留参数，设置为nullptr），stream（应用程序中维护异步操作执行顺序的stream）来规定核函数的执行配置。

```
#ifndef ASCENDC_CPU_DEBUG
// call of kernel function
void add_custom_do(uint32_t numBlocks, void* l2ctrl, void* stream, uint8_t* x, uint8_t* y, uint8_t* z)
{
    add_custom<<<numBlocks, l2ctrl, stream>>>(x, y, z);
}
#endif
```

----结束

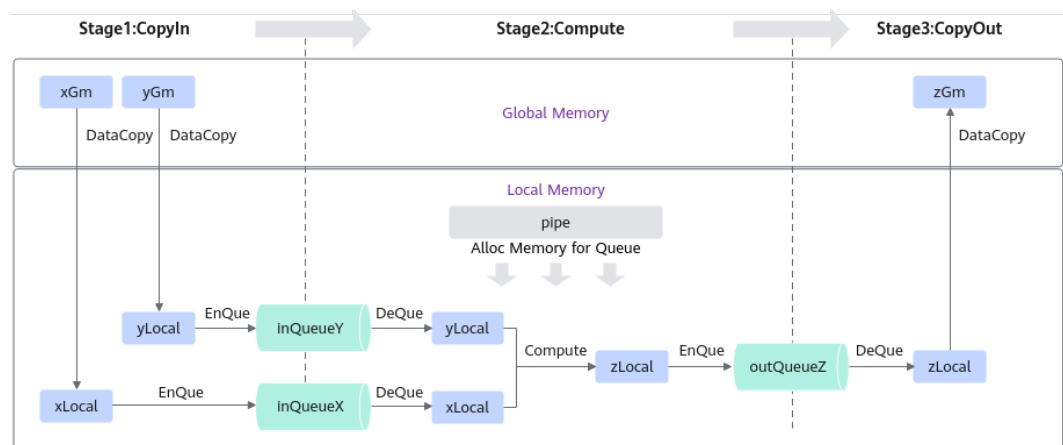
算子类实现

根据上一节介绍，核函数中会调用算子类的Init和Process函数，本节具体讲解如何基于编程范式实现算子类。

根据矢量编程范式对Add算子的实现流程进行设计的思路如下，矢量编程范式请参考[矢量编程范式](#)，设计完成后得到的Add算子实现流程图参见[图3 Add算子实现流程](#)：

- Add算子的实现流程分为3个基本任务：CopyIn，Compute，CopyOut。CopyIn任务负责将Global Memory上的输入Tensor xGm和yGm搬运至Local Memory，分别存储在xLocal，yLocal，Compute任务负责对xLocal，yLocal执行加法操作，计算结果存储在zLocal中，CopyOut任务负责将输出数据从zLocal搬运至Global Memory上的输出Tensor zGm中。
- CopyIn，Compute任务间通过VECIN队列inQueueX，inQueueY进行同步，Compute，CopyOut任务间通过VECOUT队列outQueueZ进行同步。
- 任务间交互使用到的内存、临时变量的内存统一使用Pipe内存管理对象进行管理。

图 3-5 Add 算子实现流程



算子类中主要实现上述流程，包含对外开放的初始化Init函数和核心处理函数Process，Process函数中会对上图中的三个基本任务进行调用；同时包括一些算子实现中会用到的私有成员，比如上图中的GlobalTensor（xGm、yGm、zGm）和VECIN、VECOUT队列等。KernelAdd算子类具体成员如下：

```
class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    // 初始化函数，完成内存初始化相关操作
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z){}
    // 核心处理函数，实现算子逻辑，调用私有成员函数CopyIn、Compute、CopyOut完成矢量算子的三级流水操作
    __aicore__ inline void Process(){}

private:
    // 搬入函数，完成CopyIn阶段的处理，被核心Process函数调用
    __aicore__ inline void CopyIn(){}
    // 计算函数，完成Compute阶段的处理，被核心Process函数调用
    __aicore__ inline void Compute(){}
    // 搬出函数，完成CopyOut阶段的处理，被核心Process函数调用
    __aicore__ inline void CopyOut(){}

private:
```

```
AscendC::TPipe pipe; // Pipe内存管理对象
AscendC::TQue<AscendC::TPosition::VECIN, 1> inQueueX; // 输入数据Queue队列管理对象, TPosition为
VECIN
AscendC::TQue<AscendC::TPosition::VECIN, 1> inQueueY; // 输入数据Queue队列管理对象, TPosition为
VECIN
AscendC::TQue<AscendC::TPosition::VEECOUT, 1> outQueueZ; // 输出数据Queue队列管理对象, TPosition为
VEECOUT
AscendC::GlobalTensor<half> xGm; // 管理输入输出Global Memory内存地址的对象, 其中xGm, yGm为输入,
zGm为输出
AscendC::GlobalTensor<half> yGm;
AscendC::GlobalTensor<half> zGm;
};
```

初始化函数主要完成以下内容:

- 设置输入输出Global Tensor的Global Memory内存地址。
本样例中的分配方案是: 数据整体长度TOTAL_LENGTH为1 * 2048, 使用GlobalTensor类的SetGlobalBuffer接口设定该核上Global Memory的起始地址以及长度。

```
xGm.SetGlobalBuffer((__gm__ half *)x, TOTAL_LENGTH);
```

- 通过Pipe内存管理对象为输入输出Queue分配内存。
比如, 为输入x的Queue分配内存, 可以通过如下代码段实现:

```
pipe.InitBuffer(inQueueX, 1, TOTAL_LENGTH * sizeof(half))
```

具体的初始化函数代码如下:

```
constexpr int32_t TOTAL_LENGTH = 1 * 2048; // 数据总长
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z)
{
    // 设置Global Memory的起始地址以及长度
    xGm.SetGlobalBuffer((__gm__ half *)x, TOTAL_LENGTH);
    yGm.SetGlobalBuffer((__gm__ half *)y, TOTAL_LENGTH);
    zGm.SetGlobalBuffer((__gm__ half *)z, TOTAL_LENGTH);

    // 通过Pipe内存管理对象为输入输出Queue分配内存
    pipe.InitBuffer(inQueueX, 1, TOTAL_LENGTH * sizeof(half));
    pipe.InitBuffer(inQueueY, 1, TOTAL_LENGTH * sizeof(half));
    pipe.InitBuffer(outQueueZ, 1, TOTAL_LENGTH * sizeof(half));
}
```

基于矢量编程范式, 将核函数的实现分为3个基本任务: CopyIn, Compute, CopyOut。Process函数中通过如下方式调用这三个函数。

```
__aicore__ inline void Process()
{
    CopyIn();
    Compute();
    CopyOut();
}
```

根据编程范式上面的算法分析, 将整个计算拆分成三个Stage, 用户单独编写每个Stage的代码, 三阶段流程示意图参见图3-5, 具体流程如下:

步骤1 Stage1: CopyIn函数实现。

1. 使用DataCopy接口将GlobalTensor数据拷贝到LocalTensor。
2. 使用EnQue将LocalTensor放入VECIN的Queue中。

```
__aicore__ inline void CopyIn()
{
    // 从Que中为LocalTensor分配内存
    AscendC::LocalTensor<half> xLocal = inQueueX.AllocTensor<half>();
    AscendC::LocalTensor<half> yLocal = inQueueY.AllocTensor<half>();
    // 将GlobalTensor数据拷贝到LocalTensor
    AscendC::DataCopy(xLocal, xGm, TOTAL_LENGTH);
}
```



```
AscendC::DataCopy(yLocal, yGm, TOTAL_LENGTH);  
// LocalTensor放入VECIN的Queue中  
inQueueX.Enqueue(xLocal);  
inQueueY.Enqueue(yLocal);  
}
```

步骤2 Stage2: Compute函数实现。

1. 使用DeQue从VECIN中取出LocalTensor。
2. 使用Ascend C接口Add完成矢量计算。
3. 使用EnQue将计算结果LocalTensor放入到VECOUT的Queue中。
4. 使用FreeTensor释放不再使用的LocalTensor。

```
__aicore__ inline void Compute()  
{  
    // 将Input从VECIN的Queue中取出  
    AscendC::LocalTensor<half> xLocal = inQueueX.DeQue<half>();  
    AscendC::LocalTensor<half> yLocal = inQueueY.DeQue<half>();  
    AscendC::LocalTensor<half> zLocal = outQueueZ.AllocTensor<half>();  
    // 调用Add算子进行计算  
    AscendC::Add(zLocal, xLocal, yLocal, TOTAL_LENGTH);  
    // 将计算结果LocalTensor放入到VECOUT的Queue中  
    outQueueZ.Enqueue<half>(zLocal);  
    // 释放LocalTensor  
    inQueueX.FreeTensor(xLocal);  
    inQueueY.FreeTensor(yLocal);  
}
```

步骤3 Stage3: CopyOut函数实现。

1. 使用DeQue接口从VECOUT的Queue中取出LocalTensor。
2. 使用DataCopy接口将LocalTensor拷贝到GlobalTensor上。
3. 使用FreeTensor将不再使用的LocalTensor进行回收。

```
__aicore__ inline void CopyOut()  
{  
    // 将计算结果从VECOUT的Queue中取出  
    AscendC::LocalTensor<half> zLocal = outQueueZ.DeQue<half>();  
    // 将计算结果从LocalTensor数据拷贝到GlobalTensor  
    AscendC::DataCopy(zGm, zLocal, TOTAL_LENGTH);  
    // 释放LocalTensor  
    outQueueZ.FreeTensor(zLocal);  
}
```

----结束

3.3.2.3 TBuf 的使用

在大多数算子开发时，核函数计算过程需要使用临时内存来存储运算的中间结果，这些中间结果以临时变量表示，临时变量占用的内存可以使用TBuf数据结构来管理，具体介绍请参考TBuf。下文将以输入的数据类型为bfloat16_t、在单核上运行的Add算子为例，介绍TBuf的使用方式。本样例中介绍的算子完整代码请参见[使用临时内存的Add算子样例](#)。

在Atlas A2 训练系列产品/Atlas 800I A2 推理产品上，Add接口不支持对数据类型bfloat16_t的源操作数进行求和计算。因此，需要先将算子输入的数据类型转换成Add接口支持的数据类型，再进行计算。为保证计算精度，调用Cast接口将输入bfloat16_t类型转换为float类型，再进行Add计算，并在计算结束后将float类型转换回bfloat16_t类型。

通过以上分析，得到Ascend C Add算子的设计规格如下：

- 算子类型（OpType）：Add
- 算子输入输出：

表 3-2 Add 算子输入输出规格

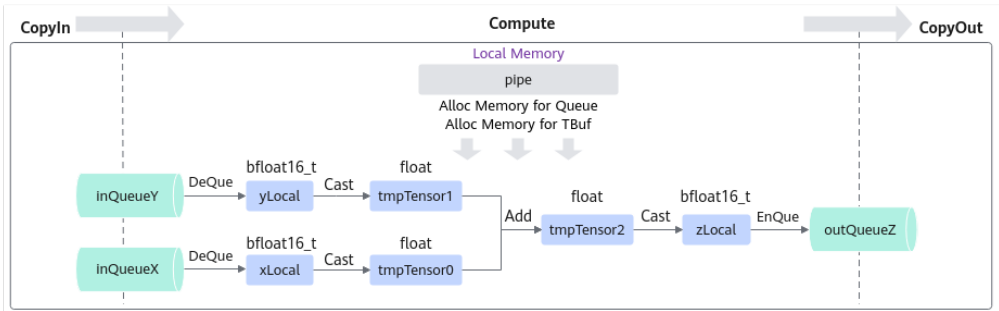
name	shape	data type	format
x（输入）	(1, 2048)	bfloat16_t	ND
y（输入）	(1, 2048)	bfloat16_t	ND
z（输出）	(1, 2048)	bfloat16_t	ND

- 核函数名称：add_custom
- 使用的主要接口：
 - DataCopy：数据搬移接口
 - Cast：矢量精度转换接口
 - Add：矢量基础算术接口
 - EnQue、DeQue等接口：Queue队列管理接口
- 算子实现文件名称：add_custom.cpp

算子类实现

该样例的CopyIn，CopyOut任务与基础矢量算子相同，Compute任务的具体流程如下图所示。

图 3-6 输入为 bfloat16_t 类型的 Add 计算流程



在Compute任务中，表示Cast转换结果、Add计算结果的临时变量均需要使用临时内存存储。与基础矢量算子实现的KernelAdd算子类相比，本样例新增三个TBuf类型的成员变量tmpBuf0、tmpBuf1、tmpBuf2，用于管理计算过程中使用的临时内存，代码如下。

```
class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z){}
    __aicore__ inline void Process(){}
private:
    __aicore__ inline void CopyIn(){}
    __aicore__ inline void Compute(){}
    __aicore__ inline void CopyOut(){}
private:
    AscendC::TPipe pipe;
```

```
AscendC::TQue<AscendC::TPosition::VECIN, 1> inQueueX, inQueueY;  
AscendC::TQue<AscendC::TPosition::VECOUT, 1> outQueueZ;  
AscendC::TBuf<AscendC::TPosition::VECCALC> tmpBuf0, tmpBuf1, tmpBuf2;  
AscendC::GlobalTensor<bfloat16_t> xGm;  
AscendC::GlobalTensor<bfloat16_t> yGm;  
AscendC::GlobalTensor<bfloat16_t> zGm;  
};
```

初始化函数阶段除原有步骤外，需要调用InitBuffer接口为TBuf变量分配内存，具体的初始化函数代码如下：

```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z)  
{  
    xGm.SetGlobalBuffer((__gm__ half *)x, TOTAL_LENGTH);  
    yGm.SetGlobalBuffer((__gm__ half *)y, TOTAL_LENGTH);  
    zGm.SetGlobalBuffer((__gm__ half *)z, TOTAL_LENGTH);  
  
    pipe.InitBuffer(inQueueX, 1, TOTAL_LENGTH * sizeof(bfloat16_t));  
    pipe.InitBuffer(inQueueY, 1, TOTAL_LENGTH * sizeof(bfloat16_t));  
    pipe.InitBuffer(outQueueZ, 1, TOTAL_LENGTH * sizeof(bfloat16_t));  
  
    pipe.InitBuffer(tmpBuf0, TOTAL_LENGTH * sizeof(float));  
    pipe.InitBuffer(tmpBuf1, TOTAL_LENGTH * sizeof(float));  
    pipe.InitBuffer(tmpBuf2, TOTAL_LENGTH * sizeof(float));  
}
```

基于矢量编程范式，核函数需要实现3个基本任务：CopyIn，Compute，CopyOut。与[基础矢量算子实现](#)相同，Process函数按顺序调用CopyIn函数，Compute函数，CopyOut函数。其中，CopyIn函数，CopyOut函数与[基础矢量算子的CopyIn函数](#)、[基础矢量算子的CopyOut函数](#)的实现没有差异，此处不过多赘述。Compute函数的实现步骤如下：

1. 使用DeQue从VECIN的Queue中取出LocalTensor。
2. 使用TBuf.Get从TBuf上获取全部长度的Tensor作为临时内存。
3. 使用Cast接口将LocalTensor转换为float类型，并存入临时内存。
4. 使用Add接口完成矢量计算，将计算结果存入临时内存。
5. 使用Cast接口将临时内存中的计算结果转换为bfloat16_t类型。
6. 使用EnQue将bfloat16_t类型的结果LocalTensor放入VECOUT的Queue中。
7. 使用FreeTensor释放不再使用的LocalTensor。

```
__aicore__ inline void Compute()  
{  
    AscendC::LocalTensor<bfloat16_t> xLocal = inQueueX.DeQue<bfloat16_t> ();  
    AscendC::LocalTensor<bfloat16_t> yLocal = inQueueY.DeQue<bfloat16_t> ();  
    AscendC::LocalTensor<bfloat16_t> zLocal = outQueueZ.AllocTensor<bfloat16_t> ();  
  
    AscendC::LocalTensor<float> tmpTensor0 = tmpBuf0.Get<float>();  
    AscendC::LocalTensor<float> tmpTensor1 = tmpBuf1.Get<float>();  
    AscendC::LocalTensor<float> tmpTensor2 = tmpBuf2.Get<float>();  
    AscendC::Cast(tmpTensor0, xLocal, AscendC::RoundMode::CAST_NONE, TOTAL_LENGTH);  
    AscendC::Cast(tmpTensor1, yLocal, AscendC::RoundMode::CAST_NONE, TOTAL_LENGTH);  
  
    AscendC::Add(tmpTensor2, tmpTensor0, tmpTensor1, TOTAL_LENGTH);  
    AscendC::Cast(zLocal, tmpTensor2, AscendC::RoundMode::CAST_RINT, TOTAL_LENGTH);  
  
    outQueueZ.EnQue<bfloat16_t>(zLocal);  
    inQueueX.FreeTensor(xLocal);  
    inQueueY.FreeTensor(yLocal);  
}
```

3.3.2.4 多核&Tiling 切分

3.3.2.4.1 概述

Ascend C核函数是运行在一个核上的处理函数，上述介绍的[基础矢量算子](#)与[TBuf的使用](#)样例均为在单核上运行的算子，不涉及Host侧Tiling实现。矢量算子实现的组成如下图所示。

为了提高算子的执行效率，通常在算子中实现多核并行计算，即对输入数据进行切分，并将不同的数据块分配到不同的核上处理。关于并行计算的具体介绍请参考[SPMD模型](#)。此外，由于单个核上内部存储Local Memory大小有限，存在无法一次完整地容纳算子的输入和输出数据的场景，因此需要每次搬运一部分输入进行计算然后搬出，再搬运下一部分输入进行计算，直到获得最终的完整结果，这个数据切分、分块计算的过程称之为**Tiling**。切分数据的算法称为Tiling算法或者Tiling策略。根据算子的shape等信息来确定数据切分算法相关参数（比如每次搬运的块大小，以及总共循环多少次）的计算程序，称之为**Tiling实现**，也叫Tiling函数（Tiling Function）。由于Tiling实现中完成的均为标量计算，AI Core并不擅长，所以我们将其独立出来放在Host侧CPU上执行。核函数内部通过解析Host侧传入的Tiling结构体获取Tiling信息，根据Tiling信息控制数据搬入、搬出Local Memory的流程；通过调用计算、数据搬运、内存管理、任务同步API，实现算子逻辑。

图 3-7 算子实现组成



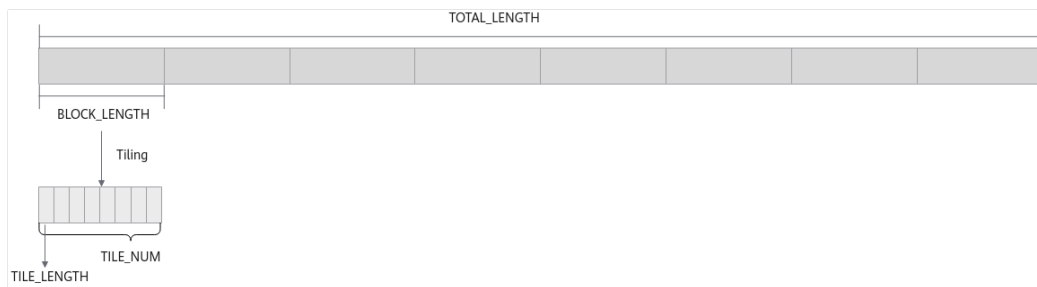
由于硬件限制，在对输入数据进行数据切分时应遵循以下几个原则：

1. 由于AI Core中Unified Buffer上的物理限制，要求Unified Buffer上的数据存储空间必须保持32字节对齐。
 - 输入数据不满足32字节对齐时，需要取输入数据长度向上对齐到32字节的长度作为输入数据总长度。
 - 进行Tiling有关计算时，以32字节为最小单位进行计算。
2. 尽可能最大利用Unified Buffer空间。

AI Core与外部存储交互时会产生性能开销，频繁的进行数据搬运会导致性能瓶颈，因此应尽可能充分利用Unified Buffer空间，减少从Global Memory上搬运数据的次数。
3. AI处理器包含多个AI Core，应该充分均衡利用多核计算能力，将计算均衡分配到多个AI Core上。

本章将基于以上原则对几种典型场景进行说明，完整代码请参见[多核Add算子样例](#)。

图 3-8 多核及 Tiling 示意图



数据切分示意如上图所示，将长度为`TOTAL_LENGTH`的算子输入分配到多个核上进行计算，每个核上计算的数据长度为`BLOCK_LENGTH`。对于每个核的计算数据，基于 Local Memory 的大小进一步切分，切分数据块的个数为`TILE_NUM`，得到的每个数据块的长度为`TILE_LENGTH`。

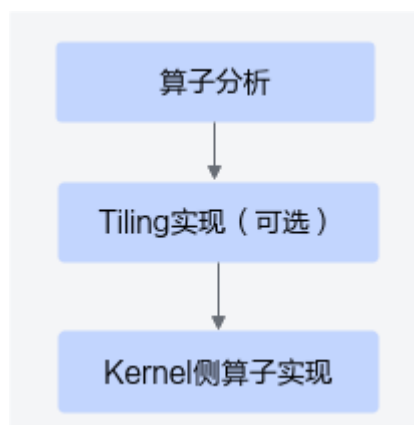
根据每个核计算的数据量是否相同、核内每个数据块的数据量是否相同，切分策略可能会存在以下几种场景：

1. 核间均分，核内均分：每个核处理的数据量相同，核内每个数据块的数据量相同。在此场景中，通过**多核Tiling**将数据均匀分配到各个核上执行，每个核上每次计算的数据长度相同。
2. 核间均分，核内不均分：每个核处理的数据量相同，核内各数据块的数据量不完全相同。此场景基于多核Tiling，核内数据不能切分为多个数据量相同且32字节对齐的数据块，需要通过**尾块Tiling**处理尾块数据的计算。
3. 核间不均分，核内均分：每个核处理的数据量不同，核内每个数据块的数据量相同。在此场景中，通过**尾核Tiling**的处理解决数据无法在各核间均匀分配的问题。
4. 核间不均分，核内不均分：每个核处理的数据量不同，核内各数据块的数据量不完全相同。该场景下需要同时考虑**尾核&尾块**，处理多核间及核内数据的合理切分。

3.3.2.4.2 多核 Tiling

基于Ascend C方式实现带有Tiling的算子的开发流程如下图所示。

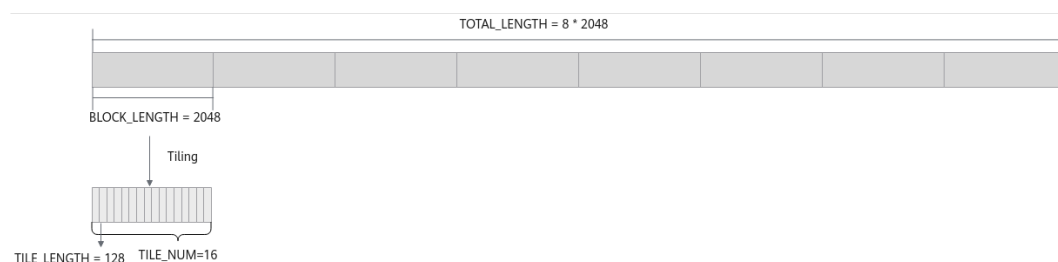
图 3-9 算子开发流程



算子分析

本样例为输入数据在核间均分、核内均分场景。本样例的Tiling策略为：数据整体长度TOTAL_LENGTH为 $8 * 2048$ ，数据平均分配到8个核上运行，每个核上计算的数据长度BLOCK_LENGTH为2048，将单核上的数据切分成16块（此处切分成16块仅用来作为Tiling的样例，并不代表性能最佳，仅供参考），每块数据的长度TILE_LENGTH为128。数据切分示意如下图所示：

图 3-10 数据切分示意图



通过以上分析，得到Ascend C Add算子的设计规格如下：

- 算子类型（OpType）：Add
- 算子输入输出：

表 3-3 Add 算子输入输出规格

name	shape	data type	format
x（输入）	(8, 2048)	half	ND
y（输入）	(8, 2048)	half	ND
z（输出）	(8, 2048)	half	ND

- 核函数名称：add_custom
- 使用的主要接口：
 - DataCopy：数据搬移接口
 - Add：矢量基础算术接口
 - EnQue、DeQue等接口：Queue队列管理接口
- 算子实现文件名称：add_custom.cpp

Tiling 实现

前述场景中算子的输入和输出均为固定shape，然而在实际的算子开发场景中，这些信息是支持动态变化的，场景会更加灵活和复杂。动态shape场景下，输入的shape是未知的。一些与输入shape相关的变量（比如每次搬运的块大小等），需要通过Tiling计算出来，然后传递到kernel侧，kernel侧使用该参数进行后续的计算。

具体实现方式为：分析设计Tiling参数、定义Tiling结构体，在Host侧通过上下文获取输入输出的shape信息，根据shape信息，计算Tiling参数并设置到对应的Tiling结构体中；通过核函数入口参数将Tiling信息传入核函数，在核函数内通过解析Tiling结构体，获取并使用相关参数来实现核函数内部逻辑，详细介绍请参考[Host侧tiling实现](#)。本节将以上述分析中的切分策略为例，说明如何实现Tiling。

基于本节的切分策略，Tiling需要定义如下参数：

- blockLength: 每个核的计算数据长度;
- tileNum: 每个核需要计算的数据块个数;
- tileLength: 每个核内每个数据块的长度。

根据确定的Tiling参数，在算子TilingData结构定义头文件中，使用C++语法定义TilingData结构体，代码如下。该头文件命名为“算子名称_tiling.h”。本章节中的算子名称为add_custom，对应头文件命名应为add_custom_tiling.h。

```
struct AddCustomTilingData {
    uint32_t blockLength;
    uint32_t tileNum;
    uint32_t tileLength;
}
```

接下来，创建一个与Tiling结构体头文件对应的cpp文件add_custom_tiling.cpp，并在该文件内完成Tiling参数的计算。由于每个核内数据被切分为16块，根据使用的核数和核内切分数，计算Tiling参数，并写入到Tiling结构体内。代码示例如下：

```
#include "add_custom_tiling.h"
constexpr int32_t CORE_NUM = 8;           // 使用的核数
constexpr int32_t TILE_NUM = 16;         // 核内切分数量
void GenerateTilingData(uint8_t* tilingBuf)
{
    uint32_t totalLength;
    // 此处省略如何获取数据总长TOTAL_LENGTH，可以根据具体情况实现。本章节仅介绍Tiling相关内容。
    AddCustomTilingData *tiling = reinterpret_cast<AddCustomTilingData *>(tilingBuf);
    uint32_t blockLength = TOTAL_LENGTH / CORE_NUM;
    uint32_t tileNum = TILE_NUM;
    uint32_t tileLength = blockLength / tileNum;

    tiling->blockLength = blockLength;
    tiling->tileNum = tileNum;
    tiling->tileLength = tileLength;
}
```

最后，在Host侧调用程序中，调用上述Tiling参数计算函数，计算出相关参数，然后传递到Kernel侧核函数。

```
extern void GenerateTilingData(uint8_t* tilingBuf);
constexpr int32_t CORE_NUM = 8;
...
uint8_t *tiling = nullptr;
size_t tilingSize = sizeof(AddCustomTilingData);
#ifdef ASCENDC_CPU_DEBUG
    tiling = (uint8_t *)AscendC::GmAlloc(tilingSize); // CPU Debug模式
...
#else
    CHECK_ACL(aclrtMallocHost((void **)&tiling, tilingSize)); // NPU模式
...
#endif
    GenerateTilingData(tiling); // 调用tiling参数计算函数
...
#ifdef ASCENDC_CPU_DEBUG
    ...
    ICPU_RUN_KF(add_custom, CORE_NUM, x, y, z,
        *reinterpret_cast<AddCustomTilingData *>(tiling)); // CPU Debug模式下核函数调用
    ....
#else
    ....
    ACLRT_LAUNCH_KERNEL(add_custom)(CORE_NUM, stream, xDevice, yDevice, zDevice, // NPU模式下核函数调用
        reinterpret_cast<AddCustomTilingData *>(tiling));
    ....

```

算子类实现

Kernel侧算子实现仍遵循[矢量算子核函数实现流程](#)，接下来重点介绍本场景中算子类实现的不同点。

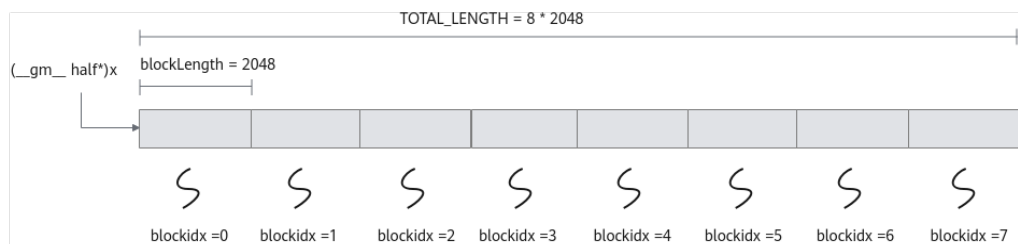
- 设置输入输出Global Tensor的Global Memory内存地址。

由于本样例中将数据分配到了多个核上进行处理，每个核处理不同的数据，因此不同核要处理的数据在Global Memory上的地址不同，在初始化函数Init中，需要获取单核所需处理的输入输出在Global Memory上的内存偏移地址，并将该偏移地址设置到GlobalTensor中。

以获取输入x在Global Memory上的内存偏移地址为例，数据整体长度TOTAL_LENGTH为 $8 * 2048$ ，平均分配到8个核上运行，每个核上处理的数据长度blockLength为2048，调用GetBlockIdx接口获取当前核的index， $x + \text{blockLength} * \text{GetBlockIdx}()$ 即为单核处理程序中x在Global Memory上的内存偏移地址，获取偏移地址后，使用GlobalTensor类的SetGlobalBuffer接口设定该核上Global Memory的起始地址以及长度，具体示意图请参考图3-11。代码如下所示：

```
xGm.SetGlobalBuffer((__gm__ half *)x + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
```

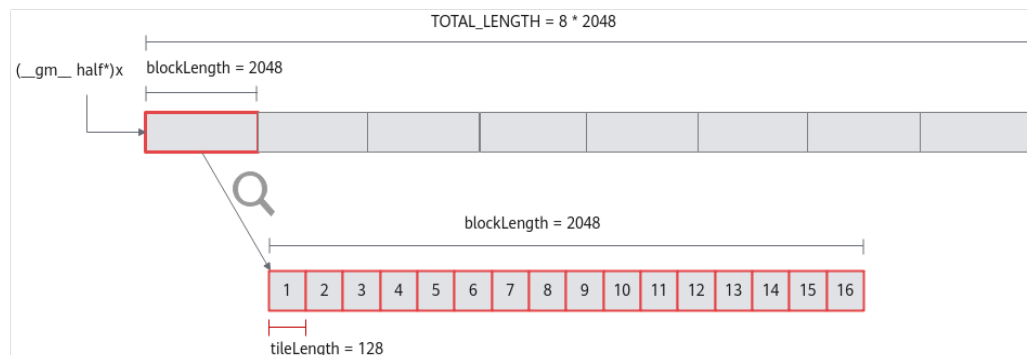
图 3-11 多核并行处理示意图



- 通过Pipe内存管理对象为输入输出Queue分配内存。

对于单核上的处理数据，可以进行数据切块（Tiling），在本示例中，仅作为参考，将单核上的数据（2048个数）切分成16块（并不意味着16块就是性能最优），每块tileLength（128）个数据。数据切分示意图如图3-12所示。

图 3-12 单核数据切分示意图



与[基础矢量算子](#)相比，在通过Pipe内存管理对象为输入输出Queue分配内存时，需使用单核内每个数据块的长度tileLength作为分配内存的长度。比如，为输入x的Queue分配内存，可以通过如下代码段实现，Pipe为inQueueX分配了一块大小

为 $\text{tileLength} * \text{sizeof}(\text{half})$ 个字节的内存块，每个内存块能容纳 tileLength （128）个 half 类型数据。

```
pipe.InitBuffer(inQueueX, 1, this->tileLength * sizeof(half))
```

具体的初始化函数代码如下：

```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData tiling)
{
    this->blockLength = tiling.blockLength;
    this->tileNum = tiling.tileNum;
    this->tileLength = tiling.tileLength;
    // 计算每个核上的地址偏移
    xGm.SetGlobalBuffer((__gm__ half *)x + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
    yGm.SetGlobalBuffer((__gm__ half *)y + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
    zGm.SetGlobalBuffer((__gm__ half *)z + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
    // pipe alloc memory to queue, the unit is Bytes
    pipe.InitBuffer(inQueueX, 1, this->tileLength * sizeof(half));
    pipe.InitBuffer(inQueueY, 1, this->tileLength * sizeof(half));
    pipe.InitBuffer(outQueueZ, 1, this->tileLength * sizeof(half));
}
```

每个核需要对 tileNum 个数据块分别进行搬入、计算、搬出处理，因此`Process`函数内将 tileNum 作为循环上限。

```
__aicore__ inline void Process()
{
    int32_t loopCount = this->tileNum;
    // tiling strategy, pipeline parallel
    for (int32_t i = 0; i < loopCount; i++) {
        CopyIn(i);
        Compute(i);
        CopyOut(i);
    }
}
```

对应的，每个核内搬入、搬出每个数据块时，需定位到每个数据块所在Global Memory上的内存偏移地址，因此在`CopyIn`和`CopyOut`函数内部使用`DataCopy`接口时，需增加每个数据块的地址偏移。`Compute`函数没有变化，与[基础向量算子](#)相同。

`CopyIn`函数实现代码如下：

```
__aicore__ inline void CopyIn(int32_t progress)
{
    ...
    // copy progress_th tile from global tensor to local tensor
    AscendC::DataCopy(xLocal, xGm[progress * this->tileLength], this->tileLength);
    AscendC::DataCopy(yLocal, yGm[progress * this->tileLength], this->tileLength);
    ...
}
```

`CopyOut`函数实现代码如下：

```
__aicore__ inline void CopyOut(int32_t progress)
{
    ...
    // copy progress_th tile from local tensor to global tensor
    AscendC::DataCopy(zGm[progress * this->tileLength], zLocal, this->tileLength);
    ...
}
```

3.3.2.4.3 尾块 Tiling

如下图中的示例，算子的输入shape为（1，2048），支持的数据类型为 half 类型，输入数据可以对齐到一个`datablock`的大小（32字节），输入数据为 $2048 * 2 / 32 = 128$ 个`datablock`，因此可以平均分配到每个核上（假设使用8个核），每个核上处理256个数，16个`datablock`。此时不需要进行尾块处理。

图 3-13 shape 对齐场景

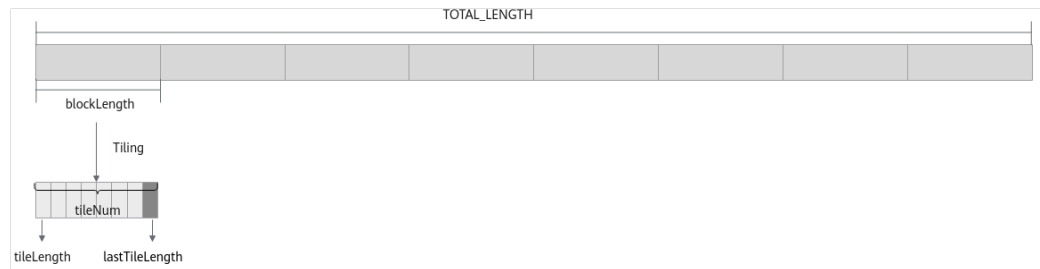


针对一些shape，比如算子的输入shape为（1，1904），支持的数据类型为half类型，输入数据可以对齐到一个datablock的大小（32字节），可以平均分配到每个核上（假设使用8个核），每个核上处理238个数，238个数无法均分到datablock上，分满14个datablock后，剩余14个数（28字节），多核切分后需要进行尾块处理。

对于不同shape的输入进行数据切分时，可能会发生Tiling后的数据平均分配到多核上，但每个核内的数据无法均分的情况。针对此种场景，在Tiling参数中增加变量lastTileLength，用来表示最后一个分块，即尾块的大小。因此，在定义算子的Tiling结构体时包含以下四个成员：

- blockLength：每个核上计算的数据长度；
- tileNum：每个核上切分的主块数据块的个数；
- tileLength：每个核上主块数据块的长度；
- lastTileLength：每个核上尾块的长度。

图 3-14 多核 Tiling 尾块示意图



Tiling 实现

算子的Tiling结构体定义如下：

```
struct AddCustomTilingData {  
    uint32_t blockLength;  
    uint32_t tileNum;  
    uint32_t tileLength;  
};
```

```
uint32_t lastTileLength;  
};
```

Host侧Tiling实现的主要内容为计算以上四个成员变量。步骤如下：

- 步骤1** 判断数据总长度totalLength是否满足32字节对齐，如不满足，则计算totalLength向上32字节对齐后的长度totalLengthAligned。

```
constexpr uint32_t BLOCK_SIZE = 32;  
// 为方便计算，这里根据数据类型定义变量alignNum作为对齐数  
uint32_t alignNum = BLOCK_SIZE / dataTypeSize;  
// totalLength为数据总量  
uint32_t totalLengthAligned = (totalLength % alignNum == 0)?  
    totalLength : ((totalLength + alignNum - 1) / alignNum) * alignNum;
```

- 步骤2** 判断totalLengthAligned是否能被使用的核数NumBlocks均分，如果可以，则计算每个核上计算数据长度blockLength。

```
constexpr uint32_t NUM_BLOCKS = 8;  
constexpr uint32_t UB_BLOCK_NUM = 20; // 此处为方便验证，使用UB_BLOCK_NUM作为Unified Buffer可用的Block数量，因此可得出可用UB空间的大小为UB_BLOCK_NUM * BLOCK_SIZE  
uint32_t blockLength, tileNum;  
if (totalLengthAligned % NUM_BLOCKS == 0) {  
    blockLength = totalLengthAligned / NUM_BLOCKS;  
}
```

- 步骤3** 计算tileNum。为了减少数据搬运开销，应尽量使用核内的Unified Buffer空间。基于每个核上的计算量以及可用Unified Buffer空间的大小，计算tileNum。

```
tileNum = blockLength / alignNum / UB_BLOCK_NUM;
```

- 步骤4** 根据计算出的tileNum，计算tileLength和lastTileLength。

如果每个核的计算量能够被当前可用Unified Buffer空间均分，则按照无尾块场景处理。

```
if ((blockLength / alignNum) % UB_BLOCK_NUM == 0) {  
    // 单核的计算量能被当前可用UB空间均分，仅有主块，无尾块  
    tileLength = UB_BLOCK_NUM * alignNum;  
    lastTileLength = 0;  
}
```

反之，按照尾块场景处理，尾块长度为单核计算数据长度 - tileNum * tileLength。

```
if (tileNum == 0) {  
    // 单核需要计算的长度小于UB可用空间，按照仅有尾块处理  
    tileLength = 0;  
    lastTileLength = ((blockLength + alignNum - 1) / alignNum) * alignNum;  
} else {  
    // 同时有主块和尾块  
    tileLength = UB_BLOCK_NUM * alignNum;  
    lastTileLength = blockLength - tileNum * tileLength;  
}
```

----结束

Host侧Tiling实现的代码如下：

```
constexpr uint32_t BLOCK_SIZE = 32;  
constexpr uint32_t NUM_BLOCKS = 8;  
constexpr uint32_t UB_BLOCK_NUM = 20; // 此处为方便验证，使用UB_BLOCK_NUM作为UB可用的Block数量，因此可得出可用UB空间的大小为UB_BLOCK_NUM * BLOCK_SIZE  
...  
uint32_t alignNum = BLOCK_SIZE / dataTypeSize; // 为方便计算，这里根据数据类型定义变量alignNum作为对齐数，dataTypeSize为运算数据的数据类型对应的字节数  
// totalLength为数据总量  
uint32_t totalLengthAligned = (totalLength % alignNum == 0)?  
    totalLength : ((totalLength + alignNum - 1) / alignNum) * alignNum;  
uint32_t blockLength, tileNum;
```

```
if (totalLengthAligned % NUM_BLOCKS == 0) {
    blockLength = totalLengthAligned / NUM_BLOCKS;
    tileNum = blockLength / alignNum / UB_BLOCK_NUM;

    if (tileNum == 0) {
        // 单核需要计算的长度小于UB可用空间，按照仅有尾块处理
        tileLength = 0;
        lastTileLength = ((blockLength + alignNum - 1) / alignNum) * alignNum;
    } else if ((blockLength / alignNum) % UB_BLOCK_NUM == 0) {
        // 单核的计算量能被当前可用UB空间均分，仅有主块，无尾块
        tileLength = UB_BLOCK_NUM * alignNum;
        lastTileLength = 0;
    } else {
        // 同时有主块和尾块
        tileLength = UB_BLOCK_NUM * alignNum;
        lastTileLength = blockLength - tileNum * tileLength;
    }
}
...
}
```

(1, 1904)形状的输入数据计算后，tiling结构体内各个变量的值如下：

```
struct AddCustomTilingData {
    uint32_t blockLength = 238; // 每个核计算238个half，8个核共计算1904个half
    uint32_t tileNum = 0; // 可用的UB空间足够，为仅有尾块的场景
    uint32_t tileLength = 0; // 没有主块，主块长度为0
    uint32_t lastTileLength = 240; // 238个half未32B对齐，对齐到240个half搬运
};
```

算子类实现

与**多核Tiling**相比，在Init函数中通过Pipe内存管理对象为输入输出Queue分配内存时，取tileLength与lastTileLength中的最大值作为分配内存的长度。例如，当单核需要计算的长度小于UB可用空间时，按照仅有尾块处理，此时tileLength为0，而lastTileLength为数据块长度。因此，需要取两者中的较大值来分配内存。

```
uint32_t initBufferLength = AscendC::Std::max(this->tileLength, this->lastTileLength);
pipe.InitBuffer(inQueueX, 1, initBufferLength * sizeof(dataType));
```

由于尾块长度为lastTileLength，与主块数据块的长度不同，因此在CopyIn函数、Compute函数、CopyOut函数中传入本次循环待处理的数据块长度参数tileLength，即待处理的主块或尾块的数据长度。

Process函数实现代码如下：

```
__aicore__ inline void Process()
{
    // 计算主块数据，对应数据块长度为tileLength
    for (uint32_t i = 0; i < this->tileNum; i++) {
        CopyIn(i, this->tileLength);
        Compute(i, this->tileLength);
        CopyOut(i, this->tileLength);
    }
    // 计算尾块数据，对应数据块长度为lastTileLength
    if (this->lastTileLength > 0) {
        CopyIn(this->tileNum, this->lastTileLength);
        Compute(this->tileNum, this->lastTileLength);
        CopyOut(this->tileNum, this->lastTileLength);
    }
}
```

CopyIn函数实现代码如下：

```
__aicore__ inline void CopyIn(int32_t progress, uint32_t tileLength)
{
    AscendC::LocalTensor<dataType> xLocal = inQueueX.AllocTensor<dataType>();
    AscendC::LocalTensor<dataType> yLocal = inQueueY.AllocTensor<dataType>();
    AscendC::DataCopy(xLocal, xGm[progress * this->tileLength], tileLength);
    AscendC::DataCopy(yLocal, yGm[progress * this->tileLength], tileLength);
}
```

```
inQueueX.Enqueue(xLocal);  
inQueueY.Enqueue(yLocal);  
}
```

Compute函数实现代码如下：

```
__aicore__ inline void Compute(int32_t progress, uint32_t tileLength)  
{  
    AscendC::LocalTensor<dataType> xLocal = inQueueX.DeQue<dataType>();  
    AscendC::LocalTensor<dataType> yLocal = inQueueY.DeQue<dataType>();  
    AscendC::LocalTensor<dataType> zLocal = outQueueZ.AllocTensor<dataType>();  
    AscendC::Add(zLocal, xLocal, yLocal, tileLength);  
    outQueueZ.Enqueue<dataType>(zLocal);  
    inQueueX.FreeTensor(xLocal);  
    inQueueY.FreeTensor(yLocal);  
}
```

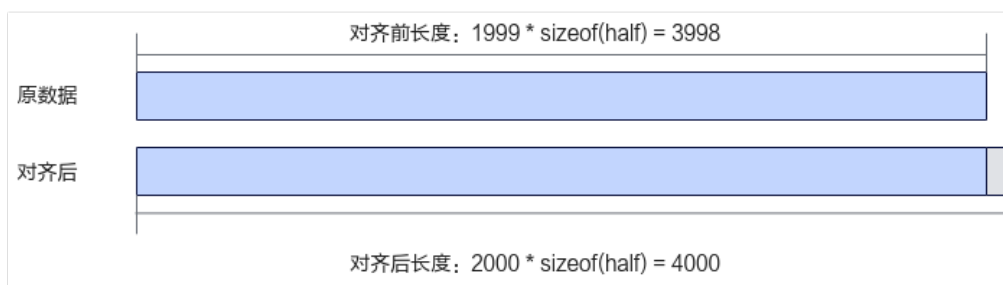
CopyOut函数实现代码如下：

```
__aicore__ inline void CopyOut(int32_t progress, uint32_t tileLength)  
{  
    AscendC::LocalTensor<dataType> zLocal = outQueueZ.DeQue<dataType>();  
    AscendC::DataCopy(zGm[progress * this->tileLength], zLocal, tileLength);  
    outQueueZ.FreeTensor(zLocal);  
}
```

3.3.2.4.4 尾核 Tiling

对于不同shape的输入进行数据切分时，可能会发生数据无法平均分配到多个核的情况。例如当算子的输入shape为[1, 1999]，使用核数为8，数据类型为half时，需要计算的数据总量为 $1 * 1999 * \text{sizeof}(\text{half}) = 3998$ 字节，3998字节既不满足32字节对齐，也无法平均分配到8个核上。因此该场景下，对数据进行多核切分后，每个核的计算数据量不同。此种情况下，应该尽可能均匀的分配数据，所有核上的计算数据量有两种情况，将计算量较多的核称为整核，计算量较少的核称为尾核。

图 3-15 数据对齐示意图



Tiling 实现

- 因为AI处理器在进行数据搬运和Vector计算时，对于搬运的数据长度和Unified Buffer首地址都有必须32字节对齐的要求，**首先待处理数据需要先保证向上对齐到32字节的大小**。该场景下后续搬运和计算的处理细节请参考[3.3.2.7 非对齐场景](#)。如下代码片段展示了将数据对齐到datablock大小的示例：

```
constexpr uint32_t SIZE_OF_HALF = 2;  
constexpr uint32_t BLOCK_SIZE = 32;  
constexpr uint32_t NUM_BLOCKS = 8;  
constexpr uint32_t ALIGN_NUM = BLOCK_SIZE / SIZE_OF_HALF;  
// shape需要对齐到的32字节，假设原totalLength为1999，向上满足32字节对齐后为2000  
uint32_t totalLengthAligned = ((totalLength + ALIGN_NUM - 1) / ALIGN_NUM) * ALIGN_NUM;
```

- 满足32字节对齐后的数据，应尽可能的均分到每个核上。如果无法均分，那么先将可以均分的部分平均分配，剩余的部分分配给部分核，会有部分核多算一个datablock。为了保证切分后的数据仍是满足32字节对齐的，以ALIGN_NUM

(ALIGN_NUM个数据为32字节)为粒度,将数据分配到所有核上。在本样例中,数据类型为half, $\text{ALIGN_NUM} = \text{BLOCK_SIZE} / \text{sizeof}(\text{half}) = 16$ 。将对齐后的数据总量按ALIGN_NUM为粒度分成x个数据块, $x = 2000 / 16 = 125$ 。

AI处理器的核数NUM_BLOCKS为8,无法将125个数据块均分到8个核上。按照以下步骤将数据块尽可能的均分到每个核上:

- 计算 $x / \text{NUM_BLOCKS} = 15$;
- 计算 $x \% \text{NUM_BLOCKS} = 5$ 。

根据上述步骤得出,如果每个核上分配15个数据块,那么将有5个数据块剩余。将这5个剩余的数据块分配到5个核上,这样可以得到5个计算16个数据块的整核和3个计算15个数据块的尾核。下图展示了数据无法均分时多核切分的示例。

图 3-16 无法均分到每个核上的示例



基于上文,设计如下的算子Tiling结构体成员:

- formerNum: 分配到数据量较多的核数,即整核的核数。
- tailNum: 分配到数据量较少的核数,即尾核的核数。
- formerLength: 整核计算的数据长度。
- tailLength: 尾核计算的数据长度。

Tiling参数的计算代码如下:

```
constexpr uint32_t NUM_BLOCKS = 8;
constexpr uint32_t SIZE_OF_HALF = 2;
constexpr uint32_t BLOCK_SIZE = 32;
// shape需要对齐到的最小单位
constexpr uint32_t ALIGN_NUM = BLOCK_SIZE / SIZE_OF_HALF;
...
uint8_t *GenerateTiling()
{
    // shape需要对齐到的datablock,假设原totalLength为1999,向上满足32字节对齐后为2000
    uint32_t totalLengthAligned = ((totalLength + ALIGN_NUM - 1) / ALIGN_NUM) * ALIGN_NUM;
    // 核心数为8,一个datablock包含16个数,那么:datablock的总数: 2000 / 16 = 125
    // 有5个核会分到16个datablock: 125 % 8 = 5,可以称之为整核
    // 有3个核会分到15个datablock: 8 - 5 = 3,可以称之为尾核
    uint32_t formerNum = (totalLengthAligned / ALIGN_NUM) % NUM_BLOCKS;
    uint32_t tailNum = NUM_BLOCKS - formerNum;
    // 整核计算的数据长度: totalLengthAligned / NUM_BLOCKS为每个核上计算的元素个数, formerLength为
```

```
上述元素个数向上32字节对齐的结果
uint32_t formerLength = ((totalLengthAligned / NUM_BLOCKS + ALIGN_NUM - 1) / ALIGN_NUM) *
ALIGN_NUM;
// 尾核计算的数据长度: totalLengthAligned / NUM_BLOCKS为每个核上计算的元素个数, tailLength 为上述
元素个数向下32字节对齐的结果
uint32_t tailLength = (totalLengthAligned / NUM_BLOCKS / ALIGN_NUM) * ALIGN_NUM;
...
}
```

算子类实现

在Kernel侧的Init函数中，计算输入在Global Memory上的内存偏移地址时，应对整核和尾核加以区分。

整核上，输入的内存偏移地址计算代码如下：

```
xGm.SetGlobalBuffer((__gm__ half *)x + formerLength * AscendC::GetBlockIdx(), formerLength);
```

尾核上，计算输入的内存偏移地址时，需在全部整核的数据长度基础上加上尾核的偏移量，代码如下：

```
xGm.SetGlobalBuffer((__gm__ half *)x + formerLength * formerNum + tailLength * (AscendC::GetBlockIdx()
- formerNum), tailLength);
```

完整的Init函数实现代码如下：

```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData tiling)
{
    if (AscendC::GetBlockIdx() < formerNum) {
        this->tileLength = formerLength;
        xGm.SetGlobalBuffer((__gm__ half *)x + formerLength * AscendC::GetBlockIdx(), formerLength);
        yGm.SetGlobalBuffer((__gm__ half *)y + formerLength * AscendC::GetBlockIdx(), formerLength);
        zGm.SetGlobalBuffer((__gm__ half *)z + formerLength * AscendC::GetBlockIdx(), formerLength);
    } else {
        this->tileLength = tailLength;
        xGm.SetGlobalBuffer((__gm__ half *)x + formerLength * formerNum + tailLength *
(AscendC::GetBlockIdx() - formerNum), tailLength);
        yGm.SetGlobalBuffer((__gm__ half *)y + formerLength * formerNum + tailLength *
(AscendC::GetBlockIdx() - formerNum), tailLength);
        zGm.SetGlobalBuffer((__gm__ half *)z + formerLength * formerNum + tailLength *
(AscendC::GetBlockIdx() - formerNum), tailLength);
    }
    pipe.InitBuffer(inQueueX, 1, this->tileLength * sizeof(half));
    pipe.InitBuffer(inQueueY, 1, this->tileLength * sizeof(half));
    pipe.InitBuffer(outQueueZ, 1, this->tileLength * sizeof(half));
}
```

其余实现与[多核Tiling](#)中的实现一致，这里不重复进行说明。

3.3.2.4.5 尾核&尾块

对于不同shape的输入进行数据切分时，可能会发生数据无法平均分配到多个核、同时每个核内的数据无法均分的情况。参考[核间均分场景下的尾块处理](#)与[核间不均分场景下的尾核处理](#)的处理方式，将两者结合起来考虑整核的尾块、尾核的尾块的处理方式。

Tiling 实现

由于本场景中核间、核内的数据均无法均分，在[核间不均分场景下的尾核处理](#)定义的Tiling结构体的基础上增加两个成员变量：

- formerLastTileLength：数据量多的核最后一个分块大小，即整核的尾块大小。计算时，先按[3.3.2.4.4 尾核Tiling](#)中提到的分核策略，切分数据量多的核。

```
// shape需要对齐到的datablock,
uint32_t totalLengthAligned = ((totalLength + ALIGN_NUM - 1) / ALIGN_NUM) * ALIGN_NUM;
// 计算整核数量
uint32_t formerNum = (totalLengthAligned / ALIGN_NUM) % NUM_BLOCKS;
// 计算整核的数据量
uint32_t formerLength = ((totalLengthAligned / NUM_BLOCKS + ALIGN_NUM - 1) / ALIGN_NUM) *
ALIGN_NUM;
```

再按[3.3.2.4.3 尾块Tiling](#)中的切分策略，计算尾块长度。

```
uint32_t formerTileNum = formerLength / ALIGN_NUM / UB_BLOCK_NUM;
if (formerTileNum == 0) {
    formerTileLength = 0;
    formerLastTileLength = formerLength / ALIGN_NUM * ALIGN_NUM;
} else if ((formerLength / ALIGN_NUM) % UB_BLOCK_NUM == 0) {
    formerTileLength = UB_BLOCK_NUM * ALIGN_NUM;
    lastTileLength = 0;
} else {
    formerTileLength = UB_BLOCK_NUM * ALIGN_NUM;
    formerLastTileLength = formerLength - formerTileNum * formerTileLength;
}
```

- **tailLastTileLength**: 数据量少的核最后一个分块大小，即尾核的尾块大小。
计算时，先按[3.3.2.4.4 尾核Tiling](#)中提到的分核策略，切分数据量少的核。

```
// 计算尾核数量
uint32_t tailNum = NUM_BLOCKS - formerNum;
// 计算尾核的数据量
uint32_t tailLength = (totalLengthAligned / NUM_BLOCKS / ALIGN_NUM) * ALIGN_NUM;
```

再按[3.3.2.4.3 尾块Tiling](#)中的切分策略，计算尾块长度。

```
uint32_t tailTileNum = tailLength / ALIGN_NUM / UB_BLOCK_NUM;
if (tailTileNum == 0) {
    tailTileLength = 0;
    tailLastTileLength = tailLength / ALIGN_NUM * ALIGN_NUM;
} else if ((tailLength / ALIGN_NUM) % UB_BLOCK_NUM == 0) {
    tailTileLength = UB_BLOCK_NUM * ALIGN_NUM;
    tailLastTileLength = 0;
} else {
    tailTileLength = UB_BLOCK_NUM * ALIGN_NUM;
    tailLastTileLength = tailLength - tailTileNum * tailTileLength;
}
```

算子类实现

Kernel侧Init函数和Process函数的实现需将[核间均分场景下的尾块处理](#)与[核间不均分场景下的尾核处理](#)的实现结合起来。

Init函数中由于整核和尾核对应的tileLength和lastTileLength不同。因此需按照[核间不均分场景下的尾核处理](#)中提到的分别处理整核和尾核。后续对主块和尾块的CopyIn、Compute、CopyOut函数的处理方式与[核间均分场景下的处理](#)方式相同。

Init函数实现代码如下：

```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData tiling)
{
    if (AscendC::GetBlockIdx() < formerNum) {
        this->tileNum = tiling.formerTileNum;
        this->tileLength = tiling.formerTileLength;
        this->lastTileLength = tiling.formerLastTileLength;

        xGm.SetGlobalBuffer((__gm__ half *)x + tiling.formerLength * AscendC::GetBlockIdx(),
            tiling.formerLength);
        yGm.SetGlobalBuffer((__gm__ half *)y + tiling.formerLength * AscendC::GetBlockIdx(),
            tiling.formerLength);
        zGm.SetGlobalBuffer((__gm__ half *)z + tiling.formerLength * AscendC::GetBlockIdx(),
            tiling.formerLength);
    } else {
```



```
this->tileNum = tiling.tailTileNum;
this->tileLength = tiling.tailTileLength;
this->lastTileLength = tiling.tailLastTileLength;

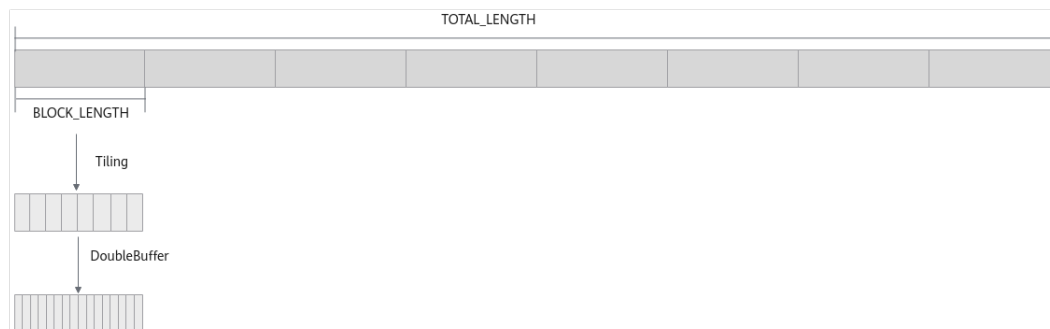
xGm.SetGlobalBuffer((__gm__ half *)x + tiling.formerLength * tiling.formerNum +
    tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum), tiling.tailLength);
yGm.SetGlobalBuffer((__gm__ half *)y + tiling.formerLength * tiling.formerNum +
    tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum), tiling.tailLength);
zGm.SetGlobalBuffer((__gm__ half *)z + tiling.formerLength * tiling.formerNum +
    tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum), tiling.tailLength);
}

// 只有尾块的场景下, tileLength为0, 因此取tileLength和lastTileLength的最大值来初始化
uint32_t initBufferLength = AscendC::Std::max(this->tileLength, this->lastTileLength);
pipe.InitBuffer(inQueueX, 1, initBufferLength * sizeof(half));
pipe.InitBuffer(inQueueY, 1, initBufferLength * sizeof(half));
pipe.InitBuffer(outQueueZ, 1, initBufferLength * sizeof(half));
}
```

3.3.2.5 DoubleBuffer 场景

因存在算子中多次搬入搬出数据的场景, 为充分利用硬件资源, 实现多流水并行, 引入**DoubleBuffer**机制。**DoubleBuffer**是通过将输入数据分成大小相等的两块, 充分利用AI Core的硬件资源, 实现数据搬入、计算、数据搬出的并行执行方式。下面以“核间不均分, 核内不均分”的样例为例, 介绍算子中DoubleBuffer的实现, 完整样例代码请参见[使用DoubleBuffer的Add算子样例](#)。

图 3-17 DoubleBuffer 数据切分示意图



Tiling 实现

使能DoubleBuffer后, 每一个数据块会分成大小相等的两块, 因此, 若要使能DoubleBuffer, 要求数据总量应该能够均分。为了简化处理, 将可用的Unified Buffer空间以32字节为粒度, 分成n块dataBlock, 如果n不是偶数, 则减1, 这样就可以保证一套代码兼容开启或不开启DoubleBuffer功能。对应步骤如下:

步骤1 判断数据总长度totalLength是否满足32字节对齐, 如不满足, 则计算totalLength向上32字节对齐后的长度totalLengthAligned。

```
constexpr uint32_t BLOCK_SIZE = 32;
// 为方便计算, 这里根据数据类型定义变量alignNum作为对齐数
uint32_t alignNum = BLOCK_SIZE / dataTypeSize;
// totalLength为数据总量
uint32_t totalLengthAligned = (totalLength % alignNum == 0)?
    totalLength : ((totalLength + alignNum - 1) / alignNum) * alignNum;
```

步骤2 根据totalLengthAligned, 计算每个核的计算数据长度blockLength, 分核策略可参照[3.3.2.4.4 尾核Tiling](#)。

步骤3 计算其余Tiling参数。

对当前Unified Buffer可用空间以32字节为粒度，进行切分，计算出数据块个数UB_BLOCK_NUM。根据是否开启DoubleBuffer计算出当前可用的最大数据块个数，记作MAX_AVAILABLE_UB_BLOCK_NUM。最后，以MAX_AVAILABLE_UB_BLOCK_NUM为粒度，对blockLength进行切分。为方便演示，如下代码直接给出UB_BLOCK_NUM，作为当前Unified Buffer可用空间包含的block（32字节）数。

```
constexpr uint32_t BUFFER_NUM = 2;
constexpr uint32_t UB_BLOCK_NUM = 21; // UB最大可以使用的block数量
constexpr uint32_t MAX_AVAILABLE_UB_BLOCK_NUM = UB_BLOCK_NUM / BUFFER_NUM * BUFFER_NUM;

tileNum = blockLength / (alignNum * MAX_AVAILABLE_UB_BLOCK_NUM);
if (tileNum == 0) {
    // 单核需要计算的长度小于UB可用空间，按照仅有尾块处理
    tileLength = 0;
    lastTileLength = (blockLength + alignNum - 1) / alignNum * alignNum;
} else if ((blockLength / alignNum) % MAX_AVAILABLE_UB_BLOCK_NUM == 0) {
    // 单核的计算量能被当前可用UB空间均分，仅有主块，无尾块
    tileLength = MAX_AVAILABLE_UB_BLOCK_NUM * alignNum;
    lastTileLength = 0;
} else {
    // 同时有主块和尾块
    tileLength = MAX_AVAILABLE_UB_BLOCK_NUM * alignNum;
    lastTileLength = blockLength - tileNum * tileLength;
}
```

----结束

算子类实现

不开启DoubleBuffer时，只需要对每个核上最后一个分块的起始地址做处理；开启DoubleBuffer后，需要处理的数据块长度变成原来的一半，所以需要对最后两个数据块的起始地址做处理。

开启DoubleBuffer，参考InitBuffer接口函数原型，将num参数配置成2，即BUFFER_NUM。

```
this->initBufferLength = AscendC::Std::max(this->tileLength, this->lastTileLength);
pipe.InitBuffer(inQueueX, BUFFER_NUM, this->initBufferLength * sizeof(dataType));
pipe.InitBuffer(inQueueY, BUFFER_NUM, this->initBufferLength * sizeof(dataType));
pipe.InitBuffer(outQueueZ, BUFFER_NUM, this->initBufferLength * sizeof(dataType));
```

同时在计算核内每个数据块的长度时，考虑DoubleBuffer场景，需要将Buffer数量，即BUFFER_NUM=2带入计算。

```
this->tileLength = tiling.tileLength / BUFFER_NUM;
```

由于无法保证尾块满足DoubleBuffer的条件，因此不对尾块进行切分。

```
this->lastTileLength = tiling.lastTileLength;
```

Init函数实现代码如下：

```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData tiling)
{
    if (tiling.isEvenCore) {
        this->blockLength = tiling.blockLength;
        this->tileNum = tiling.tileNum;
        this->tileLength = tiling.tileLength / BUFFER_NUM;
        this->lastTileLength = tiling.lastTileLength;

        xGm.SetGlobalBuffer((__gm__ dataType *)x + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
        yGm.SetGlobalBuffer((__gm__ dataType *)y + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
    }
}
```

```
>blockLength);
    zGm.SetGlobalBuffer((__gm__ dataType *)z + this->blockLength * AscendC::GetBlockIdx(), this->blockLength);
} else {
    if (AscendC::GetBlockIdx() < tiling.formerNum) {
        this->tileNum = tiling.formerTileNum;
        this->tileLength = tiling.formerTileLength / BUFFER_NUM;
        this->lastTileLength = tiling.formerLastTileLength;

        xGm.SetGlobalBuffer((__gm__ dataType *)x + tiling.formerLength * AscendC::GetBlockIdx(),
        tiling.formerLength);
        yGm.SetGlobalBuffer((__gm__ dataType *)y + tiling.formerLength * AscendC::GetBlockIdx(),
        tiling.formerLength);
        zGm.SetGlobalBuffer((__gm__ dataType *)z + tiling.formerLength * AscendC::GetBlockIdx(),
        tiling.formerLength);
    } else {
        this->tileNum = tiling.tailTileNum;
        this->tileLength = tiling.tailTileLength / BUFFER_NUM;
        this->lastTileLength = tiling.tailLastTileLength;

        xGm.SetGlobalBuffer((__gm__ dataType *)x + tiling.formerLength * tiling.formerNum +
        tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum), tiling.tailLength);
        yGm.SetGlobalBuffer((__gm__ dataType *)y + tiling.formerLength * tiling.formerNum +
        tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum), tiling.tailLength);
        zGm.SetGlobalBuffer((__gm__ dataType *)z + tiling.formerLength * tiling.formerNum +
        tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum), tiling.tailLength);
    }
}

uint32_t initBufferLength = AscendC::Std::max(this->tileLength, this->lastTileLength);
pipe.InitBuffer(inQueueX, BUFFER_NUM, initBufferLength * sizeof(dataType));
pipe.InitBuffer(inQueueY, BUFFER_NUM, initBufferLength * sizeof(dataType));
pipe.InitBuffer(outQueueZ, BUFFER_NUM, initBufferLength * sizeof(dataType));
}
```

由于开启DoubleBuffer后，切分后的主块数据块个数翻倍，在Process函数中，需要将BUFFER_NUM带入计算循环次数；尾块独立计算，不开启DoubleBuffer。后续主尾块在CopyIn、Compute、CopyOut函数中的处理，与[尾块tiling处理](#)相同。

```
__aicore__ inline void Process()
{
    // 主块进行DoubleBuffer计算，所以loopCount得乘以2
    uint32_t loopCount = this->tileNum * BUFFER_NUM;
    for (uint32_t i = 0; i < loopCount; i++) {
        CopyIn(i, this->tileLength);
        Compute(i, this->tileLength);
        CopyOut(i, this->tileLength);
    }
    // 尾块进行计算，不做DoubleBuffer操作
    if (this->lastTileLength > 0U) {
        CopyIn(loopCount, this->lastTileLength);
        Compute(loopCount, this->lastTileLength);
        CopyOut(loopCount, this->lastTileLength);
    }
}
```

3.3.2.6 Broadcast 场景

在某些场景下，可能会存在两个输入shape不相同的情况。由于Add接口只支持对shape相同的输入进行计算，因此需要先将输入进行shape变换，再进行Add计算。本节将对满足Broadcast条件的输入在算子实现中的Broadcast处理进行介绍，其他场景可以参考本章节中提供的思路。

须知

Broadcast机制通过扩展较小维度的数据，使得不同shape的输入能够进行运算，从而避免了显式的复制操作，提高了计算效率。数据进行Broadcast需满足：两个输入的维度个数相同，并且仅在某一个维度上的长度不同，某一个输入在此维度的长度为1。比如：shape为(32, 8) 和 (32, 1) 的两个输入可以进行Broadcast，因为它们都是二维，且第一个维度大小相等，而不相等的维度中第二个输入的维度为1，满足条件。

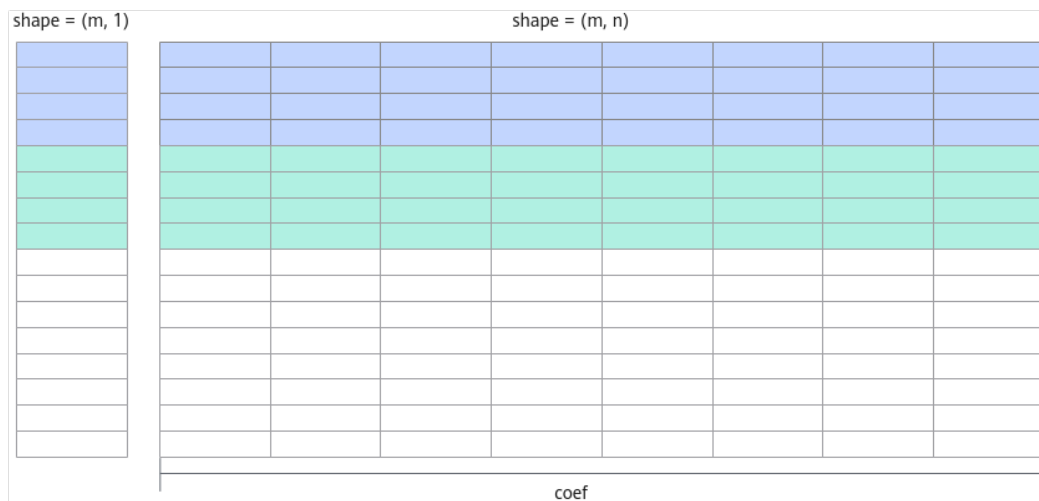
本节中将使用Broadcast接口，因此输入需满足该API相关约束。同时，由于硬件限制，该API的输入地址需满足32字节对齐。本节以输入维度为2、第二个轴（axis = 1）需要Broadcast为例进行说明。完整的样例代码请参见[输入Broadcast的Add算子样例](#)。

Tiling 实现

与输入shape相同的场景相比，在Tiling结构体中增加相应的成员变量，表示是否需要对输入进行Broadcast、需要对哪个维度进行Broadcast、Broadcast的轴需要扩充的倍数。因此新增四个Tiling结构体成员：

- xLen和yLen：表示两个输入的数据长度。
- axis：表示对输入的哪个维度进行Broadcast。
- coef：表示Broadcast的输入需要扩维的倍数。例如，x shape为(m, 1), y shape为(m, n), 则coef = n。如下图所示，图中相同颜色部分为单次计算的数据块。

图 3-18 axis=1 时 coef 示意图



Tiling结构体定义代码如下所示：

```
struct AddCustomTilingData {  
    uint32_t xLen;  
    uint32_t yLen;  
    uint32_t coef;  
    uint32_t axis;  
    ...  
};
```

设需要进行Broadcast的输入长度为shorterAxisLen；不需要进行Broadcast的输入长度为totalLength。

```
constexpr uint32_t BLOCK_SIZE = 32;
... // 读入数据
uint32_t totalLength = (xLen > yLen)? xLen : yLen;
uint32_t shorterAxisLen = (xLen < yLen)? xLen : yLen;
```

使用shorterAxisLen进行分核计算，并使用分核后的长度与coef相乘作为totalLength的分核长度。

```
constexpr uint32_t BLOCK_SIZE = 32;
if (shorterAxisLen % (NUM_BLOCKS * BUFFER_NUM) == 0) {
    uint32_t blockLength = shorterAxisLen / NUM_BLOCKS * coef;
    ...
} else {
    uint32_t formerNum = (shorterAxisLen / BUFFER_NUM) % NUM_BLOCKS;
    uint32_t tailNum = NUM_BLOCKS - formerNum;

    uint32_t formerLength = ((shorterAxisLen / BUFFER_NUM) / NUM_BLOCKS + 1) * BUFFER_NUM * coef;
    uint32_t tailLength = ((shorterAxisLen / BUFFER_NUM) / NUM_BLOCKS) * BUFFER_NUM * coef;
    ....
}
```

进行核内数据切分时，需要计算Unified Buffer数据块的数量向coef和BUFFER_NUM对齐之后的数量ubBlockAligned。

```
ubBlockAligned =
    (UB_BLOCK_NUM * alignNum / (coef * BUFFER_NUM) * (coef * BUFFER_NUM) == 0)?
    UB_BLOCK_NUM : UB_BLOCK_NUM * alignNum / (coef * BUFFER_NUM) * (coef * BUFFER_NUM);
...
tileNum = length / ubBlockAligned;
if (length % ubBlockAligned == 0 || tileNum == 0) {
    if (tileNum == 0) {
        tileNum = 1;
    }
    if (length < ubBlockAligned) {
        tileLength = length;
        lastTileLength = tileLength;
    } else {
        tileLength = ubBlockAligned;
        lastTileLength = tileLength;
    }
} else {
    tileNum = tileNum + 1;
    tileLength = ubBlockNum;
    lastTileLength = (length - (tileNum - 1) * tileLength);
}
```

算子类实现

在核函数初始化阶段，根据Tiling结构体传入的参数确定对哪个输入进行Broadcast。由于针对输入的第二个轴（axis = 1）进行Broadcast，可以计算出，对于需要进行Broadcast的输入，每个核搬入数据长度为blockLength / coef。

初始化函数代码如下：

```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z, AddCustomTilingData tiling)
{
    GM_ADDR longerInputPtr;
    GM_ADDR shorterInputPtr;
    if (tiling.xLen > tiling.yLen) {
        longerInputPtr = x;
        shorterInputPtr = y;
    } else {
        longerInputPtr = y;
        shorterInputPtr = x;
    }
    this->coef = tiling.coef;
    if (tiling.isEvenCore) {
        this->tileNum = tiling.tileNum;
        this->tileLength = tiling.tileLength / BUFFER_NUM;
    }
}
```

```
        this->lastTileLength = tiling.lastTileLength;
        xGm.SetGlobalBuffer((__gm__ dataType *)longerInputPtr + tiling.blockLength *
AscendC::GetBlockIdx(), tiling.blockLength);
        yGm.SetGlobalBuffer((__gm__ dataType *)shorterInputPtr + tiling.blockLength *
AscendC::GetBlockIdx() / this->coef, tiling.blockLength / this->coef);
        zGm.SetGlobalBuffer((__gm__ dataType *)z + tiling.blockLength * AscendC::GetBlockIdx(),
tiling.blockLength);
    } else {
        if (AscendC::GetBlockIdx() < tiling.formerNum) {
            this->tileNum = tiling.formerTileNum;
            this->tileLength = tiling.formerTileLength / BUFFER_NUM;
            this->lastTileLength = tiling.formerLastTileLength;
            xGm.SetGlobalBuffer((__gm__ dataType *)longerInputPtr + tiling.formerLength *
AscendC::GetBlockIdx(), tiling.formerLength);
            yGm.SetGlobalBuffer((__gm__ dataType *)shorterInputPtr + tiling.formerLength *
AscendC::GetBlockIdx() / this->coef, tiling.formerLength / this->coef);
            zGm.SetGlobalBuffer((__gm__ dataType *)z + tiling.formerLength * AscendC::GetBlockIdx(),
tiling.formerLength);
        } else {
            this->tileNum = tiling.tailTileNum;
            this->tileLength = tiling.tailTileLength / BUFFER_NUM;
            this->lastTileLength = tiling.tailLastTileLength;
            xGm.SetGlobalBuffer((__gm__ dataType *)longerInputPtr + tiling.formerLength * tiling.formerNum +
tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum), tiling.tailLength);
            yGm.SetGlobalBuffer((__gm__ dataType *)shorterInputPtr + tiling.formerLength *
tiling.formerNum / this->coef +
tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum) / this->coef, tiling.tailLength / this-
>coef);
            zGm.SetGlobalBuffer((__gm__ dataType *)z + tiling.formerLength * tiling.formerNum +
tiling.tailLength * (AscendC::GetBlockIdx() - tiling.formerNum), tiling.tailLength);
        }
    }
    pipe.InitBuffer(inQueueX, BUFFER_NUM, this->tileLength * sizeof(dataType));
    pipe.InitBuffer(inQueueY, BUFFER_NUM, this->coef * sizeof(dataType));
    pipe.InitBuffer(outQueueZ, BUFFER_NUM, this->tileLength * sizeof(dataType));
    pipe.InitBuffer(tmpBuf2, this->tileLength * sizeof(dataType));
}
```

由于数据是向coef对齐的，在数据拷贝的过程中可能会出现地址不满足32字节对齐的场景，因此CopyIn函数、CopyOut函数中使用DataCopyPad进行数据拷贝。

CopyIn函数实现代码如下：

```
__aicore__ inline void CopyIn(int32_t progress)
{
    AscendC::LocalTensor<dataType> xLocal = inQueueX.AllocTensor<dataType>();
    AscendC::LocalTensor<dataType> yLocal = inQueueY.AllocTensor<dataType>();
    AscendC::DataCopyExtParams copyXParams = {1, (uint32_t)(this->tileLength * sizeof(dataType)), 0, 0, 0};
    AscendC::DataCopyExtParams copyYParams = {1, (uint32_t)(this->tileLength * sizeof(dataType) / this-
>coef), 0, 0, 0};
    AscendC::DataCopyPadExtParams<dataType> padParams = {false, 0, 0, 0};
    if (progress == (this->tileNum * BUFFER_NUM - 1)) {
        AscendC::DataCopyPad<dataType>(xLocal, xGm[(progress - LAST_TWO_TILE) * this->tileLength + this-
>lastTileLength],
        copyXParams, padParams);
        AscendC::DataCopyPad<dataType>(yLocal, yGm[((progress - LAST_TWO_TILE) * this->tileLength + this-
>lastTileLength) / this->coef],
        copyYParams, padParams);
    } else {
        AscendC::DataCopyPad<dataType>(xLocal, xGm[progress * this->tileLength], copyXParams,
padParams);
        AscendC::DataCopyPad<dataType>(yLocal, yGm[progress * this->tileLength / this->coef],
copyYParams, padParams);
    }
    inQueueX.Enqueue(xLocal);
    inQueueY.Enqueue(yLocal);
}
```

CopyOut函数实现代码如下：

```
__aicore__ inline void CopyOut(int32_t progress)
{
    AscendC::LocalTensor<dataType> zLocal = outQueueZ.DeQue<dataType>();
    AscendC::DataCopyExtParams copyParams = {1, (uint32_t)(this->tileLength * sizeof(dataType)), 0, 0, 0};
    if (progress == (this->tileNum * BUFFER_NUM - 1)) {
        AscendC::DataCopyPad<dataType>(zGm[(progress - LAST_TWO_TILE) * this->tileLength + this-
>lastTileLength], zLocal, copyParams);
    } else {
        AscendC::DataCopyPad<dataType>(zGm[progress * this->tileLength], zLocal, copyParams);
    }
    outQueueZ.FreeTensor(zLocal);
}
```

在Compute函数中，调用Add接口前需要先对输入进行Broadcast。这里需要计算Broadcast前后的shape。基于前文提到的数据关系，可以计算得出Broadcast前后的shape分别为{tileLength / broadcastCoef, 1}和{tileLength / broadcastCoef, broadcastCoef}。在此基础上对输入进行Broadcast，并将计算结果存入临时空间中，然后进行Add计算。实现代码示例如下所示：

```
__aicore__ inline void Compute(int32_t progress)
{
    AscendC::LocalTensor<dataType> xLocal = inQueueX.DeQue<dataType>();
    AscendC::LocalTensor<dataType> yLocal = inQueueY.DeQue<dataType>();
    AscendC::LocalTensor<dataType> zLocal = outQueueZ.AllocTensor<dataType>();
    AscendC::LocalTensor<dataType> broadcastTmpTensor = broadcastTmpBuf.Get<dataType>();
    uint32_t dstShape[] = {this->tileLength / this->coef, this->coef};
    uint32_t srcShape[] = {this->tileLength / this->coef, 1};
    AscendC::BroadCast<dataType, 2, 1>(broadcastTmpTensor, yLocal, dstShape, srcShape);
    ...
}
```

3.3.2.7 非对齐场景

本节介绍非32字节对齐数据的更多处理方式，包括数据搬入、计算和搬出的处理。用户在实际算子开发中，可以参考如下方案介绍和算子样例灵活地处理非对齐场景。

数据搬运和 Vector 计算的对齐要求

须知

进行数据搬运和Vector计算时，对于搬运的数据长度和操作数的起始地址有如下的对齐要求：

- 使用DataCopy接口进行数据搬运，搬运的数据长度和操作数的起始地址（UB上）必须保证32字节对齐。
- 通常情况下，进行Vector计算时，操作数的起始地址必须保证32字节对齐。具体对齐要求需要查阅对应的API参考进行确认。

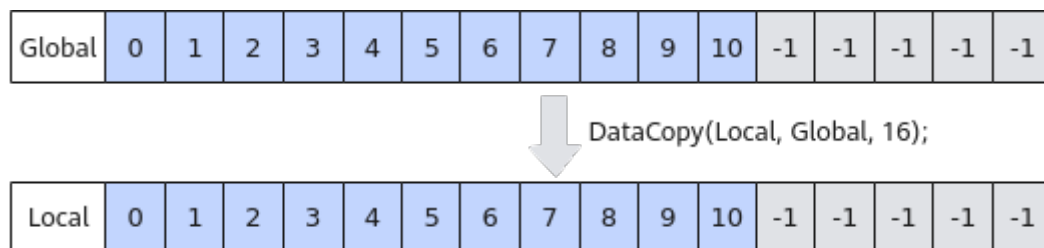
下文描述中的Global指Global Memory上的tensor，Local指Local Memory上的tensor。

下面是一些非对齐搬运和计算的例子。

- **非对齐搬入**

当需要从Global拷贝11个half数值到Local时，为保证搬运的数据长度32字节对齐，使用DataCopy拷贝16个half（32B）数据到Local上，Local[11]~Local[15]被写成无效数据-1。

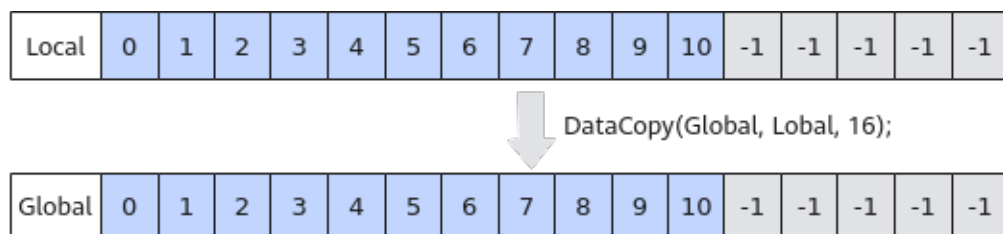
图 3-19 非对齐搬入



- 非对齐搬出

当需要从Local拷贝11个half数值到Global时，为保证搬运的数据长度32字节对齐，使用DataCopy拷贝16个half（32B）数据到Global上，Global[11]~Global[15]被写成无效数据-1。

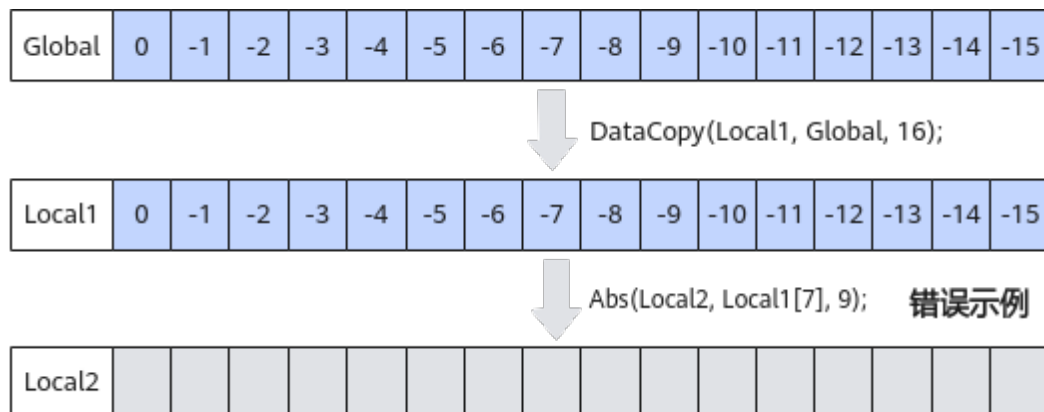
图 3-20 非对齐搬出



- 矢量计算起始地址非32字节对齐的错误示例

矢量计算时需要保证起始地址32字节对齐，如下的示例中，从Local1[7]，即LocalTensor的第8个数开始计算，起始地址不满足32字节对齐，是错误示例。

图 3-21 矢量计算起始地址非 32 字节对齐的错误示例

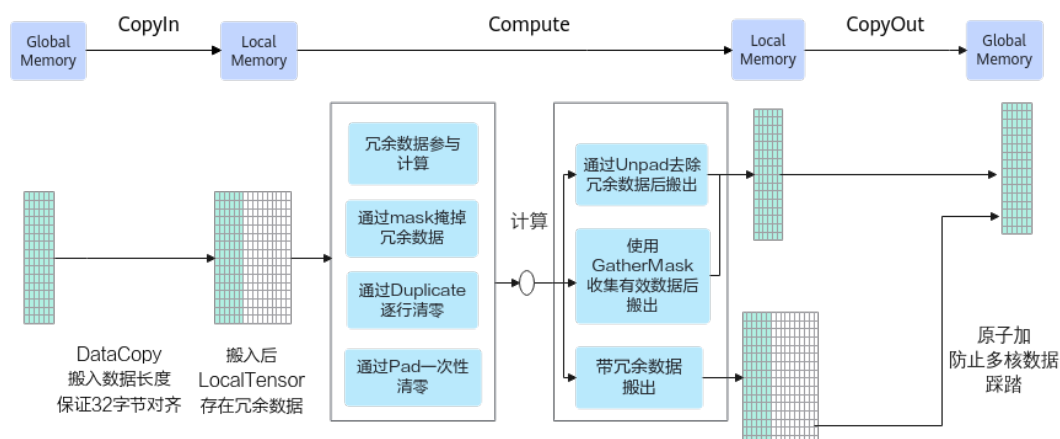


非对齐处理方案

DataCopyPad接口提供非对齐搬运的功能，如果基于该接口支持的产品开发算子（参见产品支持情况），则可以直接使用该接口解决非对齐场景下的搬运问题。使用DataCopyPad的完整示例请参考[DataCopyPad样例（工程化算子开发）](#)和[DataCopyPad样例（kernel直调）](#)。

部分型号不支持DataCopyPad接口，需要参考如下的方案处理。

图 3-22 非对齐处理方案示意图



由于搬入时搬运的数据长度必须保证32字节对齐。数据长度非对齐的情况下，从Global逐行搬运Tensor数据到Local中，Local中每行都存在冗余数据。

搬入后，进行矢量计算时对冗余数据的处理方式有以下几种：

- **冗余数据参与计算。**一般用于elementwise计算场景。
- **通过mask参数掩掉冗余数据。**一般用于轴归约计算等场景。
- **通过Duplicate逐行清零。**计算前，针对每一行数据，调用基础API Duplicate对冗余数据位置填充0值。
- **通过Pad一次性清零。**计算前，针对多行数据，可以采用高阶API Pad接口对冗余数据一次性清零。

由于搬出时搬运的数据长度和操作数的起始地址（UB上）必须保证32字节对齐，搬出时可以选择去除冗余数据或者带着冗余数据搬出的方式。

- **使用UnPad接口去除冗余数据后搬出。**待搬出的有效数据总长度满足32字节对齐时，可使用高阶API UnPad接口去除冗余数据并完整搬出。
- **使用GatherMask收集有效数据后搬出。**待搬出的有效数据总长度大于等于32字节时，可使用GatherMask重新收集有效数据，保证搬出的有效数据起始地址和数据长度32字节对齐。
- **带冗余数据搬出。**注意多核处理时开启原子加（使用SetAtomicAdd接口），防止数据踩踏。

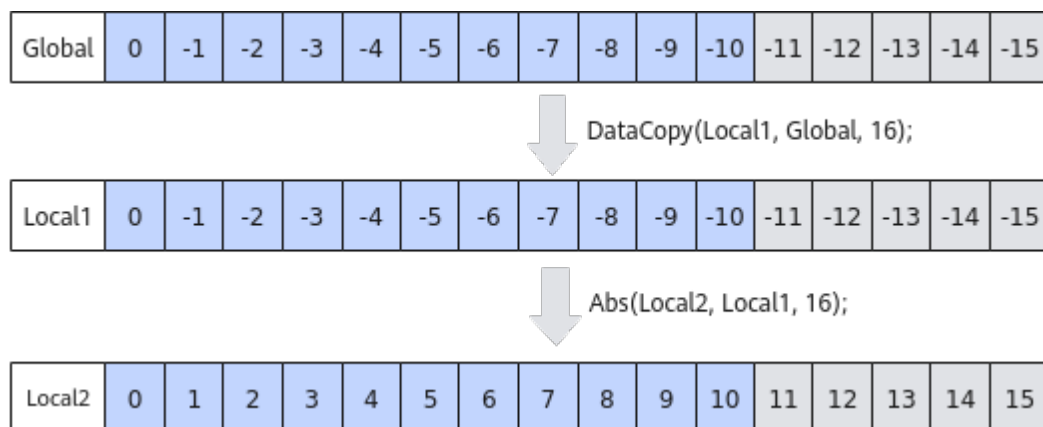
下面分别对上述几种处理方案做详细说明。

- **冗余数据参与计算**

如下图所示，对前11个half数据进行Abs计算，冗余数据可以参与计算，不影响最终结果。步骤为：

- 使用DataCopy从Global搬运16个half数据到Local1中，包含冗余数据-11~-15；
- 直接使用Abs做整块计算，不用计算尾块大小，冗余数据参与计算。

图 3-23 冗余数据参与计算

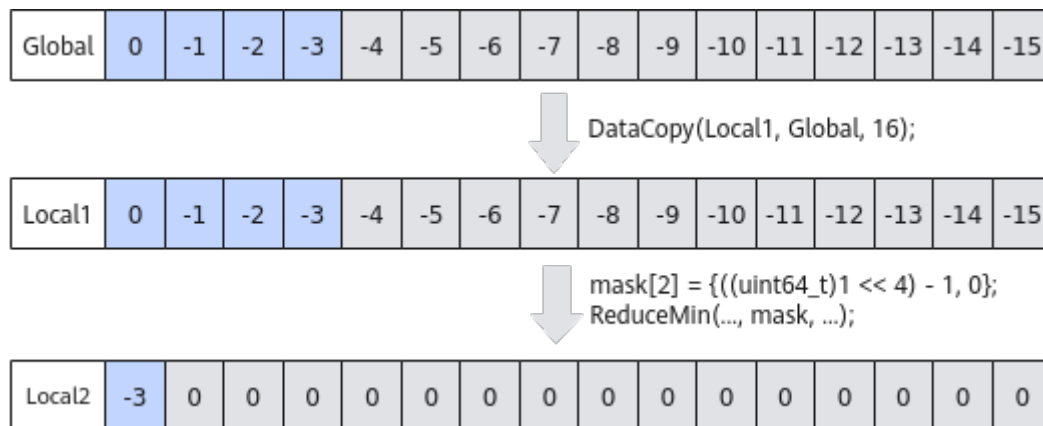


- 使用mask掩掉冗余数据

如下图所示，假设输入数据的shape为16 * 4，将输入数据搬入到UB后每行数据前4个half数据为有效数据，其余为冗余数据。为只对前4个half数据进行ReduceMin计算，可以通过设置mask参数的方法掩掉冗余数据。针对每行数据的处理步骤为：

- 使用DataCopy从Global搬运16个half数据到Local1中；
- 对归约计算的目的操作数Local2清零，如使用Duplicate等；
- 进行归约操作，将ReduceMin的mask模式设置为前4个数据有效，从而掩掉冗余数据。

图 3-24 使用 mask 掩掉脏数据



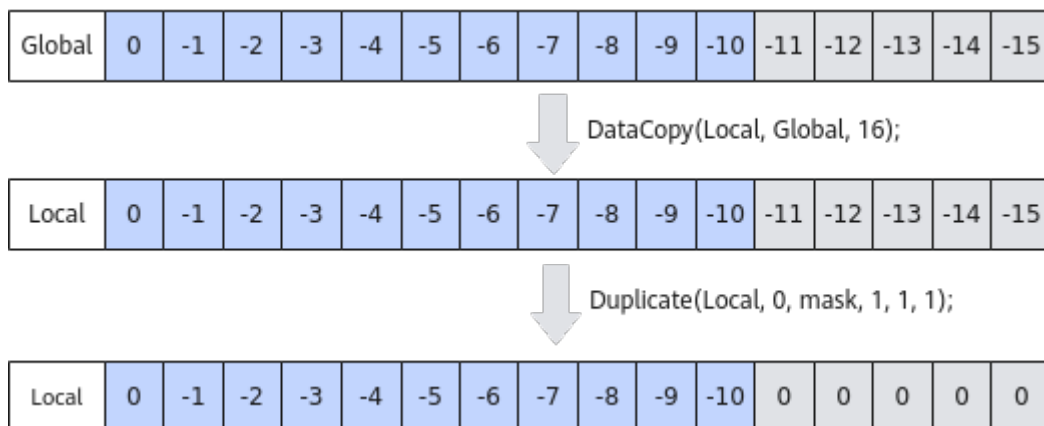
- 通过Duplicate逐行清零。

如下图所示，对于搬入后的非对齐数据，逐行进行Duplicate清零处理，步骤为：

- 使用DataCopy从Global搬运16个half数据到Local中；
- 使用基础API Duplicate，按照如下方式设置mask值，控制仅后5个元素位置有效，将冗余数据填充为0。

```
uint64_t mask0 = ((uint64_t)1 << 16) - ((uint64_t)1 << 11);  
uint64_t mask[2] = {mask0, 0};
```

图 3-25 通过 Duplicate 逐行清零

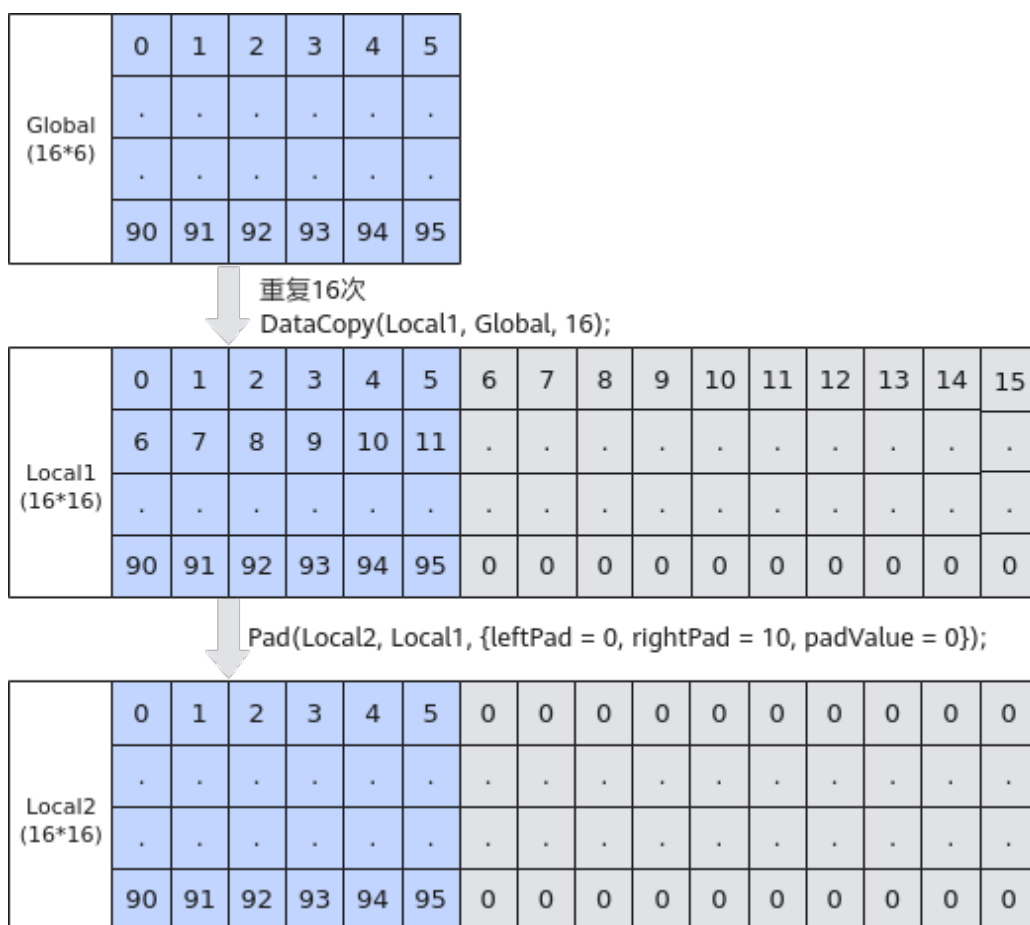


- 通过Pad一次性清零。

如下图所示，假设输入数据的shape为 $16 * 6$ ，搬入Local后大小为 $16 * 16$ ，每行都包含冗余数据，逐行清零性能较差，可以使用Pad一次性清零，步骤为：

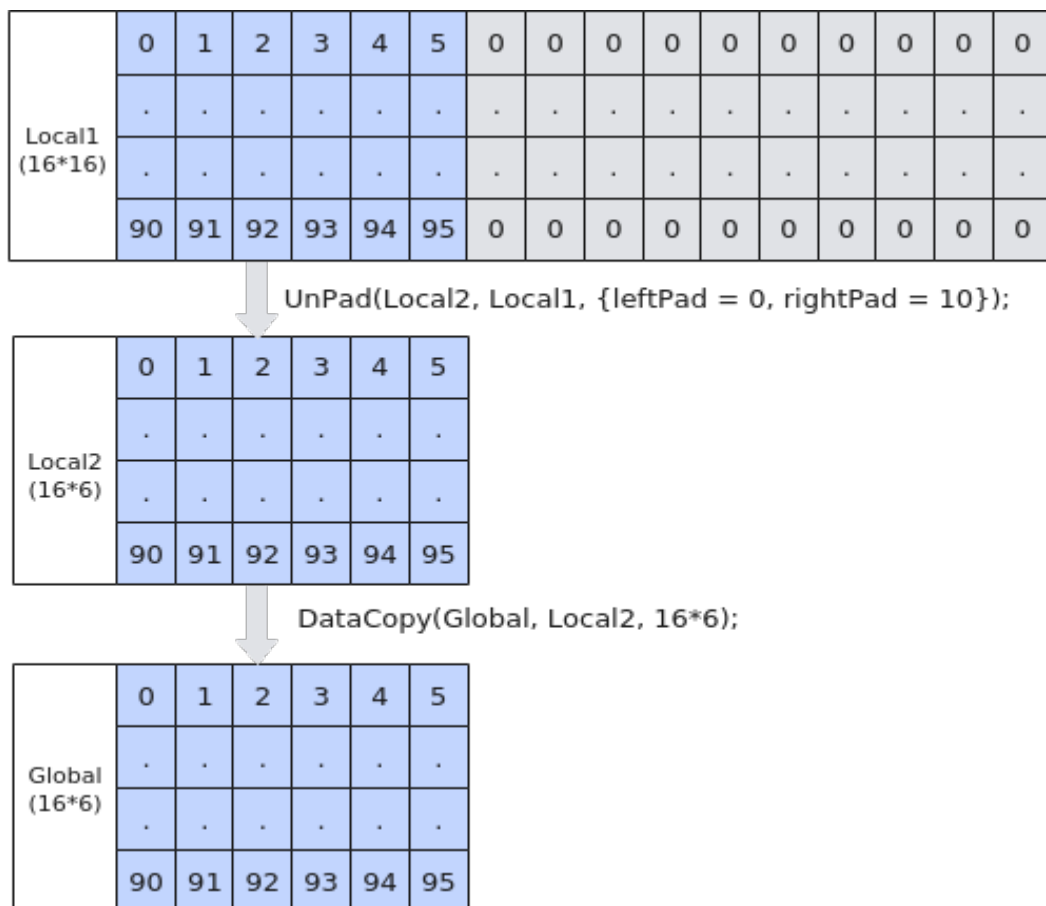
- 将 $16 * 6$ 的数据从GM上逐行搬入UB后，每行有6个有效数据；
- 使用Pad接口将冗余数据位置填充为0。（对应Pad接口使用场景为：tensor的width已32B对齐，但是有部分冗余数据）。

图 3-26 通过 Pad 一次性清零



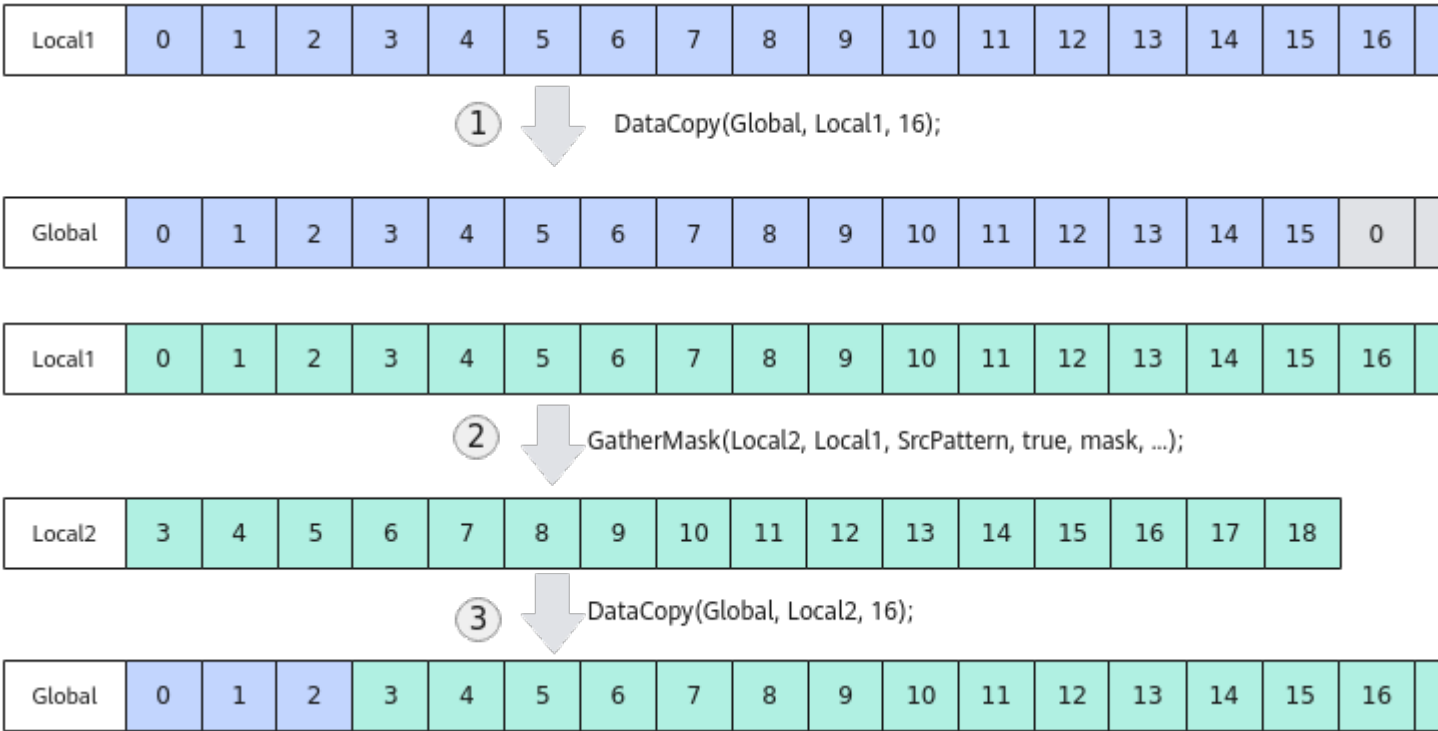
- 使用UnPad接口去除冗余数据后搬出。
如下图所示，Local内存大小为16*16，每行中只有前6个数为有效数据，要搬出的有效数据16 * 6满足32B对齐，可以使用UnPad接口去除冗余数据并完整搬出。
步骤如下：
 - 使用UnPad高阶API去除冗余值；
 - 使用DataCopy搬运出连续的16 * 6个half数据到Global中。

图 3-27 使用 UnPad 接口去除冗余数据后搬出



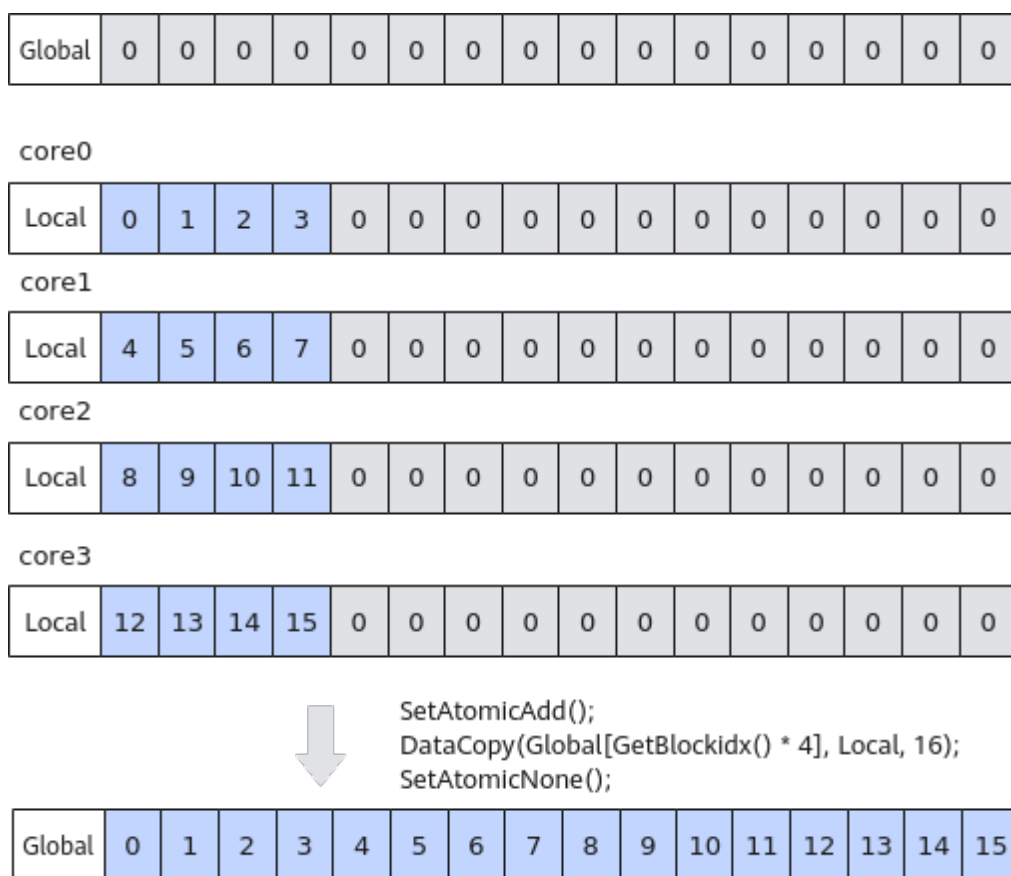
- 使用GatherMask收集有效数据后搬出。
如下图所示，为搬出19个half数据到Global中，有16-18这3个数据的搬运无法满足对齐要求，使用GatherMask对有效数据进行重新收集，收集3-18这16个数据并搬出。步骤如下：
 - 完整拷贝前16个half（32B）数据到Global中；
 - 使用GatherMask接口，将Local1[3]~[18]的数Gather到Local2中，Local2从对齐地址开始；
 - 从Local2中搬运Gather的数据（32B整数倍）到Global中。

图 3-28 使用 GatherMask 收集有效数据后搬出。



- 带冗余数据搬出
如下图所示，有4个核参与计算，每个核拷贝出4个数，每个核上拷贝的数据长度不满足32字节对齐，采用将冗余数据一起搬出的方式，步骤如下：
 - a. 将目标Global完整清零，可以通过在host清零或者在kernel侧用UB覆盖的方式处理；
 - b. 将本核内的Local数据，除了要搬出的4个有效数据，其余冗余部分清零（使用Duplicate接口）；
 - c. 使用原子累加的方式拷贝到Global，原子累加结合冗余数据清零，确保不会出现数据踩踏。

图 3-29 带冗余数据搬出



样例介绍

- 样例一：冗余数据参与计算+使用GatherMask收集有效数据后搬出。

本样例中展示了shape为128 * 18的tensor进行Abs计算的算子实现。针对每行数据的处理方案如下：

搬入后，每行数据的后14个数为冗余数据。Abs接口入参BLOCKLEN_CEIL为32个数，是18个数进行32字节对齐后的结果，有14个冗余数据参与计算。

```
AscendC::Abs(outputLocal, inputLocal, BLOCKLEN_CEIL); // main calculation
```

计算完成后，通过GatherMask的bufPattern入参控制收集18个数中的后16个数。

```
uint16_t tmpValue = 0;  
AscendC::Duplicate<uint16_t>(bufPattern, tmpValue, 16);  
bufPattern.SetValue(0, 0b1111111111111100); // select the last 14 elements of the first 16 positions  
bufPattern.SetValue(1, 0b0000000000000011); // select the first 2 elements of the next 16 positions  
uint32_t mask = 32;  
uint64_t rsvdCnt = 0;  
AscendC::LocalTensor<half> tailLocal = outQueueTail.AllocTensor<half>();  
AscendC::GatherMask(tailLocal, outputLocal, bufPattern, true, mask, {1, 1, 8, 8}, rsvdCnt);
```

首先使用DataCopy搬运前16个数，然后搬运后16个数，中间的14个数存在重复搬运。注意：因为DataCopy的目的地址存在重叠所以需要通过PipeBarrier添加流水同步。

```
uint32_t copyLenMain = TILE_LENGTH * sizeof(half) / 32 * 32 / sizeof(half);  
uint32_t offsetMain = progress * TILE_LENGTH;  
AscendC::DataCopy(dstGlobal[offsetMain], outputLocal, copyLenMain);  
AscendC::PipeBarrier<PIPE_MTE3>();  
uint32_t tailLen = 32 / sizeof(half);  
uint32_t offsetTail = offsetMain + (TILE_LENGTH - tailLen);  
AscendC::DataCopy(dstGlobal[offsetTail], tailLocal, tailLen);
```

搬入时要保证32字节对齐，所以要将输入的最后一行补齐到32字节对齐，防止访问非法数据，main.cpp中对GM上输入的长度的定义如下：

```
size_t inputByteSize = 2318 * sizeof(int16_t); // 2318 = 2304 + 32 - 18
size_t outputByteSize = 2304 * sizeof(int16_t);
```

- 样例二：通过Duplicate逐行清零+带冗余数据搬出。

本样例中展示了shape为64 * 11的tensor进行Abs计算的算子实现。共使用4个核，每个核处理16 * 11个数据。

搬入后，每行数据的后5个数为冗余数据。通过Duplicate接口对每行数据中的后5个数据进行清零。

```
// mask mode controls only the last 5 elements doing Duplicate
uint64_t mask0 = (1ul << 16) - (1ul << BLOCK_ELEMENT_NUM);
uint64_t mask[2] = {mask0, 0};
for (int32_t i = 0; i < BLOCK_GROUP_NUM; i++) {
    AscendC::Duplicate<half>(inputLocal[i * BLOCKLEN_CEIL], 0, mask, 1, 1, 1); // clear dummy data on
    inputLocal
}
AscendC::Abs(outputLocal, inputLocal, BLOCKLEN_CEIL * BLOCK_GROUP_NUM);
```

搬出时，带冗余数据搬出并开启原子累加，BLOCKLEN_CEIL中包含冗余数据。

```
AscendC::SetAtomicAdd<half>();
for (int32_t i = 0; i < BLOCK_GROUP_NUM; i++) {
    AscendC::DataCopy<half>(dstGlobal[i * BLOCK_ELEMENT_NUM], outputLocal[i * BLOCKLEN_CEIL],
    BLOCKLEN_CEIL);
}
AscendC::SetAtomicNone();
```

所以在初始化时，需要对GM数据进行清零，清零代码如下，示例中多个核都调用Fill接口进行清零，需要调用SyncAll进行核间同步。

```
AscendC::Fill<half>(dstGlobal, blockLength, 0);
```

```
pipe.InitBuffer(inQueue, BUFFER_NUM, BLOCK_GROUP_NUM * BLOCKLEN_CEIL * sizeof(half));
pipe.InitBuffer(outQueue, BUFFER_NUM, BLOCK_GROUP_NUM * BLOCKLEN_CEIL * sizeof(half));
pipe.InitBuffer(syncLocalTbuf, USE_CORE_NUM * DEFAULT_SYNCALL_NEED_SIZE * sizeof(int32_t));
AscendC::LocalTensor<int32_t> SyncLocal = syncLocalTbuf.Get<int32_t>();
AscendC::SyncAll(syncGlobal, SyncLocal, USE_CORE_NUM);
```

搬入时要保证32字节对齐，需要将输入的最后一行补齐到32字节对齐，防止访问非法数据；搬出时带冗余数据搬出，输出的最后一行也需要补齐到32字节对齐。main.cpp中对GM上输入输出的长度的定义如下：

```
// copy in borrow the next (BLOCKLEN_CEIL - BLOCK_ELEMENT_NUM) elements of srcGM
size_t inputByteSize = 709 * sizeof(int16_t);
// copy out atomic add extra (BLOCKLEN_CEIL - BLOCK_ELEMENT_NUM) zeros to dstGM
size_t outputByteSize = 709 * sizeof(int16_t);
```

- 样例三：冗余数据参与计算+使用UnPad接口去除冗余数据后搬出。

本样例中展示了shape为2048 * 14的tensor进行Abs计算的算子实现。共使用8个核，每个核处理256 * 14个数据。

搬入后，每行数据的后2个数为冗余数据。Abs接口入参BLOCK_GROUP_NUM * BLOCKLEN_CEIL为连续的16行数据，每行16个数，每行的冗余数据参与计算。

```
AscendC::Abs(inputLocal, inputLocal, BLOCK_GROUP_NUM * BLOCKLEN_CEIL); // main calculation
```

计算后，使用UnPad接口去除冗余数据后再搬出，通过unPadParams.rightPad参数控制去除每行最后的2个冗余数据。

```
unPadParams.rightPad = BLOCKLEN_CEIL - BLOCK_ELEMENT_NUM; // delete 2 dummy half each row
AscendC::UnPad<half>(outputLocal, inputLocal, unPadParams, this->tiling);
```

注意：UnPad接口需要传入tiling参数。abs_unpad_tiling.cpp中关键计算过程如下：

```
AscendC::GetUnPadMaxMinTmpSize(*ascendcPlatform, srcShape, sizeof(int16_t), tmpMaxSize,
tmpMinSize);
optiling::UnPadTiling tilingData;
AscendC::UnPadTilingFunc(srcShape, tmpMaxSize, sizeof(int16_t), tilingData);
```

main.cpp中tiling参数需要通过核函数的入参传入到kernel侧，供UnPad高阶API使用。

```
ACLRT_LAUNCH_KERNEL(abs_unpad_custom)(numBlocks, stream, xDevice, zDevice, workspaceDevice, tilingDevice);
```

搬入时要保证32字节对齐，所以要将输入的最后一行补齐到32字节对齐，防止访问非法数据，main.cpp中对GM上输入的长度的定义如下。

```
// 28674 is TOTAL_LENGTH + (BLOCKLEN_CEIL - BLOCK_ELEMENT_NUM)
// 28672 is TOTAL_LENGTH
// copy in borrow the next (BLOCKLEN_CEIL - BLOCK_ELEMENT_NUM) elements of srcGM
uint32_t oriLength = 28672;
uint32_t colNum = 14;
uint32_t maxColNum = 32 / sizeof(uint16_t);
uint32_t padLength = oriLength + maxColNum - colNum;
size_t inputByteSize = padLength * sizeof(int16_t);
size_t outputByteSize = oriLength * sizeof(int16_t);
```

- 示例四：通过Pad一次性清零+带冗余数据搬出。

本样例中展示了shape为2048 * 7的tensor进行Abs计算的算子实现。共使用8个核，每个核处理256 * 7个数据。

搬入后，每行数据的后9个数为冗余数据。每个核上通过Pad接口将256 * 9的冗余数据块整体清零后参与Abs计算。

```
AscendC::PadParams padParams = {0, BLOCKLEN_CEIL - BLOCK_ELEMENT_NUM, 0};
AscendC::Pad(outputLocal, inputLocal, padParams, this->tiling);
AscendC::Abs(outputLocal, outputLocal, BLOCK_GROUP_NUM * BLOCKLEN_CEIL); // main calculation
```

计算后带冗余数据搬出的代码解释和[样例二](#)一致。

注意：Pad接口需要传入tiling参数。abs_pad_tiling.cpp中关键计算过程如下：

```
AscendC::GetPadMaxMinTmpSize(srcShape, sizeof(int16_t), tmpMaxSize, tmpMinSize);
optiling::PadTiling tilingData;
AscendC::PadTilingFunc(srcShape, oriSrcShape, tmpMaxSize, sizeof(int16_t), tilingData);
```

main.cpp中tiling参数需要通过核函数的入参传入到kernel侧，供Pad高阶API使用。

```
ACLRT_LAUNCH_KERNEL(abs_pad_custom)(numBlocks, stream, xDevice, zDevice, workspaceDevice, tilingDevice);
```

搬入时要保证32字节对齐，需要将输入的最后一行补齐到32字节对齐，防止访问非法数据；搬出时带冗余数据搬出，输出的最后一行也需要补齐到32字节对齐。main.cpp中对GM上输入输出的长度的定义如下：

```
// 14336 is the length of input data
uint32_t oriLength = 14336;
// we must allocate more space to prevent invalid address access
uint32_t padLength = oriLength + shapePad[1] - shapeUsed[1];
size_t inputByteSize = padLength * sizeof(int16_t);
size_t outputByteSize = padLength * sizeof(int16_t);
// however, original length must be used when output to file
size_t outputFileSize = oriLength * sizeof(int16_t);
```

- 样例五：使用mask掩掉冗余数据+带冗余数据搬出。

本样例中展示了shape为16 * 4的tensor每行数据进行ReduceMin计算的算子实现。共使用4个核，每个核处理4 * 4个数据。

搬入后，每行数据的后12个数为冗余数据。通过ReduceMin的入参Mask控制只有前4个数参与计算。

```
uint64_t Mask0 = ((uint64_t)1 << BLOCK_ELEMENT_NUM) - 1; // mask mode controls only the first 4
elements do ReduceMin calculation
uint64_t Mask[2] = {Mask0, 0};
// main calculation
for (int i = 0; i < BLOCK_GROUP_NUM; i++) {
    AscendC::ReduceMin<half>(outputLocal[i * BLOCKLEN_CEIL], inputLocal[i * BLOCKLEN_CEIL],
workLocal, Mask, 1, 8, false);
}
```



```
outQueue.EnQue<half>(outputLocal);  
inQueue.FreeTensor(inputLocal);
```

计算后带冗余数据搬出的代码解释和**样例二**一致。

搬入时要保证32字节对齐，需要将输入的最后一行补齐到32字节对齐，防止访问非法数据；搬出时带冗余数据搬出，输出的最后一行也需要补齐到32字节对齐。
main.cpp中对GM上输入输出的长度的定义如下：

```
// copy in borrow the next (BLOCKLEN_CEIL - BLOCK_ELEMENT_NUM) elements of srcGM  
size_t inputByteSize = 76 * sizeof(int16_t);  
// copy out atomic add extra (BLOCKLEN_CEIL - BLOCK_ELEMENT_NUM) zeros to dstGM  
size_t outputByteSize = 76 * sizeof(int16_t);
```

3.3.3 矩阵编程（高阶 API）

3.3.3.1 基础知识

说明

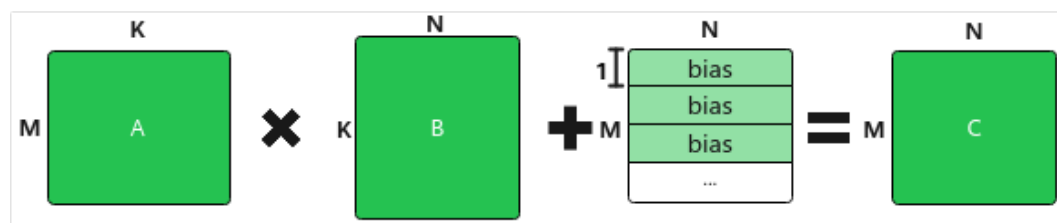
本节内容为使用高阶API进行矩阵乘法的编程指导。使用高阶API进行实际的矩阵编程时，需要通过API参考确认接口支持的产品型号。

矩阵乘法概述

Matmul的计算公式为： $C = A * B + \text{bias}$ ，其示意图如下。

- A、B为源操作数，A为左矩阵，形状为[M, K]；B为右矩阵，形状为[K, N]。
- C为目的操作数，存放矩阵乘结果的矩阵，形状为[M, N]。
- bias为矩阵乘偏置，形状为[1, N]。对A*B结果矩阵的每一行都采用该bias进行偏置。

图 3-30 Matmul 矩阵乘示意图



矩阵乘法数据流

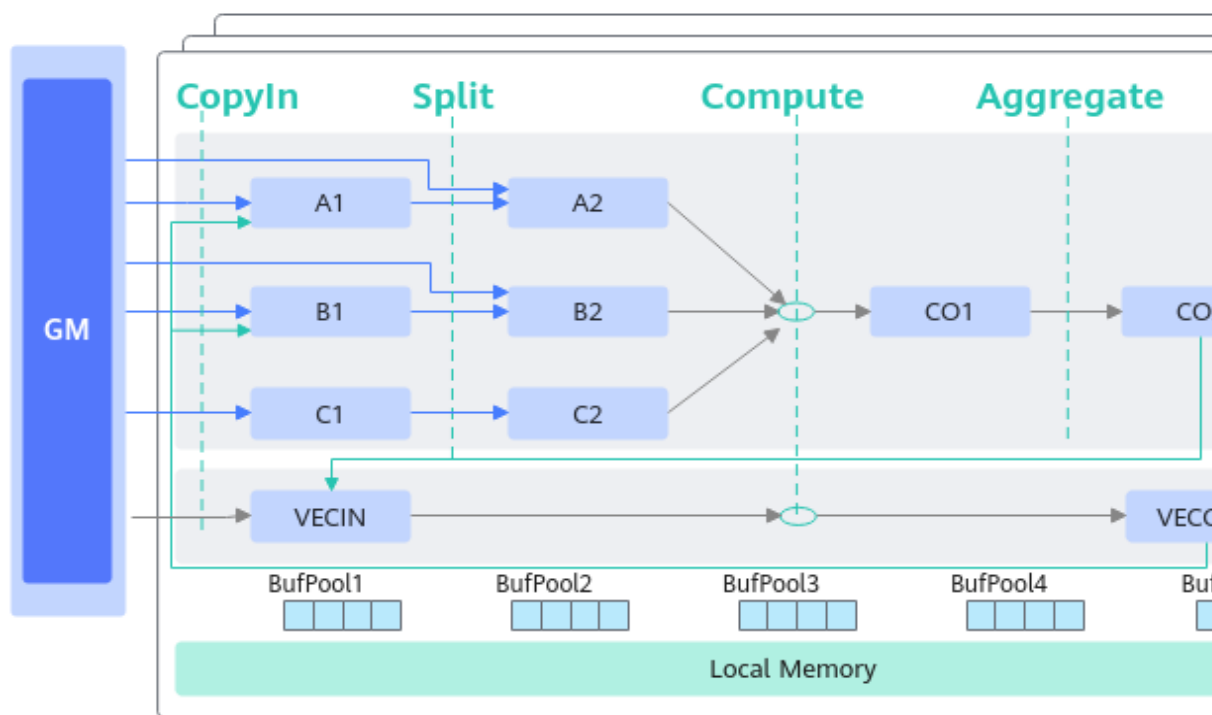
在了解矩阵乘法数据流之前，需要先回顾一下几个重要的存储**逻辑位置**的概念：

- 搬入数据的存放位置：A1，用于存放整块A矩阵，可类比CPU多级缓存中的二级缓存；
- 搬入数据的存放位置：B1，用于存放整块B矩阵，可类比CPU多级缓存中的二级缓存；
- 搬入数据的存放位置：C1，用于存放整块的矩阵乘偏置Bias矩阵，可类比CPU多级缓存中的二级缓存；
- 搬入数据的存放位置：A2，用于存放切分后的小块A矩阵，可类比CPU多级缓存中的一级缓存；

- 搬入数据的存放位置：B2，用于存放切分后的小块B矩阵，可类比CPU多级缓存中的一级缓存；
- 搬入数据的存放位置：C2，用于存放切分后的小块矩阵乘偏置Bias矩阵，可类比CPU多级缓存中的一级缓存；
- 结果数据的存放位置：CO1，用于存放小块结果C矩阵，可理解为Cube Out；
- 结果数据的存放位置：CO2，用于存放整块结果C矩阵，可理解为Cube Out；
- 搬入数据的存放位置：VECCALC，一般在计算需要临时变量时使用此位置。

矩阵乘法数据流指矩阵乘的输入输出在各存储位置间的流向。逻辑位置的数据流向如下图所示（为了简化描述，没有列出bias）：

- A矩阵从输入位置到A2的数据流如下（输入位置可以是GM或者VEECOUT）：GM->A2，GM->A1->A2；VEECOUT->A1->A2。
由于A1比A2的空间更大，数据从GM或VEECOUT可以先搬入A1进行缓存，待该数据执行Cube计算前，数据直接从A1搬入A2，这样在搬运大量数据时可以减少计算前的等待时间，提升性能，只有在搬运数据较少的场景才可能使用GM->A2的数据流。
- B矩阵从输入位置到B2的数据流如下（输入位置可以是GM或者VEECOUT）：GM->B2，GM->B1->B2；VEECOUT->B1->B2。
由于B1比B2的空间更大，数据从GM或VEECOUT可以先搬入B1进行缓存，待该数据执行Cube计算前，数据直接从B1搬入B2，这样在搬运大量数据时可以减少计算前的等待时间，提升性能，只有在搬运数据较少的场景才可能使用GM->B2的数据流。
- 完成 $A2*B2=CO1$ 计算。
- CO1数据汇聚到CO2：CO1->CO2。
- 从CO2到输出位置（输出位置可以是GM或者VEECIN）：CO2->GM/CO2->VEECIN。



数据格式

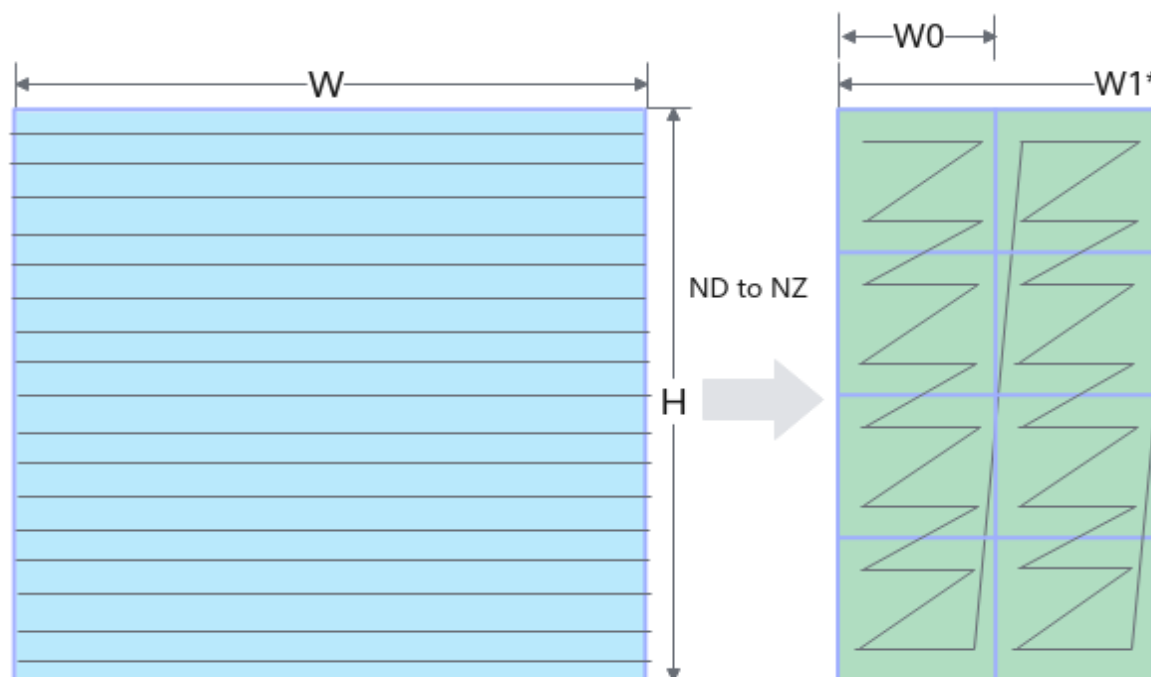
在完成Matmul矩阵乘法时，主要涉及到两种分形格式ND和NZ。

- ND：普通格式，N维张量。
- NZ：为满足AICore中Cube计算单元高性能计算的需要，引入该特殊格式。

ND -> NZ的变换过程为：

$(..., N, H, W) \rightarrow \text{pad} \rightarrow (..., N, H1*H0, W1*W0) \rightarrow \text{reshape} \rightarrow (..., N, H1, H0, W1, W0) \rightarrow \text{transpose} \rightarrow (..., N, W1, H1, H0, W0)$

如下图所示（W，H）大小的矩阵被分为（H1*W1）个分形，按照列优先排布，形状如N字形；每个分形内部有（H0*W0）个元素，按照行优先排布，形状如z字形。所以这种数据格式称为NZ（大N小Z）格式。



下面我们再通过一个具体的例子来深入理解ND和NZ格式的数据排布区别。假设分形格式为2*2，如下图所示4*4的矩阵，ND和NZ格式存储两种情况下，数据在内存中的排布格式分别为：

ND: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15

NZ: 0, 1, 4, 5, 8, 9, 12, 13, 2, 3, 6, 7, 10, 11, 14, 15

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

关于矩阵ND到NZ格式转换的样例请参考[Matmul输入矩阵ND到NZ格式转换的算子样例](#)。

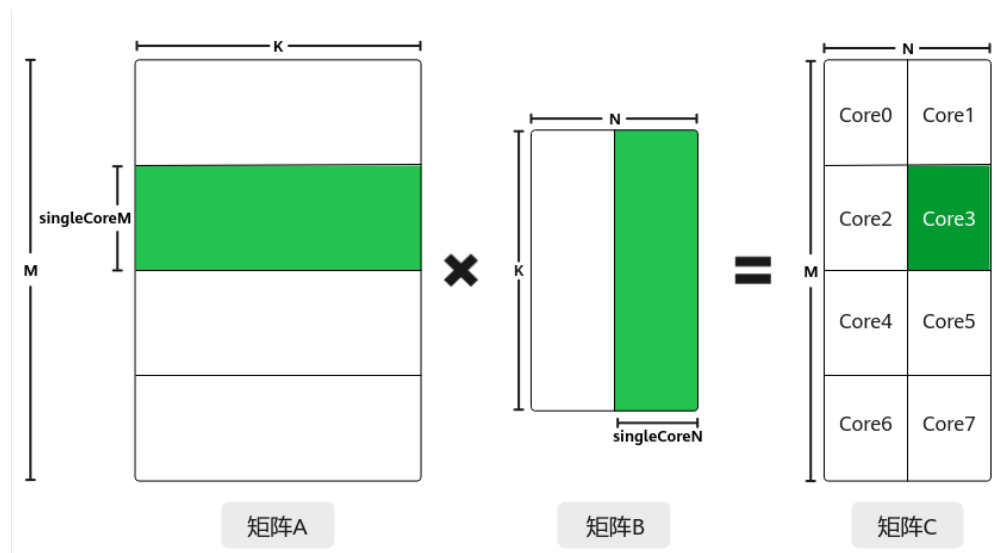
数据分块（Tiling）

- 多核切分

为了实现多核并行，需要将矩阵数据进行切分，分配到不同的核上进行处理。切分策略如下图所示：

- 对于A矩阵，沿着M轴进行切分，切分成多份的singleCoreM，单核上处理SingleCoreM * K大小的数据。
- 对于B矩阵，沿着N轴进行切分，切分成多份的singleCoreN，单核上处理K * SingleCoreN大小的数据。
- 对于C矩阵，SingleCoreM * K大小的A矩阵和K * SingleCoreN大小的B矩阵相乘得到SingleCoreM * SingleCoreN大小的C矩阵，即为单核上输出的C矩阵大小。

比如，下图中共有8个核参与计算，将A矩阵沿着M轴划分为4块，将B矩阵沿着N轴切分为两块，单核上仅处理某一分块（比如图中绿色部分为core3上参与计算的数据）：SingleCoreM * K大小的A矩阵分块和SingleCoreN * K大小的B矩阵分块相乘得到SingleCoreM * SingleCoreN大小的C矩阵分块。



- 核内切分

大多数情况下，Local Memory的存储，无法完整的容纳算子的输入与输出，需要每次搬运一部分输入进行计算然后搬出，再搬运下一部分输入进行计算，直到得到完整的最终结果，也就是需要做核内的输入切分。切分的策略如下所示：

- 对于A矩阵，沿M轴进行切分，将singleCoreM切分成多份的baseM，切分成的份数对应图示的mlter；沿K轴进行切分，切分成多份的baseK。
- 对于B矩阵，沿N轴进行切分，将singleCoreN切分成多份的baseN，切分成的份数对应图示的nlter；沿K轴进行切分，切分成多份的baseK。
- 对于C矩阵，A矩阵中baseM*baseK大小的分块和B矩阵中baseK*baseN大小的分块相乘并累加，得到C矩阵中对应位置baseM*baseN大小的分块。比如，图中结果矩阵中的绿色矩阵块5是通过如下的累加过程得到的： $a*a+b*b+c*c+d*d+e*e+f*f$ 。

- iterateOrder: 一次Iterate迭代计算出[baseM, baseN]大小的C矩阵分片。Iterate完成后，Matmul会自动偏移下一次Iterate输出的C矩阵位置，iterateOrder表示自动偏移的顺序。

- 0代表先往M轴方向偏移再往N轴方向偏移；

- 1代表先往N轴方向偏移再往M轴方向偏移。

在上图的示例中，iterateOrder取值为0。

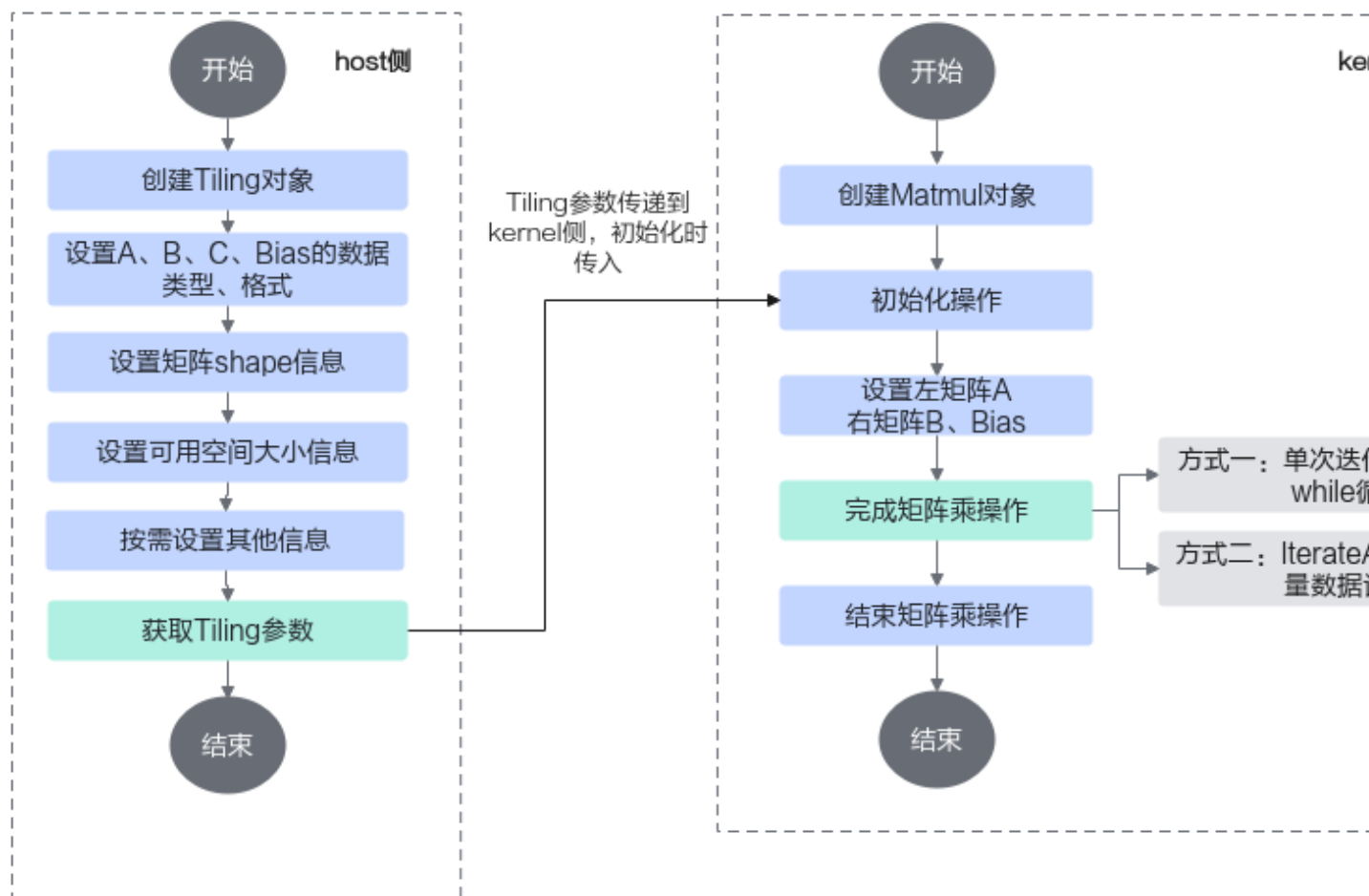
- depthA1, depthB1: A1、B1上存储的矩阵片全载A2/B2的份数，A2、B2存储大小分别是baseM * baseK, baseN * baseK, 即depthA1是A1矩阵切片含有baseM * baseK块的个数，depthB1是B1矩阵切片含有baseN * baseK块的个数。
- stepM, stepN: stepM为左矩阵在A1中缓存的buffer M方向上baseM的倍数，stepN为右矩阵在B1中缓存的buffer N方向上baseN的倍数。
- stepKa, stepKb: stepKa为左矩阵在A1中缓存的buffer K方向上baseK的倍数，stepKb为右矩阵在B1中缓存的buffer K方向上baseK的倍数。

3.3.3.2 算子实现

实现流程

上文介绍了Matmul矩阵乘的数据切分方案和数据流。Ascend C提供一组Matmul高阶API，封装了这些常用的切分和数据搬运、计算的算法逻辑，方便用户快速实现Matmul矩阵乘法的运算操作。开发者在host侧通过调用API自动获取Tiling参数，该参数传递到kernel侧后，在初始化操作时传入，通过几个简单的API即可完成矩阵乘操作。完整样例请参考[LINK](#)。

图 3-31 矩阵编程流程示意图



host侧自动获取Tiling参数的关键步骤介绍如下：

步骤1 创建Tiling对象。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
matmul_tiling::MatmulApiTiling cubeTiling(ascendcPlatform);
```

传入硬件平台信息创建PlatformAscendC对象，然后创建Tiling对象，硬件平台信息可以通过GetPlatformInfo获取。

步骤2 设置A、B、Bias的内存逻辑位置、格式和数据类型。

```
cubeTiling.SetAType(AscendC::TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT16);
cubeTiling.SetBType(AscendC::TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT16);
cubeTiling.SetCType(AscendC::TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT);
cubeTiling.SetBiasType(AscendC::TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT);
```

步骤3 设置矩阵shape信息。

```
cubeTiling.SetShape(M, N, K);
cubeTiling.SetOrgShape(M, N, K); // 设置原始完整的形状M、N、K
```

步骤4 设置可用空间大小信息。

设置Matmul计算时可用的L1 Buffer/L0C Buffer/Unified Buffer空间大小，-1表示AI处理器对应Buffer的大小。

```
cubeTiling.SetBufferSpace(-1, -1, -1);
```

步骤5 按需设置其他参数，比如设置bias参与计算。

```
cubeTiling.EnableBias(true);
```

步骤6 获取Tiling参数。

```
MatmulCustomTilingData tiling;  
if (cubeTiling.GetTiling(tiling.cubeTilingData) == -1){  
    return ge::GRAPH_FAILED;  
}
```

步骤7 Tiling参数的序列化保存等其他操作。

----结束

kernel侧使用Matmul API矩阵乘运算的具体步骤如下：

步骤1 创建Matmul对象

创建Matmul对象的示例如下：

- 纯Cube模式（只有矩阵计算）场景下，建议在代码中定义ASCENDC_CUBE_ONLY宏，避免额外的性能开销。本节内容以纯Cube模式举例。
- 默认为MIX模式（包含矩阵计算和矢量计算），该场景下通常不定义ASCENDC_CUBE_ONLY宏，如果在程序中使用了ASCENDC_CUBE_ONLY宏，则必须使用ASCEND_IS_AIC宏和ASCEND_IS_AIV宏将Cube计算和Vector计算隔离开，更多内容请参考[3.3.5 融合算子编程](#)。

```
// 纯Cube模式（只有矩阵计算）场景下，需要设置该代码宏，并且必须在#include "lib/matmul_intf.h"之前设置  
#define ASCENDC_CUBE_ONLY  
#include "lib/matmul_intf.h"  
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half> aType;  
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half> bType;  
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, float> cType;  
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, float> biasType;  
AscendC::Matmul<aType, bType, cType, biasType> mm;
```

创建对象时需要传入A、B、C、Bias的参数类型信息，类型信息通过MatmulType来定义，包括：内存逻辑位置、数据格式、数据类型。

步骤2 初始化操作。

```
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling); // 初始化
```

说明

Matmul高阶API内部实现时需要使用系统workspace（即对应本步骤中的GetSysWorkSpacePtr接口），开发者需要自行申请系统workspace的空间：

- 在host侧Tiling实现时，设置总的workspace的数值大小（包含用户workspace和系统workspace），workspace空间由框架来申请并管理。系统workspace的空间大小通过GetLibApiWorkSpaceSize获取。
size_t userWorkspaceSize = 0;
size_t systemWorkspaceSize = static_cast<size_t>(ascendcPlatform.GetLibApiWorkSpaceSize());
size_t *currentWorkspace = context->GetWorkspaceSizes(1);
currentWorkspace[0] = userWorkspaceSize + systemWorkspaceSize;
- 若算子工程不是[自定义算子工程](#)，也不是带有HAVE_WORKSPACE编译宏的[Kernel直调算子工程](#)，框架不会自动设置workspace，需要在kernel侧的Matmul初始化前，通过SetSysWorkSpace设置系统workspace。
// 使用Matmul时必须设置workspace空间
SetSysWorkspace(workspace);
if (GetSysWorkSpacePtr() == nullptr) {
 return;
}

步骤3 设置左矩阵A、右矩阵B、Bias。

```
mm.SetTensorA(gm_a); // 设置左矩阵A  
mm.SetTensorB(gm_b); // 设置右矩阵B  
mm.SetBias(gm_bias); // 设置Bias
```

步骤4 完成矩阵乘操作。

- 调用Iterate完成单次迭代计算，叠加while循环完成单核全量数据的计算。Iterate方式，可以自行控制迭代次数，完成所需数据量的计算，方式比较灵活。

```
while (mm.Iterate()) {  
    mm.GetTensorC(gm_c);  
}
```

- 调用IterateAll完成单核上所有数据的计算。IterateAll方式，无需循环迭代，使用比较简单。

```
mm.IterateAll(gm_c);
```

步骤5 结束矩阵乘操作。

```
mm.End();
```

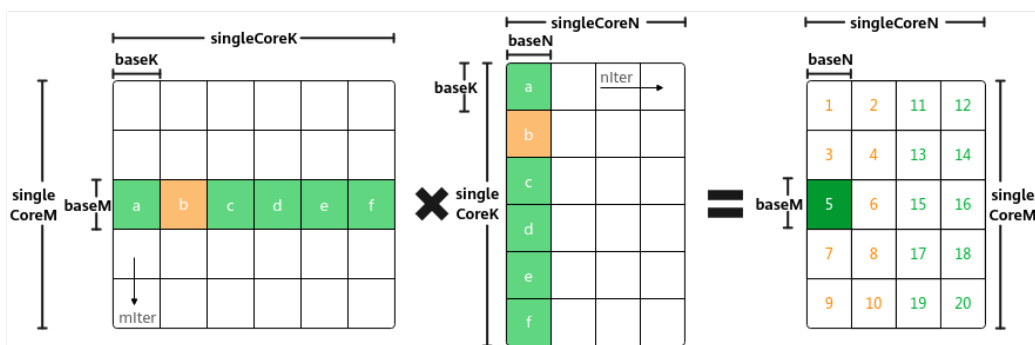
----结束

设置 Shape 信息

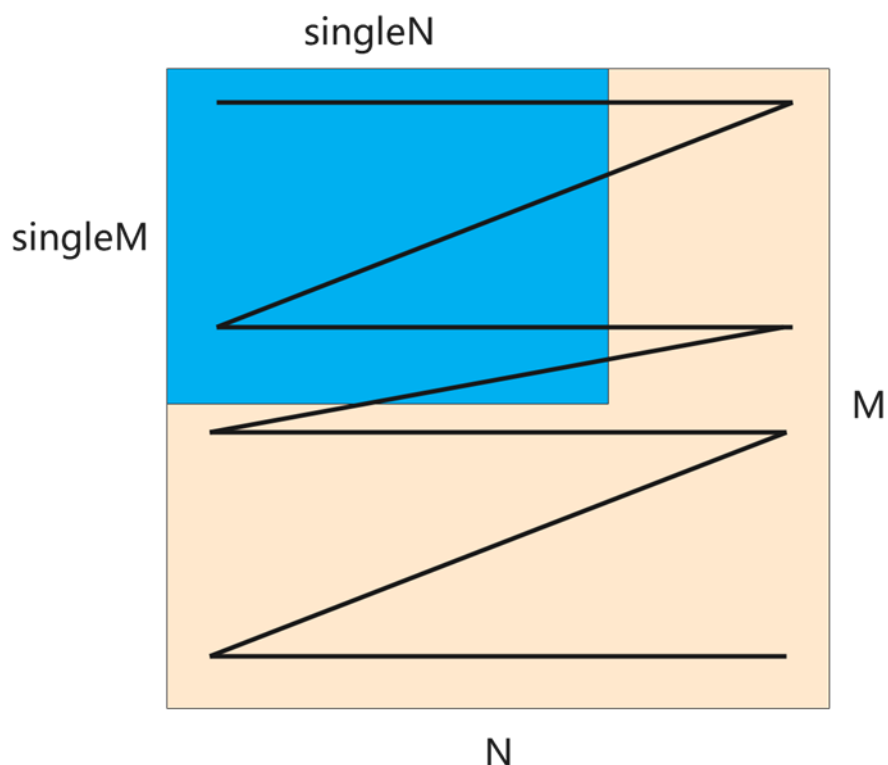
在实现Host Tiling时可以设置Shape信息，用于Tiling计算；kernel侧运行时也可以修改部分Shape信息，用于尾块设置、Matmul复用（多个Matmul计算复用一個Matmul对象）等场景。本节对涉及到的Shape概念进行介绍，并给出host侧和kernel侧设置Tiling信息的指导。

- orgShape: M、N、K
- singleCoreShape: singleCoreM、singleCoreN、singleCoreK
- singleShape: singleM、singleN、singleK
- baseShape: baseM、baseN、baseK

通过[数据分块（Tiling）](#)的介绍我们已经了解了orgShape(M、N、K)，singleCoreShape(singleCoreM、singleCoreN、singleCoreK)，baseShape(baseM、baseN、baseK)的概念，如下图所示：



除此之外，单核的Matmul Tiling时，实际参与Matmul计算的shape可以是原始shape中的一部分，singleM, singleN, singleK用于表达实际参与Matmul计算的shape，如下图所示。在单核的情况下，singleM, singleN, singleK会透传给singleCoreM, singleCoreN, singleCoreK。



- Kernel运行时设置
 - SetTail、SetSingleShape都是运行时修改singleCoreM、singleCoreN、singleCoreK，处理尾块时使用SetTail，Matmul复用（多个Matmul计算复用 一个Matmul对象）的场景可以使用SetSingleShape重新设置。
 - SetOrgShape是运行时修改M、N、K，Matmul复用的场景可以使用SetOrgShape重新设置。
- 单核Tiling时设置
 - SetOrgShape（必选）：设置M、N、K
 - SetShape（非必选）：设置singleM、singleN、singleK，等同于设置singleCoreM、singleCoreN、singleCoreK
 - SetFixSplit（非必选）：设置baseM、baseN、baseK
- 多核Tiling时设置
 - SetOrgShape（必选）：设置M、N、K
 - SetShape（非必选）：设置singleM、singleN、singleK
 - SetFixSplit（非必选）：设置baseM、baseN、baseK
 - SetSingleShape(非必选)：设置singleCoreM、singleCoreN、singleCoreK
 - SetSingleRange(非必选)：设置singleCoreM、singleCoreN、singleCoreK的范围

设置 format 格式

创建Matmul对象时需要传入A、B、C、Bias的参数类型信息，类型信息通过MatmulType来定义，包括：内存逻辑位置、数据格式、数据类型。示例如下：

```
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half> aType;
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half> bType;
```

```
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, float> cType;  
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, float> biasType;  
AscendC::Matmul<aType, bType, cType, biasType> mm;
```

针对数据格式，包括CubeFormat::ND, CubeFormat::NZ, CubeFormat::ND_ALIGN三种，ND和NZ格式在[数据格式](#)章节已经介绍，ND_ALIGN格式的介绍请参考[数据排布格式](#)。

3.3.3.3 特性场景

3.3.3.3.1 Matmul 特性介绍

除了前述[基础知识](#)和[算子实现](#)中介绍的基本计算能力外，Matmul矩阵编程还提供了适用于不同场景的处理能力及多种功能，具体场景和功能列于下表中，详细内容请见后续章节。

表 3-4 Matmul 功能特性表

特性描述	功能简介
多核对齐切分	在多核场景中，支持将矩阵数据沿M、N、K轴切分，满足M能被singleCoreM整除、N能被singleCoreN整除、K能被singleCoreK整除的对齐场景时的处理方式，从而实现多核并行计算矩阵乘。
多核非对齐切分	在多核场景中，支持将矩阵数据沿M、N、K轴切分。当出现M不能被singleCoreM整除、或N不能被singleCoreN整除、或K不能被singleCoreK整除的非对齐场景（即尾块场景）时的处理方式。
异步场景处理	MIX场景（包含矩阵计算和矢量计算）下不需要等待矩阵乘计算完成，先执行其它计算。
矩阵乘输出的量化/反量化	将矩阵乘的计算结果从CO1搬出到Global Memory时，对矩阵元素执行数据量化或反量化操作。
自定义数据搬入搬出	自定义矩阵乘计算前后的数据搬运函数。本功能支持用户实现左矩阵A、右矩阵B从Global Memory分别自定义搬入到A1、B1的过程，输出C矩阵从CO1自定义搬出到Global Memory的过程。
矩阵乘输出的Channel拆分	矩阵乘输出的Channel拆分，又称ChannelSplit。指float数据类型、NZ数据格式的输出C矩阵按照16*8的分形大小存储。
矩阵向量乘	矩阵向量乘即GEMV，指矩阵乘计算中M=1，K>1的场景，即对形状为(1, K)的左矩阵A实现矩阵乘计算。
4:2稀疏矩阵乘	4:2稀疏矩阵乘，又称Sparse Matmul。指对稀疏左矩阵A和4:2稠密化的右矩阵B实现矩阵乘计算。
上/下三角矩阵乘	忽略位于矩阵中下三角或上三角位置的元素的计算，实现矩阵中上三角或下三角位置的元素的矩阵乘计算。
TSCM输入的矩阵乘	对内存逻辑位置为TSCM的左矩阵A或右矩阵B实现矩阵乘计算。

特性描述	功能简介
矩阵乘输出的N方向对齐	矩阵乘输出的N方向对齐，又称ND_ALIGN格式输出。指对数据格式为ND_ALIGN的输出C矩阵实现N方向32字节对齐的自动补齐及输出。
单次矩阵乘局部输出	单次矩阵乘局部输出，又称Partial Output，指矩阵乘计算时不对单核K方向的计算结果做累加，直接输出计算结果。
AIC和AIV独立运行机制	AIC和AIV独立运行机制，又称双主模式。MIX场景（包含矩阵计算和矢量计算）下AIC核和AIV核独立运行代码，不依赖消息驱动。

表 3-5 BatchMatmul 功能特性表

特性描述	功能简介
Batch Matmul基础功能	Batch Matmul基础功能，支持批量处理Matmul，调用一次IterateBatch接口，计算出多个singleCoreM * singleCoreN大小的C矩阵。
Batch Matmul复用Bias矩阵	每个Batch的Matmul计算复用同一个不带Batch轴的Bias矩阵。

3.3.3.2 多核对齐切分

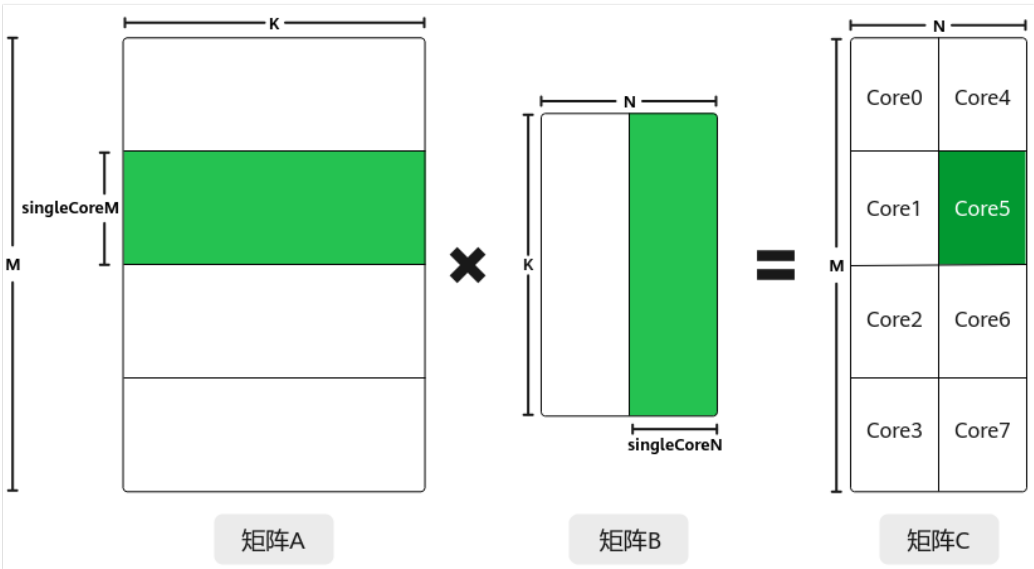
功能介绍

为了实现多核并行，提升计算效率，需要将矩阵数据进行切分，分配到不同的核上进行处理。主要的切分策略有切分K轴和不切分K轴两种。

不切分K轴、仅切分M、N轴的策略如下：

- 对于A矩阵，沿着M轴进行切分，切分成多份的singleCoreM，单核上处理SingleCoreM * K大小的数据。
- 对于B矩阵，沿着N轴进行切分，切分成多份的singleCoreN，单核上处理K * SingleCoreN大小的数据。
- 对于C矩阵，SingleCoreM * K大小的A矩阵和K * SingleCoreN大小的B矩阵相乘得到SingleCoreM * SingleCoreN大小的C矩阵，即为单核上输出的C矩阵大小。

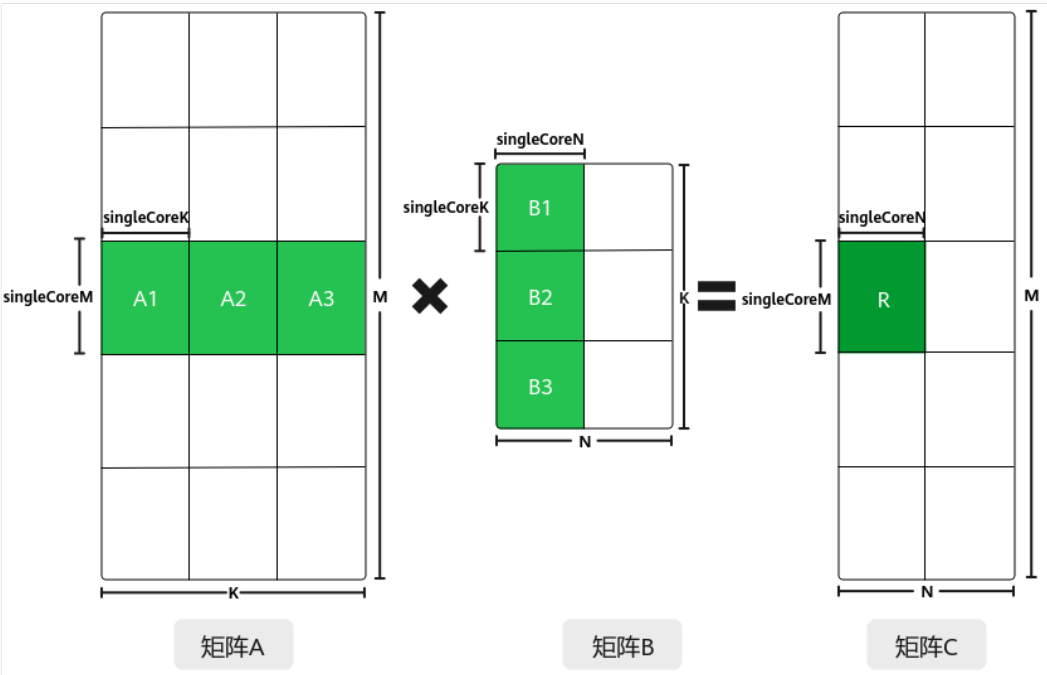
比如，下图中共有8个核参与计算，将A矩阵沿着M轴划分为4块，将B矩阵沿着N轴切分为两块，单核上仅处理某一分块（比如图中绿色部分为core5上参与计算的数据）：SingleCoreM * K大小的A矩阵分块和SingleCoreN * K大小的B矩阵分块相乘得到SingleCoreM * SingleCoreN大小的C矩阵分块。



切分M、N、K轴的策略如下图所示：

- 对于A矩阵，沿着M轴进行切分，切分成多份的singleCoreM，沿着K轴切分，切分成多份的singleCoreK，单核上处理singleCoreM * singleCoreK大小的数据。
- 对于B矩阵，沿着K轴进行切分，切分成多份的singleCoreK，沿着N轴进行切分，切分成多份的singleCoreN，单核上处理singleCoreK * singleCoreN大小的数据。
- 对于C矩阵，singleCoreM * singleCoreK大小的A矩阵与singleCoreK * singleCoreN大小的B矩阵相乘并累加得到singleCoreM * singleCoreN大小的C矩阵分块。

比如下图中，C矩阵中的R矩阵块，是通过A1*B1+A2*B2+A3*B3累加得到的，其中，A1*B1、A2*B2、A3*B3可在多个核上并行计算。



上述的切分策略会在Tiling参数中体现，比如SingleCoreM、SingleCoreN、SingleCoreK，开发者在host侧通过调用API自动获取Tiling参数，与单核场景的不同的是，多核Tiling需要使用MultiCoreMatmulTiling构造多核Tiling对象，并通过SetDim接口设置Matmul计算所用的核数。注意：这里设置的核数为Matmul计算可用的核数，仅在多核场景下设置，用于计算tiling参数；SetBlockDim为整个算子计算所用核数，是实际会加载的核数，是必须设置的。SetBlockDim的设置规则请参考[numBlocks的说明](#)。SetDim的设置规则如下：

- 纯Cube模式（只有矩阵计算）场景，本节内容以纯Cube模式举例。
SetDim设置当前AI处理器可用的核数，通过Tiling计算得到执行Matmul计算实际使用的核数，实际使用的核数小于等于AI处理器可用的核数。SetBlockDim按照实际使用的核数由用户进行配置。
- MIX模式（包含矩阵计算和矢量计算）的设置规则请参考[MIX场景核数设置规则](#)。

使用场景

多核处理Matmul矩阵计算场景。

约束说明

无

调用示例

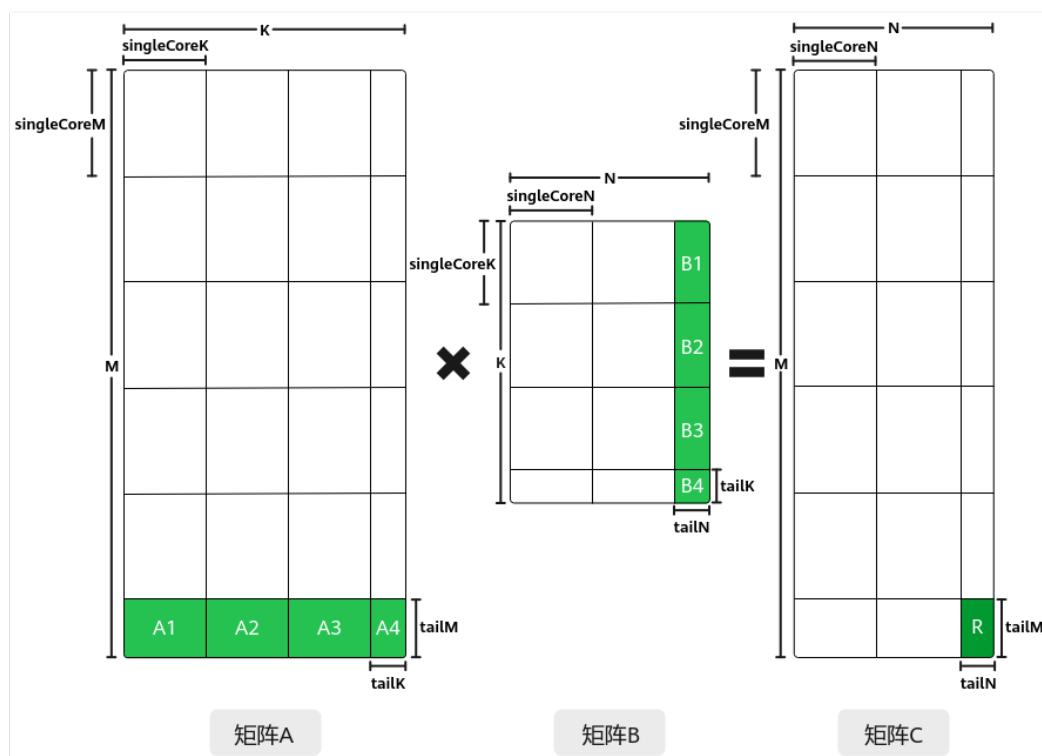
该场景的关键代码示例如下。Matmul多核对齐场景的完整样例请参考：多核切M、N的样例：[Matmul多核Kernel直调样例](#)；多核切K的样例：[多核切K场景的算子样例](#)。

```
// 构造多核Tiling对象
auto ascendcPlatform = platform_ascendc::PlatformAscendCManager::GetInstance(socVersion);
matmul_tiling::MultiCoreMatmulTiling cubeTiling(*ascendcPlatform);
// 仅包含Cube计算的算子，设置可参与矩阵乘运算的核数为当前AI处理器上的Cube核数
cubeTiling.SetDim(ascendcPlatform.GetCoreNumAic());
cubeTiling.SetAType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT16);
cubeTiling.SetBType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT16);
cubeTiling.SetCType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT);
cubeTiling.SetBiasType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT);
cubeTiling.SetOrgShape(M, N, K);
cubeTiling.SetShape(M, N, K);
cubeTiling.EnableBias(isBias);
optiling::TCubeTiling tilingData;
// 获取Tiling参数
int ret = cubeTiling.GetTiling(tilingData); // if ret = -1, gen tiling failed
```

3.3.3.3 多核非对齐切分

功能介绍

多核场景，对矩阵进行切分时，若M、N、K无法整除singleCoreM、singleCoreN、singleCoreK时，就会出现尾块，即多核非对齐场景。如下图矩阵A、B的最后一行和最后一列的矩阵块：



此时，C矩阵中的R矩阵块，依然是通过 $A1*B1 + A2*B2 + A3*B3 + A4*B4$ 累加得到的，处理 $A1*B1$ 、 $A2*B2$ 、 $A3*B3$ 、 $A4*B4$ 等尾块时，需在kernel侧设置尾块大小，在不改变原有tiling的情况下，调用SetTail接口重新设置本次计算的singleCoreM/singleCoreN/singleCoreK，在处理尾块的时候按照设置的值也就是tailM/tailN/tailK进行搬运和计算。

使用场景

多核处理Matmul矩阵计算，存在尾块的场景。

约束说明

处理尾块调用的SetTail接口，需要在Iterate/IterateAll之前调用。

调用示例

Matmul多核非对齐场景的完整样例请参考[Matmul多核非对齐切分算子样例](#)。该场景的关键代码示例如下。

```
// 处理尾块
int tailM = tiling.M - mCoreIndex * tiling.singleCoreM;
tailM = tailM < tiling.singleCoreM ? tailM : tiling.singleCoreM;
int tailN = tiling.N - nCoreIndex * tiling.singleCoreN;
tailN = tailN < tiling.singleCoreN ? tailN : tiling.singleCoreN;
// 当tailM < singleCoreM 或 tailN < singleCoreN时被认为需要处理尾块，此时可以调用SetTail接口进行设置
if (tailM < tiling.singleCoreM || tailN < tiling.singleCoreN) {
    matmulObj.SetTail(tailM, tailN);
}
```

3.3.3.4 异步场景处理

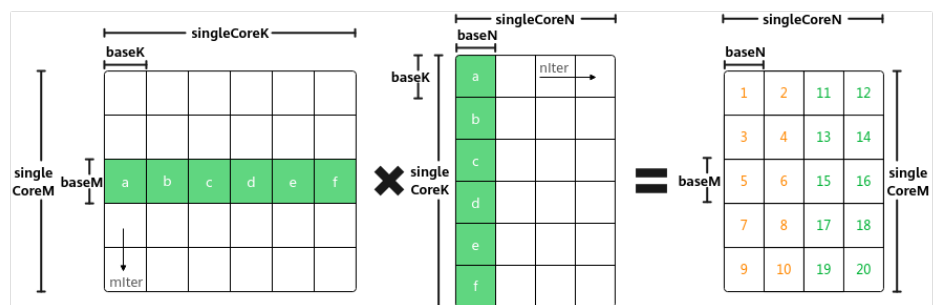
功能介绍

Matmul的Iterate和IterateAll接口在MIX场景（包含矩阵计算和矢量计算）下提供了同步和异步两种模式，纯Cube场景（只有矩阵计算）下，只支持同步模式。

同步模式指的是程序执行时，需要等待某个操作完成后才能继续执行下一步操作。异步模式指的是程序执行时，不需要等待某个操作完成就可以继续执行下一步操作。

- Iterate&GetTensorC的同步和异步

- 同步：执行完一次Iterate迭代计算后，执行GetTensorC搬运矩阵C分片，搬运完成后，才能进行下一次计算。如下图所示，C矩阵中，矩阵块1搬走后，才能计算矩阵块2，矩阵块2搬运完成后，才能计算矩阵块3。



Iterate&GetTensorC同步模式的关键代码示例如下：

```
while (mm.Iterate()) {  
    mm.GetTensorC(gm_c);  
}
```

- 异步：通过设置Iterate接口的模板参数开启异步模式。调用Iterate后，无需立即调用GetTensorC同步等待矩阵C分块搬运完成，可以先执行其它操作，待需要获取结果时再调用GetTensorC。异步模式可以减少同步等待，提高并行度，开发者对计算性能要求较高时，可以选用该方式。异步场景时，需要使用一块临时空间来缓存Iterate计算结果，否则会覆盖计算结果，调用GetTensorC时会在该临时空间中获取C的矩阵分片。临时空间通过SetWorkspace接口进行设置。SetWorkspace接口需要在Iterate接口之前调用。

Iterate&GetTensorC异步模式的关键代码示例如下：

```
mm.SetWorkspace(workspace, size); // 其中，workspace为临时空间的物理地址，size为  
singleCoreM * singleCoreN的矩阵C大小  
// 异步模式  
mm.template Iterate<false>();  
..... // 执行其他操作  
auto mIter = Ceil(singleCoreM, baseM);  
auto nIter = Ceil(singleCoreN, baseN);  
for (int i = 0; i < mIter * nIter; ++i) {  
    mm.GetTensorC<false> (gm_c);  
}
```

- IterateAll的同步和异步

- 同步：后续操作需要同步等待IterateAll执行结束。

IterateAll同步模式的关键代码示例如下：

```
mm.SetTensorA(gm_a); // 设置左矩阵A  
mm.SetTensorB(gm_b); // 设置右矩阵B  
mm.SetBias(gm_bias); // 设置Bias  
mm.IterateAll(gm_c);  
// 后续操作  
...
```


- 异步：后续操作不需要同步等待IterateAll执行结束，需要IterateAll的结果时，调用WaitIterateAll等待IterateAll异步接口返回。

IterateAll异步模式的关键代码示例如下：

```
AscendC::Matmul<aType, bType, cType, biasType> mm;  
mm.SetTensorA(queryGm[tensorACoreOffset]);  
mm.SetTensorB(keyGm[tensorBCoreOffset + sInnerStart * singleProcessSInnerSize *  
    tilingData->attentionScoreOffsetStrideParams.matmulHead], true);  
mm.SetTail(singleProcessSOuterSize, mmNNum);  
mm.template IterateAll<false>(workspaceGm[tmp_block_idx * mmResUbSize *  
    sInnerLoopTimes], 0, false, true);  
// 执行其他操作  
mm.WaitIterateAll(); // 等待IterateAll完成  
DataCopy(dstUB, GM); // 进行GM到UB的拷贝
```

使用场景

- Iterate&GetTensorC的同步：MIX场景（包含矩阵计算和矢量计算）、纯Cube场景（只有矩阵计算）。
- Iterate&GetTensorC的异步：仅MIX场景（包含矩阵计算和矢量计算）。
- IterateAll的同步：MIX场景（包含矩阵计算和矢量计算）、纯Cube场景（只有矩阵计算）。
- IterateAll的异步：仅MIX场景（包含矩阵计算和矢量计算）。

约束说明

- Iterate&GetTensorC的异步场景：
 - 传入的C矩阵地址空间大小需要保证不小于baseM * baseN。
 - SetWorkspace接口需要在Iterate接口之前调用。
 - 支持只输出到VECIN、只输出到Global Memory，同时输出到Global Memory和VECIN三种输出方式。
 - 取出C矩阵到VECIN时，数据格式仅支持NZ；取出C矩阵到GM时，数据格式支持ND或NZ。
- IterateAll的异步场景：
 - 传入的C矩阵地址空间大小需要保证不小于singleCoreM * singleCoreN。
 - 仅支持连续输出至Global Memory。

调用示例

- Iterate&GetTensorC的异步场景的完整样例请参考[异步场景样例](#)、[Iterate异步场景矩阵乘法](#)。
- IterateAll的异步场景的完整样例请参考[IterateAll异步场景矩阵乘法](#)。

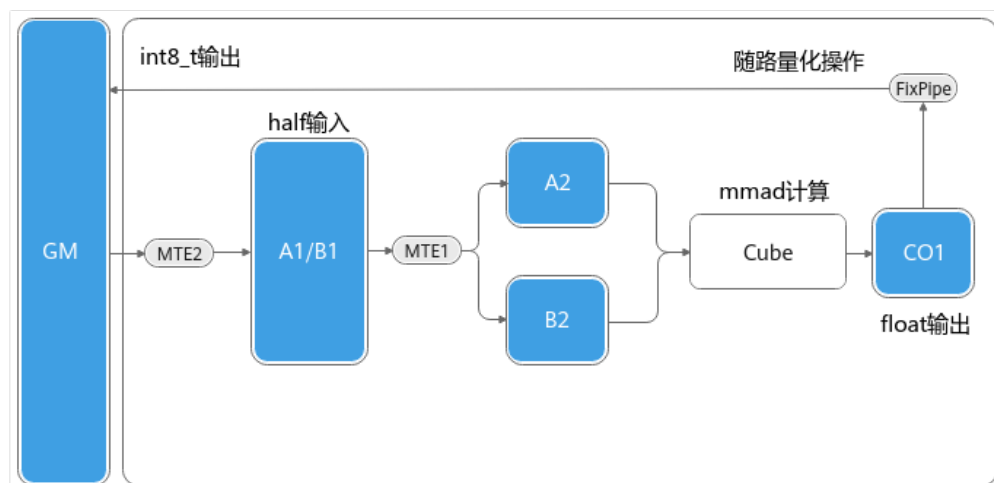
3.3.3.3.5 矩阵乘输出的量化/反量化

功能介绍

对于[特定输入输出数据类型](#)，Matmul支持将计算结果从CO1搬出到Global Memory时，对输出C矩阵元素执行数据量化或反量化操作。

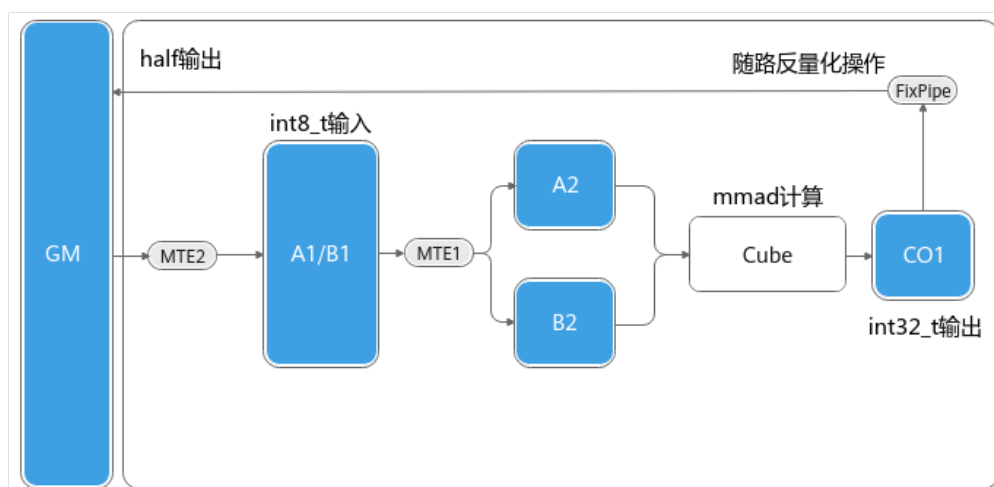
- Matmul量化场景：Matmul计算时左矩阵A、右矩阵B为half或bfloat16_t数据类型，输出C矩阵为int8_t数据类型。该场景下，C矩阵的数据从CO1搬出到Global Memory时，会执行量化操作，将最终结果量化为int8_t类型，如下图所示。

图 3-32 Matmul 量化场景示意图



- Matmul反量化场景：Matmul计算时左矩阵A、右矩阵B为int8_t或int4b_t数据类型，输出C矩阵为half数据类型，或者左矩阵A、右矩阵B为int8_t数据类型，输出C矩阵为int8_t数据类型。该场景下，C矩阵的数据从CO1搬出到Global Memory时，会执行反量化操作，将最终结果反量化为对应的half类型或int8_t类型，如下图所示。

图 3-33 Matmul 反量化场景示意图



Matmul量化/反量化包含两种模式：同一系数的量化/反量化模式、向量的量化/反量化模式，开发者在算子Tiling侧调用SetDequantType接口设置量化或反量化模式，这两种模式的具体区别为：

- 同一系数的量化/反量化模式（PER_TENSOR模式）：整个C矩阵对应一个量化参数，量化参数的shape为[1]。开发者在算子Kernel侧调用接口SetQuantScalar设置量化参数。
- 向量的量化/反量化模式（PER_CHANNEL模式）：C矩阵的shape为[m, n]，每个channel维度即C矩阵的每一列，对应一个量化参数，量化参数的shape为[n]。开发者在算子Kernel侧调用接口SetQuantVector设置量化参数。

表 3-6 量化/反量化模式对应的接口配置

模式	Tiling侧接口	Kernel侧接口
同一系数的量化/反量化	SetDequantType(DequantType::SCALAR)	SetQuantScalar(gmScalar)
向量的量化/反量化	SetDequantType(DequantType::TENSOR)	SetQuantVector(gmTensor)

使用场景

需要对矩阵计算结果进行量化/反量化操作的场景，当前该场景下，Matmul输入输出矩阵支持的数据类型如下表所示。

表 3-7 Matmul 量化/反量化支持的数据类型

A矩阵	B矩阵	C矩阵	支持平台
half	half	int8_t	Atlas A3 训练系列产品/Atlas A3 推理系列产品 Atlas A2 训练系列产品/Atlas A2 推理系列产品
bfloat16_t	bfloat16_t	int8_t	Atlas A3 训练系列产品/Atlas A3 推理系列产品 Atlas A2 训练系列产品/Atlas A2 推理系列产品
int8_t	int8_t	half	Atlas A3 训练系列产品/Atlas A3 推理系列产品 Atlas A2 训练系列产品/Atlas A2 推理系列产品
int4b_t	int4b_t	half	Atlas A3 训练系列产品/Atlas A3 推理系列产品 Atlas A2 训练系列产品/Atlas A2 推理系列产品
int8_t	int8_t	int8_t	Atlas A3 训练系列产品/Atlas A3 推理系列产品 Atlas A2 训练系列产品/Atlas A2 推理系列产品

约束说明

- SetQuantScalar和SetQuantVector接口必须在Iterate或者IterateAll接口前调用。
- 在Kernel侧与Tiling侧设置的量化/反量化模式需要保持一致：
 - Kernel侧调用SetQuantScalar接口设置同一系数的量化/反量化模式，对应Tiling侧调用SetDequantType接口配置模式为DequantType::SCALAR。

- Kernel侧调用SetQuantVector接口设置向量的量化/反量化模式，对应Tiling侧调用SetDequantType接口配置模式为DequantType::TENSOR。
- 当A、B矩阵为int8_t或int4b_t类型，C矩阵为half时，本节特性的输出结果不支持INF_NAN模式。若结果需要以INF_NAN输出，建议在调用Matmul API时将结果输出到TPosition::VECIN，同时将输出的数据类型设置为int32_t，再基于AIV核使用高阶API AscendDequant将该结果反量化为half类型。

调用示例

完整的算子样例请参考[matmul_quant算子样例](#)。

- Tiling实现

调用SetDequantType接口设置量化或反量化模式，其他实现内容与基础场景相同。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
matmul_tiling::MatmulApiTiling tiling(ascendcPlatform);
tiling.SetAType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_INT8);
tiling.SetBType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_INT8);
tiling.SetCType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_INT32);
tiling.SetBiasType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_INT32);
tiling.SetShape(M, N, K);
tiling.SetOrgShape(M, N, K);
tiling.EnableBias(true);
tiling.SetDequantType(DequantType::SCALAR); // 设置同一系数的量化/反量化模式
// tiling.SetDequantType(DequantType::TENSOR); // 设置向量的量化/反量化模式
... // 执行其他配置
```

- Kernel实现

根据具体量化模式场景，调用SetQuantScalar或SetQuantVector接口设置量化参数。其他实现内容与基础场景相同。

- 同一系数的量化/反量化模式

```
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling);
float tmp = 0.1; // 输出gm时会乘以0.1
uint64_t ans = static_cast<uint64_t>(*reinterpret_cast<int32_t*>(&tmp)); // 浮点值量化系数转换为
uint64_t类型进行设置
mm.SetQuantScalar(ans);
mm.SetTensorA(gm_a);
mm.SetTensorB(gm_b);
mm.SetBias(gm_bias);
mm.IterateAll(gm_c);
```

- 向量的量化/反量化模式

```
GlobalTensor gmQuant;
...
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling);
mm.SetQuantVector(gmQuant);
mm.SetTensorA(gm_a);
mm.SetTensorB(gm_b);
mm.SetBias(gm_bias);
mm.IterateAll(gm_c);
```

3.3.3.3.6 矩阵乘输出的 Channel 拆分

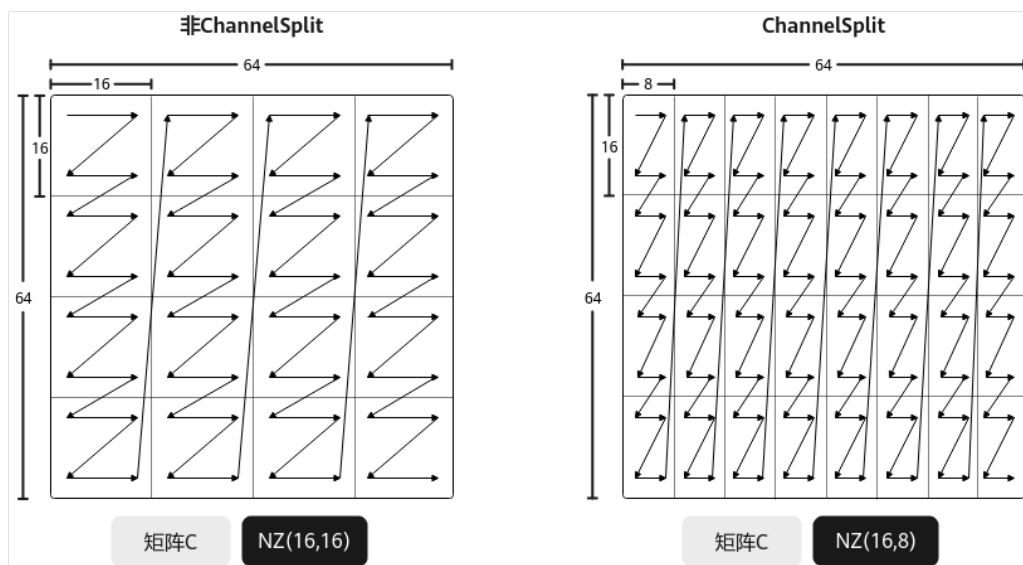
功能介绍

矩阵乘输出的Channel拆分，又称ChannelSplit。指当Matmul计算结果C矩阵的格式为**NZ**时，C矩阵采用分形存储，关于NZ格式的详细内容请参考[数据格式](#)。当C矩阵的物

理排布格式为NZ、数据类型为float时，默认情况下，每个分形内部包含 16×16 个元素，即分形的大小为 16×16 。ChannelSplit的功能为将此场景下C矩阵的每个 16×16 的分形切分为 16×8 的分形，使得C矩阵按照 16×8 的分形进行存储。

由于1个float类型数据的大小为4字节， 16×8 的分形在内轴满足32字节对齐，内轴上的数据量与一条NPU矢量计算指令处理的数据单元一致，这便于后续的其它计算。ChannelSplit功能默认不启用，用户需通过设置MatmulConfig中的isEnableChannelSplit参数为true来开启此功能。

图 3-34 ChannelSplit 功能示意图



使用场景

对于NZ格式、float类型的C矩阵，需要按 16×8 的分形存储时，使用该功能。

约束说明

开启ChannelSplit功能需满足：

- C矩阵的数据排布格式为CubeFormat::NZ。
- C矩阵的数据类型为float。
- C矩阵的内存逻辑位置为Global Memory。
- 矩阵乘结果CO1数据类型为float。

调用示例

完整的算子样例请参考[matmul_channelsplit算子样例](#)。

```
// 指定获取和修改的MatmulConfig模板
constexpr static MatmulConfigMode configMode = MatmulConfigMode::CONFIG_NORM;
// 修改模板参数isEnableChannelSplit=true，开启该MatmulConfig模板的ChannelSplit功能
constexpr static MatmulFuncParams funcParamsChannelSplit{
    false, false, false, false, 0, IterateOrder::ORDER_M, ScheduleType::INNER_PRODUCT, true, false, false,
    false, true/*isEnableChannelSplit*/
};
constexpr static MatmulConfig MM_CFG = GetMMConfig<configMode>(funcParamsChannelSplit);
Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, MM_CFG> mm;
```

```
// 常规Matmul计算，最后输出分形为16*8  
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm);  
mm.SetTensorA(gm_a);  
mm.SetTensorB(gm_b);  
mm.SetBias(gm_bias);  
mm.IterateAll(gm_c);
```

3.3.3.3.7 矩阵向量乘

功能介绍

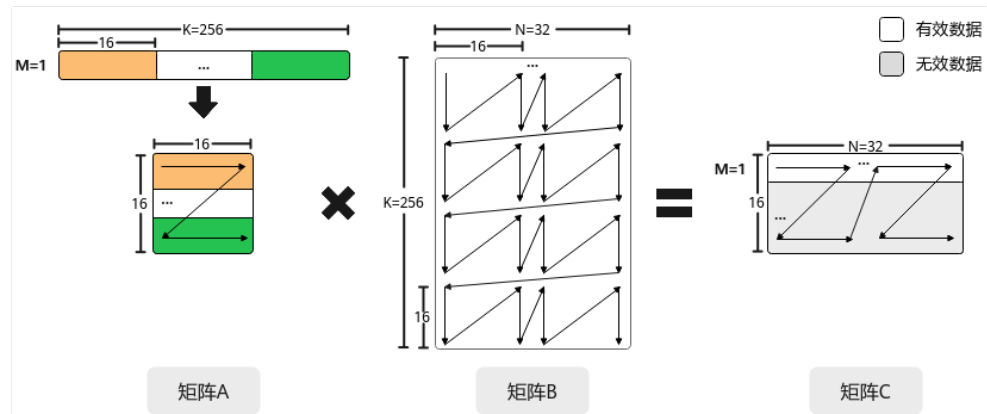
矩阵向量乘（General Matrix-Vector multiplication），即GEMV，是指Matmul计算中 $M=1$ ，形状为 $(1, K)$ 的左矩阵A与形状为 (K, N) 的右矩阵B进行矩阵乘运算的场景。Matmul支持在Tiling侧与Kernel侧通过配置A矩阵的数据格式为**VECTOR**来开启GEMV模式，从而高效处理 $M=1$ 的计算场景。若在 $M=1$ 时未开启GEMV模式，Matmul计算则将M方向作为非对齐场景进行处理。GEMV模式相较于非对齐处理方式，搬运数据量更少，性能更好。

以 $M=1$ ， $K=256$ ， $N=32$ ，左右矩阵数据类型为half的Matmul为具体示例，说明GEMV模式的Matmul API内部处理过程。

- GEMV模式

将A矩阵从A1搬运到A2时， 1×256 的向量被当作 16×16 的矩阵进行处理，调用LoadData接口一次完成 16×16 分形大小的矩阵搬运。B矩阵的搬运以及矩阵乘计算跟基础场景相同，如下图所示。

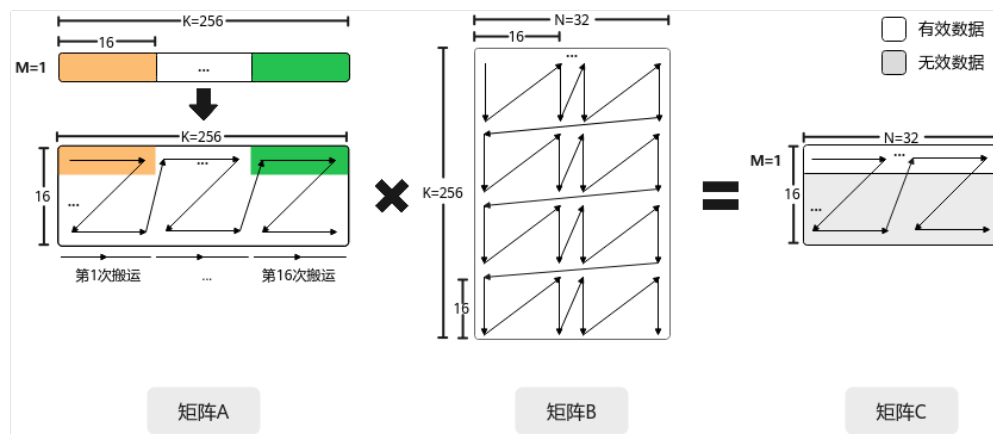
图 3-35 GEMV 模式 $M=1$ 的矩阵乘计算示意图



- 非GEMV模式

将A矩阵从A1搬运到A2时， 1×256 的向量被当作非对齐矩阵数据进行处理，将M方向对齐到32字节后进行搬运。调用LoadData接口每次搬运 16×16 分形大小的矩阵，一共搬运 $K/16=16$ 次，导致搬运数据量增加，性能相较于GEMV模式差，如下图所示。

图 3-36 非 GEMV 模式 M=1 的矩阵乘计算示意图



使用场景

形状为(1, K)的A矩阵 (M=1, K>1) 做矩阵乘计算, 即输入A矩阵的数据是向量数据。

约束说明

- 在Matmul计算中, 若要开启GEMV模式, A矩阵的原始输入形状M必须等于1。
- GEMV场景下, 左矩阵A不支持转置。
- GEMV场景下, Global Memory上的左矩阵数据需要保证16字节对齐。

调用示例

完整的算子样例请参考[matmul_gemv算子样例](#)。

- Tiling实现

调用SetAType接口, 设置A矩阵的数据格式为CubeFormat::VECTOR, 其它Tiling实现与[基础场景](#)相同。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
matmul_tiling::MatmulApiTiling tiling(ascendcPlatform);
// 调用设置A矩阵的格式为CubeFormat::VECTOR
tiling.SetAType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::VECTOR,
matmul_tiling::DataType::DT_FLOAT16);
tiling.SetBType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT16);
tiling.SetCType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT);
tiling.SetBiasType(AscendC::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT);
... // 其他实现内容
optiling::TCubeTiling tilingData;
int ret = tiling.GetTiling(tilingData);
```

- Kernel实现

相较于[基础场景](#), GEMV场景在创建Matmul对象时, 设置模板参数A_TYPE的数据格式为CubeFormat::VECTOR。

```
#include "lib/matmul_intf.h"

using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::VECTOR, half>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half>;
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, float>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, float>;
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE> mm;
```

3.3.3.3.8 4:2 稀疏矩阵乘

功能介绍

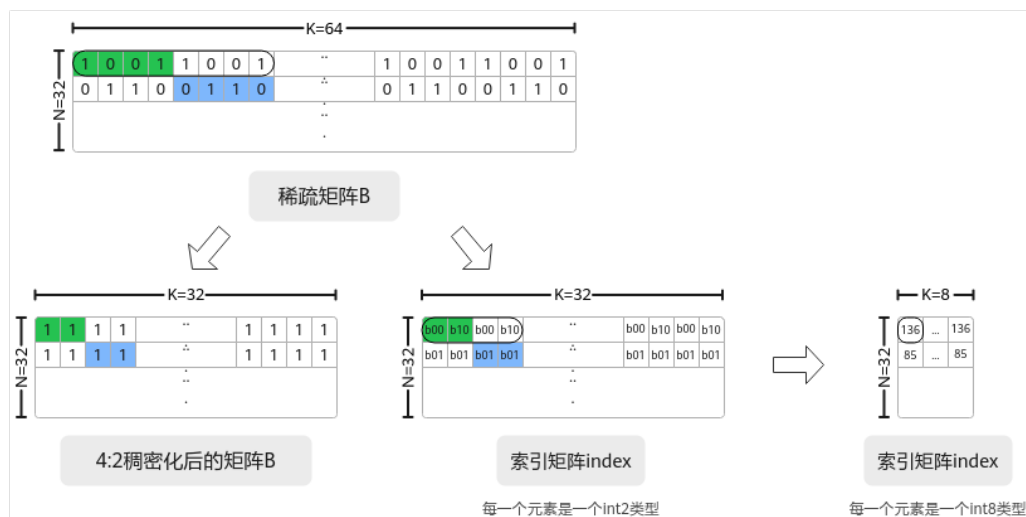
4:2稀疏矩阵乘，又称Sparse Matmul。该场景下输入的原始左矩阵A、右矩阵B为稀疏矩阵，稀疏矩阵B中每4个元素中至少有2个为零元素；在进行Matmul计算前，用户需要自行对B矩阵进行4:2稠密化，即基于原始稀疏矩阵B在每4个元素中过滤掉2个零元素，使B矩阵稠密化为稠密矩阵；Sparse Matmul场景调用Matmul API完成A矩阵与4:2稠密化后的B矩阵的矩阵乘计算。Sparse Matmul可以跳过稀疏矩阵B中的零元素，仅对非零元素进行数据搬运存储和计算，从而减少矩阵乘计算时的内存占用和计算量，提升性能。

实现流程

步骤1 数据预处理

在计算前的数据准备阶段，用户自行对原始为稀疏矩阵的B矩阵完成稠密化，稠密过程请参考稠密算法说明。稠密化过程结束后，得到4:2稠密化后的右矩阵B和索引矩阵index，稠密化后的右矩阵B和索引矩阵index将作为Sparse Matmul场景的计算输入。

图 3-37 对原始稀疏矩阵 B 进行 4:2 稠密化过程示意图



稠密化过程对于稀疏矩阵B的每4个元素，在索引矩阵index中生成2个2位索引，每个索引分别指向对应非零元素的相对位置，具体规则可参考稠密算法说明。稠密化过程生成的索引矩阵的数据类型为int2，索引矩阵在加载入Matmul前，需要拼成int8的数据类型。索引矩阵在一个int8的地址中的排布是逆序排布的，例如：索引矩阵1 2 0 1 0 2 1 0，在地址中的排布为1 0 2 1 0 1 2 0，其中1 0 2 1（对应索引矩阵前四位1 2 0 1）为一个int8，0 1 2 0（对应索引矩阵后四位0 2 1 0）为一个int8。

步骤2 使能Sparse Matmul场景

在Host侧，获取Tiling前需要通过SetSparse接口设置使能Sparse Matmul场景。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
matmul_tiling::MatmulApiTiling tiling(ascendcPlatform);
tiling.SetAType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_INT8);
tiling.SetBType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_INT8);
tiling.SetCType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
```



```
matmul_tiling::DataType::DT_INT32);  
tiling.SetBiasType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,  
matmul_tiling::DataType::DT_INT32);  
// 设置使能Sparse Matmul场景  
tiling.SetSparse(true);  
... // 其他实现内容  
optiling::TCubeTiling tilingData;  
int ret = tiling.GetTiling(tilingData);
```

步骤3 创建Matmul对象

在Kernel侧创建Matmul对象时，通过MatmulType定义A、C、Bias的参数类型信息，包括：内存逻辑位置、数据格式、数据类型。通过SparseMatmulType类型定义B矩阵的参数类型，包括：B矩阵的内存逻辑位置、索引矩阵的内存逻辑位置、数据格式、数据类型等。

```
#include "lib/matmul_intf.h"  
  
using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, AType, false>;  
// 使用SparseMatmulType定义B矩阵的参数类型信息  
using B_TYPE = AscendC::SparseMatmulType<AscendC::TPosition::GM, AscendC::TPosition::GM,  
CubeFormat::ND, BType, true>;  
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;  
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;  
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_MDL> mm;
```

步骤4 设置索引矩阵

通过SetSparseIndex接口传入稠密化过程中生成的索引矩阵。

```
mm.SetTensorA(gm_a); // 设置左矩阵A  
mm.SetTensorB(gm_b); // 设置右矩阵B  
mm.SetSparseIndex(gm_index); // 传入稠密化过程中生成的索引矩阵  
mm.SetBias(gm_bias); // 设置Bias
```

步骤5 完成矩阵乘操作

在Kernel侧，基于步骤4加载的索引矩阵，完成矩阵乘操作。Matmul API内部完成对A矩阵的稠密化，即根据索引矩阵从A矩阵的每4个元素中，选择2个对应位置元素参与计算。

```
// 调用Iterate和GetTensorC或IterateAll接口完成矩阵乘计算  
while (mm.Iterate()) {  
    mm.GetTensorC(gm_c);  
}  
// mm.IterateAll(gm_c);  
mm.End();
```

----结束

参数说明

表 3-8 SparseMatmulType 类型参数说明

参数	说明
POSITION	内存逻辑位置。 B矩阵仅支持设置为TPosition::GM。
INDEX_POSITION	索引矩阵内存逻辑位置。 仅支持设置为TPosition::GM。
CubeFormat	数据的物理排布格式，详细介绍请参考 数据格式 。 B矩阵支持设置为CubeFormat::ND，CubeFormat::NZ。

参数	说明
TYPE	B矩阵仅支持设置为int8_t数据类型。
ISTRANS	是否开启使能矩阵转置的功能。 当前只支持取值为true，表示开启使能矩阵转置的功能。
LAYOUT	表征数据的排布。Sparse Matmul场景仅支持取值为LAYOUT::NONE。 NONE：默认值，表示不使用BatchMatmul。
IBSHARE	是否使能IBShare（IntraBlock Share）。IBShare的功能是能够复用L1 Buffer上相同的A矩阵或B矩阵数据。当A矩阵和B矩阵同时使能IBShare时，表示L1 Buffer上的A矩阵和B矩阵同时复用。 Sparse Matmul场景当前仅支持该参数取值为false，表示不使能IBShare。

使用场景

左矩阵A为稀疏矩阵、右矩阵B为4:2稠密化后的矩阵的Matmul计算场景。

约束说明

- 该场景仅支持MDL模板下的纯Cube模式（只有矩阵计算）。
- 通过SetSparseIndex接口传入的索引矩阵只支持int8数据类型和[NZ数据排布格式](#)。
- 原始稀疏矩阵B中每4个元素中应保证最多2个非零元素（即最少2个零元素），如果存在3个或更多非零元素，则仅使用前2个非零元素。
- M、K、N中的任意一个值不能为0。

调用示例

Sparse Matmul场景的完整样例请参考[Sparse Matmul场景的算子样例](#)。

3.3.3.3.9 TSCM 输入的矩阵乘

功能介绍

TSCM表示L1 Buffer空间对应的逻辑内存，L1 Buffer相关内容见[存储单元](#)，开发者可以自行管理TSCM以高效利用硬件资源。比如，开发者可缓存一份TSCM数据，在不同使用场景中灵活配置为Matmul操作的A矩阵、B矩阵或Bias偏置矩阵，实现内存复用与计算效率优化。在TSCM输入场景，用户管理整块TSCM内存空间，Matmul直接使用传入的TSCM内存地址，不进行Global Memory到TSCM的数据搬入。

使用场景

用户需要自定义数据搬入到TSCM及自定义管理的场景，即需要自定义实现数据搬入功能，如非连续搬入或对搬入数据进行预处理等。用户通过自定义管理TSCM可灵活配置MTE2流水，实现跨Matmul对象的全局[DoubleBuffer](#)，MTE2相关内容见[搬运单元](#)。

约束说明

设置为TSCM输入的矩阵必须在TSCM中全载，全载即全部的矩阵数据同时搬入及保持在TSCM中。

调用示例

完整的算子样例请参考[自定义数据来源为GM的TSCM输入的Matmul算子样例](#)、[BatchMatmul自定义TSCM输入的算子样例](#)。

```
TQue<TPosition::A1, 1> scm; // 队列逻辑位置A1, 队列深度为1
pipe->InitBuffer(scm, 1, tiling.M * tiling.Ka * sizeof(A_T));
// A_TYPE的TPosition为TSCM, B_TYPE的TPosition为GM
Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE> mm1;
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm1);
mm1.Init(&tiling);
// 自定义A矩阵GM到TSCM的搬运
auto scmTensor = scm.AllocTensor<A_T>();
DataCopy(scmTensor, gm_a, tiling.M * tiling.Ka);
scm.Enqueue(scmTensor);
LocalTensor<A_T> scmLocal = scm.DeQueue<A_T>();
// A矩阵设置为TSCM输入, B矩阵为GM输入
mm1.SetTensorA(scmLocal);
mm1.SetTensorB(gm_b);
mm1.SetBias(gm_bias);
mm1.IterateAll(gm_c);
scm.FreeTensor(scmLocal);
```

3.3.3.3.10 矩阵乘输出的 N 方向对齐

功能介绍

矩阵乘输出的N方向对齐，即矩阵乘结果C矩阵按ND_ALIGN格式输出。在Matmul矩阵乘法中，常用的矩阵数据格式有ND、NZ，相关介绍可参考[数据格式](#)章节。ND_ALIGN是矩阵的另一种数据格式，该格式一般用于N方向非32字节对齐的矩阵乘计算中，配置结果C矩阵为ND_ALIGN格式后，将按照N方向32字节对齐的补齐规则输出C矩阵，详细内容请见[ND_ALIGN](#)。

以M=16，K=16，N=14，A、B矩阵数据类型为half的Matmul为具体示例，说明ND_ALIGN输出功能。当配置C矩阵为ND格式并输出到Global Memory时，按照原始N方向大小非32字节对齐输出如[图3-38](#)所示。当配置C矩阵为ND格式时，按照N方向32字节对齐输出如[图3-39](#)所示，C矩阵的N方向最后两列由下一行的实际数据进行填充补齐，以实现N方向对齐到32字节并输出。当配置C矩阵为ND_ALIGN格式时，Matmul API会在C矩阵的N方向上通过添加无效数据来填充最后两列，以确保N方向对齐至32字节并输出，如[图3-40](#)所示。

图 3-38 ND 格式 C 矩阵 N 方向非 32 字节对齐示意图

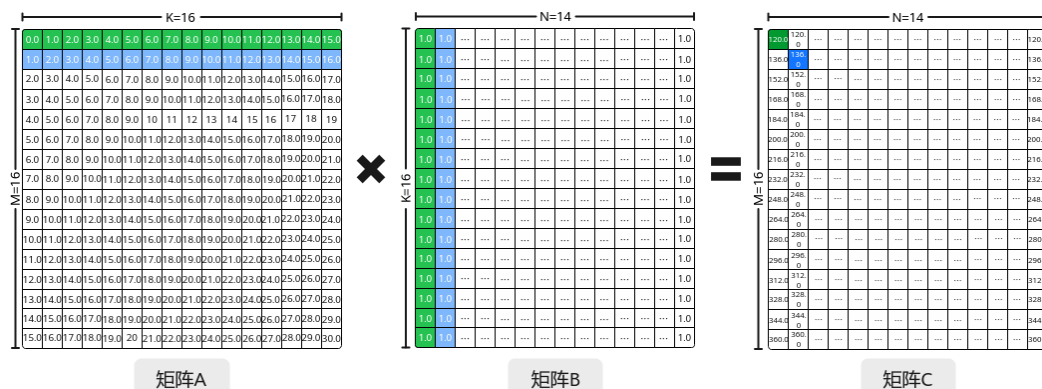


图 3-39 ND 格式 C 矩阵 N 方向 32 字节对齐示意图

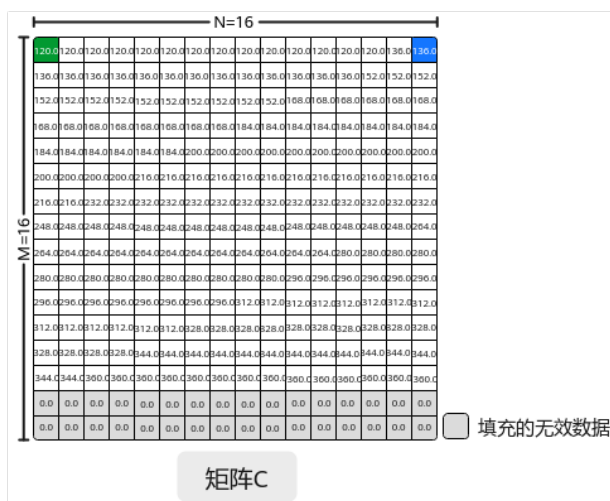


图 3-40 ND_ALIGN 格式 C 矩阵 N 方向 32 字节对齐示意图



使用场景

Matmul计算中N方向非32字节对齐，输出C矩阵的N方向要求32字节对齐的场景。

约束说明

若配置C矩阵为ND_ALIGN格式输出，则为C矩阵申请的Buffer空间为N向上32字节对齐后的空间大小。

调用示例

完整的算子样例请参考[matmul_nd_align算子样例](#)。

- Tiling实现

调用SetCType接口，设置C矩阵的数据格式为CubeFormat::ND_ALIGN，其它Tiling实现与[基础场景](#)相同。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
matmul_tiling::MatmulApiTiling tiling(ascendcPlatform);
tiling.SetAType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT16);
tiling.SetBType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT16);
// 设置C矩阵，buffer位置为GM，数据格式为ND_ALIGN
tiling.SetCType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND_ALIGN,
matmul_tiling::DataType::DT_FLOAT);
tiling.SetBiasType(AscendC::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT);
... // 其他实现内容
optiling::TCubeTiling tilingData;
int ret = tiling.GetTiling(tilingData);
```

- Kernel实现

相较于[基础场景](#)，ND_ALIGN输出功能要求在创建Matmul对象时，设置模板参数cType的数据格式为CubeFormat::ND_ALIGN。

```
#include "lib/matmul_intf.h"

typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half> aType;
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half> bType;
// 设置模板参数cType的数据格式为ND_ALIGN
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND_ALIGN, float> cType;
typedef AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, float> biasType;
AscendC::Matmul<aType, bType, cType, biasType> mm;
```

3.3.3.3.11 单次矩阵乘局部输出

功能介绍

单次矩阵乘局部输出，又称Partial Output。如[基础知识](#)中所述，一次Iterate计算过程中，会按K方向进行一次或多次基本块计算，其中的一次基本块计算为baseM*baseK和baseK*baseN大小的输入数据进行计算得到baseM*baseN大小的结果；每次基本块计算的结果进行累加后，便得到baseM*singleCoreK和singleCoreK*baseN大小的输入数据计算得到的结果baseM*baseN，并将其作为一次Iterate的最终结果输出。

开启Partial Output功能后，调用Iterate接口不会进行K轴累加，只进行单次基本块计算。用户可以通过GetTensorC接口获取对应的单片数据，最后自行进行K轴上的累加。

图 3-41 未开启 Partial Output 功能计算示意图

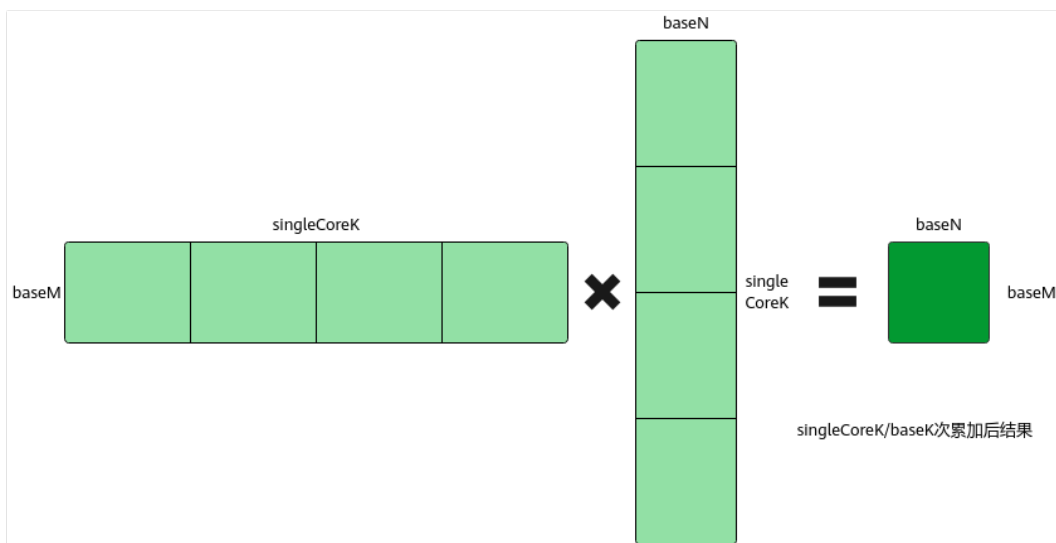
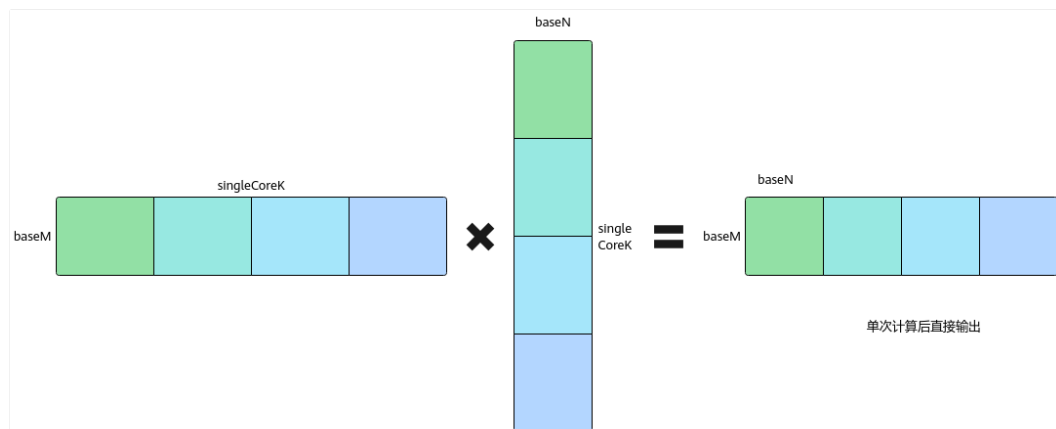


图 3-42 开启 Partial Output 功能计算示意图



使用场景

矩阵乘计算结果不需要累加，只需要输出 $\text{baseM} \times \text{baseK}$ 和 $\text{baseK} \times \text{baseN}$ 的计算结果 $\text{baseM} \times \text{baseN}$ 。例如需要先获取单次基本块计算的数据进行反量化，再累加得到最终结果。

约束说明

- 该功能仅支持MDL模板。
- 获取矩阵乘计算结果时，仅支持调用Iterate和GetTensorC接口的连续写模式，不支持非连续写模式以及IterateAll接口获取计算结果，连续写模式的介绍请参考GetTensorC。
- 该功能不支持带有Bias矩阵的Matmul计算，即不支持输入Bias矩阵。

调用示例

完整的算子样例请参考[开启Partial Output功能的算子样例](#)。

```
// 配置MDL模板, 使能Partial Output
constexpr static MatmulConfigMode configMode = MatmulConfigMode::CONFIG_MDL;
constexpr static MatmulFuncParams funcParams = {
    false, false, false, false, 0, IterateOrder::UNDEF, ScheduleType::INNER_PRODUCT, true, true,
    true /* isPartialOutput */
};
constexpr static MatmulConfig CFG_PARTIAL = GetMMConfig<configMode>(funcParams);
Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_PARTIAL> mm;
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm);
mm.Init(&tiling);
mm.SetTensorA(gmA, isTransposeA);
mm.SetTensorB(gmB, isTransposeB);
while (mm.Iterate()) {
    mm.GetTensorC(tmpGmC[dstOffset], false, true);
    dstOffset += baseM * baseN;
    // 其他操作
}
```

3.3.3.3.12 AIC 和 AIV 独立运行机制

功能介绍

AIC和AIV独立运行机制, 又称双主模式。在[分离模式](#)下, 区别于MIX模式(包含矩阵计算和矢量计算)通过消息机制驱动AIC运行, 双主模式为AIC和AIV独立运行代码, 不依赖消息驱动, 使能双主模式能够提高Matmul计算性能。默认情况下, 双主模式不使能, 需要通过MatmulConfig中的enableMixDualMaster参数开启。

使用场景

算子中的矩阵计算和矢量计算相关代码独立运行, 不依赖消息驱动时, 可以开启双主模式, 以提高Matmul计算性能。

约束说明

- 该功能仅支持Norm模板和MDL模板。
- 算子核函数的类型为MIX, 同时AIC核数:AIV核数为1:1。
- 算子核函数的类型为MIX, 同时AIC核数:AIV核数为1:2, 且A矩阵和B矩阵同时使能IBSHARE参数。
- 同一算子中所有Matmul对象的该参数取值必须保持一致。
- A、B、Bias矩阵只支持从Global Memory输入。
- 获取矩阵计算结果只支持调用IterateAll接口输出到GlobalTensor, 即计算结果放置于Global Memory的地址, 不能调用GetTensorC等接口获取结果。

调用示例

完整的算子样例请参考[使能双主模式的算子样例](#)。

```
// 修改模板参数enableMixDualMaster=true, Norm模板开启双主模式, MDL模板使用GetMDLConfig接口获取模板参数。
constexpr static MatmulConfig MM_CFG = GetNormalConfig(false, false, false,
    BatchMode::BATCH_LESS_THAN_L1, true, IterateOrder::ORDER_M, ScheduleType::OUTER_PRODUCT, false,
    true/*enableMixDualMaster*/);
Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, MM_CFG> mm;

// 常规Matmul计算
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm);
mm.SetTensorA(gm_a);
mm.SetTensorB(gm_b);
```

```
mm.SetBias(gm_bias);  
mm.IterateAll(gm_c);
```

3.3.3.13 Batch Matmul 基础功能

功能介绍

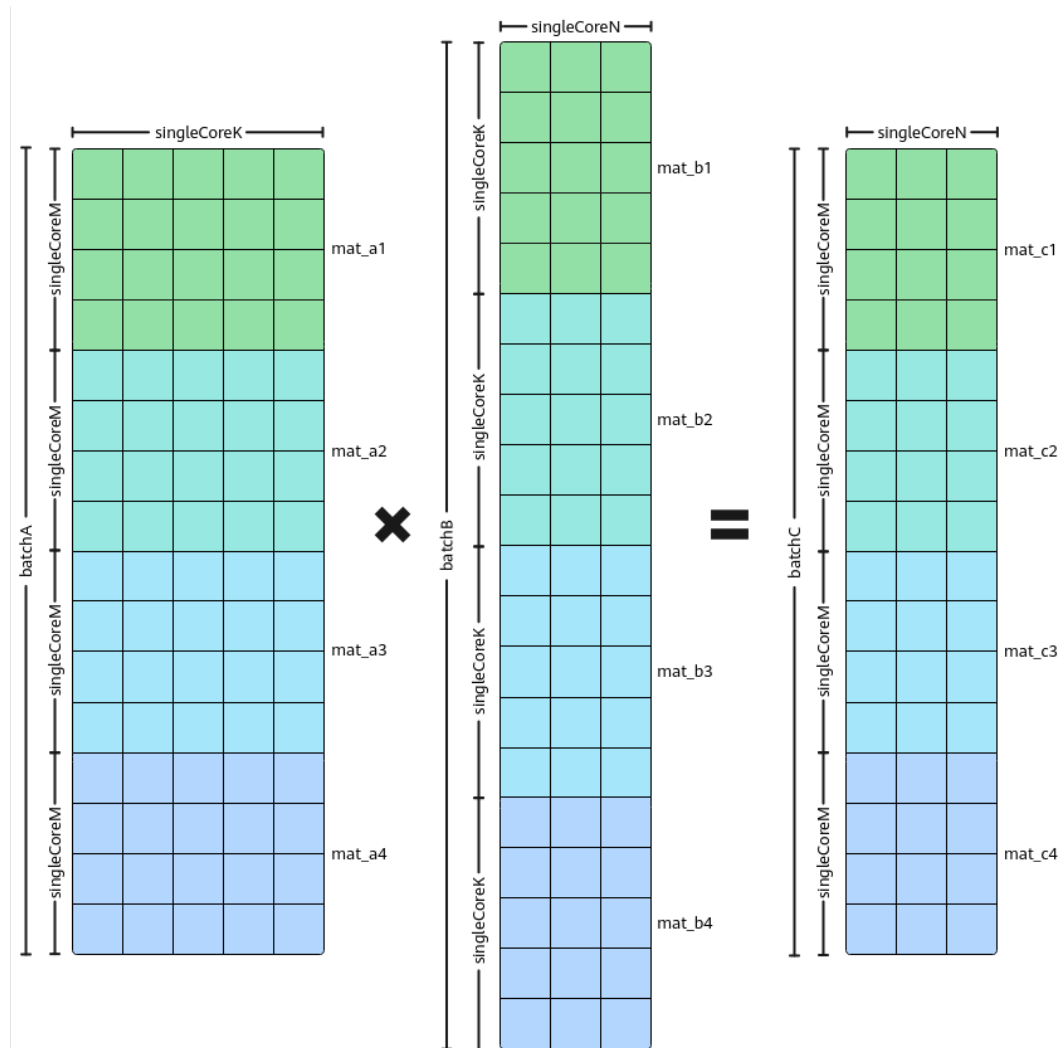
Batch Matmul是指批量处理Matmul计算的场景。该场景对外提供了IterateBatch的调用接口，调用一次IterateBatch，可以计算出多个singleCoreM * singleCoreN大小的C矩阵。

Matmul单次计算的过程需要搬入和搬出数据，当进行多次Matmul计算且单次Matmul计算的输入shape较小时，搬运开销在整体耗时中占比较大。通过IterateBatch接口批量处理Matmul，可以有效提升带宽利用率。

Batch Matmul当前支持4种Layout类型：BSNGD、SBNGD、BNGS1S2、NORMAL（BMNK的数据排布格式），相关数据排布格式请参考IterateBatch。

下图为NORMAL数据排布格式的Batch Matmul计算。整个Matmul计算一共包含4个矩阵乘操作：mat_a1*mat_b1、mat_a2*mat_b2、mat_a3*mat_b3、mat_a4*mat_b4，需要单核上计算四个singleCoreM *singleCoreN。在该场景下，如果shape较小，可以将其视为Batch Matmul场景进行批量处理，以提升性能。一次IterateBatch可同时计算出mat_c1 = mat_a1 * mat_b1、mat_c2 = mat_a2 * mat_b2、mat_c3 = mat_a3 * mat_b3、mat_c4 = mat_a4 * mat_b4。

图 3-43 NORMAL 数据排布格式的 Batch Matmul 示意图



使用场景

Matmul计算需要计算出多个singleCoreM * singleCoreN大小的C矩阵，且单次Matmul计算处理的shape较小。

约束说明

- 只支持Norm模板。
- 对于BSNGD、SBNGD、BNGS1S2 Layout格式，输入A、B矩阵按分形对齐后的多Batch数据总和应小于L1 Buffer的大小；对于NORMAL Layout格式没有这种限制，但需通过MatmulConfig配置batchMode参数，即输入A、B矩阵多Batch数据大小与L1 Buffer的大小关系；
- 对于BSNGD、SBNGD、BNGS1S2 Layout格式，称左矩阵、右矩阵的G轴分别为ALayoutInfoG、BLayoutInfoG，则ALayoutInfoG / batchA = BLayoutInfoG / batchB；对于NORMAL Layout格式，batchA、batchB必须满足倍数关系。Bias的shape(batch, n)中的batch必须与C矩阵的batch相等。
- 如果接口输出到Unified Buffer上，输出C矩阵大小BaseM*BaseN应小于分配的Unified Buffer内存大小。

- 对于BSNGD、SBNGD Layout格式，输入输出只支持ND格式数据。对于BNGS1S2、NORMAL Layout格式，输入支持ND/NZ格式数据。
- Batch Matmul不支持量化/反量化模式，即不支持SetQuantScalar、SetQuantVector接口。
- BSNGD场景，不支持一次计算多行SD，需要算子程序中循环计算。
- **异步模式**不支持IterateBatch搬运到Unified Buffer上。
- 模板参数enableMixDualMaster（默认取值为false）设置为true，即使能MixDualMaster（双主模式）场景时，不支持Batch Matmul。
- 在batch场景，A矩阵、B矩阵支持half/float/bfloat16_t/int8_t数据类型，不支持int4b_t数据类型。

调用示例

以下是NORMAL数据排布格式的Batch Matmul调用示例。BSNDG数据排布格式的Batch Matmul完整示例请参考[BatchMatmul样例](#)。

- Tiling实现

使用SetBatchInfoForNormal设置A/B/C的M/N/K轴信息和A/B矩阵的BatchNum。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
matmul_tiling::MultiCoreMatmulTiling tiling(ascendcPlatform);
int32_t M = 32;
int32_t N = 256;
int32_t K = 64;
tiling->SetDim(1);
tiling->SetAType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT16);
tiling->SetBType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT16);
tiling->SetCType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT);
tiling->SetBiasType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,
matmul_tiling::DataType::DT_FLOAT);
tiling->SetShape(M, N, K);
tiling->SetOrgShape(M, N, K);
tiling->EnableBias(true);
tiling->SetBufferSpace(-1, -1, -1);

constexpr int32_t BATCH_NUM = 3;
tiling->SetBatchInfoForNormal(BATCH_NUM, BATCH_NUM, M, N, K); // 设置矩阵排布
tiling->SetBufferSpace(-1, -1, -1);

optiling::TCubeTiling tilingData;
int ret = tiling.GetTiling(tilingData);
```

- Kernel实现

- 创建Matmul对象。

通过MatmulType设置输入输出的Layout格式为NORMAL。

```
#include "lib/matmul_intf.h"

typedef AscendC::MatmulType <AscendC::TPosition::GM, CubeFormat::ND, half, false,
LayoutMode::NORMAL> aType;
typedef AscendC::MatmulType <AscendC::TPosition::GM, CubeFormat::ND, half, true,
LayoutMode::NORMAL> bType;
typedef AscendC::MatmulType <AscendC::TPosition::GM, CubeFormat::ND, float, false,
LayoutMode::NORMAL> cType;
typedef AscendC::MatmulType <AscendC::TPosition::GM, CubeFormat::ND, float> biasType;
constexpr MatmulConfig MM_CFG = GetNormalConfig(false, false, false,
BatchMode::BATCH_LESS_THAN_L1);
AscendC::Matmul<aType, bType, cType, biasType, MM_CFG> mm;
```

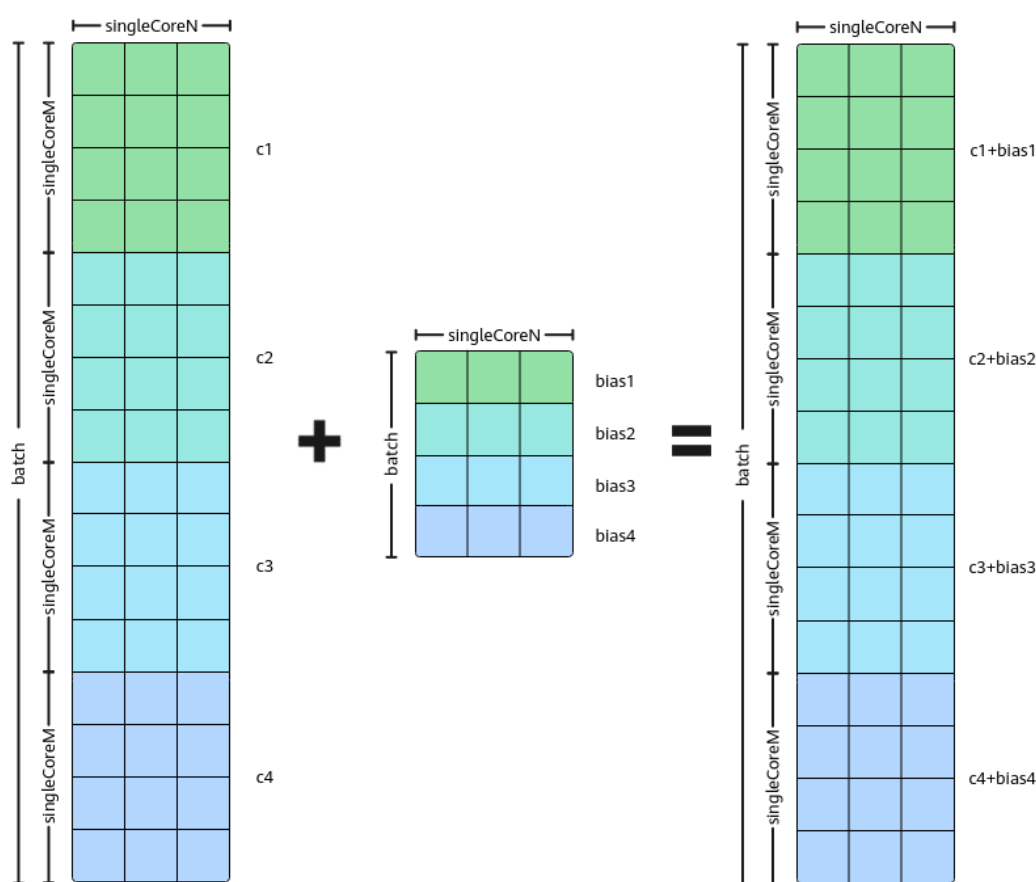
- 初始化操作。
`REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling);` // 初始化matmul对象
- 设置左矩阵A、右矩阵B、Bias。
`mm.SetTensorA(gm_a);` // 设置左矩阵A
`mm.SetTensorB(gm_b);` // 设置右矩阵B
`mm.SetBias(gm_bias);` // 设置Bias
- 完成矩阵乘操作。左矩阵每次计算batchA个MK数据，右矩阵每次计算batchB个KN数据。
`mm.IterateBatch(gm_c, batchA, batchB, false);`
- 结束矩阵乘操作。
`mm.End();`

3.3.3.3.14 Batch Matmul 复用 Bias 矩阵

功能介绍

在Batch Matmul场景中，Matmul API可以一次性计算出多个大小为 $\text{singleCoreM} * \text{singleCoreN}$ 的C矩阵。当Batch Matmul场景有Bias输入时，默认的Bias输入矩阵包含Batch轴，即Bias的大小为 $\text{Batch} * \text{N}$ 。通过开启Bias复用功能，当每个Batch计算使用的Bias数据相同时，只需输入一个不带Batch轴的Bias矩阵。Batch Matmul的Bias矩阵复用功能默认不启用，用户需要设置MatmulConfig中的isBiasBatch参数为false来开启此功能。

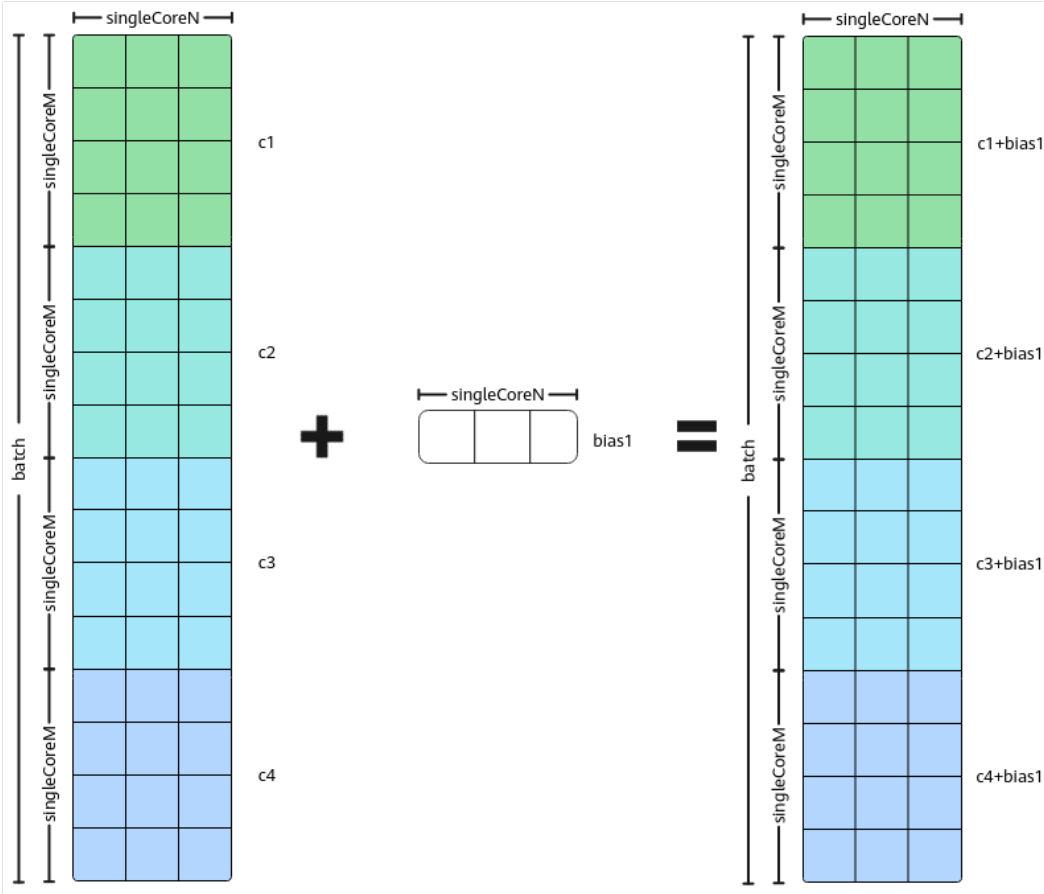
图 3-44 带有 Batch 轴的 Bias 计算示意图



如上图所示，Batch Matmul中未复用Bias矩阵的场景，每计算出一个 $\text{singleCoreM} * \text{singleCoreN}$ 大小的C矩阵，都会与 $1 * \text{singleCoreN}$ 大小的Bias矩阵相加。若不同Batch

的计算使用的Bias数据相同，则多Batch计算可以复用同一个Bias矩阵，如下图所示，此场景中调用SetBias接口时，只需设置一个1 * singleCoreN大小的Bias矩阵。

图 3-45 复用 Bias 计算示意图



使用场景

Batch Matmul中每个Batch的Matmul计算可以使用相同的Bias矩阵。

约束说明

A、B、C矩阵的Layout类型都为NORMAL时，不支持batchMode参数设为 SINGLE_LARGE_THAN_L1，即Bias复用场景下，单Batch的A、B矩阵数据总和不得超过L1 Buffer的大小。

调用示例

完整的算子样例请参考[BatchMatmul复用Bias算子样例](#)。

```
// 自定义MatmulConfig参数，将其中的isBiasBatch参数设置为false，使能BatchMatmul的Bias复用功能。
constexpr MatmulConfigMode configMode = MatmulConfigMode::CONFIG_NORM;
constexpr MatmulBatchParams batchParams = {
    false, BatchMode::BATCH_LESS_THAN_L1, false /* isBiasBatch */
};
constexpr MatmulConfig CFG_MM = GetMMConfig<configMode>(batchParams);
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_MM> mm;

REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling); // 初始化matmul对象
```

```
mm.SetTensorA(gm_a); // 设置左矩阵A
mm.SetTensorB(gm_b); // 设置右矩阵B
mm.SetBias(gm_bias); // 设置Bias, 矩阵大小为1 * singleCoreN
mm.IterateBatch(gm_c, batchA, batchB, false);
mm.End();
```

3.3.4 矩阵编程（基础 API）

3.3.4.1 耦合模式

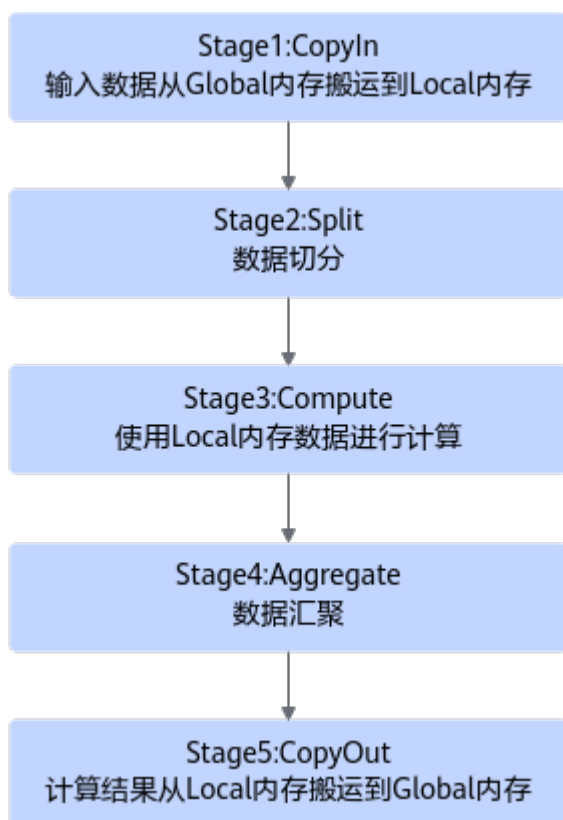
说明

本节内容为针对**耦合模式**，使用基础API进行矩阵乘法的编程指导。

编程范式

Cube编程范式把算子的实现流程分为5个基本任务：CopyIn，Split，Compute，Aggregate，CopyOut。CopyIn负责搬入操作，Split负责数据切分操作，Compute负责矩阵指令计算操作，Aggregate负责数据汇聚操作，CopyOut负责搬出操作。

图 3-46 矩阵编程基本任务设计



具体任务之间的交互流程和流程图如下。

步骤1 Stage1：CopyIn任务。

1. 使用DataCopy接口将GlobalTensor数据拷贝到LocalTensor。
2. 使用EnQue将LocalTensor放入A1/B1的Queue中。

步骤2 Stage2: Split任务。

1. 使用DeQue从A1/B1中取出LocalTensor。
2. 使用LoadData接口将LocalTensor从A1/B1中搬运到A2/B2。
3. 使用EnQue将计算结果LocalTensor放入到A2/B2的Queue中。

步骤3 Stage3: Compute任务。

1. 使用DeQue从A2/B2中取出LocalTensor。
2. 使用Mmad接口完成矩阵计算。
3. 使用EnQue将计算结果LocalTensor放入到CO1的Queue中。

步骤4 Stage4: Aggregate任务。

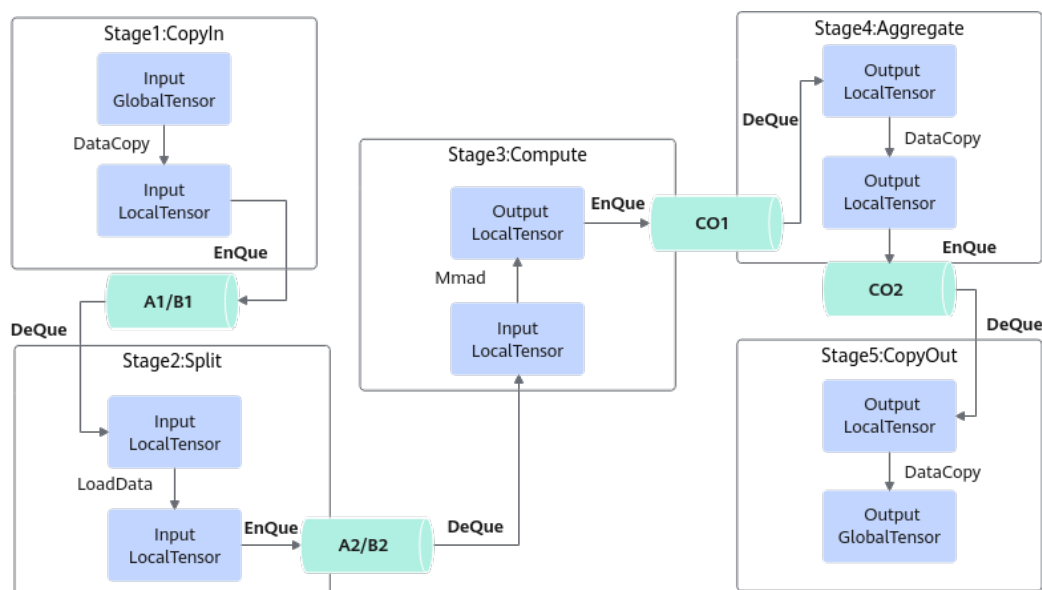
1. 使用DeQue从CO1中取出LocalTensor。
2. 使用Ascend C接口拷贝结果矩阵到CO2。
3. 使用EnQue将计算结果LocalTensor放入到CO2的Queue中。

步骤5 Stage5: CopyOut任务。

1. 使用DeQue接口从CO2的Queue中取出LocalTensor。
2. 使用DataCopy接口将LocalTensor拷贝到GlobalTensor上。

----结束

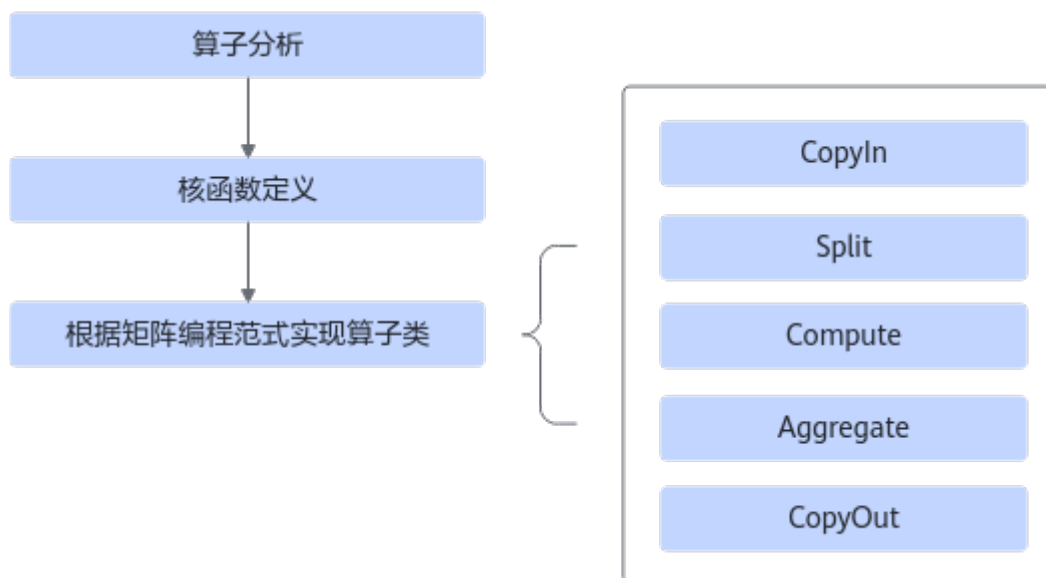
图 3-47 矩阵编程 Queue 队列



开发流程

基于Ascend C方式实现矩阵算子的流程如下图所示。

图 3-48 矩阵算子实现流程



- 算子分析：分析算子的数学表达式、输入、输出以及计算逻辑的实现，明确需要调用的Ascend C接口。
- 核函数定义：定义Ascend C算子入口函数。
- 根据矩阵编程范式实现算子类：完成核函数的内部实现，调用私有成员函数CopyIn、SplitA、SplitB、Compute、Aggregate、CopyOut完成矩阵算子的五级流水操作。

下文将以Matmul算子为例对上述步骤进行详细介绍，Matmul算子的代码框架如下，完整代码请参见[Mmad样例](#)。

```
#include "kernel_operator.h"

// 根据编程范式实现算子类
class KernelMatmul {
public:
    __aicore__ inline void Init(GM_ADDR a, GM_ADDR b, GM_ADDR c)
    {
        // ...
    }
    __aicore__ inline void Process()
    {
        CopyIn();
        SplitA();
        AscendC::LocalTensor<half> b1Local = inQueueB1.DeQue<half>();
        AscendC::LocalTensor<half> a2Local = inQueueA2.DeQue<half>();
        AscendC::LocalTensor<float> c2Local = outQueueCO2.AllocTensor<float>();
        // split matrix b into 2 parts, [32, 16] and [32, 16]
        for (int i = 0; i < 2; ++i) {
            SplitB(b1Local, i);
            Compute(a2Local);
            Aggregate(c2Local, i);
        }
        inQueueB1.FreeTensor(b1Local);
        inQueueA2.FreeTensor(a2Local);
        outQueueCO2.EnQue<float>(c2Local);
        CopyOut();
    }
private:
    __aicore__ inline void CopyIn()
    {
        // ...
    }
};
```

```
}
__aicore__ inline void SplitA()
{
    // ...
}
__aicore__ inline void SplitB(const LocalTensor<half>& b1Local, const int bSplitIdx)
{
    // ...
}
__aicore__ inline void Compute(const LocalTensor<half>& a2Local)
{
    // ...
}
__aicore__ inline void Aggregate(const LocalTensor<float>& c2Local, const int bSplitIdx)
{
    // ...
}
__aicore__ inline void CopyOut()
{
    // ...
}
private:
    // ...
};

//核函数定义
extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR c)
{
    KernelMatmul op;
    op.Init(a, b, c);
    op.Process();
}
```

算子分析

在开发算子代码之前需要分析算子的数学表达式、输入、输出以及计算逻辑的实现，明确需要调用的Ascend C接口。

步骤1 明确算子的数学表达式及计算逻辑。

Matmul算子完成矩阵乘操作，其数学表达式如下，形状为[m, k]的矩阵a和形状为[k, n]的矩阵b相乘，得到形状为[m, n]的矩阵c。为了方便，令m=k=n=32。
$$c = a * b$$

注意需要处理的数据过大时，需要对数据进行切分并分块搬运到A2、B2，分别计算后再进行汇聚。下文的计算逻辑为了展示Split和Aggregate阶段的样例，请您根据实际需要处理的数据大小决定是否需要进行切分和汇聚。

计算逻辑如下：

1. 分别搬运输入数据矩阵a、b至Local Memory A1、B1。
2. 将a矩阵从A1搬运至A2。为实现部分并行，将b矩阵切分为part1和part2，形状均为[k, n / 2]，切分后再分块搬运至B2。
3. a矩阵和b矩阵part1、part2分别做矩阵乘运算，获得矩阵c的part1和part2，形状均为[m, n / 2]。计算结果在CO1存储。
4. 将矩阵c的part1和part2分别拷贝到CO2进行合并。
5. 将合并后的输出数据从CO2搬出。

步骤2 明确输入和输出。

- Matmul算子有两个输入：a与b，输出为c。

- 本样例中算子输入支持的数据类型为half（float16），算子输出的数据类型为float32。
- 矩阵a、b、c的形状均为[32, 32]。
- 算子输入输出支持的数据格式为：ND。

步骤3 确定核函数名称和参数。

- 您可以自定义核函数名称，本样例中核函数命名为matmul_custom。
- 根据对算子输入输出的分析，确定核函数有3个参数a, b, c；a, b为输入在Global Memory上的内存地址，c为输出在Global Memory上的内存地址。

步骤4 约束分析。

由于硬件架构对矩阵乘计算的输入输出有格式约束，需要在算子实现中增加格式转换的流程。

- 搬运矩阵a、b至A1、B1时，将ND格式的矩阵a、b转换为NZ格式。
- 从A1搬运矩阵a至A2时，将NZ格式的a矩阵转换为ZZ格式；从B1搬运矩阵b到B2时将NZ格式的b矩阵转换为ZN格式。
- 将计算结果从CO2搬出时，将NZ格式的c矩阵转换为ND格式。
- 数据排布格式的相关介绍详见[数据排布格式](#)。

步骤5 确定算子实现所需接口。

- 实现外部存储和内部存储间的数据搬运，查看Ascend C API参考中的数据搬移接口，具体参考DataCopy。
- 实现矩阵数据格式转换，查看Ascend C API参考中的数据转换接口，具体参考LoadData。
- 矩阵计算过程涉及矩阵乘法，查看Ascend C API参考中的矩阵计算接口，具体参考Mmad。
- 计算中使用到的Tensor数据结构，使用Queue队列进行管理，会使用到EnQue、DeQue等接口。

----结束

通过以上分析，得到Ascend C Matmul算子的计算流程图和设计规格如下：

图 3-49 Matmul 算子的计算流程图

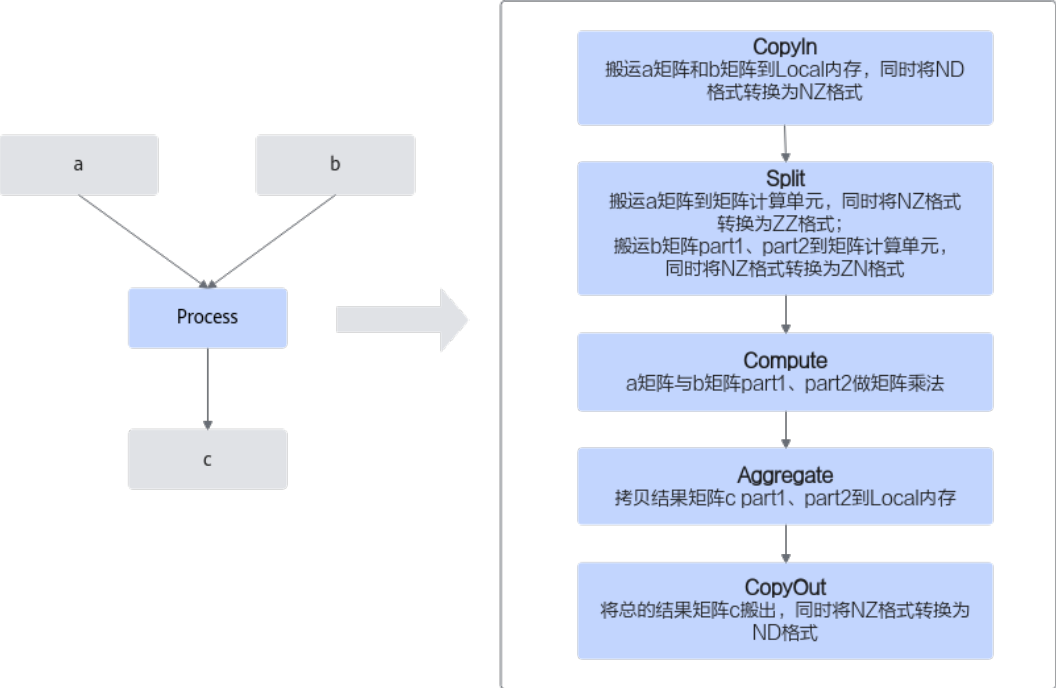


表 3-9 Ascend C Matmul 算子设计规格

算子类型 (OpType)	Matmul			
算子输入	name	shape	data type	format
	a	(m, k) = (32, 32)	half	ND
	b	(k, n) = (32, 32)	half	ND
算子输出	c	(m, n) = (32, 32)	float32	ND
核函数名称	matmul_custom			
使用的主要接口	DataCopy：数据搬移接口			
	LoadData：矩阵数据格式转换接口			
	Mmad：矩阵乘计算接口			
	EnQue、DeQue等接口：Queue队列管理接口			
算子实现文件名称	matmul_custom.cpp			

核函数定义

根据2.2.3 核函数中介绍的规则进行核函数的定义。

步骤1 函数原型定义。

本样例中，函数名为matmul_custom（核函数名称可自定义）；根据[算子分析](#)中对算子输入输出的分析，确定有3个参数a，b，c，其中a，b都为输入内存，c为输出内存。根据[2.2.3 核函数](#)中核函数的规则介绍，函数原型定义如下所示：使用__global__函数类型限定符来标识它是一个核函数，可以被<<<>>>调用；使用__aicore__函数类型限定符来标识该核函数在设备端aicore上执行；为方便起见，统一使用GM_ADDR宏修饰入参，GM_ADDR宏定义请参考[2.2.3 核函数](#)。

```
extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR c)
{
}
```

步骤2 调用算子类的Init和Process函数。

算子类的Init函数，完成内存初始化相关工作，Process函数完成算子实现的核心逻辑，具体介绍参见[算子类实现](#)。

```
extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR c)
{
    KernelMatmul op;
    op.Init(a, b, c);
    op.Process();
}
```

步骤3 对核函数进行封装，得到matmul_custom_do函数，便于主程序调用。#ifndef ASCENDC_CPU_DEBUG表示该封装函数仅在编译运行NPU侧的算子时会用到，编译运行CPU侧的算子时，可以直接调用matmul_custom函数。根据[核函数定义和调用](#)章节，调用核函数时，除了需要传入参数a，b，c，还需要传入numBlocks（核函数执行的核数），l2ctrl（保留参数，设置为nullptr），stream（应用程序中维护异步操作执行顺序的stream）来规定核函数的执行配置。

```
#ifndef ASCENDC_CPU_DEBUG
// call of kernel function
void matmul_custom_do(uint32_t numBlocks, void* l2ctrl, void* stream, uint8_t* a, uint8_t* b, uint8_t* c)
{
    matmul_custom<<<numBlocks, l2ctrl, stream>>>>(a, b, c);
}
#endif
```

----结束

算子类实现

根据上一章节介绍，核函数中会调用算子类的Init和Process函数，本章具体讲解基于编程范式实现算子类。矩阵编程范式请参考[编程范式](#)。

算子类中主要包含对外开放的初始化Init函数和核心处理函数Process以及一些实现中会用到的私有成员。KernelMatmul算子类的定义如下：

```
class KernelMatmul {
public:
    __aicore__ inline KernelMatmul(){}
    // 初始化函数，完成内存初始化相关操作
    __aicore__ inline void Init(GM_ADDR a, GM_ADDR b, GM_ADDR c){}
    // 核心处理函数，实现算子逻辑
    // 调用私有成员函数CopyIn、SplitA、SplitB、Compute、Aggregate、CopyOut完成矩阵算子的五级流水操作
    __aicore__ inline void Process(){}

private:
    __aicore__ inline void CopyND2NZ(const LocalTensor<half>& dst, const GlobalTensor<half>& src, const
uint16_t height, const uint16_t width){}
    // 搬进函数，完成编程范式中的CopyIn阶段的处理，由Process函数调用
    __aicore__ inline void CopyIn(){}
    // 搬进函数，完成编程范式中的Split阶段的处理，由Process函数调用
    __aicore__ inline void SplitA(){}
}
```

```
// 搬进函数，完成编程范式中的Split阶段的处理，由Process函数循环调用两次，分别搬运b矩阵的两个part
__aicore__ inline void SplitB(const LocalTensor<half>& b1Local, const int bSplitIdx){}
// 计算函数，完成编程范式中的Compute阶段的处理，由Process函数循环调用两次，分别计算出矩阵c的两个part
__aicore__ inline void Compute(const LocalTensor<half>& a2Local){}
// 搬出函数，完成编程范式中的Aggregate阶段的处理，由Process函数循环调用两次，分别搬出矩阵c的两个part
__aicore__ inline void Aggregate(const LocalTensor<float>& c2Local, const int bSplitIdx){}
// 搬出函数，完成编程范式中的CopyOut阶段的处理，由Process函数调用
__aicore__ inline void CopyOut(){}

private:
    AscendC::TPipe pipe; // Pipe内存管理对象，管理Queue队列的内存
    AscendC::TQue<AscendC::TPosition::A1, 1> inQueueA1; // 输入数据的队列，TPosition为A1
    AscendC::TQue<AscendC::TPosition::A2, 1> inQueueA2; // 输入数据的队列，TPosition为A2
    AscendC::TQue<AscendC::TPosition::B1, 1> inQueueB1; // 输入数据的队列，TPosition为B1
    AscendC::TQue<AscendC::TPosition::B2, 2> inQueueB2; // 输入数据的队列，TPosition为B2
    AscendC::TQue<AscendC::TPosition::CO1, 2> outQueueCO1; // 输出数据的队列，TPosition为CO1
    AscendC::TQue<AscendC::TPosition::CO2, 1> outQueueCO2; // 输出数据的队列，TPosition为CO2
    // 管理输入输出Global Memory内存地址的对象，其中aGM, bGM为输入，cGM为输出
    AscendC::GlobalTensor<half> aGM, bGM;
    AscendC::GlobalTensor<float> cGM;

    uint16_t m = 32;
    uint16_t n = 32;
    uint16_t k = 32;
    uint16_t aSize, bSize, cSize, mBlocks, nBlocks, kBlocks;
};
```

KernelMatmul构造函数实现

构造函数中对私有成员变量进行初始化，具体代码如下：

```
__aicore__ inline KernelMatmul()
{
    aSize = m * k;
    bSize = k * n;
    cSize = m * n;
    mBlocks = m / 16;
    nBlocks = n / 16;
    kBlocks = k / 16;
}
```

矩阵a的形状为[m, k]，矩阵b的形状为[k, n]，矩阵c的形状为[m,n]，此样例中m、n、k均设置为32。

aSize、bSize、cSize分别为矩阵a、b、c的数值个数。

mBlocks、nBlocks、kBlocks为m、n、k所占分形数量，half类型一个分形Shape为16 * 16，blocks计算公式为：

- $mBlocks = m / 16$
- $nBlocks = n / 16$
- $kBlocks = k / 16$

分形具体介绍可参考[2.9.2.2 数据排布格式](#)。

Init函数实现

Init函数主要完成以下内容：

- 设置输入输出Global Tensor的Global Memory内存地址。

以设置输入a在Global Memory上的内存偏移地址为例：

```
aGM.SetGlobalBuffer((__gm__ half*)a);
```

注意，因为本样例中Init函数的入参统一设置为uint8_t*，这里需要强转成具体的数据类型(__gm__ half*)，再进行偏移。

- 通过Pipe内存管理对象为输入输出Queue分配内存。

比如，为输入数据队列inQueueB2分配内存，可以通过如下代码段实现：

```
pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half) / 2);
```

此样例中将b矩阵切分为两个part，为inQueueB2分配内存时需要申请两块内存空间，每一块的大小为b矩阵大小的一半，outQueueCO1的内存初始化同理。

具体的初始化函数代码如下：

```
__aicore__ inline void Init(GM_ADDR a, GM_ADDR b, GM_ADDR c)
{
    aGM.SetGlobalBuffer((__gm__ half*)a);
    bGM.SetGlobalBuffer((__gm__ half*)b);
    cGM.SetGlobalBuffer((__gm__ float*)c);
    pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));
    pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));
    pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
    pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half) / 2);
    pipe.InitBuffer(outQueueCO1, 2, cSize * sizeof(float) / 2);
    pipe.InitBuffer(outQueueCO2, 1, cSize * sizeof(float));
}
```

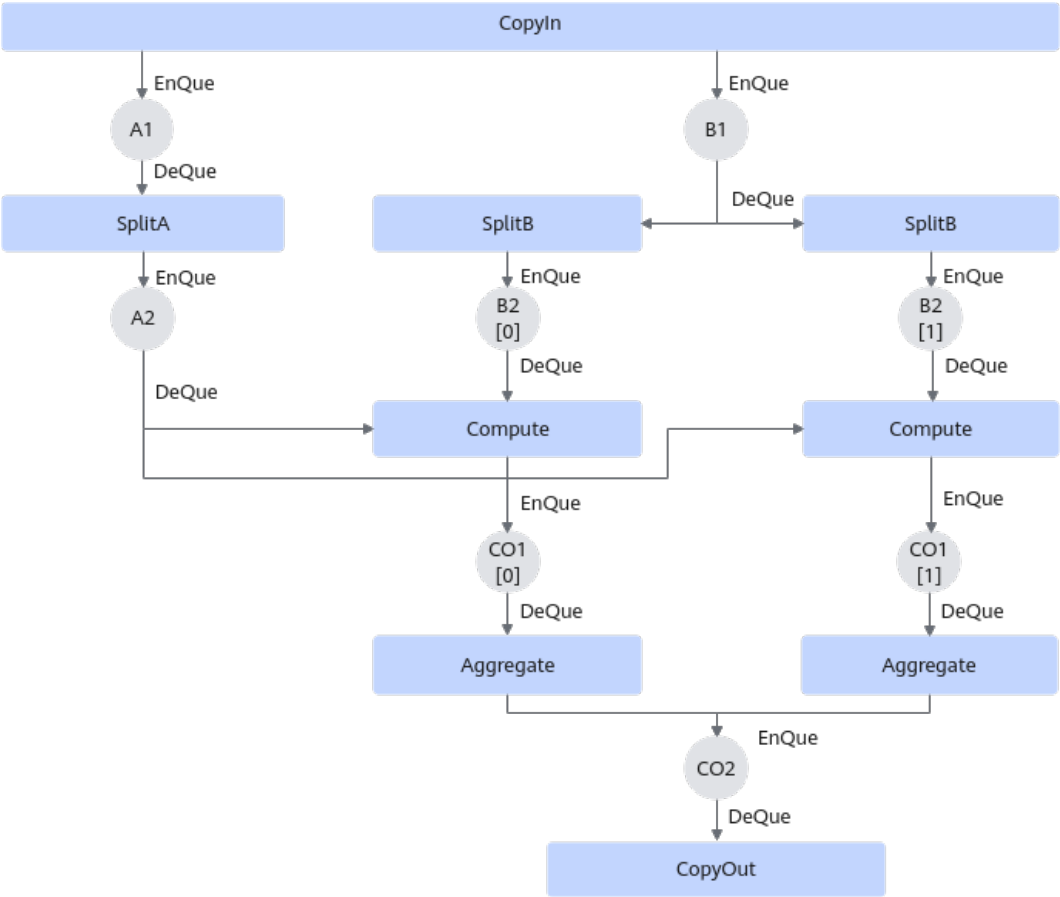
Process函数实现

基于矩阵编程范式，将核函数的实现分为5个基本阶段：CopyIn, Split, Compute, Aggregate, CopyOut。Split, Compute, Aggregate阶段需要区分a、b矩阵。Process函数中通过如下方式调用这几个函数。

```
__aicore__ inline void Process()
{
    CopyIn();
    SplitA();
    AscendC::LocalTensor<half> b1Local = inQueueB1.DeQue<half>();
    AscendC::LocalTensor<half> a2Local = inQueueA2.DeQue<half>();
    AscendC::LocalTensor<float> c2Local = outQueueCO2.AllocTensor<float>();
    // split matrix b into 2 parts, [32, 16] and [32, 16]
    for (int i = 0; i < 2; ++i) {
        SplitB(b1Local, i);
        Compute(a2Local);
        Aggregate(c2Local, i);
    }
    inQueueB1.FreeTensor(b1Local);
    inQueueA2.FreeTensor(a2Local);
    outQueueCO2.EnQue<float>(c2Local);
    CopyOut();
}
```

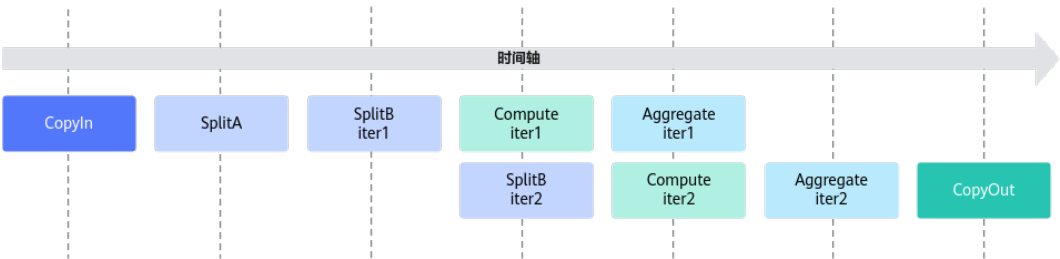
两次循环内，SplitB需要从inQueueB1中分别搬运两个part的b矩阵，Compute需要分别计算a矩阵和两个part b矩阵的乘法，Aggregate要分别搬运两个part的c矩阵，具体五个阶段数据流通示意图如下：

图 3-50 数据流通示意图



切分b矩阵，可以实现一部分的并行，本样例的流水并行示意图如下：

图 3-51 并行示意图

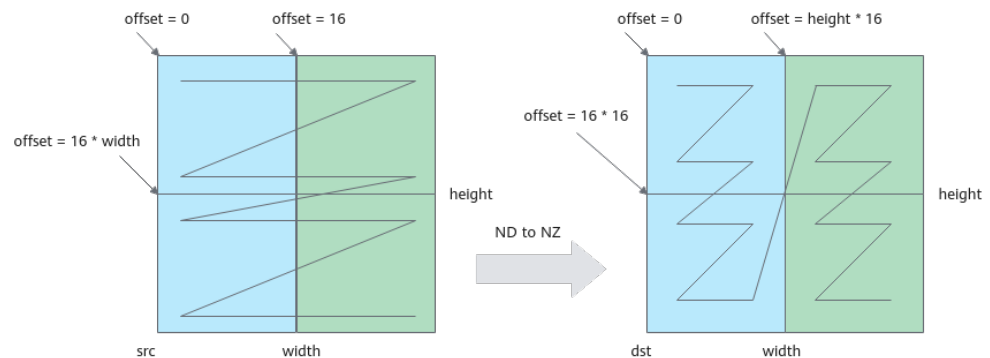


- 步骤1** Stage1：CopyIn函数实现。
1. 使用AllocTensor从A1，B1的Queue中申请a1Local和b1Local。
 2. 使用DataCopy接口将矩阵a、b搬运到Local Memory，同时将其数据格式从ND转换为NZ。
一次DataCopy指令搬运height*16个数，循环执行width/16次。DataCopy的参数设置如下：
 - blockCount设置为height，共搬运height次。
 - blockLen设置为1，一次搬运16个类型为half的数。
 - srcStride设置为width/16 - 1，源矩阵每搬运一个block需要跳跃一行。

- dstStride设置为0，目的矩阵每个block在内存中连续存储。
- 每次循环迭代，源矩阵首地址移动16个数，目的矩阵首地址移动16*height个数。

格式转换示意图如下，第一次循环搬运蓝色部分，第二次循环搬运绿色部分；图中width为32，占两个分形，height为32，占两个分形，一共搬运4个16*16分形。

图 3-52 ND to NZ 转换示意图



注意：上述ND到NZ的格式转换仅作为举例说明，开发者可根据实际情况选择合适的转换方式。

3. 使用EnQue将a1Local、b1Local分别放入A1、B1的Queue中。

具体代码如下：

```
__aicore__ inline void CopyND2NZ(const LocalTensor<half>& dst, const GlobalTensor<half>& src, const
uint16_t height, const uint16_t width)
{
    for (int i = 0; i < width / 16; ++i) {
        int srcOffset = i * 16;
        int dstOffset = i * 16 * height;
        AscendC::DataCopy(dst[dstOffset], src[srcOffset], { height, 1, uint16_t(width / 16 - 1), 0 });
    }
}

__aicore__ inline void CopyIn()
{
    AscendC::LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
    AscendC::LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();
    CopyND2NZ(a1Local, aGM, m, k);
    CopyND2NZ(b1Local, bGM, k, n);
    inQueueA1.EnQue(a1Local);
    inQueueB1.EnQue(b1Local);
}
```

步骤2 Stage2: SplitA函数实现。

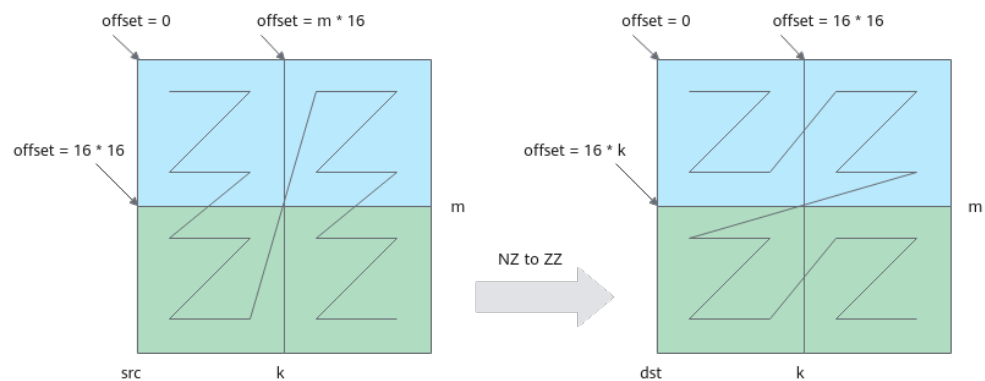
1. 使用DeQue从A1的Queue中取出a1Local。
2. 使用AllocTensor从A2的Queue中申请a2Local。
3. 使用LoadData将矩阵a搬运到A2，同时将a矩阵从NZ格式转换为ZZ格式。

搬运及格式转换示意图如下：图中k为32，占kBlocks（k/16=2）个分形，m为32，占mBlocks（m/16=2）个分形，一共搬运4个16*16分形。本示例中，调用一次LoadData接口完成两个16*16分形的搬运，循环调用两次LoadData。第一次循环搬运蓝色部分两个分形，第二次循环搬运绿色部分两个分形。

单次循环中LoadData（本样例中要完成2个分形的搬运，蓝色部分或者绿色部分）的参数设置如下：

- repeatTimes表示数据处理的迭代次数，因为LoadData每个迭代处理一个分形，所以也可以理解为待搬运分形的个数。本样例中即为k轴方向的分形个数，设置为kBlocks，表示搬运kBlocks个分形。
- srcStride表示，相邻迭代间源操作数分形首地址之间的间隔，以搬运蓝色部分分形为例：下图中左侧源操作数矩阵，第一个蓝色分形和第二个蓝色分形起始地址之间的间隔为mBlocks个分形，此处设置为mBlocks。
- dstGap使用默认值，目的矩阵两个分形连续存储。
- ifTranspose设置为false，每块分形搬运前搬运后都为Z格式，不使能转置。
- 每次循环迭代源矩阵首地址偏移 16×16 ，目的矩阵首地址偏移 $16 \times k$ 。

图 3-53 NZ to ZZ 格式转换示意图



4. 使用EnQue将计算结果a2Local放入到A2的Queue中。

具体代码如下：

```
__aicore__ inline void SplitA()
{
    int srcOffset = 0;
    int dstOffset = 0;
    AscendC::LocalTensor<half> a1Local = inQueueA1.DeQue<half>();
    AscendC::LocalTensor<half> a2Local = inQueueA2.AllocTensor<half>();

    // transform nz to zz
    for (int i = 0; i < mBlocks; ++i) {
        AscendC::LoadData2DParams loadDataParams;
        loadDataParams.repeatTimes = kBlocks;
        loadDataParams.srcStride = mBlocks;
        loadDataParams.ifTranspose = false;

        AscendC::LoadData(a2Local[dstOffset], a1Local[srcOffset], loadDataParams);

        srcOffset += 16 * 16;
        dstOffset += k * 16;
    }

    inQueueA2.EnQue<half>(a2Local);
    inQueueA1.FreeTensor(a1Local);
}
```

步骤3 Stage2: SplitB函数实现。

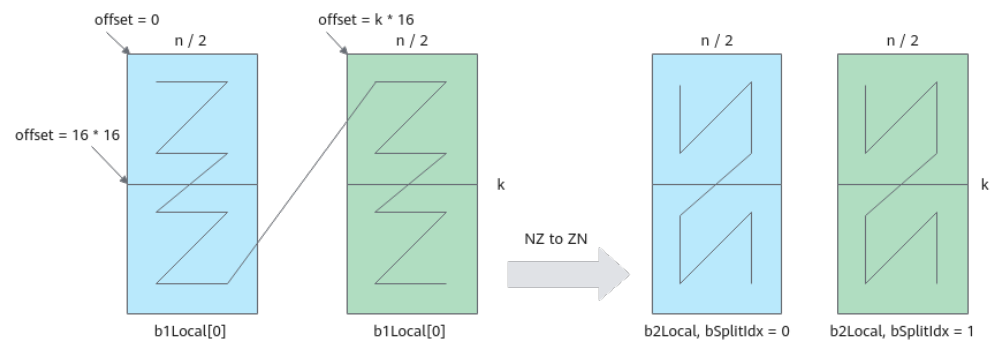
1. SplitB需要传入两个参数：使用DeQue从B1的Queue中取出的b1Local和循环迭代变量index。
2. 使用AllocTensor从B2的Queue中申请b2Local。
3. 使用LoadData将b矩阵搬运到B2，同时从NZ格式转换为ZN格式。

搬运及格式转换示意图如下：图中k为32，占kBlocks（ $k/16=2$ ）个分形，n为32，占nBlocks（ $n/16=2$ ）个分形，一共搬运4个 $16*16$ 分形。本示例中，调用一次LoadData接口完成两个 $16*16$ 分形的搬运，循环调用两次LoadData。第一次循环搬运蓝色部分两个分形，第二次循环搬运绿色部分两个分形。

单次循环中LoadData（本样例中要完成2个分形的搬运，蓝色部分或者绿色部分）的参数设置如下：

- repeatTimes表示数据处理的迭代次数，因为LoadData每个迭代处理一个分形，所以也可以理解为待搬运分形的个数。本样例中即为k轴方向的分形个数，设置为kBlocks，表示搬运kBlocks个分形。
- srcStride相邻迭代间源操作数分形首地址之间的间隔，以搬运蓝色部分分形为例：下图中左侧源操作数矩阵，第一个蓝色分形和第二个蓝色分形起始地址之间的间隔为1个分形，此处设置为1，源矩阵两个分形连续存储。
- dstGap使用默认值0，目的矩阵两个分形连续存储。
- ifTranspose设置为true，每块分形搬运前为Z格式，搬运后需要为N格式，需要使能转置。
- 每次循环迭代，源矩阵首地址需要偏移 $k*n/2$ 。

图 3-54 NZ to ZN 格式转换示意图



4. 使用EnQueue将计算结果b2Local放入到B2的Queue中。

具体代码如下：

```
__aicore__ inline void SplitB(const AscendC::LocalTensor<half>& b1Local, const int bSplitIdx)
{
    AscendC::LocalTensor<half> b2Local = inQueueB2.AllocTensor<half>();

    // transform nz to zn
    AscendC::LoadData2DParams loadDataParams;
    loadDataParams.repeatTimes = kBlocks;
    loadDataParams.srcStride = 1;
    loadDataParams.ifTranspose = true;

    AscendC::LoadData(b2Local, b1Local[bSplitIdx * bSize / 2], loadDataParams);

    inQueueB2.Enqueue<half>(b2Local);
}
```

步骤4 Stage3: Compute函数实现，完成核心的矩阵计算功能。

1. Compute函数需要传入参数a2Local，a2Local从A2的Queue中使用DeQueue取出。
2. 使用AllocTensor从CO1的Queue中申请c1Local。
3. 使用DeQueue从B2中取出b2Local。

4. 使用Mmad完成矩阵乘计算。
5. 使用EnQue将计算结果c1Local放入到CO1的Queue中。

具体代码如下：

```
__aicore__ inline void Compute(const AscendC::LocalTensor<half>& a2Local)
{
    AscendC::LocalTensor<half> b2Local = inQueueB2.DeQue<half>();
    AscendC::LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();

    AscendC::MmadParams mmadParams;
    mmadParams.m = m;
    mmadParams.n = n / 2;
    mmadParams.k = k;
    AscendC::Mmad(c1Local, a2Local, b2Local, mmadParams);

    outQueueCO1.EnQue<float>(c1Local);
    inQueueB2.FreeTensor(b2Local);
}
```

步骤5 Stage4: Aggregate函数实现，完成数据汇聚操作。

1. Aggregate需要传入两个参数：使用AllocTensor从CO2的Queue中申请的c2Local和循环迭代变量index。
2. 使用DeQue从CO1中取出c1Local。
3. 使用DataCopy将结果矩阵从CO1搬运到CO2。

DataCopy参数设置如下：

- blockCount设置为1，blockLen设置为2，连续搬运两个分形，无需格式转换。
- blockMode设置为BlockMode::BLOCK_MODE_MATRIX，表示需要按分形搬运。
- c2Local首地址偏移量设置为index * cSize / 2。

具体代码如下：

```
__aicore__ inline void Aggregate(const AscendC::LocalTensor<float>& c2Local, const int bSplitIdx)
{
    AscendC::LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();

    AscendC::DataCopyParams dataCopyParams;
    dataCopyParams.blockCount = 1;
    dataCopyParams.blockLen = 2;
    AscendC::DataCopyEnhancedParams enhancedParams;
    enhancedParams.blockMode = AscendC::BlockMode::BLOCK_MODE_MATRIX;
    AscendC::DataCopy(c2Local[bSplitIdx * cSize / 2], c1Local, dataCopyParams, enhancedParams);

    outQueueCO1.FreeTensor(c1Local);
}
```

步骤6 Stage5: CopyOut函数实现。

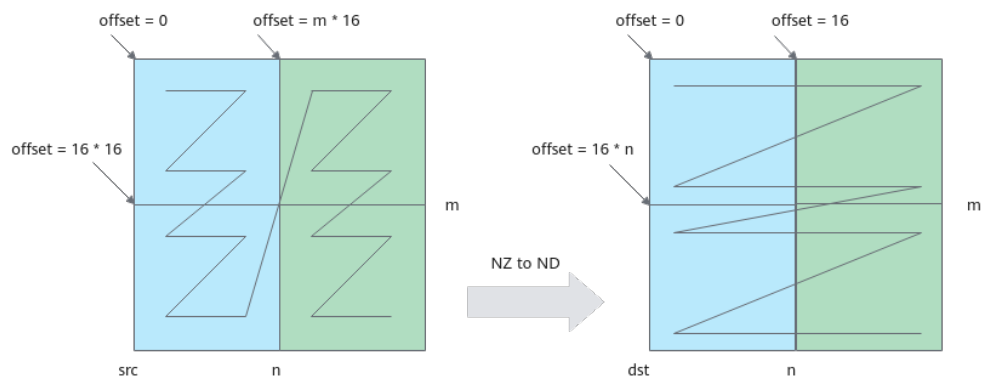
1. 使用DeQue从CO2中取出c2Local。
2. 使用DataCopy将结果矩阵从CO2搬运到Global Memory，同时需要将格式从NZ转换为ND。

每次循环移动一个分形，搬运m*16个数。DataCopy参数说明如下：

- blockCount设置为m，共搬运m次。
- blockLen设置为2，DataCopy指令一次搬运2个block，每个block16个数。
- srcStride设置为0，每两次搬运间没有间隙。

- dstStride设置为 $(nBlocks - 1) * 2$ ，每两次搬运间隔2个block。
 - 每次循环迭代，目的矩阵偏移16，源矩阵偏移 $m * 16$ 。
- 格式转换示意图如下，第一次循环搬运蓝色部分数据，第二次循环搬运绿色部分数据。

图 3-55 NZ to ND 格式转换示意图



具体代码如下：

```
__aicore__ inline void CopyOut()
{
    AscendC::LocalTensor<float> c2Local = outQueueCO2.DeQue<float>();

    // transform nz to nd
    for (int i = 0; i < nBlocks; ++i) {
        AscendC::DataCopy(cGM[i * 16], c2Local[i * m * 16], { m, 2, 0, uint16_t((nBlocks - 1) * 2) });
    }

    outQueueCO2.FreeTensor(c2Local);
}
```

----结束

3.3.4.2 分离模式

📖 说明

本节内容为针对分离模式，使用基础API进行矩阵乘法的编程指导。

针对**分离模式**，使用基础API进行矩阵乘法算子实现的编程范式和**3.3.4.1 耦合模式**一致，由于硬件架构不同，具体实现有一些差异，本节仅提供差异点说明。完整代码请参见**Mmad样例**。

• CopyIn阶段差异

- 耦合模式

在CopyIn阶段，即从GM->A1/B1（L1 Buffer）的阶段，耦合模式下可以使用DataCopy接口直接将数据从GM搬入L1 Buffer，也可以将数据从GM搬入UB，再搬入L1 Buffer。若需要ND2NZ的格式转换，需要开发者自行完成；或使用DataCopy接口提供的随路格式转换功能，但该功能会使用UB临时空间。

如下示例，直接使用了GM->A1/B1的数据搬运指令，自行完成ND2NZ的格式转换。

```
__aicore__ inline void CopyND2NZ(const AscendC::LocalTensor<half>& dst, const
AscendC::GlobalTensor<half>& src,
```

```
const uint16_t height, const uint16_t width)
{
    for (int i = 0; i < width / 16; ++i) {
        int srcOffset = i * 16;
        int dstOffset = i * 16 * height;
        AscendC::DataCopy(dst[dstOffset], src[srcOffset], { height, 1, uint16_t(width / 16 - 1),
0 });
    }
}

__aicore__ inline void CopyIn()
{
    AscendC::LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
    AscendC::LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();

    CopyND2NZ(a1Local, aGM, m, k);
    CopyND2NZ(b1Local, bGM, k, n);

    inQueueA1.Enqueue(a1Local);
    inQueueB1.Enqueue(b1Local);
}
```

– 分离模式

分离模式下数据无法经过VECIN/VECCALC/VECOUT (UB) 直接搬运到A1/B1 (L1 Buffer)，但是使用DataCopy接口提供的随路格式转换功能一条指令即可完成格式转换，无需UB作为临时空间。

示例如下：

```
__aicore__ inline void CopyIn()
{
    AscendC::LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
    AscendC::LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();

    AscendC::Nd2NzParams nd2nzA1Params;
    nd2nzA1Params.ndNum = 1;
    nd2nzA1Params.nValue = m;
    nd2nzA1Params.dValue = k;
    nd2nzA1Params.srcNdMatrixStride = 0;
    nd2nzA1Params.srcDValue = k;
    nd2nzA1Params.dstNzC0Stride = CeilCubeBlock(m) * CUBE_BLOCK;
    nd2nzA1Params.dstNzNStride = 1;
    nd2nzA1Params.dstNzMatrixStride = 0;
    AscendC::DataCopy(a1Local, aGM, nd2nzA1Params);

    AscendC::Nd2NzParams nd2nzB1Params;
    nd2nzB1Params.ndNum = 1;
    nd2nzB1Params.nValue = k;
    nd2nzB1Params.dValue = n;
    nd2nzB1Params.srcNdMatrixStride = 0;
    nd2nzB1Params.srcDValue = n;
    nd2nzB1Params.dstNzC0Stride = CeilCubeBlock(k) * CUBE_BLOCK;
    nd2nzB1Params.dstNzNStride = 1;
    nd2nzB1Params.dstNzMatrixStride = 0;
    AscendC::DataCopy(b1Local, bGM, nd2nzB1Params);

    inQueueA1.Enqueue(a1Local);
    inQueueB1.Enqueue(b1Local);
}
```

• Aggregate及CopyOut阶段差异

– 耦合模式

耦合模式中，完成矩阵乘计算后数据存放于CO1（L0C Buffer），最终搬入GM需要通过CO2（UB），且NZ2ND的格式转换需要在CO1->CO2->GM阶段中手动完成。如下样例，在Aggregate阶段将NZ格式数据从CO1搬入CO2中，在CO2->GM的阶段使用for循环调用DataCopy完成了格式转换。

```
__aicore__ inline void Aggregate(const AscendC::LocalTensor<float>& c2Local, const int
```

```
bSplitIdx)
{
    AscendC::LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();

    AscendC::DataCopyParams dataCopyParams;
    dataCopyParams.blockCount = 1;
    dataCopyParams.blockLen = 2;
    AscendC::DataCopyEnhancedParams enhancedParams;
    enhancedParams.blockMode = AscendC::BlockMode::BLOCK_MODE_MATRIX;
    AscendC::DataCopy(c2Local[bSplitIdx * cSize / 2], c1Local, dataCopyParams,
enhancedParams);

    outQueueCO1.FreeTensor(c1Local);
}
__aicore__ inline void CopyOut()
{
    AscendC::LocalTensor<float> c2Local = outQueueCO2.DeQue<float>();

    // transform nz to nd
    for (int i = 0; i < nBlocks; ++i) {
        AscendC::DataCopy(cGM[i * 16], c2Local[i * m * 16], { m, 2, 0, uint16_t((nBlocks - 1) *
2) });
    }

    outQueueCO2.FreeTensor(c2Local);
}
```

- 分离模式

分离模式下，矩阵乘的计算结果从CO1（L0C Buffer）可以通过Fixpipe通路直接写入GM，而且Fixpipe提供了随路NZ2ND的功能，方便用户做格式转换。样例如下，样例中省去了Aggregate阶段，直接CopyOut。

```
__aicore__ inline void CopyOut()
{
    AscendC::LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    AscendC::FixpipeParamsV220 fixpipeParams;
    fixpipeParams.nSize = n;
    fixpipeParams.mSize = m;
    fixpipeParams.srcStride = m;
    fixpipeParams.dstStride = n;

    fixpipeParams.ndNum = 1;
    fixpipeParams.srcNdStride = 0;
    fixpipeParams.dstNdStride = 0;
    AscendC::Fixpipe(cGM, c1Local, fixpipeParams);
    outQueueCO1.FreeTensor(c1Local);
}
```

3.3.5 融合算子编程

3.3.5.1 CV 融合

3.3.5.1.1 基础知识

说明

学习融合算子编程之前，请确保已经掌握[矩阵编程](#)相关知识。

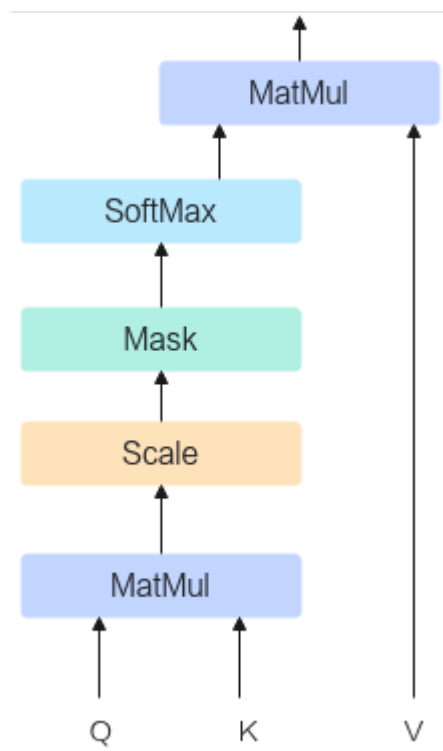
CV 融合算子

融合算子由多个独立的小算子融合而成，其功能与多个小算子的功能等价，性能方面通常优于独立的小算子。用户可以根据实际业务场景诉求，按照具体算法自由融合向

量（Vector）、矩阵（Cube）算子以达到性能上的收益。融合了Cube计算、Vector计算的算子统称为CV融合算子。

比如对于LLM大模型中最核心的一个融合算子Flash Attention，其核心实现如下图。图中的MatMul算子（Cube）、Scale算子（Vector）、Mask算子（Vector）、SoftMax算子（Vector）融合为一个大的算子Flash Attention。

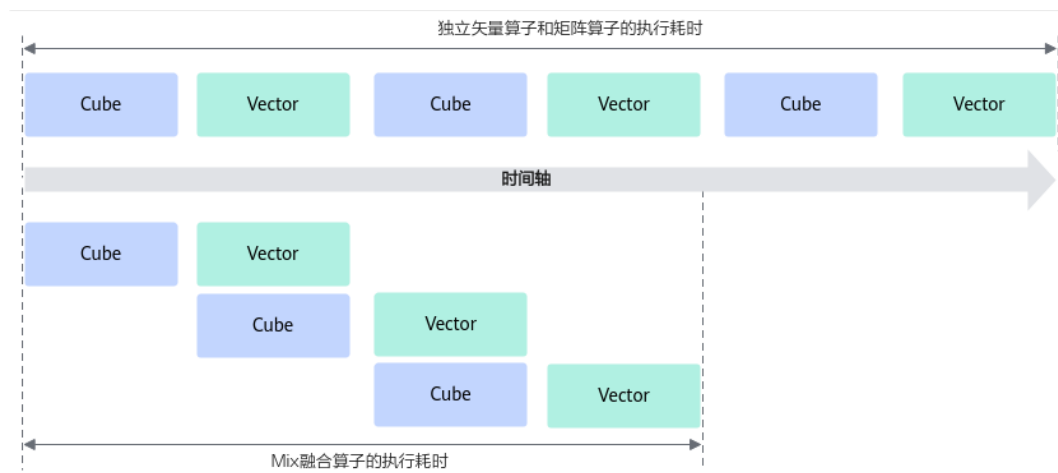
图 3-56 Flash Attention 核心实现



使用场景和优势

针对有数据依赖的矢量算子和矩阵算子，可以通过融合算子编程的方式，将矢量算子和矩阵算子进行融合，通过一个算子Kernel函数来承载，由此来获得性能上的收益。下图展示了独立矢量算子和矩阵算子、Mix融合算子的执行耗时对比，由此可以看出为什么开发Mix融合算子会带来性能上的收益。

图 3-57 独立矢量算子和矩阵算子、Mix 融合算子的执行耗时对比



- **独立的矢量算子和矩阵算子实现：**矩阵计算后的结果需要搬运到 Global Memory 上，然后由 Global Memory 搬运到 Local Memory，再进行矢量算子的计算，计算和搬运都是串行执行，另外多个算子的调度执行，会增加算子的调度耗时。
- **融合算子的实现方法：**可以对数据进行切片，再通过流水的设计，使得向量计算单元和矩阵计算单元实现并行计算；另外相比于不融合的单算子，减少了算子的调度耗时。

除了有效提升算子性能，充分发挥 AI 处理器的算力，融合算子还有如下优势：

- **减少计算量：**融合算子可以将多个算子合并为一个，简化计算过程，减少计算量，提高计算效率。
- **减少内存占用：**融合算子可以将多个算子的中间结果合并为一个，从而减少内存占用，提高内存利用率。
- **优化数据流：**融合算子可以优化数据流，减少数据在不同算子之间的传输，从而提高数据处理效率。
- **简化代码实现：**融合算子可以简化代码实现，减少代码量，提高代码可读性和可维护性。

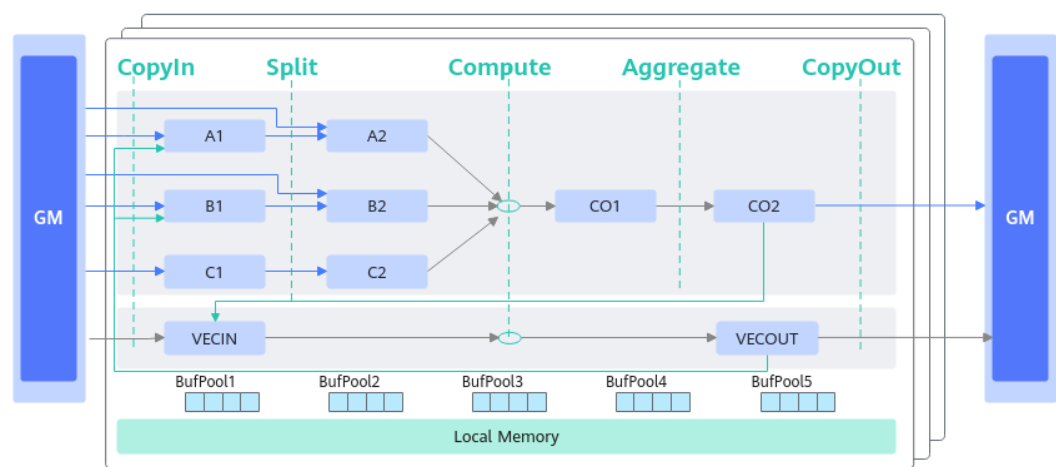
总之，融合算子是一种优化计算的有效手段，可以提高计算效率和内存利用率，优化数据流，简化代码实现。

编程范式

Ascend C 提供**融合算子的编程范式**，方便开发者基于该范式表达融合算子的数据流，快速实现自定义的融合算子。

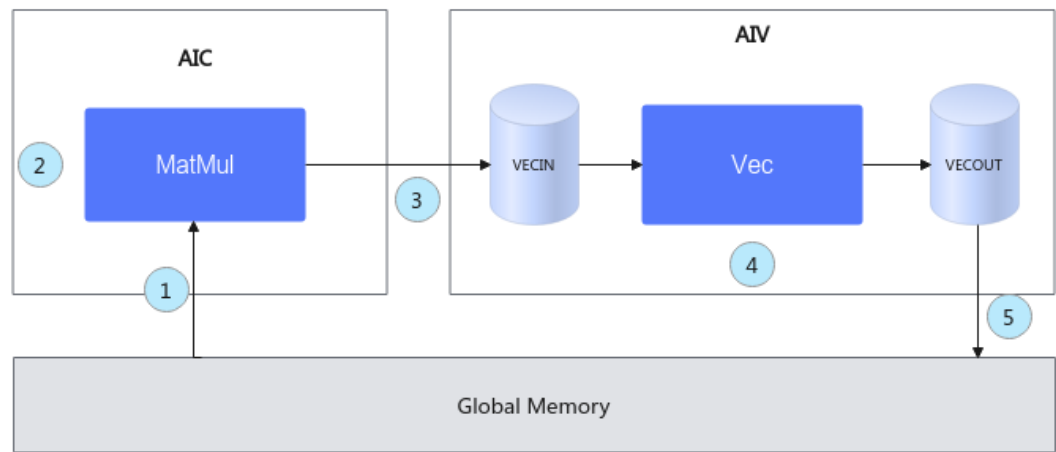
融合算子数据流指融合算子的输入输出在各存储位置间的流向。以一个典型的 Cube 和 Vector 融合算子为例，逻辑位置间的数据流向如下图所示：

- Cube 的输出可以作为 Vector 的输入：CO2->VECIN
- Vector 的输出可以作为 Cube 的输入：VECOUT->A1->A2、VECOUT->B1->B2



基于Matmul高阶API的融合算子编程范式，对上述数据流简化表达如下：

图 3-58 融合算子编程范式



1. 初始化一个MatMul对象，将输入数据从Global Memory搬运到Cube核上。
2. 进行MatMul内部的计算。
3. 将MatMul的计算结果搬运到Vector核上。
4. 进行Vector矢量计算。
5. 将输出结果搬运到Global Memory上。

整个过程的示例代码如下（伪代码）。完整样例请参考[MatmulLeakyRelu](#)。

```
template<typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::Process()
{
    // 步骤1: 初始化一个MatMul对象，将输入数据从Global Memory搬运到Cube核上。
    uint32_t computeRound = 0;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), matmulObj);
    matmulObj.Init(&tiling);
    matmulObj.SetTensorA(aGlobal);
    matmulObj.SetTensorB(bGlobal);
    matmulObj.SetBias(biasGlobal);

    while (matmulObj.template Iterate<true>()) { // 步骤2: 进行MatMul内部的计算。
        // 步骤3: 将MatMul的计算结果搬运到Vector核上。
    }
}
```

```
reluOutLocal = reluOutQueue_.AllocTensor<cType>();  
matmulObj.template GetTensorC<true>(reluOutLocal, false, true);  
// 步骤4: 进行Vector向量计算。  
AscendC::LeakyRelu(reluOutLocal, reluOutLocal, (cType)alpha, tiling.baseM * tiling.baseN);  
reluOutQueue_.EnQue(reluOutLocal);  
// 步骤5: 将输出结果搬运到Global Memory上  
reluOutQueue_.DeQue<cType>();  
...  
AscendC::DataCopy(cGlobal[startOffset], reluOutLocal, copyParam);  
reluOutQueue_.FreeTensor(reluOutLocal);  
  
computeRound++;  
}  
matmulObj.End();  
}
```

3.3.5.1.2 算子实现

下文将以Matmul+LeakyRelu融合算子的实现为例，介绍Mix融合算子的设计和实现流程。该样例仅支持在Atlas A2 训练系列产品/Atlas A2 推理系列产品上运行。

算子的设计过程分为算子分析、数据流分析、Tiling策略设计三部分。

算子分析

算子分析是指明确算子的数学表达式、输入、输出，核函数的名称等信息。

步骤1 明确算子的数学表达式及计算逻辑。该算子的计算逻辑为，先进行一个矩阵乘操作，然后将矩阵乘的结果与一个alpha参数进行LeakyRelu操作。数学表达式如下：

```
c = LeakyRelu(a * b + bias, alpha);
```

步骤2 明确输入和输出。

- MatMul+LeakyRelu算子输入为a、b、bias，输出为c。alpha作为激活函数LeakyRelu的系数，为固定值，可以在算子实现中直接使用常数值参与计算。
- 本样例中算子输入a、b支持的数据类型为half（float16），算子输入bias支持的数据类型为float32，算子输出c的数据类型为float32。
- 输入矩阵a的形状为[M, K]，输入矩阵b的形状为[K, N]，输出矩阵c的形状为[M, N]，输入bias的形状为[1, N]。
- 算子输入输出支持的数据格式为：ND。

步骤3 确定核函数名称和参数。

- 您可以自定义核函数名称，本样例中核函数命名为matmul_leakyrelu_custom。
- 根据对算子输入输出的分析，确定核函数的参数a, b, bias, c；a, b, bias为输入在Global Memory上的内存地址，c为输出在Global Memory上的内存地址。

----结束

通过以上分析，得到Ascend C MatMul+LeakyRelu算子的设计规格如下：

- 算子类型（OpType）：MATMUL_LEAKYRELU
- 算子输入输出：

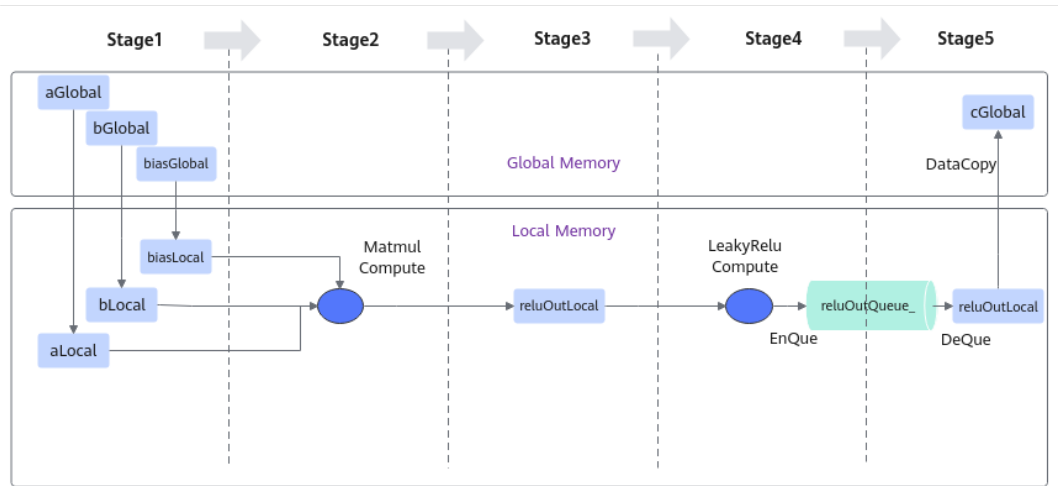
表 3-10 MATMUL_LEAKYRELU 算子输入输出规格

name	shape	data type	format
a (输入)	[M, K]	half	ND
b (输入)	[K, N]	half	ND
bias (输入)	[1, N]	float32	ND
z (输出)	[M, N]	float32	ND

- 核函数名称: matmul_leakyrelu_custom

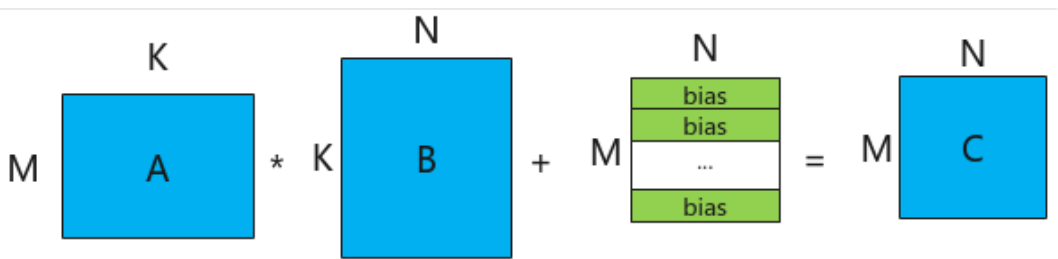
数据流分析

进行算子的数据流分析：数据流向为在Cube核上完成Matmul计算后将数据搬运至Vector核进行LeakyRelu计算。根据上述数据流并结合融合算子的编程范式，规划并行的流水任务。如下图所示：



- 步骤1 将输入数据从Global Memory搬运到Cube核。
- 步骤2 进行MatMul内部的计算，计算公式和计算示意图如下：
注：bias的shape为[1, N]，对A*B结果矩阵的每一行都采用该bias进行偏置。

图 3-59 Matmul 矩阵乘示意图

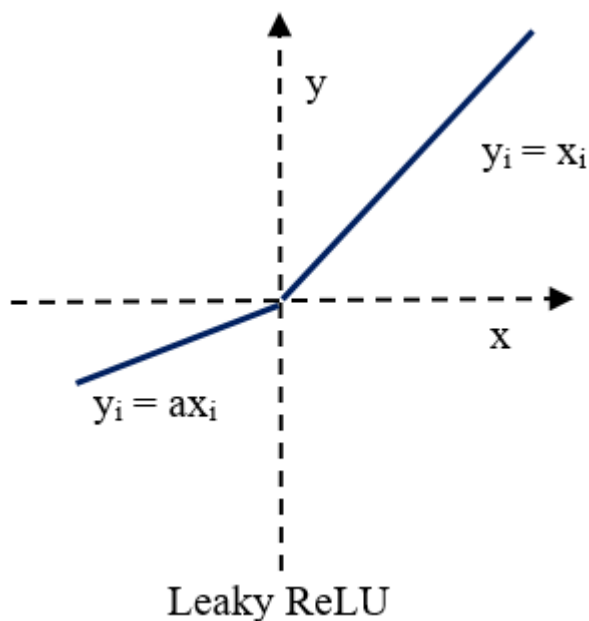


- 步骤3 将MatMul的计算结果搬运到Vector核。

步骤4 进行Vector矢量计算，该样例中进行LeakyReLU计算。

Leaky ReLU（带泄露线性整流函数）激活函数，是人工神经网络中一种常用的激活函数，其数学表达式和函数图像如下所示：

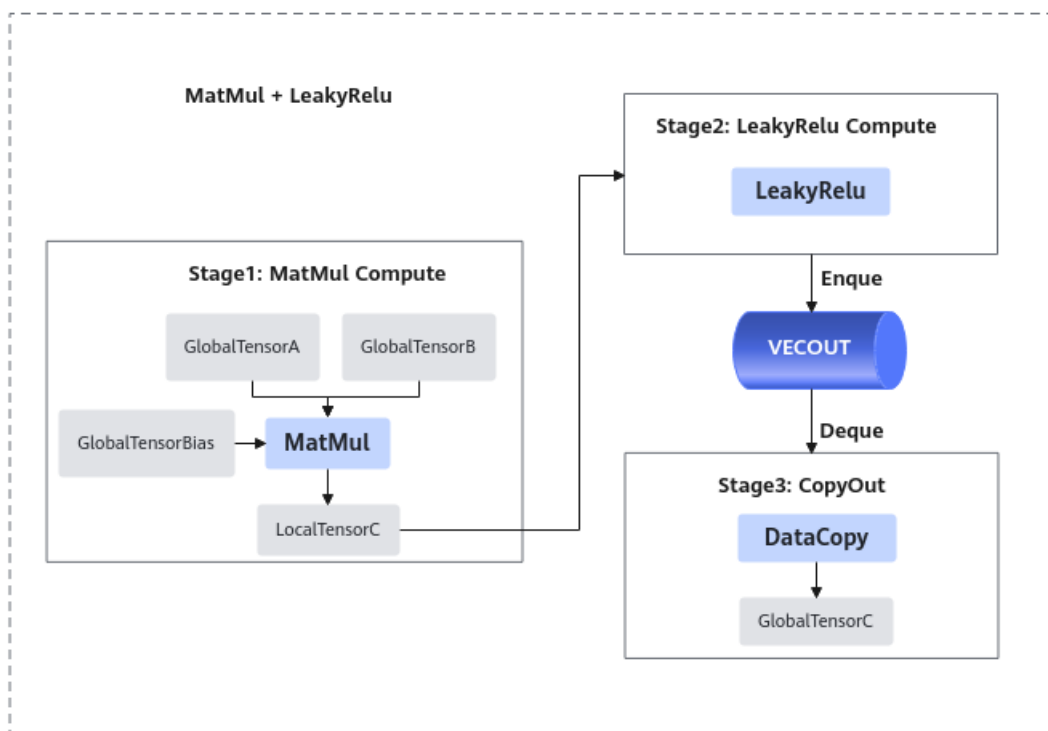
$$f(x_i) = \begin{cases} x_i & \text{if } x_i \geq 0 \\ ax_i & \text{if } x_i < 0 \end{cases}$$



步骤5 将输出结果搬运到Global Memory。

----结束

前三步的内容都封装在Matmul高阶API内，本样例中可以简化为3个stage。如下图所示：



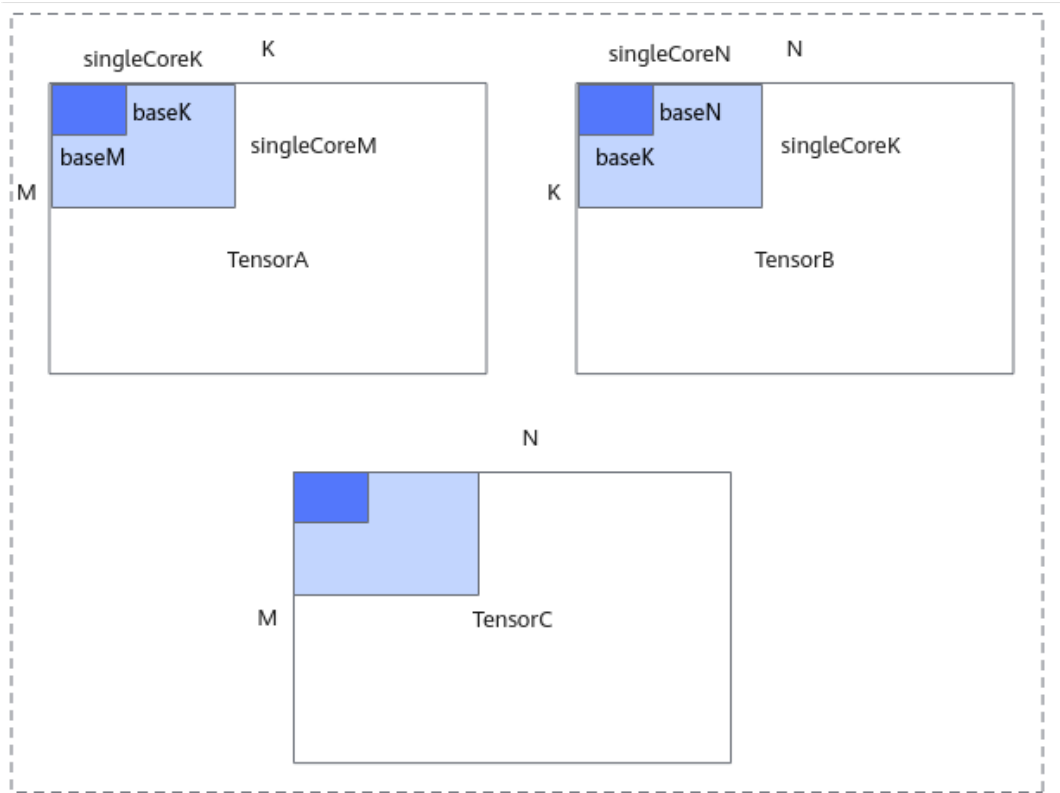
根据上述分析，明确实现过程中会使用到Matmul高阶API接口，LeakyRelu Vector计算接口、DataCopy、EnQue、DeQue接口。

Tiling 策略设计

Tiling策略的设计主要包括多核切分和核内切分策略。

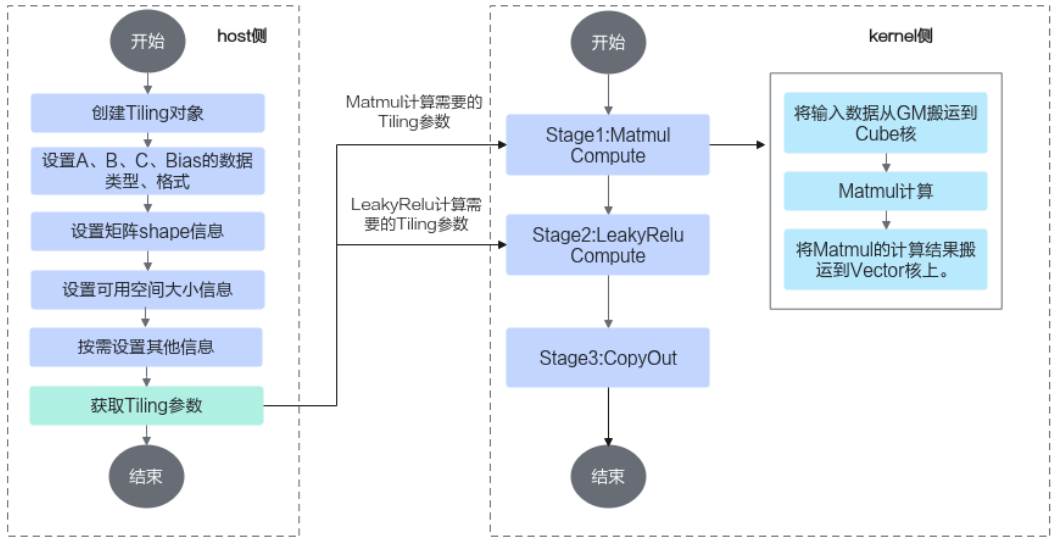
- 多核切分: 根据当前核数，对输入shape的M, K, N进行多核切分，得到单核内shape大小singleCoreM, singleCoreK, singleCoreN。
- 核内切分: 根据Local Memory的大小约束，对单核内的shape大小进一步切分，得到A、B、C矩阵参与一次矩阵乘指令的shape大小baseM, baseN, baseK。切分时需要注意：GetTensorC的结果如果放在LocalMemory（UB）上，需要注意，baseM * baseN的大小不能超出UB的限制。

切分策略示意图如下，更多切分策略相关原理请参考[数据分块（Tiling）](#)。



算子实现

在[矩阵编程章节](#)，我们得知Ascend C提供一组Matmul高阶API，封装了常用的切分和数据搬运、计算的算法逻辑，方便用户快速实现Matmul矩阵乘法的运算操作。融合算子中矩阵编程部分的实现与之类似，开发者在host侧通过调用API自动获取Tiling参数，该参数传递到kernel侧后，在初始化操作时传入，通过几个简单的API即可完成矩阵乘操作。再结合上文的融合算子的编程范式，融合算子实现的步骤如下。完整样例请参考[MatmulLeakyRelu](#)。



kernel侧实现的代码框架如下，在完成Matmul对象的初始化、左矩阵A、右矩阵B、Bias的设置后，通过单次Iterate叠加while循环的方式完成后续的Matmul计算、LeakyRelu计算、CopyOut流程。

```
template<typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::Process(){
    uint32_t computeRound = 0;
    // Matmul对象初始化
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), matmulObj);
    // 设置Matmul的输入（包括左矩阵、右矩阵、bias）
    matmulObj.Init(&tiling);
    matmulObj.SetTensorA(aGlobal);
    matmulObj.SetTensorB(bGlobal);
    matmulObj.SetBias(biasGlobal);
    // 调用matmul iterate获取一块[baseM, baseN]的计算结果
    while (matmulObj.template Iterate<true>())
    {
        MatmulCompute();
        LeakyReluCompute();
        CopyOut(computeRound);
        computeRound++;
    }
    matmulObj.End();
}
```

Matmul计算、LeakyRelu计算、CopyOut的具体实现代码如下：

步骤1 Matmul计算：

1. 将输入数据从Global Memory搬运到Cube核。
2. 进行MatMul内部的计算。
3. 将MatMul的计算结果搬运到Vector核。

```
template<typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::Process(){
    uint32_t computeRound = 0;
    // ...
    // 调用matmul iterate获取一块[baseM, baseN]的计算结果
    while (matmulObj.template Iterate<true>())
    {
        MatmulCompute();
        // ...
        computeRound++;
    }
    matmulObj.End();
}

template<typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::MatmulCompute(){
    reluOutLocal = reluOutQueue_.AllocTensor<cType>();
    // 调用GetTensorC将Matmul的计算结果搬运到Vector核。
    matmulObj.template GetTensorC<true>(reluOutLocal, false, true);
}
```

步骤2 LeakyRelu计算。

```
// 调用LeakyRule接口进行计算
template<typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::LeakyReluCompute(){
    AscendC::LeakyRelu(reluOutLocal, reluOutLocal, (cType)alpha, tiling.baseM * tiling.baseN);
    reluOutQueue_.EnQue(reluOutLocal);
}
```

步骤3 CopyOut，将输出结果搬运到Global Memory。

```
// 将结果搬运到GM
template<typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::CopyOut(uint32_t count){
```

```
reluOutQueue_.DeQue<cType>();  
const uint32_t roundM = tiling.singleCoreM / tiling.baseM;  
const uint32_t roundN = tiling.singleCoreN / tiling.baseN;  
uint32_t startOffset = (count % roundM * tiling.baseM * tiling.N + count / roundM * tiling.baseN);  
AscendC::DataCopyParams copyParam = {(uint16_t)tiling.baseM,  
    (uint16_t)(tiling.baseN * sizeof(cType) / DEFAULT_CO_SIZE), 0,  
    (uint16_t)((tiling.N - tiling.baseN) * sizeof(cType) / DEFAULT_CO_SIZE)};  
AscendC::DataCopy(cGlobal[startOffset], reluOutLocal, copyParam);  
reluOutQueue_.FreeTensor(reluOutLocal);  
}
```

----结束

host侧自动获取Tiling参数的关键步骤介绍如下：

步骤1 创建Tiling对象。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());  
matmul_tiling::MultiCoreMatmulTiling cubeTiling(ascendcPlatform);
```

创建对象时需要传入硬件平台信息，硬件平台信息可以通过GetPlatformInfo获取。

步骤2 设置A、B、Bias的数据类型和格式。

设置示例如下，其中TPosition::LCM是Unified Buffer上的逻辑位置，等同于TPosition::VECCALC，关于TPosition的详细内容请参考TPosition。

```
cubeTiling.SetAType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,  
    matmul_tiling::DataType::DT_FLOAT16);  
cubeTiling.SetBType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,  
    matmul_tiling::DataType::DT_FLOAT16);  
cubeTiling.SetCType(matmul_tiling::TPosition::LCM, matmul_tiling::CubeFormat::ND,  
    matmul_tiling::DataType::DT_FLOAT);  
cubeTiling.SetBiasType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,  
    matmul_tiling::DataType::DT_FLOAT);
```

步骤3 设置矩阵shape信息。

```
cubeTiling.SetShape(M, N, K);  
cubeTiling.SetOrgShape(M, N, K);
```

步骤4 设置可用空间大小信息。

设置Matmul计算时可用的L1 Buffer/L0C Buffer/Unified Buffer空间大小，-1表示AI处理器对应Buffer的大小。

```
cubeTiling.SetBufferSpace(-1, -1, -1);
```

步骤5 按需设置其他参数，比如设置bias参与计算。

```
cubeTiling.SetBias(true);
```

步骤6 获取Tiling参数。

```
MatmulLeakyreluCustomTilingData tiling;  
if (cubeTiling.GetTiling(tiling.cubeTilingData) == -1){  
    return ge::GRAPH_FAILED;  
}
```

步骤7 Tiling参数的序列化保存等其他操作。

----结束

 说明

- 特别的对于多核场景，需要通过SetDim接口设置Matmul计算所用的核数，MIX模式（包含矩阵计算和矢量计算）的设置规则如下：
 - **分离模式**：Matmul API都是从AIV侧发起的，调用Iterate计算时在AIV侧只会起到通知的作用，通知AIC去做矩阵计算，计算完成后AIC告知AIV计算完成。这个架构下，SetBlockDim设置为实际计算会用到的AI Core（AIC、AIV组合）的数量，SetDim设置为实际计算会用到的AIV的数量。例如，SetBlockDim时可以设置为20，启动20个AI Core（AIC AIV的组合），SetDim设置为40，表示按照40个AIV进行切分。
 - **耦合模式**：SetBlockDim加载的核数就是Matmul API实际计算会用到的核数，SetDim和SetBlockDim设置的值是一样的。
 - Matmul高阶API内部实现时需要使用系统workspace，开发者需要：
 - 在host侧Tiling实现时，设置总的workspace的数值大小（包含用户workspace和系统workspace），workspace空间由框架来申请并管理。系统workspace的空间大小通过GetLibApiWorkSpaceSize获取。

```
size_t userWorkspaceSize = 0;
size_t systemWorkspaceSize = ascendcPlatform.GetLibApiWorkSpaceSize();
size_t *currentWorkspace = context->GetWorkspaceSizes(1);
currentWorkspace[0] = userWorkspaceSize + systemWorkspaceSize;
```
 - 若算子工程不是**自定义算子工程**，也不是带有**HAVE_WORKSPACE**编译宏的Kernel直调算子工程，kernel侧需要在Matmul初始化前，通过SetSysWorkSpace设置系统workspace。

```
// 使用Matmul时必须设置workspace空间
SetSysWorkspace(workspace);
if (GetSysWorkSpacePtr() == nullptr) {
    return;
}
```
 - 上文介绍的实现方法，AIC侧和AIV侧的代码隔离和核间同步由框架来完成，开发者无需关心。除该方法外，开发者也可以选择底层编码的方式在分离模式下实现融合算子，这种方式将更加灵活。采用底层编码方式时，需要注意以下几点：
 - 通过**ASCEND_IS_AIV**和**ASCEND_IS_AIC**实现AIV和AIC代码之间的隔离。
 - 自行实现AIC和AIV核之间的同步：比如Matmul + LeakyRelu算子样例中，需要确保在AIC完成矩阵计算后，AIV再进行LeakyRelu的计算。
 - 使用高阶API Matmul时需要设置ASCENDC_CUBE_ONLY，表示仅在AIC侧调用Matmul API。
 - 使用设置Kernel类型接口设置Kernel类型为KERNEL_TYPE_MIX_xxx，同时启用AIV核和AIC核。

```
#define ASCENDC_CUBE_ONLY // 指定Matmul运行在AIC上
KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_MIX_AIC_1_2); // 设置Kernel类型为
KERNEL_TYPE_MIX_xxx
if ASCEND_IS_AIC {
    ...
    // AIC核进行Matmul计算
    // AIC核完成计算后，通过AscendC::CrossCoreSetFlag<modelId, pipe>(flagId)发送同步flag
}
if ASCEND_IS_AIV {
    ...
    // AIV核通过AscendC::CrossCoreWaitFlag(flagId)接收同步flag
    // AIV核进行LeakyRelu计算
}
```
- 完整样例请参考**BareMix**样例。

3.3.5.2 通算融合

3.3.5.2.1 基础知识

说明

本节内容为通算融合算子的理论背景和开发指导，学习本节内容之前，请确保已经掌握[矩阵编程](#)和《[HCCL集合通信库用户指南](#)》中的相关知识。

通算融合算子一般支持如下产品型号：

Atlas A3 训练系列产品/Atlas A3 推理系列产品

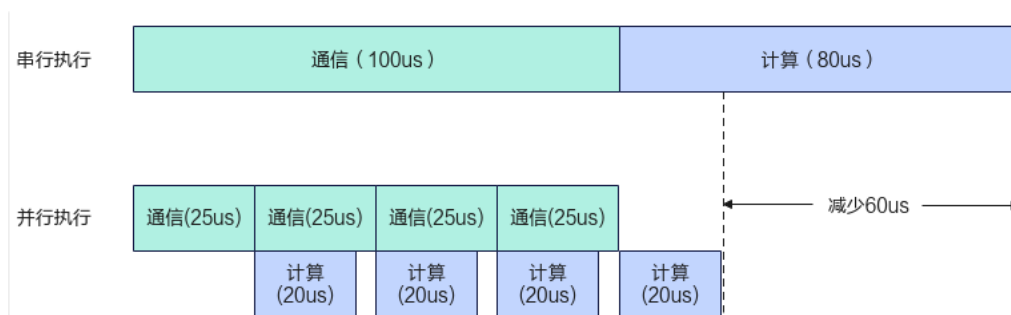
Atlas A2 训练系列产品/Atlas A2 推理系列产品

通算融合算子

相比于一般的计算或搬运类算子，通算融合算子将原本串行的通信和计算操作融合在一起，通过在算子内部进行数据切分，实现了计算和通信任务在算子内的并行执行，从而提升算子性能。通算融合算子统称为MC²算子，即Matrix Computation & Communication。

如下图所示，串行的通信算子和计算算子的理想执行耗时为两个算子执行时间的加和，而在融合通信和计算任务得到的通算融合算子内，将需要通信和计算的数据进行切分，一次通信和计算的数据量减少，整个通信和计算任务分多次进行，使得计算与通信流水并行，理论执行耗时大大缩短，从而带来性能收益。

图 3-60 通信计算融合前后的理论执行耗时对比示意图



使用场景和优势

随着模型规模的增长，单设备上的训练和推理在计算能力、内存容量和能效等方面面临瓶颈，因此分布式并行计算成为必选技术路径。对于大模型分布式训练和推理过程中的通信和计算任务，可根据通信和计算的依赖关系分为两类：

- 弱依赖计算通信任务

通信或计算的结果不会立即被对方使用，两者虽有依赖，但在两者中间可以调度执行其他无依赖的计算或通信任务。如[图2](#)所示，通信1与计算1-2、计算4有依赖关系，通信1与计算1-1、计算2-1、计算2-2、计算3无依赖关系；通信2与计算2-2、计算4有依赖关系，通信2与计算2-1、计算3无依赖关系。因此，通信1和通信2都有较大的流水空间，可以被与它们无依赖的计算任务所掩盖。如[图3](#)所示，通信1和通信2均可以被无依赖的计算任务掩盖。在模型中，此类无依赖的通信和计算可以实现任务级并行，无需做算子融合。因此，弱依赖计算通信任务不适用通算融合场景。

图 3-61 弱依赖计算通信任务示意图

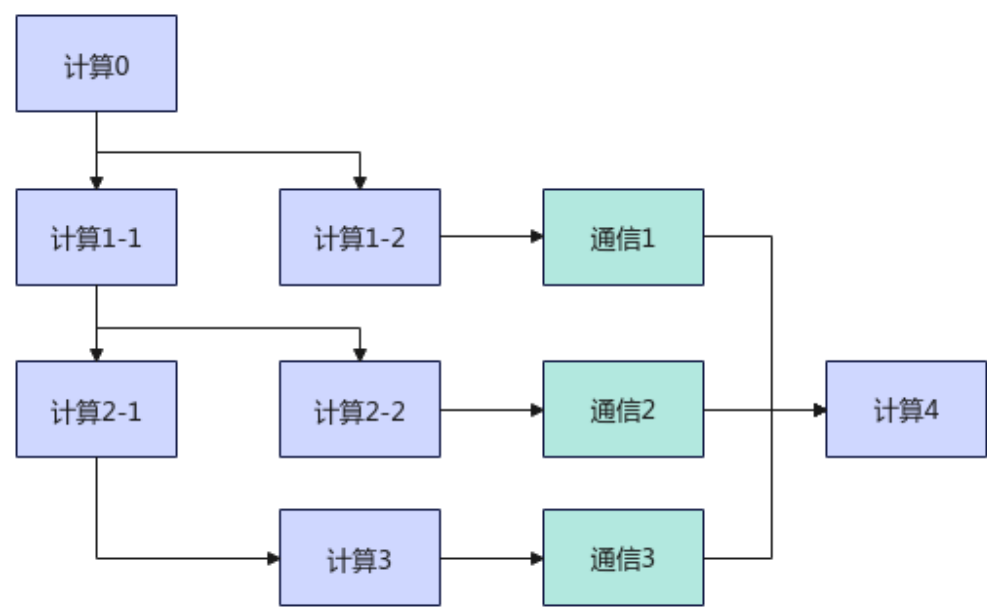
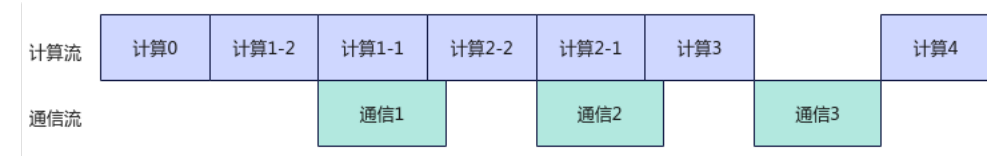


图 3-62 弱依赖计算通信任务的调度模拟示意图



- 强依赖计算通信任务
通信或计算的结果立即被对方使用，两者间存在紧密依赖关系。如下图所示，计算通信任务必须串行执行，在通信1和通信2执行过程中，硬件计算资源闲置，此类通信计算模式若在模型中大量出现，将导致算力利用率低，通信成为主要性能瓶颈。强依赖计算通信任务适合融合为通算融合算子，利用通算融合技术提升性能。

图 3-63 强依赖计算通信任务示意图

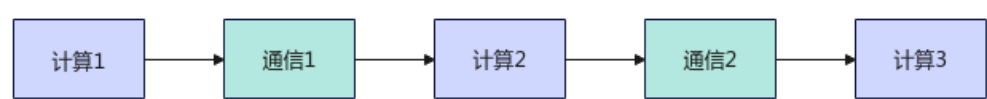
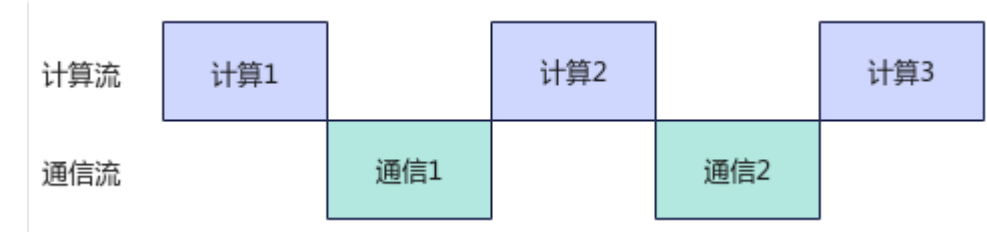


图 3-64 强依赖计算通信任务的调度模拟示意图



通算融合技术与网络模型结构密切相关，一般而言，符合上述强依赖计算通信任务都有可能通过通算融合算子实现性能提升。

3.3.5.2.2 算子实现

在通算融合类算子的实现中，通信操作使用Hccl高阶API，矩阵乘计算操作使用Matmul高阶API。关于更多集合通信的内容和相关概念请参考《[HCCL集合通信库用户指南](#)》。通算融合算子的开发过程与一般算子相同，但请注意，当前通算融合算子暂不支持[Kernel直调和入图（GE图）开发](#)，仅支持[单算子API调用](#)。

下文将以AllGatherMatmulCustom算子（简称AllGatherMatmul）的实现为例，从算子分析、数据流分析、创建算子工程、原型定义、Tiling实现、Kernel实现、编译与运行等方面介绍通算融合算子的设计和实现流程。本样例中算子的完整代码请参见[AllGatherMatmul样例](#)。该样例仅支持在Atlas A2 训练系列产品/Atlas A2 推理系列产品上运行。

算子分析

算子分析是指明确算子的数学表达式、输入、输出，核函数的名称等信息。

步骤1 明确算子的数学表达式及通信计算逻辑。

AllGatherMatmul算子实现了AllGather通信和Matmul矩阵乘法的融合。算子逻辑为：对输入的通信矩阵a做AllGather通信得到Matmul计算的左矩阵，即通信结果gather_out，将gather_out和右矩阵b做Matmul运算得到输出c。对应的数学表达式为：

```
gather_out = AllGather(a)
c = gather_out * b
```

步骤2 明确输入、输出和属性。

- a、b为源操作数，a为通信的输入矩阵，形状为[M, K]；b为Matmul的右矩阵，形状为[K, N]。在样例中，M、K、N分别固定为512、5120、640。
- gather_out为目的操作数，存放AllGather通信结果，形状为[M * rankDim, K]，其中，rankDim为通信域内的卡数，在样例中固定为8。
- c为目的操作数，存放Matmul运算结果，形状为[M * rankDim, N]。
- 算子输入输出的数据类型为float16，**format**为：ND。
- group为算子属性，表示通信域名称，明确算子运行时所在的通信域。

步骤3 确定核函数名称和参数。

- 本样例中核函数命名为all_gather_matmul_custom。
- 根据对算子输入输出的分析，确定核函数的参数aGM，bGM，cGM，gatherOutGM；aGM，bGM为输入在Global Memory上的内存地址，cGM，gatherOutGM为输出在Global Memory上的内存地址。注意，核函数的参数和单算子API调用的输入输出在命名上存在区别，原因是核函数的参数是输入输出在Global Memory上的内存地址，而单算子API调用时输入输出的类型是aclTensor，两者并不完全一致。

步骤4 确定算子实现所需接口。

- 算子涉及AllGather通信，查看Ascend C API参考中的通信相关接口，需要使用Hccl高阶API来实现AllGather通信。
- 算子涉及Matmul左右矩阵在外部存储和内部存储间的数据搬运，查看Ascend C API参考中的数据搬运接口，需要使用DataCopy来实现数据搬运。
- 计算过程涉及矩阵计算操作，查看Ascend C API参考中的矩阵计算相关接口，需要使用Matmul高阶API实现矩阵乘计算。

----结束

表 3-11 AllGatherMatmulCustom 算子规格

算子类型 (OpType)	AllGatherMatmulCustom			
算子输入输出	name	shape	data type	format
	a	[512, 5120]	float16	ND
	b	[5120, 640]	float16	ND
	c	[4096, 640]	float16	ND
	gather_out	[4096, 5120]	float16	ND
算子属性	group (char*) , Host侧标识通信域的字符串, 表示通信域名称。			
核函数名称	all_gather_matmul_custom			

数据流分析

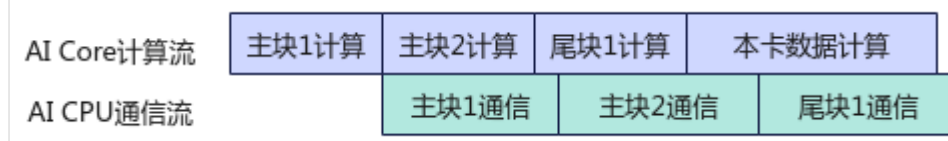
AllGatherMatmul算子的数据在卡间进行AllGather通信, 在卡内进行Matmul计算, 通信和计算按照数据切分后的主块、尾块分多次进行, 流水互相掩盖。分析过程中, 假定通信矩阵的切分策略为按M轴进行切分, 切分后主块数 (tileCnt) 为2, 尾块数 (tailCnt) 为1, 则可得到通信计算掩盖示意图如下。

图 3-65 AllGatherMatmul 通信计算掩盖示意图



AllGather的功能为将通信域内所有卡的输入按照卡id重新排序, 然后拼接起来, 再将结果发送到所有卡。因此, AllGather的结果中包含本卡数据, 即本卡输入的通信矩阵a, 算子无需等待这部分数据的通信完成, 也无需对数据进行切分, 可直接基于完整的通信矩阵a进行Matmul计算。AllGatherMatmul算子首先做本卡数据的Matmul计算, 这样做的好处在于主块1的通信能与Matmul计算互相掩盖, 同时, 主块1、主块2、尾块1的计算无需再包括对本卡数据的Matmul计算, 可以减少后续主尾块的计算量, 增加通信计算的掩盖率, 从而提高性能。注意, 不是所有的通算融合算子都适合首先进行本卡数据的Matmul计算。因为AllGatherMatmul算子的通信在计算之前, 所以先进行本卡数据的Matmul计算, 可以实现本卡数据计算和第一次通信之间的互相掩盖。如果是计算在通信前的算子, 如MatmulAllReduce, 建议将本卡数据的计算放在最后, 与最后一次通信互相掩盖, 如下图所示。

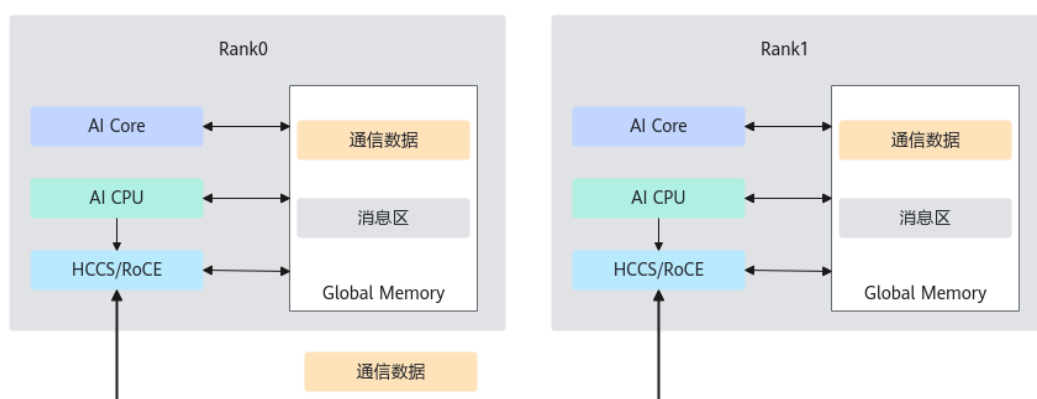
图 3-66 MatmulAllReduce 通信计算掩盖示意图



AllGatherMatmul算子逻辑分析：

步骤1 AI Core将要执行的通信信息写入Global Memory中的消息区，实现任务下发。消息区是特定地址的Global Memory，AI Core和AI CPU通过向其写入和轮询读取来实现消息在两者间的传递，这些操作统一封装于Hccl高阶API中。

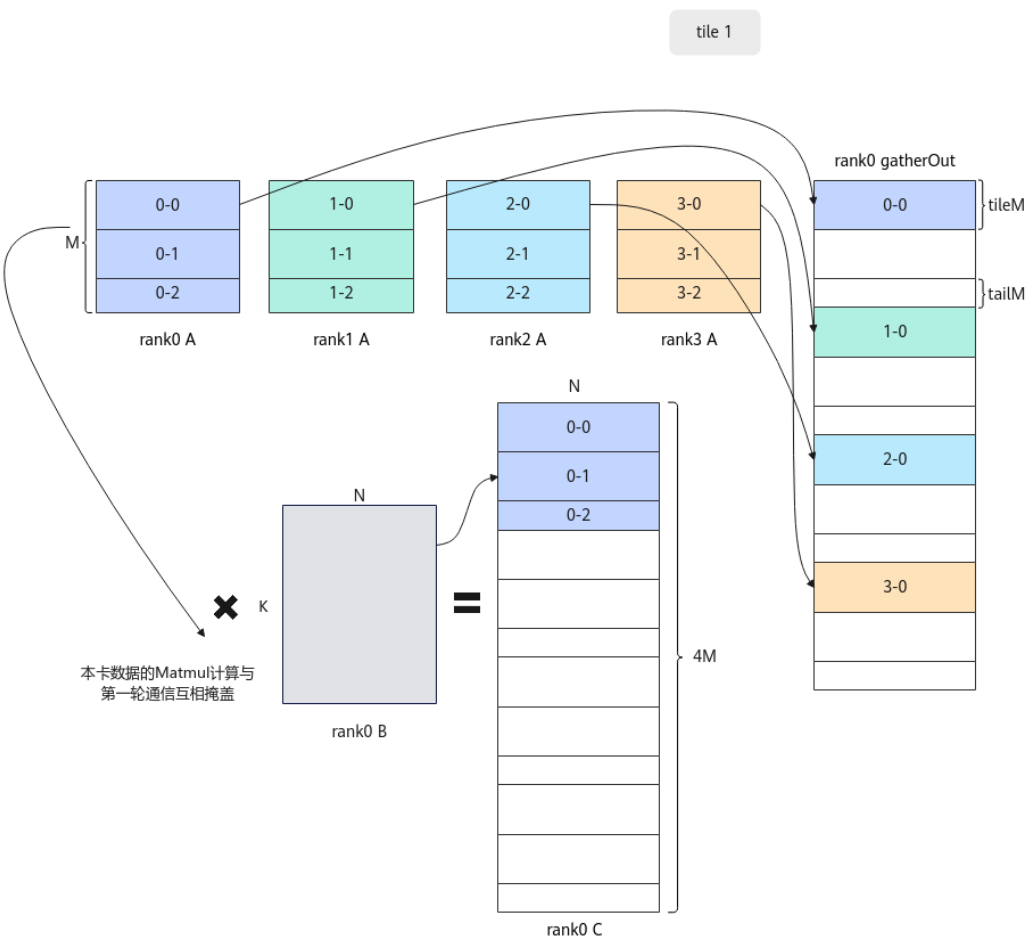
图 3-67 通算融合算子通信流程示意图



步骤2 AI CPU从消息区读取到所有通信任务信息，开始基于HCCS（华为缓存一致性系统，用于CPU/NPU之间的高速互联）或RoCE（承载在融合以太网上的RDMA技术，即跨越以太网的RDMA通信方式）等链路执行第一轮AllGather集合通信任务。与此同时，AI Core开始对本卡数据进行Matmul计算。

下图是通信卡数为4时，第一轮通信与本卡计算的示意图。tile 1表示图示为第一轮通信和与其相互掩盖的矩阵乘计算的处理流程。图中切分后的小矩阵中形如X-Y的数字表示它所在的数据块对应于第X张卡第Y块数据。

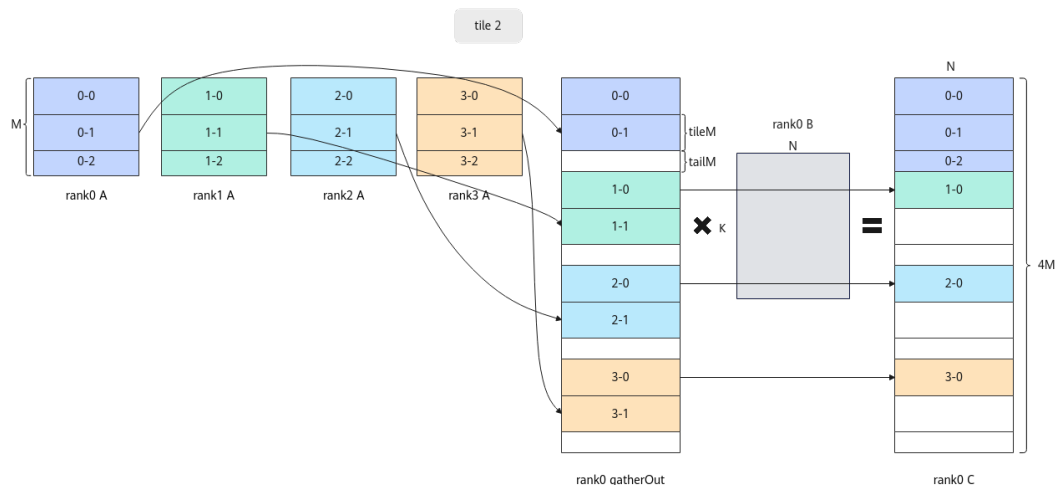
图 3-68 AllGatherMatmul 第一轮通信与 rank0 上的本卡数据矩阵乘示意图



步骤3 AI CPU完成第一轮通信任务后，向消息区写入第一轮通信任务已完成的消息，并开始执行第二轮通信任务。同时，AI Core在完成本卡数据的Matmul计算后，通过轮询消息区等到第一轮通信任务已完成的消息，开始进行第一轮通信结果即主块1的Matmul计算。

下图是通信卡数为4时，第二轮通信与rank0计算的示意图。tile 2表示图示为第二轮通信和与其相互掩盖的矩阵乘计算的处理流程。

图 3-69 AllGatherMatmul 第二轮通信与 rank0 上主块 1 的矩阵乘示意图



步骤4 类似步骤3，逐步完成剩余所有数据块的通信和计算。

----结束

创建算子工程

创建通算融合算子的算子工程与一般算子相同，具体请参考[2.10.2.2 创建算子工程](#)章节。本样例基于如下原型定义json文件，使用自定义算子工程生成工具msOpGen，为AllGatherMatmul算子创建算子工程。

```
[
  {
    "op": "AllGatherMatmulCustom",
    "input_desc": [
      {
        "name": "a",
        "param_type": "required",
        "format": [
          "ND"
        ],
        "type": [
          "float16"
        ]
      },
      {
        "name": "b",
        "param_type": "required",
        "format": [
          "ND"
        ],
        "type": [
          "float16"
        ]
      }
    ],
    "output_desc": [
      {
        "name": "c",
        "param_type": "required",
        "format": [
          "ND"
        ],
        "type": [
          "float16"
        ]
      }
    ]
  }
]
```

```
    },  
    {  
        "name": "gather_out",  
        "param_type": "required",  
        "format": [  
            "ND"  
        ],  
        "type": [  
            "float16"  
        ]  
    }  
],  
"attr": [  
    {  
        "name": "group",  
        "type": "string",  
        "default_value": "",  
        "param_type": "required"  
    }  
]  
}  
]
```

算子原型定义

相比于一般算子，通算融合算子在实现[算子原型定义](#)时，有如下约束：

- 必须定义一个表示算子通信域名称的属性。通信域是集合通信执行的上下文，管理对应的通信实体（例如一个NPU就是一个通信实体）和通信所需的资源。
- 必须通过原型注册中的MC2接口注册该算子为通算融合算子，并通过HcclGroup接口配置该算子的通信域名称。

AllGatherMatmul算子使用"group"属性表示该算子的通信域名称，其在算子原型中定义如下：

```
this->Attr("group").AttrType(REQUIRED).String(); // "group"为通算融合算子的属性，表示通信域名称，原型定义中的String类型对应单算子API中的char*类型
```

```
...  
this->MC2().HcclGroup("group"); // 将"group"属性配置为该算子的通信域名称
```

AllGatherMatmul算子的完整原型定义如下：

```
namespace ops {  
class AllGatherMatmulCustom : public OpDef {  
public:  
    explicit AllGatherMatmulCustom(const char *name) : OpDef(name)  
    {  
        this->Input("a")  
            .ParamType(REQUIRED)  
            .DataType({ge::DT_FLOAT16})  
            .Format({ge::FORMAT_ND})  
            .UnknownShapeFormat({ge::FORMAT_ND});  
        this->Input("b")  
            .ParamType(REQUIRED)  
            .DataType({ge::DT_FLOAT16})  
            .Format({ge::FORMAT_ND})  
            .UnknownShapeFormat({ge::FORMAT_ND})  
            .IgnoreContiguous();  
        this->Output("c")  
            .ParamType(REQUIRED)  
            .DataType({ge::DT_FLOAT16})  
            .Format({ge::FORMAT_ND})  
            .UnknownShapeFormat({ge::FORMAT_ND});  
        this->Output("gather_out")  
            .ParamType(REQUIRED)  
            .DataType({ge::DT_FLOAT16})  
            .Format({ge::FORMAT_ND})  
    }  
};  
}
```

```
.UnknownShapeFormat({ge::FORMAT_ND});  
  
this->Attr("group").AttrType(REQUIRED).String();  
  
this->AICore().SetTiling(AllGatherMatmulCustomTilingFunc); // 注册  
AllGatherMatmulCustomTilingFunc为Tiling入口函数  
this->AICore().AddConfig("ascendxxx"); // ascendxxx请修改为对应的AI处理器型号。  
this->MC2().HcclGroup("group");  
}  
};  
OP_ADD(AllGatherMatmulCustom);  
}
```

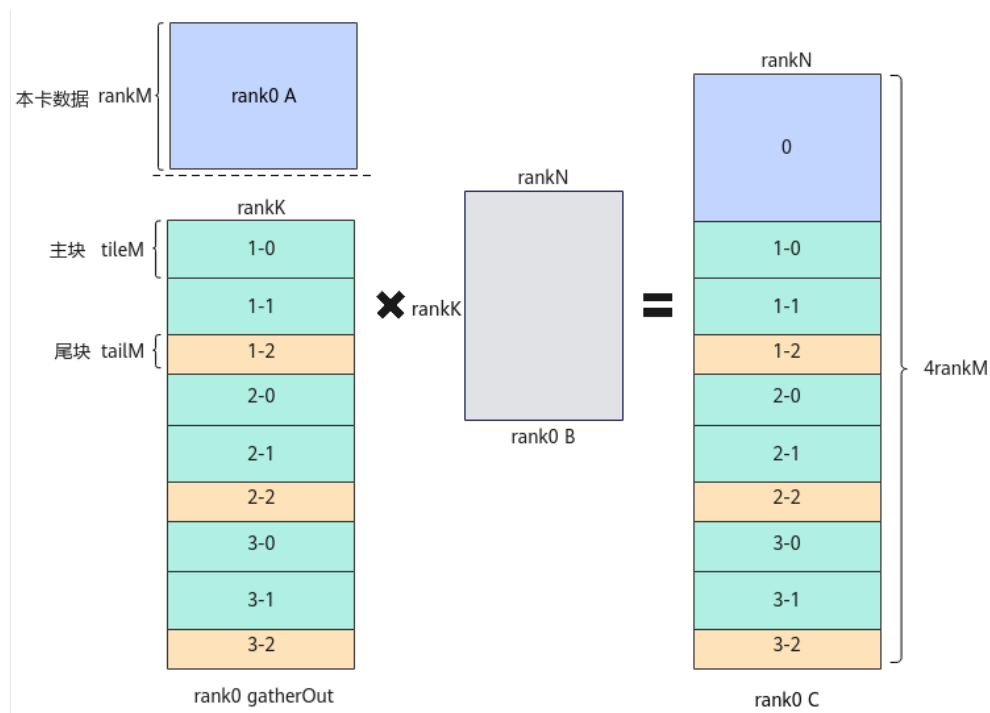
Tiling 实现

通算融合算子Tiling策略的设计主要包括通信切分策略、Matmul多核切分和核内切分策略。

- 通信切分策略：每轮通信数据块的大小，对通算融合算子的性能有较大影响。样例中按照主块M轴长度448对通信矩阵A的M轴进行切分。具体场景中如何确定切分策略请参考[MC²算子性能调优案例](#)。
- Matmul多核切分和核内切分：
 - 多核切分：根据当前核数，对输入shape的M、K、N进行多核切分，得到单核内shape大小singleCoreM、singleCoreK、singleCoreN。
 - 核内切分：根据Local Memory的大小约束，对单核内的shape大小进一步切分，得到A、B、C矩阵参与一次矩阵乘指令的shape大小baseM、baseN、baseK。

如上所述，通信矩阵被切分为主块、尾块，主块、尾块的通信结果以及本卡数据需要分别进行Matmul计算。如下图，主块、尾块和本卡数据在M轴的长度分别为tileM、tailM和rankM，即Matmul计算时的左矩阵存在三种不同的形状，因此需要分别以通信矩阵主块、尾块和本卡数据块的大小为矩阵乘原始的输入形状，调用Matmul高阶API提供的Tiling接口，得到对应这三种形状的多核切分和核内切分策略。这里，singleCoreM、baseM等概念和相关原理的介绍请参考[3.3.3.1 基础知识](#)。

图 3-70 AllGatherMatmul 算子在 rank0 的矩阵乘示意图



下面给出Tiling实现的关键步骤：

步骤1 定义AllGatherMatmul算子的Tiling结构体。

通信和Matmul融合得到的通算融合算子的Tiling结构体一般包括如下三个部分：

- Hccl高阶API的Tiling结构体。定义Mc2InitTiling和Mc2CcTiling参数。Mc2InitTiling参数用于初始化通信任务配置，必须定义为算子Tiling结构体的第一个参数。Mc2CcTiling为具体每个通信任务的参数配置，由于AllGatherMatmul算子中只有AllGather一个通信任务，因此仅需定义一个Mc2CcTiling参数。
- Matmul高阶API的Tiling结构体TCubeTiling。一般而言，主块、尾块和本卡数据的shape是不同的，由于TCubeTiling只能存储对一个输入形状进行Tiling计算得到的结果，因此需要分别定义主块、尾块和本卡数据块的Tiling结构体，来存放它们的多核切分和核内切分策略。
- AllGatherMatmul算子额外需要的自定义结构体AllGatherMatmulTiling。

AllGatherMatmul算子的完整Tiling结构体定义如下：

```
struct AllGatherMatmulTiling {
    uint32_t rankM;    // A矩阵M轴的长度
    uint32_t rankN;    // B矩阵N轴的长度
    uint32_t rankK;    // A、B矩阵K轴的长度
    uint32_t tileNum;   // 主块个数
    uint32_t tailM;     // 尾块的M轴长度
    uint32_t tailNum;   // 尾块个数（0或1）
};

class AllGatherMatmulCustomTilingData {
public:
    Mc2InitTiling mc2InitTiling;
    Mc2CcTiling mc2CcTiling;
    TCubeTiling localTiling;
    TCubeTiling tileTiling;
    TCubeTiling tailTiling;
    AllGatherMatmulTiling cfg;
};
```

步骤2 获取AllGatherMatmul算子的Tiling结构体对象指针。

```
AllGatherMatmulCustomTilingData *tiling = context->GetTilingData<AllGatherMatmulCustomTilingData>();
```

context为TilingContext的对象指针，该指针由框架自动从注册的Tiling入口函数AllGatherMatmulCustomTilingFunc传入，用于保存算子Tiling计算的上下文。在AllGatherMatmul算子的Tiling实现中，通过该上下文context获取计算Tiling所需要的输入输出shape、输入属性等参数，然后将Tiling结果（例如TilingKey、TilingData）保存至上下文中，供后续算子执行时使用。

步骤3 设置算子自定义的Tiling结构体参数。

```
tiling->cfg.tileNum = rankM / TILE_M; // TILE_M在样例中为常量448，表示通信数据块切分后的主块在M轴的长度  
tiling->cfg.tailM = rankM % TILE_M;  
tiling->cfg.tailNum = (rankM % TILE_M == 0) ? 0 : 1;  
tiling->cfg.rankM = rankM;  
tiling->cfg.rankN = rankN;  
tiling->cfg.rankK = rankK;
```

步骤4 设置Matmul高阶API Tiling结构体。

通过matmul_tiling::MultiCoreMatmulTiling获取TCubeTiling结构体，首先创建多核Tiling对象mmTiling，然后设置A、B、C的参数类型信息，M、N、K形状信息等，最后调用GetTiling接口，获取Tiling信息，具体方法可详见Matmul Tiling类。AllGatherMatmul算子中将上述逻辑封装为matmulTilingFunc函数，再分别根据主块、尾块和本卡数据的形状大小，调用matmulTilingFunc函数，得到对应的TCubeTiling参数。

```
// 封装设置TCubeTiling结构体的函数为matmulTilingFunc  
auto matmulTilingFunc = [&](int64_t m, int64_t n, int64_t k, TCubeTiling &cubeTiling) -> bool {  
    matmul_tiling::MultiCoreMatmulTiling mmTiling;  
    mmTiling.SetAType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,  
matmul_tiling::DataType::DT_FLOAT16);  
    mmTiling.SetBType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,  
matmul_tiling::DataType::DT_FLOAT16);  
    mmTiling.SetCType(matmul_tiling::TPosition::GM, matmul_tiling::CubeFormat::ND,  
matmul_tiling::DataType::DT_FLOAT16);  
    mmTiling.SetBias(false);  
    mmTiling.SetDim(aicCoreNum);  
    mmTiling.SetShape(m, n, k);  
    mmTiling.SetOrgShape(m, n, k);  
    mmTiling.SetBufferSize(L1_BUFFER_SIZE);  
    if (mmTiling.GetTiling(cubeTiling) != 0) {  
        return false;  
    }  
    return true;  
};  
// 设置本卡数据的Matmul TCubeTiling结构体  
if (!matmulTilingFunc(rankM, rankN, rankK, tiling->localTiling)) {  
    ERROR_LOG("Get local matmul tiling failed");  
    return ge::GRAPH_FAILED;  
}  
// 设置主块的Matmul TCubeTiling结构体  
if (!matmulTilingFunc(TILE_M, rankN, rankK, tiling->tileTiling)) {  
    ERROR_LOG("Get tile matmul tiling failed");  
    return ge::GRAPH_FAILED;  
}  
// 设置尾块的Matmul TCubeTiling结构体  
if (!matmulTilingFunc(rankM % TILE_M, rankN, rankK, tiling->tailTiling)) {  
    ERROR_LOG("Get tail matmul tiling failed");  
    return ge::GRAPH_FAILED;  
}
```

步骤5 设置Hccl高阶API Tiling结构体。

根据通信任务类型、算法配置等，创建一个Mc2CcTilingConfig类对象，通过向GetTiling方法传入算子Tiling结构体中mc2InitTiling和mc2CcTiling成员的引用，完成

需要传递给Kernel侧的Mc2InitTiling参数和Mc2CcTiling参数的获取。Hccl高阶API Tiling结构体的具体使用方法详见Hccl Tiling使用说明。

```
uint32_t opType = HCCL_CMD_ALLGATHER; // 设置通信任务类型
std::string algConfig = "AllGather=level0:doubling"; // 设置通信算法，该参数为预留字段，配置后不生效
uint32_t reduceType = HCCL_REDUCE_SUM; // 设置Reduce操作类型，仅对有归约操作的通信任务有效，作为AllGather通信，可以直接配置默认值HCCL_REDUCE_SUM
AscendC::Mc2CcTilingConfig mc2CcTilingConfig(group, opType, algConfig, reduceType);
mc2CcTilingConfig.GetTiling(tiling->mc2InitTiling);
mc2CcTilingConfig.SetSkipLocalRankCopy(0); // 输出gatherOut需带有本卡的A矩阵，因此设置为0
mc2CcTilingConfig.GetTiling(tiling->mc2CcTiling);
```

----结束

Kernel 实现

在AllGatherMatmul算子的Kernel实现中，需要对本卡数据、通信主块、通信尾块共三种形状的左矩阵进行Matmul运算，为避免重复代码，有必要抽象出一个通用的适用于不同输入形状的Matmul计算函数。设计该Matmul计算函数前，需要考虑Matmul计算需要的基本信息，罗列如下：

- 输入A、B矩阵和输出C矩阵的地址。
- TCubeTiling结构体：包含矩阵A、B、C的形状、数据类型等信息，以及A、B矩阵进行Matmul运算时在核间和核内的切分策略。

除了上述Matmul运算所需的信息外，为了快速实现Matmul矩阵乘法，可以使用Matmul高阶API中的Matmul对象来执行计算。如果Matmul对象在Matmul计算函数中定义，每次调用该函数时都会实例化Matmul对象并释放资源，这将导致较大的运行时开销。因此，将该对象也作为Matmul计算函数的参数，以实现对象的复用。

综上所述，在Kernel实现中定义的适用于不同输入形状的Matmul计算函数如下。其中Matmul计算函数函数名定义为MatmulKernel，入参aGM、bGM、cGM表示需要运算的原始输入输出矩阵的地址，入参tiling表示TCubeTiling结构体，入参mm对应Matmul高阶API的实现类。MATMUL_TYPE是特化了MatmulType模板的类型别名。

```
using MATMUL_TYPE = MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half>;
__aicore__ inline void MatmulKernel(GM_ADDR aGM, GM_ADDR bGM, GM_ADDR cGM, TCubeTiling &tiling,
    Matmul<MATMUL_TYPE, MATMUL_TYPE, MATMUL_TYPE> &mm)
```

MatmulKernel函数的实现步骤如下。

步骤1 TCubeTiling结构体存储了Matmul计算所需的核数，在无需计算的核上直接返回，结束计算。

```
if (GetBlockIdx() >= tiling.usedCoreNum) {
    return;
}
```

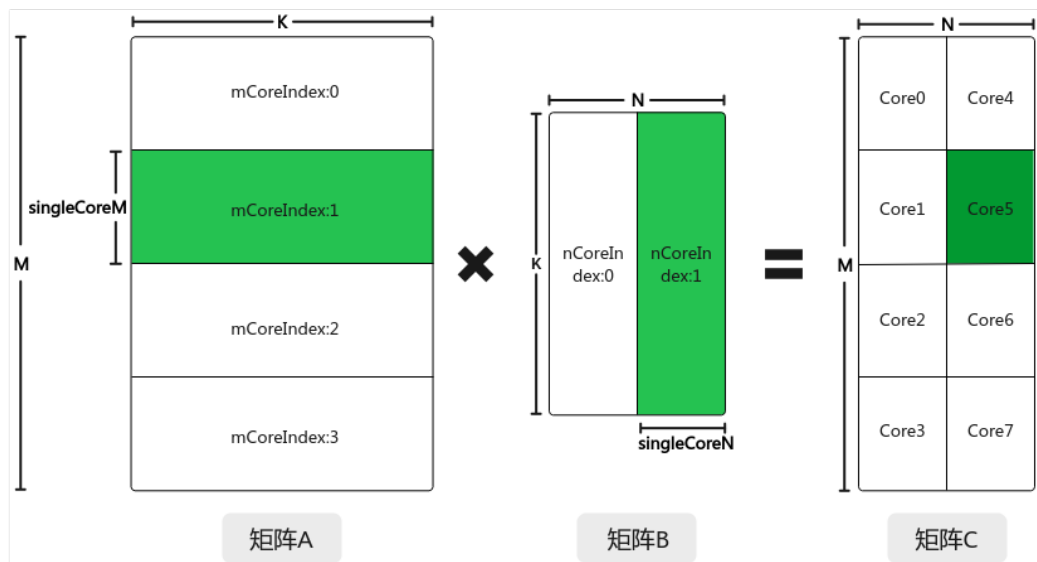
步骤2 Matmul高阶API要求使用GlobalTensor作为输入输出矩阵，因此，根据函数输入的A、B、C矩阵在Global Memory的地址，分别定义aGlobal、bGlobal、cGlobal三个GlobalTensor。

```
GlobalTensor<half> aGlobal, bGlobal, cGlobal;
aGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ half *>(aGM), tiling.M * tiling.Ka);
bGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ half *>(bGM), tiling.Ka * tiling.N);
cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ half *>(cGM), tiling.M * tiling.N);
```

步骤3 为了实现多核并行，提升计算效率，将矩阵数据进行切分，切分后的数据分配到不同的核上进行处理。这里采用了不切分K轴、仅切分M、N轴的切分策略，示意图如下。在这种场景下，每个核需要计算待处理的矩阵数据相对于原始矩阵的偏移量，并将偏移后的矩阵作为传入A、B、C矩阵时的入参。同时，为支持分核后的尾块数据的处

理，每个核需要计算实际处理的singleCoreM、singleCoreN大小，并在下一步中通过调用Matmul高阶API进行设置。

图 3-71 Matmul 计算分核示意图



```
int mSingleBlocks = (tiling.M + tiling.singleCoreM - 1) / tiling.singleCoreM;
int mCoreIndex = GetBlockIdx() % mSingleBlocks;
int nCoreIndex = GetBlockIdx() / mSingleBlocks;
// 计算当前核需要处理的矩阵数据相对于原始矩阵的偏移
int offsetA = mCoreIndex * tiling.Ka * tiling.singleCoreM;
int offsetB = nCoreIndex * tiling.singleCoreN;
int offsetC = mCoreIndex * tiling.N * tiling.singleCoreM + nCoreIndex * tiling.singleCoreN;
// 计算当前核的singleCoreM/singleCoreN，作为后续SetTail接口的入参
int tailM = Std::min(tiling.M - mCoreIndex * tiling.singleCoreM, tiling.singleCoreM);
int tailN = Std::min(tiling.N - nCoreIndex * tiling.singleCoreN, tiling.singleCoreN);
```

步骤4 调用Matmul高阶API设置Matmul计算的原始完整的形状、当前核处理的输入输出矩阵的地址和计算的actualsingleCoreM、singleCoreN的大小，并完成矩阵乘运算。

```
mm.SetOrgShape(tiling.M, tiling.N, tiling.Ka, tiling.Kb);
mm.SetTensorA(aGlobal[offsetA]);
mm.SetTensorB(bGlobal[offsetB]);
mm.SetTail(tailM, tailN);
mm.IterateAll(cGlobal[offsetC]);
```

----结束

AllGatherMatmul算子的核函数定义如下，aGM、bGM、cGM、gatherOutGM参数含义如[算子分析](#)中所述，workspaceGM和tilingGM分别表示workspace空间和tiling数据在Global Memory的地址。

```
extern "C" __global__ __aicore__ void all_gather_matmul_custom(GM_ADDR aGM, GM_ADDR bGM,
GM_ADDR cGM, GM_ADDR gatherOutGM, GM_ADDR workspaceGM, GM_ADDR tilingGM)
```

下面介绍AllGatherMatmul算子主流程实现的具体步骤。

步骤1 Matmul计算依赖AIC核，因此控制算子逻辑仅运行于AIC中。通过ASCEND_IS_AIV宏，判断如果当前核为AIV核，直接返回，结束当前核的运行。

```
if ASCEND_IS_AIV {
    return;
}
```

步骤2 注册算子Tiling结构体、获取Tiling，并初始化TPipe。

```
REGISTER_TILING_DEFAULT(AllGatherMatmulCustomTilingData);  
GET_TILING_DATA(tilingData, tilingGM);  
TPipe pipe;
```

步骤3 定义并赋值后续计算所需变量。

```
auto &&localTiling = tilingData.localTiling;  
auto &&tileTiling = tilingData.tileTiling;  
auto &&tailTiling = tilingData.tailTiling;  
const auto tileNum = tilingData.cfg.tileNum;           // 主块数量  
const auto tailNum = tilingData.cfg.tailNum;           // 尾块数量  
const auto aTileEleCnt = tileTiling.M * tileTiling.Ka; // 通信矩阵主块元素数  
const auto aTileSize = tileTiling.M * tileTiling.Ka * sizeof(half); // 通信矩阵主块字节数  
const auto cTileSize = tileTiling.M * tileTiling.N * sizeof(half); // 通信矩阵主块对应输出矩阵的字节数  
const auto aTailEleCnt = tailTiling.M * tailTiling.Ka; // 通信矩阵尾块元素数  
const auto aRankEleCnt = localTiling.M * localTiling.Ka; // 通信矩阵元素数  
const auto aRankSize = localTiling.M * localTiling.Ka * sizeof(half); // 通信矩阵字节数  
const auto cRankSize = localTiling.M * localTiling.N * sizeof(half); // 通信矩阵对应输出矩阵的字节数
```

步骤4 初始化hccl对象并下发AllGather通信任务。

```
Hccl hccl;  
GM_ADDR contextGM = GetHcclContext<HCCL_GROUP_ID_0>();  
hccl.InitV2(contextGM, &tilingData);  
hccl.SetCcTilingV2(offsetof(AllGatherMatmulCustomTilingData, mc2CcTiling));  
auto handleId = hccl.AllGather<true>(aGM, gatherOutGM, aTileEleCnt,  
HcclDataType::HCCL_DATA_TYPE_FP16, aRankEleCnt, tileNum);  
auto tailHandleId = hccl.AllGather<true>(aGM + tileNum * aTileSize, gatherOutGM + tileNum * aTileSize,  
aTailEleCnt,  
HcclDataType::HCCL_DATA_TYPE_FP16, aRankEleCnt, tailNum);
```

步骤5 初始化Matmul对象，对本卡数据进行Matmul计算。

```
Matmul<MATMUL_TYPE, MATMUL_TYPE, MATMUL_TYPE> mm;  
REGIST_MATMUL_OBJ(GetTPipePtr(), GetSysWorkSpacePtr(), mm);  
mm.Init(&localTiling);  
MatmulKernel(aGM, bGM, cGM + hccl.GetRankId() * cRankSize, localTiling, mm);
```

步骤6 逐轮等待主块的通信完成，并对其进行Matmul计算。

```
auto aAddr = gatherOutGM;  
auto cAddr = cGM;  
mm.Init(&tileTiling);  
for (uint32_t i = 0; i < tileNum; i++) {  
    hccl.Wait(handleId);  
    for (uint32_t rankId = 0; rankId < hccl.GetRankDim(); rankId++) {  
        if (rankId == hccl.GetRankId())  
            continue;  
        MatmulKernel(aAddr + rankId * aRankSize, bGM, cAddr + rankId * cRankSize, tileTiling, mm);  
    }  
    aAddr += aTileSize;  
    cAddr += cTileSize;  
}
```

步骤7 等待尾块的通信完成，并对其进行Matmul计算。

```
aAddr = gatherOutGM + tileNum * aTileSize;  
cAddr = cGM + tileNum * cTileSize;  
if (tailNum > 0) {  
    mm.Init(&tailTiling);  
    hccl.Wait(tailHandleId);  
    for (uint32_t rankId = 0; rankId < hccl.GetRankDim(); rankId++) {  
        if (rankId == hccl.GetRankId())  
            continue;  
        MatmulKernel(aAddr + rankId * aRankSize, bGM, cAddr + rankId * cRankSize, tailTiling, mm);  
    }  
}
```

步骤8 释放资源。

```
mm.End();  
hccl.Finalize();
```

----结束

整合前述代码，完整Kernel代码如下。

```
#define ASCENDC_CUBE_ONLY  
#include "kernel_operator.h"  
#include "lib/matmul_intf.h"  
#include "all_gather_matmul_custom_tiling.h"  
using namespace AscendC;  
using MATMUL_TYPE = MatmulType<AscendC::TPosition::GM, CubeFormat::ND, half>;  
  
__aicore__ inline void MatmulKernel(GM_ADDR aGM, GM_ADDR bGM, GM_ADDR cGM, TCubeTiling &tiling,  
    Matmul<MATMUL_TYPE, MATMUL_TYPE, MATMUL_TYPE> &mm)  
{  
    if (GetBlockIdx() >= tiling.usedCoreNum) {  
        return;  
    }  
  
    GlobalTensor<half> aGlobal, bGlobal, cGlobal;  
    aGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ half*>(aGM), tiling.M * tiling.Ka);  
    bGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ half*>(bGM), tiling.Ka * tiling.N);  
    cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ half*>(cGM), tiling.M * tiling.N);  
  
    int mSingleBlocks = (tiling.M + tiling.singleCoreM - 1) / tiling.singleCoreM;  
    int mCoreIdx = GetBlockIdx() % mSingleBlocks;  
    int nCoreIdx = GetBlockIdx() / mSingleBlocks;  
    int offsetA = mCoreIdx * tiling.Ka * tiling.singleCoreM;  
    int offsetB = nCoreIdx * tiling.singleCoreN;  
    int offsetC = mCoreIdx * tiling.N * tiling.singleCoreM + nCoreIdx * tiling.singleCoreN;  
    int tailM = Std::min(tiling.M - mCoreIdx * tiling.singleCoreM, tiling.singleCoreM);  
    int tailN = Std::min(tiling.N - nCoreIdx * tiling.singleCoreN, tiling.singleCoreN);  
  
    mm.SetOrgShape(tiling.M, tiling.N, tiling.Ka, tiling.Kb);  
    mm.SetTensorA(aGlobal[offsetA]);  
    mm.SetTensorB(bGlobal[offsetB]);  
    mm.SetTail(tailM, tailN);  
    mm.IterateAll(cGlobal[offsetC]);  
}  
extern "C" __global__ __aicore__ void all_gather_matmul_custom(GM_ADDR aGM, GM_ADDR bGM,  
    GM_ADDR cGM,  
    GM_ADDR gatherOutGM, GM_ADDR workspaceGM,  
    GM_ADDR tilingGM)  
{  
    if ASCEND_IS_AIV {  
        return;  
    }  
    REGISTER_TILING_DEFAULT(AllGatherMatmulCustomTilingData);  
    GET_TILING_DATA(tilingData, tilingGM);  
    TPipe pipe;  
  
    auto &&localTiling = tilingData.localTiling;  
    auto &&tileTiling = tilingData.tileTiling;  
    auto &&tailTiling = tilingData.tailTiling;  
    const auto tileNum = tilingData.cfg.tileNum; // 主块数量  
    const auto tailNum = tilingData.cfg.tailNum; // 尾块数量  
    const auto aTileEleCnt = tileTiling.M * tileTiling.Ka; // 通信矩阵主块元素数  
    const auto aTileSize = tileTiling.M * tileTiling.Ka * sizeof(half); // 通信矩阵主块字节数  
    const auto cTileSize = tileTiling.M * tileTiling.N * sizeof(half); // 输出矩阵主块字节数  
    const auto aTailEleCnt = tailTiling.M * tailTiling.Ka; // 通信矩阵尾块元素数  
    const auto aRankEleCnt = localTiling.M * localTiling.Ka; // 单卡通信矩阵元素数  
    const auto aRankSize = localTiling.M * localTiling.Ka * sizeof(half); // 单卡通信矩阵字节数  
    const auto cRankSize = localTiling.M * localTiling.N * sizeof(half); // 单卡输出矩阵字节数  
  
    Hccl hccl;  
    GM_ADDR contextGM = GetHcclContext<HCCL_GROUP_ID_0>();  
    hccl.InitV2(contextGM, &tilingData);  
    hccl.SetCcTilingV2(offsetof(AllGatherMatmulCustomTilingData, mc2CcTiling));
```

```
auto handleId = hccl.AllGather<true>(aGM, gatherOutGM, aTileEleCnt,
HcclDataType::HCCL_DATA_TYPE_FP16, aRankEleCnt, tileNum);
auto tailHandleId = hccl.AllGather<true>(aGM + tileNum * aTileSize, gatherOutGM + tileNum * aTileSize,
aTailEleCnt,
HcclDataType::HCCL_DATA_TYPE_FP16, aRankEleCnt, tailNum);

Matmul<MATMUL_TYPE, MATMUL_TYPE, MATMUL_TYPE> mm;
REGIST_MATMUL_OBJ(GetTPipePtr(), GetSysWorkSpacePtr(), mm);
mm.Init(&localTiling);
MatmulKernel(aGM, bGM, cGM + hccl.GetRankId() * cRankSize, localTiling, mm);

auto aAddr = gatherOutGM;
auto cAddr = cGM;
mm.Init(&tileTiling);
for (uint32_t i = 0; i < tileNum; i++) {
    hccl.Wait(handleId);
    for (uint32_t rankId = 0; rankId < hccl.GetRankDim(); rankId++) {
        if (rankId == hccl.GetRankId())
            continue;
        MatmulKernel(aAddr + rankId * aRankSize, bGM, cAddr + rankId * cRankSize, tileTiling, mm);
    }
    aAddr += aTileSize;
    cAddr += cTileSize;
}

aAddr = gatherOutGM + tileNum * aTileSize;
cAddr = cGM + tileNum * cTileSize;
if (tailNum > 0) {
    mm.Init(&tailTiling);
    hccl.Wait(tailHandleId);
    for (uint32_t rankId = 0; rankId < hccl.GetRankDim(); rankId++) {
        if (rankId == hccl.GetRankId())
            continue;
        MatmulKernel(aAddr + rankId * aRankSize, bGM, cAddr + rankId * cRankSize, tailTiling, mm);
    }
}

mm.End();
hccl.Finalize();
}
```

编译和运行

下面从编译、安装、运行三个步骤对AllGatherMatmul样例作简要介绍。

步骤1 编译

参考[AllGatherMatmul样例](#)中生成自定义算子工程、编译算子的命令，运行install.sh脚本完成编译。

样例目录结构如下，AllGatherMatmulCustom目录为必要的算子实现，install.sh脚本使用msOpGen在21_all_gather_matmul_custom目录下创建一个CustomOp目录，并将算子实现文件复制到对应目录下，再调用msOpGen生成的编译入口脚本build.sh编译算子。

```
├── 21_all_gather_matmul_custom
│   ├── AclNNInvocation          // 通过aclnn调用的方式调用AllGatherMatmulCustom算子
│   ├── AllGatherMatmulCustom     // AllGatherMatmulCustom算子工程
│   ├── all_gather_matmul_custom.json // AllGatherMatmulCustom算子的原型定义json文件
│   ├── all_gather_matmul_demo_def.h // AllGatherMatmulCustom算子参数配置
│   └── install.sh                // 脚本，调用msOpGen生成自定义算子工程，并编译
```

msOpGen生成的CustomOp目录结构如下。

```
├── CustomOp                      // msOpGen生成的AllGatherMatmul自定义算子工程
│   ├── cmake
│   └── op_host                   // host侧实现文件
```

```
├── op_kernel      // kernel侧实现文件
├── scripts        // 自定义算子工程打包相关脚本所在目录
├── build.sh       // 编译入口脚本
├── CMakeLists.txt // 算子工程的CMakeLists.txt
└── CMakePresets.json // 编译配置项
```

步骤2 安装

部署自定义算子包前，请确保环境中存在自定义算子包默认部署路径的环境变量ASCEND_OPP_PATH。

```
# 查看环境变量输出
echo $ASCEND_OPP_PATH

# 若无输出，则需设置环境变量，ASCEND_INSTALL_PATH为CANN软件包安装路径
source [ASCEND_INSTALL_PATH]/set_env.bash
# 例如 source /usr/local/Ascend/cann/set_env.sh
```

然后执行如下命令，切换目录为编译出的自定义算子安装包所在目录，并安装自定义算子包。

```
cd CustomOp/build_out
./custom_opp_<target os>_<target architecture>.run
```

命令执行成功后，自定义算子包中的相关文件将部署至环境变量ASCEND_OPP_PATH指向的vendors/customize目录中。

步骤3 运行

切换目录为AclNNInvocation目录，执行run.sh脚本运行单算子样例。

```
cd ../../AclNNInvocation
bash run.sh
```

样例中的AclNNInvocation目录提供了完整的单算子API调用的示例代码。完成前两个步骤自定义算子的编译部署后，会自动生成单算子API，该API可以直接在应用程序中调用。算子API的形式一般为“两段式接口”，形如：

```
// 获取算子使用的workspace空间大小
aclnnStatus aclnnAllGatherMatmulCustomGetWorkspaceSize(
    const aclTensor *a,
    const aclTensor *b,
    char *group,
    const aclTensor *cOut,
    const aclTensor *gatherOutOut,
    uint64_t *workspaceSize,
    aclOpExecutor **executor);
// 执行算子
aclnnStatus aclnnAllGatherMatmulCustom(
    void *workspace,
    uint64_t workspaceSize,
    aclOpExecutor *executor,
    const aclrtStream stream);
```

其中aclnnAllGatherMatmulCustomGetWorkspaceSize为第一段接口，主要用于计算本次API调用计算过程中需要的workspace内存大小。按照该workspaceSize大小申请Device侧内存，然后调用第二段接口aclnnAllGatherMatmulCustom执行计算。详细内容请参考[单算子API调用](#)章节。

在通算融合场景，单算子API调用的程序中需要调用HCCL接口参考中的接口创建通信域，并在多线程上执行AllGatherMatmul算子。以下给出main函数和线程调用函数中关键步骤的代码示例，仅供参考。

```
int main(int argc, char **argv)
{
    // 1.AscendCL初始化
    if (aclInit(NULL) != ACL_SUCCESS) {
        ERROR_LOG("aclInit failed");
    }
```



```
        return FAILED;
    }
    // 2.通信域创建
    HcclComm comms[RANK_DIM]; // RANK_DIM为卡数，示例中为8
    int32_t devices[RANK_DIM];
    for (int32_t i = 0; i < RANK_DIM; i++) {
        devices[i] = i;
    }
    if (HcclCommInitAll(RANK_DIM, devices, comms) != HCCL_SUCCESS) {
        ERROR_LOG("Hccl comm init failed.");
        (void)aclFinalize();
        return FAILED;
    }
    // 3.创建多线程以在通信域的所有卡上都调用AllGatherMatmul算子
    std::vector<std::unique_ptr<std::thread>> threads(RANK_DIM);
    for (uint32_t rankId = 0; rankId < RANK_DIM; rankId++) {
        threads[rankId].reset(new(std::nothrow) std::thread(&RunOp, rankId, std::ref(comms[rankId])));
    }
    for (uint32_t rankId = 0; rankId < RANK_DIM; rankId++) {
        threads[rankId]->join();
    }
    // 4.AscendCL去初始化
    (void)aclFinalize();
    return SUCCESS;
}
```

在main函数中，通过HcclCommInitAll接口在当前进程统一创建了RANK_DIM张卡的通信域，一张卡对应后续创建的一个线程。每个线程都调用RunOp函数，该函数负责卡上运行时资源申请和单算子API的二阶段接口调用。RunOp函数的代码示例如下。

```
bool RunOp(uint32_t rankId, HcclComm &comm)
{
    // 1.申请当前线程的context、stream等资源
    aclrtContext context;
    aclrtCreateContext(&context, rankId);
    aclrtStream stream;
    aclrtCreateStream(&stream);
    aclrtSetCurrentContext(context);

    // 2.获取当前线程对应卡的通信域名称
    char group[128] = {0};
    HcclGetCommName(comm, group);

    // 3.申请device侧内存存放算子的输入输出
    // .....

    // 4.计算workspace大小并申请内存
    size_t workspaceSize = 0;
    aclOpExecutor *handle = nullptr;
    auto ret = aclnnAllGatherMatmulCustomGetWorkspaceSize(a, b, group, c, gatherOut, &workspaceSize,
    &handle);
    void *workspace = nullptr;
    if (workspaceSize != 0) {
        aclrtMalloc(&workspace, workspaceSize);
    }

    // 5.执行算子
    ret = aclnnAllGatherMatmulCustom(workspace, workspaceSize, handle, stream);

    // 6.同步等待
    ret = aclrtSynchronizeStreamWithTimeout(stream, 10000); // 10000ms 流同步超时

    // 7.释放算子输入、输出和workspace等device侧内存
    // .....

    // 8.释放通信域、context、stream等资源
    (void)HcclCommDestroy(comm);
    (void)aclrtDestroyStream(stream);
    (void)aclrtDestroyContext(context);
    (void)aclrtResetDevice(rankId);
}
```

```
    return true;  
}
```

----结束

3.3.5.2.3 特性场景

重执行

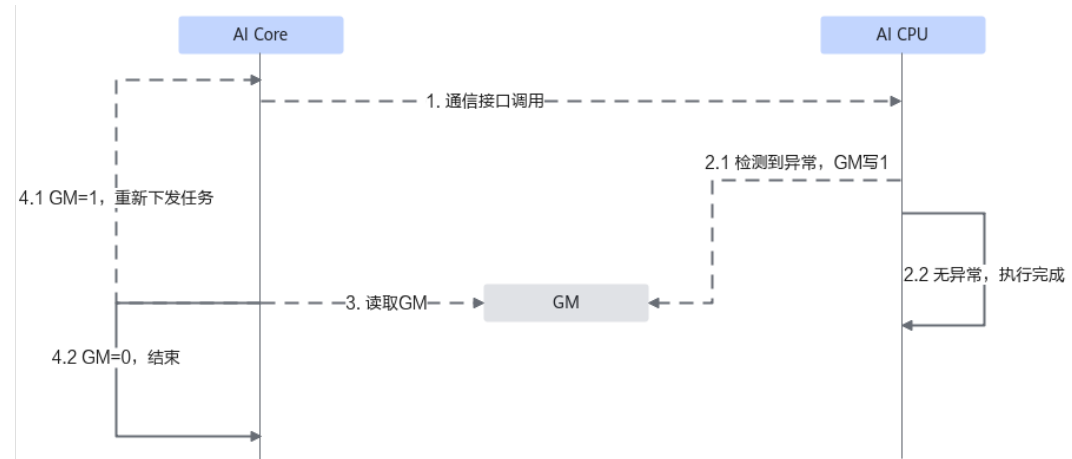
为避免执行通信任务的环境中硬件闪断导致发生通信中断，通算融合算子可通过配置编译宏与环境变量，开启重执行能力。通算融合算子开启重执行后，AI CPU在检测到环境异常时，通过下图示意的机制，通知AI Core重新下发通信任务，避免由于硬件闪断造成的通信中断，提升通信稳定性。

当前，该能力的支持情况如下：

Atlas A2 训练系列产品/Atlas A2 推理系列产品不支持通算融合算子的重执行。

Atlas A3 训练系列产品/Atlas A3 推理系列产品支持通算融合算子的重执行。

图 3-72 通信任务重执行机制



开启重执行的条件如下：

- 通算融合算子的输出内存地址和输入内存地址不相同。
- 通算融合算子仅存在Server间通信（Server为计算节点，通常是8卡或16卡的昇腾NPU设备组成的服务器形态的统称）。
- 算子编译时，配置编译宏AICORE_EXCEPTION_RESTART，如下所示。具体的编译宏配置阶段和方式请参考[支持自定义编译选项](#)。
`add_ops_compile_options(ALL OPTIONS -DAICORE_EXCEPTION_RESTART)`
- 配置HCCL重执行环境变量HCCL_OP_RETRY_ENABLE，开启重执行的检测和上报能力，该环境变量的说明请参考《[环境变量参考](#)》“集合通信 > 可靠性相关 > HCCL_OP_RETRY_ENABLE”。请在算子执行前设置该环境变量，具体配置如下：
server间L1需配置为1，不支持跨超节点，L2配置为0。
`export HCCL_OP_RETRY_ENABLE="L1:1, L2:0"`

注意，开启重执行后，若AI Core第一次下发通信任务后通信中断，默认只重执行一次。若需修改重执行次数或重传间隔时间，请参考《[环境变量参考](#)》“集合通信 > 可靠性相关 > HCCL_OP_RETRY_PARAMS”。

3.4 功能调试

3.4.1 运行正常

运行正常是算子开发后进行调测的前提，是后续进行性能优化的基础。Ascend C提供了多种调测方法和调测工具，供开发者使用，具体内容请参考[2.7.2 功能调试](#)。

3.4.2 精度正常

性能优化的基础是算子运行得到正确的计算结果。评判计算结果正确性需要有一定的评判标准，即使用已知的正确的输出和实际结果进行比较。优化过程中，每次迭代修改后，都需要验证性能优化的新结果是否满足精度评判标准。

下文先介绍几个会影响精度正常的因素：

- 正确插入同步，同步机制是并行计算架构的核心特点之一，对于有数据依赖的场景需要正确插入同步。
- 正确计算偏移地址，多核并行计算时，计算数据在内存上的正确偏移对保证计算结果的正确性至关重要。
- 浮点数计算，在涉及浮点计算的情况下，精度正常不能期望数值逐bit一致。因为浮点数的计算本身不满足交换律、结合律，另外不同硬件对浮点数的支持不同，都可能导致精度结果存在差异。

然后介绍编码过程中需要严格遵守的规则（禁止修改kernel函数参数），防止出现不必要的精度问题。

正确插入同步

- 核内同步
AI Core内部包括MTE1、MTE2、MTE3、Cube、Vector、Scalar等多条流水线。Ascend C框架默认使能auto sync（自动插入同步）编译选项，编译器可以正常插入同步；Ascend C编程模型也会帮助开发者完成部分流水线的同步控制。流水类型的详细介绍、同步类型的分类、编译器自动同步的约束限制、何时需要开发者手动插入同步可参考同步控制。
- 核间同步
上文描述的都是核内同步的情况。特别的，当算子使用多核同步时（多核同步概念可参考多核同步），逻辑核数NumBlocks必须保证不大于实际运行该算子的AI处理器核数，否则框架插入同步会出现异常，导致Kernel“卡死”现象。

【反例】

```
// 存在多核同步逻辑的代码中NumBlocks大于CoreNum  
// 比如在Tiling计算中没有做该校验  
FlashAttentionScoreApiTiling(tilingData);  
FlashAttentionScoreGetTensorSize(tilingData);  
CoreNum = ascendcPlatform.GetCoreNum();  
context->SetBlockDim(CoreNum + 1);
```

【正例】

```
FlashAttentionScoreApiTiling(tilingData);  
FlashAttentionScoreGetTensorSize(tilingData);  
// 在Kernel中有使用多核同步指令时，Host设置NumBlocks需要保证不大于CoreNum  
CoreNum = ascendcPlatform.GetCoreNum();  
context->SetBlockDim(CoreNum);
```

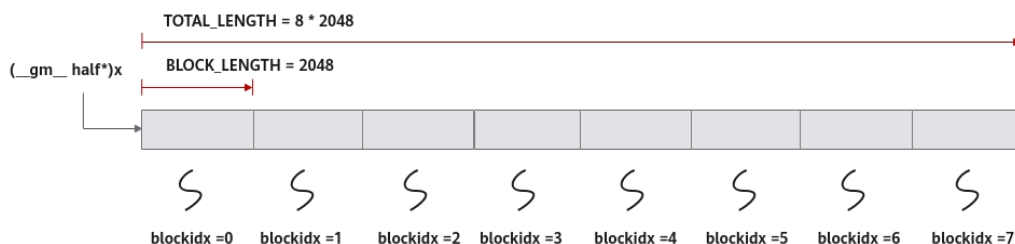
正确计算偏移地址

算子使能多核计算时，需要在Tiling的时候确定单核的计算量，Kernel侧根据单核计算量进行地址的偏移。

比如如下样例的分配方案：数据整体长度TOTAL_LENGTH为8 * 2048个元素，平均分配到8个核上运行，每个核上处理的数据大小BLOCK_LENGTH为2048。 $x + \text{BLOCK_LENGTH} * \text{GetBlockIdx}()$ 即为单核处理程序中输入x在Global Memory上的内存偏移地址，获取偏移地址后，使用GlobalTensor类的SetGlobalBuffer接口设定该核上Global Memory的起始地址以及长度。具体示意图请参考图3-73。

```
xGm.SetGlobalBuffer((_gm_half*x) + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
```

图 3-73 多核并行处理示意图



浮点数计算

- 每个浮点算术运算都涉及一定量的舍入。因此，算术运算的执行顺序很重要。如果A、B和C是浮点值，则(A+B)+C不能保证像数学计算中那样等于A+(B+C)。当并行计算时，可能会更改操作的顺序，因此并行结果可能与顺序结果不匹配。这种情况带来的精度差异是浮点值计算固有存在的。
- 某些型号的AI处理器指令支持的数据类型有限，当发现某个API支持的数据类型不能满足需求时，应更倾向于先将数据转换到更高的精度进行计算后，再将计算结果转换到目标精度，来防止精度损失。例如，某款AI处理器Vector计算类API不支持bfloat16的计算，需要先使用Cast接口转换成float数据类型，进行计算后再使用Cast接口转换回bfloat16数据类型。

【正确示例】

```
// dst = src0 + src1, src0、src1、dst均为bfloat16类型, tmp0、tmp1、tmp2均为float类型
...
Cast(tmp0Tensor, src0Tensor, RoundMode::CAST_NONE, computeSize);
Cast(tmp1Tensor, src1Tensor, RoundMode::CAST_NONE, computeSize);
Add(tmp2Tensor, tmp0Tensor, tmp1Tensor, computeSize);
Cast(dstTensor, tmp2Tensor, RoundMode::CAST_FLOOR, computeSize);
...
```

- 昇腾AI处理器都遵循IEEE 754标准进行二进制浮点表示，除了一些小的例外。这些例外可能会导致与在主机系统上计算的IEEE 754值不同的结果。例如，使用了复合指令的API Axy，源操作数中每个元素与标量求积后和目的操作数中的对应元素相加，该复合指令将乘加操作组合到单个指令执行，计算结果可能与分别执行这两个操作的单指令得到的结果略有不同。开发者在使用此类API时，需要考虑这种精度差异。

禁止修改 Kernel 函数参数

禁止修改Kernel函数参数，不能对函数参数重新进行赋值和修改。例如：

FlashAttentionKernel函数定义如下，其参数query、key、tilingData等为指针类型，该指针本身禁止修改。对于算子输入参数，指针指向的内容不可以修改；作为一

个例外，算子输出参数，指针指向的内容可以进行修改。特别要强调一下，为了实现静态编译，无论是对tilingData指针本身，还是对tilingData指针指向的内容均禁止修改。

```
__aiCore__ __global__ void FlashAttentionKernel(__gm__ uint8_t* query, __gm__ uint8_t* key, ..., __gm__
uint8_t* attention, ..., __gm__ uint8_t* tilingData) {
    .....
}
```

【反例】

// 对Kernel函数参数重新赋值、对TilingData内容进行修改是不允许的，以下是错误示例

```
query = tmpQueryPtr;  
key = tmpKeyPtr;  
tilingData = tmpTilingDataPtr;  
tilingData[0] = 2;
```

【正例】

```
// 输入参数仅进行读操作
inputQueryGMTensor.SetGlobalBuffer(query);

// 输出参数attention指针本身是只读，但其指向的内存可以读写
outputAttentionGMTensor.SetGlobalBuffer(attention);
...
DataCopy(outputAttentionGMTensor, outputAttentionLocalTensor, count);
```

3.4.3 算子调试

从[异构计算](#)章节可以了解到Ascend C算子主要包括Tiling和Kernel实现两部分组成。

Tiling实现运行在Host侧CPU上，一般使用传统的调测手段（比如gdb工具）即可完成调试。

Kernel实现运行在Device侧NPU上，Ascend C提供了多种调试方式，包括孪生调试、上板调试等，具体的调试方法请参考[算子调试](#)。

3.5 性能分析

3.5.1 获取性能数据

在进行性能优化之前，需要拿到准确的性能数据，了解性能现状，并根据性能现状分析下一步的优化方向。Ascend C提供了多种性能测试方法，包括上板Profiling、单原子性能仿真流水图等手段。

上板 Profiling

如下命令行是一个算子上板性能数据采集的示例，可以根据自身的需要灵活组合配置参数。示例中--output为可选参数，用于指定收集到的性能数据的存放路径；\$HOME/projects/MyApp/out/main为算子可执行文件。

```
msprof op --output=$HOME/projects/output $HOME/projects/MyApp/out/main
```

如下示例则展示了部分性能数据文件的样例：

图 3-74 PipeUtilization.csv（计算单元和搬运单元耗时占比）文件示例

[illegible]

详细的字段说明和性能分析工具的具体使用方法请参考上板性能数据采集。

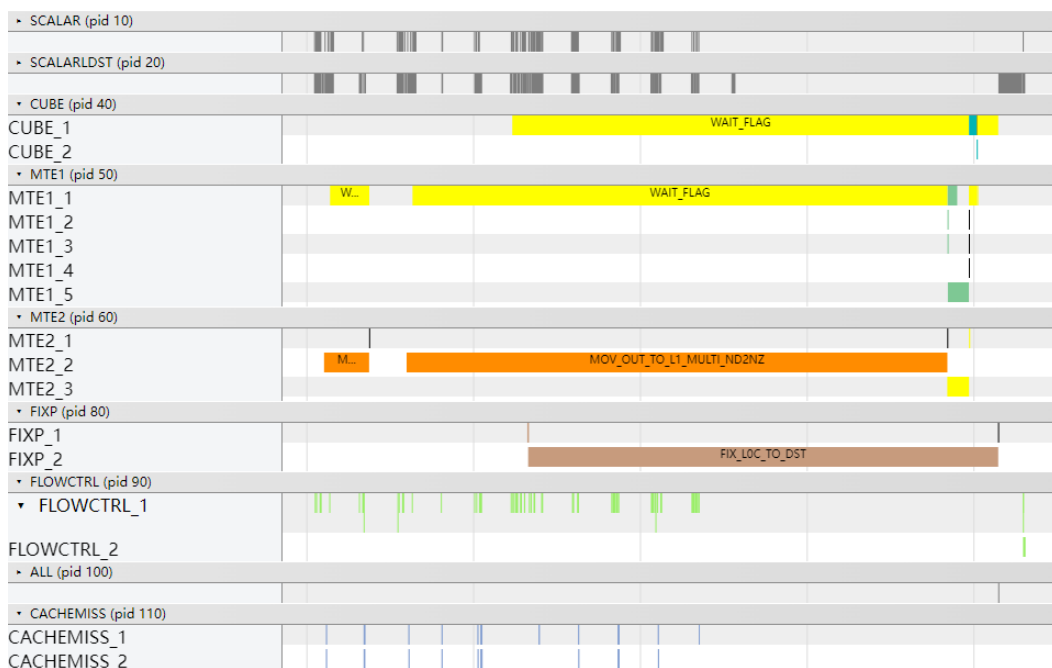
算子仿真流水图

算子调优工具msProf支持仿真环境下的性能数据采集和自动解析。使用msProf工具获取仿真流水图的具体方式请参考指令流水图。

支持以下两种可视化呈现方式：

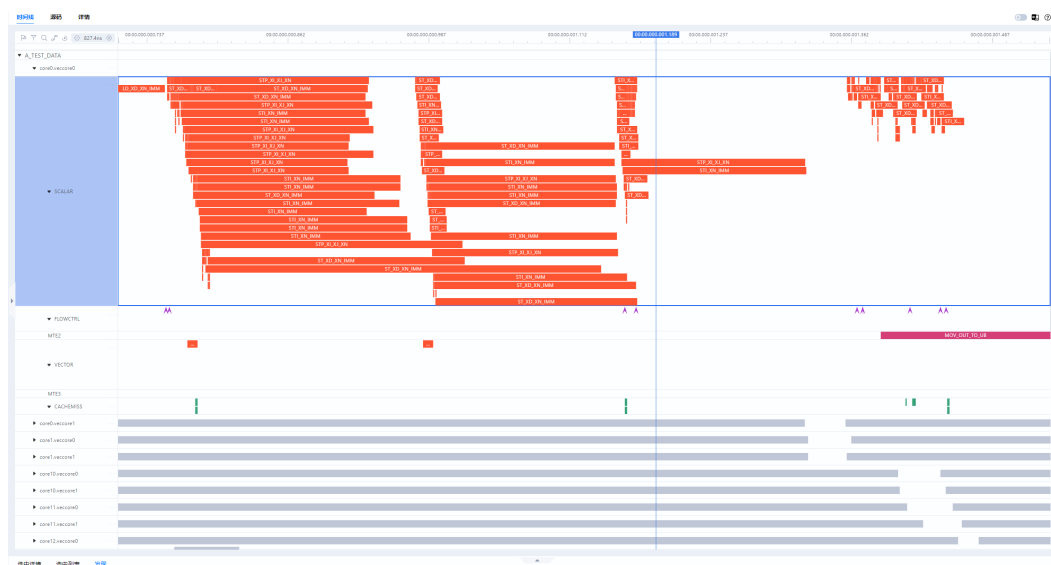
- Chrome浏览器

在Chrome浏览器中输入“chrome://tracing”地址，并将通过msprof op simulator生成指令流水图文件（trace.json）拖到空白处打开，键盘上输入快捷键（W：放大，S：缩小，A：左移，D：右移）可进行查看。



- 指令流水图支持MindStudio Insight可视化呈现，MindStudio Insight工具以时序图方式为用户提供指令在昇腾AI处理器上的运行情况，用户可通过分析时序图中的指令详情、指令执行时间、指令关联代码的调用栈及指令/流水间同步连线等信息，识别微观指令的时序优化点。

图 3-75 时间线界面



说明

本文部分样例中展示的算子仿真流水图和上述两种可视化呈现方式不一致，但是其中的关键字段含义是对应的，开发者可以参考指令流水图查看具体字段的含义。

3.5.2 分析性能数据

理论参数

理论性能为算子实际性能的理想目标。不同的硬件平台的硬件规格各异，理论性能可以帮助我们了解硬件的潜能，从而设定性能优化的目标。

- 搬运相关流水（MTE1/MTE2/MTE3等）的理论耗时 = 搬运数据量（单位：Byte） / 理论带宽。例如：某款AI处理器的GM峰值带宽约为1.8TB/s，想要进行一次float数据类型、4096 * 4096大小的矩阵搬运，搬运的理论耗时是 $\text{sizeof(float)} * 4096 * 4096 / 1.8\text{TB/s} = 37.28\mu\text{s}$ （按照1TB = 10^{12} Byte来计算）。

说明

- 搬运指令同时存在时，会存在共享带宽的情况，并不能每条都以接近理论带宽的速率搬运数据。比如，当MTE2/MTE3同时进行GM读写时，搬运流水线的耗时应该是（MTE2搬运量 + MTE3搬运量） / GM带宽。
- 搬运不同大小的数据块时，对带宽的利用率（有效带宽/理论带宽）不一样。针对每次搬运数据量较小的情况，实测性能达不到理论带宽。
- 计算相关流水（Cube/Vector/Scalar等）的理论耗时 = 计算数据量（单位：Element） / 理论算力。例如：某款AI处理器对float数据类型的Vector理论峰值算力为11.06TOPS，想要进行一次32K float类型的Element单指令计算，计算的理论耗时是 $32\text{K} / 11.06\text{TOPS} = 0.003\mu\text{s}$ （按照1K = 1000来计算）。

查找瓶颈

获取性能数据后，和理论数值差异较大的地方、耗时较长的流程被认为是“瓶颈点”。下文将介绍如何通过性能数据找到瓶颈点和对应的优化方向。

- 方法一：通过上板Profiling分析流水情况

查看上板Profiling解析后的op_summary_*.csv文件分析流水情况。注：“*”表示时间戳。

图 3-76 op_summary_*.csv 示例一

E		F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	aa	ab	ac	ad	ae	af	ag	ah	ai	aj	ak	al	am	an	ao	ap	aq	ar	as	at	au	av	aw	ax	ay	az	ba	bb	bc	bd	be	bf	bg	bh	bi	bj	bk	bl	bm	bn	bo	bp	bq	br	bs	bt	bu	bv	bw	bx	by	bz	ca	cb	cc	cd	ce	cf	cg	ch	ci	cj	ck	cl	cm	cn	co	cp	cq	cr	cs	ct	cu	cv	cw	cx	cy	cz	da	db	dc	dd	de	df	dg	dh	di	dj	dk	dl	dm	dn	do	dp	dq	dr	ds	dt	du	dv	dw	dx	dy	dz	ea	eb	ec	ed	ee	ef	eg	eh	ei	ej	ek	el	em	en	eo	ep	eq	er	es	et	eu	ev	ew	ex	ey	ez	fa	fb	fc	fd	fe	ff	fg	fh	fi	fj	fk	fl	fm	fn	fo	fp	fq	fr	fs	ft	fu	fv	fw	fx	fy	fz	ga	gb	gc	gd	ge	gf	gg	gh	gi	gj	gk	gl	gm	gn	go	gp	gq	gr	gs	gt	gu	gv	gw	gx	gy	gz	ha	hb	hc	hd	he	hf	hg	hh	hi	hj	hk	hl	hm	hn	ho	hp	hq	hr	hs	ht	hu	hv	hw	hx	hy	hz	ia	ib	ic	id	ie	if	ig	ih	ii	ij	ik	il	im	in	io	ip	iq	ir	is	it	iu	iv	iw	ix	iy	iz	ja	jb	jc	jd	je	jf	jj	jk	jl	jm	jn	jo	jp	jq	jr	js	jt	ju	jv	jw	jx	ky	kz	la	lb	lc	ld	le	lf	lg	lh	li	lj	lk	ll	lm	ln	lo	lp	lq	lr	ls	lt	lu	lv	lw	lx	ly	lz	ma	mb	mc	md	me	mf	mg	mh	mi	mj	mk	ml	mm	mn	mo	mp	mq	mr	ms	mt	mu	mv	mw	mx	my	mz	na	nb	nc	nd	ne	nf	ng	nh	ni	nj	nk	nl	nm	nn	no	np	nq	nr	ns	nt	nu	nv	nw	nx	ny	nz	oa	ob	oc	od	oe	of	og	oh	oi	oj	ok	ol	om	on	oo	op	oq	or	os	ot	ou	ov	ow	ox	oy	oz	pa	pb	pc	pd	pe	pf	pg	ph	pi	pj	pk	pl	pm	pn	po	pp	pq	pr	ps	pt	pu	pv	pw	px	py	pz	qa	qb	qc	qd	qe	qf	qg	qh	qi	qj	qk	ql	qm	qn	qo	qp	qq	qr	qs	qt	qu	qv	qw	qx	qy	qz	ra	rb	rc	rd	re	rf	rg	rh	ri	rj	rk	rl	rm	rn	ro	rp	rq	rr	rs	rt	ru	rv	rw	rx	ry	rz	sa	sb	sc	sd	se	sf	sg	sh	si	sj	sk	sl	sm	sn	so	sp	sq	sr	ss	st	su	sv	sw	sx	sy	sz	ta	tb	tc	td	te	tf	tg	th	ti	tj	tk	tl	tm	tn	to	tp	tq	tr	ts	tt	tu	tv	tw	tx	ty	tz	ua	ub	uc	ud	ue	uf	ug	uh	ui	uj	uk	ul	um	un	uo	up	uq	ur	us	ut	uu	uv	uw	ux	uy	uz	va	vb	vc	vd	ve	vf	vg	vh	vi	vj	vk	vl	vm	vn	vo	vp	vq	vr	vs	vt	vu	vv	vw	vx	vy	vz	wa	wb	wc	wd	we	wf	wg	wh	wi	wj	wk	wl	wm	wn	wo	wp	wq	wr	ws	wt	wu	wv	ww	wx	wy	wz	xa	xb	xc	xd	xe	xf	xg	xh	xi	xj	xk	xl	xm	xn	xo	xp	xq	xr	xs	xt	xu	xv	xw	xx	xy	xz	ya	yb	yc	yd	ye	yf	yg	yh	yi	yj	yk	yl	ym	yn	yo	yp	yq	yr	ys	yt	yu	yv	yw	yx	yy	yz	za	zb	zc	zd	ze	zf	zg	zh	zi	zj	zk	zl	zm	zn	zo	zp	zq	zr	zs	zt	zu	zv	zw	zx	zy	zz																							
OP Type	Task Type	Task Start Time(s)	Task Duration(s)	Block Dim	Input Shapes	Input Data Types	Input Tensor Shape (x,y,z)	accout int (x,y,z)	acc.float (x,y,z)	acc.msc (x,y,z)	acc.wc (x,y,z)	acc.scalar (x,y,z)	acc.int (x,y,z)	acc.time(s)	acc.mem (MB)	acc.flops (G)	acc.aa (x,y,z)	acc.ab (x,y,z)	acc.ac (x,y,z)	acc.ad (x,y,z)	acc.ae (x,y,z)	acc.af (x,y,z)	acc.ag (x,y,z)	acc.ah (x,y,z)	acc.ai (x,y,z)	acc.aj (x,y,z)	acc.ak (x,y,z)	acc.al (x,y,z)	acc.am (x,y,z)	acc.an (x,y,z)	acc.ao (x,y,z)	acc.ap (x,y,z)	acc.aq (x,y,z)	acc.ar (x,y,z)	acc.as (x,y,z)	acc.at (x,y,z)	acc.au (x,y,z)	acc.av (x,y,z)	acc.aw (x,y,z)	acc.ax (x,y,z)	acc.ay (x,y,z)	acc.az (x,y,z)	acc.ba (x,y,z)	acc.bb (x,y,z)	acc.bc (x,y,z)	acc.bd (x,y,z)	acc.be (x,y,z)	acc.bf (x,y,z)	acc.bg (x,y,z)	acc.bh (x,y,z)	acc.bi (x,y,z)	acc.bj (x,y,z)	acc.bk (x,y,z)	acc.bl (x,y,z)	acc.bm (x,y,z)	acc.bn (x,y,z)	acc.bo (x,y,z)	acc.bp (x,y,z)	acc.bq (x,y,z)	acc.br (x,y,z)	acc.bs (x,y,z)	acc.bt (x,y,z)	acc.bu (x,y,z)	acc.bv (x,y,z)	acc.bw (x,y,z)	acc.bx (x,y,z)	acc.by (x,y,z)	acc.bz (x,y,z)	acc.ca (x,y,z)	acc.cb (x,y,z)	acc.cc (x,y,z)	acc.cd (x,y,z)	acc.ce (x,y,z)	acc.cf (x,y,z)	acc.cg (x,y,z)	acc.ch (x,y,z)	acc.ci (x,y,z)	acc.cj (x,y,z)	acc.ck (x,y,z)	acc.cl (x,y,z)	acc.cm (x,y,z)	acc.cn (x,y,z)	acc.co (x,y,z)	acc.cp (x,y,z)	acc.cq (x,y,z)	acc.cr (x,y,z)	acc.cs (x,y,z)	acc.ct (x,y,z)	acc.cu (x,y,z)	acc.cv (x,y,z)	acc.cw (x,y,z)	acc.cx (x,y,z)	acc.cy (x,y,z)	acc.cz (x,y,z)	acc.da (x,y,z)	acc.db (x,y,z)	acc.dc (x,y,z)	acc.dd (x,y,z)	acc.de (x,y,z)	acc.df (x,y,z)	acc.dg (x,y,z)	acc.dh (x,y,z)	acc.di (x,y,z)	acc.dj (x,y,z)	acc.dk (x,y,z)	acc.dl (x,y,z)	acc.dm (x,y,z)	acc.dn (x,y,z)	acc.do (x,y,z)	acc.dp (x,y,z)	acc.dq (x,y,z)	acc.dr (x,y,z)	acc.ds (x,y,z)	acc.dt (x,y,z)	acc.du (x,y,z)	acc.dv (x,y,z)	acc.dw (x,y,z)	acc.dx (x,y,z)	acc.dy (x,y,z)	acc.dz (x,y,z)	acc.ea (x,y,z)	acc.eb (x,y,z)	acc.ec (x,y,z)	acc.ed (x,y,z)	acc.ee (x,y,z)	acc.ef (x,y,z)	acc.eg (x,y,z)	acc.eh (x,y,z)	acc.ei (x,y,z)	acc.ej (x,y,z)	acc.ek (x,y,z)	acc.el (x,y,z)	acc.em (x,y,z)	acc.en (x,y,z)	acc.eo (x,y,z)	acc.ep (x,y,z)	acc.eq (x,y,z)	acc.er (x,y,z)	acc.es (x,y,z)	acc.et (x,y,z)	acc.eu (x,y,z)	acc.ev (x,y,z)	acc.ew (x,y,z)	acc.ex (x,y,z)	acc.ey (x,y,z)	acc.ez (x,y,z)	acc.fa (x,y,z)	acc.fb (x,y,z)	acc.fc (x,y,z)	acc.fd (x,y,z)	acc.fe (x,y,z)	acc.ff (x,y,z)	acc.fg (x,y,z)	acc.fh (x,y,z)	acc.fi (x,y,z)	acc.fj (x,y,z)	acc.fk (x,y,z)	acc.fl (x,y,z)	acc.fm (x,y,z)	acc.fn (x,y,z)	acc.fo (x,y,z)	acc.fp (x,y,z)	acc.fq (x,y,z)	acc.fr (x,y,z)	acc.fs (x,y,z)	acc.ft (x,y,z)	acc.fu (x,y,z)	acc.fv (x,y,z)	acc.fw (x,y,z)	acc.fx (x,y,z)	acc.fy (x,y,z)	acc.fz (x,y,z)	acc.ga (x,y,z)	acc.gb (x,y,z)	acc.gc (x,y,z)	acc.gd (x,y,z)	acc.ge (x,y,z)	acc.gf (x,y,z)	acc.gg (x,y,z)	acc.gh (x,y,z)	acc.gi (x,y,z)	acc.gj (x,y,z)	acc.gk (x,y,z)	acc.gl (x,y,z)	acc.gm (x,y,z)	acc.gn (x,y,z)	acc.go (x,y,z)	acc.gp (x,y,z)	acc.gq (x,y,z)	acc.gr (x,y,z)	acc.gs (x,y,z)	acc.gt (x,y,z)	acc.gu (x,y,z)	acc.gv (x,y,z)	acc.gw (x,y,z)	acc.gx (x,y,z)	acc.gy (x,y,z)	acc.gz (x,y,z)	acc.ha (x,y,z)	acc.hb (x,y,z)	acc.hc (x,y,z)	acc.hd (x,y,z)	acc.he (x,y,z)	acc.hf (x,y,z)	acc.hg (x,y,z)	acc.hh (x,y,z)	acc.hi (x,y,z)	acc.hj (x,y,z)	acc.hk (x,y,z)	acc.hl (x,y,z)	acc.hm (x,y,z)	acc.hn (x,y,z)	acc.ho (x,y,z)	acc.hp (x,y,z)	acc.hq (x,y,z)	acc.hr (x,y,z)	acc.hs (x,y,z)	acc.ht (x,y,z)	acc.hu (x,y,z)	acc.hv (x,y,z)	acc.hw (x,y,z)	acc.hx (x,y,z)	acc.hy (x,y,z)	acc.hz (x,y,z)	acc.ia (x,y,z)	acc.ib (x,y,z)	acc.ic (x,y,z)	acc.id (x,y,z)	acc.ie (x,y,z)	acc.if (x,y,z)	acc.ig (x,y,z)	acc.ih (x,y,z)	acc.ii (x,y,z)	acc.ij (x,y,z)	acc.ik (x,y,z)	acc.il (x,y,z)	acc.im (x,y,z)	acc.in (x,y,z)	acc.io (x,y,z)	acc.ip (x,y,z)	acc.iq (x,y,z)	acc.ir (x,y,z)	acc.is (x,y,z)	acc.it (x,y,z)	acc.iu (x,y,z)	acc.iv (x,y,z)	acc.iw (x,y,z)	acc.ix (x,y,z)	acc.iy (x,y,z)	acc.iz (x,y,z)	acc.ja (x,y,z)	acc.jb (x,y,z)	acc.jc (x,y,z)	acc.jd (x,y,z)	acc.je (x,y,z)	acc.jf (x,y,z)	acc.jg (x,y,z)	acc.jh (x,y,z)	acc.ji (x,y,z)	acc.jj (x,y,z)	acc.jk (x,y,z)	acc.jl (x,y,z)	acc.jm (x,y,z)	acc.jn (x,y,z)	acc.jo (x,y,z)	acc.jp (x,y,z)	acc.jq (x,y,z)	acc.jr (x,y,z)	acc.js (x,y,z)	acc.jt (x,y,z)	acc.ju (x,y,z)	acc.jv (x,y,z)	acc.jw (x,y,z)	acc.jx (x,y,z)	acc.jy (x,y,z)	acc.jz (x,y,z)	acc.ka (x,y,z)	acc.kb (x,y,z)	acc.kc (x,y,z)	acc.kd (x,y,z)	acc.ke (x,y,z)	acc.kf (x,y,z)	acc.kg (x,y,z)	acc.kh (x,y,z)	acc.ki (x,y,z)	acc.kj (x,y,z)	acc.ka (x,y,z)	acc.kl (x,y,z)	acc.km (x,y,z)	acc.kn (x,y,z)	acc.ko (x,y,z)	acc.kp (x,y,z)	acc.kq (x,y,z)	acc.kr (x,y,z)	acc.ks (x,y,z)	acc.kt (x,y,z)	acc.ku (x,y,z)	acc.kv (x,y,z)	acc.kw (x,y,z)	acc.kx (x,y,z)	acc.ky (x,y,z)	acc.kz (x,y,z)	acc.la (x,y,z)	acc.lb (x,y,z)	acc.lc (x,y,z)	acc.ld (x,y,z)	acc.le (x,y,z)	acc.lf (x,y,z)	acc.lg (x,y,z)	acc.lh (x,y,z)	acc.li (x,y,z)	acc.lj (x,y,z)	acc.lk (x,y,z)	acc.ll (x,y,z)	acc.lm (x,y,z)	acc.ln (x,y,z)	acc.lo (x,y,z)	acc.lp (x,y,z)	acc.lq (x,y,z)	acc.lr (x,y,z)	acc.ls (x,y,z)	acc.lt (x,y,z)	acc.lu (x,y,z)	acc.lv (x,y,z)	acc.lw (x,y,z)	acc.lx (x,y,z)	acc.ly (x,y,z)	acc.lz (x,y,z)	acc.ma (x,y,z)	acc.mb (x,y,z)	acc.mc (x,y,z)	acc.md (x,y,z)	acc.me (x,y,z)	acc.mf (x,y,z)	acc.mg (x,y,z)	acc.mh (x,y,z)	acc.mi (x,y,z)	acc.mj (x,y,z)	acc.mk (x,y,z)	acc.ml (x,y,z)	acc.mm (x,y,z)	acc.mn (x,y,z)	acc.mo (x,y,z)	acc.mp (x,y,z)	acc.mq (x,y,z)	acc.mr (x,y,z)	acc.ms (x,y,z)	acc.mt (x,y,z)	acc.mu (x,y,z)	acc.mv (x,y,z)	acc.mw (x,y,z)	acc.mx (x,y,z)	acc.my (x,y,z)	acc.mz (x,y,z)	acc.na (x,y,z)	acc.nb (x,y,z)	acc.nc (x,y,z)	acc.nd (x,y,z)	acc.ne (x,y,z)	acc.nf (x,y,z)	acc.ng (x,y,z)	acc.nh (x,y,z)	acc.ni (x,y,z)	acc.nj (x,y,z)	acc.nk (x,y,z)	acc.nl (x,y,z)	acc.nm (x,y,z)	acc.nn (x,y,z)	acc.no (x,y,z)	acc.np (x,y,z)	acc.nq (x,y,z)	acc.nr (x,y,z)	acc.ns (x,y,z)	acc.nt (x,y,z)	acc.nu (x,y,z)	acc.nv (x,y,z)	acc.nw (x,y,z)	acc.nx (x,y,z)	acc.ny (x,y,z)	acc.nz (x,y,z)	acc.oa (x,y,z)	acc.ob (x,y,z)	acc.oc (x,y,z)	acc.od (x,y,z)	acc.oe (x,y,z)	acc.of (x,y,z)	acc.og (x,y,z)	acc.oh (x,y,z)	acc.oi (x,y,z)	acc.oj (x,y,z)	acc.ok (x,y,z)	acc.ol (x,y,z)	acc.om (x,y,z)	acc.on (x,y,z)	acc.oo (x,y,z)	acc.op (x,y,z)	acc.oq (x,y,z)	acc.or (x,y,z)	acc.os (x,y,z)	acc.ot (x,y,z)	acc.ou (x,y,z)	acc.ov (x,y,z)	acc.ow (x,y,z)	acc.ox (x,y,z)	acc.oy (x,y,z)	acc.oz (x,y,z)	acc.pa (x,y,z)	acc.pb (x,y,z)	acc.pc (x,y,z)	acc.pd (x,y,z)	acc.pe (x,y,z)	acc.pf (x,y,z)	acc.pg (x,y,z)	acc.ph (x,y,z)	acc.pi (x,y,z)	acc.pj (x,y,z)	acc.pk (x,y,z)	acc.pl (x,y,z)	acc.pm (x,y,z)	acc.pn (x,y,z)	acc.po (x,y,z)	acc.pp (x,y,z)	acc.pq (x,y,z)	acc.pr (x,y,z)	acc.ps (x,y,z)	acc.pt (x,y,z)	acc.pu (x,y,z)	acc.pv (x,y,z)	acc.pw (x,y,z)	acc.px (x,y,z)	acc.py (x,y,z)	acc.pz (x,y,z)	acc.qa (x,y,z)	acc.qb (x,y,z)	acc.qc (x,y,z)	acc.qd (x,y,z)	acc.qe (x,y,z)	acc.qf (x,y,z)	acc.qg (x,y,z)	acc.qh (x,y,z)	acc.qi (x,y,z)	acc.qj (x,y,z)	acc.qk (x,y,z)	acc ql (x,y,z)	acc.qm (x,y,z)	acc.qn (x,y,z)	acc.qo (x,y,z)	acc.qp (x,y,z)	acc.qq (x,y,z)	acc.qr (x,y,z)	acc.qs (x,y,z)	acc.qt (x,y,z)	acc.qu (x,y,z)	acc.qv (x,y,z)	acc.qw (x,y,z)	acc.qx (x,y,z)	acc.qy (x,y,z)	acc.qz (x,y,z)	acc.ra (x,y,z)	acc.rb (x,y,z)	acc.rc (x,y,z)	acc.rd (x,y,z)	acc.re (x,y,z)	acc.rf (x,y,z)	acc.rg (x,y,z)	acc.rh (x,y,z)	acc.ri (x,y,z)	acc.rj (x,y,z)	acc.rk (x,y,z)	acc.rl (x,y,z)	acc.rm (x,y,z)	acc.rn (x,y,z)	acc.ro (x,y,z)	acc.rp (x,y,z)	acc.rq (x,y,z)	acc.rr (x,y,z)	acc.rs (x,y,z)	acc.rt (x,y,z)	acc.ru (x,y,z)	acc.rv (x,y,z)	acc.rw (x,y,z)	acc.rx (x,y,z)	acc.ry (x,y,z)	acc.rz (x,y,z)	acc.sa (x,y,z)	acc.sb (x,y,z)	acc.sc (x,y,z)	acc.sd (x,y,z)	acc.se (x,y,z)	acc.sf (x,y,z)	acc.sg (x,y,z)	acc.sh (x,y,z)	acc.si (x,y,z)	acc.sj (x,y,z)	acc.sk (x,y,z)	acc.sl (x,y,z)	acc.sm (x,y,z)	acc.sn (x,y,z)	acc.so (x,y,z)	acc.sp (x,y,z)	acc.sq (x,y,z)	acc.sr (x,y,z)	acc.ss (x,y,z)	acc.st (x,y,z)	acc.su (x,y,z)	acc.sv (x,y,z)	acc.sw (x,y,z)	acc.sx (x,y,z)	acc.sy (x,y,z)	acc.sz (x,y,z)	acc.ta (x,y,z)	acc.tb (x,y,z)	acc.tc (x,y,z)	acc.td (x,y,z)	acc.te (x,y,z)	acc.tf (x,y,z)	acc.tg (x,y,z)	acc.th (x,y,z)	acc.ti (x,y,z)	acc.tj (x,y,z)	acc.tk (x,y,z)	acc.tl (x,y,z)	acc.tm (x,y,z)	acc.tn (x,y,z)	acc.to (x,y,z)	acc.tp (x,y,z)	acc tq (x,y,z)	acc.tr (x,y,z)	acc.ts (x,y,z)	acc.tt (x,y,z)	acc.tu (x,y,z)	acc.tv (x,y,z)	acc.tw (x,y,z)	acc.tx (x,y,z)	acc.ty (x,y,z)	acc.tz (x,y,z)	acc.ua (x,y,z)	acc.ub (x,y,z)	acc.uc (x,y,z)	acc.ud (x,y,z)	acc.ue (x,y,z)	acc.uf (x,y,z)	acc.ug (x,y,z)	acc.uh (x,y,z)	acc.ui (x,y,z)	acc.uj (x,y,z)	acc.uk (x,y,z)	acc.ul (x,y,z)	acc.um (x,y,z)	acc.un (x,y,z)	acc.uo (x,y,z)	acc.up (x,y,z)	acc.uq (x,y,z)	acc.ur (x,y,z)	acc.us (x,y,z)	acc.ut (x,y,z)	acc.uu (x,y,z)	acc.uv (x,y,z)	acc.uw (x,y,z)	acc.ux (x,y,z)	acc.uy (x,y,z)	acc.uz (x,y,z)	acc.va (x,y,z)	acc.vb (x,y,z)	acc.vc (x,y,z)	acc.vd (x,y,z)	acc.ve (x,y,z)	acc.vf (x,y,z)	acc.vg (x,y,z)	acc.vh (x,y,z)	acc.vi (x,y,z)	acc.vj (x,y,z)	acc.vk (x,y,z)	acc.vl (x,y,z)	acc.vm (x,y,z)	acc.vn (x,y,z)	acc.vo (x,y,z)	acc.vp (x,y,z)	acc.vq (x,y,z)	acc.vr (x,y,z)	acc.vs (x,y,z)	acc.vt (x,y,z)	acc.vu (x,y,z)	acc.vv (x,y,z)	acc.vw (x,y,z)	acc.vx (x,y,z)	acc.vy (x,y,z)	acc.vz (x,y,z)	acc.wa (x,y,z)	acc.wb (x,y,z)	acc.wc (x,y,z)	acc.wd (x,y,z)	acc.we (x,y,z)	acc.wf (x,y,z)	acc.wg (x,y,z)	acc.wh (x,y,z)	acc.wi (x,y,z)	acc.wj (x,y,z)	acc.wk (x,y,z)	acc.wl (x,y,z)	acc.wm (x,y,z)	acc.wn (x,y,z)	acc.wo (x,y,z)	acc.wp (x,y,z)	acc.wq (x,y,z)	acc.wr (x,y,z)	acc.ws (x,y,z)	acc.wt (x,y,z)	acc.wu (x,y,z)	acc.wv (x,y,z)	acc.ww (x,y,z)	acc.wx (x,y,z)	acc.wy (x,y,z)	acc.wz (x,y,z)	acc.xa (x,y,z)	acc.xb (x,y,z)	acc.xc (x,y,z)	acc.xd (x,y,z)	acc.xe (x,y,z)	acc.xf (x,y,z)	acc.xg (x,y,z)	acc.xh (x,y,z)	acc.xi (x,y,z)	acc.xj (x,y,z)	acc.xk (x,y,z)	acc.xl (x,y,z)	acc.xm (x,y,z)	acc.xn (x,y,z)	acc.xo (x,y,z)	acc.xp (x,y,z)	acc.xq (x,y,z)	acc.xr (x,y,z)	acc.xs (x,y,z)	acc.xt (x,y,z)	acc.xu (x,y,z)	acc.xv (x,y,z)	acc.xw (x,y,z)	acc.xx (x,y,z)	acc.xy (x,y,z)	acc.xz (x,y,z)	acc.ya (x,y,z)	acc.yb (x,y,z)	acc.yc (x,y,z)	acc.yd (x,y,z)	acc.ye (x,y,z)	acc.yf (x,y,z)	acc.yg (x,y,z)	acc.yh (x,y,z)	acc.yi (x,y,z)	acc.yj (x,y,z)	acc.yk (x,y,z)	acc.yl (x,y,z)	acc.ym (x,y,z)	acc.yn (x,y,z)	acc.yo (x,y,z)	acc.yp (x,y,z)	acc.yq (x,y,z)	acc.yr (x,y,z)	acc.ys (x,y,z)	acc.yt (x,y,z)	acc.yu (x,y,z)	acc.yv (x,y,z)	acc.yw (x,y,z)	acc.yx (x,y,z)	acc.yy (x,y,z)	acc.yz (x,y,z)	acc.za (x,y,z)	acc.zb (x,y,z)	acc.zc (x,y,z)	acc.zd (x,y,z)	acc.ze (x,y,z)	acc.zf (x,y,z)	acc.zg (x,y,z)	acc.zh (x,y,z)	acc.zi (x,y,z)	acc.zj (x,y,z)	acc.zk (x,y,z)	acc.zl (x,y,z)	acc.zm (x,y,z)	acc.zn (x,y,z)	acc.zo (x,y,z)	acc.zp (x,y,z)	acc.zq (x,y,z)	acc.zr (x,y,z)	acc.zs (x,y,z)	acc.zt (x,y,z)	acc.zu (x,y,z)	acc.zv (x,y,z)	acc.zw (x,y,z)	acc.zx (x,y,z)	acc.zy (x,y,z)	acc.zz (x,y,z)

每条流水线的利用率理想情况下应为100%，没有达到100%的流水就可能有提升空间。上图示例中为某款AI处理器上获取的数据，可以看到Cube算子MatMulV2，Cube流水的利用率aic_mac_ratio在80%左右，初步判断没有充分发挥算力；MTE2流水的利用率aic_mte2_ratio已经在95%左右，判断MTE2是最长的流水。

然后比较最长的流水和理论的差距：输入左右矩阵的shape分别为（2048，12288）、（12288，6144），数据类型为bfloat16；Bias输入的shape为（6144），数据类型为float。由此可以计算出总共需要搬运的数据量，继而通过[理论参数](#)中介绍的搬运流水理论耗时计算方法计算出理论值为 $(\text{sizeof}(\text{bfloat16}) * (2048 * 12288 + 12288 * 6144) + \text{sizeof}(\text{float}) * 6144) / 1.8\text{TB/s} \approx 111.8\mu\text{s}$ （按照 $1\text{TB} = 10^{12}\text{Byte}$ 来计算），与实际性能数据aic_mte2_time存在比较大的差距。经分析输入数据的总大小已经超过L1的空间（512KB），做MatMul计算会存在输入矩阵数据重复搬运的情况，重复搬运的次数是否合理，需要结合[流水优化](#)和[Tiling优化](#)手段进行优化，可参考[方法三](#)、[查看仿真流水图分析各条流水的情况](#)进一步分析。

图 3-77 op_summary_*.csv 示例二

	G	H	J	M	N	P	Q	R	AG	AH	AI	AJ	AK	AL	AM	AN	AO	AP
t	Start Time(us)	Duration(us)	Block Dim	Input Shapes	Input Data Types	Input Formats	Output Shapes	Output Data Types	Output Formats	av time(us)	all cycles	av vec t (me/us)	av vec ratio	av scalar time(us)	av scalar ratio	av time2 ratio	av time3 ratio	av time3 ratio
1	1.7e-15	350.3		40 '8192, '8192'	FLOAT	FORMAT '8192, '8192'	DT B16	FORMAT 1	346.83	24971839	19.7136	0.0568	1.8592	0.0054	343.7998	0.9913	137.559	0.3967
2	1.7e-15	349.92		40 '8192, '8192'	FLOAT	FORMAT '8192, '8192'	DT B16	FORMAT 1	346.2	24926318	19.7137	0.0569	1.8618	0.0054	342.772	0.9913	137.621	0.3971

上图示例中算子输入的shape为（8192, 8192），数据类型为float。由此可以计算出总共需要搬运的数据量，继而通过[理论参数](#)中介绍的搬运流水理论耗时计算方法计算出理论值为 $\text{sizeof(float)} * (8192 * 8192) / 0.8\text{TB/s} \approx 335.5\mu\text{s}$ （按照 $1\text{TB} = 10^{12}\text{Byte}$ 来计算，不同的AI处理器其理论带宽有差异），与实际性能数据aiv_mte2_time相符，可以判断该算子基本是一个搬运MTE2 bound（达到上限）的算子。本示例中总体执行时间Duration为350us，和MTE2的实际耗时持平，说明该算子已经调优完成。如果MTE2耗时和总体执行时间有较大差距，那么下一步优化方向主要是[流水优化](#)结合[Tiling优化](#)，使得其他的流水尽量隐藏在MTE2的流水中，可参考[方法三、查看仿真流水图分析各条流水的情况](#)进行进一步分析。

- **方法二：通过上板Profiling分析Tiling情况**

查看上板Profiling解析后的op_summary_*.csv文件分析Tiling情况。

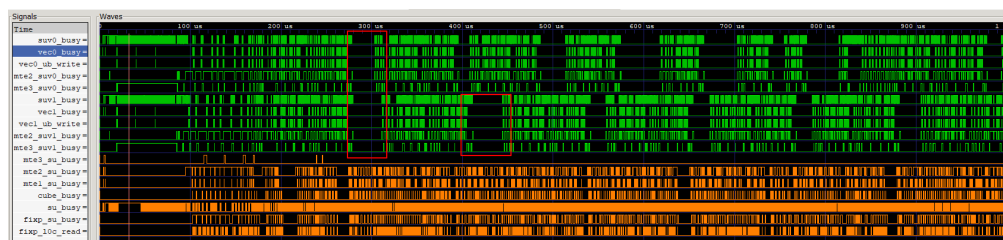
图 3-78 op_summary_*.csv 示例

OP Type	Task Type	Task Start Time(us)	Task Duration(us)	Block Dim	Input Shapes	Input Data Types	Input For
Mul	AI_VECTOR_COR	1706851252899848.070	1122.96	48	"14592,12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902039.430	3.24	12	"12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902057.450	3.42	12	"12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902076.310	3.06	12	"12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902095.030	3	12	"12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902113.930	2.98	12	"12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902132.930	2.96	12	"12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902286.730	31.72	48	"1536,12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902361.790	3.3	12	"12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252902876.890	392.28	48	"4608,12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252903611.830	2.18	5	"4608,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252904186.570	492.82	48	"12288,6144,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252905129.550	1.92	6	"6144,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252905703.770	491.86	48	"6144,12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252906650.210	2.76	12	"12288,"	FLOAT,FLOAT	FOR
Mul	AI_VECTOR_COR	1706851252906667.790	2.96	12	"12288,"	FLOAT,FLOAT	FOR

上图示例中为某款AI处理器上获取的数据，通过硬件平台可以查看该AI处理器有48个Vector核，Mul算子是一个纯Vector算子，但是有些场景没有用满所有Vector核（Block Dim < 48），造成算力浪费。那么下一步的主要优化方向为[Tiling优化](#)。

- 方法三：通过仿真流水图分析流水情况

图 3-79 仿真流水图示例



上图示例中为某款AI处理器上获取的数据，可以看到Vector核的相关流水（vec0的MTE2、MTE3，vec1的MTE2、MTE3等）有规律性的断流现象。可以结合算子逻辑分析，是否存在数据依赖等因素导致断流。那么下一步的主要优化方向为[流水优化](#)，其次结合[Tiling优化](#)和[内存优化](#)等手段进一步提升Vector流水利用率。

- 方法四：通过上板Profiling查看头开销

头开销是算子执行计算前产生的时延，包含核启动、核取址TLB MISS、同地址访问（由于硬件限制，多核同时访问相同内存地址冲突带来额外的时延）以及变量资源初始化带来的时延。以Atlas A2 训练系列产品/Atlas A2 推理系列产品为例，满核头开销约为20~21微秒。对于推理领域等本身延迟为微秒级别的算子，头开销是一个值得优化的对象。

通过上板Profiling数据（空Kernel时的TaskDuration数据）可以看到每个核的启动开销耗时，继而通过使用恰当的核数和算子Kernel Type等方法来不断的实践，尝试找到最优的配置，具体优化方向可以参考[3.6.3 头尾开销优化](#)。

3.6 SIMD 算子性能优化

3.6.1 优化建议总览表

表 3-12 性能优化建议总览表

分类	分类描述	优化建议
Tiling 策略	提供Tiling相关的优化建议，便于开发者选择合适的Tiling切分策略。	核间负载均衡
头尾开销优化	提供降低算子头尾开销（算子执行计算前后产生的时延）的优化建议。	设置合适的核数和算子Kernel类型
		限制TilingData结构大小
		避免TPipe在对象内创建和初始化
		核函数内删除Workspace相关冗余操作
流水编排	通过任务并行化、异步调度等方法，提升硬件资源利用率，实现更高的吞吐率。	使能DoubleBuffer
		使能Iterate或IterateAll异步接口避免AIC/AIV同步依赖
内存访问	通过控制搬运的数据块大小和GM地址等来实现搬运效率的最大化；通过Buffer的共享与复用、数据压缩精简、使用专用存储空间、访存调度优化等方法来减少内存占用，提升计算效率。	尽量一次搬运较大的数据块
		GM地址尽量512B对齐
		高效的使用搬运API
		避免同地址访问
		设置合理的L2 CacheMode
		算子与高阶API共享临时Buffer
		纯搬运类算子VECIN和VECOUT建议复用
		通过缩减Tensor ShapeInfo维度，优化栈空间
		避免Unified Buffer的bank冲突
矢量计算	矢量计算相关优化建议。	L2 Cache切分
		通过Unified Buffer融合实现连续vector计算
		Vector算子灵活运用Counter模式
矩阵计算	矩阵计算相关优化建议。	选择低延迟指令，优化归约操作性能
		通过BT Buffer实现高效的bias计算
		通过FP Buffer存放量化参数实现高效随路量化

分类	分类描述	优化建议
		通过L0C Buffer数据暂存实现高效的矩阵乘结果累加
		较小矩阵长驻L1 Buffer，仅分次搬运较大矩阵
		Matmul使能AtomicAdd选项

3.6.2 Tiling 策略

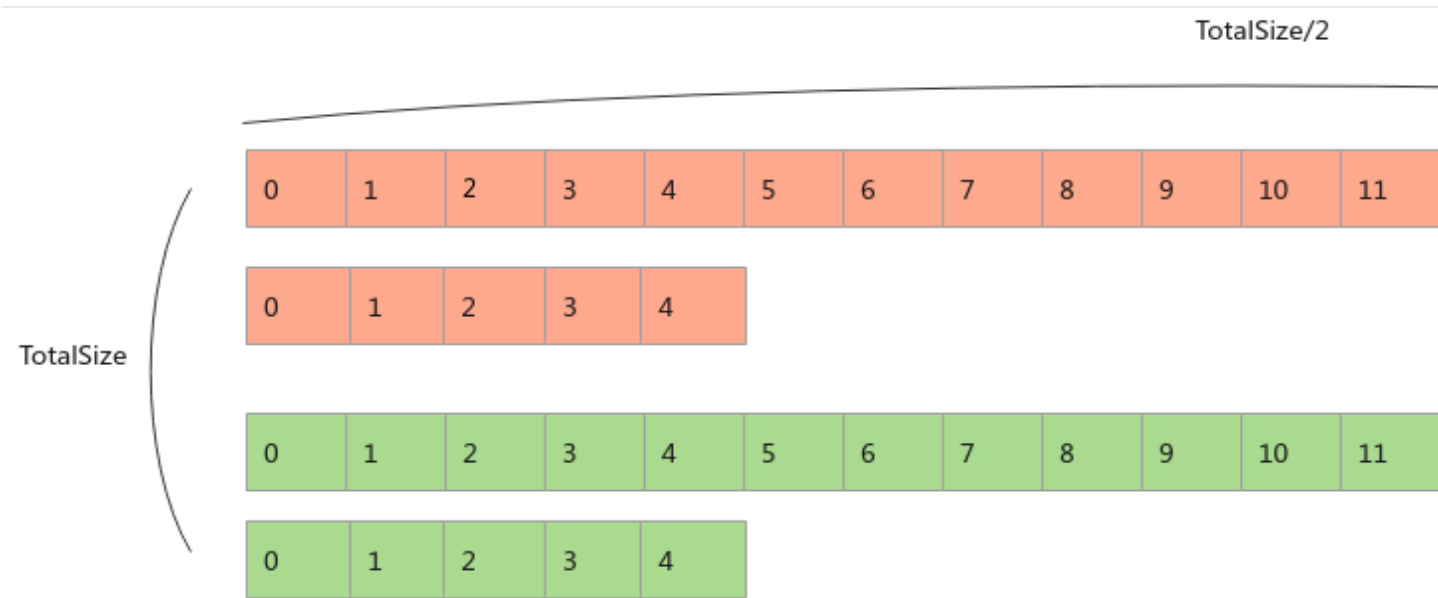
3.6.2.1 核间负载均衡

【优先级】：中

【描述】AI处理器的物理核数是固定的，当L2 Cache切分之后，可能发生部分核有计算拖尾的情况，即每次所有核计算量除以每个核处理的数据量不能被核数整除，导致最后需要部分尾核来计算尾块数据。而在尾核计算时，部分核始终处于空闲状态，从而使得算子的整体性能变差。如图1，假设总的数据量为TotalSize，L2 Cache切分之后分为两份 $TotalSize / 2$ ，每个核每次的计算量为 $TotalSize / 2 / 25$ ，即需要25个核进行处理，由于AI处理器的核数为20，因此每次计算时，1到5核的每个核需要多算一份数据，导致发生拖尾的情况。

【反例】

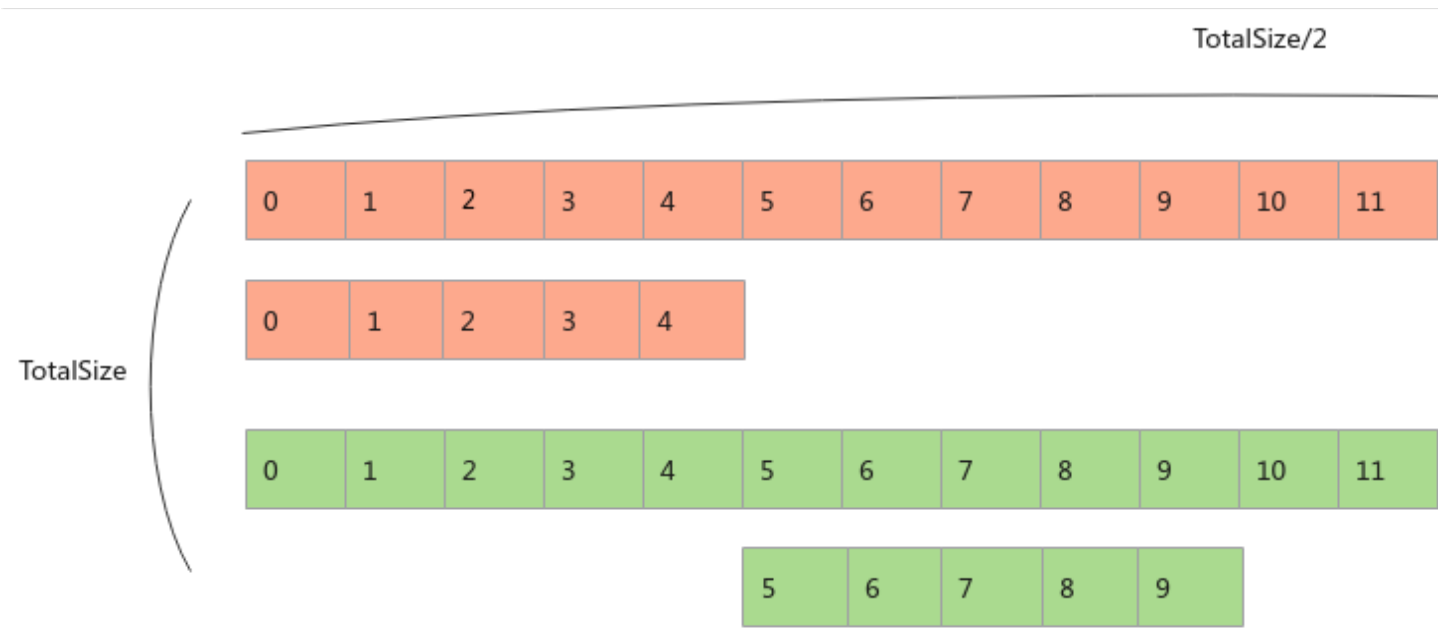
图 3-80 计算拖尾示意图



【正例】

针对上述切分策略，调整拖尾核的位置后可以达到全局负载最优，如图2所示。完成所有计算时，1到10核多一次数据块的计算，可以实现全局负载最优。

图 3-81 核间负载均衡示意图



3.6.3 头尾开销优化

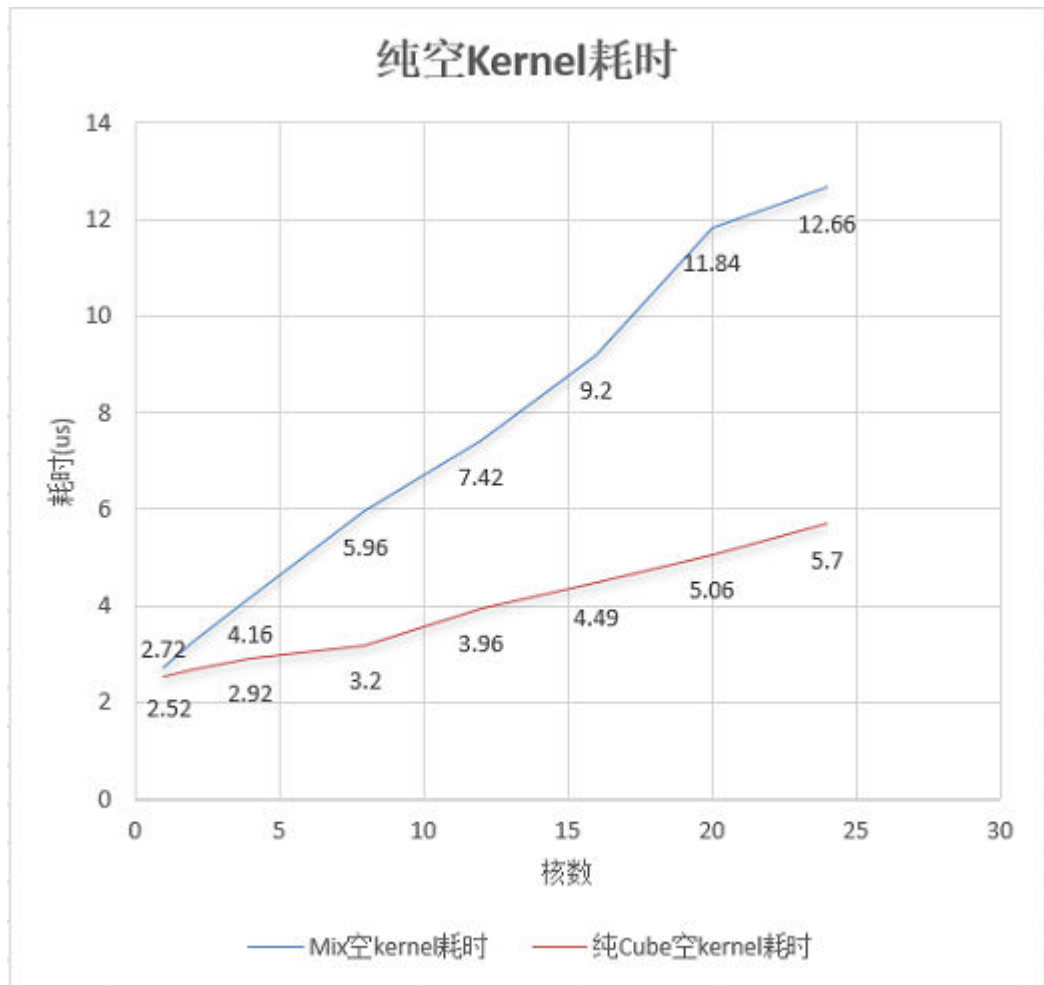
3.6.3.1 设置合适的核数和算子 Kernel 类型

在算子执行过程中，可能会因为以下几个原因产生额外的启动开销或者头开销：

1. **核启动**：每个核在启动时需要进行初始化操作，加载必要的配置和资源。
2. **核取址TLB MISS**：当核在访问内存时，如果Translation Lookaside Buffer（TLB）中没有对应的页表项，就需要从内存中加载页表项，这会导致额外的延迟。
3. **同地址访问冲突**：由于硬件限制，多个核同时访问相同的内存地址时可能会发生冲突，导致额外的时延。
4. **变量资源初始化**：在算子执行前，需要初始化一些变量和资源，这也可能带来额外的性能开销。

头开销会随着使用的核数增加而增加。下图展示了这部分头开销随启动核数的变化情况。

图 3-82 头开销随启动核数的变化



对于整体耗时在微秒级别且单核计算量耗时较少的算子，可以通过减少启动核数并增加单核计算量的方式来获得性能提升。这种优化方式的本质是在头开销耗时和单核计算量耗时之间进行权衡。为了达到最佳性能，开发者需要通过实践尝试，找到最合适的核数设置。

- 对于自定义算子工程，可以在TilingFunc（算子工程提供的在Host侧计算Tiling的默认函数）中通过SetBlockDim接口来设置算子使用的核数，具体设置方法请参考SetBlockDim；对于Kernel直调工程，可以在<<<>>>调用时指定算子使用的核数。
- 此外，算子的Kernel类型也会影响算子启动的核数。以纯Vector算子为例，如果以混合启动的方式执行该算子，调度器会同时启动Vector核和Cube核。然而，此时Cube核并没有实际的计算指令，但仍会产生核启动和核初始化的头开销。因此，建议设置合适的Kernel类型以最小化头开销。

通常，算子工程会通过算子使用的指令自动识别算子类型，但该功能无法区分AIC和AIV的配比，默认按照AIV:AIC为1:2的配比下发任务。此外，自动识别功能可能失效，因为其依赖于编译优化的结果。所以推荐用户手动设置算子的Kernel类型。具体设置方法请参考设置Kernel类型。

3.6.3.2 限制 TilingData 结构大小

【优先级】中

【描述】TilingData结构是Tiling切分信息的载体，当Host侧按照Tiling切分策略计算完Tiling后，算子会以入参的方式将Tiling切分信息从Host侧传递到Device侧，此时Tiling信息存放在GM上。调用GET_TILING_DATA宏后，会将Tiling信息从GM拷贝到AI处理器的栈空间上，期间会有拷贝开销，由于GM访问效率较低，同时考虑到栈空间限制，需要限制TilingData结构大小。拷贝耗时为us级别，在小shape的场景下，进行此类优化收益会更加明显。

限制TilingData结构大小，可以从以下方面考虑：

- 减少不必要的TilingData结构变量；
- 根据Tiling的数据范围选择合适的变量类型；
- 合理排布TilingData结构；
- TilingData整体结构要求8字节补齐。

【反例】

- 如下的示例中存在TilingData结构变量冗余的情况：NumBlocks信息已经通过SetBlockDim接口进行设置，可以在Kernel侧调用GetBlockNum接口获取，无需通过TilingData结构传递。
- 此外，变量的数据类型也不合理：formerNum和tailNum分别为计算整块数据的核数和计算尾块数据的核数，不会超过NUM_BLOCKS的值，使用uint8_t类型即可；formerLength等变量根据其计算逻辑，不会超出uint32_t的范围，使用uint32_t类型即可。

```
// Tiling结构体定义
BEGIN_TILING_DATA_DEF(TilingDataUnalign)
    TILING_DATA_FIELD_DEF(uint64_t, numBlocks);
    TILING_DATA_FIELD_DEF(uint64_t, formerNum);
    TILING_DATA_FIELD_DEF(uint64_t, tailNum);
    TILING_DATA_FIELD_DEF(uint64_t, formerLength);
    TILING_DATA_FIELD_DEF(uint64_t, tailLength);
    TILING_DATA_FIELD_DEF(uint64_t, alignNum);
END_TILING_DATA_DEF;

// Host侧Tiling函数计算Tiling结构信息
constexpr uint32_t NUM_BLOCKS = 8;
constexpr uint32_t SIZE_OF_HALF = 2;
constexpr uint32_t BLOCK_SIZE = 32;
constexpr uint32_t ALIGN_NUM = BLOCK_SIZE / SIZE_OF_HALF;
static ge::graphStatus TilingFunc(gert::TilingContext *context)
{
    TilingDataUnalign tiling;
    uint32_t totalLength = context->GetInputTensor(0)->GetShapeSize();
    // NumBlocks信息已经通过SetBlockDim接口进行设置
    context->SetBlockDim(NUM_BLOCKS);
    uint32_t totalLengthAligned = ((totalLength + ALIGN_NUM - 1) / ALIGN_NUM) * ALIGN_NUM;
    // formerNum、tailNum保证不超过0-NUM_BLOCKS数据范围
    uint32_t formerNum = (totalLengthAligned / ALIGN_NUM) % NUM_BLOCKS;
    uint32_t tailNum = NUM_BLOCKS - formerNum;
    // formerLength等变量根据其计算逻辑，不会超出uint32_t的范围
    uint32_t formerLength = ((totalLengthAligned / NUM_BLOCKS + ALIGN_NUM - 1) / ALIGN_NUM) *
        ALIGN_NUM;
    uint32_t tailLength = (totalLengthAligned / NUM_BLOCKS / ALIGN_NUM) * ALIGN_NUM;
    ...
}
```

【正例】

Tiling变量无冗余，变量数据类型最小化。

```
BEGIN_TILING_DATA_DEF(TilingDataUnalign)
    TILING_DATA_FIELD_DEF(uint8_t, formerNum);
    TILING_DATA_FIELD_DEF(uint8_t, tailNum);
    TILING_DATA_FIELD_DEF(uint32_t, formerLength);
```

```
TILING_DATA_FIELD_DEF(uint32_t, tailLength);  
TILING_DATA_FIELD_DEF(uint32_t, alignNum);  
END_TILING_DATA_DEF;
```

【反例】

如下的示例中TilingData结构不合理：由于AI处理器访存需要8字节对齐，在用户定义TilingData结构后，Ascend C工程框架会按照8字节对齐的方式对字节进行补齐，并保证整体TilingData结构满足8字节对齐要求。如下TilingData结构formerNum和tailNum变量都会补充3个字节，整体TilingData结构会因为8字节对齐再补充4个字节，该TilingData结构共计补充10个字节。

```
BEGIN_TILING_DATA_DEF(TilingDataUnalign)  
TILING_DATA_FIELD_DEF(uint8_t, formerNum); // 需补充3个字节，使得formerLength变量访问无误  
TILING_DATA_FIELD_DEF(uint32_t, formerLength);  
TILING_DATA_FIELD_DEF(uint8_t, tailNum); // 需补充3个字节，使得tailLength变量访问无误  
TILING_DATA_FIELD_DEF(uint32_t, tailLength);  
TILING_DATA_FIELD_DEF(uint32_t, alignNum); // 需补充4个字节，使得下个TilingData结构访问无误  
END_TILING_DATA_DEF;
```

【正例】

如下的示例中，对Tiling参数的排布进行了调整，字节排布合理，只需要补充2个字节，达到了减小TilingData结构的目的。

```
BEGIN_TILING_DATA_DEF(TilingDataUnalign)  
TILING_DATA_FIELD_DEF(uint8_t, formerNum);  
TILING_DATA_FIELD_DEF(uint8_t, tailNum); // 需补充2个字节，使得formerLength变量访问无误  
TILING_DATA_FIELD_DEF(uint32_t, formerLength);  
TILING_DATA_FIELD_DEF(uint32_t, tailLength);  
TILING_DATA_FIELD_DEF(uint32_t, alignNum);  
END_TILING_DATA_DEF;
```

3.6.3.3 避免 TPIPE 在对象内创建和初始化

【优先级】中

【编译器背景知识】创建类对象时，会分配内存空间，用于存储类中的相关成员变量或函数。当类中变量需要参与计算时，变量值从内存被加载到寄存器，计算完成后，变量从寄存器存储回内存。Scalar常量折叠和常量传播是编译器编译时的优化方式，优化前编译器会判断变量是否只初始化过一次或只赋值过一次，若满足此编译优化的前提条件，变量值将会尽量驻留在寄存器中，从而在后续使用变量时，将减少读取内存的操作，提升运行性能。

【描述】TPipe是用来管理全局内存和同步的框架，用户可以调用TPipe的接口，为TQue/TBuf进行内存分配。在编写Ascend C算子过程中，经常用一个类存放计算所需的相关变量，这里称类名为KernelExample。当TPipe对象在KernelExample类的实现中定义并初始化时，TPipe对象的内存空间在整个KernelExample对象的内存空间之中；需要注意的是，创建TPipe对象时，对象初始化会设置全局变量的TPipe指针，这导致KernelExample对象的内存有被外部污染的风险，此时编译器的编译优化将采取保守策略，不会对KernelExample对象中的Scalar变量进行常量折叠和常量传播。因此，在任何场景下，我们都建议将TPipe对象创建于KernelExample类外部，使得TPipe对象的内存空间独立于KernelExample类对象的内存空间，触发编译器对KernelExample类内Scalar的编译优化，减少算子Scalar指令耗时。

【反例】

代码中TPipe对象由KernelExample类内部创建并初始化，影响编译器Scalar折叠优化，在NPU侧导致Scalar不必要的增加。

```
template <typename ComputeT> class KernelExample {  
public:
```

```
__aicore__ inline KernelExample() {}

__aicore__ inline void Init(...)
{
    ...
    pipe.InitBuffer(xxxBuf, BUFFER_NUM, xxxSize);
    ...
}

private:
    ...
    TPipe pipe;
    ...
};

extern "C" __global__ __aicore__ void example_kernel(...)
{
    ...
    KernelExample<float> op;
    op.Init(...);
    ...
}
```

【正例】

改为由Kernel入口函数创建TPipe对象，在KernelExample类中保存TPipe指针使用。

```
template <typename ComputeT> class KernelExample {
public:
    __aicore__ inline KernelExample() {}

    __aicore__ inline void Init(..., TPipe* pipeIn)
    {
        ...
        pipe = pipeIn;
        pipe->InitBuffer(xxxBuf, BUFFER_NUM, xxxSize);
        ...
    }

private:
    ...
    TPipe* pipe;
    ...
};

extern "C" __global__ __aicore__ void example_kernel(...)
{
    ...
    TPipe pipe;
    KernelExample<float> op;
    op.Init(..., &pipe);
    ...
}
```

【性能对比】

图 3-83 aiv_scalar_time 优化前后对比

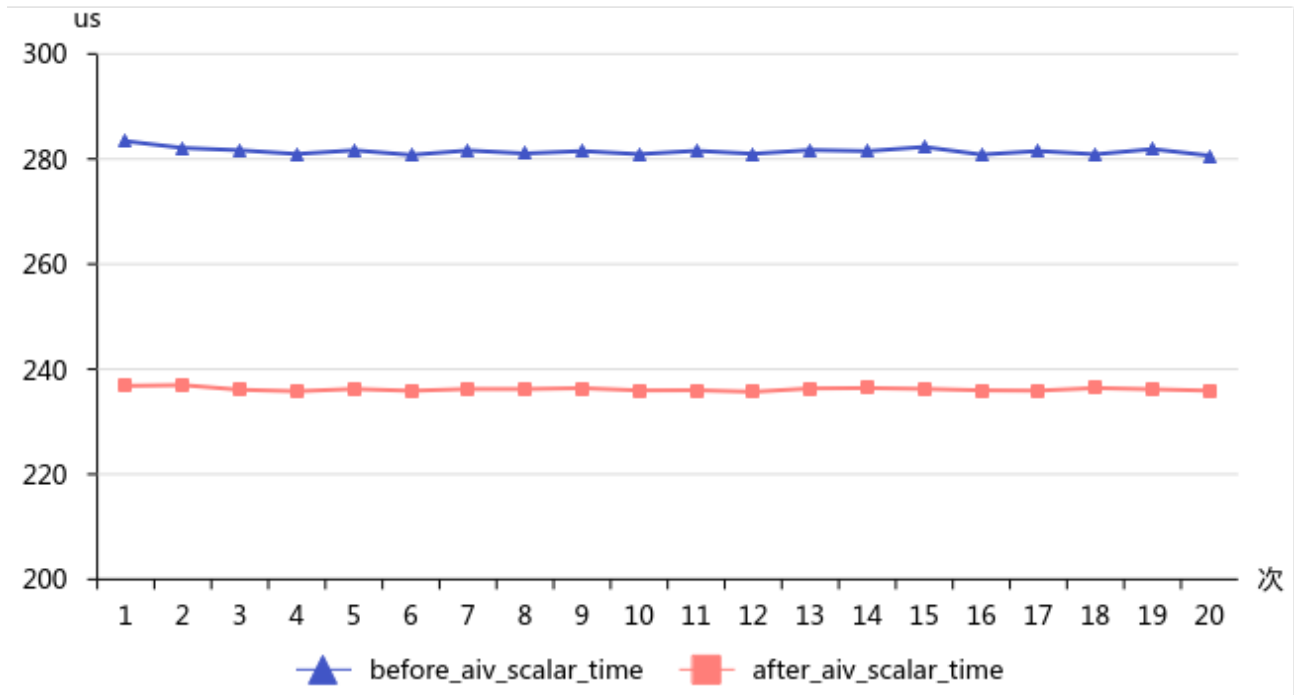
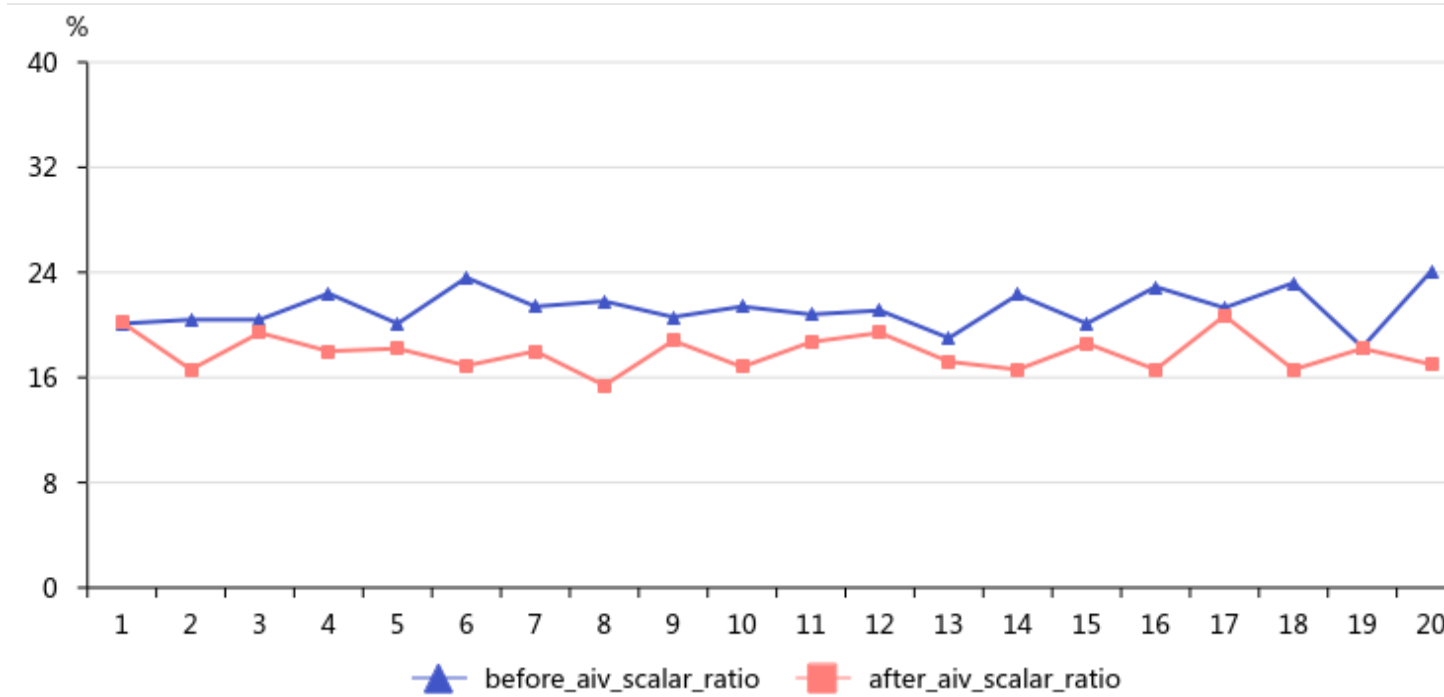


图 3-84 aiv_scalar_ratio 优化前后对比



通过性能数据对比可以看出，Scalar优化效果显著，平均时间从281us减少到236us，下降17%；平均scalar_time时延占比从21%下降到17%。因此在Scalar bound（达到上限）的场景下可以使用此优化措施。

3.6.3.4 核函数内删除 Workspace 相关冗余操作

【优先级】中

【描述】在Ascend C算子工程中，编写核函数时传入的参数workspace已经直接赋值为用户Workspace，因此无需再通过SetSysWorkspace和GetUserWorkspace来设置和获取Workspace。减少这些冗余判断后，编译器可以在不使用该参数的情况下进一步优化未用到的workspace变量。

【反例】

fast_gelu函数的参数workspace等价于用户workspace，且不为空，仍然对workspace进行判空，并且设置SetSysWorkspace和GetUserWorkspace来获取用户Workspace。

```
template <uint64_t schMode, uint64_t dType>
__global__ __aicore__ void fast_gelu(GM_ADDR x, GM_ADDR y, GM_ADDR workspace, GM_ADDR tiling)
{
    // 反例，冗余判断
    if (workspace == nullptr) {
        return;
    }
    SetSysWorkspace(workspace);
    GM_ADDR userWS = GetUserWorkspace(workspace);
    if (userWS == nullptr) {
        return;
    }
    REGISTER_TILING_DEFAULT(EleBaseTilingDataV2);
    GET_TILING_DATA_WITH_STRUCT(EleBaseTilingDataV2, tilingData, tiling);
    KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_AIV_ONLY);
    TPipe pipe;
    if constexpr (dType == static_cast<uint64_t>(TPL_FP16)) {
        ElementwiseSch<schMode, FastGeluDag::FastGeluNeedCast<half>::OpDag> sch(&tilingData, &pipe);
        sch.Init(x, y);
        sch.Process();
    } else if constexpr (dType == static_cast<uint64_t>(TPL_BF16)) {
        ElementwiseSch<schMode, FastGeluDag::FastGeluNeedCast<bfloat16_t>::OpDag> sch(&tilingData,
&pipe);
        sch.Init(x, y);
        sch.Process();
    } else if constexpr (dType == static_cast<uint64_t>(TPL_FP32)) {
        ElementwiseSch<schMode, FastGeluDag::FastGeluNoCast<float>::OpDag> sch(&tilingData, &pipe);
        sch.Init(x, y);
        sch.Process();
    }
}
```

【正例】

fast_gelu函数中删除对workspace参数进行空指针判断，也无需设置SetSysWorkspace和通过GetUserWorkspace来获取Workspace。

```
template <uint64_t schMode, uint64_t dType>
__global__ __aicore__ void fast_gelu(GM_ADDR x, GM_ADDR y, GM_ADDR workspace, GM_ADDR tiling)
{
    REGISTER_TILING_DEFAULT(EleBaseTilingDataV2);
    GET_TILING_DATA_WITH_STRUCT(EleBaseTilingDataV2, tilingData, tiling);
    KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_AIV_ONLY);
    TPipe pipe;
    if constexpr (dType == static_cast<uint64_t>(TPL_FP16)) {
        ElementwiseSch<schMode, FastGeluDag::FastGeluNeedCast<half>::OpDag> sch(&tilingData, &pipe);
        sch.Init(x, y);
        sch.Process();
    } else if constexpr (dType == static_cast<uint64_t>(TPL_BF16)) {
        ElementwiseSch<schMode, FastGeluDag::FastGeluNeedCast<bfloat16_t>::OpDag> sch(&tilingData,
&pipe);
        sch.Init(x, y);
        sch.Process();
    } else if constexpr (dType == static_cast<uint64_t>(TPL_FP32)) {

```

```
ElementwiseSch<schMode, FastGeluDag::FastGeluNoCast<float>::OpDag> sch(&tilingData, &pipe);  
sch.Init(x, y);  
sch.Process();  
}  
}
```

3.6.4 流水编排

3.6.4.1 使能 DoubleBuffer

【优先级】中

【描述】执行于AI Core上的指令队列主要包括如下几类，Vector指令队列（V）、Cube指令队列（M）、Scalar指令队列（S）和搬运指令队列（MTE1/MTE2/MTE3）。不同指令队列间的相互独立性和可并行执行特性，是DoubleBuffer优化机制的基石。

以纯Vector计算为例，矢量计算前后的CopyIn、CopyOut过程使用搬运指令队列（MTE2/MTE3），Compute过程使用Vector指令队列（V），不同指令队列可并行执行，意味着CopyIn、CopyOut过程和Compute过程是可以并行的。如图3-85所示，考虑一个完整的数据搬运和计算过程，CopyIn过程将数据从Global Memory搬运到Local Memory，Vector计算单元完成Compute计算后，经过CopyOut过程将计算结果搬回Global Memory。

图 3-85 数据搬运与 Vector 计算过程

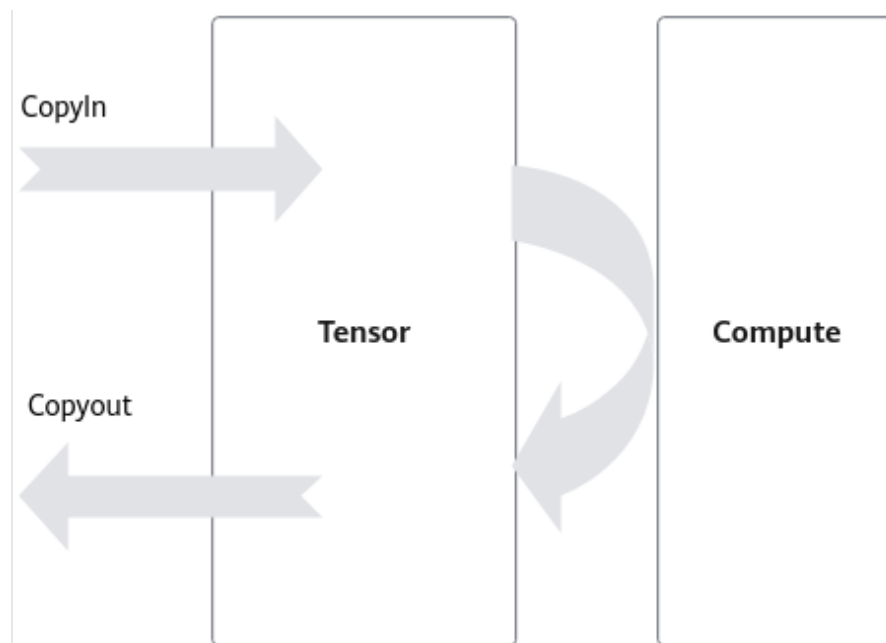
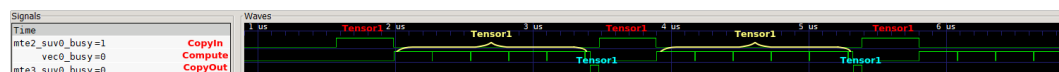


图 3-86 未使能 DoubleBuffer 的流水图



在此过程中，数据搬运与Vector计算串行执行，Vector计算单元不可避免存在资源闲置问题，假设CopyIn、Compute、CopyOut三阶段分别耗时相同均为 t ，则Vector的利用率仅为 $1/3$ ，等待时间过长，Vector利用率严重不足。

为减少Vector等待时间，使能DoubleBuffer机制将待处理的数据一分为二，例如Tensor1、Tensor2。如图3-87所示，当Vector单元对Tensor1中数据进行Compute计算时，Tensor2数据流可以执行CopyIn的过程；而当Vector切换到计算Tensor2时，Tensor1数据流可以执行CopyOut的过程。由此，数据的进出搬运和Vector计算实现并行执行，Vector闲置问题得以有效缓解。

总体来说，DoubleBuffer是基于MTE指令队列与Vector指令队列的独立性和可并行性，通过将数据搬运与Vector计算并行执行以隐藏大部分的数据搬运时间，并降低Vector指令的等待时间，最终提高Vector单元的利用效率。通过为队列申请内存时设置内存块的个数为2，使能DoubleBuffer，实现数据并行，简单代码示例如下：

```
pipe.InitBuffer(inQueueX, 2, 256);
```

图 3-87 DoubleBuffer 机制

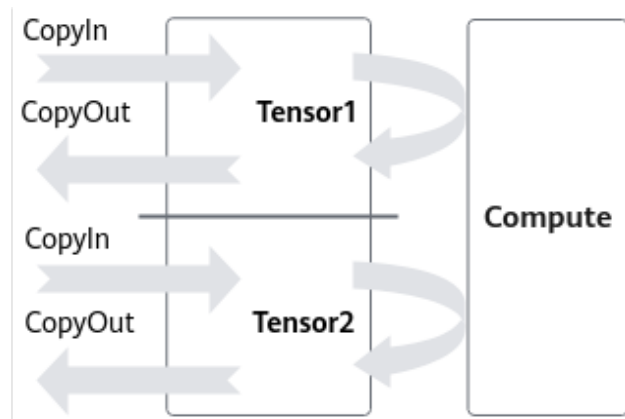
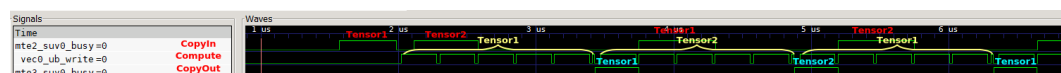


图 3-88 使能 DoubleBuffer 的流水图



需要注意：

多数情况下，采用DoubleBuffer能有效提升Vector的利用率，缩减算子执行时间。然而，DoubleBuffer机制缓解Vector闲置问题，并不代表它总能带来明显的整体性能提升。例如：

- 当数据搬运时间较短，而Vector计算时间较长时，由于数据搬运在整个计算过程中的时间占比较低，DoubleBuffer机制带来的性能收益会偏小。
- 当原始数据较小且Vector可一次性完成所有数据量的计算时，强行使用DoubleBuffer会降低Vector计算资源的利用率，最终效果可能适得其反。

因此，DoubleBuffer的使用需综合考虑Vector算力、数据量大小、搬运与计算时间占比等多种因素。

【反例】

```
_aicore__ inline void Init(__gm__ uint8_t* src0Gm, __gm__ uint8_t* src1Gm, __gm__ uint8_t* dstGm)
{
    src0Global.SetGlobalBuffer((__gm__ half*)src0Gm);
    src1Global.SetGlobalBuffer((__gm__ half*)src1Gm);
    dstGlobal.SetGlobalBuffer((__gm__ half*)dstGm);
    // 不使能DoubleBuffer,占用的物理空间是1 * sizeSrc0 * sizeof(half)
    // 3个InitBuffer执行后总空间为1 * (sizeSrc0 * sizeof(half) + sizeSrc1 * sizeof(half) + sizeDst0 *
    sizeof(half))
```

```
pipe.InitBuffer(inQueueSrc0, 1, sizeSrc0 * sizeof(half));  
pipe.InitBuffer(inQueueSrc1, 1, sizeSrc1 * sizeof(half));  
pipe.InitBuffer(outQueueDst, 1, sizeDst0 * sizeof(half));  
}  
__aicore__ inline void Process()  
{  
    // 需要round*2次循环才能处理完数据  
    for (uint32_t index = 0; index < round * 2; ++index) {  
        CopyIn(index);  
        Compute();  
        CopyOut(index);  
    }  
}
```

【正例】

```
__aicore__ inline void Init(__gm__ uint8_t* src0Gm, __gm__ uint8_t* src1Gm, __gm__ uint8_t* dstGm)  
{  
    src0Global.SetGlobalBuffer((__gm__ half*)src0Gm);  
    src1Global.SetGlobalBuffer((__gm__ half*)src1Gm);  
    dstGlobal.SetGlobalBuffer((__gm__ half*)dstGm);  
    // InitBuffer中使用2表示使能DoubleBuffer,占用的物理空间是2 * sizeSrc0 * sizeof(half)  
    // 3个InitBuffer执行后总空间为2 * (sizeSrc0 * sizeof(half) + sizeSrc1 * sizeof(half) + sizeDst0 *  
    sizeof(half))  
    pipe.InitBuffer(inQueueSrc0, 2, sizeSrc0 * sizeof(half));  
    pipe.InitBuffer(inQueueSrc1, 2, sizeSrc1 * sizeof(half));  
    pipe.InitBuffer(outQueueDst, 2, sizeDst0 * sizeof(half));  
}  
__aicore__ inline void Process()  
{  
    // 开启DoubleBuffer的前提是循环次数 >= 2  
    for (uint32_t index = 0; index < round; ++index) {  
        CopyIn(index);  
        Compute();  
        CopyOut(index);  
    }  
}
```

3.6.4.2 使能 Iterate 或 IterateAll 异步接口避免 AIC/AIV 同步依赖

【优先级】高

【描述】在MIX场景，即AIC（AI Cube核）和AIV（AI Vector核）混合编程中，调用Matmul Iterate或者IterateAll时，AIV发送消息到AIC启动Matmul计算。若通过Iterate<true>同步方式，如图1 同步方式消息发送示意图，每次调用都会触发一次消息发送，而通过Iterate<false>异步方式，如图2 异步方式消息发送示意图，仅第一次需要发送消息，后续无需发送消息，从而减少Cube与Vector核间交互，减少核间通信开销。因此，MIX场景推荐使用Iterate<false>或者IterateAll<false>异步接口（注意：使用异步接口时需要设置Workspace）。

图 3-89 同步方式消息发送示意图

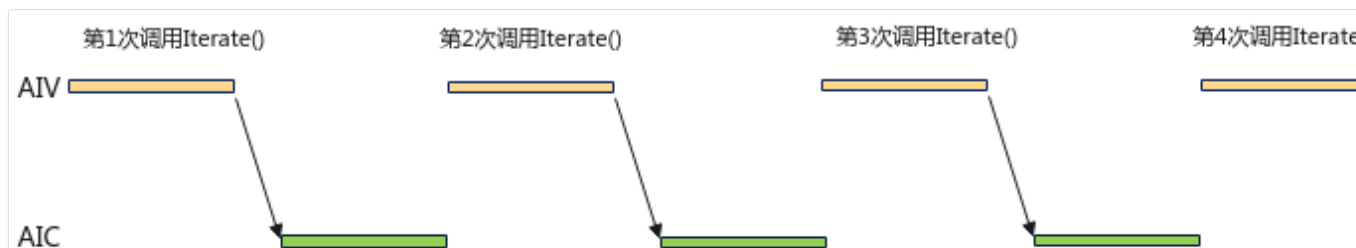
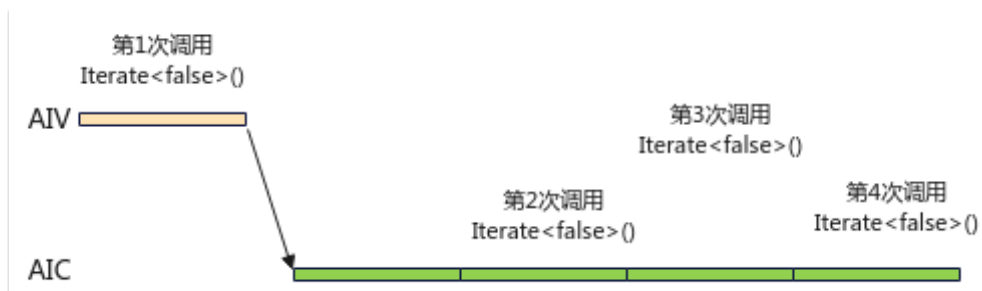


图 3-90 异步方式消息发送示意图



【反例】

MIX场景使用Iterate接口的同步方式。

```
TQueBind<TPosition::CO2, TPosition::VECIN> qVecIn;  
TQueBind<TPosition::VECIN, TPosition::VEECOUT> qVecOut;  
mm.SetTensorA(gmA);  
mm.SetTensorB(gmB);  
int16_t scalar = 2;  
  
while(mm.template Iterate()){  
    auto cInUB = qVecIn.AllocTensor<float>();  
    mm.GetTensorC(cInUB);  
    qVecIn.Enqueue(cInUB);  
    cInUB = qVecIn.DeQueue<float>();  
    auto cOutUB = qVecOut.AllocTensor<float>();  
    Muls(cOutUB, cInUB, scalar, baseM*baseN);  
    qVecIn.FreeTensor(cInUB);  
    ...  
}
```

【正例】

MIX场景使用Iterate接口的异步方式。

```
TQueBind<TPosition::CO2, TPosition::VECIN> qVecIn;  
TQueBind<TPosition::VECIN, TPosition::VEECOUT> qVecOut;  
mm.SetTensorA(gmA);  
mm.SetTensorB(gmB);  
mm.SetWorkspace(workspace, size);//其中，workspace为临时空间的物理地址，size为  
singleCoreM*singleCoreN大小的矩阵C占用的内存大小：singleCoreM*singleCoreN*sizeof(float)  
int16_t scalar = 2;  
  
while(mm.template Iterate<false>()){  
    auto cInUB = qVecIn.AllocTensor<float>();  
    mm.GetTensorC(cInUB);  
    qVecIn.Enqueue(cInUB);  
    cInUB = qVecIn.DeQueue<float>();  
    auto cOutUB = qVecOut.AllocTensor<float>();  
    Muls(cOutUB, cInUB, scalar, baseM*baseN);  
    qVecIn.FreeTensor(cInUB);  
    ...  
}
```

3.6.5 内存访问

3.6.5.1 尽量一次搬运较大的数据块

【优先级】高

【描述】搬运不同大小的数据块时，对带宽的利用率（有效带宽/理论带宽）不一样。根据实测经验，单次搬运数据长度16KB以上时，通常能较好地发挥出带宽的最佳性

能。因此对于单次搬运，应考虑尽可能的搬运较大的数据块。下图展示了某款AI处理器上实测的不同搬运数据量下带宽的变化图。

说明

测试数据与处理器型号相关，且实际测试时可能会存在略微抖动，具体带宽数值并不一定和下文的测试数据严格一致。

图 3-91 UB->GM 方向不同单次搬运数据量下实际占用带宽的变化

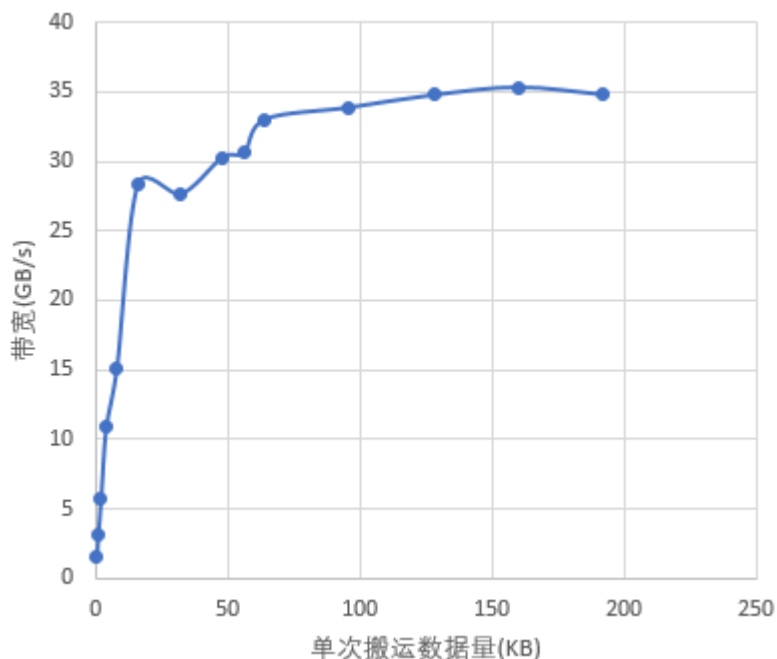
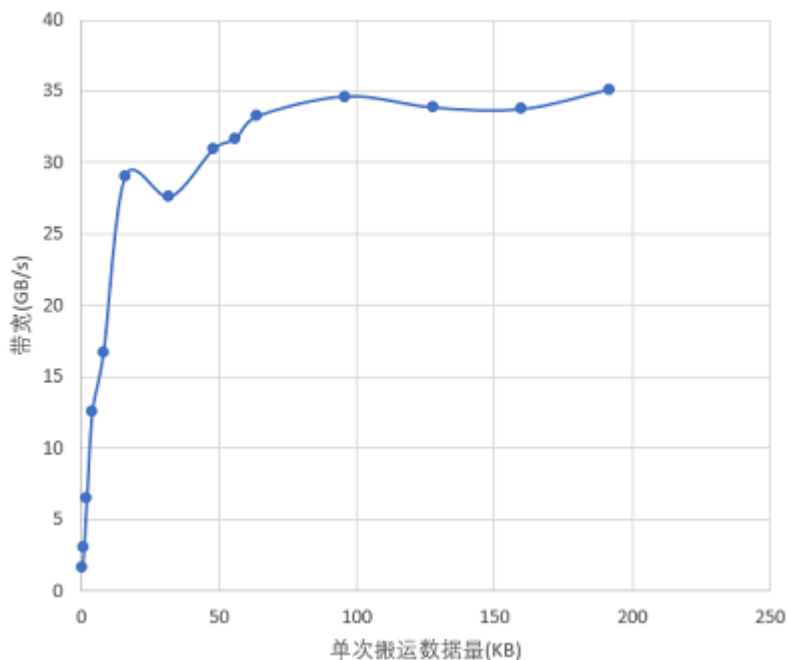


图 3-92 GM->UB 方向不同单次搬运数据量下实际占用带宽的变化



3.6.5.2 GM 地址尽量 512B 对齐

【优先级】高

【描述】由于AI处理器内部设计约束，从GM向Local Memory搬运数据时，保证GM地址512B对齐可以最有效的发挥出带宽的效率。如下图示例，展示了在512B对齐以及32B对齐情况下单核的带宽效率：搬运同等数据量，带宽差距最大的情况，32B对齐场景只能达到512B对齐场景的70%。

说明

- 本性能优化手段仅针对Atlas A2 训练系列产品/Atlas A2 推理系列产品生效。
- 测试数据与处理器型号相关，且实际测试时可能会存在略微抖动，具体带宽数值并不一定和下文的测试数据严格一致。

图 3-93 GM->UB 方向 512B 对齐和 32B 对齐实测带宽的差异对比

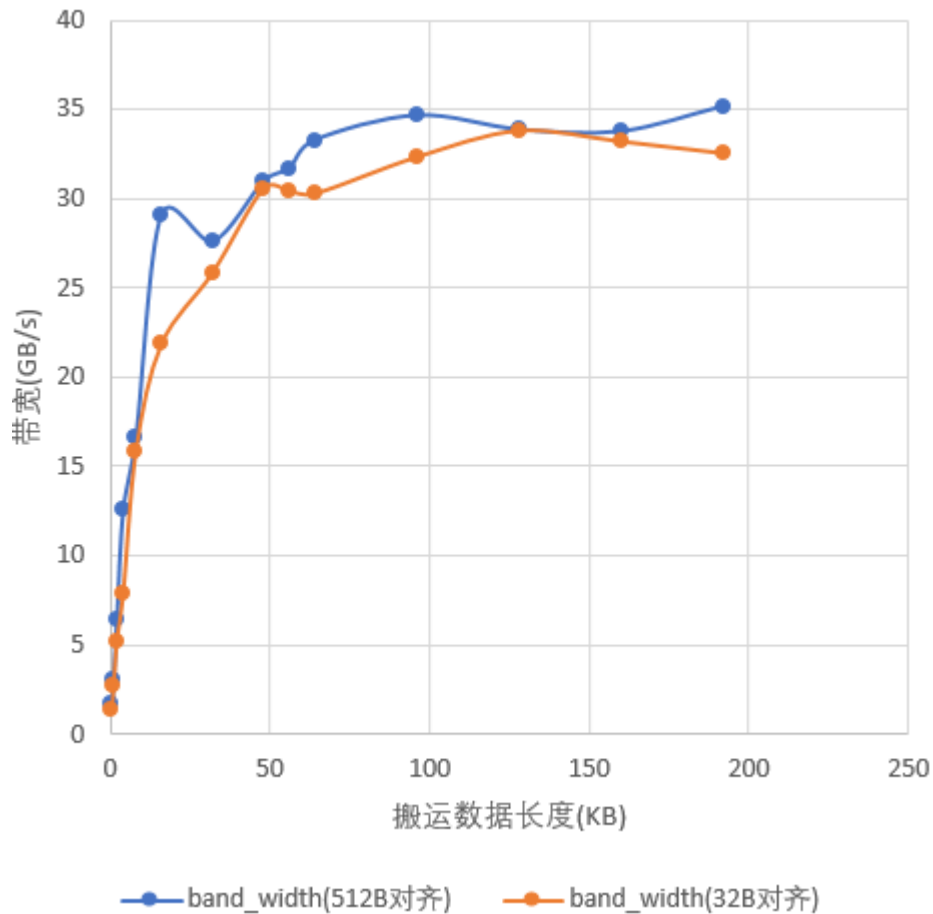
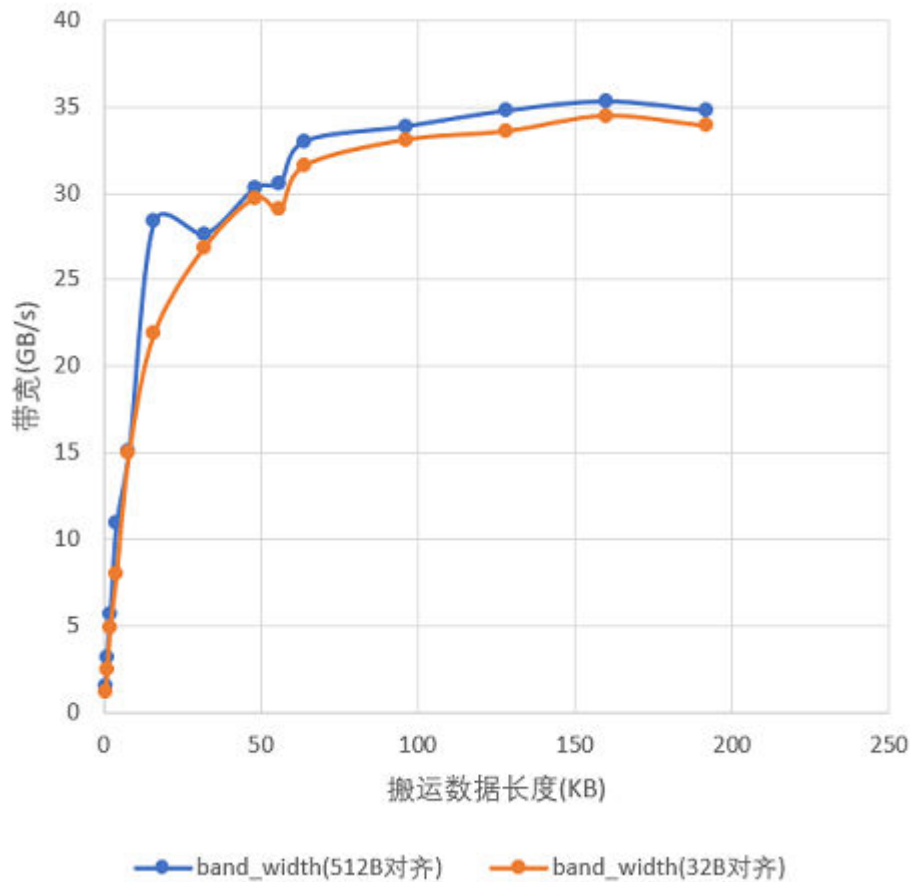


图 3-94 UB->GM 方向 512B 对齐和 32B 对齐实测带宽的差异对比

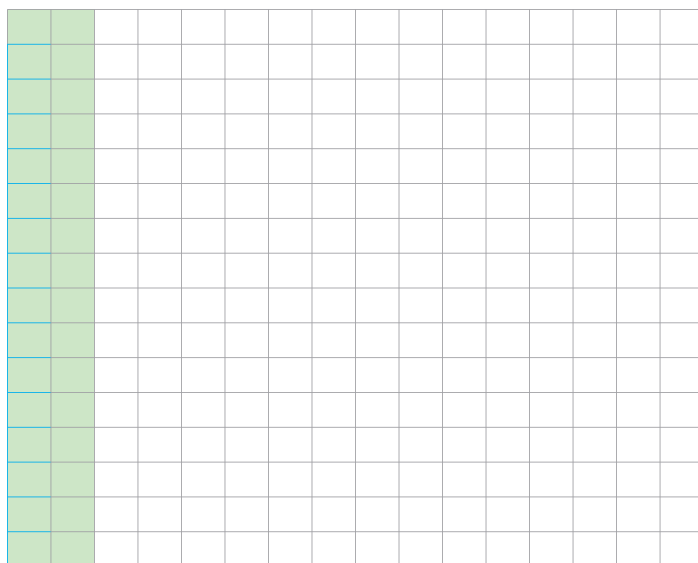


3.6.5.3 高效的使用搬运 API

【优先级】高

【描述】在使用搬运API时，应该尽可能地通过配置搬运控制参数实现连续搬运或者固定间隔搬运，避免使用for循环，二者效率差距极大。如下图示例，图片的每一行为16KB，需要从每一行中搬运前2KB，针对这种场景，使用for循环遍历每行，每次仅能搬运2KB。若直接配置DataCopyParams参数（包含srcStride/dstStride/blockLen/blockCount），则可以达到一次搬完的效果，每次搬运32KB；参考[3.6.5.1 尽量一次搬运较大的数据块](#)章节介绍的搬运数据量和实际带宽的关系，建议一次搬完。

图 3-95 待搬运数据排布

**【反例】**

// 搬运数据存在间隔，从GM上每行16KB中搬运2KB数据，共16行

```
LocalTensor<float> tensorIn;
```

```
GlobalTensor<float> tensorGM;
```

```
...
```

```
constexpr int32_t copyWidth = 2 * 1024 / sizeof(float);
```

```
constexpr int32_t imgWidth = 16 * 1024 / sizeof(float);
```

```
constexpr int32_t imgHeight = 16;
```

```
// 使用for循环，每次只能搬运2K，重复16次
```

```
for (int i = 0; i < imgHeight; i++) {
```

```
    DataCopy(tensorIn[i * copyWidth], tensorGM[i * imgWidth], copyWidth);
```

```
}
```

【正例】

```
LocalTensor<float> tensorIn;
```

```
GlobalTensor<float> tensorGM;
```

```
...
```

```
constexpr int32_t copyWidth = 2 * 1024 / sizeof(float);
```

```
constexpr int32_t imgWidth = 16 * 1024 / sizeof(float);
```

```
constexpr int32_t imgHeight = 16;
```

```
// 通过DataCopy包含DataCopyParams的接口一次搬完
```

```
DataCopyParams copyParams;
```

```
copyParams.blockCount = imgHeight;
```

```
copyParams.blockLen = copyWidth / 8; // 搬运的单位为DataBlock(32Byte)，每个DataBlock内有8个float
```

```
copyParams.srcStride = (imgWidth - copyWidth) / 8; // 表示两次搬运src之间的间隔，单位为DataBlock
```

```
copyParams.dstStride = 0; // 连续写，两次搬运之间dst的间隔为0，单位为DataBlock
```

```
DataCopy(tensorGM, tensorIn, copyParams);
```

3.6.5.4 避免同地址访问

【优先级】高**📖 说明**

该性能优化指导适用于如下产品型号：

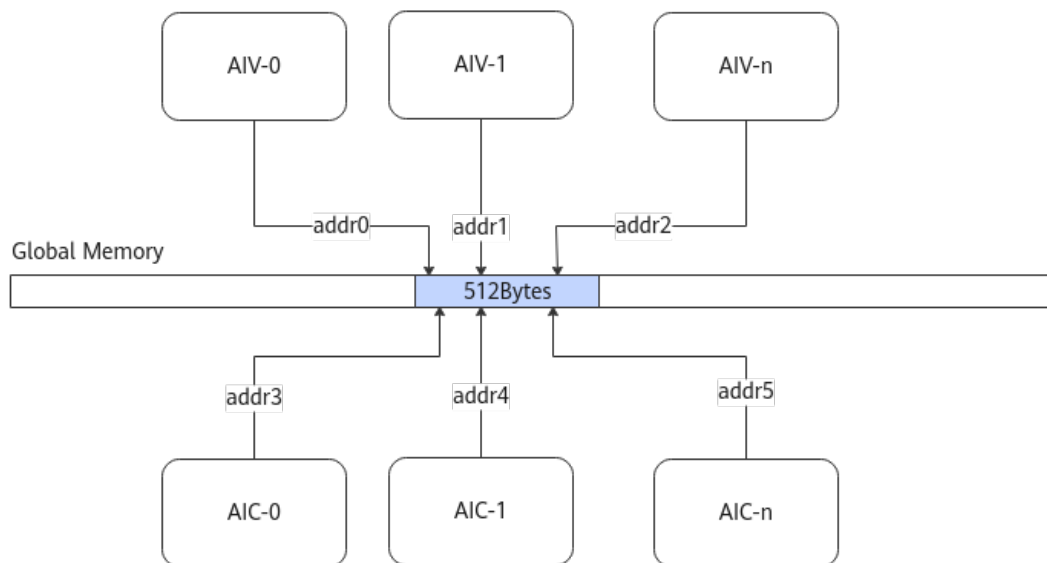
- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品

【描述】 MTE2、MTE3、Scalar等单元访问Global Memory数据时，其地址请求会按照512字节粒度对齐后进行处理。当同时访问Global Memory的数据，且地址处于连续

的512字节范围内时，由于数据一致性的原因，多个请求会被串行处理，进而影响数据搬运效率。

当前算子执行机制保证用户Kernel入参（包括Workspace/Tiling）的地址512字节对齐，因此开发者只需要根据地址的偏移量即可判断两个地址是否会落入连续的512字节范围内。

如下图所示，AI Core内的各个核对Global Memory的数据同时发出读写请求，尽管addr0~addr5是多个不同的地址，但因为落在连续的512字节范围内，被视为同一个地址请求，此时这几个数据请求会被串行处理，数据访问效率会降低。同地址访问的影响受同时访问的核数影响，同地址访问的核数越多时，串行导致的性能劣化越严重。



避免同地址访问的方法主要有两种：**调整数据访问顺序**和**修改切分策略**。下文介绍配套的样例请参考[避免同地址访问样例](#)。

调整数据访问顺序

以一个形状为 (8192, 128) 的float类型输入进行Adds计算为例。

为了体现同地址冲突的影响，上述场景设计中每一行的数据大小为512字节（128个float），每个核每一轮计算处理512 * 8字节的数据，并进行全核同步（实际场景中并不需要），每一轮计算都需要等待所有核完成当前数据块的计算后，再进行下一轮。

实现方案	原始实现	优化实现
实现方法	使用16个核参与计算，按列方向进行切分，每个核总计算数据量为8192 * 8；单核执行循环16次，每次计算的数据量为512 * 8；每个核的循环顺序如下图所示，列方向0~15表示每个核的数据块执行顺序。 由于多个核同时访问同一行数据（512字节），导致同地址冲突的发生。	使用16个核参与计算，按列方向进行切分，每个核总计算数据量为8192 * 8；单核执行循环16次，每次计算的数据量为512 * 8；每个核的循环顺序如下图所示，列方向0~15表示每个核的数据块执行顺序。 由于每个核每一轮处理的地址在不同行，不会同时访问连续的512字节，所以不会导致同地址访问冲突。

实现方案	原始实现																优化实现																
示意图		0核	1核	2核	3核	4核	5核	6核	7核	8核	9核	10核	11核	0核	1核	2核	3核	4核	5核	6核	7核	8核	9核	10核	11核								
	0x0000-0x03FFF	0	0	0	0	0	0	0	0	0	0	0	0	0	1	2	3	4	5	6	7	8	9	10	11								
	0x0400-0x07FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	2	3	4	5	6	7	8	9	10	11	12								
	0x0800-0x0BFFF	2	2	2	2	2	2	2	2	2	2	2	2	2	3	4	5	6	7	8	9	10	11	12	13								
	0x0C00-0x0FFFF	3	3	3	3	3	3	3	3	3	3	3	3	3	4	5	6	7	8	9	10	11	12	13	14								
	0x1000-0x13FFF	4	4	4	4	4	4	4	4	4	4	4	4	4	5	6	7	8	9	10	11	12	13	14	15								
	0x1400-0x17FFF	5	5	5	5	5	5	5	5	5	5	5	5	5	6	7	8	9	10	11	12	13	14	15	0								
	0x1800-0x1BFFF	6	6	6	6	6	6	6	6	6	6	6	6	6	7	8	9	10	11	12	13	14	15	0	1								
	0x1C00-0x1FFFF	7	7	7	7	7	7	7	7	7	7	7	7	7	8	9	10	11	12	13	14	15	0	1	2								
	0x2000-0x23FFF	8	8	8	8	8	8	8	8	8	8	8	8	8	9	10	11	12	13	14	15	0	1	2	3								
	0x2400-0x27FFF	9	9	9	9	9	9	9	9	9	9	9	9	9	10	11	12	13	14	15	0	1	2	3	4								
	0x2800-0x2BFFF	10	10	10	10	10	10	10	10	10	10	10	10	10	11	12	13	14	15	0	1	2	3	4	5								
	0x2C00-0x2FFFF	11	11	11	11	11	11	11	11	11	11	11	11	11	12	13	14	15	0	1	2	3	4	5	6								
	0x3000-0x33FFF	12	12	12	12	12	12	12	12	12	12	12	12	12	13	14	15	0	1	2	3	4	5	6	7								
	0x3400-0x37FFF	13	13	13	13	13	13	13	13	13	13	13	13	13	14	15	0	1	2	3	4	5	6	7	8								
	0x3800-0x3BFFF	14	14	14	14	14	14	14	14	14	14	14	14	14	15	0	1	2	3	4	5	6	7	8	9								
	0x3C00-0x3FFFF	15	15	15	15	15	15	15	15	15	15	15	15	15	0	1	2	3	4	5	6	7	8	9	10								
		128								128								128															
示例代码	<pre>for (int32_t i = 0; i < tiling->loopOneCore; i++) { AscendC::SyncAll(); CopyIn(i); Compute(); AscendC::SyncAll(); CopyOut(i); }</pre>																<pre>for (int32_t i = 0; i < tiling->loopOneCore; i++) { int32_t newProgress = (i + AscendC::GetBlockIdx()) % tiling- >loopOneCore; AscendC::SyncAll(); CopyIn(newProgress); Compute(); AscendC::SyncAll(); CopyOut(newProgress); }</pre>																

修改切分策略

仍以 一个形状为 (8192, 128) 的float类型输入进行Adds计算为例。

为了体现同地址冲突的影响，上述场景设计中每一行的数据大小为512字节（128个float），每个核每一轮计算处理512 * 8字节的数据，并进行全核同步（实际场景中并不需要），每一轮计算都需要等待所有核完成当前数据块的计算后，再进行下一轮。

实现方案	原始实现	优化实现																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
实现方法	使用16个核参与计算，按列方向进行切分，每个核总计算数据量为8192 * 8；单核执行循环16次，每次计算的数据量为512 * 8；每个核的循环顺序如下图所示，列方向0~15表示每个核的数据块执行顺序。 由于多个核同时访问同一行数据（512字节），导致同地址冲突的发生。	使用16个核参与计算，按行方向进行切分，每个核总计算数据量为512 * 128；单核执行循环16次，每次计算的数据量为512 * 8；每个核的循环顺序如下图所示（行方向），均为从块0~块15。 由于每个核每一轮处理的地址在不同行，不会同时访问连续的512字节，所以不会导致同地址访问冲突。																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
示意图	<table><tr><th></th><th>0核</th><th>1核</th><th>2核</th><th>3核</th><th>4核</th><th>5核</th><th>6核</th><th>7核</th><th>8核</th><th>9核</th><th>10核</th><th>11核</th><th>12核</th><th>13核</th><th>14核</th><th>15核</th></tr><tr><td>0x0000-0x03FFF</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0x04000-0x07FFF</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>0x08000-0x0BFFF</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td></tr><tr><td>0x0C000-0x0FFFF</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td></tr><tr><td>0x10000-0x13FFF</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td></tr><tr><td>0x14000-0x17FFF</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td></tr><tr><td>0x18000-0x1BFFF</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td></tr><tr><td>0x1C000-0x1FFFF</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>0x20000-0x23FFF</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td><td>8</td></tr><tr><td>0x24000-0x27FFF</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td><td>9</td></tr><tr><td>0x28000-0x2BFFF</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td><td>10</td></tr><tr><td>0x2C000-0x2FFFF</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td><td>11</td></tr><tr><td>0x30000-0x33FFF</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td><td>12</td></tr><tr><td>0x34000-0x37FFF</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td><td>13</td></tr><tr><td>0x38000-0x3BFFF</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td></tr><tr><td>0x3C000-0x3FFFF</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td><td>15</td></tr></table> 128 128		0核	1核	2核	3核	4核	5核	6核	7核	8核	9核	10核	11核	12核	13核	14核	15核	0x0000-0x03FFF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0x04000-0x07FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0x08000-0x0BFFF	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	0x0C000-0x0FFFF	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	0x10000-0x13FFF	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	0x14000-0x17FFF	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	0x18000-0x1BFFF	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	0x1C000-0x1FFFF	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	0x20000-0x23FFF	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	0x24000-0x27FFF	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	0x28000-0x2BFFF	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	0x2C000-0x2FFFF	11	11	11	11	11	11	11	11	11	11	11	11	11	11	11	11	0x30000-0x33FFF	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	0x34000-0x37FFF	13	13	13	13	13	13	13	13	13	13	13	13	13	13	13	13	0x38000-0x3BFFF	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	0x3C000-0x3FFFF	15	15	15	15	15	15	15	15	15	15	15	15	15	15	15	15	<table><tr><th></th><th>0核</th><th>1核</th><th>2核</th><th>3核</th><th>4核</th><th>5核</th><th>6核</th><th>7核</th><th>8核</th><th>9核</th><th>10核</th><th>11核</th><th>12核</th><th>13核</th><th>14核</th><th>15核</th></tr><tr><td>0x0000-0x03FFF</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>0x04000-0x07FFF</td><td>1</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x08000-0x0BFFF</td><td>2</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x0C000-0x0FFFF</td><td>3</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x10000-0x13FFF</td><td>4</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x14000-0x17FFF</td><td>5</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x18000-0x1BFFF</td><td>6</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x1C000-0x1FFFF</td><td>7</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x20000-0x23FFF</td><td>8</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x24000-0x27FFF</td><td>9</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x28000-0x2BFFF</td><td>10</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x2C000-0x2FFFF</td><td>11</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x30000-0x33FFF</td><td>12</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x34000-0x37FFF</td><td>13</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x38000-0x3BFFF</td><td>14</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr><tr><td>0x3C000-0x3FFFF</td><td>15</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td></tr></table> 8 128		0核	1核	2核	3核	4核	5核	6核	7核	8核	9核	10核	11核	12核	13核	14核	15核	0x0000-0x03FFF	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	0x04000-0x07FFF	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x08000-0x0BFFF	2	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x0C000-0x0FFFF	3	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x10000-0x13FFF	4	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x14000-0x17FFF	5	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x18000-0x1BFFF	6	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x1C000-0x1FFFF	7	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x20000-0x23FFF	8	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x24000-0x27FFF	9	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x28000-0x2BFFF	10	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x2C000-0x2FFFF	11	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x30000-0x33FFF	12	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x34000-0x37FFF	13	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x38000-0x3BFFF	14	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0x3C000-0x3FFFF	15	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	0核	1核	2核	3核	4核	5核	6核	7核	8核	9核	10核	11核	12核	13核	14核	15核																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x0000-0x03FFF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x04000-0x07FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x08000-0x0BFFF	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x0C000-0x0FFFF	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x10000-0x13FFF	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x14000-0x17FFF	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x18000-0x1BFFF	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x1C000-0x1FFFF	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x20000-0x23FFF	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x24000-0x27FFF	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x28000-0x2BFFF	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x2C000-0x2FFFF	11	11	11	11	11	11	11	11	11	11	11	11	11	11	11	11																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x30000-0x33FFF	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x34000-0x37FFF	13	13	13	13	13	13	13	13	13	13	13	13	13	13	13	13																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x38000-0x3BFFF	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x3C000-0x3FFFF	15	15	15	15	15	15	15	15	15	15	15	15	15	15	15	15																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
	0核	1核	2核	3核	4核	5核	6核	7核	8核	9核	10核	11核	12核	13核	14核	15核																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x0000-0x03FFF	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x04000-0x07FFF	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x08000-0x0BFFF	2	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x0C000-0x0FFFF	3	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x10000-0x13FFF	4	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x14000-0x17FFF	5	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x18000-0x1BFFF	6	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x1C000-0x1FFFF	7	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x20000-0x23FFF	8	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x24000-0x27FFF	9	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x28000-0x2BFFF	10	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x2C000-0x2FFFF	11	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x30000-0x33FFF	12	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x34000-0x37FFF	13	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x38000-0x3BFFF	14	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
0x3C000-0x3FFFF	15	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				

实现方案	原始实现	优化实现
示例代码	<pre>__aicore__ inline void Init(GM_ADDR x, GM_ADDR z, AddsCustomTilingData* tilingPtr) { tiling = tilingPtr; xGm.SetGlobalBuffer((__gm__ float *)x + AscendC::GetBlockIdx() * tiling->tileN); zGm.SetGlobalBuffer((__gm__ float *)z + AscendC::GetBlockIdx() * tiling->tileN); // we disable the L2 cache mode to highlight the influence of the gm address conflict xGm.SetL2CacheHint(AscendC::CacheMode::CA CHE_MODE_DISABLE); zGm.SetL2CacheHint(AscendC::CacheMode::CA CHE_MODE_DISABLE); pipe.InitBuffer(inQueueX, BUFFER_NUM, tiling->tileM * tiling->tileN * sizeof(float)); pipe.InitBuffer(outQueueZ, BUFFER_NUM, tiling->tileM * tiling->tileN * sizeof(float)); }</pre>	<pre>__aicore__ inline void Init(GM_ADDR x, GM_ADDR z, AddsCustomTilingData* tilingPtr) { tiling = tilingPtr; // change the tile method from column split to row split xGm.SetGlobalBuffer((__gm__ float *)x + AscendC::GetBlockIdx() * tiling->tileM * tiling- >n); zGm.SetGlobalBuffer((__gm__ float *)z + AscendC::GetBlockIdx() * tiling->tileM * tiling- >n); // we disable the L2 cache mode to highlight the influence of the gm address conflict xGm.SetL2CacheHint(AscendC::CacheMode::CAC HE_MODE_DISABLE); zGm.SetL2CacheHint(AscendC::CacheMode::CAC HE_MODE_DISABLE); pipe.InitBuffer(inQueueX, BUFFER_NUM, tiling->tileM * tiling->tileN * sizeof(float)); pipe.InitBuffer(outQueueZ, BUFFER_NUM, tiling->tileM * tiling->tileN * sizeof(float)); }</pre>

📖 说明

你可以通过执行如下命令行，通过msprof工具获取上述示例的性能数据并进行对比。

```
msprof op --launch-count=3 --output=./prof ./execute_adds_op
```

重点关注PipeUtilization.csv中的aiv_mte2_time(us)和aiv_mte3_time(us)搬运指令耗时。

3.6.5.5 设置合理的 L2 CacheMode

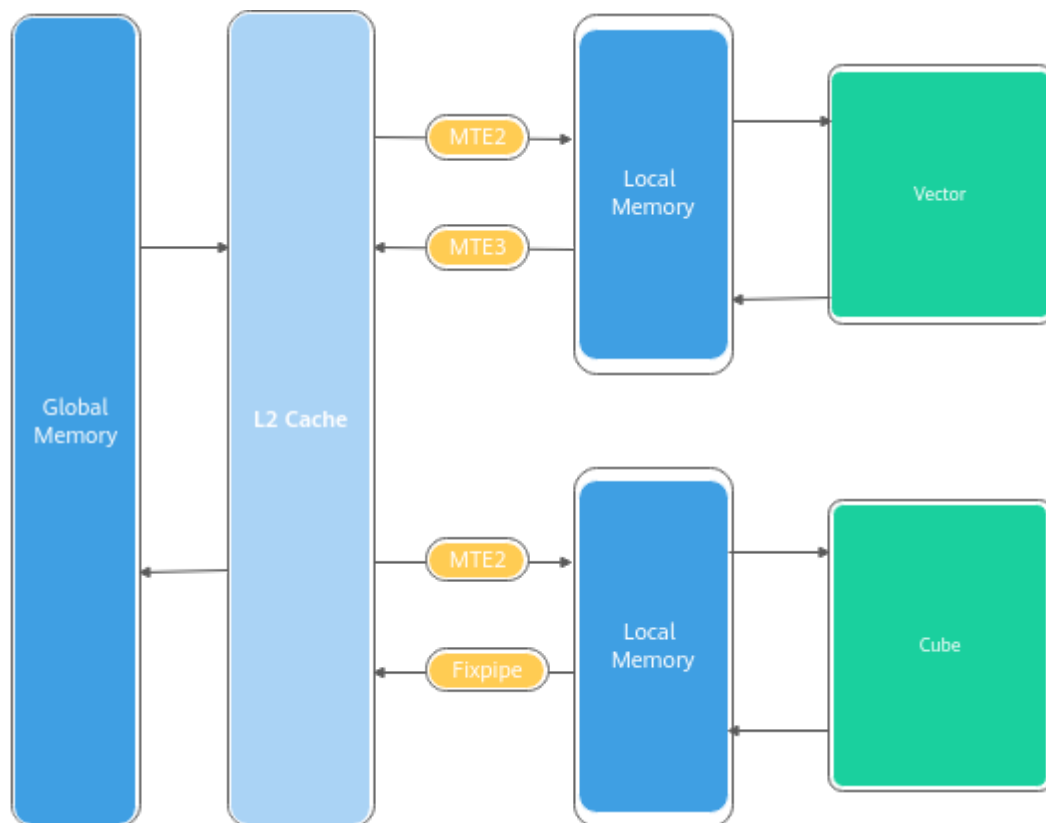
【优先级】高

📖 说明

该性能优化指导适用于如下产品型号：

- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品

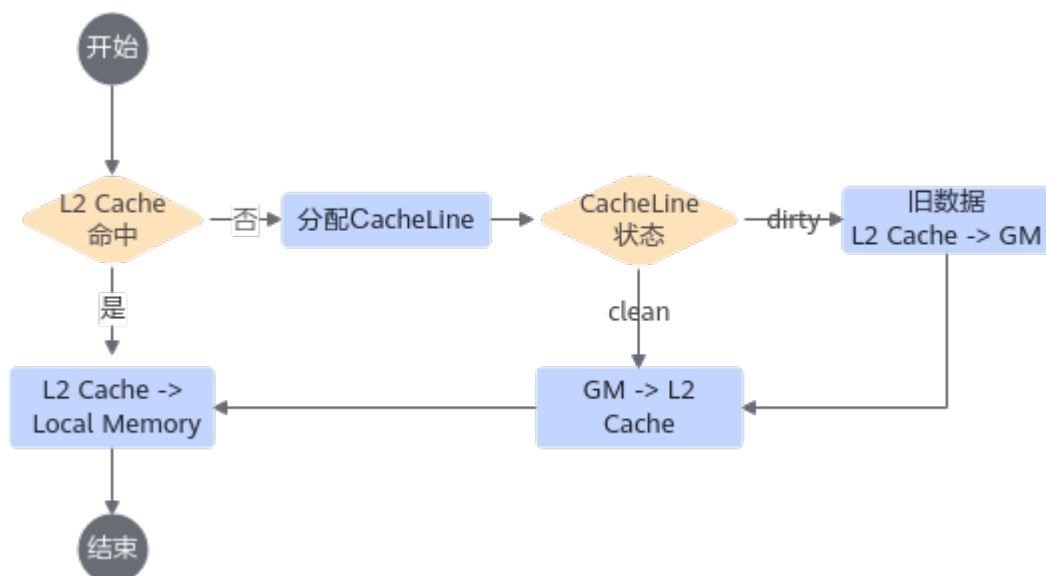
【描述】L2 Cache常用于缓存频繁访问的数据，其物理位置如下图所示：



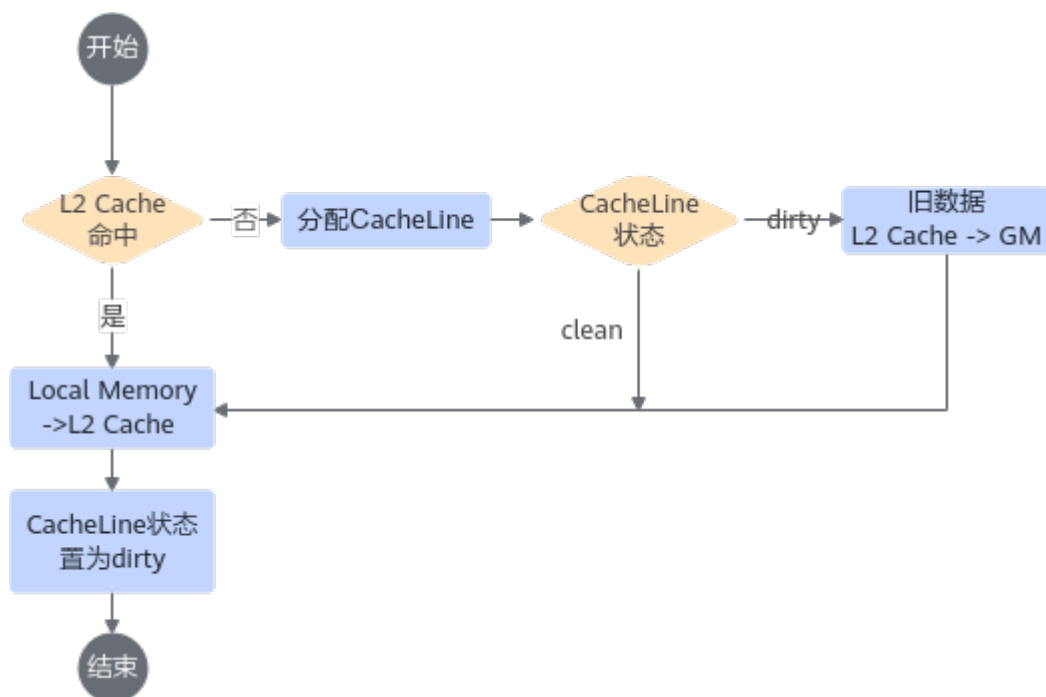
L2 Cache的带宽相比GM的带宽有数倍的提升，因此当数据命中L2 Cache时，数据的搬运耗时会优化数倍。通常情况下，L2 Cache命中率越高，算子的性能越好，在实际访问中需要通过设置合理的L2 CacheMode来保证重复读取的数据尽量缓存在L2 Cache上。

L2 Cache 访问的原理及 CacheMode 介绍

数据通过MTE2搬运单元搬入时，L2 Cache访问的典型流程如下：



数据通过MTE3或者Fixpipe搬运单元搬出时，L2 Cache访问的典型流程如下：



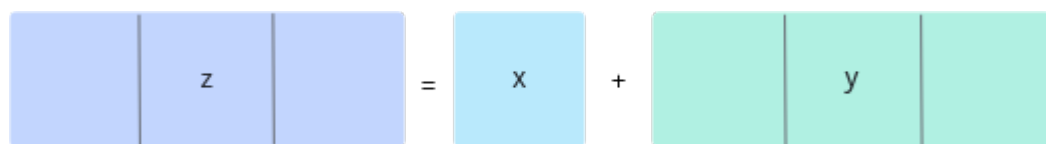
从上面的流程可以看出，当数据访问总量超出L2 Cache容量时，AI Core会对L2 Cache进行数据替换。由于Cache一致性的要求，替换过程中旧数据需要先写回GM（此过程中会占用GM带宽），旧数据写回后，新的数据才能进入L2 Cache。

开发者可以针对访问的数据设置其CacheMode，对于只访问一次的Global Memory数据设置其访问状态为不进入L2 Cache，这样可以更加高效的利用L2 Cache缓存需要重复读取的数据，避免一次性访问的数据替换有效数据。

设置 L2 CacheMode 的方法

Ascend C基于GlobalTensor提供了SetL2CacheHint接口，用户可以根据需要指定CacheMode。

考虑如下场景，构造两个Tensor的计算，x的输入Shape为(5120, 5120)，y的输入Shape为(5120, 15360)，z的输出Shape为(5120, 15360)，由于两个Tensor的Shape不相等，x分别与y的3个数据块依次相加。该方案主要为了演示CacheMode的功能，示例代码中故意使用重复搬运x的实现方式，真实设计中并不需要采用这个方案。下文完整样例请参考[设置合理L2 CacheMode样例](#)。



实现方案	原始实现	优化实现
实现方法	总数据量700MB，其中：x：100MB；y：300MB；z：300MB。 使用40个核参与计算，按列方向切分。 x、y、z 对应GlobalTensor的CacheMode均设置为CACHE_MODE_NORMAL，需要经过L2 Cache，需要进入L2 Cache的总数据量为700MB。	总数据量700MB，其中：x：100MB；y：300MB；z：300MB。 使用40个核参与计算，按列方向切分。 x对应的GlobalTensor的CacheMode设置为CACHE_MODE_NORMAL；y和z对应的GlobalTensor的CacheMode设置为CACHE_MODE_DISABLE。只有需要频繁访问的x，设置为需要经过L2 Cache。需要进入L2 Cache的总数据量为100MB。
示例代码	<pre>xGm.SetGlobalBuffer((__gm__ float *)x + AscendC::GetBlockIdx() * TILE_N); yGm.SetGlobalBuffer((__gm__ float *)y + AscendC::GetBlockIdx() * TILE_N); zGm.SetGlobalBuffer((__gm__ float *)z + AscendC::GetBlockIdx() * TILE_N);</pre>	<pre>xGm.SetGlobalBuffer((__gm__ float *)x + AscendC::GetBlockIdx() * TILE_N); yGm.SetGlobalBuffer((__gm__ float *)y + AscendC::GetBlockIdx() * TILE_N); zGm.SetGlobalBuffer((__gm__ float *)z + AscendC::GetBlockIdx() * TILE_N); // disable the L2 cache mode of y and z yGm.SetL2CacheHint(AscendC::CacheMode::CACHE_ MODE_DISABLE); zGm.SetL2CacheHint(AscendC::CacheMode::CACHE_ MODE_DISABLE);</pre>

📖 说明

你可以通过执行如下命令行，通过msprof工具获取上述示例的性能数据并进行对比。

```
msprof op --launch-count=2 --output=./prof ./execute_add_op
```

重点关注Memory.csv中的aiv_gm_to_ub_bw(GB/s)和aiv_main_mem_write_bw(GB/s)写带宽的速率。

3.6.5.6 算子与高阶 API 共享临时 Buffer

【优先级】高

【描述】如果算子使用的高阶API需要传入临时Buffer，如SoftMax，该临时空间会挤占算子其他计算的空间，从而导致单次计算搬运的数据量变少，搬运的次数变多。此场景可通过共享临时Buffer空间，提升单次搬运的数据量，减少搬运的次数，提升内存使用效率。

【反例】

SoftMax高阶API计算需要临时Buffer空间，算子在进行其他计算时拥有独立临时Buffer。UB空间是固定的，假设可以给SoftMax和Add能分配临时空间为64KB，SoftMax计算需要的临时Buffer空间tmpSoftmaxBuf占用32KB，则存储Add计算结果的LocalTensor tmpSumBuf最多只能分配32KB。如果src0Tensor计算的数据量是512KB，则需要搬运 $512 / 32 = 16$ 次。

```
...  
constexpr int32_t blockLen = 32 * 1024;  
TBuf<TPosition::VECCALC> tmpSoftmaxBuf;  
pipe.InitBuffer(tmpSoftmaxBuf, softmaxBufSize * sizeof(uint8_t)); // 单独分配Softmax的临时Buf 32KB  
TBuf<TPosition::VECCALC> tmpSumBuf;
```

```
pipe.InitBuffer(tmpSumBuf, sumBufSize * sizeof(T)); // 单独分配Add的临时Buf, 且softmaxBufSize *
sizeof(uint8_t) + sumBufSize * sizeof(T) <= 64KB
...
for (int i = 0; i < 16; i++) {
    ...
    LocalTensor<uint8_t> tmpSoftmaxTensor = tmpSoftmaxBuf.Get<uint8_t>(softmaxBufSize);
    SoftMax<T, true, true>(dstTensor, expSumTensor, dstMaxTensor, srcTensor, tmpSoftmaxTensor, tiling);
    ...
    DataCopy(src0Tensor, src0Gm[i * blockLen / sizeof(T)], Params);
    ...
    LocalTensor<T> tmpSumTensor = tmpSumBuf.Get<T>(sumBufSize);
    Add<T>(tmpSumTensor, src0Tensor, src1Tensor, count);
    ...
}
...
```

【正例】

SoftMax高阶API计算需要临时Buffer空间, 算子在进行其他计算时可以共享此临时Buffer, 按照上述假设只需要搬运 $512 / 64 = 8$ 次。

```
...
constexpr int32_t blockLen = 64 * 1024;
TBuf<TPosition::VECCALC> tmpSharedBuf;
pipe.InitBuffer(tmpSharedBuf, bufferSize); // 共享分配bufferSize = MAX(softmaxBufSize * sizeof(uint8_t),
sumBufSize * sizeof(T)) <= 64KB
...
for (int i = 0; i < 8; i++) {
    ...
    LocalTensor<uint8_t> tmpSharedTensor = tmpSharedBuf.Get<uint8_t>(softmaxBufSize);
    SoftMax<T, true, true>(dstTensor, expSumTensor, dstMaxTensor, srcTensor, tmpSharedTensor, tiling);
    ...
    DataCopy(src0Tensor, src0Gm[i * blockLen / sizeof(T)], Params);
    ...
    LocalTensor<T> tmpSumTensor = tmpSharedBuf.Get<T>(sumBufSize);
    Add<T>(tmpSumTensor, src0Tensor, src1Tensor, count);
    ...
}
...
```

3.6.5.7 纯搬运类算子 VECIN 和 VECOUT 建议复用

【优先级】高

【描述】纯搬运类算子在执行时并不涉及实际vector计算, 若存在冗余的vector指令, 会导致算子整体执行时间变长。这种场景可以使用Ascend C针对纯搬运类算子提供的TQueBind接口, 该接口可以将VECIN与VECOUT绑定, 省略将数据从VECIN拷贝到VECOUT的步骤, 从而避免vector的无谓消耗。

【反例】

此段代码为了保证数据搬入和数据搬出之间的流水同步, 存在LocalTensor -> LocalTensor的DataCopy指令。

```
template <typename ComputeT> class KernelExample {
public:
    ...
    __aicore__ inline void Process(...)
    {
        for (int i = 0; i < iLen; ++i) {
            ...
            auto iLocal = QueI.AllocTensor<ComputeT>();
            DataCopy(iLocal, inGm[i * 32], size);
            QueI.Enqueue(iLocal);
            iLocal = QueI.DeQueue<ComputeT>();
            for (int j = 0; j < jLen; ++j) {
                ...
                auto oLocal = QueO.AllocTensor<ComputeT>();
```

```
        DataCopy(oLocal, iLocal, size); // LocalTensor -> LocalTensor的DataCopy指令,以实现数据从VECIN
到VECOUT的搬运
        QueO.Enqueue(oLocal);

        auto oLocal = QueO.DeQueue<ComputeT>();
        DataCopyPad(outGm[j], oLocal, ...);
        QueO.FreeTensor(oLocal);
    }
    QueI.FreeTensor(iLocal);
}
}

private:
...
TQue<TPosition::VECIN, BUFFER_NUM> QueI;
TQue<TPosition::VECOUT, BUFFER_NUM> QueO;
...
};

extern "C" __global__ __aicore__ void example_kernel(...)
{
    ...
    op.Process(...);
}
```

【正例】

将LocalTensor -> LocalTensor的DataCopy指令替换为TQueBind接口，减少将VECIN拷贝到VECOUT的步骤，从而避免了冗余拷贝。

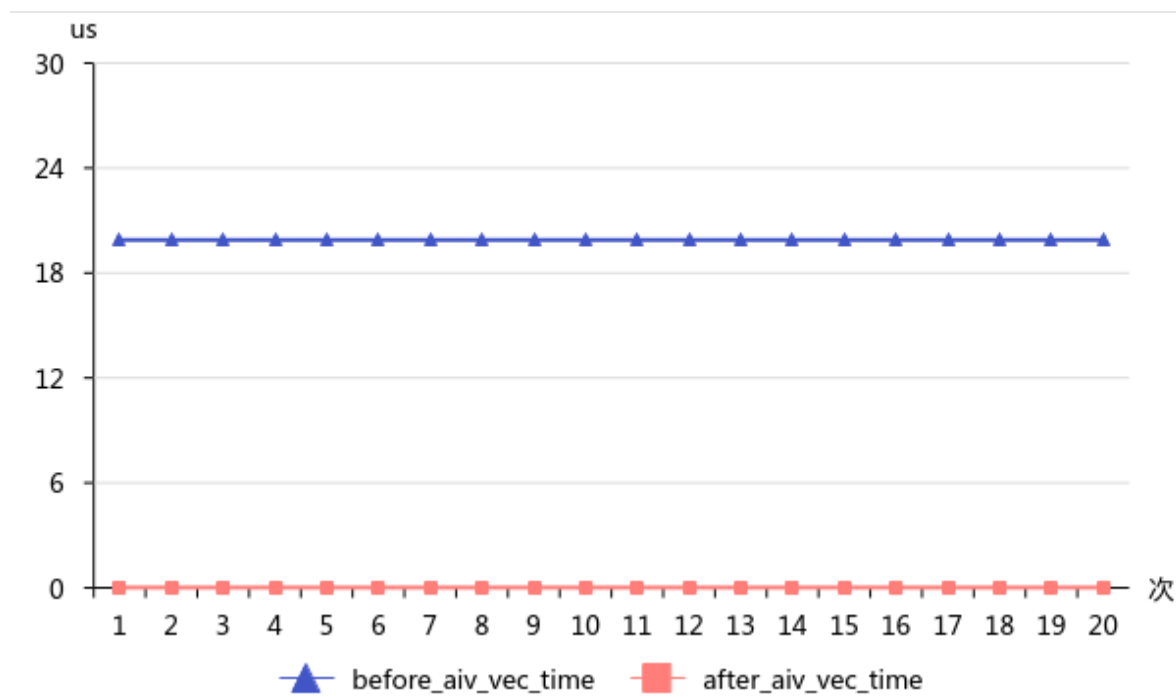
```
template <typename ComputeT> class KernelExample {
public:
    ...
    __aicore__ inline void Process(...)
    {
        for (int i = 0; i < iLen; ++i) {
            ...
            auto bindLocal = queBind.AllocTensor<ComputeT>();
            DataCopy(bindLocal, inGm[i * 32], size);
            queBind.Enqueue(bindLocal);
            bindLocal = queBind.DeQueue<ComputeT>();
            for (int j = 0; j < jlen; ++j) {
                ...
                DataCopyPad(outGm[j], bindLocal, ...);
            }
            queBind.FreeTensor(bindLocal);
        }
    }
}

private:
...
TQueBind<TPosition::VECIN, TPosition::VECOUT, BUFFER_NUM> queBind; // 使用TQueBind替换原来的
QueI, QueO
...
};

extern "C" __global__ __aicore__ void example_kernel(...)
{
    ...
    op.Process(...);
}
```

【性能对比】

图 3-96 aiv_vec_time 优化前后对比



如上图所示，将反例中DataCopy指令替换为TQueBind之后有明显优化。由于省略了数据从VECIN拷贝到VECOUT的步骤，aiv_vec_time几乎缩减为0。

3.6.5.8 通过缩减 Tensor ShapeInfo 维度，优化栈空间

【优先级】中

【描述】GlobalTensor和LocalTensor中通过ShapeInfo类型的成员变量来保存shape信息，SetShapeInfo/GetShapeInfo可以设置或者获取ShapeInfo，在算子实现内部用于shape信息保存和传递。默认情况下支持的最大维度为8。在不使用上述ShapeInfo功能的情况下，不需要这些信息，可以通过K_MAX_SHAPE_DIM宏将其设置为0。经实测减小K_MAX_SHAPE_DIM值，可缩减栈空间，减少scalar指令和cache miss几率，提升算子性能。

```
...
#ifndef K_MAX_SHAPE_DIM
#define K_MAX_SHAPE_DIM 8
#endif
...
struct ShapeInfo {
public:
    ...
    uint32_t shape[K_MAX_SHAPE_DIM];
    uint32_t originalShape[K_MAX_SHAPE_DIM];
};

template <typename T> class GlobalTensor {
...
private:
    ShapeInfo shapeInfo_;
}

template <typename T> class LocalTensor {
...
private:
```

```
ShapeInfo shapeInfo_;  
}  
...
```

【反例】

算子无需使用ShapeInfo，但未对ShapeInfo大小进行限制（使用默认值8），导致浪费 $K_MAX_SHAPE_DIM * sizeof(uint32_t) * 2 * 4$ 字节的栈空间。2表示有shape和originalShape两个数组，4表示该样例中使用了GlobalTensor和LocalTensor共4个Tensor。

```
...  
#include "kernel_operator.h" ...  
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR x, GM_ADDR z, GM_ADDR  
workspace, GM_ADDR tiling)  
{  
    ...  
    GlobalTensor<T> dataIn;  
    GlobalTensor<T> dataOut;  
    LocalTensor<T> vecIn;  
    LocalTensor<T> vecOut;  
    ...  
}  
...
```

【正例】

算子无需使用ShapeInfo，设置#define K_MAX_SHAPE_DIM 0，有效缩减了 $K_MAX_SHAPE_DIM * sizeof(uint32_t) * 2 * 4$ 大小的栈空间。

```
#define K_MAX_SHAPE_DIM 0  
...  
#include "kernel_operator.h" //需注意定义K_MAX_SHAPE_DIM宏的位置须在包含Ascend C相关头文件之前  
...  
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR x, GM_ADDR z, GM_ADDR  
workspace, GM_ADDR tiling)  
{  
    ...  
    GlobalTensor<T> dataIn;  
    GlobalTensor<T> dataOut;  
    LocalTensor<T> vecIn;  
    LocalTensor<T> vecOut;  
    ...  
}  
...
```

3.6.5.9 避免 Unified Buffer 的 bank 冲突

3.6.5.9.1 避免 bank 冲突（NPU 架构版本 220x）

【优先级】高

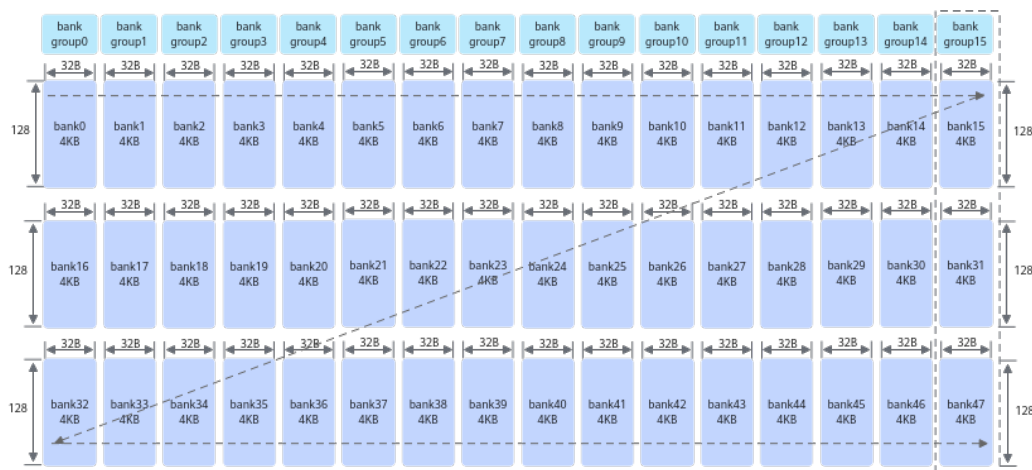
说明

该性能优化建议适用于如下产品型号：

- Atlas A3 训练系列产品/Atlas A3 推理系列产品
- Atlas A2 训练系列产品/Atlas A2 推理系列产品

【描述】为了提高数据访问的效率和吞吐量，Unified Buffer采用了bank（大小相等的内存模块）结构设计。Unified Buffer总大小为192K，划分为48个bank。每个bank由128行组成，每行长度为32B。这48个bank进一步组织为16个bank group，每个bank group包含3个bank，例如bank15、bank31和bank47组成一个bank group。

图 3-97 bank 结构示意图（图中箭头方向表示内存排布的顺序）



每个bank可以独立地进行数据的读写操作，允许多个数据请求同时进行。然而，当多个读写操作试图同时访问同一个bank或bank group时，由于硬件资源的限制，这些操作必须排队等待，会导致bank冲突，引起性能下降。

具体来说，Vector计算单元每拍（一个指令周期）能够从每个bank group中读取或写入一行数据。如果同一个API中的多个操作试图同时访问同一个bank或bank group，Vector计算单元无法在同一个周期内处理所有请求，导致这些请求排队等待。这种排队增加了数据访问的延迟，降低了系统的整体性能。

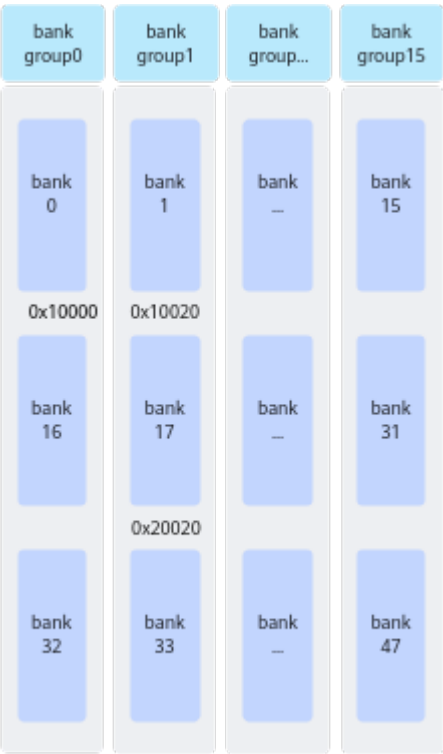
bank 冲突的典型场景

bank冲突主要可以分为以下三种场景：

- **读写冲突：**读操作和写操作同时尝试访问同一个bank。
- **写写冲突：**多个写操作同时尝试访问同一个bank group。
- **读读冲突：**多个读操作同时尝试访问同一个bank group。

下文给出了一些具体的示例，假设，0x10000地址在bank16上，0x10020在bank17上，0x20020在bank33上，如下图所示：

图 3-98 地址分配示意图



- 读写冲突示例
Vector指令的源操作数src和目的操作数dst同时读写到同一个bank时造成读写冲突，具体分析如下：

表 3-13 读写冲突示例

序号	src地址	dst地址	bank	bank group	结论
示例 1	0x10020	0x10000	bank_id0 != bank_id1	bank_group_id0 != bank_group_id1	src地址和dst地址分别属于bank16和bank17，故无冲突。
示例 2	0x10020	0x10E20	bank_id0 == bank_id1	bank_group_id0 == bank_group_id1	src地址和dst地址的地址都在bank17，故存在冲突。

- 写写冲突示例
Vector指令目的操作数dst对应的8个DataBlock（block0-block7）同时写到一个bank group时造成写写冲突，具体分析如下：

表 3-14 写写冲突示例

序号	dst 地址	blk_stride	block0_addr	block1_addr	block2_addr	...	结论
示例 1	0x1FE00	16	0x1FE00	0x20000	0x20200	...	8个DataBlock均在一个bank group下，故全部冲突，8拍完成一个Repeat的写入。
示例 2	0x1FE00	8	0x1FE00	0x1FF00	0x20000	...	block0和block2在一个bank group，存在冲突，4拍完成一个Repeat的写入。

- 读读冲突

- Vector指令多个源操作数同时读到同一个bank group时造成读读冲突，具体分析如下：

表 3-15 双 src 场景读读冲突示例

序号	src0地址	src1地址	bank	bank group	结论
示例 1	0x10020	0x20020	bank_id0 != bank_id1	bank_group_id0 == bank_group_id1	存在冲突。
示例 2	0x10020	0x10000	bank_id0 != bank_id1	bank_group_id0 != bank_group_id1	无冲突。

- Vector指令某一个源操作数对应的8个DataBlock（block0-block7）读到同一个bank group时造成读读冲突，具体分析如下：

表 3-16 单 src 场景读读冲突示例

序号	src地址	blk_stride	block0_addr	block1_addr	block2_addr	...	结论
示例 1	0x1FE00	16	0x1FE00	0x20000	0x20200	...	8个DataBlock均在一个bank group下，故全部冲突，8拍完成一个Repeat的读操作。
示例 2	0x1FE00	8	0x1FE00	0x1FF00	0x20000	...	block0和block2在同一个bank group下，存在冲突，4拍完成一个Repeat。

📖 说明

通过msProf工具可以进行资源冲突占比的相关性能数据采集。
工具的具体使用方法请参考算子调优（msProf）。资源冲突占比文件性能数据文件说明请参考ResourceConflictRatio（资源冲突占比）。

如何避免 bank 冲突

避免bank冲突的方法有两种：**优化计算逻辑**和**优化地址分配**。

- 优化计算逻辑**
对一个shape为(8, 16, 16)的输入做(1, 0, 2)的transpose操作，输出shape为(16, 8, 16)。通过将计算逻辑由“跳读，连续写”修改为“连续读，跳写”可避免冲突问题。实现方案对比如下：

实现方案	原始实现	优化实现
实现方法	跳读，连续写 同一Repeat内输入的8个DataBlock都在同一个bank group而发生读读冲突。	连续读，跳写 同一个Repeat内输入的8个DataBlock不在同一个bank group内，避免了读读冲突。
示意图		

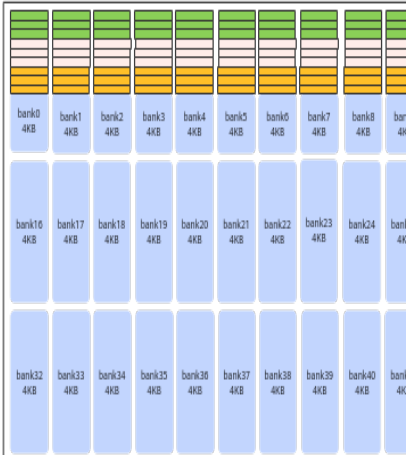
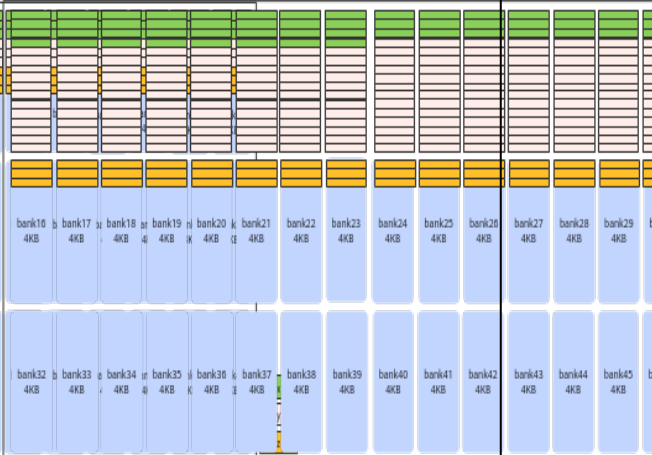
实现方案	原始实现	优化实现
示例代码	<pre>uint64_t mask = 128; UnaryRepeatParams params; params.dstBlkStride = 1; params.srcBlkStride = 16; for(uint32_t i=0; i<16; i++) { AscendC::Adds(dstLocal[i * 128], srcLocal[i * 16], 0, mask, 1, params); }</pre>	<pre>uint64_t mask = 128; UnaryRepeatParams params; params.dstBlkStride = 8; params.srcBlkStride = 1; for(uint32_t i=0; i<8; i++) { AscendC::Adds(dstLocal[i * 16], srcLocal[i * 256], 0, mask, 2, params); }</pre>

- 优化地址分配

实现连续4096个float元素的加法 $z = x + y$ ，通过在内存分配时适当扩大内存，保证在一个Repeat内，x和y不会同时出现在同一个bank group内，x/y和z不会同时出现同一个bank内。完整样例可参考[避免bank冲突样例](#)。

实现方案对比如下：

实现方案	原始实现	优化实现
实现方法	不做地址优化，直接使用InitBuffer分配内存，各个Tensor的地址分别为： x：起始地址0x0，tensor长度为4096 * sizeof(float)字节 y：起始地址0x4000，tensor长度为4096 * sizeof(float)字节 z：起始地址0x8000，tensor长度为4096 * sizeof(float)字节 在一个Repeat内，x和y同时读同一个bank group，x/y和z同时读写同一个bank。	优化地址，使用InitBuffer分配内存时适当扩大内存申请，各个Tensor的地址分别为： x：起始地址0x0，tensor长度为(4096 * sizeof(float) + 256)字节 y：起始地址0x4100，tensor长度为(64 * 1024 - (4096 * sizeof(float) + 256))字节 z：起始地址0x10000，tensor长度为4096 * sizeof(float) 字节 x多申请256字节，避免一个Repeat内x/y同时读同一个bank group；y多申请空间，确保z不会和x/y落入同一个bank

实现方案	原始实现	优化实现
示意图		
示例代码	<pre>pipe.InitBuffer(inQueueX, 1, 4096 * sizeof(float)); pipe.InitBuffer(inQueueY, 1, 4096 * sizeof(float)); pipe.InitBuffer(outQueueZ, 1, 4096 * sizeof(float));</pre>	<pre>pipe.InitBuffer(inQueueX, 1, 4096 * sizeof(float) + 256); // 多申请256字节 pipe.InitBuffer(inQueueY, 1, 64 * 1024 - (4096 * sizeof(float) + 256)); //多申请空间，确保z不会和 x/y落入同一个bank， 64 * 1024是16个bank group 的空间， 4096 * sizeof(float) + 256是x所占的空间 pipe.InitBuffer(outQueueZ, 1, 4096 * sizeof(float));</pre>

3.6.5.10 L2 Cache 切分

【优先级】：高

【描述】假设，AI处理器的L2 Cache大小为192MB，L2 Cache读写混合带宽约为7TB/s，而GM的带宽约为1.6TB/s，两者之间存在较大差距。搬入或搬出相同数据量的情况下，访问L2 Cache读写数据比GM更快。若数据无法命中L2 Cache，即需要访问的数据不在L2 Cache内，导致需要去GM上读写，带宽利用效率较低，最终算子搬入或搬出数据变为算子整个运行过程的性能瓶颈。切分策略建议：当输入和输出数据的数据量超过L2 Cache大小时，Tiling中使能L2 Cache切分策略。

【反例】

假设输入数据大小为InputTotalSize，L2 Cache大小为L2CacheSize，InputTotalSize = L2CacheSize * 2，总核数为20个核，数据未切分，整体一次完成计算。假设20个核一次可以处理共L2CacheSize的数据，则每个核至少两次读取输入数据。

图 3-99 未使能 L2 Cache 切分

384MB

0	0	1	1	2	2	3	3	4	4	5	5	6
10	10	11	11	12	12	13	13	14	14	15	15	16

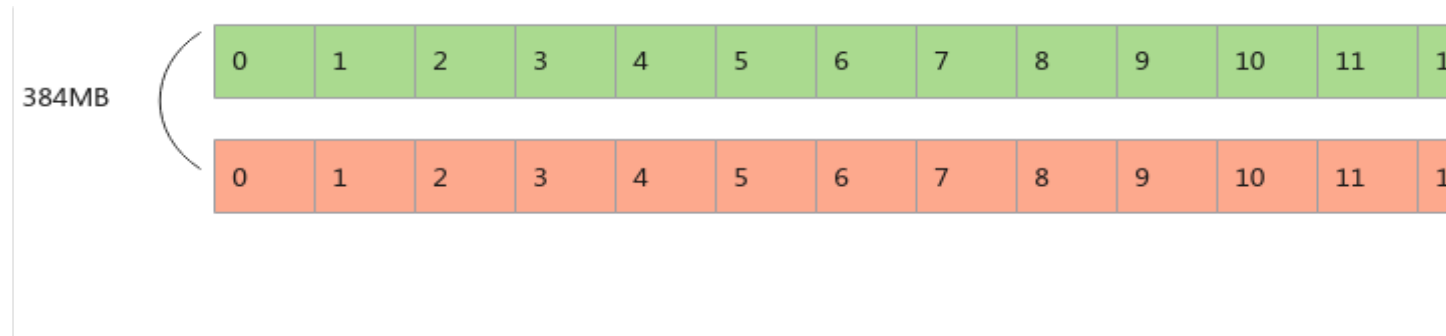
```
constexpr int32_t TOTAL_LENGTH = InputTotalSize / sizeof(half);
constexpr int32_t USE_CORE_NUM = 20;
constexpr int32_t TILE_NUM = 2;
constexpr int32_t BLOCK_LENGTH = TOTAL_LENGTH / USE_CORE_NUM;
constexpr int32_t TILE_LENGTH = BLOCK_LENGTH / TILE_NUM;

class KernelSample {
public:
    __aicore__ inline KernelSample() {}
    __aicore__ inline void Init(GM_ADDR x)
    {
        xGm.SetGlobalBuffer((__gm__ half*)x + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
        yGm.SetGlobalBuffer((__gm__ half*)y + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
        pipe.InitBuffer(inQueueX, 1, BLOCK_LENGTH * sizeof(half));
        pipe.InitBuffer(inQueueY, 1, BLOCK_LENGTH * sizeof(half));
    }
    __aicore__ inline void Process()
    {
        // 示例演示对输入数据加2的运算
        constexpr int32_t loopCount = 2;
        for (int32_t i = 0; i < loopCount; i++) {
            // 外层的每次循环对输入数据进行加1的运算
            for (int32_t j = 0; j < TILE_NUM; j++) {
                // 内层循环分别处理每个核第0块和第1块数据
                CopyIn(j);
                Compute();
                CopyOut(j);
            }
        }
    }
private:
    __aicore__ inline void CopyIn(int32_t process)
    {
        LocalTensor<half> xLocal = inQueueX.AllocTensor<half>();
        // 对于每个核，除了首次读取外，读取第0块数据时，L2 Cache内缓存的是第1块数据；
        // 对于每个核，读取第1块数据时，L2 Cache内缓存的是第0块数据；
        // 每个核需要4次读取GM上的数据
        DataCopy(xLocal, xGm[process * TILE_LENGTH], TILE_LENGTH);
        inQueueX.Enqueue(xLocal);
    }
    __aicore__ inline void Compute()
    {
        LocalTensor<half> yLocal = inQueueY.AllocTensor<half>();
        LocalTensor<half> xLocal = inQueueX.DeQueue<half>();
        Adds(yLocal, xLocal, 1, TILE_LENGTH);
        inQueueY.Enqueue<half>(yLocal);
        inQueueX.FreeTensor(xLocal);
    }
    __aicore__ inline void CopyOut(int32_t process)
    {
        LocalTensor<half> yLocal = inQueueY.DeQueue<half>();
        DataCopy(yGm[process * TILE_LENGTH], yLocal, TILE_LENGTH);
        inQueueY.FreeTensor(yLocal);
    }
}
```

```
}  
...
```

【正例】

假设输入数据大小为InputTotalSize，L2 Cache大小为L2CacheSize，InputTotalSize = L2CacheSize * 2，能使用的总核数为20个核，输入数据均等切分成2份数据，则整体分两次进行计算，每次的计算量为L2CacheSize，第一次20个核先计算前L2CacheSize个数据，第二次20个核计算后L2CacheSize个数据。每次计算前读取的数据能够命中L2 Cache，提升算子性能。

图 3-100 使能 L2 Cache 切分

```
constexpr int32_t TOTAL_LENGTH = InputTotalSize / sizeof(half);  
constexpr int32_t TILE_NUM = 2;  
constexpr int32_t USE_CORE_NUM = 20;  
constexpr int32_t TILE_LENGTH = TOTAL_LENGTH / TILE_NUM;  
constexpr int32_t BLOCK_LENGTH = TILE_LENGTH / USE_CORE_NUM;  
  
class KernelSample {  
public:  
    __aicore__ inline KernelSample() {}  
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, int32_t index)  
    {  
        xGm.SetGlobalBuffer((__gm__ half*)x + BLOCK_LENGTH * GetBlockIdx() + index * TILE_LENGTH,  
        BLOCK_LENGTH);  
        yGm.SetGlobalBuffer((__gm__ half*)y + BLOCK_LENGTH * GetBlockIdx() + index * TILE_LENGTH,  
        BLOCK_LENGTH);  
        pipe.InitBuffer(inQueueX, 1, BLOCK_LENGTH * sizeof(half));  
        pipe.InitBuffer(inQueueY, 1, BLOCK_LENGTH * sizeof(half));  
    }  
    __aicore__ inline void Process()  
    {  
        // 示例演示对输入数据加2的运算  
        constexpr int32_t loopCount = 2;  
        for (int32_t i = 0; i < loopCount; i++) {  
            // 每次循环对输入数据进行加1的运算  
            CopyIn();  
            Compute();  
            CopyOut();  
        }  
    }  
private:  
    __aicore__ inline void CopyIn()  
    {  
        LocalTensor<half> xLocal = inQueueX.AllocTensor<half>();  
        // 对于每个核，除了首次读取外，第二次读取可以命中L2 Cache;  
        // 每个核2次读取GM上的数据，2次访问L2 Cache读数据  
        DataCopy(xLocal, xGm, BLOCK_LENGTH);  
        inQueueX.Enqueue(xLocal);  
    }  
    __aicore__ inline void Compute()  
    {
```

```
LocalTensor<half> yLocal = inQueueY.AllocTensor<half>();
LocalTensor<half> xLocal = inQueueX.DeQue<half>();
Adds(yLocal, xLocal, 1, BLOCK_LENGTH);
inQueueY.EnQue<half>(yLocal);
inQueueX.FreeTensor(xLocal);
}
__aicore__ inline void CopyOut()
{
    LocalTensor<half> yLocal = inQueueY.DeQue<half>();
    DataCopy(yGm, yLocal, BLOCK_LENGTH);
    inQueueY.FreeTensor(yLocal);
}
}
...

extern "C" __global__ __aicore__ void simple_kernel(__gm__ uint8_t* srcGm, __gm__ uint8_t* dstGm)
{
    AscendC::KernelSample op;
    // 输入数据均等切分成2份数据进行计算
    for (int32_t i = 0; i < TILE_NUM; i++) {
        op.Init(srcGm, dstGm, i);
        op.Process();
    }
}
...
```

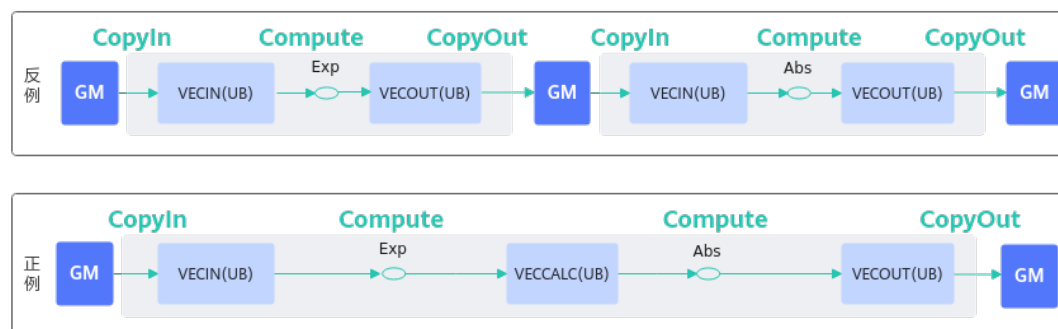
3.6.6 矢量计算

3.6.6.1 通过 Unified Buffer 融合实现连续 vector 计算

【优先级】高

【描述】算子实现中涉及多次vector计算，且前一次计算输出是后一次计算输入的情况下，可将前一次计算输出暂存在UB（Unified Buffer）上直接作为下一次计算的输入，不需要将前一次的计算输出从UB搬运到GM后再从GM搬运到UB。这种UB Buffer融合的方式可以减少搬入搬出次数，实现连续vector计算，提升内存使用效率。数据流图对比如下：

图 3-101 数据流图对比



【反例】

该算子的计算逻辑为进行Exp计算后再进行Abs计算。计算过程中先把源操作数从GM搬运到UB进行Exp计算，Exp计算完成后将Exp的结果从UB搬运到GM；再从GM中把Exp的结果搬运到UB上作为Abs计算的输入，Abs计算完成后将目的操作数结果从UB搬运到GM。整个过程从GM搬进搬出共4次。当需要进行的vector计算为n次时，从GM搬进搬出共需要2n次。

```
class KernelSample {
public:
    __aicore__ inline KernelSample() {}
    __aicore__ inline void Init(__gm__ uint8_t* src0Gm, __gm__ uint8_t* dstGm)
    {
        src0Global.SetGlobalBuffer((__gm__ float*)src0Gm);
        dstGlobal.SetGlobalBuffer((__gm__ float*)dstGm);
        pipe.InitBuffer(inQueueSrc0, 1, 1024 * sizeof(float));
        pipe.InitBuffer(outQueueDst, 1, 1024 * sizeof(float));
    }
    __aicore__ inline void Process()
    {
        CopyIn();
        Compute();
        CopyOut();
        CopyIn1();
        Compute1();
        CopyOut1();
    }

private:
    __aicore__ inline void CopyIn()
    {
        LocalTensor<float> src0Local = inQueueSrc0.AllocTensor<float>();
        DataCopy(src0Local, src0Global, 1024);
        inQueueSrc0.EnQue(src0Local);
    }
    __aicore__ inline void Compute()
    {
        LocalTensor<float> src0Local = inQueueSrc0.DeQue<float>();
        LocalTensor<float> dstLocal = outQueueDst.AllocTensor<float>();
        Exp(dstLocal, src0Local, 1024);
        outQueueDst.EnQue<float>(dstLocal);
        inQueueSrc0.FreeTensor(src0Local);
    }
    __aicore__ inline void CopyOut()
    {
        LocalTensor<float> dstLocal = outQueueDst.DeQue<float>();
        DataCopy(dstGlobal, dstLocal, 1024);
        outQueueDst.FreeTensor(dstLocal);
    }
    __aicore__ inline void CopyIn1()
    {
        LocalTensor<float> src0Local = inQueueSrc0.AllocTensor<float>();
        DataCopy(src0Local, dstGlobal, 1024);
        inQueueSrc0.EnQue(src0Local);
    }
    __aicore__ inline void Compute1()
    {
        LocalTensor<float> src0Local = inQueueSrc0.DeQue<float>();
        LocalTensor<float> dstLocal = outQueueDst.AllocTensor<float>();
        Abs(dstLocal, src0Local, 1024);
        outQueueDst.EnQue<float>(dstLocal);
        inQueueSrc0.FreeTensor(src0Local);
    }
    __aicore__ inline void CopyOut1()
    {
        LocalTensor<float> dstLocal = outQueueDst.DeQue<float>();
        DataCopy(dstGlobal, dstLocal, 1024);
        outQueueDst.FreeTensor(dstLocal);
    }
}

private:
    TPipe pipe;
    TQue<TPosition::VECIN, 1> inQueueSrc0;
    TQue<TPosition::VECOUT, 1> outQueueDst;
    GlobalTensor<float> src0Global, dstGlobal;
};
```

【正例】

使用UB Buffer融合方式后，在UB上进行连续vector计算时，前一次的结果可直接作为后一次计算的输入，继续在UB上进行计算，不需要中间的搬进搬出，只需在开始计算时将源操作数搬运到UB，以及全部计算结束后将最终结果从UB搬运到GM，共2次搬进搬出。

```
class KernelSample {
public:
    __aicore__ inline KernelSample() {}
    __aicore__ inline void Init(__gm__ uint8_t* src0Gm, __gm__ uint8_t* dstGm)
    {
        src0Global.SetGlobalBuffer((__gm__ float*)src0Gm);
        dstGlobal.SetGlobalBuffer((__gm__ float*)dstGm);
        pipe.InitBuffer(inQueueSrc0, 1, 1024 * sizeof(float));
        pipe.InitBuffer(outQueueDst, 1, 1024 * sizeof(float));
    }
    __aicore__ inline void Process()
    {
        CopyIn();
        Compute();
        CopyOut();
    }

private:
    __aicore__ inline void CopyIn()
    {
        LocalTensor<float> src0Local = inQueueSrc0.AllocTensor<float>();
        DataCopy(src0Local, src0Global, 1024);
        inQueueSrc0.EnQue(src0Local);
    }
    __aicore__ inline void Compute()
    {
        LocalTensor<float> src0Local = inQueueSrc0.DeQue<float>();
        LocalTensor<float> dstLocal = outQueueDst.AllocTensor<float>();
        Exp(dstLocal, src0Local, 1024);
        Abs(dstLocal, dstLocal, 1024);
        outQueueDst.EnQue<float>(dstLocal);
        inQueueSrc0.FreeTensor(src0Local);
    }
    __aicore__ inline void CopyOut()
    {
        LocalTensor<float> dstLocal = outQueueDst.DeQue<float>();
        DataCopy(dstGlobal, dstLocal, 1024);
        outQueueDst.FreeTensor(dstLocal);
    }

private:
    TPipe pipe;
    TQue<TPosition::VECIN, 1> inQueueSrc0;
    TQue<TPosition::VECOUT, 1> outQueueDst;
    GlobalTensor<float> src0Global, dstGlobal;
};
```

3.6.6.2 Vector 算子灵活运用 Counter 模式

【优先级】高

【描述】Normal模式下，通过迭代次数repeatTimes和掩码mask，控制Vector算子中矢量计算API的计算数据量；当用户想要指定API计算的总元素个数时，首先需要自行判断是否存在不同的主块和尾块，主块需要将mask设置为全部元素参与计算，并且计算主块所需迭代次数，然后根据尾块中剩余元素个数重置mask，再进行尾块的运算，在此过程中涉及大量Scalar计算。

Counter模式下，用户不需要计算迭代次数以及判断是否存在尾块，将mask模式设置为Counter模式后，只需要设置mask为{0, 总元素个数}，然后调用相应的API，处理逻辑更简便，减少了指令数量和Scalar计算量，同时更加高效地利用了指令单次执行的并发能力，进而提升性能。

提示：Normal模式和Counter模式、掩码的介绍可参考[2.5.2.3.1 如何使用掩码操作API](#)。

以下反例和正例中的代码均以AddCustom算子为例，修改其中Add接口的调用代码，以说明Counter模式的优势。

```
AscendC::Add(zLocal, xLocal, yLocal, this->tileLength);
```

【反例】

输入数据类型为half的xLocal, yLocal，数据量均为15000。Normal模式下，每个迭代内参与计算的元素个数最多为256B/sizeof(half)=128个，所以15000次Add计算会被分为：主块计算15000/128=117次迭代，每次迭代128个元素参与计算；尾块计算1次迭代，该迭代15000-117*128=24个元素参与计算。从代码角度，需要计算主块的repeatTimes、尾块元素个数；主块计算时，设置mask值为128，尾块计算时，需要设置mask值为尾块元素个数24；这些过程均涉及Scalar计算。

```
uint32_t ELE_SIZE = 15000;
AscendC::BinaryRepeatParams binaryParams;

uint32_t numPerRepeat = 256 / sizeof(DTYPE_X); // DTYPE_X为half数据类型
uint32_t mainRepeatTimes = ELE_SIZE / numPerRepeat;
uint32_t tailEleNum = ELE_SIZE % numPerRepeat;

AscendC::SetMaskNorm();
AscendC::SetVectorMask<DTYPE_X, AscendC::MaskMode::NORMAL>(numPerRepeat); // 设置normal模式
mask，使每个迭代计算128个数
AscendC::Add<DTYPE_X, false>(zLocal, xLocal, yLocal, AscendC::MASK_PLACEHOLDER, mainRepeatTimes,
binaryParams); // MASK_PLACEHOLDER值为0，此处为mask占位，实际mask值以SetVectorMask设置的为准
if (tailEleNum > 0) {
    AscendC::SetVectorMask<DTYPE_X, AscendC::MaskMode::NORMAL>(tailEleNum); // 设置normal模式
mask，使每个迭代计算24个数
    // 偏移tensor的起始地址，在xLocal和yLocal的14976个元素处，进行尾块计算
    AscendC::Add<DTYPE_X, false>(zLocal[mainRepeatTimes * numPerRepeat], xLocal[mainRepeatTimes *
numPerRepeat],
        yLocal[mainRepeatTimes * numPerRepeat], AscendC::MASK_PLACEHOLDER, 1, binaryParams);
}
AscendC::ResetMask(); // 还原mask值
```

【正例】

输入数据类型为half的xLocal, yLocal，数据量均为15000。Counter模式下，只需要设置mask为所有参与计算的元素个数15000，然后直接调用Add指令，即可完成所有计算，不需要繁琐的主尾块计算，代码较为简练。

当要处理多达15000个元素的矢量计算时，Counter模式的优势更明显，不需要反复修改主块和尾块不同的mask值，减少了指令条数以及Scalar计算量，并充分利用了指令单次执行的并发能力。

```
uint32_t ELE_SIZE = 15000;
AscendC::BinaryRepeatParams binaryParams;
AscendC::SetMaskCount();
AscendC::SetVectorMask<DTYPE_X, AscendC::MaskMode::COUNTER>(ELE_SIZE); // 设置counter模式mask，
总共计算15000个数
AscendC::Add<DTYPE_X, false>(zLocal, xLocal, yLocal, AscendC::MASK_PLACEHOLDER, 1,
binaryParams); // MASK_PLACEHOLDER值为0，此处为mask占位，实际mask值以SetVectorMask
设置的为准
AscendC::ResetMask(); // 还原mask值
```

【性能对比】

图 3-102 Normal 模式和 Counter 模式下的 Scalar 执行时间对比

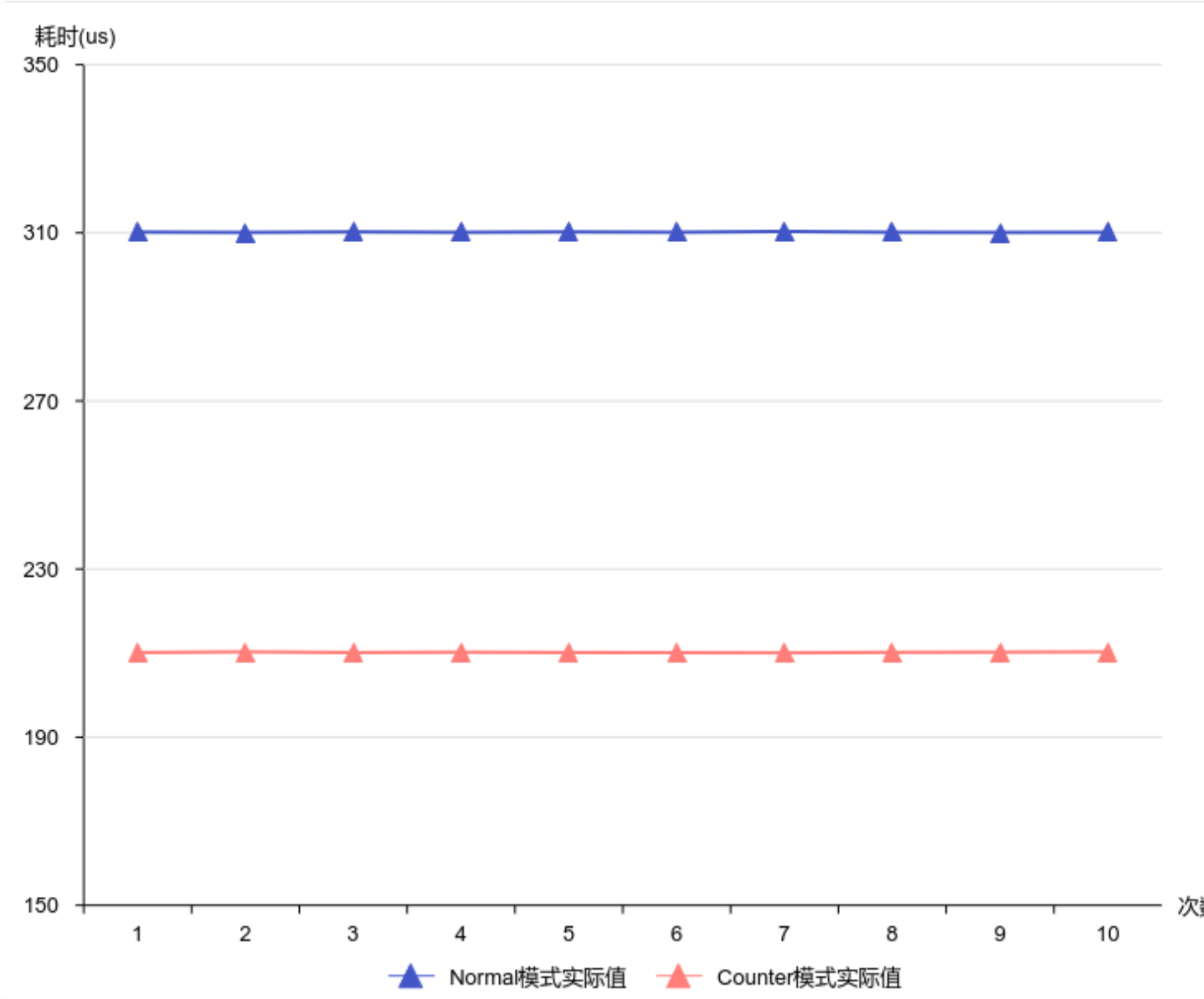
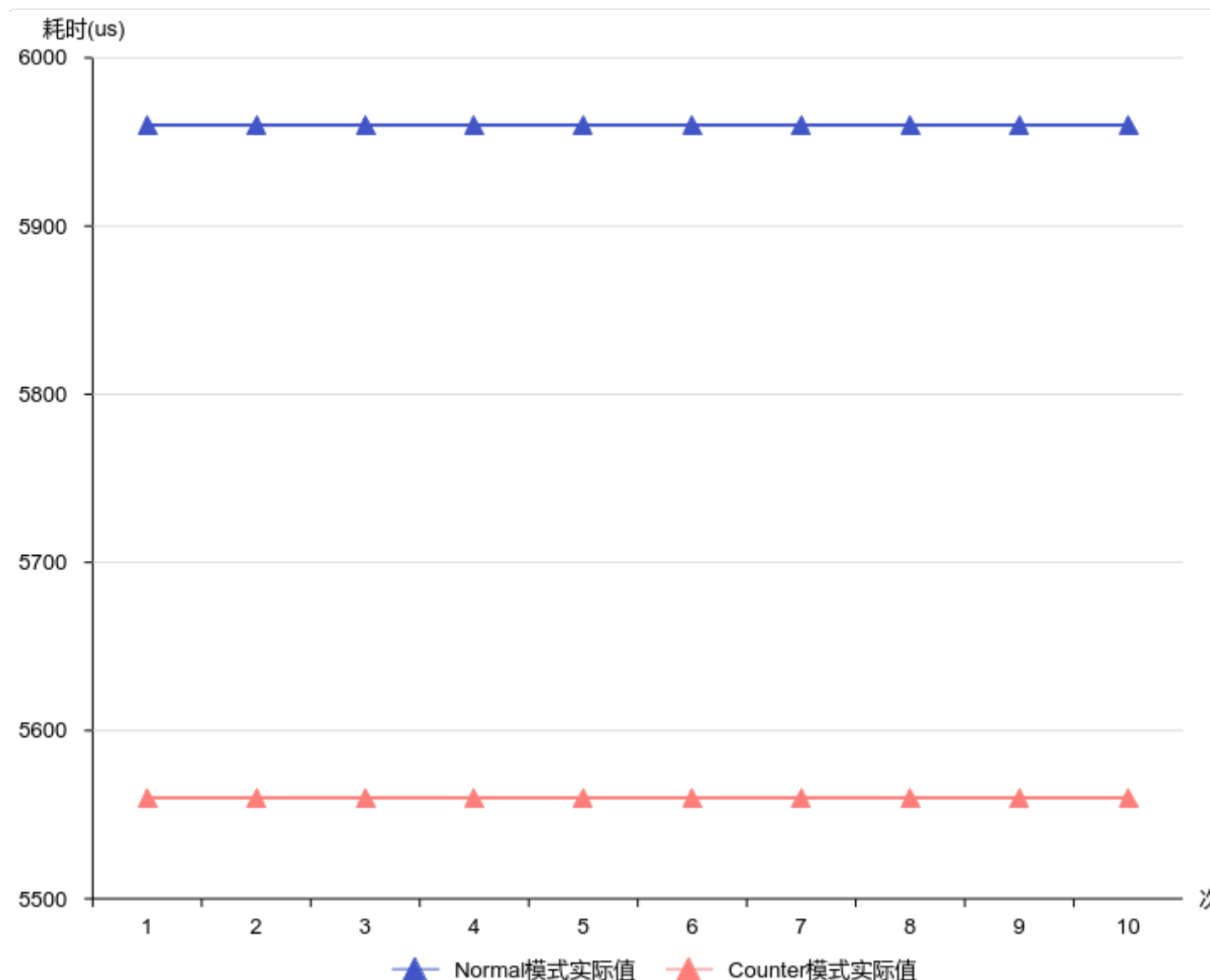


图 3-103 Normal 模式和 Counter 模式下的 Vector 执行时间对比



以上性能数据是分别循环运行1000次反例和正例代码得到的Scalar和Vector执行时间。从上述两幅性能对比图和示例代码可以看到，使用Counter模式能够大幅度简化代码，易于维护，同时能够降低Scalar和Vector计算耗时，获得性能提升。

3.6.6.3 选择低延迟指令，优化归约操作性能

【优先级】高

【描述】

指令执行延迟（Instruction Execution Latency）指的是一条指令从开始执行到完全完成（即所有操作结束，结果可用）所消耗的时间，它直接影响程序的响应速度和实时性。在延迟敏感的场景中，降低指令执行延迟是提升性能的关键。下文以归约操作为例，介绍了几种归约方案的性能对比，便于开发者在使用归约指令时，能够根据具体的数据规模和场景，选择性能更高的方案。

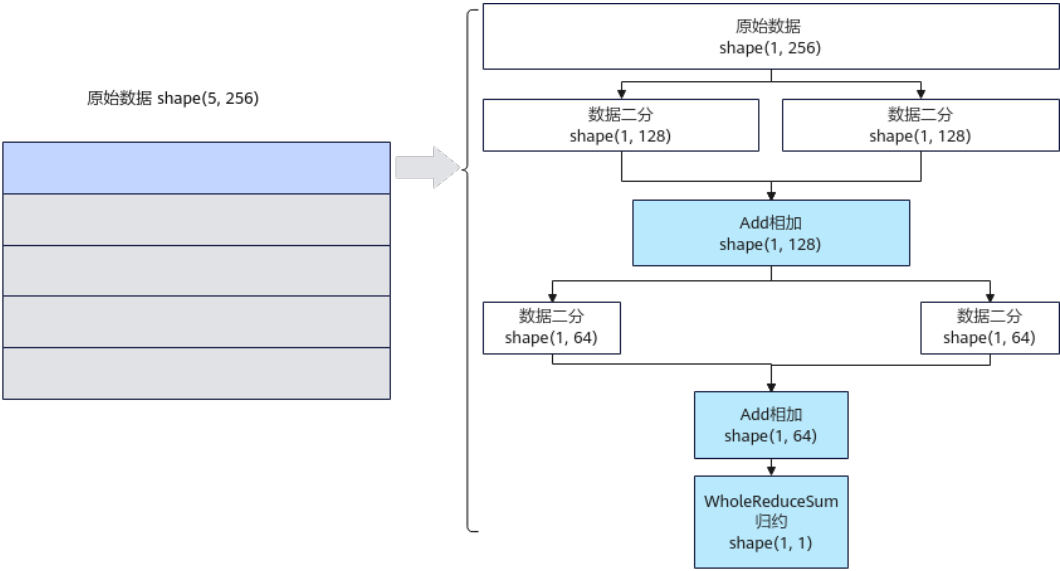
二分累加方案和归约类指令方案的对比

根据单指令性能测试数据（开发者可以自行测试）分析，WholeReduceSum等归约指令的延迟时间约为Add指令的2-5倍。因此，对于连续数据的归约操作，可以采用Add指令和WholeReduceSum指令的组合，以优化整体性能。该方案简称为**二分累加方案**，具体方案说明如下：

- 二分累加：将数据一分为二，使用Add指令将两部分数据相加；将相加后的结果再次一分为二，继续使用Add指令进行累加，重复此过程。
- 当二分累加后的数据量小于等于256Byte（一条指令一个Repeat的数据操作量），使用WholeReduceSum指令，一次执行得到归约结果。

假设输入数据的数据类型为float，shape为(5, 256)，下图展示了一行数据的执行过程：

图 3-104 二分累加方案示意图



将以上过程，针对每一行，各自执行，得到最终归约结果，即shape为(m, k)的数据，归约完成后，shape为(m, 1)。

由于ReduceSum接口是由多种指令组合实现，通常来说，数据量较大，循环次数较多的场景，二分累加方案性能 > WholeReduceSum单指令操作性能 > ReduceSum接口性能。而小数据量或者特殊shape下的场景，需要拆分开来，依据指令执行时间和指令执行数目等条件，具体问题具体分析。

下文给出了二分累加方案和归约类指令方案的核心代码片段和性能数据对比。完整样例请参考[ReduceCustom](#)。

【性能数据】

输入shape为30000，数据类型为float时，如下示例的性能数据对比如下，数据单位为cycle，使用GetSystemCycle接口获取。

二分累加方案	WholeReduceSum单指令操作
172	242

【二分累加方案】

```
__aicore__ inline void BinaryReduceSumImpl(const AscendC::LocalTensor<float>& dst, const
AscendC::LocalTensor<float>& src, const uint32_t bsLength, const uint32_t hLength)
{
    // src为二维数据, shape为(bsLength, hLength), dst的shape为(bsLength,1)
    AscendC::BinaryRepeatParams binaryParams;
    AscendC::UnaryRepeatParams unaryParams;
    AscendC::SetMaskCount();
    for (uint32_t i = 0; i < bsLength; i++) {
        AscendC::LocalTensor<float> srcTmp = src[i * hLength];
        AscendC::LocalTensor<float> dstTmp = dst[i * hLength];
        uint32_t totalNum = hLength / 16 * 16;
        uint32_t remaining = hLength - totalNum;
        AscendC::LocalTensor<float> remainingTensor = srcTmp[totalNum];
        while (totalNum > ONE_REPEAT_FLOAT_SIZE) {
            uint32_t halfNum = AscendC::DivCeil(totalNum, 16) * DEFAULT_REP_STRIDE;
            AscendC::SetVectorMask<uint8_t, AscendC::MaskMode::COUNTER>(0, totalNum - halfNum);
            AscendC::Add<float, false>(dstTmp, srcTmp, srcTmp[halfNum], AscendC::MASK_PLACEHOLDER, 1,
binaryParams);
            totalNum = halfNum;
            srcTmp = dstTmp;
        }
        if (remaining != 0 && hLength > ONE_REPEAT_FLOAT_SIZE) {
            AscendC::SetVectorMask<uint8_t, AscendC::MaskMode::COUNTER>(0, remaining);
            AscendC::Add<float, false>(dstTmp, dstTmp, remainingTensor, AscendC::MASK_PLACEHOLDER, 1,
binaryParams);
        }
        AscendC::SetVectorMask<uint8_t, AscendC::MaskMode::COUNTER>(0, totalNum);
        AscendC::WholeReduceSum<float, false>(dstTmp, srcTmp, AscendC::MASK_PLACEHOLDER, 1,
DEFAULT_BLK_STRIDE,
        DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
    }
    AscendC::ResetMask();
    AscendC::SetMaskNorm();
}
```

【WholeReduceSum单指令操作】

```
__aicore__ inline void WholeReduceSumImpl(const AscendC::LocalTensor<float>& dst, const
AscendC::LocalTensor<float>& src, const uint32_t bsLength, const uint32_t hLength)
{
    // src为二维数据, shape为(bsLength, hLength), dst的shape为(bsLength,1)
    AscendC::SetMaskCount();
    for (uint32_t i = 0; i < bsLength; i++) {
        uint32_t totalNum = hLength;
        AscendC::LocalTensor<float> srcTmp = src[i * hLength];
        AscendC::LocalTensor<float> dstTmp = dst[i * hLength];
        while (totalNum > 1) {
            AscendC::SetVectorMask<uint8_t, AscendC::MaskMode::COUNTER>(0, totalNum);
            AscendC::WholeReduceSum<float, false>(dstTmp, srcTmp, AscendC::MASK_PLACEHOLDER, 1,
DEFAULT_BLK_STRIDE,
            DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
            totalNum = AscendC::DivCeil(totalNum, ONE_REPEAT_FLOAT_SIZE);
            srcTmp = dstTmp;
        }
    }
    AscendC::ResetMask();
    AscendC::SetMaskNorm();
}
```

BlockReduceSum 和 WholeReduceSum 归约方案对比

进一步测试分析可知, 单指令BlockReduceSum的执行效率优于WholeReduceSum, 因此, 根据不同的shape, 通过不同的指令组合, 可以达到更佳的性能。

例如数据类型为float，shape大小为256的数据，可以通过如下三种方式得到归约结果：

- 使用两次WholeReduceSum；
- 使用三次BlockReduceSum；
- 一次BlockReduceSum操作加一次WholeReduceSum操作。

通过分析单指令性能数据（开发者可以自行测试）可知：一次BlockReduceSum操作加一次WholeReduceSum操作性能优于两次WholeReduceSum，同时也优于三次BlockReduceSum的方案。

下文给出了上面三种方式的核心代码片段和性能数据对比。完整样例请参考[ReduceCustom](#)。

【性能数据】

输入shape为256，数据类型为float。如下示例的性能数据如下：

表 3-17 两次 WholeReduceSum、三次 BlockReduceSum、一次 BlockReduceSum 加一次 WholeReduceSum，三种归约操作的性能数据（循环 100 次的时间总和）

两次WholeReduceSum	三次BlockReduceSum	一次BlockReduceSum加一次WholeReduceSum
13us	13.94us	8.44us

【两次WholeReduceSum操作】

```
...
pipe.InitBuffer(calcBuf, totalLength * sizeof(DTYPE));
AscendC::LocalTensor<DTYPE> tempTensor1 = calcBuf.Get<DTYPE>();
const uint32_t repeatNum = (totalLength * sizeof(DTYPE) + REP_LEN - 1) / REP_LEN;
AscendC::SetMaskCount();
AscendC::SetVectorMask<DTYPE>(0, totalLength);
AscendC::WholeReduceSum<DTYPE, false>(tempTensor1, xLocal, 1, AscendC::MASK_PLACEHOLDER,
    DEFAULT_BLK_STRIDE, DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
AscendC::PipeBarrier<PIPE_V>();
AscendC::SetVectorMask<DTYPE>(0, repeatNum);
AscendC::WholeReduceSum<DTYPE, false>(zLocal, tempTensor1, 1, AscendC::MASK_PLACEHOLDER,
    DEFAULT_BLK_STRIDE, DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
AscendC::PipeBarrier<PIPE_V>();
AscendC::SetMaskNorm();
...
```

【三次BlockReduceSum操作】

```
...
static constexpr uint32_t BLK_LEN = 32;
TBuf<TPosition::VECCALC> calcBuf;
constexpr uint32_t c0Count = BLK_LEN / sizeof(DTYPE_X);
const uint32_t blockNum0 = (totalLength + c0Count - 1) / c0Count;
const uint32_t blockNum1 = (blockNum0 + c0Count - 1) / c0Count;
AscendC::SetMaskCount();
AscendC::SetVectorMask<DTYPE_X>(0, totalLength);
AscendC::BlockReduceSum<DTYPE_X, false>(tempTensor1, xLocal, AscendC::MASK_PLACEHOLDER, 1,
    DEFAULT_BLK_STRIDE, DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
AscendC::PipeBarrier<PIPE_V>();
AscendC::SetVectorMask<DTYPE_X>(0, blockNum0);
AscendC::BlockReduceSum<DTYPE_X, false>(tempTensor1, tempTensor1, AscendC::MASK_PLACEHOLDER, 1,
    DEFAULT_BLK_STRIDE, DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
AscendC::PipeBarrier<PIPE_V>();
```

```
AscendC::SetVectorMask<DTYPE_X>(0, blockNum1);
AscendC::BlockReduceSum<DTYPE_X, false>(zLocal, tempTensor1, AscendC::MASK_PLACEHOLDER, 1,
    DEFAULT_BLK_STRIDE, DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
AscendC::PipeBarrier<PIPE_V>();
AscendC::SetMaskNorm();
...
```

【 BlockReduceSum + WholeReduceSum操作 】

```
...
pipe.InitBuffer(calcBuf, totalLength * sizeof(DTYPE));
AscendC::LocalTensor<DTYPE> tempTensor1 = calcBuf.Get<DTYPE>();
constexpr uint32_t c0Count = BLK_LEN / sizeof(DTYPE);
const uint32_t blockNum0 = (totalLength + c0Count - 1) / c0Count;
AscendC::SetMaskCount();
AscendC::SetVectorMask<DTYPE>(0, totalLength);
AscendC::BlockReduceSum<DTYPE, false>(tempTensor1, xLocal, 1, AscendC::MASK_PLACEHOLDER,
    DEFAULT_BLK_STRIDE, DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
AscendC::PipeBarrier<PIPE_V>();
AscendC::SetVectorMask<DTYPE>(0, blockNum0);
AscendC::WholeReduceSum<DTYPE, false>(zLocal, tempTensor1, 1, AscendC::MASK_PLACEHOLDER,
    DEFAULT_BLK_STRIDE, DEFAULT_BLK_STRIDE, DEFAULT_REP_STRIDE);
AscendC::PipeBarrier<PIPE_V>();
AscendC::SetMaskNorm();
...
```

3.6.7 矩阵计算

3.6.7.1 通过 BT Buffer 实现高效的 bias 计算

【 优先级 】高

【 描述 】算子中进行带bias的矩阵乘计算时，可将bias数据搬运至C2(Bias Table Buffer)上，调用一次Mmad接口实现矩阵乘加bias的计算，或者直接调用Matmul高阶API完成功能。相比于先将矩阵乘的结果从CO1(L0C)搬运到GM上，再搬运到UB上进行加bias的过程，减少了数据搬运的次数，可提升内存使用效率。数据流图对比如下：

图 3-105 反例数据流图

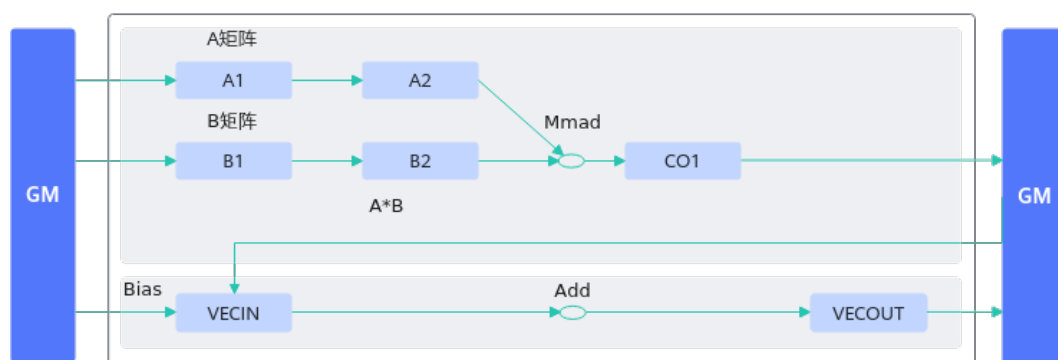
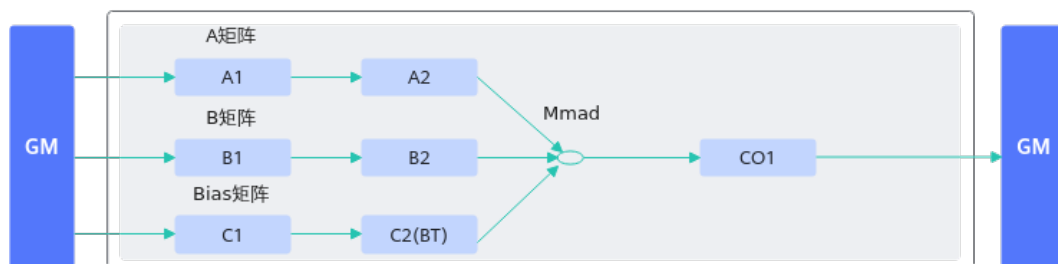


图 3-106 正例数据流图



【反例】

该算子进行带bias的矩阵乘计算时，过程如下：

- 将矩阵乘的计算结果从CO1(L0C)搬运到workspace(GM)上；
- 从workspace搬运到UB上；
- 在UB上进行加bias的运算；
- 最后将结果搬运到GM。

当循环n次该计算过程，则分别增加了n次CO1->workspace、workspace->UB的搬运。

```
// 该样例仅做示例说明，非完整代码，省略了部分同步控制代码
public:
    __aicore__ inline KernelSample()
    {
        aSize = m * k;
        bSize = k * n;
        cSize = m * n;
    }
    __aicore__ inline void Init(__gm__ uint8_t *a, __gm__ uint8_t *b, __gm__ uint8_t *bias, __gm__ uint8_t *c)
    {
        aGM.SetGlobalBuffer((__gm__ half *)a);
        bGM.SetGlobalBuffer((__gm__ half *)b);
        cGM.SetGlobalBuffer((__gm__ float *)c);
        biasGM.SetGlobalBuffer((__gm__ float *)bias);
        pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
        pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half));
        pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));
        pipe.InitBuffer(inQueueBias, 1, n * sizeof(float));
        pipe.InitBuffer(inQueueSrc0, 1, cSize * sizeof(float));
        pipe.InitBuffer(outQueueDst, 1, cSize * sizeof(float));
    }
    __aicore__ inline void Process()
    {
        CopyIn();
        SplitA();
        SplitB();
        Compute();
        CopyOut();
        CopyIn1();
        Compute1();
        CopyOut1();
    }
private:
    __aicore__ inline void CopyIn()
    {
        LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
        LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();
        LocalTensor<float> biasLocal = inQueueBias.AllocTensor<float>();
```



```
Nd2NzParams dataCopyA1Params;
dataCopyA1Params.ndNum = 1;
dataCopyA1Params.nValue = m;
dataCopyA1Params.dValue = k;
dataCopyA1Params.srcNdMatrixStride = 0;
dataCopyA1Params.srcDValue = k;
dataCopyA1Params.dstNzC0Stride = m;
dataCopyA1Params.dstNzNStride = 1;
dataCopyA1Params.dstNzMatrixStride = 0;
DataCopy(a1Local, aGM, dataCopyA1Params);

Nd2NzParams dataCopyB1Params;
dataCopyB1Params.ndNum = 1;
dataCopyB1Params.nValue = k;
dataCopyB1Params.dValue = n;
dataCopyB1Params.srcNdMatrixStride = 0;
dataCopyB1Params.srcDValue = n;
dataCopyB1Params.dstNzC0Stride = k;
dataCopyB1Params.dstNzNStride = 1;
dataCopyB1Params.dstNzMatrixStride = 0;
DataCopy(b1Local, bGM, dataCopyB1Params);
// 将bias搬运到UB
DataCopy(biasLocal, biasGM, n);

inQueueA1.Enqueue(a1Local);
inQueueB1.Enqueue(b1Local);
inQueueBias.Enqueue(biasLocal);
}
__aicore__ inline void SplitA()
{
    ...
}
__aicore__ inline void SplitB()
{
    ...
}
__aicore__ inline void Compute()
{
    LocalTensor<half> a2Local = inQueueA2.DeQue<half>();
    LocalTensor<half> b2Local = inQueueB2.DeQue<half>();
    LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();
    MmadParams mmadParams;
    mmadParams.m = m;
    mmadParams.n = n;
    mmadParams.k = k;
    // 矩阵乘
    Mmad(c1Local, a2Local, b2Local, mmadParams); // m*n
    outQueueCO1.Enqueue<float>(c1Local);
    inQueueA2.FreeTensor(a2Local);
    inQueueB2.FreeTensor(b2Local);
}
__aicore__ inline void CopyOut()
{
    LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    GM_ADDR usrWorkspace = AscendC::GetUserWorkspace(workspace);
    xGm.SetGlobalBuffer((__gm__ float *) (usrWorkspace));
    FixpipeParamsV220 fixpipeParams;
    fixpipeParams.nSize = n;
    fixpipeParams.mSize = m;
    fixpipeParams.srcStride = m;
    fixpipeParams.dstStride = n;
    fixpipeParams.ndNum = 1;
    fixpipeParams.srcNdStride = 0;
    fixpipeParams.dstNdStride = 0;
    // 将矩阵乘的计算结果从CO1搬运到workspace
    Fixpipe(xGm, c1Local, fixpipeParams);
    outQueueCO1.FreeTensor(c1Local);
}
```

```
__aicore__ inline void CopyIn1()
{
    PipeBarrier<PIPE_ALL>();
    // 将矩阵乘的计算结果从workspace搬运到UB
    LocalTensor<float> src0Local = inQueueSrc0.AllocTensor<float>();
    DataCopy(src0Local, xGm, cSize);
    inQueueSrc0.Enqueue(src0Local);
}
__aicore__ inline void Compute1()
{
    LocalTensor<float> src0Local = inQueueSrc0.DeQueue<float>();
    LocalTensor<float> biasLocal = inQueueBias.DeQueue<float>();
    LocalTensor<float> dstLocal = outQueueDst.AllocTensor<float>();
    BinaryRepeatParams addRepeatParams;
    addRepeatParams.dstRepStride = 8;
    addRepeatParams.src0RepStride = 8;
    addRepeatParams.src1RepStride = 0;
    // 加bias的运算
    Add(dstLocal, src0Local, biasLocal, 32, m, addRepeatParams);
    outQueueDst.Enqueue<float>(dstLocal);
    inQueueSrc0.FreeTensor(src0Local);
    inQueueBias.FreeTensor(biasLocal);
}
__aicore__ inline void CopyOut1()
{
    ...
}
private:
    TPipe pipe;
    TQue<TPosition::A1, 1> inQueueA1;
    TQue<TPosition::A2, 1> inQueueA2;
    TQue<TPosition::B1, 1> inQueueB1;
    TQue<TPosition::B2, 1> inQueueB2;
    TQue<TPosition::VECIN, 1> inQueueBias;
    TQue<TPosition::VECOIN, 1> inQueueSrc0;
    TQue<TPosition::VECOOUT, 1> outQueueDst;

    GlobalTensor<half> aGM;
    GlobalTensor<half> bGM;
    GlobalTensor<float> cGM;
    GlobalTensor<float> biasGM;
    uint16_t m = 32, k = 32, n = 32;
    uint16_t aSize, bSize, cSize;
    ...
```

【正例】

该算子进行带bias的矩阵乘计算时，先将bias搬运到BT上，调用一次Mmad接口实现矩阵乘加bias的计算。

```
...
// 该样例仅做示例说明，非完整代码，省略了部分同步控制代码
public:
    __aicore__ inline KernelSample()
    {
        aSize = m * k;
        bSize = k * n;
        cSize = m * n;
    }
    __aicore__ inline void Init(__gm__ uint8_t *a, __gm__ uint8_t *b, __gm__ uint8_t *bias, __gm__ uint8_t *c)
    {
        aGM.SetGlobalBuffer((__gm__ half *)a);
        bGM.SetGlobalBuffer((__gm__ half *)b);
        cGM.SetGlobalBuffer((__gm__ float *)c);
        biasGM.SetGlobalBuffer((__gm__ float *)bias);
        pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
        pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half));
        pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));
    }
```

```
pipe.InitBuffer(inQueueC1, 1, n * sizeof(float));
pipe.InitBuffer(outQueueC2, 1, n * sizeof(float));
}
__aicore__ inline void Process()
{
    CopyIn();
    SplitA();
    SplitB();
    SplitBias();
    Compute();
    CopyOut();
}
private:
__aicore__ inline void CopyIn()
{
    LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
    LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();
    LocalTensor<float> bias1Local = inQueueC1.AllocTensor<float>();

    Nd2NzParams dataCopyA1Params;
    dataCopyA1Params.ndNum = 1;
    dataCopyA1Params.nValue = m;
    dataCopyA1Params.dValue = k;
    dataCopyA1Params.srcNdMatrixStride = 0;
    dataCopyA1Params.srcDValue = k;
    dataCopyA1Params.dstNzC0Stride = m;
    dataCopyA1Params.dstNzNStride = 1;
    dataCopyA1Params.dstNzMatrixStride = 0;
    DataCopy(a1Local, aGM, dataCopyA1Params);

    Nd2NzParams dataCopyB1Params;
    dataCopyB1Params.ndNum = 1;
    dataCopyB1Params.nValue = k;
    dataCopyB1Params.dValue = n;
    dataCopyB1Params.srcNdMatrixStride = 0;
    dataCopyB1Params.srcDValue = n;
    dataCopyB1Params.dstNzC0Stride = k;
    dataCopyB1Params.dstNzNStride = 1;
    dataCopyB1Params.dstNzMatrixStride = 0;
    DataCopy(b1Local, bGM, dataCopyB1Params);
    // 将bias从GM搬运到L1
    DataCopy(bias1Local, biasGM, n);

    inQueueA1.Enqueue(a1Local);
    inQueueB1.Enqueue(b1Local);
    inQueueC1.Enqueue(bias1Local);
}
__aicore__ inline void SplitA()
{
    ...
}
__aicore__ inline void SplitB()
{
    ...
}
__aicore__ inline void SplitBias()
{
    LocalTensor<float> bias1Local = inQueueC1.DeQue<float>();
    LocalTensor<float> bias2Local = outQueueC2.AllocTensor<float>();
    // 将bias从L1搬运到BT
    DataCopy(bias2Local, bias1Local, { 1, (uint16_t)(n * sizeof(float) / 64), 0, 0 });
    outQueueC2.Enqueue<float>(bias2Local);
    inQueueC1.FreeTensor(bias1Local);
}
__aicore__ inline void Compute()
{
    LocalTensor<half> a2Local = inQueueA2.DeQue<half>();
    LocalTensor<half> b2Local = inQueueB2.DeQue<half>();
    LocalTensor<float> bias2Local = outQueueC2.DeQue<float>();
```

```
LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();
MmadParams mmadParams;
mmadParams.m = m;
mmadParams.n = n;
mmadParams.k = k;
mmadParams.cmatrixInitVal = false;
// 矩阵乘
Mmad(c1Local, a2Local, b2Local, bias2Local, mmadParams);
outQueueCO1.EnQue<float>(c1Local);
inQueueA2.FreeTensor(a2Local);
inQueueB2.FreeTensor(b2Local);
outQueueC2.FreeTensor(bias2Local);
}
__aicore__ inline void CopyOut()
{
    LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    FixpipeParamsV220 fixpipeParams;
    fixpipeParams.nSize = n;
    fixpipeParams.mSize = m;
    fixpipeParams.srcStride = m;
    fixpipeParams.dstStride = n;

    fixpipeParams.ndNum = 1;
    fixpipeParams.srcNdStride = 0;
    fixpipeParams.dstNdStride = 0;
    Fixpipe(cGM, c1Local, fixpipeParams);
    outQueueCO1.FreeTensor(c1Local);
}
private:
    TPIPE pipe;
    TQue<TPosition::A1, 1> inQueueA1;
    TQue<TPosition::A2, 1> inQueueA2;
    TQue<TPosition::B1, 1> inQueueB1;
    TQue<TPosition::B2, 1> inQueueB2;
    TQue<TPosition::CO1, 1> outQueueCO1;
    TQue<TPosition::C1, 1> inQueueC1;
    TQue<TPosition::C2, 1> outQueueC2;

    GlobalTensor<half> aGM;
    GlobalTensor<half> bGM;
    GlobalTensor<float> cGM;
    GlobalTensor<float> biasGM;
    uint16_t m = 32, k = 32, n = 32;
    uint16_t aSize, bSize, cSize;
```

3.6.7.2 通过 FP Buffer 存放量化参数实现高效随路量化

【优先级】高

【描述】算子实现中对矩阵乘结果进行量化计算时，可将量化参数搬运到C2PIPE2GM（Fixpipe Buffer）上，调用一次Fixpipe接口实现矩阵乘结果的量化计算。相比于将矩阵乘的结果从CO1（L0C）搬运到GM，再从GM搬运到UB，在UB进行量化计算的过程，数据搬运的次数更少，内存使用效率更高。

📖 说明

本性能优化手段仅针对Atlas A2 训练系列产品/Atlas A2 推理系列产品生效。

图 3-107 反例数据流图

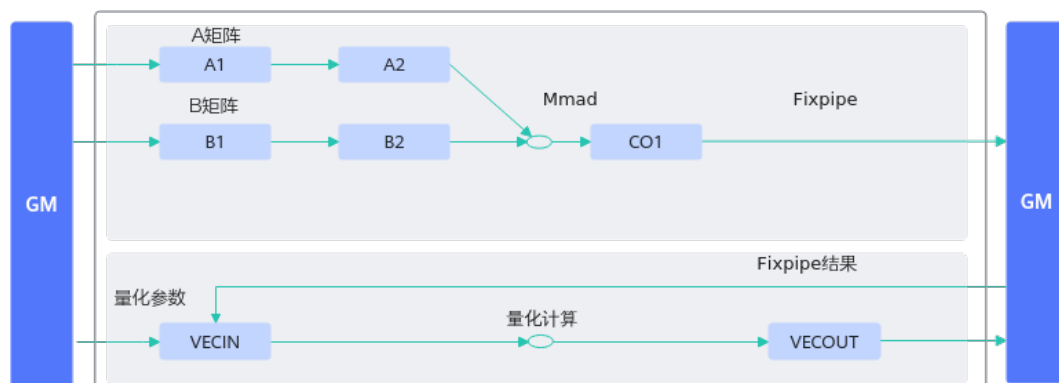
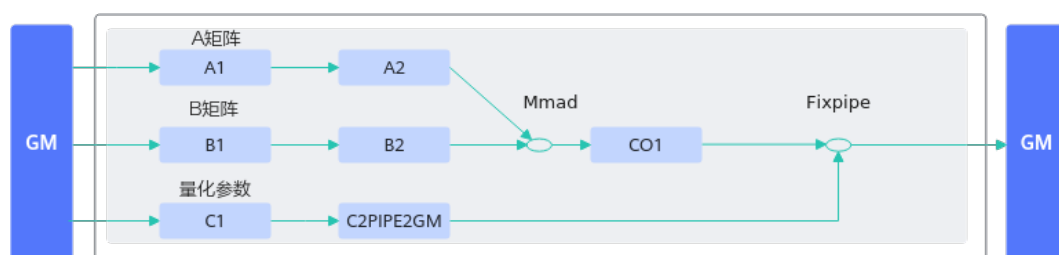


图 3-108 正例数据流图



【反例】

对矩阵乘结果进行量化计算的过程如下：

- 将矩阵乘的结果从CO1搬运到workspace上；
- 再从workspace搬运到UB上；
- 将量化参数搬运到UB上，和矩阵乘的结果一起在UB上进行一系列量化计算；
- 将最终量化结果从UB搬运到GM上。

相比于正确示例多增加了CO1->workspace、workspace->UB的搬运过程和量化的vector计算。

```
...  
// 该样例仅做示例说明，非完整代码，省略了部分同步控制代码  
public:  
    __aicore__ inline KernelSample()  
    {  
        aSize = m * k;  
        bSize = k * n;  
        cSize = m * n;  
    }  
    __aicore__ inline void Init(__gm__ uint8_t *a, __gm__ uint8_t *b, __gm__ uint8_t *c, __gm__ uint8_t  
*deqTensor)  
    {  
        aGM.SetGlobalBuffer((__gm__ half *)a);  
        bGM.SetGlobalBuffer((__gm__ half *)b);  
        cGM.SetGlobalBuffer((__gm__ float *)c);  
        deqGM.SetGlobalBuffer((__gm__ half *)deqTensor);  
        pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));  
        pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));  
        pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));  
        pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half));  
        pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));  
        pipe.InitBuffer(inQueueSrc0, 1, cSize * sizeof(float));  
        pipe.InitBuffer(inQueueTmp, 1, cSize * sizeof(half));  
    }
```

```
pipe.InitBuffer(inQueueDeq, 1, cSize * sizeof(half));
pipe.InitBuffer(outQueueDst, 1, cSize * sizeof(int8_t));
}
__aicore__ inline void Process()
{
    CopyIn();
    SplitA();
    SplitB();
    Compute();
    CopyOut();
    CopyIn1();
    Compute1();
    CopyOut1();
}
private:
__aicore__ inline void CopyIn()
{
    LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
    LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();
    LocalTensor<half> deqLocal = inQueueDeq.AllocTensor<half>();

    Nd2NzParams dataCopyA1Params;
    dataCopyA1Params.ndNum = 1;
    dataCopyA1Params.nValue = m;
    dataCopyA1Params.dValue = k;
    dataCopyA1Params.srcNdMatrixStride = 0;
    dataCopyA1Params.srcDValue = k;
    dataCopyA1Params.dstNzC0Stride = m;
    dataCopyA1Params.dstNzNStride = 1;
    dataCopyA1Params.dstNzMatrixStride = 0;
    DataCopy(a1Local, aGM, dataCopyA1Params);

    Nd2NzParams dataCopyB1Params;
    dataCopyB1Params.ndNum = 1;
    dataCopyB1Params.nValue = k;
    dataCopyB1Params.dValue = n;
    dataCopyB1Params.srcNdMatrixStride = 0;
    dataCopyB1Params.srcDValue = n;
    dataCopyB1Params.dstNzC0Stride = k;
    dataCopyB1Params.dstNzNStride = 1;
    dataCopyB1Params.dstNzMatrixStride = 0;
    DataCopy(b1Local, bGM, dataCopyB1Params);
    // 将量化参数搬运到UB
    DataCopy(deqLocal, deqGM, cSize);

    inQueueA1.Enqueue(a1Local);
    inQueueB1.Enqueue(b1Local);
    inQueueDeq.Enqueue(deqLocal);
}
__aicore__ inline void SplitA()
{
    ...
}
__aicore__ inline void SplitB()
{
    ...
}
__aicore__ inline void Compute()
{
    LocalTensor<half> a2Local = inQueueA2.DeQueue<half>();
    LocalTensor<half> b2Local = inQueueB2.DeQueue<half>();
    LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();
    MmadParams mmadParams;
    mmadParams.m = m;
    mmadParams.n = n;
    mmadParams.k = k;
    // 矩阵乘
    Mmad(c1Local, a2Local, b2Local, mmadParams); // m*n
    outQueueCO1.Enqueue<float>(c1Local);
}
```

```
inQueueA2.FreeTensor(a2Local);
inQueueB2.FreeTensor(b2Local);
}
__aicore__ inline void CopyOut()
{
    LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    GM_ADDR usrWorkspace = AscendC::GetUserWorkspace(workspace);
    xGm.SetGlobalBuffer((__gm__ float *) (usrWorkspace));
    FixpipeParamsV220 fixpipeParams;
    fixpipeParams.nSize = n;
    fixpipeParams.mSize = m;
    fixpipeParams.srcStride = m;
    fixpipeParams.dstStride = n;
    fixpipeParams.ndNum = 1;
    fixpipeParams.srcNdStride = 0;
    fixpipeParams.dstNdStride = 0;
    // 将矩阵乘的计算结果从CO1搬运到workspace
    Fixpipe(xGm, c1Local, fixpipeParams);
    outQueueCO1.FreeTensor(c1Local);
}
__aicore__ inline void CopyIn1()
{
    // 将矩阵乘的计算结果从workspace搬运到UB
    LocalTensor<float> src0Local = inQueueSrc0.AllocTensor<float>();
    DataCopy(src0Local, xGm, cSize);
    inQueueSrc0.EnQue(src0Local);
}
__aicore__ inline void Compute1()
{
    LocalTensor<float> src0Local = inQueueSrc0.DeQue<float>();
    LocalTensor<half> tmpLocal = inQueueTmp.AllocTensor<half>();
    LocalTensor<half> deqLocal = inQueueDeq.DeQue<half>();
    LocalTensor<int8_t> dstLocal = outQueueDst.AllocTensor<int8_t>();
    // 量化计算
    Cast(tmpLocal, src0Local, RoundMode::CAST_NONE, cSize);
    LocalTensor<half> tmpHalfBuffer = src0Local.ReinterpretCast<half>();
    Mul(tmpHalfBuffer, tmpLocal, deqLocal, cSize);
    Cast(dstLocal, tmpHalfBuffer, RoundMode::CAST_NONE, cSize);
    outQueueDst.EnQue<int8_t>(dstLocal);
    inQueueSrc0.FreeTensor(src0Local);
    inQueueTmp.FreeTensor(tmpLocal);
    inQueueDeq.FreeTensor(deqLocal);
}
__aicore__ inline void CopyOut1()
{
    ...
}
private:
    TPipe pipe;
    TQue<TPosition::A1, 1> inQueueA1;
    TQue<TPosition::A2, 1> inQueueA2;
    TQue<TPosition::B1, 1> inQueueB1;
    TQue<TPosition::B2, 1> inQueueB2;
    TQue<TPosition::CO1, 1> outQueueCO1;
    TQue<TPosition::VECIN, 1> inQueueDeq;
    TQue<TPosition::VECIN, 1> inQueueSrc0;
    TQue<TPosition::VECCALC, 1> inQueueTmp;
    TQue<TPosition::VECCOUT, 1> outQueueDst;

    GlobalTensor<half> aGM;
    GlobalTensor<half> bGM;
    GlobalTensor<float> cGM;
    GlobalTensor<float> biasGM;
    uint16_t m = 32, k = 32, n = 32;
    uint16_t aSize, bSize, cSize;
    ...
}
```

【正例】

该算子对矩阵乘的结果进行量化计算时，可将量化参数搬运到FB(Fixpipe Buffer)上，调用一次Fixpipe接口实现矩阵乘结果的量化计算。

```
...
public:
    __aicore__ inline KernelSample()
    {
        aSize = m * k;
        bSize = k * n;
        cSize = m * n;
        QuantSize = n;
    }
    __aicore__ inline void Init(__gm__ uint8_t *a, __gm__ uint8_t *b, __gm__ uint8_t *c, __gm__ uint8_t
*deqTensor)
    {
        aGM.SetGlobalBuffer((__gm__ half *)a);
        bGM.SetGlobalBuffer((__gm__ half *)b);
        cGM.SetGlobalBuffer((__gm__ float *)c);
        deqGM.SetGlobalBuffer((__gm__ uint64_t *)deqTensor);
        pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
        pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half));
        pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));
        pipe.InitBuffer(inQueueDeq1, 1, QuantSize * sizeof(uint64_t));
        pipe.InitBuffer(inQueueDeq, 1, QuantSize * sizeof(uint64_t));
    }
    __aicore__ inline void Process()
    {
        CopyIn();
        SplitA();
        SplitB();
        SplitDeq();
        Compute();
        CopyOut();
    }
private:
    __aicore__ inline void CopyIn()
    {
        LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
        LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();
        LocalTensor<uint64_t> deq1Local = inQueueDeq1.AllocTensor<uint64_t>();

        Nd2NzParams dataCopyA1Params;
        dataCopyA1Params.ndNum = 1;
        dataCopyA1Params.nValue = m;
        dataCopyA1Params.dValue = k;
        dataCopyA1Params.srcNdMatrixStride = 0;
        dataCopyA1Params.srcDValue = k;
        dataCopyA1Params.dstNzC0Stride = m;
        dataCopyA1Params.dstNzNStride = 1;
        dataCopyA1Params.dstNzMatrixStride = 0;
        DataCopy(a1Local, aGM, dataCopyA1Params);

        Nd2NzParams dataCopyB1Params;
        dataCopyB1Params.ndNum = 1;
        dataCopyB1Params.nValue = k;
        dataCopyB1Params.dValue = n;
        dataCopyB1Params.srcNdMatrixStride = 0;
        dataCopyB1Params.srcDValue = n;
        dataCopyB1Params.dstNzC0Stride = k;
        dataCopyB1Params.dstNzNStride = 1;
        dataCopyB1Params.dstNzMatrixStride = 0;
        DataCopy(b1Local, bGM, dataCopyB1Params);
        // 将量化参数搬运到L1上
        DataCopy(deq1Local, deqGM, QuantSize);

        inQueueA1.EnQue(a1Local);
        inQueueB1.EnQue(b1Local);
    }
}
```



```
    inQueueDeq.Enqueue(deq1Local);
}
__aicore__ inline void SplitA()
{
    ...
}
__aicore__ inline void SplitB()
{
    ...
}
__aicore__ inline void SplitDeq()
{
    LocalTensor<uint64_t> deq1Local = inQueueDeq1.DeQue<uint64_t>();
    LocalTensor<uint64_t> deqLocal = inQueueDeq.AllocTensor<uint64_t>();
    // 将量化参数从L1->FB
    DataCopy(deqLocal, deq1Local, { 1, (uint16_t)(QuantSize * sizeof(uint64_t) / 128), 0, 0 });
    inQueueDeq.Enqueue<uint64_t>(deqLocal);
    inQueueDeq1.FreeTensor(deq1Local);
}
__aicore__ inline void Compute()
{
    LocalTensor<half> a2Local = inQueueA2.DeQue<half>();
    LocalTensor<half> b2Local = inQueueB2.DeQue<half>();
    LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();
    MmadParams mmadParams;
    mmadParams.m = m;
    mmadParams.n = n;
    mmadParams.k = k;
    // 矩阵乘
    Mmad(c1Local, a2Local, b2Local, mmadParams); // m*n
    outQueueCO1.Enqueue<float>(c1Local);
    inQueueA2.FreeTensor(a2Local);
    inQueueB2.FreeTensor(b2Local);
}
__aicore__ inline void CopyOut()
{
    LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    LocalTensor<uint64_t> deqLocal = inQueueDeq.DeQue<uint64_t>();
    SetFixpipeNz2ndFlag(1, 0, 0);
    DataCopyCO12DstParams dataCopyParams;
    dataCopyParams.nSize = n;
    dataCopyParams.mSize = m;
    dataCopyParams.srcStride = m;
    dataCopyParams.dstStride = n;
    dataCopyParams.quantPre = QuantMode_t::VQF322B8_PRE;
    dataCopyParams.nz2ndEn = true;
    // 将矩阵乘进行量化后的计算结果搬出
    DataCopy(cGM, c1Local, DataCopyCO12DstParams);
    outQueueCO1.FreeTensor(c1Local);
}
}

private:
    TPipe pipe;
    TQue<QuePosition::A1, 1> inQueueA1;
    TQue<QuePosition::A2, 1> inQueueA2;
    TQue<QuePosition::B1, 1> inQueueB1;
    TQue<QuePosition::B2, 1> inQueueB2;
    TQue<QuePosition::C1, 1> inQueueDeq1;
    TQue<QuePosition::C2PIPE2GM, 1> inQueueDeq;
    TQue<QuePosition::CO1, 1> outQueueCO1;
    GlobalTensor<half> aGM;
    GlobalTensor<half> bGM;
    GlobalTensor<float> cGM;
    GlobalTensor<uint64_t> deqTensorGM;
    uint16_t m = 32, k = 32, n = 32;
    uint16_t aSize, bSize, cSize, QuantSize;
    ...
}
```

3.6.7.3 通过 L0C Buffer 数据暂存实现高效的矩阵乘结果累加

【优先级】高

【描述】算子实现中对矩阵乘的结果进行累加时（比如矩阵 $A1 * B1 + A2 * B2 \dots$ 结果的累加），可将前一次矩阵乘的结果暂存在CO1（L0C）上，调用Mmad接口实现矩阵乘结果累加。相比于每次矩阵乘的结果从CO1搬运到GM上，再搬运到UB上进行累加计算，可减少数据搬运的次数，提升内存使用效率。

图 3-109 反例数据流图

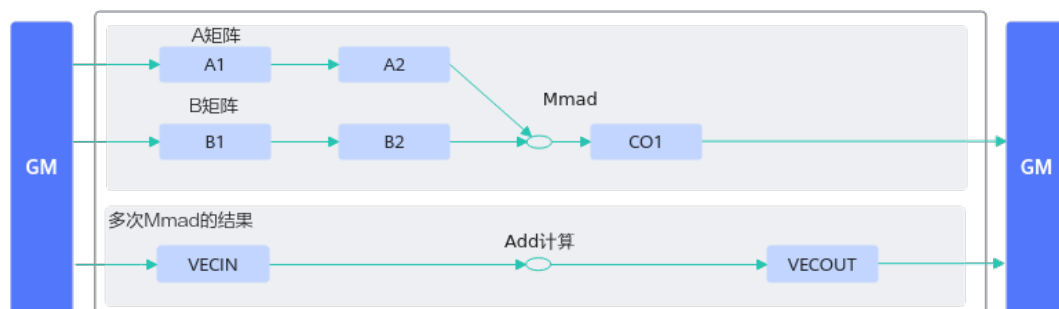
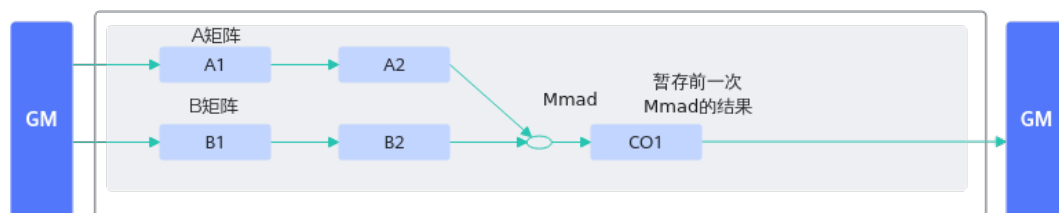


图 3-110 正例数据流图



【反例】

优化前，算子进行2次矩阵乘结果累加的过程如下：

- 将前一次矩阵乘的计算结果从CO1搬运到workspace上，再从workspace搬运到UB上；
- 下一次矩阵乘计算重复完成上述步骤将结果搬运到UB上；
- 在UB上将2次矩阵乘的结果相加。

当需要累加n次矩阵乘时，分别增加了n次CO1->workspace、workspace->UB搬运以及n次Add运算。

```
...  
// 该样例仅做示例说明，非完整代码，省略了部分同步控制代码  
public:  
    __aicore__ inline KernelSample()  
    {  
        aSize = m * k;  
        bSize = k * n;  
        cSize = m * n;  
    }  
    __aicore__ inline void Init(__gm__ uint8_t *a, __gm__ uint8_t *b, __gm__ uint8_t *c)  
    {  
        aGM.SetGlobalBuffer((__gm__ half *)a);  
        bGM.SetGlobalBuffer((__gm__ half *)b);  
        cGM.SetGlobalBuffer((__gm__ float *)c);  
        pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));  
        pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));  
    }
```

```
pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half));
pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));
pipe.InitBuffer(inQueueSrc0, 1, cSize * sizeof(float));
pipe.InitBuffer(inQueueSrc1, 1, cSize * sizeof(float));
pipe.InitBuffer(outQueueDst, 1, cSize * sizeof(float));

}
__aicore__ inline void Process()
{
    // 第一次矩阵乘计算
    CopyIn();
    SplitA();
    SplitB();
    Compute();
    // 将第一次矩阵乘的结果搬出
    CopyOut();
    // 将第一次矩阵乘的结果搬运到UB
    CopyIn1();
    // 第二次矩阵乘计算
    Compute1();
    // 将第一次矩阵乘的结果搬出
    CopyOut1();
    // 将第二次矩阵乘的结果搬运到UB
    CopyIn1();
    // 将两次矩阵乘的结果累加
    Compute2();
    CopyOut2();
}
private:
__aicore__ inline void CopyIn()
{
    LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
    LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();

    Nd2NzParams dataCopyA1Params;
    dataCopyA1Params.ndNum = 1;
    dataCopyA1Params.nValue = m;
    dataCopyA1Params.dValue = k;
    dataCopyA1Params.srcNdMatrixStride = 0;
    dataCopyA1Params.srcDValue = k;
    dataCopyA1Params.dstNzC0Stride = m;
    dataCopyA1Params.dstNzNStride = 1;
    dataCopyA1Params.dstNzMatrixStride = 0;
    DataCopy(a1Local, aGM, dataCopyA1Params);

    Nd2NzParams dataCopyB1Params;
    dataCopyB1Params.ndNum = 1;
    dataCopyB1Params.nValue = k;
    dataCopyB1Params.dValue = n;
    dataCopyB1Params.srcNdMatrixStride = 0;
    dataCopyB1Params.srcDValue = n;
    dataCopyB1Params.dstNzC0Stride = k;
    dataCopyB1Params.dstNzNStride = 1;
    dataCopyB1Params.dstNzMatrixStride = 0;
    DataCopy(b1Local, bGM, dataCopyB1Params);

    inQueueA1.EnQue<half>(a1Local);
    inQueueB1.EnQue<half>(b1Local);
}
__aicore__ inline void SplitA()
{
    ...
}
__aicore__ inline void SplitB()
{
    ...
}
__aicore__ inline void Compute()
```

```
{
    LocalTensor<half> a2Local = inQueueA2.DeQue<half>();
    LocalTensor<half> b2Local = inQueueB2.DeQue<half>();
    LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();
    MmadParams mmadParams;
    mmadParams.m = m;
    mmadParams.n = n;
    mmadParams.k = k;
    // 矩阵乘
    Mmad(c1Local, a2Local, b2Local, mmadParams);
    outQueueCO1.EnQue<float>(c1Local);
    inQueueA2.EnQue<half>(a2Local);
    inQueueB2.EnQue<half>(b2Local);
}
__aicore__ inline void CopyOut()
{
    LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    GM_ADDR usrWorkspace = AscendC::GetUserWorkspace(workspace);
    xGm.SetGlobalBuffer((__gm__ float *) (usrWorkspace));
    FixpipeParamsV220 fixpipeParams;
    fixpipeParams.nSize = n;
    fixpipeParams.mSize = m;
    fixpipeParams.srcStride = m;
    fixpipeParams.dstStride = n;
    fixpipeParams.ndNum = 1;
    fixpipeParams.srcNdStride = 0;
    fixpipeParams.dstNdStride = 0;
    // 将矩阵乘的计算结果从CO1搬运到workspace
    Fixpipe(xGm, c1Local, fixpipeParams);
    outQueueCO1.EnQue<float>(c1Local);
}
__aicore__ inline void CopyIn1()
{
    LocalTensor<float> src0Local = inQueueSrc0.AllocTensor<float>();
    // 将矩阵乘的计算结果从workspace搬运到UB
    DataCopy(src0Local, xGm, cSize);
    inQueueSrc0.EnQue<float>(src0Local);
}
__aicore__ inline void Compute1()
{
    LocalTensor<half> a2Local = inQueueA2.DeQue<half>();
    LocalTensor<half> b2Local = inQueueB2.DeQue<half>();
    LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    MmadParams mmadParams;
    mmadParams.m = m;
    mmadParams.n = n;
    mmadParams.k = k;
    // 矩阵乘
    Mmad(c1Local, a2Local, b2Local, mmadParams);
    outQueueCO1.EnQue<float>(c1Local);
    inQueueA2.FreeTensor(a2Local);
    inQueueB2.FreeTensor(b2Local);
}
__aicore__ inline void CopyOut1()
{
    LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    FixpipeParamsV220 fixpipeParams;
    fixpipeParams.nSize = n;
    fixpipeParams.mSize = m;
    fixpipeParams.srcStride = m;
    fixpipeParams.dstStride = n;
    fixpipeParams.ndNum = 1;
    fixpipeParams.srcNdStride = 0;
    fixpipeParams.dstNdStride = 0;
    // 将矩阵乘的计算结果从CO1搬运到workspace
    Fixpipe(xGm, c1Local, fixpipeParams);
    outQueueCO1.FreeTensor(c1Local);
}
__aicore__ inline void CopyIn2()
```

```
{
    PipeBarrier<PIPE_ALL>();
    LocalTensor<float> src1Local = inQueueSrc1.AllocTensor<float>();
    // 将矩阵乘的计算结果从workspace搬运到UB
    DataCopy(src1Local, xGm, cSize);
    inQueueSrc1.EnQue<float>(src1Local);
}
__aicore__ inline void Compute2()
{
    LocalTensor<float> src0Local = inQueueSrc0.DeQue<float>();
    LocalTensor<float> src1Local = inQueueSrc1.DeQue<float>();
    LocalTensor<float> dstLocal = outQueueDst.AllocTensor<float>();
    // 两次矩阵乘的结果相加
    Add(dstLocal, src0Local, src1Local, cSize);
    outQueueDst.EnQue<float>(dstLocal);
    inQueueSrc0.FreeTensor(src0Local);
    inQueueSrc1.FreeTensor(src1Local);
}
__aicore__ inline void CopyOut2()
{
    ...
}
private:
    TPipe pipe;
    TQue<TPosition::A1, 1> inQueueA1;
    TQue<TPosition::A2, 1> inQueueA2;
    TQue<TPosition::B1, 1> inQueueB1;
    TQue<TPosition::B2, 1> inQueueB2;
    TQue<TPosition::CO1, 1> outQueueCO1;
    TQue<TPosition::VECIN, 1> inQueueSrc0;
    TQue<TPosition::VECIN, 1> inQueueSrc1;
    TQue<TPosition::VECOU, 1> outQueueDst;

    GlobalTensor<half> aGM;
    GlobalTensor<half> bGM;
    GlobalTensor<float> cGM;
    uint16_t m = 32, k = 32, n = 32;
    uint16_t aSize, bSize, cSize;
...
}
```

【正例】

通过优化，算子对矩阵乘结果累加时，可将前一次矩阵乘的结果暂存在L0C上，通过Mmad接口参数cmatrixInitVal和cmatrixSource配置C矩阵的初始值，只调用2次Mmad接口实现2次矩阵乘结果累加。

```
...
// 该样例仅做示例说明，非完整代码，省略了部分同步控制代码
public:
    __aicore__ inline KernelSample()
    {
        aSize = m * k;
        bSize = k * n;
        cSize = m * n;
    }
    __aicore__ inline void Init(__gm__ uint8_t *a, __gm__ uint8_t *b, __gm__ uint8_t *c)
    {
        aGM.SetGlobalBuffer((__gm__ half *)a);
        bGM.SetGlobalBuffer((__gm__ half *)b);
        cGM.SetGlobalBuffer((__gm__ float *)c);
        pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
        pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half));
        pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));
    }
    __aicore__ inline void Process()
    {
        CopyIn();
    }
}
```

```
SplitA();
SplitB();
Compute();
CopyOut();
}
private:
__aicore__ inline void CopyIn()
{
    LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
    LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();

    Nd2NzParams dataCopyA1Params;
    dataCopyA1Params.ndNum = 1;
    dataCopyA1Params.nValue = m;
    dataCopyA1Params.dValue = k;
    dataCopyA1Params.srcNdMatrixStride = 0;
    dataCopyA1Params.srcDValue = k;
    dataCopyA1Params.dstNzC0Stride = m;
    dataCopyA1Params.dstNzNStride = 1;
    dataCopyA1Params.dstNzMatrixStride = 0;
    DataCopy(a1Local, aGM, dataCopyA1Params);

    Nd2NzParams dataCopyB1Params;
    dataCopyB1Params.ndNum = 1;
    dataCopyB1Params.nValue = k;
    dataCopyB1Params.dValue = n;
    dataCopyB1Params.srcNdMatrixStride = 0;
    dataCopyB1Params.srcDValue = n;
    dataCopyB1Params.dstNzC0Stride = k;
    dataCopyB1Params.dstNzNStride = 1;
    dataCopyB1Params.dstNzMatrixStride = 0;
    DataCopy(b1Local, bGM, dataCopyB1Params);

    inQueueA1.EnQue(a1Local);
    inQueueB1.EnQue(b1Local);
}
__aicore__ inline void SplitA()
{
    ...
}
__aicore__ inline void SplitB()
{
    ...
}
__aicore__ inline void Compute()
{
    LocalTensor<half> a2Local = inQueueA2.DeQue<half>();
    LocalTensor<half> b2Local = inQueueB2.DeQue<half>();
    LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();
    MmadParams mmadParams;
    mmadParams.m = m;
    mmadParams.n = n;
    mmadParams.k = k;
    // 第一次矩阵乘
    Mmad(c1Local, a2Local, b2Local, mmadParams);
    PipeBarrier<PIPE_M>();
    // 第二次矩阵乘累加第一次矩阵乘的结果
    mmadParams.cmatrixInitVal = false;
    Mmad(c1Local, a2Local, b2Local, c1Local, mmadParams);
    outQueueCO1.EnQue<float>(c1Local);
    inQueueA2.FreeTensor(a2Local);
    inQueueB2.FreeTensor(b2Local);
}
__aicore__ inline void CopyOut()
{
    LocalTensor<float> c1Local = outQueueCO1.DeQue<float>();
    FixpipeParamsV220 fixpipeParams;
    fixpipeParams.nSize = n;
    fixpipeParams.mSize = m;
```

```
fixpipeParams.srcStride = m;  
fixpipeParams.dstStride = n;  
  
fixpipeParams.ndNum = 1;  
fixpipeParams.srcNdStride = 0;  
fixpipeParams.dstNdStride = 0;  
Fixpipe(cGM, c1Local, fixpipeParams);  
outQueueCO1.FreeTensor(c1Local);  
}  
private:  
    TPipe pipe;  
    TQue<TPosition::A1, 1> inQueueA1;  
    TQue<TPosition::A2, 1> inQueueA2;  
    TQue<TPosition::B1, 1> inQueueB1;  
    TQue<TPosition::B2, 1> inQueueB2;  
    TQue<TPosition::CO1, 1> outQueueCO1;  
  
    GlobalTensor<half> aGM;  
    GlobalTensor<half> bGM;  
    GlobalTensor<dst_T> cGM;  
    uint16_t m = 32, k = 32, n = 32;  
    uint16_t aSize, bSize, cSize;
```

3.6.7.4 较小矩阵长驻 L1 Buffer，仅分次搬运较大矩阵

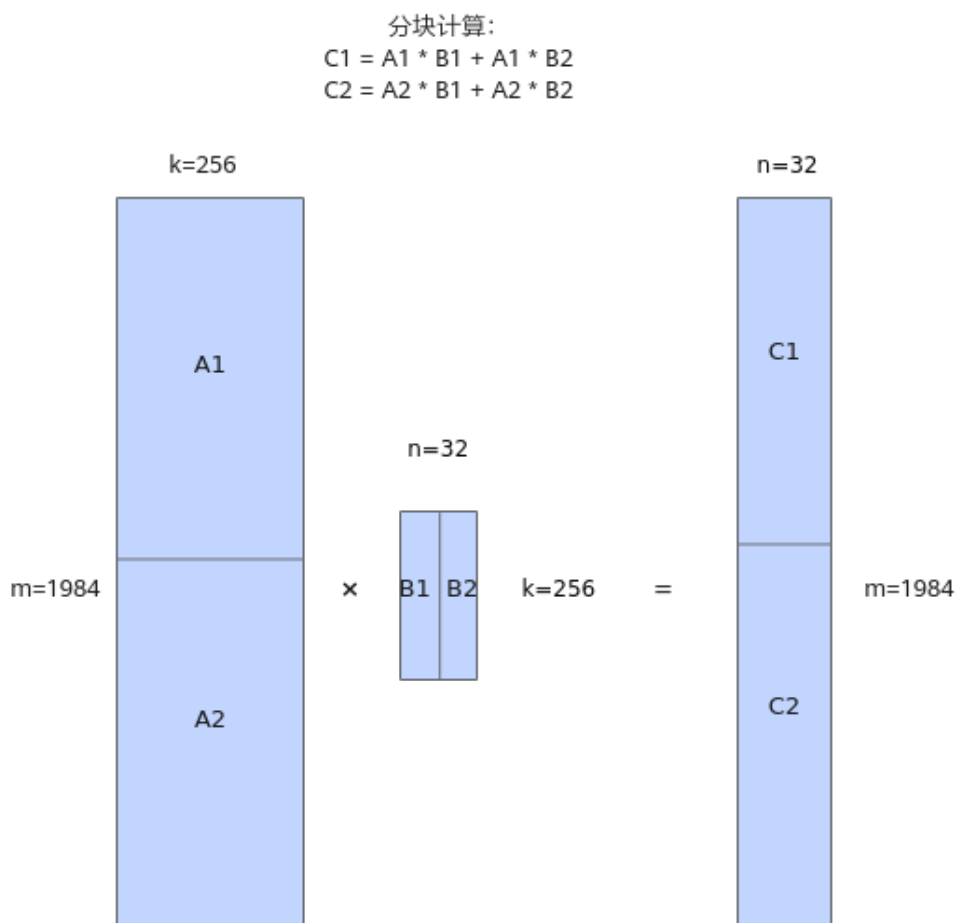
【优先级】高

【描述】在进行cube计算时，当L1无法全载左右矩阵时，可以让较小的矩阵长驻于L1上，只分次搬运较大的矩阵，减少搬运次数。

【反例】

假设L1的大小为512K，左矩阵和右矩阵的大小分别为992K、16K，数据类型为half，单次无法将左右矩阵全部载入L1中。开发者规划的切分策略为：不切K轴，将左矩阵平均分成两块A1、A2，shape大小均为[992, 256]；将右矩阵平均分成两块，shape大小均为[256, 16]。计算时的加载顺序如下：先加载A1矩阵至L1，将B1、B2依次加载并计算；然后再加载A2至L1，将B1、B2依次加载并计算。

图 3-111 反例切分策略图示



```
...
public:
__aicore__ inline KernelSample()
{
    aSize = baseM * baseK;
    bSize = baseK * baseN;
    cSize = m * n;
}
__aicore__ inline void Init(__gm__ uint8_t *a, __gm__ uint8_t *b, __gm__ uint8_t *c)
{
    aGM.SetGlobalBuffer((__gm__ half *)a);
    bGM.SetGlobalBuffer((__gm__ half *)b);
    cGM.SetGlobalBuffer((__gm__ float *)c);
    pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));
    pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));
    pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
    pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half));
    pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));
}
__aicore__ inline void Process()
{
    for (uint32_t i = 0; i < 2; i++) {
        CopyInA1(i);
        SplitA();
        for (uint32_t j = 0; j < 2; j++) {
            CopyInB1(j);
            SplitB();
            Compute(i, j);
        }
    }
}
```



```
    }  
    CopyOut();  
}  
private:  
__aicore__ inline void CopyInA1(uint32_t i)  
{  
    LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();  
    // 左矩阵a1/a2分块载入A1  
    Nd2NzParams dataCopyA1Params;  
    dataCopyA1Params.ndNum = 1;  
    dataCopyA1Params.nValue = baseM;  
    dataCopyA1Params.dValue = baseK;  
    dataCopyA1Params.srcNdMatrixStride = 0;  
    dataCopyA1Params.srcDValue = baseK;  
    dataCopyA1Params.dstNzC0Stride = baseM;  
    dataCopyA1Params.dstNzNStride = 1;  
    dataCopyA1Params.dstNzMatrixStride = 0;  
    DataCopy(a1Local, aGM[i * baseM * baseK], dataCopyA1Params);  
    inQueueA1.Enqueue(a1Local);  
}  
__aicore__ inline void SplitA()  
{  
    LocalTensor<half> a1Local = inQueueA1.DeQue<half>();  
    LocalTensor<half> a2Local = inQueueA2.AllocTensor<half>();  
    // 左矩阵a1/a2分块从A1->A2  
    LoadData2dParams loadL0AParams;  
    loadL0AParams.repeatTimes = baseM * baseK * sizeof(half) / 512;  
    loadL0AParams.srcStride = 1;  
    loadL0AParams.dstGap = 0;  
    LoadData(a2Local, a1Local, loadL0AParams);  
    inQueueA2.Enqueue(a2Local);  
    inQueueA1.FreeTensor(a1Local);  
}  
__aicore__ inline void CopyInB1(uint32_t j)  
{  
    LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();  
    // 右矩阵分块b1/b2载入B1  
    Nd2NzParams dataCopyB1Params;  
    dataCopyB1Params.ndNum = 1;  
    dataCopyB1Params.nValue = baseK;  
    dataCopyB1Params.dValue = baseN;  
    dataCopyB1Params.srcNdMatrixStride = 0;  
    dataCopyB1Params.srcDValue = n;  
    dataCopyB1Params.dstNzC0Stride = baseK;  
    dataCopyB1Params.dstNzNStride = 1;  
    dataCopyB1Params.dstNzMatrixStride = 0;  
    DataCopy(b1Local, bGM[j * baseN], dataCopyB1Params);  
    inQueueB1.Enqueue(b1Local);  
}  
__aicore__ inline void SplitB()  
{  
    LocalTensor<half> b1Local = inQueueB1.DeQue<half>();  
    LocalTensor<half> b2Local = inQueueB2.AllocTensor<half>();  
    // 右矩阵分块b1/b2从B1->B2  
    LoadData2dTransposeParams loadL0BParams;  
    loadL0BParams.startIndex = 0;  
    loadL0BParams.repeatTimes = baseK / nBlockSize;  
    loadL0BParams.srcStride = 1;  
    loadL0BParams.dstGap = 1;  
    LoadDataWithTranspose(b2Local, b1Local, loadL0BParams);  
    inQueueB2.Enqueue(b2Local);  
    inQueueB1.FreeTensor(b1Local);  
}  
__aicore__ inline void Compute(uint32_t i, uint32_t j)  
{  
    LocalTensor<half> a2Local = inQueueA2.DeQue<half>();  
    LocalTensor<half> b2Local = inQueueB2.DeQue<half>();  
    LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();  
    // 矩阵乘
```

```
        mmadParams.m = baseM;
        mmadParams.n = baseN;
        mmadParams.k = baseK;
        Mmad(c1Local[i * baseM * baseN + j * m * baseN], a2Local, b2Local, mmadParams);
        outQueueCO1.Enqueue<float>(c1Local);
        inQueueA2.FreeTensor(a2Local);
        inQueueB2.FreeTensor(b2Local);
    }
    __aicore__ inline void CopyOut()
    {
        ...
    }
private:
    TPipe pipe;
    TQue<TPosition::A1, 1> inQueueA1;
    TQue<TPosition::A2, 1> inQueueA2;
    TQue<TPosition::B1, 1> inQueueB1;
    TQue<TPosition::B2, 1> inQueueB2;
    TQue<TPosition::CO1, 1> outQueueCO1;

    GlobalTensor<half> aGM;
    GlobalTensor<half> bGM;
    GlobalTensor<float> cGM;
    uint16_t m = 1984, k = 256, n = 32;
    uint16_t baseM = 992, baseK = 256, baseN = 16;
    uint16_t aSize, bSize, cSize;
    uint16_t nBlockSize = 16;
...

```

【正例】

该示例中，将较小的右矩阵一次性搬入L1并长存于L1上，循环内不断搬运A矩阵，当循环次数为2时，共需要3次搬运。

```
...
public:
    __aicore__ inline KernelSample()
    {
        aSize = baseM * baseK;
        bSize = baseK * n;
        cSize = m * n;
    }
    __aicore__ inline void Init(__gm__ uint8_t *a, __gm__ uint8_t *b, __gm__ uint8_t *c)
    {
        aGM.SetGlobalBuffer((__gm__ half *)a);
        bGM.SetGlobalBuffer((__gm__ half *)b);
        cGM.SetGlobalBuffer((__gm__ float *)c);
        pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));
        pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
        pipe.InitBuffer(inQueueB2, 2, bSize * sizeof(half));
        pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));
    }
    __aicore__ inline void Process()
    {
        CopyInB1();
        SplitB();
        for (uint32_t i = 0; i < 2; i++) {
            CopyInA1(i);
            SplitA();
            for (uint32_t j = 0; j < 2; j++) {
                Compute(i, j);
            }
        }
        CopyOut();
    }
private:
    __aicore__ inline void CopyInB1()
    {
        LocalTensor<half> b1Local = inQueueB1.AllocTensor<half>();
    }

```

```
// 右矩阵全载入B1
Nd2NzParams dataCopyB1Params;
dataCopyB1Params.ndNum = 1;
dataCopyB1Params.nValue = baseK;
dataCopyB1Params.dValue = n;
dataCopyB1Params.srcNdMatrixStride = 0;
dataCopyB1Params.srcDValue = n;
dataCopyB1Params.dstNzC0Stride = baseK;
dataCopyB1Params.dstNzNStride = 1;
dataCopyB1Params.dstNzMatrixStride = 0;
DataCopy(b1Local, bGM, dataCopyB1Params);
inQueueB1.Enqueue(b1Local);
}
__aicore__ inline void SplitB()
{
    LocalTensor<half> b1Local = inQueueB1.DeQueue<half>();
    LocalTensor<half> b2Local = inQueueB2.AllocTensor<half>();
    // 右矩阵全部从B1->B2
    LoadData2dTransposeParams loadLOBParams;
    loadLOBParams.startIndex = 0;
    loadLOBParams.repeatTimes = baseK / nBlockSize;
    loadLOBParams.srcStride = 1;
    loadLOBParams.dstGap = 1;
    for (int blockNum = 0; blockNum < (n / nBlockSize); blockNum++) {
        LoadDataWithTranspose(b2Local[blockNum * 16 * nBlockSize], b1Local[blockNum * baseK *
nBlockSize], loadLOBParams);
    }
    inQueueB2.Enqueue(b2Local);
    inQueueB1.FreeTensor(b1Local);
}
__aicore__ inline void CopyInA1(uint32_t i)
{
    LocalTensor<half> a1Local = inQueueA1.AllocTensor<half>();
    // 左矩阵a1/a2分块载入A1
    Nd2NzParams dataCopyA1Params;
    dataCopyA1Params.ndNum = 1;
    dataCopyA1Params.nValue = baseM;
    dataCopyA1Params.dValue = baseK;
    dataCopyA1Params.srcNdMatrixStride = 0;
    dataCopyA1Params.srcDValue = baseK;
    dataCopyA1Params.dstNzC0Stride = baseM;
    dataCopyA1Params.dstNzNStride = 1;
    dataCopyA1Params.dstNzMatrixStride = 0;
    DataCopy(a1Local, aGM[i * baseM * baseK], dataCopyA1Params);
    inQueueA1.Enqueue(a1Local);
}
__aicore__ inline void SplitA()
{
    LocalTensor<half> a1Local = inQueueA1.DeQueue<half>();
    LocalTensor<half> a2Local = inQueueA2.AllocTensor<half>();
    // 左矩阵a1/a2分块从A1->A2
    LoadData2dParams loadLOAParams;
    loadLOAParams.repeatTimes = baseM * baseK * sizeof(half) / 512;
    loadLOAParams.srcStride = 1;
    loadLOAParams.dstGap = 0;
    LoadData(a2Local, a1Local, loadLOAParams);
    inQueueA2.Enqueue(a2Local);
    inQueueA1.FreeTensor(a1Local);
}
__aicore__ inline void Compute(uint32_t i, uint32_t j)
{
    LocalTensor<half> a2Local = inQueueA2.DeQueue<half>();
    LocalTensor<half> b2Local = inQueueB2.DeQueue<half>();
    LocalTensor<float> c1Local = outQueueCO1.AllocTensor<float>();
    // 矩阵乘
    mmadParams.m = baseM;
    mmadParams.n = baseN;
    mmadParams.k = baseK;
    Mmad(c1Local[i * baseM * baseN + j * m * baseN], a2Local, b2Local, mmadParams);
}
```

```
    outQueueCO1.Enqueue<float>(c1Local);
    inQueueA2.FreeTensor(a2Local);
    inQueueB2.FreeTensor(b2Local);
}
__aicore__ inline void CopyOut()
{
    ...
}
private:
    TPipe pipe;
    TQue<TPosition::A1, 1> inQueueA1;
    TQue<TPosition::A2, 1> inQueueA2;
    TQue<TPosition::B1, 1> inQueueB1;
    TQue<TPosition::B2, 1> inQueueB2;
    TQue<TPosition::CO1, 1> outQueueCO1;

    GlobalTensor<half> aGM;
    GlobalTensor<half> bGM;
    GlobalTensor<float> cGM;
    uint16_t m = 1984, k = 256, n = 32;
    uint16_t baseM = 992, baseK = 256, baseN = 16;
    uint16_t aSize, bSize, cSize;
    uint16_t nBlockSize = 16;
    ...
```

3.6.7.5 Matmul 使能 AtomicAdd 选项

【优先级】中

【描述】对于Matmul得到的结果矩阵C(m, n)，若后续需要和GM上的矩阵D(m, n)进行Add操作，则可以在GetTensorC接口或者IterateAll接口的GM通路上，将enAtomic参数设为1，开启AtomicAdd累加操作，在搬出矩阵C到GM时，矩阵C的结果将直接累加到矩阵D的GM地址上，从而实现与矩阵D的Add操作。

【反例】

将Matmul的结果矩阵C和GM上的矩阵D分别搬到UB上，做完Add操作后，结果再搬出到GM。这样至少要多分配一块UB内存给矩阵D，假设在分离架构的处理器上执行，将多做三次搬运操作（矩阵C从GM搬到UB、矩阵D从GM搬到UB、Add结果从UB搬出到GM）。

```
template <class A_TYPE, class B_TYPE, class C_TYPE, class BIAS_TYPE>
__aicore__ inline void MatMulKernel(...)
{
    ...
    AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_MDL> mm;
    TPipe pipe;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm);

    mm.SetTensorA(gm_a);
    mm.SetTensorB(gm_b);
    mm.SetBias(gm_bias);
    mm.IterateAll(gm_c);

    DataCopy(local_c, gm_c, c_size);
    DataCopy(local_d, gm_d, d_size);
    event_t eventIdMTE2ToV = static_cast<event_t>(GetTPipePtr()->FetchEventID(HardEvent::MTE2_V));
    SetFlag<HardEvent::MTE2_V>(eventIdMTE2ToV);
    WaitFlag<HardEvent::MTE2_V>(eventIdMTE2ToV);
    Add(local_d, local_d, local_c, d_size);
    DataCopy(gm_d, local_d, d_size);
    ...
}

extern "C" __global__ __aicore__ void example_kernel(...)
{
    ...
}
```

```
typedef AscendC::MatmulType<TPosition::GM, CubeFormat::ND, half> aType;  
typedef AscendC::MatmulType<TPosition::GM, CubeFormat::ND, half> bType;  
typedef AscendC::MatmulType<TPosition::GM, CubeFormat::ND, float> cType;  
typedef AscendC::MatmulType<TPosition::GM, CubeFormat::ND, float> biasType;  
MatMulKernel<aType, bType, cType, biasType>(...);  
...  
}
```

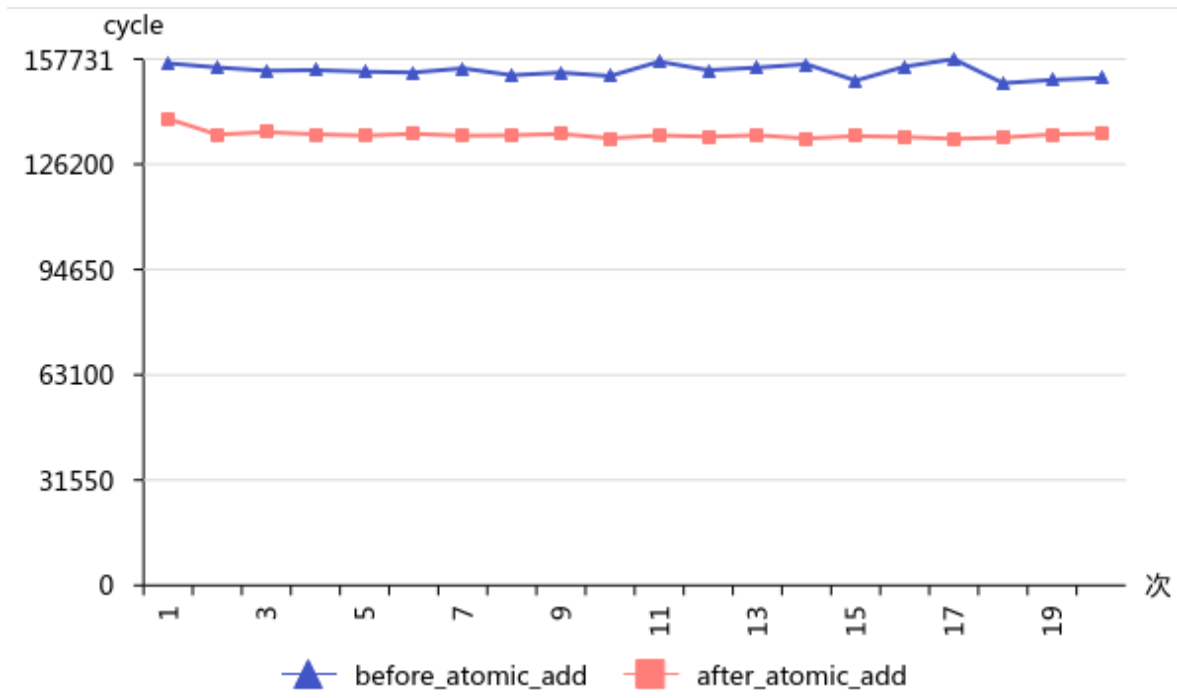
【正例】

计算Matmul结果时，调用IterateAll接口或者GetTensorC接口搬运到矩阵D的GM地址上，同时将接口中enAtomic参数设为1，搬出到GM时，Matmul结果矩阵C会累加到矩阵D上，从而得到两个矩阵Add后的结果。

```
template <class A_TYPE, class B_TYPE, class C_TYPE, class BIAS_TYPE>  
__aicore__ inline void MatMulKernel(...)  
{  
    ...  
    AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_MDL> mm;  
    TPipe pipe;  
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm);  
  
    mm.SetTensorA(gm_a);  
    mm.SetTensorB(gm_b);  
    mm.SetBias(gm_bias);  
  
    mm.IterateAll(gm_d, 1); // IterateAll接口中的enAtomic设为1  
    // while (mm.Iterate()) {  
    //     mm.GetTensorC(gm_d, 1); // GetTensorC接口中的enAtomic设为1  
    // }  
    ...  
}  
  
extern "C" __global__ __aicore__ void example_kernel(...)  
{  
    ...  
    typedef AscendC::MatmulType<TPosition::GM, CubeFormat::ND, half> aType;  
    typedef AscendC::MatmulType<TPosition::GM, CubeFormat::ND, half> bType;  
    typedef AscendC::MatmulType<TPosition::GM, CubeFormat::ND, float> cType;  
    typedef AscendC::MatmulType<TPosition::GM, CubeFormat::ND, float> biasType;  
    MatMulKernel<aType, bType, cType, biasType>(...);  
    ...  
}
```

【性能对比】

图 3-112 Matmul 使能 AtomicAdd 选项前后性能对比



以矩阵维度M=64, N=256, K=256, 矩阵D为(64, 256)为例, Matmul使能AtomicAdd选项前后的性能对比如上图所示, 平均cycle数从开启AtomicAdd选项前的154181变为开启后的135054, 性能优化12.4%。因此在这种场景下, 使能AtomicAdd选项能获取更优的性能。

3.7 优秀实践

3.7.1 FlashAttention 算子性能调优案例

案例介绍

本案例中的算子FlashAttentionScoreGrad, 用于训练场景下计算注意力的反向输出, 即FlashAttentionScore算子的反向计算。

已知注意力的正向计算公式为:

$$Y = \text{Dropout}(\text{Softmax}(\text{Mask}(\frac{QK^T + pse}{\sqrt{d}}), \text{atten_mask}), \text{keep_prob})V$$

为方便表达, 以变量S和P表示计算公式:

$$S = \text{Mask}(\frac{QK^T + pse}{\sqrt{d}}, \text{atten_mask})$$

$$P = \text{Dropout}(\text{Softmax}(S), \text{keep_prob})$$

$$Y = PV$$

则注意力的反向计算公式为：

$$dV = P^T dY$$

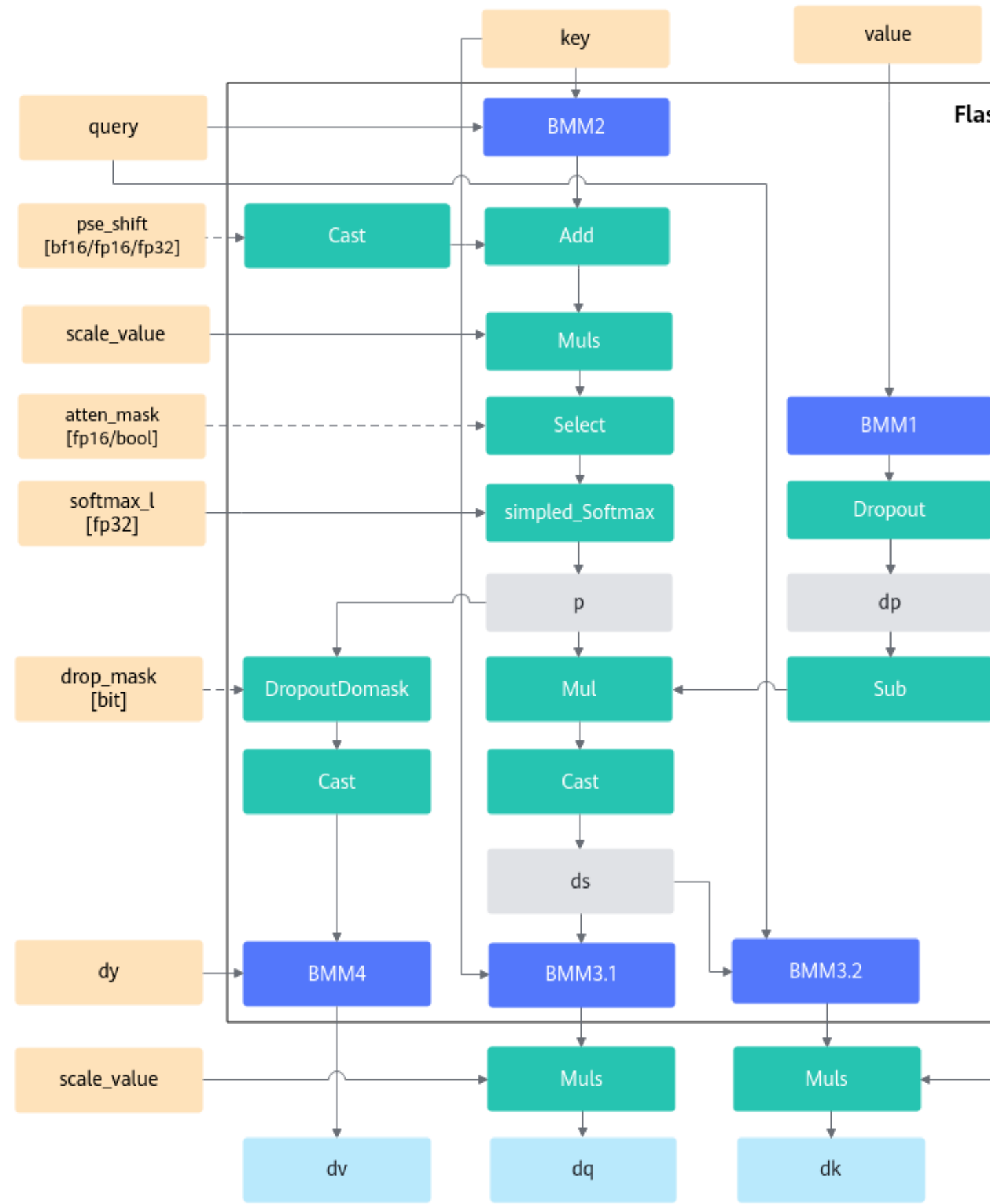
$$dQ = \frac{((dS) * K)}{\sqrt{d}}$$

$$dK = \frac{((dS)^T * Q)}{\sqrt{d}}$$

$$d(pse) = dS * \sqrt{d}$$

计算流程图如下：

图 3-113 算子计算流程



按照FlashAttention反向计算流程的实现，简介整体计算流程如下。对本算子的算法感兴趣的用户可简单了解，无需重点关注。

1. 重计算p，本步骤重计算了FlashAttention流程中的softmax结果p，计算结果保存在ub中。

$$p = \text{SimplifiedSoftmax}(\text{Mask}(\text{Matmul}(\text{query}, \text{key}^T) + \text{pse}) * \text{scale})$$

2. 计算dp，该计算包含matmul计算和dropout计算，matmul计算中，左矩阵为dy，右矩阵为转置后的value。

$$dp = \text{Dropout}(\text{Matmul}(dy, \text{value}^T))$$

3. 计算ds，本计算中，FlashSoftmaxGrad计算的入参为dy、正向输出attention_in，该结果与dp做减操作，最终的结果与p相乘得到结果ds。

$$ds = p * \text{Sub}(dp, \text{FlashSoftmaxGrad}(dy, \text{attention_in}))$$

4. 计算dq，本计算将ds结果与key做matmul计算，并将结果与scale相乘得到结果dq。

$$dq = \text{Matmul}(ds, \text{key}) * \text{scale}$$

5. 计算dk，本计算将转置后的ds结果与query做matmul计算，并将结果与scale相乘得到结果dk。

$$dk = \text{Matmul}(ds^T, \text{query}) * \text{scale}$$

6. 计算dv，本计算将p的结果做drop计算，转置后与dy做matmul计算。

$$dv = \text{Matmul}(\text{DropOut}(p)^T, dy)$$

本案例的验证平台为Atlas A2 训练系列产品/Atlas A2 推理系列产品，以两个场景为例，第一个场景的输入维度信息为：B=1，N1=12，N2=12，S1=6144，S2=6144，D=128，causal场景，即atten_mask的形状为下三角，如[图3-114](#)。第二个场景的输入维度信息为：B=24，N1=5，N2=5，S1=9216，S2=9216，D=64，不带atten_mask和drop_mask输入。主要涉及的优化手段包括tiling基本块大小调整，核间负载均衡，CV流水并行，MTE2流水优化以及FixPipe流水优化等优化手段。

图 3-114 causal 场景 atten_mask 形状

0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	1
0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	0	1	1	1
0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0

获取性能数据

流水优化分析工具包括CAModel和Profiling工具，分别从两个方面分析：第一个是从Profiling工具生成的Profiling数据中分析各项流水的占比，第二个是从CAModel工具生成的打点图分析各流水并行情况。

分析主要瓶颈点

通过观察分析流水图和Profiling数据，结合优化经验来判断性能瓶颈点。在优化过程中不同阶段可能会出现不同的瓶颈点，需要不断优化以达到最佳性能。

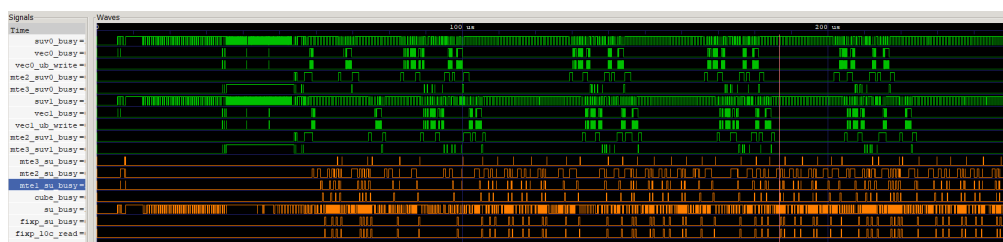
- 根据优化经验，循环间会存在一些不必要的性能开销，循环越多性能可能越差；满足UB最大空间限制的情况下，UB切分的基本块越大，循环越少。算子中通过InitBuffer接口分配UB buffer大小。

```
pipe->InitBuffer(ubBuffer, 120 * 1024);
pipe->InitBuffer(tmpBuffer, 30 * 1024);
pipe->InitBuffer(vecClc3, 8 * 1024);
```

如上代码所示，InitBuffer接口的第二个参数表示buffer占用的大小，所有buffer大小的和即为占用的总空间。这里 $120 * 1024 + 30 * 1024 + 8 * 1024 = 158KB < UB\ Size$ ，没有充分利用UB空间。

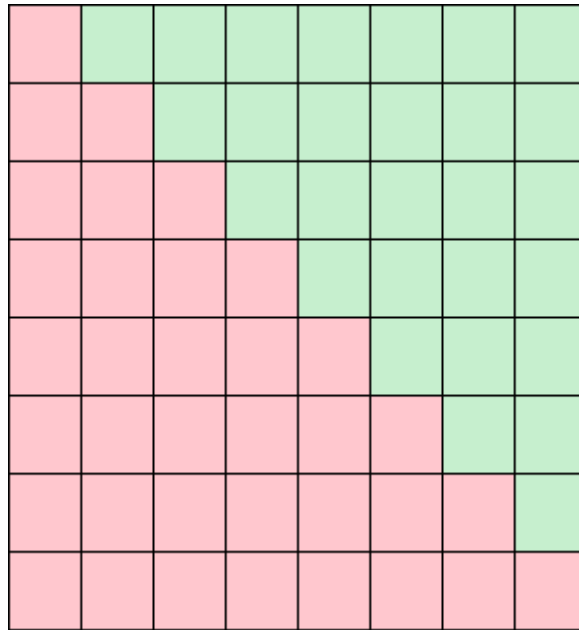
- 观察如下流水图，绿色为Vector侧的流水，橙色为Cube侧的流水，可以看出两侧的流水都存在大段的空隙，CV之间流水很大程度上未并行，需要考虑CV流水优化。

图 3-115 优化前算子流水



- 对于上述场景一，causal场景下可能存在核间分布不均匀的情况，如下图经过 `atten_mask` 掩码后，红色部分是算子需要计算的部分，绿色无需计算；如果不按照基本块的个数来分核，按照第一根轴的大小8（行）来分核，假设平均分到9个核上，每个核做 $\text{ceil}(8 / 9) = 1$ 行，则第1个核只需做1个基本块，但是第8个核需要做8个基本块的计算，出现明显的负载不均衡。因此需要考虑将红色块均匀分到多个核上计算。

图 3-116 causal 场景 atten_mask 形状



- 场景一的Profiling数据如下，aic_fixpipe_ratio占比极高，可能存在FixPipe bound。

图 3-117 场景一 Profiling 数据

[illegible]

- 场景二的Profiling数据如下，mte2_ratio占比高，可能存在MTE2 bound。

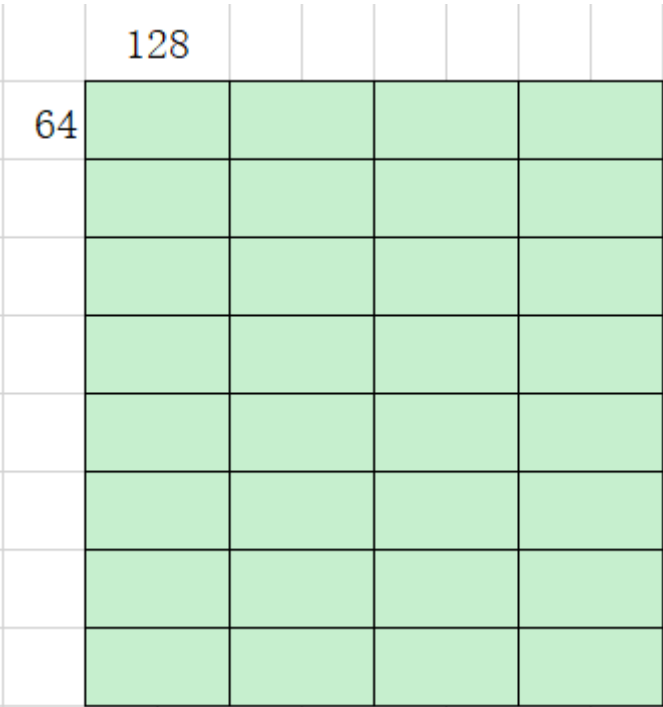
图 3-118 场景二 Profiling 数据

2	Output Sh	Output Da	Output Fo	Context ID	aic	tim	aic	total	c	aic	mac	tim	aic	mac	ra	aic	scalar	aic	scalar	aic	mt1	t	aic	mt1	aic	mt2	t	aic	mt2	ratio	aic	fixp1	aic	fixp1	ratio	aic	icache	av	time	(u)	aic	total	c	aic	vec	tim	aic	vec	ra	aic	scalar	av	scalar	av	mt2	t	aic	mt2
3	N/A	N/A	N/A	0	28782.41	1.04E+09	3326	089	0.1156	28226	0.1	0.9807	3064	068	0.1065	15044	492	0.5227	22642	57	0.7867	0.0012	28772	29	207E+09	6043	186	0.21	3085	892	0.1073	5507	29	0.191																								

设计优化方案

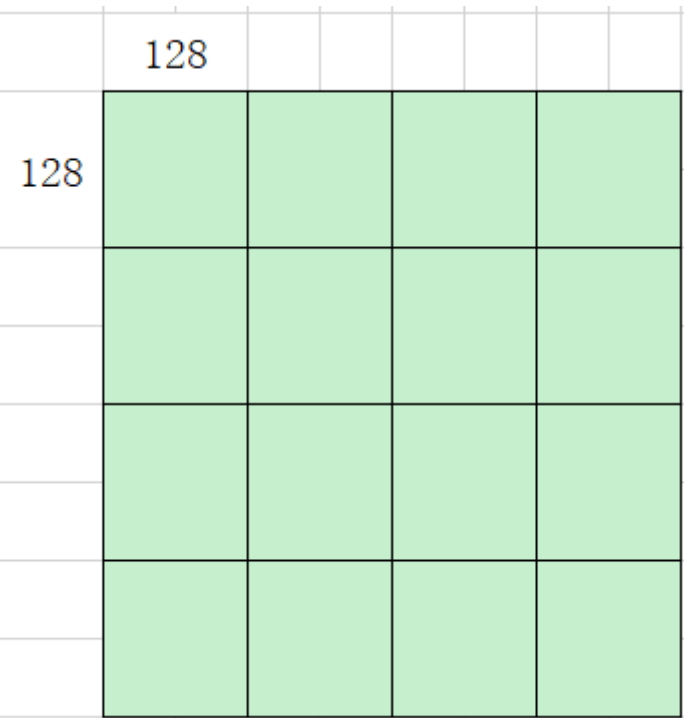
- 优化点一：调整tiling基本块
在满足UB空间大小限制的前提下，tiling基本块的切分应尽可能大。如下图为优化前按照(64, 128)切分计算，总共需要循环计算32次。

图 3-119 优化前计算基本块及次数



考虑到UB空间没有用满，基本块调整到(128, 128)，如下图优化后只需循环计算16次，切分后算子性能提升一倍。

图 3-120 优化后计算基本块及次数

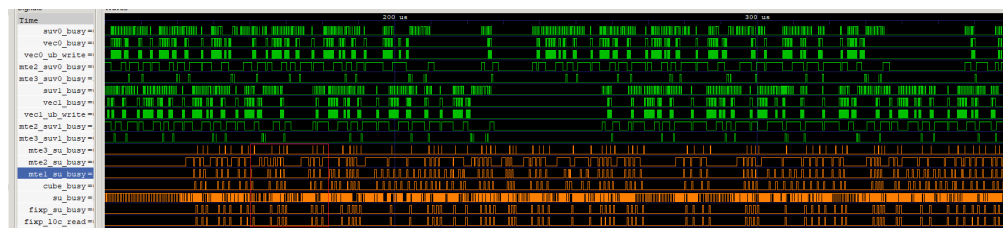


- 优化点二：CV流水并行
由于FAG算子中Cube计算比Vector计算快且存在依赖性，同时为了减少CV之间的通信次数，通过缓存机制使matmul提前计算多块，这里的缓存机制指的是将mm

一次性计算多个基本块缓存到GM上。如下代码中，SetTail设置的singleCoreM和singleCoreN大小分别为BaseM，BaseN的倍数，即matmul一次发起多个基本块的计算，实现matmul结果的缓存，Vector侧分多次取matmul的结果。

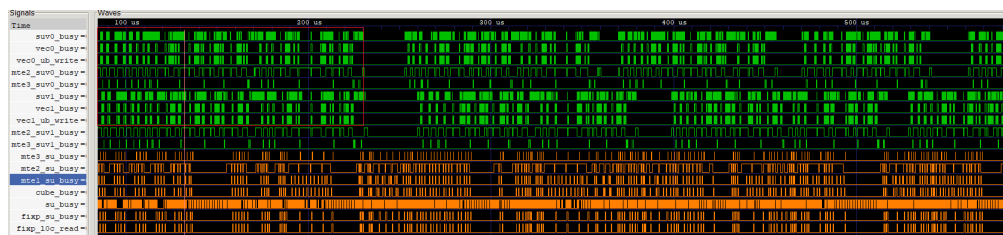
```
mm3.SetTail(s2CvExtend, -1, preS1Extend);  
mm3.SetTensorA(mulWorkSpaceGm[pingpongIdx * coreNum * cubeBaseMN + cBlockIdx *  
cubeBaseMN], true);  
mm3.SetTensorB(queryGm[mm2aTensorOffsetCv]);  
mm3.template IterateAll<false>(dkWorkSpaceGm[bTensorOffsetCv], true);
```

图 3-121 完成 mm1/mm2/mm3 缓存的流水



如上图是实现mm1、mm2和mm3缓存的流水图，并行度提高，CV的间隔减小，提升了算子性能。

图 3-122 Vector 等 Cube 流水的间隔插入 Vector 计算



基于缓存mm1/mm2/mm3的优化后，在本轮Vector计算等Cube流水的间隔，插入下一轮循环的Vector计算，如上图所示，这样使Vector流水与Cube流水之间的并行度更高，反映到流水图中为Vector计算更密集。原计算过程伪代码与在CV间隔中插入下一轮Vector计算的伪代码，分别如以下两段所示。

```
// 原计算过程伪代码  
// mm1计算;  
dropout();  
Sub();  
// mm2计算;  
Softmax();  
AttenMask();  
...  
// 在Vector等Cube流水的间隔中，插入下一轮循环的Vector计算伪代码  
// mm1计算;  
dropout();  
Sub();  
dropout(); // 下一轮循环的Vector计算  
Sub(); // 下一轮循环的Vector计算  
// mm2计算;  
Softmax();  
AttenMask();  
...
```

- 优化点三：每个核负载均衡

图 3-123 causal 场景优化前每个核计算量

0							
1	1						
2	2	2					
3	3	3	3				
4	4	4	4	4			
5	5	5	5	5	5		
6	6	6	6	6	6	6	
7	7	7	7	7	7	7	7

图 3-124 causal 场景优化后每个核计算量

0							
0	0						
0	1	1					
1	1	2	2				
2	2	3	3	3			
3	4	4	4	4	5		
5	5	5	6	6	6	6	
7	7	7	7	8	8	8	8

尽量实现每个核的计算量均匀，负载均衡。优化前的分核及每个核的计算量如图 11 causal 场景优化前每个核计算量所示，按照第一根轴的大小 8（行）来分核，平均分到 9 个核上，每个核计算 $\text{ceil}(8/9)=1$ 行，第 1 个核只计算 1 个基本块，但是第 8 个核计算 8 个基本块。优化后如图 12 causal 场景优化后每个核计算量所示，红色块总共 36 个基本块，均分到每个核上，每个核的计算量为 4 个基本块，性能提升一倍。

- 优化点四：FIXPIPE 优化

从采集的 Profiling 数据来看，Cube FixPipe 占比高达 81%，出现了很严重的 bound（达到上限）。CAModel 工具打印发现存在很多异常的 128B 搬运，排查代码，发现 workspace 地址未 512B 对齐。

图 3-125 场景一优化前 Profiling 数据

9	Context ID	aic	simd	total_c/aic	mac	b1aic	mac	r1aic	scalar	f1aic	scalar	f1aic	mte1_f1aic	mte1_r1aic	mte2_f1aic	mte2_ratio	aic_fixpipe	aic_fixpipe_ratio	aic_licache	aiy_time(us)	aiv_total_c/aiv	vec	tin	aiv_vec	rat	aiv_sclar	f1aiv	scalar	f1aiv	mte2_f1aiv	mte2_r1aiv	mte3_f1aiv	mte3_r1aiv	icache
10		0	2067376	7.44E+09	26507.62	0.1282	204910.1	0.9912	23847.1	0.1153	57073.96	0.2761	166511.6	0.8151	0.0025	206734.5	1.49E-10	55978.65	0.2708	24983.12	0.1208	38686.49	0.1871	26993.82	0.1306	0.001								

代码实现中使用SetGlobalBuffer接口设置workspace的起始地址，如果起始地址不是按照512B对齐，搬运效率会很低，可以强制**GM地址512Byte对齐**来避免这个情况。下面代码中ADDR_ALIGN_SIZE即为512。

```
// init workspace address
syncGlobal.SetGlobalBuffer((__gm__ int32_t*)workspace);
uint64_t workspaceOffsets = SYNC_GLOBAL_WORKSPACE_SIZE;
dqWorkSpaceGm.SetGlobalBuffer((__gm__ float*)workspace + workspaceOffsets / sizeof(T2));
workspaceOffsets = (workspaceOffsets + qPostBlockTotal * sizeof(float) + ADDR_ALIGN_SIZE) /
ADDR_ALIGN_SIZE * ADDR_ALIGN_SIZE; dkWorkSpaceGm.SetGlobalBuffer((__gm__ float*)workspace
+ workspaceOffsets / sizeof(T2));
workspaceOffsets = (workspaceOffsets + kvPostBlockTotal * sizeof(float) + ADDR_ALIGN_SIZE) /
ADDR_ALIGN_SIZE * ADDR_ALIGN_SIZE; dvWorkSpaceGm.SetGlobalBuffer((__gm__ float*)workspace
+ workspaceOffsets / sizeof(T2));
workspaceOffsets = (workspaceOffsets + kvPostBlockTotal * sizeof(float) + ADDR_ALIGN_SIZE) /
ADDR_ALIGN_SIZE * ADDR_ALIGN_SIZE;
// matmul1 and matmul2 workspace size
matmulWorkspaceSize = cubeBaseMN * sizeof(float);
mm1WorkspaceGm.SetGlobalBuffer((__gm__ T2*)(workspace + workspaceOffsets + cBlockIdx *
matmulWorkspaceSize)); mm2WorkspaceGm.SetGlobalBuffer((__gm__ T2*)(workspace +
workspaceOffsets + coreNum * matmulWorkspaceSize + cBlockIdx * matmulWorkspaceSize)); // drop
workspace offset
workspaceOffsets = (workspaceOffsets + coreNum * cubeBaseMN * sizeof(float) * INPUT_NUMS +
ADDR_ALIGN_SIZE) / ADDR_ALIGN_SIZE * ADDR_ALIGN_SIZE;
dropWorkSpaceGm.SetGlobalBuffer((__gm__ T1*)workspace + workspaceOffsets / sizeof(T1));
// mul workspace offset
workspaceOffsets = (workspaceOffsets + coreNum * cubeBaseMN * sizeof(half) * 2 +
ADDR_ALIGN_SIZE) / ADDR_ALIGN_SIZE * ADDR_ALIGN_SIZE;
mulWorkSpaceGm.SetGlobalBuffer((__gm__ T1*)workspace + workspaceOffsets / sizeof(T1));
```

修改代码，workspace地址经过512B对齐后，FixPipe时间减半。

图 3-126 场景一优化后 Profiling 数据

[illegible]

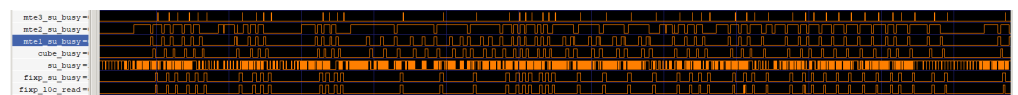
- 优化点五：MTE2优化

结合如下的Profiling数据和流水图，可以看出MTE2 bound，且部分MTE2搬运时间异常。

图 3-127 场景二 Profiling 数据

Output Sh	Output De	Output For	Context ID	alcove_tir	alc_total_c	alc_mac_tir	alc_mac_re	alc_sclar	alc_sclar	alc_mte1_t	alc_mte1	alc_mte2_t	alc_mte2_ratio	alc_fixpice	alc_fixpice_ratio	alc_icache	alc_time(us)	alc_total_c	alc_vec_tir	alc_vec_re	alc_sclar	alc_sclar	alc_mte2_t	alc_mte2
N/A	N/A	N/A	0	28782.41	1.04e+09	3326.089	0.1156	28226.01	0.9807	3064.068	0.1065	15044.92	0.6227	22642.57	0.7867	0.0012	28772.29	2.07e+09	6043.186	0.21	3085.892	0.1073	5507.29	0.191

图 3-128 场景二流水图



将输入数据排布格式从BSH更改为BNSD后，数据搬运连续，不需要跳地址读取数据，搬运效率提升一倍，部分异常搬运时长降低了一半。

验证优化方案性能收益

- 调整tiling基本块：理论评估Vector切块越大，计算和搬运循环次数越少，同时能够充分利用搬运带宽和Vector算力。基本块大小从(64, 128)增大到(128, 128)后，性能提升一倍，实测与理论分析一致。

- CV流水并行：CV流水掩盖的时间即为提升的性能，符合预期的收益。
- 核间负载均衡：优化前负载最多的核计算量减少的倍数，即为预期提升的性能；案例中优化前负载最多的核的计算量大小为8块，优化后为4块，实际性能提升一倍，符合预期的收益。
- FixPipe优化：从Profiling数据看出FixPipe占比0.8，优化后占比0.55，实测算子性能提升45%，与理论分析一致。
- MTE2优化：从Profiling数据看出MTE2占比0.52，优化后占比减少一半，实测算子性能提升30%，与理论分析一致。

总结

融合算子场景，可参考此优化。

3.7.2 GroupedMatmul 算子性能调优案例

案例介绍

本案例对分组Matmul即GroupedMatmul算子的per-token量化场景进行性能分析和优化，GroupedMatmul算子计算过程（通过python代码表达）为：

```
offset = 0
for i in range(g):
    mmOut = x[offset:offset + groupList[i]] * weight[i] + bias[i]
    y[offset:offset + groupList[i]] = Gelu(mmOut * scale[i] * pertokenScale[offset:offset + groupList[i]])
    offset += groupList[i]
```

验证平台为Atlas A2 训练系列产品/Atlas A2 推理系列产品。

优化分析以如下算子规格为例：

表 3-18 算子规格

input	shape	data type	format
x	(1024,1024)	int8	ND
weight	(8,1024,8192)	int8	NZ
bias	(8,8192)	int32	ND
groupList	8	int64	ND
scale	(8,8192)	float	ND
pertokenScale	1024	float	ND
y	(1024,8192)	float16	ND

主要介绍以下优化方法：

- 对于Vector计算占比较高（Vector Bound）的场景，将AI Core中的AIC核和AIV核启动比例设置为1:2；
- 优化CV并行流水，减少Cube和Vector计算间的空闲等待时间；

- 优化Vector计算流水，提高Vector并行计算速度。

获取性能数据

固定8核测试，即当前性能和后续优化tiling中numBlocks固定设置为8。

通过msProf算子调优工具获取算子性能数据：

- 获取真实环境执行的性能数据（指令的cycle占比数据 ArithmeticUtilization.csv ），包含各个流水的占比情况；
- 获取仿真性能数据（指令流水图），包含各个流水的占用区间，可观察流水间依赖情况，从而优化并行效率。

分析主要瓶颈点

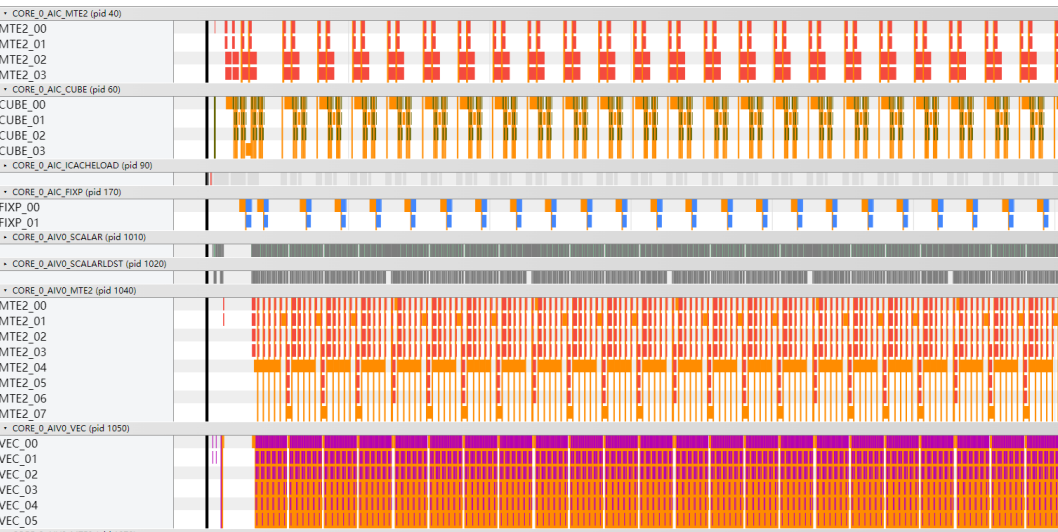
固定8核进行测试的情况下，通过msprof op命令获取指令的cycle占比数据如下：

图 3-129 指令的 cycle 占比数据 ArithmeticUtilization.csv（性能总耗时为 218.1us）

Op Name	OP Type	Task Dura	aicore_tim	aic_mte2	aic_mte2	aiv_time(u	aiv_vec_tir	aiv_vec_ra	aiv_mte2	aiv_mte2_ratio
quant_grc	quant_grc	218.064	215.38	95.262	0.442	212.04	91.333	0.431	27.016	0.127

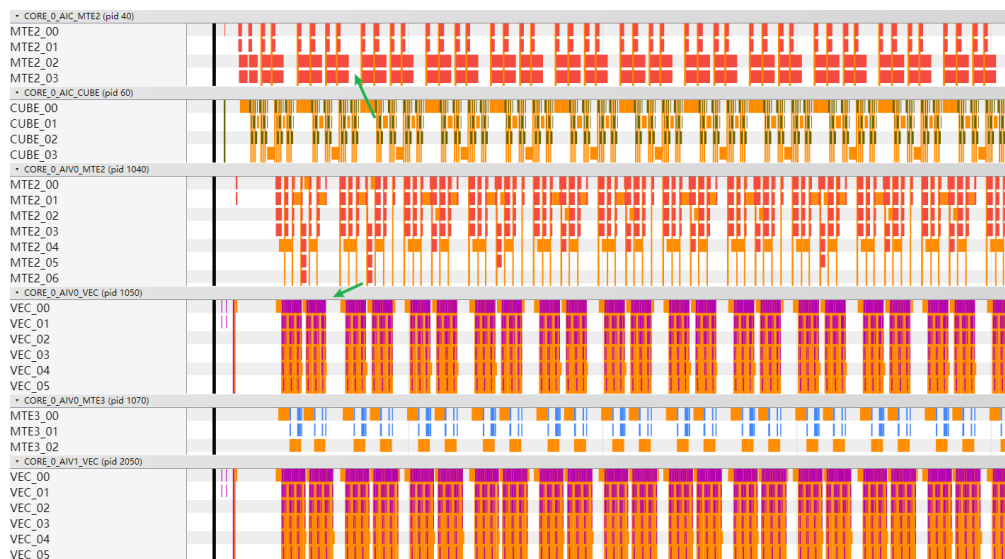
通过msprof op simulator获取到的指令流水图如下图所示：

图 3-130 指令流水图

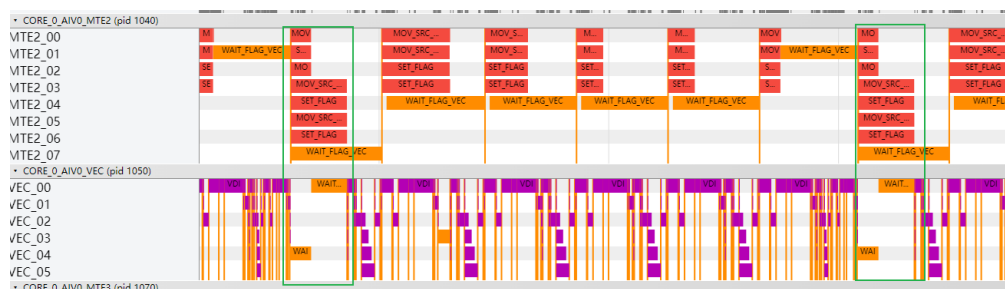


结合上述两种数据（真实数据和仿真数据）进行性能分析：

- Vector计算bound，当前为减少核启动开销设置为1:1；
- 实际优化过程中，对上述问题进行优化、Vector计算占比下降后，Cube和Vector计算各自都有间隙，相互之间都有等待耗时；



- Vector计算没有开启double buffer，计算和数据搬运部分没有并行。



设计优化方案

- 将AI Core中的AIC核和AIV核启动比例设置为1:2。每次AIC输出的数据，由两个AIV并行计算对应的反量化和激活函数；在Vector侧代码的循环里，AIV0和AIV1交替进行计算（前提条件，循环次数不为1）。代码示例如下：

```
uint32_t vecCount = 0;
uint32_t taskRatio = GetTaskRatio();
for (uint32_t offsetN = 0; offsetN < curCubeSingleN; offsetN += mnConfig.baseN) {
    if (unlikely(offsetN + mnConfig.baseN >= curCubeSingleN)) {
        curVecBaseN = curCubeSingleN - offsetN;
    }
    uint32_t alignBaseN = Ceil(curVecBaseN, uint32_t(8)) * 8; // 8: num int32_t in 32B ub block
    DataCopyScale(curVecBaseN, alignBaseN, scaleOffset + offsetN);
    uint32_t curVecBaseM = vecBaseM;
    uint64_t mmOutOffset = mnConfig.workSpaceOffset + offsetN * mnConfig.baseM;
    CrossCoreWaitFlag(SYNC_AIC_TO_AIV);
    for (uint32_t offsetM = 0; offsetM < curCubeSingleM; offsetM += vecBaseM) {
        vecCount++;
        if (vecCount % taskRatio != subBlockIdx) {
            continue; // AIV0和AIV1交替进行计算
        }
        if (unlikely(offsetM + vecBaseM >= curCubeSingleM)) {
            curVecBaseM = curCubeSingleM - offsetM;
        }
        // 使用AscendDequant接口做perchannel反量化
        LocalTensor<CT::T> mmOutLocal = vecInQueue.AllocTensor<CT::T>();
        DataCopyPad2D(mmOutLocal, mmOutGm[mmOutOffset + offsetM * curVecBaseN],
                      curVecBaseM, curVecBaseN, curVecBaseN);
        vecInQueue.Enqueue(mmOutLocal);
        ComputeDequantAndActivate(mnConfig, curVecBaseM, alignBaseN, curVecBaseN, offsetM);
        LocalTensor<DTYPE_Y> yLocal = vecOutQueue.DeQue<DTYPE_Y>();
        DataCopyPad2D(yGm[outOffset + offsetM * tiling->n + offsetN], yLocal,
```

```
        curVecBaseM, curVecBaseN, alignBaseN, tiling->n);  
        vecOutQueue.FreeTensor(yLocal);  
    }  
    ...  
}
```

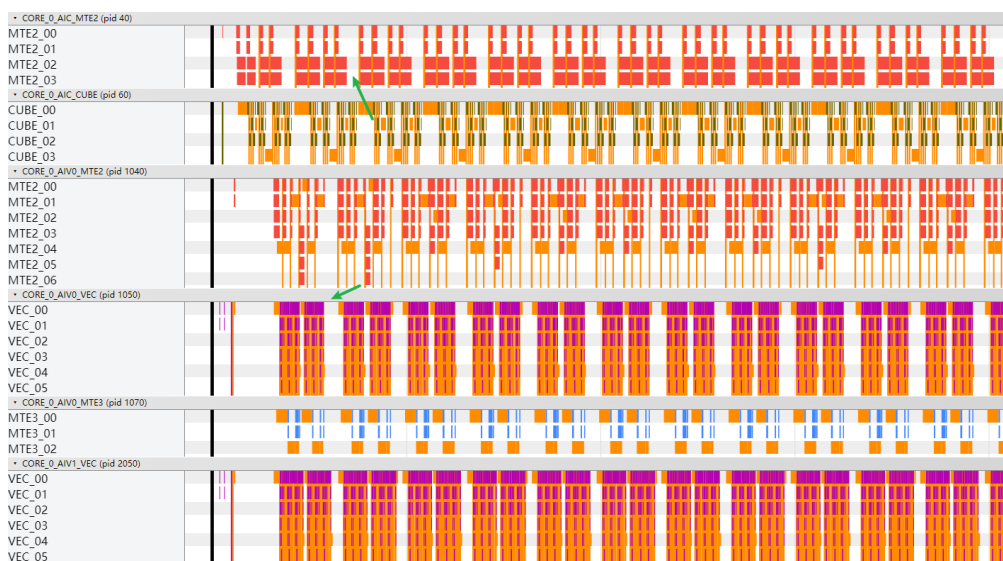
- AIC和AIV启动比例设置为1:2后，出现Cube和Vector计算各自都有间隙、相互之间都有等待耗时的情况。分析原因是因为Vector和Cube计算存在使用一份workspace进行数据传递的场景，通过4份workspace的方案进行优化：host按4倍baseM * baseN申请workspace，Cube侧代码在计算前可以跳过前4轮的等待。

```
if ASCEND_IS_AIC {  
    if (cubeCount >= tiling->parallNum) { // tiling->parallNum设置为4  
        CrossCoreWaitFlag(SYNC_AIV_TO_AIC);  
    }  
    mm.SetOrgShape(mnConfig.m, tiling->n, tiling->k);  
    mm.SetSingleShape(curSingleM, curSingleN, tiling->k);  
    mm.SetTensorA(xGm[xOffset]);  
    auto weightSlice = weightGm[weightOffset];  
    if (mnConfig.numBlocksM == 1) {  
        weightSlice.SetL2CacheHint(CacheMode::CACHE_MODE_DISABLE);  
    }  
    mm.SetTensorB(weightSlice);  
    uint64_t workspaceOffset = mnConfig.workSpaceOffset;  
    while (mm.Iterate()) {  
        mm.GetTensorC(mmOutGm[workspaceOffset], 0, true);  
        CrossCoreSetFlag<2, PIPE_FIX>(SYNC_AIC_TO_AIV);  
        workspaceOffset += (mnConfig.baseM * mnConfig.baseN);  
    }  
}  
cubeCount++;
```

- Vector计算开启double buffer，InitBuffer指定分配内存块个数为2。
pipe->InitBuffer(scaleInQueue, 2, tiling->mmTilingData.baseN * sizeof(DTYPE_SCALE));
pipe->InitBuffer(perTokenScaleInQueue, 2, tiling->mmTilingData.baseM * sizeof(float));
pipe->InitBuffer(vecInQueue, 2, tiling->ubCalSize * sizeof(cT::T));
pipe->InitBuffer(vecOutQueue, 2, tiling->ubCalSize * sizeof(DTYPE_Y));

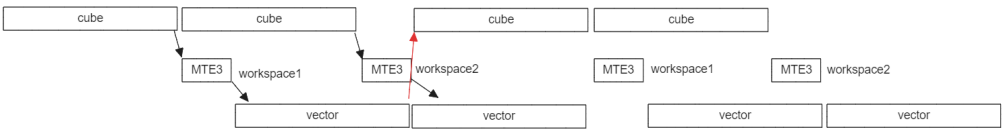
验证优化方案性能收益

- 将AI Core中的AIC和AIV启动比例设置为1:2后，执行总耗时从218.1us下降为154.2us。指令流水图显示Cube计算间等待变小。



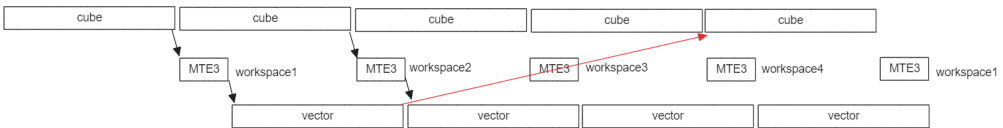
- 如上图所示，Vector计算已经不处于bound状态，但Cube和Vector计算都有间隙，未被充分利用（上述两个箭头的位置）。分析原因如下：

Vector计算在等Cube计算输出的数据，Cube侧需要等Vector计算完释放workspace以存放下一轮的计算结果，当前为了让Cube、Vector计算流水并行，workspace使用了两份空间：

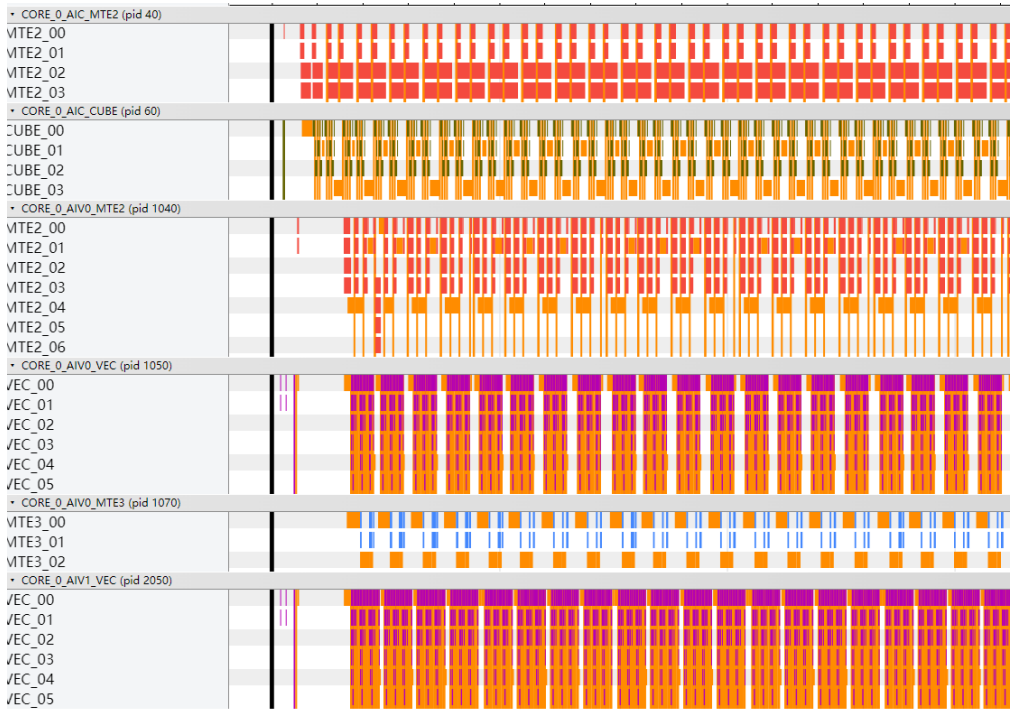


因为Vector和Cube计算存在使用一份workspace进行数据传递的场景，存在数据依赖，所以会有等待的间隔。

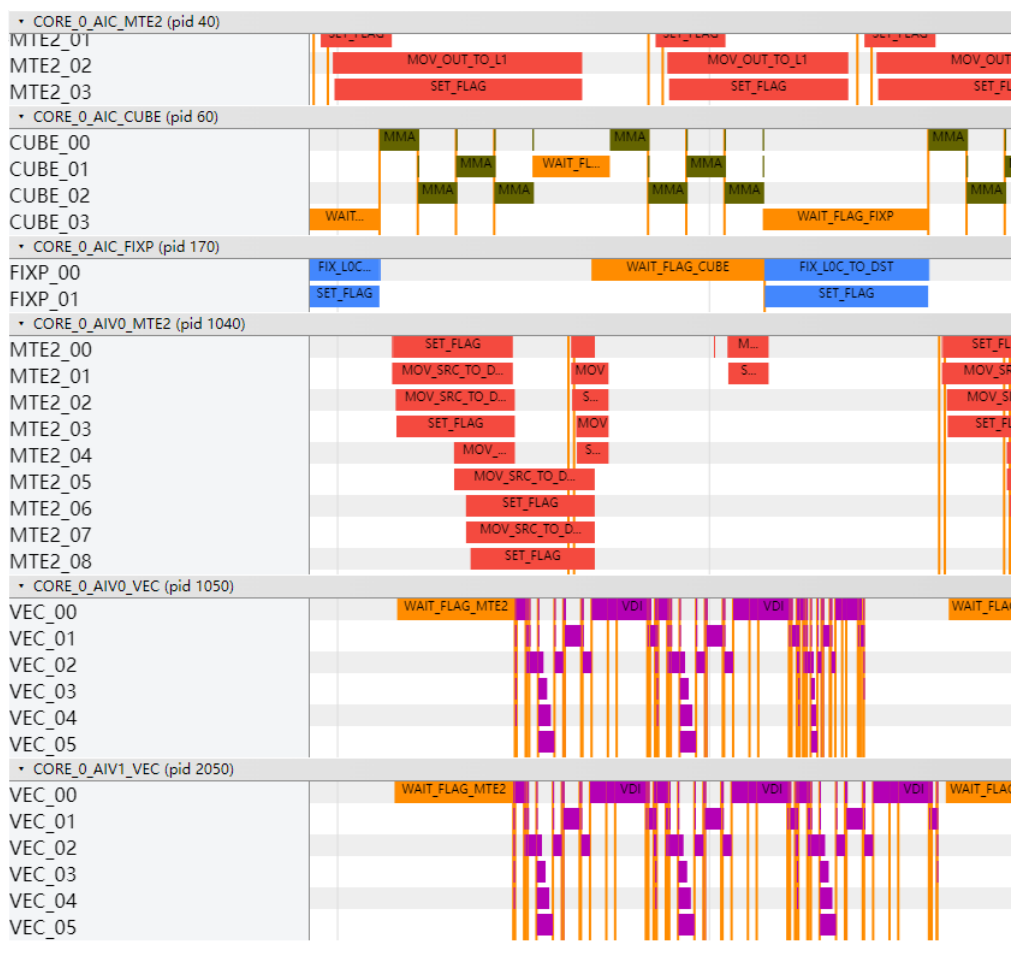
可以采用4份workspace进行优化：



优化后，总耗时由154.2us下降为131.8us。指令流水图显示Vector、Cube计算各自间隙明显减小。



- Vector计算开启double buffer，优化后执行总耗时从131.8us下降为128.1us。



总结

- 在Vector计算为主要瓶颈点时，将AI Core中的AIC核和AIV核启动比例设置为1:2；
- Cube、Vector计算时间接近，且两者都有因相互等待导致的间隙时，采用4份workspace优化；
- 观察数据搬运是否与计算相互掩盖，多轮计算没有数据依赖，且buffer够大时，开启double buffer，增加并行效率。

3.7.3 MC²算子性能调优案例

案例介绍

MC²通算融合算子的性能收益主要来自于通信、计算的并行执行，即将输入数据切分为多个子块，子块的计算和通信任务形成两条流水线，通过两条流水线上任务的并行执行，实现流水掩盖，从而提升算子性能。如下图所示，MC²算子先做Matmul计算、后通信的场景，输入矩阵沿M轴被切分为两块，第二块数据的Matmul计算和第一块数据的通信可以并行执行，从而达到计算和通信时间相互掩盖的目的。本节的所有图示中MM代表Matmul计算，hcom代表通信任务。



本案例将介绍如何分析通算融合算子的性能收益、如何制定较好的数据切分策略。更多MC²算子的完整样例请参考[MatmulAllReduce样例](#)、[MatmulReduceScatter样例](#)、[AllGatherMatmul样例](#)。

获取性能数据

通过msProf算子调优工具获取算子性能数据：

- 获取[真实环境执行的性能数据](#)，包含各个流水的占比情况；
- 获取仿真性能数据（指令流水图），包含各个流水的占用区间，可观察流水间依赖情况，从而优化并行效率。

分析主要瓶颈点

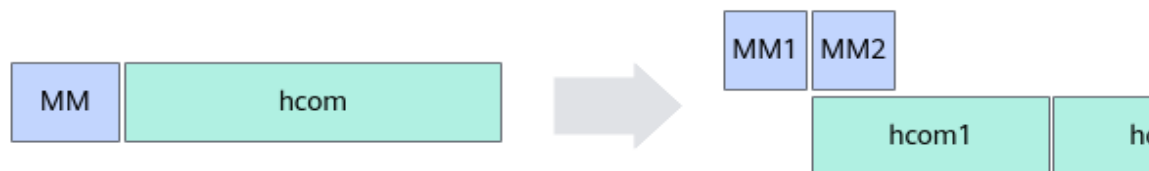
MC²算子性能收益公式为：

融合前算子串行耗时 = 融合前计算算子耗时 + 融合前通信算子耗时

MC²算子收益 = (融合前算子串行耗时 - 融合后MC²算子耗时) / 融合前算子串行耗时

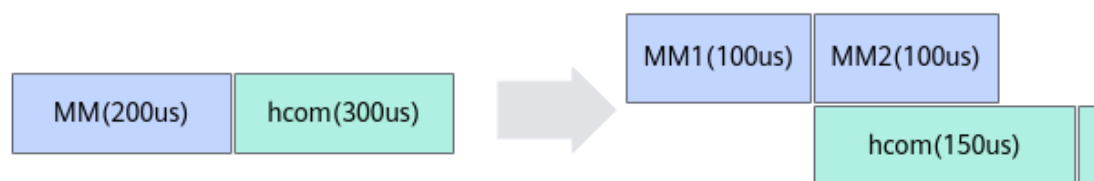
融合后MC²算子的执行耗时，受以下因素制约，从而影响算子性能收益。

- 因素一：计算和通信的执行时间差异
若计算和通信任务的执行时间相差不大，则融合后，MC²算子的计算和通信并行执行，实现流水掩盖效果，性能收益较大。
若计算和通信任务的执行时间差异较大，则融合后，MC²算子内计算和通信并行执行，能够掩盖的时间较少，算子整体执行耗时与未切分串行时的算子执行耗时接近，此时无法获得较大性能收益。



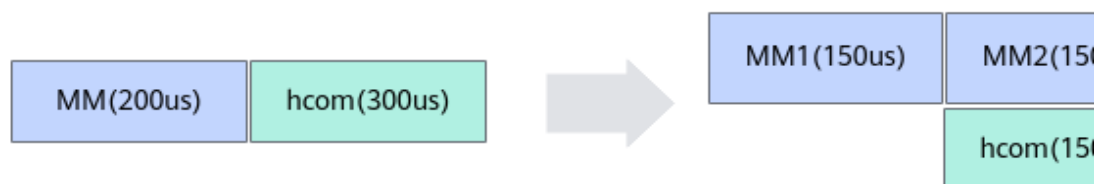
- 因素二：数据切分导致的计算或通信的执行时间膨胀
当对输入数据进行切分后，原本的整块数据被切分成若干小数据块，对若干小数据块分别做Matmul计算或者执行通信任务，此时相比切分前，计算或者通信任务的执行时间可能发生膨胀（即执行时间变长）。该膨胀产生的原因包括：切分后的数据块过小导致计算或通信的效率降低、切分的数据块过多导致增加额外的调度开销、并行执行后计算和通信对L2 Cache或device存储内存的访问冲突等。以Matmul计算为例，简单说明数据切分后执行时间可能发生的膨胀情况。
 - 未发生膨胀：

数据切分前，Matmul执行时间为200us，将Matmul的输入均匀切分为两块，假设切分后，每块数据的Matmul执行时间都是100us，通过计算的并行执行，下图实际性能收益为100us。



- 发生一般程度的膨胀：

数据切分前，Matmul执行时间为200us，将Matmul的输入均匀切分为两块，假设切分后，每块数据的Matmul执行时间都是150us，通过计算的并行执行，下图实际性能收益为50us。



- 发生严重程度的膨胀：

数据切分前，Matmul执行时间为200us，将Matmul的输入均匀切分为两块，假设切分后，每块数据的Matmul执行时间都是200us，通过计算的并行执行，下图实际性能收益为劣化50us。



综合上述分析，计算和通信执行时间较均衡的场景，有更好的流水掩盖和性能收益；同时，性能收益也受到数据切分导致的执行时间膨胀的影响。下文将介绍如何制定数据切分策略，以达到最佳流水掩盖效果。

设计优化方案

以Atlas A2 训练系列产品/Atlas A2 推理系列产品，输入数据格式为ND、数据类型为half的MatmulAllReduce算子为例，该算子中计算执行在前，通信任务执行在后。假定Matmul计算中左矩阵的形状为[M, K]，右矩阵的形状为[K, N]，算子中的通信对象为Matmul的输出矩阵，则通信任务的输入shape为[M, N]。因为K轴只在Matmul计算中存在，当K轴较大时，计算量大，计算执行时间大于通信执行时间，此时计算为算子的瓶颈（bound）；反之，当K轴较小时，计算量小，计算执行时间小于通信执行时间，此时通信为算子的瓶颈。在制定数据切分策略前，对原始矩阵分别执行计算和通信任务，根据两个任务的执行时间判定bound场景。

该算子的数据切分策略应满足如下要求：

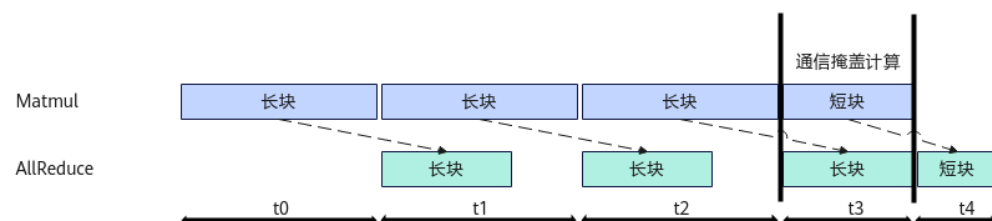
- 只切分M轴。因为通信任务调用的Hccl API要求分块数据内存连续，若按N轴切分，则每行数据都被切断，导致通信数据的内存不连续，不满足通信要求；若按M轴切分，则每行数据都是内存连续的，满足通信要求。
- 若A表示长块、B表示短块，只能切出A或B连续排布的形式，例如AAAB、BAAA等情况。

如上文所述，数据切分的目标是达成尽可能多的流水掩盖。根据计算与通信任务的执行时间差异，实际场景可以分解为如下两个具体场景，两个场景有各自细分的切分目标。

- 计算bound:

对于同一个切分后的数据块，计算执行耗时大于通信执行耗时，此时，**计算连续**，且**通信的尾块要短**，如下图所示。

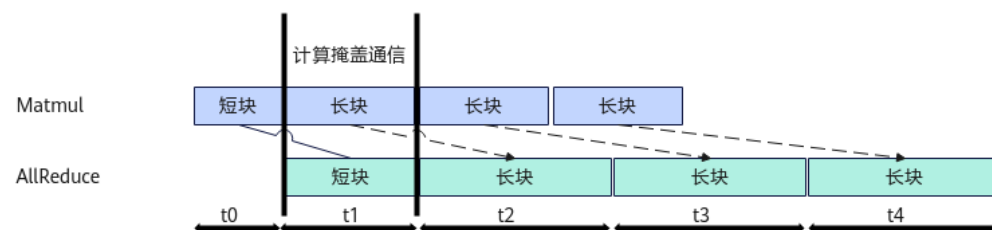
图 3-131 计算 bound 示意图



- 通信bound:

对于同一个切分后的数据块，通信执行耗时大于计算执行耗时，此时，**通信连续**，且**计算的头块要短**，如下图所示。

图 3-132 通信 bound 示意图

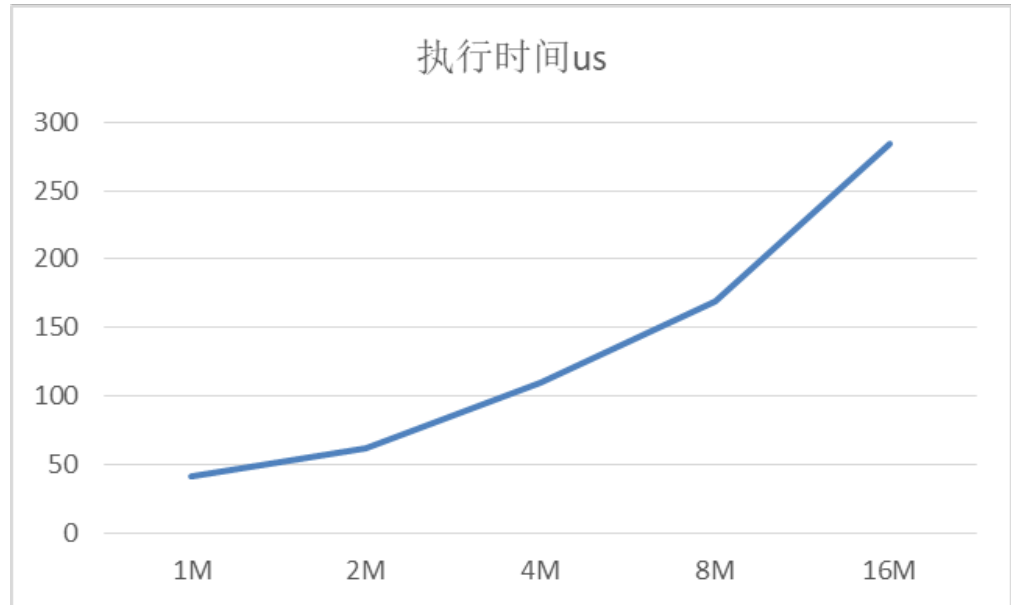


前置工作:

在进行最终的数据切分前，需要做的前置工作有：判定bound场景、分别对Matmul计算和AllReduce通信的数据量与执行时间关系做公式拟合。具体步骤如下。

1. 将输入数据分别单独执行Matmul计算和AllReduce通信任务，利用msProf工具分别采集执行时间，判定时间较长的任务为对应的bound场景。例如通信执行时间大于计算执行时间，则为**通信bound场景**。
2. 将输入数据按M轴切分，分别切成M轴为256、512、768、1024、2048的若干数据块，这里可以根据实际情况调整切块大小。
3. 将步骤2得到的数据块分别做AllReduce通信，利用msProf工具采集执行时间，得到每个数据块的执行时间 $t_1, t_2, \dots, t_n$ 。然后，作图分析数据量与对应的执行时间的关系，拟合得到公式 $t = \text{CostComm}(m)$ ，其中 m 表示数据块的M轴长度， t 表示数据块的通信执行时间， CostComm 表示拟合得到的 m 和 t 的映射关系。该映射关系一般为线性，若不满足线性关系，可以采用分段拟合的方式。示例如下：

m 值	数据量	执行时间us
1	1M	42
2	2M	62.1
4	4M	109.8
8	8M	168.8
16	16M	284.8



数据量 $x = m * N * \text{sizeof}(\text{dataType})$ ，单位是Bytes。该拟合公式表示为：

- x 小于8MB: $t = -0.9698202 * x * x + 27.0622573 * x + 14.769$ ，单位是us。
- x 大于等于8MB: $t = 13.58491263 * x + 61.508333$ ，单位是us。

4. 将步骤2切分得到的各数据块做Matmul计算，按与步骤3相同的方式采集各数据块的计算执行时间 t ，并拟合得到 M 轴长度和计算执行时间关系的公式 $t = \text{CostMM}(m)$ ， CostMM 是拟合得到的 m 和 t 的映射关系。

切分算法步骤：

1. 根据输入矩阵的shape: M 、 K 、 N ，按照经验值设置合适的 M 轴方向切分的短块长度。如下表达式中， a 、 b 、 c 表示根据经验值给出的短块长度的备选，将 K 、 N 带入如下三个表达式后，取 a 、 b 、 c 的最小值 m_0 为选定的短块长度。
 - $a * K * N \geq 4 * 1024 * 1024 * 1024$ ， a 取不等式的最小值
 - $b * K * N / 1024 + b * N \geq 6 * 1024 * 1024$ ， b 取不等式的最小值
 - $c \geq 3 * 128$ ， c 取不等式的最小值
 - $m_0 = \min(a, b, c)$
2. 根据短块长度 m_0 和拟合公式，分别得到计算执行时间 $t_0 = \text{CostMM}(m_0) * 1.15$ ，通信执行时间 $t_1 = \text{CostComm}(m_0) * 1.15$ 。
 注意：通信和计算并行执行时，可能出现抢占内存带宽的情况，导致执行时间增加，一般按经验在拟合公式中乘以1.15的系数，用户可以根据实测情况调整该系数。
3. 根据短块的长度，按图1或图2配平通算，得到长块的长度，长块长度尽量对齐128个元素，以保证计算亲和。本案例为通信bound场景，这里的配平，即将短块的通信时间和长块的计算时间匹配相等：将 t_1 作为长块的计算执行时间，带入 $t_1 = \text{CostMM}(m_1)$ 公式，计算得到 m_1 ，即为长块的长度。
4. 根据短块长度 m_0 、长块长度 m_1 、原始 M 轴长度 M ，得出长块的切块个数 $\text{count} = (M - m_0) / m_1$ 。该式一般不能整除，此时需要做如下处理：
 - 将结果的小数部分舍弃，保留整数部分作为切块个数。
 - 由于舍弃了小数部分， M 轴长度有剩余，因此需要调整长块长度 $m_1 = (M - m_0) / \text{count}$ 。

- 为保持计算亲和，将长块长度m1调整至128对齐，即向下取128倍数的整数，更新长块长度m1。
- 由于调整m1后，M轴长度有剩余，因此调整短块长度 $m0 = M - (m1 * \text{count})$ 。
- 最终得到短块长度m0，长块长度m1，长块个数count。

验证优化方案性能收益

- 制定切分策略并验证性能收益

本MatmulAllReduce案例中，给定的输入矩阵Shape为M=4096，K=3072，N=8192，数据类型为half，分核数为8，通过融合前msProf工具采集，得到该输入的Matmul计算执行时间为803us，AllReduce通信的执行时间为1071us，总耗时1874us，属于通信bound场景。按照上述的切分算法，本案例的具体切分情况如下：

- 根据经验值选定短块（bound场景中短块为头块，即切分后的第一个数据块）M方向长度m0为384，则通信数据量x为： $384 * 8192 * 2 / 1024 / 1024 = 6\text{MB}$ 。按[通信拟合公式](#)估算通信的执行时间为143us。考虑可能会发生内存带宽冲突，因此再乘以1.15的系数，得出通信执行时间164us。
- 计算长块M方向长度。根据短块通信时间，配平长块的计算执行时间同样为164us，按计算拟合公式估算出该长度m1为768。
- 根据M=4096、m0=384、m1=768，计算长块个数： $(4096 - 384) / 768 = 4.83$ ，向下取整为4。
- 根据短块长度m0=384，长块个数4，调整长块m1长度： $(4096 - 384) / 4 = 928$ ，向下按128对齐，调整m1为896。
- 根据长块长度m1=896，长块个数4，调整短块m0长度： $4096 - 896 * 4 = 512$ 。
- 最终得到将原始输入矩阵切分为5个数据块，长度分别为：{512, 896, 896, 896, 896}。

如下代码所示，在算子的Tiling代码中设置制定好的切分策略。按该切分策略测试，融合后该算子的执行时间为1262us，则融合算子的性能收益为 $(1874 - 1262) / 1874 = 32.7\%$ 。

```
MatmulAllReduceCustomTilingData *tiling = context->GetTilingData<MatmulAllReduceCustomTilingData>();
tiling->param.rankDim = 8;
tiling->param.tileM = 512; // 短块大小
tiling->param.tileNum = 1; // 短块个数
tiling->param.tailM = 896; // 长块大小
tiling->param.tailNum = 4; // 长块个数
tiling->param.rankM = 4096;
tiling->param.rankN = 8192;
tiling->param.rankK = 4096;
tiling->param.isTransposeA = 0;
tiling->param.isTransposeB = 0;
tiling->param.cToFloatLen = 0;
tiling->param.nd2NzWorkLen = true;
tiling->param.dataType = static_cast<uint8_t>(HCCL_DATA_TYPE_MAP.at(aType));
```

- 针对切分膨胀做调整

上文提到，切分数据会引起计算或通信执行时间的膨胀，使实测结果与理论值有偏差。比如切块数量较多时，执行时间的膨胀对性能影响较大，可能导致性能收益变小或者出现性能劣化，因此最终需要根据上述理论切分策略，结合实测，对切分策略做调整。

总结

MC²算子通过数据切分后计算和通信的并行执行，获得性能收益，但受数据切分后执行时间膨胀的影响。对MC²算子进行性能调优的主要方式是制定数据切分策略，开发人员需要根据理论推导找到理想切分策略，然后根据实测结果调整，最终找到最优切分策略。

3.7.4 Matmul 性能调优案例

3.7.4.1 Matmul 性能优化策略总览

本节提供了一系列包含Matmul计算的算子性能调优案例，开发者可根据实际应用场景，参考相关案例中的优化方法和思路，应用于具体实践中。案例分为如下五类，各分类的简介请参见如下表格，详细内容请阅读后续章节。

- Tiling优化

表 3-19 Tiling 优化策略总览

分类	适用场景	相关案例
Tiling优化：优化Tiling分核及基本块切分策略。	数据量足够多的大Shape场景。	Matmul算子优化Tiling策略

- 并行度优化

表 3-20 并行度优化策略总览

分类	适用场景	相关案例
多核间任务并行：合理地将数据分配给不同的核来执行任务。	矩阵的K轴较大、M轴和N轴相比K轴较小的场景。	Matmul高阶API使能多核切K
多核间数据访问并行：优化多核数据并行访问机制，如多核场景同一内存数据的地址访问冲突优化，实现多核数据访问效率提升。	多核执行Matmul，输入矩阵的K轴较大且K轴非全载的场景。	Matmul高阶API使能多核K轴错峰访问内存
单核内流水并行：利用不同指令队列间的相互独立性和可并行执行特性，优化核内流水并行度。	1. 算子的MMAD流水和FIXPIPE流水之间串行执行，同步等待的时间在算子整体执行耗时中占比较高。 2. MTE2 Bound且MTE2流水和其他流水串行执行。	1. Matmul高阶API使能UnitFlag 2. Matmul高阶API使能NBuffer33模板

- 内存优化

表 3-21 内存优化策略总览

分类	适用场景	相关案例
内存共享与复用：通过Buffer的共享与缓存复用，减少重复的数据搬运带来的开销。	MIX场景下，多个AIV的A矩阵或B矩阵GM地址相同，且多个AIV复用的A矩阵或B矩阵在L1 Buffer上全载。	Matmul高阶API使能IBShare模板共享A和B矩阵数据 Matmul高阶API使能IBShare模板共享B矩阵数据
内存对齐：确保处理的数据满足特定的对齐要求，针对非对齐数据使用不同的搬运策略，以提升数据搬运的效率。	输入矩阵内轴非256字节对齐，且数据量较大的场景。	AIV核上的ND2NZ格式转换

- Scalar优化

表 3-22 Scalar 优化策略总览

分类	适用场景	相关案例
Tiling常量化：在Kernel编译期间完成Matmul Tiling的计算，由变量转化为常量扩散到系统中，减少Scalar提升性能。	Matmul初始化时的Scalar计算较多，影响指令头开销。 Matmul迭代之间的Scalar计算较多，阻塞MTE2流水。	Matmul高阶API使能Tiling全量常量化
纯Cube模式：减少消息处理机制带来额外的Scalar开销。	相较于MIX模式，没有矢量计算，只有矩阵计算的场景。	Matmul高阶API使能纯Cube模式

- 搬运优化

表 3-23 搬运优化策略总览

分类	适用场景	相关案例
搬运吞吐量优化：通过合理控制搬运数据块的大小，提升带宽利用效率，实现搬运效率的提升。	1. MTE2循环搬运次数多的大shape场景。 2. 输入和输出的数据量超过L2 Cache大小的场景。	1. Matmul 高阶API使能MDL模板 2. Matmul高阶API使能L2 Cache切分
预加载搬运：预加载需要搬运的数据块，减少流水之间的间隙。	MTE2流水间隙较大，且M或N数值较大的场景。	Matmul高阶API使能MTE2 Preload

设计优化方案

- 优化点一：优化分核逻辑

由Profiling数据看出分核数为4，启动更多的核同时计算，可以提高计算并行度。当前案例使用的AI处理器共20个核，每个核中包含1个Cube Core和2个Vector Core。NPU调用程序中设置numBlocks为实际使用的核数20。

```
// 代码片段
uint32_t numBlocks = 20; // 优化前numBlocks为4
CHECK_ACL(aclInit(nullptr));
int32_t deviceId = 0;
CHECK_ACL(aclrtSetDevice(deviceId));
aclrtStream stream = nullptr;
CHECK_ACL(aclrtCreateStream(&stream));

uint8_t *aHost;
uint8_t *aDevice;
CHECK_ACL(aclrtMallocHost((void **)&aHost, aFileSize));
CHECK_ACL(
    aclrtMalloc((void **)&aDevice, aFileSize, ACL_MEM_MALLOC_HUGE_FIRST));
ReadFile("./input/x1_gm.bin", aFileSize, aHost, aFileSize);
// PrintData(aHost, 16, printDataType::HALF);
CHECK_ACL(aclrtMemcpy(aDevice, aFileSize, aHost, aFileSize,
    ACL_MEMCPY_HOST_TO_DEVICE));

uint8_t *bHost;
uint8_t *bDevice;
CHECK_ACL(aclrtMallocHost((void **)&bHost, bFileSize));
CHECK_ACL(
    aclrtMalloc((void **)&bDevice, bFileSize, ACL_MEM_MALLOC_HUGE_FIRST));
ReadFile("./input/x2_gm.bin", bFileSize, bHost, bFileSize);
// PrintData(bHost, 16, printDataType::HALF);
CHECK_ACL(aclrtMemcpy(bDevice, bFileSize, bHost, bFileSize,
    ACL_MEMCPY_HOST_TO_DEVICE));

uint8_t *workspaceHost;
uint8_t *workspaceDevice;
CHECK_ACL(aclrtMallocHost((void **)&workspaceHost, workspaceSize));
CHECK_ACL(aclrtMalloc((void **)&workspaceDevice, workspaceSize,
    ACL_MEM_MALLOC_HUGE_FIRST));

uint8_t *tilingHost;
uint8_t *tilingDevice;
CHECK_ACL(aclrtMallocHost((void **)&tilingHost, tilingFileSize));
CHECK_ACL(aclrtMalloc((void **)&tilingDevice, tilingFileSize,
    ACL_MEM_MALLOC_HUGE_FIRST));
CHECK_ACL(aclrtMemcpy(tilingHost, tilingFileSize, GenerateTiling(),
    tilingFileSize, ACL_MEMCPY_HOST_TO_HOST));
// PrintData(tilingHost, 16, printDataType::UINT32_T);
CHECK_ACL(aclrtMemcpy(tilingDevice, tilingFileSize, tilingHost,
    tilingFileSize, ACL_MEMCPY_HOST_TO_DEVICE));

uint8_t *cHost;
uint8_t *cDevice;
CHECK_ACL(aclrtMallocHost((void **)&cHost, cFileSize));
CHECK_ACL(
    aclrtMalloc((void **)&cDevice, cFileSize, ACL_MEM_MALLOC_HUGE_FIRST));

// ACLRT_LAUNCH_KERNEL(matmul_custom)
// (numBlocks, stream, aDevice, bDevice, cDevice, workspaceDevice, tilingDevice);
matmul_custom_do(numBlocks, stream, aDevice, bDevice, cDevice, workspaceDevice, tilingDevice);
```

由于Matmul API都是从Vector侧发起的，当前案例使用的AI处理器中Cube Core和Vector Core的配比为1 : 2，所以在Matmul tiling计算中需要按照2倍的numBlocks数切分，即Vector Core数。NPU调用程序中设置的实际运行核数是20核，所以Tiling代码中设置Tiling API按照40个核进行数据切分，如下代码所示。

```
int usedCoreNum = 40; // 优化前usedCoreNum是8
int runMode = 1;
```



```
int32_t baseM = 64; // 64
int32_t baseN = 64; // 64
optiling::TCubeTiling tilingData;
MultiCoreMatmulTiling tilingApi;
tilingApi.SetDim(usedCoreNum);
```

图 3-134 优化分核逻辑后 Profiling 数据

aicore_time	aic_total_c	aic_mac_t	aic_mac_r	aic_scalar	aic_scalar	aic_mte1_t	aic_mte1_r	aic_mte2_t	aic_mte2_r	aic_fixpipe	aic_fixpipe
2532.53	93703665	706.283	0.279	1225.87	0.484	1201.55	0.474	2452.38	0.968	724.456	0.286

修改代码后，算子执行时间从12045us下降到2532us，约等于(20核 / 4核) = 5倍的性能提升。

- 优化点二：优化基本块

当前Tiling中设置的base块为 [baseM, baseN, baseK] = [64, 64, 256]，这种基本块Cube计算cycle少，计算访存比（即计算量与需要数据量的比值）低；搬出一次Matmul结果到GM的base块是64 * 64，由于输出格式是ND，数据类型是float，搬出下一次Matmul结果的起始地址需要偏移一个baseN的大小，即64 * 4 = 256字节，导致fixpipe搬出时GM地址非512byte对齐，那么需要设置更优的基本块。

针对当前shape较大的场景，基本块的选择原则为计算访存比最大，即在Cube计算量最大的情况下，访存的数据量最小。在输入为fp16类型的情况下，Cube执行单元1 cycle能算16 * 16 * 16个数。根据经验，[baseM, baseN, baseK] = [128, 256, 64]和[128, 128, 128]两种切分方案均满足搬出时GM地址512Byte对齐（每搬出一次Matmul结果时，地址分别偏移256 * 4byte和128 * 4byte），Cube计算cycle数一致，为(128 * 64 * 256) / (16 * 16 * 16) = (128 * 128 * 128) / (16 * 16 * 16) = 512cycle。针对[baseM, baseN, baseK] = [128, 256, 64]，计算访存比为512cycle / (128 * 64 * 2 + 256 * 64 * 2) = 512cycle / 48KB；针对[baseM, baseN, baseK] = [128, 128, 128]，计算访存比为512cycle / (128 * 128 * 2 + 128 * 128 * 2) = 512cycle / 64KB；可见[128, 256, 64]基本块方案的计算访存比更高，计算密度更大，同样的计算量，需要的数据量最小，最大限度提高Cube单元的计算量。

修改Tiling代码，通过SetFixSplit()接口设置baseM和baseN，tiling函数会自动计算出最优baseK，这里得到64。

```
int32_t baseM = 128; // 优化前baseM是64
int32_t baseN = 256; // 优化前baseN是64

optiling::TCubeTiling tilingData;
MultiCoreMatmulTiling tilingApi;
tilingApi.SetDim(usedCoreNum);
tilingApi.SetAType(leftPos, leftFormat, leftDtype, bool(transposeA));
tilingApi.SetBType(rightPos, rightFormat, rightDtype, bool(transposeB));
tilingApi.SetCType(resPos, resFormat, resDtype);
tilingApi.SetBiasType(biasPos, biasFormat, biasDtype);

tilingApi.SetOrgShape(M, N, K);
tilingApi.SetShape(M, N, K);
tilingApi.SetFixSplit(baseM, baseN, -1);
```

使能这组基本块后，MTE2耗时（对应aic_mte2_time）从2452us降低到808us，MTE2性能提升3倍。

图 3-135 优化基本块后 Profiling 数据

aicore_time	aic_total_c	aic_mac_t	aic_mac_r	aic_scalar	aic_scalar	aic_mte1_t	aic_mte1_r	aic_mte2_t	aic_mte2_r	aic_fixpipe	aic_fixpipe
835.63	30918284	615.797	0.737	618.704	0.74	415.068	0.497	808.522	0.968	87.006	0.104

- 优化点三：使能大包搬运

当前带宽利用率为：totalSize / mte2Time = totalCnt * dtype / mte2Time，代入数据计算为2491GB/s。未使能大包搬运的情况下，矩阵从GM搬运到L1一次只搬

运1个基本块。通过模板参数使能大包搬运，一次搬运多个基本块，提高MTE2带宽利用率。

```
// 原始matmul对象定义:
Matmul<AscendC::MatmulType<TPosition::GM, CubeFormat::ND, A_T>,
      AscendC::MatmulType<TPosition::GM, CubeFormat::ND, B_T>,
      AscendC::MatmulType<TPosition::GM, CubeFormat::ND, C_T>,
      AscendC::MatmulType<TPosition::GM, CubeFormat::ND, BiasT>>>
mm;
// 通过在定义matmul对象的模板参数里加上CFG_MDL参数使能大包搬运功能:
Matmul<AscendC::MatmulType<TPosition::GM, CubeFormat::ND, A_T>,
      AscendC::MatmulType<TPosition::GM, CubeFormat::ND, B_T>,
      AscendC::MatmulType<TPosition::GM, CubeFormat::ND, C_T>,
      AscendC::MatmulType<TPosition::GM, CubeFormat::ND, BiasT>, CFG_MDL>>
mm;
```

从下图可以看到，使能大包搬运后，MTE2耗时从808us下降到591us，带宽利用率代入数据计算为3406GB/s，利用率提升36%+，Cube利用率达到80%+。

图 3-136 使能大包搬运后 Profiling 数据

aicore_tim	aic_total_c	aic_mac_t	aic_mac_r	aic_scalar_t	aic_scalar_r	aic_mte1_t	aic_mte1_r	aic_mte2_t	aic_mte2_r	aic_fixpipe	aic_fixpipe_r
710.46	26286857	581.815	0.819	472.152	0.665	481.476	0.678	591.088	0.832	26.882	0.038

验证优化方案性能收益

- 优化分核逻辑，实际收益4.75倍，约等于(20核 / 4核) = 5倍收益，并且考虑到核的启动开销，可以认为收益一致。
- 优化基本块，实际收益约3倍，理论评估代入上述分析公式，收益为(1 / 64 + 1 / 64) / (1 / 128 + 1 / 256)，约等于2.7倍，考虑到cache缓存的影响，认为收益一致。
- 大包搬运，实际收益25%+，与经验值一致。

总结

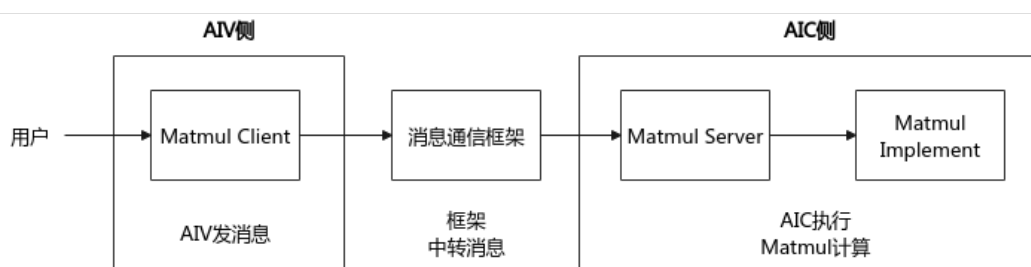
优化点一和优化点二的适用场景，需要shape足够大，数据量足够多，才能分满核和使能最优的基本块。大shape场景下，MTE2 Bound算子可参考此案例的优化手段。

3.7.4.3 Matmul 高阶 API 使能纯 Cube 模式

案例介绍

本案例呈现了在矩阵乘算子场景中，使能Matmul高阶API的纯Cube模式对算子性能的提升效果。如下图所示，Matmul API默认使用MIX模式，即用户从AIV侧发起消息，通过消息通信框架中转消息后，在AIC侧执行Matmul计算。这套消息处理机制会带来额外的Scalar性能开销。相较于MIX模式，纯Cube模式可以直接跳过消息通信框架，完成Matmul计算，提升算子性能。

图 3-137 默认 MIX 模式的 Matmul 流程示意图



- **使能纯Cube模式的适用场景**
非融合算子，只有矩阵计算的场景。即相较于MIX模式（包含矩阵计算和向量计算），没有向量计算的场景。本案例的算子规格如下：

表 3-24 算子用例规格

輸入	Shape	Data type	Format
a	128, 64	float16	ND
b	64, 30720	float16	ND

当前案例使用的AI处理器共24个核，每个核中包含1个AIC核和2个AIV核。

Tiling参数如下:

- 原始shape: M=128, N=30720, K=64。
- 单核shape:
 - MIX场景: 按48个AIV核进行切分, singleCoreM=128, singleCoreN=640, singleCoreK=64。
 - 纯Cube场景: 按24个AIC核进行切分, singleCoreM=128, singleCoreN=1280, singleCoreK=64。
- 基本块shape: baseM=128, baseN=256, baseK=64。
- L1相关Tiling参数: stepM=1, stepN=1, stepKa=4, stepKb=4, depthA1=8, depthB1=8。

获取性能数据

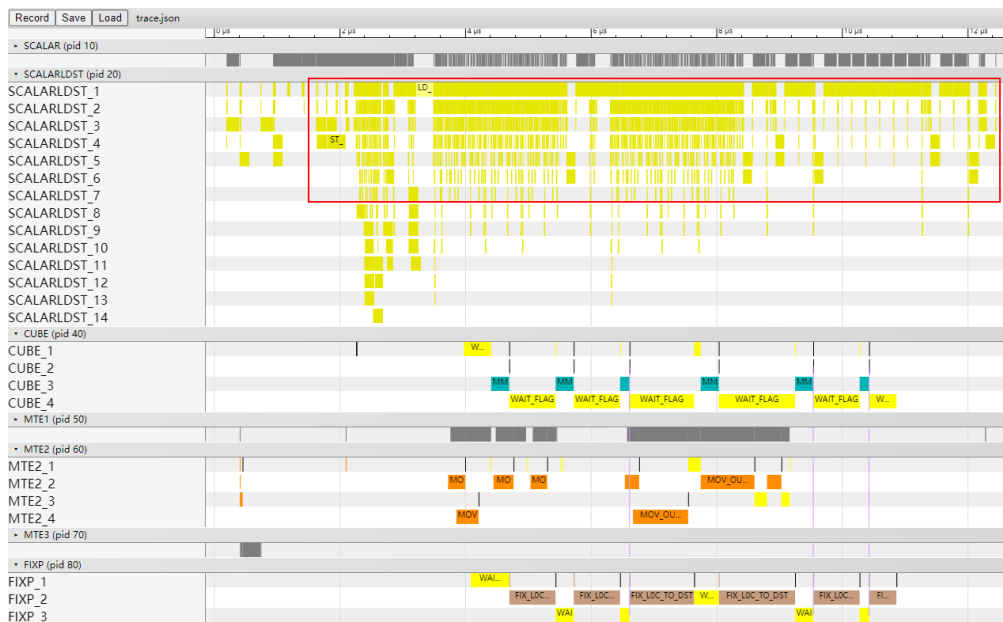
使用msProf工具获取**量子仿真流水图**和**上板Profiling**数据。因为纯Cube模式主要优化Scalar流水性能，可以重点分析Scalar的流水情况。

分析主要瓶颈点

- 优化前的Profiling数据如下，从C列的aic_time数据可以看出，多个核中最大算子执行耗时为17.85us。从G列的aic_scalar_time数据可以看出，Scalar平均耗时为15.02us，性能瓶颈在Scalar流水。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	GROUP ID	NAME	AGE	SEX	HEIGHT	WEIGHT	HAIR	SKIN	OCULAR	EAR	NOSE	MOUTH	TEETH	NECK	THORAX	ABDOMEN	PELVIS	FEET	TOES	HAIR
2	1	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
3	2	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
4	3	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
5	4	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
6	5	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
7	6	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
8	7	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
9	8	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
10	9	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
11	10	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
12	11	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
13	12	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
14	13	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
15	14	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
16	15	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
17	16	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
18	17	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
19	18	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
20	19	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
21	20	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
22	21	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
23	22	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
24	23	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
25	24	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
26	25	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
27	26	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
28	27	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
29	28	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
30	29	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
31	30	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
32	31	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
33	32	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
34	33	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
35	34	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
36	35	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
37	36	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
38	37	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
39	38	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
40	39	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
41	40	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
42	41	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
43	42	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
44	43	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
45	44	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
46	45	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
47	46	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
48	47	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
49	48	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
50	49	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
51	50	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
52	51	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
53	52	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
54	53	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
55	54	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
56	55	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
57	56	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
58	57	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
59	58	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
60	59	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
61	60	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
62	61	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
63	62	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
64	63	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
65	64	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
66	65	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
67	66	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
68	67	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
69	68	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
70	69	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
71	70	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
72	71	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
73	72	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
74	73	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
75	74	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
76	75	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
77	76	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
78	77	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
79	78	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
80	79	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
81	80	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
82	81	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
83	82	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
84	83	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
85	84	male	20.0	M	1.78	75.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD	MD
86	85	female	20.0	F	1.63	50.0	BRN	WHT	BLU	MD	MD	MD	MD	MD	MD	MD	MD	MD		

- 优化前的流水图如下，由于默认为MIX模式，每次Matmul计算均涉及消息通信框架对消息进行处理，Scalar流水重，性能开销较大，如下图红框所示。



设计优化方案

默认MIX模式下，用户在AIV侧发起消息，通过消息通信框架中转消息后，在AIC侧执行Matmul计算。基于这样的流程，用户使用Matmul高阶API编写算子代码时，可以使用REGIST_MATMUL_OBJ宏，无需区分AIV和AIC，但也因这套消息处理机制导致了额外的性能开销，如图1 默认MIX模式的Matmul流程示意图所示。

实现默认MIX模式的具体步骤如下：

步骤1 Kernel侧，定义Matmul对象。

```
#include "lib/matmul_intf.h"

using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, AType>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BType>;
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_NORM> matmulObj;
```

步骤2 Host侧，Matmul多核Tiling对象调用SetDim接口设置参与运算的核数。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendCManager::GetInstance();
matmul_tiling::MultiCoreMatmulTiling cubeTiling(*ascendcPlatform);
int32_t numBlocks = ascendcPlatform->GetCoreNumAiv(); // MIX模式使用GetCoreNumAiv获取AI处理器可用的核数。
cubeTiling.SetDim(numBlocks);
```

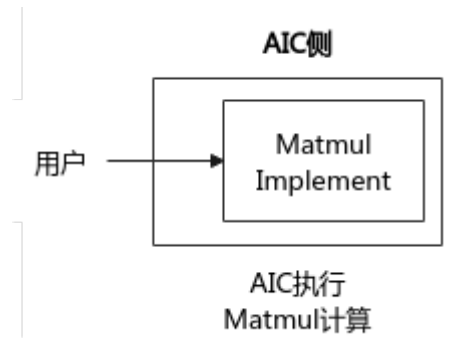
步骤3 调用核函数，参考核函数定义和调用，设置核函数的numBlocks参数配置。

```
matmul_custom_do(ascendcPlatform->GetCoreNumAic(), stream, x1, x2, bias, y, workspaceDevice,
tilingDevice); // MIX模式下，启动时，按照AIV和AIC组合启动，numBlocks用于设置启动多少个AI Core。
```

----结束

在没有矢量计算的算子场景下，可以跳过消息通信框架的机制，使能纯Cube模式完成Matmul计算，减少消息通信的性能开销，提升算子性能。

图 3-138 纯 Cube 模式的 Matmul 流程示意图



Matmul API 使能纯 Cube 模式的完整样例请参考 [Matmul API 性能优化样例](#)。使能纯 Cube 模式的主要步骤如下：

步骤1 Kernel 侧，在定义 Matmul 对象的代码中，包含 matmul_intf.h 头文件前设置 ASCENDC_CUBE_ONLY 宏。

```
#define ASCENDC_CUBE_ONLY // 在#include "lib/matmul_intf.h"前，设置ASCENDC_CUBE_ONLY宏
#include "lib/matmul_intf.h"
```

```
using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, AType>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BType>;
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_NORM> matmulObj;
```

步骤2 Host 侧，Matmul 多核 Tiling 对象调用 SetDim 接口设置参与运算的核数。

```
auto ascendcPlatform = platform_ascendc::PlatformAscendCManager::GetInstance();
matmul_tiling::MultiCoreMatmulTiling cubeTiling(*ascendcPlatform);
int32_t numBlocks = ascendcPlatform->GetCoreNumAic(); // 纯Cube模式使用GetCoreNumAic接口获取AI处理器可用的核数。
cubeTiling.SetDim(numBlocks);
```

步骤3 调用核函数，参考 [核函数定义和调用](#)，设置核函数的 numBlocks 参数配置。

```
matmul_custom_do(ascendcPlatform->GetCoreNumAic(), stream, x1, x2, bias, y, workspaceDevice,
tilingDevice); // 仅包含Cube计算的算子，numBlocks用于设置启动多少个AIC。
```

步骤4 Kernel 侧，核函数实现中增加 AIV 侧返回分支。

```
extern "C" __global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b,
GM_ADDR bias, GM_ADDR c, GM_ADDR workspace, GM_ADDR tilingGm)
{
    if (g_coreType == AscendC::AIV) { // 纯Cube模式，AIV侧直接return
        return;
    }
    ...
    // 其他代码
}
```

----结束

验证优化方案性能收益

- 优化后的 Profiling 数据如下，从 C 列的 aic_time 数据来看，多个核中最大算子执行耗时为 11.21us，较优化前的 17.85us 有较大提升。从 G 列的 aic_scalar_time 数据来看，Scalar 平均耗时从优化前的 15.02us 降低至 5.17us。

J	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	0	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
2	0	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
3	1	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
4	2	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
5	3	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
6	4	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
7	5	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
8	6	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
9	7	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
10	8	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
11	9	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
12	10	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
13	11	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
14	12	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
15	13	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
16	14	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
17	15	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
18	16	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
19	17	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
20	18	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
21	19	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
22	20	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
23	21	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
24	22	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
25	23	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

- 优化后的流水图如下。对比优化前的流水图，红框所示位置的Scalar流水明显变稀疏。纯Cube模式相较于MIX模式，减少了对消息通信的处理，优化了整体Scalar性能开销。



总结

在只有矩阵计算，没有矢量计算的场景下，可以考虑使能纯Cube模式，优化Matmul计算中的消息通信性能开销，提升算子性能。

3.7.4.4 Matmul 高阶 API 使能 MDL 模板

案例介绍

本案例呈现了在矩阵乘算子场景中，使用Matmul高阶API进行矩阵乘法计算，使能MDL模板对算子性能的提升效果。在MDL模板中，MTE2流水从Global Memory到A1/B1的数据搬运为一次性大包搬运，即一次MTE2能搬运多个Matmul计算的基本块，提升带宽利用率，使后续的MTE1流水尽可能复用A1/B1内基本块的缓存数据，减少MTE2的搬运次数。MDL模板的详细介绍请参考MatmulConfig。

- MDL模板的适用场景
一般适用于MTE2循环搬运次数多的大shape场景，MDL模板在A1/B1中缓存多次计算需要的数据，避免MTE2频繁搬运。
- MDL模板的约束条件
MDL模板的TCubeTiling结构体需要满足TCubeTiling约束条件和MDL模板补充约束条件，具体请参考TCubeTiling结构体。

本案例的算子规格如下：

表 3-25 算子规格

输入	Shape	Data type	Format
a	128, 1024	float16	ND
b	1024, 30720	float16	ND

当前案例使用的AI处理器共24个核，每个核中包含1个AIC核和2个AIV核。

Tiling参数如下：

- 原始shape：M=128, N=30720, K=1024。
- 单核shape：按24个AIC核进行切分，singleCoreM=128，singleCoreN=1280，singleCoreK=1024。

对于B矩阵，沿着N轴进行切分，切分成24份的singleCoreN，单核上处理K * SingleCoreN大小的数据。对于A矩阵，M轴不进行切分即singleCoreM=M，单核上处理singleCoreM * K大小的数据。总共24个核参与计算。

- 基本块shape：baseM=128，baseN=256，baseK=64。
- L1相关Tiling参数：stepM=1，stepN=1，stepKa=4，stepKb=4，depthA1=8，depthB1=8。

获取性能数据

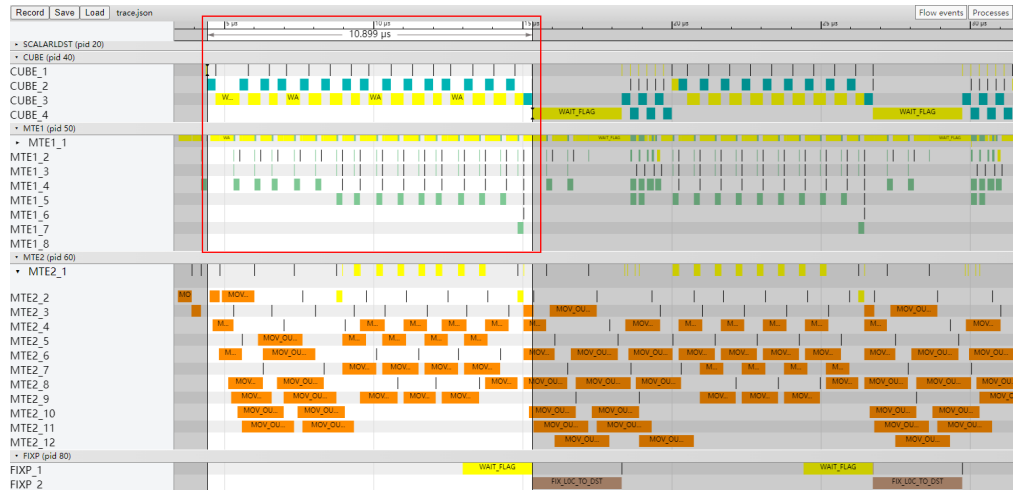
使用msProf工具获取[算子仿真流水图](#)和[上板Profiling](#)数据，因为MDL模板主要优化MTE2搬运效率，重点分析MTE2的流水情况。

分析主要瓶颈点

- 优化前的Profiling数据如下，Matmul默认为Norm模板。从C列的aic_time数据可以看出，多个核中最大算子执行耗时为83.68us。从C列的aic_time、L列的aic_mte2_time和M列的aic_mte2_ratio几组数据来看，MTE2平均耗时75.64us，耗时占比达到92%以上，因此需要优化MTE2流水的耗时。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	0000	id	sub 2Dx16 16	80 100x16x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16	80 100x 100x16
2	0	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
3	1	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
4	2	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
5	3	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
6	4	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
7	5	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
8	6	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
9	7	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
10	8	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
11	9	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
12	10	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
13	11	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
14	12	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
15	13	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
16	14	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
17	15	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
18	16	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
19	17	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
20	18	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
21	19	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
22	20	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
23	21	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
24	22	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443
25	23	subNo	81.94044	151025	23.411802	0.288768	22.085494	0.170122	0.104454	0.36865	761.931334	78.402102	0.000494	0.000013	882.148743	4.142321	0.000008	128.701443	0.000008	128.701443

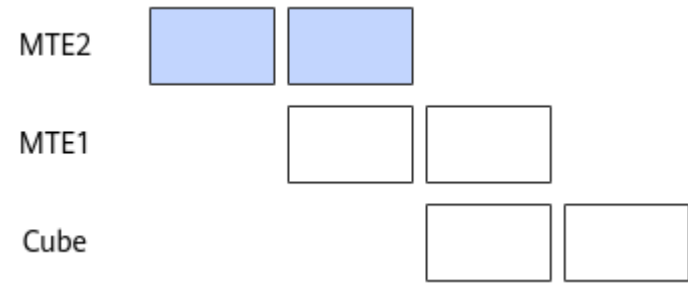
- 优化前的流水图如下，MTE2分多次从Global Memory搬运基本块到A1/B1。由于输入的矩阵Shape较大，MTE2循环搬运的次数多，但每次只搬运1个基本块，导致带宽利用率低，整体的MTE2搬运耗时长。进而影响后续的MTE1和MMAD流水，导致流水之间同步等待时间偏长。如红框所示，第一个基本块（baseM*baseN）的计算需要调用16次MMAD指令（singleCoreK/baseK=16），从左侧的第1个MMAD指令调用开始，到右侧的第16个MMAD指令调用结束，期间耗时10.899us，其中大部分是流水同步等待耗时。



设计优化方案

下图是默认的Norm模板的Matmul计算流水示意图，MTE2分多次从Global Memory搬运基本块到A1或B1，每次只搬运一个基本块。Norm模板的优势为启动开销小，可以提前启动MTE1流水；Norm模板的劣势为在大Shape场景，MTE2搬运次数多，搬运带宽利用率低，整体性能开销大。

图 3-139 默认 Norm 模板流水示意图



实现Norm模板的具体步骤如下：

步骤1 创建Matmul对象，使用默认的Norm模板参数CFG_NORM。

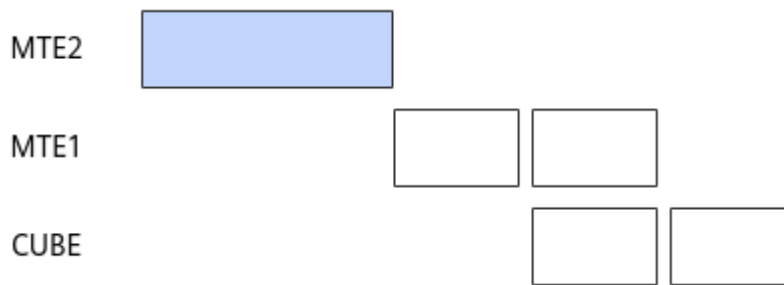
```
#define ASCENDC_CUBE_ONLY
#include "lib/matmul_intf.h"

using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, AType>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BType>;
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_NORM> matmulObj; // 使用CFG_NORM定义Matmul对象
```

----结束

下图是MDL模板的Matmul计算流水示意图，MTE2一次性从Global Memory搬运多个基本块到A1或B1，每次搬运stepM * stepKa个基本块到A1或搬运stepN * stepKb个基本块到B1。MDL模板的优势为MTE2一次性搬运多个基本块，带宽利用率高，后续的MTE1流水能尽可能复用A1或B1的缓存数据，MTE2重复搬运次数少。MDL模板的劣势为MTE2头开销时间较长，MTE1流水需要等待MTE2流水完成后才启动，MTE1启动时间晚。

图 3-140 MDL 模板流水示意图



Matmul API使能MDL模板的完整样例请参考[Matmul API性能优化样例](#)。使能MDL模板的主要步骤如下：

步骤1 创建Matmul对象，使用默认的MDL模板参数CFG_MDL。

```
#define ASCENDC_CUBE_ONLY
#include "lib/matmul_intf.h"
```

```
using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, AType>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BType>;
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_MDL> matmulObj; // 使用CFG_MDL定义Matmul对象
```

----结束

验证优化方案性能收益

- 优化后的Profiling数据如下，从C列的aic_time数据可以看出，多个核中最大算子执行耗时为53.4us，相较于优化前的83.68us有较大提升。从L列的aic_mte2_time数据可以看出，MTE2平均耗时下降较多，从优化前的75.64us降低至46.24us。

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
23	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- 优化后的流水图如下，MDL模板相较于默认的Norm模板，MTE2可以一次性搬运多个基本块，整体的MTE2搬运次数减少了。同时因为MTE2一次搬运多个基本块到A1/B1，后续的MTE1流水能尽量复用A1/B1的缓存数据，减少了流水同步等待，提升了算子整体性能。如红框所示，第一个基本块（baseM*baseN）的计算需要调用16次MMAD指令（singleCoreK/baseK=16），从左侧的第1个MMAD指令调用开始，到右侧的第16个MMAD指令调用结束耗时约5.198us，较优化前的10.899us提升较大，其中流水同步等待时间大幅减少。



总结

大Shape输入、MTE2搬运次数多，且MTE1流水等MTE2流水的同步等待耗时较长的场景下，可以使能MDL模板。通过实现MTE2从Global Memory一次性搬入多个基本块到A1或B1，使后续的MTE1流水能尽量复用A1/B1的缓存数据，减少MTE2的搬运次数，从而提升算子性能。

3.7.4.5 Matmul 高阶 API 使能 UnitFlag

案例介绍

本案例呈现了在矩阵乘算子场景中，使用Matmul高阶API进行矩阵乘法计算，使能UnitFlag功能对算子性能的提升效果。UnitFlag功能为AIC核中MMAD计算指令和FIXPIPE数据搬运指令提供了基于内存访问的细粒度同步，使计算与搬运流水并行。使能UnitFlag功能的方式为将MatmulConfig中的enUnitFlag参数设置为true。enUnitFlag参数的详细介绍请参考MatmulConfig。

- 使能UnitFlag的适用场景
算子的MMAD流水和FIXPIPE流水之间串行执行，FIXPIPE等待MMAD计算完成才搬出结果，这个指令同步等待的时间在算子整体执行耗时中占比较高。这种场景可以使能UnitFlag功能，以获得MMAD和FIXPIPE流水并行的性能收益。如果算子原本的MMAD、FIXPIPE流水可以被其他流水掩盖（比如MTE2 Bound），这时使能UnitFlag功能总体收益很小。
- 使能UnitFlag的约束条件
 - UnitFlag功能仅支持Norm、IBShare、MDL三个模板。
 - 使能UnitFlag功能时，不支持算子内同时存在CO1(L0C)搬出到Global Memory和A1(L1)搬出到Global Memory的两种流水。
 - 使能UnitFlag功能时，若同时使能L0C累加功能，不支持多次Iterate计算、一次GetTensorC输出。

本案例的算子规格如下：

表 3-26 算子规格

输入	Shape	Data type	Format
a	128, 64	float16	ND

输入	Shape	Data type	Format
b	64, 30720	float16	ND

当前案例使用的AI处理器共20个核，每个核包含1个AIC核和2个AIV核。

算子的Tiling参数如下：

- 原始shape：M=128, N=30720, K=64。
 - 单核shape：按20个AIC核进行切分，singleCoreM=128, singleCoreN=1536, singleCoreK=64。
- 对于B矩阵，沿着N轴进行切分，切分成20份singleCoreN，单核上处理K * SingleCoreN大小的数据。对于A矩阵，M轴不进行切分即singleCoreM=M，单核上处理singleCoreM * K大小的数据。总共20个核参与计算。
- 基本块shape：baseM=128, baseN=256, baseK=64。
 - L1相关Tiling参数：stepM=1, stepN=1, stepKa=4, stepKb=4, depthA1=8, depthB1=8。

获取性能数据

使用msProf工具获取[算子仿真流水图](#)和[上板Profiling](#)数据。因为UnitFlag功能主要优化MMAD和FIXPIPE流水串行问题，所以获取性能数据后重点分析Cube、FIXPIPE的流水情况。

分析主要瓶颈点

- 优化前的流水图如下。如下图中红框所示，每一轮MMAD计算流水和FIXPIPE数据搬出流水之间都是串行执行的，完成MMAD计算后才开始FIXPIPE数据搬出，考虑实现MMAD与FIXPIPE之间流水并行来优化算子性能。



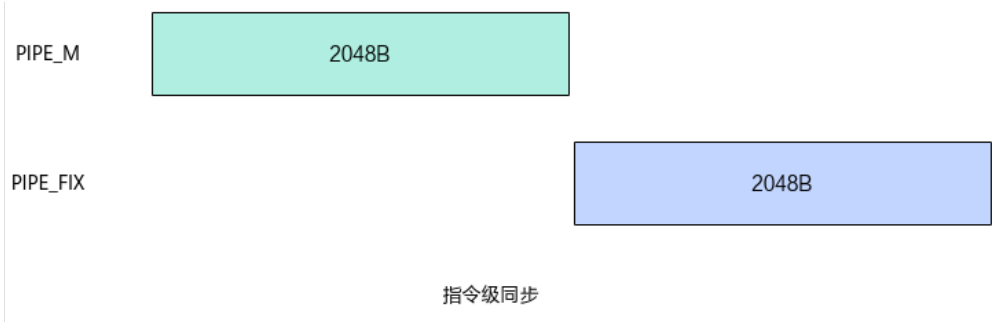
- 优化前的Profiling数据如下，从C列的aic_time数据可以看出，多个核中最大算子执行耗时为37.39us。

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	P	Q
1	block_id	sub_block_id	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scaler_time(us)	aic_scaler_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_fixpipe_time(us)	aic_fixpipe_ratio	
2	0	cube0	37.081818	61185	29.732534	0.801031	15.361818	0.414269	23.261818	0.627311	25.78906	0.695389	0.001212	0.000032	4.222636	0.1139
3	1	cube0	37.366062	61654	30.055151	0.804344	15.609091	0.417734	23.529091	0.629691	26.121819	0.699079	0.001212	0.000032	4.24303	0.113553
4	2	cube0	36.990303	61034	29.70303	0.802996	15.457576	0.417882	23.255152	0.628682	25.852728	0.698906	0.001212	0.000033	4.231515	0.114395
5	3	cube0	37.250908	61464	29.84424	0.801119	15.730303	0.42228	24.411213	0.630353	25.544243	0.687795	0.001212	0.000033	4.215757	0.113172
6	4	cube0	37.270302	61496	29.969696	0.804117	15.617576	0.419035	23.583637	0.632773	25.811516	0.692549	0.001212	0.000033	4.233333	0.113585
7	5	cube0	37.34	61611	29.949091	0.802065	15.644849	0.418984	23.772728	0.636656	26.015151	0.697148	0.001212	0.000032	4.217576	0.112951
8	6	cube0	37.39212	61697	30.06303	0.80394	15.729697	0.420669	23.605091	0.631392	26.205454	0.700328	0.001212	0.000032	4.216393	0.112751
9	7	cube0	37.237576	61442	30.138182	0.809349	15.596788	0.418999	23.527273	0.631815	26.075151	0.700238	0.001212	0.000033	4.217576	0.112821
10	8	cube0	37.226665	61424	30.141519	0.809654	15.6206	0.419755	23.454243	0.630259	25.430906	0.683127	0.001212	0.000033	4.221212	0.113092
11	9	cube0	37.24485	61454	29.879999	0.802259	15.726666	0.422251	23.496967	0.628535	25.818181	0.687832	0.001212	0.000033	4.221212	0.11363
12	10	cube0	37.071514	61168	29.705454	0.801301	15.628485	0.421577	23.441818	0.63234	25.810303	0.688138	0.001212	0.000033	4.229997	0.114096
13	11	cube0	37.367771	61656	29.956364	0.801674	15.590182	0.417429	23.518182	0.629379	25.569099	0.677097	0.001212	0.000032	4.212121	0.112722
14	12	cube0	37.201817	61383	30.159393	0.810697	15.540606	0.417738	23.350304	0.627666	25.81394	0.693889	0.001212	0.000033	4.226666	0.113615
15	13	cube0	37.313938	61568	29.918182	0.801796	15.650303	0.419422	23.551516	0.631172	25.297575	0.677966	0.001212	0.000032	4.22	0.113094
16	14	cube0	37.333333	61633	30.071516	0.800666	15.732121	0.42117	23.567079	0.630444	25.555151	0.684146	0.001212	0.000032	4.208485	0.112967
17	15	cube0	37.067272	61161	29.976969	0.808718	15.569697	0.420039	23.645454	0.637906	25.347879	0.683834	0.001212	0.000033	4.211515	0.113618
18	16	cube0	37.289696	61528	30.038788	0.805552	15.653334	0.419776	23.352121	0.626235	25.808994	0.695618	0.001212	0.000033	4.214545	0.113022
19	17	cube0	37.230301	61463	30.041819	0.80465	15.605454	0.418935	23.378181	0.627597	25.845454	0.695517	0.001212	0.000033	4.228363	0.113385
20	18	cube0	37.33091	61596	30.142424	0.807439	15.56	0.418813	23.595798	0.632037	25.207273	0.675239	0.001212	0.000032	4.20606	0.11287
21	19	cube0	37.1513	61663	30.176363	0.80747	15.62606	0.418128	23.579393	0.630946	25.627274	0.685743	0.001212	0.000032	4.221818	0.112969

设计优化方案

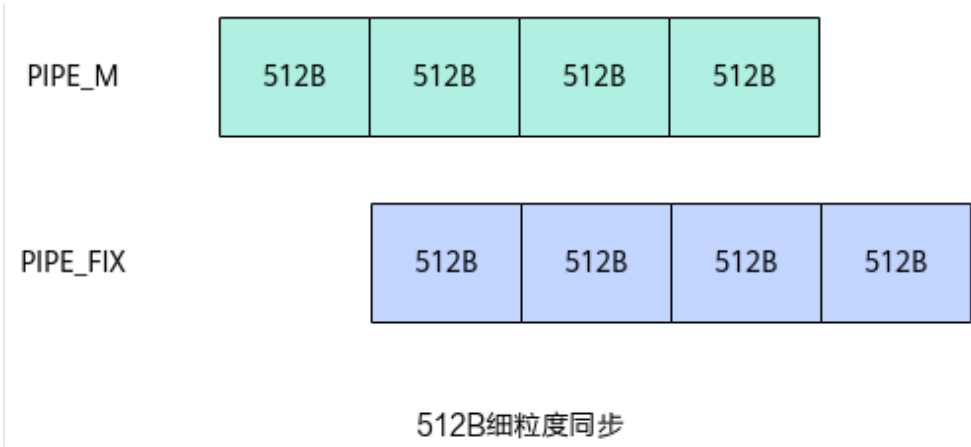
如下图所示，未开启UnitFlag功能时，MMAD和FIXPIPE是指令级别的同步，FIXPIPE指令需要等MMAD指令执行完成才进行结果搬出，MMAD和FIXPIPE之间流水串行。

图 3-141 未开启 UnitFlag 功能



如下图所示，开启UnitFlag功能时，MMAD和FIXPIPE指令是512B大小的细粒度同步。在一条MMAD指令执行过程中，每当完成一个512B数据结果的计算，FIXPIPE立即开始搬出该512B的数据，从而实现MMAD和FIXPIPE之间的流水并行，提升算子性能。

图 3-142 开启 UnitFlag 功能



Matmul API使能UnitFlag功能的完整样例请参考[Matmul API性能优化样例](#)。使能UnitFlag功能的主要步骤如下：

步骤1 自定义MatmulConfig模板参数，将其中的enUnitFlag参数设置为true，使能UnitFlag功能。

```
__aicore__ inline constexpr MatmulConfig GetCustomMDLCFG()
{
    auto mmCfg = CFG_MDL;
    mmCfg.enUnitFlag = true;
    return mmCfg;
}
constexpr static MatmulConfig CUSTOM_CFG_MDL = GetCustomMDLCFG();
```

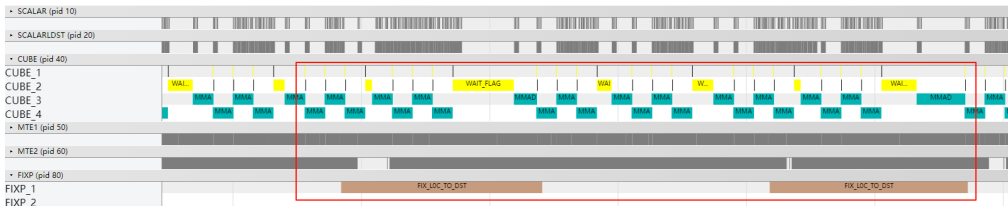
步骤2 基于自定义的MatmulConfig模板参数，创建Matmul对象。

```
using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, AType>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BType>;
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CUSTOM_CFG_MDL > matmulObj;
```

----结束

验证优化方案性能收益

- 优化后的流水图如下，MMAD计算流水和FIXPIPE数据搬出流水之间实现了流水并行。



- 优化后的Profiling数据如下，从C列的aic_time数据可以看出，多个核中最大算子执行耗时为34.66us，较优化前的37.39us有约7.3%的性能提升。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	P	Q
1	block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_fixpipe_time(us)	aic_fixpipe_ratio		
2	0	cube0	33.975151	56059	30.435908	0.895681	15.521818	0.456888	23.843636	0.701796	25.042425	0.737081	0.001212	0.000036	19.341211	0.569275
3	1	cube0	33.990007	55920	30.450455	0.897157	15.399394	0.454381	23.263031	0.686409	25.246941	0.744921	0.001212	0.000036	19.29697	0.569385
4	2	cube0	34.104244	56272	30.777576	0.902456	15.52303	0.455164	23.64303	0.693258	24.845454	0.731447	0.001212	0.000036	19.452122	0.570372
5	3	cube0	34.37394	56717	30.891516	0.89889	15.886484	0.461643	23.854546	0.693972	24.893333	0.724192	0.001212	0.000036	19.443029	0.565633
6	4	cube0	34.073534	56221	30.4	0.892183	15.566061	0.456884	23.418787	0.687205	24.889091	0.730457	0.001212	0.000036	19.309696	0.56671
7	5	cube0	34.124241	56305	30.469091	0.892887	15.524242	0.454933	23.451512	0.68676	24.950302	0.731161	0.001212	0.000036	19.301819	0.565634
8	6	cube0	34.097576	56261	30.572727	0.896625	15.886061	0.460304	23.527273	0.689995	25.069696	0.735234	0.001212	0.000036	19.174545	0.562345
9	7	cube0	34.370911	56712	30.889697	0.901626	15.839699	0.460767	23.61212	0.687799	24.833333	0.715774	0.001212	0.000036	19.512121	0.567957
10	8	cube0	34.237876	56492	30.820606	0.900188	15.772121	0.460667	23.693333	0.692027	24.734545	0.722439	0.001212	0.000036	19.586061	0.572063
11	9	cube0	34.22121	56485	30.551516	0.892745	15.924242	0.461826	23.47394	0.685947	24.867772	0.729595	0.001212	0.000036	19.533333	0.570796
12	10	cube0	33.907879	55948	30.389696	0.896243	15.611515	0.46041	23.284243	0.686691	25.255152	0.744817	0.001212	0.000036	19.21394	0.566651
13	11	cube0	34.656364	57183	30.970909	0.893657	15.888485	0.458488	23.775152	0.686026	24.510303	0.707238	0.001212	0.000036	19.656364	0.566602
14	12	cube0	34.187878	56410	30.838787	0.902209	15.665485	0.458217	23.524242	0.686037	24.826666	0.729206	0.001212	0.000036	19.544849	0.571889
15	13	cube0	34.21212	56450	30.439394	0.899725	15.624845	0.456811	23.704243	0.692861	24.533333	0.717095	0.001212	0.000036	19.445454	0.568379
16	14	cube0	34.353939	56654	30.930304	0.900342	15.904848	0.462297	23.661818	0.688766	24.37394	0.709495	0.001212	0.000036	19.437576	0.565803
17	15	cube0	33.852726	55957	30.502424	0.901053	15.615798	0.461285	23.546667	0.695662	24.817879	0.733899	0.001212	0.000036	19.292605	0.569444
18	16	cube0	34.05062	56110	30.552122	0.898432	15.514545	0.456229	23.409697	0.688398	24.847273	0.733613	0.001212	0.000036	19.257576	0.566298
19	17	cube0	34.058788	56197	30.600607	0.898464	15.696394	0.46095	23.459394	0.686791	24.821818	0.731729	0.001212	0.000036	19.342424	0.567913
20	18	cube0	34.18	56397	30.763021	0.900053	15.565454	0.455397	23.610909	0.696633	24.82909	0.730347	0.001212	0.000036	19.529697	0.571112
21	19	cube0	33.933334	55990	30.579393	0.901161	15.622424	0.460386	23.390909	0.68932	25.064848	0.73865	0.001212	0.000036	19.304848	0.568905
22																

总结

在算子的MMAD计算流水和FIXPIPE数据搬出流水串行且未被其他流水掩盖（比如MTE2 Bound）时，考虑使能UnitFlag功能，实现MMAD计算流水和FIXPIPE数据搬出流水的流水并行，提升算子性能。

3.7.4.6 Matmul 高阶 API 使能 Tiling 全量常量

案例介绍

本案例呈现了在使用Matmul高阶API进行矩阵乘法计算时，使能Matmul Tiling全量常量对算子性能的提升效果。Matmul API在初始化和迭代过程中有大量Scalar计算，Matmul初始化时的Scalar计算影响指令头开销，Matmul迭代间的Scalar计算可能阻塞MTE2流水。在调用Matmul API实现矩阵乘法时，使用MatmulApiStaticTiling参数替代TCubeTiling变量参数，将Scalar计算提前到编译期进行，以减少运行时的Scalar计算开销，实现算子性能的提升。

- Matmul Tiling常量化的适用场景：
 - Matmul初始化时的Scalar计算较多，影响指令头开销。
 - Matmul迭代之间的Scalar计算较多，阻塞MTE2流水。
- Matmul Tiling常量化需要在编译期确定部分Tiling参数，根据确定参数的不同，分为全量常量化和部分常量化两种场景，使用Matmul Tiling常量化需要满足两种场景中任一场景的条件：
 - 全量常量化：能够确定常量singleCore Shape（singleCoreM/singleCoreN/singleCoreK）和常量base Shape（basicM/basicN/basicK，也称baseM/baseN/baseK）。
 - 部分常量化：能够确定常量base Shape（basicM/basicN/basicK，也称baseM/baseN/baseK）。

其中，全量常量化场景比部分常量化场景可以减少更多的Scalar计算开销。

本案例的算子规格如下:

表 3-27 算子规格

輸入	Shape	Data type	Format
a	128, 64	float16	ND
b	64, 30720	float16	ND

当前案例使用的AI处理器共24个核，每个核中包含1个AIC核和2个AIV核。

Tiling参数如下:

- 原始shape: M=128, N=30720, K=64。
- 单核shape: 按24个AIC核进行切分, singleCoreM=128, singleCoreN=1280, singleCoreK=64。

对于B矩阵，沿着N轴进行切分，切分成24份的singleCoreN，单核上处理K * singleCoreN大小的数据。对于A矩阵，M轴不进行切分即singleCoreM=M，单核上处理singleCoreM * K大小的数据。总共24个核参与计算。

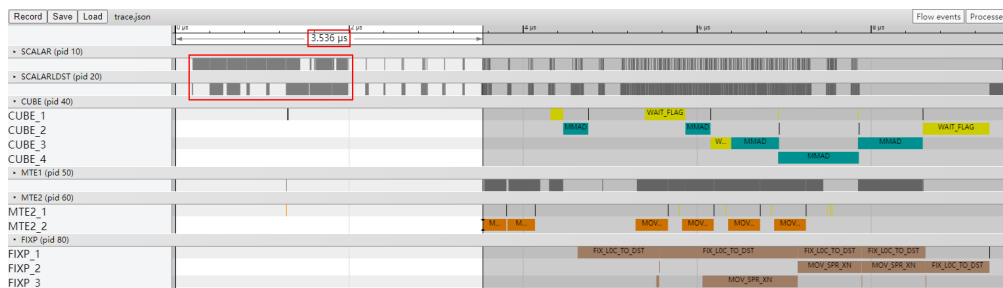
- 基本块shape: baseM=128, baseN=256, baseK=64。
- L1相关Tiling参数: stepM=1, stepN=1, stepKa=4, stepKb=4, depthA1=8, depthB1=8。

获取性能数据

使用msProf工具获取[算子仿真流水图](#)和[上板Profiling](#)数据。相较于基础场景，Tiling常量化在编译期间将部分或全部Tiling参数由变量转化为常数值，在算子执行时直接使用常量化的Tiling参数，可以减少Scalar性能开销，所以重点分析Scalar流水。

分析主要瓶颈点

- 优化前的流水图如下，默认不使能Tiling常量化，Tiling参数需要从Host侧拷贝到Kernel侧，导致Matmul初始化时的Scalar计算较多，第一个MTE2指令开始于3.536us左右，MTE2前的指令头开销在算子整个流水中占比较大，因此需要优化Scalar计算。



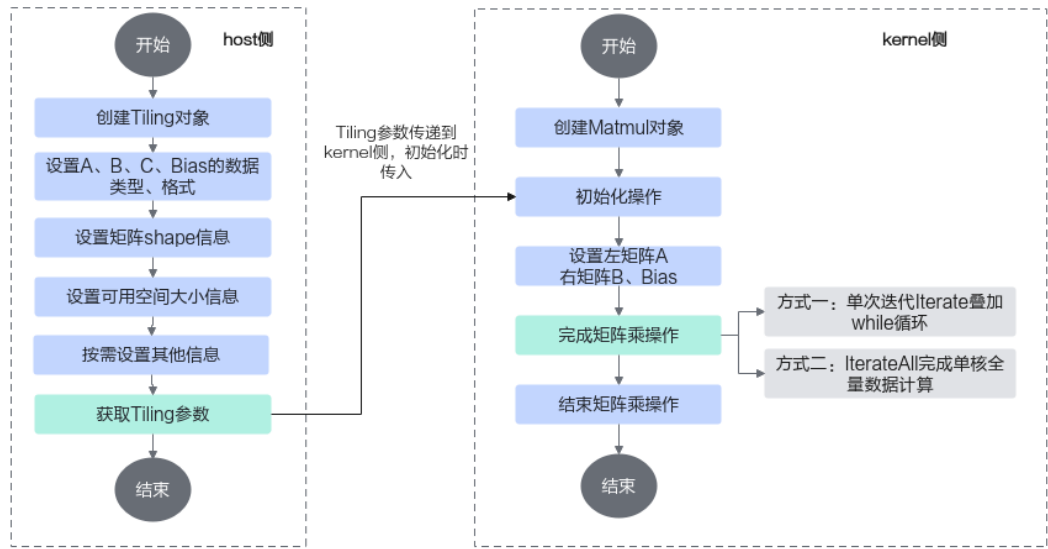
- 优化前的Profiling数据如下，从C列的aic_time数据来看，多个核中最大算子执行耗时为10.62us，从G列的aic_scalar_time数据来看，Scalar平均耗时6.32us。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	DOOR ID	DOOR TYPE	DOOR STATUS	DOOR COLOR	DOOR MATERIAL	DOOR WEIGHT	DOOR HEIGHT	DOOR WIDTH	DOOR DEPTH	DOOR SURFACE AREA	DOOR VOLUME	DOOR MASS	DOOR DENSITY	DOOR STRENGTH	DOOR RESISTANCE	DOOR IMPACT	DOOR ENERGY	DOOR MOMENT	DOOR TORQUE	DOOR POWER
2	1	DOOR	OPEN	RED	STEEL	150.0	2.0	1.0	0.5	0.75	0.375	0.375	7.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
3	2	DOOR	CLOSED	BLUE	ALUMINUM	120.0	1.8	0.9	0.4	0.6	0.288	0.288	5.76	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
4	3	DOOR	OPEN	GREEN	WOOD	180.0	2.2	1.2	0.6	0.84	0.42	0.42	8.4	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
5	4	DOOR	CLOSED	YELLOW	BRASS	90.0	1.5	0.7	0.3	0.45	0.225	0.225	4.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
6	5	DOOR	OPEN	PURPLE	IRON	200.0	2.5	1.5	0.8	1.0	0.5	0.5	10.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
7	6	DOOR	CLOSED	ORANGE	COPPER	110.0	1.7	0.8	0.4	0.68	0.34	0.34	6.8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
8	7	DOOR	OPEN	PINK	STEEL	160.0	2.1	1.1	0.5	0.84	0.42	0.42	8.4	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
9	8	DOOR	CLOSED	BROWN	ALUMINUM	130.0	1.9	1.0	0.5	0.75	0.375	0.375	7.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
10	9	DOOR	OPEN	WHITE	WOOD	190.0	2.4	1.3	0.7	0.96	0.48	0.48	9.6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
11	10	DOOR	CLOSED	BLACK	BRASS	80.0	1.4	0.6	0.3	0.42	0.21	0.21	4.2	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
12	11	DOOR	OPEN	GRAY	IRON	210.0	2.6	1.6	0.9	1.05	0.525	0.525	10.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
13	12	DOOR	CLOSED	PURPLE	COPPER	100.0	1.6	0.7	0.3	0.6	0.3	0.3	6.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
14	13	DOOR	OPEN	PINK	STEEL	170.0	2.3	1.2	0.6	0.96	0.48	0.48	9.6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
15	14	DOOR	CLOSED	BROWN	ALUMINUM	140.0	2.0	1.1	0.6	0.8	0.4	0.4	8.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
16	15	DOOR	OPEN	WHITE	WOOD	185.0	2.4	1.4	0.8	1.08	0.54	0.54	10.8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
17	16	DOOR	CLOSED	BLACK	BRASS	95.0	1.5	0.7	0.3	0.45	0.225	0.225	4.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
18	17	DOOR	OPEN	GRAY	IRON	220.0	2.7	1.7	1.0	1.125	0.5625	0.5625	11.25	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
19	18	DOOR	CLOSED	PURPLE	COPPER	105.0	1.7	0.8	0.4	0.68	0.34	0.34	6.8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
20	19	DOOR	OPEN	PINK	STEEL	175.0	2.3	1.3	0.7	1.05	0.525	0.525	10.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
21	20	DOOR	CLOSED	BROWN	ALUMINUM	145.0	2.0	1.1	0.6	0.8	0.4	0.4	8.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
22	21	DOOR	OPEN	WHITE	WOOD	195.0	2.4	1.4	0.8	1.12	0.56	0.56	11.2	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
23	22	DOOR	CLOSED	BLACK	BRASS	100.0	1.6	0.7	0.3	0.6	0.3	0.3	6.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
24	23	DOOR	OPEN	GRAY	IRON	230.0	2.8	1.8	1.1	1.26	0.63	0.63	12.6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25	24	DOOR	CLOSED	PURPLE	COPPER	110.0	1.7	0.8	0.4	0.68	0.34	0.34	6.8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
26	25	DOOR	OPEN	PINK	STEEL	180.0	2.4	1.4	0.8	1.12	0.56	0.56	11.2	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
27	26	DOOR	CLOSED	BROWN	ALUMINUM	150.0	2.1	1.2	0.6	0.9	0.45	0.45	9.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
28	27	DOOR	OPEN	WHITE	WOOD	200.0	2.5	1.5	0.8	1.25	0.625	0.625	12.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
29	28	DOOR	CLOSED	BLACK	BRASS	105.0	1.7	0.8	0.4	0.68	0.34	0.34	6.8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
30	29	DOOR	OPEN	GRAY	IRON	240.0	2.9	1.9	1.2	1.35	0.675	0.675	13.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
31	30	DOOR	CLOSED	PURPLE	COPPER	115.0	1.8	0.9	0.4	0.72	0.36	0.36	7.2	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
32	31	DOOR	OPEN	PINK	STEEL	185.0	2.5	1.5	0.8	1.25	0.625	0.625	12.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
33	32	DOOR	CLOSED	BROWN	ALUMINUM	155.0	2.2	1.3	0.6	0.99	0.495	0.495	9.9	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
34	33	DOOR	OPEN	WHITE	WOOD	205.0	2.6	1.6	0.9	1.38	0.69	0.69	13.8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
35	34	DOOR	CLOSED	BLACK	BRASS	110.0	1.8	0.9	0.4	0.72	0.36	0.36	7.2	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
36	35	DOOR	OPEN	GRAY	IRON	250.0	3.0	2.0	1.3	1.5	0.75	0.75	15.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
37	36	DOOR	CLOSED	PURPLE	COPPER	120.0	1.9	1.0	0.5	0.75	0.375	0.375	7.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
38	37	DOOR	OPEN	PINK	STEEL	190.0	2.6	1.6	0.9	1.38	0.69	0.69	13.8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
39	38	DOOR	CLOSED	BROWN	ALUMINUM	160.0	2.3	1.4	0.7	1.1	0.55	0.55	11.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
40	39	DOOR	OPEN	WHITE	WOOD	210.0	2.7	1.7	1.0	1.425	0.7125	0.7125	14.25	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
41	40	DOOR	CLOSED	BLACK	BRASS	115.0	1.9	1.1	0.5	0.825	0.4125	0.4125	8.25	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
42	41	DOOR	OPEN	GRAY	IRON	260.0	3.1	2.1	1.4	1.575	0.7875	0.7875	15.75	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
43	42	DOOR	CLOSED	PURPLE	COPPER	125.0	2.0	1.1	0.6	0.8	0.4	0.4	8.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
44	43	DOOR	OPEN	PINK	STEEL	195.0	2.7	1.7	1.0	1.425	0.7125	0.7125	14.25	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
45	44	DOOR	CLOSED	BROWN	ALUMINUM	165.0	2.4	1.5	0.7	1.2	0.6	0.6	12.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
46	45	DOOR	OPEN	WHITE	WOOD	215.0	2.8	1.8	1.1	1.53	0.765	0.765	15.3	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
47	46	DOOR	CLOSED	BLACK	BRASS	120.0	2.0	1.2	0.6	0.8	0.4	0.4	8.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
48	47	DOOR	OPEN	GRAY	IRON	270.0	3.2	2.2	1.5	1.68	0.84	0.84	16.8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
49	48	DOOR	CLOSED	PURPLE	COPPER	130.0	2.1	1.3	0.7	0.825	0.4125	0.4125	8.25	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50	49	DOOR	OPEN	PINK	STEEL	200.0	2.8	1.8	1.1	1.575	0.7875	0.7875	15.75	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
51	50	DOOR	CLOSED	BROWN	ALUMINUM	170.0	2.5	1.6	0.8	1.3	0.65	0.65	13.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
52	51	DOOR	OPEN	WHITE	WOOD	220.0	2.9	1.9	1.2	1.65	0.825	0.825	16.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
53	52	DOOR	CLOSED	BLACK	BRASS	125.0	2.2	1.4	0.7	0.875	0.4375	0.4375	8.75	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
54	53	DOOR	OPEN	GRAY	IRON	280.0	3.3	2.3	1.6	1.725	0.8625	0.8625	17.25	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
55	54	DOOR	CLOSED	PURPLE	COPPER	135.0	2.3	1.5	0.8	0.9375	0.46875	0.46875	9.375	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
56	55	DOOR	OPEN	PINK	STEEL	205.0	2.9	1.9	1.2	1.65	0.825	0.825	16.5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
57	56	DOOR	CLOSED	BROWN	ALUMINUM	175.0	2.6	1.7	0.9	1.365	0.6775	0.6775	13.65	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
58	57	DOOR	OPEN	WHITE	WOOD	225.0	3.0	2.0	1.3	1.8	0.9	0.9	18.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
59	58	DOOR	CLOSED	BLACK	BRASS	130.0	2.4	1.6	0.9	0.975	0.4875	0.4875	9.75	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
60	59	DOOR	OPEN	GRAY	IRON	290.0	3.4	2.4	1.7	1.875	0.9375	0.9375	18.75	0.000000	0.00					

设计优化方案

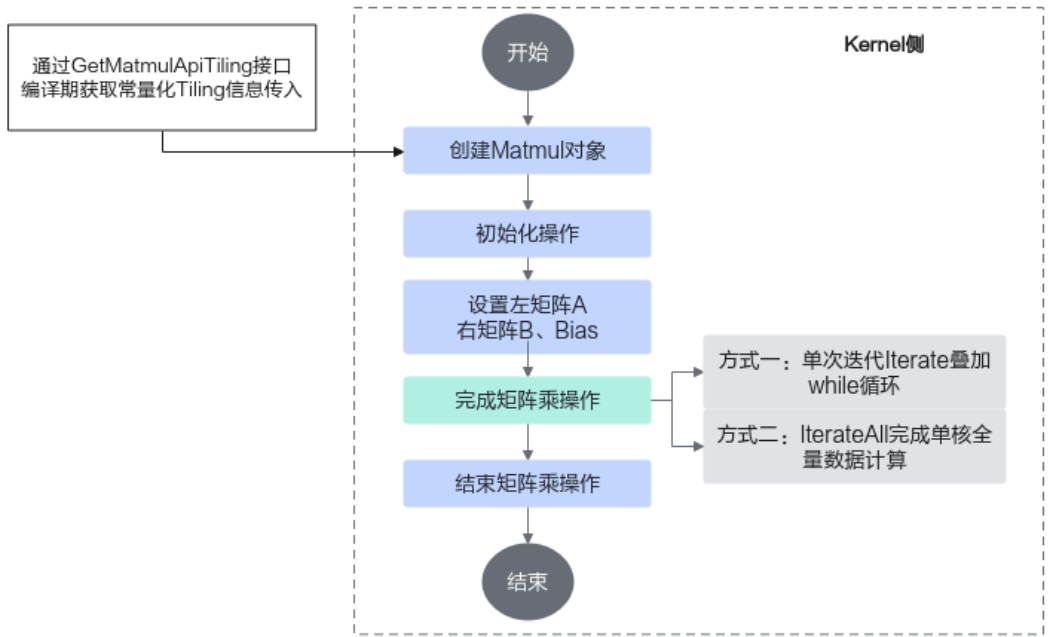
如下图所示，默认不使能Tiling常量化功能时，开发者在host侧创建Tiling对象，通过调用API自动获取Tiling参数。然后将Tiling参数从Host侧传递到Kernel侧，在Kernel侧初始化操作时传入。在算子执行时，使用Tiling变量参数完成矩阵乘操作。

图 3-143 默认不使能 Tiling 常量化的 Matmul 计算流程示意图



如下图所示，使能Tiling常量化功能时，开发者只需要在Kernel侧创建Matmul对象时，调用GetMatmulApiTiling接口在编译期获取常量化Tiling信息，即可完成Tiling常量化。在算子执行时，使用常量化的Tiling参数完成矩阵乘操作，减少Scalar计算开销。

图 3-144 使能 Tiling 常量化的 Matmul 计算流程示意图



Matmul API使能Tiling全量常量化的完整样例请参考[Matmul Tiling常量化的算子样例](#)。使能Tiling全量常量化功能的步骤如下：

- 步骤1** 调用获取MatmulConfig模板的接口GetMMConfig时，使用常数值设置MatmulShapeParams，得到带有常量化参数的自定义MatmulConfig模板CUSTOM_CFG。

```
constexpr int32_t MAX_M = 10000; // custom matmul kernel support max value of M Dim shape
constexpr int32_t MAX_N = 10000; // custom matmul kernel support max value of N Dim shape
constexpr int32_t MAX_K = 10000; // custom matmul kernel support max value of K Dim shape
constexpr int32_t BASE_M = 128; // BASE_M * BASE_K * sizeof(typeA) <= L0A size
constexpr int32_t BASE_N = 256; // BASE_N * BASE_K * sizeof(typeB) <= L0B size
constexpr int32_t BASE_K = 64; // BASE_M * BASE_N * sizeof(typeC) <= L0C size
constexpr MatmulShapeParams shapeParams = { MAX_M,
                                              MAX_N,
                                              MAX_K,
                                              BASE_M,
                                              BASE_N,
                                              BASE_K };
constexpr MatmulConfig CUSTOM_CFG =
    GetMMConfig<MatmulConfigMode::CONFIG_MDL>(shapeParams);
```

- 步骤2** 创建Matmul对象。首先调用GetMatmulApiTiling接口，将Tiling信息常量化，得到常量化模板参数CONSTANT_CFG，包括常量化的Matmul Tiling信息和MatmulConfig模板。创建Matmul对象时，使用常量化模板参数CONSTANT_CFG。

```
using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, aType>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, bType>;
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, cType>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, biasType>;
constexpr static auto CONSTANT_CFG = AscendC::GetMatmulApiTiling<A_TYPE, B_TYPE, C_TYPE,
    BIAS_TYPE>(CUSTOM_CFG);
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CONSTANT_CFG> matmulObj;
```

- 步骤3** 初始化操作。全量常量化时，可以在REGIST_MATMUL_OBJ接口的入参传递Tiling参数的位置，使用空指针替代。部分常量化时，在Kernel侧使用REGIST_MATMUL_OBJ接口初始化Matmul对象时，仍需要使用Tiling。

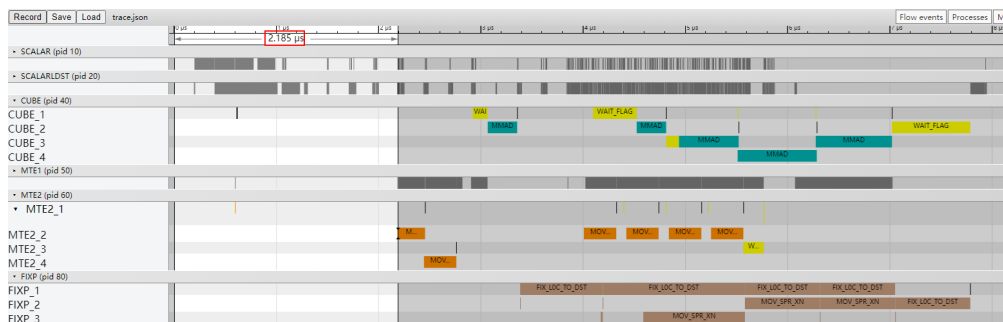
```
// 全量常量化场景，初始化操作示例
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), matmulObj, (TCubeTiling*)nullptr);

// 部分常量化场景，初始化操作示例
REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), matmulObj, &tiling);
```

----结束

验证优化方案性能收益

- 优化后的流水图如下，通过使能Tiling全量常量化，无需将Tiling参数从Host侧拷贝到Kernel侧，在编译期完成Tiling常量化，减少了Matmul初始化时的Scalar计算。从0us起到第一个MTE2指令发起，这之间的时间为Matmul初始化时间，Matmul初始化时间从优化前的3.536us减少到2.185us，性能有所提升。



- 优化后的Profiling数据如下，从C列的aic_time数据来看，多个核中最大算子执行耗时为7.87us，相较于优化前的10.62us提升了25.9%。从G列的aic_scalar_time数据来看，Scalar平均耗时3.38us，相较于优化前的6.32us提升了46.5%。

id	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	0	auto	0.020376	12794	0.040948	0.000232	2.782729	0.000202	0.045057	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
2	1	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
3	2	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
4	3	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
5	4	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
6	5	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
7	6	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
8	7	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
9	8	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
10	9	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
11	10	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
12	11	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
13	12	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
14	13	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
15	14	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
16	15	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
17	16	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
18	17	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
19	18	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
20	19	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
21	20	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
22	21	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
23	22	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
24	23	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
25	24	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
26	25	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
27	26	auto	0.020376	12408	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001
28	27	auto	7.047567	10038	0.040948	0.040948	2.829570	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001	0.020001

总结

算子在调用Matmul API完成矩阵乘计算时，若Matmul初始化时的Scalar计算较多，影响了指令头开销，或Matmul迭代间的Scalar计算较多，阻塞了MTE2流水。在这两类场景下，满足上文提及的Tiling常量化使能条件（**全量常量化**或**部分常量化**），可以考虑使能Tiling常量化，减少Scalar计算开销，提升算子性能。

3.7.4.7 Matmul 高阶 API 使能 L2 Cache 切分

案例介绍

本案例呈现了在Matmul计算过程中，输入和输出的数据总量超过L2 Cache大小时，通过L2 Cache数据切分对算子性能的提升效果。使能L2 Cache切分的完整样例请参考**L2 Cache切分的算子样例**。

本案例使用的AI处理器的L2 Cache大小为192MB，L2 Cache纯读带宽约为GM的3到4倍，两者之间存在较大差距。在搬入或搬出相同数据量的情况下，访问L2 Cache内的数据比访问GM更快。若数据无法命中L2 Cache，即需要访问的数据不在L2 Cache内，导致需要访问GM进行读写，带宽利用率较低，最终算子搬入或搬出数据成为算子整个运行过程的性能瓶颈。

- 使能L2 Cache切分的适用场景
输入和输出的数据量超过L2 Cache的大小。

本案例的算子规格如下：

表 3-28 算子规格

输入	Shape	Data type	Format
a	30720, 1024	float16	ND
b	4096, 1024	float16	ND

获取性能数据

使用msProf工具获取**算子仿真流水图**和**上板Profiling**数据。因为L2 Cache切分功能主要利用带宽更大的L2 Cache，减少MTE2数据搬运开销，所以重点分析MTE2的流水。

分析主要瓶颈点

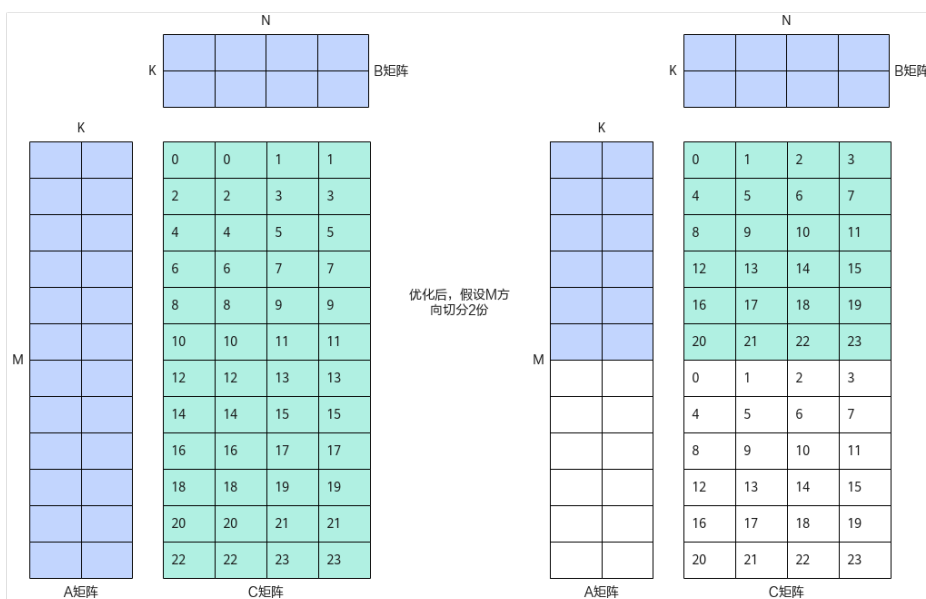
当前案例基于Tiling全量常量化进一步优化，Tiling全量常量化请参考[3.7.4.6 Matmul 高阶API使能Tiling全量常量化](#)案例。优化前的Profiling数据如下，C列的aic_time是867us，K列的aic_mte2_time是861.9us，MTE2占比为99%，MTE2数据搬运是当前算子性能的瓶颈。

Block ID	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	block_id	sub_block	aic_time(us)	aic_total_c	aic_cube	taic_cube	taic_scalar	taic_scalar	taic_mte1	taic_mte1	taic_mte2	taic_mte2	taic_mte3	taic_mte3	taic_fixpipe	aic_fixpipe
2	0	cube0	867.0195	1560635	743.8261	0.857912	275.6295	0.317905	553.735	0.638665	861.9689	0.994175	0.001111	0.000001	134.3511	0.154957
3	1	cube0	866.4894	1559681	744.5584	0.859281	276.6311	0.319255	555.375	0.640948	859.5522	0.991994	0.001111	0.000001	134.8006	0.155571
4	2	cube0	866.7955	1560232	743.9828	0.858314	276.9461	0.319506	553.6789	0.638765	861.8239	0.994264	0.001111	0.000001	134.7617	0.155471
5	3	cube0	869.3867	1564896	743.9955	0.855771	276.0161	0.317484	557.9583	0.641784	861.2433	0.990633	0.001111	0.000001	136.5989	0.157121
6	4	cube0	866.4155	1559548	742.9822	0.857536	276.0467	0.318608	558.6055	0.644732	857.1456	0.989301	0.001111	0.000001	133.7272	0.154345
7	5	cube0	867.6189	1561714	744.0605	0.857589	276.8322	0.319071	557.2839	0.642314	858.1094	0.989904	0.001111	0.000001	135.7422	0.156454
8	6	cube0	867.2844	1561112	742.9639	0.856655	275.4828	0.317638	560.2095	0.645935	856.4794	0.987542	0.001111	0.000001	133.0506	0.153411
9	7	cube0	866.1272	1559029	743.3489	0.858244	275.8489	0.318485	557.39	0.643543	853.5717	0.985504	0.001111	0.000001	133.0356	0.153598
10	8	cube0	867.3917	1561305	742.9006	0.856476	275.4478	0.317559	558.1	0.643423	856.1794	0.987074	0.001111	0.000001	135.6339	0.15637
11	9	cube0	868.9461	1564103	744.1939	0.856433	278.135	0.320083	551.4667	0.634638	864.5028	0.994887	0.001111	0.000001	134.5694	0.154865
12	10	cube0	866.6683	1560003	743.4911	0.857873	276.8239	0.319412	553.7267	0.638914	861.4861	0.994021	0.001111	0.000001	134.4906	0.155181
13	11	cube0	867.2733	1561092	743.6395	0.857445	277.1039	0.319512	553.0695	0.637711	862.9461	0.995011	0.001111	0.000001	135.2517	0.15595
14	12	cube0	867.4889	1561480	743.6367	0.857229	276.1606	0.318345	554.3283	0.639003	861.1934	0.992743	0.001111	0.000001	135.9372	0.156702
15	13	cube0	866.1033	1558986	743.5217	0.858468	276.5183	0.319267	553.9489	0.639588	861.5122	0.994699	0.001111	0.000001	135.82	0.156817
16	14	cube0	866.7022	1560064	744.0677	0.858505	276.3206	0.318818	555.5745	0.641021	859.2367	0.991386	0.001111	0.000001	133.2717	0.153769
17	15	cube0	869.4316	1564977	742.7806	0.854329	275.4756	0.316846	557.4161	0.641127	860.0016	0.989154	0.001111	0.000001	134.0194	0.154146
18	16	cube0	865.9283	1558671	743.1533	0.858216	274.7839	0.317329	557.99	0.644384	854.0439	0.986275	0.001111	0.000001	134.7578	0.155622
19	17	cube0	867.5294	1561553	743.7022	0.857265	276.2833	0.318471	555.9783	0.640875	858.4106	0.989489	0.001111	0.000001	135.6844	0.156403
20	18	cube0	867.3689	1561264	743.2472	0.856899	276.0567	0.318269	555.0605	0.639936	861.1278	0.992805	0.001111	0.000001	135.155	0.155822
21	19	cube0	866.1895	1559141	744.1061	0.859057	277.1489	0.319963	553.74	0.639283	861.4194	0.994493	0.001111	0.000001	135.1089	0.155981
22	20	cube0	867.2083	1560975	743.8272	0.857726	276.3544	0.318671	553.2839	0.638006	861.2522	0.993132	0.001111	0.000001	134.2833	0.154846
23	21	cube0	868.8367	1563906	744.0677	0.856395	276.9517	0.318761	554.5561	0.638274	862.7622	0.993009	0.001111	0.000001	134.3883	0.154676
24	22	cube0	866.3228	1559381	743.9667	0.858764	277.1939	0.319966	552.5367	0.637795	861.8406	0.994826	0.001111	0.000001	134.2789	0.154999
25	23	cube0	867.2939	1561129	744.1261	0.857986	276.6233	0.31895	554.9172	0.639826	861.7117	0.993564	0.001111	0.000001	134.7367	0.155353

设计优化方案

- 优化点一：调整切块大小和计算次数
 - 优化前，输入数据不进行切分，所有核一次计算全部数据。如下图所示，图中数字表示核id，24个核一次计算A和B矩阵的所有数据。
 - 优化后，输入数据被切分多次，所有核分多次计算，每个核单次计算只依赖切分后的数据量。L2 Cache切分方案确保单次计算的数据都在L2 Cache缓存中，搬运输入数据的效率更高。

图 3-145 优化点一示意图

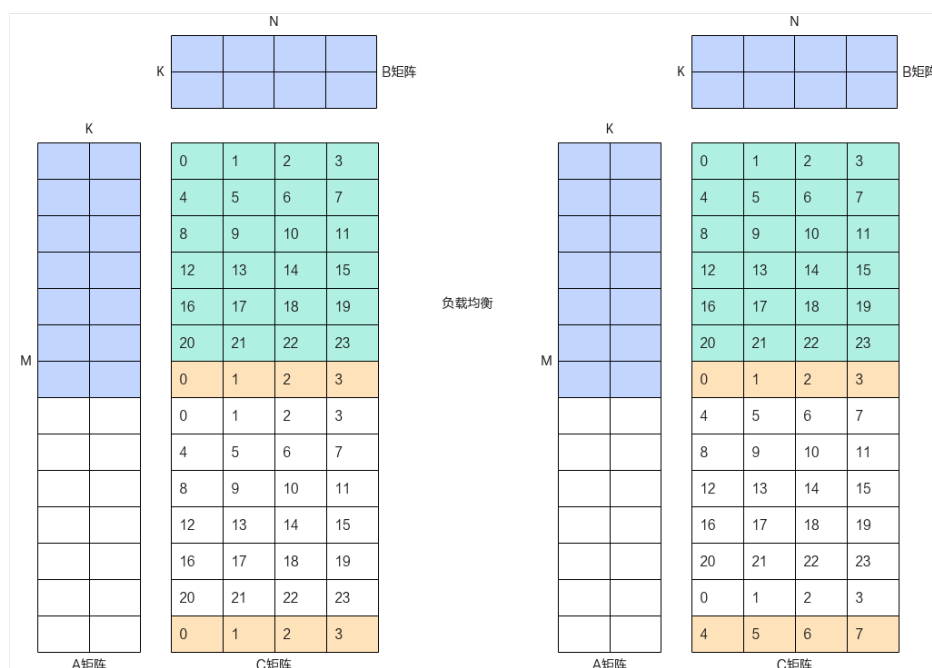


- 优化点二：选择拖尾较小的L2 Cache切分方案
结合[3.6.2.1 核间负载均衡](#)的原理，AI处理器的物理核数固定，当数据进行L2 Cache切分之后，可能出现部分核有计算拖尾的情况，即每次所有核总计算量除以

每个核单次处理的数据量不能被核数整除，导致每次计算的最后需要部分尾核计算剩余数据。而在尾核计算时，部分核始终处于空闲状态，导致算子的整体性能变差。下图中标黄的数据块就是尾块数据，左边方案由于拖尾，每次计算中0、1、2、3核多执行一次处理剩余数据。为达到全局负载最优，调整拖尾核的位置，如右边方案所示，完成所有计算时，0到7核均多一次数据块的计算。

在实际场景中，满足切分后的数据量小于L2 Cache大小的前提下，拖尾越小越好。基于这个原则可以确定L2 Cache切分块数。

图 3-146 优化点二示意图

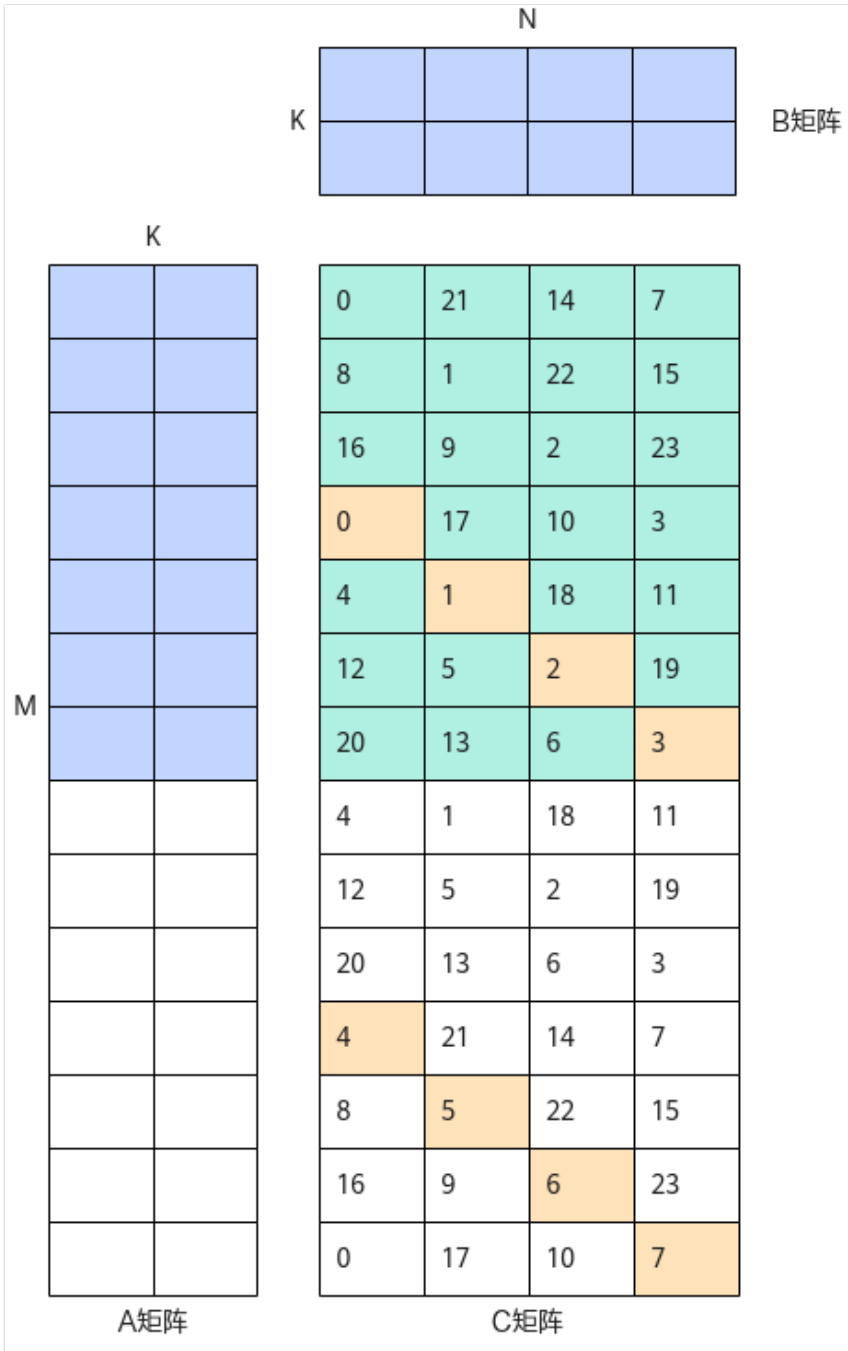


- 优化点三：错位分核，减少左右矩阵同地址冲突问题

同地址冲突：多核并发执行Matmul计算时，如果多核同时访问输入矩阵的相同地址，会导致地址冲突，影响性能。

在M和N方向，将矩阵数据L2 Cache切分为大数据块，然后在数据块间错位分核，即将每个数据块依次沿对角线分配给不同的核处理，从而有效减少同地址冲突的问题。比如，在处理同一行的尾块数据0，1，2，3时，如果顺序分配执行的核，多核会同时读同一行左矩阵数据，导致读读冲突。若按照对角线方式分配执行的核，在对角线上的尾块数据被分配给核0，1，2，3计算，多核访问不同行的左矩阵数据，将减少同地址冲突的次数。

图 3-147 优化点三示意图



Matmul API使能L2 Cache切分的完整样例请参考[L2 Cache切分的算子样例](#)。实现L2 Cache切分的关键步骤如下：

步骤1 判断是否需要进行L2 Cache切分。如果数据总量超过设定的L2 Cache大小，则计算L2 Cache切分数目。

```
bool smallDim = mTileNum_ < L1_MIN_UST_DIM && nTileNum_ < L1_MIN_UST_DIM;
if (smallDim || (!EnableL2Tile())) { // 判断计算数据总量是否小于L2Cache阈值
    mL2TileNum_ = mTileNum_;
    nL2TileNum_ = nTileNum_;
    mL2BlockNum_ = 1;
    nL2BlockNum_ = 1;
    return; // 不需要切分，提前返回
}
```

```
}  
InitL2TileTail(); // 计算L2切分
```

步骤2 基于负载均衡原则，计算L2 Cache切分的份数，m方向L2 Cache切分数：
mL2TileNum_，n方向L2 Cache切分数：nL2TileNum_。

```
int64_t mConflict = INT64_MAX;  
int64_t nConflict = INT64_MAX;  
constexpr bool isNMajor = l1N > l1M; // 根据shape大小，判断主维度  
for (int64_t i = maxMajor; i >= L1_MIN_UST_DIM; i--) {  
    for (int64_t j = maxMinor; j >= minMinor; j--) {  
        if (GetTotalSize(j * l1M, i * l1N, k_) <= L2_TILE_THRESHOLD) { // 确保分块小于L2Cache阈值  
            uint64_t mConflictTmp = AscendC::Ceil(blockNum_, mL2TileNumTailTmp); // 计算负载冲突值  
            uint64_t nConflictTmp = AscendC::Ceil(blockNum_, nL2TileNumTailTmp);  
            if (mConflict >= mConflictTmp && nConflict >= nConflictTmp) { // 若冲突值更小，更新分块数  
量  
                mConflict = mConflictTmp;  
                nConflict = nConflictTmp;  
                mL2TileNum_ = curMajorDim;  
                nL2TileNum_ = curMinorDim;  
            }  
        }  
    }  
}  
}
```

步骤3 错位分核。输入当前数据块的下标，获取按对角线分配的核的下标。

```
__aicore__ inline BlockCoord GetBlockCoord(int64_t tileIdx) {  
    GetCommonTileIndex(tileIdx);  
    int64_t mTileIdx = newBlockIdx_ % mL2TileNumTmp_;  
    mTileIdx = mTileIdx + mL2Idx_ * mL2TileNum_;  
    int64_t nTileIdx = 0;  
    if (mL2TileNumTmp_ != 0 && nL2TileNumTmp_ != 0) {  
        int64_t tmp = newBlockIdx_ / CalcLcm(mL2TileNumTmp_, nL2TileNumTmp_);  
        nTileIdx = (newBlockIdx_ + tmp) % nL2TileNumTmp_;  
    }  
    nTileIdx = nTileIdx + nL2Idx_ * nL2TileNum_;  
    return {mTileIdx * l1M, nTileIdx * l1N, 0};  
}
```

步骤4 设置左右矩阵，根据前序步骤计算的L2 Cache切分数和执行核的下标，循环多次计算Matmul。

```
L2CacheOpt l2Opt(shapes, blockNum); matmulObj.SetOrgShape(shapes.m, shapes.n, shapes.k);  
for (int64_t tileIdx = curBlockIdx; tileIdx < l2Opt.GetTileNum(); tileIdx += blockNum) {  
    auto blockShape = l2Opt.GetBlockShape(tileIdx); // 获取单次计算L2切分块大小  
    if (Get<0>(blockShape) <= 0 ||  
        Get<1>(blockShape) <= 0){  
        return;  
    }  
    auto blockCoord = l2Opt.GetBlockCoord(tileIdx);  
    // 获取当前执行计算的核的下标blockCoord  
    matmulObj.SetTail(Get<0>(blockShape), Get<1>(blockShape), Get<2>(blockShape));  
    const auto& offsetCoord = CalcOffset(shapes, blockCoord); // 基于下标计算矩阵偏移  
    int64_t offsetA = Get<0>(offsetCoord);  
    int64_t offsetB = Get<1>(offsetCoord);  
    int64_t offsetC = Get<2>(offsetCoord);  
    matmulObj.SetTensorA(aGlobal[offsetA], false);  
    matmulObj.SetTensorB(bGlobal[offsetB], false);  
    if (shapes.isBias) {  
        matmulObj.SetBias(biasGlobal);  
    }  
    matmulObj.IterateAll(cGlobal[offsetC]); // 计算L2切分块  
}  
matmulObj.End();
```

----结束

验证优化方案性能收益

优化后的Profiling数据如下，C列的aic_time为805.6us，相比于优化前，总执行时间降低了约7.1%，MTE2搬运时间降低了约10.7%。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	block_id	sub_block	aic_time(us)	aic_total_c	aic_cube_t	aic_cube_r	aic_scalar_t	aic_scalar_r	aic_mte1_t	aic_mte1_r	aic_mte2_t	aic_mte2_r	aic_mte3_t	aic_mte3_r	aic_fixpipe_t	aic_fixpipe_r
2	0	cube0	805.7633	1450374	739.6234	0.917916	284.7161	0.35335	571.9861	0.709869	760.5233	0.943855	0.001111	0.000001	126.0561	0.156443
3	1	cube0	802.1606	1443889	739.5	0.921885	284.6589	0.354865	572.6211	0.713849	757.5767	0.94442	0.001111	0.000001	125.6339	0.156619
4	2	cube0	799.1456	1438462	739.3761	0.925208	284.6972	0.356252	571.7355	0.715434	754.8494	0.944571	0.001111	0.000001	127.3744	0.159388
5	3	cube0	816.015	1468827	740.37	0.9073	284.8033	0.349017	567.6578	0.695646	795.1061	0.974377	0.001111	0.000001	125.8111	0.154177
6	4	cube0	807.84	1454112	740.4239	0.916548	284.7967	0.352541	568.8739	0.704191	783.5178	0.969892	0.001111	0.000001	126.3439	0.156397
7	5	cube0	804.7416	1448535	739.8639	0.919381	284.8022	0.353905	569.0039	0.707064	777.0356	0.965571	0.001111	0.000001	126.69	0.157429
8	6	cube0	818.2178	1472792	741.01	0.905639	285.4111	0.34882	568.1128	0.69433	796.1216	0.972995	0.001111	0.000001	126.7339	0.15489
9	7	cube0	814.1428	1465457	740.8483	0.909974	285.1306	0.350222	569.9772	0.700095	790.7106	0.971219	0.001111	0.000001	125.4344	0.154069
10	8	cube0	807.8306	1454095	740.3572	0.916476	285.1772	0.353016	568.4178	0.703635	782.9439	0.969193	0.001111	0.000001	126.5244	0.156622
11	9	cube0	803.1967	1445754	739.1906	0.920311	285.0422	0.354885	571.3561	0.711353	769.2961	0.957793	0.001111	0.000001	126.6961	0.15774
12	10	cube0	803.5383	1446369	738.8417	0.919485	284.7495	0.354369	571.6644	0.711434	760.8555	0.946881	0.001111	0.000001	126.62	0.157578
13	11	cube0	808.0139	1454425	739.0767	0.914683	285.2744	0.353056	571.0839	0.706775	766.0711	0.948092	0.001111	0.000001	124.3628	0.153912
14	12	cube0	805.0511	1449092	739.3517	0.918391	284.6683	0.353603	571.41	0.709781	764.96	0.950201	0.001111	0.000001	126.4333	0.15705
15	13	cube0	803.7078	1446674	739.2683	0.919822	284.4889	0.353971	570.6089	0.709971	767.9094	0.955459	0.001111	0.000001	126.0356	0.156818
16	14	cube0	803.0511	1445492	739.6194	0.921012	284.525	0.354305	571.5584	0.711733	774.99	0.965057	0.001111	0.000001	125.2233	0.155934
17	15	cube0	817.5378	1471568	741.0917	0.906492	285.6128	0.349357	566.67	0.693142	797.8073	0.975866	0.001111	0.000001	127.4817	0.155934
18	16	cube0	806.5061	1451711	739.1672	0.916505	285.1239	0.35353	571.2061	0.708248	769.6733	0.95433	0.001111	0.000001	126.5522	0.156914
19	17	cube0	800.7289	1441312	738.7955	0.922654	284.2606	0.355002	572.0061	0.714357	756.9067	0.945272	0.001111	0.000001	123.9328	0.154775
20	18	cube0	788.9984	1420197	738.9989	0.936629	284.01	0.359963	573.4628	0.726824	737.2867	0.934459	0.001111	0.000001	123.3733	0.156367
21	19	cube0	806.2889	1451320	739.1967	0.916789	284.4944	0.352844	571.7455	0.709108	768.0311	0.952551	0.001111	0.000001	125.0094	0.155043
22	20	cube0	805.7067	1450272	738.7505	0.916898	284.7761	0.353449	572.4617	0.710509	768.0133	0.953217	0.001111	0.000001	125.3339	0.155558
23	21	cube0	804.2167	1447590	739.4839	0.919508	284.6667	0.353968	571.2539	0.710323	773.0367	0.961229	0.001111	0.000001	125.2983	0.155802
24	22	cube0	797.9305	1436275	739.5328	0.926813	284.2545	0.35624	572.7439	0.717787	745.6906	0.934531	0.001111	0.000001	125.525	0.157313
25	23	cube0	805.1544	1449278	739.39	0.918321	284.7444	0.353652	572.1	0.710547	764.0517	0.94895	0.001111	0.000001	125.0522	0.155315

总结

在Matmul计算数据量超过L2 Cache大小的场景下，可以考虑使能L2 Cache切分，提高L2 Cache命中率，利用L2 Cache高带宽特性提升算子性能。

3.7.4.8 Matmul 高阶 API 使能多核切 K

案例介绍

本案例呈现了在矩阵乘算子场景中，使用Matmul高阶API进行矩阵乘法计算，使能多核切K功能对算子性能的提升效果。为了实现算子在多核上并行执行，提升计算效率，需要将矩阵数据进行切分，切分后的数据块被分配到不同的核上处理。通常情况下，切分矩阵数据时仅切分M、N轴，不切分K轴。若M和N较小，切分M和N轴较困难，此时需要考虑K轴切分；使能多核切K功能后，该场景下可以对矩阵的K轴进行切分，从而使算子在多核上并行执行。由于K轴较大，在该场景下不切分K轴通常会导致单核的输入数据量过大，使能K轴切分后，切分策略能够更有效地平衡输出带宽和输入带宽。

- 使能多核切K的适用场景
 - 矩阵的K轴较大，M轴和N轴相比K轴较小，可以在K轴进行切分，使算子并行执行的核数更多。
 - 矩阵的M轴、N轴和K轴均较大时，可以在K轴进行切分，使切分策略更好地平衡输入和输出带宽。
- 使能多核切K的约束条件
 - 使能多核切K的场景，获取C矩阵结果时仅支持输出到Global Memory。
 - 使能多核切K的场景，需在Kernel侧代码中首次将C矩阵分片的结果写入Global Memory之前，先对Global Memory进行清零，在获取C矩阵分片的结果时，开启AtomicAdd累加。如果不预先清零Global Memory，可能会因为累加Global Memory中的原始无效数据而产生精度问题。
 - 使能多核切K的场景，不支持Bias参与矩阵乘计算。

本案例的算子规格如下：

表 3-29 算子规格

输入	Shape	Data type	Format
a	16, 1024	float16	ND
b	1024, 16	float16	ND

当前案例使用的AI处理器共24个核，算子中使能高阶API Matmul的纯Cube模式。
Tiling参数如下：

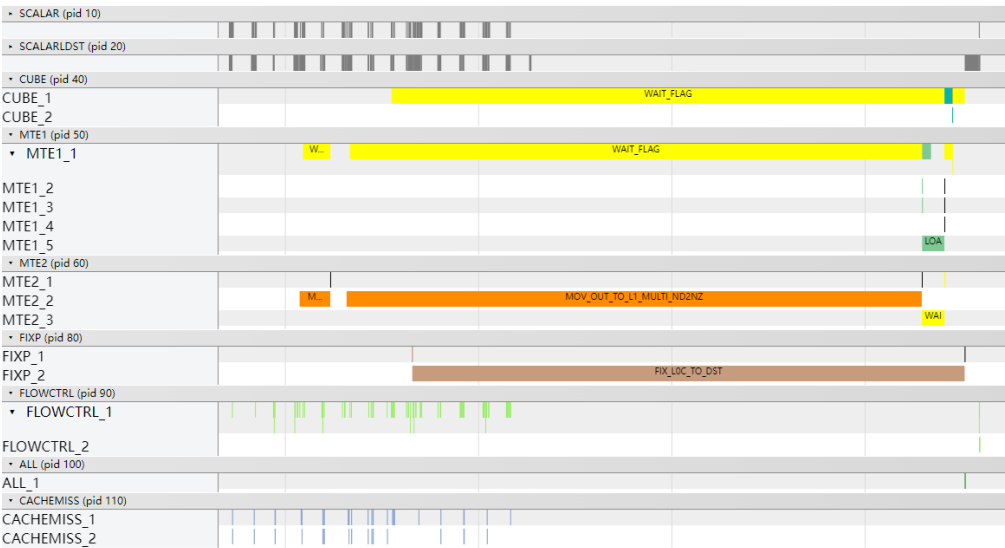
- 原始shape：M=16, N= 16, K=1024。
- 单核shape：未开启多核切K时，singleCoreM=16, singleCoreN=16, singleCoreK=1024；开启多核切K后，singleCoreM=16, singleCoreN=16, singleCoreK=512。

获取性能数据

使用msProf工具获取算子仿真流水图和上板Profiling数据。

分析主要瓶颈点

- 优化前的流水图如下，由于未使能多核切K，且M和N非常小，原始矩阵数据未进行切分，所有数据在单核上进行计算。



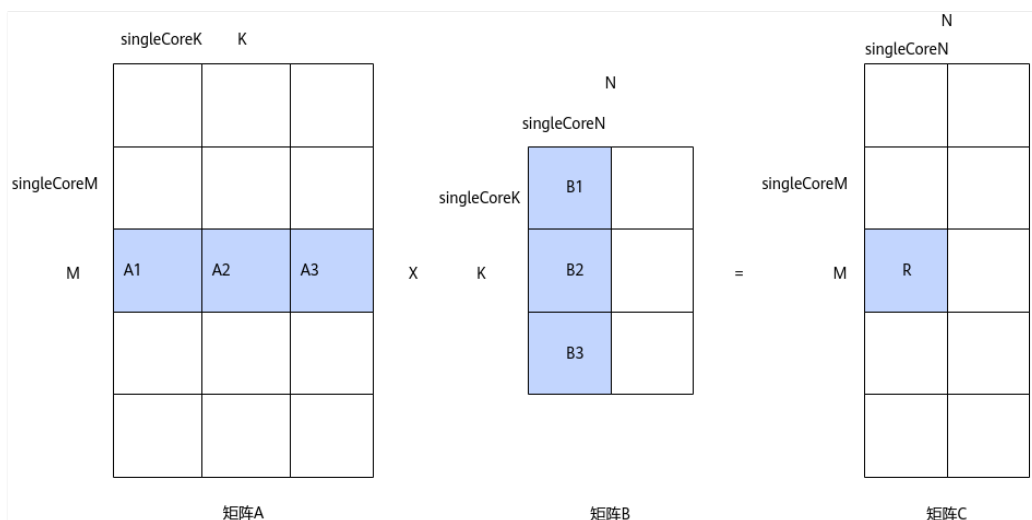
- 优化前的Profiling数据如下，可以看到算子只在单核上执行，aic_time耗时约19.60us，其中aic_mte2_time的平均耗时约为13.72us，aic_mte2_ratio占比较高。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_fixpipe_time(us)	aic_fixpipe_ratio
2	0	cube0	19.598379	36257	0.08973	0.004578	4.004865	0.204347	0.235135	0.011996	13.718919	0.700003	0.001081	0.000055	13.476757	0.667647
3																
4																

设计优化方案

使能多核切K后，矩阵的K方向数据可以进行切分。如下图所示，C矩阵中的R矩阵块，是通过A1*B1+A2*B2+A3*B3累加得到的，其中，A1*B1、A2*B2、A3*B3可在多个核上并行计算。

图 3-148 开启多核切 K



使能多核切K功能的方式为：在GetTiling接口前调用EnableMultiCoreSplitK接口，使能多核切K，并在Kernel实现中，对C矩阵的Global Memory地址清零后开启AtomicAdd。使能多核切K的完整样例请参考[多核切K场景的算子样例](#)。具体步骤如下：

- Tiling实现

通过GetTiling接口获取TCubeTiling结构体前，调用EnableMultiCoreSplitK接口且入参为true，使能多核切K。

```
cubeTiling.SetOrgShape(M, N, K);
cubeTiling.SetShape(M, N, K);
cubeTiling.EnableBias(isBias);
cubeTiling.SetBufferSpace(-1, -1, -1);
// tiling enable split K
cubeTiling.EnableMultiCoreSplitK(true);
if (cubeTiling.GetTiling(tilingData) == -1) {
    std::cout << "Generate tiling failed." << std::endl;
    return {};
}
```

- Kernel实现

调用Fill接口，对C矩阵的Global Memory地址清零。

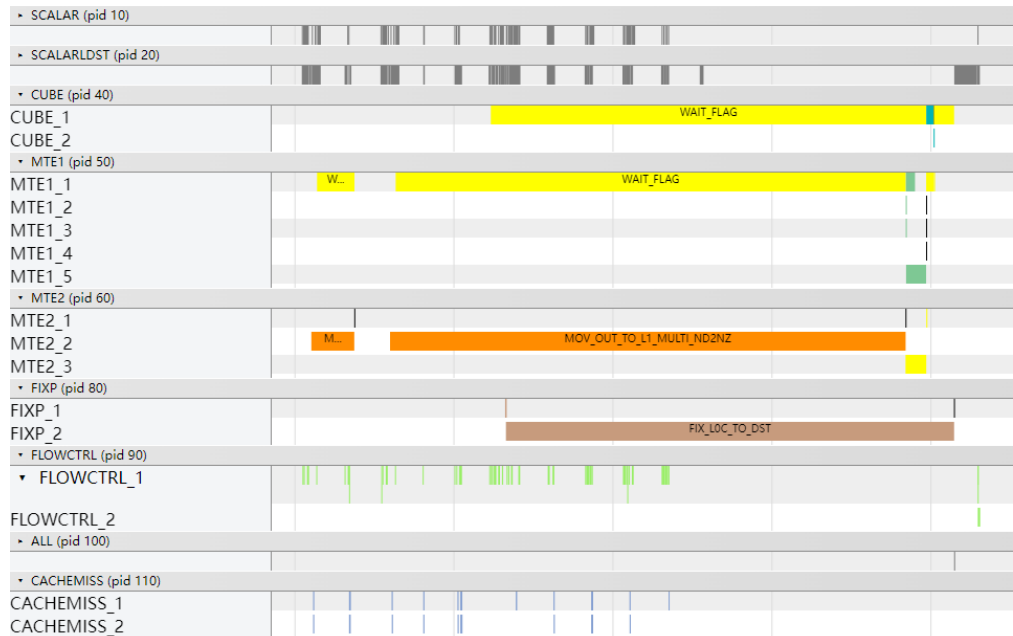
```
cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ cType*>(c), tiling.M * tiling.N);
// clear gm
Fill(cGlobal, tiling.M * tiling.N, (cType)0);
```

调用IterateAll接口，开启AtomicAdd累加，完成矩阵乘操作。

```
// set AtomicAdd
uint8_t enAtomic = 1;
matmulObj.IterateAll(cGlobal, enAtomic);
```

验证优化方案性能收益

- 优化后的流水图如下，开启多核切K后，切分原始矩阵的K方向，单核处理K方向的数据量由原来的1024变为512，单核处理的数据量减半，MTE2流水变短。



- 优化后的Profiling数据如下，可以看到算子在两个核上执行，aic_time平均耗时约为13.70us，较优化前的19.60us有较大提升。

J	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_fixpipe_time(us)	aic_fixpipe_ratio
2	0	cube0	13.406487	24802	0.055676	0.004153	4.048649	0.301892	0.131151	0.009788	7.414595	0.553306	0.001081	0.000081	6.994459	0.522095
3	1	cube0	13.99081	25883	0.057297	0.004095	3.942703	0.281807	0.131351	0.009388	7.90973	0.565352	0.001081	0.000077	7.394595	0.528532
4																
5																

总结

当算子使用Matmul API完成矩阵计算时，原始矩阵的M和N方向无法进行有效切分，且结果输出到Global Memory时，可以考虑使能多核切K功能，实现多核并行，提升计算效率。

3.7.4.9 Matmul 高阶 API 使能多核 K 轴错峰访问内存

案例介绍

本案例呈现在矩阵乘算子场景中，使用Matmul高阶API进行矩阵乘法计算，使能多核K轴错峰访问Device内存对算子性能的提升效果。在多核并行执行Matmul计算时，如果输入矩阵A或B的内存位置位于GM，并且参与多核计算的矩阵相同，那么将出现多核同时访问相同GM地址的情况，导致地址访问冲突，从而影响算子性能。若使能多核K轴错峰访问Device内存，切分的矩阵K轴方向对应的不同核将尽量从不同的GM起始地址开始访问和搬运数据，缓解地址访问冲突，提升算子性能。

图 3-149 访问地址冲突示意图

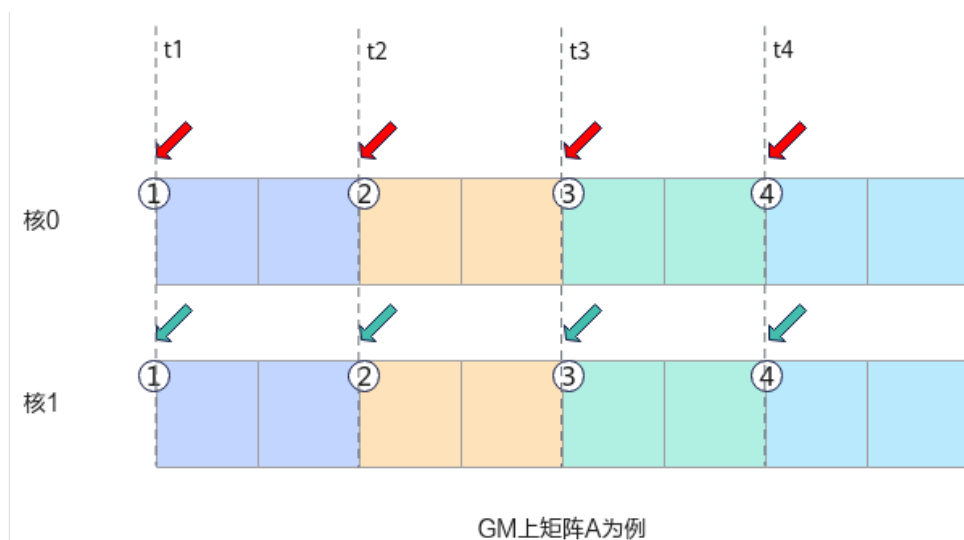
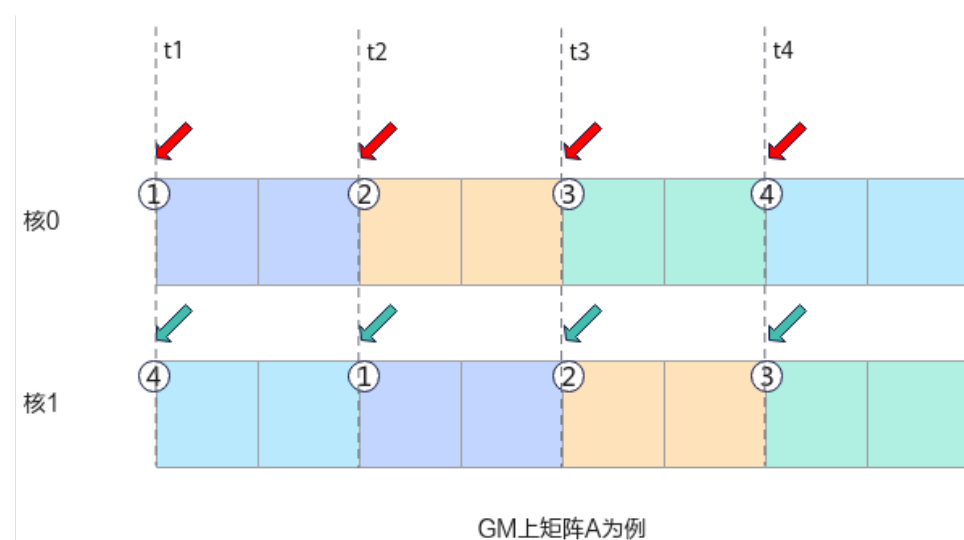


图 3-150 缓解地址冲突示意图



- 使能多核K轴错峰访问内存的适用场景：
多核执行Matmul，且输入矩阵的K轴较大。
- 使能多核K轴错峰访问内存的约束条件：
 - 输入矩阵的K轴非全载，K轴非全载即矩阵的K方向数据不能同时搬入及保持在L1 Buffer中。
 - 仅支持MDL模板。
 - 在多核上执行Matmul计算。
 - A矩阵或B矩阵的内存位置位于GM。

本案例的算子规格如下：

表 3-30 算子用例规格

输入	Shape	Data type	Format
a	768, 6144	float16	ND
b	6144, 2048	float16	ND

获取性能数据

使用msProf工具获取[算子仿真流水图](#)和[上板Profiling](#)数据，重点分析MTE2的流水。

分析主要瓶颈点

优化前的Profiling数据（PipeUtilization.csv）如下所示，aic_mte2_ratio平均达到0.93，MTE2在算子整体执行时长中占比较高，算子当前为MTE2 Bound。本案例中，矩阵按M和N方向切分，单核shape[singleCoreM, singleCoreN, singleCoreK]为[128, 512, 6144]，基本块shape[baseM, baseN, baseK]为[128, 256, 64]，每次加载A矩阵的数据时，多核有概率同时访问同一GM地址，引发地址冲突，导致MTE2搬运效率降低，MTE2执行耗时增加。

block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio
0	cube0	97.181084	179785	55.972973	0.575966	55.87027	0.574909	40.527027	0.417026	91.097298	0.937397
1	cube0	97.074051	179587	56.074596	0.577648	56.449188	0.581506	40.240002	0.414529	90.012436	0.927255
2	cube0	97.836754	180998	56.035675	0.572747	57.169189	0.584332	40.281082	0.411717	90.380539	0.923789
3	cube0	97.211891	179842	55.773514	0.573731	58.014053	0.596779	40.107025	0.412573	90.107025	0.926914
4	cube0	97.215675	179849	55.769188	0.573665	58.704323	0.603857	40.13081	0.412802	91.602699	0.942263
5	cube0	97.799461	180929	55.758919	0.570135	57.950272	0.592542	40.439999	0.413499	90.181625	0.922108
6	cube0	96.235672	178036	55.756218	0.579372	57.925404	0.601912	40.931892	0.42533	87.956215	0.913967
7	cube0	95.905945	177426	56.257298	0.586588	57.697838	0.601609	40.234596	0.419521	89.915138	0.937535
8	cube0	95.840541	176935	55.976215	0.585277	58.24054	0.608952	40.447567	0.422912	89.642159	0.937282
9	cube0	96.136757	177853	55.721622	0.579608	57.837296	0.601615	40.551891	0.421815	89.26973	0.918168
10	cube0	96.471352	178472	55.834595	0.578769	56.701622	0.587756	40.582703	0.420671	89.286484	0.915158
11	cube0	95.916214	177445	55.696217	0.580676	56.547028	0.589546	40.754593	0.424898	88.122704	0.918747
12	cube0	97.657837	180667	55.989189	0.57332	56.523243	0.578789	40.53838	0.415106	91.364326	0.935555
13	cube0	97.375679	180145	56.216217	0.577313	56.554596	0.580788	40.586487	0.416803	91.158379	0.936151
14	cube0	96.977295	179408	56.20108	0.579528	57.469189	0.592605	40.312431	0.415689	91.942703	0.948085
15	cube0	97.511353	180396	55.836758	0.572618	57.203243	0.586632	40.582703	0.416184	90.81189	0.931296
16	cube0	97.162704	179751	55.779461	0.574083	57.581081	0.592625	40.311893	0.414891	90.610268	0.932562
17	cube0	97.564323	180494	56.042702	0.574418	57.611351	0.590496	40.426487	0.414357	91.854057	0.941472
18	cube0	96.501625	178528	55.714054	0.577338	57.474052	0.595576	40.663784	0.421379	89.143784	0.923754
19	cube0	96.171349	177917	55.944324	0.581715	57.375675	0.596598	40.725407	0.423467	89.225403	0.927775
20	cube0	96.37027	178285	55.652973	0.577491	57.369728	0.595305	40.465946	0.419901	89.97081	0.933595
21	cube0	96.087029	177761	55.860542	0.581354	57.825405	0.601802	40.457836	0.421054	89.34919	0.929878
22	cube0	96.438377	178411	55.589188	0.576422	57.696758	0.598276	40.523243	0.420198	89.070267	0.923598
23	cube0	95.808647	177246	55.984863	0.58434	57.725945	0.602513	40.36108	0.421268	89.839996	0.937702

MTE2的搬运效率还可以通过查看其带宽利用率进行验证，如下图所示，通过分析Memory.csv，发现MTE2平均带宽利用率只有34.4%。

验证优化方案性能收益

算子Tiling参数不变，优化后的Profiling数据（PipeUtilization.csv）如下所示。可以看到，MTE2耗时显著降低，MTE2的平均耗时从90us降低到69.87us，最大耗时从91.94us降低到75.82us。

block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio
0	cube0	78.071892	144433	53.95892	0.691144	58.42865	0.748395	43.446487	0.556493	69.223785	0.886667
1	cube0	77.507568	143389	54.162704	0.698805	57.375134	0.740252	43.763783	0.564639	68.304863	0.881267
2	cube0	78.468651	145167	53.987568	0.688015	56.552975	0.720708	43.652973	0.556311	68.991348	0.879222
3	cube0	79.827568	147681	53.952972	0.675869	57.167568	0.716138	43.250271	0.541796	69.38324	0.869164
4	cube0	80.668648	149237	53.994595	0.669338	58.288109	0.722562	43.816216	0.543163	69.952972	0.867164
5	cube0	84.896217	157058	54.560001	0.642667	58.983784	0.694775	42.615135	0.501967	75.561081	0.890041
6	cube0	78.974052	146102	53.936214	0.682961	58.467567	0.740339	43.268108	0.547877	68.018379	0.861275
7	cube0	78.737297	145664	54.020542	0.686086	58.810268	0.746918	43.918919	0.557791	67.791893	0.860988
8	cube0	78.758919	145704	53.978378	0.685362	58.65892	0.744791	43.52919	0.552689	68.038918	0.863888
9	cube0	79.076218	146291	53.92054	0.681881	58.251892	0.736655	43.400002	0.548838	67.408112	0.852445
10	cube0	79.098381	146332	54.211349	0.685366	58.585407	0.740665	43.184864	0.545964	69.188652	0.874716
11	cube0	82.610809	152830	54.591892	0.660832	59.13892	0.715874	42.374054	0.512936	74.741081	0.904737
12	cube0	78.178917	144631	53.951351	0.690101	56.943783	0.728378	43.131893	0.551707	69.227028	0.885495
13	cube0	77.577301	143518	53.989189	0.695941	56.585945	0.729414	43.571735	0.561651	68.375137	0.881381
14	cube0	79.400002	146890	54.136215	0.681816	57.812431	0.728116	44.243782	0.557226	68.981079	0.868779
15	cube0	79.317841	146738	54.033512	0.681228	57.177296	0.720863	43.816757	0.55242	68.676216	0.865836
16	cube0	80.390808	148723	54.009731	0.67184	57.611893	0.716648	43.364864	0.539426	70.575766	0.877907
17	cube0	84.824326	156925	54.648109	0.64425	58.284866	0.687124	42.815136	0.504751	76.032433	0.896352
18	cube0	78.860001	145891	54.025948	0.685087	57.82	0.733198	43.699459	0.55414	68.462166	0.868148
19	cube0	79.150269	146428	54.017296	0.682465	58.177296	0.735023	43.815674	0.553576	68.165947	0.861222
20	cube0	79.089188	146315	53.98811	0.682623	57.604324	0.728346	43.797298	0.553771	68.152435	0.861716
21	cube0	78.625946	145458	53.952972	0.686198	57.782703	0.734906	44.067028	0.560464	67.874054	0.863253
22	cube0	80.194595	148360	53.974052	0.673039	57.955677	0.722688	43.76865	0.545781	70.025406	0.873194
23	cube0	84.536758	156393	54.593513	0.645796	58.799461	0.695549	43.023785	0.508936	75.827026	0.896971

MTE2的带宽利用率（Memory.csv）如下所示，平均带宽利用率提升到41.7%。

read_main_memory_datas(KB)	write_main_memory_datas(KB)	GM_to_L1_datas(KB)	GM_to_L1_bw_usage_rate(%)
9216.875	256.25	9216	42.642628
9216.875	256.25	9216	42.953102
9216.875	256.25	9216	42.42701
9216.875	256.25	9216	41.704773
9216.875	256.25	9216	41.269943
9216.875	256.25	9216	39.214825
9216.875	256.25	9216	42.155495
9216.875	256.25	9216	42.282257
9216.875	256.25	9216	42.270645
9216.875	256.25	9216	42.101028
9216.875	256.25	9216	42.089233
9216.875	256.25	9216	40.299694
9216.875	256.25	9216	42.584248
9216.875	256.25	9216	42.91449
9216.875	256.25	9216	41.929348
9216.875	256.25	9216	41.972782
9216.875	256.25	9216	41.412575
9216.875	256.25	9216	39.248062
9216.875	256.25	9216	42.216465
9216.875	256.25	9216	42.061646
9216.875	256.25	9216	42.094128
9216.875	256.25	9216	42.342133
9216.875	256.25	9216	41.513901
9216.75	256.125	9216	39.381569

查看OpBasicInfo.csv文件，优化后算子整体耗时为85.68us，耗时从98.72us降低到85.68us，性能提升13.2%。

总结

在多核执行Matmul的场景，当输入矩阵K轴较大（一般大于4096）时，可以尝试使用MDL模板并开启K轴错峰访问内存的功能，缓解地址访问冲突，提升MTE2搬运效率，进而优化算子性能。

3.7.4.10 Matmul 高阶 API 使能 NBuffer33 模板

案例介绍

本案例呈现了在矩阵乘算子场景中，使用Matmul高阶API进行矩阵乘法计算，使能NBuffer33模板对算子性能的提升效果。NBuffer33模板的实现为单核计算的A矩阵切分为3x3个基本块，该3x3个A矩阵的基本块全载和保持在L1 Buffer中，每次与3x1个B矩阵的基本块计算矩阵乘，同时DoubleBuffer并行搬入下次计算所需的3x1个B矩阵基本块，直到singleCoreN方向的矩阵乘计算完成。针对MTE2 Bound场景，通过NBuffer33算法的切分数据方式，错开搬运流水，减少单次搬运的数据量，平衡MTE2和FixPipe的数据流量，让两者带宽均匀分布。NBuffer33模板的详细介绍请参考MatmulPolicy。

- 使能NBuffer33模板的适用场景
MTE2 Bound的场景，Tiling参数满足约束条件时，可以使能NBuffer33模板。
- 使能NBuffer33模板的约束条件
 - 仅支持MatmulConfig为MDL模板。
 - A矩阵、B矩阵的内存逻辑位置只支持TPosition::GM。
 - 仅支持纯Cube模式（只有矩阵计算），暂不支持MIX模式（包含矩阵计算和矢量计算）。
 - 仅支持通过IterateAll接口获取Matmul的计算结果C矩阵。
 - stepM、stepKa、stepKb小于等于3，且满足：
 $\text{stepKa} = \text{stepKb} = \text{Ceil}(\text{singleCoreK} / \text{baseK})$ 。
 - A矩阵全载的基本块大小与B矩阵载入的基本块大小之和不超过L1 Buffer的大小。

本案例的算子规格如下：

表 3-31 算子规格

输入	Shape	Data type	Format
a	256, 192	float16	ND
b	192, 512	float16	ND

当前案例使用的AI处理器共24个核，算子中使能高阶API Matmul的纯Cube模式，使用MDL模板，Tiling参数如下：

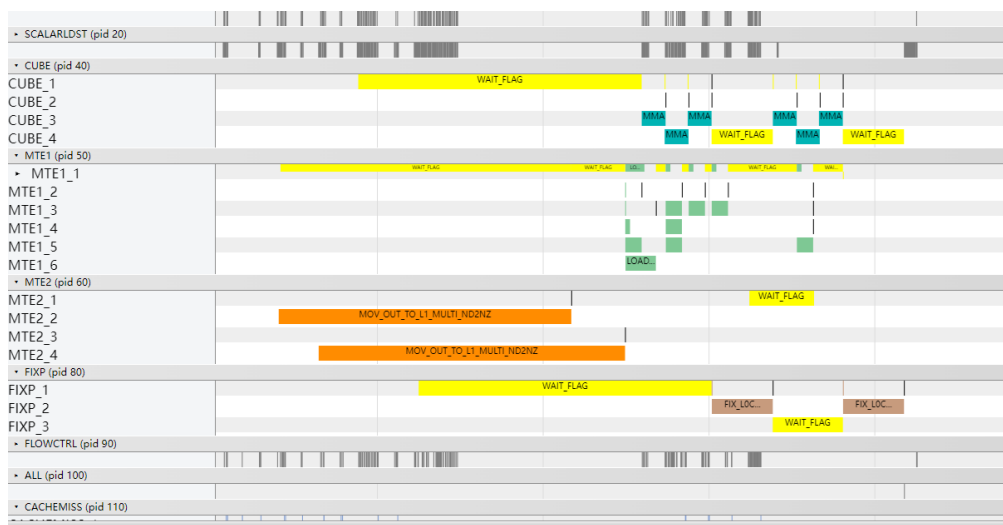
- 原始shape：M=256, N=512, K=192。
- 单核shape：singleCoreM=256, singleCoreN=256, singleCoreK=192。
- 基本块shape：baseM=128, baseN=256, baseK=64。
- L1缓存相关Tiling参数：stepM=2, stepN=1, stepKa=3, stepKb=3。

获取性能数据

使用msProf工具获取[算子仿真流水图](#)和[上板Profiling](#)数据，重点分析Cube、Fixpipe的流水情况。

分析主要瓶颈点

- 优化前的流水图如下，MatmulPolicy的默认模板下A、B矩阵全载，A、B矩阵都只搬运一次。此时MTE2执行时间较长，且流水整体呈串行。



- 优化前的Profiling数据如下，aic_time平均耗时34.01us。

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_fixpipe_time(us)	aic_fixpipe_ratio	aic_icc
2	0	cube0	33.996379	62897	1.707027	0.050039	5.69973	0.163058	1.160541	0.034135	10.097837	0.287009	0.001081	0.000032	15.794054	0.464553	
3	1	cube0	34.034595	62964	1.707027	0.050156	5.121081	0.150467	1.160541	0.034099	9.201622	0.270361	0.001081	0.000032	16.530811	0.485706	
4																	

设计优化方案

使能NBuffer33模板：在GetTiling接口前，调用SetMatmulConfigParams接口开启NBuffer33模式，使获取的Tiling满足要求；Kernel侧在创建Matmul对象时使能NBuffer33模板。使能NBuffer33模板的完整样例请参考[使能NBuffer33模板策略的样例](#)。具体步骤如下：

- Tiling实现

调用GetTiling接口获取TCubeTiling结构体前，开启NBuffer33模式。

```
matmul_tiling::MatmulConfigParams matmulConfigParams(1, false,
    matmul_tiling::ScheduleType::N_BUFFER_33, /* NBuffer33模式 */
    matmul_tiling::MatrixTraverse::NOSET, false);
cubeTiling.SetMatmulConfigParams(matmulConfigParams);
if (cubeTiling.GetTiling(tilingData) == -1) {
    std::cout << "Generate tiling failed." << std::endl;
    return {};
}
```

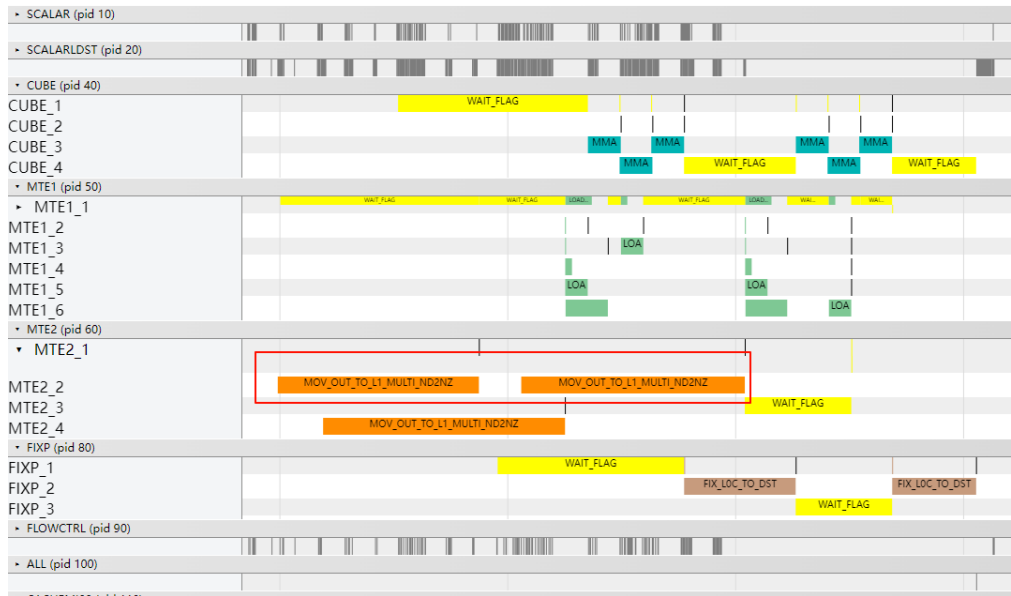
- Kernel实现

设置模板参数MatmulPolicy为NBuffer33模板策略，创建Matmul对象。

```
AscendC::MatmulImpl<
    AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, aType>,
    AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, bType>,
    AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, cType>,
    AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, biasType>, CFG_MD,
    AscendC::MatmulCallBackFunc<nullptr, nullptr, nullptr>,
    AscendC::Impl::Detail::NBuffer33MatmulPolicy> matmulObj;
```

验证优化方案性能收益

- 优化后的流水图如下，Tiling参数不变，但由于stepM为2，NBuffer33模式会将左矩阵数据的搬运拆分为两次。可以看到，第一次MTE2结束后的计算过程（包括MTE1、MMAD和FIXPIPE）可以和第二次MTE2并行。分块搬运数据可以减少一次搬运数据导致的部分头开销，优化加载数据的性能。



- 优化后的Profiling数据如下，aic_time平均耗时32.66us，较优化前的34.01us有所提升。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	BlockId	SubBlockId	AicTime(us)	AicTotalCycles	AicCubeTime(us)	AicCubeRatio	AicScalarTime(us)	AicScalarRatio	AicMTE1Time(us)	AicMTE1Ratio	AicMTE2Time(us)	AicMTE2Ratio	AicMTE3Time(us)	AicMTE3Ratio	AicFixPipeTime(us)	AicFixPipeRatio	AicAllTime(us)
2	0	cube0	32.561623	60239	1.707027	0.052425	4.908108	0.150733	1.143243	0.03511	10.978919	0.337174	0.001081	0.000033	16.62973	0.510716	32.661623
3	1	cube0	32.753513	60594	1.707027	0.052117	5.009189	0.152936	1.159459	0.0354	10.99627	0.323332	0.001081	0.000033	16.128109	0.492408	32.753513
4																	
5																	

总结

MTE2 Bound的场景，Tiling参数满足stepM、stepKa、stepKb小于等于3的条件时，可以考虑使能NBuffer33模板，切分矩阵将搬运流水错开，减少单次搬运的数据量，平衡MTE2和FixPipe的数据流量。

3.7.4.11 Matmul 高阶 API 使能 IBShare 模板共享 B 矩阵数据

案例介绍

本案例呈现了在矩阵乘算子场景中，使用Matmul高阶API进行矩阵乘法计算，B矩阵使能IBShare对算子性能的提升效果。IBShare功能通过共享L1 Buffer上相同的A矩阵或B矩阵数据，减少重复的MTE2数据搬运开销，提升算子性能。该功能支持A矩阵和B矩阵其中一个矩阵使能IBShare，也支持A矩阵和B矩阵同时使能IBShare。

- 使能IBShare的适用场景
MIX场景（包含矩阵计算和矢量计算）下，多个AIV的A矩阵或B矩阵GM地址相同，且多个AIV复用的A矩阵或B矩阵在L1 Buffer上全载。
- 使能IBShare的约束条件
 - A矩阵和B矩阵同时使能IBShare的场景，同一算子中其它Matmul对象的A矩阵和B矩阵也必须同时使能IBShare。
 - A矩阵和B矩阵同时使能IBShare的场景，获取矩阵计算结果时，只支持调用IterateAll接口，且只支持输出到Global Memory。

本案例的算子规格如下：

表 3-32 算子规格

输入	Shape	Data type	Format
a	64, 384	float16	ND
b	384, 256	float16	ND

当前案例使用的AI处理器共20个核，每个核中包含1个AIC核和2个AIV核。因为输入shape较小，本案例以单核为示例，参考SetDim接口在MIX模式下的使用，在Tiling程序中设置参与运算的核数为2。Tiling参数如下：

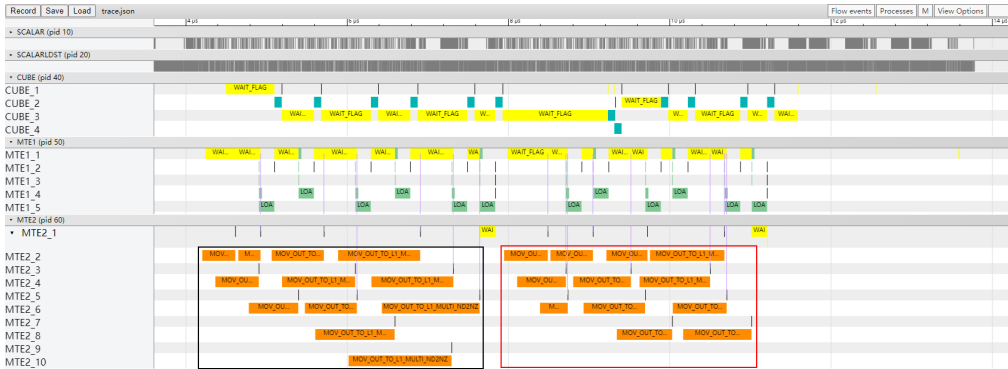
- 原始shape：M=64, N= 256, K=384。
- 单核shape：singleCoreM=32, singleCoreN=256, singleCoreK=384。A矩阵拆成两半，一半在AIV0上处理，一半在AIV1上处理；AIV0和AIV1使用的B矩阵数据相同。
- 基本块shape：baseM=32, baseN=256, baseK=64。
- L1缓存相关Tiling参数：stepM=1, stepN=1, stepKa=6, stepKb=6。

获取性能数据

使用msProf工具获取算子仿真流水图和上板Profiling数据，因为IBShare功能主要是通过共享L1 Buffer上相同的A矩阵或B矩阵数据，减少重复的MTE2数据搬运开销，所以重点分析MTE2的流水情况。

分析主要瓶颈点

- 优化前的流水图如下，不使能IBShare模板，默认使用的Norm模板。黑框标识AIV0发起的MTE2搬运流水：MTE2总共搬运了12次，其中A矩阵搬运了6次（stepM*stepKa=6），B矩阵搬运了6次（stepN*stepKb=6）。红框标识的AIV1发起的MTE2搬运流水，跟AIV0基本一致。在该案例中，因为AIV1使用的B矩阵跟AIV0使用的B矩阵数据相同，且singleCoreN=baseN*stepN，singleCoreK=baseK*stepKb，即B矩阵可以在L1全载。考虑在AIV0搬入B矩阵到L1 Buffer后，将B矩阵数据缓存在L1 Buffer上等待AIV1进行复用，进而节省B矩阵的MTE2重复搬运开销。



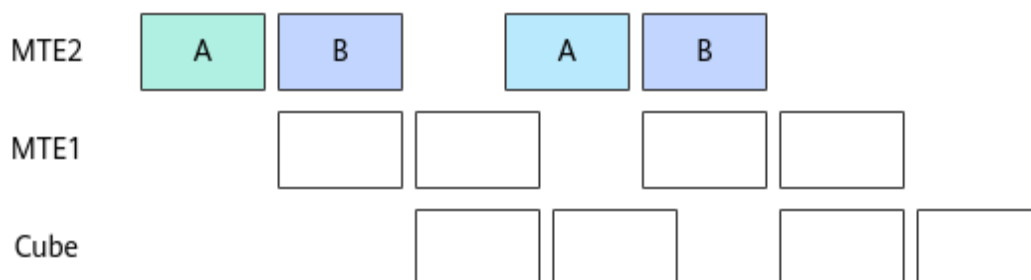
- 优化前的Profiling数据如下，C列的aic_time是10.29us，K列的aic_mte2_time是5.56us。

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time	aic_scalar_ratio	aic_atel_time	aic_atel_ratio	aic_mte2_time	aic_mte2_ratio	aic_atel_ratio	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
2	0	cube0	10.29909	19980	1.30333	0.028959	9.92727	0.233619	2.559788	0.059862	5.567079	0.123733	0.944262	0.00765	4.658182	0.100813
3	0	vector0	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA
4	0	vector1	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA	RA

设计优化方案

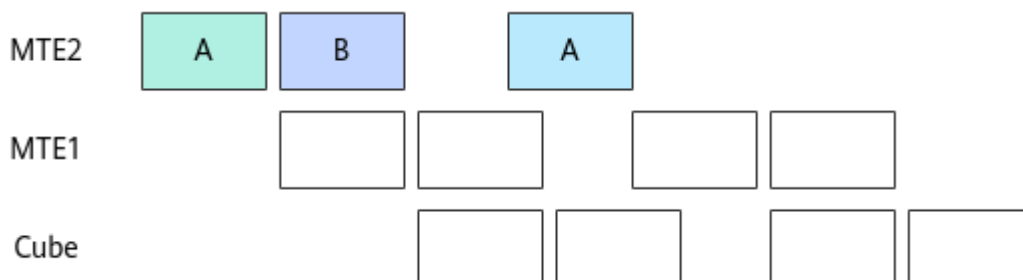
下图是不使能IBShare模板（默认使用Norm模板）的Matmul计算流水示意图。MTE2分多次从Global Memory搬运基本块到A1或B1，即使前后两次搬运的B矩阵基本块数据是相同的数据，也会重复搬运。

图 3-151 不使能 IBShare 模板的 Matmul 流水示意图



下图是使能IBShare模板的Matmul计算流水示意图。MTE2分多次从Global Memory搬运基本块到A1或B1，若前后两次搬运的B矩阵基本块数据相同，不会重复搬运，第一次搬运到B1内的数据会被复用。

图 3-152 使能 IBShare 模板的 Matmul 流水示意图



Matmul API使能IBShare模板共享B矩阵的完整样例请参考[仅B矩阵使能IBShare样例](#)。使能IBShare功能的主要步骤如下：

步骤1 创建Matmul对象。

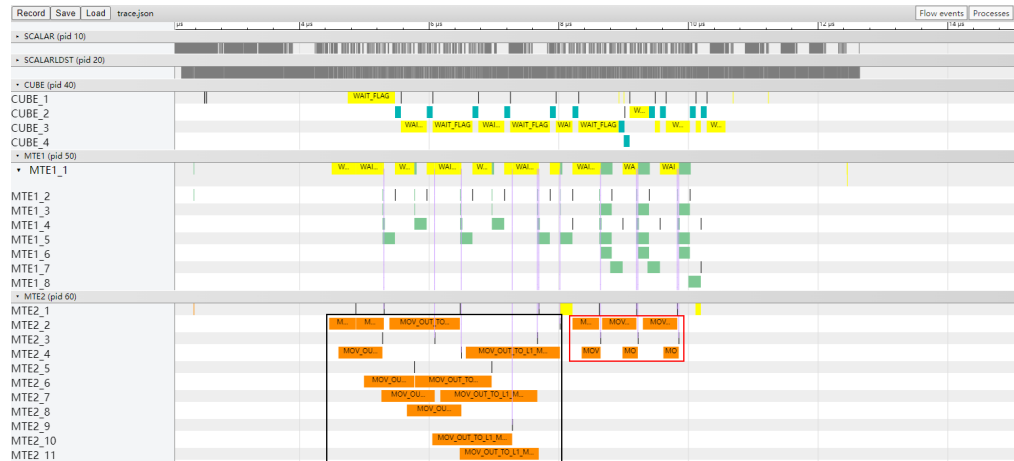
```
#define ASCENDC_CUBE_ONLY
#include "lib/matmul_intf.h"

using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, AType>;
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BType, false,
LayoutMode::NONE, true>; // 设置B矩阵的IBSHARE参数为true
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_IBSHARE_NORM> matmulObj; // 使用默认的
IBShare模板参数CFG_IBSHARE_NORM定义Matmul对象
```

----结束

验证优化方案性能收益

- 优化后的流水图如下，黑框标识的AIV0发起的MTE2搬运流水，与优化前一致。红框标识的AIV1发起的MTE2搬运流水，相较于优化前的A矩阵和B矩阵一共12次MTE2数据搬运，减少到了仅6次A矩阵的MTE2数据搬运，省去了B矩阵的6次MTE2数据搬运开销。



- 优化后的Profiling数据如下，C列的aic_time是9.93us，较优化前的10.29us提升了3.55%。K列的aic_mte2_time是4.71us，较优化前的5.56us提升了15.46%。

[illegible]

总结

MIX场景（包含矩阵计算和矢量计算）下，若多个AIV的A矩阵或B矩阵GM地址相同，且多个AIV复用的A矩阵/B矩阵在L1 Buffer上全载。可以考虑使能IBShare模板，通过共享L1 Buffer上相同的A矩阵或B矩阵数据，减少重复的MTE2数据搬运开销，提升算子性能。

3.7.4.12 Matmul 高阶 API 使能 IBShare 模板共享 A 和 B 矩阵数据

案例介绍

本案例呈现了在融合算子场景中，使用Matmul高阶API进行矩阵乘法计算时，A矩阵和B矩阵同时启用IBShare对性能的提升效果。

该案例的关键优化措施包括：

- 分核逻辑：以Cube核视角分核，Matmul计算结果输出到GM，提供给Vector核进行后续计算。
- 开启IBShare：A矩阵和B矩阵同时开启IBShare。

- 开启IBShare: A矩阵和B矩阵同时开启IBShare。

本案例的算子规格如下:

表 3-33 算子规格

輸入	Shape	Data type	Format
x	128,384	float16	ND
y	384,256	float16	ND

开启IBShare和未开启IBShare的完整样例请参考[MatmulABshare样例](#)和[MatmulNoABshare样例](#)。

获取性能数据

使用msProf工具获取算子的Profiling的数据，重点分析MTE2，Cube，Scalar的流水情况。

分析主要瓶颈点

图 3-153 优化前 Profiling 数据

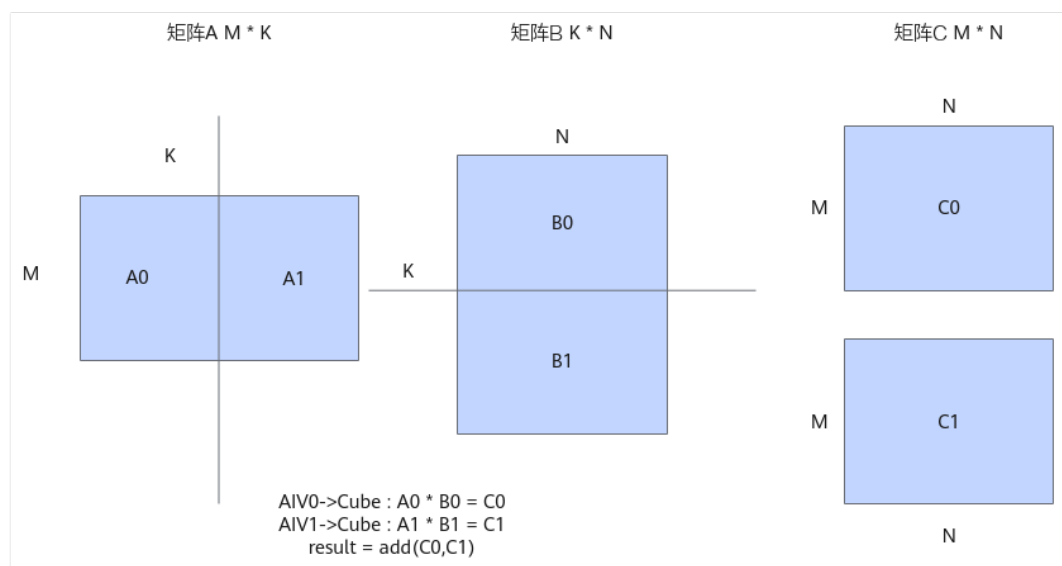
Op Name	Task Type	Task Duration	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size	Allocated size
matmul_noABshare_custom	MIX	AIC	26.661	2.7	263	48589	2.75	0.105	25.753	0.981	4.953	1.989	9.854	0.375	0.118	0.572	0.005	20.74	76.729	0.042	0.002	4.941	0.238	
matmul_noABshare_custom	MIX	AIC	27.341	2.98	2.9	50142	2.75	0.105	26.393	0.98	4.954	1.84	10.048	0.373	0.1506	0.565	0.007	21.09	78033	0.042	0.002	4.934	0.234	
matmul_noABshare_custom	MIX	AIC	27.441	2.98	2.9	50142	2.75	0.105	26.393	0.98	4.954	1.84	10.048	0.373	0.1506	0.565	0.007	21.09	78033	0.042	0.002	4.934	0.234	
matmul_noABshare_custom	MIX	AIC	27.498	2.8	2.8	49614	2.75	0.101	26.777	0.988	4.953	1.83	10.17	0.375	0.1537	0.567	0.005	21.29	78788	0.042	0.002	4.884	0.229	
matmul_noABshare_custom	MIX	AIC	26.8	2.82	2.82	46870	2.75	0.104	26.122	0.989	4.955	1.887	9.959	0.376	0.1539	0.57	0.006	20.91	77352	0.042	0.002	5.004	0.239	
matmul_noABshare_custom	MIX	AIC	26.8	2.82	2.82	46870	2.75	0.104	26.122	0.989	4.955	1.887	9.959	0.376	0.1539	0.57	0.006	20.91	77352	0.042	0.002	5.004	0.239	
matmul_noABshare_custom	MIX	AIC	26.881	2.92	2.95	49389	2.75	0.104	25.956	0.981	4.953	1.887	10.169	0.384	0.1538	0.58	0.005	20.85	77148	0.042	0.002	4.774	0.227	
matmul_noABshare_custom	MIX	AIC	27.041	2.78	2.62	49225	2.75	0.103	26.051	0.98	4.954	1.86	10.25	0.385	0.1598	0.579	0.006	21.07	77959	0.042	0.002	4.748	0.225	
matmul_noABshare_custom	MIX	AIC	27.041	2.78	2.62	49225	2.75	0.103	26.051	0.98	4.954	1.86	10.25	0.385	0.1598	0.579	0.006	21.07	77959	0.042	0.002	4.748	0.225	
matmul_noABshare_custom	MIX	AIC	27.502	3.02	3.72	53034	2.75	0.101	26.672	0.981	4.954	1.82	10.349	0.38	0.1529	0.571	0.005	21.27	79243	0.042	0.002	4.774	0.227	

通过分析以上Profiling数据可以看出，算子执行多次的平均耗时为27.11us，aic_scalar_time的平均耗时为26.27us，当前性能瓶颈点为Cube的Scalar流水。

设计优化方案

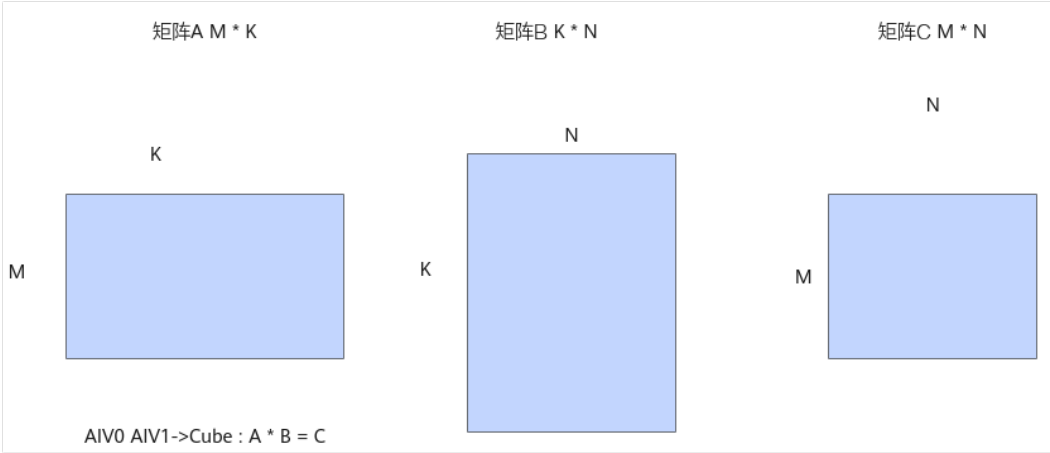
A矩阵和B矩阵均未开启IBShare时，数据需要根据K轴、M轴或N轴进行切分计算。这里以K轴切分为例，未开启IBShare之前，算子以AIV Block为视角进行tiling切分，AIV0发起A0*B0的计算，AIV1发起A1*B1的计算。

图 3-154 未开启 IBShare

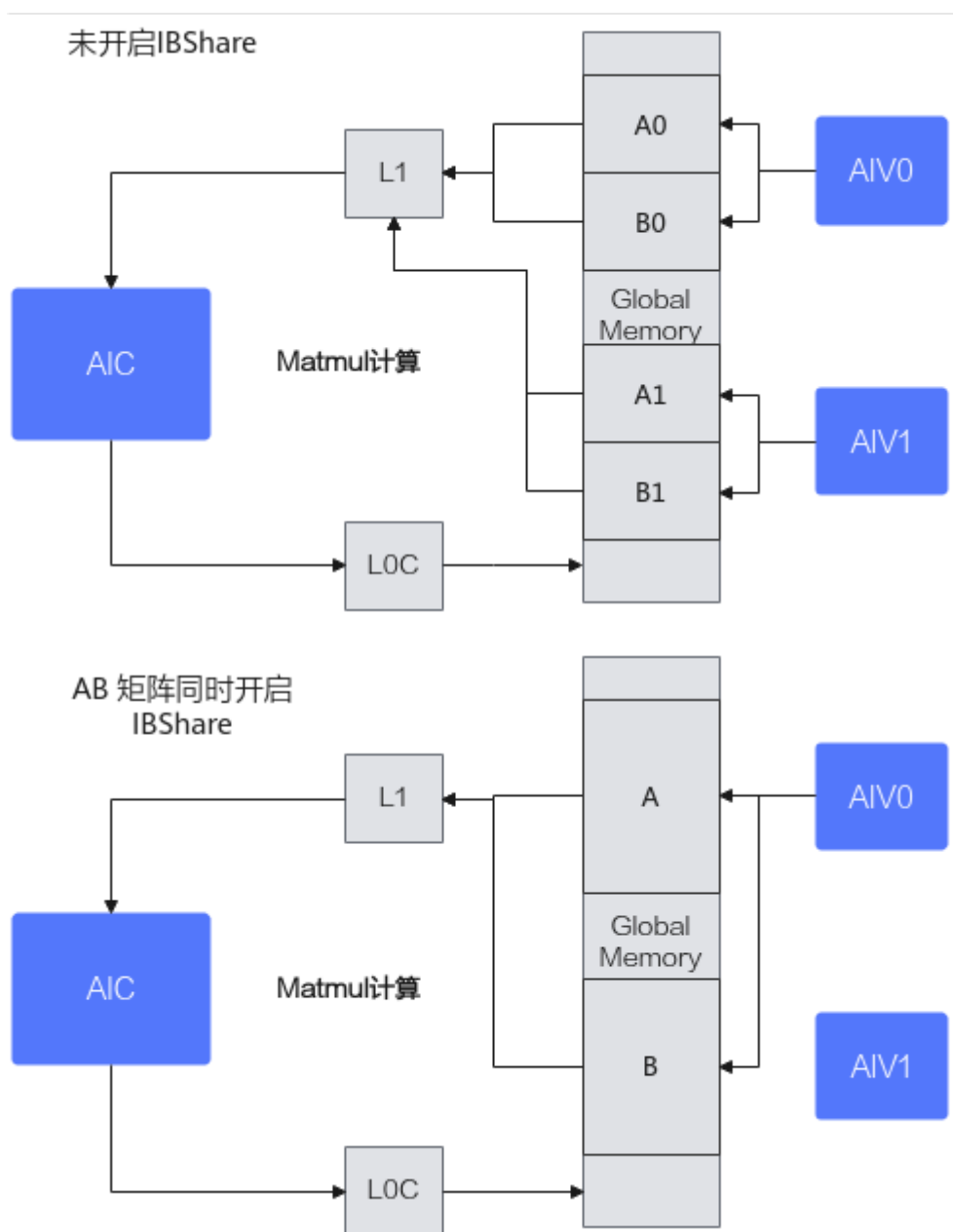


当A矩阵和B矩阵都启用IBShare时，可以一次性加载到L1 Buffer上，省去了切分，分开搬运的过程，同时Cube计算单元完全由AIV0单核驱动，发起一次计算，计算的结果由AIV0和AIV1共享，从而减少Cube响应的次数，减少Scalar计算。

图 3-155 开启 IBShare



开启IBShare和不开启IBShare的数据交互对比示意图如下：



通过设置A和B矩阵MatmulType的IBShare均为true，开启该优化，具体代码如下：

```
constexpr bool isABshare = true;
template <typename aType, typename bType, typename cType> class MatmulABshareKernel {
public:
    __aicore__ inline MatmulABshareKernel(){};
    __aicore__ inline void Init(GM_ADDR a, GM_ADDR b, GM_ADDR c, GM_ADDR workspace,
                               const TCubeTiling &tiling, AscendC::TPipe *pipe);
    __aicore__ inline void Process(AscendC::TPipe *pipe);
    __aicore__ inline void CalcOffset(int32_t blockIdx, const TCubeTiling &tiling, int32_t &offsetA, int32_t
    &offsetB,
                                     int32_t &offsetC);
    AscendC::Matmul<AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, aType, false,
    LayoutMode::NONE, isABshare>,
    AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, bType, false, LayoutMode::NONE,
    isABshare>,
    AscendC::MatmulType<AscendC::TPosition::VECIN, CubeFormat::ND, cType>>
    matmulObj;
```

```
AscendC::GlobalTensor<aType> aGlobal;  
AscendC::GlobalTensor<bType> bGlobal;  
AscendC::GlobalTensor<cType> cGlobal;  
TCubeTiling tiling;  
};  
template <typename aType, typename bType, typename cType>  
__aicore__ inline void MatmulABshareKernel<aType, bType, cType>::Init(GM_ADDR a, GM_ADDR b,  
GM_ADDR c,  
GM_ADDR workspace,const TCubeTiling &tiling, AscendC::TPipe  
*pipe)  
{  
    this->tiling = tiling;  
    aGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ aType *>(a), tiling.M * tiling.Ka);  
    bGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ bType *>(b), tiling.Kb * tiling.N);  
    cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ cType *>(c), tiling.M * tiling.N);  
    int32_t offsetA, offsetB, offsetC;  
    CalcOffset(AscendC::GetBlockIdx(), tiling, offsetA, offsetB, offsetC); // calculate offset  
    aGlobal = aGlobal[offsetA];  
    bGlobal = bGlobal[offsetB];  
    cGlobal = cGlobal[offsetC];  
}  
template <typename aType, typename bType, typename cType>  
__aicore__ inline void  
MatmulABshareKernel<aType, bType, cType>::CalcOffset(int32_t blockIdx, const TCubeTiling &tiling,  
int32_t &offsetA, int32_t &offsetB, int32_t &offsetC)  
{  
    offsetA = 0;  
    offsetB = 0;  
    offsetC = 0;  
}
```

验证优化方案性能收益

优化后执行多次的平均耗时：22.44us，较优化前有较大提升。

图 3-156 优化后 Profiling 数据

Op Name	Task Type	Task Duration	Task V/aicore_aic	total_aic	mac_aic	m/aic	scalar_time/aic	sca_aic	mte_aic	mte2_aic	mte/aic	fixpip/aic	fixpip/aic	icache/aiv	time/aiv	total_c/aiv	vec_time/aiv	vec_rate/aiv	scalar/aiv	scalar_		
matmul_ABshare_custom	MIX_AIC	24.46	2596	241	44557	2.486	0.103	19.46	0.908	4.851	0.201	7.862	0.326	9.533	0.396	0.029	21.81	80694	0.022	0.001	3.831	0.176
matmul_ABshare_custom	MIX_AIC	22.1	2.8	21.7	40167	2.486	0.115	19.718	0.908	4.856	0.224	8.074	0.372	9.759	0.45	0.026	19.5	72157	0.022	0.001	3.761	0.193
matmul_ABshare_custom	MIX_AIC	22.6	2.76	22.3	41213	2.487	0.112	19.883	0.892	4.856	0.218	8.002	0.359	9.719	0.436	0.028	20.08	74306	0.022	0.001	3.912	0.195
matmul_ABshare_custom	MIX_AIC	22	2.86	21.7	40052	2.487	0.115	19.363	0.894	4.847	0.224	7.766	0.359	9.394	0.434	0.027	19.6	72531	0.022	0.001	4.002	0.204
matmul_ABshare_custom	MIX_AIC	21.66	10.58	21.3	39367	2.487	0.117	19.229	0.904	4.857	0.228	7.713	0.362	9.426	0.443	0.027	19.3	71392	0.022	0.001	3.937	0.204
matmul_ABshare_custom	MIX_AIC	21.5	13.88	21.1	39076	2.486	0.118	19.179	0.908	4.867	0.23	7.807	0.37	9.685	0.459	0.026	19.21	71082	0.022	0.001	3.944	0.205
matmul_ABshare_custom	MIX_AIC	21.98	12.86	21.6	39950	2.486	0.115	19.512	0.904	4.845	0.224	7.894	0.366	9.542	0.442	0.027	19.48	72059	0.022	0.001	3.824	0.196
matmul_ABshare_custom	MIX_AIC	22.28	12.94	21.9	40443	2.486	0.114	19.52	0.893	4.843	0.222	8.095	0.37	9.774	0.447	0.027	19.69	72852	0.022	0.001	3.798	0.193
matmul_ABshare_custom	MIX_AIC	23.101	12.7	22.8	42096	2.486	0.109	20.162	0.886	4.852	0.213	8.286	0.364	9.943	0.437	0.027	20.38	75399	0.022	0.001	3.704	0.182
matmul_ABshare_custom	MIX_AIC	22.78	22.6	22.4	41463	2.486	0.111	20.363	0.91	4.835	0.216	8.186	0.365	9.862	0.44	0.026	19.97	73878	0.022	0.001	3.929	0.197

总结

融合算子场景下，Matmul A矩阵和B矩阵同时开启IBShare，以Cube核视角分核，可以有效减少Cube侧的Scalar开销，提升性能。

3.7.4.13 AIV 核上的 ND2NZ 格式转换

案例介绍

本案例展示了在矩阵乘算子场景中，使用Matmul高阶API进行计算，对内轴（内轴即矩阵的行方向）非256字节对齐的输入矩阵，在AIV核上进行ND2NZ格式转换对算子性能提升的效果。为提升Cube单元的计算效率，ND格式的输入矩阵在执行Cube计算前会先转换为NZ格式，ND格式和NZ格式的具体内容可参考[数据格式](#)。Matmul API内部使用随路ND2NZ指令同时进行格式转换以及数据搬运。但在数据非256字节对齐时，随路ND2NZ指令存在带宽利用率低的问题。因此输入矩阵的内轴非256字节对齐时，在进行Matmul计算前，利用AIV核上Vector计算单元完成ND格式到NZ格式的转换，可以避免随路非对齐数据搬运存在的效率低的问题，从而提升算子性能。

- AIV核上的ND2NZ格式转换的适用场景
输入矩阵内轴非256字节对齐，且数据量较大影响随路格式转换的效率。

本案例的算子规格如下：

表 3-34 算子规格

输入	Shape	Data type	Format
a	1024, 1024	float16	ND
b	1024, 4095	float16	ND

当前案例使用的AI处理器共24个核，算子中使能高阶API Matmul的纯Cube模式。使用MDL模板，Tiling参数如下：

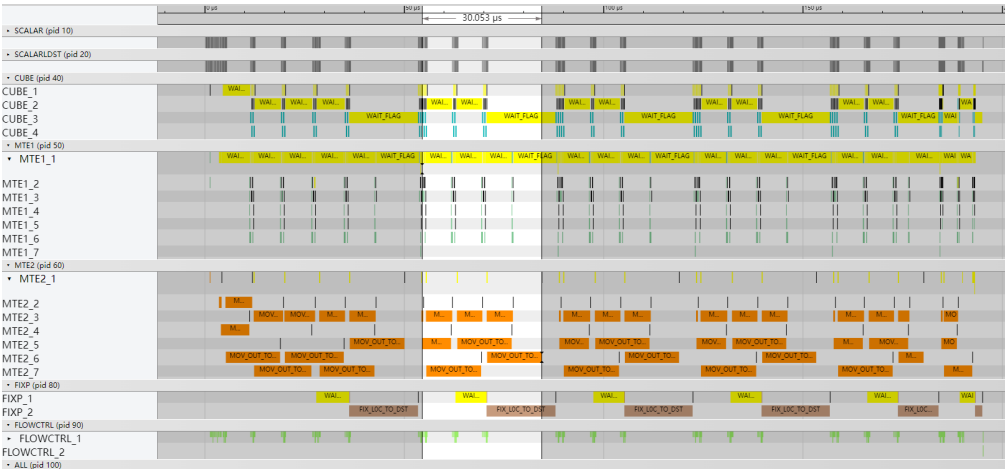
- 原始shape：M=1024, N= 4095, K=1024。
- 单核shape：singleCoreM=128, singleCoreN=1408, singleCoreK=1024。
- 基本块shape：baseM=128, baseN=256, baseK=64。
- L1缓存相关Tiling参数：stepM=1, stepN=1, stepKa=4, stepKb=4。

获取性能数据

使用msProf工具获取[算子仿真流水图](#)和[上板Profiling数据](#)，重点分析MTE2的流水。

分析主要瓶颈点

- 优化前的Cube流水图如下，由于使用了随路ND2NZ指令，在MTE2数据搬运过程中进行数据格式的转换，导致MTE2整体占比较高。



- 优化前的Profiling数据如下，可以看到只使用Cube单元执行计算，aic_time最大耗时149.04us，其中aic_mte2_ratio占比很高。

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_fixp_time(us)	aic_fixp_ratio
2	0	cubeb0	146.770813	271536	25.302344	0.1724	15.518757	0.103877	19.120201	0.130271	137.265411	0.935236	0.001081	0.000007	44.987026	0.300376
3	1	cubeb0	147.20108	272322	25.431892	0.17277	15.783243	0.107222	19.125406	0.129927	136.702164	0.928676	0.001081	0.000007	44.648109	0.300314
4	2	cubeb0	146.638749	271278	25.200001	0.171853	16.196756	0.110455	19.065405	0.130018	134.101074	0.914512	0.001081	0.000007	45.814953	0.311432
5	3	cubeb0	146.13298	270346	25.118919	0.171891	17.397298	0.118777	18.92054	0.129475	133.7146	0.91502	0.001081	0.000007	45.288108	0.309773
6	4	cubeb0	146.461075	270963	25.228107	0.172251	17.389189	0.118729	18.989189	0.129653	135.286386	0.923784	0.001081	0.000007	46.283783	0.316014
7	5	cubeb0	147.120544	272173	25.403784	0.172673	17.611351	0.119707	18.984324	0.129659	134.533508	0.914444	0.001081	0.000007	45.319458	0.308043
8	6	cubeb0	146.650454	271320	25.297838	0.172484	17.236676	0.117522	18.981081	0.129423	134.307704	0.915745	0.001081	0.000007	44.936647	0.307119
9	7	cubeb0	147.360535	272617	25.280001	0.171552	17.397839	0.118063	18.912973	0.128345	135.111893	0.91688	0.001081	0.000007	45.314594	0.307508
10	8	cubeb0	148.939195	275462	26.312433	0.176701	18.130812	0.121757	19.118378	0.12839	136.95459	0.919719	0.001081	0.000007	44.56216	0.296257
11	9	cubeb0	149.05243	275730	26.965541	0.174181	17.865702	0.120654	19.047039	0.127795	135.60554	0.908607	0.001081	0.000007	45.632431	0.306169
12	10	cubeb0	148.67044	275044	26.18973	0.176157	16.801081	0.113007	19.012974	0.127885	136.249725	0.914442	0.001081	0.000007	45.120541	0.30349
13	11	cubeb0	147.55513	272977	25.620108	0.173264	16.372972	0.110962	19.07946	0.125804	134.450896	0.91119	0.001081	0.000007	44.488109	0.301366
14	12	cubeb0	147.6427	273139	25.695135	0.174036	16.272452	0.110215	19.088108	0.125096	137.085403	0.919484	0.001081	0.000007	44.513512	0.301495
15	13	cubeb0	146.95784	271872	25.637837	0.175818	16.062702	0.109801	19.121622	0.130116	136.176224	0.926635	0.001081	0.000007	44.957298	0.305982
16	14	cubeb0	149.042077	275726	26.243784	0.176084	17.125946	0.114908	19.255496	0.123997	137.633113	0.923424	0.001081	0.000007	45.327569	0.304128
17	15	cubeb0	147.550278	272968	26.005405	0.176248	16.989729	0.115145	19.305486	0.130847	135.220535	0.916437	0.001081	0.000007	45.396755	0.30767
18	16	cubeb0	134.483246	248794	21.948648	0.163207	13.971892	0.103893	16.842163	0.125236	121.12162	0.900645	0.001081	0.000008	41.740002	0.310373
19	17	cubeb0	134.522156	248866	22.011892	0.16363	14.162703	0.10543	16.807566	0.124843	121.19043	0.900167	0.001081	0.000008	40.794594	0.302556
20	18	cubeb0	134.323242	248498	21.988649	0.1637	13.918379	0.103619	16.898188	0.125587	120.627571	0.900399	0.001081	0.000008	40.942162	0.304803
21	19	cubeb0	135.361618	250419	22.224865	0.164189	14.327027	0.105843	16.772344	0.123545	121.504868	0.897632	0.001081	0.000008	40.897839	0.302136
22	20	cubeb0	135.399486	250489	22.418788	0.16556	14.425466	0.105848	16.751355	0.123894	121.551888	0.898023	0.001081	0.000008	41.00108	0.303316
23	21	cubeb0	134.051895	247996	22.169189	0.165378	14.4	0.107421	16.827026	0.125526	119	0.897716	0.001081	0.000008	42.005405	0.313552
24	22	cubeb0	135.788108	251134	22.26504	0.163984	14.551934	0.10721	16.748864	0.1235	121.797295	0.89723	0.001081	0.000008	42.975877	0.316584
25	23	cubeb0	135.127583	249966	22.227028	0.164489	14.616476	0.10817	16.693136	0.123107	121.138754	0.897842	0.001081	0.000008	41.464325	0.306853

设计优化方案

对于ND格式的输入矩阵，不再使用随路ND2NZ指令进行格式转换，而是利用Vector计算单元的能力完成数据格式转换。首先使用DataCopyPad接口，将非对齐的矩阵数据搬入Unified Buffer，使用Duplicate接口填充需要补为对齐位置的数据，再逐行调用Copy接口实现数据从ND到NZ格式的重排，将重排后的NZ数据写入workspace内存，最后直接读取workspace上的NZ数据，进行Matmul计算。

AIV核上的ND2NZ格式转换的完整样例请参考[Matmul输入矩阵ND到NZ格式转换的算子样例](#)。实现AIV核上的ND2NZ格式转换的主要步骤如下：

步骤1 创建Matmul对象时，定义内轴非256字节对齐的B矩阵的Format为NZ格式。

```
using A_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, ATYPE, true>;  
// 使用CubeFormat::NZ定义矩阵B的类型信息  
using B_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, AscendC::TPosition::GM, CubeFormat::NZ, BType, true>;  
using C_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, CType>;  
using BIAS_TYPE = AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, BiasType>;  
AscendC::Matmul<A_TYPE, B_TYPE, C_TYPE, BIAS_TYPE, CFG_MDL> matmulObj;
```

步骤2 利用Vector计算单元实现ND2NZ格式转换。如下代码中MatrixBtoNZ为将B矩阵的ND格式转换为NZ格式的函数，该函数的具体实现请参考完整样例代码。

```
// Vector ND2NZ  
if ASCEND_IS_AIV {  
    pipe->InitBuffer(ubBuf, TOTAL_UB_SIZE);  
    MatrixBtoNZ<typename B_TYPE::T>(tempGM, bGMNZ, tiling, isTransB, ubBuf, tiling.baseK,  
        tiling.baseN); // ND2NZ格式转换函数  
    SyncAll();  
    // CV SYNC  
    NotifyEvent<PIPE_MTE3>(4);  
    return;  
}  
if ASCEND_IS_AIC {  
    WaitEvent(4); // 等待Vector完成ND2NZ格式转换  
}
```

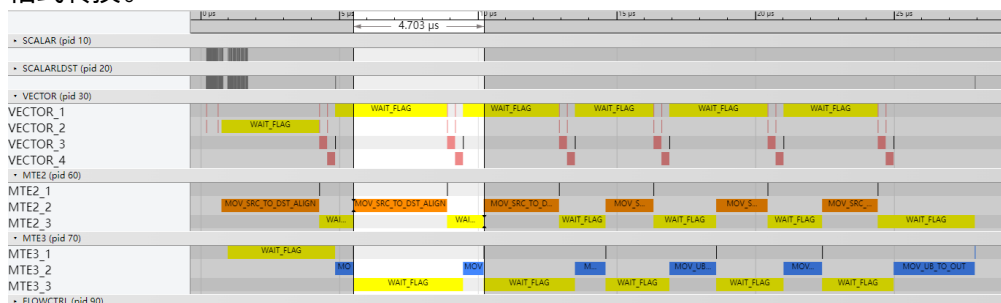
步骤3 设置左矩阵A、右矩阵B、Bias，完成矩阵乘操作。

```
matmulObj.SetTail(tailM, tailN, shapes.k);  
matmulObj.SetTensorA(aGlobal, false);  
matmulObj.SetTensorB(bGlobal, false);  
if (shapes.isBias) {  
    matmulObj.SetBias(biasGlobal);  
}  
matmulObj.IterateAll(cGlobal);
```

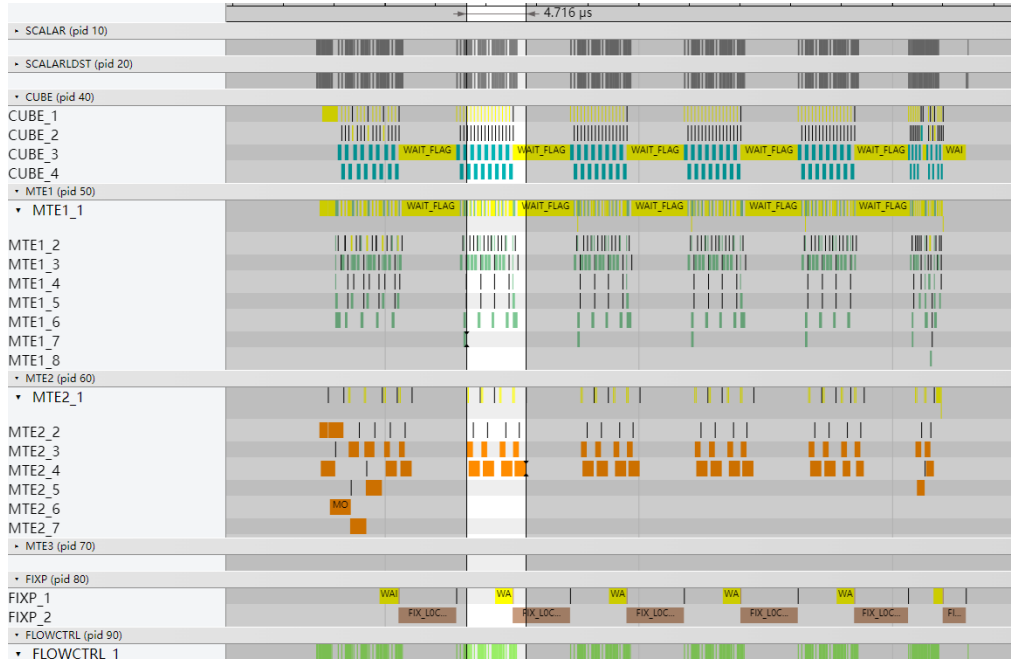
----结束

验证优化方案性能收益

- 优化后的Vector流水图如下所示，利用Vector计算单元的能力，完成B矩阵的数据格式转换。



- 优化后的Cube流水图如下所示，不使用随路ND2NZ指令对B矩阵进行格式转换后，MTE2的占比明显下降。



- 优化后的Profiling数据如下，可以看到同时使用Cube单元和Vector单元，aic_time最大耗时90.95us，其中aic_mte2_ratio占比明显降低。

J	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	block_i	sub_block_j	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)	ac_time(us)
2	0	0	89.361084	65319	24.743244	0.266056	16.994995	0.182738	19.984986	0.214900	24.003242	0.258099	0.384895	0.004138	39.840342	0.433393
3	1	cube0	90.238612	169927	24.97948	0.268596	18.931892	0.203569	19.523785	0.209933	24.832343	0.284766	0.623784	0.006707	41.491894	0.486149
4	2	cube0	89.394992	165390	24.302162	0.262114	19.532433	0.210026	20.028486	0.215382	23.172432	0.249196	0.75027	0.008097	39.99891	0.433009
5	3	cube0	88.974594	164603	24.02919	0.258378	14.525405	0.156187	19.864864	0.213601	24.353242	0.281854	1.531892	0.016472	39.81081	0.438073
6	4	cube0	88.905945	164476	24.395676	0.263319	17.89496	0.150487	20.157297	0.216745	23.936756	0.257384	0.448649	0.004824	39.430899	0.432987
7	5	cube0	89.673537	164017	24.105946	0.260064	19.914695	0.203983	19.971891	0.214752	23.330811	0.230869	0.604865	0.006504	39.748695	0.432954
8	6	cube0	89.884865	166287	24.454594	0.262953	19.276135	0.207726	19.911352	0.214101	24.033314	0.258425	0.572973	0.006161	39.838921	0.438375
9	7	cube0	90.948647	168256	24.961081	0.266399	19.572432	0.210486	20.08919	0.216013	24.253313	0.28679	1.227568	0.0132	40.616756	0.436739
10	8	cube0	89.742165	166023	24.647168	0.265020	15.08243	0.163185	19.967138	0.215019	23.576245	0.253208	0.7	0.007537	40.658458	0.447188
11	9	cube0	89.453514	165489	24.367027	0.262011	17.215675	0.185115	19.906487	0.214048	23.442162	0.252086	1.28973	0.013888	40.426487	0.434693
12	10	cube0	89.071027	164863	24.220541	0.260438	14.544324	0.156391	19.954994	0.214596	23.437296	0.252034	1.98488	0.01343	40.191891	0.432171
13	11	cube0	89.355133	165307	24.481001	0.263237	16.712081	0.179797	20.149711	0.216664	23.868456	0.256832	1.865846	0.013139	40.701891	0.43245
14	12	cube0	89.508652	165591	24.66054	0.265167	17.28108	0.195818	20.131351	0.216466	23.154584	0.248974	1.66849	0.017942	39.764324	0.427573
15	13	cube0	89.558922	165684	24.541622	0.263888	17.562162	0.198884	19.769845	0.212337	24.246234	0.260907	1.516216	0.019529	40.74055	0.438001
16	14	cube0	89.789132	166110	24.749189	0.26612	17.528109	0.188474	19.798378	0.212886	25.122162	0.270311	1.871892	0.020128	40.262161	0.432926
17	15	cube0	89.854502	165326	24.292433	0.261209	17.569947	0.188881	19.932974	0.214333	24.163784	0.258626	1.751892	0.018838	39.874054	0.438753
18	16	cube0	83.293276	154092	21.811352	0.234531	14.96301	0.160761	17.540544	0.186538	20.864324	0.225638	1.654554	0.017786	38.987556	0.387169
19	17	cube0	82.602699	152815	21.984324	0.236391	15.315676	0.164685	17.488108	0.188044	20.912432	0.224865	1.464486	0.015554	38.471352	0.392165
20	18	cube0	83.06324	153667	22.12108	0.237861	15.601622	0.167759	17.61027	0.189338	20.975136	0.225639	1.485757	0.015664	38.613514	0.393694
21	19	cube0	83.905317	153364	22.28873	0.238459	16.018919	0.172246	17.481892	0.189285	20.939999	0.221935	1.847027	0.016183	37.868649	0.403964
22	20	cube0	84.018921	155435	22.344866	0.240267	15.998379	0.170206	17.624865	0.189515	22.002023	0.2482162	1.015937	0.017022	38.970272	0.39753
23	21	cube0	82.38524	152439	21.724525	0.233995	15.822702	0.170137	17.622162	0.189486	20.298378	0.216282	1.414054	0.015205	38.352974	0.394892
24	22	cube0	83.114052	153751	21.857296	0.2361	15.727027	0.169108	17.764864	0.191235	20.312973	0.218419	1.576157	0.016954	38.55027	0.393014
25	23	cube0	82.678917	152956	22.031893	0.236902	15.920244	0.171218	17.662163	0.189916	21.113514	0.227027	1.468649	0.015792	38.192432	0.389166

J	A	B	R	S	T	U	V	W	X	Y	Z	AA
1	block_i	sub_block_j	av_time(us)	av_time(us)	av_vec_time(us)	av_vec_ratio	av_scalar_time(us)	av_scalar_ratio	av_mte2_time(us)	av_mte2_ratio	av_mte3_time(us)	av_mte3_ratio
3	0	vector0	23.316757	43136	3.211351	0.034531	3.737838	0.040192	9.872433	0.106155	1.715676	0.018448
4	0	vector1	23.806486	44042	3.418919	0.036763	5.122162	0.055077	10.32	0.110968	1.751892	0.018838
6	1	vector0	24.982162	46217	3.334054	0.03585	8.053514	0.086597	9.388649	0.100953	1.647027	0.01771
7	1	vector1	23.506487	43487	3.075676	0.033072	8.027027	0.085312	10.415135	0.111991	1.44973	0.015588
9	2	vector0	24.52	45582	3.477297	0.03739	7.518919	0.078698	12.079918	0.129881	1.681622	0.018082
10	2	vector1	22.841621	42257	3.182162	0.034217	5.856757	0.062976	9.004324	0.096821	1.59027	0.0171
12	3	vector0	24.848108	45969	3.282703	0.035298	8.347568	0.089759	9.210811	0.099041	1.682703	0.018094
13	3	vector1	22.888649	42344	3.244324	0.034485	6.65027	0.071508	10.204325	0.109724	1.585405	0.017047
15	4	vector0	24.28108	44920	3.580541	0.0385	2.456757	0.026417	13.63946	0.146661	2.107027	0.022656
16	4	vector1	22.974054	42502	3.085405	0.033176	4.242162	0.045615	11.473514	0.123371	1.558378	0.016757
18	5	vector0	23.206486	42932	3.21027	0.034519	3.841081	0.041302	12.422703	0.133577	1.521622	0.016362
19	5	vector1	22.888107	42343	3.326487	0.035769	2.882162	0.030991	13.616756	0.146417	1.576216	0.016949
21	6	vector0	22.763243	42112	3.048108	0.032775	4.849189	0.052142	12.732432	0.136908	1.523784	0.016385
22	6	vector1	24.455135	45242	3.207561	0.03449	5.392973	0.057989	13.402163	0.144109	1.545405	0.016617
24	7	vector0	24.542162	45403	3.576216	0.038454	3.468649	0.037297	15.167027	0.163086	1.735135	0.018657
25	7	vector1	23.295135	43096	3.07027	0.033014	4.998378	0.053746	11.543243	0.124121	1.460541	0.015705
27	8	vector0	23.976757	44357	2.984324	0.03209	3.31027	0.035594	12.548108	0.124926	1.571351	0.016896
28	8	vector1	22.463783	41558	2.822162	0.030346	2.698011	0.028933	11.334595	0.121877	1.372973	0.014763
30	9	vector0	21.669189	40088	2.730811	0.029364	2.721081	0.029259	11.132973	0.119709	1.391892	0.014967
31	9	vector1	21.612972	39984	2.715135	0.029195	2.892432	0.031101	10.368649	0.111491	1.336216	0.014368
33	10	vector0	21.669729	40089	2.720541	0.029253	2.981622	0.03206	10.813514	0.116274	1.32973	0.014298
34	10	vector1	22.544325	41707	2.768108	0.029765	3.632973	0.039064	8.875135	0.095432	1.330811	0.01431
36	11	vector0	22.905947	42376	2.766486	0.029747	4.143784	0.044557	9.794595	0.105318	1.331892	0.014321
37	11	vector1	22.767567	42120	2.713514	0.029178	4.421081	0.047539	10.637298	0.11438	1.309189	0.014077
39	12	vector0	23.767567	43970	2.988649	0.032136	3.673513	0.0395	12.227568	0.131479	1.441081	0.015495
40	12	vector1	22.28054	41219	2.689189	0.028916	4.261081	0.045818	8.683243	0.093368	1.318378	0.014176
42	13	vector0	22.165945	41007	2.525405	0.027155	5.431892	0.058407	8.858919	0.095257	1.226486	0.013188
43	13	vector1	22.76919	42123	2.518378	0.027079	5.98973	0.064406	7.783784	0.083697	1.257297	0.013519
45	14	vector0	22.184864	41042	2.554595	0.027469	5.322703	0.057233	8.442163	0.090776	1.281081	0.013775
46	14	vector1	23.545406	43559	2.627027	0.028248	5.897838	0.063418	8.081621	0.086899	1.255135	0.013496
48	15	vector0	24.5	45325	2.828108	0.03041	5.932433	0.06379	9.247567	0.099436	1.377297	0.01481
49	15	vector1	23.477297	43433	2.870811	0.030809	4.3	0.046237	9.989108	0.107399	1.339459	0.014403
51	16	vector0	23.311892	43127	2.925946	0.031462	3.943784	0.042406	10.364696	0.111253	1.448108	0.015571
52	16	vector1	23.274054	43057	2.67946	0.028811	5.709189	0.061389	10.528108	0.113205	1.243784	0.013374
54	17	vector0	23.76217	43246	2.61027	0.028067	6.518378	0.07009	10.144324	0.109079	1.264865	0.013601
55	17	vector1	24.602161	45514	2.757297	0.029648	6.621081	0.071194	10.597298	0.113949	1.253513	0.013479
57	18	vector0	23.484865	43447	2.785946	0.029956	5.418378	0.058262	11.457297	0.123197	1.337297	0.01438

表 3-35 端到端性能对比

优化方法	总耗时(us)	AIC_MTE2平均耗时(us)	AIV_MTE2平均耗时(us)
随路ND2NZ	149.82	130.77	0
Vector侧ND2NZ	93.76	22.85	10.31

从上表中执行时间的对比，可以看出：不使用随路ND2NZ指令后，总耗时大幅下降，端到端性能提升明显。

总结

对于矩阵乘计算中矩阵内轴非256字节对齐的场景，随路ND2NZ指令的带宽利用率低，影响算子性能，通过在AIV核上进行ND2NZ的数据重排，提升算子整体性能。值得注意的是，带宽利用率与数据量有关，如果矩阵数据总量太小，即使是在AIV核上进行的ND2NZ转换也无法明显提升有效带宽，反而会因为引入了多核同步，导致算子端到端的性能劣化。

3.7.4.14 Matmul 高阶 API 使能 MTE2 Preload

案例介绍

本案例呈现了在矩阵乘算子场景中，使用Matmul高阶API进行矩阵乘法计算，使能MTE2 Preload对算子性能的提升效果。通过MatmulConfig中的doMTE2Preload参数开启矩阵M或N方向的预加载功能，预加载即在MTE2间隙提前加载A矩阵/B矩阵数据，开启预加载功能后，可以减少MTE2间隙，提升算子性能。doMTE2Preload参数的详细介绍请参考MatmulConfig。

- 使能MTE2 Preload的适用场景
MTE2流水间隙较大，且M或N数值较大时。
- 使能MTE2 Preload的约束条件
 - 仅在使用MDL模板和SpecialMDL模板时，MTE2 Preload有效。
 - 开启M或N方向预加载功能时，需保证K方向数据全载，且M或N方向开启DoubleBuffer。
 - K方向数据全载的条件是 $\text{singleK} \leq \text{baseK} * \text{stepK}$ 。
 - M方向开启DoubleBuffer的条件是 $\text{depthA1} = \text{stepM} * \text{stepK} * 2$ 。
 - N方向开启DoubleBuffer的条件是 $\text{depthB1} = \text{stepN} * \text{stepK} * 2$ 。

本案例的算子规格如下：

表 3-36 算子规格

输入	Shape	Data type	Format
a	128, 512	float16	ND
b	512, 24576	float16	ND

当前案例使用的AI处理器共24个核，算子中使能高阶API Matmul的纯Cube模式。使用MDL模板，Tiling参数如下：

- 原始shape：M=128, N= 24576, K=512。
- 单核shape：singleCoreM=128, singleCoreN=1024, singleCoreK=512。
- 基本块shape：baseM=128, baseN=128, baseK=64。
- L1缓存相关Tiling参数：stepM=1, stepN=1, stepKa=8, stepKb=8, depthA1=8, depthB1=16。

获取性能数据

使用msProf工具获取算子仿真流水图和上板Profiling数据，重点分析Cube，Fixpipe的流水情况。

分析主要瓶颈点

- 优化前的流水图如下，M和K方向全载，因此A矩阵只搬运一次。由于N较大，B矩阵会搬运多次，可以看到单次MTE2间存在间隙。



- 优化前的Profiling数据如下，aic_time平均耗时30.88us。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	a
1	block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_fusepipe_time(us)	aic_fusepipe_ratio	a
2	0	cube0	31.768845	58767	9.196279	0.299567	13.129729	0.413327	8.007027	0.252083	22.472973	0.707455	0.001081	0.000034	3.852973	0.121269	2
3	1	cube0	29.982344	55469	9.201622	0.306862	13.216675	0.440769	7.981081	0.266185	21.58919	0.720042	0.001081	0.000036	3.891931	0.125114	3
4	2	cube0	31.208109	57735	9.189189	0.294449	11.256675	0.360665	7.979459	0.259685	21.611893	0.692509	0.001081	0.000035	3.588108	0.114974	4
5	3	cube0	32.209918	59642	9.20973	0.28971	11.220108	0.348278	7.983108	0.247159	21.463783	0.665772	0.001081	0.000034	4.022162	0.134761	5
6	4	cube0	31.523785	58319	9.193514	0.291637	11.486486	0.364375	7.988108	0.252765	20.431351	0.648125	0.001081	0.000034	4.314595	0.139886	6
7	5	cube0	31.552433	58372	9.184865	0.291098	12.891892	0.407319	7.962973	0.253324	20.488108	0.649335	0.001081	0.000034	4.152973	0.131621	7
8	6	cube0	31.195135	57711	9.199189	0.294571	12.257638	0.424997	7.996757	0.256346	20.163784	0.647017	0.001081	0.000035	3.856077	0.123425	8
9	7	cube0	30.879999	57128	9.201622	0.29798	13.108648	0.424503	7.998378	0.259015	20.156216	0.652727	0.001081	0.000035	3.871892	0.125385	9
10	8	cube0	32.02919	59254	9.21027	0.287559	13.146486	0.410453	7.968108	0.248776	21.05027	0.657221	0.001081	0.000034	3.962162	0.124329	10
11	9	cube0	31.615675	58489	9.191351	0.290701	13.331938	0.421242	7.98	0.252406	20.781891	0.657645	0.001081	0.000034	3.694554	0.116942	11
12	10	cube0	32.132973	59446	9.205846	0.286495	13.581081	0.422652	8.014054	0.249403	20.723783	0.644938	0.001081	0.000034	4.092973	0.127376	12
13	11	cube0	29.874594	55268	9.191892	0.307683	13.097837	0.438427	7.993514	0.267569	20.149731	0.674477	0.001081	0.000036	3.816157	0.121065	13
14	12	cube0	28.229731	52225	9.217838	0.326529	13.568787	0.480323	8.009729	0.283734	18.656216	0.669871	0.001081	0.000038	3.715476	0.115323	14
15	13	cube0	28.163242	52102	9.193514	0.305437	12.526486	0.444781	7.972433	0.283079	18.969649	0.673525	0.001081	0.000038	3.868649	0.137365	15
16	14	cube0	28.445946	52625	9.204325	0.323572	11.555135	0.406214	8.019676	0.281786	19.247568	0.676687	0.001081	0.000038	3.837838	0.134917	16
17	15	cube0	29.574594	54713	9.2	0.311078	11.74973	0.397291	8.009729	0.270831	19.703243	0.666222	0.001081	0.000037	3.601081	0.121763	17
18	16	cube0	31.488486	58250	9.204325	0.292326	11.350811	0.360498	8.008108	0.254335	21.241081	0.674609	0.001081	0.000034	3.801622	0.120738	18
19	17	cube0	32.003784	59207	9.204325	0.287801	12.138378	0.378779	8.018379	0.250545	21.242163	0.668739	0.001081	0.000034	3.886487	0.121438	19
20	18	cube0	32.123783	59429	9.200541	0.286409	12.188108	0.378788	7.988108	0.248666	21.461622	0.668091	0.001081	0.000034	3.775135	0.117518	20
21	19	cube0	31.948109	59104	9.21027	0.288288	12.393514	0.387926	7.979459	0.249783	21.673513	0.678397	0.001081	0.000034	3.704324	0.115948	21
22	20	cube0	30.874553	56717	9.204325	0.300069	13.152432	0.42078	8.013432	0.261122	20.572432	0.678678	0.001081	0.000035	3.97027	0.126174	22
23	21	cube0	31.55946	58385	9.185946	0.291068	13.133513	0.416151	7.985405	0.253027	20.634594	0.653832	0.001081	0.000034	4.080541	0.129297	23
24	22	cube0	30.74	56959	9.194594	0.299108	12.906487	0.41898	8.002703	0.260335	20.114595	0.654346	0.001081	0.000035	4.19027	0.136313	24
25	23	cube0	31.054865	57470	9.208108	0.296416	13.138678	0.422847	8.02	0.258189	20.695135	0.666191	0.001081	0.000035	3.78054	0.121498	25

设计优化方案

使能MTE2 Preload功能：在创建Matmul对象时，开启doMTE2Preload开关。使能MTE2 Preload的完整样例请参考[M/N方向预加载Matmul算子样例](#)。具体步骤如下：

- 步骤1** 配置MDL模板参数，将其中的doMTE2Preload参数设置为2，使能N方向Preload功能。

```
// preloadMode = 2
static constexpr MatmulConfig MM_CFG = GetMDLConfig(false, false, preloadMode);
```

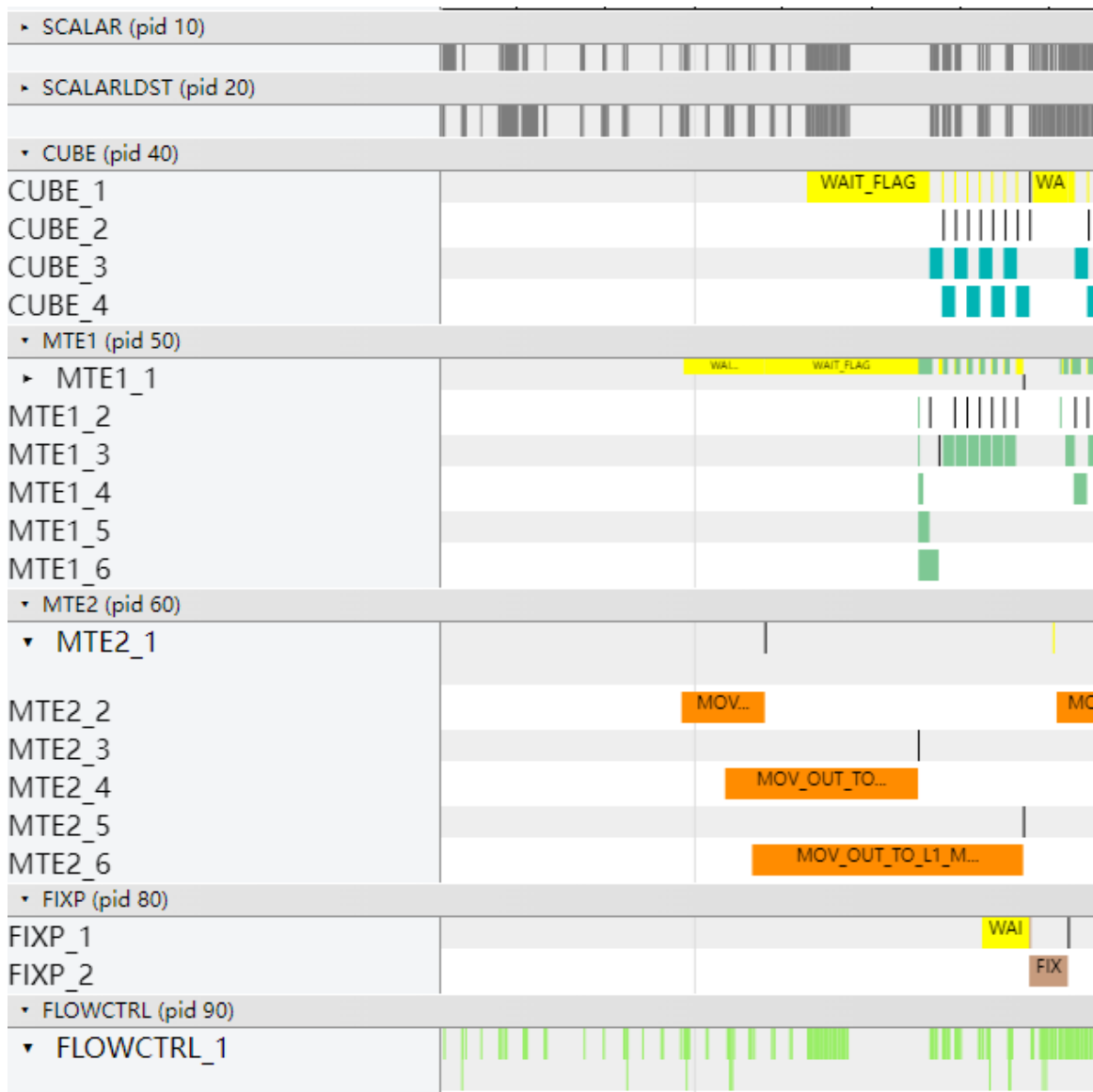
- 步骤2** 基于自定义MatmulConfig模板参数，创建Matmul对象。

```
AscendC::Matmul<AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, aType>,
AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, bType>,
AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, cType>,
AscendC::MatmulType<AscendC::TPosition::GM, CubeFormat::ND, biasType>, MM_CFG> matmulObj;
```

----结束

验证优化方案性能收益

- 优化后的流水图如下，Tiling参数不变，可以看到，下一次计算使用的B矩阵数据提前加载，MTE2间的间隙缩短。



- 优化后的Profiling数据如下，aic_time平均耗时28.50us，较优化前的30.88us有所下降。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_fixp_time(us)	aic_fixp_ratio
2	0	cube0	27.154055	50235	9.220541	0.339564	10.607568	0.390644	8.141622	0.298831	21.851892	0.604738	0.001081	0.00004	3.956216	0.145695
3	1	cube0	29.081568	53812	9.218919	0.318937	10.876751	0.372931	8.115125	0.27899	21.889649	0.725319	0.001081	0.000037	4.068649	0.159676
4	2	cube0	29.901081	53252	9.210511	0.319808	11.245243	0.390376	8.110811	0.281015	20.957538	0.724203	0.001081	0.000038	4.464865	0.155024
5	3	cube0	28.784864	53215	9.225406	0.320718	13.236217	0.460152	8.091892	0.281312	22.166487	0.77061	0.001081	0.000038	3.966486	0.137893
6	4	cube0	29.303244	54211	9.242162	0.315397	13.102703	0.447142	8.09946	0.276401	21.36973	0.729362	0.001081	0.000037	4.074054	0.139031
7	5	cube0	29.458378	54490	9.227568	0.313241	12.417297	0.42152	8.076216	0.274157	21.511351	0.730229	0.001081	0.000037	4.230811	0.14362
8	6	cube0	29.811352	55151	9.221081	0.309314	12.81027	0.429711	8.038379	0.269642	22.273514	0.747149	0.001081	0.000036	3.923243	0.131602
9	7	cube0	29.476797	54532	9.221622	0.312844	13.588919	0.459887	8.050311	0.271124	21.639649	0.733793	0.001081	0.000037	4.129189	0.140083
10	8	cube0	29.187567	53997	9.214595	0.315703	12.225406	0.418857	8.065513	0.277293	20.841621	0.714058	0.001081	0.000037	4.114594	0.140971
11	9	cube0	29.291351	54189	9.197297	0.313994	11.661622	0.398125	8.084865	0.276015	21.222162	0.72452	0.001081	0.000037	4.148649	0.141634
12	10	cube0	28.902702	53395	9.201081	0.318452	12.171192	0.422395	8.063243	0.279947	20.789188	0.721179	0.001081	0.000038	4.45027	0.154509
13	11	cube0	28.069189	51928	9.21027	0.328127	11.31027	0.402943	8.077338	0.287783	21.811567	0.777211	0.001081	0.000039	3.716216	0.132395
14	12	cube0	28.537838	49095	9.264524	0.349099	11.672433	0.439541	8.102703	0.305326	20.20973	0.761544	0.001081	0.000041	3.96973	0.150341
15	13	cube0	28.856716	53346	9.225406	0.31993	12.016778	0.416789	8.063243	0.280321	21.761621	0.754677	0.001081	0.000037	3.907027	0.135453
16	14	cube0	28.776756	53237	9.221081	0.320435	12.075135	0.419614	8.084865	0.280951	21.661081	0.752728	0.001081	0.000038	3.836216	0.13331
17	15	cube0	26.109109	48300	9.216216	0.353002	12.376757	0.474058	8.063784	0.308661	20.038378	0.767616	0.001081	0.000041	4.027567	0.154285
18	16	cube0	27.276217	50451	9.238379	0.338697	12.672973	0.464616	8.094595	0.296764	21.458378	0.786707	0.001081	0.00004	4.048108	0.148412
19	17	cube0	28.963244	52842	9.196217	0.32136	12.220541	0.427841	8.087027	0.283127	21.49946	0.752697	0.001081	0.000038	4.041622	0.141497
20	18	cube0	28.718919	53056	9.21675	0.32134	12.045405	0.420009	8.122162	0.28321	20.755675	0.725121	0.001081	0.000038	4.343784	0.151463
21	19	cube0	28.843784	53361	9.197338	0.318885	12.64054	0.438241	8.082162	0.280205	21.076757	0.730721	0.001081	0.000037	4.412433	0.152977
22	20	cube0	29.457296	54496	9.2	0.312317	12.6	0.427738	8.046108	0.273213	21.963244	0.746275	0.001081	0.000037	4.152973	0.140983
23	21	cube0	29.57513	54711	9.263783	0.313246	12.538918	0.425992	8.059919	0.272505	22.304646	0.754272	0.001081	0.000037	4.256406	0.144907
24	22	cube0	26.867567	49705	9.237838	0.343829	12.392432	0.461241	8.075135	0.309553	20.104324	0.748275	0.001081	0.00004	3.812973	0.141917
25	23	cube0	27.228107	50372	9.210811	0.338283	12.561622	0.461348	8.112973	0.297963	20.13027	0.739319	0.001081	0.00004	3.782162	0.138172

总结

当MTE2流水间隙较大，且M或N数值较大时，可以考虑使能MTE2 Preload功能，提前加载A矩阵或B矩阵数据。